scikit-rf

Object Oriented RF Engineering

scikit-rf Documentation

scikit-rf Developers

Nov 02, 2026

CONTENTS

1 Tutorials **3**
- 1.1 Installation . . . 3
- 1.2 Introduction . . . 4
- 1.3 Configuring frequency units . . . 14
- 1.4 Networks . . . 17
- 1.5 Connecting Networks . . . 33
- 1.6 Plotting . . . 37
- 1.7 NetworkSet . . . 56
- 1.8 Circuits . . . 67
- 1.9 Calibration . . . 77
- 1.10 De-embedding . . . 81
- 1.11 Quality Factors . . . 91
- 1.12 Media . . . 99
- 1.13 Vector Fitting . . . 112
- 1.14 Parametric Vector Fitting . . . 119
- 1.15 Virtual Instruments . . . 119

2 Examples **123**
- 2.1 Network Theory . . . 123
- 2.2 Matching . . . 274
- 2.3 Metrology and Calibration . . . 278
- 2.4 NetworkSets . . . 486
- 2.5 Plotting . . . 497
- 2.6 Circuit . . . 557
- 2.7 Mixed Mode . . . 581
- 2.8 Vector Fitting . . . 590
- 2.9 Parametric Vector Fitting . . . 611
- 2.10 Q-factor . . . 615
- 2.11 Taper . . . 623
- 2.12 Instrument Control . . . 628
- 2.13 Interactive . . . 628

3 API **633**
- 3.1 network (*skrf.network*) . . . 633
- 3.2 frequency (*skrf.frequency*) . . . 836
- 3.3 networkSet (*skrf.networkSet*) . . . 846
- 3.4 plotting (*skrf.plotting*) . . . 868
- 3.5 Media . . . 881
- 3.6 taper (*skrf.taper*) . . . 1136
- 3.7 calibration (*skrf.calibration.calibration*) . . . 1148

3.8 deembedding (*skrf.calibration.deembedding*) . . . 1394
3.9 virtual instruments (*skrf.vi*) . . . 1443
3.10 vna (*skrf.vi.vna*) . . . 1452
3.11 vectorFitting (skrf.vectorFitting) . . . 1455
3.12 Qfactor (*skrf.qfactor*) . . . 1474
3.13 time (*skrf.time*) . . . 1481
3.14 circuit (*skrf.circuit*) . . . 1485
3.15 device (*skrf.media.device*) . . . 1508
3.16 mathFunctions (*skrf.mathFunctions*) . . . 1514
3.17 tlineFunctions (*skrf.tlineFunctions*) . . . 1531
3.18 instances (*skrf.instances*) . . . 1546
3.19 constants (*skrf.constants*) . . . 1547
3.20 data (*skrf.data*) . . . 1549
3.21 util (*skrf.util*) . . . 1558
3.22 IO . . . 1569

4 Contributing to scikit-rf **1593**
4.1 Sponsoring the Project . . . 1593
4.2 Contributing to the Code . . . 1593
4.3 Contributing to the Documentation . . . 1596
4.4 Join the **scikit-rf** team! . . . 1596

5 Acknowledging or Citing scikit-rf **1599**
5.1 In Publications . . . 1599
5.2 In Presentations or Apps . . . 1599

6 Publications using or citing scikit-rf **1601**
6.1 2026 . . . 1601
6.2 2025 . . . 1601
6.3 2024 . . . 1602
6.4 2023 . . . 1605
6.5 2022 . . . 1606
6.6 2021 . . . 1607
6.7 2020 . . . 1607
6.8 2019 . . . 1607
6.9 2018 . . . 1607
6.10 2012-2018 . . . 1608

7 Glossary **1609**

8 License **1611**

Bibliography **1613**

Python Module Index **1617**

Index **1619**

scikit-rf (aka *skrf*) is an Open Source, BSD-licensed package for RF/Microwave engineering developed and maintained for all supported versions of the Python programming language (currently 3.8+). It provides a modern, object-oriented library which is both flexible and scalable. The documentation below is broken up into three sections; narrative tutorials, practical examples, and a reference API.

CHAPTER

ONE

TUTORIALS

These tutorials are meant to give an overview of how skrf can be used. Each page can be followed along in a browser, or downloaded as an ipython notebook

1.1 Installation

1.1.1 anaconda (recommended)

The easiest way to install **scikit-rf** is to first install anaconda, a binary package manager for python modules. Once anaconda is setup you can install scikit-rf by entering the following into a terminal (do not forget the `-c conda-forge` option):

```
conda install -c conda-forge  scikit-rf
```

1.1.2 pip

Using the Python Package Index (PyPi), you can install **scikit-rf** using:

```
pip install scikit-rf
```

1.1.3 git

The bleeding-edge development version of **scikit-rf** may be installed from our github page. You can use git directly, or download a zipfile.

```
git clone git@github.com:scikit-rf/scikit-rf.git
cd scikit-rf
python setup.py install
```

1.1.4 Optional Modules

Some features of scikit-rf won't be available until you install additional modules. You can install these using conda or pip.

Instrument Control

To use the vi module, you must install all of its dependencies. This can be done with

$ pip install scikit-rf[visa]

which will install the following:

- pyvisa

- pyvisa-py

Warning about instrument communications

The above should be enough to communicate with many instruments. However, some of us have had difficult to resolve issues with certain instrument configurations that require particular setups. For example, to communicate with Keysight's N5227B PNA over GPIB, we had to install the Keysight IO Libraries Suite

When using the virtual instrument module, you have the option of specifying the backend that PyVISA uses to communicate with instruments. By default, this is PyVISA-py, the fully Python VISA implementation, but in the case above with the Keysight Analyzer, we had to specify the exact .dll of the VISA driver. More information can be found here on VISA backends, but if you have issues, please submit a detailed bug report!

Excel file export

- xlrd
- xlwt http://www.python-excel.org/ - for xls reading/writing

[ ]:

Download This Notebook: `Introduction.ipynb`

1.2 Introduction

This is a brief introduction to **scikit-rf** (aka `skrf`). The intended audience are those who have a working python stack, and are somewhat familiar with python. If you are completely new to python, see scipy's Getting Started. First, import the scikit-rf module `skrf`, as `rf`

[1]:
```
import skrf as rf
```

If this produces an error, please see the *installation* tutorial.

1.2.1 Networks

The central object in `skrf` is a N-port microwave *Network* object. A *Network* can be created in a number of ways, one way is from data stored in a touchstone file.

[2]:
```
ring_slot = rf.Network('data/ring slot.s2p')
```

If you can't find `ring slot.s2p`, then just import it from the `skrf.data` module.

[3]:
```
from skrf.data import ring_slot  # noqa: F811
```

A short description of the network will be printed out if entered onto the command line

[4]:
```
ring_slot
```

```
[4]: 2-Port Network: 'ring slot',  75.0-110.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]
```

The basic attributes of a microwave *Network* are provided by the following properties,

- `Network.s` : Scattering Parameter matrix.
- `Network.z0` : Port Impedance matrix.
- `Network.frequency` : Frequency Object.

The *Network* object has numerous other properties and methods which can found in it's docstring. If you are using IPython/Jupyter, then these properties and methods can be 'tabbed' out on the command line.

```
In [1]: ring_slot.s<TAB>
ring_slot.s               ring_slot.s_arcl         ring_slot.s_im
ring_slot.s11             ring_slot.s_arcl_unwrap  ring_slot.s_mag
...
```

Other way to build Network are detailed in the *tutorial on Networks*.

Linear Operations

Element-wise mathematical operations on the s-parameters are accessible through overloaded operators. To illustrate, we load a couple `Networks` stored in the `skrf.data` module.

```
[5]: short = rf.data.wr2p2_short
     delayshort = rf.data.wr2p2_delayshort
```

The complex difference between their s-parameters is computed with

```
[6]: short - delayshort
[6]: 1-Port Network: 'wr2p2,short',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j]
```

This returns a new *Network*. Other arithmetic operators are overloaded as well,

```
[7]: short/delayshort
[7]: 1-Port Network: 'wr2p2,short',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j]
```

Cascading and De-embedding

Cascading and de-embeding 2-port Networks can also be done though operators. Cascading is done through the power operator, `**`.

```
[8]: short = rf.data.wr2p2_short
     line = rf.data.wr2p2_line

     delayshort = line ** short
```

De-embedding can be accomplished by cascading the *inverse* of a network. The inverse of a network is accessed through the property `Network.inv`.

```
[9]: short = line.inv ** delayshort
```

For more information on the functionality provided by the *Network* object, such as interpolation, stitching, n-port connections, and IO support see the *Networks* tutorial.

Finding minimum (or maximum) of a Network quantity

Often, it is desirable to get the minimum (or maximum) value of a Network quantity (s-parameters, z-parameters, etc.) and the frequency at which this occurs. In `scikit-rf`, Network quantities are stored as Numpy arrays of shape (N_F, N_p, N_p) where N_F is the number of frequency points and N_p is the number of ports of a Network:

[10]:
```
print(type(line.s))  # s-parameters are stored as a Numpy array
```
```
<class 'numpy.ndarray'>
```

[11]:
```
print(line.s.shape)  # line is a 2-port Network defined on 201 frequency points
```
```
(201, 2, 2)
```

The frequency points are defined in the `frequency` parameter of a Network:

[12]:
```
print(line.frequency)  # returns a Frequency object
```
```
330.0-500.0 GHz, 201 pts
```

The frequency values are given by the `frequency.f` parameter, or simply `.f`:

[13]:
```
line.f[0:10]  # the 10 first frequency points. Same than line.frequency.f[0:10]
```
[13]:
```
array([3.3000e+11, 3.3085e+11, 3.3170e+11, 3.3255e+11, 3.3340e+11,
       3.3425e+11, 3.3510e+11, 3.3595e+11, 3.3680e+11, 3.3765e+11])
```

Being a Numpy array, finding the minimum (or maximum) value of a the magnitude of the S_{21} parameter can be performed using the `min()` (or `max()`) method:

[14]:
```
import numpy as np

rs = rf.data.ring_slot  # another 2-port example

print(rs.s_mag[:,1,0].min())  # or .max() for maximum. Watch out that Python indexing␣
↪starts at 0!
```
```
0.5101255034355594
```

Finding the frequency at which the magnitude of the S_{11} parameter is minimum can be performed using the Numpy function `argmin <https://numpy.org/doc/stable/reference/generated/numpy.argmin.html?highlight=argmin#numpy.argmin>`__:

[15]:
```
f_match = rs.f[np.argmin(rs.s_mag[:,0,0])]  # frequency for min(|S11|)
print(f_match)
```
```
85850000000.0
```

1.2.2 Plotting

> **Note**
>
> The plotting infrastructure in skrf is under refactoring at the moment to allow for multiple backends, and headless setups. The following assumes you are using *matplotlib* with an interactive session.

skrf has a function which updates your matplotlib rcParams so that plots appear like the ones shown in these tutorials.

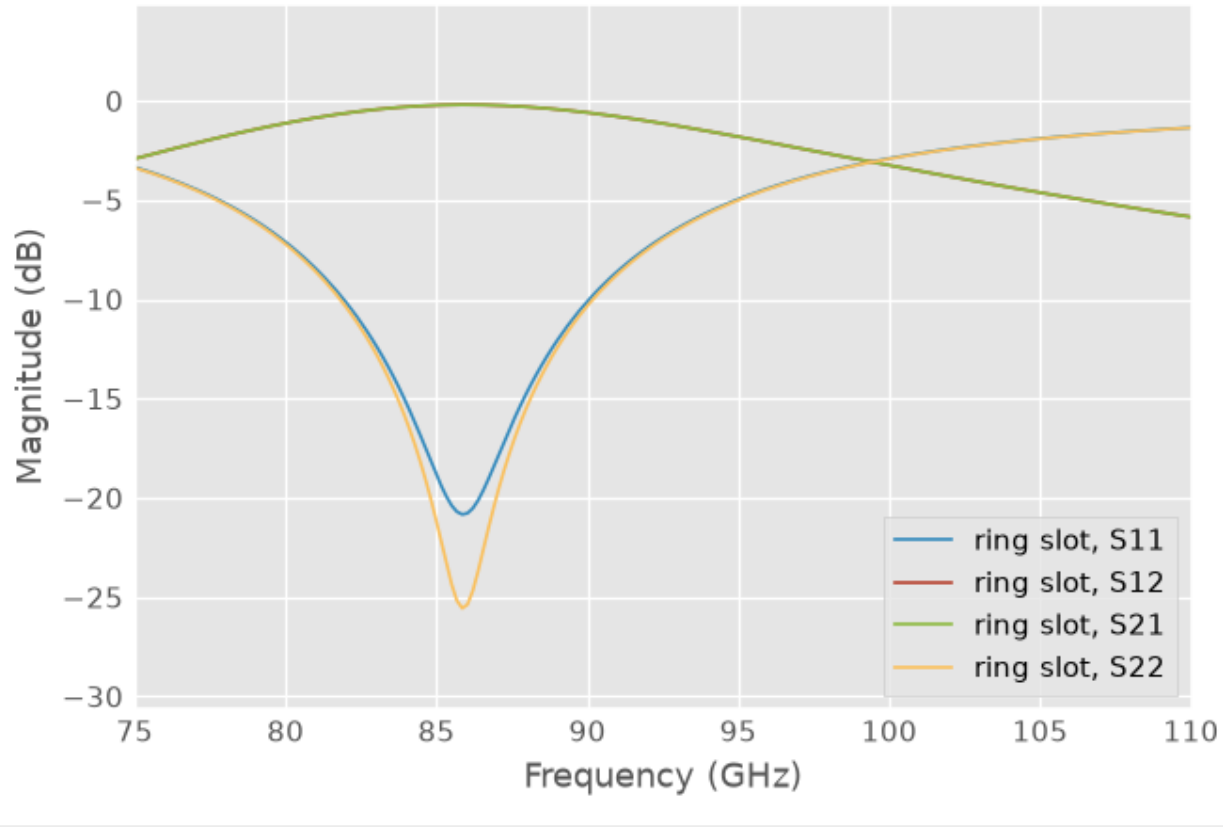
[16]:
```
# display plots in notebook
%matplotlib inline
import matplotlib.pyplot as plt

rf.stylely()
```

The methods of the *Network* class provide convenient ways to plot components of the network parameters,

- `Network.plot_s_db()` : plot magnitude of s-parameters in log scale
- `Network.plot_s_deg()` : plot phase of s-parameters in degrees
- `Network.plot_s_smith()` : plot complex s-parameters on Smith Chart

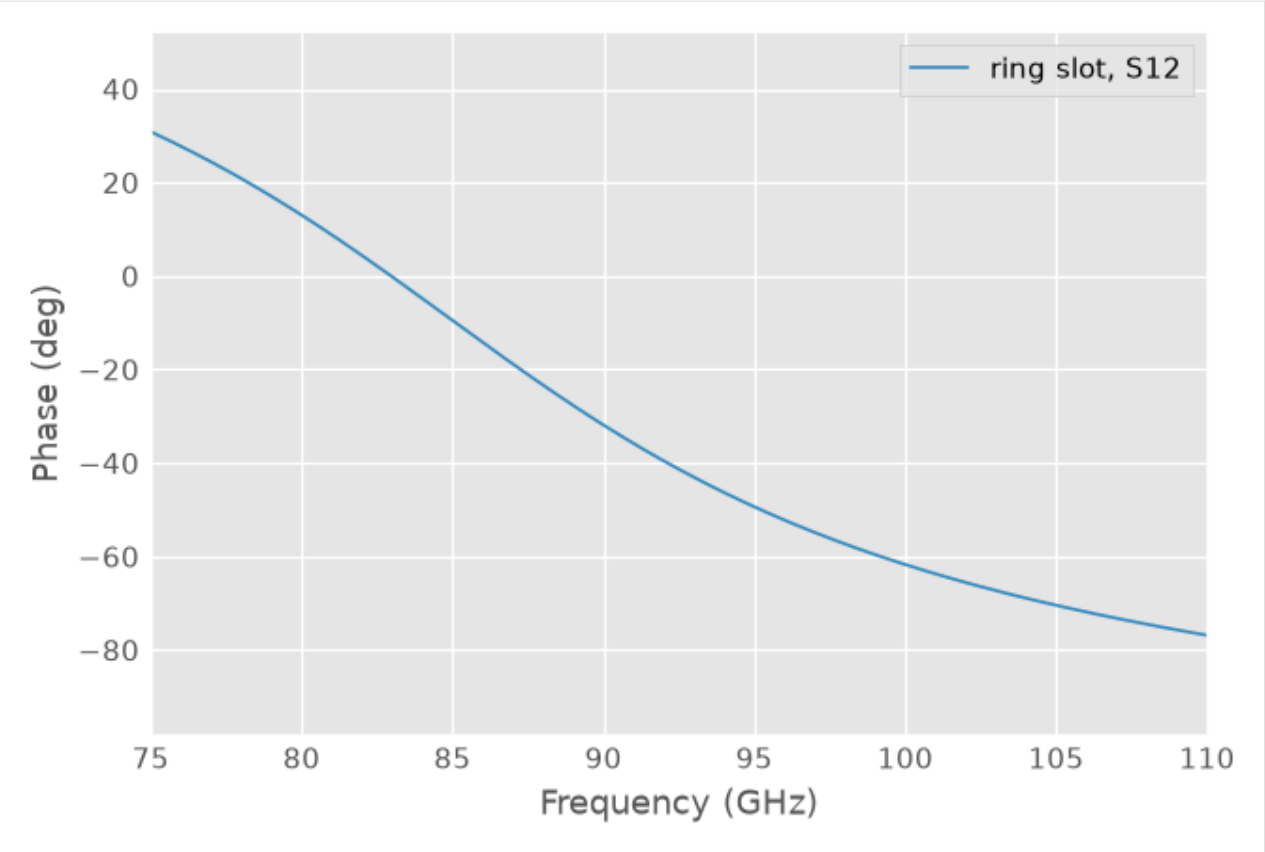
To plot all four s-parameters of the `ring_slot` in Mag, Phase, and on the Smith Chart.

[17]:
```
ring_slot.plot_s_db()
```



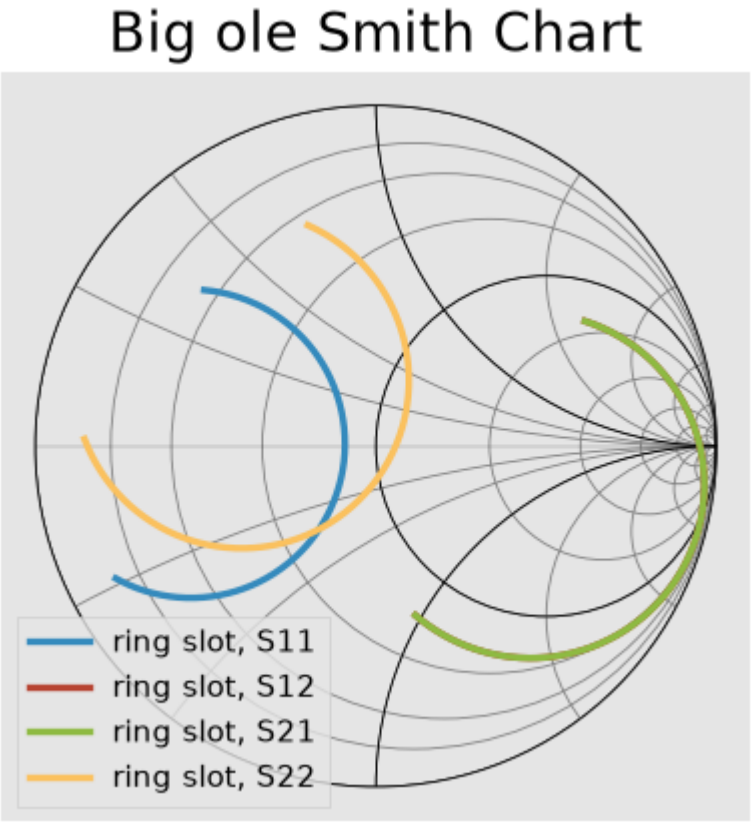
Or plot the phase of S_{12}

[18]:
```
ring_slot.plot_s_deg(m=0,n=1)
```



[19]:
```
ring_slot.plot_s_smith(lw=2)
plt.title('Big ole Smith Chart')
```

[19]:
```
Text(0.5, 1.0, 'Big ole Smith Chart')
```



For more detailed information about plotting see the *Plotting* tutorial

1.2.3 NetworkSet

The *NetworkSet* object represents an unordered set of networks and provides methods for calculating statistical quantities and displaying uncertainty bounds.

A *NetworkSet* is created from a list or dict of *Networks*'s. This can be done quickly with `rf.io.read_all()` , which loads all skrf-readable objects in a directory. The argument `contains` is used to load only files which match a given substring.

[20]:
```
rf.io.read_all('data/', contains='ro')
```

[20]:
```
{'ro,1': 1-Port Network: 'ro,1',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j],
 'ro,2': 1-Port Network: 'ro,2',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j],
 'ro,3': 1-Port Network: 'ro,3',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]}
```

This dictionary can be passed directly to the *NetworkSet* constructor,

[21]:
```
from skrf.networkSet import NetworkSet

ro_dict = rf.io.read_all('data/', contains='ro')
ro_ns = NetworkSet(ro_dict, name='ro set') # name is optional
ro_ns
```

[21]:
```
3-Networks NetworkSet: [1-Port Network: 'ro,1',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j],␣
↪1-Port Network: 'ro,2',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j], 1-Port Network: 'ro,3
```

(continues on next page)

(continued from previous page)

```
↪',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]]
```

NetworkSet's are list-like.

Statistical Properties

Statistical quantities can be calculated by accessing properties of the *NetworkSet*. For example, to calculate the complex average of the set, access the `mean_s` property

[22]:
```
ro_ns.mean_s
```

[22]:
```
1-Port Network: 'ro set',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]
```

The returned results are stored in a *Network*s s-parameters, regardless of the type of the output. Similarly, to calculate the complex standard deviation of the set,

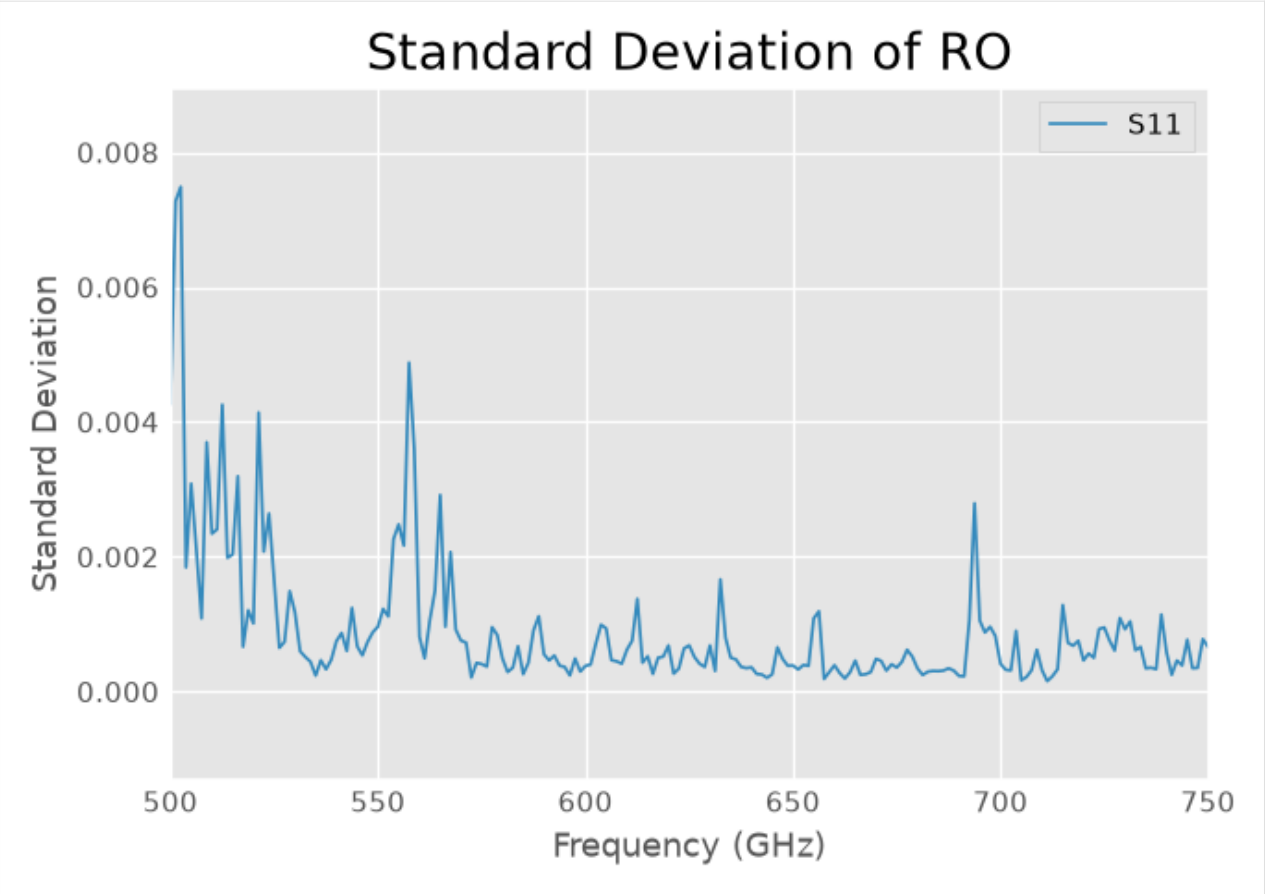
[23]:
```
ro_ns.std_s
```

[23]:
```
1-Port Network: 'ro set',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]
```

Because these methods return a *Network* object the results can be saved or plotted in the same way as you would with a *Network*. To plot the magnitude of the standard deviation of the set,

[24]:
```
ro_ns.std_s.plot_s_mag(label='S11')
plt.ylabel('Standard Deviation')
plt.title('Standard Deviation of RO')
```

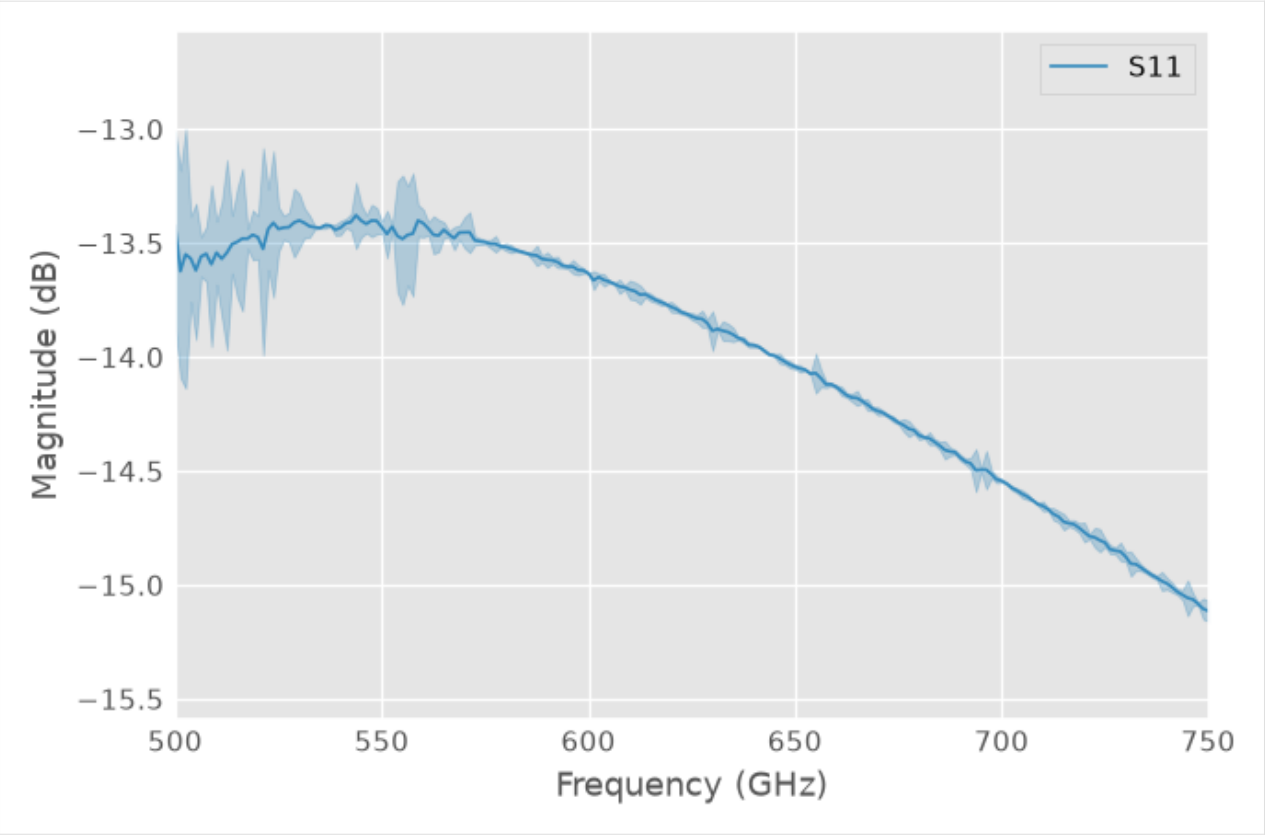
[24]:
```
Text(0.5, 1.0, 'Standard Deviation of RO')
```



Plotting Uncertainty Bounds

Uncertainty bounds on any network parameter can be plotted through the methods

[25]:
```
ro_ns.plot_uncertainty_bounds_s_db(label='S11');
```



See the *networkset* tutorial for more information.

1.2.4 Virtual Instruments

The *skrf.vi* module provides classes to communicate with some instruments. At the moment, this is only VNAs, but more support may be added in the future. See the *Virtual Instrument* tutorial for more information.

An example of using the `PNA` class to retrieve some s-parameter data and plot it

```
from skrf.vi.vna.keysight import PNA
instr = PNA(address="TCPIP0::10.0.0.1::INSTR")

freq = Frequency(start=2.3e9, stop=2.6e9, npoints=401, unit='hz')
instr.frequency = freq

ntwk = instr.get_snp_network(ports=(1,2))
ntwk.s21.plot_s_db()
```

1.2.5 Calibration

Calibrations are performed through a *Calibration* class. In most cases, creating a *Calibration* object requires at least two pieces of information:

- a list of measured *Network*'s
- a list of ideal *Network*'s

The *Network* elements in each list must all be similar (same #ports, frequency info, etc) and must be aligned to each other, meaning the first element of ideals list must correspond to the first element of measured list.

Below is an example script illustrating how to create a *Calibration* .

One Port Calibration

```
import skrf as rf
from skrf.calibration import OnePort

my_ideals = rf.io.read_all('ideals/')
my_measured = rf.io.read_all('measured/')
duts = rf.io.read_all('measured/')

## create a Calibration instance
cal = OnePort(
    ideals = [my_ideals[k] for k in ['short','open','load']],
    measured = [my_measured[k] for k in ['short','open','load']],
    )

caled_duts = [cal.apply_cal(dut) for dut in duts.values()]
```

See the *Calibration* tutorial for more details and examples.

1.2.6 Transmission Line Media

Simple transmission-line based networks can be created through methods of the *Media* class, which represents a transmission line object for a given medium. Once constructed, a *Media* object contains the necessary properties such as `propagation constant` and `characteristic impedance`, that are needed to generate microwave circuits.

The basic usage looks something like this,

CPW

[26]:
```
from skrf import Frequency
from skrf.media import CPW, Coaxial

freq = Frequency(75, 110, 101, 'GHz')
cpw =  CPW(freq, w = 10e-6, s = 5e-6, ep_r = 10.6)
cpw
```

[26]:
```
Coplanar Waveguide Media.  75.0-110.0 GHz. 101 points
 W= 1.00e-05m, S= 5.00e-06m
```

[27]:
```
cpw.line(d=90,unit='deg', name='line')
```

[27]:
```
2-Port Network: 'line',  75.0-110.0 GHz, 101 pts, z0=[50.02832417+0.j 50.02832417+0.j]
```

Coax

[28]:
```
freq = Frequency(1, 10, 101, 'GHz')
coax = Coaxial(frequency = freq, Dint = 1e-3, Dout = 2e-3)
coax
```

[28]:
```
Coaxial Media.  Coaxial Media.  1.0-10.0GHz, 101 points.
Dint = 1.00 mm, Dout = 2.00 mm
z0 = (41.7, -0.1j)-(41.6, -0.0j) Ohm
z0_port is not defined.
```

[ ]:

Download This Notebook: `Frequency_units.ipynb`

1.3 Configuring frequency units

A frequency's input unit tells scikit-rf what the supplied numbers mean. Its display unit controls how those frequencies are represented. Internally, `Frequency.f` always stores values in Hz; `Frequency.f_scaled` expresses the same frequencies in `Frequency.unit`.

Set `rf.constants.FREQ_UNIT_DEFAULT` to choose a default unit for newly created frequency objects and networks, including networks read from Touchstone files. An explicit `Network(..., f_unit=...)` overrides this default for that network. This setting applies to the current Python process and needs no configuration file. The examples below save the current setting and restore it at the end.

[1]:
```
from io import StringIO

import numpy as np

import skrf as rf

original_unit = rf.constants.FREQ_UNIT_DEFAULT
```

1.3.1 Keeping input units

The factory setting is `None`. It preserves explicit constructor units and file units. When `unit` is omitted from `Frequency` or `Frequency.from_f`, the input is interpreted in Hz.

[2]:
```
rf.constants.FREQ_UNIT_DEFAULT = None

omitted = rf.Frequency(1, 2, 2)
original = rf.Frequency(1000, 2000, 2, unit="MHz")

print("Omitted unit:", omitted.unit, omitted.f_scaled)
print("Explicit unit:", original.unit, original.f_scaled)
assert omitted.unit == "Hz"
assert original.unit == "MHz"
np.testing.assert_allclose(omitted.f, [1, 2])
np.testing.assert_allclose(original.f, [1e9, 2e9])
```

```
Omitted unit: Hz [1. 2.]
Explicit unit: MHz [1000. 2000.]
```

1.3.2 Choosing a common unit

With a global unit of GHz, omitted Frequency input units are interpreted as GHz. An explicit input unit still determines the meaning of the input values, but the resulting object uses GHz for its representation.

All three constructions below describe the same 1-2 GHz band. For each object, `f` is `[1e9, 2e9]`, `unit` is `"GHz"`, and `f_scaled` is `[1, 2]`.

[3]:
```
rf.constants.FREQ_UNIT_DEFAULT = "GHz"
```

(continues on next page)

(continued from previous page)

```
implicit = rf.Frequency(1, 2, 2)
explicit = rf.Frequency(1000, 2000, 2, unit="MHz")
from_values = rf.Frequency.from_f([1000, 2000], unit="MHz")

for frequency in (implicit, explicit, from_values):
    print(frequency.f, "Hz ->", frequency.f_scaled, frequency.unit)
    assert frequency.unit == "GHz"
    np.testing.assert_allclose(frequency.f, [1e9, 2e9])
    np.testing.assert_allclose(frequency.f_scaled, [1, 2])
```

```
[1.e+09 2.e+09] Hz -> [1. 2.] GHz
[1.e+09 2.e+09] Hz -> [1. 2.] GHz
[1.e+09 2.e+09] Hz -> [1. 2.] GHz
```

1.3.3 Reading Touchstone data

A Touchstone header defines the unit of the file's frequency values. scikit-rf uses that unit to read the data. An explicit `f_unit` chooses the resulting network's display unit, overriding the global default. If `f_unit` is omitted, the global default applies; when it is `None`, the file unit is retained. The absolute frequencies, scattering parameters and impedances are unchanged. For files containing noise data, the noise frequency axis uses the same selected display unit.

This small Touchstone example contains two frequencies, 1000 and 2000 MHz. We use an in-memory file so the notebook needs no downloads or data files. For a file on disk, use `rf.Network("example.s1p")` in the same way.

[4]:
```
touchstone_data = "# MHz S RI R 50\n1000 0.1 0\n2000 0.2 0\n"

with StringIO(touchstone_data) as stream:
    stream.name = "example.s1p"
    rf.constants.FREQ_UNIT_DEFAULT = None
    file_units = rf.Network(stream)

with StringIO(touchstone_data) as stream:
    stream.name = "example.s1p"
    rf.constants.FREQ_UNIT_DEFAULT = "GHz"
    configured = rf.Network(stream)

with StringIO(touchstone_data) as stream:
    stream.name = "example.s1p"
    overridden = rf.Network(stream, f_unit="Hz")

print("File units:", file_units.frequency.f_scaled, file_units.frequency.unit)
print("Configured units:", configured.frequency.f_scaled, configured.frequency.unit)
print("Explicit override:", overridden.frequency.f_scaled, overridden.frequency.unit)
assert file_units.frequency.unit == "MHz"
assert configured.frequency.unit == "GHz"
assert overridden.frequency.unit == "Hz"
np.testing.assert_allclose(configured.frequency.f_scaled, [1, 2])
np.testing.assert_allclose(overridden.frequency.f_scaled, [1e9, 2e9])
for network in (configured, overridden):
    np.testing.assert_allclose(network.f, [1e9, 2e9])
    np.testing.assert_array_equal(network.f, file_units.f)
    np.testing.assert_array_equal(network.s, file_units.s)
    np.testing.assert_array_equal(network.z0, file_units.z0)
```

```
File units: [1000. 2000.] MHz
Configured units: [1. 2.] GHz
Explicit override: [1.e+09 2.e+09] Hz
```

1.3.4 Supplying raw frequencies to a Network

`Network(f=...)` keeps its existing input convention: raw numbers are in Hz unless `f_unit` specifies another input unit. This differs from omitting `unit` in a Frequency constructor, which uses the configured unit for input. An explicit `f_unit` also sets the resulting Network's display unit. When it is omitted, the display unit follows the global setting, or Hz if it is `None`.

[5]:
```
raw_hz = rf.Network(f=[1e9, 2e9], s=[0.1, 0.2])
raw_mhz = rf.Network(f=[1000, 2000], f_unit="MHz", s=[0.1, 0.2])

assert raw_hz.frequency.unit == "GHz"
assert raw_mhz.frequency.unit == "MHz"
np.testing.assert_allclose(raw_hz.frequency.f_scaled, [1, 2])
np.testing.assert_allclose(raw_mhz.frequency.f_scaled, [1000, 2000])
for network in (raw_hz, raw_mhz):
    print(network.frequency.f_scaled, network.frequency.unit)
    np.testing.assert_allclose(network.f, [1e9, 2e9])
```

```
[1. 2.] GHz
[1000. 2000.] MHz
```

1.3.5 Existing objects and copies

Changing the global setting does not change existing objects. The `original` frequency created earlier still uses MHz, and its copies and slices retain that unit. You can explicitly assign `.unit` to change an individual object's representation without changing its absolute frequencies.

Constructing a new Network from an existing Frequency applies an explicit `f_unit` to the network's copy. When `f_unit` is omitted, the global setting applies, or the source unit is retained if the setting is `None`. The source object is unchanged.

[6]:
```
copied = original.copy()
sliced = original[1:]
network = rf.Network(frequency=original, s=[0.1, 0.2])

assert original.unit == copied.unit == sliced.unit == "MHz"
assert network.frequency.unit == "GHz"
np.testing.assert_array_equal(network.f, original.f)

copied.unit = "kHz"
print("Original:", original.f_scaled, original.unit)
print("Modified copy:", copied.f_scaled, copied.unit)
print("New network:", network.frequency.f_scaled, network.frequency.unit)
assert original.unit == "MHz"
np.testing.assert_array_equal(copied.f, original.f)
```

```
Original: [1000. 2000.] MHz
Modified copy: [1000000. 2000000.] kHz
New network: [1. 2.] GHz
```

1.3.6 Restoring the previous setting

Restore the setting saved at the start so that later work in this kernel uses the same configuration as before the tutorial. The objects already created above keep their units.

[7]:
```
rf.constants.FREQ_UNIT_DEFAULT = original_unit
```

Download This Notebook: `Networks.ipynb`

1.4 Networks

1.4.1 Introduction

This tutorial gives an overview of the microwave network analysis features of **skrf**. For this tutorial, and the rest of the scikit-rf documentation, it is assumed that **skrf** has been imported as `rf`. Whether or not you follow this convention in your own code is up to you.

[1]:
```
import numpy as np

import skrf as rf
```

If this produces an import error, please see *Installation*.

1.4.2 Creating Networks

skrf provides an object for a N-port microwave *Network*. A *Network* can be created in a number of ways:

- from a Touchstone file
- from S-parameters
- from Z-parameters
- from other RF parameters (Y, ABCD, T, etc.)

Some examples for each situation is given below.

Creating Network from Touchstone file

Touchstone file (`.sNp` files, with `N` being the number of ports) is a *de facto* standard to export N-port network parameter data and noise data of linear active devices, passive filters, passive devices, or interconnect networks. Creating a Network from a Touchstone file is simple:

[2]:
```
from skrf import Frequency, Network

ring_slot = Network('data/ring slot.s2p')
```

Note that some software, such as ANSYS HFSS, add additional information to the Touchstone standard, such as comments, simulation parameters, Port Impedance or Gamma (wavenumber). These data are also imported if detected.

A short description of the network will be printed out if entered onto the command line

[3]:
```
ring_slot
```

[3]:
```
2-Port Network: 'ring slot',  75.0-110.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]
```

Creating Network from s-parameters

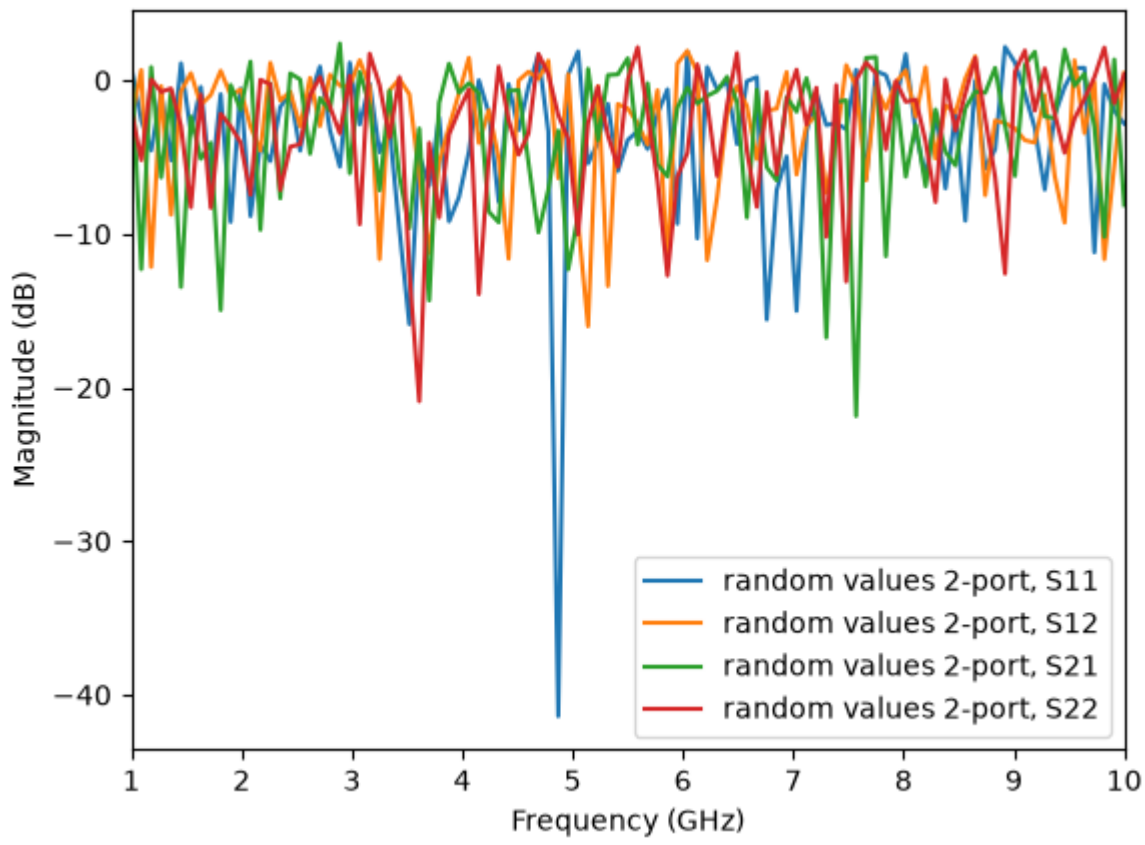
Networks can also be created by directly passing values for the `frequency`, `s`-parameters (and optionally the port impedance `z0`).

The scattering matrix of a N-port Network is expected to be a Numpy array of shape `(nb_f, N, N)`, where `nb_f` is the number of frequency points and `N` the number of ports of the network.

[4]:
```
# dummy 2-port network from Frequency and s-parameters
freq = Frequency(1, 10, 101, 'ghz')
rng = np.random.default_rng()
s = rng.uniform(size=(101, 2, 2)) + 1j*rng.uniform(size=(101, 2, 2))  # random complex␣
↪numbers
# if not passed, will assume z0=50. name is optional but it's a good practice.
ntwk = Network(frequency=freq, s=s, name='random values 2-port')
ntwk
```

[4]:
```
2-Port Network: 'random values 2-port',  1.0-10.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]
```

[5]:
```
ntwk.plot_s_db()
```



Often, s-parameters are stored in separate arrays. In such case, one needs to forge the s-matrix:

[6]:
```
# let's assume we have separate arrays for the frequency and s-parameters
f = np.array([1, 2, 3, 4]) # in GHz
S11 = rng.uniform(size=4)
S12 = rng.uniform(size=4)
S21 = rng.uniform(size=4)
S22 = rng.uniform(size=4)

# Before creating the scikit-rf Network object, one must forge the Frequency and S-
↪matrix:
freq2 = rf.Frequency.from_f(f, unit='GHz')

# forging S-matrix as shape (nb_f, 2, 2)
# there is probably smarter way, but less explicit for the purpose of this example:
s = np.zeros((len(f), 2, 2), dtype=complex)
s[:,0,0] = S11
s[:,0,1] = S12
s[:,1,0] = S21
s[:,1,1] = S22

# constructing Network object
ntw = rf.Network(frequency=freq2, s=s)

print(ntw)
```

```
2-Port Network: '',  1.0-4.0 GHz, 4 pts, z0=[50.+0.j 50.+0.j]
```

If necessary, the characteristic impedance can be passed as a scalar (same for all frequencies), as a list or an array:

[7]:
```
ntw2 = rf.Network(frequency=freq, s=s, z0=25, name='same z0 for all ports')
print(ntw2)
ntw3 = rf.Network(frequency=freq, s=s, z0=[20, 30], name='different z0 for each port')
print(ntw3)
ntw4 = rf.Network(frequency=freq, s=s, z0=rng.uniform(size=(4,2)), name='different z0␣
↪for each frequencies and ports')
print(ntw4)
```

```
2-Port Network: 'same z0 for all ports',  1.0-10.0 GHz, 101 pts, z0=[25.+0.j 25.+0.j]
2-Port Network: 'different z0 for each port',  1.0-10.0 GHz, 101 pts, z0=[20.+0.j 30.+0.
↪j]
2-Port Network: 'different z0 for each frequencies and ports',  1.0-10.0 GHz, 101 pts,␣
↪z0=[0.78793375+0.j 0.98447722+0.j]
```

From z-parameters

As networks are also defined from their Z-parameters, there is `from_z()` method of the Network:

[8]:
```
# 1-port network example
z = np.full((len(freq), 1, 1), 10j)  # replicate z=10j for all frequencies

ntw = rf.Network(frequency=freq, z=z)
print(ntw)
```

```
1-Port Network: '',  1.0-10.0 GHz, 101 pts, z0=[50.+0.j]
```

From other network parameters: Y, ABCD, H, T

It is also possible to generate Networks from other kind of RF parameters: Y, ABCD, H or T using the `y=`, `a=`, `h=` or `t=` parameters respectively when creating a `Network`.

For example, the ABCD parameters of a series-impedance is:

$$\begin{bmatrix} 1 & Z \\ 0 & 1 \end{bmatrix}$$

so the associated Network can be created like:

[9]:
```
z = 20
abcd = np.array([[1, z],
                 [0, 1]])

a = np.tile(abcd, (len(freq),1,1))
ntw = Network(frequency=freq, a=a)
print(ntw)
```

```
2-Port Network: '',  1.0-10.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]
```

Note that convenience functions are also available for converting some parameters into another if required:

from:nbsphinx-math:*to*	S	Z	Y	ABCD	T	H
S		s2z	s2y	s2a	s2t	s2h
Z	z2s		z2y	z2a	z2t	z2h
Y	y2s	y2z			y2t	
ABCD	a2s	a2z				
T	t2s	t2z	t2y			
H	h2s	h2z				

[10]:
```
# example: converting a -> s
s = rf.network.a2s(a)
# checking that these S-params are the same
np.all(ntw.s == s)
```

[10]:
```
np.True_
```

1.4.3 Basic Properties

The basic attributes of a microwave *Network* are provided by the following properties :

- `Network.s` : Scattering Parameter matrix.
- `Network.z0` : Port Characteristic Impedance matrix.
- `Network.frequency` : Frequency Object.

The *Network* object has numerous other properties and methods. If you are using IPython, then these properties and methods can be 'tabbed' out on the command line.

```
In [1]: ring_slot.s<TAB>
ring_slot.line.s              ring_slot.s_arcl         ring_slot.s_im
ring_slot.line.s11            ring_slot.s_arcl_unwrap  ring_slot.s_mag
...
```

All of the network parameters are represented internally as complex `numpy.ndarray`. The s-parameters are of shape `(nfreq, nport, nport)`:

```
[11]: np.shape(ring_slot.s)
```

```
[11]: (201, 2, 2)
```

1.4.4 Slicing

You can slice the `Network.s` attribute any way you want.

```
[12]: ring_slot.s[:11,1,0]  # get first 10 values of S21
```

```
[12]: array([0.6134571 +0.36678139j, 0.6218194 +0.36403169j,
       0.63024301+0.36109574j, 0.63872415+0.3579682j ,
       0.64725874+0.35464377j, 0.65584238+0.35111711j,
       0.66447037+0.34738295j, 0.6731377 +0.34343602j,
       0.68183901+0.33927115j, 0.69056862+0.33488321j,
       0.6993205 +0.3302672j ])
```

Slicing by frequency can also be done directly on Network objects like so

```
[13]: ring_slot[0:10] #  Network for the first 10 frequency points
```

```
[13]: 2-Port Network: 'ring slot_subset',  75.0-76.575 GHz, 10 pts, z0=[50.+0.j 50.+0.j]
```

or with a human friendly string,

```
[14]: ring_slot['80-90ghz']
```

```
[14]: 2-Port Network: 'ring slot',  80.075-90.05 GHz, 58 pts, z0=[50.+0.j 50.+0.j]
```

Notice that slicing directly on a Network **returns a Network**. So, a nice way to express slicing in both dimensions is

```
[15]: ring_slot.s11['80-90ghz']
```

```
[15]: 1-Port Network: 'ring slot',  80.075-90.05 GHz, 58 pts, z0=[50.+0.j]
```

1.4.5 Plotting

Amongst other things, the methods of the *Network* class provide convenient ways to plot components of the network parameters,

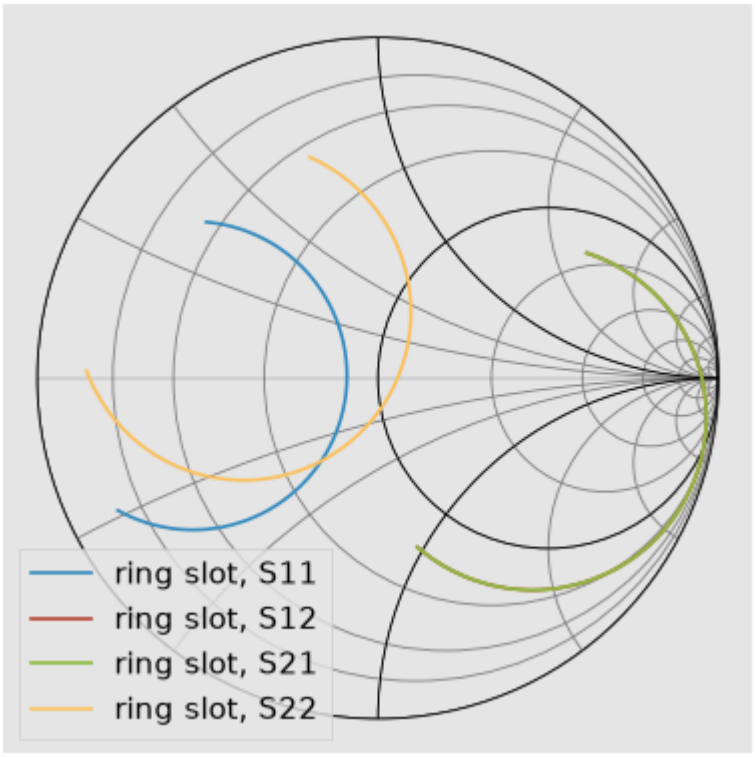
- `Network.plot_s_db` : plot magnitude of s-parameters in log scale
- `Network.plot_s_deg` : plot phase of s-parameters in degrees
- `Network.plot_s_smith` : plot complex s-parameters on Smith Chart
- …

If you would like to use skrf's plot styling,

```
[16]: %matplotlib inline
      rf.stylely()
```

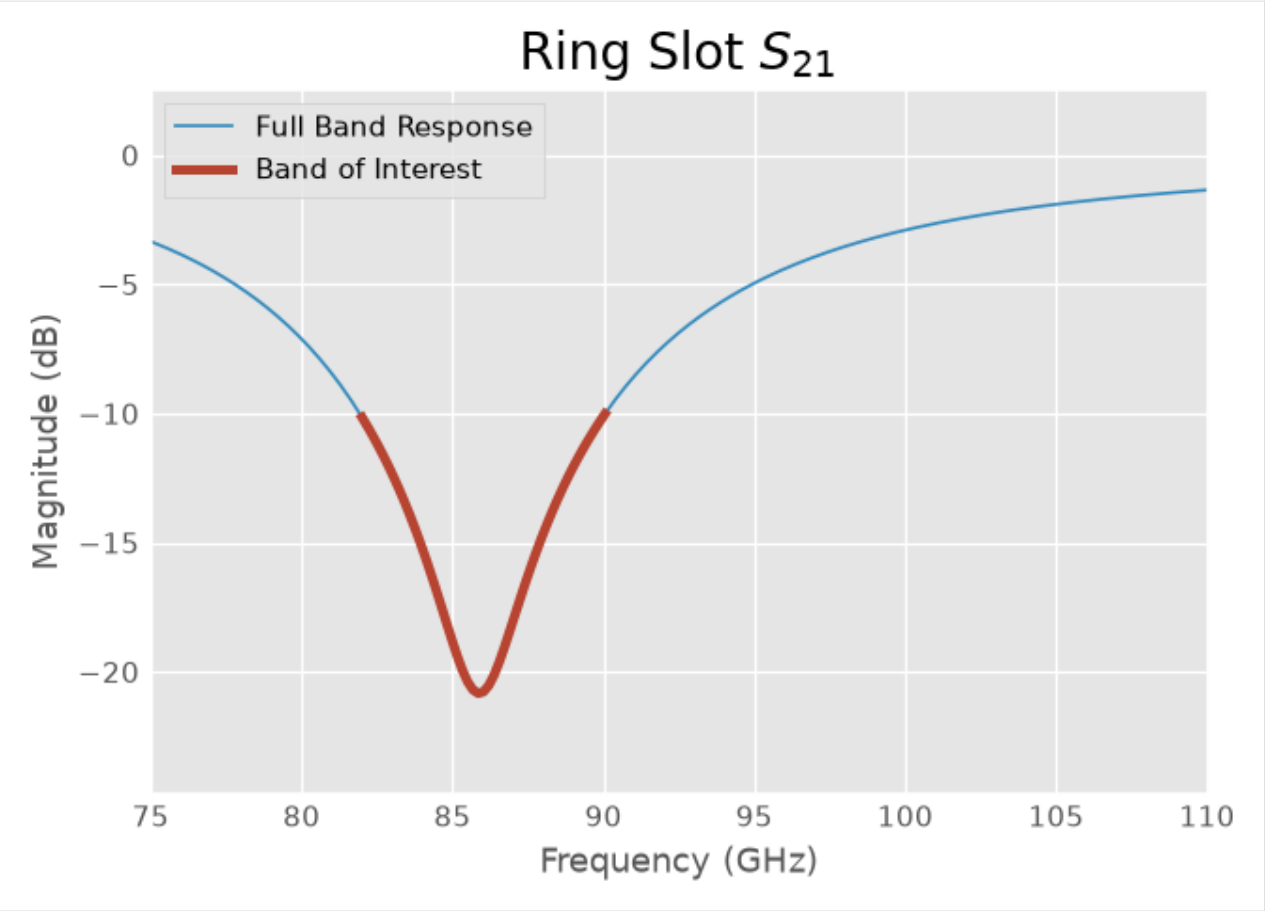
To plot all four s-parameters of the `ring_slot` on the Smith Chart.

[17]:
```
ring_slot.plot_s_smith()
```



Combining this with the slicing features,

[18]:
```
from matplotlib import pyplot as plt

plt.title('Ring Slot $S_{21}$')

ring_slot.s11.plot_s_db(label='Full Band Response')
ring_slot.s11['82-90ghz'].plot_s_db(lw=3,label='Band of Interest')
```



For more detailed information about plotting see *Plotting*.

1.4.6 Arithmetic Operations

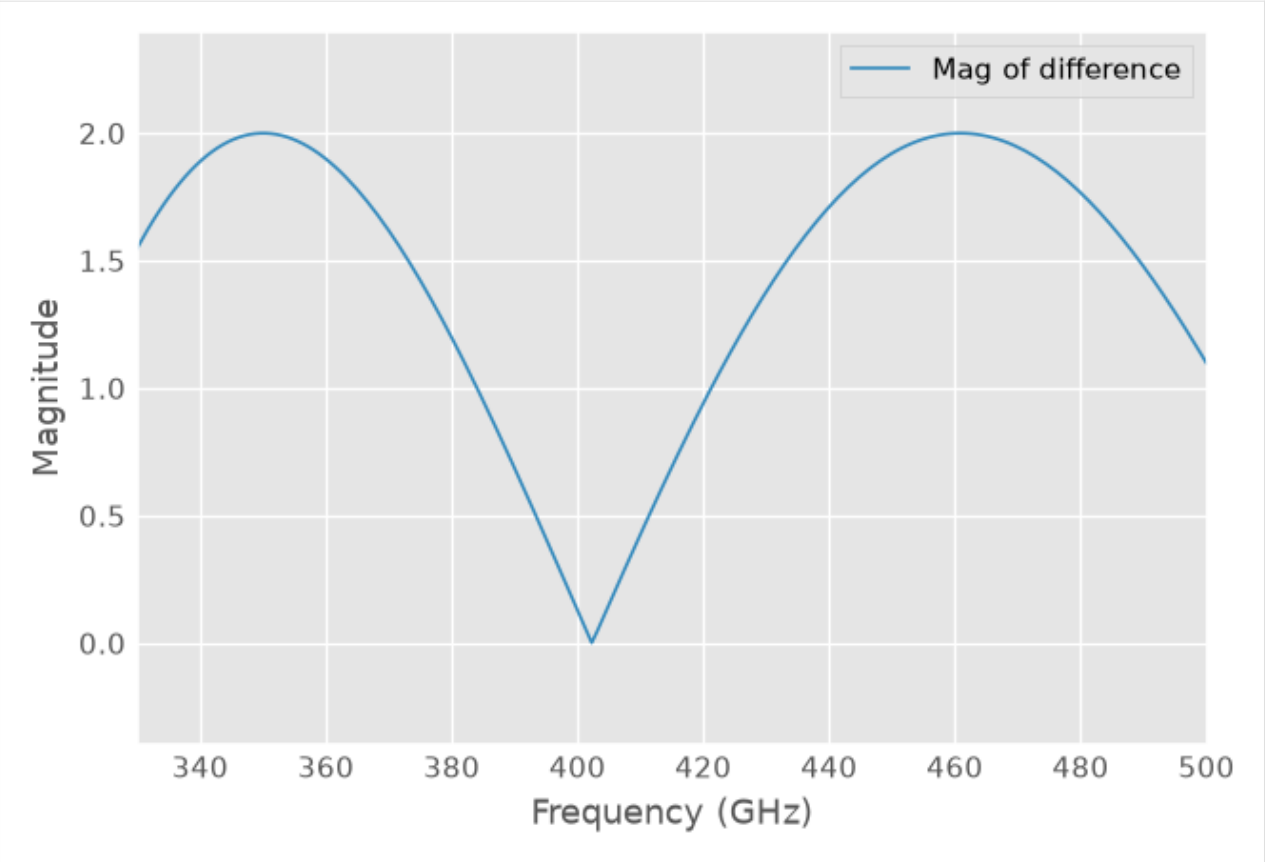
Element-wise mathematical operations on the scattering parameter matrices are accessible through overloaded operators. To illustrate their usage, load a couple Networks stored in the `data` module.

[19]:
```
from skrf.data import wr2p2_delayshort as delayshort
from skrf.data import wr2p2_short as short

short - delayshort
short + delayshort
short * delayshort
short / delayshort
```

[19]:
```
1-Port Network: 'wr2p2,short',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j]
```

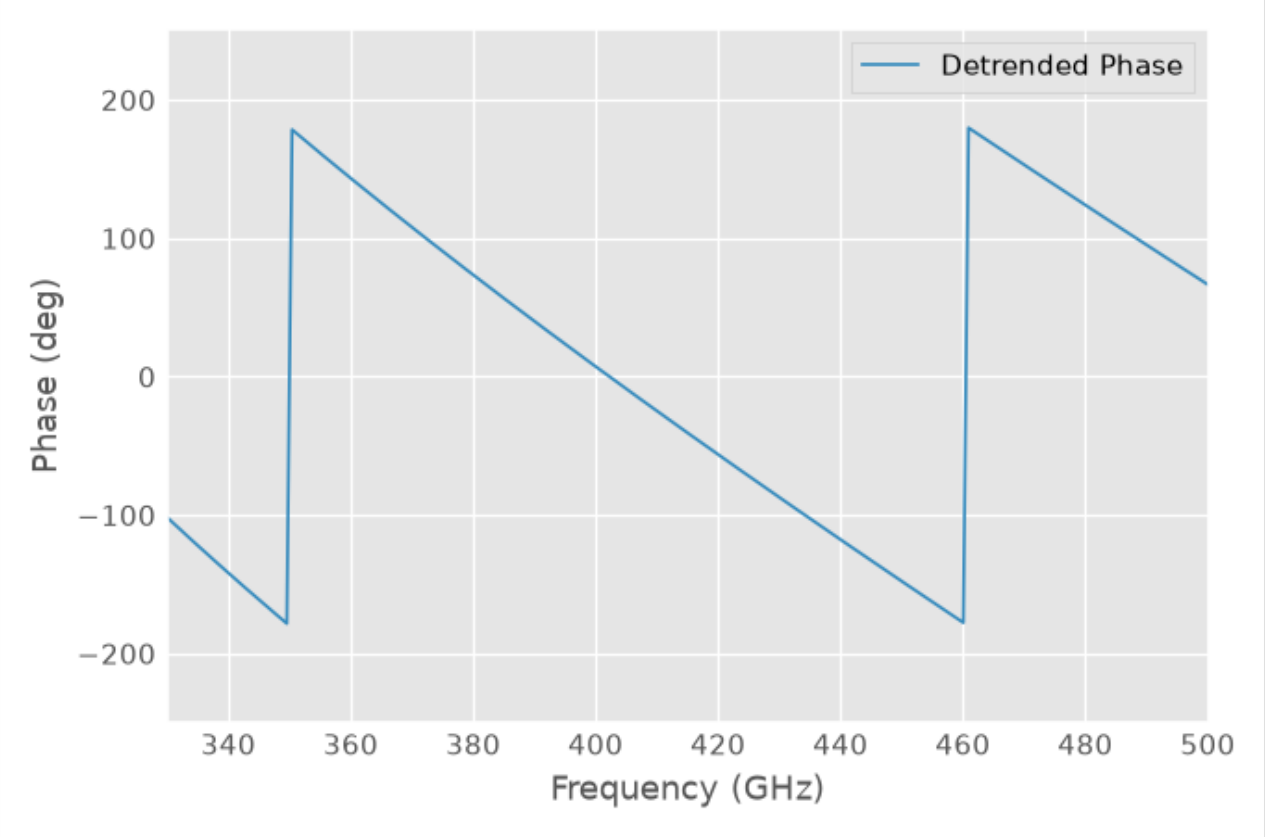
All of these operations return *Network* types. For example, to plot the complex difference between `short` and `delay_short`,

[20]:
```
difference = (short - delayshort)
difference.plot_s_mag(label='Mag of difference')
```



Another common application is calculating the phase difference using the division operator,

[21]:
```
(delayshort/short).plot_s_deg(label='Detrended Phase')
```



Linear operators can also be used with scalars or an `numpy.ndarray` that have the same length as the *Network*.

[22]:
```
hopen = (short**-1)
hopen.s[:3,...]
```

[22]:
```
array([[[1.-0.j]],

       [[1.-0.j]],

       [[1.-0.j]]])
```

[23]:
```
rando =  hopen *rng.uniform(size=len(hopen))
rando.s[:3,...]
```

[23]:
```
array([[[0.26460475+0.j]],

       [[0.82743788+0.j]],

       [[0.36568375+0.j]]])
```

1.4.7 Comparison of Network

Comparison operators also work with networks:

[24]:
```
short == delayshort
```

[24]:
```
False
```

[25]:
```
short != delayshort
```

[25]:
```
True
```

1.4.8 Cascading and De-embedding

Cascading and de-embeding 2-port Networks can also be done through operators. The `cascade` function can be called through the power operator, `**`. To calculate a new network which is the cascaded connection of the two individual Networks `line` and `short`,

[26]:
```
short = rf.data.wr2p2_short
line = rf.data.wr2p2_line
delayshort = line ** short
```

De-embedding can be accomplished by cascading the *inverse* of a network. The inverse of a network is accessed through the property `Network.inv`. To de-embed the `short` from `delay_short`,

[27]:
```
short_2 = line.inv ** delayshort

short_2 == short
```

[27]:
```
True
```

The cascading operator also works for to 2N-port Networks. This is illustrated in *this example on balanced Networks*. For example, assuming you want to cascade three 4-port Network `ntw1`, `ntw2` and `ntw3`, you can use: `resulting_ntw = ntw1 ** ntw2 ** ntw3`. Note that the port scheme assumed by the `**` cascading operator with 4-port networks is:

```
   ntw1    ntw2    ntw3
  +----+  +----+  +----+
0-|0  2|--|0  2|--|0  2|-2
1-|1  3|--|1  3|--|1  3|-3
  +----+  +----+  +----+
```

More documentation on Connecting Network is available here: *Connecting Networks*.

1.4.9 Connecting Multi-ports

skrf supports the connection of arbitrary ports of N-port networks. It accomplishes this using an algorithm called sub-network growth*[1]*, available through the function `connect()`.

As an example, terminating one port of an ideal 3-way splitter can be done like so,

[28]:
```
tee = rf.data.tee
tee
```

[28]:
```
3-Port Network: 'tee',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j 50.+0.j]
```

To connect port `1` of the tee, to port `0` of the delay short,

[29]:
```
terminated_tee = rf.network.connect(tee, 1, delayshort, 0)
terminated_tee
```

[29]:
```
2-Port Network: 'tee',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]
```

Note that this function takes into account port impedances. If two connected ports have different port impedances, an appropriate impedance mismatch is inserted.

More information on connecting Networks is available here: *connecting Networks*.

For advanced connections between many arbitrary N-port Networks, the `Circuit` object is more relevant since it allows defining explicitly the connections between ports and Networks. For more information, please refer to the *Circuit documentation*.

1.4.10 Port names

The ports of a *Network* can be given names with `Network.port_names`. A named port can then be used anywhere a port index is accepted, and the names follow the ports they describe through operations which reorder, drop or combine ports.

> **Version note**
>
> Port names can only be used this way in versions after 2.0.1. In version 2.0.1 and earlier, Network.port_names is a plain attribute which most operations do not keep up to date, and it cannot be used to address a port.

Names are given either when the *Network* is created, or by assigning to the property afterwards. There has to be a name for every port.

[30]:
```
freq = rf.Frequency(1, 10, 11, unit='GHz')
media = rf.media.DefinedGammaZ0(frequency=freq, z0_port=50)

amp = media.attenuator(-10, name='amp')
amp.port_names = ['in', 'out']
amp.port_names
```

[30]:
```
['in', 'out']
```

[31]:
```
s = np.zeros((freq.npoints, 2, 2), dtype=complex)
s[:, 0, 1] = s[:, 1, 0] = 10 ** (-3 / 20)

pad = rf.Network(frequency=freq, s=s, name='pad', port_names=['rf_in', 'rf_out'])
pad.port_names
```

[31]:
```
['rf_in', 'rf_out']
```

A *Network* without named ports has `port_names` set to `None`.

[32]:
```
print(media.attenuator(-3).port_names)
```

```
None
```

Connecting by name

`connect()` takes a port name in place of a port index and the output `Network` keeps the names of the ports which are left over.

[33]:
```
cascade = rf.network.connect(amp, 'out', pad, 'rf_in')
cascade.port_names
```

[33]:
```
['in', 'rf_out']
```

This is the same connection as the one made by indices, names are just another way of selecting a port.

[34]:
```
cascade == rf.network.connect(amp, 1, pad, 0)
```

[34]:
```
True
```

`innerconnect()`, `parallelconnect()`, `subnetwork()` and `renumber()` accept names the same way.

[35]:
```
tee = rf.data.tee.copy()
tee.port_names = ['sum', 'a', 'b']

tee.subnetwork(['a', 'sum']).port_names
```

[35]:
```
['a', 'sum']
```

In `renumber()` a name refers to the port which currently carries it, so the call below swaps the ports named `sum` and `b`.

[36]:
```
tee.renumbered(['sum', 'b'], ['b', 'sum']).port_names
```

[36]:
```
['b', 'a', 'sum']
```

Names follow the ports

A one-port taken out of a network is either a reflection coefficient, which is one of the ports and keeps its name, or a transmission coefficient, which is not any single port of the network and therefore has no name.

[37]:
```
print(amp.s11.port_names) # port 'in'
print(amp.s22.port_names) # port 'out'
print(amp.s21.port_names) # neither port
```

```
['in']
['out']
None
```

The same one-ports can be picked out by name instead of by number.

[38]:
```
amp['in', 'in'].port_names
```

[38]:
```
['in']
```

Mixed mode

`se2gmm()` replaces pairs of single ended ports with a differential and a common mode port. Each new port is named after the pair it is made of, so the names stay unique and say where the port came from.

[39]:
```
line_p = media.line(50, 'deg', name='p')
line_n = media.line(70, 'deg', name='n')

pair = rf.network.concat_ports([line_p, line_n], port_order='second')
pair.port_names = ['in+', 'in-', 'out+', 'out-']
pair.port_names
```

[39]:
```
['in+', 'in-', 'out+', 'out-']
```

[40]:
```
mm = pair.copy()
mm.se2gmm(p=2)
mm.port_names
```

[40]:
```
['d(in+,in-)', 'd(out+,out-)', 'c(in+,in-)', 'c(out+,out-)']
```

The differential insertion loss is then addressable by name.

[41]:
```
mm['d(in+,in-)', 'd(out+,out-)'].s_db[:, 0, 0]
```

[41]:
```
array([-0.13297082, -0.13297082, -0.13297082, -0.13297082, -0.13297082,
       -0.13297082, -0.13297082, -0.13297082, -0.13297082, -0.13297082,
       -0.13297082])
```

`gmm2se()` reverses the conversion and restores the single ended names.

[42]:
```
mm.gmm2se(p=2)
mm.port_names
```

[42]:
```
['in+', 'in-', 'out+', 'out-']
```

Duplicate names

Several ports may share a name, which happens for instance when a Touchstone file names them that way, or when `connect()` joins two networks which both have a port called `in`. Such a name cannot be used to address a port, since it does not identify one, and those ports have to be given by index instead.

[43]:
```
amp2 = amp.copy()
amp2.name = 'amp2'

# the two ports named 'out' are the ones consumed, both 'in' ports are left
back_to_back = rf.network.connect(amp, 'out', amp2, 'out')
back_to_back.port_names
```

[43]:
```
['in', 'in']
```

Asking for a port by an ambiguous name raises rather than silently returning the first match.

[44]:
```
try:
    back_to_back['in', 'in']
except ValueError as err:
    print(err)
```

```
Port name 'in' is ambiguous, network has ports ['in', 'in']. Use the port index instead
```

1.4.11 Interpolation and Concatenation

A common need is to change the number of frequency points of a *Network*. To use the operators and cascading functions the networks involved must have matching frequencies, for instance. If two networks have different frequency information, then an error will be raised,

[45]:
```
from skrf.data import wr2p2_line1 as line1

line1
```

[45]:
```
2-Port Network: 'wr2p2,line1',  330.0-500.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]
```

```
line1+line
```

(continues on next page)

(continued from previous page)

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-49-82040f7eab08> in <module>()
----> 1 line1+line

/home/alex/code/scikit-rf/skrf/network.py in __add__(self, other)
    500
    501         if isinstance(other, Network):
--> 502             self.__compatible_for_scalar_operation_test(other)
    503             result.s = self.s + other.s
    504         else:

/home/alex/code/scikit-rf/skrf/network.py in __compatible_for_scalar_operation_test(self,
↪ other)
    701         '''
    702         if other.frequency  != self.frequency:
--> 703             raise IndexError('Networks must have same frequency. See `Network.
↪interpolate`')
    704
    705         if other.s.shape != self.s.shape:

IndexError: Networks must have same frequency. See `Network.interpolate`
```

This problem can be solved by interpolating one of Networks along the frequency axis using `Network.resample`.

[46]:
```
line1.resample(201)
line1
```

[46]:
```
2-Port Network: 'wr2p2,line1',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]
```

And now we can do things

[47]:
```
line1 + line
```

[47]:
```
2-Port Network: 'wr2p2,line1',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]
```

You can also interpolate from a `Frequency` object. For example,

[48]:
```
line.interpolate(line1.frequency)
```

[48]:
```
2-Port Network: 'wr2p2,line',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]
```

A related application is the need to combine Networks which cover different frequency ranges. Two Networks can be concatenated (aka stitched) together using `stitch`, which concatenates networks along their frequency axis. To combine a WR-2.2 Network with a WR-1.5 Network,

[49]:
```
from skrf.data import wr1p5_line, wr2p2_line

big_line = rf.network.stitch(wr2p2_line, wr1p5_line)
big_line
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪frequency.py:287: InvalidFrequencyWarning: Frequency values are not monotonously␣
↪increasing!
```

(continues on next page)

(continued from previous page)

```
To get rid of the invalid values call `drop_non_monotonic_increasing`
  temp_freq.check_monotonic_increasing()
```

[49]:
```
2-Port Network: 'wr2p2,line',  330.0-750.0 GHz, 402 pts, z0=[50.+0.j 50.+0.j]
```

1.4.12 Reading and Writing

For long term data storage, **skrf** has support for reading and partial support for writing touchstone file format. Reading is accomplished with the Network initializer as shown above, and writing with the method `Network.write_touchstone()`.

For **temporary** data storage, **skrf** object can be pickled with the functions `skrf.io.general.read` and `skrf.io.general.write`. The reason to use temporary pickles over touchstones is that they store all attributes of a network, while touchstone files only store partial information.

[50]:
```
rf.io.write('data/myline.ntwk',line) # write out Network using pickle
```

[51]:
```
ntwk = Network('data/myline.ntwk') # read Network using pickle
```

> **Warning**
>
> Pickling methods can't support long term data storage because they require the structure of the object being written to remain unchanged. something that cannot be guaranteed in future versions of skrf. (see http://docs.python.org/2/library/pickle.html)

Frequently there is an entire directory of files that need to be analyzed. `rf.read_all` creates Networks from all files in a directory quickly. To load all **skrf** files in the `data/` directory which contain the string `'wr2p2'`.

[52]:
```
dict_o_ntwks = rf.io.read_all(rf.data.pwd, contains = 'wr2p2')
dict_o_ntwks
```

[52]:
```
{'wr2p2,delayshort': 1-Port Network: 'wr2p2,delayshort',  330.0-500.0 GHz, 201 pts,␣
↪z0=[50.+0.j],
 'wr2p2,line': 2-Port Network: 'wr2p2,line',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.
↪+0.j],
 'wr2p2,line1': 2-Port Network: 'wr2p2,line1',  330.0-500.0 GHz, 101 pts, z0=[50.+0.j 50.
↪+0.j],
 'wr2p2,short': 1-Port Network: 'wr2p2,short',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j]}
```

Other times you know the list of files that need to be analyzed. `rf.read_all` also accepts a files parameter. This example file list contains only files within the same directory, but you can store files however your application would benefit from.

[53]:
```
import os

dict_o_ntwks_files = rf.io.read_all(
    files=[os.path.join(rf.data.pwd, test_file) for test_file in ['ntwk1.s2p', 'ntwk2.s2p
↪']]
)
dict_o_ntwks_files
```

[53]:
```
{'ntwk1': 2-Port Network: 'ntwk1',  1.0-10.0 GHz, 91 pts, z0=[50.+0.j 50.+0.j]}
```

1.4.13 Other Parameters

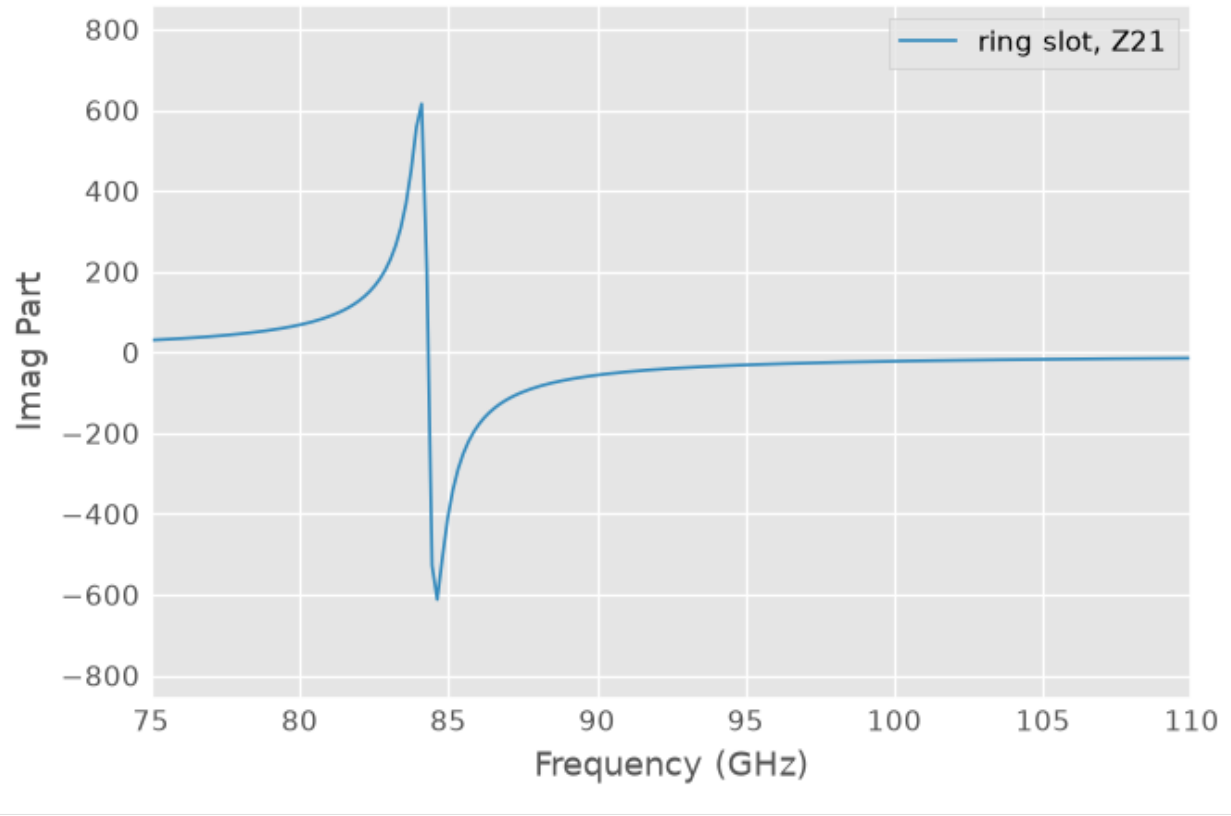
This tutorial focuses on s-parameters, but other network representations are available as well. Impedance and Admittance Parameters can be accessed through the parameters `Network.z` and `Network.y`, respectively. Scalar components of complex parameters, such as `Network.z_re`, `Network.z_im` and plotting methods are available as well.

Other parameters are only available for 2-port networks, such as wave cascading parameters (`Network.t`), and ABCD-parameters (`Network.a`) or Hybrid parameters (`Network.h`).

[54]:
```
ring_slot.z[:3,...]
```

[54]:
```
array([[[0.88442687+28.15350224j, 0.94703504+30.46757222j],
        [0.94703504+30.46757222j, 1.0434417 +43.45766805j]],

       [[0.91624901+28.72415928j, 0.98188607+31.09594438j],
        [0.98188607+31.09594438j, 1.08168411+44.17642274j]],

       [[0.94991736+29.31694632j, 1.01876516+31.74874257j],
        [1.01876516+31.74874257j, 1.12215451+44.92215712j]]])
```

[55]:
```
ring_slot.plot_z_im(m=1,n=0)
```



1.4.14 References

[1] Compton, R.C.; , “Perspectives in microwave circuit analysis,” Circuits and Systems, 1989., Proceedings of the 32nd Midwest Symposium on , vol., no., pp.716-718 vol.2, 14-16 Aug 1989. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=101955&isnumber=3167

[ ]:

1.5 Connecting Networks

scikit-rf supports the connection of arbitrary ports of N-port networks. It accomplishes this using an algorithm called sub-network growth*[1]*, available through the function `connect()`. Note that this function takes into account port impedances. If two connected ports have different port impedances, an appropriate impedance mismatch is inserted. This capability is illustrated here with situations often encountered.

[1]:
```
import skrf as rf
```

1.5.1 Cascading 2-port and 1-port Networks

A common problem is to connect two Networks one to the other, also known as cascading Networks, which creates a new Network. The figure below illustrates sile simple situations, where the port numbers are identified in gray:

or,

Let's illustrate this by connecting a transmission line (2-port Network) to a short-circuit (1-port Network) to create a delay short (1-port Network):

Cascading Networks being a frequent operation, it can done conveniently through the ** operator or with the `cascade` function:

[2]:
```
line = rf.data.wr2p2_line  # 2-port
short = rf.data.wr2p2_short  # 1-port

delayshort = line ** short  # --> 1-port Network
print(delayshort)
```

```
1-Port Network: 'wr2p2,line',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j]
```

or, equivalently using the `cascade()` function:

[3]:
```
delayshort2 = rf.network.cascade(line, short)
print(delayshort2 == delayshort)  # the result is the same
```

```
True
```

It is of course possible to connect two 2-port Networks together using the `connect()` function. The `connect()` function requires the Networks and the port numbers to connect together. In our example, the port 1 of the line is connected to the port 0 of the short:

[4]:
```
delayshort3 = rf.network.connect(line, 1, short, 0)
print(delayshort3 == delayshort)
```

```
True
```

One often needs to cascade a chain Networks together:

or,

which can be realized using chained ** or the convenient function `cascade_list`:

[5]:
```
line1 = rf.data.wr2p2_line  # 2-port
line2 = rf.data.wr2p2_line  # 2-port
line3 = rf.data.wr2p2_line  # 2-port
line4 = rf.data.wr2p2_line  # 2-port
short = rf.data.wr2p2_short  # 1-port

chain1 = line1 ** line2 ** line3 ** line4 ** short

chain2 = rf.network.cascade_list([line1, line2, line3, line4, short])

print(chain1 == chain2)
```

```
True
```

1.5.2 Cascacing 2N-port Networks

The cascading operator ** also works for to 2N-port Networks, width the following port scheme:

It also works for multiple 2N-port Network. For example, assuming you want to cascade three 4-port Network `ntw1`, `ntw2` and `ntw3`, you can use:

```
resulting_ntw = ntw1 ** ntw2 ** ntw3
```

This is illustrated in *this example on balanced Networks*.

1.5.3 Cascading Multi-port Networks

To make specific connections between multi-port Networks, three solutions are available, which mostly depends of the complexity of the circuit one wants to build:

- For reduced number of connection(s): the `connect()` function
- For intermediate complexity, where you might need to connect multiple Networks in parallel to the same intersection, we offer the `parallelconnect()` method. This method provides a balance between the simplicity of `connect()` and the flexibility of `Circuit` object. For more information, please refer to the `` `paralleconnect() documentation <../api/generated/skrf.network.paralleconnect.html#skrf.network.paralleconnect>`__
- For more advanced connections between many arbitrary N-port Networks, the `Circuit` object is more relevant since it allows defining explicitly the connections between ports and Networks. For more information, please refer to the *Circuit documentation*.

As an example, terminating one of the port of an a 3-port Network, such as an ideal 3-way splitter:

can be done like:

[6]:
```
tee = rf.data.tee
```

To connect port `1` of the tee, to port `0` of the delay short,

[7]:
```
terminated_tee = rf.network.connect(tee, 1, delayshort, 0)
terminated_tee
```

[7]:
```
2-Port Network: 'tee',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]
```

`parallelconnect()` method also could handle this situation with just a slight change in syntax.

[8]:
```
terminated_tee_par = rf.network.parallelconnect([tee, delayshort], [1, 0])
terminated_tee_par
```

[8]:
```
2-Port Network: '',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]
```

In the previous example, the port #2 of the 3-port Network `tee` becomes the port #1 of the resulting 2-port Network.

1.5.4 Multiple Connections of Multi-port Networks

Keeping track of the port numbering when using multiple time the `connect` function can be tedious (this is the reason why the *Circuit object* can be simpler to use).

Let's illustrate this with the following example: connecting the port #1 of a tee-junction (3-port) to the port #0 of a transmission line (2-port):

To keep track of the port scheme after the connection operation, let's change the port characteristic impedances (in red in the figure above):

[9]:
```
tee.z0 = [1, 2, 3]
line.z0 = [10, 20]
# the resulting network is:
rf.network.connect(tee, 1, line, 0)
```

[9]:
```
3-Port Network: 'tee',  330.0-500.0 GHz, 201 pts, z0=[ 1.+0.j 20.+0.j  3.+0.j]
```

1.5.5 Networks Connections from the Intersection Perspective

In the previous example, we briefly introduced the `parallelconnect()` method. In this section, We will detail the application scenarios of the `parallelconnect()` method with multiple examples and provide recommendations for comparing the three solutions.

Firstly, let's consider the simplest example: inner-connecting any two ports within an N-port `Network`. This will result in an (N-2)-port `Network`.

Here, we can use the `innerconnect()` method to connect any two ports within the N-port `Network`

```
# Innerconnect the m'th and n'th ports of the N-Port Network
inner_network = rf.network.innerconnect(nport_network, m, n)
```

`parallelconnect()` method could handle inner-connect situation like this:

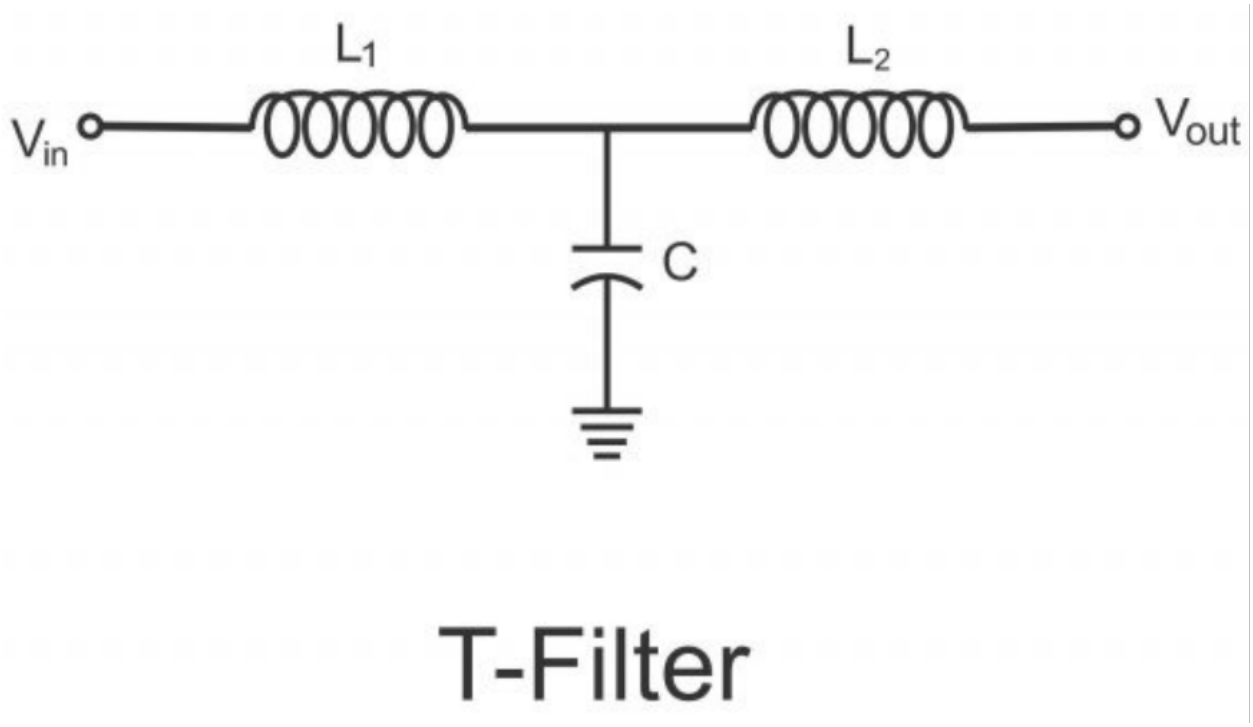
```
inner_network_par = rf.network.parallelconnect(nport_network, [[m, n]])
```

If you anticipate the need to inner-connect more than 2 ports, the `innerconnect()` method will not suffice for this task. You'll need to look into using `tee` (`splitter`) or `Circuit` object to achieve your goal. However, `parallelconnect()` offers a straightforward solution for such cases,

```
# An example of inner-connect a list of ports of a N-port Network
ports_list = [[m, n, ..., y, z]]
inner_network2_par = rf.network.parallelconnect(nport_network, ports_list)
```

The second example involves connecting two multi-port Networks. As this cases has been previously demonstrated, we will not delve into it again here.

Moving on, let's consider the parallel connection of multiple multi-port `Networks`. A common application of this is in the construction of a `T-type filter circuit`, figure taken from electroniclinic.com



Let's ignore the specific details and just compare the three methods in implementing this example, you can use:

```
# 1. Connect() method with tee
t_type_filter_ntwk = rf.network.connect(L1, 1, tee, 0)
t_type_filter_ntwk = rf.network.connect(t_type_filter_ntwk, 1, C, 0)
t_type_filter_ntwk = rf.network.connect(t_type_filter_ntwk, 2, L2, 0)
t_type_filter_ntwk

# 2. Circuit object method
cnx = [
    [(port1, 0), (L1, 0)],
    [(port2, 0), (C , 1)],
    [(port3, 0), (L2, 1)],
    [(L1, 1), (C, 0), (L2, 0)]
]
t_type_filter_ckt = rf.cicuit.Circuit(cnx)
t_type_filter_ntwk = t_type_filter_ckt.network
t_type_filter_ntwk

# 3. Parallelconnect() method
t_type_filter_ntwk = rf.network.parallelconnect([L1, C, L2], [1, 0, 0])
t_type_filter_ntwk
```

It can be seen that the `parallelconnect()` method can realize the construction of `T-type filter` circuit very clearly and concisely.

In this section, we compared the `parallelconnect()` method and compared it with the `connect()/innerconnect()` method and the `Circuit` object for connecting `Networks` from an intersection perspective.

The `parallelconnect()` method excels in handling multiple parallel connections with minimal effort, making it ideal for scenarios requiring simplicity and efficiency. In contrast, the `connect()/innerconnect()` method is better suited for simpler, sequential connections, while the `Circuit` object is for complex, multi-layered connections with greater control.

From the intersection perspective, the `Circuit` object is best for managing complex networks with multiple intersections, a task that exceeds the capabilities of other methods, which are designed for single intersection scenarios. And `innerconnect()/connect()` methods are limited to handling connections between individual or pairs of Networks, `parallelconnect()` removes the restriction on the number of Networks and can efficiently establish connections for multiple Networks in a single line of code, making it particularly advantageous for complex circuits with parallel connections at multiple points.

1.5.6 References

[1] Compton, R.C.; , "Perspectives in microwave circuit analysis," Circuits and Systems, 1989., Proceedings of the 32nd Midwest Symposium on , vol., no., pp.716-718 vol.2, 14-16 Aug 1989. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=101955&isnumber=3167

[ ]:

Download This Notebook: `Plotting.ipynb`

1.6 Plotting

1.6.1 Introduction

This tutorial describes **skrf's** plotting features. If you would like to use skrf's matplotlib interface with skrf styling, start with this

[1]:
```
%matplotlib inline
```

[2]:
```
from matplotlib import pyplot as plt  # for advanced smith chart only

import skrf as rf
```

1.6.2 Plotting Methods

Plotting functions are implemented as methods of the `Network` class.

- `Network.plot_s_re`
- `Network.plot_s_im`
- `Network.plot_s_mag`
- `Network.plot_s_db`
- ...

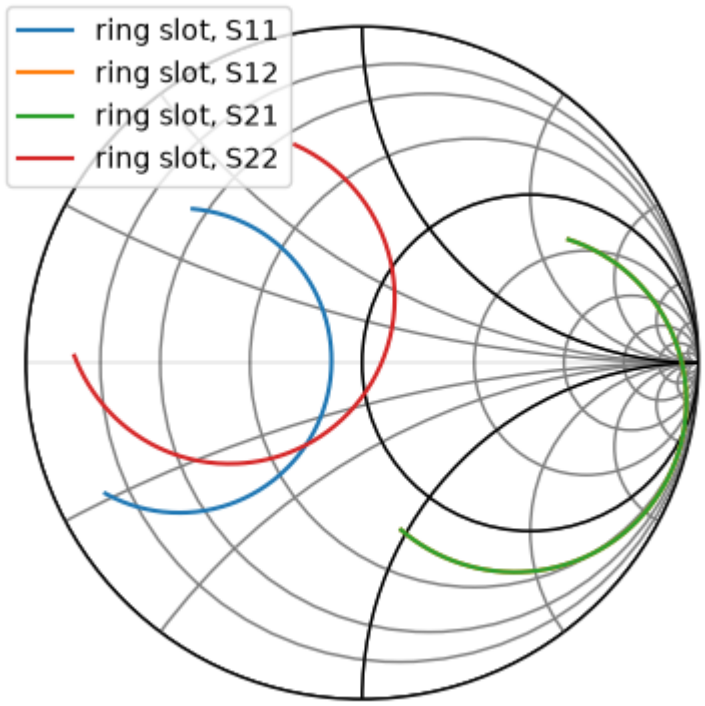
Similar methods exist for Impedance (`Network.z`) and Admittance Parameters (`Network.y`),

- `Network.plot_z_re`
- `Network.plot_z_im`
- ...
- `Network.plot_y_re`
- `Network.plot_y_im`
- ...

1.6.3 Smith Chart

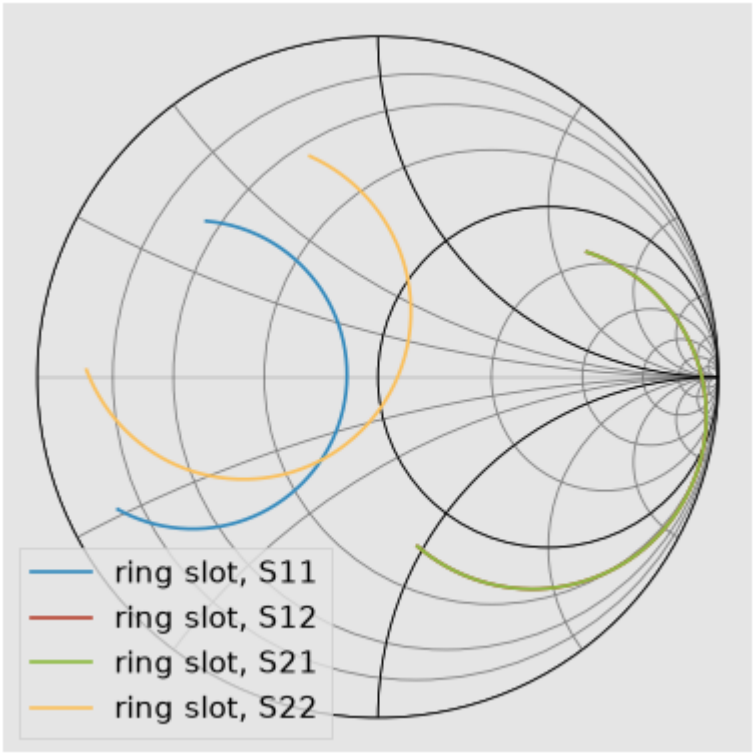
As a first example, load a *Network* and plot all four s-parameters on the Smith chart.

[3]:
```
from skrf import Network

ring_slot = Network('data/ring slot.s2p')
ring_slot.plot_s_smith()
```

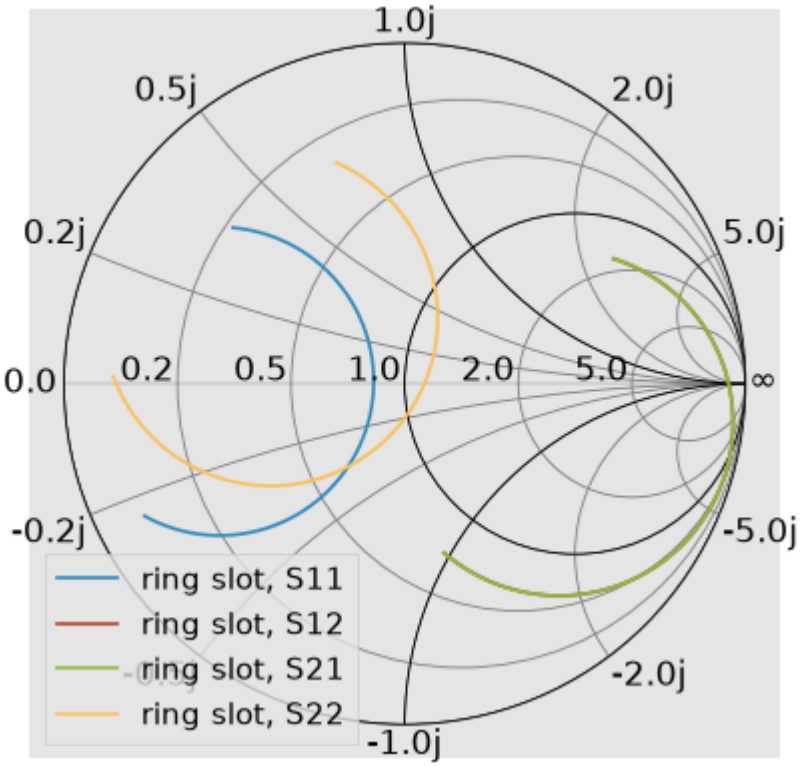


scikit-rf includes a convenient command to make nicer figures quick:

[4]:
```
rf.stylely()  # nicer looking. Can be configured with different styles
ring_slot.plot_s_smith()
```

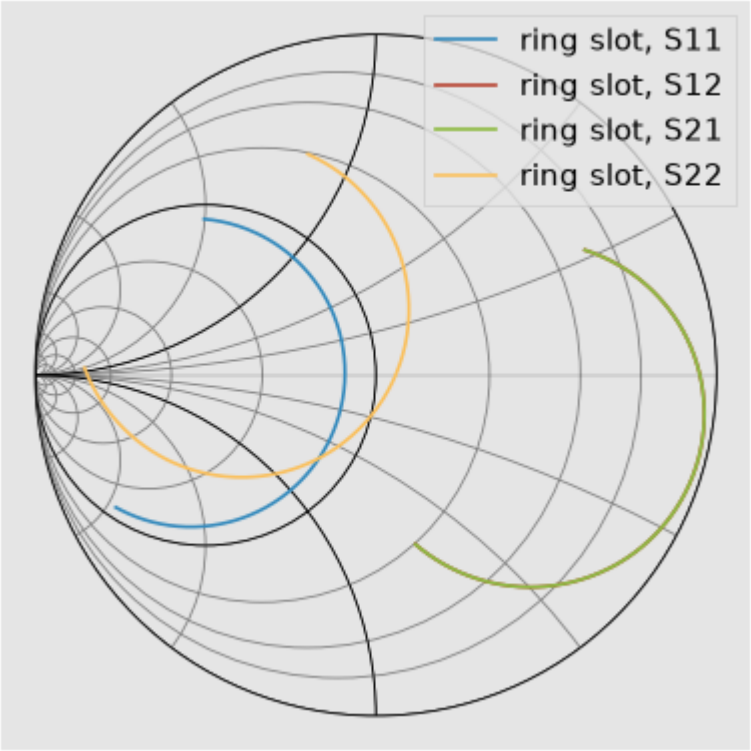


[5]:
```
ring_slot.plot_s_smith(draw_labels=True)
```



Another common option is to draw admittance contours, instead of impedance. This is controlled through the `chart_type` argument.

[6]:
```
ring_slot.plot_s_smith(chart_type='y')
```



Advanced Smith Chart with markers

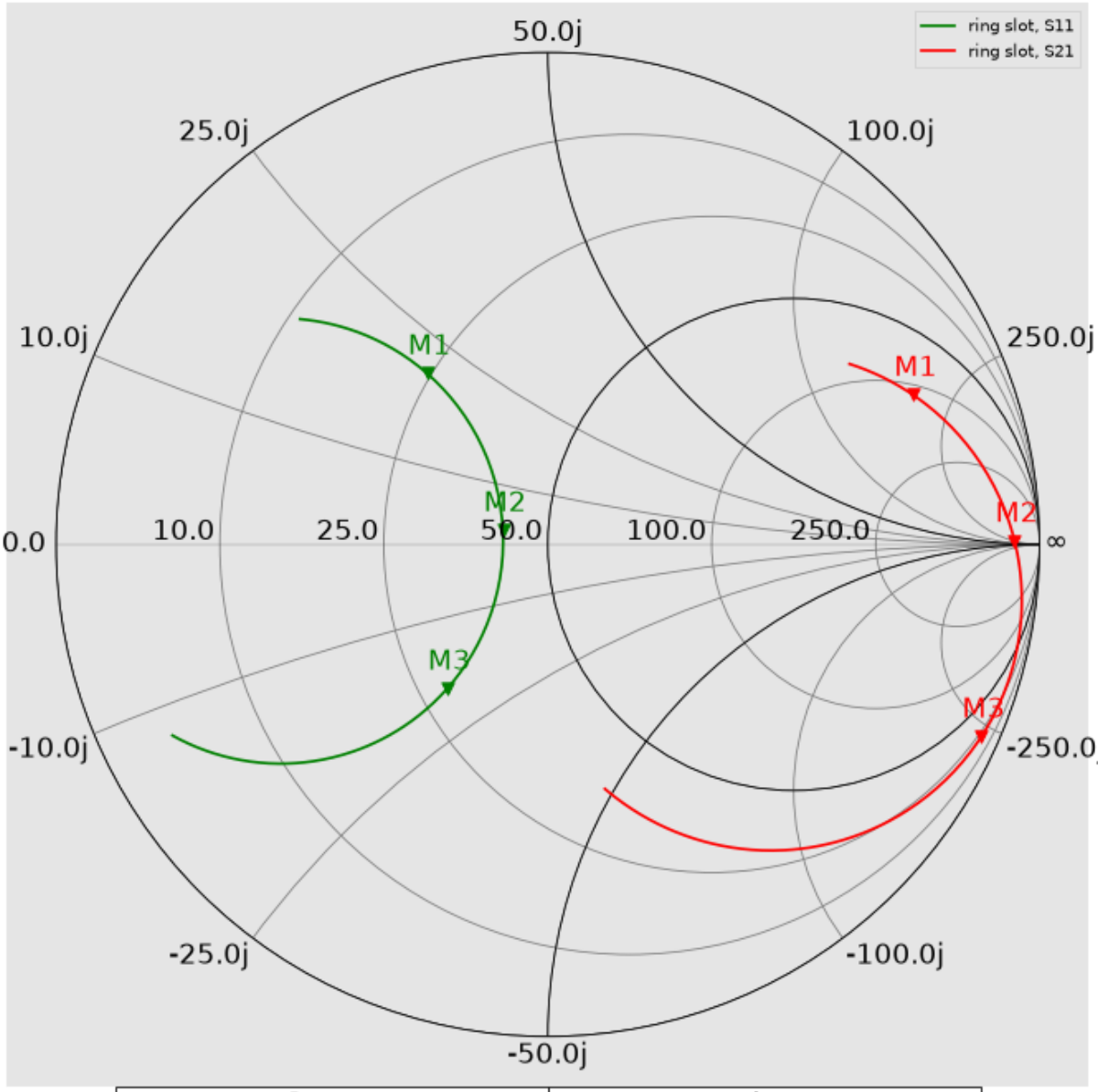
The Smith Chart is a convenient plot that shows complex numbers with real and imaginary parts from zero to infinity with a zoom on a real reference in the center region. It is, however, not straightforward to figure out what point is related to which frequency because there is no frequency axis. A common workaround is to provide some frequency markers related to a table.

[7]:
```
# prepare markers
lines = [
    {'marker_idx': [30, 60, 90], 'color': 'g', 'm': 0, 'n': 0, 'ntw': ring_slot},
    {'marker_idx': [15, 45, 75], 'color': 'r', 'm': 1, 'n': 0, 'ntw': ring_slot},
]

# prepare figure
fig, ax = plt.subplots(1, 1, figsize=(7,8))

# impedance smith chart
rf.plotting.smith(ax = ax, draw_labels = True, ref_imm = 50.0, chart_type = 'z')

# plot data
col_labels = ['Frequency', 'Real Imag']
row_labels = []
row_colors = []
```

(continues on next page)

(continued from previous page)

```
cell_text = []
for l in lines:
    m = l['m']
    n = l['n']
    l['ntw'].plot_s_smith(m=m, n=n, ax = ax, color=l['color'])
    #plot markers
    for i, k in enumerate(l['marker_idx']):
        x = l['ntw'].s.real[k, m, n]
        y = l['ntw'].s.imag[k, m, n]
        z = l['ntw'].z[k, m, n]
        z = f'{z.real:.4f} + {z.imag:.4f}j ohm'
        f = l['ntw'].frequency.f_scaled[k]
        f_unit = l['ntw'].frequency.unit
        row_labels.append(f'M{i + 1}')
        row_colors.append(l['color'])
        ax.scatter(x, y, marker = 'v', s=20, color=l['color'])
        ax.annotate(row_labels[-1], (x, y), xytext=(-7, 7), textcoords='offset points',␣
↪color=l['color'])
        cell_text.append([f'{f:.3f} {f_unit}', z])
leg1 = ax.legend(loc="upper right", fontsize= 6)

# plot the table
the_table = ax.table(cellText=cell_text,
                     colWidths=[0.4] * 2,
                     rowLabels=row_labels,
                     colLabels=col_labels,
                     rowColours=row_colors,
                     loc='bottom')
the_table.auto_set_font_size(False)
the_table.set_fontsize(6)
#the_table.scale(1.5, 1.5)
```



	Frequency	Real Imag
M1	80.250 GHz	4.3030 + 66.3993j ohm
M2	85.500 GHz	45.9921 + -226.3701j ohm
M3	90.750 GHz	1.5410 + -44.4414j ohm
M1	77.625 GHz	1.7870 + 43.3261j ohm
M2	82.875 GHz	35.5576 + 203.8466j ohm
M3	88.125 GHz	4.9655 + -82.3531j ohm

Advanced Smith Chart with background

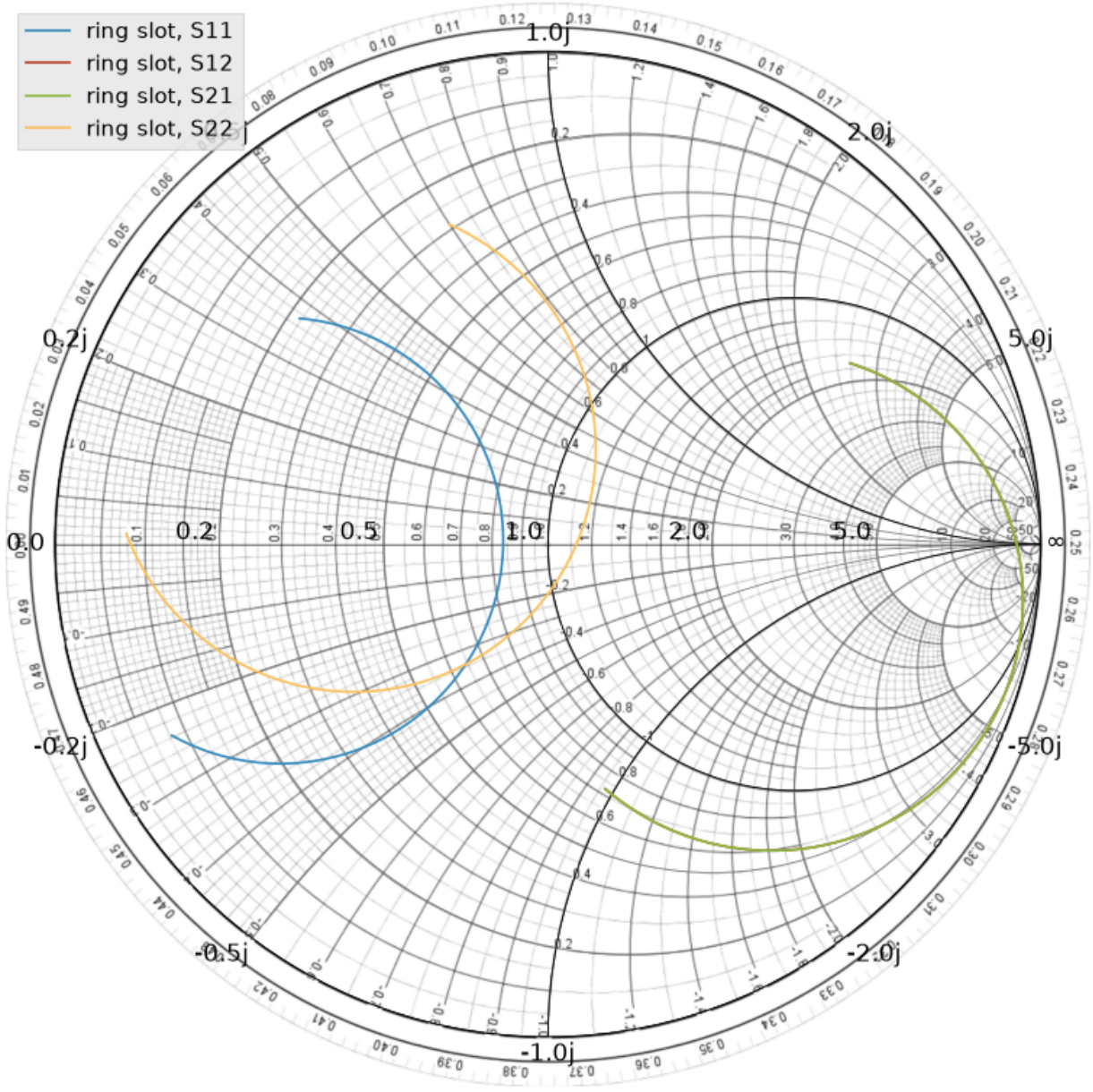
The Smith Chart can be drawn above a high-resolution Smith Chart background (or another image upon your fantasy).

[8]:
```
# prepare figure
fig, ax = plt.subplots(1, 1, figsize=(8,8))
```

(continues on next page)

(continued from previous page)

```
background = plt.imread('figures/smithchart.png')

# tweak background position
ax.imshow(background, extent=[-1.185, 1.14, -1.13, 1.155])
rf.plotting.smith(ax = ax, draw_labels = True, ref_imm = 1.0, chart_type = 'z')

ring_slot.plot_s_smith(ax = ax)
```



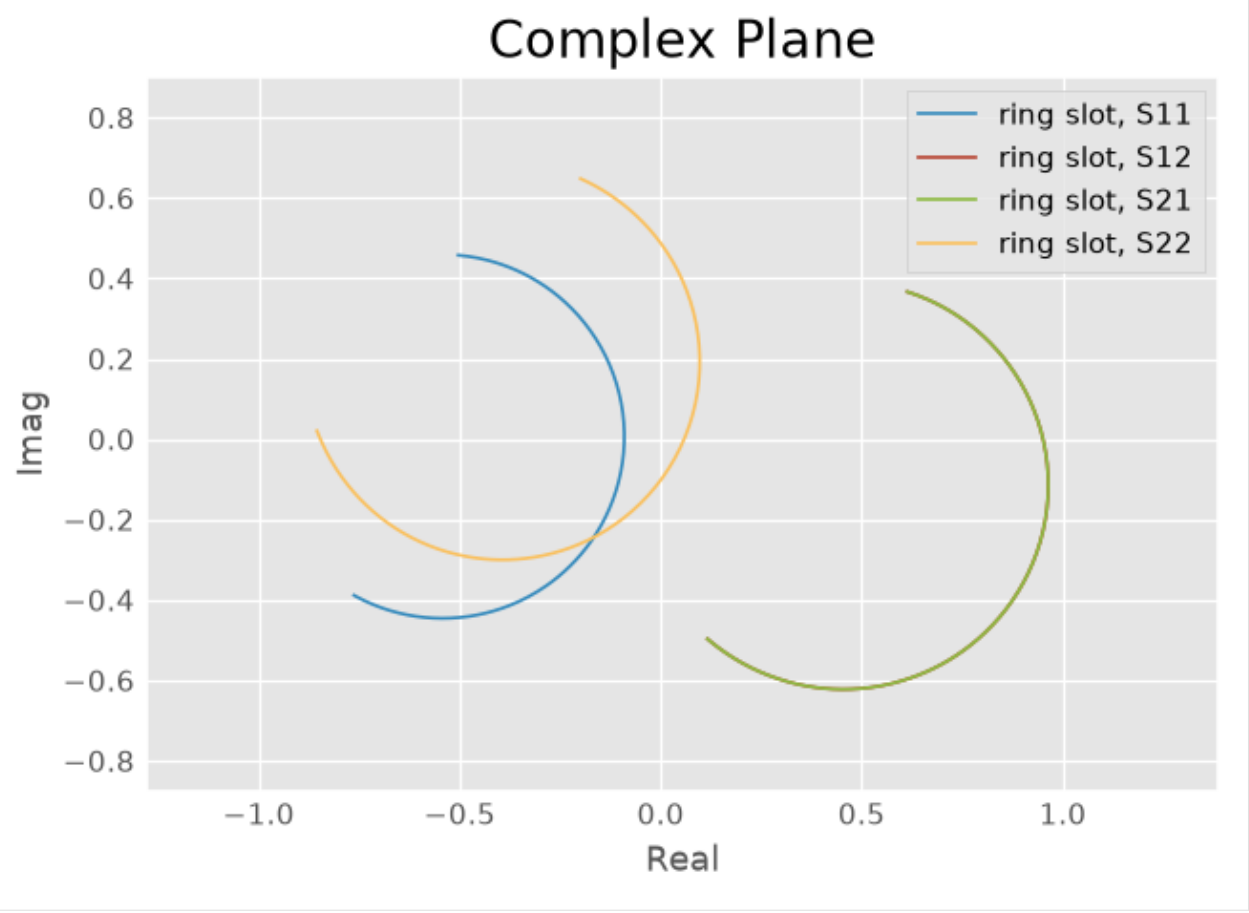
See `skrf.plotting.smith()` for more info on customizing the Smith Chart.

1.6.4 Complex Plane

Network parameters can also be plotted in the complex plane without a Smith Chart through `Network.plot_s_complex`.

[9]:
```
from matplotlib import pyplot as plt

ring_slot.plot_s_complex()

plt.axis('equal') # otherwise circles won't be circles
```

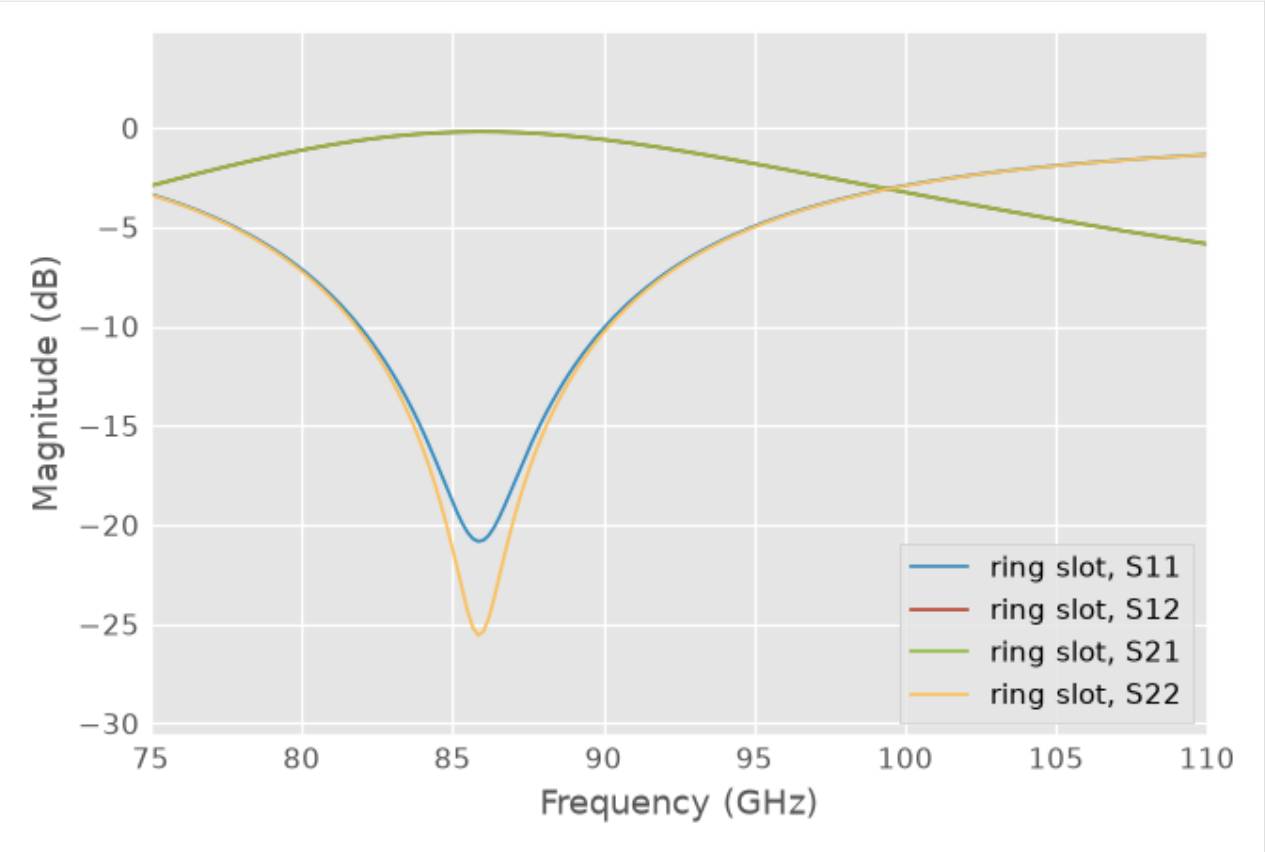
[9]:
```
(np.float64(-0.855165798772),
 np.float64(0.963732138327),
 np.float64(-0.8760764348472001),
 np.float64(0.9024032182652001))
```



1.6.5 Log-Magnitude

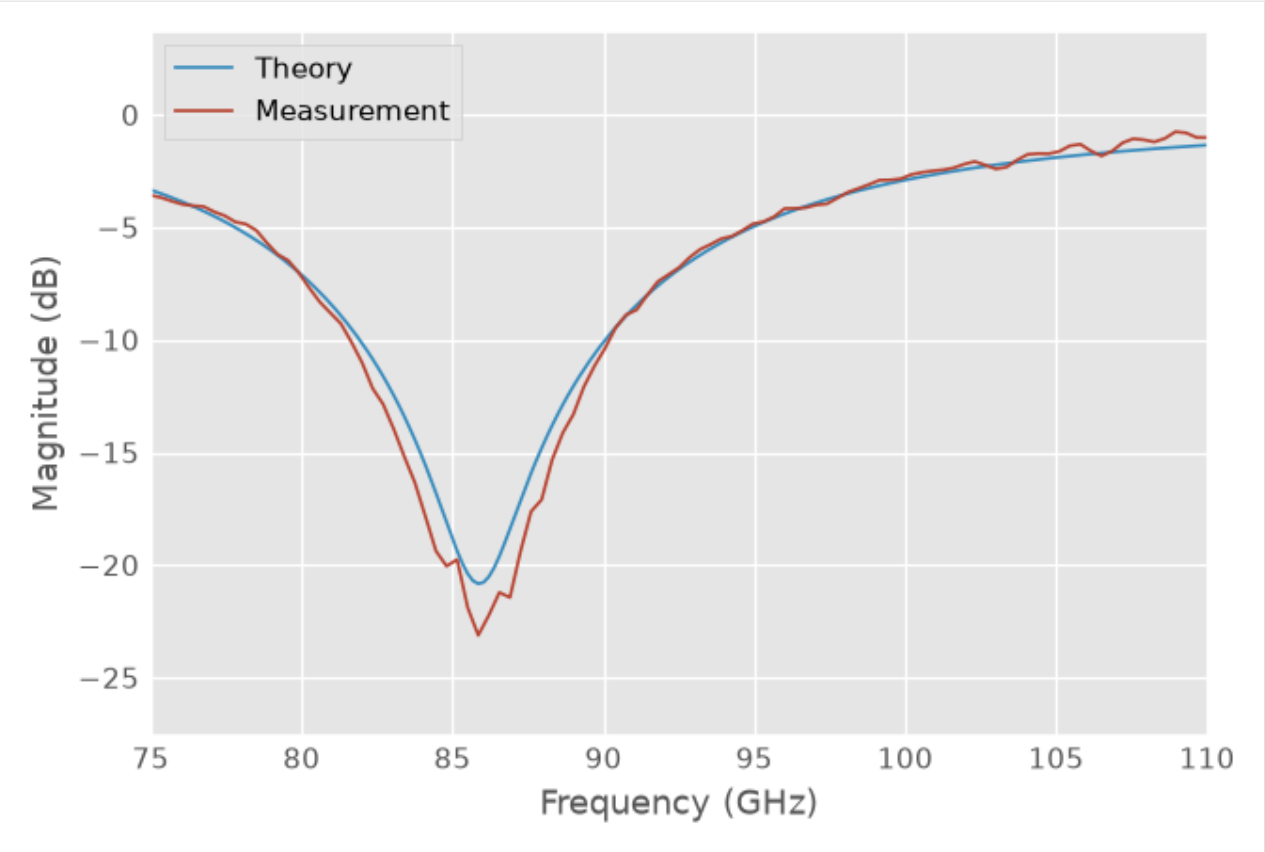
Scalar components of the complex network parameters can be plotted vs frequency as well. To plot the log-magnitude of the s-parameters vs. frequency,

[10]:
```
ring_slot.plot_s_db()
```



When no arguments are passed to the plotting methods, all parameters are plotted. Single parameters can be plotted by passing indices `m` and `n` to the plotting commands (indexing start from 0). Comparing the simulated reflection coefficient off the ring slot to a measurement,

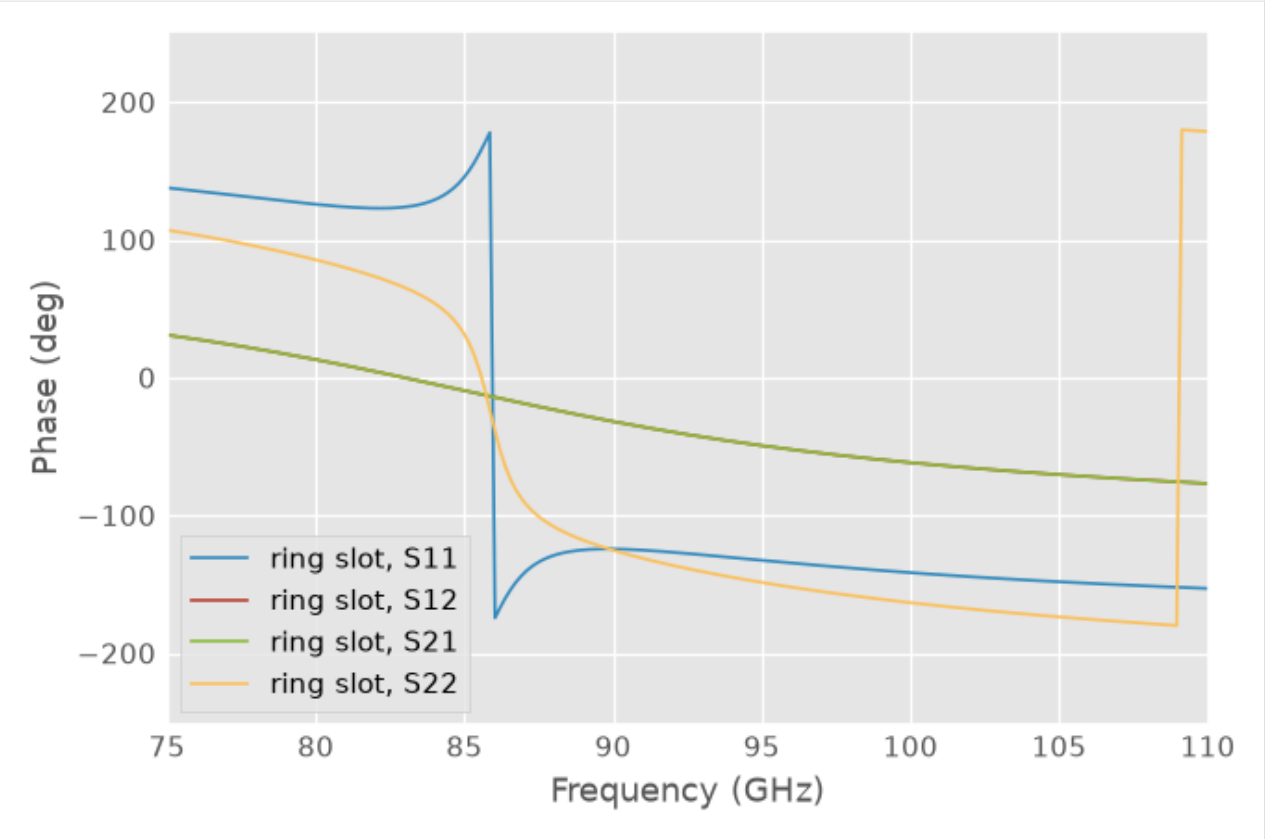
[11]:
```
from skrf.data import ring_slot_meas

ring_slot.plot_s_db(m=0,n=0, label='Theory')
ring_slot_meas.plot_s_db(m=0,n=0, label='Measurement')
```



1.6.6 Phase

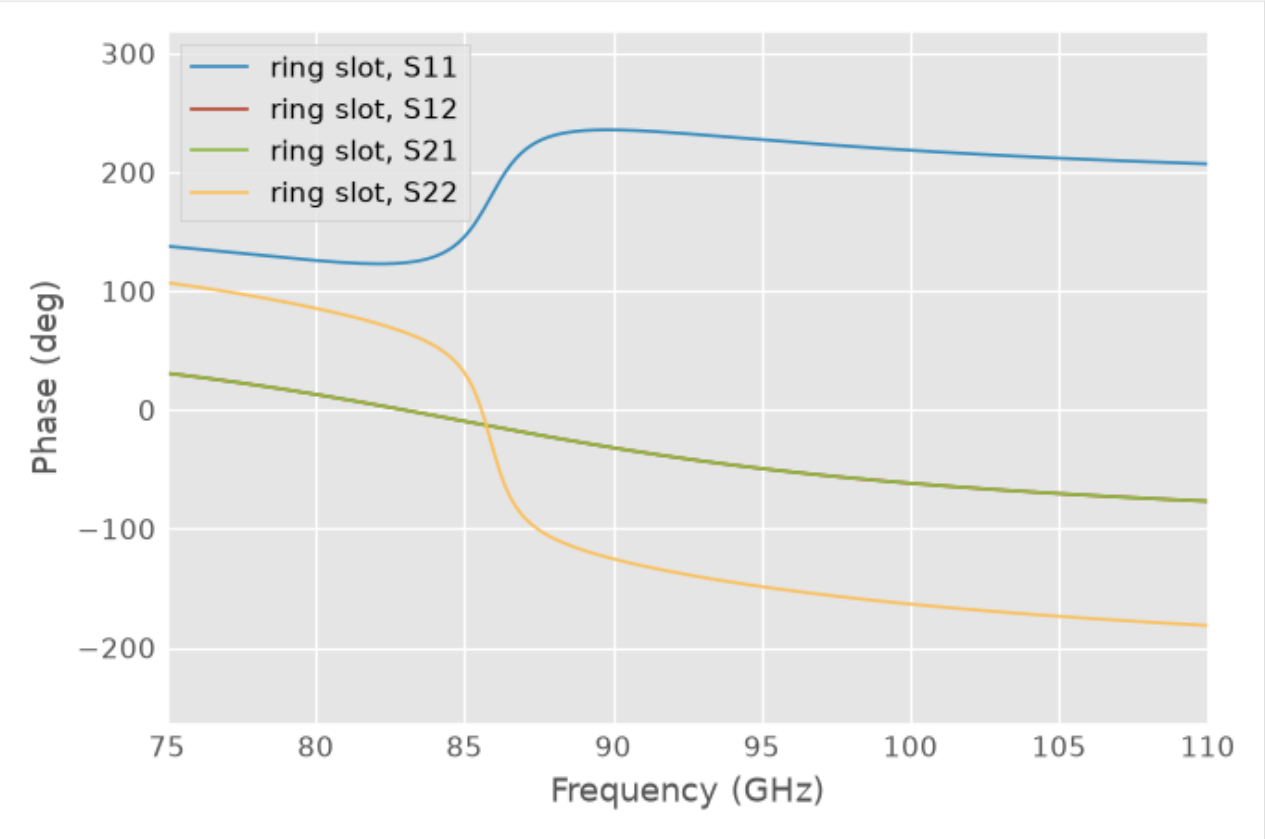
Plot phase,

[12]:
```
ring_slot.plot_s_deg()
```



Or unwrapped phase,

[13]:
```
ring_slot.plot_s_deg_unwrap()
```



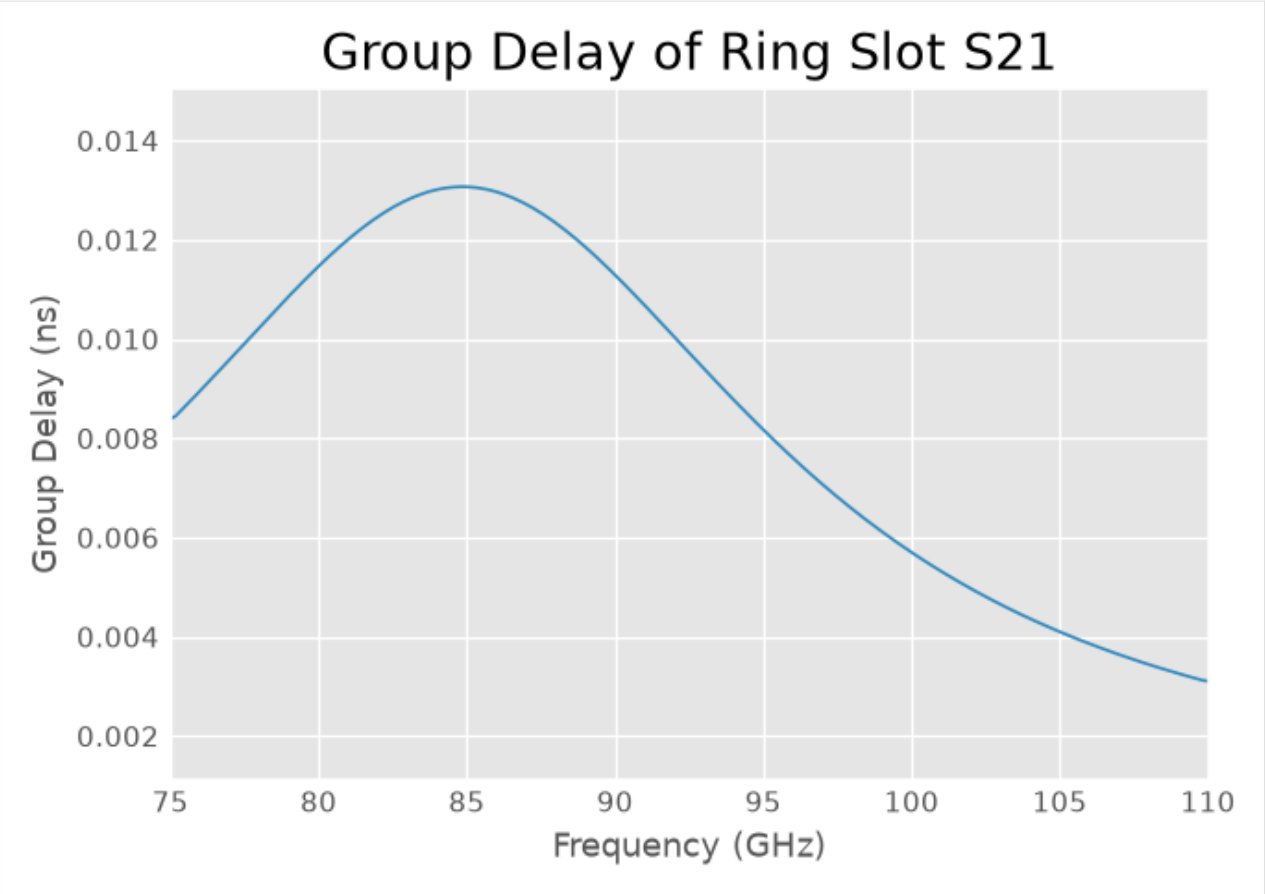
Phase is radian (rad) is also available

1.6.7 Group Delay

A Network has a `plot()` method which creates a rectangular plot of the argument vs frequency. This can be used to make plots are aren't 'canned'. For example group delay

[14]:
```
gd = abs(ring_slot.s21.group_delay) *1e9 # in ns

ring_slot.plot(gd)
plt.ylabel('Group Delay (ns)')
plt.title('Group Delay of Ring Slot S21')
```

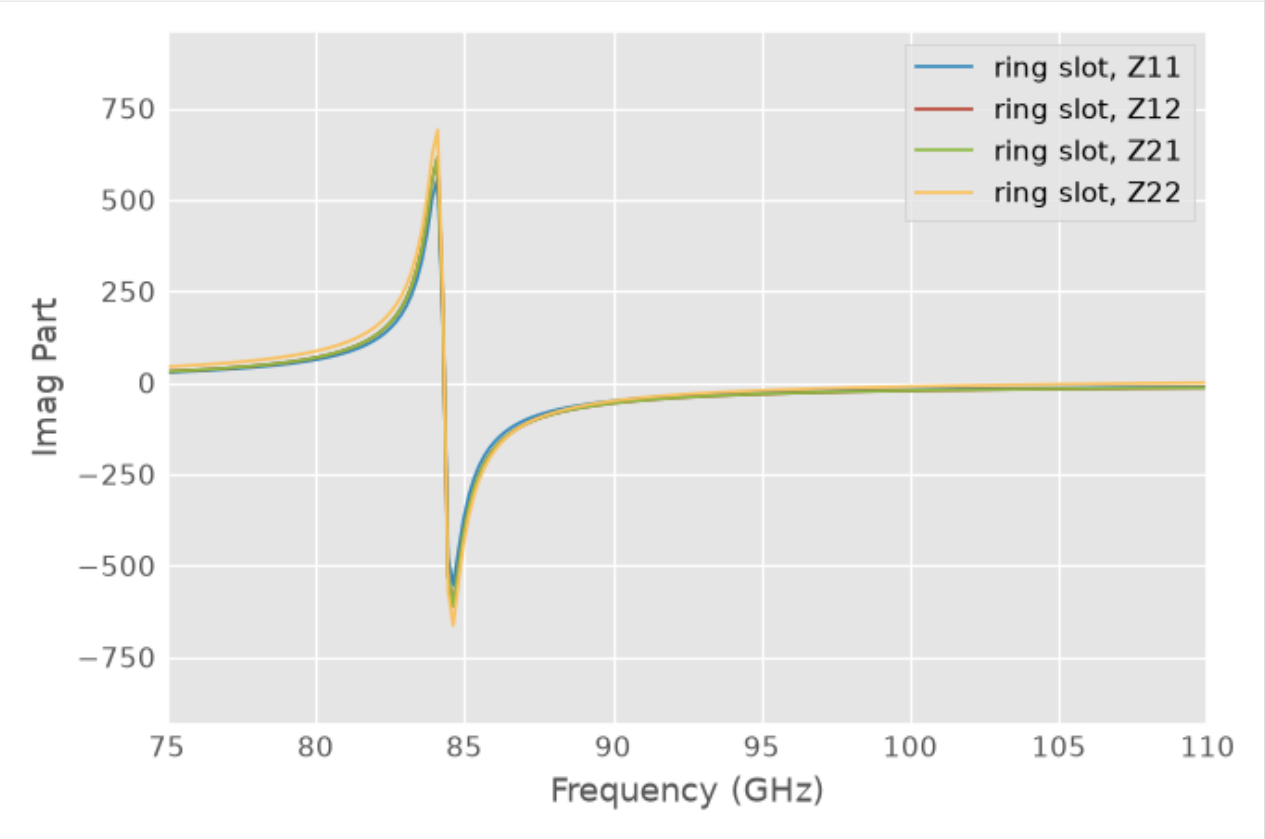
[14]:
```
Text(0.5, 1.0, 'Group Delay of Ring Slot S21')
```



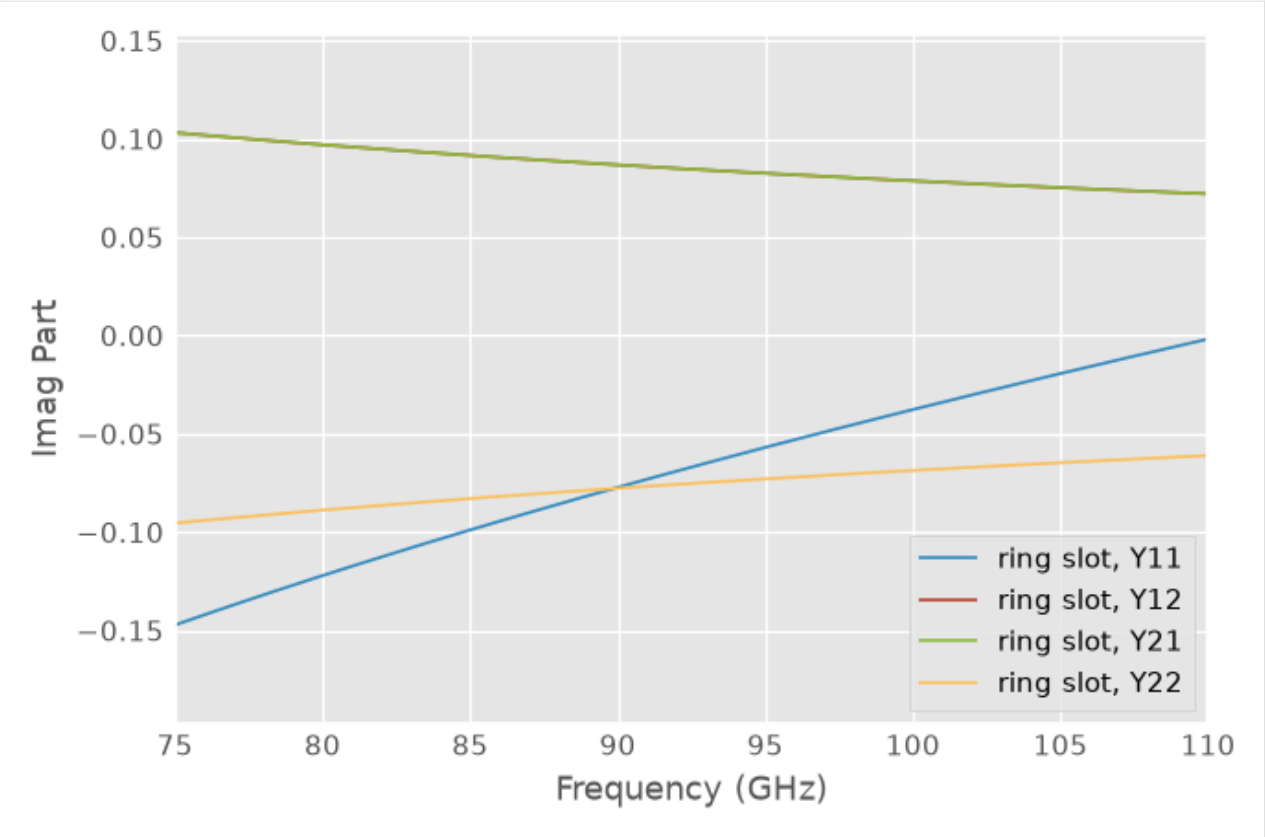
1.6.8 Impedance, Admittance

The components the Impedance and Admittance parameters can be plotted similarly,

[15]:
```
ring_slot.plot_z_im()
```



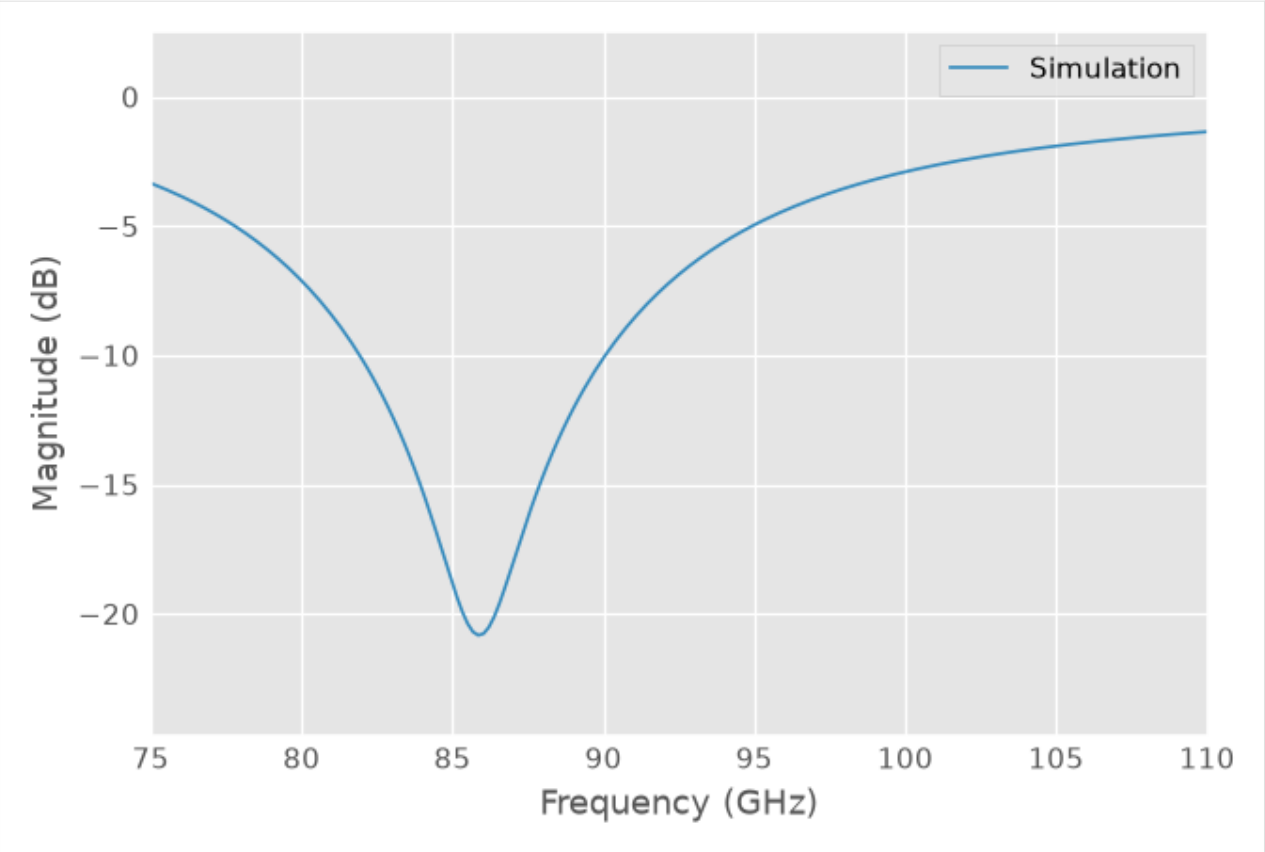
[16]:
```
ring_slot.plot_y_im()
```



1.6.9 Customizing Plots

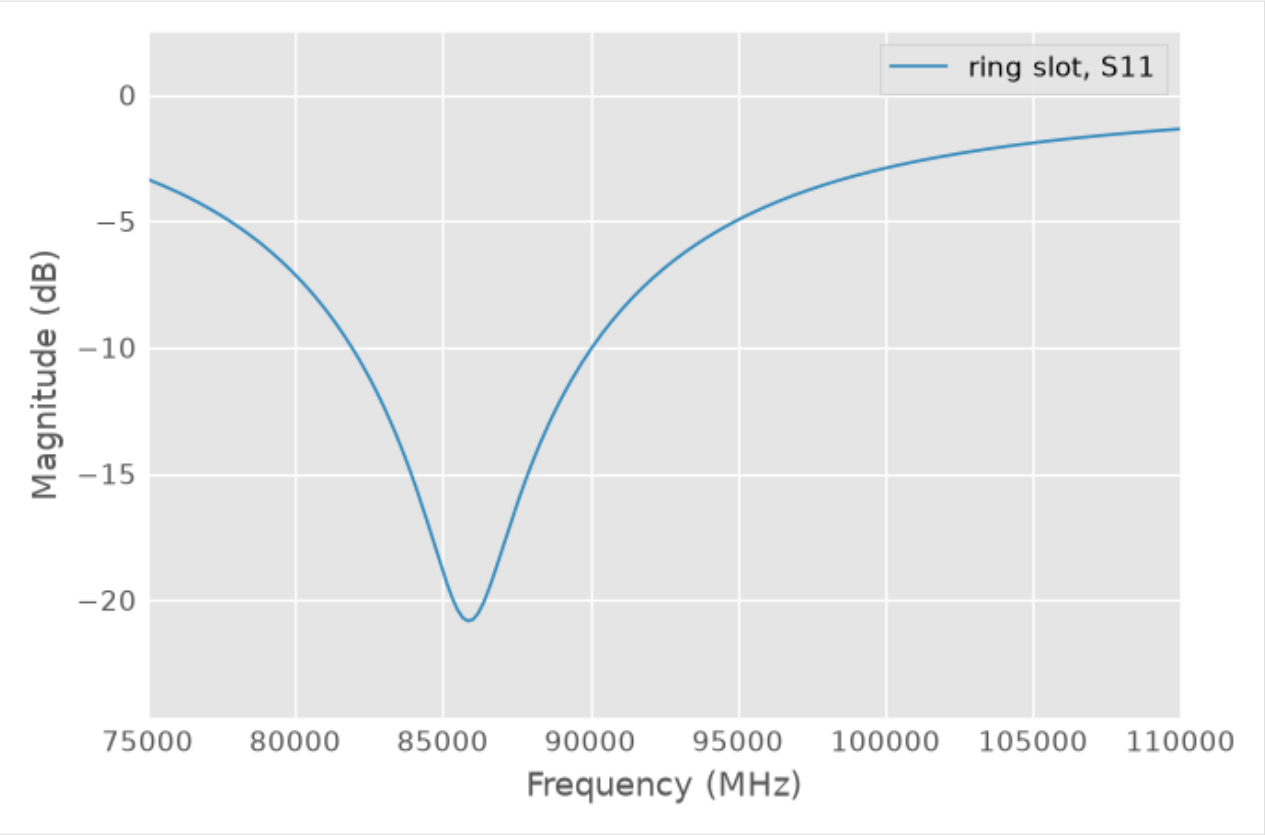
The legend entries are automatically filled in with the Network's `Network.name`. The entry can be overridden by passing the `label` argument to the plot method.

[17]:
```
ring_slot.plot_s_db(m=0,n=0, label = 'Simulation')
```



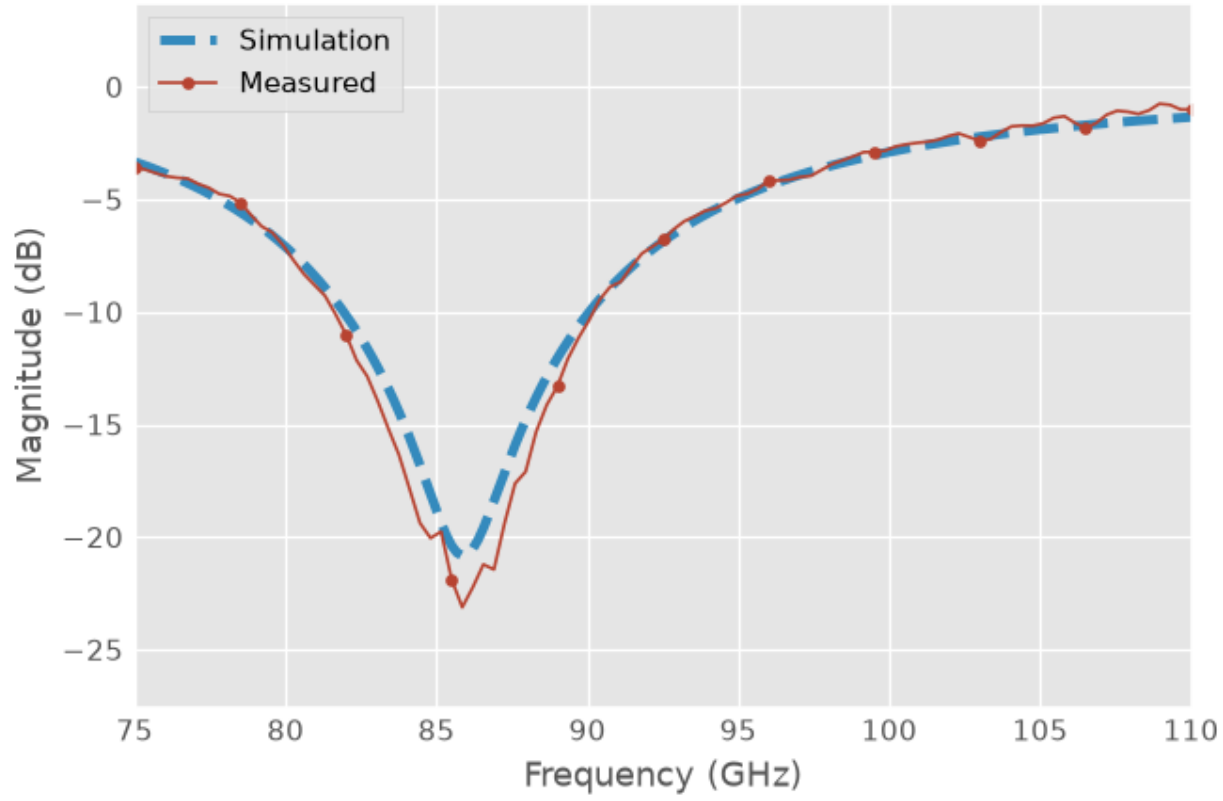
The frequency unit used on the x-axis is automatically filled in from the Networks `Network.frequency.unit` attribute. To change the label, change the frequency's `unit`.

[18]:
```
ring_slot.frequency.unit = 'mhz'
ring_slot.plot_s_db(0,0)
```



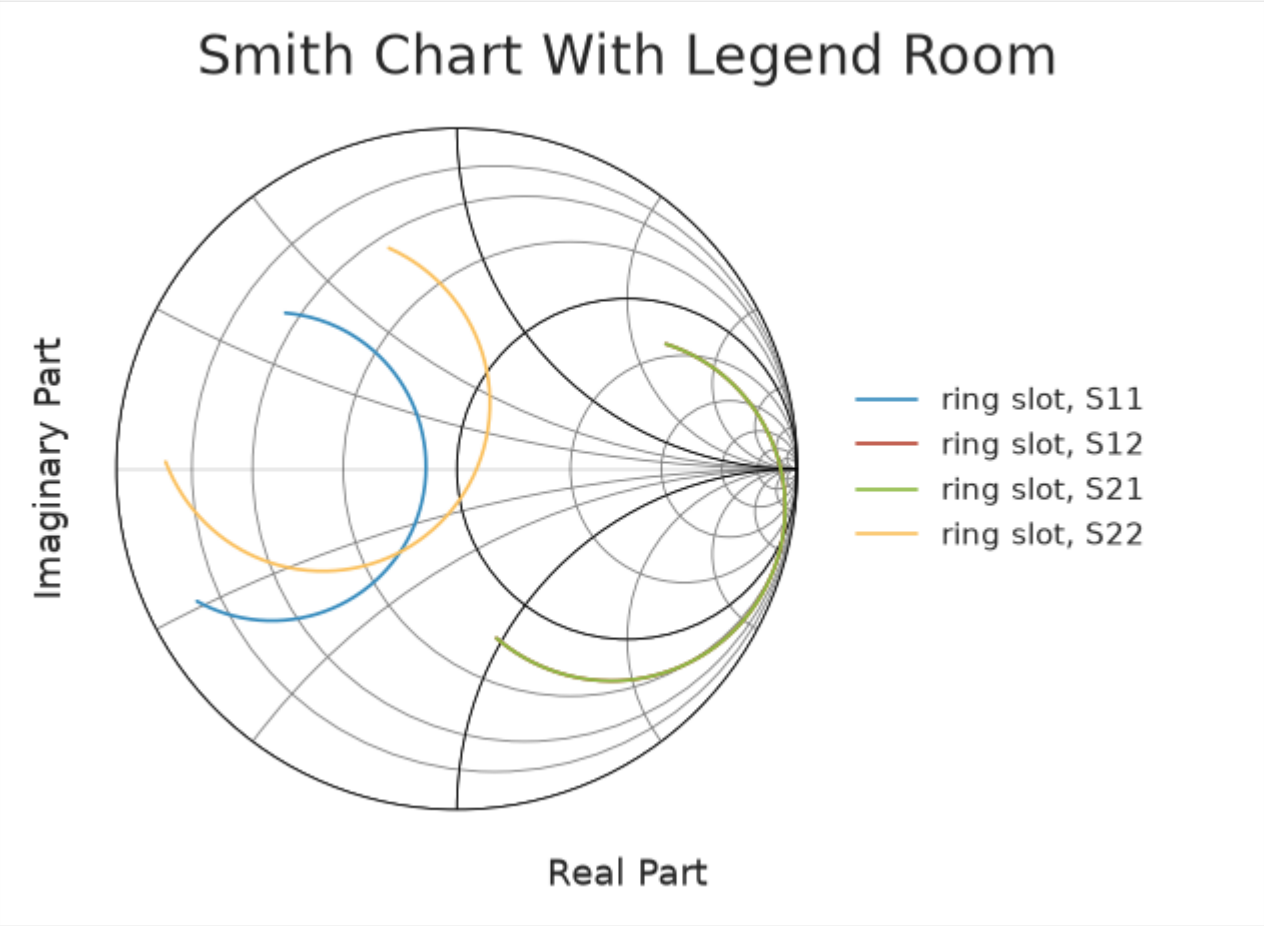
Other key word arguments given to the plotting methods are passed through to the matplotlib `matplotlib.pyplot.plot` function.

[19]:
```
ring_slot.frequency.unit='ghz'
ring_slot.plot_s_db(m=0,n=0, linewidth = 3, linestyle = '--', label = 'Simulation')
ring_slot_meas.plot_s_db(m=0,n=0, marker = 'o', markevery = 10,label = 'Measured')
```



All components of the plots can be customized through matplotlib functions, and `styles` can be used with a context manager.

[20]:
```
from matplotlib import pyplot as plt
from matplotlib import style

mpl_style = "seaborn-ticks"
mpl_style = mpl_style if mpl_style in style.available else "seaborn-v0_8-ticks"

with style.context(mpl_style):
    ring_slot.plot_s_smith()
    plt.xlabel('Real Part')
    plt.ylabel('Imaginary Part')
    plt.title('Smith Chart With Legend Room')
    plt.axis([-1.1,2.1,-1.1,1.1])
    plt.legend(loc=5)
```



1.6.10 Saving Plots

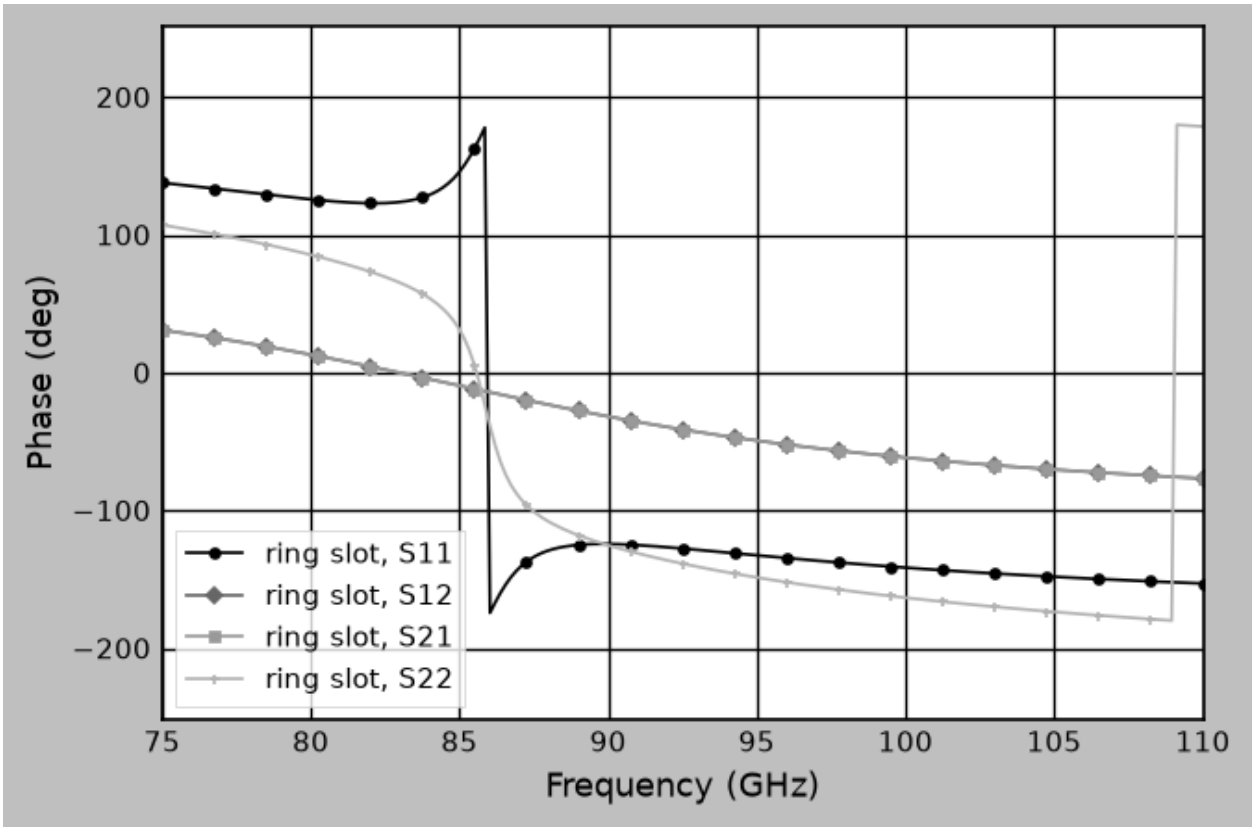
Plots can be saved in various file formats using the GUI provided by the matplotlib. However, skrf provides a convenience function, called `skrf.plotting.save_all_figs`, that allows all open figures to be saved to disk in multiple file formats, with filenames pulled from each figure's title,

[21]:
```
from skrf.plotting import save_all_figs

save_all_figs('data/', format=['png','eps','pdf'])
```

1.6.11 Adding Markers Post Plot

A common need is to make a color plot, interpretable in greyscale print. The `skrf.plotting.add_markers_to_lines` adds different markers each line in a plots *after* the plot has been made, which is usually when you remember to add them.

[22]:
```
from skrf import plotting

with plt.style.context('grayscale'):
    ring_slot.plot_s_deg()
    plotting.add_markers_to_lines()
    plt.legend() # have to re-generate legend
```



[ ]:

Download This Notebook: `NetworkSet.ipynb`

1.7 NetworkSet

1.7.1 Introduction

The *NetworkSet* object represents an unordered set of networks. It provides methods iterating and slicing the set, sorting by datetime, calculating statistical quantities, and displaying uncertainty bounds on plots.

1.7.2 Creating a *NetworkSet*

Lets take a look in the `data/` folder, there are some redundant measurements of a network called `ro`, which is a *radiating open* waveguide.

```
ls data/ro*

-a----       14/02/2021     12:35           8031 ro,1.s1p
```

(continues on next page)

(continued from previous page)

```
-a----        14/02/2021     12:35           8030 ro,2.s1p
-a----        14/02/2021     12:35           8031 ro,3.s1p
-a----        14/02/2021     12:35          46592 ro_spreadsheet.xls
```

The files `ro,1.s1p` , `ro,2.s1p`, ... are redundant measurements on which we would like to calculate statistics using the *NetworkSet* class.

A *NetworkSet* is created from a list or dict of *Network*'s. So first we need to load all of the touchstone files into `Networks`. This can be done quickly with `rf.io.read_all`, The argument `contains` is used to load only files which match a given substring.

[1]:
```
import skrf as rf

rf.io.read_all(rf.data.pwd, contains='ro')
```

[1]:
```
{'ro,1': 1-Port Network: 'ro,1',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j],
 'ro,2': 1-Port Network: 'ro,2',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j],
 'ro,3': 1-Port Network: 'ro,3',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]}
```

This can be passed directly to the *NetworkSet* constructor,

[2]:
```
from skrf.networkSet import NetworkSet

ro_dict = rf.io.read_all(rf.data.pwd, contains='ro')
ro_ns = NetworkSet(ro_dict, name='ro set')
ro_ns
```

[2]:
```
3-Networks NetworkSet: [1-Port Network: 'ro,1',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j],␣
↪1-Port Network: 'ro,2',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j], 1-Port Network: 'ro,3
↪',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]]
```

A NetworkSet can also be constructed directly from:

- a directory containing Touchstone files: `NetworkSet.from_dir()`,
- a zipfile of touchstones files: `NetworkSet.from_zip()`,
- a dictionary of s-parameters: `NetworkSet.from_s_dict()`,
- a (G)MDIF (.mdf) file: `NetworkSet.from_mdif()`,
- a CITI (.cti) file: `NetworkSet.from_citi()`.

1.7.3 Accessing Network Methods

The *Network* elements in a *NetworkSet* can be accessed like the elements of list,

[3]:
```
ro_ns[0]
```

[3]:
```
1-Port Network: 'ro,1',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]
```

Most *Network* methods are also methods of *NetworkSet*. These methods are called on each *Network* element individually. For example to plot the log-magnitude of the s-parameters of each Network.

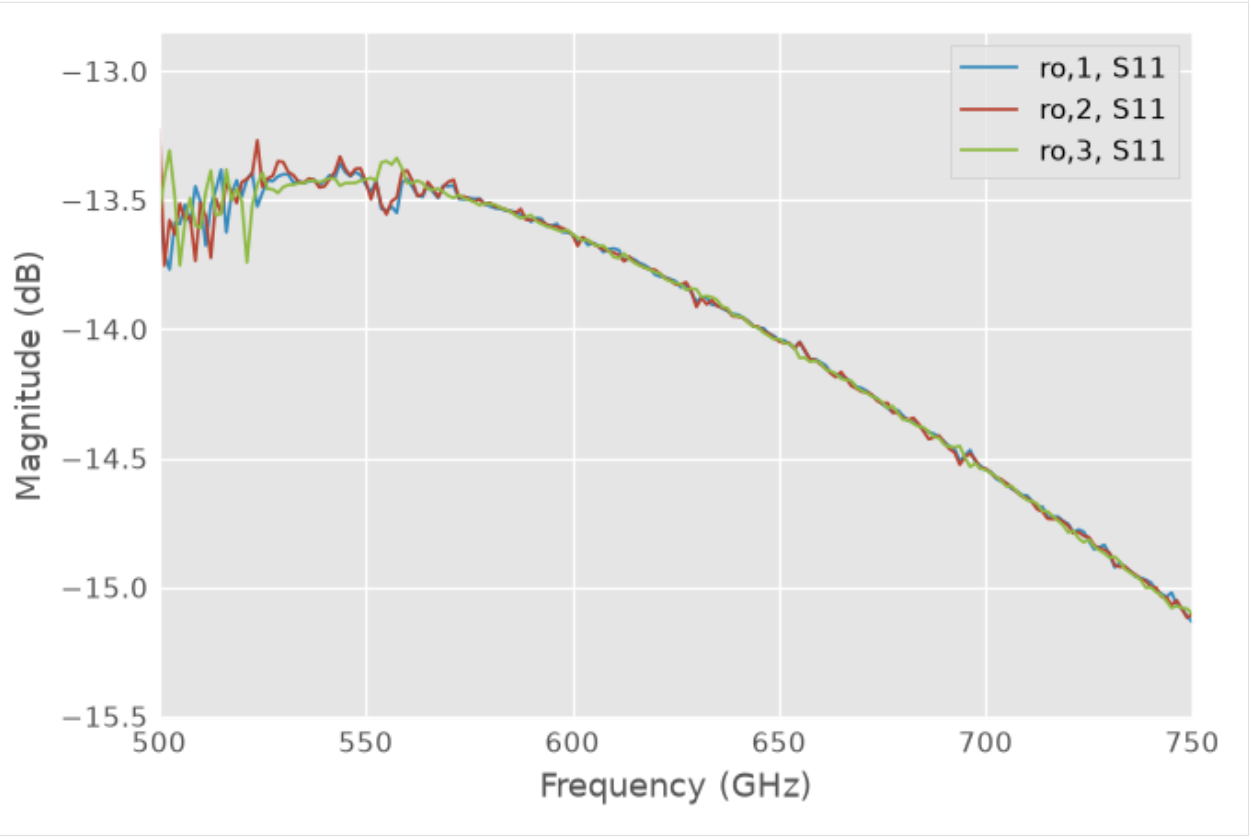
[4]:
```
%matplotlib inline
import skrf as rf
```

(continues on next page)

(continued from previous page)

```
rf.stylely()

ro_ns.plot_s_db()
```

[4]:
```
[None, None, None]
```



1.7.4 Statistical Properties

Statistical quantities can be calculated by accessing properties of the NetworkSet. To calculate the complex average of the set, access the `mean_s` property

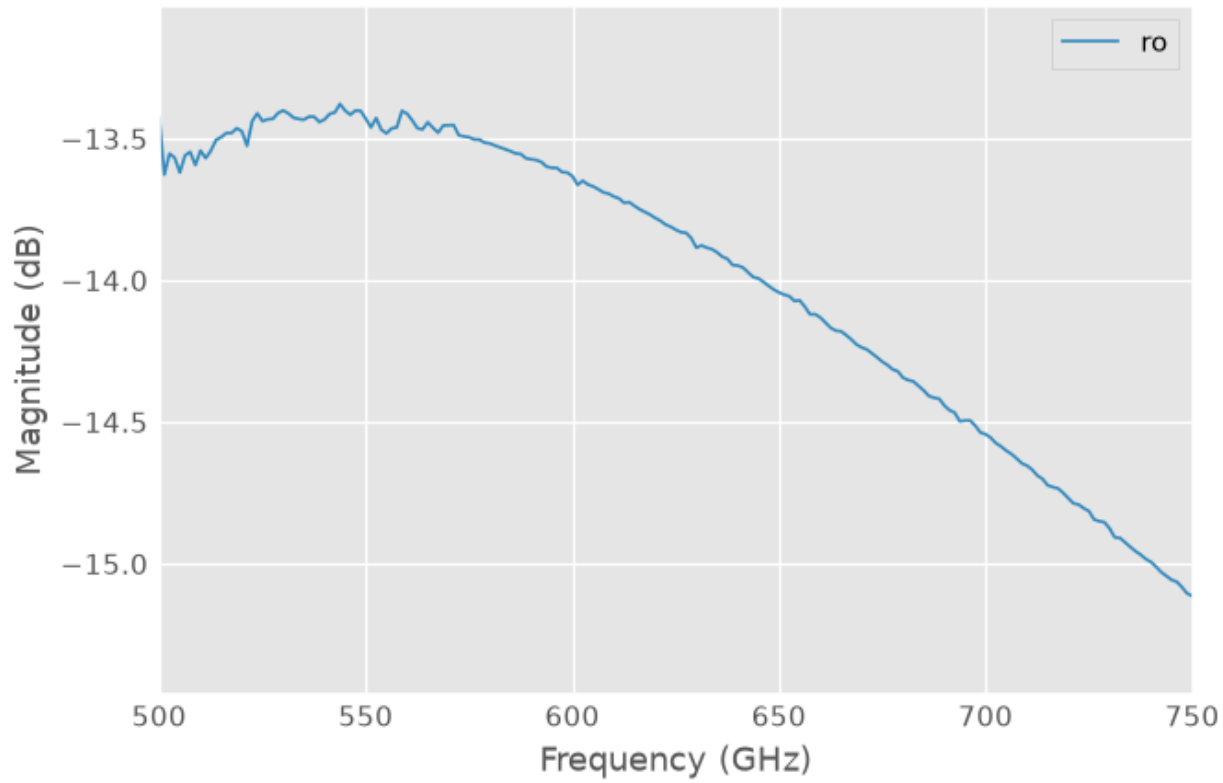
[5]:
```
ro_ns.mean_s
```

[5]:
```
1-Port Network: 'ro set',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]
```

> **Note**
>
> Because the statistical operator methods are generated upon initialization their API is not explicitly documented in this manual.

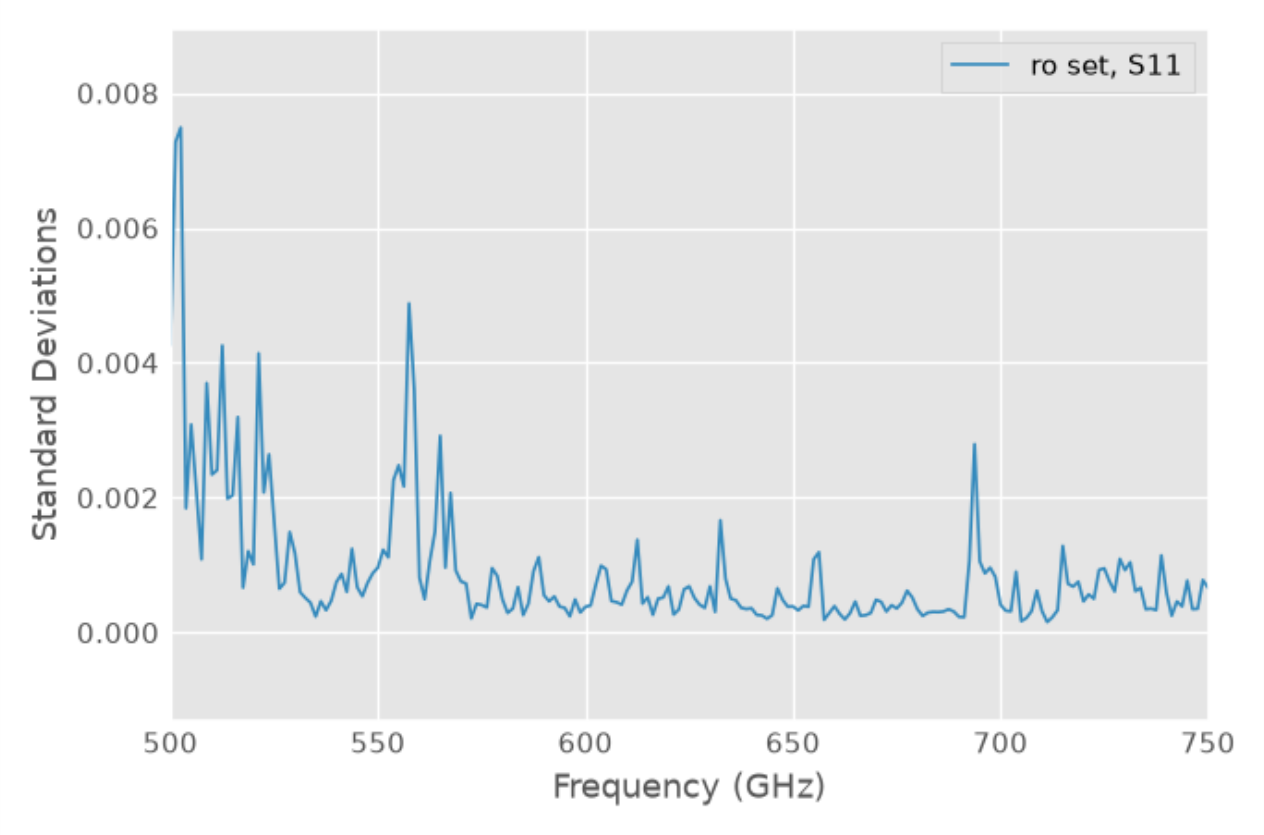
The naming convention of the statistical operator properties are `NetworkSet.{function}_{parameter}`, where `function` is the name of the statistical function, and `parameter` is the Network parameter to operate on. These methods return a *Network* object, so they can be saved or plotted in the same way as you would with a Network. To plot the log-magnitude of the complex mean response

[6]:
```
ro_ns.mean_s.plot_s_db(label='ro')
```



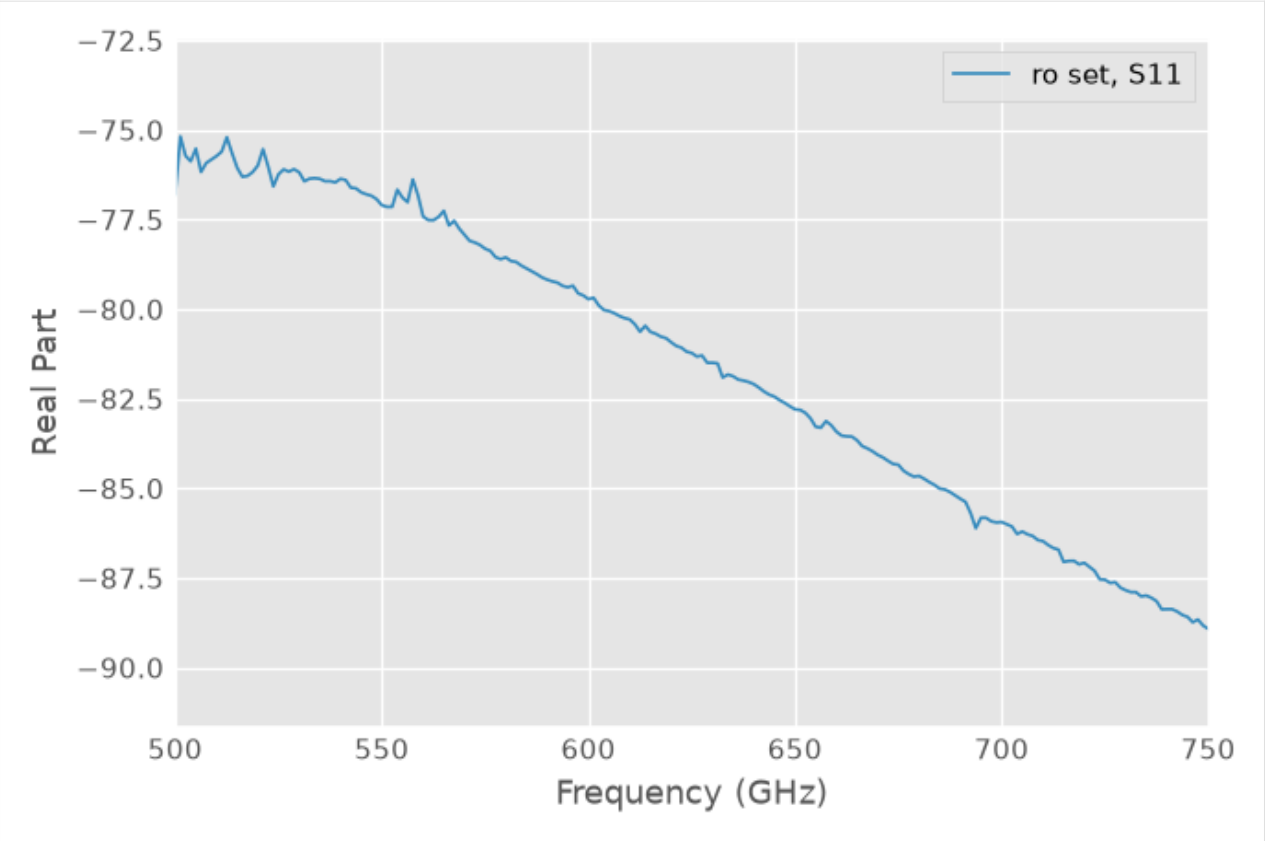
Or to plot the standard deviation of the complex s-parameters,

[7]:
```
ro_ns.std_s.plot_s_re(y_label='Standard Deviations')
```



Using these properties it is possible to calculate statistical quantities on the scalar components of the complex network parameters. To calculate the mean of the phase component,

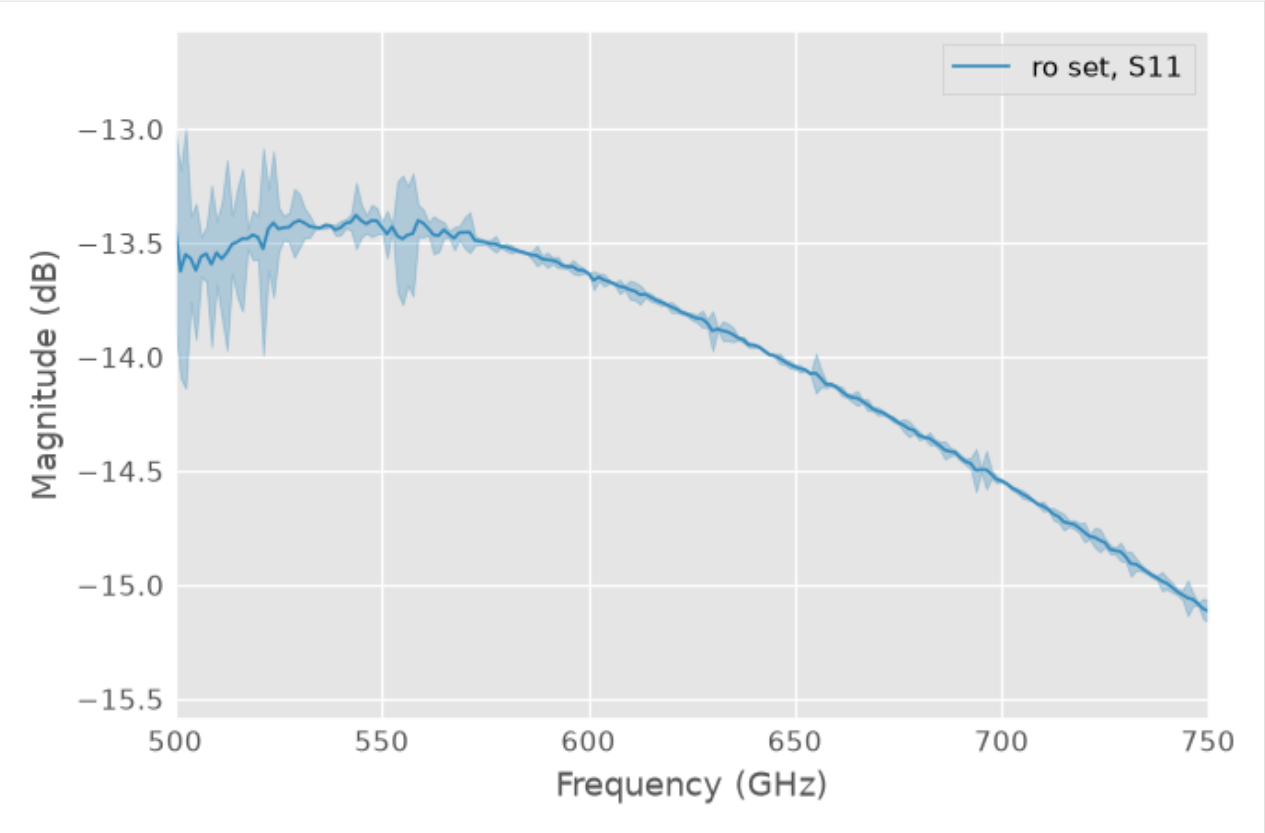
[8]:
```
ro_ns.mean_s_deg.plot_s_re()
```



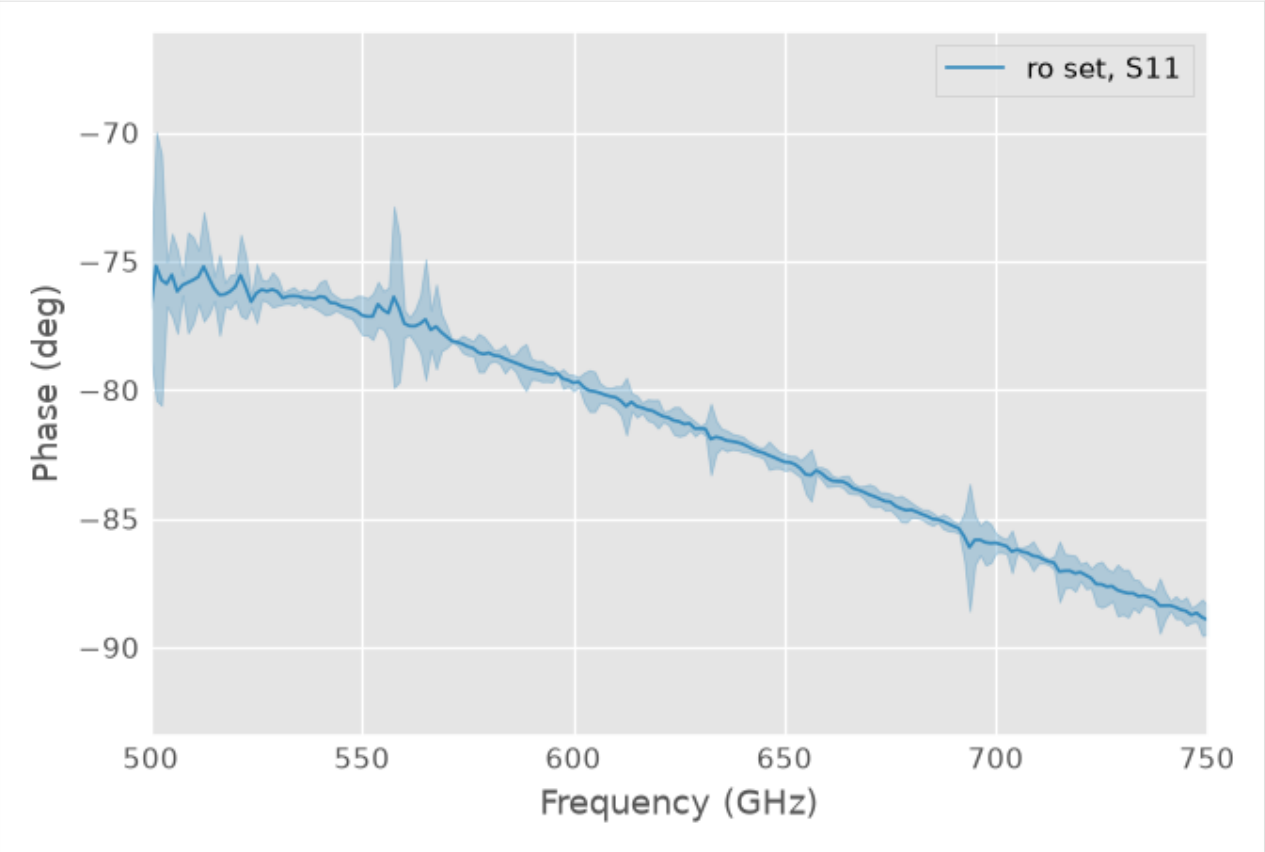
1.7.5 Plotting Uncertainty Bounds

Uncertainty bounds can be plotted through the methods

[9]:
```
ro_ns.plot_uncertainty_bounds_s_db()
```



[10]:
```
ro_ns.plot_uncertainty_bounds_s_deg()
```



Note

The uncertainty bounds plotted above are calculated **after** the complex number has been projected onto the specified scalar component. Thus, the first plot represents uncertainty in the magnitude component **only**.

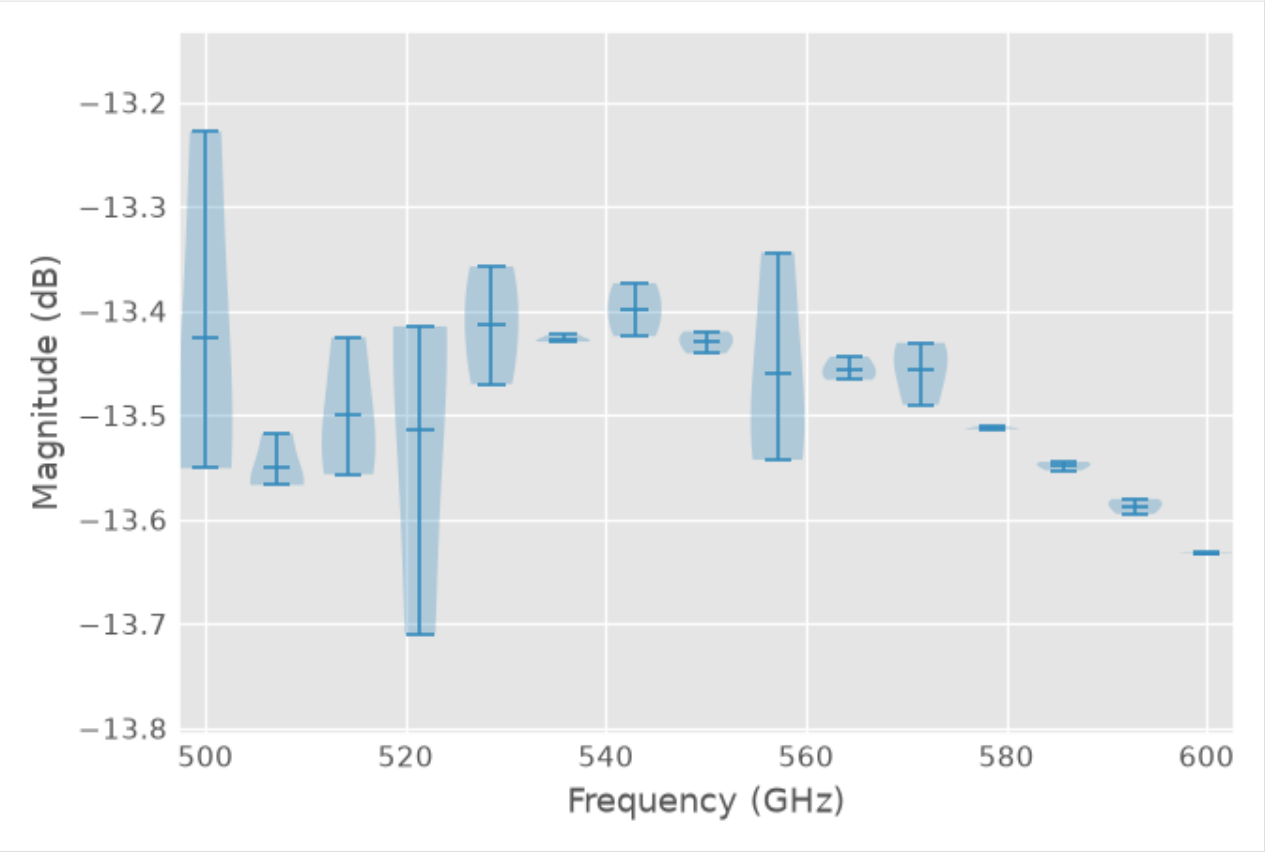
1.7.6 Plotting Violin Plots

Violin plots are used to get an idea of the distribution at a glance by displaying a probability density at each point, along with the minimum, maximum, and average values by default. Let's interpolate the frequencies to reduce clutter.

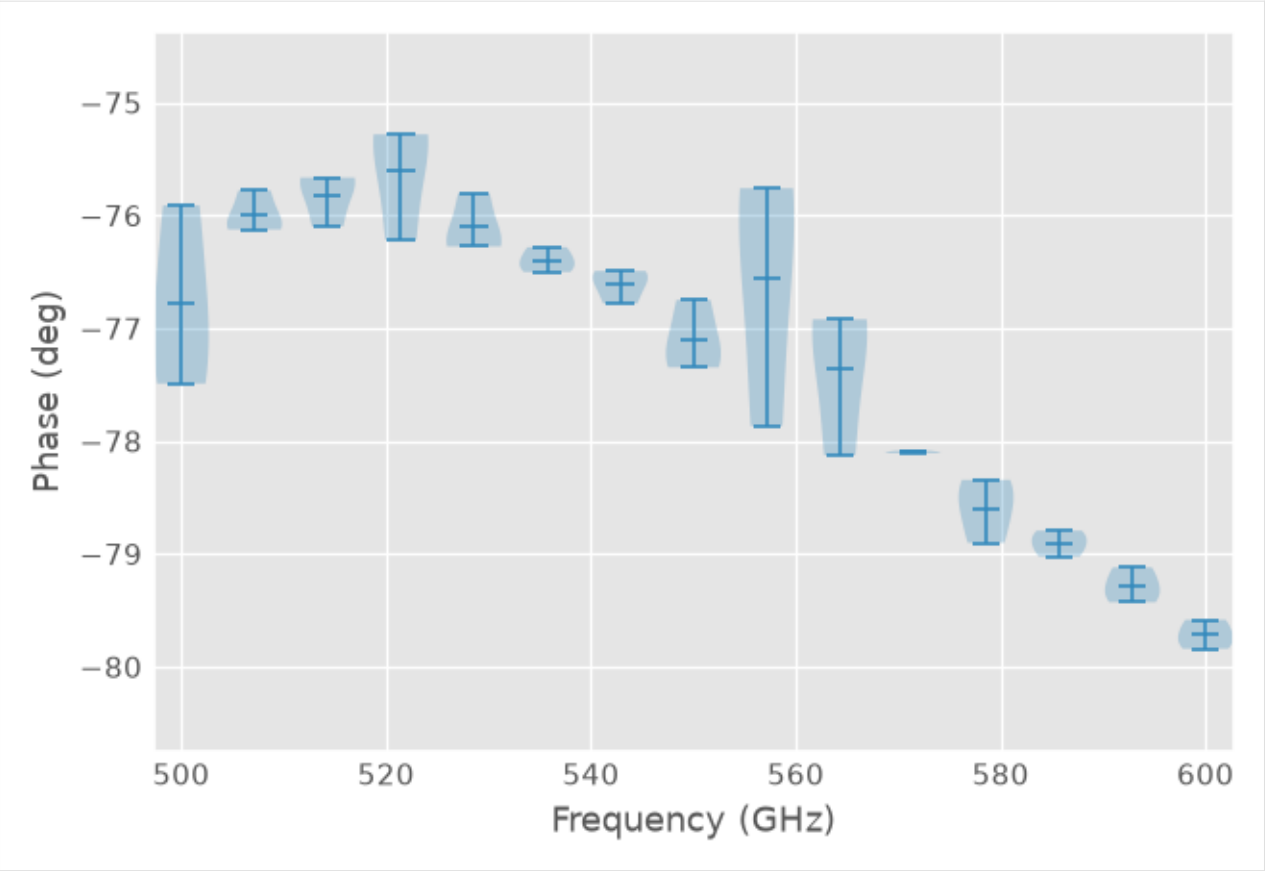
[11]:
```
ro_ns_interp = ro_ns.interpolate_frequency(rf.Frequency(500, 600, 15, "GHz"))
```

Violin plots are available for most common network parameters, except for those in the time domain. To plot the log-magnitude and phase of the s-parameters

[12]:
```
ro_ns_interp.plot_violin("s_db")
```



[13]:
```
ro_ns_interp.plot_violin("s_deg")
```



1.7.7 Reading and Writing

To write all *Network*s of a *NetworkSet* out to individual touchstones,

[14]:
```
ro_ns.write_touchstone(dir='data/')
```

[14]:
```
[None, None, None]
```

For temporary data storage, *NetworkSet*s can be saved and read from disk using the functions `rf.io.read` and `rf.io.write`

[15]:
```
rf.io.write('ro set.ns', ro_ns)
```

[16]:
```
ro_ns = rf.io.read('ro set.ns')
ro_ns
```

[16]:
```
3-Networks NetworkSet: [1-Port Network: 'ro,1',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j],␣
↪1-Port Network: 'ro,2',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j], 1-Port Network: 'ro,3
↪',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]]
```

1.7.8 Export to Excel, csv, or html

*NetworkSet*s can also be exported to other filetypes. The format of the output; real/imag, mag/phase is adjustable, as is the output type; csv, excel, html. For example to export mag/phase for each network into an Excel spreadsheet for your boss[s]

[17]:
```
ro_ns.write_spreadsheet('data/ro_spreadsheet.xls', form='db')
```

More info on this can be found in the function, `skrf.io.general.network_2_spreadsheet`

1.7.9 Named Parameters

If all the `Network` objects of a `NetworkSet` have a `params` property containing a dictionary of the named parameters and values associated to each Network, it is possible to select the Networks corresponding to a subset of named parameters using the `.sel()` method.

The following example illustrates this feature.

[18]:
```
# dummy named parameters and values 'a', 'X' and 'c'
import numpy as np

params = [
        {'a':0, 'X':10, 'c':'A'},
        {'a':1, 'X':10, 'c':'A'},
        {'a':2, 'X':10, 'c':'A'},
        {'a':1, 'X':20, 'c':'A'},
        {'a':0, 'X':20, 'c':'A'},
        ]
# create a NetworkSet made of dummy Networks, each define for set of parameters
freq1 = rf.Frequency(75, 110, 101, 'ghz')
rng = np.random.default_rng()
ntwks_params = [rf.Network(frequency=freq1, s=rng.uniform(size=(len(freq1),2,2)),
                           name=f'ntwk_{m}', comment=f'ntwk_{m}', params=params)
                for (m, params) in enumerate(params) ]
ns = rf.networkSet.NetworkSet(ntwks_params)
print(ns)
```

```
5-Networks NetworkSet: [2-Port Network: 'ntwk_0',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j␣
↪50.+0.j], 2-Port Network: 'ntwk_1',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j], 2-
↪Port Network: 'ntwk_2',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j], 2-Port Network:
↪ 'ntwk_3',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j], 2-Port Network: 'ntwk_4', ␣
↪75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]]
```

Selecting the sub-NetworkSet matching scalar parameters can be made as:

[19]:
```
ns.sel({'a': 1})
```

[19]:
```
2-Networks NetworkSet: [2-Port Network: 'ntwk_1',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j␣
↪50.+0.j], 2-Port Network: 'ntwk_3',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]]
```

[20]:
```
ns.sel({'a': 0, 'X': 10})
```

[20]:
```
1-Networks NetworkSet: [2-Port Network: 'ntwk_0',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j␣
↪50.+0.j]]
```

Selecting the sub-NetworkSet matching a range of parameters also works:

[21]:
```
ns.sel({'a': 0, 'X': [10,20]})
```

[21]:
```
2-Networks NetworkSet: [2-Port Network: 'ntwk_0',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j␣
↪50.+0.j], 2-Port Network: 'ntwk_4',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]]
```

[22]:
```
ns.sel({'a': [0,1], 'X': [10,20]})
```

[22]:
```
4-Networks NetworkSet: [2-Port Network: 'ntwk_0',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j␣
↪50.+0.j], 2-Port Network: 'ntwk_1',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j], 2-
↪Port Network: 'ntwk_3',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j], 2-Port Network:
↪ 'ntwk_4',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]]
```

The various named parameter keys and values of the NetworkSet can be retrieved using the `dims` and `coords` properties:

[23]:
```
ns.dims
```

[23]:
```
dict_keys(['a', 'X', 'c'])
```

[24]:
```
ns.coords
```

[24]:
```
{'a': [0, 1, 2], 'X': [10, 20], 'c': ['A']}
```

1.7.10 Interpolating between the Networks of a NetworkSet

It is possible to create new Networks interpolated from the Networks contained in a `NetworkSet`. If no `params` properties have been defined for each Network of the NetworkSet, the `interpolate_from_network()` method can be used to specify a interpolating parameter.

[25]:
```
param_x = [1, 2, 3]  # a parameter associated to each Network
x0 = 1.5  # parameter value to interpolate for
interp_ntwk = ro_ns.interpolate_from_network(param_x, x0)
print(interp_ntwk)
```

```
1-Port Network: 'ro,1',  500.0-750.0 GHz, 201 pts, z0=[50.+0.j]
```

An illustrated example is given in the *Examples section* of the documentation.

It is also possible to interpolate using a named parameter when they have been defined:

[26]:
```
# Interpolated Network for a=1.2 within X=10 Networks:
ns.interpolate_from_params('a', 1.2, {'X': 10})
```

[26]:
```
2-Port Network: 'ntwk_0',  75.0-110.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]
```

.. $_{c}ircuit$ ::

||

Download This Notebook: :download:‘Circuit.ipynb‘

1.8 Circuits

1.8.1 Introduction

The Circuit class represents a circuit of arbitrary topology, consisting of an arbitrary number of N-ports Networks connected together. Like in an electronic circuit simulator, the circuit must have one (or more) `Port` connected to the circuit. The `Circuit` object allows one retrieving the M-ports `Network` (and thus its network parameters: S, Z, etc.), where M is the number of ports defined. Moreover, the `Circuit` object also allows calculating the scattering matrix S of the entire circuit, that is the “internal” scattering matrices for the various intersections in the circuit. The calculation algorithm is based on ref [1].

The figure below illustrates a network with 2 ports, `Network` elements N_i and intersections:

one must must define the connection list ("netlist") of the circuit. This connection list is defined as a List of List of interconnected Tuples `(network, port_number)`:

```
connexions = [
    [(network1, network1_port_nb), (network2, network2_port_nb), (network2, network2_
↪port_nb), ...],
    ...
]
```

For example, the connection list to construct the above circuit could be:

```
connexions = [
    [(port1, 0), (network1, 0), (network4, 0)],
    [(network1, 1), (network2, 0), (network5, 0)],
    [(network1, 2), (network3, 0)],
    [(network2, 1), (network3, 1)],
    [(network2, 2), (port2, 0)],
    [(network5, 1), (ground1, 0)],
    [(network5, 2), (open1, 0)]
]
```

where we have assumed that `port1`, `port2`, `ground1`, `open1` and all the `network1` to `network5` are scikit-rf Networks objects with same `Frequency`. Networks can have different (real) characteristic impedances: mismatch are taken into account. Convenience methods are provided to create ports and grounded connexions:

- Circuit.Port()
- Circuit.Ground()
- Circuit.SeriesImpedance()
- Circuit.ShuntAdmittance()
- Circuit.Open()

Warnings

Circuit requires distinct `Network` names for each `Network`. So, in case you would like to use multiple times the same `Network`, you have to duplicate this `Network` (for example using the `.copy()` method) and to specify a distinct `.name` property for each copy, like:

```
# Assuming network1 is the Network you would like to duplicate:
network2 = network1.copy()
network1.name = 'My first Network'
network2.name = 'My second network'
```

In addition, a set `(network, port_number)` should appear only once in the connection list. In case you would like to connect multiple networks to a single ones (like grounding multiple networks), there is two solutions:

- connecting the N networks to a single `Ground` object once like in:

```
# in the connection list:
...
    [(network1,1),(network2,1),(network3,1),(gnd,0)]
...
```

- Or create as many distinct `Ground` objects (also with unique `name` properties as mentioned above) as necessary.

```
# in the connection list:
...
    [(network1,1), (gnd1,0)],
    [(network2,1), (gnd2,0)],
    [(network3,1), (gnd3,0)],
...
```

Once the connection list is defined, the `Circuit` with:

```
resulting_circuit = rf.circuit.Circuit(connexions)
```

`resulting_circuit` is a Circuit object.

The resulting 2-ports `Network` is obtained with the Circuit.network parameter:

```
resulting_network = resulting_circuit.network
```

Note that it is also possible to create manually a circuit of multiple `Network` objects using the connecting methods of `scikit-rf`. Although the `Circuit` approach to build a multiple `Network` may appear to be more verbose than the 'classic' way for building a circuit, as the circuit complexity increases, in particular when components are connected in parallel, the `Circuit` approach is interesting as it increases the readability of the code. Moreover, `Circuit` circuit topology can be plotted using its `plot_graph` method, which is useful to rapidly control if the circuit is built as expected.

1.8.2 Examples

Loaded transmission line

Assume that a 50Ω lossless transmission line is loaded with a $Z_L = 75\Omega$ impedance.

If the transmission line electric length is $\theta = 0$, then one would thus expect the reflection coefficient to be:

$$\rho = s = \frac{Z_L - Z_0}{Z_L + Z_0} = 0.2$$

[1]:
```
import skrf as rf
from skrf.circuit import Circuit

rf.stylely()
```

[2]:
```
Z_0 = 50
Z_L = 75
theta = 0

# the necessary Frequency description
freq = rf.Frequency(start=1, stop=2, unit='GHz', npoints=3)

# The combination of a transmission line + a load can be created
# using the convenience delay_load method
# important: all the Network must have the parameter "name" defined
tline_media = rf.media.DefinedGammaZ0(freq, z0=Z_0)
delay_load = tline_media.delay_load(rf.tlineFunctions.zl_2_Gamma0(Z_0, Z_L), theta, unit=
↪'deg', name='delay_load')
```

(continues on next page)

(continued from previous page)

```
# the input port of the circuit is defined with the Circuit.Port method
port1 = Circuit.Port(freq, 'port1', z0=Z_0)

# connection list
cnx = [
    [(port1, 0), (delay_load, 0)]
]
# building the circuit
cir = Circuit(cnx)

# getting the resulting Network from the 'network' parameter:
ntw = cir.network
print(ntw)
```

```
1-Port Network: '',  1.0-2.0 GHz, 3 pts, z0=[50.+0.j]
```

[3]:
```
# as expected the reflection coefficient is:
print(ntw.s[0])
```

```
[[0.2+0.j]]
```

It is also possible to build the above circuit using a series impedance Network, then shorted:

To do so, one would need to use the `Ground()` method to generate the required `Network` object.

[4]:
```
port1 = Circuit.Port(freq, 'port1', z0=Z_0)
# piece of transmission line and series impedance
trans_line = tline_media.line(theta, unit='deg', name='trans_line')
load = tline_media.resistor(Z_L, name='delay_load')
# ground network (short)
ground = Circuit.Ground(freq, name='ground')

# connection list
cnx = [
    [(port1, 0), (trans_line, 0)],
    [(trans_line, 1), (load, 0)],
    [(load, 1), (ground, 0)]
]
# building the circuit
cir = Circuit(cnx)
# the result if the same :
print(cir.network.s[0])
```

```
[[0.2+0.j]]
```

LC Filter

Here we model a low-pass LC filter, with example values taken from rf-tools.com :

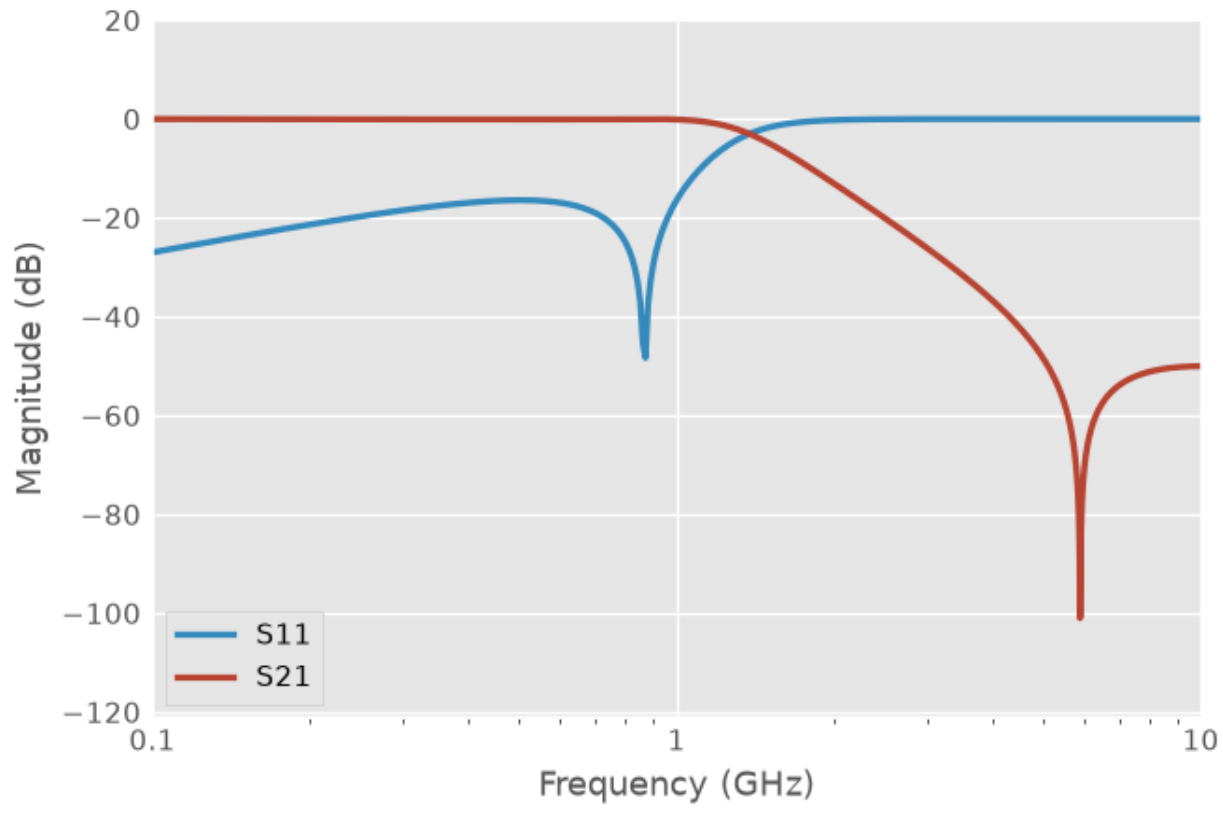
[5]:
```
freq = rf.Frequency(start=0.1, stop=10, unit='GHz', npoints=1001)
tl_media = rf.media.DefinedGammaZ0(freq, z0=50, gamma=1j*freq.w/rf.constants.c)
```

(continues on next page)

(continued from previous page)

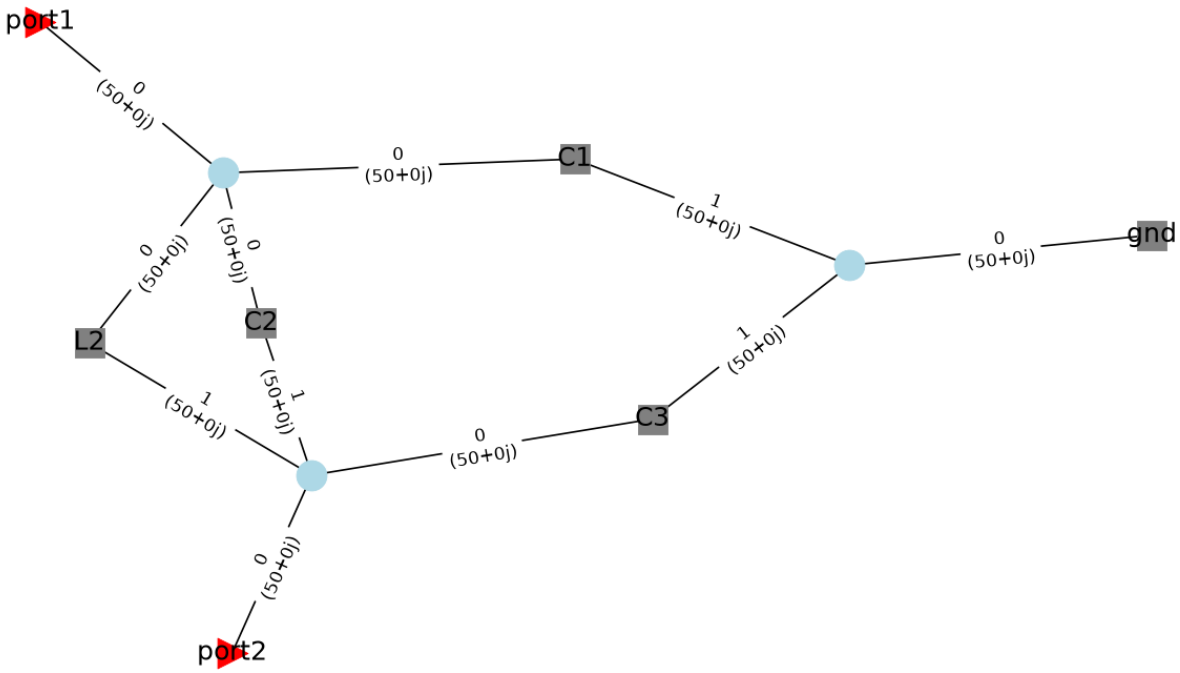
```
C1 = tl_media.capacitor(3.222e-12, name='C1')
C2 = tl_media.capacitor(82.25e-15, name='C2')
C3 = tl_media.capacitor(3.222e-12, name='C3')
L2 = tl_media.inductor(8.893e-9, name='L2')
RL = tl_media.resistor(50, name='RL')
gnd = Circuit.Ground(freq, name='gnd')
port1 = Circuit.Port(freq, name='port1', z0=50)
port2 = Circuit.Port(freq, name='port2', z0=50)

cnx = [
    [(port1, 0), (C1, 0), (L2, 0), (C2, 0)],
    [(L2, 1), (C2, 1), (C3, 0), (port2, 0)],
    [(gnd, 0), (C1, 1), (C3, 1)],
]
cir = Circuit(cnx)
ntw = cir.network
```

[6]:
```
ntw.plot_s_db(m=0, n=0, lw=2, logx=True)
ntw.plot_s_db(m=1, n=0, lw=2, logx=True)
```



When building a `Circuit` made of few networks, it can be useful to represent the connection graphically, in order to check for possible errors. This is possible using the Circuit.plot_graph() method. Ports are indicated by triangles, Network with squares and interconnections with circles. It is possible to display the network names as well as their associated ports (and characteristic impedances):

[7]:
```
cir.plot_graph(network_labels=True, network_fontsize=15,
               port_labels=True, port_fontsize=15,
              edge_labels=True, edge_fontsize=10)
```



Circuit Reduction

Here we model a slightly more complex band-pass LC filter to demonstrate circuit reduction, with example values taken from markimicrowave.com :

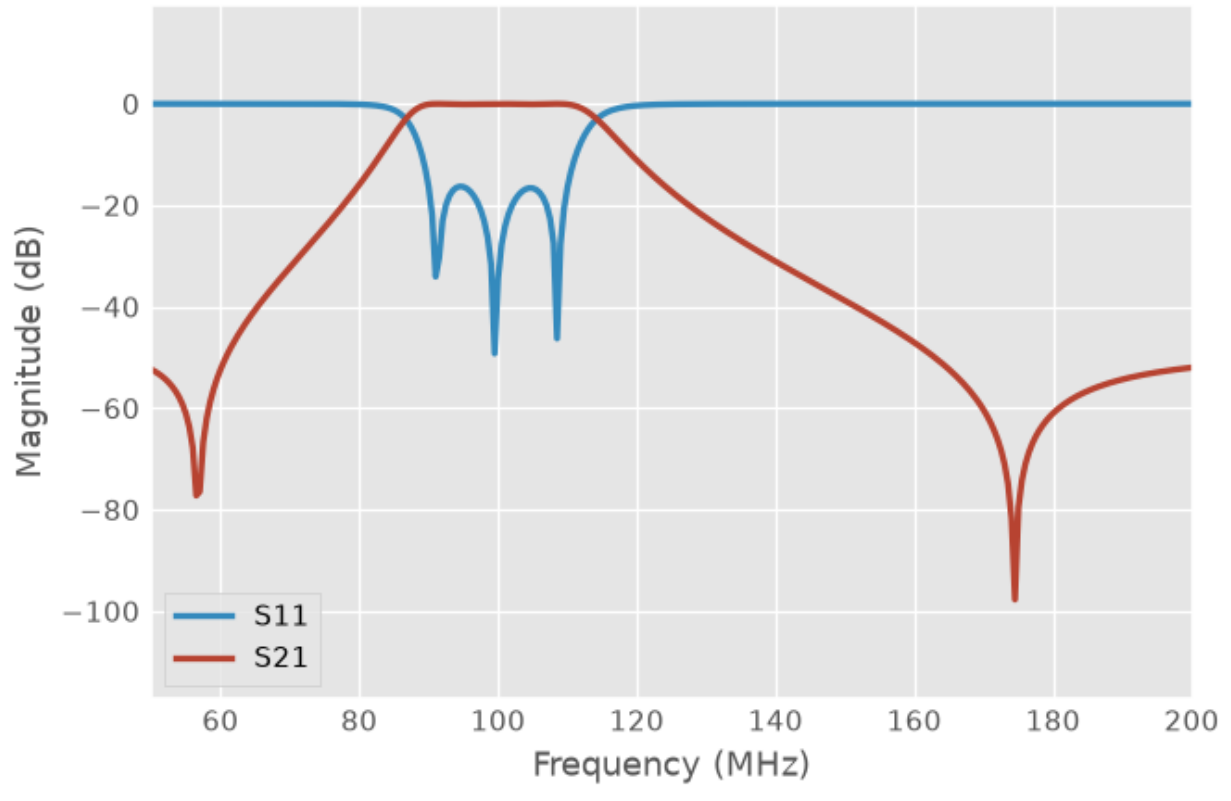
[8]:
```
freq = rf.Frequency(50, 200, 301, 'mhz')
tl_media = rf.media.DefinedGammaZ0(frequency=freq, z0=50)
gnd = Circuit.Ground(frequency=freq,name='ground')
C1 = tl_media.capacitor(6.353e-12, name='C1')
L1 = tl_media.inductor(402.7e-9, name='L1')
C2 = tl_media.capacitor(61.08e-12, name='C2')
L2 = tl_media.inductor(13.63e-9, name='L2')
C3 = tl_media.capacitor(187.8e-12, name='C3')
L3 = tl_media.inductor(41.89e-9, name='L3')
C4 = tl_media.capacitor(6.353e-12, name='C4')
L4 = tl_media.inductor(402.27e-9, name='L4')
```

(continues on next page)

(continued from previous page)

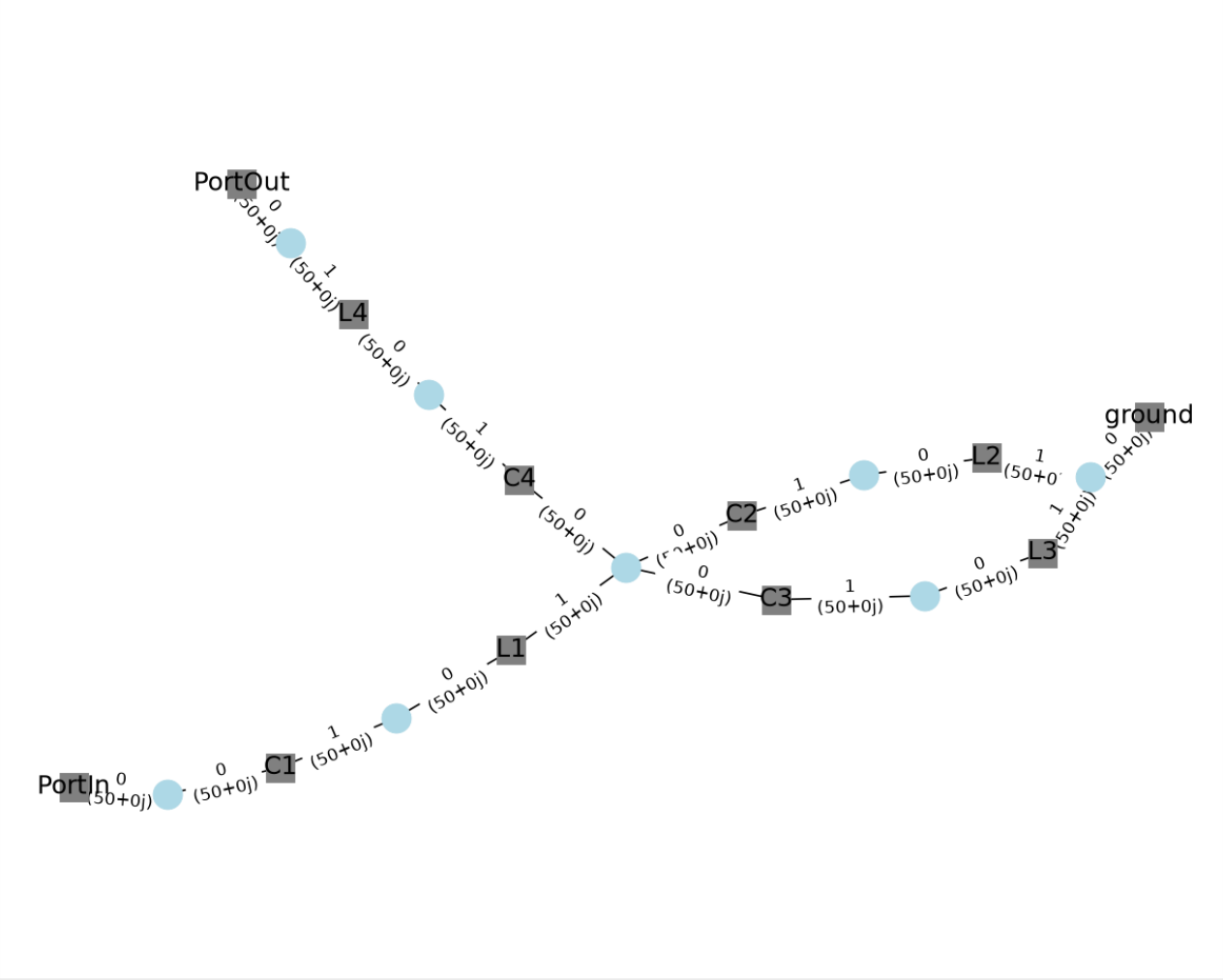
```
Port1 = Circuit.Port(frequency=freq, name='PortIn')
Port2 = Circuit.Port(frequency=freq, name='PortOut')
cnx = [
    [(Port1, 0), (C1, 0)],
    [(C1, 1), (L1, 0)],
    [(L1, 1), (C2, 0), (C3, 0), (C4, 0)],
    [(C2, 1), (L2, 0)],
    [(C3, 1), (L3, 0)],
    [(L2, 1), (L3, 1), (gnd, 0)],
    [(C4, 1), (L4, 0)],
    [(L4, 1), (Port2, 0)]
]
cir = Circuit(cnx)
ntw = cir.network
```

[9]:
```
ntw.plot_s_db(m=0, n=0, lw=2)
ntw.plot_s_db(m=1, n=0, lw=2)
```



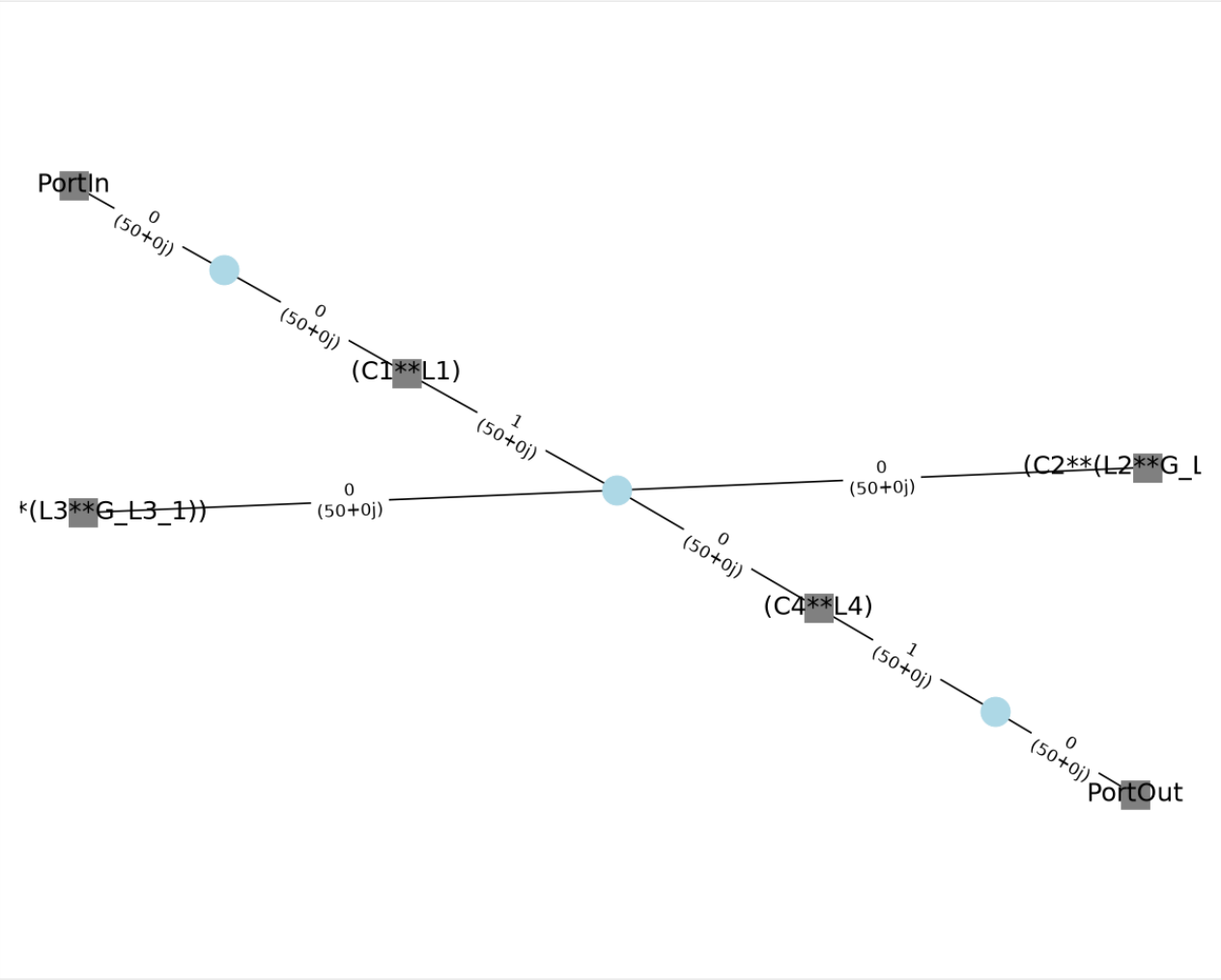
As in the previous example, we can use Circuit.plot_graph() method to represent the connection graphically.

[10]:
```
cir.plot_graph(network_labels=True, network_fontsize=15,
               port_labels=True, port_fontsize=15,
              edge_labels=True, edge_fontsize=10)
```



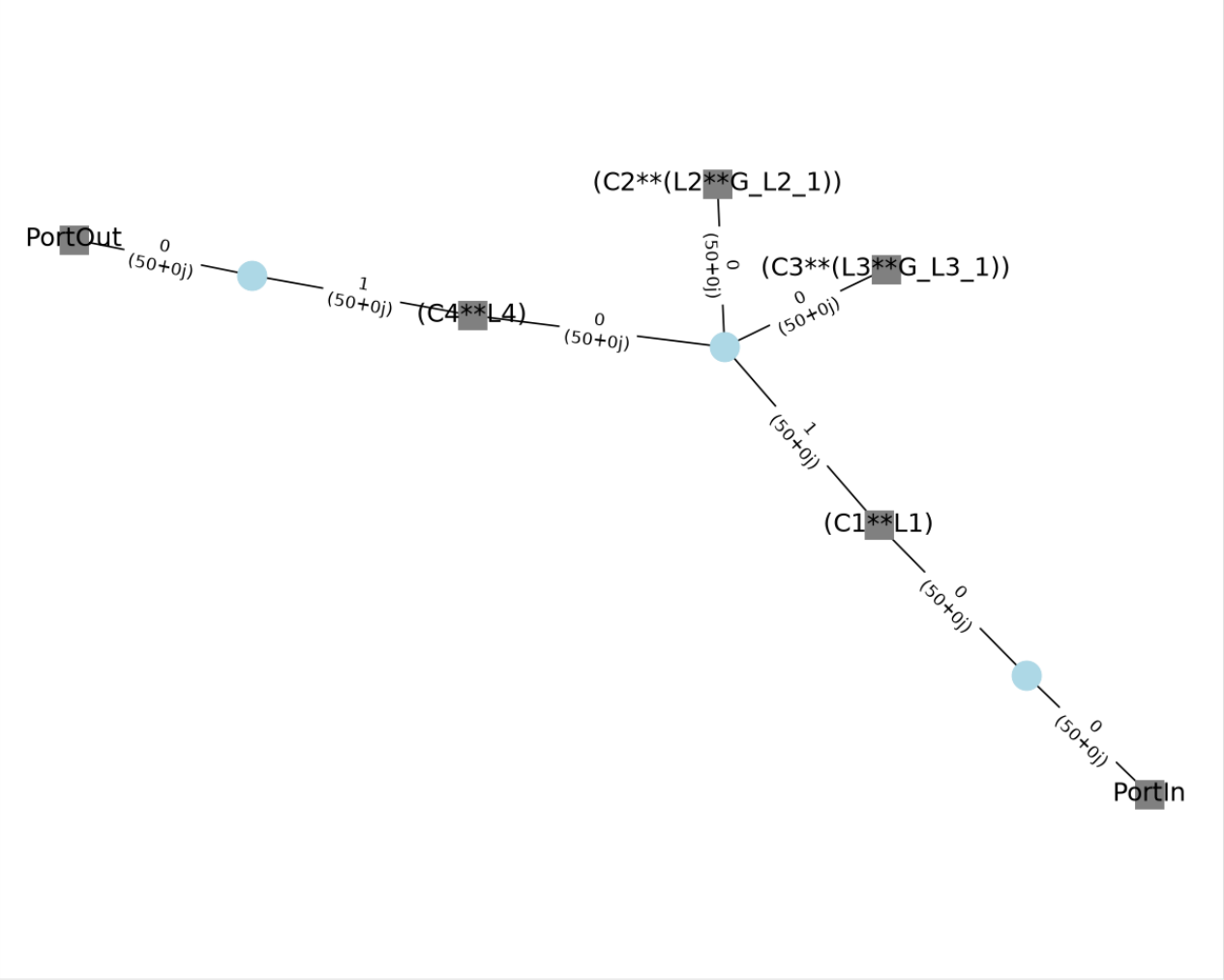
When focusing solely on `Ports` and disregarding internal `Networks`, we could utilize rf.reduce_circuit() to streamline the circuit. This method will automatically series-connect adjacent networks through a `rf.connect()` approach, retaining only `Ports` and connections containing more than 2 `Networks` for circuit analysis.

[11]:
```
reduced_cnx = rf.circuit.reduce_circuit(cnx)
reduced_cir = Circuit(reduced_cnx)
reduced_cir.plot_graph(network_labels=True, network_fontsize=15,
               port_labels=True, port_fontsize=15,
              edge_labels=True, edge_fontsize=10)
```



Additionally, optional parameters can influence the behavior of circuit reduction. For instance, `split_ground=True` indicates splitting the shared `Ground` in the circuit:

[12]:
```
fully_reduced_cnx = rf.circuit.reduce_circuit(cnx, split_ground=True)
fully_reduced_cir = Circuit(fully_reduced_cnx)

fully_reduced_cir.plot_graph(network_labels=True, network_fontsize=15,
                 port_labels=True, port_fontsize=15,
                edge_labels=True, edge_fontsize=10)
```



Finally, the Circuit constructor offers an option for automatic circuit simplification. Setting `auto_reduce=True` enables the creation of a significantly streamlined `Circuit` object, enhancing efficiency for Port's S-parameter calculations.

[13]:
```
from timeit import timeit

import numpy as np

fully_reduced_ntw = fully_reduced_cir.network
auto_reduced_cir = Circuit(cnx, auto_reduce=True)
auto_reduced_ntw = auto_reduced_cir.network
n_times = 10
print("Unreduced Circuit calculate Network in "
      f"{timeit(lambda: cir.network, number=n_times)/n_times:.4f}s")
print("Auto-reduced Circuit calculate Network in "
      f"{timeit(lambda: auto_reduced_cir.network, number=n_times)/n_times:.4f}s")

print("Manually reduced circuit has the same s-parameters as the original circuit: "
      f"{np.allclose(ntw.s, fully_reduced_ntw.s)}")
print("Auto reduced circuit has the same s-parameters as the original circuit: "
      f"{np.allclose(ntw.s, auto_reduced_ntw.s)}")
```

```
Unreduced Circuit calculate Network in 0.0015s
Auto-reduced Circuit calculate Network in 0.0005s
Manually reduced circuit has the same s-parameters as the original circuit: True
Auto reduced circuit has the same s-parameters as the original circuit: True
```

1.8.3 References

[1] Hallbjörner, P., 2003. Method for calculating the scattering matrix of arbitrary microwave networks giving both internal and external scattering. Microw. Opt. Technol. Lett. 38, 99–102. https://doi.org/10/d27t7m

[ ]:

Download This Notebook: `Calibration.ipynb`

1.9 Calibration

1.9.1 Introduction

This tutorial describes how to use **scikit-rf** to calibrate data taken from a VNA. For an introduction to VNA calibration see this article by Rumiantsev and Ridler [1]., for an outline of different calibration algorithms see Doug Ryttings presentation [2]. If you like to read books, then you may want to checkout [3].

What follows are examples of how to calibrate a device under test (DUT), assuming you have measured an acceptable set of standards, and have a corresponding set ideal responses. This may be referred to as *offline* calibration, because it is not occurring onboard the VNA itself. One benefit of this technique is that it provides maximum flexibility for non-conventional calibrations, and preserves all raw data.

There are many calibration algorithms implemented in scikit-rf. The choice between them depends of the media, the available calibration standards and the desired trade-off between accuracy and effort. The calibration algorithms available in scikit-rf are listed in the *Calibration* API's page.

1.9.2 Creating a Calibration

Calibrations are performed through a *Calibration* class. In General, *Calibration* objects require two arguments:

- a list of measured `Network`'s
- a list of ideal `Network`'s

The `Network` elements in each list must all be similar (same number of ports, frequency info, etc) and must be aligned to each other, meaning the first element of ideals list must correspond to the first element of measured list. Self calibration algorithms, such as TRL, do not require predefined ideal responses.

1.9.3 One-Port Example

This example code is written to be instructive, not concise. To construct a one-port calibration you need to have measured at least three standards and have their known *ideal* responses in the form of `Networks`. These `Network` can be created from touchstone files, a format all modern VNA's support.

In the following script the measured and ideal touchstone files for a conventional short-open-load (SOL) calkit are in folders `measured/` and `ideal/`, respectively. These are used to create a `OnePort` Calibration and correct a measured DUT

```
import skrf as rf
from skrf.calibration import OnePort

## created necessary data for Calibration class

# a list of Network types, holding 'ideal' responses
my_ideals = [\
        rf.Network('ideal/short.s1p'),
        rf.Network('ideal/open.s1p'),
        rf.Network('ideal/load.s1p'),
        ]

# a list of Network types, holding 'measured' responses
my_measured = [\
        rf.Network('measured/short.s1p'),
        rf.Network('measured/open.s1p'),
        rf.Network('measured/load.s1p'),
        ]

## create a Calibration instance
cal = OnePort(\
        ideals = my_ideals,
        measured = my_measured,
        )


## run, and apply calibration to a DUT

# run calibration algorithm
cal.run()

# apply it to a dut
dut = rf.Network('my_dut.s1p')
dut_caled = cal.apply_cal(dut)
dut_caled.name =  dut.name + ' corrected'

# plot results
dut_caled.plot_s_db()
# save results
dut_caled.write_touchstone()
```

Concise One-port

This example achieves the same task as the one above except in a more concise *programmatic* way.

```
import skrf as rf
from skrf.calibration import OnePort

my_ideals = rf.io.load_all_touchstones('ideals/')
```

(continues on next page)

(continued from previous page)

```
my_measured = rf.io.load_all_touchstones('measured/')


## create a Calibration instance
cal = OnePort(\
    ideals = [my_ideals[k] for k in ['short','open','load']],
    measured = [my_measured[k] for k in ['short','open','load']],
    )

## what you do with 'cal' may  may be similar to above example
```

1.9.4 Two-port Calibrations

Naturally, two-port calibration is more involved than one-port. **skrf** supports a few different two-port algorithms. The traditional `SOLT` algorithm uses the 12-term error model. This algorithms is straightforward, and similar to the OnePort example.

The `EightTerm` calibration is based on the algorithm described in [4], by R.A.Speciale. It can be constructed from any number of standards, providing that some fundamental constraints are met. In short, you need three two-port standards; one must be transmissive, and one must provide a known impedance and be reflective. Note, that the word *8-term* is used in the literature to describe a specific error model used by a variety of calibration algorithms, like TRL, LRM, etc. The `EightTerm` class, is an implementation of the algorithm cited above, which does not use any self-calibration.

One important detail of using the 8-term error model formulation is that switch-terms may need to be measured in order to achieve a high quality calibration (thanks to Dylan Williams for pointing this out). These are described next.

Switch-terms

Originally described by Roger Marks[5], switch-terms account for the fact that the 8-term (aka *error-box*) model is overly simplified. The two error networks change slightly depending on which port is being excited. This is due to the internal switch within the VNA. Hence, switch terms describe the imperfect load when the source is switch to the other port.

Switch terms can be directly measured with a low insertion loss transmissive standard (like a thru) connected between the cables connected to the instrument ports. Sometime, a custom measurement configuration is available on the VNA itself.

skrf has support for measuring switch terms in the `skrf.vi.vna` module, see the HP8510's `skrf.vi.vna.hp.HP8510C.get_switch_terms`, or PNA's `skrf.vi.vna.keysight.get_switch_terms` . Without switch-term measurements, your calibration quality will vary depending on properties of you VNA.

Using one-port ideals in two-port Calibration

Commonly, you have data for ideal data for reflective standards in the form of one-port touchstone files (ie `.s1p`). To use this with skrf's two-port calibration method you need to create a two-port network that is a composite of the two networks. The function `skrf.network.two_port_reflect` does this

```
short = rf.Network('ideals/short.s1p')
shorts = rf.network.two_port_reflect(short, short)
```

SOLT Example

Two-port calibration is accomplished in an identical way to one-port, except all the standards are two-port networks. This is even true of reflective and load standards (S21=S12=0). So if you measure reflective or load standards you should measure two of them simultaneously, and store information in a two-port. For example, connect a short to port-1 and a short to port-2, and save a two-port measurement as 'short,short.s2p' or similar.

Alternatively, for a `SOLT`, `TwelveTerm`, or `EightTerm` calibration, it's also acceptable to measure one port at a time while leaving the other unused port open. Later, two one-port networks can be combined to a two-port network using `skrf.network.two_port_reflect`, explained above.

By default, no isolation calibration takes place. For reflective and even load standards, S_{21} and S_{12} are ignored. If isolation calibration is needed, one must explicitly tell scikit-rf to do so. Specifically, an optional parameter `isolation` is used for this purpose. To calibrate isolation, set this parameter to a two-port network, representing the measurement result with loads on both ports.

It should be noted that the less common (but still *supported* `SixteenTerm` calibration is a notable exception: this calibration is specifically designed to eliminate port leakages better than the standard `SOLT` model. Measuring a combination of standards at both port simultaneously is *required*.

```
import skrf as rf
from skrf.calibration import SOLT



# a list of Network types, holding 'ideal' responses
my_ideals = [
    rf.Network('ideal/short, short.s2p'),
    rf.Network('ideal/open, open.s2p'),
    rf.Network('ideal/load, load.s2p'),
    rf.Network('ideal/thru.s2p'),
    ]

# a list of Network types, holding 'measured' responses
my_measured = [
    rf.Network('measured/short, short.s2p'),
    rf.Network('measured/open, open.s2p'),
    rf.Network('measured/load, load.s2p'),
    rf.Network('measured/thru.s2p'),
    ]


## create a SOLT instance
cal = SOLT(
    ideals = my_ideals,
    measured = my_measured,
    rf.Network('measured/load, load.s2p'),
    # isolation calibration is optional, it can be removed.
    )


## run, and apply calibration to a DUT

# run calibration algorithm
cal.run()
```

(continues on next page)

(continued from previous page)

```
# apply it to a dut
dut = rf.Network('my_dut.s2p')
dut_caled = cal.apply_cal(dut)

# plot results
dut_caled.plot_s_db()
# save results
dut_caled.write_touchstone()
```

1.9.5 Saving and Recalling a Calibration

Calibration's can be written-to and read-from disk using the temporary storage container of pickles. Writing can be accomplished by using `Calibration.write`, or `rf.io.write()`, and reading is done with `rf.io.read()`. Since these functions rely on pickling, they are not recommended for long-term data storage. Currently there is no way to achieve long term storage of a Calibration object, other than saving the script used to generate it.

1.9.6 References

[1] Andrej Rumiantsev and Nick Ridler, "VNA Calibration", IEEE Microwave Magazine 2008

[2] Doug Rytting, "Network Analyzer, Error Models and Calibration Methods"

[3] J. P. Dunsmore, "Handbook of microwave component measurements: with advanced vna techniques". Hoboken, N.J: Wiley, 2012.

[4] R.A. Speciale, "A Generalization of the TSD Network-Analyzer Calibration Procedure, Covering n-Port Scattering-Parameter Measurements, Affected by Leakage Errors," Microwave Theory and Techniques, IEEE Transactions on , vol.25, no.12, pp. 1100- 1115, Dec 1977. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1129282&isnumber=25047

[5] Roger B. Marks, "Formulations of the Basic Vector Network Analyzer Error Model including Switch-Terms," ARFTG Conference Digest-Fall, 50th , vol.32, no., pp.115-126, Dec. 1997. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4119948&isnumber=4119931

[ ]:

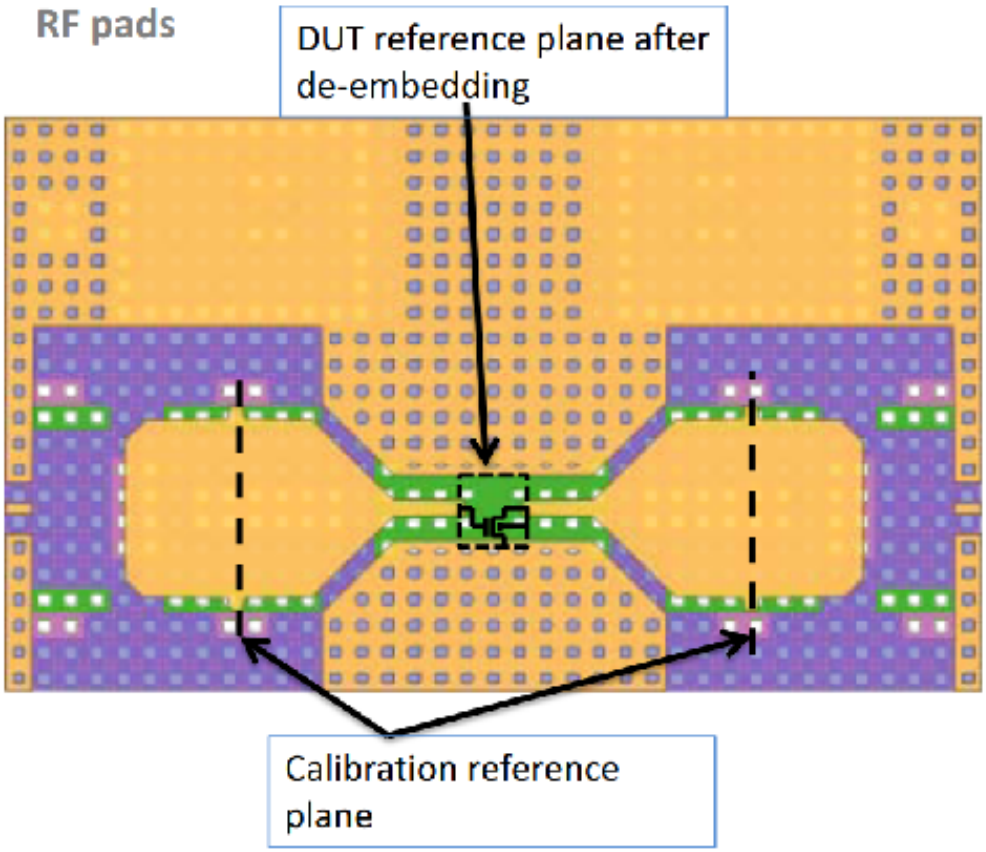
1.10 De-embedding

This is a short tutorial on how de-embedding can be performed using scikit-rf. We will start by introducing the concept of de-embedding and why it is required, with a simple use-case scenario. Next, we will go through what kinds of de-embedding exist, and how to choose the right one for your application. Lastly, we will see how code can be written to quickly perform de-embedding on your s-parameter datasets.

1.10.1 What is de-embedding and how does it differ from calibration?

Let's start with an example. Consider the measurement of integrated RF transistors built on a silicon wafer which are vital to the development of compact models for integrated circuit design. The typical measurement setup for such a scenario involves the use of RF probes that have a co-axial connector at one end, and a ground-signal-ground (GSG) probe tip at the other. This probe tip lands on GSG pads constructed on-wafer that provides access to the transistor under test. To measure an RF transistor, the actual device-under-test (DUT) is connected to GSG pads at two of the three terminals of the transistor via metal interconnects, while connecting the third terminal to RF ground. In common-source measurement configuration of a FET, this means that the Gate and Drain terminals are connected to the RF probes, and the Source terminal is grounded. Now, s-parameters are measured as a function of terminal voltages on the transistor.

But what is the calibration reference plane for this transistor s-parameter measurement? The effects of the cabling and probe transitions can be removed by performing a standard calibration using methods such as Thru-Reflect-Line (TRL) or Short-Open-Load-Thru (SOLT) to move the calibration reference to the probe tips, or in other words, to the GSG pads implemented on-wafer. The use of an Impedance Standard Substrates (ISS) provides a set of well-defined calibration standards that can be used to establish such a reference plane for measurement. However, there is still a substantial "test-fixture" involving metal interconnects that need to be removed before accessing the real terminals of the RF transistor whose measurement we actually want. **De-embedding refers to the removal of extraneous effects that a test fixture can have on the measurement of a device under test (DUT).** The figure below (obtained from here) shows an example of the "test-fixture" that often has to be removed in on-wafer measurement applications.



The de-embedding process differs from calibration in that no well known standards are used either because they cannot exist in certain environments, or are not practical to implement given space and cost constraints. De-embedding uses several dummy structures that help remove unwanted test fixture effects, but does not provide enough information to deduce a complete error box network like those obtained using standard calibration techniques. Since the test fixture itself may contain various transitions and interconnects before reaching the DUT, de-embedding is useful when simple scalar port extension of the reference plane is not applicable.

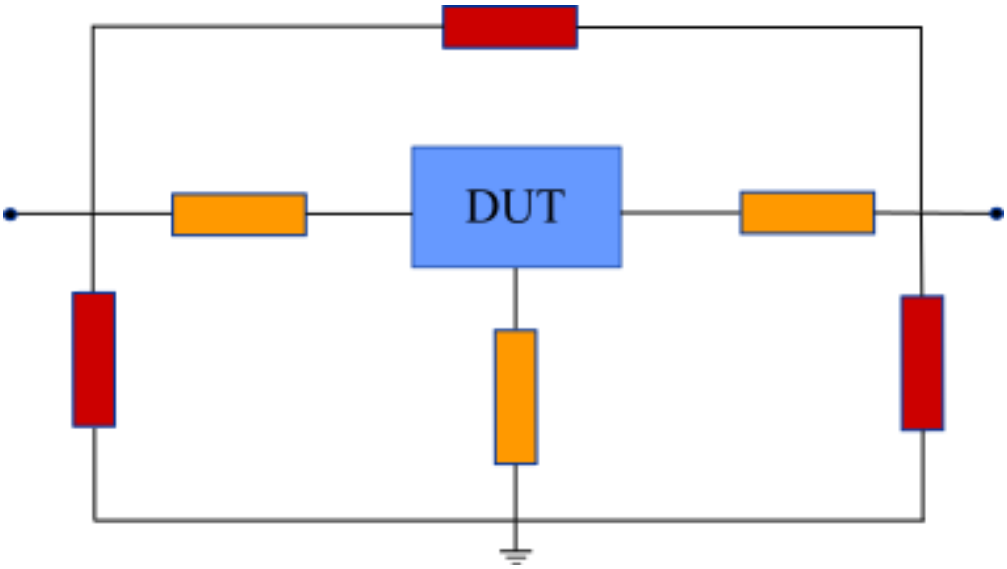
For a basic introduction to this topic, refer this article.

1.10.2 Open-Short De-embedding

In the last few decades of on-wafer RF measurements, the Open-Short de-embedding method has been the work horse of the RFIC industry where the frequencies of transistor operation have predominantly been in the range of a few gigahertz. As transistor measurements are extended to higher frequencies, more sophisticated de-embedding methods are required when the simplifying assumptions of Open-Short de-embedding break down. It is not easy to define a frequency limit up to which Open-Short de-embedding is valid because it depends on the layout techniques adopted in the design of on-wafer pad cages. If proper design techniques are adopted, Open-Short de-embedding should be

applicable at least to 10 GHz if not more.

The accuracy of Open-Short de-embedding depends on the validity of assumption that the test-fixture parasitics are a combination of parallel conductances (in red) and series impedances (in orange) as shown in the figure below. The lumped element model is usually representative of an on-wafer test-fixture, because first there is the shunt pad capacitance in the GSG pads, followed by the series impedance of the interconnect lines. To move the reference plane of the measurements from the plane of calibration (the outer terminals of the network below) to the DUT, two dummy structures - Open and Short - are needed in addition to the DUT. To create an Open dummy, the DUT is simply removed from the test fixture, while the three terminals are shorted together to implement the Short dummy. With the help of these dummies, the real measurements of the DUT can be extracted.

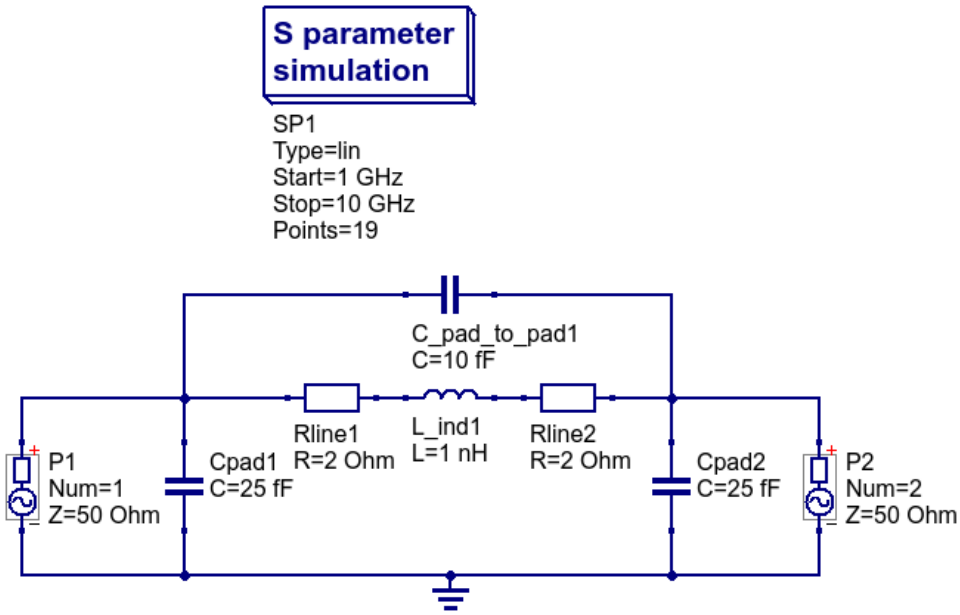


The detailed procedure of OpenShort de-embedding is shown in the figure below. The measurement result for the open dummy are clearly similar to those for parallel parasitics. The parallel parasitics are extracted simply by measuring the open dummies (step i). On the other hand, the short dummy includes both parallel and series parasitics. The series parasitics are extracted by subtracting the parallel parasitics from the short dummy in the Y-parameter domain (step ii). By subtracting the parallel and series parasitics from the DUT measurement results in the Y- and Z-parameter domains, respectively, a DUT without parasitic components can be extracted (step iii).

The reader is referred to this detailed presentation for intricate details regarding the design of on-wafer test structures and best practices. From here on, we will focus on explaining how the de-embedding class in scikit-rf works.

1.10.3 De-embedding with Scikit-RF

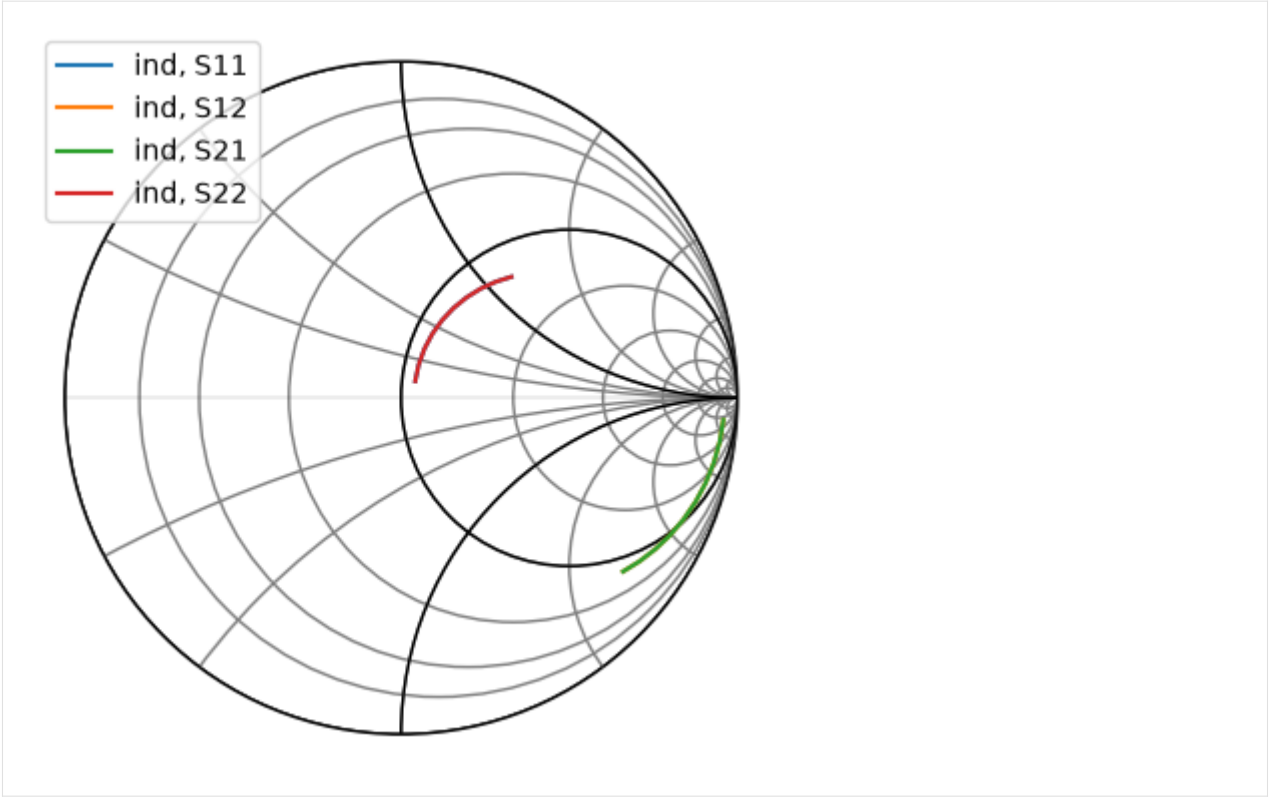
In this section, let us build up a concrete example to demonstrate how de-embedding in scikit-rf works. Assume that the device under test is a 1nH inductor whose measurements we are interested in. Since this inductor must be placed in an on-wafer test-fixture, let us assume that the pad capacitance at each port is 25fF, the pad-pad capacitance is 10fF, and the resistance of interconnect lines from each pad are 2-ohms each. The resulting network whose measurements are available at the external reference plane P1-P2, is shown below.



The goal is to accurately extract the actual 1nH inductor by removing all other extraneous parasitic circuit elements.

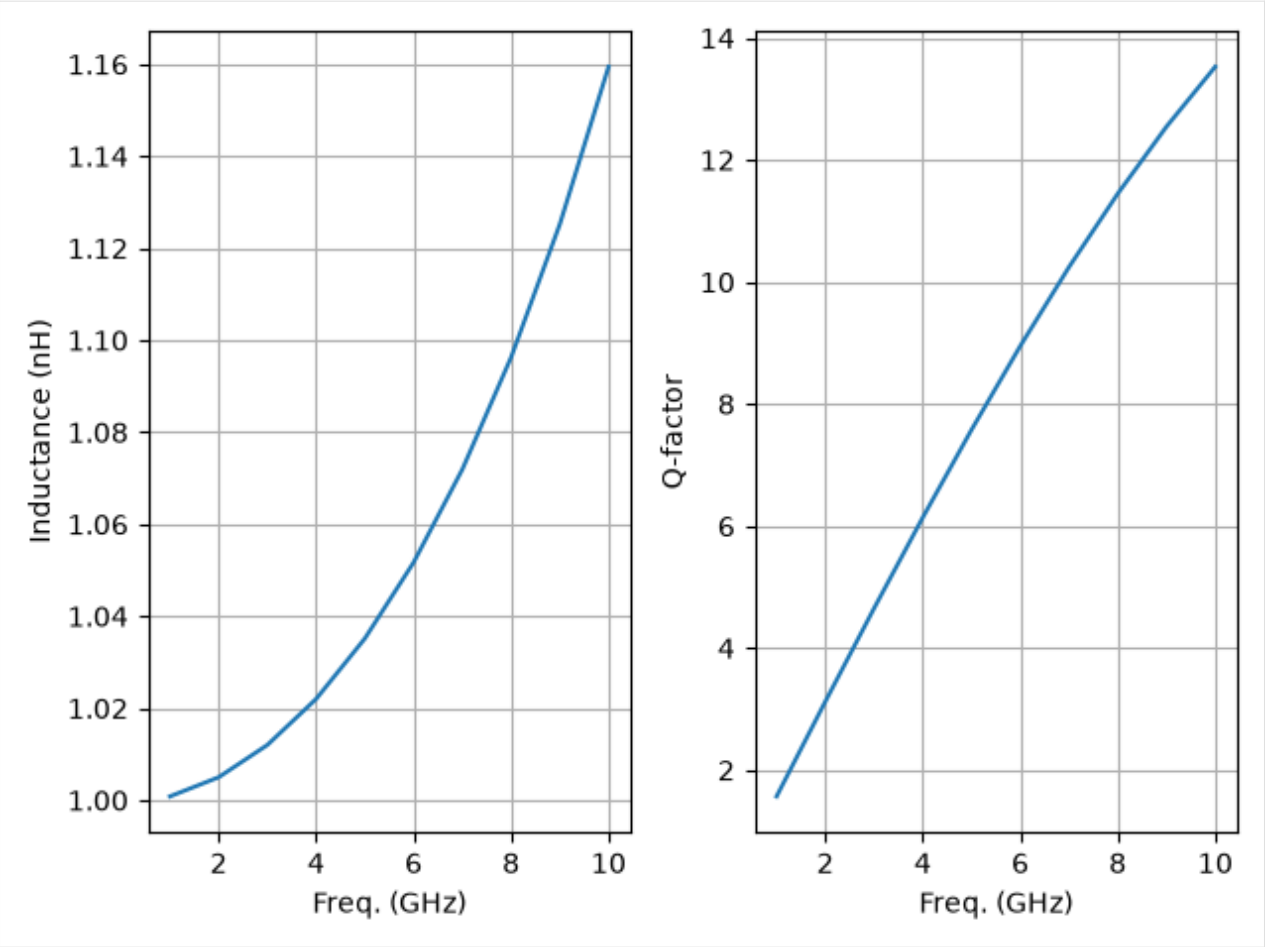
[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf
```

[2]:
```
# Look at the raw inductor measurement with parasitics included
# From S11/S22, it is clear that it is not a pure inductance.
raw_ind = rf.data.ind
raw_ind.plot_s_smith()
```

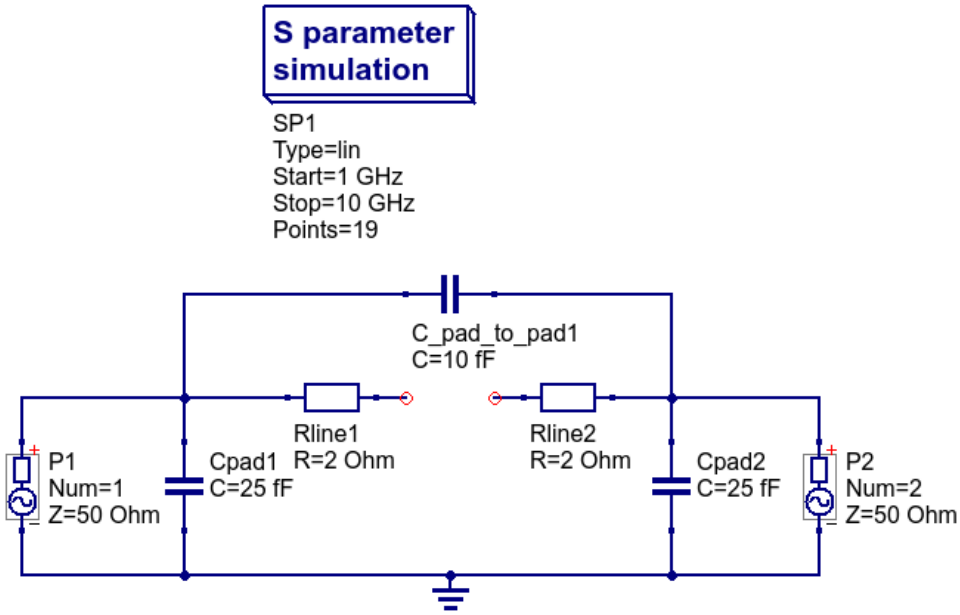


If we were to extract inductance directly from this measurement, we will have an inductance value that changes with frequency, and the quality factor will also be influenced by the presence of parasitic resistors.

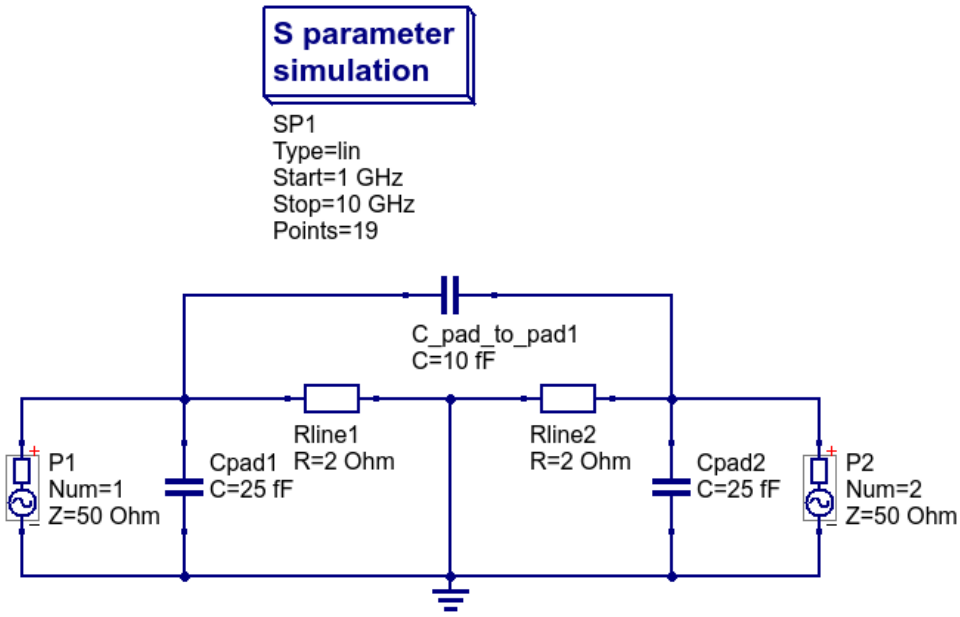
[3]:
```
# plot the inductance and q-factor of the raw inductor measurement
Lraw_nH = 1e9 * np.imag(1/raw_ind.y[:,0,0])/2/np.pi/raw_ind.f
Qraw = np.abs(np.imag(1/raw_ind.y[:,0,0])/np.real(1/raw_ind.y[:,0,0]))

fig, (ax1, ax2) = plt.subplots(1,2)
ax1.plot(raw_ind.f*1e-9, Lraw_nH)
ax1.grid()
ax1.set_ylabel("Inductance (nH)")
ax1.set_xlabel("Freq. (GHz)")
ax2.plot(raw_ind.f*1e-9, Qraw)
ax2.grid()
ax2.set_ylabel("Q-factor")
ax2.set_xlabel("Freq. (GHz)")
fig.tight_layout()
```



To create an open dummy structure, the DUT, which is the 1nH inductor here is simply removed from the DUT test structure resulting in the circuit shown below.



To create a short dummy structure, the internal terminals of the test-fixture that would otherwise connect to the DUT are shorted to ground as shown below.

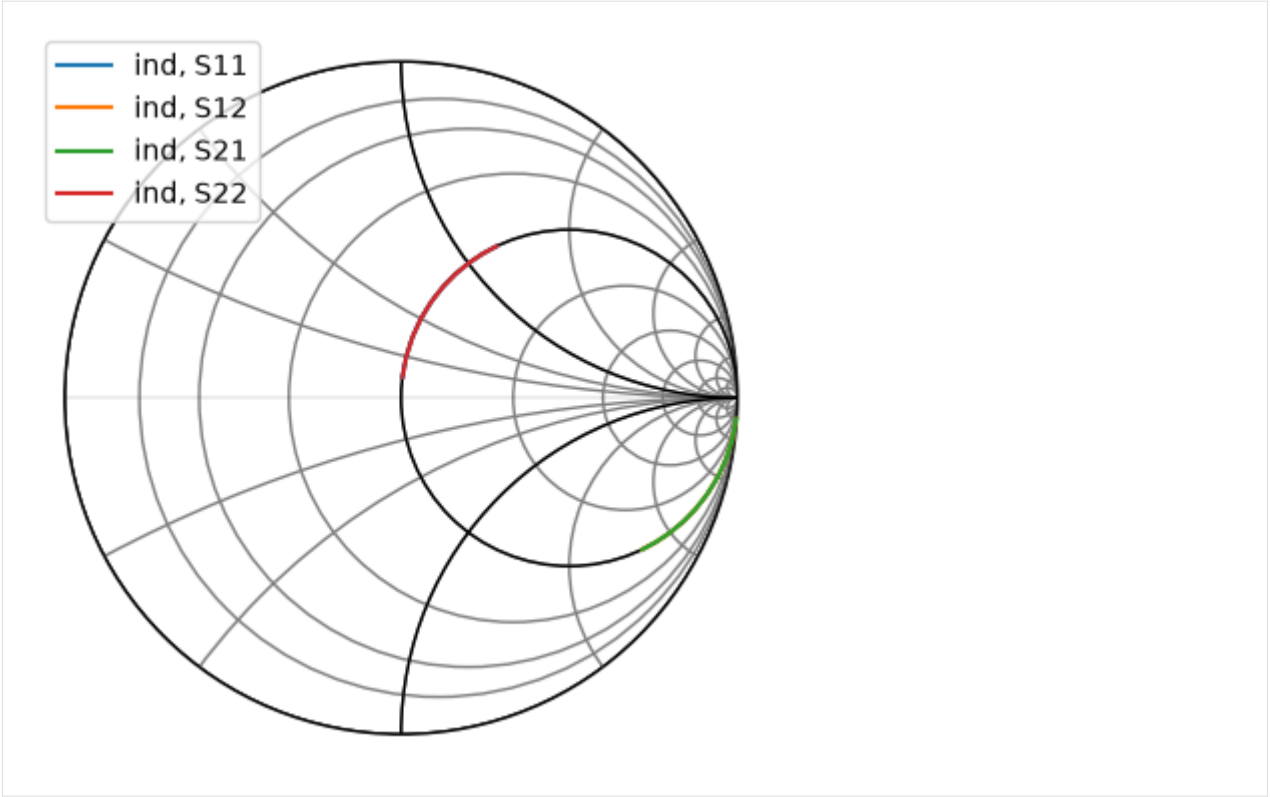


[4]:
```
# load in 2-ports short/open dummy networks
open_nw = rf.data.open_2p
short_nw = rf.data.short_2p
```

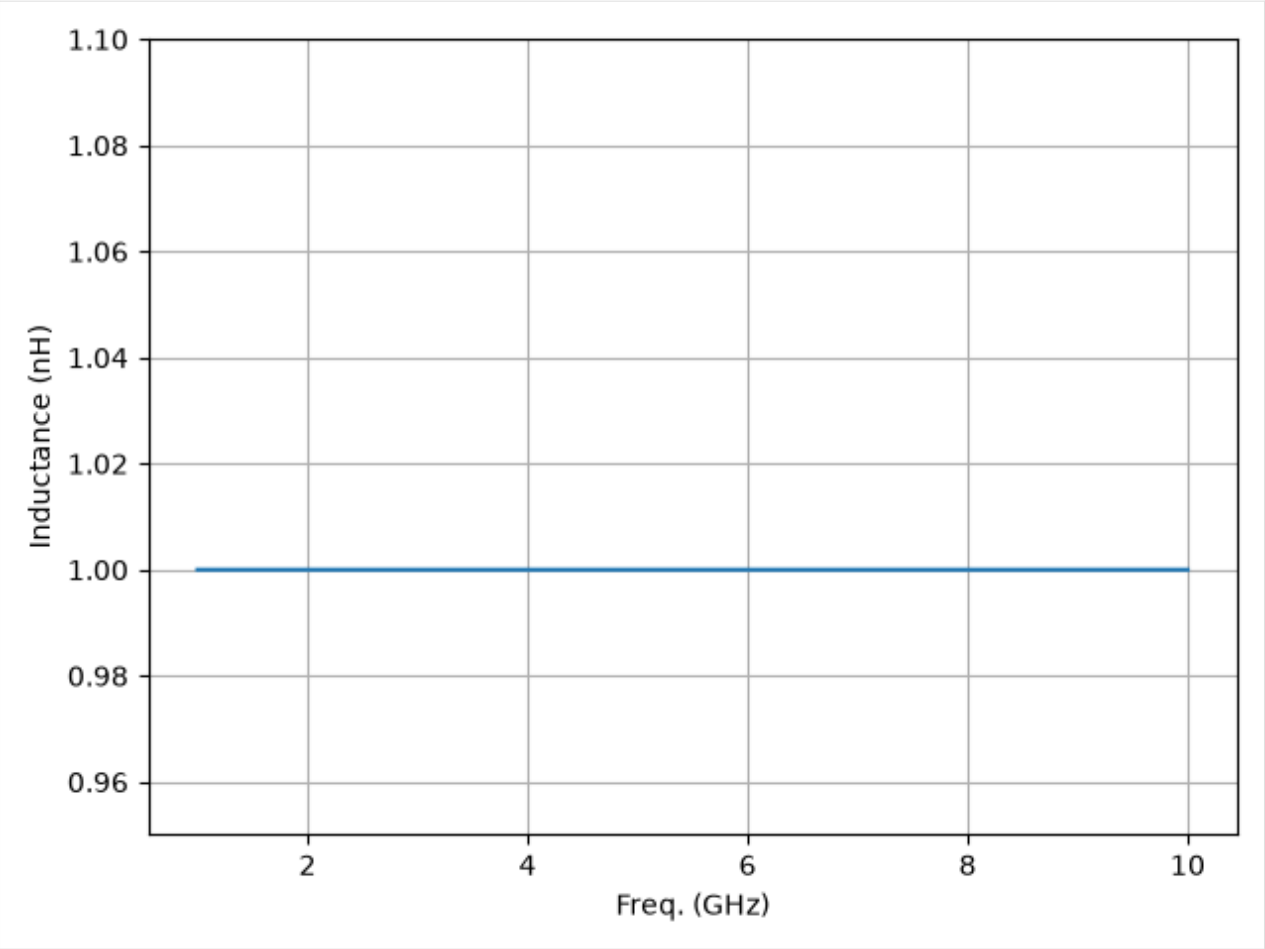
To perform Open-Short de-embedding with the available dummy measurements, we create a de-embedding object as an instance of the `OpenShort` class, while providing the open and short network objects as arguments to the de-embedding object creation.

To get the de-embedded network, we apply the `deembed` method on the network object on which we want to perform the de-embedding operation. This is shown below.

[5]:
```
from skrf.calibration.deembedding import OpenShort

dm = OpenShort(dummy_open=open_nw, dummy_short=short_nw, name='tutorial')

actual_ind = dm.deembed(raw_ind)
actual_ind.plot_s_smith()
```



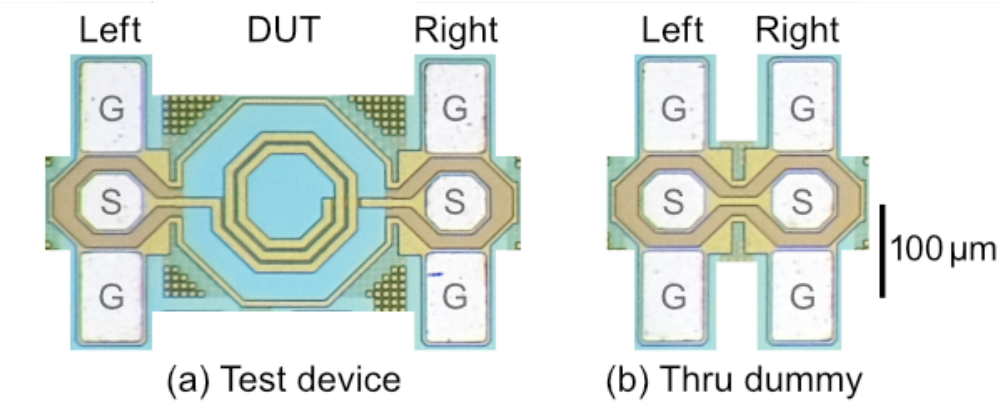
[6]:
```
# plot the inductance of the de-embedded measurement
# we ignore plotting Q-factor here, because an ideal lossless inductor has infinite Q

Lactual_nH = 1e9 * np.imag(1/actual_ind.y[:,0,0])/2/np.pi/actual_ind.f

fig, ax1 = plt.subplots(1,1)
ax1.plot(actual_ind.f*1e-9, Lactual_nH)
ax1.grid()
ax1.set_ylim(0.95, 1.1)
ax1.set_ylabel("Inductance (nH)")
ax1.set_xlabel("Freq. (GHz)")
fig.tight_layout()
```



From the plot above, it is seen that the actual inductor value is accurately extracted even in the presence of unwanted parasitic elements due to proper de-embedding of the test-fixture.

1.10.4 Through-only de-embedding

At higher frequencies above 10 GHz, the fringe capacitance of the open dummy and the parasitic inductance of the short dummy cause errors. For high frequencies, several methods using a through (thru) dummy have been proposed, such as `SplitPi`[1], `SplitTee`[2], `AdmittanceCancel`[3], and `ImpedanceCancel`[4]. The thru dummy is a direct connection of the left and right test pads. The example of the test device and thru dummy is shown in the figure below, where G and S indicate ground pads and signal pads.



Split methods

Split methods are based on lumped elements assumption as well as the open or short methods. The `SplitPi` and `SplitTee` methods assume that the embedding-networks are -network and T-network, respectively, as shown in the figure below. The values of the left and right parasitics (i.e. Z and Y) are extracted from the thru dummy. The parasitics are removed by multiplying by the inverse of the left and right ABCD matrix. **These methods can apply to only 2-port DUTs**.

Immittance cancel methods

The `AdmittanceCancel` (a.k.a Mangan's method) and `ImpedanceCancel` methods are also based on lumped elements assumption as shown in the figure below. However, unlike the split methods, they do NOT directly calculate the immittance of the left and right parasitics. These methods de-embed the DUTs by left-right mirroring operation which is called 'swapping'. The `AdmittanceCancel` method consists of two steps. First, an matrix defined as 'H' is calculated by multiplying the ABCD matrix of the test device by inverse matrix of thru dummy as shown in figure(c). Then, the de-embedded Y matrix is obtained by taking the average of the left-right mirrored and un-mirrored Y-parameters of H. Similarly, in the case of the `ImpedanceCancel` method, it takes the average of the Z-parameters. **These methods can apply to only symmetric 2-port DUTs (i.e. S11=S22 and S12=S21)**, but more accurate than the split methods in the characterization of transmission lines at mmW frequencies. A comparison between the split methods and immittance cancel methods is described in [4].

1.10.5 Other De-embedding Methods

Other simple forms of de-embedding included in scikit-rf are `Open`, `Short` and `ShortOpen` methods, which may be applicable depending on the equivalent circuit of the parasitic network of the test-fixture. As an example, it may be desirable to only remove the pad contact resistance from the measurement for which `Short` de-embedding can be used. In certain measurements, only the pad capacitance might need to be removed, for which the `Open` de-embedding method would be more appropriate. To remove pad contact resistance and pad capacitance in one operation, the `ShortOpen` method would be the most suitable.

Many other sophisticated methods of de-embedding have been reported in literature to extend the accuracy of DUT measurements up to high frequencies. While they can be done using the network operations of scikit-rf, their inclusion as a built-in de-embedding class within the package is welcomed as an open-source contribution to the project.

1.10.6 References

[1]: L. Nan et al., "Experimental Characterization of the Effect of Metal Dummy Fills on Spiral Inductors," in 2007 IEEE Radio Frequency Integrated Circuits (RFIC) Symposium, Jun. 2007, pp. 307–310.

[2]: M. J. Kobrinsky et al., "Experimental validation of crosstalk simulations for on-chip interconnects using S-parameters," IEEE Transactions on Advanced Packaging, vol. 28, no. 1, pp. 57–62, Feb. 2005.

[3]: A. M. Mangan et al., "De-embedding transmission line measurements for accurate modeling of IC designs," IEEE Trans. Electron Devices, vol. 53, no. 2, pp. 235–241, Feb. 2006.

[4]: S. Amakawa et al., "Comparative analysis of on-chip transmission line de-embedding techniques," in 2015 IEEE International Symposium on Radio-Frequency Integration Technology, Aug. 2015, pp. 91–93.

[ ]:

1.11 Quality Factors

1.11.1 Resonant Circuits and Q-factor

Resonant circuits are used extensively in oscillators, tuned amplifiers, filters, etc. At a particular frequency, *a resonant frequency* f_r (or ω_r), resonators presents a maximum (or a minimum) impedance (ex: open or short circuit)[1]. The quality factor Q, or Q-factor, is a unitless figure of merit of the losses of a resonant passive circuit, defined as [2,3]:

$$Q = 2\pi \left.\frac{\text{Average Energy Stored}}{\text{Energy Loss per Second}}\right|_{\omega=\omega_r} = \omega_r \left.\frac{\text{Average Energy Stored}}{\text{Average Power Loss}}\right|_{\omega=\omega_r}$$

From this definition, lower loss implies a higher Q. Resonators having higher Q-factors resonate with greater amplitudes (at the resonant frequency) but have a smaller range of frequencies BW around that frequency for which they resonate.

1.11.2 Loaded of Unloaded Q-factors

Actually, there are three Q factors than can be defined depending upon which loss is being considered [2]:

- Unloaded Q:

$$Q_0 = \omega_r \left.\frac{\text{Energy Stored in the Resonant Circuit}}{\text{Power Loss in the Resonant Circuit}}\right|_{\omega=\omega_r}$$

- External Q:

$$Q_e = \omega_r \left.\frac{\text{Energy Stored in the Resonant Circuit}}{\text{Power Loss in the External Circuit}}\right|_{\omega=\omega_r}$$

- Loaded Q:

$$Q_L = \omega_r \left.\frac{\text{Energy Stored in the Resonant Circuit}}{\text{Total Power Loss}}\right|_{\omega=\omega_r}$$

The *loaded* Q-factor Q_L, describes energy dissipation within the entire resonant system comprising of the resonator itself and the instrument used for observing resonances [3]. The term *loading* refers to the effect that the external circuit has on measured quantities. Any loss mechanisms due to the external circuitry will have the effect of lowering the Q.

The *unloaded* Q-factor Q_0 is a characteristic of the resonator itself, in the absence of any loading effects caused by external circuitry (uncoupled). For most applications, the quantity that is desired is the unloaded Q-factor Q_0 which is determined by energy dissipation associated with the resonator only and therefore gives the best description of the resonant mode. Direct measurement of the unloaded Q of a resonator is generally not possible because of the loading effect of the measurement system. However, it is possible to estimate Q_0 from measurements of the frequency response of the loaded resonator.

The energy dissipation in the external circuit is characterized by the *external* Q-factor Q_e and these three Q-factors are related by the relationship (deduced from the three expressions above):

$$\frac{1}{Q_L} = \frac{1}{Q_e} + \frac{1}{Q_0}$$

If one defines the *coupling factor* $\beta = Q_0/Q_e$ then:

$$Q_0 = (1+\beta)Q_L$$

Three cases can be distinguished:

1. $\beta < 1$: The resonator is said to be *undercoupled* to the feeding circuitry
2. $\beta = 1$: The resonator is said to be *critically coupled*
3. $\beta > 1$: The resonator is said to be *overcoupled*

While the measurements of the loaded Quality factor Q_L is straightforward, obtaining uncertainty below 1% (which is considered to be low for Q-factor measurement) requires attention to several aspects of the experimental procedure. In addition, finding the unloaded Q_0 from measured S-parameters consists in first finding the coupling factor β, then measure Q_L from the 3dB bandwidth and using the relationships above.

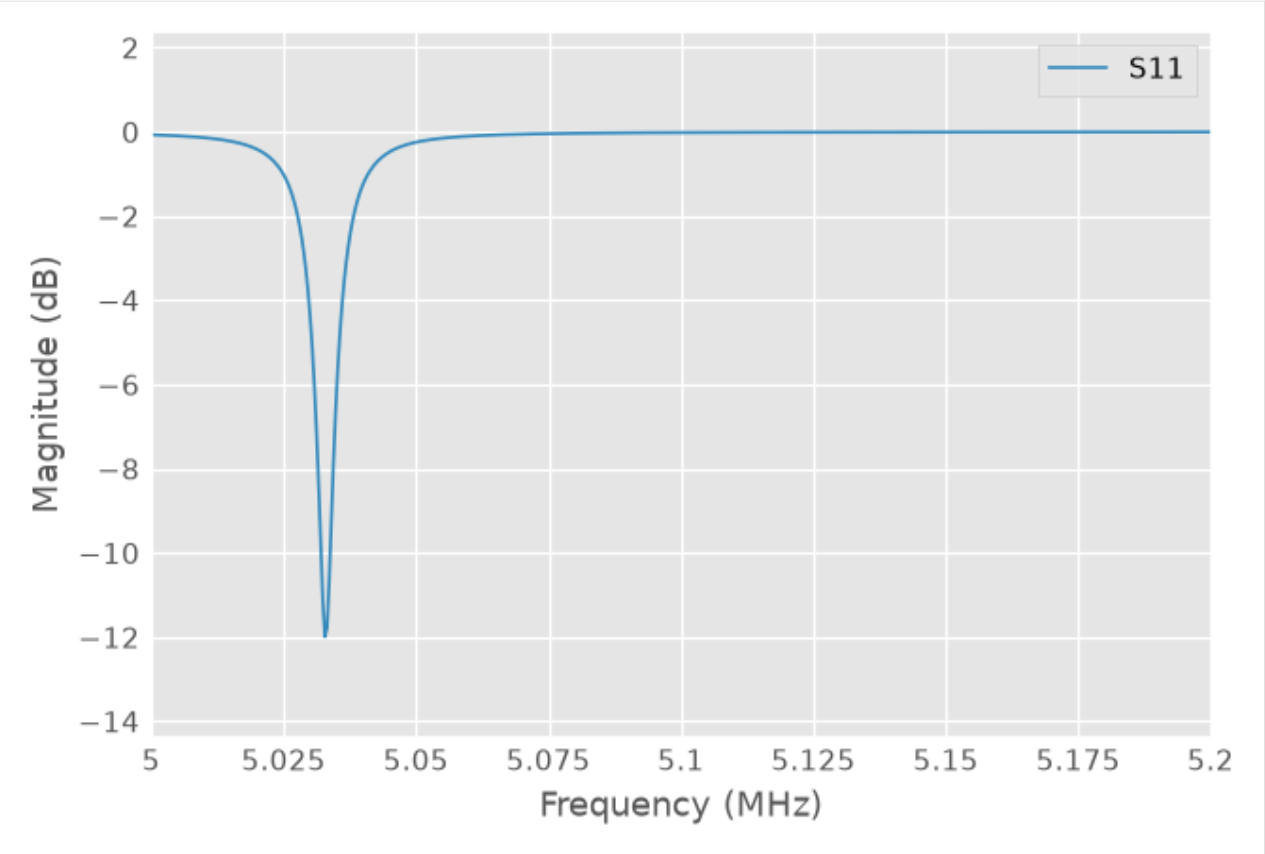
Fortunately, `scikit-rf` implements automatic methods for determining *loaded* and *unloaded* Q-factors from frequency-domain S-parameters. The implemented methods are described in detail in [4], and can be applied to measurements of transmission or reflection.

1.11.3 Example with a Parallel RLC Circuit

To illustrate the usage of the `Qfactor` class of `scikit-rf`, a parallel RLC circuit as illustrated above is used as an canonical example for which analytical formulas are available for benchmarking.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf

rf.stylely()
```

[2]:
```
C = 1e-6  # F
L = 1e-9  # H
R = 30  # Ohm
Z0 = 50  # Ohm

freq = rf.Frequency(5, 5.2, npoints=501, unit='MHz')
media = rf.media.DefinedGammaZ0(frequency=freq, z0=Z0)  # ideal line (no loss)
rng = np.random.default_rng()
random_d = rng.uniform(-np.pi, np.pi)  # a random length for the sake of the example

resonator = media.line(d=random_d, unit='rad') \
                **media.shunt_inductor(L) ** media.shunt_capacitor(C) \
                ** media.shunt(media.resistor(R)**media.short()) ** media.open()

resonator.plot_s_db()
```



Analytical formulas for this case are available and given by [2,3]:

$$f_{\text{res}} = \frac{1}{2\pi\sqrt{LC}}$$

$$Q = \omega_r RC = \frac{RC}{\sqrt{LC}}$$

[3]:
```
def f_res_RLC(L, C):
    return 1/(2*np.pi*np.sqrt(L*C))

def Q_RLC(R, L, C):
    return R * C / np.sqrt(L*C)
```

The unloaded Q-factor Q_0 is a characteristic of the resonator itself, in the absence of any loading effects caused by the external circuitry. In practice, of course, the resonator is connected and coupled to this circuitry which have the effect to lower the Q-factor of the loaded circuit Q_L.

Hence, when the resonator is connected to the transmission line, it is loaded with the port impedance Z_0 (assuming no loss in the transmission line), which is connected in parallel to the intrinsic resonator resistor. The equivalent resonator load is therefore $R \parallel Z_0 = (RZ_0)/(R + Z_0)$:

[4]:
```
print(f'Theoretical Resonant Frequency: {f_res_RLC(L, C)/1e6} MHz')
print(f'Theoretical Loaded Q: Q_L = {Q_RLC((R*Z0)/(R+Z0), L, C)}')  # Req = R//Z0
print(f'Theoretical Unloaded Q: Q_0 = {Q_RLC(R, L, C)}')
```

```
Theoretical Resonant Frequency: 5.032921210448704 MHz
Theoretical Loaded Q: Q_L = 592.9270612815711
Theoretical Unloaded Q: Q_0 = 948.6832980505137
```

First, let's create a `Qfactor` object, passing the resonator Network and the type of resonator we are dealing with:

[5]:
```
Q = rf.qfactor.Qfactor(resonator, res_type='reflection')
```

Note that in case of multiple resonances (*like in this example*), it is recommended to also pass the estimated resonance frequency and eventually the estimated (order of magnitude) Q-factor.

Then, we use the `fit` method to fit the loaded Q-factor Q_L and resonant frequency. The returned results will be useful to deduce the unloaded Q-factor.

[6]:
```
res = Q.fit()
print(f'Fitted Resonant Frequency: f_L = {Q.f_L/1e6} MHz')
print(f'Fitted Loaded Q-factor: Q_L = {Q.Q_L}')
```

```
Fitted Resonant Frequency: f_L = 5.032919422706801 MHz
Fitted Loaded Q-factor: Q_L = 592.9272284660565
```

Note that these results are also shown if you print the `Qfactor` object:

[7]:
```
Q
```

[7]:
```
Q-factor of Network None. (fitted: f_L=5.033MHz, Q_L=592.927)
```

Finally, the unloaded Q-factor is deduced from the fitting properties:

[8]:
```
Q0 = Q.Q_unloaded(res)
print(f'Fitted Unloaded Q-factor: Q_0 = {Q0}')
```

```
Fitted Unloaded Q-factor: Q_0 = 948.6836129993933
```

Note that passing the result is optional. Calling the `.fit()` method will store internally the optimized results and use it if a specific optimized result is not passed:

[9]:
```
Q0 = Q.Q_unloaded()  # will use the latest optimized results performed with .fit()
print(f'Fitted Unloaded Q-factor: Q_0 = {Q0}')
```

```
Fitted Unloaded Q-factor: Q_0 = 948.6836129993933
```

Note that the analytical results and the fitted ones match well, the relative error is small:

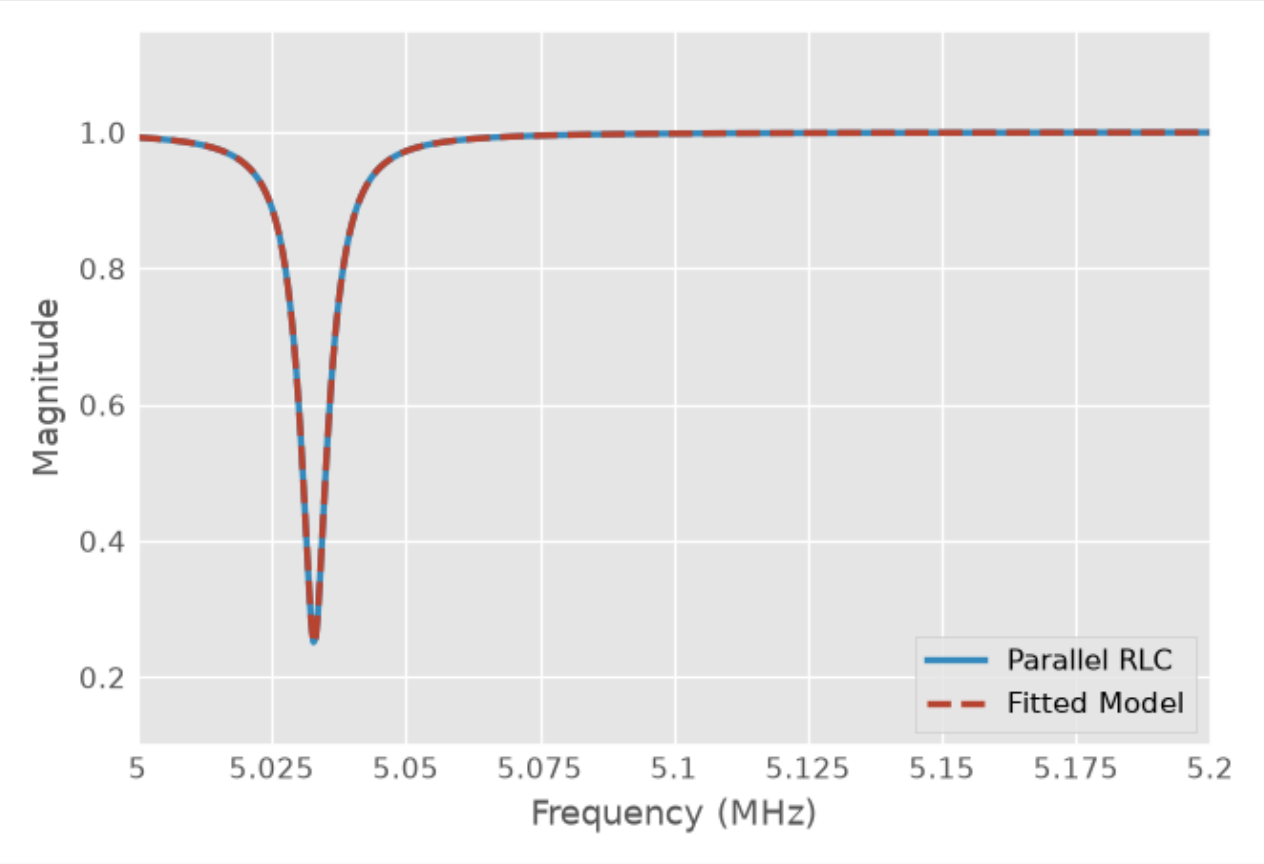
[10]:
```
print('Relative Error on Q_0:', (Q_RLC(R, L, C) - Q0)/Q_RLC(R, L, C))
```

```
Relative Error on Q_0: -3.3198526866053205e-07
```

It is also possible to generate a Network from the fitted results. For example, we can compare the fitted resonator model for more frequency points to the initial RLC resonator:

[11]:
```
new_freq = rf.Frequency(5, 5.2, npoints=5001, unit='MHz')
fitted_network = Q.fitted_network(res, frequency=new_freq)
```

[12]:
```
resonator.plot_s_mag(label='Parallel RLC ', lw=2)
fitted_network.plot_s_mag(label='Fitted Model', lw=2, ls='--')
```



1.11.4 Q-Circle

Close to a resonance, a resonator can be represented by an equivalent circuit model [1,4].

The Scattering-parameters (reflection or transmission) of RF circuits including resonator(s) have the form of circles in the complex plane $(\Re(s), \Im(s))$. S-parameters can be expressed as a function of the frequency f, the loaded quality factor Q_L [1,4]:

$$S(f) = S_D + d\frac{e^{-2j\delta}}{1 + jQ_L\mathcal{F}}$$

where :

- S_D is the detuned S-parameter measured at frequencies far above or below the resonance
- Q_L is the loaded Q-factor
- f_L is the loaded resonant frequency
- f_0 is the unloaded resonant frequency
- d is the diameter of the Q-circle
- δ is a real valued constant that defines the orientation of the Q-circle
- $\mathcal{F}$ is the fractional offset frequency given by:

$$\mathcal{F} = \frac{f}{f_L} - \frac{f_L}{f}$$

The fractional frequency $\mathcal{F}$ is a convenient way to express the frequency when dealing with resonant circuits: $\mathcal{F} = 0$ at the resonant frequency and when $f < f_L$, $\mathcal{F}$ is negative, while it is positive when $f > f_L$. If $\Delta f = f - f_L$ is the frequency deviation from the resonance, then close to the resonance:

$$\mathcal{F} \approx 2\frac{\Delta f}{f_L}$$

For example, if the source frequency is 10% below the resonant frequency ($0.9f/f_L$), then $\mathcal{F} \approx -0.2$.

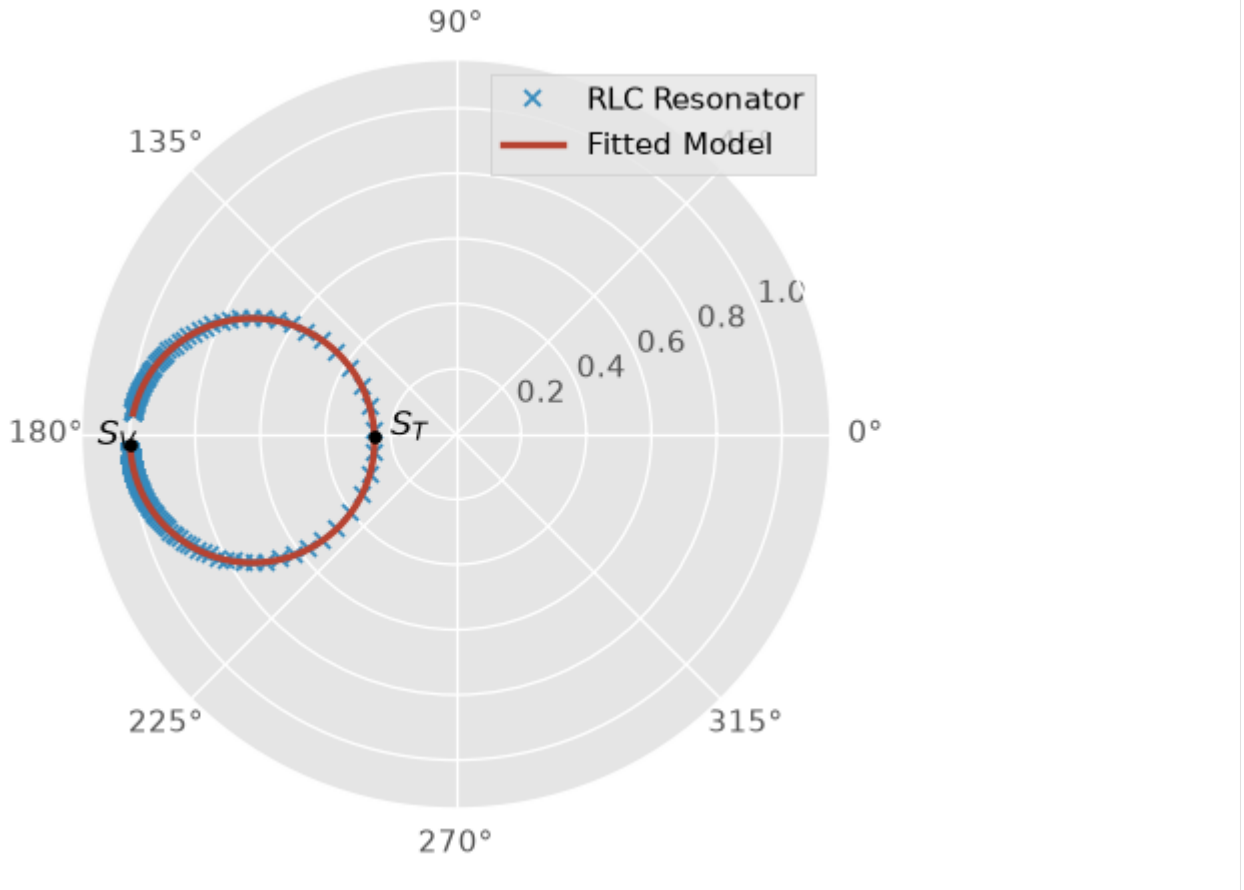
The parameters (diameters and tuned and off-tuned S-parameters) of the Q-circle can eventually be obtained from:

[13]:
```
diam, S_V, S_T = Q.Q_circle()
```

Use a polar plane to check the results:

[14]:
```
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
resonator.plot_s_polar(ax=ax, label="RLC Resonator", ls='', marker='x', ms=5)
fitted_network.plot_s_polar(ax=ax, label="Fitted Model", lw=2)
ax.plot(np.angle(S_V), np.abs(S_V), 'ko')
ax.plot(np.angle(S_T), np.abs(S_T), 'ko')
ax.text(np.angle(S_T), 0.8*np.abs(S_T), '$S_T$')
ax.text(np.angle(S_V), 1.1*np.abs(S_V), '$S_V$')
```

[14]:
```
Text(-3.1095630970814456, 1.1000000000000003, '$S_V$')
```



1.11.5 Bandwidth

Another definition of the Q-factor is the frequency-to-bandwidth ratio of a transmission resonator [6]:

$$Q_L = \frac{\omega_r}{\Delta\omega} = \frac{f_r}{\Delta f}$$

where f_r is the resonant frequency, $\Delta f = BW$ is the resonance width or *fractional bandwidth* BW. Note that this definition is only an approximation, which can be not accurate when there is significant leakage (see [4] for more information). The definition of the bandwidth also depends on the resonator type:

- transmission: bandwidth, also known as *full width at half maximum* (FWHM), is the bandwidth over which the dissipated power is half the maximum value at the resonant frequency (3 dB below the max value).
- reflection: while it is possible to define the bandwidth as the 3-dB points below the max reflection coefficient [6], a more proper definition of the bandwidth consists in measuring the difference between frequencies f_1 and f_2, which belong to the two points inclined by 45 degrees on each side of the centerline from S_T to the origin (see [4,8] for additional details).

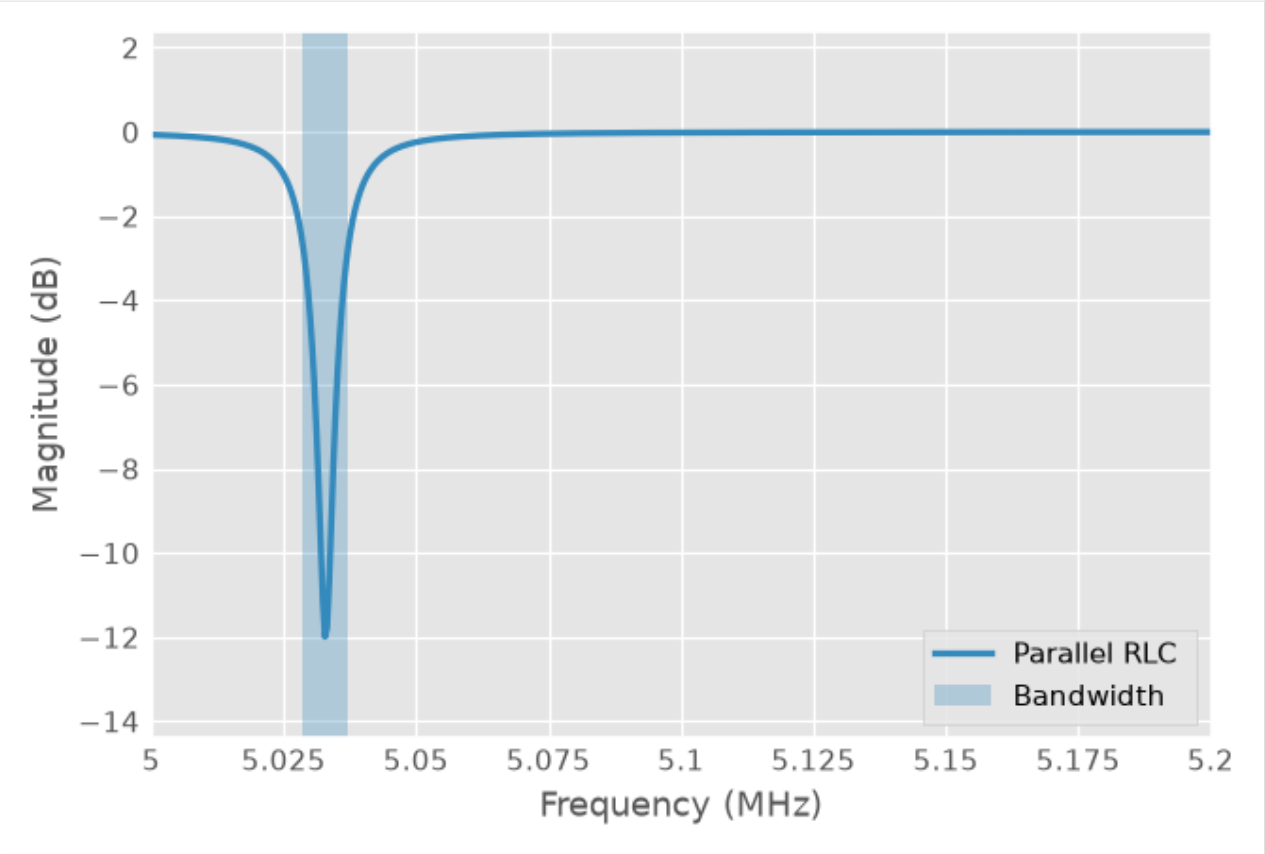
Using the previous relation, the bandwidth BW can also be obtained from the `Qfactor` class via the `.BW` parameter.

[15]:
```
BW = Q.BW
print(f'Bandwidth: {BW} Hz')
```

```
Bandwidth: 8488.258223066108 Hz
```

[16]:
```
fig, ax = plt.subplots()
resonator.plot_s_db(label='Parallel RLC ', lw=2, ax=ax)
ax.axvspan(xmin=Q.f_L-Q.BW/2, xmax=Q.f_L+Q.BW/2, alpha=0.3, label='Bandwidth')
ax.legend()
```

[16]:
```
<matplotlib.legend.Legend at 0x771170ffc5c0>
```



1.11.6 References

- [1] B. A. Galwas, "Scattering Matrix Description of Microwave Resonators," in IEEE Transactions on Microwave Theory and Techniques, vol. 31, no. 8, pp. 669-671, Aug. 1983, doi: 10.1109/TMTT.1983.1131566.
- [2] Peter A. Rizzi, "Microwave Engineering: Passive Circuits", Prentice-Hall, 1988
- [3] David M. Pozar, "Microwave Engineering", 4th Edition, Éditeur Wiley, 2011
- [4] A. P. Gregory, "Q-factor Measurement by using a Vector Network Analyser", National Physical Laboratory Report MAT 58 (2021), https://eprintspublications.npl.co.uk/9304/
- [5] Microwaves101, "Resonance of RLC Circuits", https://www.microwaves101.com/encyclopedias/resonance-of-rlc-circuits
- [6] Green, Estill I. (October 1955). "The Story of Q", American Scientist. 43: 584–594. http://www.collinsaudio.com/Prosound_Workshop/The_story_of_Q.pdf
- [7] R. S. Kwok and J.-F. Liang, 'Characterization of high-Q resonators for microwave filter applications', IEEE Transactions on Microwave Theory and Techniques, vol. 47, no. 1, pp. 111–114, Jan. 1999, doi: 10.1109/22.740093.
- [8] Darko Kajfez, "Q factor measurements, analog and digital", https://people.engineering.olemiss.edu/darko-kajfez/assets/rfqmeas2b.pdf

[ ]:

Download This Notebook: `Media.ipynb`

1.12 Media

1.12.1 Introduction

Scikit-rf supports S-Parameters simulations of transmission line models such as hollow waveguide, coaxial, microstripline and freespace among others.

The *Media* object stores the properties of a media, such as the *propagation constant* and the *characteristic impedance*. The network objects - like a line of a given length - are generated from the media.

This tutorial illustrates how to create networks using several kinds of media. It also explains the most common pitfalls regarding the network *port impedance* and *characteristic impedance*.

The singular of the Latin word *media* is *medium*, which means middle or center. For clarity's sake, *media* is used for both singular and plural in scikit-rf.

1.12.2 RG58C/U coaxial cable

Two arguments are common to all media constructors:

- `frequency` (required)
- `z0_port` (optional)

The frequency axes `frequency` is a `Frequency` object defining the media band.

The *port impedance* `z0_port` is optional and is used to renormalize from the media *characteristic impedance* `z0` to the *port impedance* of a simulated measurement.

If no *port impedance* `z0_port` is specified, the lines have the raw *characteristic impedance* `z0` of the media.

An example with an RG58C/U flexible coaxial cable media follows.

Simulating a VNA measurement

[1]:
```
# various initialization
%matplotlib inline
import matplotlib.pyplot as plt

import skrf as rf

# import the desired media and the frequency axis
from skrf import Frequency
from skrf.media import Coaxial

rf.stylely()


# frequency
f_rg58 = Frequency(1, 5, 101, 'GHz')

# media with z0_port the port impedance of the VNA
```

(continues on next page)

(continued from previous page)

```
rg58 = Coaxial(f_rg58, Dint = 0.91e-3, Dout = 2.95e-3, epsilon_r = 2.3, z0_port = 50)
print(rg58)
```

```
Coaxial Media.  Coaxial Media.  1.0-5.0GHz, 101 points.
Dint = 0.91 mm, Dout = 2.95 mm
z0 = (46.6, -0.1j)-(46.5, -0.0j) Ohm
z0_port = (50.0, 0.0j)-(50.0, 0.0j) Ohm
```

The Media has a *characteristic impedance* `z0` of approximately 47 Ohm. The *port impedance* `z0_port` is 50 Ohm. The propagation constant `gamma` is also computed from the Media parameters. These properties defines the transmission line model.

[2]:
```
print(f'z0 = {rg58.z0[0]}')
print(f'z0_port = {rg58.z0_port[0]}')
print(f'gamma = {rg58.gamma[0]}')
```

```
z0 = (46.557679343366004-0.05941866252040144j)
z0_port = 50.0
gamma = (0.040617114947926004+31.82566778491332j)
```

Let's create a line network corresponding to a 100 millimeter length of coax.

[3]:
```
rg58_line = rg58.line(100, unit = 'mm', name = '100 mm, z0 Ohm')
print(rg58_line)
```

```
2-Port Network: '100 mm, z0 Ohm',  1.0-5.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]
```

The *characteristic impedance* `z0`of the media has been renormalized to *port impedance* `z0_port`. The network has a port impedance `z0` equal to `z0_port`.

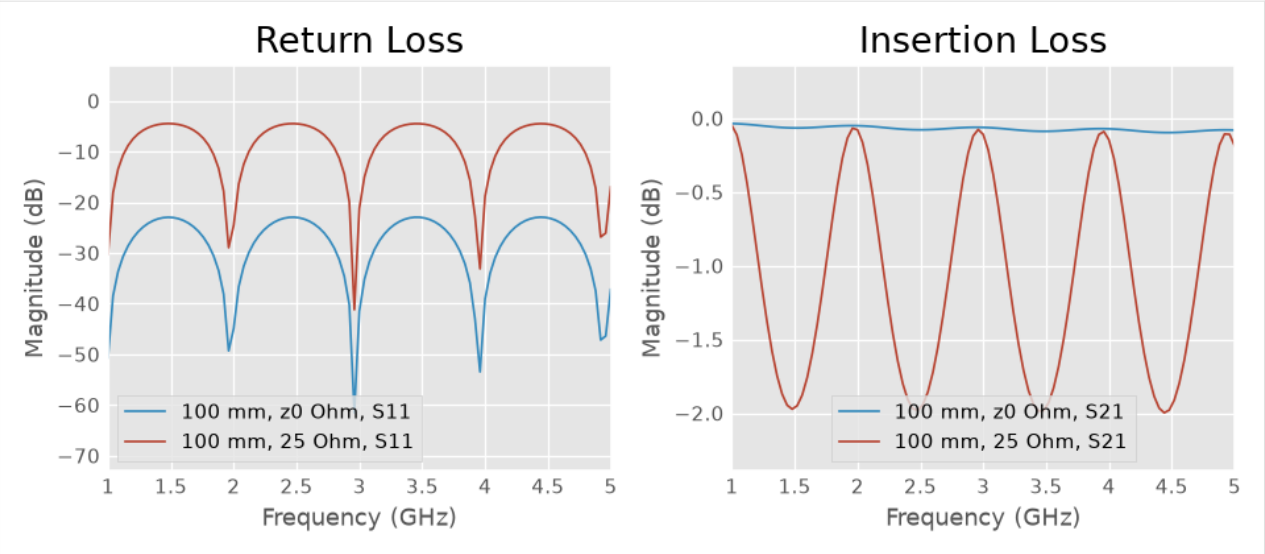
In some cases, lines of arbitrary impedance are required without creating multiples media. The impedance of the line can be overridden at construction. The resulting network is renormalized to `z0_port`.

[4]:
```
rg58_25ohm_line = rg58.line(100, unit = 'mm', z0 = 25, name = '100 mm, 25 Ohm')
print(rg58_25ohm_line)
```

```
2-Port Network: '100 mm, 25 Ohm',  1.0-5.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]
```

Let's plot the two lines S-Parameters.

[5]:
```
fig, axes = plt.subplots(1, 2, figsize = (8, 3.5))

# return loss
rg58_line.plot_s_db(0, 0, ax = axes[0])
rg58_25ohm_line.plot_s_db(0, 0, ax = axes[0])
axes[0].set_title('Return Loss')
rg58_line.plot_s_db(1, 0, ax = axes[1])
rg58_25ohm_line.plot_s_db(1, 0, ax = axes[1])
axes[1].set_title('Insertion Loss')
plt.tight_layout()
```



The return loss and insertion loss show the effect of the mismatch between the *port impedance* `z0_port` and the *characteristic impedance* `z0` of the line, that is either defined by the geometry or forced to an arbitrary value.

1.12.3 Basic examples

Let's create two networks, a length of 50-Ohm planar microstripline and a length of WR-10 rectangular waveguide.

Microstripline	Rectangular Waveguide

First of all, create two *Media* objects with the model parameters.

Media objects

Microstripline			WR-10 Rectangular Waveguide		
Track width	`w`	3 mm	Aperture width	`a`	100 mil
Track thickness	`t`	35 um	Aperture height	`b`	50 mil
Substrate height	`h`	1.6 mm			
Relative permittivity (FR-4)	`ep_r`	4.5	Relative permittivity (air)	`ep_r`	1.0
Resistivity (copper)	`rho`	1.68e-08 Ohm * m	Resistivity (copper)	`rho`	1.68e-08 Ohm * m

[6]:
```
# various initialization
%matplotlib inline
import matplotlib.pyplot as plt

import skrf as rf

# import the desired media and the frequency axis
```

(continues on next page)

(continued from previous page)

```
from skrf import Frequency
from skrf.constants import to_meters
from skrf.media import MLine, RectangularWaveguide

rf.stylely()

# create frequency axes
f_mlin = Frequency(0.1, 10,1001, 'GHz')
f_wr10 = Frequency(75, 110, 1001,'GHz')

# create media from parameters
mlin  =  MLine(f_mlin, w = 3e-3, h = 1.6e-3, t = 35e-6, ep_r = 4.5,  rho = 1.68e-08)
print(mlin)
wr10  = RectangularWaveguide(f_wr10, a = to_meters(100, 'mil'), b = to_meters(50, 'mil'),
↪ ep_r = 1.0,  rho = 1.68e-08)
print(wr10)
```

```
Microstripline Media.  0.1-10.0 GHz.  1001 points
 W= 3.00e-03m, H= 1.60e-03m
Rectangular Waveguide Media.  75.0-110.0 GHz.  1001 points
 a= 2.54e-03m, b= 1.27e-03m
```

Line creation

Secondly, use the *Media* objects to generate networks corresponding to a 100 millimeter length of both media.

[7]:
```
# create the transmission line networks
mlin_100 = mlin.line(100e-3, unit = 'm', name = 'mlin 100mm')
print(mlin_100)
wr10_100 = wr10.line(100e-3, unit = 'm', name = 'wr10 100mm')
print(wr10_100)
```

```
2-Port Network: 'mlin 100mm',  0.1-10.0 GHz, 1001 pts, z0=[49.66276711+0.j 49.66276711+0.
↪j]
2-Port Network: 'wr10 100mm',  75.0-110.0 GHz, 1001 pts, z0=[610.32096719+0.12861446j␣
↪610.32096719+0.12861446j]
```

Plotting the results

The S-Parameters of the lines are plotted in the figure below.

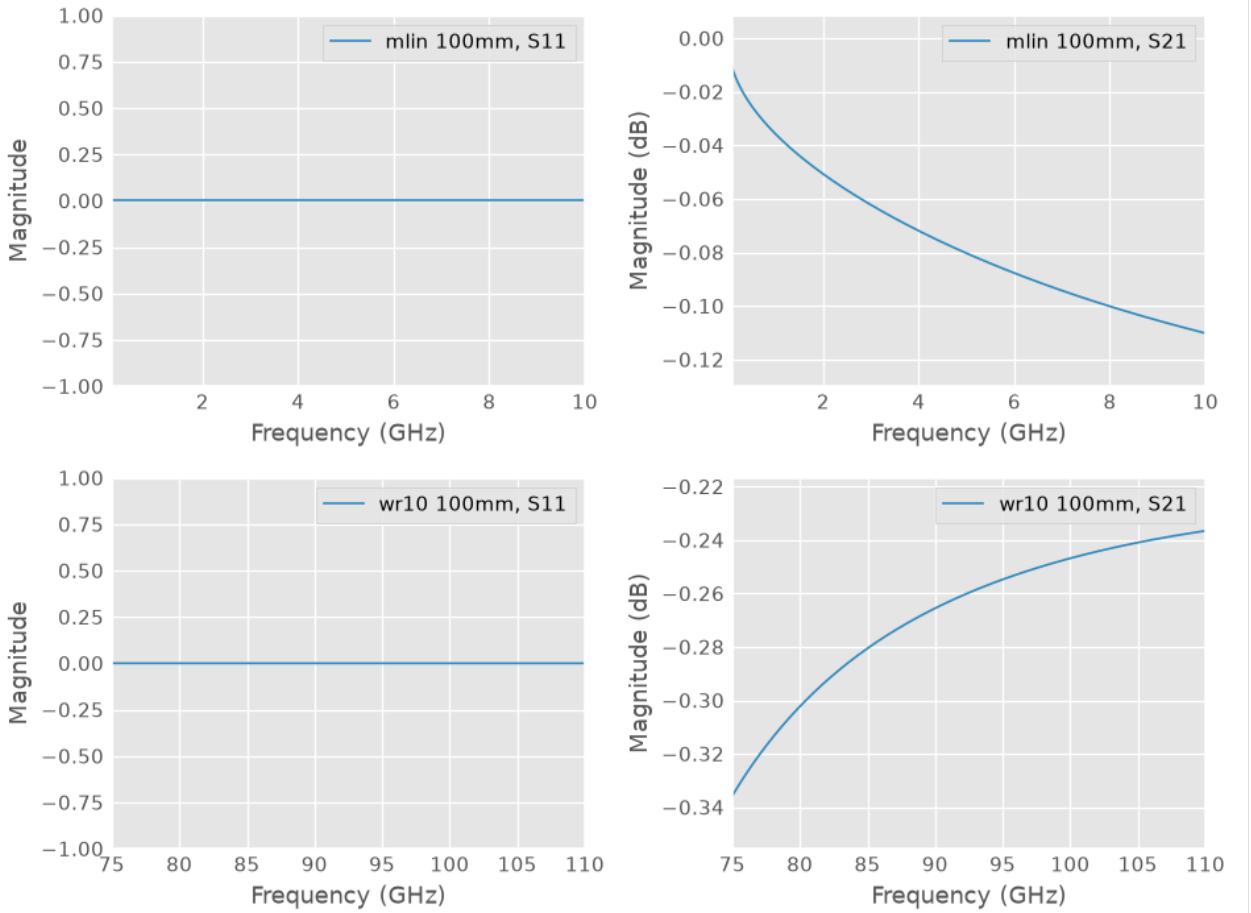
[8]:
```
# prepare figure
fig1, axes = plt.subplots(2, 2, figsize = (8, 6))

# plot miscrostipline
mlin_100.plot_s_mag(0, 0, ax = axes[0,0])
mlin_100.plot_s_db(1, 0, ax = axes[0,1])

# plot rectangular waveguide
wr10_100.plot_s_mag(0, 0, ax = axes[1,0])
wr10_100.plot_s_db(1, 0, ax = axes[1,1])

# resize plot nicely
```

(continues on next page)

(continued from previous page)

```
axes[0, 0].set_ylim((-1, 1))
axes[1, 0].set_ylim((-1, 1))
fig1.tight_layout()
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪mathFunctions.py:303: RuntimeWarning: divide by zero encountered in log10
  out = 20 * np.log10(z)
```



The insertion loss S21 of the transmission line is frequency dependent, but the S11 magnitude is constant and zero. The absolute magnitude of S11 was plotted instead of dB because log(0) is undefined.

Why does return loss S11 not show the shape observed on vector network analyzer measurements?

This is because no *port impedance* `z0_port` was specified at media construction. In this case, the *characteristic impedance* `z0` is used as *port impedance* and the network is perfectly matched with itself on the whole frequency band.

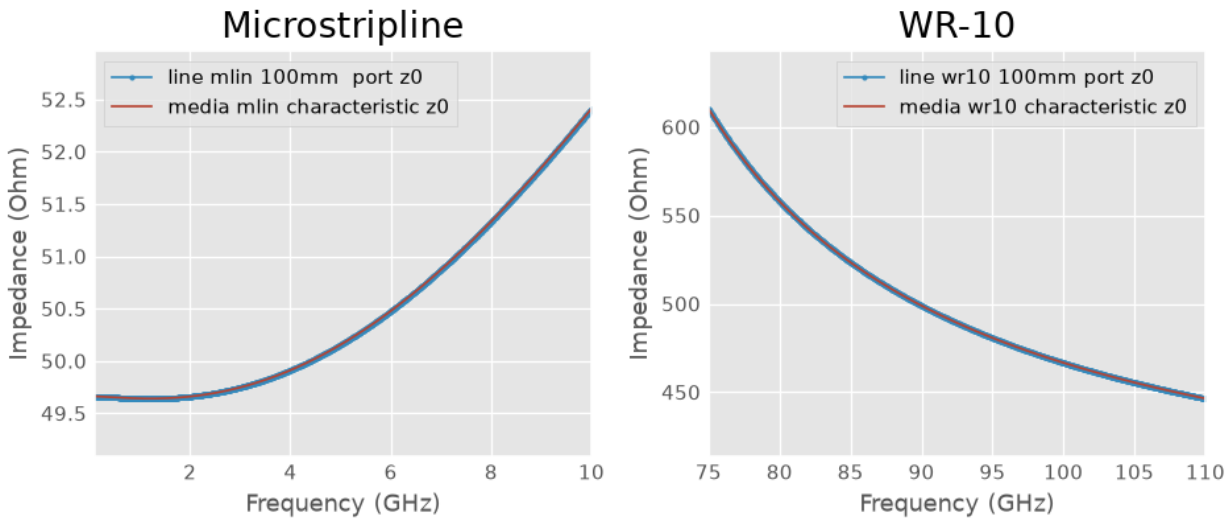
The *characteristic impedance* is usually a frequency-dependent parameter. Having a frequency-dependent *port impedance* is often encountered in electromagnetic simulations. Real-world measurements use a constant *port impedance* instead.

The *port impedance* `z0` of the line network and the *characteristic impedance* `z0` of the media are plotted below. These parameters are frequency-dependent. In both cases, the *port impedance* of the line equals the *characteristic impedance* of the media.

The *port impedance* of `Network` object is also `z0` but it is not a *characteristic impedance*. The `Network` S-Parameters

are normalized to its *port impedance*.

[9]:
```
# prepare figure
fig2, axes = plt.subplots(1, 2, figsize = (8, 3.5))

# plot miscrostipline
axes[0].plot(mlin_100.frequency.f_scaled, mlin_100.z0[:, 0].real, marker = '.',
             label = f'line {mlin_100.name}  port z0')
axes[0].plot(mlin.frequency.f_scaled, mlin.z0.real,
             label = 'media mlin characteristic z0')
axes[0].set_ylabel('Impedance (Ohm)')
axes[0].set_xlabel(f'Frequency ({mlin.frequency.unit})')
axes[0].set_title('Microstripline')
axes[0].legend()

# plot rectangular waveguide
axes[1].plot(wr10_100.frequency.f_scaled, wr10_100.z0[:, 0].real, marker = '.',
             label = f'line {wr10_100.name} port z0')
axes[1].plot(wr10.frequency.f_scaled, wr10.z0.real,
             label = 'media wr10 characteristic z0')
axes[1].set_ylabel('Impedance (Ohm)')
axes[1].set_xlabel(f'Frequency ({wr10.frequency.unit})')
axes[1].set_title('WR-10')
axes[1].legend()

# resize plot nicely
fig2.tight_layout()
```



Measured-like microstripline

S-Parameters measurement of a microstriplines is done with a VNA (Vector Network Analyzer). The VNA has a known *port impedance* - usually 50 Ohm - and coaxial connectivity.

A coaxial to microstripline transition is used and the VNA is calibrated at the end of the coaxial cable. In this example let's assume an ideal transition (no length, perfect match on both sides).

Thus, the transition is only a mismatch between VNA *port impedance* `z0_port` and transmission line *characteristic impedance* `z0`.

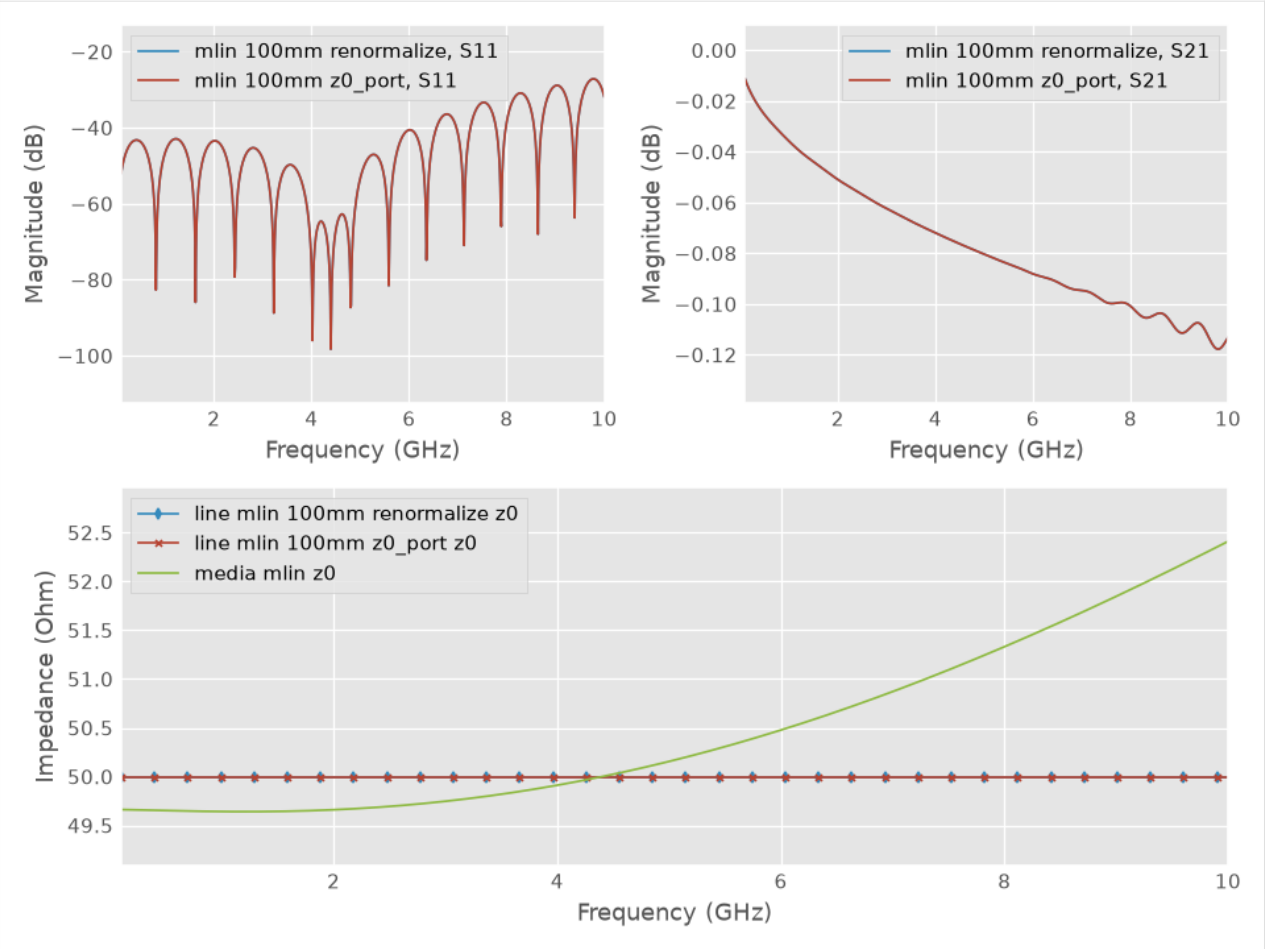
To get S-Parameter network with 50-Ohm *port impedance*, either the *port impedance* `z0_port` can be specified at media object construction or the line can be renormalized from *characteristic* to *port impedance*.

[10]:
```
# renormalization method
mlin_100_measured1 = mlin_100.copy()
mlin_100_measured1.renormalize([50, 50])
mlin_100_measured1.name = f'{mlin_100.name} renormalize'
print(mlin_100_measured1)

# port impedance specified at media construction method
mlin_meas  =  MLine(f_mlin, w=3e-3, h=1.6e-3, t=35e-6, ep_r=4.5,  rho=1.68e-08, z0_
↪port=50)
mlin_100_measured2 = mlin_meas.line(100e-3, unit = 'm', name = 'mlin 100mm z0_port')
print(mlin_100_measured2)
```

```
2-Port Network: 'mlin 100mm renormalize',  0.1-10.0 GHz, 1001 pts, z0=[50.+0.j 50.+0.j]
2-Port Network: 'mlin 100mm z0_port',  0.1-10.0 GHz, 1001 pts, z0=[50.+0.j 50.+0.j]
```

The plot below shows that the *characteristic impedance* of the microstripline is now embedded within a network with 50-Ohm *port impedance*. This is equivalent to a VNA measurement with an ideal coaxial to microstripline transition.

[11]:
```
# prepare figure
fig3, axes = plt.subplots(2, 2, figsize = (8, 6))
gs = axes[1, 0].get_gridspec()
for ax in axes[1, :]:
    ax.remove()
axbig = fig3.add_subplot(gs[1, :])

# plot return loss
mlin_100_measured1.plot_s_db(0, 0, ax=axes[0,0])
mlin_100_measured2.plot_s_db(0, 0, ax=axes[0,0])

# plot insertion loss
mlin_100_measured1.plot_s_db(1, 0, ax=axes[0,1])
mlin_100_measured2.plot_s_db(1, 0, ax=axes[0,1])

# plot port and characteristic impedances
axbig.plot(mlin_100_measured1.frequency.f_scaled, mlin_100_measured1.z0[:, 0].real,
           marker = 'd', markevery = 30, label = f'line {mlin_100_measured1.name} z0')
axbig.plot(mlin_100_measured2.frequency.f_scaled, mlin_100_measured2.z0[:, 0].real,
           marker = 'x', markevery = 30, label = f'line {mlin_100_measured2.name} z0')
axbig.plot(mlin.frequency.f_scaled, mlin.z0.real, label = 'media mlin z0')
axbig.set_ylabel('Impedance (Ohm)')
axbig.set_xlabel(f'Frequency ({mlin.frequency.unit})')
axbig.legend()

# resize plot nicely
fig3.tight_layout()
```



Measured-like WR-10 waveguide

S-Parameters measurement of a hollow waveguide is done with a VNA (Vector Network Analyzer). The VNA has a known port impedance - usually 50 Ohm - and coaxial connectivity.

A coaxial to waveguide transition is used. The transition is calibrated at the waveguide interface. Thus, VNA *port impedance* override the *characteristic impedance* of the waveguide.

The measurement will store the *port impedance* instead of the *characteristic impedance*, which is lost. This is not normalization. The actual *characteristic impedance* of the line is not measured. This method is specific to hollow waveguides.

To get an S-Parameters network with 50-Ohm port impedance, either the port impedance `z0_override` can be specified at media object construction or the impedance can be overridden manually.

[12]:
```
# override method
wr10_100_measured1 = wr10_100.copy()
wr10_100_measured1.z0[:,:] = 50
wr10_100_measured1.name = f'{wr10_100.name} override'
print(wr10_100_measured1)

# port impedance at media construction method
wr10_meas  =  RectangularWaveguide(f_wr10, a=to_meters(100, 'mil'), b=to_meters(50, 'mil
```

(continues on next page)

(continued from previous page)

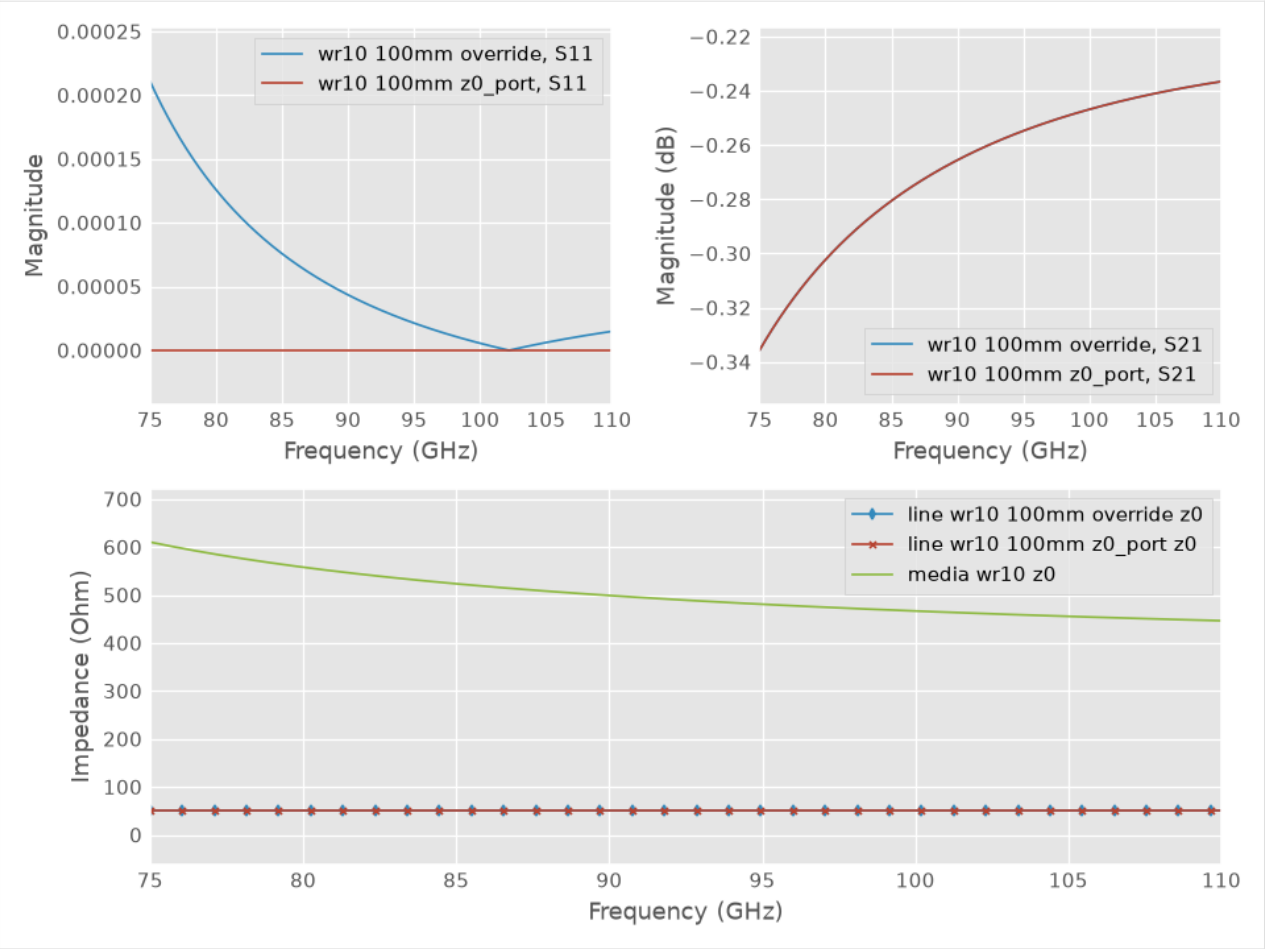
```
↪'), ep_r=1.0,  rho=1.68e-08,
                                    z0_override = 50)
wr10_100_measured2 = wr10_meas.line(100e-3, unit = 'm', name = 'wr10 100mm z0_port')
print(wr10_100_measured2)
```

```
2-Port Network: 'wr10 100mm override',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j 50.+0.j]
2-Port Network: 'wr10 100mm z0_port',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j 50.+0.j]
```

The plot below shows that the port impedance of the rectangular waveguide measurement is now 50 Ohm. The insertion loss S21 is equal for both methods. However, there is a slight S11 difference between manual override and port impedance specification at construction.

This is because the default s-parameter definition for network is `s_def = 'power'` and that the characteristic impedance has an imaginary part. For this reason, perfect match is not zero but complex conjugate. The manual override method does not gives the expected results.

[13]:
```
# prepare figure
fig4, axes = plt.subplots(2, 2, figsize = (8, 6))
gs = axes[1, 0].get_gridspec()
for ax in axes[1, :]:
    ax.remove()
axbig = fig4.add_subplot(gs[1, :])

# plot return loss
wr10_100_measured1.plot_s_mag(0, 0, ax=axes[0,0])
wr10_100_measured2.plot_s_mag(0, 0, ax=axes[0,0])

# plot insertion loss
wr10_100_measured1.plot_s_db(1, 0, ax=axes[0,1])
wr10_100_measured2.plot_s_db(1, 0, ax=axes[0,1])

# plot port and characteristic impedances
axbig.plot(wr10_100_measured1.frequency.f_scaled, wr10_100_measured1.z0[:, 0].real,
           marker = 'd', markevery = 30, label = f'line {wr10_100_measured1.name} z0')
axbig.plot(wr10_100_measured2.frequency.f_scaled, wr10_100_measured2.z0[:, 0].real,
           marker = 'x', markevery = 30, label = f'line {wr10_100_measured2.name} z0')
axbig.plot(wr10.frequency.f_scaled, wr10.z0.real, label = 'media wr10 z0')
axbig.set_ylabel('Impedance (Ohm)')
axbig.set_xlabel(f'Frequency ({mlin.frequency.unit})')
axbig.legend()

# resize plot nicely
fig4.tight_layout()
```



Parameter variation

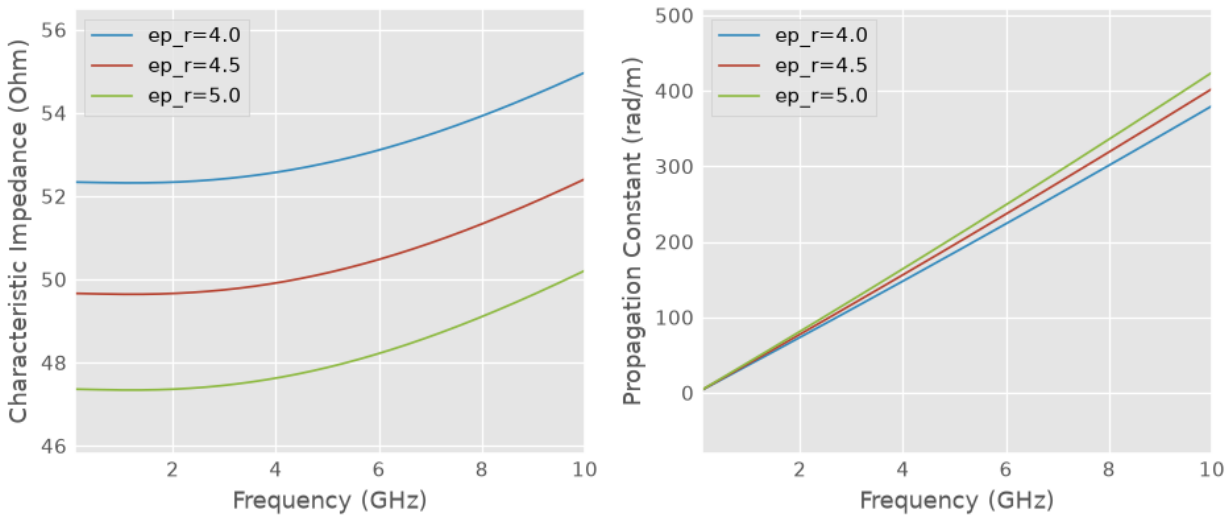
The construction parameters of the media can be varied. If, for example, we want to know how much the *characteristic impedance* and *propagation constant* of a microstripline would be affected by a change in relative permittivity of the substrate:

[14]:
```
#prepare
fig5, axes = plt.subplots(1, 2, figsize = (8, 3.5))

# plot
for ep_r in [4.0, 4.5, 5.0]:
    ml = MLine(f_mlin, w=3e-3, h=1.6e-3, t=35e-6, ep_r=ep_r,  rho=1.68e-08, z0_port=50)
    axes[0].plot(f_mlin.f_scaled, ml.z0.real, label=f'ep_r={ep_r:.1f}')
    axes[1].plot(f_mlin.f_scaled, ml.beta, label=f'ep_r={ep_r:.1f}')

axes[0].set_xlabel(f'Frequency ({f_mlin.unit})')
axes[0].set_ylabel('Characteristic Impedance (Ohm)')
axes[0].legend()
axes[1].set_xlabel(f'Frequency ({f_mlin.unit})')
axes[1].set_ylabel('Propagation Constant (rad/m)')

axes[1].legend()
```

(continues on next page)

(continued from previous page)

```
# resize plot nicely
fig5.tight_layout()
```



More detailed examples illustrating how to create various kinds of *Media* objects follow. The list of supported media is in the *Media* API page. The network creation and connection syntax of **skrf** are cumbersome if you need to doing complex circuit design. **skrf**'s synthesis capabilities lend themselves more to scripted applications such as calibration, optimization or batch processing.

1.12.4 Slab of Si in Freespace

A plane-wave in freespace from 10 to 20 GHz.

[15]:
```
from skrf.media import Freespace

freq = Frequency(10, 20, 101, 'GHz')
air =  Freespace(freq)
air
```

[15]:
```
Freespace  Media.  10-20 GHz.  101 points
```

[16]:
```
air.z0[:2] # 377 Ohm baby!
```

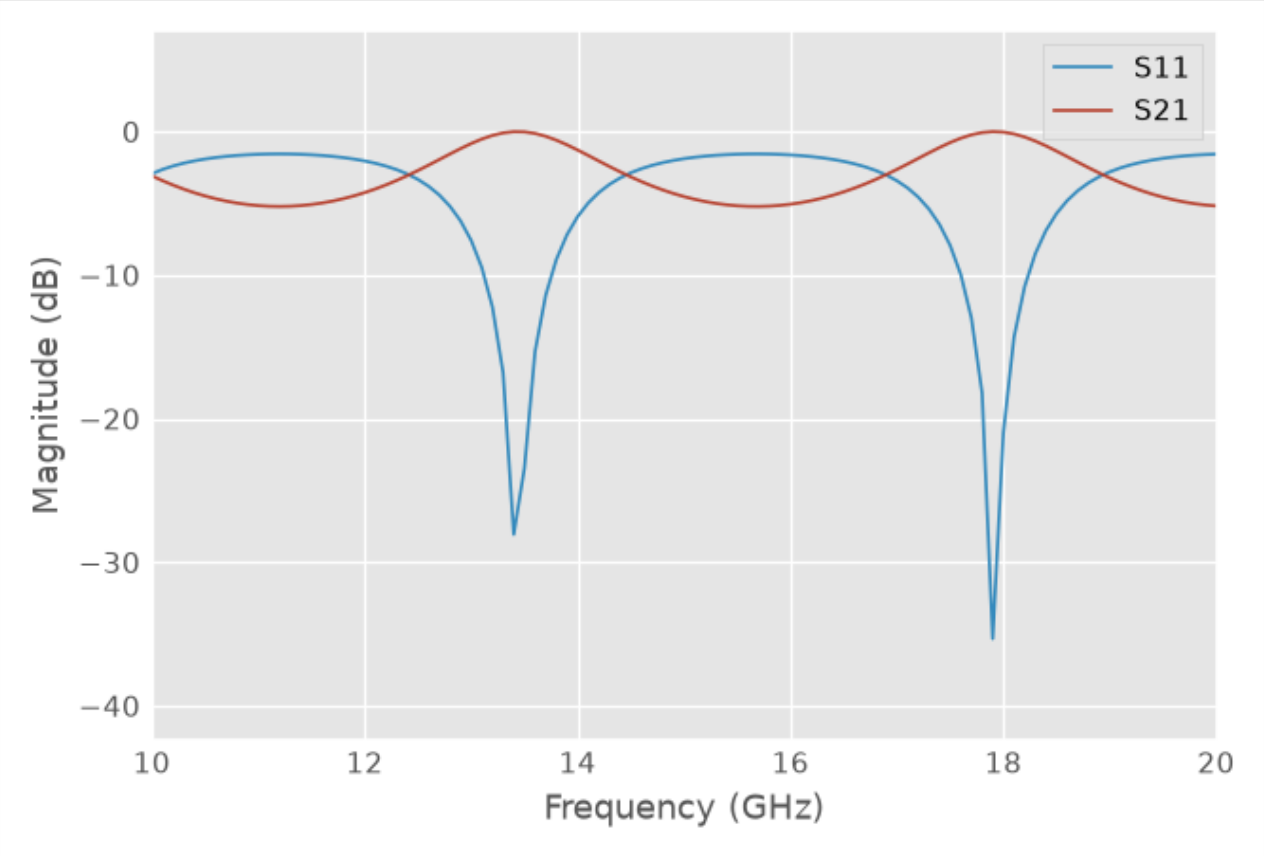
[16]:
```
array([376.73031341+0.j, 376.73031341+0.j])
```

[17]:
```
# plane wave in Si
si = Freespace(freq, ep_r = 11.2)
si.z0[:3] # ~110 Ohm
```

[17]:
```
array([112.56971215+0.j, 112.56971215+0.j, 112.56971215+0.j])
```

Simulate a 1cm slab of Si in half-space,

[18]:
```
slab = air.thru() ** si.line(1, unit = 'cm') ** air.thru()
slab.plot_s_db(n = 0)
```



1.12.5 Network Synthesis

Networks are created through methods of a Media object. To create a 1-port short for a microstripline,

```
[19]: mlin_meas.short(name = 'short')
```

```
[19]: 1-Port Network: 'short',  0.1-10.0 GHz, 1001 pts, z0=[50.+0.j]
```

Or to create a 90° section of microstripline,

```
[20]: mlin_meas.line(d = 90, unit = 'deg', name = 'line')
```

```
[20]: 2-Port Network: 'line',  0.1-10.0 GHz, 1001 pts, z0=[50.+0.j 50.+0.j]
```

Note

Simple circuits like `Media.short` and `Media.open` are ideal short and open with $\Gamma = -1$ and $\Gamma = 1$. They dont account the sophisticated effects of the discontinuities. Eventually, these more complex networks could be implemented with methods specific to a given Media, e.g. `MLine.mlin_short`, should the need arise.

1.12.6 Building Circuits

Complex circuits can be made by connecting a series of networks. Let's build a 90° microstripline delay short.

[21]:
```
delay_short = mlin_meas.line(d = 90, unit = 'deg') ** mlin_meas.short()
delay_short.name = 'delay short'
delay_short
```

[21]:
```
1-Port Network: 'delay short',  0.1-10.0 GHz, 1001 pts, z0=[50.+0.j]
```

When `Networks` with more than 2 ports need to be connected together, use `rf.connect()`. To create a two-port network for a shunted delayed open, you can create an ideal 3-way splitter (a 'tee') and connect the delayed open to one of its ports,

[22]:
```
tee = mlin_meas.tee()
delay_open = mlin_meas.delay_open(40, unit = 'deg')
shunt_open = rf.network.connect(tee, 1, delay_open, 0)
```

Adding networks in shunt is so common, there is a `Media.shunt()` function to do this,

[23]:
```
mlin_meas.shunt(delay_open)
```

[23]:
```
2-Port Network: '',  0.1-10.0 GHz, 1001 pts, z0=[50.+0.j 50.+0.j]
```

If a specific circuit is created frequently, it may make sense to use a function to create the circuit. This can be done most quickly using `lambda`

[24]:
```
delay_short = lambda d: mlin_meas.line(d, unit = 'deg') ** mlin_meas.short()  # noqa:␣
↪E731
delay_short(90)
```

[24]:
```
1-Port Network: '',  0.1-10.0 GHz, 1001 pts, z0=[50.+0.j]
```

A more useful example may be to create a function for a shunt-stub tuner, that will work for any media object.

[25]:
```
def shunt_stub(med, d0, d1):
    return med.line(d0, unit = 'deg')**med.shunt_delay_open(d1, unit = 'deg')

shunt_stub(mlin_meas, 10, 90)
```

[25]:
```
2-Port Network: '',  0.1-10.0 GHz, 1001 pts, z0=[50.+0.j 50.+0.j]
```

This approach lends itself to design optimization.

> **Note**
>
> To connect complex circuits, you might be interested to use the `skrf.circuit` module.

1.12.7 Design Optimization

The abilities of `scipy`'s optimizers can be used to automate network design. In this example, scikit-rf is used to automate the single stub impedance matching network design. First, we create a 'cost' function that returns something we want to minimize, such as the reflection coefficient magnitude at the band center. Then, one of scipy's minimization algorithms is used to determine the optimal parameters of the stub lengths to minimize this cost.

[26]:
```
from scipy.optimize import fmin

from skrf.media import CPW
```

(continues on next page)

(continued from previous page)

```
freq = Frequency(75, 110, 101, 'GHz')
cpw = CPW(freq, w = 10e-6, s = 5e-6, ep_r = 10.6, z0_port = 50)

# the load we are trying to match
load = cpw.load(.2+.2j)

# single stub circuit generator function
def shunt_stub(med, d0, d1):
    return med.line(d0, unit = 'deg') ** med.shunt_delay_open(d1, unit = 'deg')


# define the cost function we want to minimize (this uses sloppy namespace)
def cost(d):
    # prevent negative length lines, returning high cost
    if d[0] < 0 or d[1] < 0:
        return 1e3
    return (shunt_stub(cpw, d[0], d[1]) ** load)[100].s_mag.squeeze()

# initial guess of optimal delay lengths in degrees
d0 = 120,40 # initial guess

#determine the optimal delays
d_opt = fmin(cost, (120, 40))

d_opt
```

```
Optimization terminated successfully.
         Current function value: 0.232798
         Iterations: 63
         Function evaluations: 122
```

[26]:
```
array([227.04547462,  12.72478816])
```

[ ]:
```
```

1.13 Vector Fitting

This is a brief introduction to Vector Fitting. The concept and its algorithm was proposed in 1999 by Bjørn Gustavsen and Adam Semlyen [1]. See the Vector Fitting website for more information [2]. The main application of Vector Fitting is to model the original sampled frequency responses of an active or passive device in a circuit simulator. Also see the API documentation and the application examples for more information about the implementation in scikit-rf.

1.13.1 Mathematical Description

The idea of Vector Fitting is to fit a set of rational model functions to a set of sampled frequency responses $\underline{\underline{\mathbf{H}}}_{\text{sampled}}$, for example from a S, Y or Z matrix. The model function $\underline{\underline{\mathbf{H}}}(\underline{\mathrm{s}})$ is defined in the Laplace domain with $\underline{\mathrm{s}} = \sigma + \mathrm{j}\omega$:

$$\underline{\underline{\mathbf{H}}}(\underline{\mathrm{s}}) = \mathbf{d} + \underline{\mathrm{s}}\mathbf{e} + \sum_{k=1}^{K} \frac{\underline{\underline{\mathbf{c}}}_k}{\underline{\mathrm{s}} - \underline{p}_k} \tag{1.1}$$

For the desired fit, that model function shall match the given frequency responses at each of its frequency samples ω_n:

$$\underline{\underline{\mathbf{H}}}(\underline{\mathrm{s}} = \mathrm{j}\omega_n) \stackrel{!}{=} \underline{\underline{\mathbf{H}}}_{\text{sampled}}(\omega_n) \tag{1.2}$$

Generally, $\underline{\mathbf{H}}(\underline{s})$ is a vector holding the individual complex frequency responses $\underline{H}_1(\underline{s})$, $\underline{H}_2(\underline{s})$, ..., $\underline{H}_M(\underline{s})$ of the model. All elements in $\underline{\mathbf{H}}(\underline{s})$ share a common set of complex poles $\underline{p}_k$, but have individual sets of complex residues $\underline{\mathbf{c}}_k$, real constants $\mathbf{d}$ and real proportional coefficients $\mathbf{e}$, which are therefore vectors as well:

$$\underline{\mathbf{p}} = \begin{pmatrix} \underline{p}_1 & \underline{p}_2 & \underline{p}_3 & \cdots & \underline{p}_K \end{pmatrix} \tag{1.3}$$

$$\underline{\mathbf{c}} = \begin{pmatrix} \underline{c}_{1,1} & \underline{c}_{2,1} & \underline{c}_{3,1} & \cdots & \underline{c}_{K,1} \\ \underline{c}_{1,2} & \underline{c}_{2,2} & \underline{c}_{3,2} & \cdots & \underline{c}_{K,2} \\ \vdots & & & & \\ \underline{c}_{1,M} & \underline{c}_{2,M} & \underline{c}_{3,M} & \cdots & \underline{c}_{K,M} \end{pmatrix} \tag{1.4}$$

$$\mathbf{d} = \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_M \end{pmatrix} \tag{1.5}$$

$$\mathbf{e} = \begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_M \end{pmatrix} \tag{1.6}$$

The number of poles, K, required for a good fit of $\underline{\mathbf{H}}(\underline{s})$ to $\underline{\mathbf{H}}_{\text{sampled}}$ depends on the shape of the responses.

As an example, the goal could be to fit the rational model function to the S matrix of a 2-port ($M = 4$) sampled at N frequencies ω_n:

$$\begin{pmatrix} \underline{S}_{11}(\omega_1) \\ \underline{S}_{12}(\omega_1) \\ \underline{S}_{21}(\omega_1) \\ \underline{S}_{22}(\omega_1) \\ \vdots \\ \underline{S}_{11}(\omega_\mathrm{N}) \\ \underline{S}_{12}(\omega_\mathrm{N}) \\ \underline{S}_{21}(\omega_\mathrm{N}) \\ \underline{S}_{22}(\omega_\mathrm{N}) \end{pmatrix} \overset{!}{=} \begin{pmatrix} d_{11} + \mathrm{j}\omega_1 e_{11} + \sum_{k=1}^{K} \frac{\underline{c}_{k,11}}{\mathrm{j}\omega_1 - \underline{p}_k} \\ d_{12} + \mathrm{j}\omega_1 e_{12} + \sum_{k=1}^{K} \frac{\underline{c}_{k,12}}{\mathrm{j}\omega_1 - \underline{p}_k} \\ d_{21} + \mathrm{j}\omega_1 e_{21} + \sum_{k=1}^{K} \frac{\underline{c}_{k,21}}{\mathrm{j}\omega_1 - \underline{p}_k} \\ d_{22} + \mathrm{j}\omega_1 e_{22} + \sum_{k=1}^{K} \frac{\underline{c}_{k,22}}{\mathrm{j}\omega_1 - \underline{p}_k} \\ \vdots \\ d_{11} + \mathrm{j}\omega_\mathrm{N} e_{11} + \sum_{k=1}^{K} \frac{\underline{c}_{k,11}}{\mathrm{j}\omega_\mathrm{N} - \underline{p}_k} \\ d_{12} + \mathrm{j}\omega_\mathrm{N} e_{12} + \sum_{k=1}^{K} \frac{\underline{c}_{k,12}}{\mathrm{j}\omega_\mathrm{N} - \underline{p}_k} \\ d_{21} + \mathrm{j}\omega_\mathrm{N} e_{21} + \sum_{k=1}^{K} \frac{\underline{c}_{k,21}}{\mathrm{j}\omega_\mathrm{N} - \underline{p}_k} \\ d_{22} + \mathrm{j}\omega_\mathrm{N} e_{22} + \sum_{k=1}^{K} \frac{\underline{c}_{k,22}}{\mathrm{j}\omega_\mathrm{N} - \underline{p}_k} \end{pmatrix} \tag{1.7}$$

During the Vector Fitting process, the model parameters $\underline{\mathbf{p}}$, $\underline{\mathbf{c}}$, $\mathbf{d}$ and $\mathbf{e}$ will be optimized in an iterative manner until a good fit is achieved.

1.13.2 Equivalent Electrical Circuits

The benefit of Vector Fitting the sampled frequency responses is the easy representation of the rational basis function with equivalent electrical circuits. A detailed derivation can be found in [3] and [4].

Constant and Proportional Term

The constant and proportional terms $\mathbf{d} + \underline{s}\mathbf{e}$ can be represented in an electrical circuit by equivalent impedances $\underline{Z}_{\mathrm{RL}}(\underline{s})$ or equivalent admittances $\underline{Y}_{\mathrm{RC}}(\underline{s})$ build out of a series RL or parallel RC circuits.

Target response of the constant and proportional term:

$$\underline{H}_{\mathrm{target}}(\underline{s}) = d_i + \underline{s} e_i \tag{1.8}$$

Impedance of a series RL circuit:

$$\underline{Z}_{\mathrm{RL}}(\underline{s}) = R + \underline{s} L \tag{1.9}$$

This impedance matches the target response if $R = d_i$ and $L = e_i$.

Admittance of a parallel RC circuit:

$$\underline{Y}_{\mathrm{RC}}(\underline{s}) = \frac{1}{R} + \underline{s} C \tag{1.10}$$

This admittance matches the target response if $R = \frac{1}{d_i}$ and $C = e_i$.

Real Poles and Residues

The individual terms $\frac{c_{k,i}}{\underline{s} - p_{k,i}}$ of the fit with a real pole $p_{k,i}$ and a real residue $c_{k,i}$ can be represented in an electrical circuit by equivalent impedances $\underline{Z}_{\mathrm{RC}}(\underline{s})$ or equivalent admittances $\underline{Y}_{\mathrm{RL}}(\underline{s})$ build out of a parallel RC or series RL circuits.

Target response of the real pole-residue term:

$$\underline{H}_{\mathrm{target}}(\underline{s}) = \frac{c_{k,i}}{\underline{s} - p_{k,i}} \tag{1.11}$$

Impedance of a parallel RC circuit:

The parallel RC circuit is the same as above for the constant and proportional term, but this time its impedance $\underline{Z}_{\mathrm{RC}}(\underline{s})$ is used instead of its admittance:

$$\underline{Z}_{\mathrm{RC}}(\underline{s}) = \frac{\frac{1}{C}}{\underline{s} + \frac{1}{RC}} \tag{1.12}$$

This impedance matches the target response if $C = \frac{1}{c_{k,i}}$ and $R = \frac{c_{k,i}}{-p_{k,i}}$.

Admittance of a series RL circuit:

The series RL circuit is the same as above for the constant and proportional term, but this time its admittance $\underline{Y}_{\mathrm{RL}}(\underline{s})$ is used instead of its impedance:

$$\underline{Y}_{\mathrm{RL}}(\underline{s}) = \frac{\frac{1}{L}}{\underline{s} + \frac{R}{L}} \tag{1.13}$$

This admittance matches the target response if $L = \frac{1}{c_{k,i}}$ and $R = \frac{-p_{k,i}}{c_{k,i}}$.

Complex Conjugate Poles and Residues

In Vector Fitting, complex poles $\underline{p}_k = p'_k + \mathrm{j}p''_k$ and residues $\underline{c}_k = c'_k + \mathrm{j}c''_k$ always come in complex conjugate pairs $(\underline{p}_k, \underline{p}_k^*)$ and $(\underline{c}_k, \underline{c}_k^*)$. The target response for an equivalent electrical circuit of such a complex conjugate pair is therefore:

$$\underline{H}_{\mathrm{target}}(\underline{\mathrm{s}}) = \frac{\underline{c}_k}{\underline{\mathrm{s}} - \underline{p}_k} + \frac{\underline{c}_k^*}{\underline{\mathrm{s}} - \underline{p}_k^*} = \frac{2c'_k\underline{\mathrm{s}} - 2(c'_k p'_k + c''_k p''_k)}{\underline{\mathrm{s}}^2 - 2p'_k\underline{\mathrm{s}} + |\underline{p}_k|^2} \tag{1.14}$$

There are different ways to build an equivalent circuit out of passive or active components, which can be dimensioned to match this target frequency response. Four such circuits are presented and analyzed in [3], two of them are presented here.

Impedance of a parallel RLC circuit

A parallel RLC circuit with a second resistor in series with the inductor provides an impedance $\underline{Z}_{\mathrm{RCL}}(\underline{\mathrm{s}})$ that can be matched to the target response.

$$\underline{Z}_{\mathrm{RCL}}(\underline{\mathrm{s}}) = \frac{\frac{1}{C}\underline{\mathrm{s}} + \frac{R_1}{LC}}{\underline{\mathrm{s}}^2 + (\frac{1}{R_2 C})\underline{\mathrm{s}} + \frac{1}{LC}(1 + \frac{R_1}{R_2})} \tag{1.15}$$

This impedance matches the target response if $C = \frac{1}{2c'_k}$, $L = \frac{2c'_k}{(p''_k)^2 + (\frac{c''_k p''_k}{c'_k})^2}$, $R_1 = \frac{-2(c'_k p'_k + c''_k p''_k)}{(p''_k)^2 + (\frac{c''_k p''_k}{c'_k})^2}$, and $R_2 = \frac{(2c'_k)^2}{-2(c'_k p'_k - c''_k p''_k)}$.

In some cases, the calculated resistances can be negative, meaning that the passive circuit is required to provide gain, which is not normally possible. This is not uncommon if the modelled network is an active network, but it can also happen for purely passive networks. Some circuit simulators allow negative resistances by implementing them with the help of some kind of controlled source. To have full control over the behavior, the synthesized equivalent circuits already include voltage-controlled current sources in parallel with the resistors to realize negative effective resistances.

The effective resistance $R_{i,eff}$ of a resistor R_i in parallel with a current source controlled by the voltage V_i over the resistor equals $-R_i$ if the transconductance of the controlled current source is $g_{T,i} = \frac{-2}{R_i}$.

This way, both resistances R_1 and R_2 can be realized with both positive and negative values. In case of positive resistances, we simply put $g_{T,i} = 0$.

Admittance of a series RLC circuit

A series RLC circuit combined with a parallel current source controlled by the voltage over the capacitor provides an admittance $\underline{Y}_{\mathrm{RCL,I}}(\underline{\mathrm{s}})$ that can be matched to the target response.

$$\underline{Y}_{\mathrm{RCL,I}}(\underline{\mathrm{s}}) = \frac{1/L\underline{\mathrm{s}} + b}{\underline{\mathrm{s}}^2 + R/L\underline{\mathrm{s}} + 1/(LC)} \tag{1.16}$$

This admittance matches the target response if $L = \frac{1}{2c'_k}$, $R = \frac{-p'_k}{c'_k}$, $C = \frac{2c'_k}{|\underline{p}_k|^2}$ and $b = -2(c'_k p'_k + c''_k p''_k)$ with $g_{\mathrm{m}} = bLC = \frac{b}{|\underline{p}_k|^2}$.

1.13.3 Equivalent Circuit of a Vector Fitted N-Port

Case 1: Vector Fitted S Parameters

The equivalent circuit of a N-port with a Vector Fitted scattering matrix consists of an interface network and a transfer network for each port.

For the realization of the interface network, sometimes also called port network, one can either use a current source ($I_{\mathrm{src,i}}$) with a parallel resistor R_i (Norton equivalent), or a voltage source ($V_{\mathrm{src,i}}$) with a series resistor R_i (Thévenin equivalent). Solving Kirchhoff's current law for the Norton equivalent interface network, one gets $V_i = R_i(I_i + I_{src,i})$. Solving Kirchhoff's voltage law for the Thévenin equivalent interface network, one gets $V_i = R_i I_i + V_{src,i}$.

Comparing both equations with the definitions of the incident (a_i) and reflected (b_i) power waves at port i, one can see that the source terms $I_{\mathrm{src,i}}$ and $V_{\mathrm{src,i}}$ are proportional to the reflected power wave b_i, where R_i is the reference impedance (resistance) of port i:

$$b_i = \frac{1}{2\sqrt{R_i}}(V_i - R_i I_i) = \frac{\sqrt{R_i}}{2} I_{src,i} = \frac{1}{2\sqrt{R_i}} V_{src,i} \tag{1.17}$$

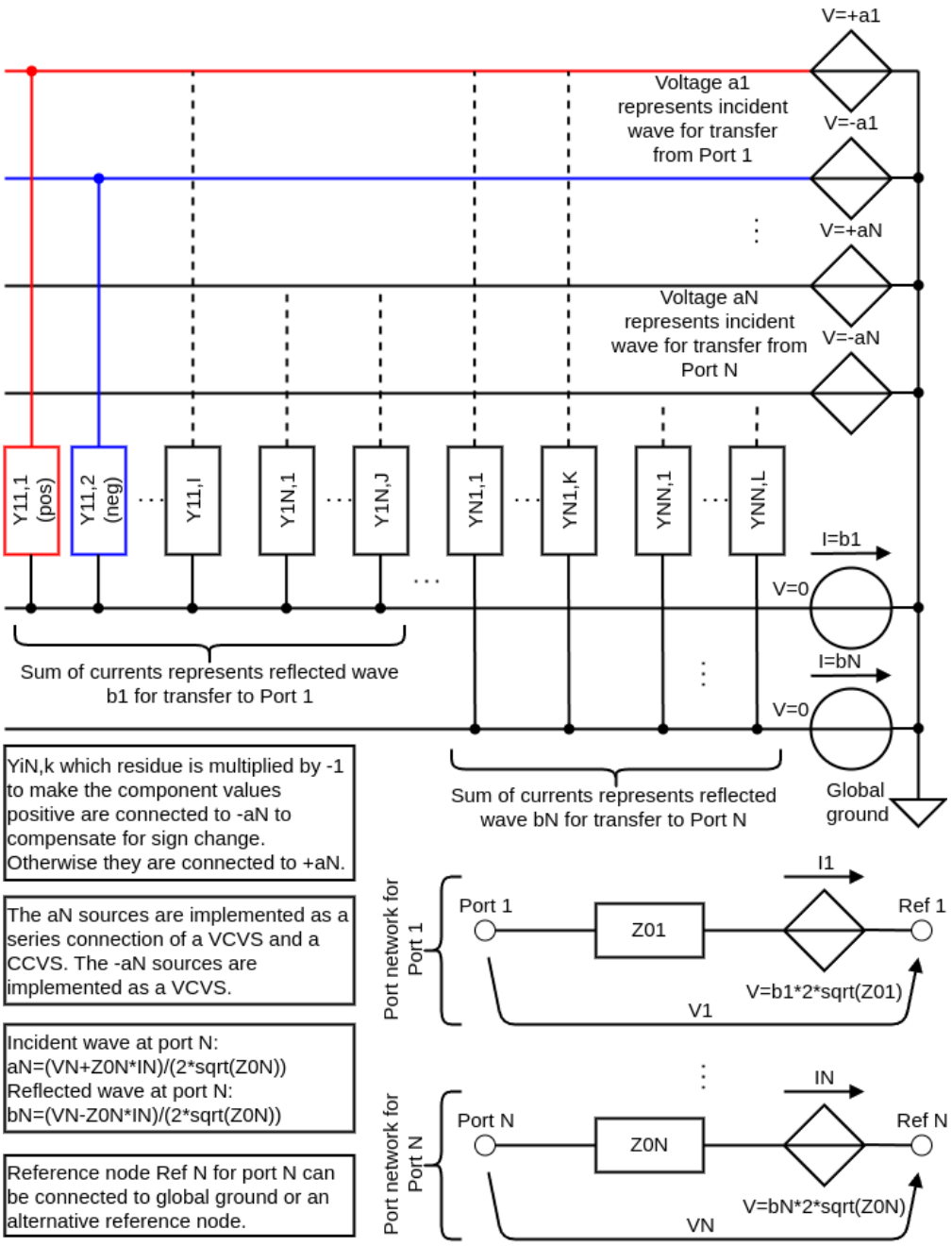
The incident power wave a_i is calculated from the port voltage V_i and the port current I_i, regardless of the type of interface network:

$$a_i = \frac{1}{2\sqrt{R_i}}(V_i + R_i I_i) \tag{1.18}$$

Expressing Scattering Responses Using Admittances

The figure below shows the structure of the interface and transfer networks of one such port i with the external port voltage V_i and port current I_i. The individual frequency responses $\underline{S}_{i,j}$ of the Vector Fit are reproduced with the equivalent admittances $\underline{Y}_{\mathrm{S},i,j}$ based on the fitting parameters $\underline{\mathbf{p}}$, $\underline{\mathbf{c}}$, $\mathbf{d}$ and $\mathbf{e}$ as described above.

This equivalent circuit topology has some room for optimizations. For example, the controlled sources for the incident power waves, which are driving the stacks of parallel admittances, can be realized differentially and the resulting currents can be summed in a single node for the transfer back to the respective ports for the reflected power waves. This is shown in the following schematic:



Expressing Scattering Responses Using Impedances

The figure below shows the structure of the interface and transfer networks of one such port i with the external port voltage V_i and port current I_i. The individual frequency responses $\underline{S}_{i,j}$ of the Vector Fit are reproduced with the equivalent impedances $\underline{Z}_{\mathrm{S},i,j}$ based on the fitting parameters $\underline{\mathbf{p}}$, $\underline{\mathbf{c}}$, $\mathbf{d}$ and $\mathbf{e}$ as described above. The impedances are driven by controlled current sources, which produce a current proportional to the incident power wave a_i with $g_{a,i} = \frac{1}{2\sqrt{R_i}}$ and $r_{a,i} = \frac{\sqrt{R_i}}{2}$. The voltages $V_{j,i,k}$ across the individual impedances $Z_{j,i,k}$, which represent a fraction of the total scattering response modelled by a single pole-residue pair, are transferred to the corresponding voltage-controlled current source at the port interfaces with a gain of $g_{b,j} = \frac{\sqrt{R_i}}{2}$, as derived above.

Direct implementation of the State-Space Equations

The direct synthesis of an equivalent circuit from the state-space model is fairly straightforward, as the circuit topology is derived from the state-space equations:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) \tag{1.19}$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t) + \mathbf{E}\dot{\mathbf{u}}(t) \tag{1.20}$$

In the case of scattering parameters, the input vector $\mathbf{u}$ represents the incident waves $\mathbf{a}$ and the output vector $\mathbf{y}$ represents the reflected waves $\mathbf{b}$. Regardless of the parameter representation, the state vector $\mathbf{x}$ is an internal variable defining the dynamics of the network.

In the equivalent circuit, the values of the circuit elements are given by the coefficients in the state-space matrices A, B, C, D (and E). Thanks to the sparsity of these matrices in case of a vector-fitted pole/residue model, the number of non-zero circuit elements is much lower than in the fully populated case, resulting in a compact netlist.

As shown in the schematic below, there is one interface network for each port, transferring the individual inputs and states with their respective gains to the output. The state networks depend on the pole type: a real pole is represented by one state, a complex conjugate pole pair is represented by two states. In case of the optional proportional term E, another differentiation network is required for each port.

The individual gains of the controlled sources are listed below, resulting mostly from the definition of the scattering parameters as a function of the port voltages and currents:

- $g_{D,i,j} = \frac{2}{\sqrt{R_{0,i}}} d_{i,j} \frac{1}{2\sqrt{R_{0,j}}}$
- $f_{D,i,j} = \frac{2}{\sqrt{R_{0,i}}} d_{i,j} \frac{\sqrt{R_{0,j}}}{2}$
- $g_{E,i,j} = \frac{2}{\sqrt{R_{0,i}}} e_{i,j}$
- $g_{C,i,j,k} = \frac{2}{\sqrt{R_{0,i}}} c_{i,j,k}$
- $g_{a,j} = \frac{1}{2\sqrt{R_{0,j}}}$
- $f_{a,j} = \frac{\sqrt{R_{0,j}}}{2}$

For more details on the state-space circuit synthesis, refer to "Passive Macromodeling" by S. Grivet-Talocia and B. Gustavsen [4].

Implementation in scikit-rf

The implementation in scikit-rf has changed several times in the past, experimenting with equivalent admittances, equivalent impedances, and direct state-space circuit synthesis, as shown above. Currently, scikit-rf uses a direct implementation of the state-space equations. Comparing the simulation runtimes in ngspice, Xyce, and LTspice, there are large differences between the types of equivalent netlists. While the exact reasons for the performance differences are difficult to investigate, it seems like the kind of controlled sources has a big impact in any simulation based on modified nodal analysis (MNA). Voltage-controlled current sources usually perform better than controlled voltage sources. Counterintuitively, the number of controlled sources is not always critical for most MNA circuit solvers, as long as the controlling signal is already present in the solution vector of the linear system. The file size of the synthesized equivalent netlist is therefore not representative of the performance in the simulator.

Case 2: Vector Fitted Y Parameters

Not implemented. See Y Parameters for a general equivalent circuit.

Case 3: Vector Fitted Z Parameters

Not implemented. See Z Parameters for a general equivalent circuit.

1.13.4 References

[1] B. Gustavsen and A. Semlyen, "Rational approximation of frequency domain responses by vector fitting," in IEEE Transactions on Power Delivery, vol. 14, no. 3, pp. 1052-1061, July 1999, DOI: 10.1109/61.772353.

[2] https://www.sintef.no/projectweb/vectorfitting/

[3] G. Antonini, "SPICE equivalent circuits of frequency-domain responses," in IEEE Transactions on Electromagnetic Compatibility, vol. 45, no. 3, pp. 502-512, Aug. 2003, DOI: 10.1109/TEMC.2003.815528.

[4] "Passive Macromodeling" by S. Grivet-Talocia and B. Gustavsen, Wiley, 2016, DOI: 10.1002/9781119140931.

1.14 Parametric Vector Fitting

The implementation for the parametric vector fitting is based on the algorithm proposed in [1]

1.14.1 Mathematical Description

The parametric model function $\underline{\underline{\mathbf{H}}}(\underline{s}; \lambda)$ is defined in the Laplace domain with $\underline{s} = \sigma + j\omega$ and for a parameter (or a vector of parameters) λ:

$$\underline{\underline{\mathbf{H}}}(\underline{s}; \lambda) = \frac{\mathbf{Q}_0(\lambda) + \sum_{k=1}^{K} \frac{\mathbf{Q}_k(\lambda)}{\underline{s} - \underline{p}_k}}{\mathbf{R}_0(\lambda) + \sum_{k=1}^{K} \frac{\mathbf{R}_k(\lambda)}{\underline{s} - \underline{p}_k}} \tag{1.21}$$

This model is based on a set of K poles $\underline{p}_k$, which do not depend on the value(s) of the parameter(s). In contrast, the numerator coefficients $\mathbf{Q}_k(\lambda)$ and the denominator coefficients $\mathbf{R}_k(\lambda)$ do depend on the parameters. They get calculated on a given grid of parameter samples using regular vector fitting. These coefficients can then be interpolated to calculate the model response at any parameter value between the gridpoints.

1.14.2 References

[1] P. Triverio, S. Grivet-Talocia and M. S. Nakhla, "A Parameterized Macromodeling Strategy With Uniform Stability Test", IEEE Transactions on Advanced Packaging, vol. 32, no. 1, pp. 205-215, Feb. 2009, DOI: https://doi.org/10.1109/TADVP.2008.2007913

Download This Notebook: `VirtualInstruments.ipynb`

1.15 Virtual Instruments

1.15.1 Introduction

Warning

The *vi* modules depends on PyVISA as well as some additional libraries. Make sure you've followed the instructions in the *installation tutorial* to ensure a working setup.

Additionally, PyVISA can use a number of different backends to communicate with instruments. The installation of these is up to the user, but there are further instructions in the PyVISA documentation By default, PyVISA uses it's Python VISA implementation, but certain instrument configurations require their manufacturer's specific VISA library be installed and used to communicate with instruments. For example, the Keysight IO libraries suite has to be used to communicate with their instruments over GPIB.

The virtual instrument module defines a consistent high-level interface for interacting with different types of instruments that can be controlled with SCPI (or other means). Primarily for scikit-rf, this is vector network analyzers (VNAs), but we are actively working on adding support for spectrum analyzers.

For information on specific instruments, consult the *vi* module's documentation.

The currently supported instruments are (organized by manufacturer):

Rohde & Schwarz

- ZNA
- ZVA

Keysight

- FieldFox
- PNA

NanoVNA

- NanoVNA V2

Tip

You can help grow scikit-rf by adding support for more instruments! It's really not that complicated. Check out the *vi module documentation* to learn how.

1.15.2 Examples

Note

You can download this Jupyter notebook and run it directly on your PC: `VirtualInstruments.ipynb`

Make sure to comment out any code that is not needed for your instrument.

Connecting to an instrument

```
[ ]: from skrf import Frequency
     from skrf.vi.vna.keysight import PNA
     from skrf.vi.vna.nanovna import NanoVNAv2
     from skrf.vi.vna.rohde_schwarz import ZNA
```

(continues on next page)

(continued from previous page)

```
# Rohde & Schwarz ZNA over ethernet
instr = ZNA(address="TCPIP::10.0.0.5::INSTR")
ch1 = instr.ch1

# Keysight PNA over ethernet
instr = PNA(address="TCPIP::10.0.0.5::INSTR")
ch1 = instr.ch1

# NanoVNA V2 over USB
instr = NanoVNAv2("ASRL/dev/ttyACM0::INSTR")  # Linux
instr = NanoVNAv2("ASRL1::INSTR")  # Windows
ch1 = instr  # NanoVNA does not support channels

print(instr.id)
```

Changing the frequency settings of an instrument

[ ]:
```
freq = Frequency(start=1, stop=2, npoints = 101, unit="GHz")  # create frequency object
ch1.frequency = freq  # apply frequency
```

Creating measurements on the instrument

[ ]:
```
# not supported on NanoVNA
ch1.create_measurement(name="Temp_S11", parameter="S11")
ch1.create_measurement(name="Temp_S22", parameter="S22")
```

Getting a 2-port network and plotting

[ ]:
```
ntwk = ch1.get_snp_network(ports=(1,2))
ntwk.plot_s_db()
```

Deleting measurements on the instrument

[ ]:
```
# not supported on NanoVNA
ch1.delete_measurement(name="Temp_S11")
ch1.delete_measurement(name="Temp_S22")
```

CHAPTER

TWO

EXAMPLES

These examples are meant to give an overview of how skrf can be used. Each page can be followed along in a browser, or downloaded as an ipython notebook

2.1 Network Theory

2.1.1 Balanced Network De-embedding

Demonstration of *balanced*, i.e. 2N-port, network de-embedding.

Setup

[1]:
```
import matplotlib.pyplot as plt

import skrf as rf
from skrf.media import CPW
from skrf.network import concat_ports, connect

rf.stylely()
```

[2]:
```
# base parameters
freq = rf.Frequency(1e-3,10,1001,'ghz')
cpw  = CPW(freq, w=0.6e-3, s=0.25e-3, ep_r=10.6, z0_port=50)
```

Build fixture network

- short length of mismatched line with connector-like input shunt capacitance
- some crosstalk added with nudge

[3]:
```
"""
             11
    0----+-=======-2
         |
         = c1
         |
        GND

             11
    1----+-=======-3
         |
```

(continues on next page)

(continued from previous page)

```
         = c1
         |
        GND
"""


l1 = cpw.line(20, unit='mm')
c1 = cpw.shunt_capacitor(C=0.15e-12)
l1 = connect(c1, 1, l1, 0)
li = concat_ports([l1, l1], port_order='second')
Fix = li
Fix.name = 'Fix'
Fix.nudge(1e-4)
Left = Fix
# flip fixture for right side
Right = Fix.flipped()
```

Build DUT network

- some length of mismatched lines
- some crosstalk added with nudge

[4]:
```
"""
        l2
    0-=======-2
        l2
    1-=======-3
"""
l2 = cpw.line(50, 'mm')
DUT = concat_ports([l2, l2], port_order='second')
DUT.name = 'DUT'
DUT.nudge(1e-5)
```

Build the measurement

- cascade Left, DUT and Right
- add some noise

[5]:
```
"""
            Left        Meas         Right
             l1          l2           l1
    0----+-=======-2 0-=======-2 0-=======-+----2
         |                                  |
         = c1                               = c1
         |                                  |
        GND                                GND

             l1          l2           l1
    1----+-=======-3 1-=======-3 1-=======-+----3
         |                                  |
         = c1                               = c1
```

(continues on next page)

(continued from previous page)

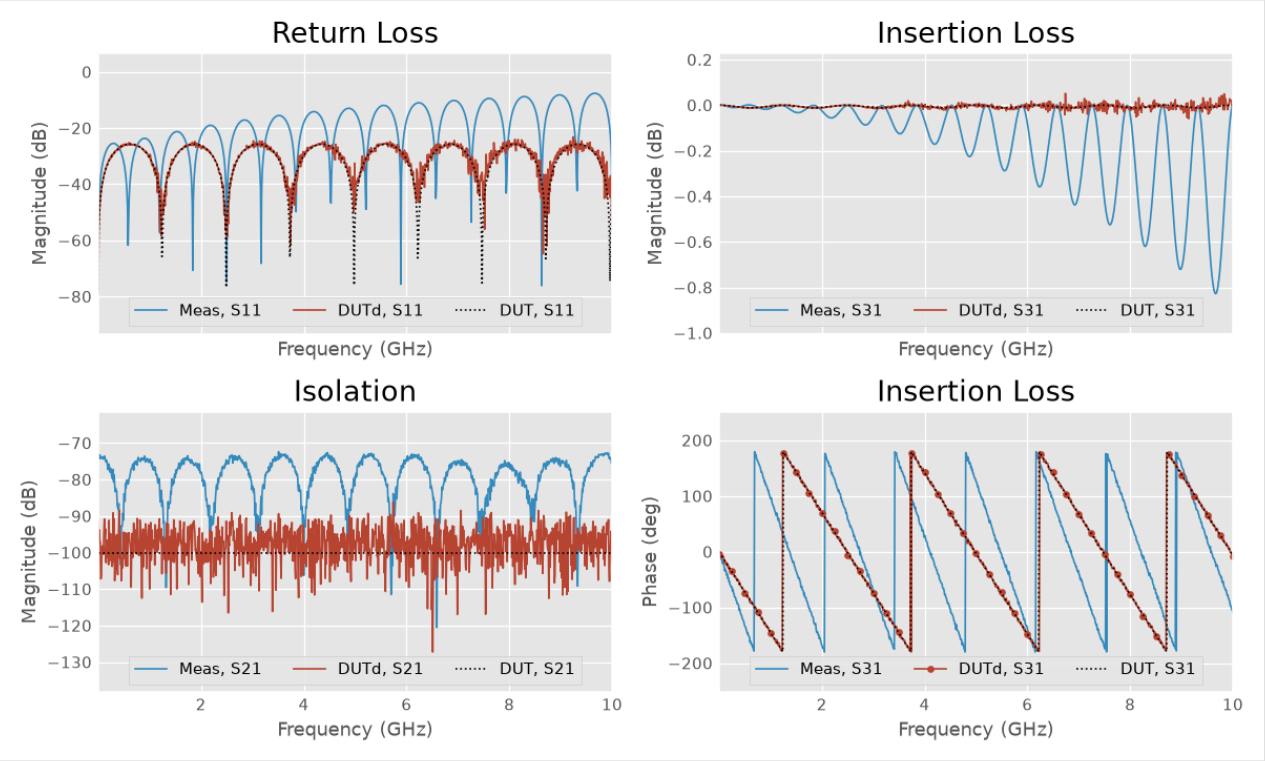
```
        |                                   |
       GND                                 GND
"""
Meas = Left ** DUT ** Right
Meas.name = 'Meas'
Meas.add_noise_polar(1e-5, 2)
```

Perform de-embedding

[6]:
```
DUTd = Left.inv ** Meas ** Right.inv
DUTd.name = 'DUTd'
```

Display results

[7]:
```

fig, axarr = plt.subplots(2,2, sharex=True, figsize=(10,6))

ax = axarr[0,0]
Meas.plot_s_db(m=0, n=0, ax=ax)
DUTd.plot_s_db(m=0, n=0, ax=ax)
DUT.plot_s_db(m=0, n=0, ax=ax, ls=':', color='0.0')
ax.set_title('Return Loss')
ax.legend(loc='lower center', ncol=3)
ax.grid(True)

ax = axarr[0,1]
Meas.plot_s_db(m=2, n=0, ax=ax)
DUTd.plot_s_db(m=2, n=0, ax=ax)
DUT.plot_s_db(m=2, n=0, ax=ax, ls=':', color='0.0')
ax.set_title('Insertion Loss')
ax.legend(loc='lower center', ncol=3)
ax.grid(True)

ax = axarr[1,0]
Meas.plot_s_db(m=1, n=0, ax=ax)
DUTd.plot_s_db(m=1, n=0, ax=ax)
DUT.plot_s_db(m=1, n=0, ax=ax, ls=':', color='0.0')
ax.set_title('Isolation')
ax.legend(loc='lower center', ncol=3)
ax.grid(True)

ax = axarr[1,1]
Meas.plot_s_deg(m=2, n=0, ax=ax)
DUTd.plot_s_deg(m=2, n=0, ax=ax, marker='o', markevery=25)
DUT.plot_s_deg(m=2, n=0, ax=ax, ls=':', color='0.0')
ax.set_title('Insertion Loss')
ax.legend(loc='lower center', ncol=3)
ax.grid(True)

fig.tight_layout()
```



[ ]:

2.1.2 Balun & Transformer Designs

[1]:
```
import matplotlib.pyplot as plt

import skrf as rf
from skrf.circuit import Circuit
from skrf.constants import c

rf.stylely()
```

This tutorial explains a few passive RF circuits, such as ferrite-loaded baluns & transformers using transmission lines, within the scikit-rf environment. We tested these circuits in Windows Anaconda, Android Pydroid 3 and iOS Juno.

Balun

Baluns are balanced-to-unbalanced circuits. In other words, baluns convert differential to single-ended circuits, and vice versa.

Transformers, as the name indicates, transform one RF impedance to another.

Both baluns and transformers can be built using transmission lines or lumped elements. Baluns and transformers are used extensively, but not exclusively, in RF matching circuits, combiners, dividers, amplifiers, mixers. At times, baluns and transformers are used in antenna designs, antenna emulators and/or antenna tuners (aka couplers).

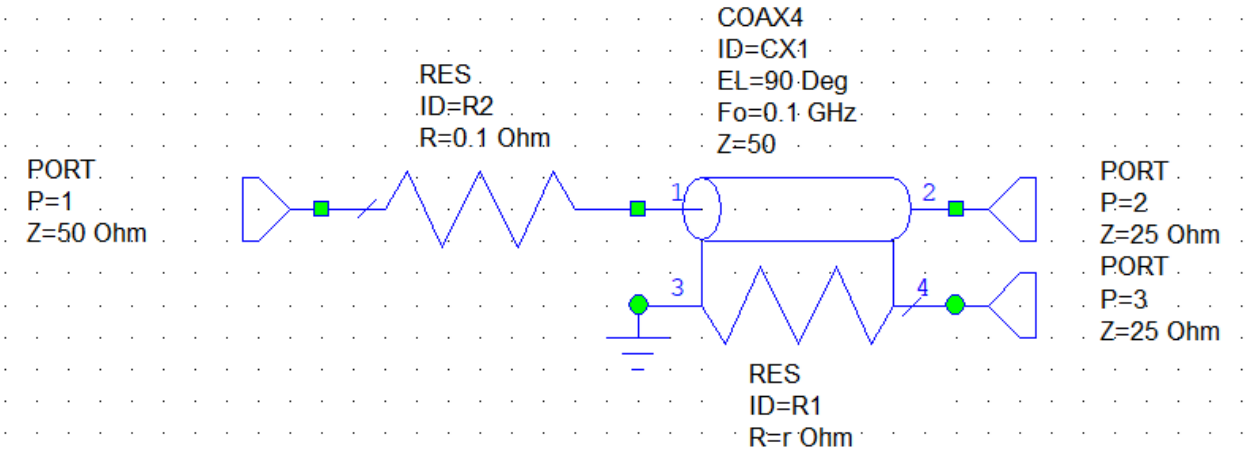


Fig.1 Ideal Schematic of a Coax Balun

In Fig.1 above, a coaxial balun is depicted and is comprised of a 4-port (outer conductor floating) coax and a resistor. In this circuit, the resistor is employed to model the effect of a ferrite sleeve (or bead) around the coax. The resistor acts to discourage current flow on the outside surface of the outer conductor, which makes the coax behave exactly like a twisted pair of wires over a bandwidth of interest. In some cases, a lumped inductor can replace the resistor to model the frequency-dependent behavior of the ferrite material of the sleeve. The actual value of the resistor, or the inductor, is a function of the frequency range of the design and the characteristics of the ferrite material. Often, the value of the resistor, or inductor, is derived from experimental measurements in the lab. We use 333-Ohms here as a reasonable value to demonstrate the usage of scikit-rf only.

In reality, a 4-port transmission line is implemented in scikit-rf exactly as shown in the nodal diagram of figure below:

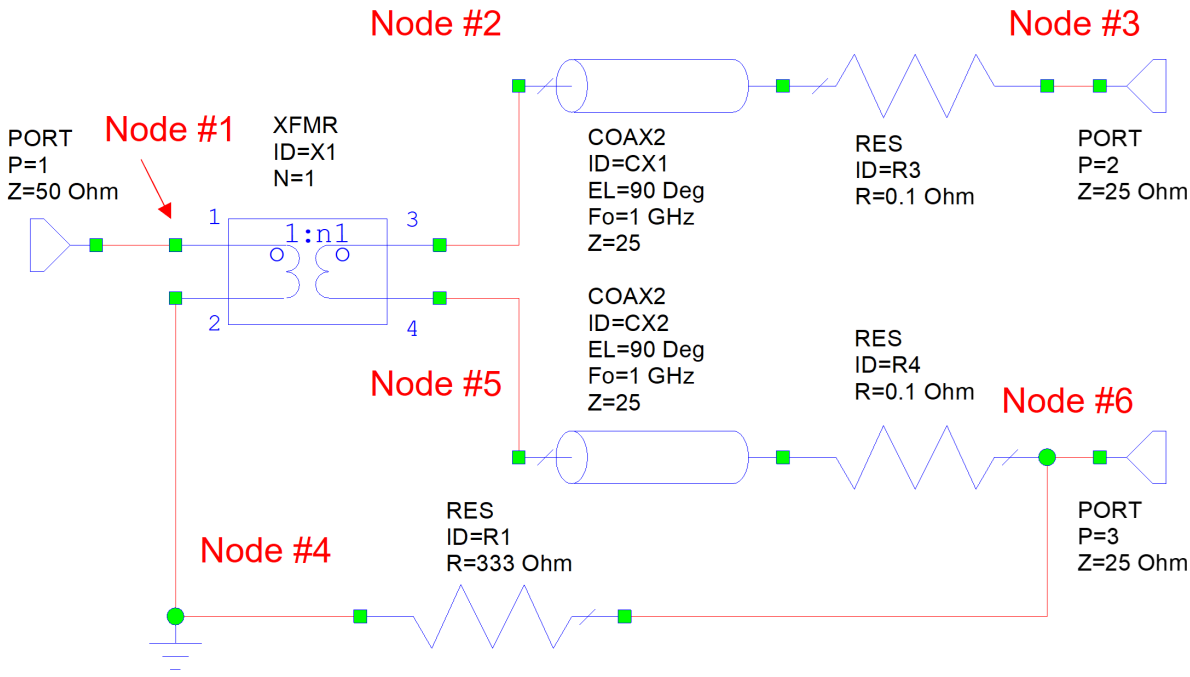


Fig.2 Nodal Connections

Using Fig.2 as a guide, we can create a transmission-line balun as follows. First, we add some insertion loss to the trandmission lines in the firm of a 0.1-Ohm resistor. Then we connect all the elements (transformer, two 25-Ohms lines and the ferrite resistor) together to create the balun. The sum port is 50-Ohms single-ended. The differential output ports are 25-Ohms each.

[2]:
```
# need this ideal s4p file to create a coax with a floating outer conductor
xformer = r"Balun_Transformer_Designs/Ideal_4port_xformer_hi_res.s4p"
xformer_ntwk = rf.Network(xformer)
xformer_ntwk1 = xformer_ntwk["0.001-2ghz"]  # band center is thus 1 GHz

freq = xformer_ntwk1.frequency
z0_ports = 25

beta = freq.w / c
line_branch = rf.media.DefinedGammaZ0(frequency=freq, z0=z0_ports, gamma=0+beta*1j)

# d is 75 mm in length, which is quarterwave at 1 GHz in air
#d = line_branch.theta_2_d(90, deg=True, bc=True)
branch1 = line_branch.line(90, unit="deg", name="branch1")

lossy_line = line_branch.resistor(0.1, name="res")
```

The RF circuit is build using the scikit-rf Circuit capabilities:

[3]:
```
port1 = Circuit.Port(freq, name="port1", z0=25)
port2 = Circuit.Port(freq, name="port2", z0=25)

# creating a 25-Ohm 2-port transmission line, with realistic insertion loss
# insertion loss is equivalent to a 0.1-Ohm series resistor
connections = [
    [(port1, 0), (lossy_line, 0)],
    [(lossy_line, 1), (branch1, 0)],
    [(branch1, 1), (port2, 0)],
]

C = Circuit(connections)

line1 = C.network
line1.name = "top line"
line2 = C.network
line2.name = "bottom line"

line = rf.media.DefinedGammaZ0(frequency=freq, z0=50)
ferrite_resistor = line.resistor(333, name="res")

port1a = Circuit.Port(frequency=freq, name="port1a", z0=50)
port2a = Circuit.Port(frequency=freq, name="port2a", z0=25)
port3a = Circuit.Port(frequency=freq, name="port3a", z0=25)
ground = Circuit.Ground(frequency=freq, name="ground", z0=50)

# creating a 4-port floating coax by feeding two 25-Ohm lines (twisted pair of wires)
# with an ideal transformer, to ensure the differential nature of the twisted pair, as␣
↪per Fig.2
# the 333-Ohm resistor models the loading of the ferrite sleeve,
# upon the outer conductor of the 4-port coax line
conn = [
    [(port1a, 0), (xformer_ntwk1, 0)],
    [(xformer_ntwk1, 1), (line1, 0)],
```

(continues on next page)

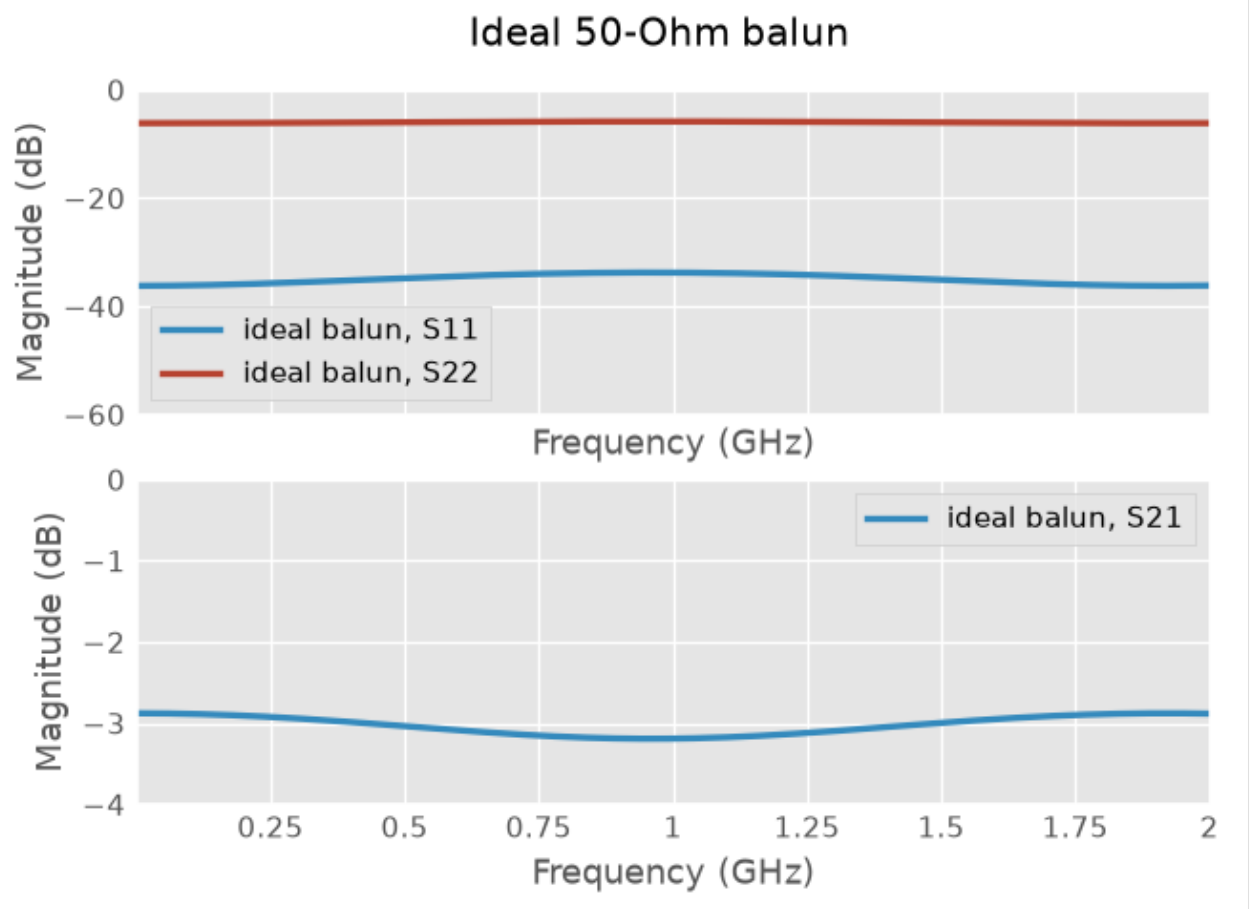
(continued from previous page)

```
    [(line1, 1), (port2a, 0)],
    [(ground, 0), (xformer_ntwk1, 2), (ferrite_resistor, 0)],
    [(xformer_ntwk1, 3), (line2, 0)],
    [(line2, 1), (ferrite_resistor, 1), (port3a, 0)],
]

C1 = Circuit(conn)
balun = C1.network
balun.name = "ideal balun"
```

Let's have a look the the S-parameters of this ideal balun:

[4]:
```
fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

balun.plot_s_db(ax=ax1, m=0, n=0, lw=2)  # S11
balun.plot_s_db(ax=ax1, m=1, n=1, lw=2)  # S22
ax1.set_ylim(-60, 0)

balun.plot_s_db(ax=ax2, m=1, n=0, lw=2)  # S21
ax2.set_ylim(-4, 0)

fig.suptitle("Ideal 50-Ohm balun")
```

[4]:
```
Text(0.5, 0.98, 'Ideal 50-Ohm balun')
```

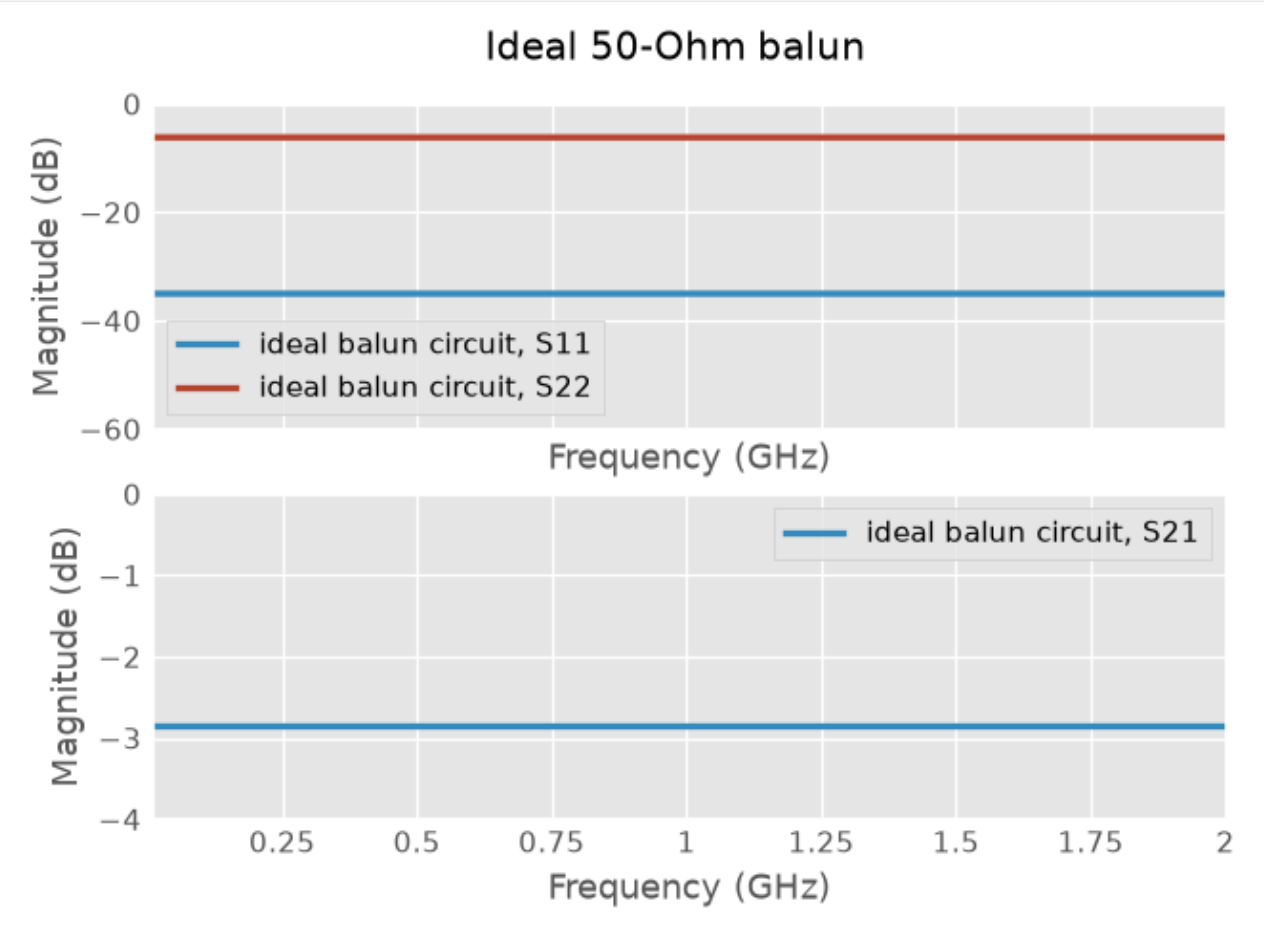


As can be seen from graph above, the sum port return loss S11 (blue trace, upper graph) is -35dB. The S22 (red trace, upper graph) is -6dB, indicating the typical return loss of a terminal within a differential pair. The S21 (blue trace, lower graph) is -3dB, as expected, to show that the energy entering the sum port splits evenly between the two differential ports.

The above example of a balun design was implemented in scikit-rf version 1.3 (also compatible with earlier versions). For scikit-rf version 1.4, the script below employs a newly created Media element, called line_floating, to simplify the circuit.

[5]:
```
freq1 = rf.Frequency(start=0.01, stop=2, unit=('GHz'), npoints=2001)
beta = freq1.w / c

line1 = rf.media.DefinedGammaZ0(frequency=freq1, z0=50, gamma=0+beta*1j)

ferrite_resistor = line1.resistor(333, name='res') # model ferrite sleeve as a parallel␣
↪333 Ohm resistor
Rp_circuit = ferrite_resistor

# define a floating 4-port transmission line, using a new element in ver 1.4 of Scikit-rf
line4p = line1.line_floating(90, unit='deg', z0=50, name='line4p')
port1 = Circuit.Port(frequency=freq1, name='port1', z0=50)
port2 = Circuit.Port(frequency=freq1, name='port2', z0=25)
port3 = Circuit.Port(frequency=freq1, name='port3', z0=25)
ground = Circuit.Ground(frequency=freq1, name='ground', z0=50)

connections = [[(port1, 0), (line4p, 0)],
    [(port2, 0), (line4p, 1)],
    [(port3, 0), (line4p, 3), (Rp_circuit, 0)],
    [(line4p, 2), (Rp_circuit, 1), (ground, 0)]]

circuit = Circuit(connections)
balun_circuit = circuit.network
balun_circuit.name = 'ideal balun circuit'

fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True)

balun_circuit.plot_s_db(ax=ax1, m=0, n=0, lw=2)  # S11
balun_circuit.plot_s_db(ax=ax1, m=1, n=1, lw=2)  # S22
ax1.set_ylim(-60, 0)

balun_circuit.plot_s_db(ax=ax2, m=1, n=0, lw=2)  # S21
ax2.set_ylim(-4, 0)

fig.suptitle("Ideal 50-Ohm balun")
```

[5]:
```
Text(0.5, 0.98, 'Ideal 50-Ohm balun')
```



Guanella transformer

The next circuit to be explored is a Guanella transformer, which is a 4-to-1 impedance transformer. In this example, a 100-Ohms impedance is transformed to 25-Ohms using two 50-Ohms transmission lines. This example is implemented in scikit-rf version 1.4.

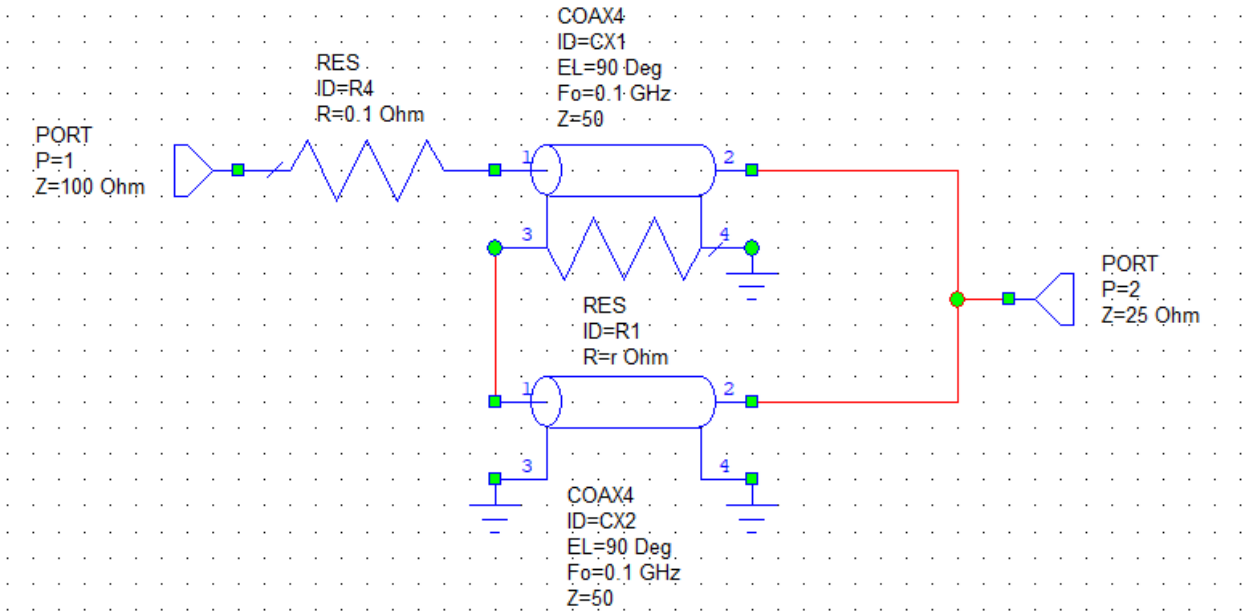


Fig.3 Ideal Guanella Transformer

Using Fig.3 as a nodal guide, a Guanella transformer can be created. Note that on the left-hand side of the circuit, the two transmission lines are connected in RF series to form the 100-Ohms port. On the opposite end, the right-hand side is connected in parallel to form the 25-Ohms port. Also, the ferrite sleeve is modeled as an inductor in this example.

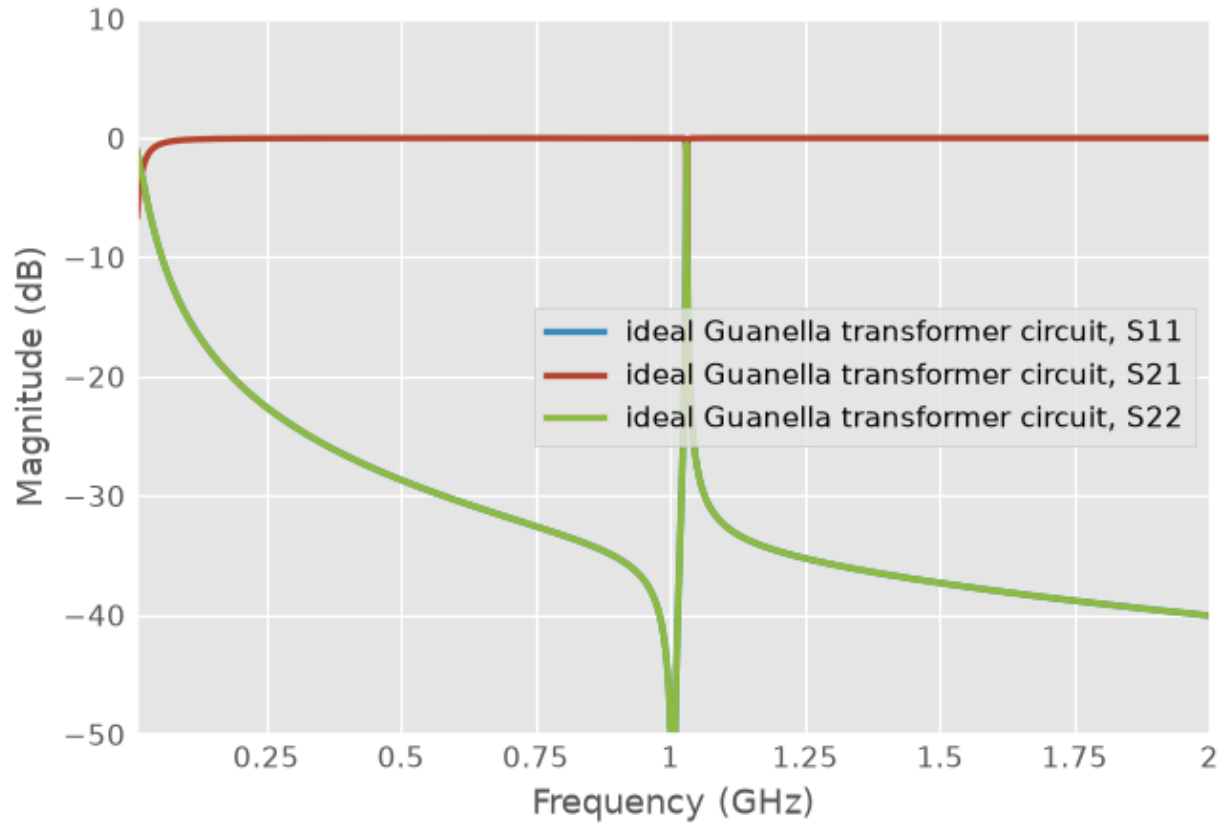
[6]:
```
freq = rf.Frequency(start=0.01, stop=2, unit=('GHz'), npoints=2001)
beta = freq.w/c

line = rf.media.DefinedGammaZ0(frequency=freq, z0=50, gamma=0+beta*1j)
ferrite_inductor = line.inductor(100e-9, name='ind') # model ferrite sleeve as a␣
↪parallel 100 nH inductor

# define a floating 4-port transmission line
line4p_a = line.line_floating(90, unit='deg', z0=50, name='line4p_a')
line4p_b = line.line_floating(90, unit='deg', z0=50, name='line4p_b')

port1 = Circuit.Port(frequency=freq, name='port1', z0=100)
port2 = Circuit.Port(frequency=freq, name='port2', z0=25)
ground1 = Circuit.Ground(frequency=freq, name='ground1', z0=50)
ground2 = Circuit.Ground(frequency=freq, name='ground2', z0=50)

connections = [[(port1, 0), (line4p_a, 0)],
    [(port2, 0), (line4p_a, 1), (line4p_b, 1)],
    [(line4p_a, 2), (line4p_b, 0), (ferrite_inductor, 0)],
    [(line4p_a, 3), (ferrite_inductor, 1), (ground1, 0), (line4p_b, 3)],
    [(line4p_b, 2), (ground2, 0)]]

circuit = Circuit(connections)
transf_circuit = circuit.network
transf_circuit.name = 'ideal Guanella transformer circuit'

transf_circuit.plot_s_db(m=0, n=0, lw=2)
```

(continues on next page)

(continued from previous page)

```
transf_circuit.plot_s_db(m=1, n=0, lw=2)
transf_circuit.plot_s_db(m=1, n=1, lw=2)

plt.ylim(-50,10)
```

[6]: (-50.0, 10.0)



In this Guanella transformer example, the ferrite sleeve is modeled as an inductor shunting across the outer conductor of the upper coax. As can be seen in the S-parameter plot immediately above, the transformer works well between 200 and 1000 MHz. Below 200 MHz, the ferrite-sleeve inductor does not provide enough impedance to "choke" to coax line properly. At 1000 MHz, this circuit has a sharp resonance which constrains the upper frequency limit to 1 GHz.

Marchand balun

The final circuit that we discuss here is the triax Marchand balun, which does not employ any ferrite material. In this circuit, two triax sections are cascaded and the balanced output is taken from the middle. Fig.4 shows a sketch of a Marchand balun, as per the description in the original Marchand's paper.

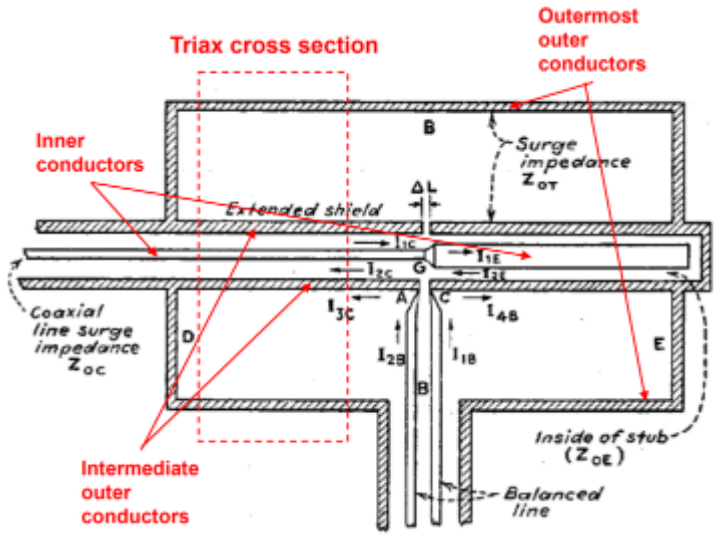


Fig.4 Original Marchand Balun Drawing

As can be seen in Fig.4, the triax cross section has an inner conductor that is shielded by an intermediate circular outer conductor. In turn, the circular outermost outer conductor envelopes the intermediate conductor.

Fig.5 Shows an electrical circuit which models the triax Marchand balun. Two nested coax lines are employed to emulate one triax structure.

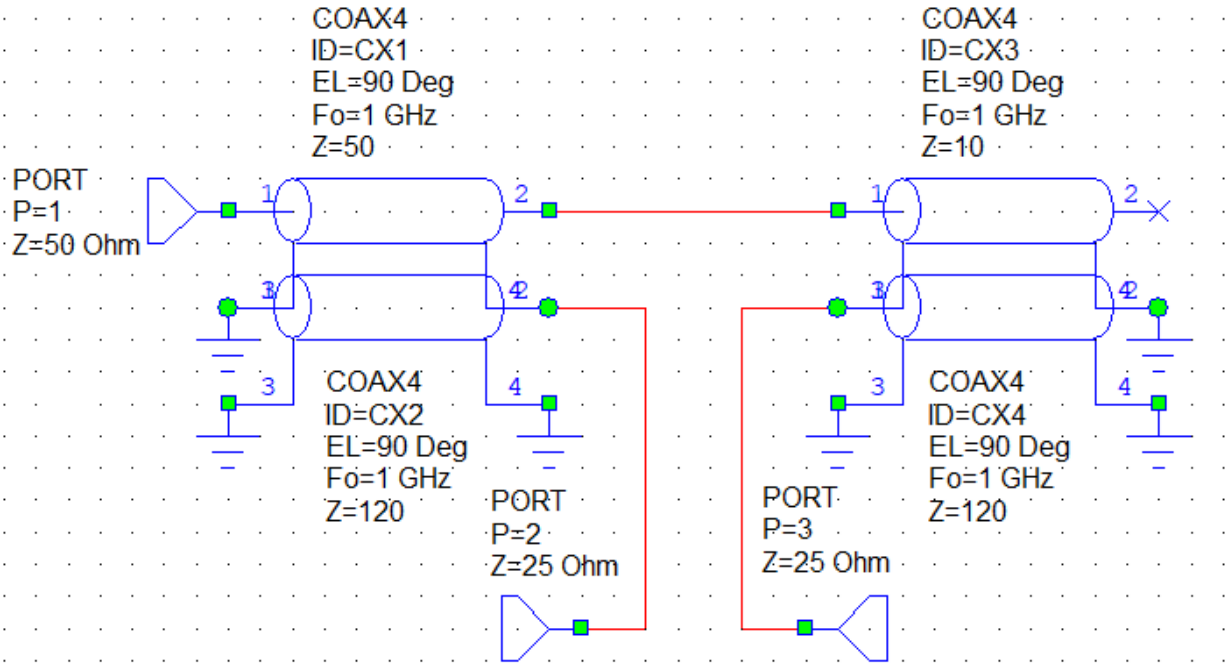


Fig.5 Ideal Marchand Balun Circuit

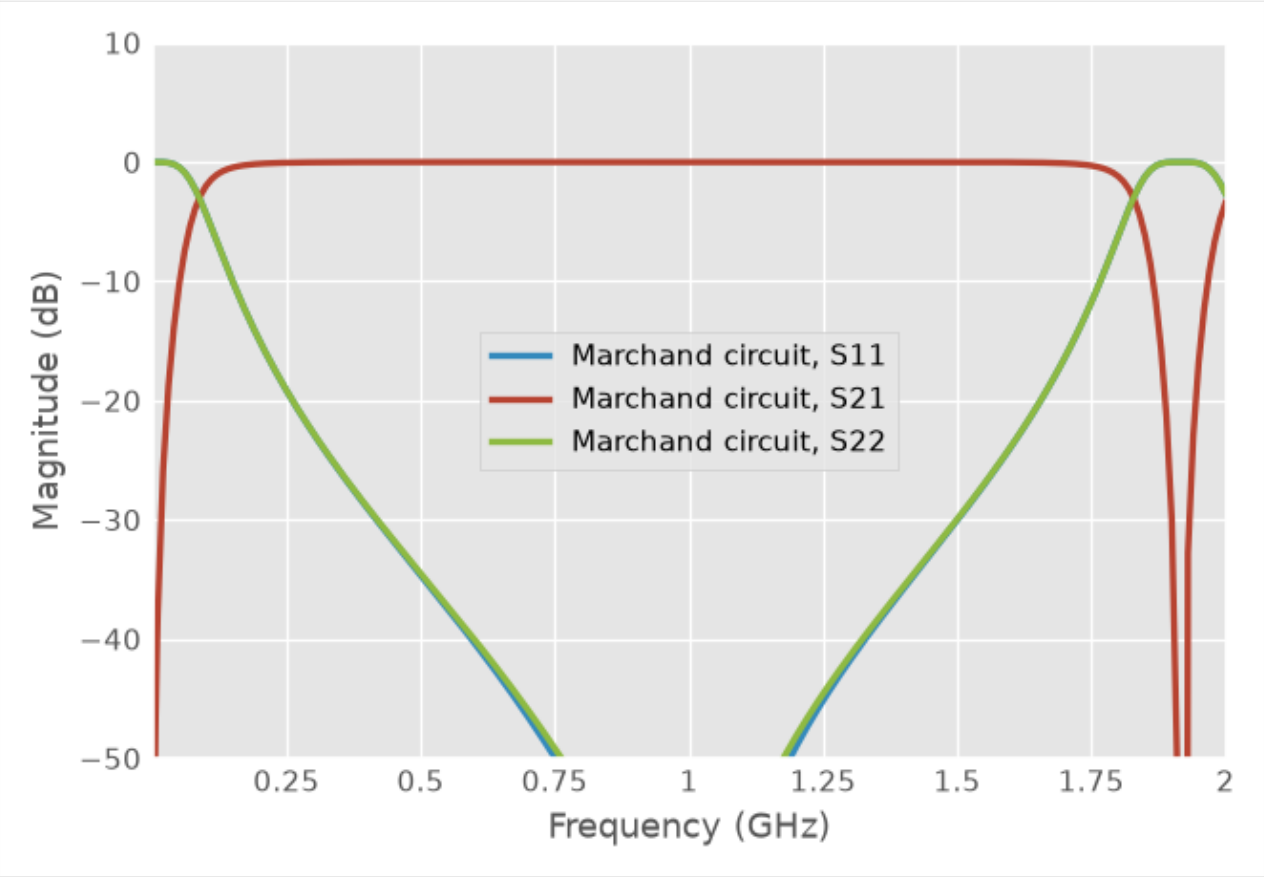
As seen in Fig.5, coax CX1 and CX2 form the first triax. The inner most coax impedance CX1 is 50-Ohms. The outer conductor of CX1 becomes the inner conductor of the 120-Ohms CX2.

The second triax is formed by the 10-Ohms CX3 and the 120-Ohms CX4. Similar to the previous triax, the outer conductor of CX3 becomes the inner conductor of CX4.

The ideal transformer in Fig.5 is needed to show that the output of the Marchand balun is indeed differential.

[7]:
```
# ideal transformer to convert differential ports to 50-Ohms single-ended
xformer = r"Balun_Transformer_Designs/Ideal_4port_xformer_hi_res.s4p"
xformer_ntwk = rf.Network(xformer)
xformer_ntwk1a = xformer_ntwk["0.001-2ghz"]

freq = xformer_ntwk1a.frequency
beta = freq.w/c

line = rf.media.DefinedGammaZ0(frequency=freq, z0=50, gamma=0+beta*1j)
C1 = line.capacitor(0.106e-12, name='cap')

# define a floating 4-port transmission line
line4p_a1 = line.line_floating(90, unit='deg', z0=50, name='line4p_a1')
line4p_b1 = line.line_floating(90, unit='deg', z0=120, name='line4p_b1')
line4p_c1 = line.line_floating(90, unit='deg', z0=10, name='line4p_c1')
line4p_d1 = line.line_floating(90, unit='deg', z0=120, name='line4p_d1')

port1a = Circuit.Port(frequency=freq, name='port1', z0=50)
port2a = Circuit.Port(frequency=freq, name='port2', z0=50)
ground1a = Circuit.Ground(frequency=freq, name='ground1', z0=50)
ground2a = Circuit.Ground(frequency=freq, name='ground2', z0=50)

connections = [[(port1a, 0), (line4p_a1, 0)],
    [(ground1a, 0), (line4p_a1, 2), (line4p_b1, 0), (line4p_b1, 2), (line4p_b1, 3)],
    [(line4p_a1, 1), (line4p_c1, 0)],
    [(line4p_a1, 3), (line4p_b1, 1), (xformer_ntwk1a, 0)],
    [(line4p_c1, 2), (xformer_ntwk1a, 2), (line4p_d1, 0)],
    [(line4p_d1, 1), (line4p_c1, 3), (line4p_d1, 2), (line4p_d1, 3), (C1, 1), (xformer_
↪ntwk1a, 3), (ground2a, 0)],
    [(line4p_c1, 1), (C1, 0)],
    [(xformer_ntwk1a, 1), (port2a, 0)]]

circuit1 = Circuit(connections)
Marchand_circuit = circuit1.network
Marchand_circuit.name = 'Marchand circuit'

Marchand_circuit.plot_s_db(m=0, n=0, lw=2)
Marchand_circuit.plot_s_db(m=1, n=0, lw=2)
Marchand_circuit.plot_s_db(m=1, n=1, lw=2)

plt.ylim(-50,10)
```

[7]:
```
(-50.0, 10.0)
```



In the S-parameter plot of the Marchand balun above, the bandwidth of this circuit is from 250 to 1600 MHz (-20dB return-loss bandwidth). This multi-octave performance is achieved without the use of any ferrite sleeve. The compromise here is the complexity of the balun's construction.

Using these three simple examples, one can build many other interesting transmission-line circuits in scikit-rf. Furthermore, these circuits can be built on an iphone (Juno), or an Android tablet (Pydroid 3), or a laptop (Anaconda). Such ease of access allows design ideas to be quickly & accurately captured and tested where or whenever they are conceived. The possibilities and potentials are endless.

2.1.3 CPW media example

A coplanar waveguide (CPW) is a planar transmission line where the return path is a plane located on both sides of a strip conductor on the same layer. The characteristic impedance depends on the strip width and the gap with the return plane, giving two degrees of freedom to the designer.

A great advantage of CPW is the convenience to perform shunts or series connections with components because both the signal and the return path are on the same side, thus no via is required. On the other hand, the large coplanar planes require a lot of area.

The coplanar waveguide has quasi-TEM mode and less dispersion than microstrip. To avoid the propagation of parasitic slotline mode, the planes on both sides of the strip have to be connected together. This could be done using air bridges on MMICs or by adding a conductor plane on the backside of the substrate and connecting all planes together with via fences.

When using conductor backing, the line could behave like a microstrip line instead, should the height of the substrate be small and the gap with coplanar planes be big.

[1]:
```
import matplotlib.pyplot as plt

import skrf as rf
from skrf.calibration.deembedding import IEEEP370_SE_NZC_2xThru
from skrf.media import CPW

rf.stylely()
```

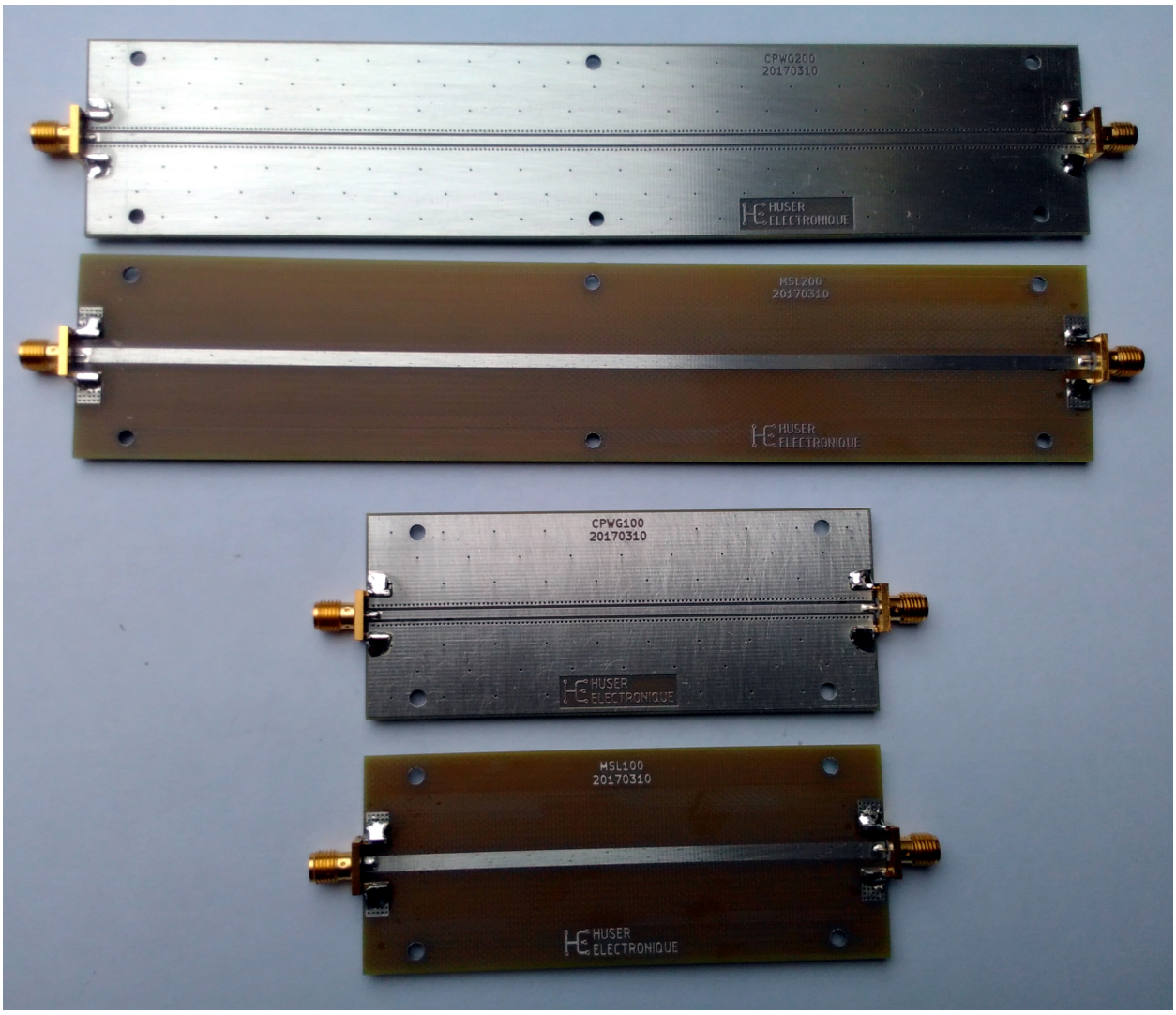
Measurement of two CPWG lines with different lengths

The measurement was performed on the 21st of March 2017 on an Anritsu MS46524B 20GHz Vector Network Analyser. The setup is a linear frequency sweep from 1 MHz to 10 GHz with 10'000 points. Output power is 0 dBm, IF bandwidth is 1kHz and neither averaging nor smoothing were used.

CPWGxxx is an L long, W wide, with a G wide gap to top ground, T thick copper coplanar waveguide with conductor backing on an H height substrate with top and bottom ground plane. A closely spaced vias wall is placed on both sides of the line and the top and bottom ground planes are connected by many vias.

Name	L (mm)	W (mm)	G (mm)	H (mm)	T (um)	Substrate
CPW100	100	1.70	0.50	1.55	50	FR-4
CPW200	200	1.70	0.50	1.55	50	FR-4

The milling of the artwork is performed mechanically with a lateral wall of 45°. The relative permittivity of the dielectric was assumed to be approximately 4.5 for design purposes.



Looking at measurements

We look at measurement insertion loss phase and magnitude as well as return loss phase and magnitude.

The insertion loss of CPWG200 is twice that of CPWG100 and is mostly linear up to 6 GHz. The phase shows also a factor two. Both statements agree with the fact that CPWG200 has twice the length of CPWG100.

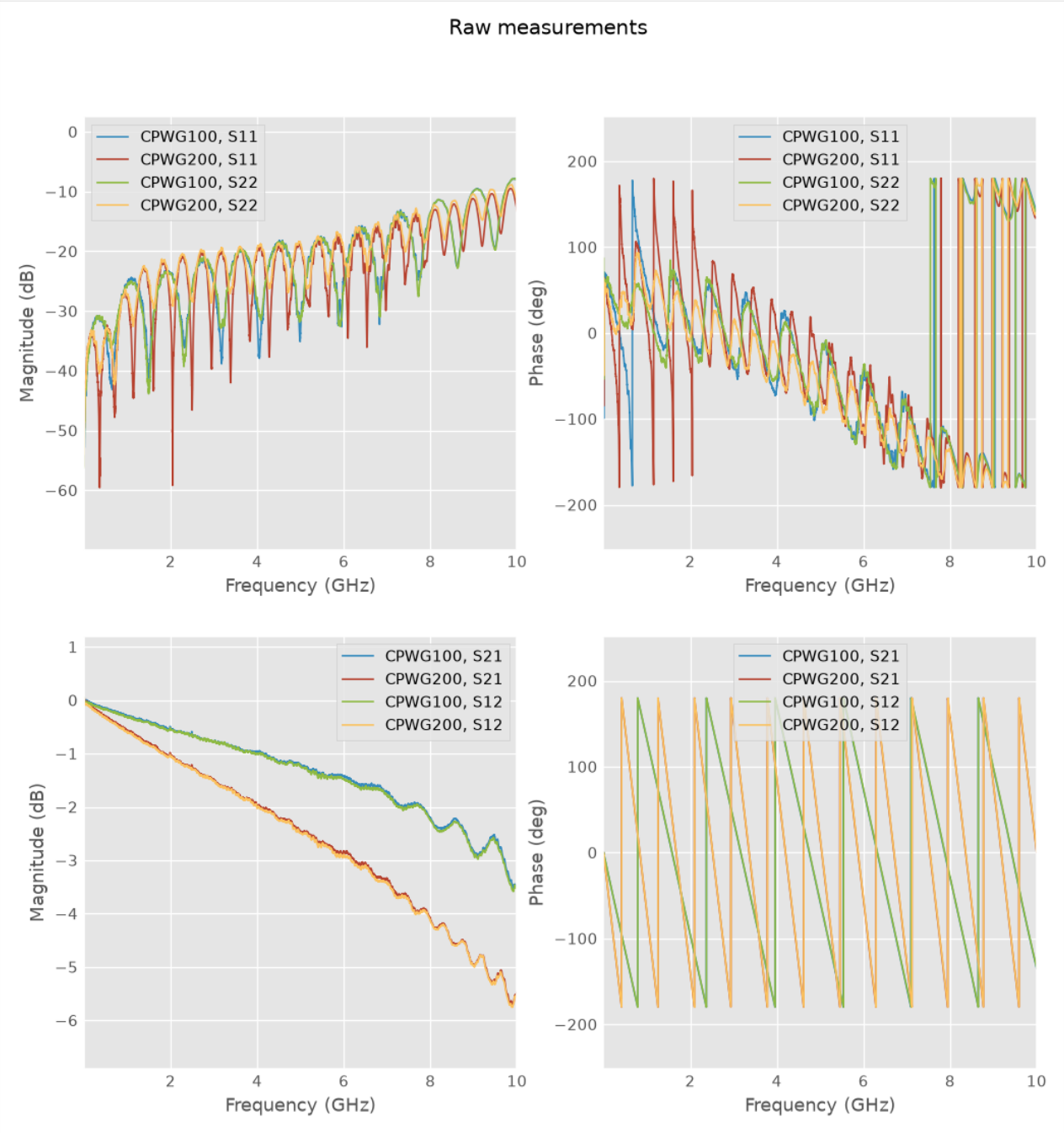
The return loss insertion and phase have the same order of magnitude for the two networks. Insertion loss is below -20 dB at low frequency which indicates a good match of the impedance. The distance between magnitude minima is two times shorter with CPWG200 related to CPWG100, which again agrees with the physical length difference.

[2]:
```
# Load raw measurements
TL100 = rf.Network('CPWG100.s2p')
TL200 = rf.Network('CPWG200.s2p')

# plot them all
plt.figure(figsize=(10, 10))
plt.suptitle('Raw measurements')
plt.subplot(2, 2, 1)
TL100.plot_s_db(0, 0)
```

(continues on next page)

(continued from previous page)

```
TL200.plot_s_db(0, 0)
TL100.plot_s_db(1, 1)
TL200.plot_s_db(1, 1)
plt.subplot(2, 2, 2)
TL100.plot_s_deg(0, 0)
TL200.plot_s_deg(0, 0)
TL100.plot_s_deg(1, 1)
TL200.plot_s_deg(1, 1)
plt.subplot(2, 2, 3)
TL100.plot_s_db(1, 0)
TL200.plot_s_db(1, 0)
TL100.plot_s_db(0, 1)
TL200.plot_s_db(0, 1)
plt.subplot(2, 2, 4)
TL100.plot_s_deg(1, 0)
TL200.plot_s_deg(1, 0)
TL100.plot_s_deg(0, 1)
TL200.plot_s_deg(0, 1)
```



Removing connectors effects

The SMA connectors cause a discontinuity in the signal path. The waves have to go through a coaxial to planar transition with a complex three-dimensional geometry in the launch region.

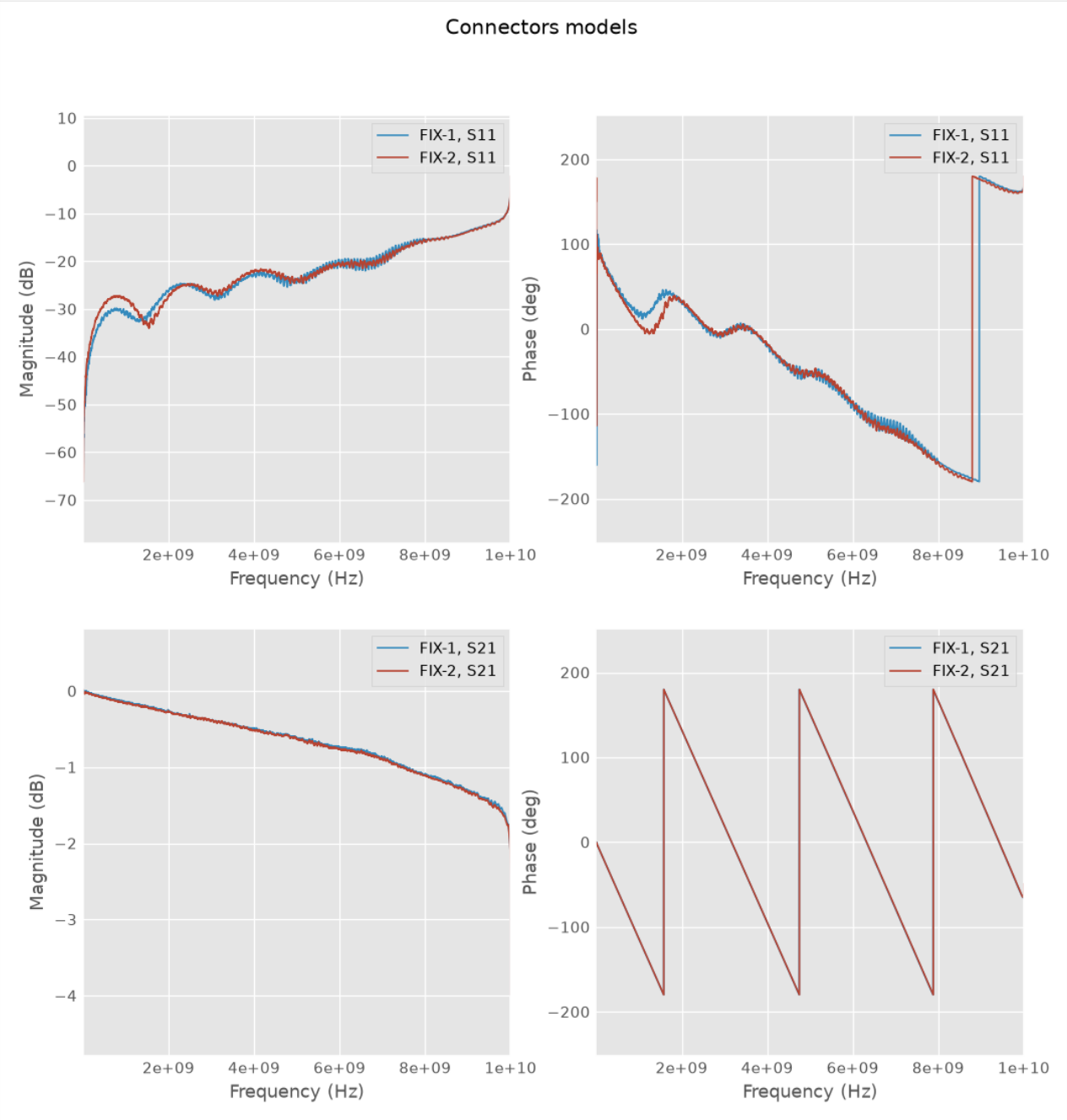
By splitting the 100 mm line into two halves and deembed them from the 200 mm line, the connector and 50 mm of the line will be removed on both sides. This will remove the effects caused by the connectors.

[3]:
```
# deembedding using IEEEP370 impedance corrected 2xthru method
dm = IEEEP370_SE_NZC_2xThru(dummy_2xthru = TL100, name = '2xthru')
```

(continues on next page)

(continued from previous page)

```
fix1 = dm.s_side1
fix1.name = 'FIX-1'
fix2 = dm.s_side2
fix2.name = 'FIX-2'
d_dut = dm.deembed(TL200)
d_dut.name = 'd_DUT'

# plot them all
plt.figure(figsize=(10, 10))
plt.suptitle('Connectors models')
plt.subplot(2, 2, 1)
fix1.plot_s_db(0, 0)
fix2.plot_s_db(0, 0)
plt.subplot(2, 2, 2)
fix1.plot_s_deg(0, 0)
fix2.plot_s_deg(0, 0)
plt.subplot(2, 2, 3)
fix1.plot_s_db(1, 0)
fix2.plot_s_db(1, 0)
plt.subplot(2, 2, 4)
fix1.plot_s_deg(1, 0)
fix2.plot_s_deg(1, 0)
```



Comparison of deembedded dut with CPW media results

We now compare the results of our deembedded measurements with simulated results from CPW media.

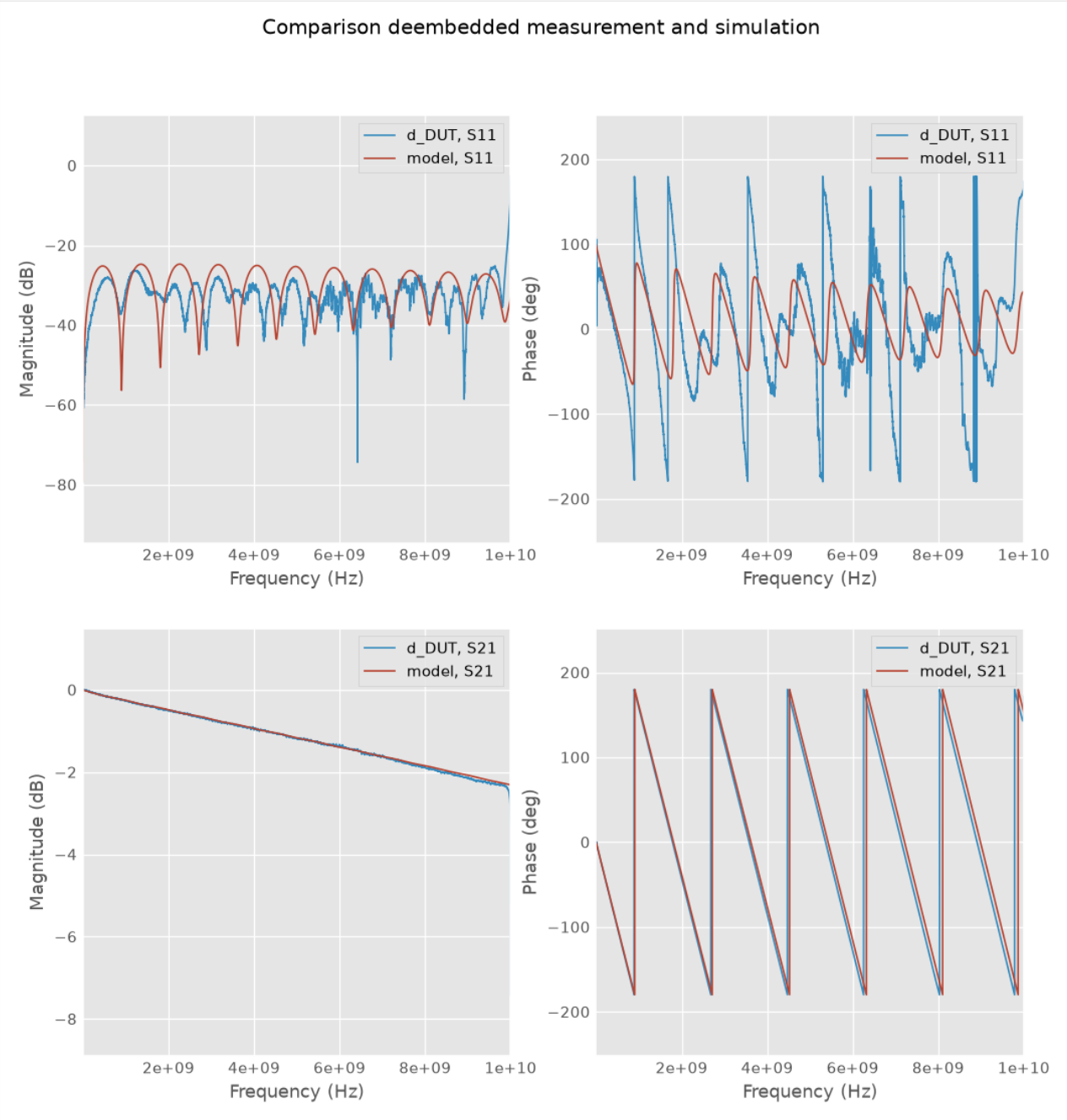
The ep_r and tanD values are taken from the "Correlating microstripline model to measurement" notebook in the same directory.

The insertion loss agreement is very good both in phase and magnitude.

The return loss overall magnitude seems correct and indicates a good match. Return losses below -20 dB are tricky to measure because the signal becomes small. Thus we should not expect to have a good agreement on the shape here.

[4]:
```
ep_r = 4.421
tanD = 0.0167

cpw = CPW(frequency=d_dut.frequency, w = 1.7e-3, s = 0.5e-3, t = 50e-6, h = 1.55e-3,
        ep_r = ep_r, tand = tanD, rho = 1.7e-8, z0_port = 50., has_metal_backside = True)
l = cpw.line(d = 100.0e-3, unit = 'm')
l.name = 'model'

# plot them all
plt.figure(figsize=(10, 10))
plt.suptitle('Comparison deembedded measurement and simulation')
plt.subplot(2,2,1)
d_dut.plot_s_db(0,0)
l.plot_s_db(0,0,)
plt.subplot(2,2,2)
d_dut.plot_s_deg(0,0)
l.plot_s_deg(0,0)
plt.subplot(2,2,3)
d_dut.plot_s_db(1,0)
l.plot_s_db(1,0)
plt.subplot(2,2,4)
d_dut.plot_s_deg(1,0)
l.plot_s_deg(1,0)
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/media/
↪cpw.py:240: RuntimeWarning: Conductor loss calculation invalid for lineheight t (5e-
↪05)  < 3 * skin depth (6.56212641250373e-05)
  self.alpha_conductor, self.alpha_dielectric = self.analyse_loss(
```



Checking deembedding

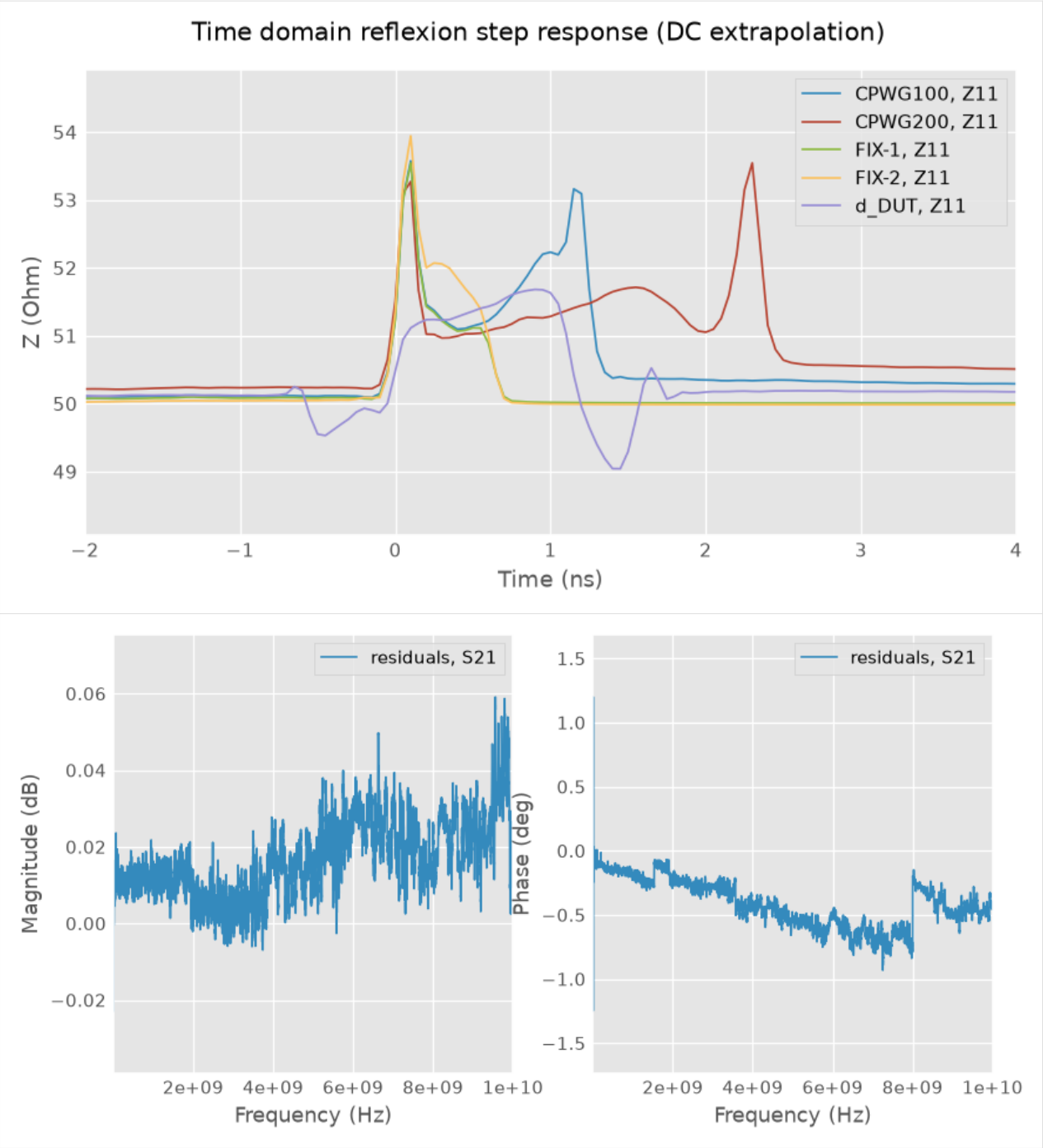
Two basic checks can be done to ensure the consistency of the model of the sides generated by IEEEP370 2xthru algorithm.

1. Plot time step response of 2xthru, side1, and side2. The sides should look like the two halves of 2xthru. This is the case here. Plot time step response of fixture-dut-fixture it should have the same left and right shape than 2xthru. Here we can see a small impedance mismatch which will cause a bounce in dut shape. Plot dut and see if it looks like the middle portion of fixture-dut-fixture. Here everything seems OK outside the small bounce left and right of dut.

2. Looks at the residuals by deembedding two sides models from 2xthru. Residual insertion magnitude shall be ±

0.1 dB and residual insertion loss phase shall be within $\pm$ 1°. This is the case for both here on the full bandwidth.

[5]:
```
# compute residuals
res = dm.deembed(TL100)
res.name = 'residuals'
res.s += 1e-15 # avoid numeric singularities

# extrapolate to dc for time step
TL100_dc = TL100.extrapolate_to_dc(kind='linear')
TL200_dc = TL200.extrapolate_to_dc(kind='linear')
fix1_dc = fix1.extrapolate_to_dc(kind='cubic')
fix2_dc = fix2.extrapolate_to_dc(kind='cubic')
d_dut_dc = d_dut.extrapolate_to_dc(kind='cubic')

# plot them all
# time domain
plt.figure(figsize=(8, 4))
plt.suptitle('Time domain reflexion step response (DC extrapolation)')
TL100_dc.plot_z_time_step(0, 0)
TL200_dc.plot_z_time_step(0, 0)
fix1_dc.plot_z_time_step(0, 0)
fix2_dc.plot_z_time_step(0, 0)
d_dut_dc.plot_z_time_step(0, 0)
plt.xlim(-2, 4)

# residuals frequency domain
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
res.plot_s_db(1,0)
plt.subplot(1, 2, 2)
res.plot_s_deg(1,0)
```



[ ]:

2.1.4 Circular waveguide attenuation

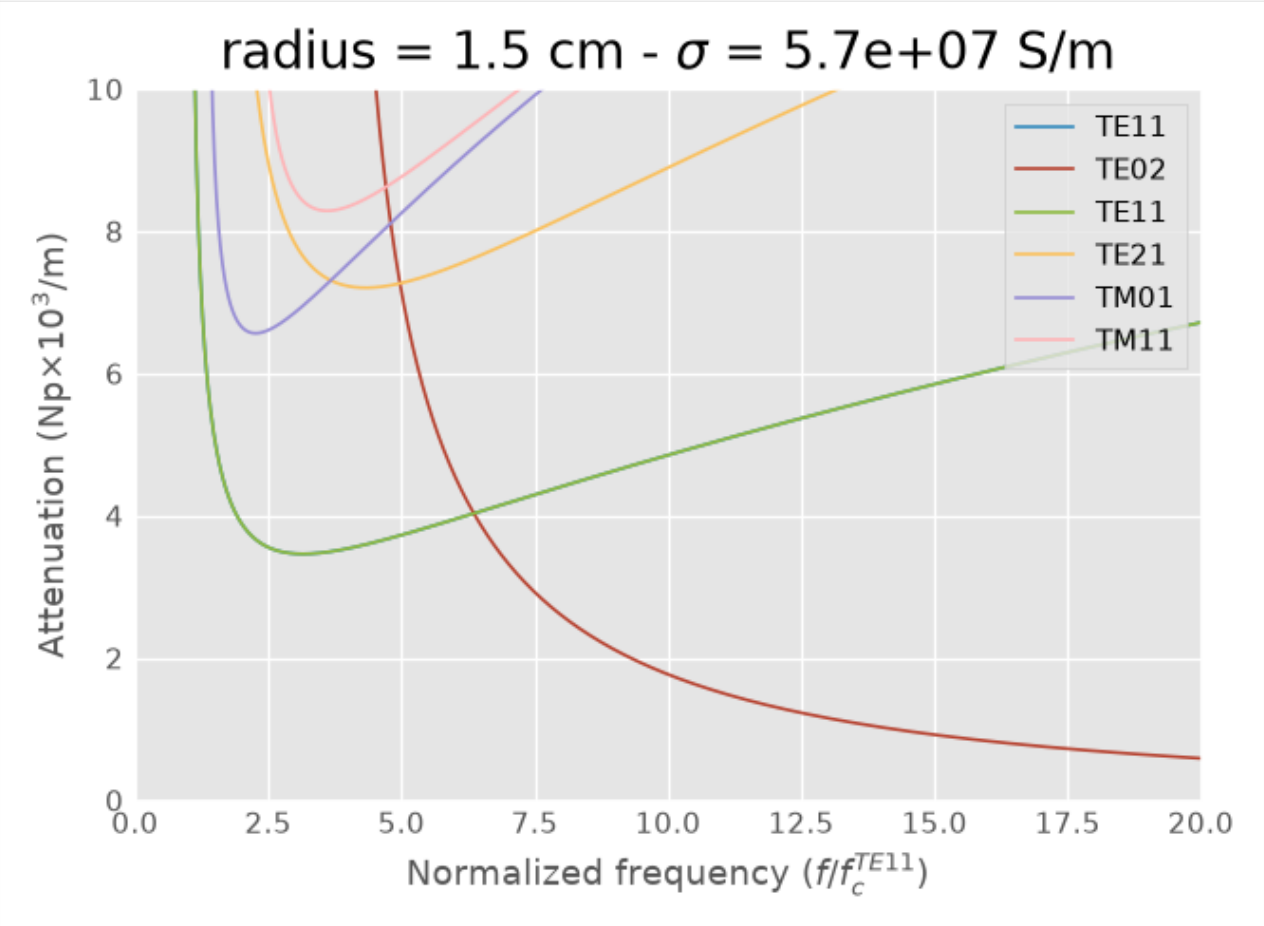
The aim of this example is to reproduce the attenuation coefficients as presented in Balanis' Advanced Engineering Electromagnetics – Third edition, Constantine A. Balanis, Wiley, 2024.

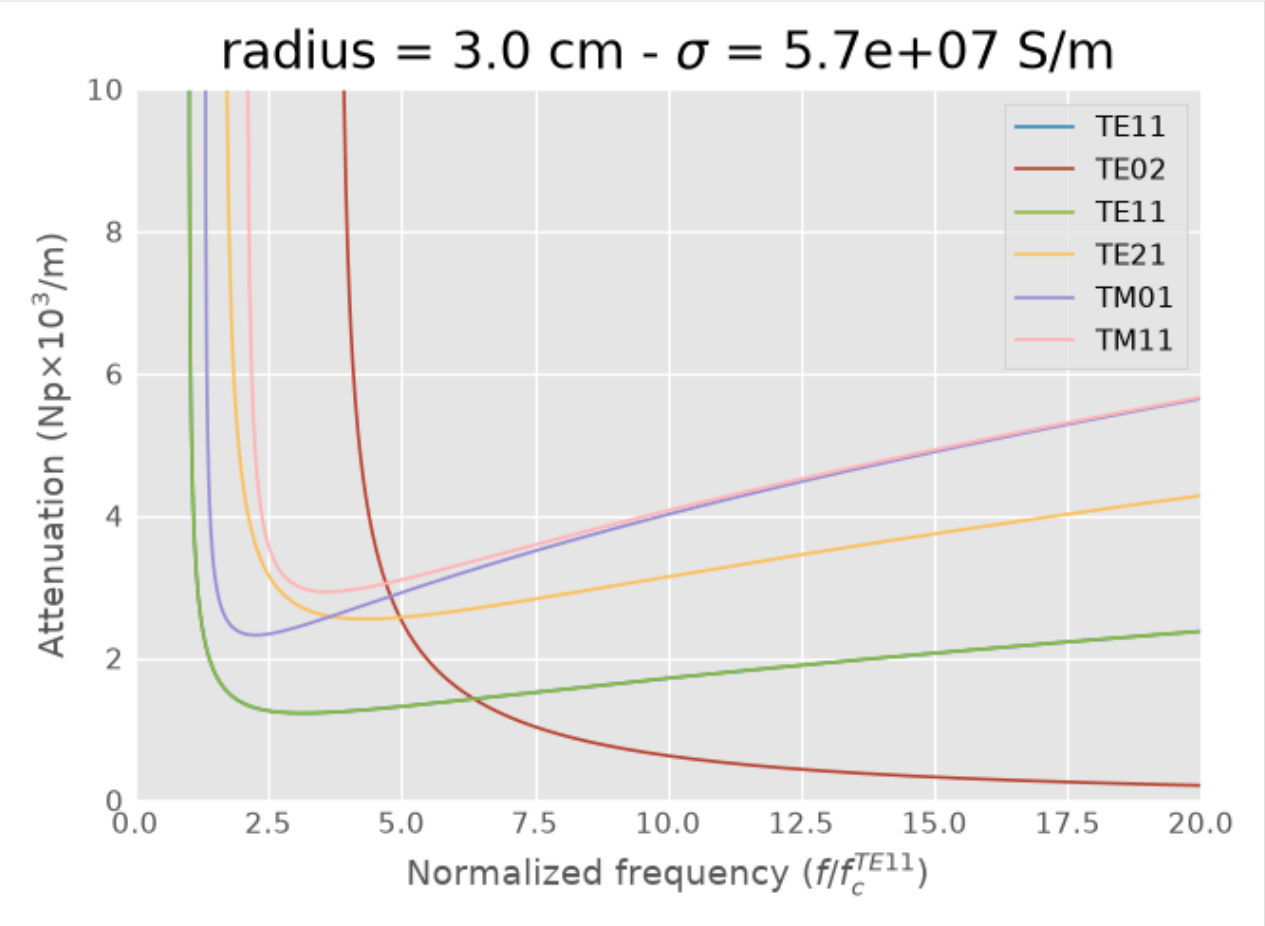
The 2 figures are shown on page 495.

[1]:
```
%matplotlib inline
import matplotlib.pyplot as plt

import skrf as rf
from skrf.frequency import Frequency
from skrf.media import CircularWaveguide

rf.stylely()
# plot formatting
plt.rcParams['lines.linewidth'] = 1
```

[2]:
```
sigma = 5.7e7
radius_list = [1.5e-2, 3e-2]
mode_list = [
    ['TE', 1, 1],
    ['TE', 0, 2],
    ['TE', 1, 1],
    ['TE', 2, 1],
    ['TM', 0, 1],
    ['TM', 1, 1]
]
```

[3]:
```
for radius in radius_list:
    fig, ax = plt.subplots()
    wg = CircularWaveguide(r=radius, m=1, n=1)
    fcTE11 = wg.f_cutoff
    fmax = fcTE11 * 20
    for mode_type, m, n in mode_list:
        wg_plot = CircularWaveguide(r=radius, m=m, n=n, mode_type=mode_type, rho=1/sigma)
        wg_plot.frequency = Frequency(wg_plot.f_cutoff+1, fmax, 1001, 'Hz')
        label = "%s%d%d" % (mode_type, m, n)
        plt.plot(wg_plot.frequency.f/fcTE11, (wg_plot.alpha)*1e3, label=label)

    ax.set_ylim(0, 10)
    ax.set_xlim(0, 20)
    ax.legend(loc='upper right')
    ax.set_xlabel('Normalized frequency ($f/f_c^{TE11}$)')
    ax.set_ylabel('Attenuation (Np×$10^3$/m)')
    ax.set_title(r'radius = %.1f cm - $\sigma$ = %g S/m' % (radius*1e2, sigma))
```


radius = 1.5 cm - σ = 5.7e+07 S/m
Attenuation (Np×10^3/m)
Normalized frequency (f/f_c^TE11)
TE11
TE02
TE11
TE21
TM01
TM11
0
2
4
6
8
10
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0



2.1.5 Correlating microstripline model to measurement

Target

The aim of this example is to correlate the microstripline model to the measurement over 4 frequency decades from 1MHz to 5GHz.

Plan

1. Two different lengths of microstripline are measured;
2. Multiline method is used to compute the frequency dependent relative permittivity and loss angle of the dielectric;
3. Microstripline model is fitted to the computed parameters by optimization;
4. Checking the results by embedding the connectors and comparison against measurement;

[1]:
```
%load_ext autoreload
%autoreload 2
import matplotlib.pyplot as plt
import numpy as np
from numpy import absolute, log10, real, sum
from scipy.optimize import minimize

import skrf as rf
```

(continues on next page)

(continued from previous page)

```
rf.stylely()
```

Measurement of two microstripline with different lengths

The measurement where performed the 21th March 2017 on a Anritsu MS46524B 20GHz Vector Network Analyser. The setup is a linear frequency sweep from 1MHz to 10GHz with 10'000 points. Output power is 0dBm, IF bandwidth is 1kHz and neither averaging nor smoothing are used.

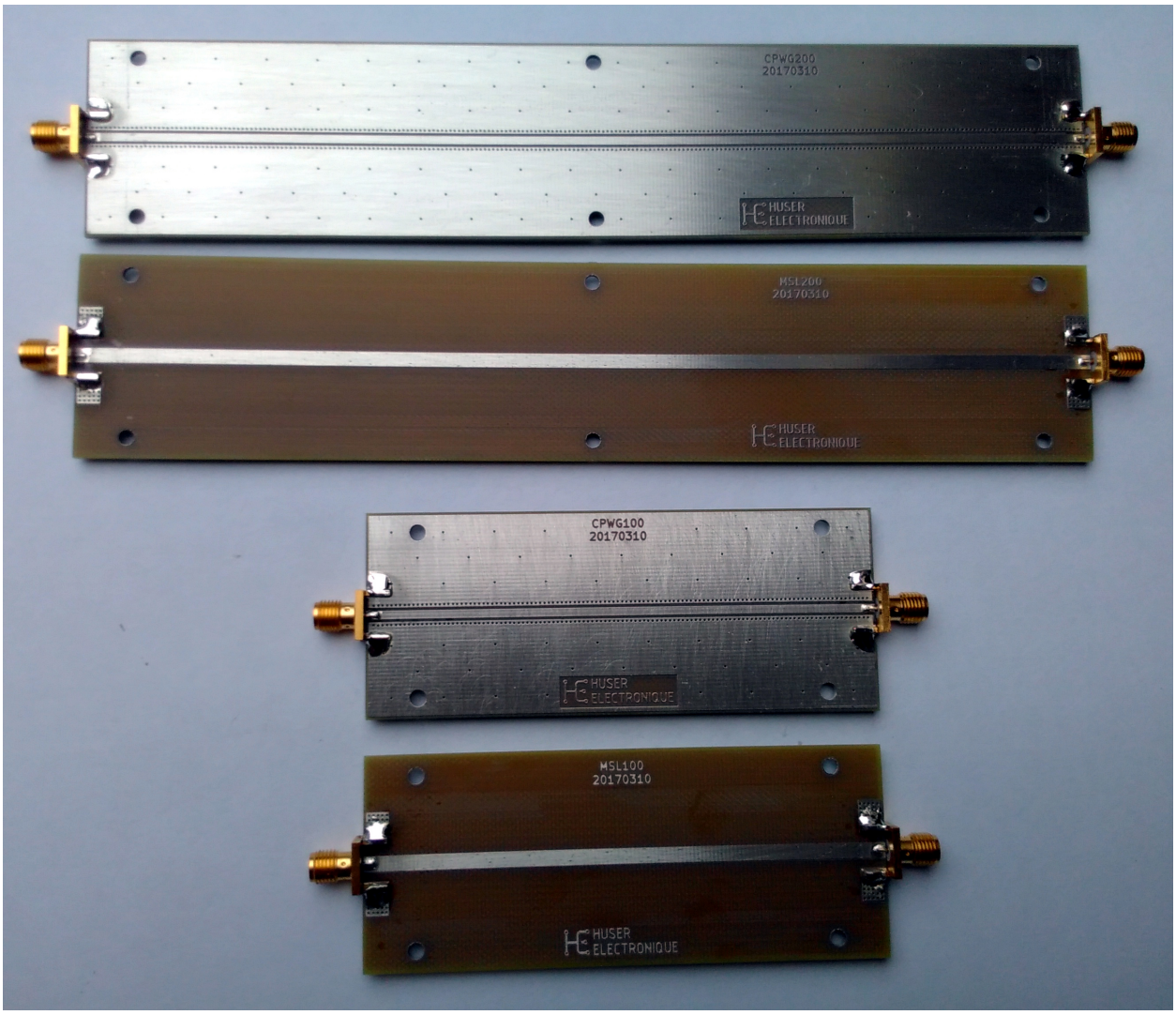
The frequency range of interest is limited from 1MHz to 5GHz, but the measurement are up to 10GHz.

MSLxxx is a L long, W wide, T thick copper microstripline on a H height substrate with bottom ground plane.

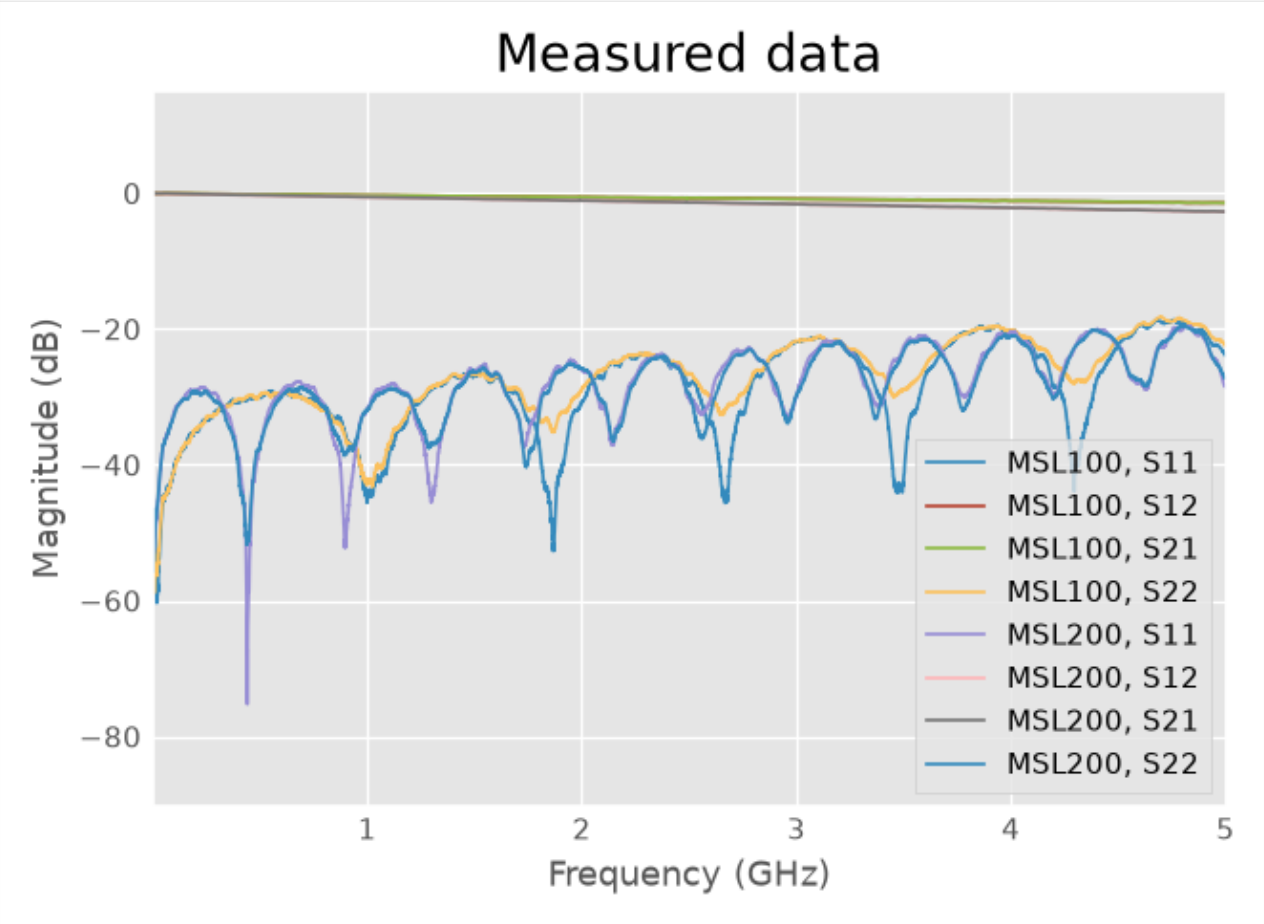
Name	L (mm)	W (mm)	H (mm)	T (um)	Substrate
MSL100	100	3.00	1.55	50	FR-4
MSL200	200	3.00	1.55	50	FR-4

The milling of the artwork is performed mechanically with a lateral wall of 45°. A small top ground plane chunk connected by a vias array to bottom ground is provided to solder the connector top ground legs and provide some coplanar-like transition from coax to microstrip.

The relative permittivity of the dielectric was assumed to be approximately 4.5 for design purpose.



[2]:
```
# Load raw measurements
MSL100_raw = rf.Network('MSL100.s2p')
MSL200_raw = rf.Network('MSL200.s2p')

# Keep only the data from 1MHz to 5GHz
MSL100 = MSL100_raw['1-5000mhz']
MSL200 = MSL200_raw['1-5000mhz']

plt.figure()
plt.title('Measured data')
MSL100.plot_s_db()
MSL200.plot_s_db()
plt.show()
```



The measured data shows that the electrical length of MSL200 is approximately twice the one of MSL100. The frequency spacing between Return Loss dips is approximately the half for MSL200 compared to MSL100. This is coherent with the physical dimensions if the small connector length is neglected.

The MSL200 Insertion Loss is also about twice than MSL100, which is coherent as a longer path bring more attenuation.

Return Loss under -20dB is usually considered to be fair for microstripline, it correspond to 1% of the power being reflected.

Dielectric effective relative permittivity extraction by multiline method

The phase of the measurements transmission parameter are subtracted. Because connectors are present on both DUTs, their length effect is canceled and the remaining phase difference is related to the difference of the DUTs length.

Knowing the physical length ΔL and the phase $\Delta\phi$, the effective relative permittivity constant $\epsilon_{r,eff}$ can be computed from the relation

$$\begin{cases} \lambda = \frac{c_0}{f \cdot \sqrt{\epsilon_{r,eff}}} \\ \phi = \frac{2\pi L}{\lambda} \end{cases} \implies \epsilon_{r,eff} = \left(\frac{\Delta\phi \cdot c_0}{2\pi f \cdot \Delta L} \right)^2$$

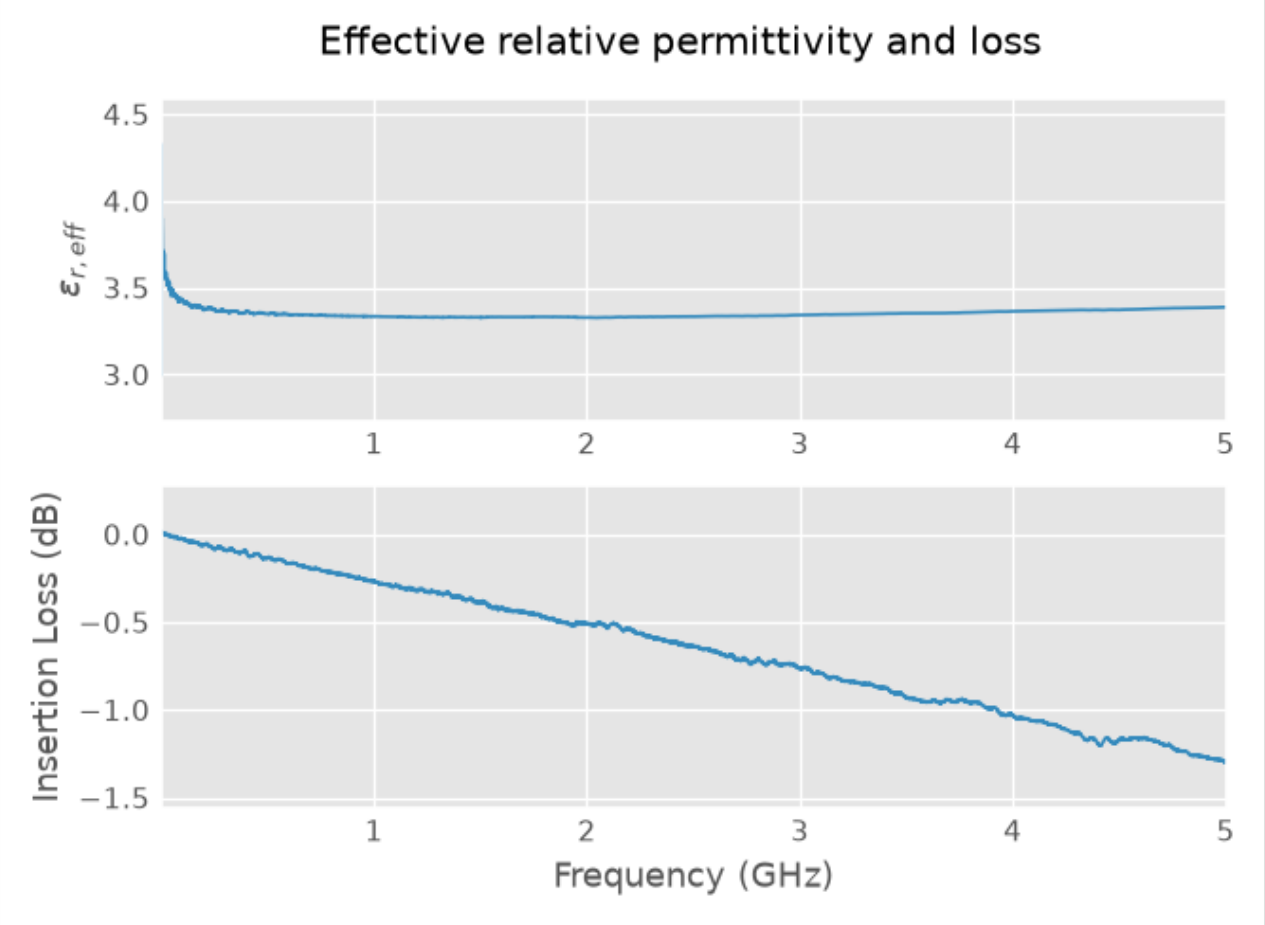
In the same idea, the difference of Insertion Loss of the two DUT gives the Insertion Loss of the difference of the length and cancel connectors effects.

[3]:
```
c0      = 3e8
f       = MSL100.f
deltaL  = 0.1
```

(continues on next page)

(continued from previous page)

```
deltaPhi = np.unwrap(np.angle(MSL100.s[:,1,0])) - np.unwrap(np.angle(MSL200.s[:,1,0]))
Er_eff   = np.power(deltaPhi * c0 / (2 * np.pi * f * deltaL), 2)
Loss_mea = 20 * log10(absolute(MSL200.s[:,1,0] / MSL100.s[:,1,0]))

plt.figure()
plt.suptitle('Effective relative permittivity and loss')
plt.subplot(2,1,1)
plt.plot(f * 1e-9, Er_eff)
plt.ylabel(r'$\epsilon_{r,eff}$')

plt.subplot(2,1,2)
plt.plot(f * 1e-9, Loss_mea)
plt.xlabel('Frequency (GHz)')
plt.ylabel('Insertion Loss (dB)')
plt.show()
```



The effective relative permittivity of the geometry shows a dispersion effect at low frequency which can be modelled by a wideband Debye model such as *Djordjevic/Svensson* implementation of skrf microstripline media. The value then increase slowly with frequency which correspond roughly to the *Kirschning and Jansen* dispersion model.

The Insertion Loss seems proportional to frequency, which indicate a predominance of the dielectric losses. Conductor losses are related to the square-root of frequency. Radiation losses are neglected.

Fit microstripline model to the computed parameters by optimization

Effective relative permittivity

Microstrip media model with the physical dimensions of the measured microstriplines is fitted to the computed $\epsilon_{r,eff}$ by optimization of ϵ_r and tand of the substrate at 1GHz. The dispersion model used to account for frequency variation of the parameters are *Djordjevic/Svensson* and *Kirschning and Jansen*.

[4]:
```
from skrf.media import MLine

W   = 3.00e-3
H   = 1.55e-3
T   = 50e-6
L   = 0.1
Er0   = 4.5
tand0 = 0.02
f_epr_tand = 1e9
x0 = [Er0, tand0]

def model(x, freq, Er_eff, L, W, H, T, f_epr_tand, Loss_mea):
    ep_r = x[0]
    tand = x[1]
    m = MLine(frequency=freq, z0_port=50, w=W, h=H, t=T,
        ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tand, rough=0.15e-6,
        f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
        diel='djordjevicsvensson', disp='kirschningjansen')
    DUT  = m.line(L, 'm')
    Loss_mod = 20 * log10(absolute(DUT.s[:,1,0]))
    return sum((real(m.ep_reff_f) - Er_eff)**2) + 0.01*sum((Loss_mod - Loss_mea)**2)

res = minimize(model, x0, args=(MSL100.frequency, Er_eff, L, W, H, T, f_epr_tand, Loss_
↪mea),
               bounds=[(4.2, 4.7), (0.001, 0.1)])
Er   = res.x[0]
tand = res.x[1]

print(f'Er={Er:.3f}, tand={tand:.4f} at {f_epr_tand * 1e-9:.1f} GHz.')
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/media/
↪mline.py:276: RuntimeWarning: Conductor loss calculation invalid for lineheight t (5e-
↪05)  < 3 * skin depth (6.585246130908566e-05)
  self.alpha_conductor, self.alpha_dielectric = self.analyse_loss(
```

```
Er=4.421, tand=0.0167 at 1.0 GHz.
```

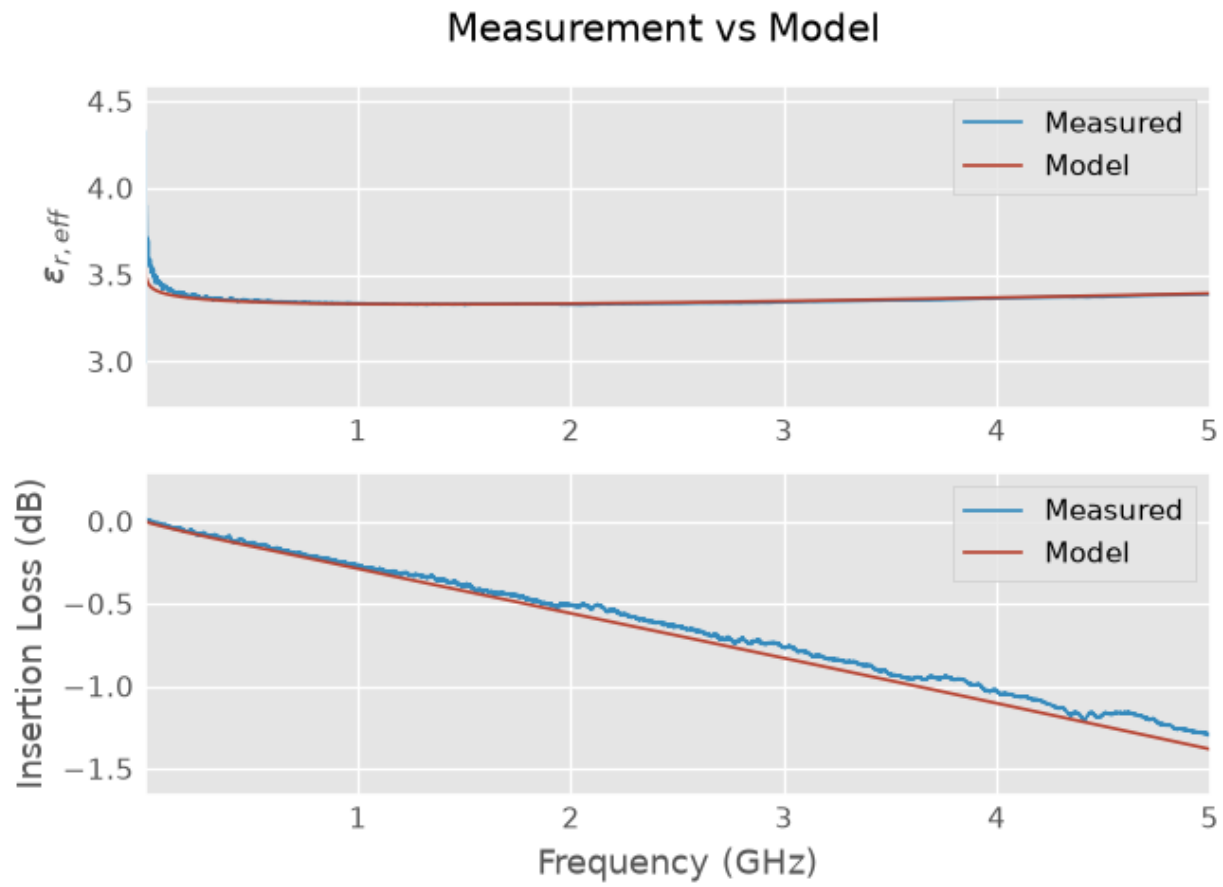
As a sanity check, the model data are compared with the computed parameters

[5]:
```
m = MLine(frequency=MSL100.frequency, z0_port=50, w=W, h=H, t=T,
        ep_r=Er, mu_r=1, rho=1.712e-8, tand=tand, rough=0.15e-6,
        f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
        diel='djordjevicsvensson', disp='kirschningjansen')
DUT  = m.line(L, 'm')
DUT.name = 'DUT'
Loss_mod = 20 * log10(absolute(DUT.s[:,1,0]))
```

(continues on next page)

(continued from previous page)

```
plt.figure()
plt.suptitle('Measurement vs Model')
plt.subplot(2,1,1)
plt.plot(f * 1e-9, Er_eff, label='Measured')
plt.plot(f * 1e-9, real(m.ep_reff_f), label='Model')
plt.ylabel(r'$\epsilon_{r,eff}$')
plt.legend()

plt.subplot(2,1,2)
plt.plot(f * 1e-9, Loss_mea, label='Measured')
plt.plot(f * 1e-9, Loss_mod, label='Model')
plt.xlabel('Frequency (GHz)')
plt.ylabel('Insertion Loss (dB)')
plt.legend()
plt.show()
```

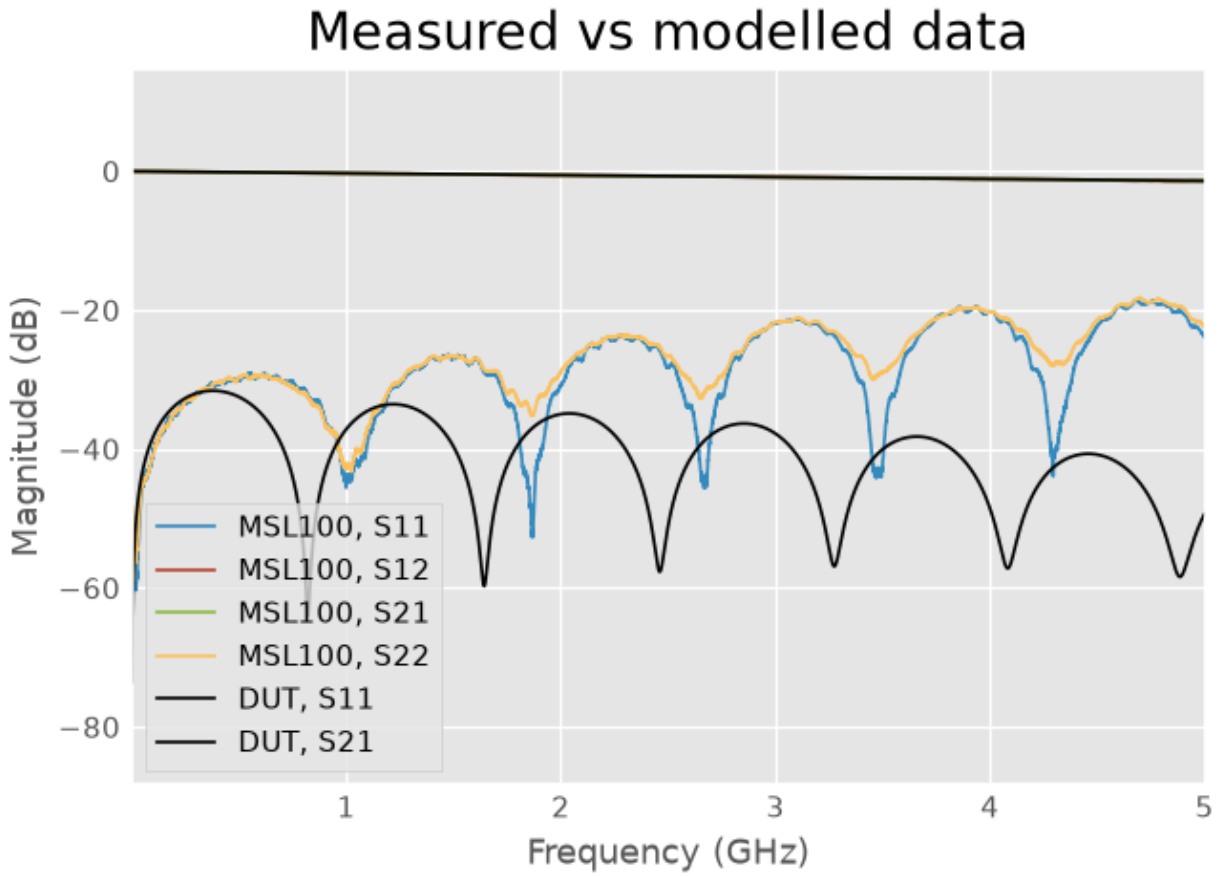


The model results shows a reasonable agreement with the measured $\epsilon_{r,eff}$ and Insertion Loss values.

Checking the results

If the model is now plotted against the measurement of the same length, the plot shows no agreement. This is because the connector effects are not captured by the model.

[6]:
```
plt.figure()
plt.title('Measured vs modelled data')
MSL100.plot_s_db()
DUT.plot_s_db(0, 0, color='k')
DUT.plot_s_db(1, 0, color='k')
plt.show()
```



Connector delay and loss estimation

The delay of the connector is estimated by fitting a line to its phase contribution vs frequency.

The phase and loss of the two connector are computed by subtracting phase and loss computed without the connectors to the measurement of the same length.

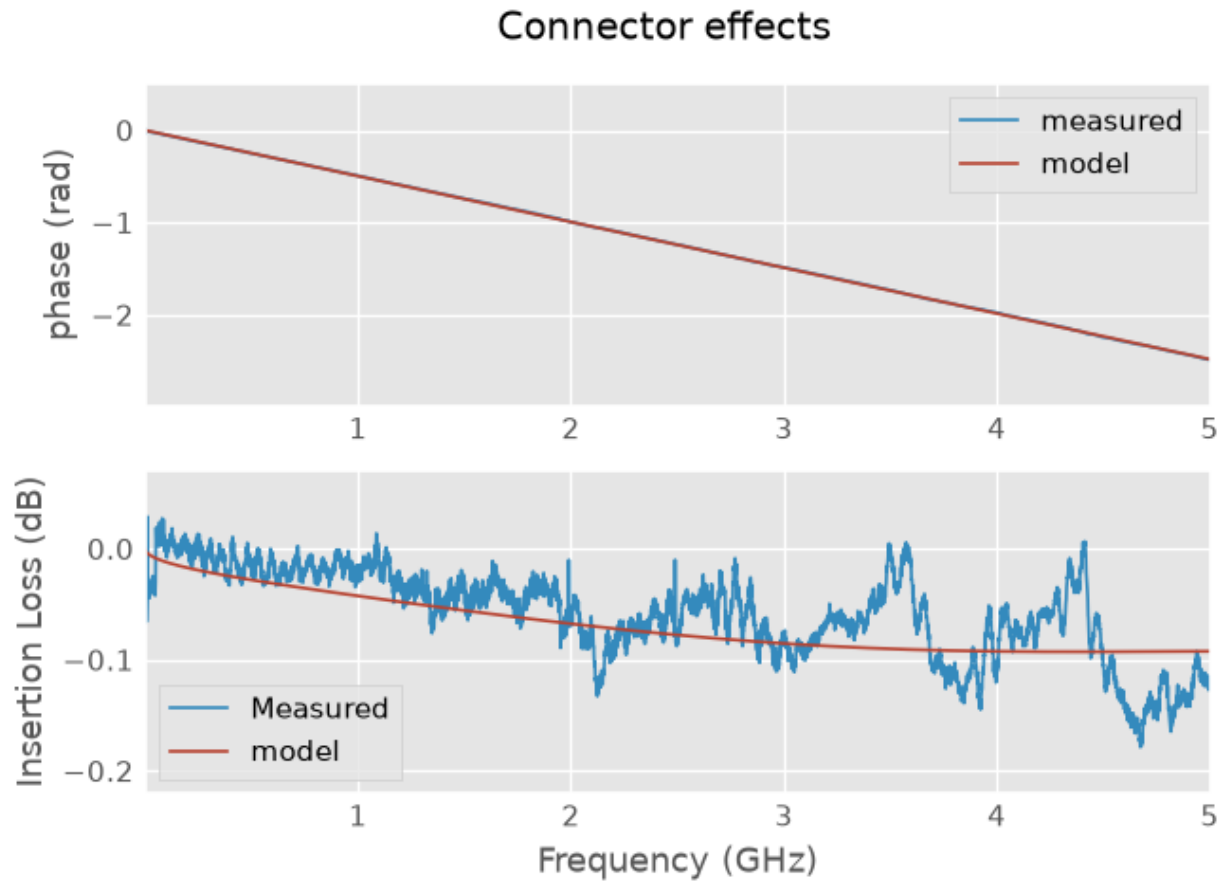
[7]:
```
phi_conn = np.unwrap(np.angle(MSL100.s[:,1,0])) + deltaPhi
z = np.polyfit(f, phi_conn, 1)
p = np.poly1d(z)
delay = -z[0]/(2*np.pi)/2
print(f'Connector delay: {delay * 1e12:.0f} ps')

loss_conn_db = 20 * log10(absolute(MSL100.s[:,1,0])) - Loss_mea
alpha = 1.6*np.log(10)/20 * np.sqrt(f/1e9)
beta  = 2*np.pi*f/c0
```

(continues on next page)

(continued from previous page)

```
gamma = alpha + 1j*beta
mf = rf.media.DefinedGammaZ0(m.frequency, z0_port=50, z0=53.5, gamma=gamma)
left = mf.line(delay*1e9, 'ns')
right = left.flipped()
check = left ** right

plt.figure()
plt.suptitle('Connector effects')
plt.subplot(2,1,1)
plt.plot(f * 1e-9, phi_conn, label='measured')
plt.plot(f * 1e-9, np.unwrap(np.angle(check.s[:,1,0])), label='model')
plt.ylabel('phase (rad)')
plt.legend()

plt.subplot(2,1,2)
plt.plot(f * 1e-9, loss_conn_db, label='Measured')
plt.plot(f * 1e-9, 20*np.log10(np.absolute(check.s[:,1,0])), label='model')
plt.xlabel('Frequency (GHz)')
plt.ylabel('Insertion Loss (dB)')
plt.legend()
plt.show()
```

```
Connector delay: 39 ps
```



The phase of the model shows a good agreement, while the Insertion Loss seems to have a reasonable agreement and is small whatsoever.

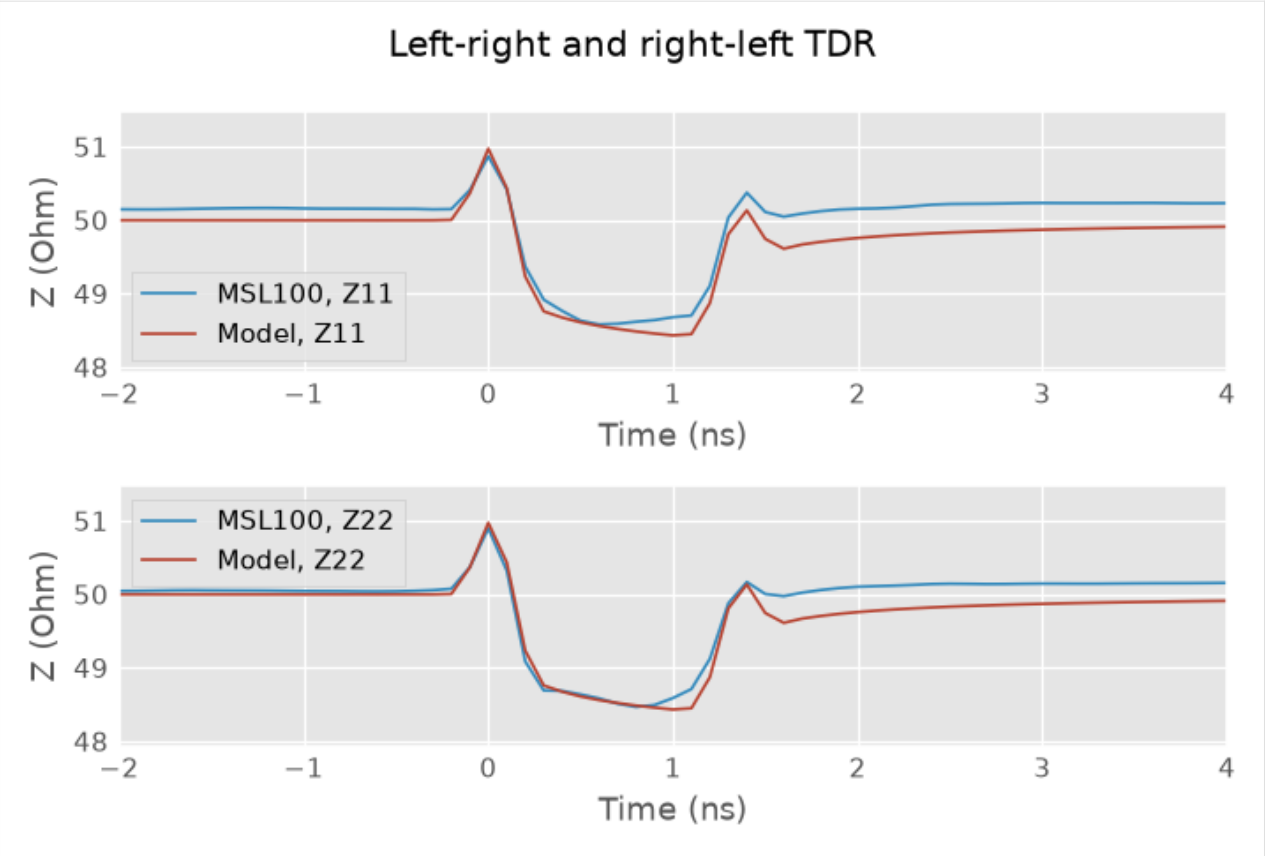
Connector impedance adjustment by time-domain reflectometry

Time-domain step responses of measurement and model are used to adjust the connector model characteristic impedance.

The plots shows the connector having an inductive behaviour (positive peak) and the microstripline being a bit too much capacitive (negative plateau).

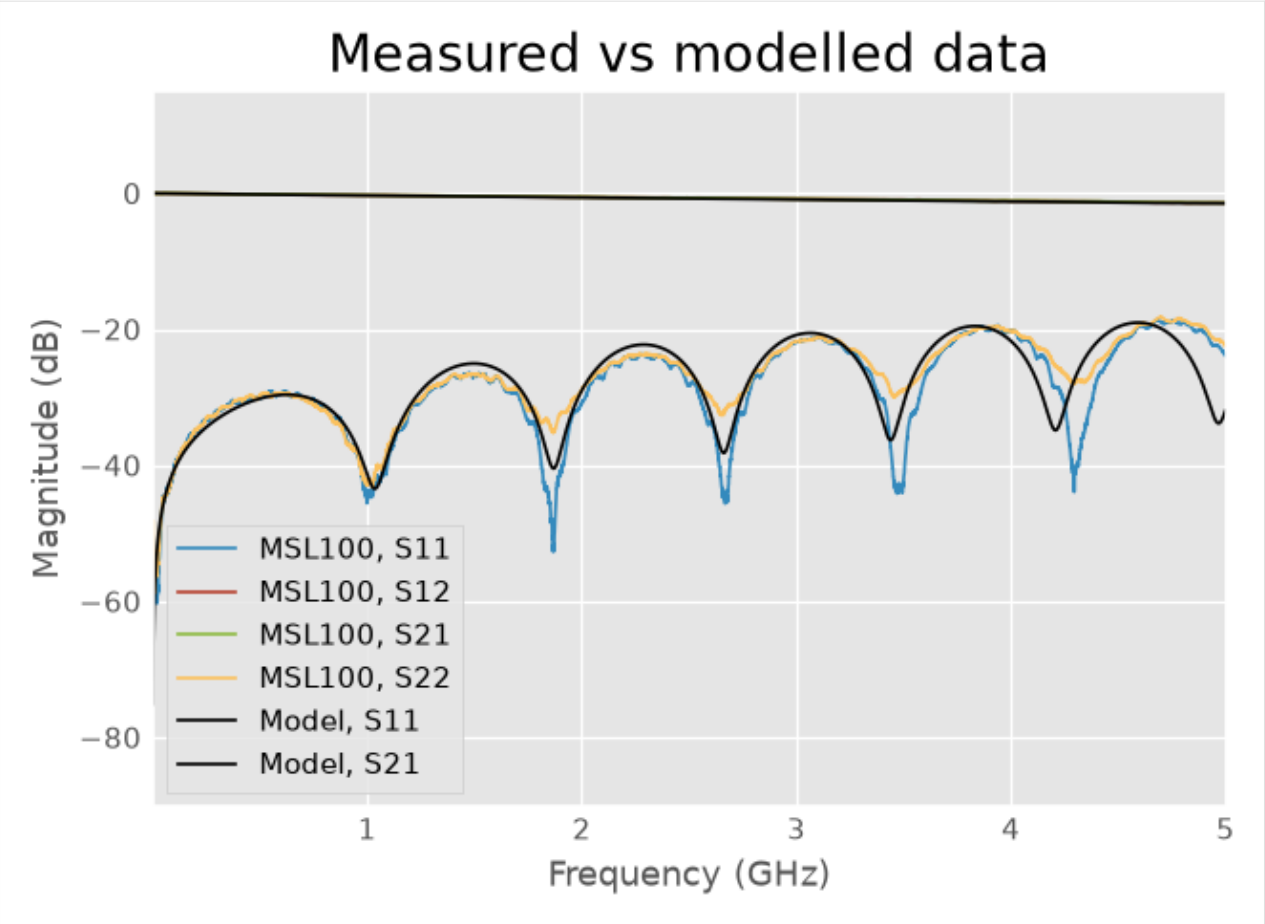
Characteristic impedance of the connector is tuned by trial-and-error until a reasonable agreement is achieved. Optimization could have been used instead.

[8]:
```
mod = left ** DUT ** right
mod.name = 'Model'

MSL100_dc = MSL100.extrapolate_to_dc(kind='cubic')
DUT_dc = mod.extrapolate_to_dc(kind='cubic')

plt.figure()
plt.suptitle('Left-right and right-left TDR')
plt.subplot(2,1,1)
MSL100_dc.plot_z_time_step(0, 0)
DUT_dc.plot_z_time_step(0, 0)
plt.xlim(-2, 4)

plt.subplot(2,1,2)
MSL100_dc.plot_z_time_step(1, 1)
DUT_dc.plot_z_time_step(1, 1)
plt.xlim(-2, 4)

plt.tight_layout()
plt.show()
```



Final comparison

[9]:
```
plt.figure()
plt.title('Measured vs modelled data')
MSL100.plot_s_db()
mod.name = 'Model'
mod.plot_s_db(0, 0, color='k')
mod.plot_s_db(1, 0, color='k')

plt.show()
```



The plot shows a decent agreement between the model and the measured data. The model is a good representation of the DUT between 1MHz and 5 GHz.

At higher frequency, the model begin to deviate from the measurement. The model does not capture effects such as radiation loss or complex copper roughness. Smaller geometries such as the top ground plane chunk may also begin to contribute as they become electrically long with the increase of frequency.

As a comparison, the 5GHz wavelength is 60mm in the air and the MSL100 line is 100mm long. The DUT itself is electrically long above some GHz.

[ ]:

2.1.6 DC Extrapolation for Time Domain

When converting S-parameters to time domain the frequency points should be equally spaced and start from 0 Hz. Usually VNA doesn't measure down to DC and DC point should be added afterwards.

When a DC point measurement is added it might be the case that frequency point spacing between the DC and the first measured point is not equal to the spacing between the other measured points. Since time domain conversion relies on Fourier transform that assumes regularly spaced points, the measurements should also be resampled to be equally spaced.

[1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
```

(continues on next page)

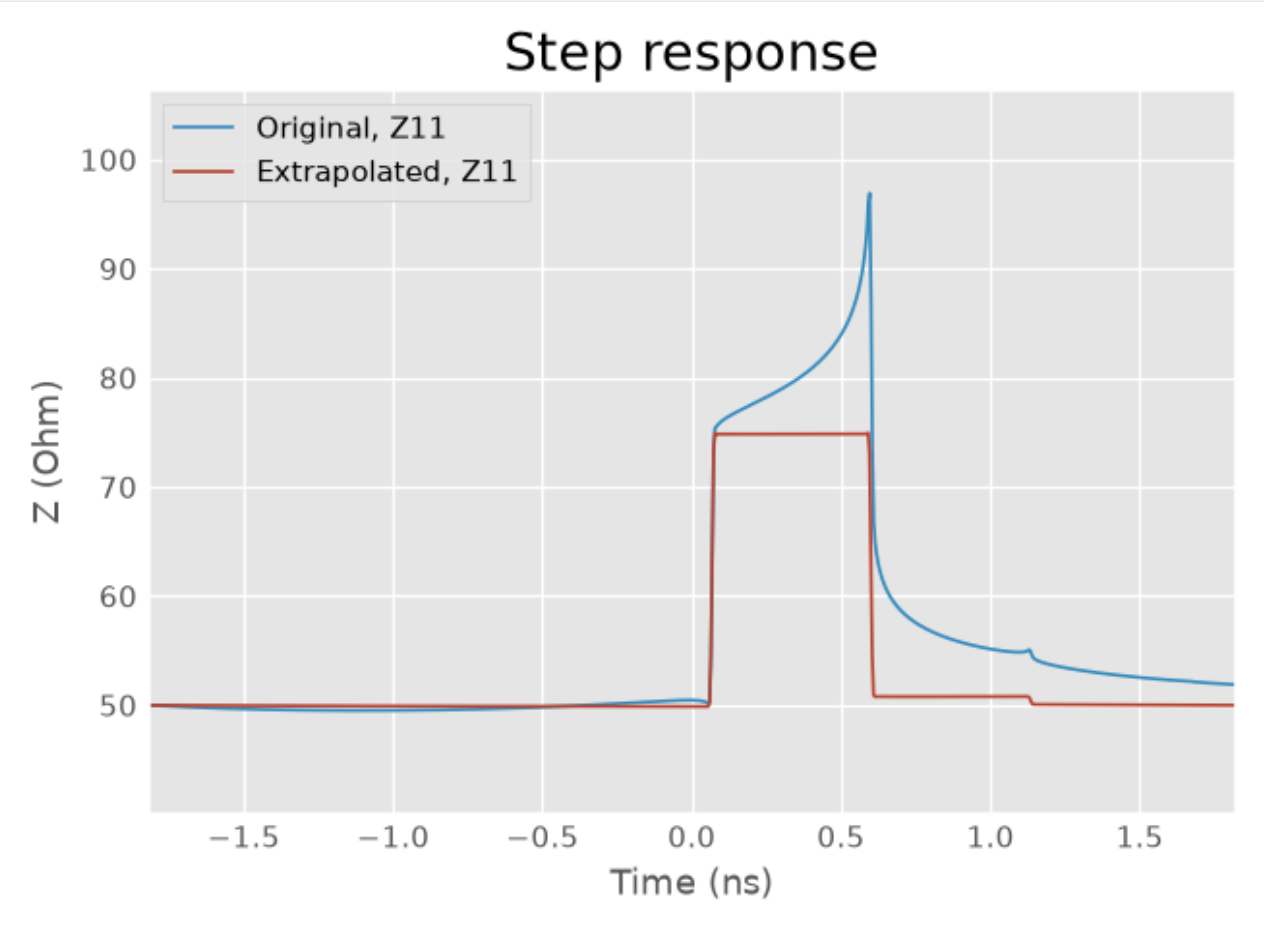
(continued from previous page)

```
import skrf as rf
from skrf.media import Coaxial
from skrf.plotting import stylely

stylely()
```

[2]:
```
freq = rf.Frequency(0.11, 110, 401, 'GHz')
coax1mm = Coaxial(freq, Dint=0.44e-3, Dout=1.0e-3, sigma=1e20, z0_port = 50)

X = coax1mm.line(10, unit='mm', z0=50, name='X')
Y = coax1mm.line(80, unit='mm', z0=75, name='Y')
dut = X**Y**X
dut.name = 'Original'

dut_dc = dut.extrapolate_to_dc(dc_sparam=[[0,1],[1,0]])
dut_dc.name = 'Extrapolated'
```

[3]:
```
plt.figure()
plt.title('Step response')
dut.plot_z_time_step(0, 0)
dut_dc.plot_z_time_step(0, 0)
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/network.
↪py:5251: RuntimeWarning: Frequency doesn't begin from 0. Step response will not be␣
↪correct.
  x, y = self.step_response(**t_func_kwargs)
```



Interpolation method comparison

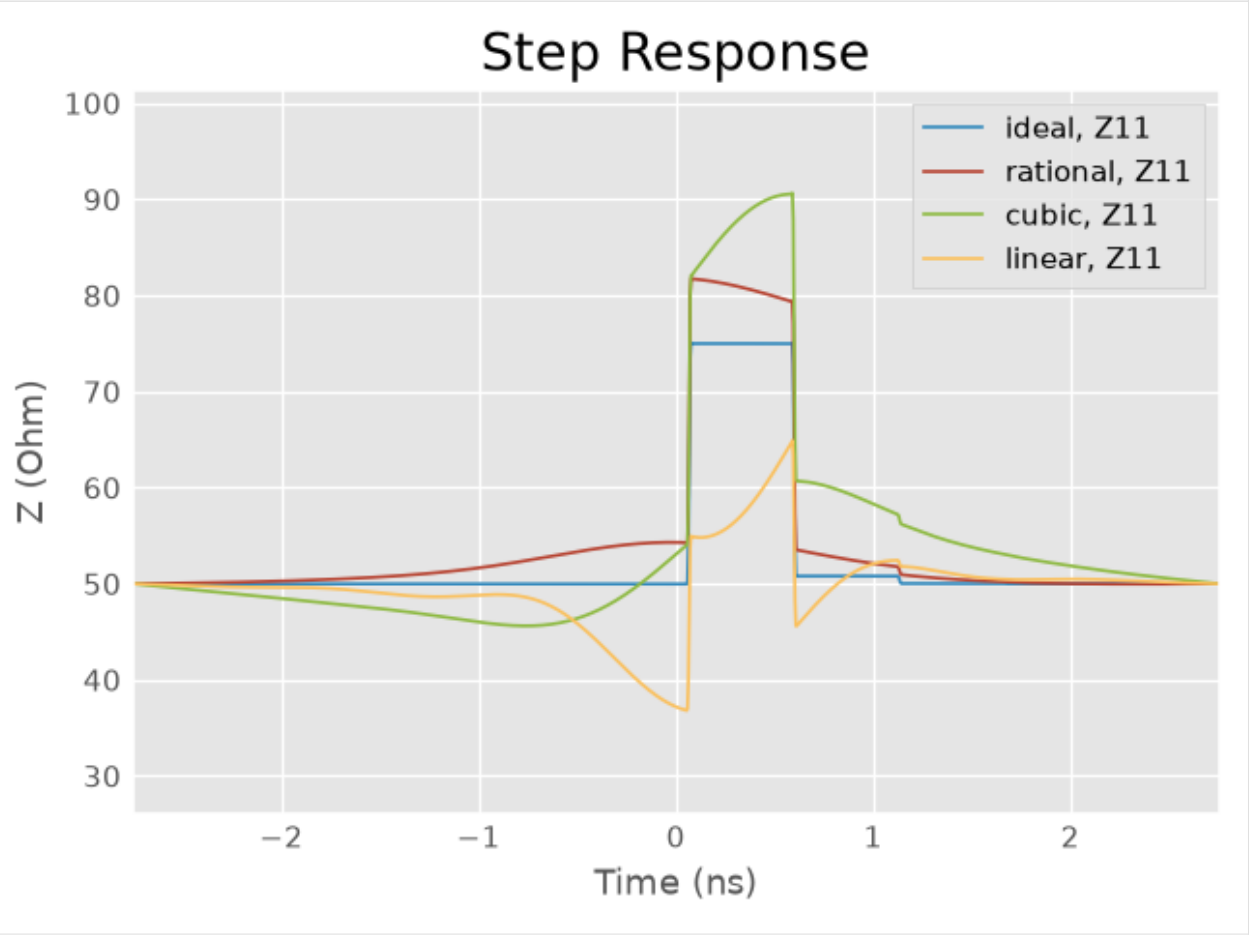
[4]:
```
# Frequency points for measurements to be interpolated
# Frequencies from 0 to 1 GHz are missing
freq = rf.Frequency(1, 110, 601, 'GHz')
# Frequency for ideal
freq2 = rf.Frequency(0, 110, 602, 'GHz')
coax1mm = Coaxial(freq, z0_port=50, Dint=0.44e-3, Dout=1.0e-3, sigma=1e20)
coax1mm2 = Coaxial(freq2, z0_port=50, Dint=0.44e-3, Dout=1.0e-3, sigma=1e20)

# Generate the DUT
X = coax1mm.line(10, 'mm', z0=50, name='X')
Y = coax1mm.line(80, 'mm', z0=75, name='Y')
dut = X**Y**X

# DUT with full frequencies for comparison
X2 = coax1mm2.line(10, 'mm', z0=50, name='X')
Y2 = coax1mm2.line(80, 'mm', z0=75, name='Y')
dut_ideal = X2**Y2**X2
dut_ideal.name = 'ideal'

# Extrapolate to DC with different methods
```

(continues on next page)

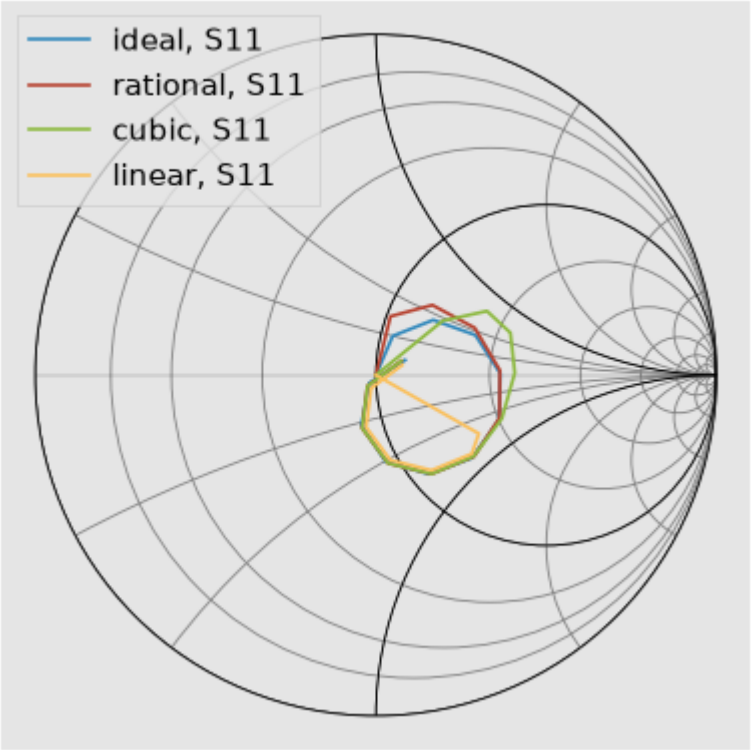
(continued from previous page)

```
dut_dc_rational = dut.extrapolate_to_dc(kind='rational', dc_sparam=[[0,1],[1,0]])
dut_dc_rational.name = 'rational'
dut_dc_linear = dut.extrapolate_to_dc(kind='linear', dc_sparam=[[0,1],[1,0]])
dut_dc_linear.name = 'linear'
dut_dc_cubic = dut.extrapolate_to_dc(kind='cubic', dc_sparam=[[0,1],[1,0]])
dut_dc_cubic.name = 'cubic'
```

[5]:
```
plt.figure()
plt.title('Step Response')
dut_ideal.plot_z_time_step(0, 0)
dut_dc_rational.plot_z_time_step(0, 0)
dut_dc_cubic.plot_z_time_step(0, 0)
dut_dc_linear.plot_z_time_step(0, 0)
```



[6]:
```
dut_ideal.s11['0-2ghz'].plot_s_smith()
dut_dc_rational.s11['0-2ghz'].plot_s_smith()
dut_dc_cubic.s11['0-2ghz'].plot_s_smith()
dut_dc_linear.s11['0-2ghz'].plot_s_smith()
```

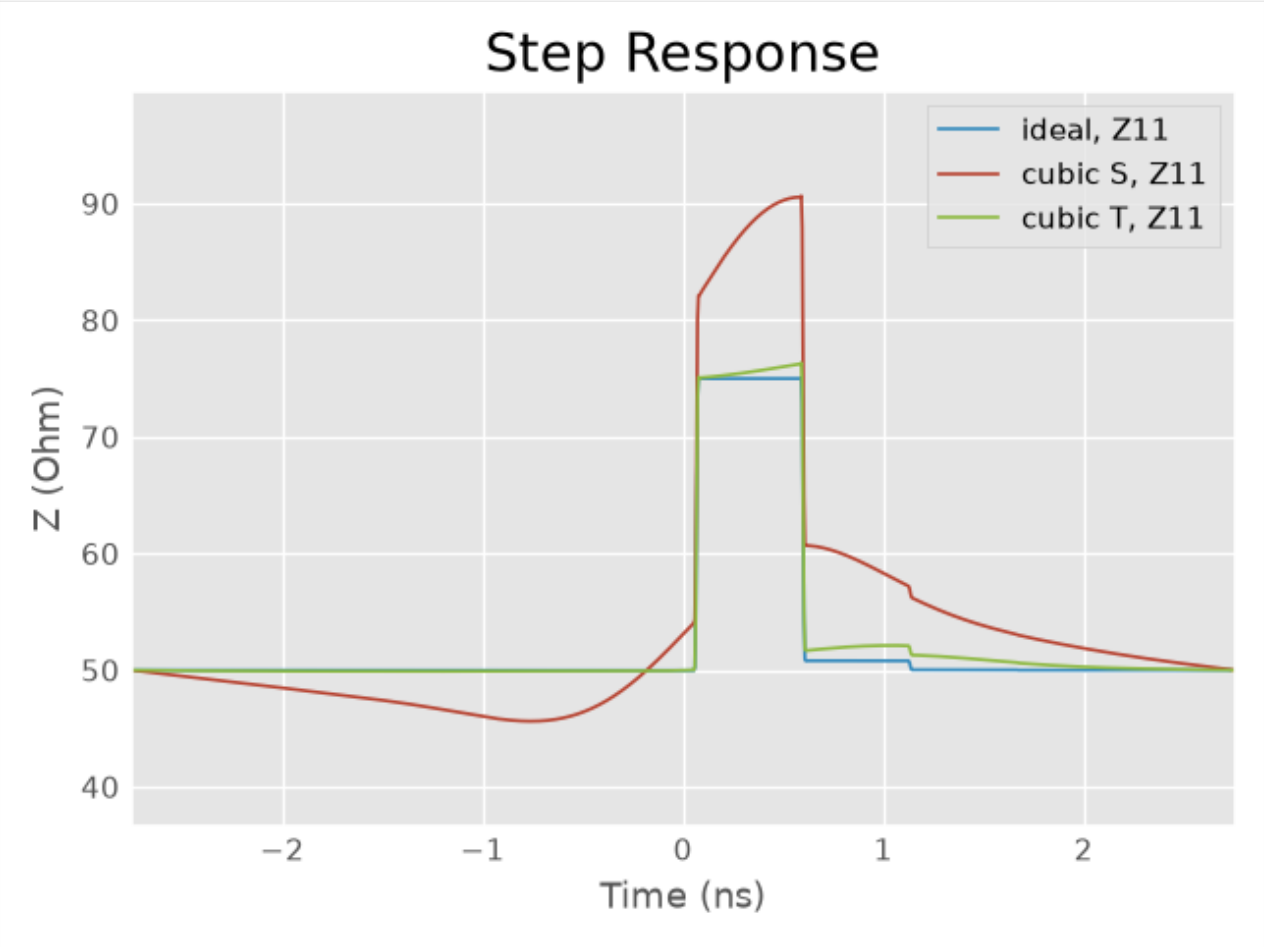


Interpolation basis

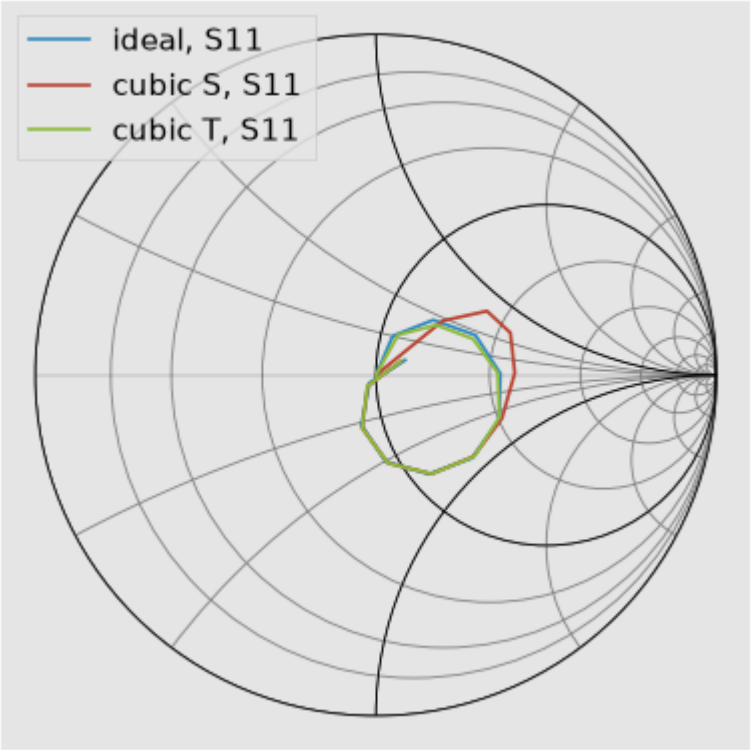
By default S-parameters are interpolated, but it's possible to interpolate also other parameters such as T, ABCD, Z or Y-parameters.

Usually S-parameters are the best choice, but in this case T-parameters give better results. Note that T-parameters are defined only for two-ports with non-zero S21. It probably isn't the best choice in most cases but for well matched transmissive two-ports it might be worth trying.

[7]:
```
dut_dc_cubic_t = dut.extrapolate_to_dc(kind='cubic', dc_sparam=[[0,1],[1,0]], basis='t')
dut_dc_cubic_t.name = 'cubic T'
dut_dc_cubic.name = 'cubic S'

plt.figure()
plt.title('Step Response')
dut_ideal.plot_z_time_step(0, 0)
dut_dc_cubic.plot_z_time_step(0, 0)
dut_dc_cubic_t.plot_z_time_step(0, 0)
```



[8]:
```
dut_ideal.s11['0-2ghz'].plot_s_smith()
dut_dc_cubic.s11['0-2ghz'].plot_s_smith()
dut_dc_cubic_t.s11['0-2ghz'].plot_s_smith()
```



[ ]:

2.1.7 DefinedAEpTandZ0 media example

[1]:
```
%load_ext autoreload
%autoreload 2
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.ticker import AutoMinorLocator
from numpy import log, sum
from scipy.optimize import minimize

import skrf as rf
import skrf.mathFunctions as mf
from skrf.calibration import NISTMultilineTRL

rf.stylely()
```

Measurement of two CPWG lines with different lengths

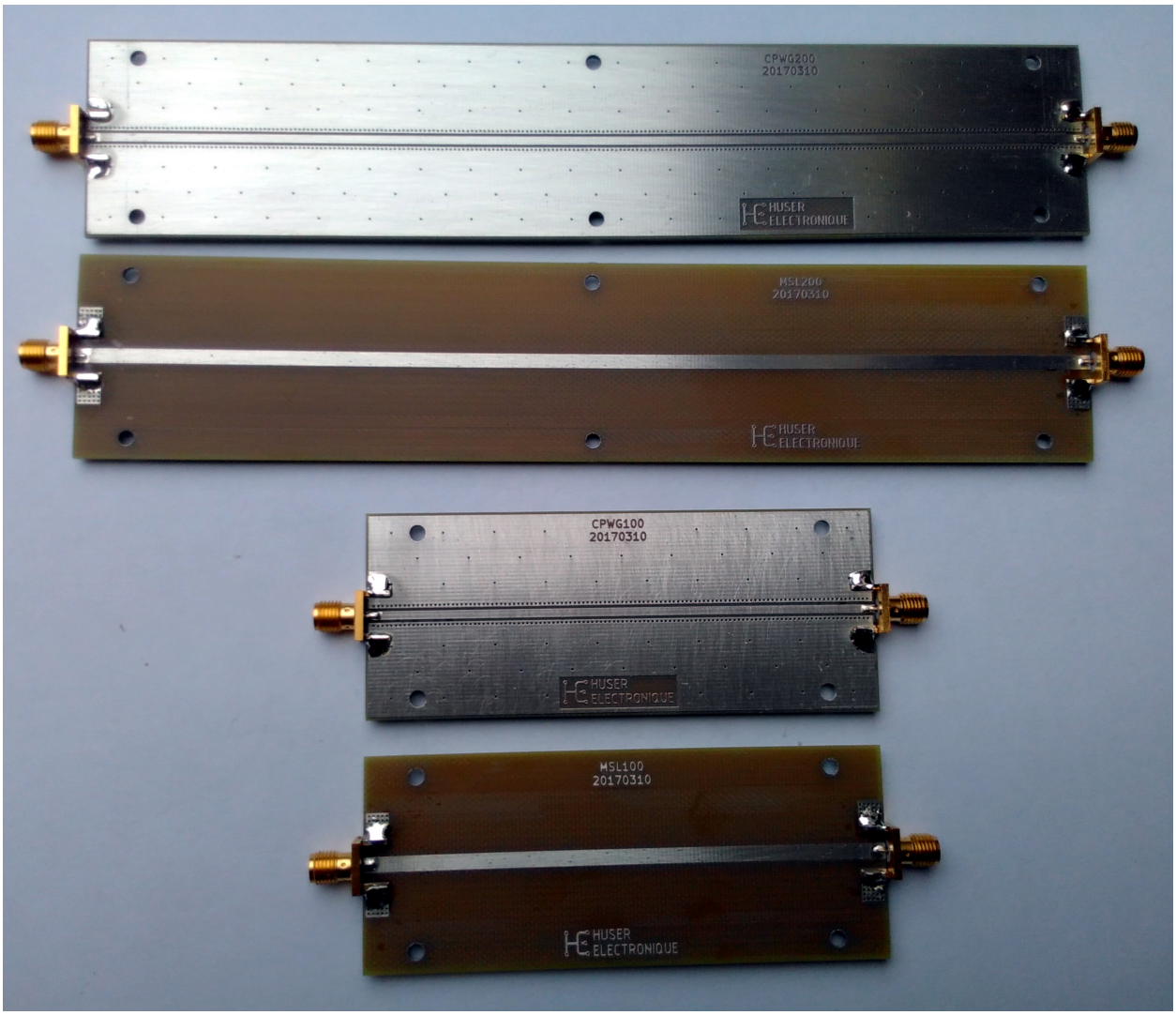
The measurement where performed the 21th March 2017 on a Anritsu MS46524B 20GHz Vector Network Analyser. The setup is a linear frequency sweep from 1MHz to 10GHz with 10'000 points. Output power is 0dBm, IF bandwidth is 1kHz and neither averaging nor smoothing are used.

CPWGxxx is a L long, W wide, with a G wide gap to top ground, T thick copper coplanar waveguide on ground on a H height substrate with top and bottom ground plane. A closely spaced via wall is placed on both side of the line and the top and bottom ground planes are connected by many vias.

Name	L (mm)	W (mm)	G (mm)	H (mm)	T (um)	Substrate
MSL100	100	1.70	0.50	1.55	50	FR-4
MSL200	200	1.70	0.50	1.55	50	FR-4

The milling of the artwork is performed mechanically with a lateral wall of 45°.

The relative permittivity of the dielectric was assumed to be approximately 4.5 for design purpose.

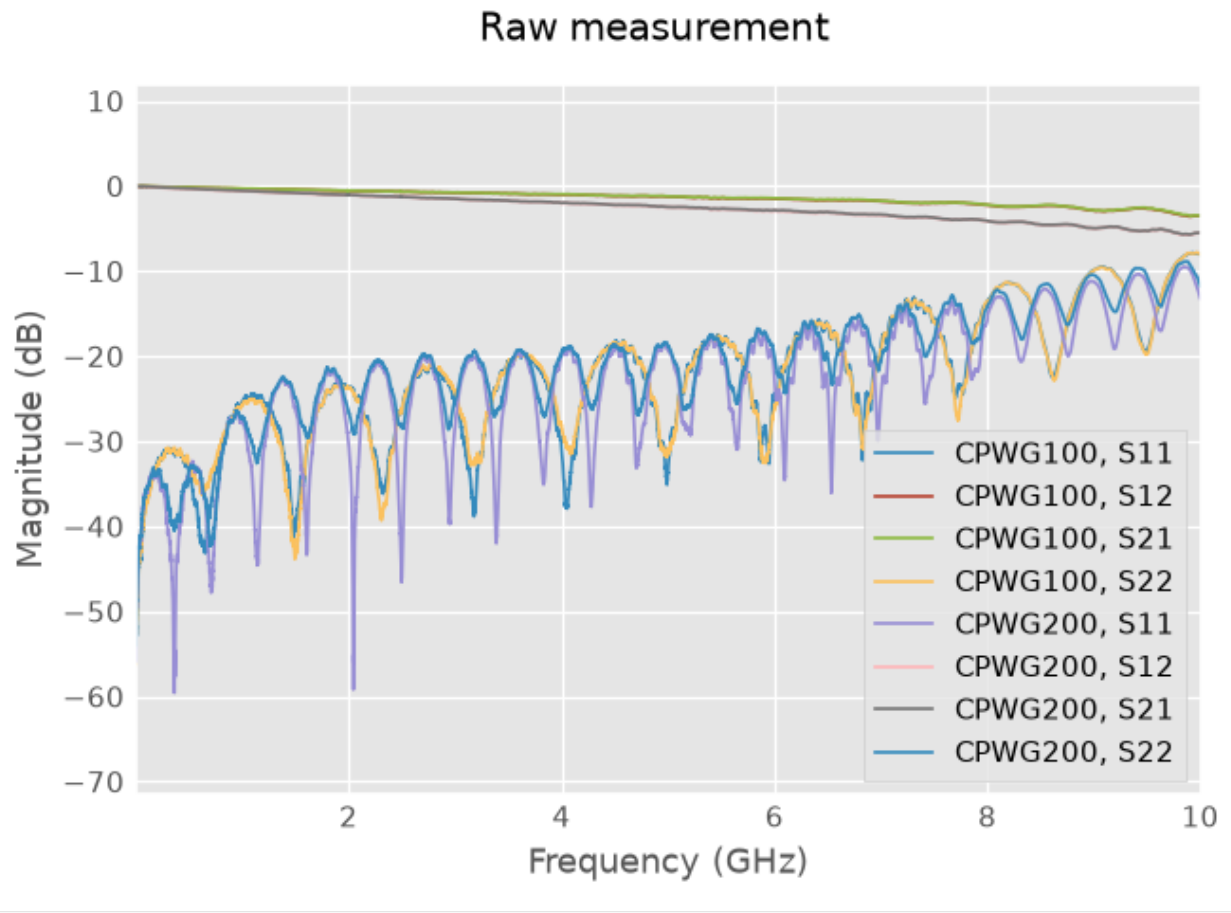


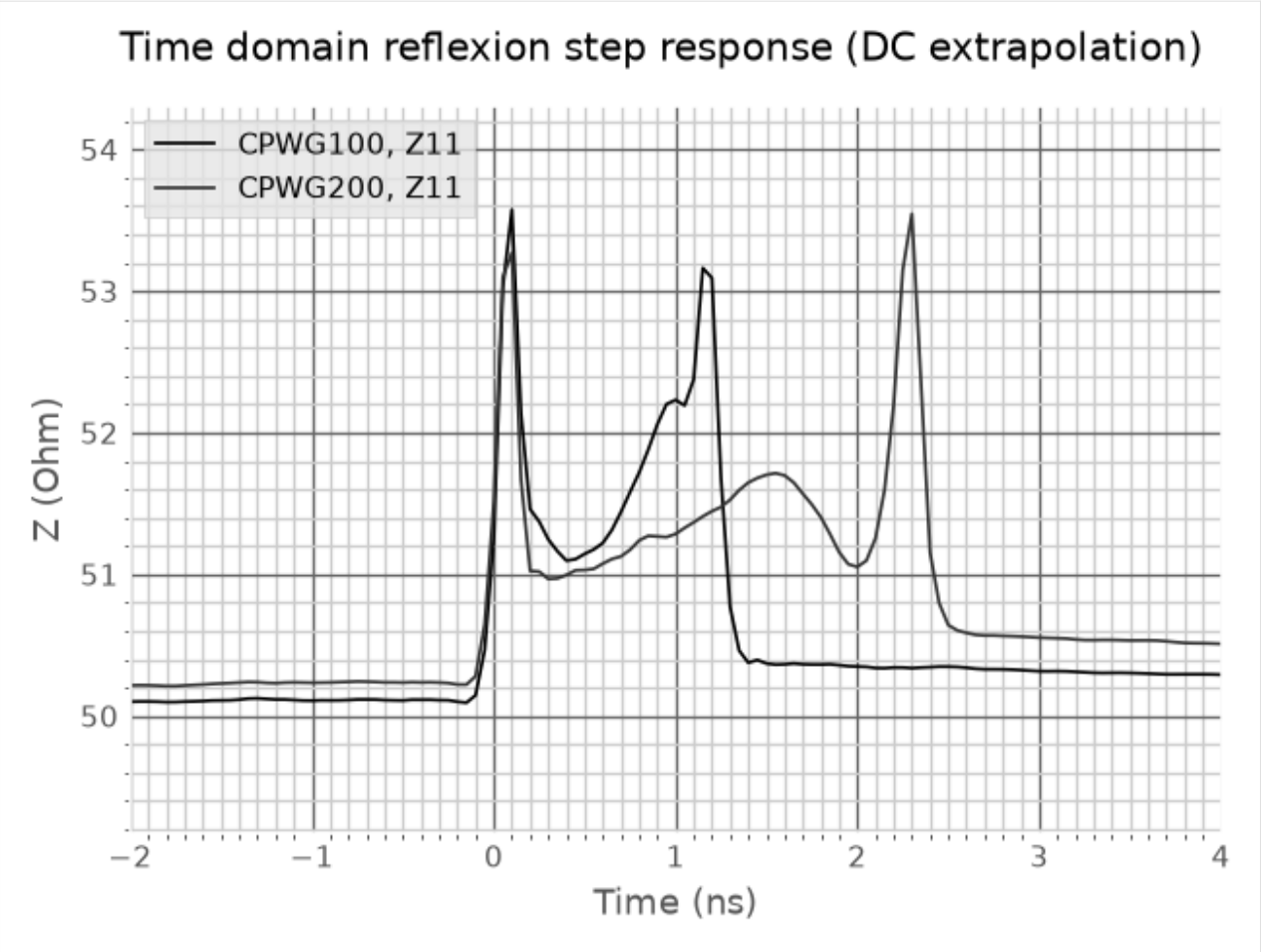
[2]:
```
# Load raw measurements
TL100 = rf.Network('CPWG100.s2p')
TL200 = rf.Network('CPWG200.s2p')
TL100_dc = TL100.extrapolate_to_dc(kind='linear')
TL200_dc = TL200.extrapolate_to_dc(kind='linear')

plt.figure()
plt.suptitle('Raw measurement')
TL100.plot_s_db()
TL200.plot_s_db()
```

(continues on next page)

(continued from previous page)

```
plt.figure()
t0 = -2
t1 = 4
plt.suptitle('Time domain reflexion step response (DC extrapolation)')
ax = plt.subplot(1, 1, 1)
TL100_dc.plot_z_time_step(0, 0, ax=ax, color='0.0')
TL200_dc.plot_z_time_step(0, 0, ax=ax, color='0.2')
ax.set_xlim(t0, t1)
ax.xaxis.set_minor_locator(AutoMinorLocator(10))
ax.yaxis.set_minor_locator(AutoMinorLocator(5))
ax.patch.set_facecolor('1.0')
ax.grid(True, color='0.8', which='minor')
ax.grid(True, color='0.4', which='major')
plt.show()
```





Impedance from the line and from the connector section may be estimated on the step response. The line section is not flat, there is some variation in the impedance which may be induced by manufacturing tolerances and dielectric inhomogeneity.

Note that the delay on the reflexion plot are twice the effective section delays because the wave travel back and forth on the line.

Connector discontinuity is about 50 ps long. TL100 line plateau (flat impedance part) is about 450 ps long.

[3]:
```
Z_conn = 53.2    # ohm, connector impedance
Z_line = 51.4    # ohm, line plateau impedance
d_conn = 0.05e-9 # s,   connector discontinuity delay
d_line = 0.45e-9 # s,   line plateau delay, without connectors
```

Dielectric effective relative permittivity extraction by multiline method

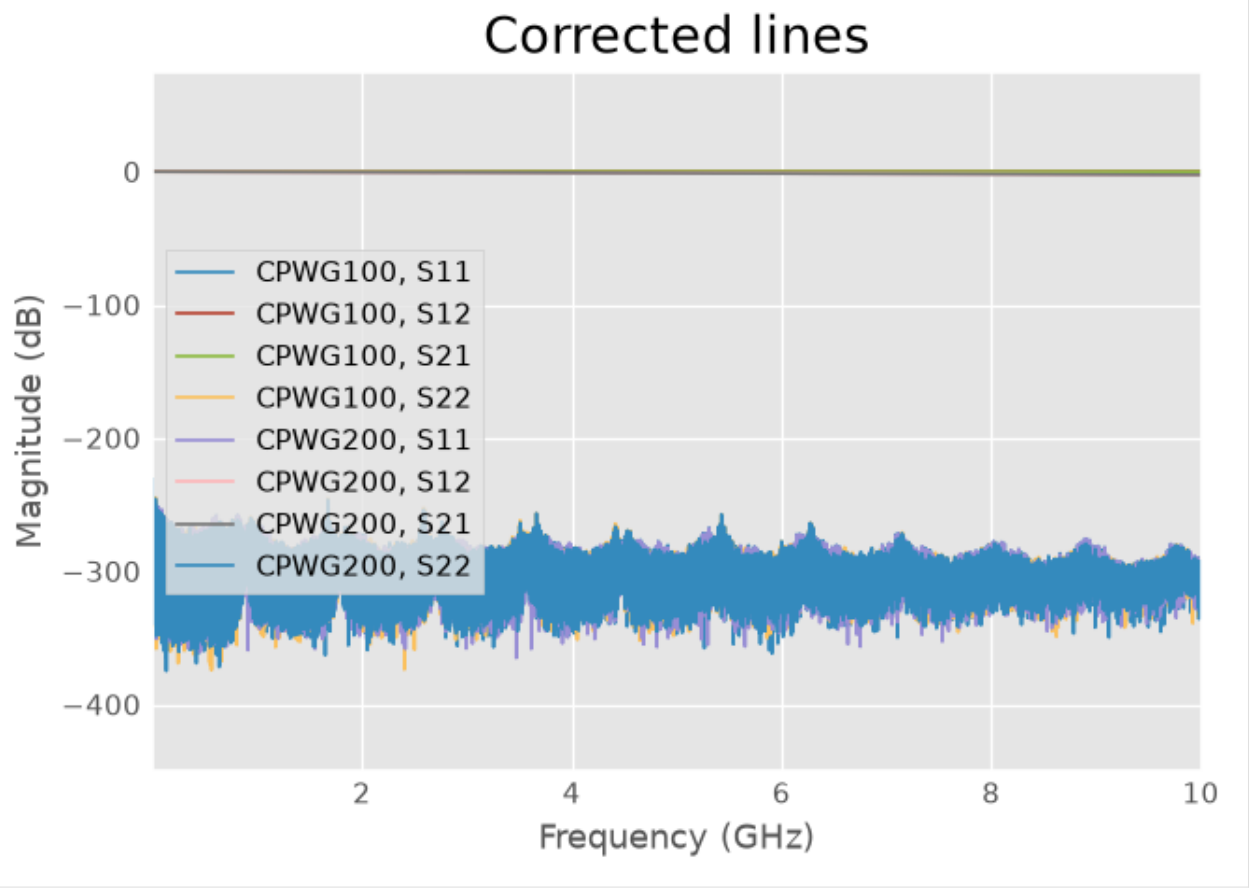
[4]:
```
#Make the missing reflect measurement
#This is possible because we already have existing calibration
#and know what the open measurement would look like at the reference plane
#'refl_offset' needs to be set to -half_thru - connector_length.
reflect = TL100.copy()
reflect.s[:,0,0] = 1
reflect.s[:,1,1] = 1
```

(continues on next page)

(continued from previous page)

```
reflect.s[:,1,0] = 0
reflect.s[:,0,1] = 0

# Perform NISTMultilineTRL algorithm. Reference plane is at the center of the thru.
cal = NISTMultilineTRL([TL100, reflect, TL200], [1], [0, 100e-3], er_est=3.0, refl_
↪offset=[-56e-3])

plt.figure()
plt.title('Corrected lines')
cal.apply_cal(TL100).plot_s_db()
cal.apply_cal(TL200).plot_s_db()
plt.show()
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:2948: UserWarning: No switch terms provided
  EightTerm.__init__(self,
```



Calibration results shows a very low residual noise floor. The error model is well fitted.

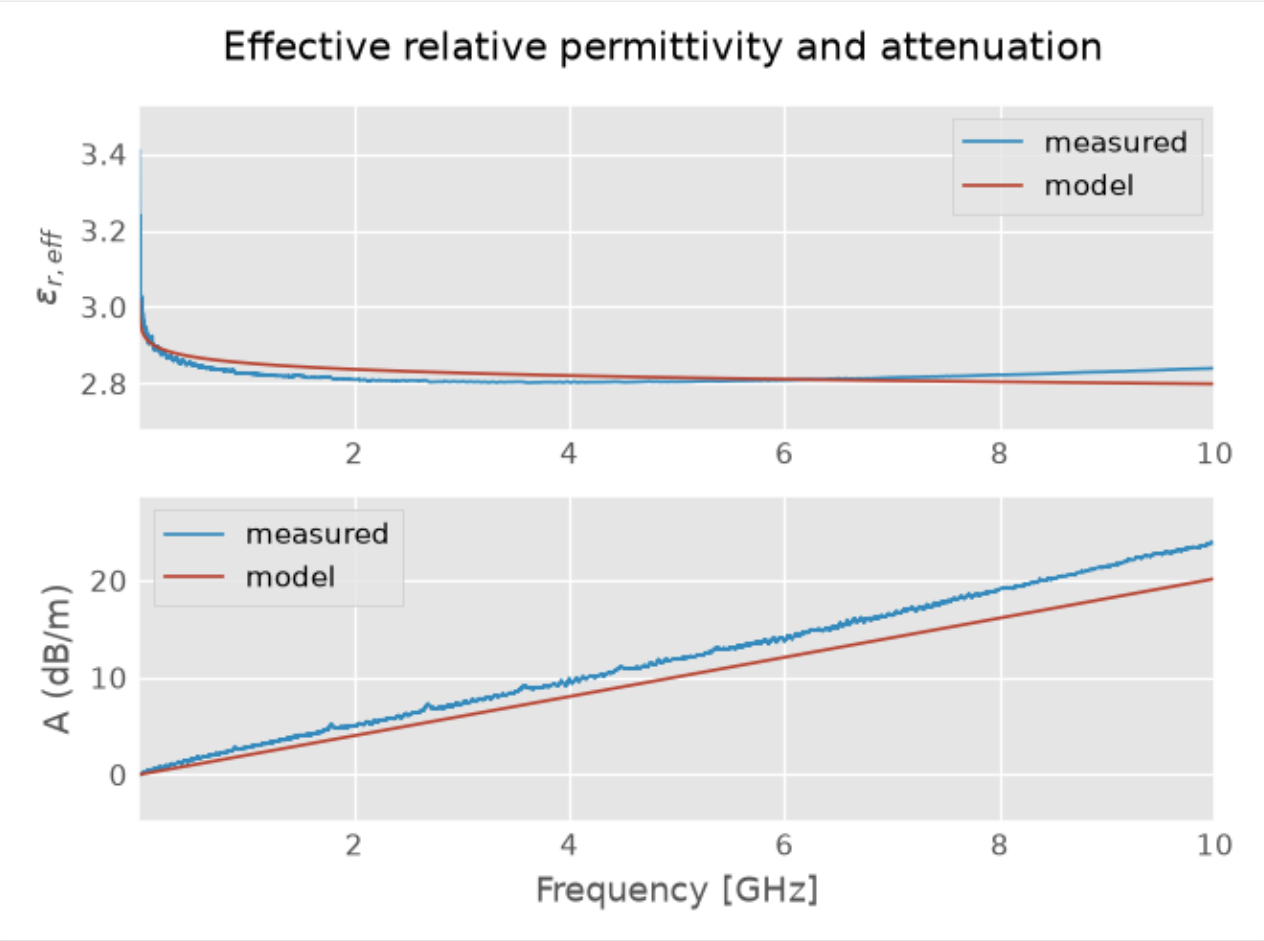
[5]:
```
from skrf.media import DefinedAEpTandZ0

freq  = TL100.frequency
f     = TL100.frequency.f
f_ghz = TL100.frequency.f/1e9
```

(continues on next page)

(continued from previous page)

```
L     = 0.1
A     = 0.0
f_A   = 1e9
ep_r0 = 2.0
tanD0 = 0.001
f_ep  = 1e9
x0 = [ep_r0, tanD0]

ep_r_mea = cal.er_eff.real
A_mea    = 20/log(10)*cal.gamma.real

def model(x, freq, ep_r_mea, A_mea, f_ep):
    ep_r, tanD = x[0], x[1]
    m = DefinedAEpTandZ0(frequency=freq, ep_r=ep_r, tanD=tanD, z0=50,
                                f_low=1e3, f_high=1e18, f_ep=f_ep, model=
↪'djordjevicsvensson')
    ep_r_mod = m.ep_r_f.real
    A_mod = m.alpha * log(10)/20
    return sum((ep_r_mod - ep_r_mea)**2)  + 0.001*sum((20/log(10)*A_mod - A_mea)**2)

res = minimize(model, x0, args=(TL100.frequency, ep_r_mea, A_mea, f_ep),
               bounds=[(2, 4), (0.001, 0.013)])
ep_r, tanD = res.x[0], res.x[1]

print(f'epr={ep_r:.3f}, tand={tanD:.4f} at {f_ep * 1e-9:.1f} GHz.')

m = DefinedAEpTandZ0(frequency=freq, ep_r=ep_r, tanD=tanD, z0=50,
                                f_low=1e3, f_high=1e18, f_ep=f_ep, model=
↪'djordjevicsvensson')

plt.figure()
plt.suptitle('Effective relative permittivity and attenuation')
plt.subplot(2,1,1)
plt.ylabel(r'$\epsilon_{r,eff}$')
plt.plot(f_ghz, ep_r_mea, label='measured')
plt.plot(f_ghz, m.ep_r_f.real, label='model')
plt.legend()

plt.subplot(2,1,2)
plt.xlabel('Frequency [GHz]')
plt.ylabel('A (dB/m)')
plt.plot(f_ghz, A_mea, label='measured')
plt.plot(f_ghz, 20/log(10)*m.alpha, label='model')
plt.legend()
plt.show()
```

```
epr=2.853, tand=0.0130 at 1.0 GHz.
```



Relative permittivity $\epsilon_{e,eff}$ and attenuation A shows a reasonable agreement.

A better agreement could be achieved by implementing the Kirschning and Jansen microstripline dispersion model or using a linear correction.

Connectors effects estimation

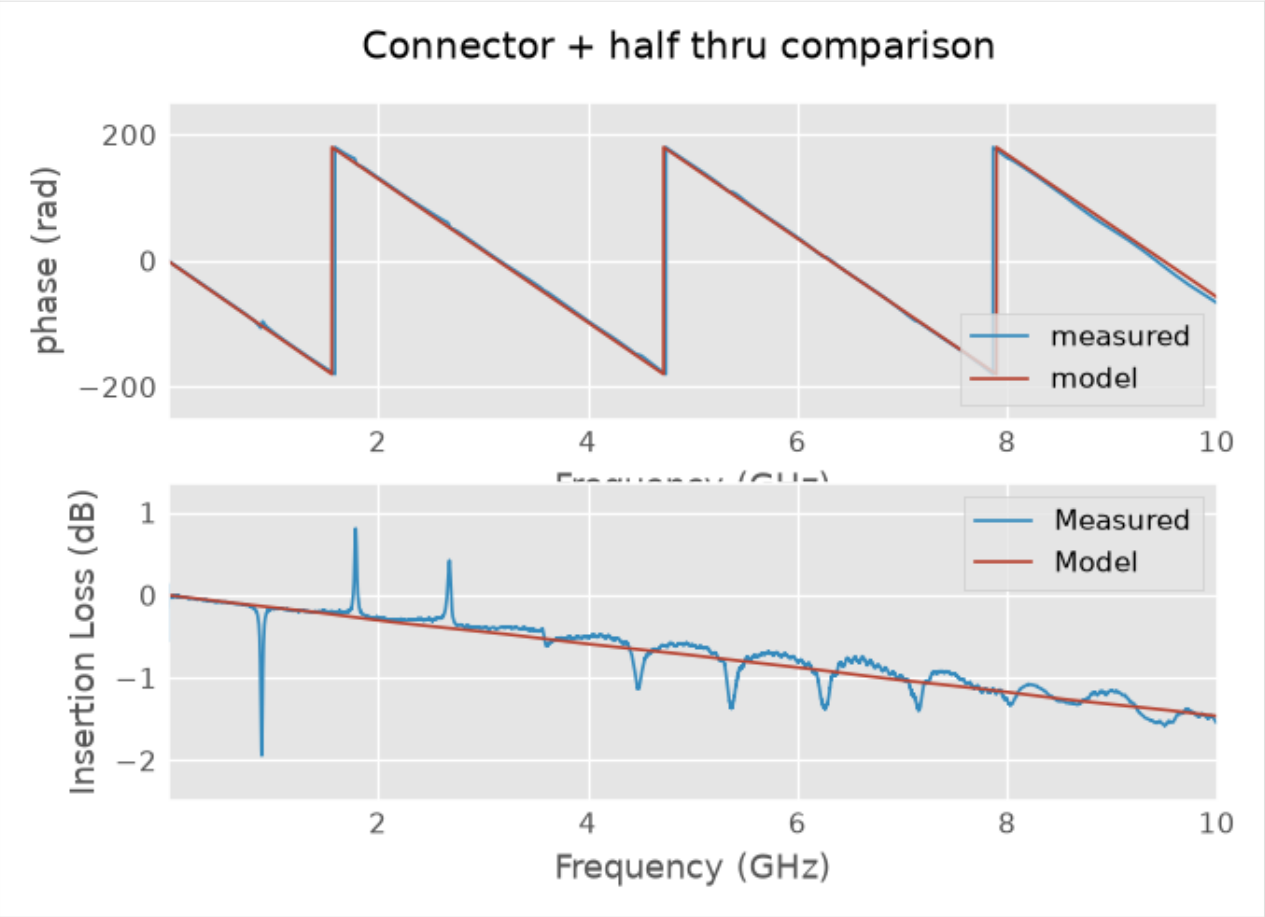
[6]:
```
# note: a half line is embedded in connector network
coefs = cal.coefs
r = mf.sqrt_phase_unwrap(coefs['forward reflection tracking'])
s1 = np.array([[coefs['forward directivity'],r],
        [r, coefs['forward source match']]]).transpose()

conn = TL100.copy()
conn.name = 'Connector'
conn.s = s1

# delay estimation,
phi_conn = (np.angle(conn.s[:500,1,0]))
z = np.polyfit(f[:500], phi_conn, 1)
p = np.poly1d(z)
delay = -z[0]/(2*np.pi)
print(f'Connector + half thru delay: {delay * 1e12:.0f} ps')
```

(continues on next page)

(continued from previous page)

```
print(f'TDR read half thru delay: {d_line/2 * 1e12:.0f} ps')
d_conn_p = delay - d_line/2
print(f'Connector delay: {d_conn_p * 1e12:.0f} ps')


# connector model with guessed loss
half = m.line(d_line/2, 's', z0=Z_line)
mc = DefinedAEpTandZ0(m.frequency, ep_r=1, tanD=0.025, z0=50,
                              f_low=1e3, f_high=1e18, f_ep=f_ep, model=
↪'djordjevicsvensson')
left = mc.line(d_conn_p, 's', z0=Z_conn)
right = left.flipped()
check = mc.thru() ** left ** half ** mc.thru()

plt.figure()
plt.suptitle('Connector + half thru comparison')
plt.subplot(2,1,1)
conn.plot_s_deg(1, 0, label='measured')
check.plot_s_deg(1, 0, label='model')
plt.ylabel('phase (rad)')
plt.legend()

plt.subplot(2,1,2)
conn.plot_s_db(1, 0, label='Measured')
check.plot_s_db(1, 0, label='Model')
plt.xlabel('Frequency (GHz)')
plt.ylabel('Insertion Loss (dB)')
plt.legend()
plt.show()
```

```
Connector + half thru delay: 316 ps
TDR read half thru delay: 225 ps
Connector delay: 91 ps
```



Connector + thru plots shows a reasonable agreement between calibration results and model.

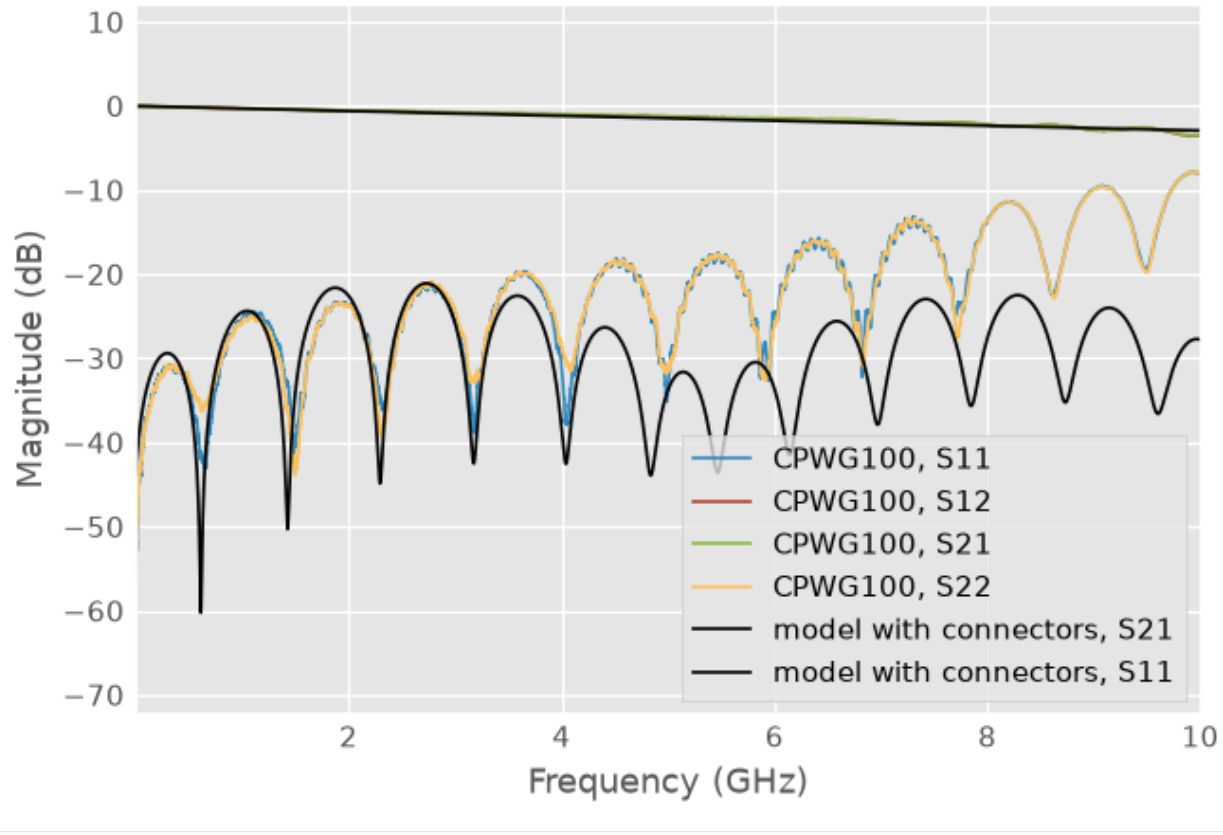
Final check

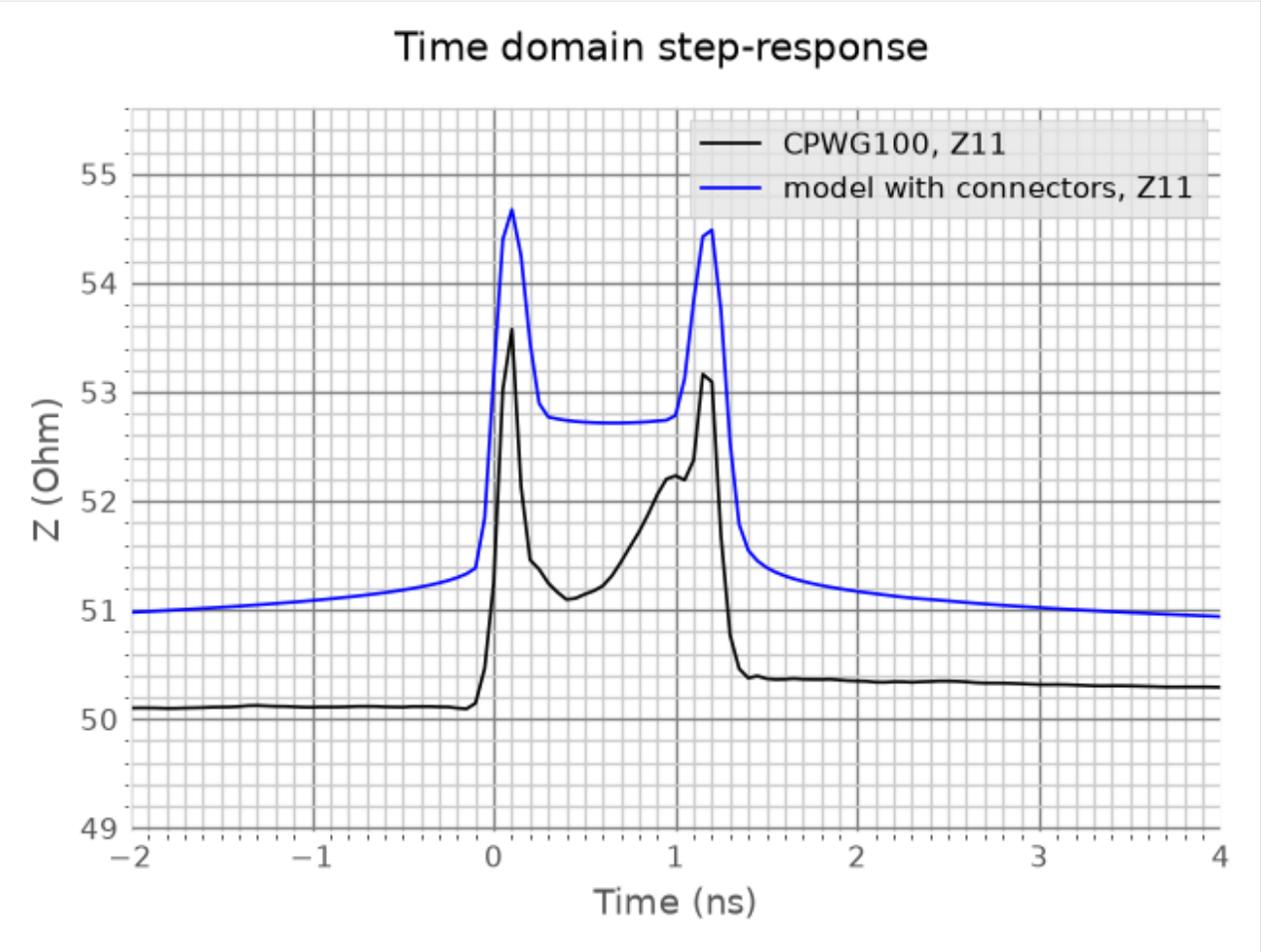
[7]:
```
DUT = m.line(d_line, 's', Z_line)
DUT.name = 'model'

Check = m.thru() ** left ** DUT ** right ** m.thru()
Check.name = 'model with connectors'

plt.figure()
TL100.plot_s_db()
Check.plot_s_db(1,0, color='k')
Check.plot_s_db(0,0, color='k')
plt.show()

Check_dc = Check.extrapolate_to_dc(kind='linear')

plt.figure()
plt.suptitle('Time domain step-response')
ax = plt.subplot(1,1,1)
TL100_dc.plot_z_time_step(0, 0, ax=ax, color='k')
Check_dc.plot_z_time_step(0, 0, ax=ax, color='b')
```

(continues on next page)

(continued from previous page)

```
t0 = -2
t1 = 4
ax.set_xlim(t0, t1)
ax.xaxis.set_minor_locator(AutoMinorLocator(10))
ax.yaxis.set_minor_locator(AutoMinorLocator(5))
ax.patch.set_facecolor('1.0')
ax.grid(True, color='0.8', which='minor')
ax.grid(True, color='0.5', which='major')
```





The plots shows a reasonable agreement between model and measurement up to 4 GHz.

Further works may include implementing CPWG medium or modeling the line by more sections to account the impedance variation vs. position.

[ ]:

2.1.8 IEEEP370 Deembedding

Table of Content

- *Single-ended*
  - *Single-Ended simulation of 2xThru, DUT and Fixture-DUT-Fixture*
  - *Single-Ended S-parameters quality checking*
  - *IEEEP370_SE_NZC_2xthru without impedance correction*
  - *IEEEP370_SE_ZC_2xthru with impedance correction*
  - *Single Ended Comparison with AICC De-Embedding Utility*
- *Mixed mode*
  - *Mixed mode simulation of 2xThru, DUT and Fixture-DUT-Fixture*
  - *Mixed mode S-parameters quality checking*

- *IEEEP370_MM_NZC_2xthru without impedance correction*
- *IEEEP370_MM_ZC_2xthru with impedance correction*
- *Mixed mode Comparison with AICC De-Embedding Utility*

Introduction

The IEEE 370-2020 standard (*Standard for Electrical Characterization of Printed Circuit Board and Related Interconnects at Frequencies up to 50 GHz*) provides good practices for ensuring the quality of measured data for high-frequency electrical interconnects at frequencies up to 50 GHz. It recommends methods and processes for ensuring the accuracy and consistency of measured data for signals with frequency content up to 50 GHz, in particular for removing test fixture and instrumentation effects.

In the examples below, and following the standard proposal on labeling, a 2-port DUT has one fixture attached to each of its ports. The fixture attached to the single-ended DUT port 1 is labeled as FIX-1. The fixture attached to the single-ended DUT port 2 is labeled as FIX-2. The cascade of the DUT and fixtures on both ends is named the composite structure or the FIX-DUT-FIX structure.

The combination of the two fixtures, such as FIX-1 connected to FIX-2, is referred to as a FIX-FIX structure or a *2x-Thru*.

The purpose of this notebook is to illustrate the `scikit-rf` implementation of the deembedding methods proposed within the IEEE P370 standard to remove the test fixture effects.

First, let's make the necessary Python import statements:

[1]:
```
import matplotlib.pyplot as plt
from scipy.constants import c, pi

import skrf as rf
from skrf.calibration.deembedding import (
    IEEEP370,
    IEEEP370_FD_QM,
    IEEEP370_FER,
    IEEEP370_TD_QM,
    IEEEP370_MM_NZC_2xThru,
    IEEEP370_MM_ZC_2xThru,
    IEEEP370_SE_NZC_2xThru,
    IEEEP370_SE_ZC_2xThru,
)
from skrf.media import DefinedGammaZ0, MLine
from skrf.network import concat_ports

rf.stylely()
```

Single-ended

The algorithms can be used with 2-port single-ended networks.

Single-Ended simulation of 2xThru, DUT and Fixture-DUT-Fixture

We use `scikit-rf` `MLine` media to simulate microstrip line artifacts. This is a convenient way to generate synthetic S-parameter networks without leaving Python to operate another software.

- `dut` is a a Beatty structure with a 3xWidth microstrip section connected left and right by two uniform 1xWidth microstrip lines.
- `fdf` is FIX-DUT-FIX, prolonging the DUT left and right with a microstrip line and a planar-to-coaxial connector.
- `s2xthru` is FIX-FIX and is the cascade of the two prolonging lines and connectors without the DUT. For example purpose, the width of the lines is altered by 8% to show the effect of impedance mismatch on the deembedding process.

The target is to get the left and right fixtures models and retrieve the DUT by deembedding them from FIX-DUT-FIX.

The microstrip lines are inspired by *this example*.

[2]:
```
directory = 'ieeep370deembedding/'
freq = rf.Frequency(10e-3, 10, 1000, 'GHz')
W   = 3.00e-3
H   = 1.55e-3
T   = 50e-6
ep_r = 4.459
tanD = 0.0183
f_epr_tand = 1e9

# microstrip segments
MSL1 = MLine(frequency=freq, z0_port=50, w=W, h=H, t=T,
        ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tanD, rough=0.15e-6,
        f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
        diel='djordjevicsvensson', disp='kirschningjansen')

# capacitive 3 x width Beatty structure
MSL2 = MLine(frequency=freq, z0_port=50, w=3*W, h=H, t=T,
        ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tanD, rough=0.15e-6,
        f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
        diel='djordjevicsvensson', disp='kirschningjansen')

# microstrip segment with a 10% variation of width
MSL3 = MLine(frequency=freq, z0_port=50, w=0.92*W, h=H, t=T,
        ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tanD, rough=0.15e-6,
        f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
        diel='djordjevicsvensson', disp='kirschningjansen')

# connector SMA edge PCB mount soldered
CONN = DefinedGammaZ0(frequency=freq, gamma = 1j* 2 * pi * freq.f / c, z0_port = 50)
z_conn = 53.2    # ohm, connector impedance
d_conn = 50      # ps,   connector discontinuity delay
connector = CONN.line(50, 'ps', z0 = z_conn)

# building DUT
dut =    MSL1.line(20e-3, 'm') \
      ** MSL2.line(20e-3, 'm') \
```

(continues on next page)

(continued from previous page)

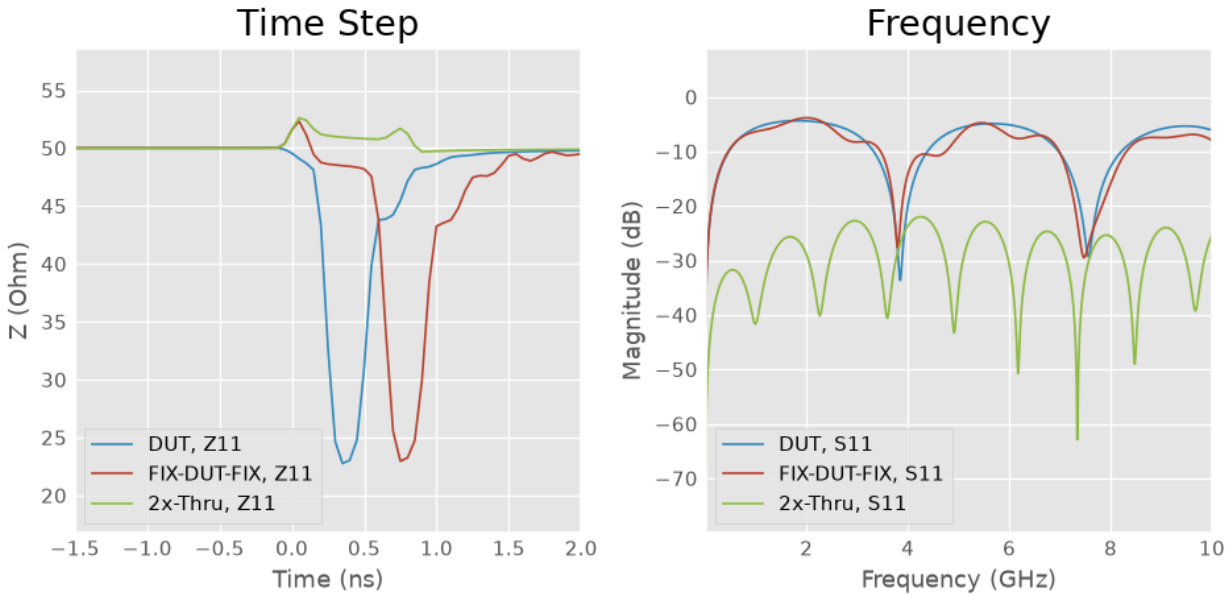
```
      ** MSL1.line(20e-3, 'm')
dut.name = 'DUT'

# building FIXTURE-DUT-FIXTURE
thru1 = MSL1.line(25e-3, 'm')
thru3 = MSL3.line(25e-3, 'm')
fdf     = connector ** thru1 ** dut ** thru1 ** connector
fdf.name = 'FIX-DUT-FIX'

# building FIXTURE-FIXTURE with a 8% width variation from FIXTURE-DUT-FIXTURE(2xthru)
s2xthru = connector ** thru3 ** thru3 ** connector
s2xthru.name = '2x-Thru'

# extrapolate to DC for time step
dut_dc = IEEEP370.extrapolate_to_dc(dut)
fdf_dc = IEEEP370.extrapolate_to_dc(fdf)
s2xthru_dc = IEEEP370.extrapolate_to_dc(s2xthru)
# set True to write .s2p files
if False:
    s2xthru.write_touchstone(directory + 'se_2xthru_nonoise.s2p')
    fdf.write_touchstone(directory + 'se_fdf_nonoise.s2p')
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/media/
↪mline.py:276: RuntimeWarning: Conductor loss calculation invalid for lineheight t (5e-
↪05)  < 3 * skin depth (2.0824376726482413e-05)
  self.alpha_conductor, self.alpha_dielectric = self.analyse_loss(
```

In the figure below, the time shift of DUT caused by the fixtures is visible on FIX-DUT-FIX trace. The 2x-Thru trace shows the awaited impedance mismatch with the FIX-DUT-FIX. The connectors add an impedance bump at the ends of 2x-Thru and FIX-DUT-FIX impedance step response. The reference DUT is the Beatty structure alone, without fixtures.

The _dc networks were extrapolated to DC for the time-domain step plot.

[3]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 4))

axs[0].set_title('Time Step')
dut_dc.plot_z_time_step(0, 0, ax = axs[0])
fdf_dc.plot_z_time_step(0, 0, ax = axs[0])
s2xthru_dc.plot_z_time_step(0, 0, ax = axs[0])
axs[0].set_xlim((-1.5, 2))
axs[0].legend(loc = 'lower left')

axs[1].set_title('Frequency')
dut.plot_s_db(0, 0, ax = axs[1])
fdf.plot_s_db(0, 0, ax = axs[1])
s2xthru.plot_s_db(0, 0, ax = axs[1])
axs[1].legend(loc = 'lower left')
fig.tight_layout()
```



Single-Ended S-parameters quality checking

First, check the quality metrics of the data to ensure they do not violate passivity, reciprocity, or causality. Initial quality checking in the frequency domain results in percentage scores and is informative only. The results are evaluated as good, acceptable, inconclusive or poor. Inconclusive or poor results may indicate the need to recalibrate the VNA and perform the measurements again.

This quick check may raise potential issues that question measurement, simulation, or de-embedding process.

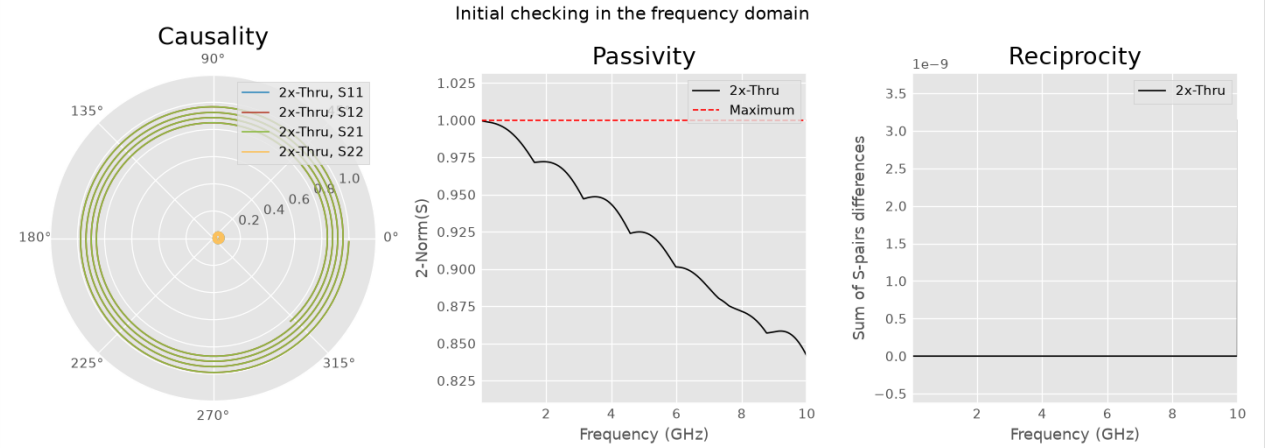
[4]:
```
fd_qm = IEEEP370_FD_QM()
print(s2xthru.name)
qm_2xthru = fd_qm.check_se_quality(s2xthru)
fd_qm.print_qm(qm_2xthru)
print(fdf.name)
qm_fdf = fd_qm.check_se_quality(fdf)
fd_qm.print_qm(qm_fdf)
```

```
2x-Thru
causality is good (100.00%)
passivity is good (100.00%)
reciprocity is good (100.00%)
FIX-DUT-FIX
causality is good (100.00%)
passivity is good (100.00%)
reciprocity is good (100.00%)
```

All the results are in the best class, which is not surprising for synthetic data.

Using the `verbose = True` parameter, it is possible to get an insight into the checking process. The causality check verifies that the complex S-parameters rotate clockwise in the complex plane. The passivity check verifies that the 2-Norm of S-parameters is smaller or equal to 1 at each frequency. The reciprocity check integrates the absolute difference between S_{ij} and S_{ji} pairs. For further details, refer to IEEE 370-2020.

[5]:
```
qm_2xthru = fd_qm.check_se_quality(s2xthru, verbose = True)
```



Application-based quality checking in the time domain targets interconnect applications with specified data rate and rise time. It is an advanced process that targets giving a physical estimation in millivolt based on peak distortion analysis between the input data and ideal re-constructed data.

If necessary, the original S-parameters are extrapolated to a frequency of three times the desired data rate. Causal, passive, and reciprocal models are reconstructed and stimulated by a pulse. The differences between the original extrapolated signal and the ideal responses in the time domain are integrated on unit intervals to give metrics in millivolts. The results are evaluated as good, acceptable, inconclusive or poor.

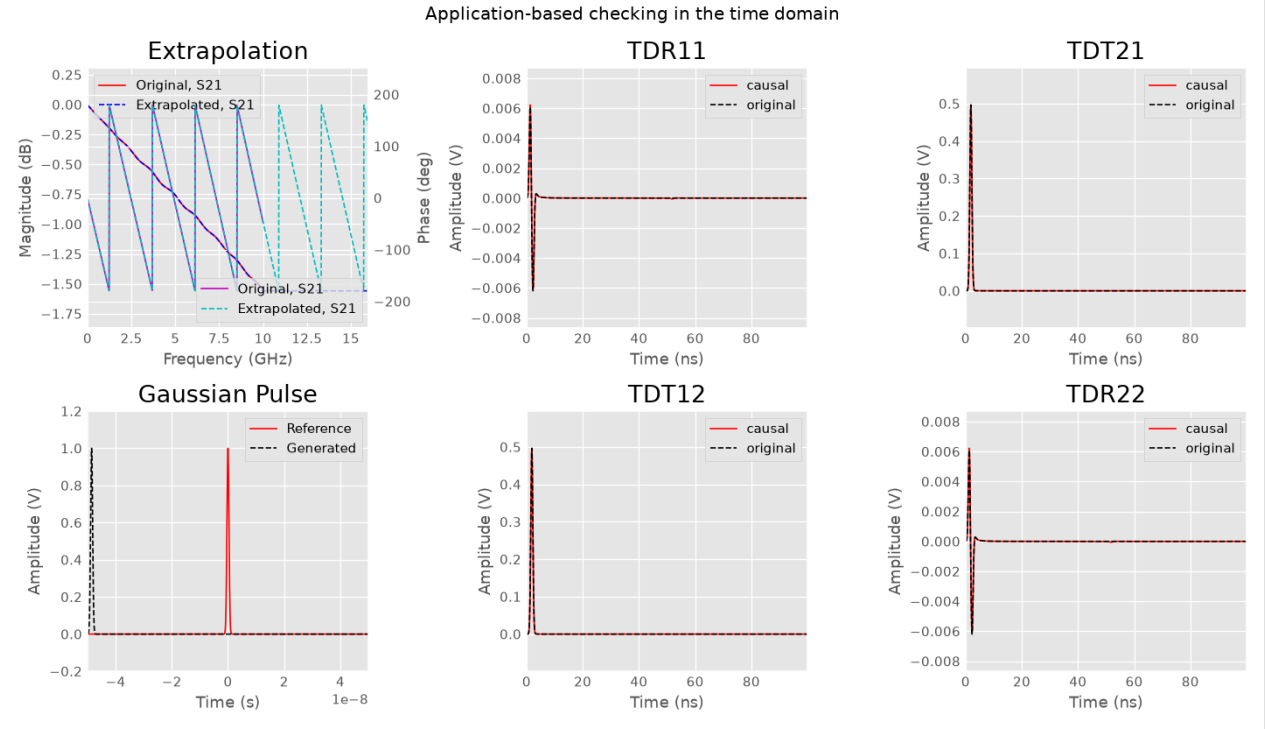
[6]:
```
td_qm = IEEEP370_TD_QM(1e9, # bps
                       32,  # unit interval samples number
                       0.4, # rise-time from 20% to 80% in fraction of width
                       1,   # gaussian pulse
                       1,   # constant extrapolation
                       verbose = False)
print(s2xthru.name)
qm_2xthru = td_qm.check_se_quality(s2xthru)
td_qm.print_qm(qm_2xthru)
print(fdf.name)
qm_fdf = td_qm.check_se_quality(fdf)
td_qm.print_qm(qm_fdf)
```

```
2x-Thru
causality in the time domain is good (0.2 mV)
passivity in the time domain is good (0.0 mV)
reciprocity in the time domain is good (0.0 mV)
FIX-DUT-FIX
causality in the time domain is good (1.9 mV)
passivity in the time domain is good (0.0 mV)
reciprocity in the time domain is good (0.0 mV)
```

All the results are in the best class, which is not surprising for synthetic data.

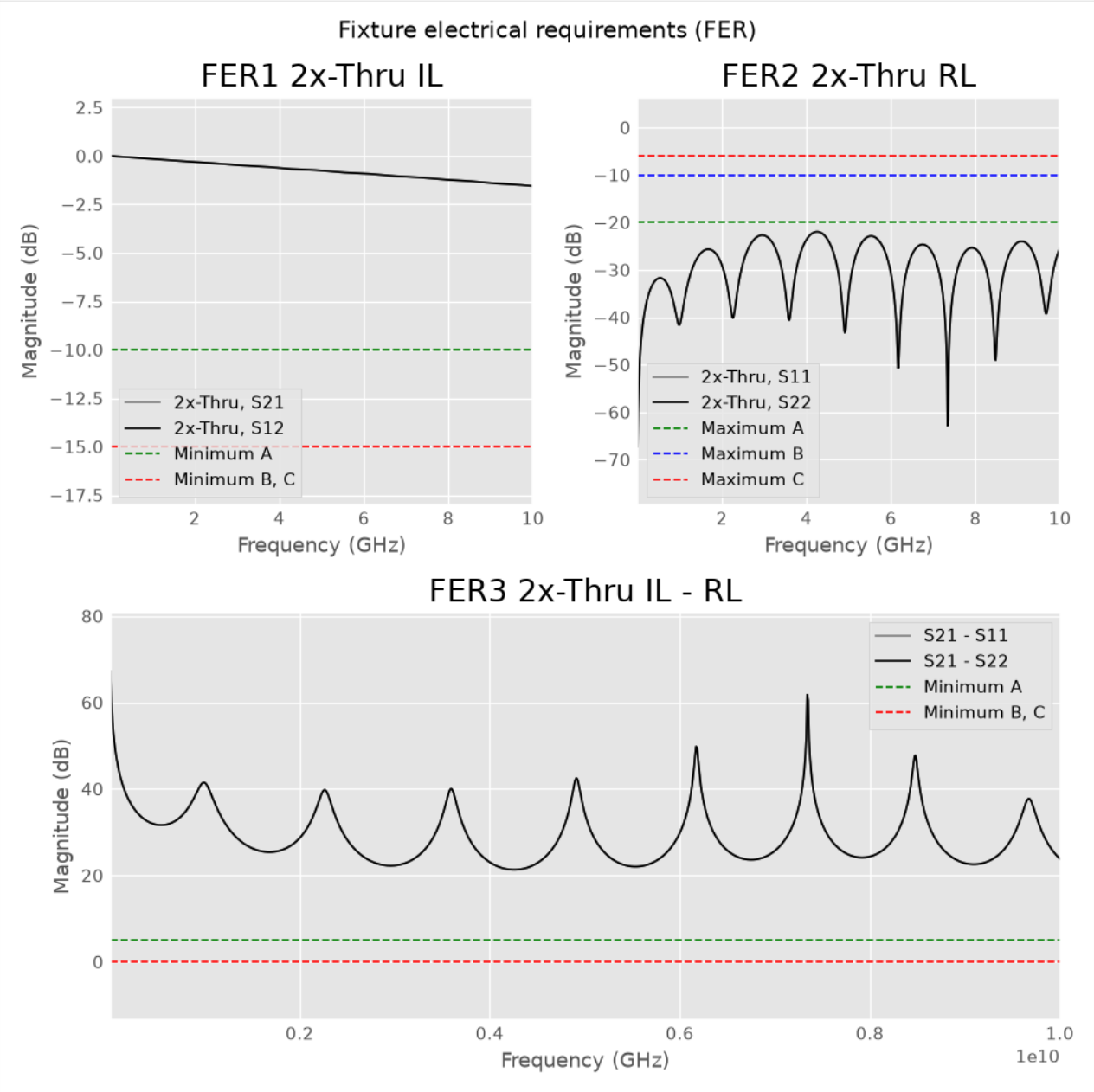
Using the `verbose = True` parameter, it is possible to get an insight into the checking process. The extrapolation, the pulse, and the time responses of the original and the causality-enforced models are plotted. For further details, refer to IEEE 370-2020.

[7]:
```
qm_2xthru = td_qm.check_se_quality(s2xthru, verbose = True)
```

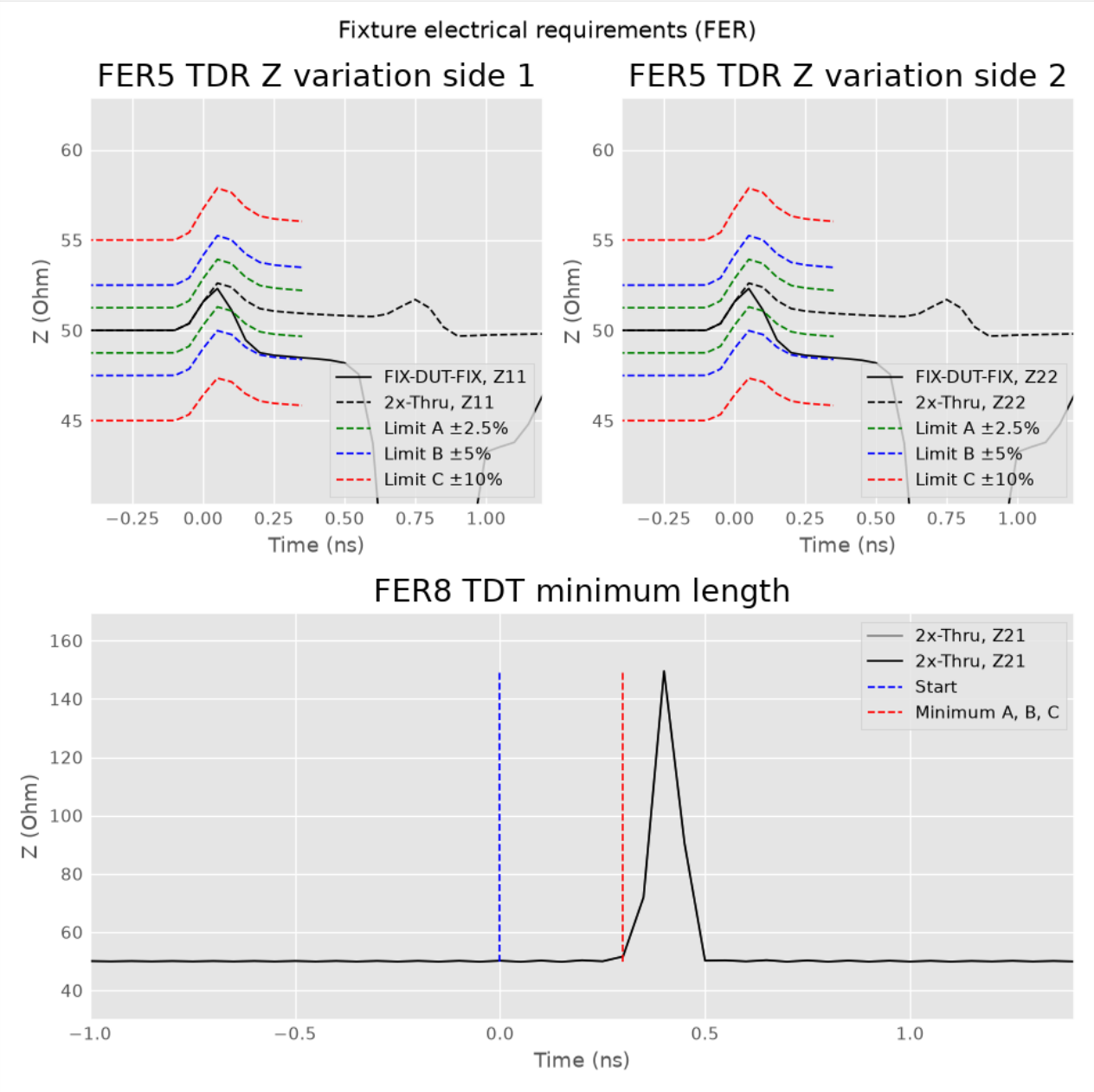


Before exercising the deembedding algorithms, the compliance of input data with IEEE 370 fixture electrical requirements (FER) shall be verified.

[8]:
```
fer = IEEEP370_FER()
fig = fer.plot_fd_se_fer(s2xthru)
```



[9]:
```
fig = fer.plot_td_se_fer(s2xthru, fdf)
```



The performance of the 2x-Thru is deemed class B compliant for frequencies below 10 GHz. This is due to the artificial impedance mismatch between 2x-Thru and FIX-DUT-FIX that fails the FER5 A limit.

IEEEP370_SE_NZC_2xthru without impedance correction

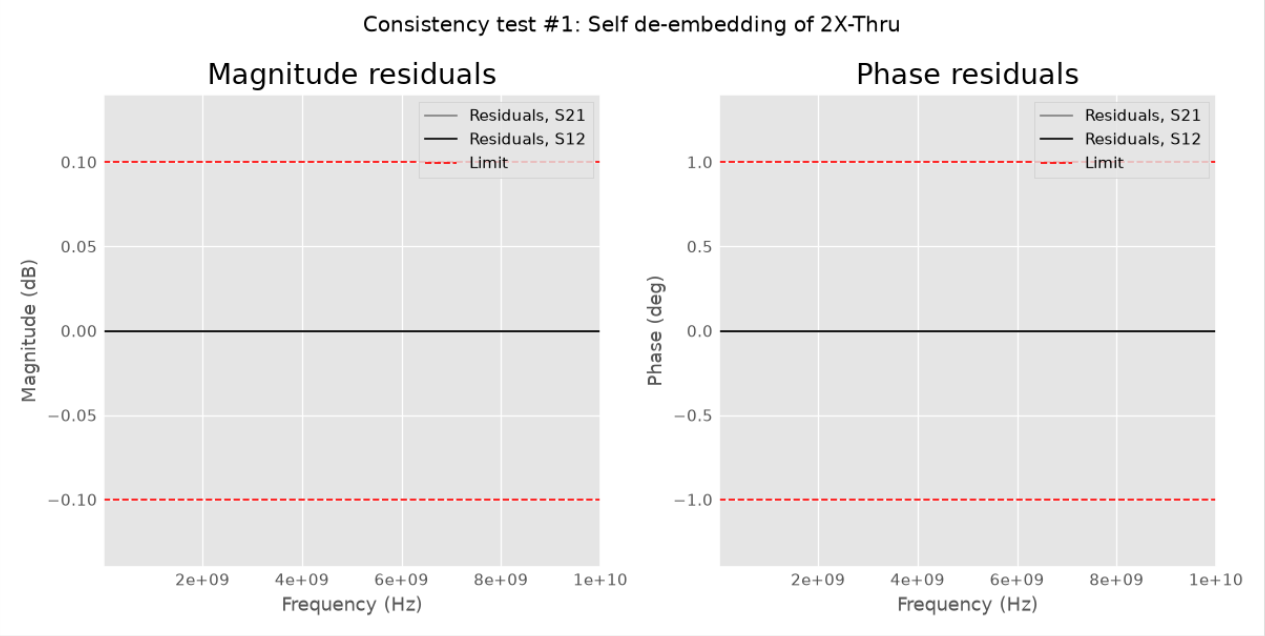
This method takes the 2x-Thru as input. It is a simple and efficient bisection algorithm that cannot correct for the difference of impedance between the lines of FIX-FIX and FIX-DUT-FIX. This unwanted difference should be minimized. It can appear because of materials inhomogeneities, manufacturing processes, or if the artifacts are not built on the same board.

The S-parameters bisection is done by time gating S11 and S22, taking the proper square root of the S21 corrected by return loss, and remixing the parameters according to the fixture signal flow graph. This method gives crude results but is robust.

IEEE 370 recommends the following consistency tests:

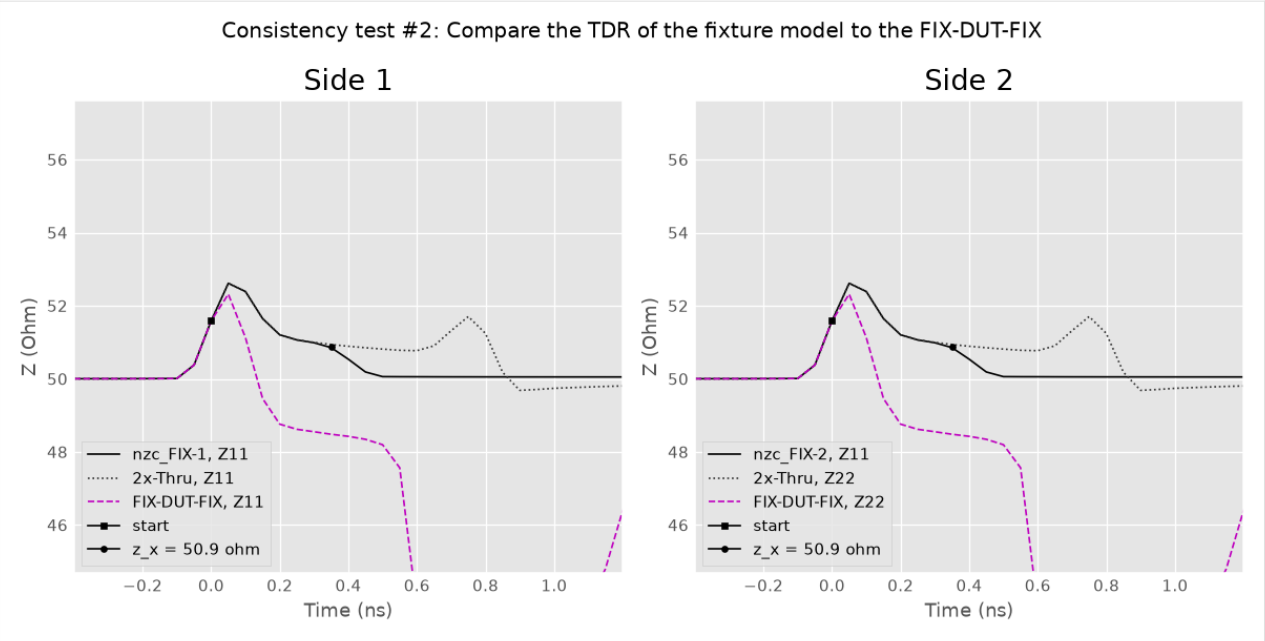
- Self de-embedding of 2x-Thru with absolute magnitude of residual insertion loss < ±0.1 dB and residual phase < ±1°
- Compare the TDR of the fixture model to the FIX-DUT-FIX

[10]:
```
dm_nzc = IEEEP370_SE_NZC_2xThru(dummy_2xthru = s2xthru, name = '2xthru')
nzc_fix1 = dm_nzc.s_side1
nzc_fix1.name = 'nzc_FIX-1'
nzc_fix2 = dm_nzc.s_side2
nzc_fix2.name = 'nzc_FIX-2'
nzc_d_dut = dm_nzc.deembed(fdf)
nzc_d_dut.name = 'nzc_DUT'
nzc_fix1_dc = IEEEP370.extrapolate_to_dc(nzc_fix1)
nzc_d_dut_dc = IEEEP370.extrapolate_to_dc(nzc_d_dut)

fig, ax = dm_nzc.plot_check_residuals()
```



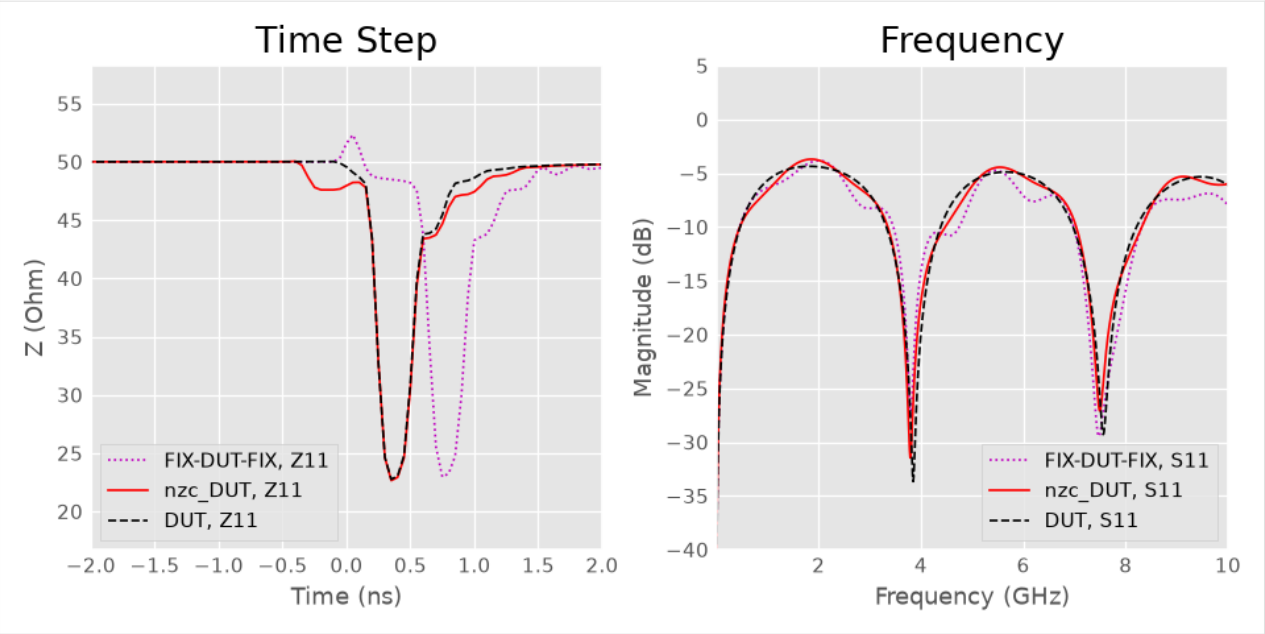
The agreement between fixtures models and 2xthru is excellent, as shown by the magnitude and phase of residuals being much smaller than IEEEP370 ±0.1 dB and ±1° limits.

[11]:
```
fig, ax = dm_nzc.plot_check_impedance(fdf)
```



Both fixtures show a close agreement with 2x-Thru from the start to the middle. However, the 5% artificial impedance mismatch added between 2x-Thru and FIX-DUT-FIX will deteriorate the DUT deembedding performance.

[12]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('Time Step')
fdf_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dotted', color = 'm')
nzc_d_dut_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'r')
dut_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dashed', color = 'k')
axs[0].set_xlim((-2, 2))
axs[0].legend(loc = 'lower left')

axs[1].set_title('Frequency')
fdf.plot_s_db(0, 0, ax = axs[1], color = 'm', linestyle = 'dotted')
nzc_d_dut.plot_s_db(0, 0, ax = axs[1], color = 'r')
dut.plot_s_db(0, 0, ax = axs[1], linestyle = 'dashed', color = 'k')
axs[1].set_ylim((-40, 5))
fig.tight_layout()
```



The NZC deembedding algorithm has removed the delay of the fixtures. As expected, the impedance mismatch between 2x-Thru and FIX-DUT-FIX deteriorates the de-embedding by leaving an impedance bounce before time zero in the time domain. It is also visible as a ripple in the frequency domain.

IEEEP370_SE_ZC_2xThru with impedance correction

This method takes 2x-Thru and the FIX-DUT-FIX as inputs. The algorithm computes the length of the fixtures by halving the delay of 2x-Thru in time domain transmission. The propagation constant `gamma` is also determined from the 2xThru. It then peels the FIX-DUT-FIX time domain impedance profile iteratively in cycles of determining start impedance and deembedding a single time sample long transmission line.

Because the 2x-Thru is used only for length and propagation constant determination, the effect of an impedance mismatch with the FIX-DUT-FIX is reduced. If another FIX-DUT-FIX has to be de-embedded with the same fixtures, an impedance mismatch between the two FIX-DUT-FIX would have an adverse effect on deembedding performance.

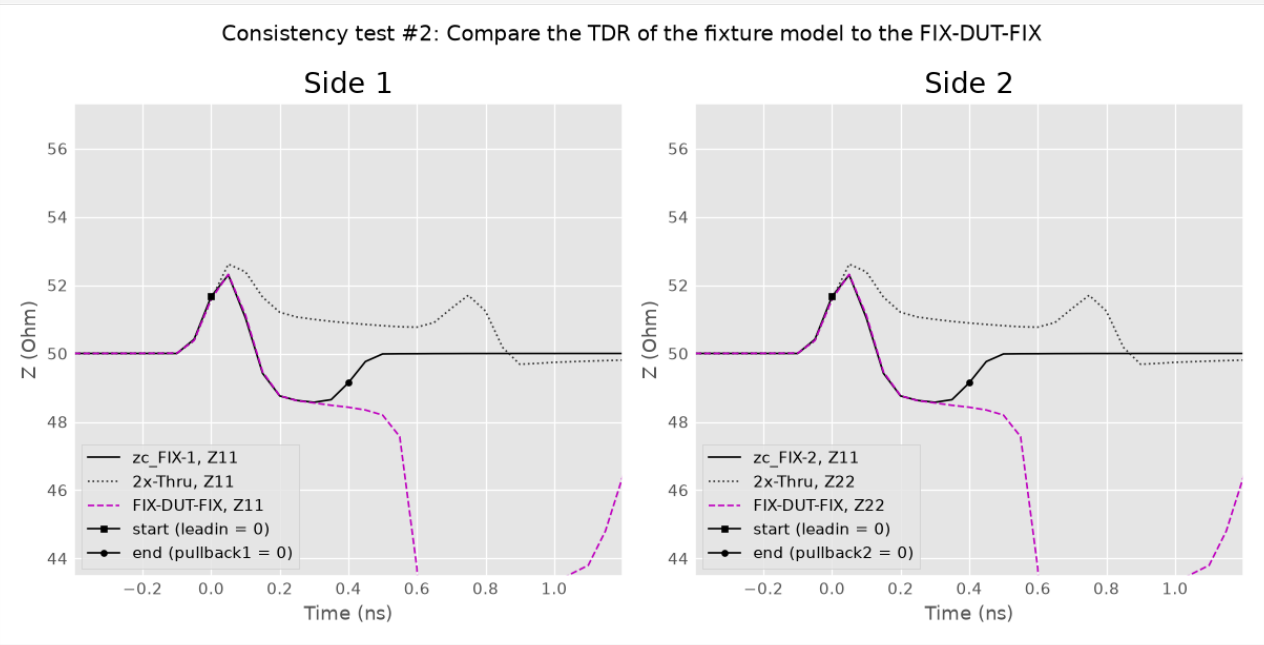
This method is more advanced than `IEEEP370_SE_NZC_2xThru` 2x-Thru S-parameters bisection. It has more options and often gives better results at the expense of more processing power.

[13]:
```
dm_zc  = IEEEP370_SE_ZC_2xThru(dummy_2xthru = s2xthru, dummy_fix_dut_fix = fdf,
                           bandwidth_limit = 10e9, pullback1 = 0, pullback2 = 0,
                           leadin = 0, NRP_enable = False,
                           name = 'zc2xthru')
zc_d_dut = dm_zc.deembed(fdf)
zc_d_dut.name = 'zc_DUT'
zc_fix1 = dm_zc.s_side1
zc_fix1.name = 'zc_FIX-1'
zc_fix2 = dm_zc.s_side2
zc_fix2.name = 'zc_FIX-2'
zc_fix1_dc = IEEEP370.extrapolate_to_dc(zc_fix1)
zc_d_dut_dc = IEEEP370.extrapolate_to_dc(zc_d_dut)

fig, ax = dm_zc.plot_check_residuals()
```

The agreement between fixtures models and 2xthru is good, as shown by the magnitude and phase of residuals being smaller than IEEE 370 ± 0.1 dB and $\pm 1°$ limits.

The residuals are higher than than with the NZC algorithm. This is the awaited result because the fixtures are modeled on the FIX-DUT-FIX impedance profile that has an artificially added impedance mismatch with the 2x-Thru.

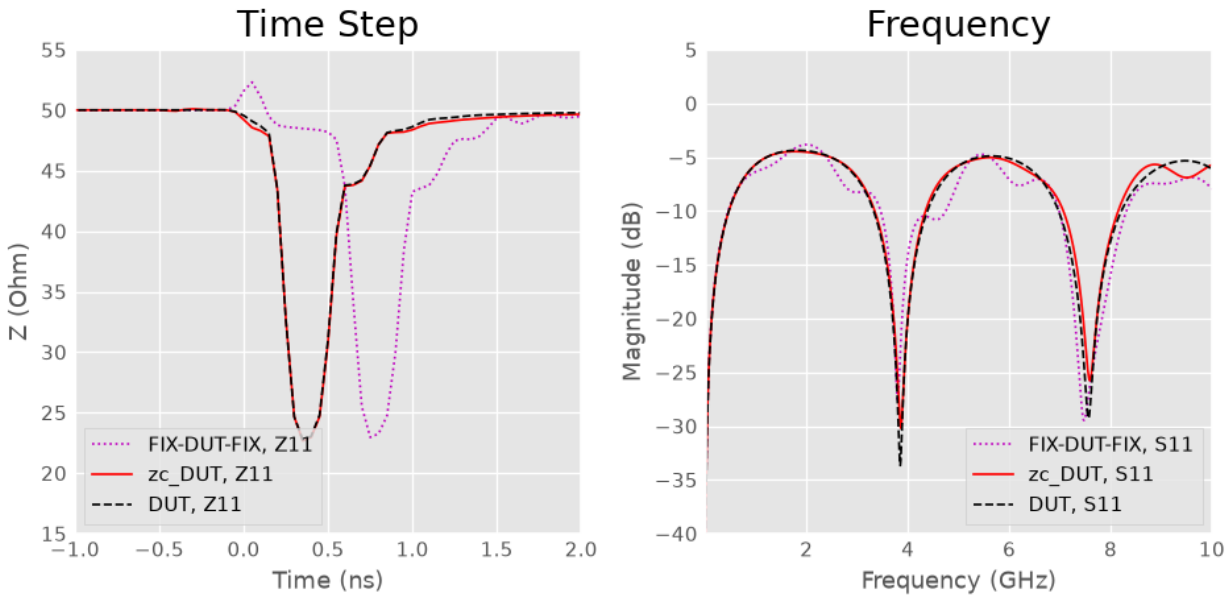
[14]:
```
fig, ax = dm_zc.plot_check_impedance()
```



Both fixtures show a close agreement with FIX-DUT-FIX from the start to the middle. The 2x-Thru was used for fixture length and propagation constant gamma determination only.

The `leadin` option is useful if there is a need to capture several non-reference impedance samples before the time zero. The `pullback1` and `pullback2` options can shorten the fixtures by an integer number of time samples.

[15]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('Time Step')
fdf_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dotted', color = 'm')
zc_d_dut_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'r')
dut_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dashed', color = 'k')
axs[0].set_xlim((-1, 2))
axs[0].set_ylim((15, 55))
axs[0].legend(loc = 'lower left')

axs[1].set_title('Frequency')
fdf.plot_s_db(0, 0, ax = axs[1], color = 'm', linestyle = 'dotted')
zc_d_dut.plot_s_db(0, 0, ax = axs[1], color = 'r')
dut.plot_s_db(0, 0, ax = axs[1], linestyle = 'dashed', color = 'k')
axs[1].set_ylim((-40, 5))
fig.tight_layout()
```



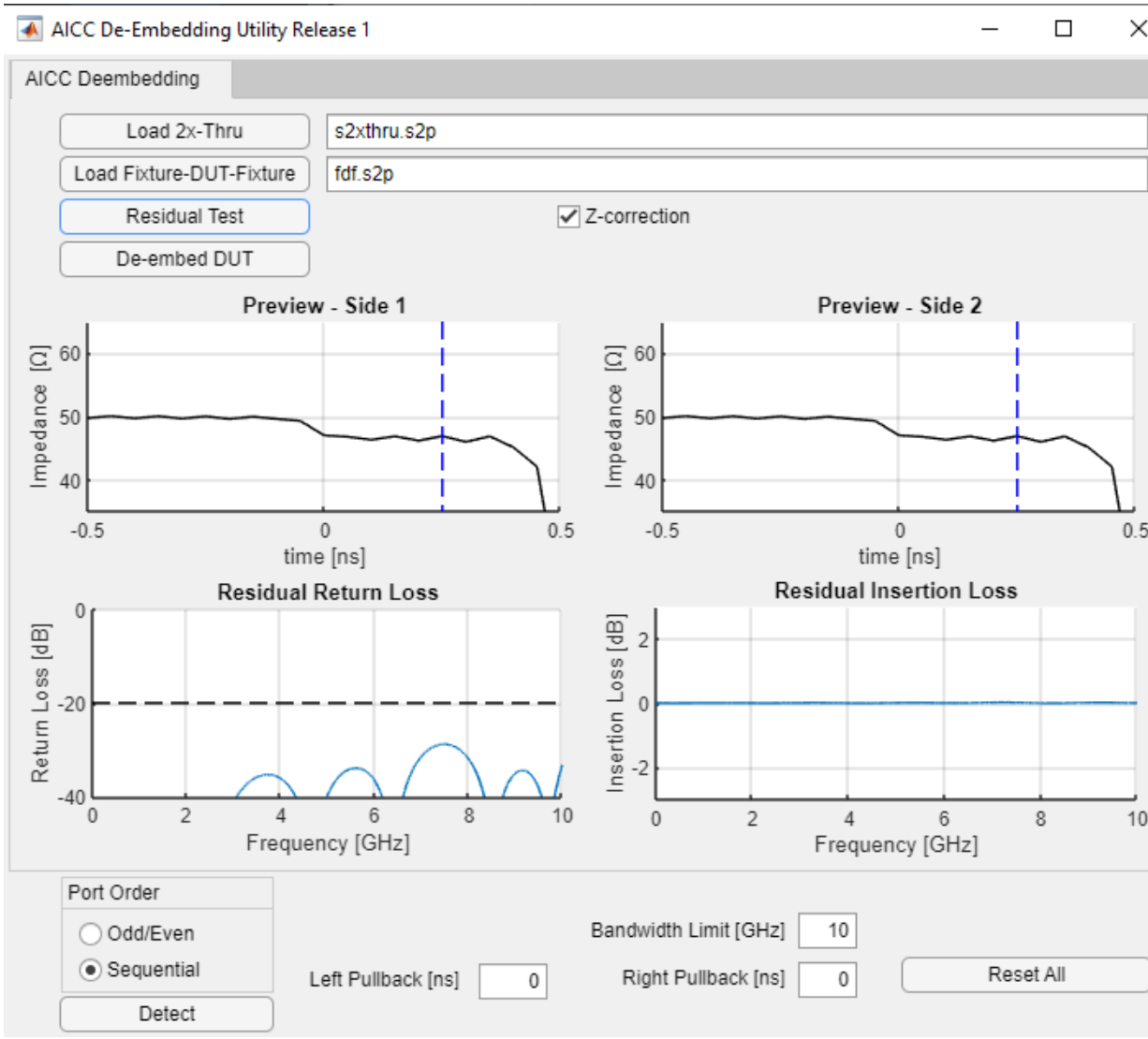
As expected, the ZC deembedding algorithm shows a better agreement with the reference DUT than NZC, both in the time and frequency domains. This is because it is not impacted by the impedance mismatch between 2xThru and FIX-DUT-FIX of this example. This impedance difference should be minimized as much as possible in practical designs.

Single Ended Comparison with AICC De-Embedding Utility

A set of reference Matlab or Octave codes that implement the IEEEP370 NZC and ZC deembedding algorithms are available with an open-source BSD-3-Clause license on the IEEE repository

However, not everyone has access to Matlab and RF Toolbox. Maybe, this is one of the reasons why you are reading this text, looking forward to using `scikit-rf` and Python.

A free compiled binary of Matlab routine with a GUI is available on Amphenol website with the name "ACS De-



embedding Utility".

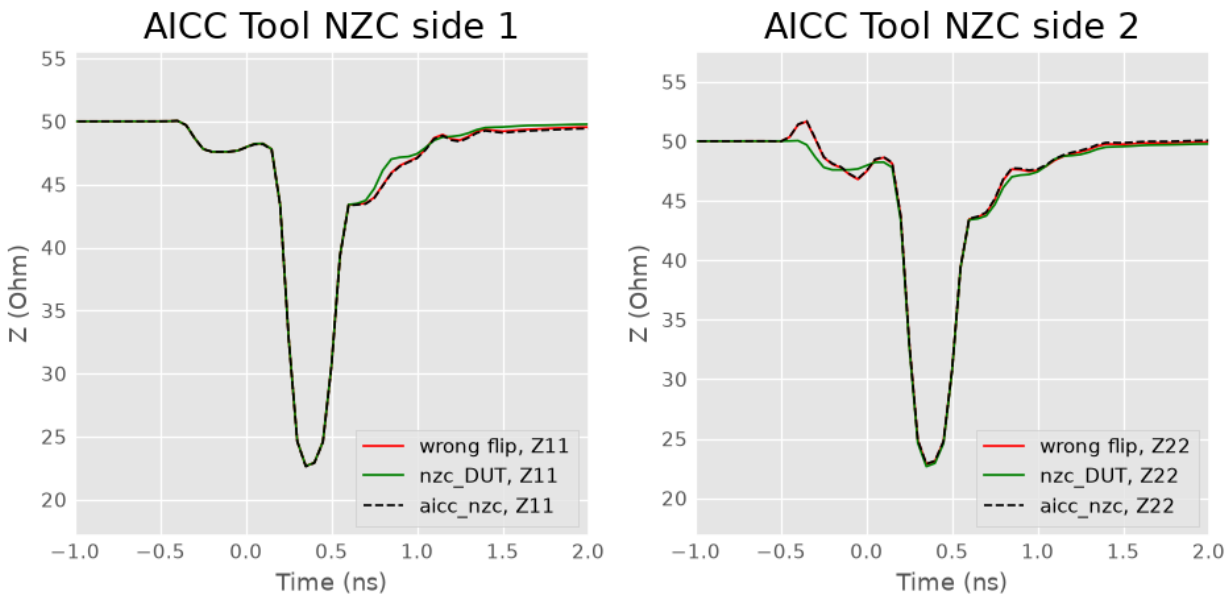
Let's compare the output of this tool on `scikit-rf` port of deembedding algorithms as a consistency check.

[16]:
```
# read AICC generated files
nzc_ref = rf.Network(directory + 'deembedded_SE_NZC_se_fdf.s2p')
nzc_ref.name = 'aicc_nzc'
zc_ref = rf.Network(directory + 'deembedded_SE_ZC_se_fdf.s2p')
zc_ref.name = 'aicc_zc'
nzc_ref_dc = IEEEP370.extrapolate_to_dc(nzc_ref)
zc_ref_dc = IEEEP370.extrapolate_to_dc(zc_ref)

# make purposely wrong NZC side 2 flip for comparison with AICC Tool
nzc_wrong_d_dut = dm_nzc.s_side1.inv ** fdf ** dm_nzc.s_side2.inv
nzc_wrong_d_dut.name = 'wrong flip'
nzc_wrong_d_dut_dc = IEEEP370.extrapolate_to_dc(nzc_wrong_d_dut)

# compute absolute differences
# division of complex reflexion coeff is like subtraction in dB and deg
delta_nzc = nzc_ref / nzc_wrong_d_dut
```

(continues on next page)

(continued from previous page)

```
delta_zc = zc_ref / zc_d_dut

# plot them all
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('AICC Tool NZC side 1')
nzc_wrong_d_dut_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'r')
nzc_d_dut_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'g')
nzc_ref_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dashed', color = 'k')
axs[0].legend(loc = 'lower right')
axs[0].set_xlim((-1, 2))

axs[1].set_title('AICC Tool NZC side 2')
nzc_wrong_d_dut_dc.plot_z_time_step(1, 1, ax = axs[1], color = 'r')
nzc_d_dut_dc.plot_z_time_step(1, 1, ax = axs[1], color = 'g')
nzc_ref_dc.plot_z_time_step(1, 1, ax = axs[1], linestyle = 'dashed', color = 'k')
axs[1].legend(loc = 'lower right')
axs[1].set_xlim((-1, 2))

plt.tight_layout()
```



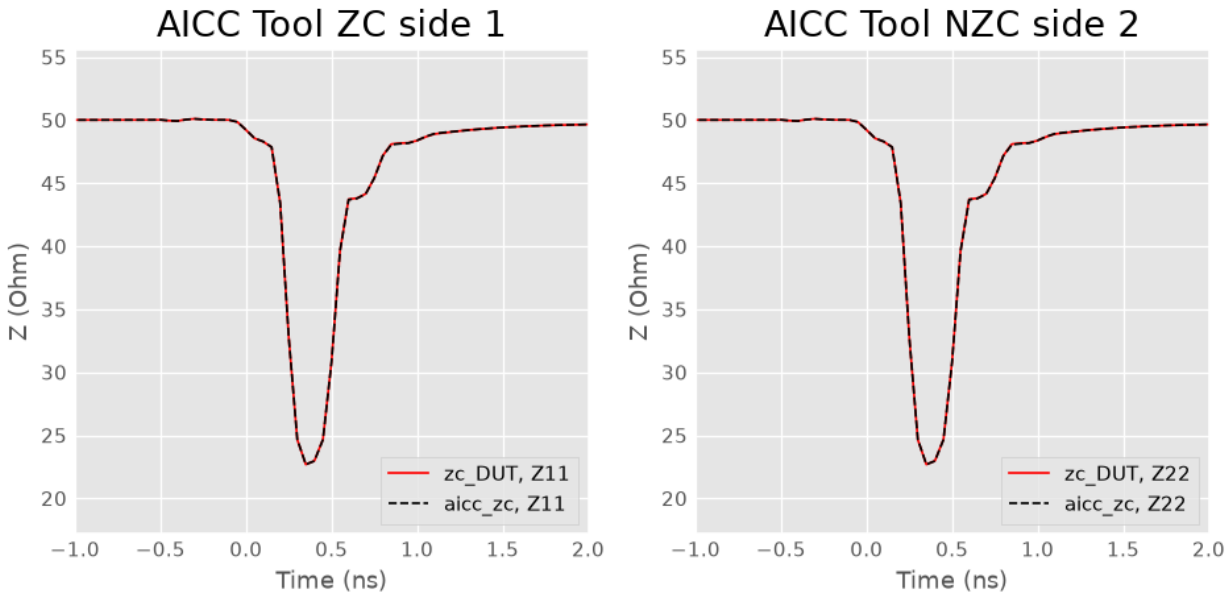
When comparing `scikit-rf` and `AICC` results, NZC algorithm exhibits a small discrepancy after the DUT section (green curve). This is because `AICC` FIX-2 has a wrong flip, which is demonstrated by the results of side 1 and side 2 being different for a symmetrical FIX-DUT-FIX (dashed black curve). If this error is propagated to `scikit-rf` for comparison's sake, the agreement is a perfect fit (red curve). The version of `AICC` used is 1.0.0.

[17]:
```
# plot them all
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('AICC Tool NZC side 1')

axs[0].set_title('AICC Tool ZC side 1')
zc_d_dut_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'r')
zc_ref_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dashed', color = 'k')
```

(continues on next page)

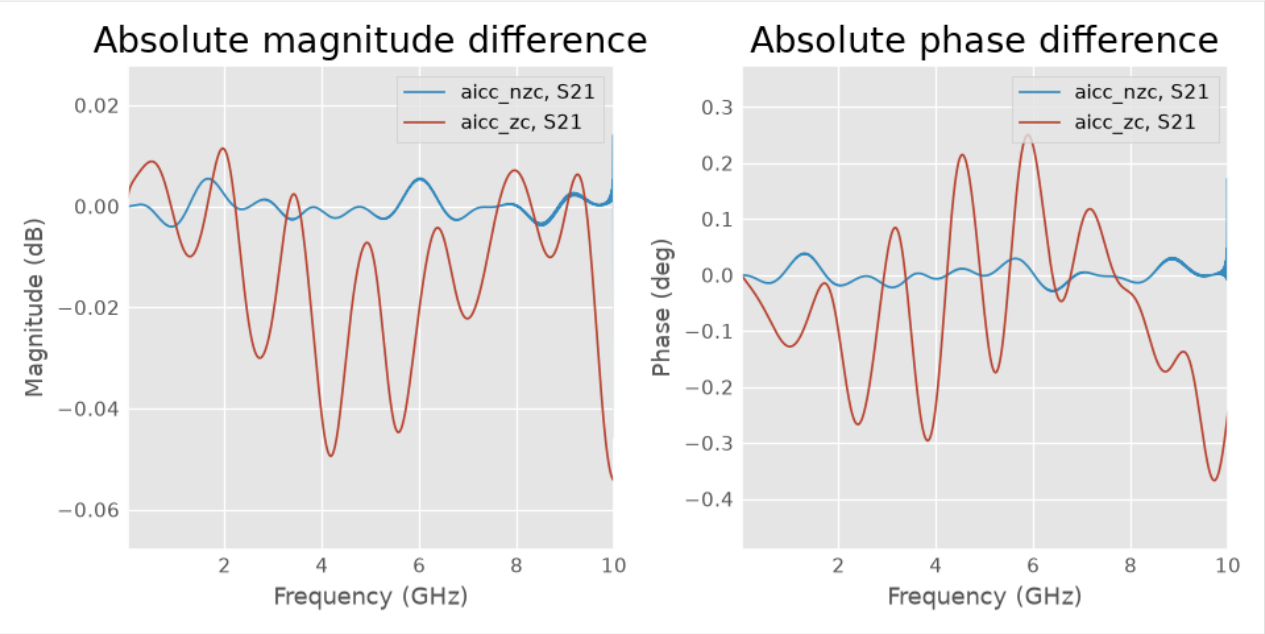
(continued from previous page)

```
axs[0].legend(loc = 'lower right')
axs[0].set_xlim((-1, 2))

axs[1].set_title('AICC Tool NZC side 2')
zc_d_dut_dc.plot_z_time_step(1, 1, ax = axs[1], color = 'r')
zc_ref_dc.plot_z_time_step(1, 1, ax = axs[1], linestyle = 'dashed', color = 'k')
axs[1].legend(loc = 'lower right')
axs[1].set_xlim((-1, 2))

plt.tight_layout()
```



When comparing `scikit-rf` and `AICC` results, ZC is visually a perfect fit in the time domain for both sides.

[18]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('Absolute magnitude difference')
delta_nzc.plot_s_db(1,0, ax = axs[0])
delta_zc.plot_s_db(1,0, ax = axs[0])
axs[0].legend(loc = 'upper right')

axs[1].set_title('Absolute phase difference')
delta_nzc.plot_s_deg(1,0, ax = axs[1])
delta_zc.plot_s_deg(1,0, ax = axs[1])
axs[1].legend(loc = 'upper right')

plt.tight_layout()
```



Both NZC (with flip error) and ZC absolute differences of magnitude and phase are within ± 0.06 dB and $\pm 0.4°$ up to 10 GHz, this is excellent.

In conclusion, for the single-ended case studied, `scikit-rf` gives consistent results with `AICC De-embedding Utility` implementation of IEEE 370 deembedding routines. It also solves the NZC FIX-2 flip error in `AICC De-embedding Utility` version 1.0.0.

Mixed mode

ZC and NZC deembedding algorithms are compatible with differential 4-port networks. Single-ended to generalized mixed-modes transformation is used to separate differential and common modes subnetworks and model the fixture individually. The differential and common modes fixtures are then merged and transformed into a 4-port single-ended again. This approach assumes differential to common mode transformation is insignificant.

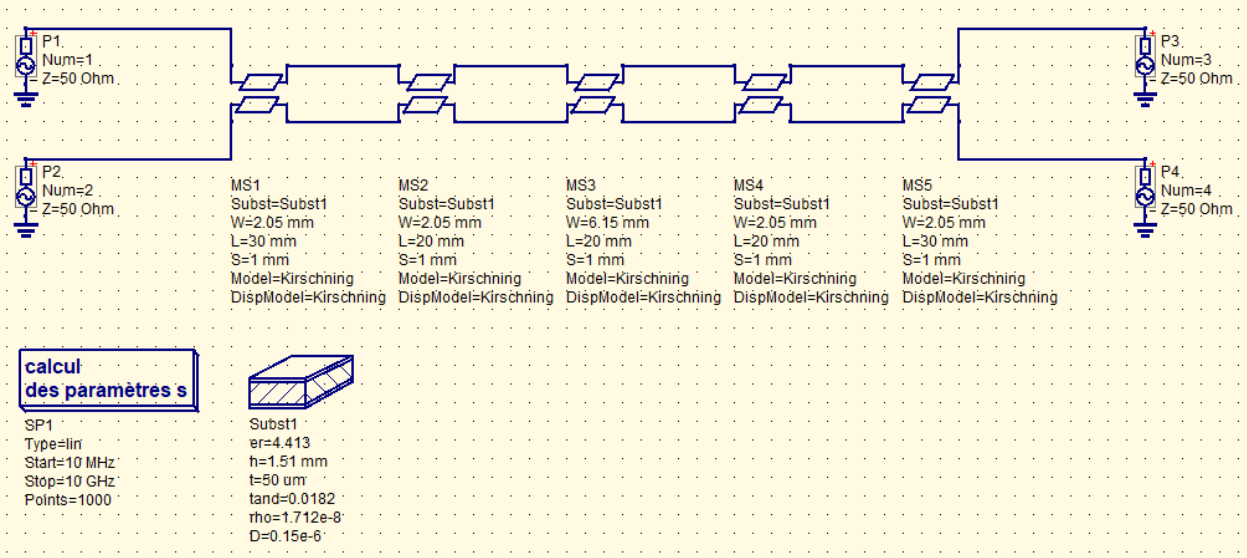
Mixed mode simulation of 2xThru, DUT and Fixture-DUT-Fixture

We use Qucs to simulate coupled microstrip line artifacts. This is a free simulator that can generate S-parameters of equation-based RF devices, among other things.

- `diff_dut` is a a Beatty structure with a 3xWidth coupled microstrip section connected left and right by two uniform 1xWidth coupled microstrip lines.
- `diff_fdf` is FIX-DUT-FIX, prolonging the DUT left and right with a coupled microstrip line and a planar-to-coaxial connector.
- `diff_2xthru` is FIX-FIX and is the cascade of the two prolonging lines and connectors without the DUT. For example purpose, the width of the lines is altered to show the effect of impedance mismatch on the deembedding process.

The target is to get the left and right fixtures models and retrieve the DUT by deembedding them from FIX-DUT-FIX.

The files and `qucs` sources are located in the directory `ieeep370deembedding` next to this notebook.



In the figure below, the time shift of DUT caused by the fixtures is visible on FIX-DUT-FIX trace. The 2x-Thru trace shows the awaited impedance mismatch with the FIX-DUT-FIX. The connectors add an impedance bump at the ends of 2x-Thru and FIX-DUT-FIX impedance step response. The reference DUT is the Beatty structure alone, without fixtures.

The common mode impedances are significantly mismatched from 25 ohm with this coupled lines geometry.

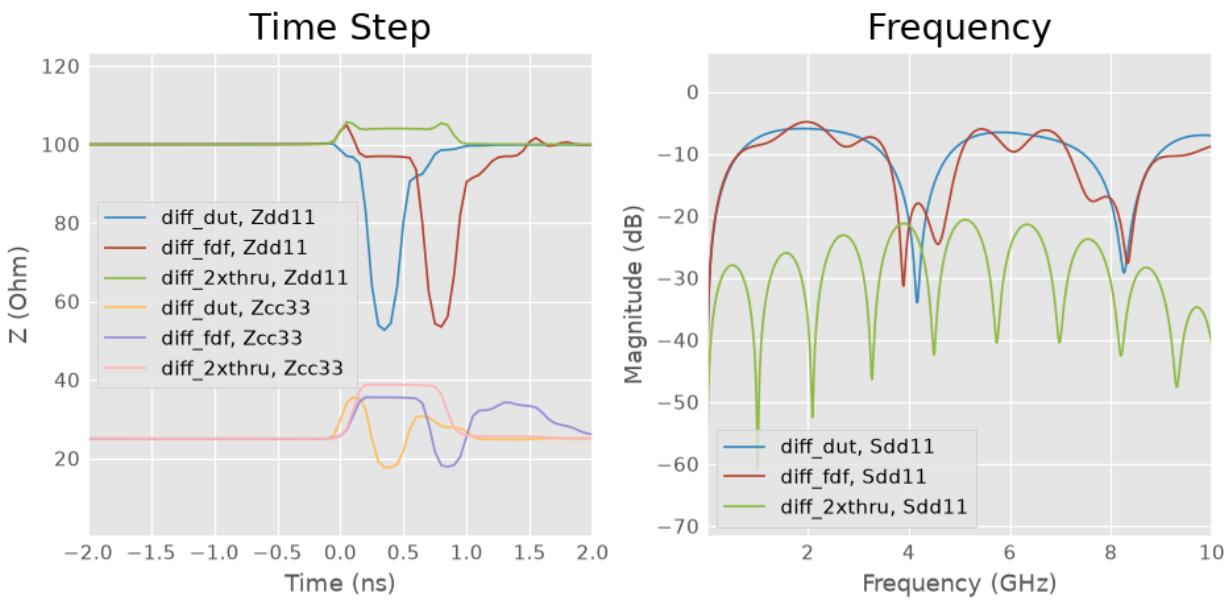
[19]:
```
# load single-ended data
se_dut = rf.Network(directory + 'diff_dut.s4p')
se_2xthru = rf.Network(directory + 'diff_2xthru_with_connectors.s4p')
se_2xthru.name = 'diff_2xthru'
#se_2xthru = se_2xthru.delay(10, 'ps', 3) # skew
se_fdf = rf.Network(directory + 'diff_fdf_with_connectors.s4p')
se_fdf.name = 'diff_fdf'
#se_fdf = se_fdf.delay(10, 'ps', 0) # skew

# transform to mixed-modes
mm_dut = se_dut.copy()
mm_dut.se2gmm(p = 2)
mm_2xthru = se_2xthru.copy()
mm_2xthru.se2gmm(p = 2)
mm_fdf = se_fdf.copy()
mm_fdf.se2gmm(p = 2)

# extrapolate to DC for time step
mm_dut_dc = IEEEP370.extrapolate_to_dc(mm_dut)
mm_2xthru_dc = IEEEP370.extrapolate_to_dc(mm_2xthru)
mm_fdf_dc = IEEEP370.extrapolate_to_dc(mm_fdf)

# set True to write .s4p files
if False:
    connectors = concat_ports([connector, connector], port_order = 'second')
    se_2xthru = connectors ** rf.Network(directory + 'diff_2xthru.s4p') ** connectors
    se_fdf = connectors ** rf.Network(directory + 'diff_fdf.s4p') ** connectors
    se_2xthru.write_touchstone(directory + 'diff_2xthru_with_connectors.s4p')
    se_fdf.write_touchstone(directory + 'diff_fdf_with_connectors.s4p')
```

(continues on next page)

(continued from previous page)

```
# looking at the newtorks
fig, axs = plt.subplots(1, 2, figsize=(8, 4))

axs[0].set_title('Time Step')
mm_dut_dc.plot_z_time_step(0, 0, ax = axs[0])
mm_fdf_dc.plot_z_time_step(0, 0, ax = axs[0])
mm_2xthru_dc.plot_z_time_step(0, 0, ax = axs[0])
mm_dut_dc.plot_z_time_step(2, 2, ax = axs[0])
mm_fdf_dc.plot_z_time_step(2, 2, ax = axs[0])
mm_2xthru_dc.plot_z_time_step(2, 2, ax = axs[0])
axs[0].set_xlim((-2, 2))
axs[0].legend(loc = 'center left')

axs[1].set_title('Frequency')
mm_dut.plot_s_db(0, 0, ax = axs[1])
mm_fdf.plot_s_db(0, 0, ax = axs[1])
mm_2xthru.plot_s_db(0, 0, ax = axs[1])
axs[1].legend(loc = 'lower left')

fig.tight_layout()
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪mathFunctions.py:303: RuntimeWarning: divide by zero encountered in log10
  out = 20 * np.log10(z)
```



Mixed mode S-parameters quality checking

First, check the quality metrics of the data to ensure they do not violate passivity, reciprocity, or causality. Initial quality checking in the frequency domain results in percentage scores and is informative only. The results are evaluated as good, acceptable, inconclusive or poor. Inconclusive or poor results may indicate the need to recalibrate the VNA and perform the measurements again.

This quick check may raise potential issues that question measurement, simulation, or de-embedding process.

Only the differential and the common modes are checked.

[20]:
```
fd_qm = IEEEP370_FD_QM()
print(mm_2xthru.name)
qm_2xthru = fd_qm.check_mm_quality(se_2xthru)
fd_qm.print_qm(qm_2xthru)
print(mm_fdf.name)
qm_fdf = fd_qm.check_mm_quality(se_fdf)
fd_qm.print_qm(qm_fdf)
```

```
diff_2xthru
Differential mode
causality is good (100.00%)
passivity is good (100.00%)
reciprocity is good (100.00%)
Common mode
causality is good (100.00%)
passivity is good (100.00%)
reciprocity is good (100.00%)
diff_fdf
Differential mode
causality is good (100.00%)
passivity is good (100.00%)
reciprocity is good (100.00%)
Common mode
causality is good (100.00%)
passivity is good (100.00%)
reciprocity is good (100.00%)
```

All the results are in the best class, which is not surprising for synthetic data.

Application-based quality checking in the time domain targets interconnect applications with specified data rate and rise time. It is an advanced process that targets giving a physical estimation in millivolt based on peak distortion analysis between the input data and ideal re-constructed data.

Only the differential and the common modes are checked.

[21]:
```
td_qm = IEEEP370_TD_QM(1e9, # bps
                       32,  # UI samples
                       0.4, # rise-time in fraction of UI
                       1,   # Gaussian pulse,
                       2,   # zero padding extrapolation
                       verbose = False)
print(mm_2xthru.name)
qm_2xthru = td_qm.check_mm_quality(se_2xthru)
td_qm.print_qm(qm_2xthru)
print(mm_fdf.name)
qm_fdf = td_qm.check_mm_quality(se_fdf)
td_qm.print_qm(qm_fdf)
```

```
diff_2xthru
Differential mode
causality in the time domain is good (0.65 mV)
passivity in the time domain is good (0.0 mV)
```

(continues on next page)

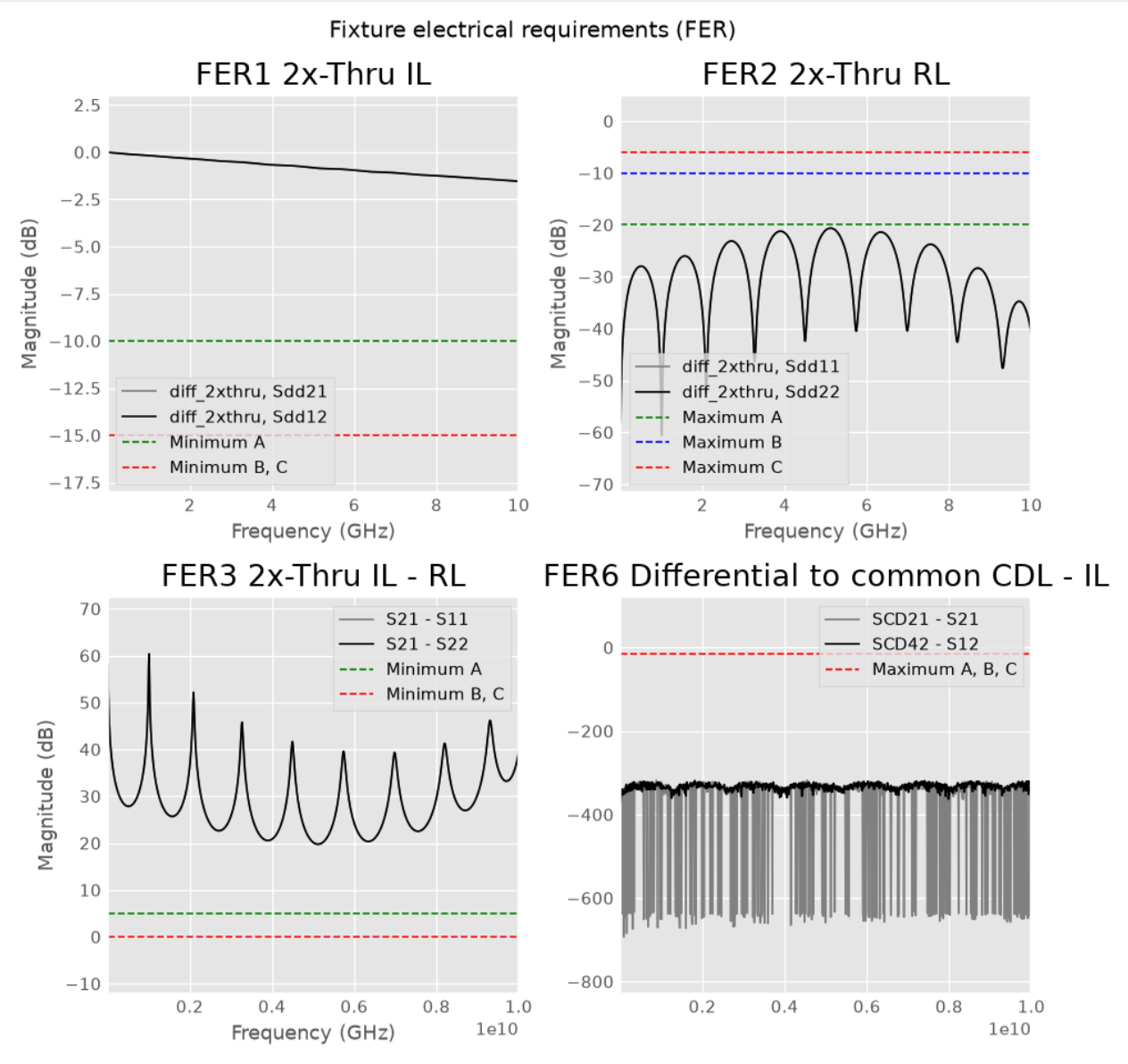
(continued from previous page)

```
reciprocity in the time domain is good (0.0 mV)
Common mode
causality in the time domain is good (1.95 mV)
passivity in the time domain is good (0.0 mV)
reciprocity in the time domain is good (0.0 mV)
diff_fdf
Differential mode
causality in the time domain is good (3.1 mV)
passivity in the time domain is good (0.0 mV)
reciprocity in the time domain is good (0.0 mV)
Common mode
causality in the time domain is good (2.3 mV)
passivity in the time domain is good (0.0 mV)
reciprocity in the time domain is good (0.0 mV)
```

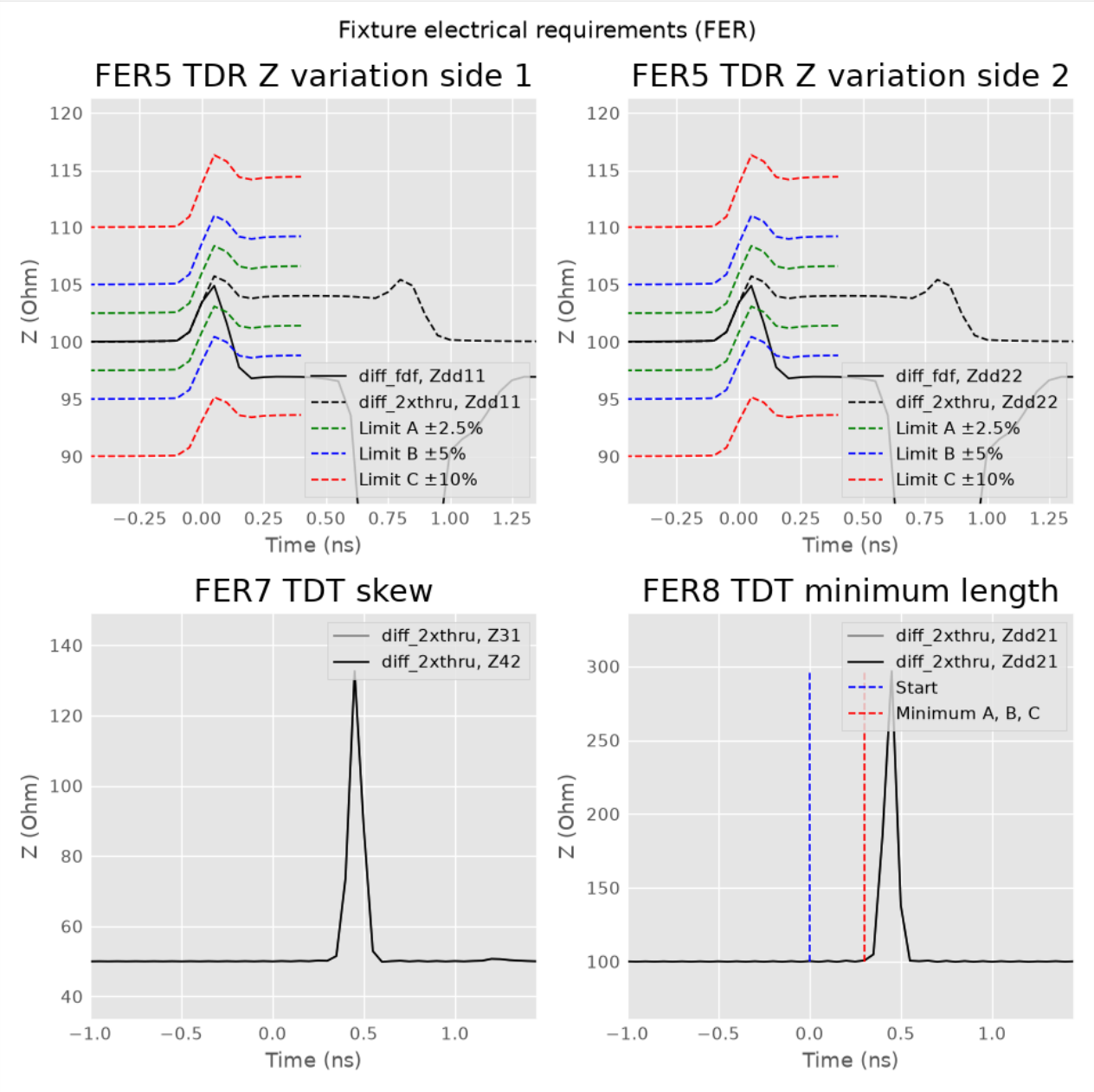
All the results are in the best class, which is not surprising for synthetic data.

Before exercising the deembedding algorithms, the compliance of input data with IEEE 370 fixture electrical requirements (FER) shall be verified.

[22]:
```
fer = IEEEP370_FER()
fig = fer.plot_fd_mm_fer(se_2xthru)
```



[23]:
```
fig = fer.plot_td_mm_fer(se_2xthru, se_fdf)
```



The performance of the 2x-Thru is deemed class C compliant for frequencies below 10 GHz. This is due to the artificial impedance mismatch between 2x-Thru and FIX-DUT-FIX that fails the FER5 A and B limits.

IEEEP370_MM_NZC_2xThru without impedance correction

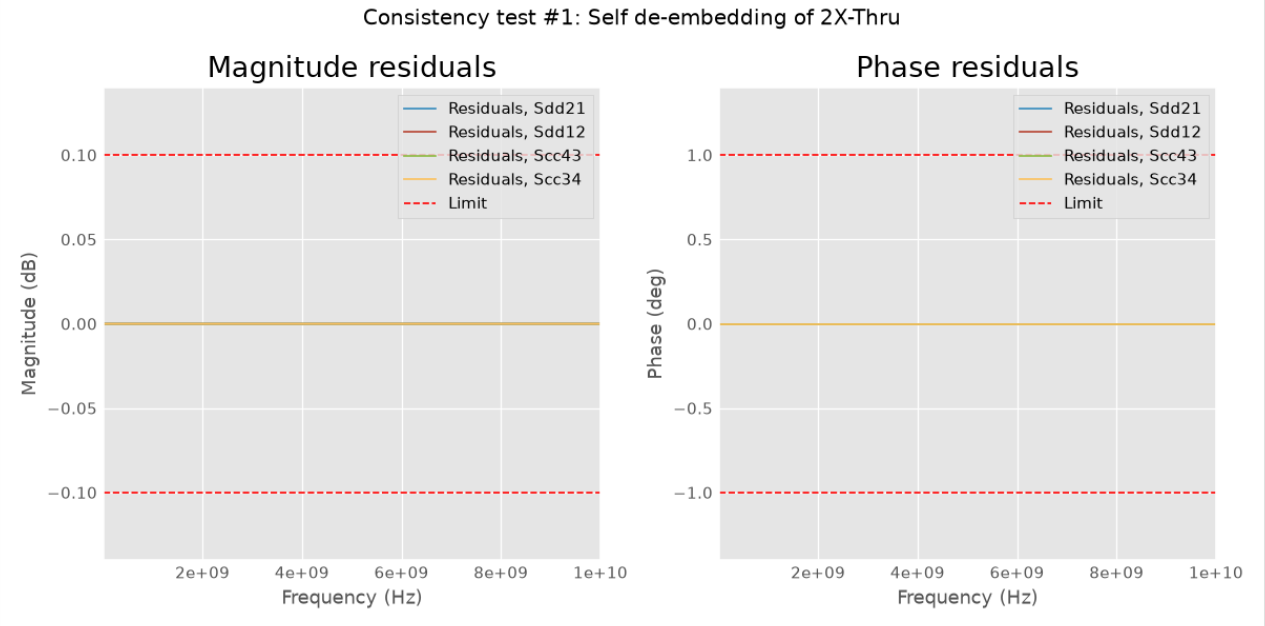
This method takes the 2x-Thru as input. It is a simple and efficient bisection algorithm that cannot correct for the difference of impedance between the lines of FIX-FIX and FIX-DUT-FIX. This unwanted difference should be minimized. It can appear because of materials inhomogeneities, manufacturing processes, or if the artifacts are not built on the same board.

The S-parameters bisection is done by time gating S11 and S22, taking the proper square root of the S21 corrected by return loss, and remixing the parameters according to the fixture signal flow graph. This method gives crude results but is robust.

IEEE 370 recommends the following consistency tests:

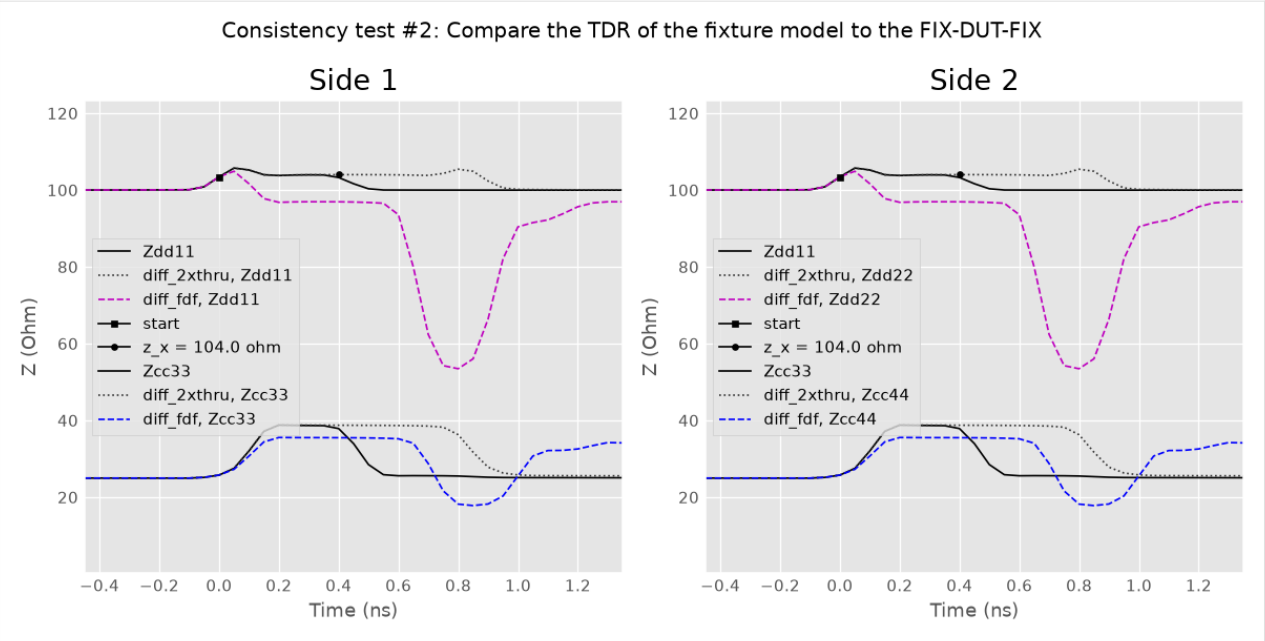
- Self de-embedding of 2x-Thru with absolute magnitude of residual insertion loss < ±0.1 dB and residual phase < ±1°
- Compare the TDR of the fixture model to the FIX-DUT-FIX

[24]:
```
# de-embedding
z0 = 50
dm_mmnzc  = IEEEP370_MM_NZC_2xThru(dummy_2xthru = se_2xthru, z0 = z0, name = 'mmnzc')
mm_nzc_side1 = dm_mmnzc.se_side1.copy()
mm_nzc_side1.name = 'FIX-1-2'
mm_nzc_side1.se2gmm(p = 2)
se_d_dut_nzc = dm_mmnzc.deembed(se_fdf)
se_d_dut_nzc.name = 'd_dut_nzc'
mm_d_dut_nzc = se_d_dut_nzc.copy()
mm_d_dut_nzc.se2gmm(p = 2)
mm_nzc_side1_dc = IEEEP370.extrapolate_to_dc(mm_nzc_side1)
mm_d_dut_nzc_dc = IEEEP370.extrapolate_to_dc(mm_d_dut_nzc)

fig, ax = dm_mmnzc.plot_check_residuals()
```



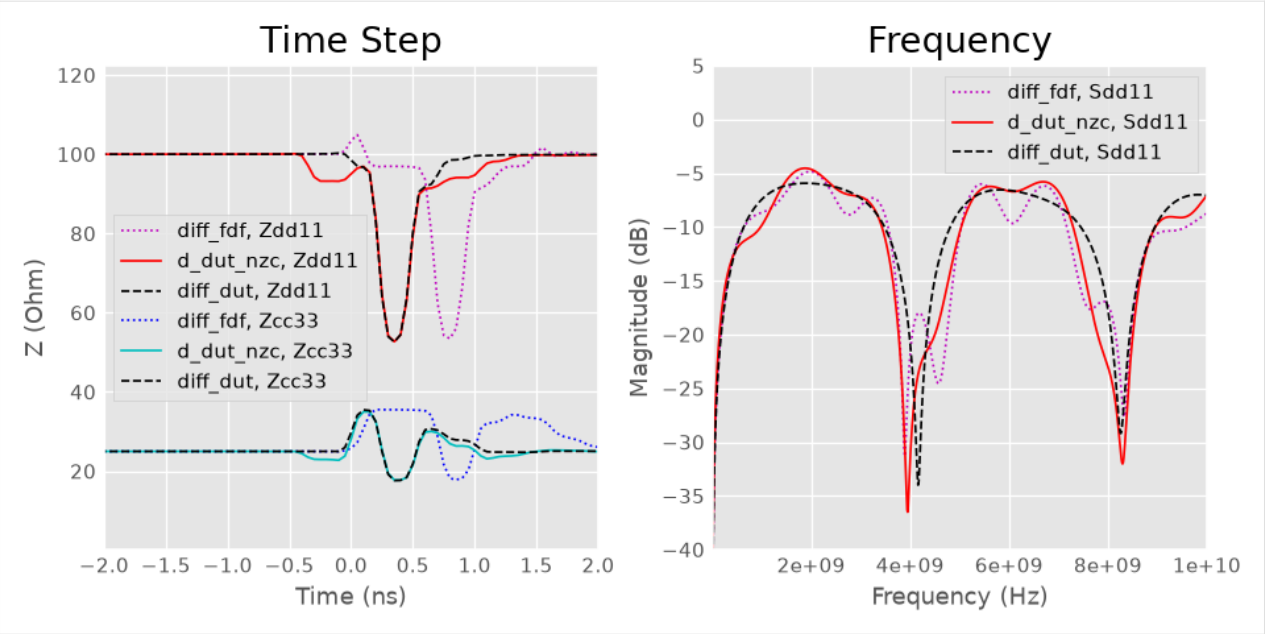
The agreement between fixtures models and 2xthru is excellent, as shown by the magnitude and phase of residuals being much smaller than IEEEP370 ±0.1 dB and ±1° limits.

[25]:
```
fig, ax = dm_mmnzc.plot_check_impedance(se_fdf)
```



Both fixtures show a close agreement with 2x-Thru from the start to the middle. However, the artificial impedance mismatch added between 2x-Thru and FIX-DUT-FIX will deteriorate the DUT deembedding performance.

[26]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('Time Step')
mm_fdf_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dotted', color = 'm')
mm_d_dut_nzc_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'r')
mm_dut_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dashed', color = 'k')
mm_fdf_dc.plot_z_time_step(2, 2, ax = axs[0], linestyle = 'dotted', color = 'b')
mm_d_dut_nzc_dc.plot_z_time_step(2, 2, ax = axs[0], color = 'c')
mm_dut_dc.plot_z_time_step(2, 2, ax = axs[0], linestyle = 'dashed', color = 'k')
axs[0].set_xlim((-2, 2))
axs[0].legend(loc = 'center left')

axs[1].set_title('Frequency')
mm_fdf.plot_s_db(0, 0, ax = axs[1], color = 'm', linestyle = 'dotted')
mm_d_dut_nzc.plot_s_db(0, 0, ax = axs[1], color = 'r')
mm_dut.plot_s_db(0, 0, ax = axs[1], linestyle = 'dashed', color = 'k')
axs[1].set_ylim((-40, 5))
fig.tight_layout()
```



The NZC deembedding algorithm has removed the delay of the fixtures. As expected, the impedance mismatch between 2x-Thru and FIX-DUT-FIX deteriorates the de-embedding by leaving an impedance bounce before time zero in the time domain. It is also visible as a ripple in the frequency domain.

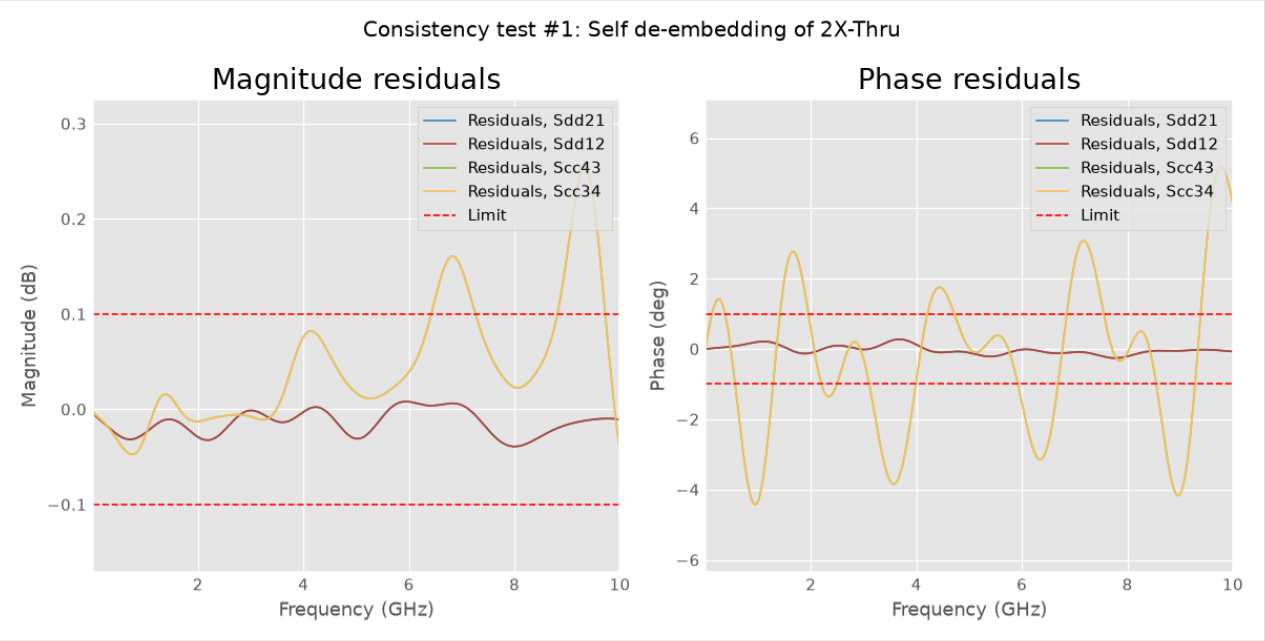
IEEEP370_MM_ZC_2xThru with impedance correction

This method take 2x-Thru and FIX-DUT-FIX as inputs. It makes a correction for the (unwanted) difference of impedance between the lines of FIX-FIX and FIX-DUT-FIX.

The algorithm computes the length of the fixtures by halving the delay of 2x-Thru in time domain transmission. The propagation constant gamma is also determined from the 2xThru. It then peels the FIX-DUT-FIX time domain impedance profile iteratively in cycles of determining start impedance and deembedding a single time sample long transmission line.

IEEEP370_MM_ZC_2xThru has more options and often give better results than IEEEP370_MM_NZC_2xThru, but it consumes more processing power.

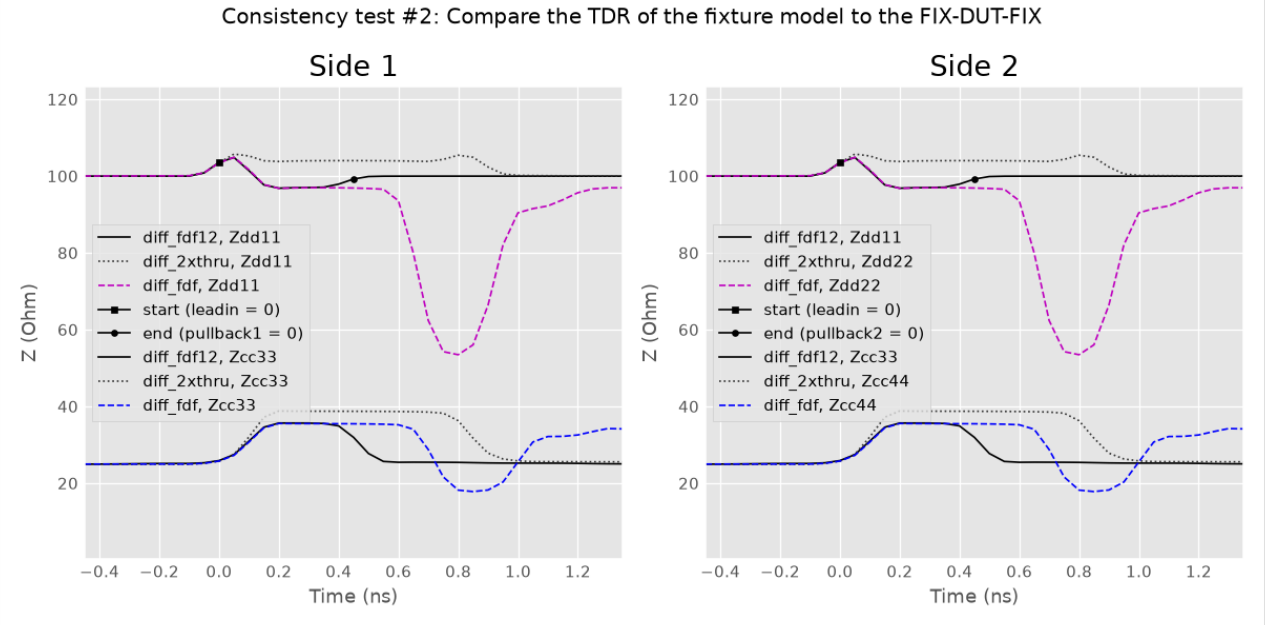
[27]:
```
# de-embedding
dm_mmzc  = IEEEP370_MM_ZC_2xThru(dummy_2xthru = se_2xthru, dummy_fix_dut_fix = se_fdf,
                         bandwidth_limit = 10e9, pullback1 = 0, pullback2 = 0,
                         leadin = 0, NRP_enable = False, name = 'mmzc')
mm_zc_side1 = dm_mmzc.se_side1.copy()
mm_zc_side1.name = 'FIX-1-2'
mm_zc_side1.se2gmm(p = 2)
se_d_dut_zc = dm_mmzc.deembed(se_fdf)
se_d_dut_zc.name = 'd_dut_zc'
mm_d_dut_zc = se_d_dut_zc.copy()
mm_d_dut_zc.se2gmm(p = 2)
mm_zc_side1_dc = IEEEP370.extrapolate_to_dc(mm_zc_side1)
mm_d_dut_zc_dc = IEEEP370.extrapolate_to_dc(mm_d_dut_zc)

fig, ax = dm_mmzc.plot_check_residuals()
```



The agreement between fixtures models and 2xthru is good, as shown by the magnitude and phase of residuals differential mode being smaller than IEEE 370 ± 0.1 dB and $\pm 1°$ limits. The common mode residuals are within ± 0.3 dB and $\pm 6°$ which is higher but not catastrophic.

The residuals are higher than than with the NZC algorithm. This is the awaited result because the fixtures are modeled on the FIX-DUT-FIX impedance profile that has an artificially added impedance mismatch with the 2x-Thru. This is especially true for common mode.

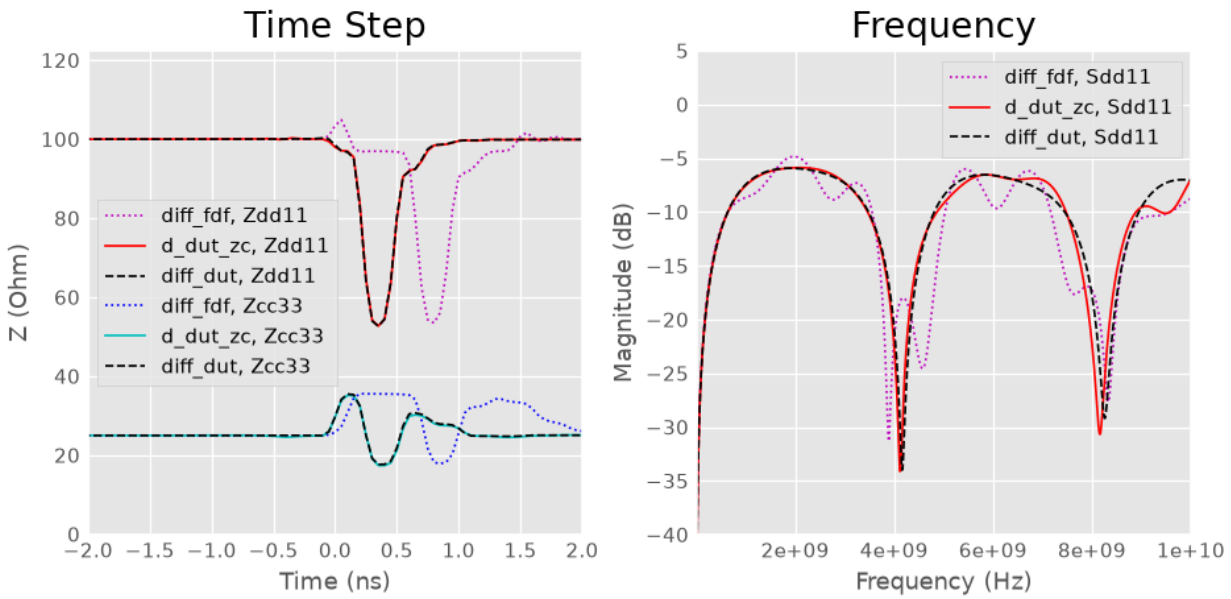
[28]:
```
fig, ax = dm_mmzc.plot_check_impedance()
```



Both fixtures show a close agreement with FIX-DUT-FIX from the start to the middle. The 2x-Thru was used for fixture length and propagation constant gamma determination only.

The `leadin` option is useful if there is a need to capture several non-reference impedance samples before the time zero. The `pullback1` and `pullback2` options can shorten the fixtures by an integer number of time samples.

[29]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('Time Step')
mm_fdf_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dotted', color = 'm')
mm_d_dut_zc_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'r')
mm_dut_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dashed', color = 'k')
mm_fdf_dc.plot_z_time_step(2, 2, ax = axs[0], linestyle = 'dotted', color = 'b')
mm_d_dut_zc_dc.plot_z_time_step(2, 2, ax = axs[0], color = 'c')
mm_dut_dc.plot_z_time_step(2, 2, ax = axs[0], linestyle = 'dashed', color = 'k')
axs[0].set_xlim((-2, 2))
axs[0].legend(loc = 'center left')

axs[1].set_title('Frequency')
mm_fdf.plot_s_db(0, 0, ax = axs[1], color = 'm', linestyle = 'dotted')
mm_d_dut_zc.plot_s_db(0, 0, ax = axs[1], color = 'r')
mm_dut.plot_s_db(0, 0, ax = axs[1], linestyle = 'dashed', color = 'k')
axs[1].set_ylim((-40, 5))
fig.tight_layout()
```



As expected, the ZC deembedding algorithm shows a better agreement with the reference DUT than NZC, both in the time and frequency domains. This is because it is not impacted by the impedance mismatch between 2xThru and FIX-DUT-FIX of this example. This impedance difference should be minimized as much as possible in practical designs.

Mixed mode comparison with AICC De-Embedding Utility

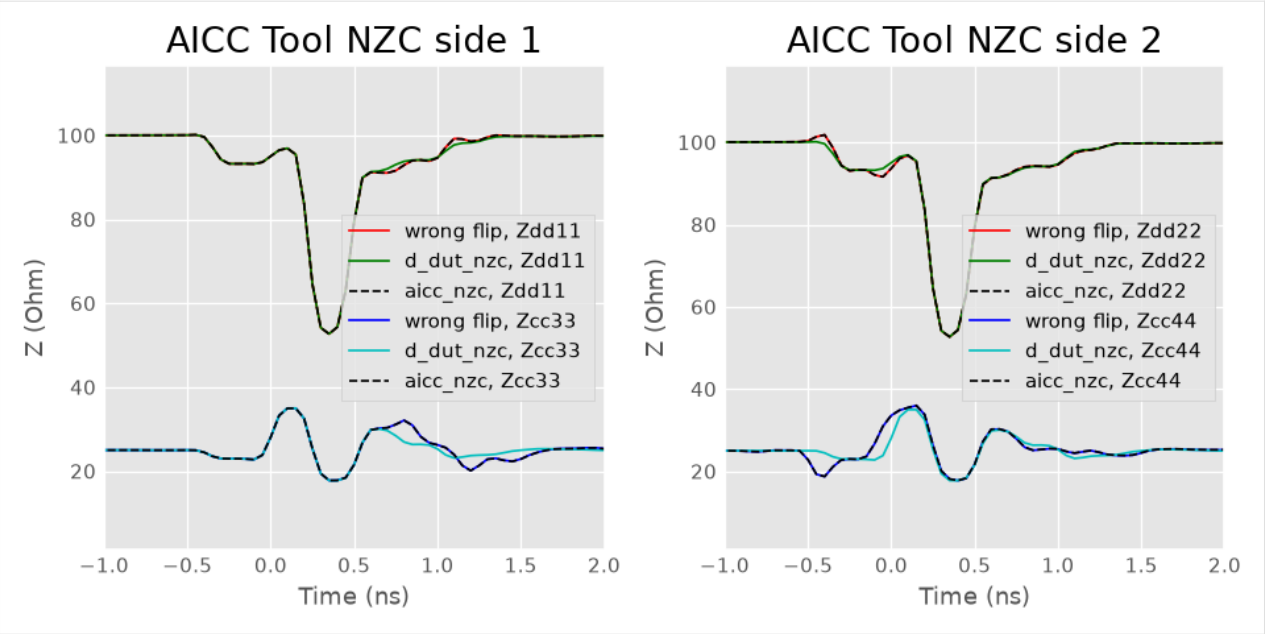
Let's compare the deembedded DUTs from `scikit-rf` and `AICC De-Embedding Utility` to conclude this notebook.

[30]:
```
# load single-ended data
se_ref_nzc = rf.Network(directory + 'deembedded_MM_NZC_diff_fdf_with_connectors.s4p')
se_ref_nzc.name = 'aicc_nzc'
se_ref_zc = rf.Network(directory + 'deembedded_MM_ZC_diff_fdf_with_connectors.s4p')
```

(continues on next page)

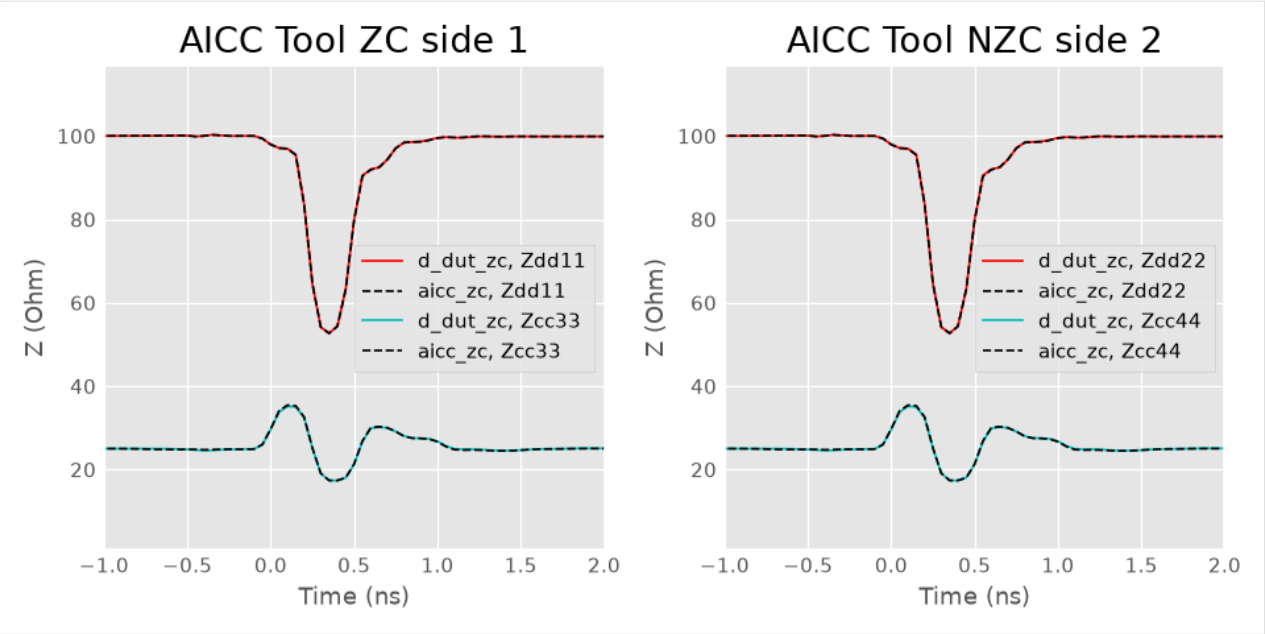
(continued from previous page)

```
se_ref_zc.name = 'aicc_zc'

# transform to mixed-modes
mm_ref_nzc = se_ref_nzc.copy()
mm_ref_nzc.se2gmm(p = 2)
mm_ref_zc = se_ref_zc.copy()
mm_ref_zc.se2gmm(p = 2)

# extrapolate to DC for time step
mm_ref_nzc_dc = IEEEP370.extrapolate_to_dc(mm_ref_nzc)
mm_ref_zc_dc = IEEEP370.extrapolate_to_dc(mm_ref_zc)

# make purposely wrong NZC side 2 flip for comparison with AICC Tool
se_nzc_wrong_d_dut = dm_mmnzc.se_side1.inv ** se_fdf ** dm_mmnzc.se_side2.inv
mm_nzc_wrong_d_dut = se_nzc_wrong_d_dut.copy()
mm_nzc_wrong_d_dut.se2gmm(p=2)
mm_nzc_wrong_d_dut.name = 'wrong flip'
mm_nzc_wrong_d_dut_dc = IEEEP370.extrapolate_to_dc(mm_nzc_wrong_d_dut)

# compute absolute differences
# division of complex reflexion coeff is like subtraction in dB and deg
delta_nzc = mm_ref_nzc / mm_nzc_wrong_d_dut
delta_zc = mm_ref_zc / mm_d_dut_zc

# plot them all
fig, axs = plt.subplots(1, 2, figsize=(8, 4))

axs[0].set_title('AICC Tool NZC side 1')
mm_nzc_wrong_d_dut_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'r')
mm_d_dut_nzc_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'g')
mm_ref_nzc_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dashed', color = 'k')
mm_nzc_wrong_d_dut_dc.plot_z_time_step(2, 2, ax = axs[0], color = 'b')
mm_d_dut_nzc_dc.plot_z_time_step(2, 2, ax = axs[0], color = 'c')
mm_ref_nzc_dc.plot_z_time_step(2, 2, ax = axs[0], linestyle = 'dashed', color = 'k')
axs[0].legend(loc = 'center right')
axs[0].set_xlim((-1, 2))

axs[1].set_title('AICC Tool NZC side 2')
mm_nzc_wrong_d_dut_dc.plot_z_time_step(1, 1, ax = axs[1], color = 'r')
mm_d_dut_nzc_dc.plot_z_time_step(1, 1, ax = axs[1], color = 'g')
mm_ref_nzc_dc.plot_z_time_step(1, 1, ax = axs[1], linestyle = 'dashed', color = 'k')
mm_nzc_wrong_d_dut_dc.plot_z_time_step(3, 3, ax = axs[1], color = 'b')
mm_d_dut_nzc_dc.plot_z_time_step(3, 3, ax = axs[1], color = 'c')
mm_ref_nzc_dc.plot_z_time_step(3, 3, ax = axs[1], linestyle = 'dashed', color = 'k')
axs[1].legend(loc = 'center right')
axs[1].set_xlim((-1, 2))

fig.tight_layout()
```



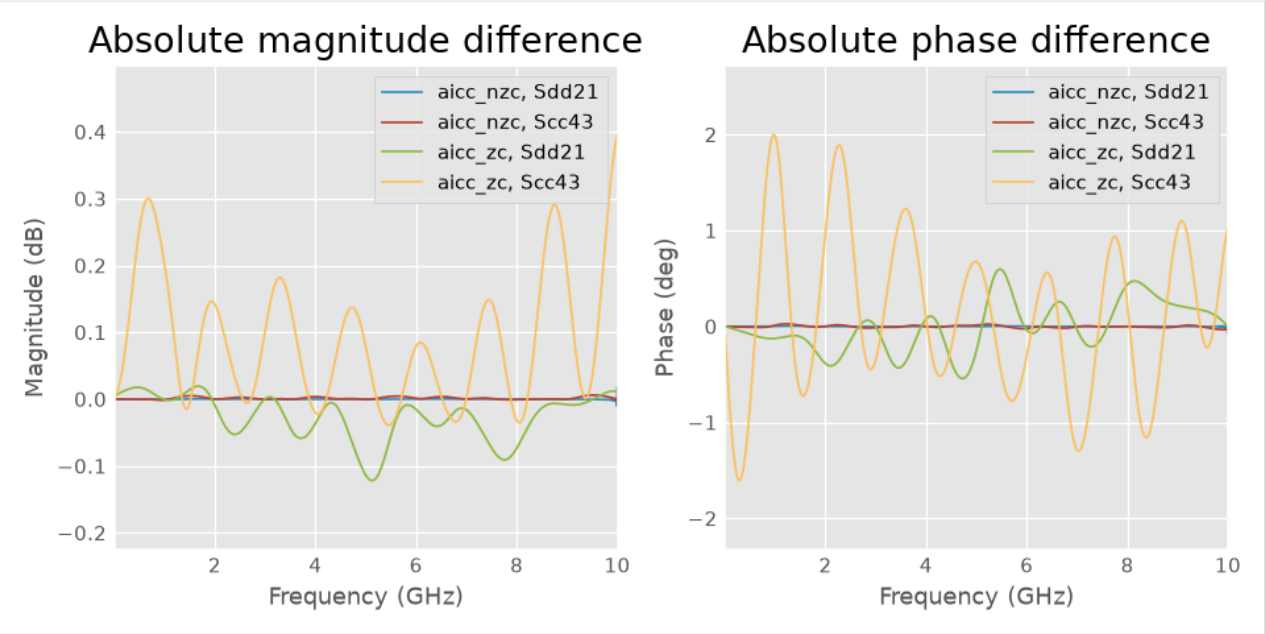
When comparing `scikit-rf` and `AICC` results, NZC algorithm exhibits a small discrepancy after the DUT section (green curve). This is because `AICC` FIX-2 has a wrong flip, which is demonstrated by the results of side 1 and side 2 being different for a symmetrical FIX-DUT-FIX (dashed black curve). If this error is propagated to `scikit-rf` for comparison's sake, the agreement is a perfect fit (red curve). The version of `AICC` used is 1.0.0.

[31]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('AICC Tool NZC side 1')

axs[0].set_title('AICC Tool ZC side 1')
mm_d_dut_zc_dc.plot_z_time_step(0, 0, ax = axs[0], color = 'r')
mm_ref_zc_dc.plot_z_time_step(0, 0, ax = axs[0], linestyle = 'dashed', color = 'k')
mm_d_dut_zc_dc.plot_z_time_step(2, 2, ax = axs[0], color = 'c')
mm_ref_zc_dc.plot_z_time_step(2, 2, ax = axs[0], linestyle = 'dashed', color = 'k')
axs[0].legend(loc = 'center right')
axs[0].set_xlim((-1, 2))

axs[1].set_title('AICC Tool NZC side 2')
mm_d_dut_zc_dc.plot_z_time_step(1, 1, ax = axs[1], color = 'r')
mm_ref_zc_dc.plot_z_time_step(1, 1, ax = axs[1], linestyle = 'dashed', color = 'k')
mm_d_dut_zc_dc.plot_z_time_step(3, 3, ax = axs[1], color = 'c')
mm_ref_zc_dc.plot_z_time_step(3, 3, ax = axs[1], linestyle = 'dashed', color = 'k')
axs[1].legend(loc = 'center right')
axs[1].set_xlim((-1, 2))

fig.tight_layout()
```



When comparing `scikit-rf` and `AICC` results, ZC is visually a perfect fit in the time domain for both sides.

[32]:
```
fig, axs = plt.subplots(1, 2, figsize=(8, 4))
axs[0].set_title('Absolute magnitude difference')
delta_nzc.plot_s_db(1, 0, ax = axs[0])
delta_nzc.plot_s_db(3, 2, ax = axs[0])
delta_zc.plot_s_db(1, 0, ax = axs[0])
delta_zc.plot_s_db(3, 2, ax = axs[0])
axs[0].legend(loc = 'upper right')

axs[1].set_title('Absolute phase difference')
delta_nzc.plot_s_deg(1, 0, ax = axs[1])
delta_nzc.plot_s_deg(3, 2, ax = axs[1])
delta_zc.plot_s_deg(1, 0, ax = axs[1])
delta_zc.plot_s_deg(3, 2, ax = axs[1])
axs[1].legend(loc = 'upper right')

fig.tight_layout()
```



The agreement between `scikit-rf` and `AICC De-Embedding Utility` is better than ± 0.05 dB and $\pm 0.5°$ for the NZC algorithm. For ZC it keep within ± 0.4 dB and $\pm 2°$ on full bandwidth, that is good enough.

In conclusion, for the specific mixed mode case studied, we can say that `scikit-rf` gives consistent results with `AICC De-embedding Utility` implementation in IEEEP370 deembedding routines. It also solves the NZC FIX-2 flip error in `AICC De-embedding Utility` version 1.0.0.

2.1.9 LNA Example

Let's design a LNA using Infineon's BFU520 transistor. First we need to import scikit-rf and a bunch of other utilities:

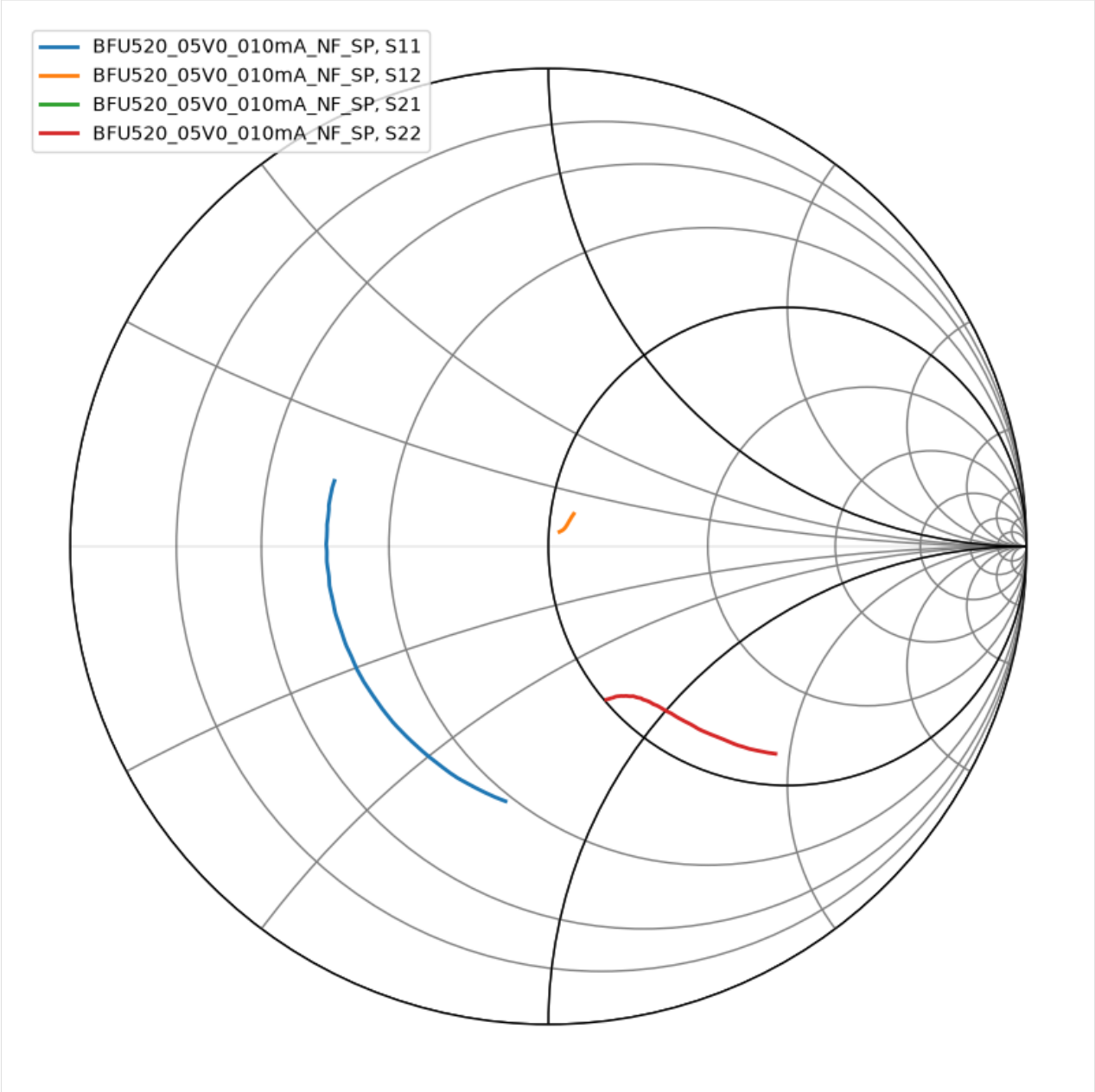
[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf
from skrf.media import DistributedCircuit
from skrf.network import s2z

plt.rcParams['figure.figsize'] = [10, 10]

f = rf.Frequency(0.4, 2, 101, 'GHz')
tem = DistributedCircuit(f, z0=50)
```

[2]:
```
# import the scattering parameters/noise data for the transistor
bjt = rf.Network('BFU520_05V0_010mA_NF_SP.s2p').interpolate(f)
print(bjt)
```

```
2-Port Network: 'BFU520_05V0_010mA_NF_SP',  0.4-2.0 GHz, 101 pts, z0=[50.+0.j 50.+0.j]
```

Let's plot the smith chart for it:

[3]:
```
bjt.plot_s_smith(lw=2)
```



Now let's calculate the source and load stability curves. I'm slightly misusing the `Network` type to plot the curves; normally the curves you pass in to `Network` should be a function of frequency, but it also works to draw these circles as long as you don't try to use any other functions on them

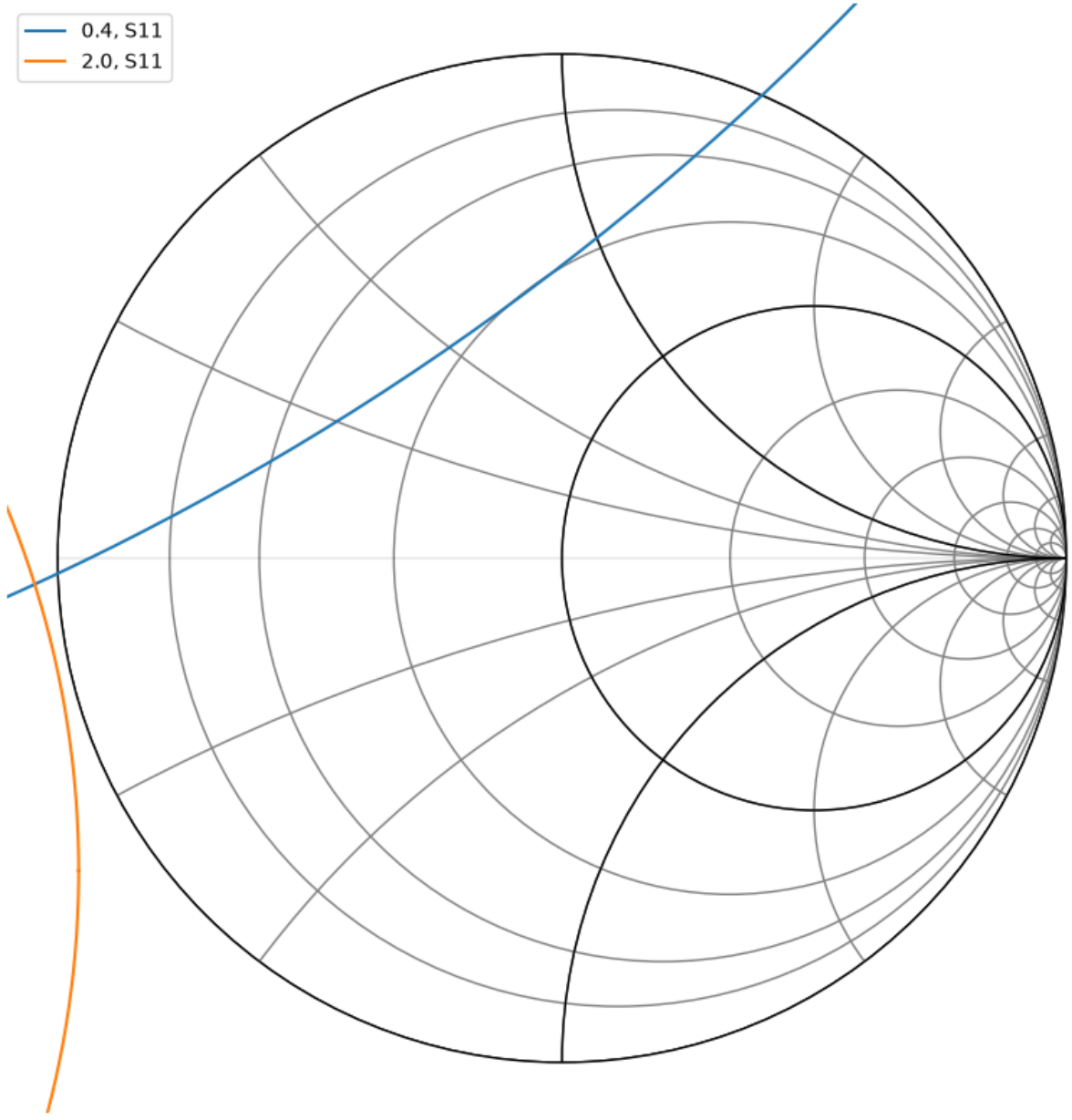
[4]:
```
sqabs = lambda x: np.square(np.absolute(x))  # noqa: E731

delta = bjt.s11.s*bjt.s22.s - bjt.s12.s*bjt.s21.s
rl = np.absolute((bjt.s12.s * bjt.s21.s)/(sqabs(bjt.s22.s) - sqabs(delta)))
cl = np.conj(bjt.s22.s - delta*np.conj(bjt.s11.s))/(sqabs(bjt.s22.s) - sqabs(delta))

rs = np.absolute((bjt.s12.s * bjt.s21.s)/(sqabs(bjt.s11.s) - sqabs(delta)))
cs = np.conj(bjt.s11.s - delta*np.conj(bjt.s22.s))/(sqabs(bjt.s11.s) - sqabs(delta))
```

(continues on next page)

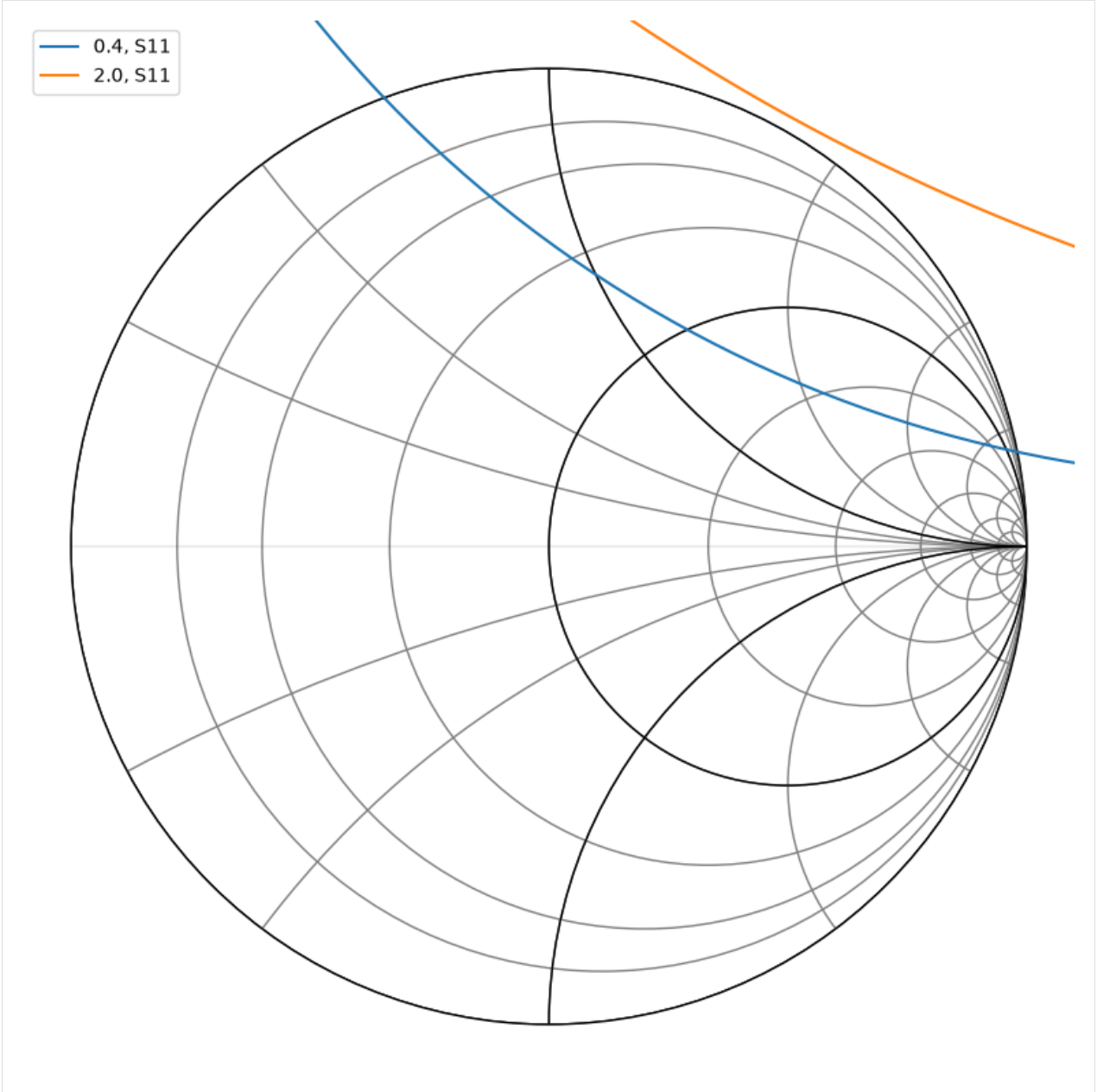
(continued from previous page)

```
def calc_circle(c, r):
    theta = np.linspace(0, 2*np.pi, 1000)
    return c + r*np.exp(1.0j*theta)
```

[5]:
```
for i, f in enumerate(bjt.f):
    # decimate it a little
    if i % 100 != 0:
        continue
    n = rf.Network(name=str(f/1.e+9), s=calc_circle(cs[i][0, 0], rs[i][0, 0]))
    n.plot_s_smith()
```



[6]:
```
for i, f in enumerate(bjt.f):
    # decimate it a little
    if i % 100 != 0:
        continue
    n = rf.Network(name=str(f/1.e+9), s=calc_circle(cl[i][0, 0], rl[i][0, 0]))
    n.plot_s_smith()
```

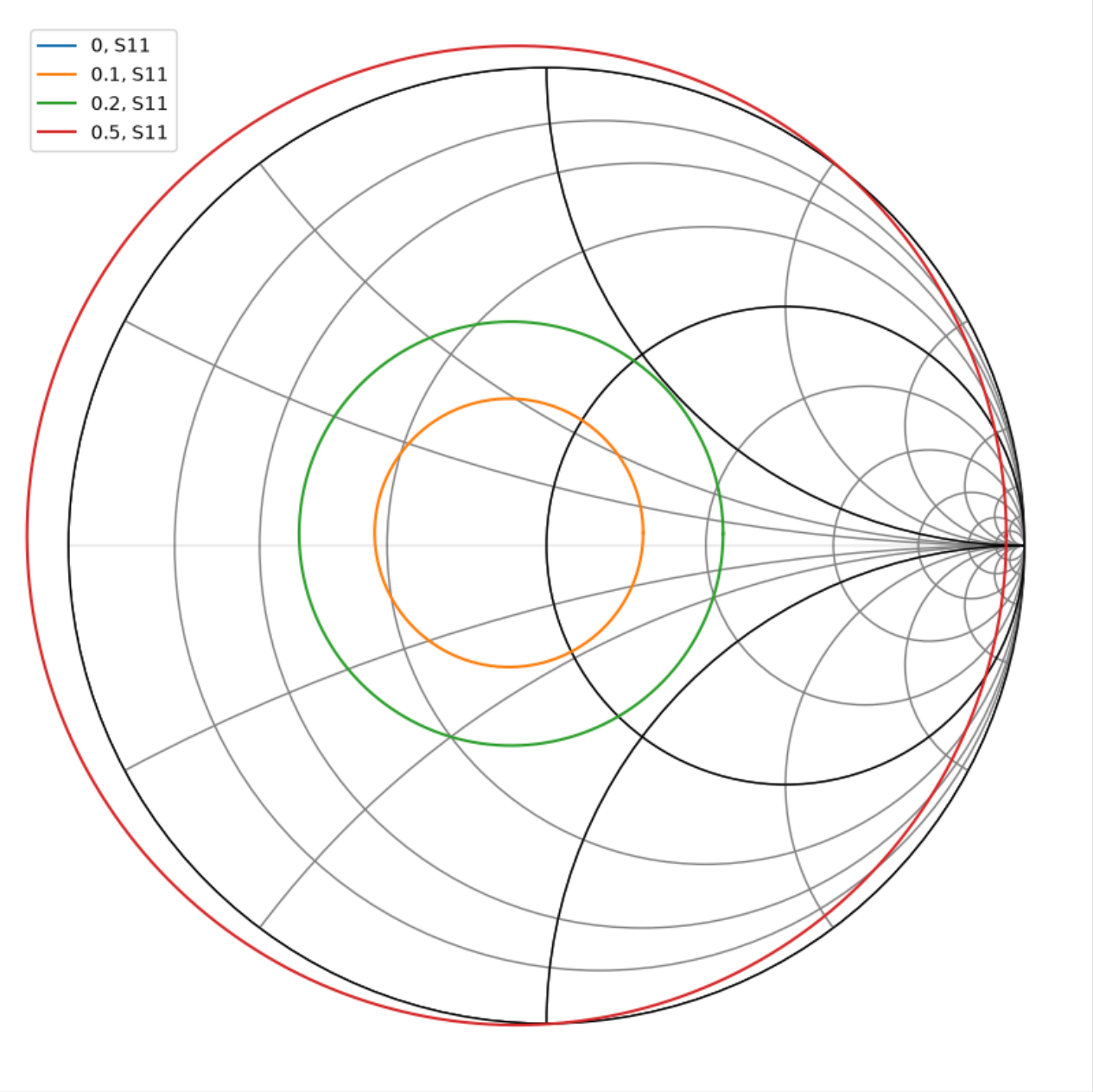


So we can see that we need to avoid inductive loads near short circuit in the input matching network and high impedance inductive loads on the output.

Let's draw some constant noise circles. First we grab the noise parameters for our target frequency from the network model:

[7]:
```
idx_915mhz = rf.util.find_nearest_index(bjt.f, 915.e+6)

# we need the normalized equivalent noise and optimum source coefficient to calculate␣
↪the constant noise circles
rn = bjt.rn[idx_915mhz]/50
gamma_opt = bjt.g_opt[idx_915mhz]
fmin = bjt.nfmin[idx_915mhz]

for nf_added in [0, 0.1, 0.2, 0.5]:
    nf = 10**(nf_added/10) * fmin

    N = (nf - fmin)*abs(1+gamma_opt)**2/(4*rn)
    c_n = gamma_opt/(1+N)
    r_n = 1/(1-N)*np.sqrt(N**2 + N*(1-abs(gamma_opt)**2))

    n = rf.Network(name=str(nf_added), s=calc_circle(c_n, r_n))
    n.plot_s_smith()

print("the optimum source reflection coefficient is ", gamma_opt)
```

```
the optimum source reflection coefficient is  (-0.08373015564356305+0.
↪028299611051606704j)
```



So we can see from the chart that just leaving the input at 50 ohms gets us under 0.1 dB of extra noise, which seems pretty good. I'm actually not sure that these actually correspond to the noise figure level increments I have listed up there, but the circles should at least correspond to increasing noise figures

So let's leave the input at 50 ohms and figure out how to match the output network to maximize gain and stability. Let's see what matching the load impedance with an unmatched input gives us:

[8]:
```
gamma_s = 0.0

gamma_l = np.conj(bjt.s22.s - bjt.s21.s*gamma_s*bjt.s12.s/(1-bjt.s11.s*gamma_s))
gamma_l = gamma_l[idx_915mhz, 0, 0]
is_gamma_l_stable = np.absolute(gamma_l - cl[idx_915mhz]) > rl[idx_915mhz]
```

(continues on next page)

(continued from previous page)

```
gamma_l, is_gamma_l_stable
```

```
[8]: (np.complex128(0.24318880200879378+0.3424018477949188j), array([[ True]]))
```

This looks like it may be kind of close to the load instability circles, so it might make sense to pick a load point with less gain for more stability, or to pick a different source impedance with more noise.

But for now let's just build a matching network for this and see how it performs:

```
[9]: def calc_matching_network_vals(z1, z2):
         flipped = np.real(z1) < np.real(z2)
         if flipped:
             z2, z1 = z1, z2

         # cancel out the imaginary parts of both input and output impedances
         z1_par = 0.0
         if abs(np.imag(z1)) > 1e-6:
             # parallel something to cancel out the imaginary part of
             # z1's impedance
             z1_par = 1/(-1j*np.imag(1/z1))
             z1 = 1/(1./z1 + 1/z1_par)
         z2_ser = 0.0
         if abs(np.imag(z2)) > 1e-6:
             z2_ser = -1j*np.imag(z2)
             z2 = z2 + z2_ser

         Q = np.sqrt((np.real(z1) - np.real(z2))/np.real(z2))
         x1 = -1.j * np.real(z1)/Q
         x2 = 1.j * np.real(z2)*Q

         x1_tot = 1/(1/z1_par + 1/x1)
         x2_tot = z2_ser + x2
         if flipped:
             return x2_tot, x1_tot
         else:
             return x1_tot, x2_tot

     z_l = s2z(np.array([[[gamma_l]]]))[0,0,0]
     # note that we're matching against the conjugate;
     # this is because we want to see z_l from the BJT side
     # if we plugged in z the matching network would make
     # the 50 ohms look like np.conj(z) to match against it, so
     # we use np.conj(z_l) so that it'll look like z_l from the BJT's side
     z_par, z_ser = calc_matching_network_vals(np.conj(z_l), 50)
     z_l, z_par, z_ser
```

```
[9]: (np.complex128(59.68243025631687+49.623296920134344j),
      -567.3436907385121j,
      50.46880393432915j)
```

Let's calculate what the component values are:

```
[10]: c_par = np.real(1/(2j*np.pi*915e+6*z_par))
      l_ser = np.real(z_ser/(2j*np.pi*915e+6))
```

(continues on next page)

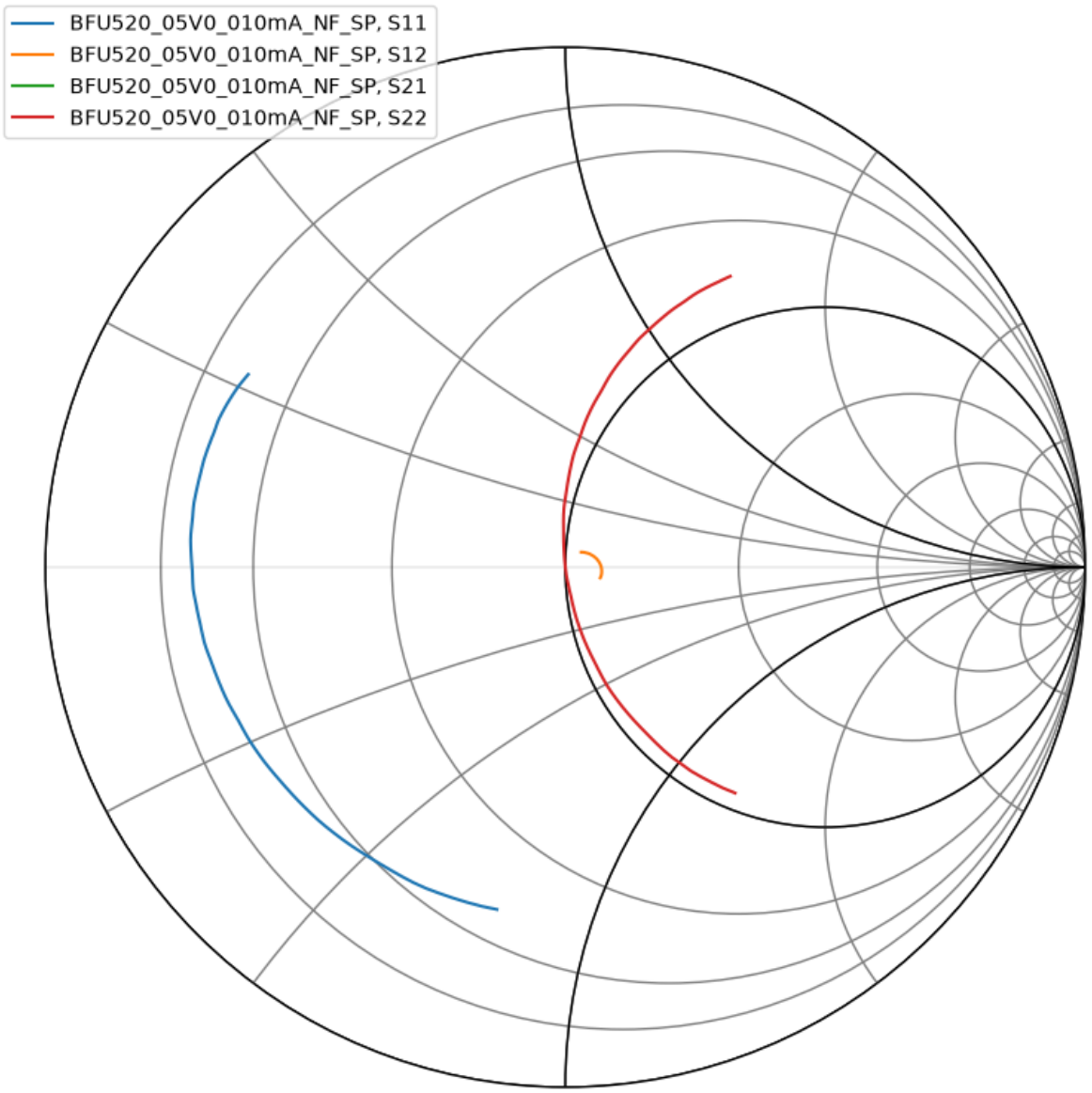
(continued from previous page)

```
print(c_par, l_ser)
```

```
3.065863450236221e-13 8.778535101731343e-09
```

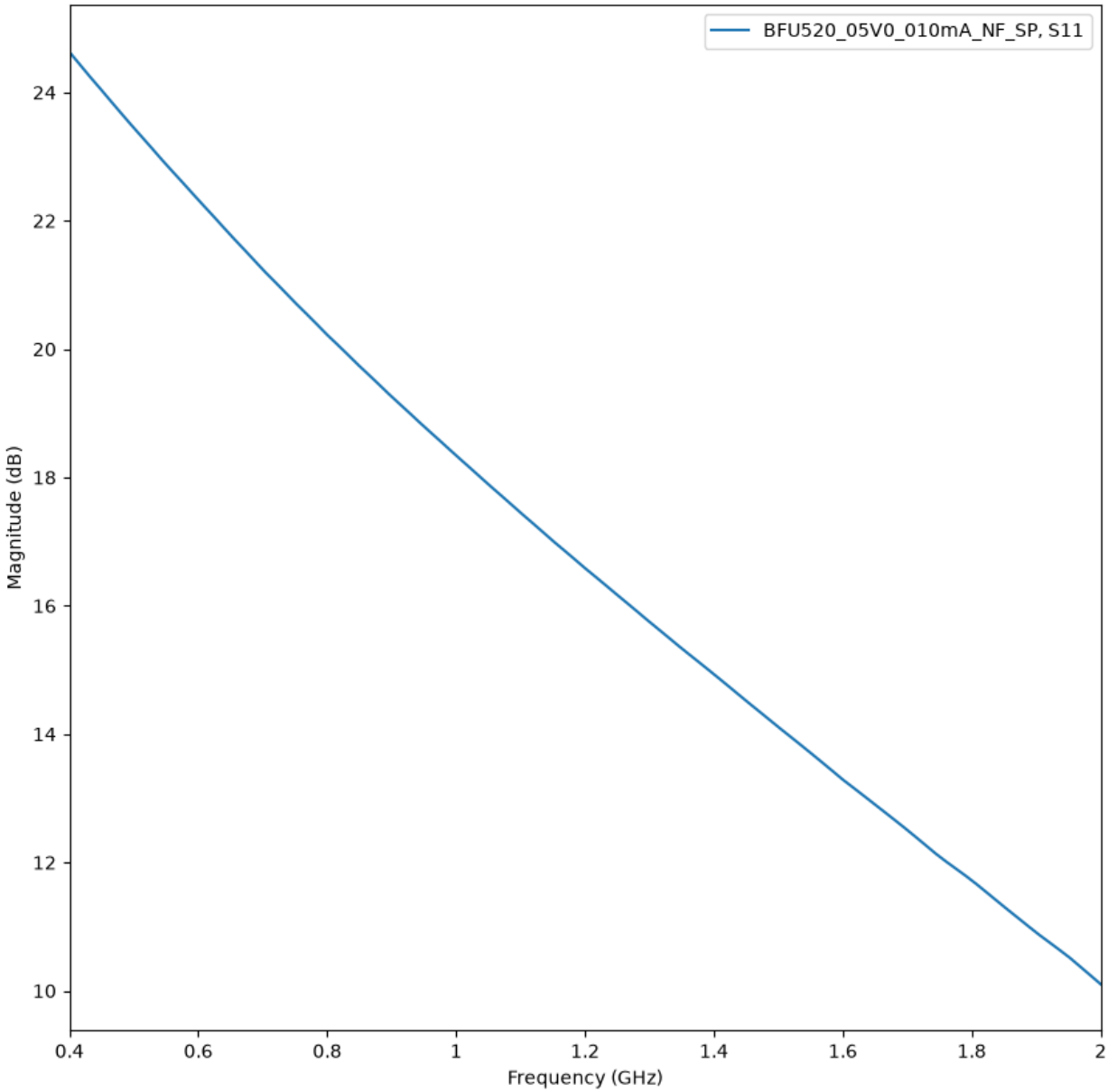
The capacitance is kind of low but the inductance seems reasonable. Let's test it out:

[11]:
```
output_network = tem.shunt_capacitor(c_par) ** tem.inductor(l_ser)

amplifier = bjt ** output_network

amplifier.plot_s_smith()
```



That looks pretty reasonable; let's take a look at the S21 to see what we got:

[12]:
```
amplifier.s21.plot_s_db()
```



So about 18 dB gain; let's see what our noise figure is:

[13]:
```
10*np.log10(amplifier.nf(50.)[idx_915mhz])
```

[13]:
```
np.float64(0.9590591473171773)
```

So 0.96 dB NF, which is reasonably close to the BJT tombstone optimal NF of 0.95 dB

[ ]:

2.1.10 Properties of Rectangular Waveguide

Introduction

This example demonstrates how to use scikit-rf to calculate some properties of rectangular waveguide. For more information regarding the theoretical basis for these calculations, see the *References*.

Object Creation

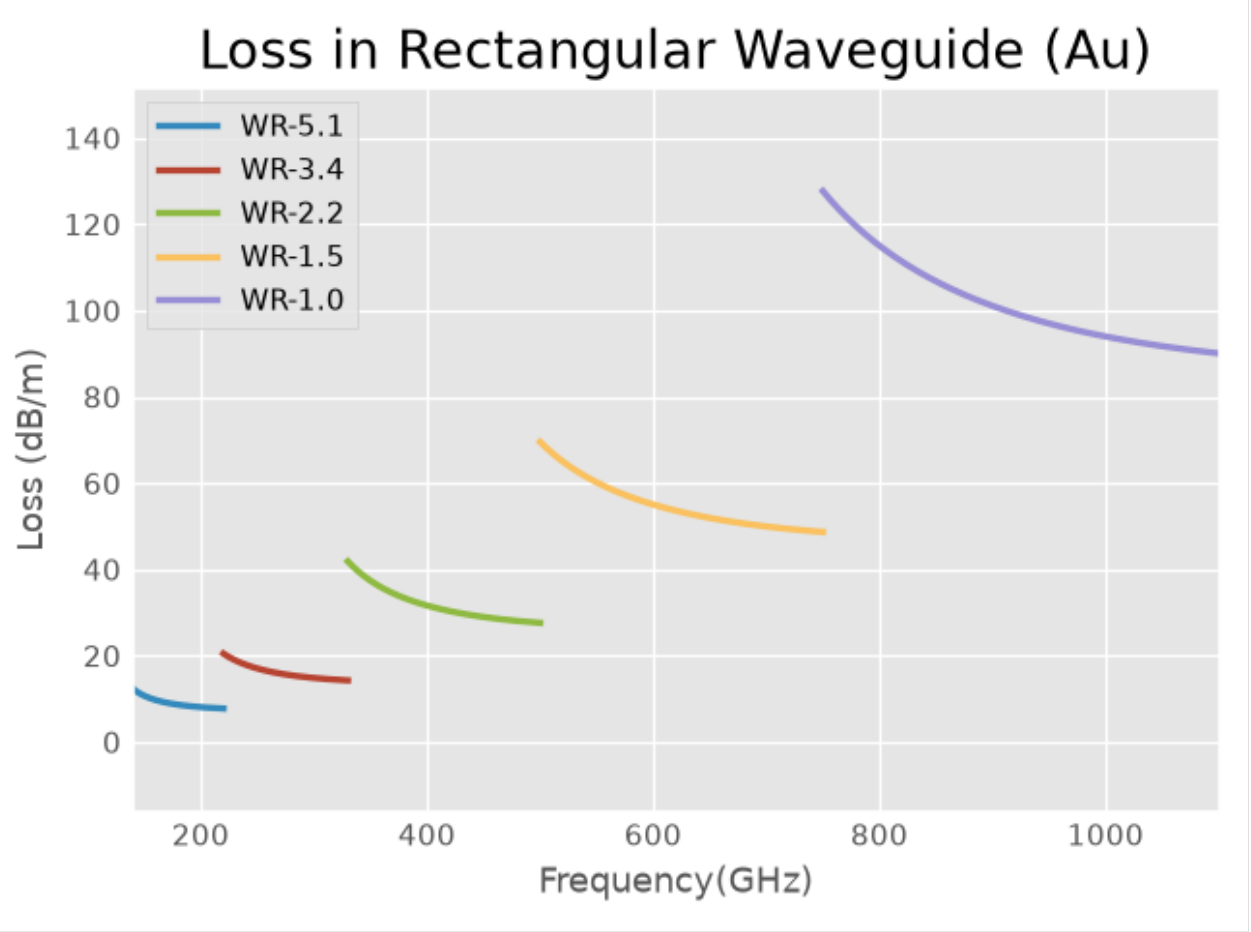
This first section imports necessary modules and creates several `RectangularWaveguide` objects for some standard waveguide bands.

[1]:
```
%matplotlib inline
# imports
import matplotlib.pyplot as plt
import numpy as np
from scipy.constants import c, mil

import skrf as rf
from skrf.frequency import Frequency
from skrf.mathFunctions import np_2_db
from skrf.media import Freespace, RectangularWaveguide

rf.stylely()
# plot formatting
plt.rcParams['lines.linewidth'] = 2
```

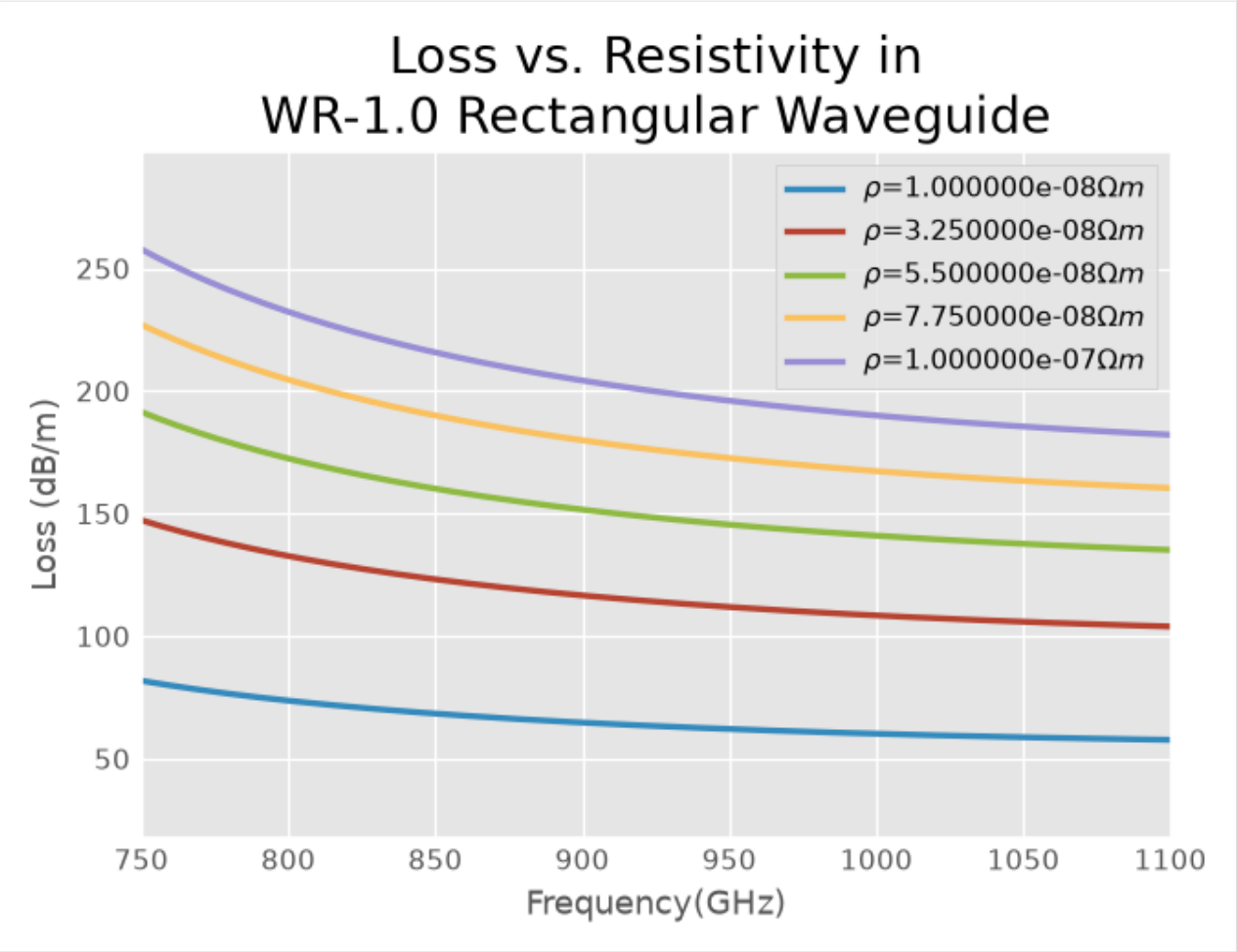
[2]:
```
# create frequency objects for standard bands
f_wr5p1  = Frequency(140,220,1001, 'GHz')
f_wr3p4  = Frequency(220,330,1001, 'GHz')
f_wr2p2 = Frequency(330,500,1001, 'GHz')
f_wr1p5 = Frequency(500,750,1001, 'GHz')
f_wr1   = Frequency(750,1100,1001, 'GHz')

# create rectangular waveguide objects
wr5p1 = RectangularWaveguide(f_wr5p1.copy(), a=51*mil, b=25.5*mil, rho = 'au')
wr3p4 = RectangularWaveguide(f_wr3p4.copy(), a=34*mil, b=17*mil, rho = 'au')
wr2p2 = RectangularWaveguide(f_wr2p2.copy(), a=22*mil, b=11*mil, rho = 'au')
wr1p5 = RectangularWaveguide(f_wr1p5.copy(), a=15*mil, b=7.5*mil, rho = 'au')
wr1   = RectangularWaveguide(f_wr1.copy(), a=10*mil, b=5*mil, rho = 'au')

# add names to waveguide objects for use in plot legends
wr5p1.name = 'WR-5.1'
wr3p4.name = 'WR-3.4'
wr2p2.name = 'WR-2.2'
wr1p5.name = 'WR-1.5'
wr1.name = 'WR-1.0'

# create a list to iterate through
wg_list = [wr5p1, wr3p4,wr2p2,wr1p5,wr1]

# creat a freespace object too
freespace = Freespace(Frequency(125,1100, 1001, 'GHz'))
freespace.name = 'Free Space'
```

Conductor Loss

[3]:
```
fig, ax = plt.subplots()

for wg in wg_list:
    wg.frequency.plot(np_2_db(wg.alpha), label=wg.name)

ax.legend()
ax.set_xlabel('Frequency(GHz)')
ax.set_ylabel('Loss (dB/m)')
ax.set_title('Loss in Rectangular Waveguide (Au)');
```

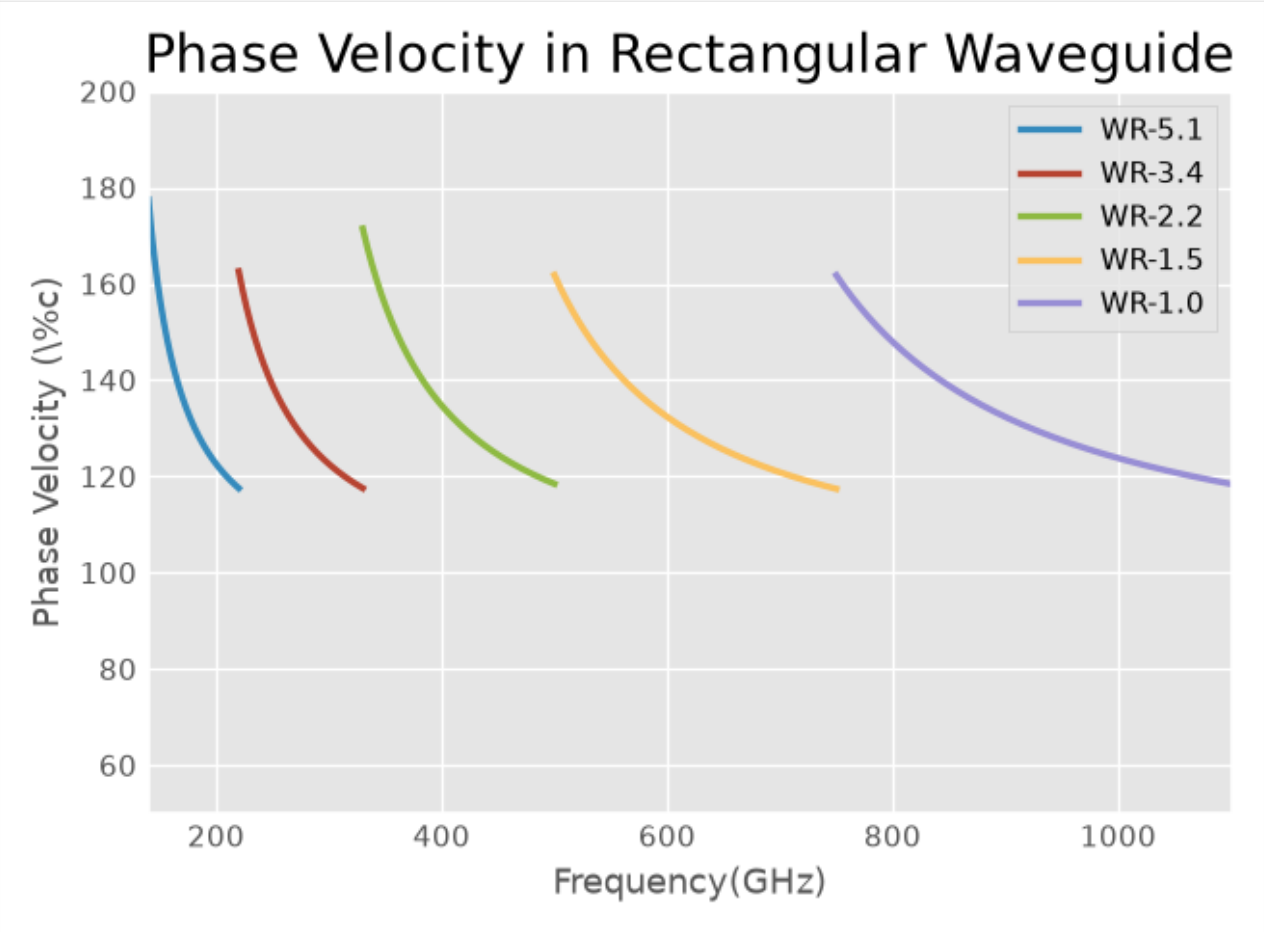


[4]:
```
fig, ax = plt.subplots()
resistivity_list = np.linspace(1,10,5)*1e-8 # ohm meter
for rho in resistivity_list:
    wg = RectangularWaveguide(f_wr1.copy(), a=10*mil, b=5*mil,
                              rho = rho)
    wg.frequency.plot(np_2_db(wg.alpha),label=rf'$ \rho $={rho:e}$ \Omega m$')

ax.legend()
ax.set_xlabel('Frequency(GHz)')
ax.set_ylabel('Loss (dB/m)')
ax.set_title('Loss vs. Resistivity in\nWR-1.0 Rectangular Waveguide');
```

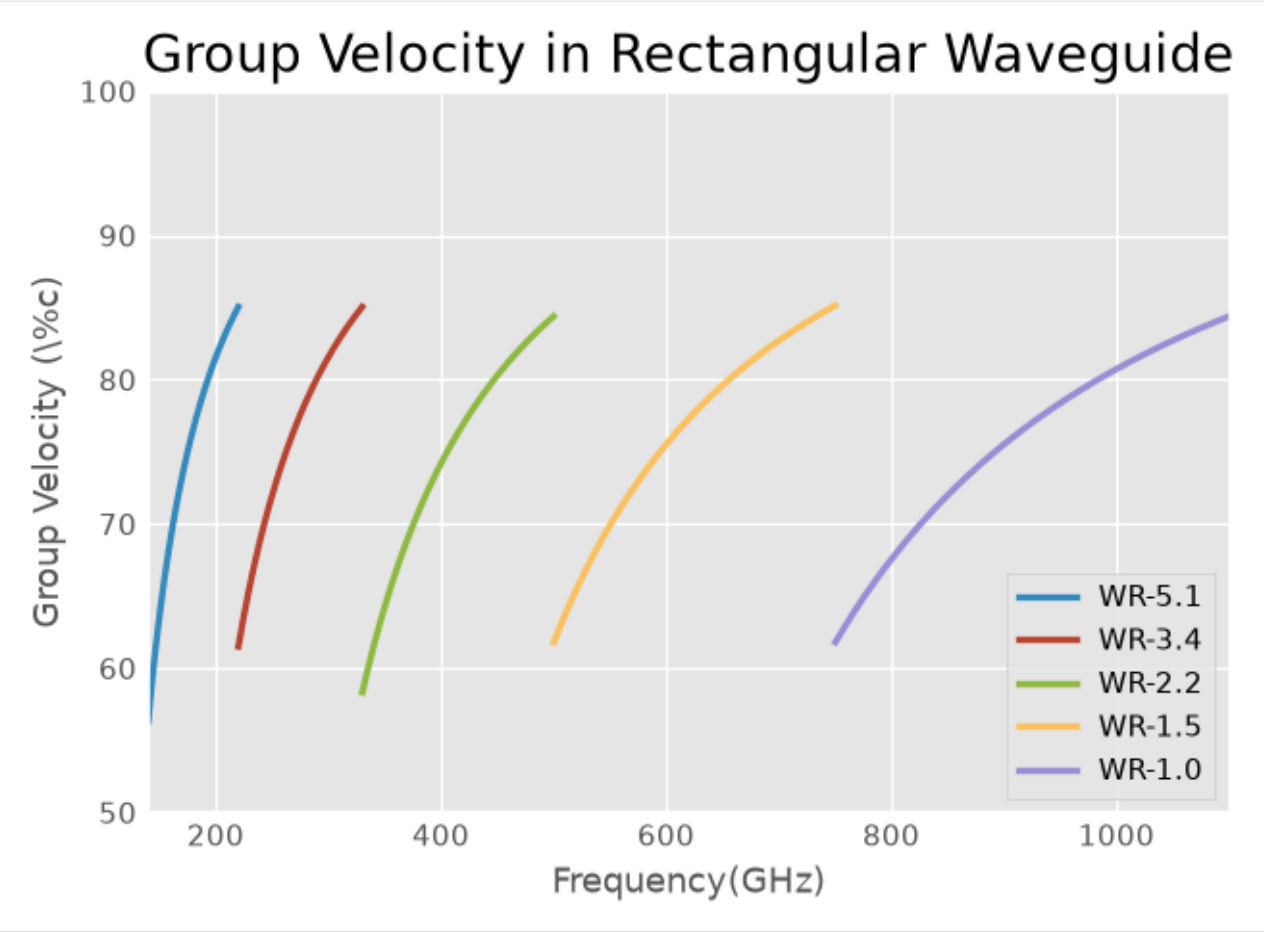


Phase and Group Velocity

[5]:
```
fig, ax = plt.subplots()
for wg in wg_list:
    wg.frequency.plot(100*wg.v_p.real/c, label=wg.name )

ax.legend()
ax.set_ylim(50,200)
ax.set_xlabel('Frequency(GHz)')
ax.set_ylabel(r'Phase Velocity (\%c)')
ax.set_title('Phase Velocity in Rectangular Waveguide');
```



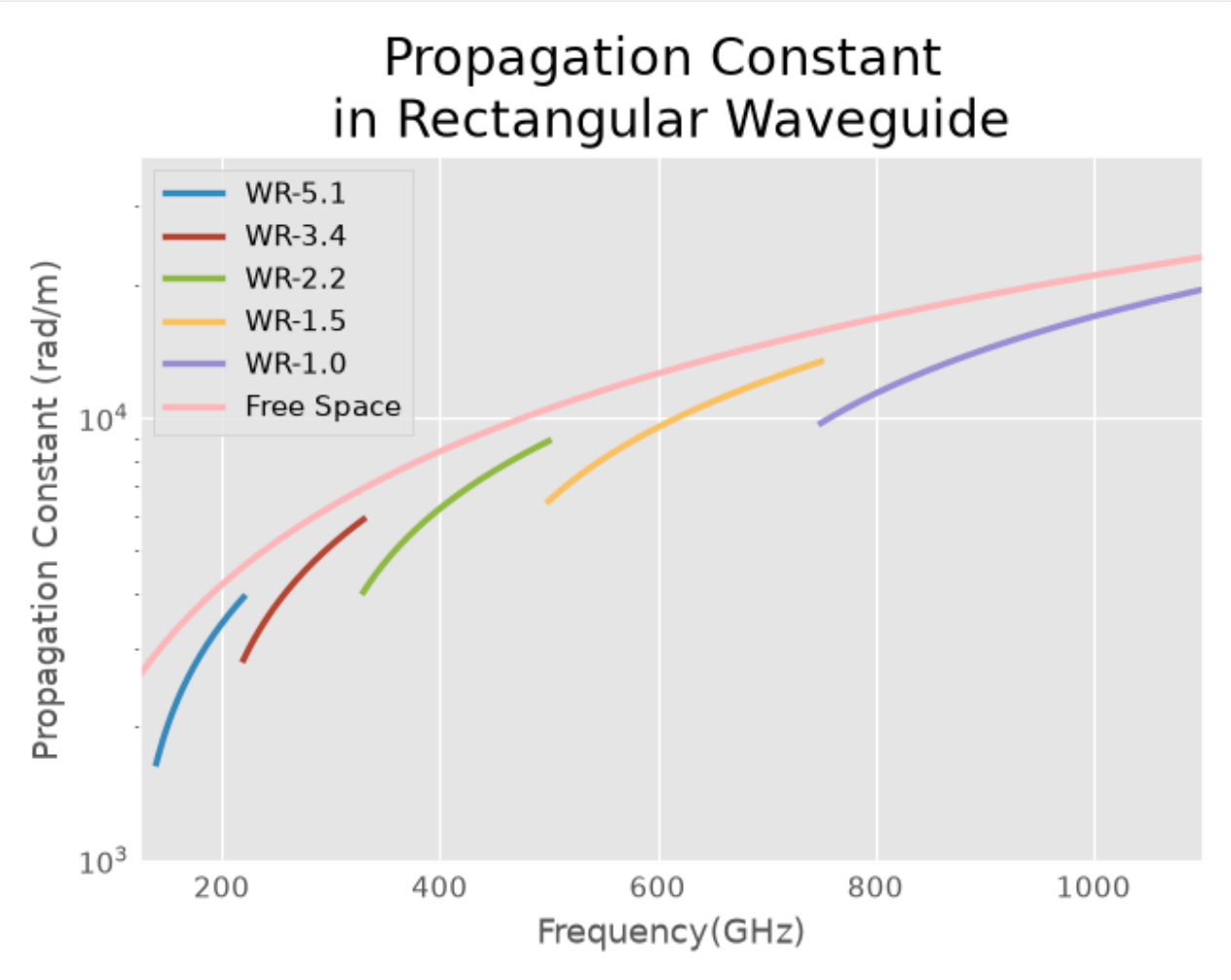
[6]:
```
fig, ax = plt.subplots()
for wg in wg_list:
    plt.plot(wg.frequency.f_scaled[1:],
             100/c*np.diff(wg.frequency.w)/np.diff(wg.beta),
             label=wg.name )

ax.legend()
ax.set_ylim(50,100)
ax.set_xlabel('Frequency(GHz)')
ax.set_ylabel(r'Group Velocity (\%c)')
ax.set_title('Group Velocity in Rectangular Waveguide');
```



Propagation Constant

[7]:
```
fig, ax = plt.subplots()
for wg in wg_list+[freespace]:
    wg.frequency.plot(wg.beta,   label=wg.name)

ax.legend()
ax.set_xlabel('Frequency(GHz)')
ax.set_ylabel('Propagation Constant (rad/m)')
ax.set_title('Propagation Constant \nin Rectangular Waveguide')
ax.semilogy()
```

[7]:
```
[]
```



References

- [1] https://www.microwaves101.com/encyclopedias/waveguide-mathematics
- [2] http://en.wikipedia.org/wiki/Waveguide_(electromagnetism)
- [3] R. F. Harrington, Time-Harmonic Electromagnetic Fields (IEEE Press Series on Electromagnetic Wave Theory). Wiley-IEEE Press, 2001.
- [4] http://www.ece.rutgers.edu/~orfanidi/ewa (see Chapter 9)

2.1.11 Renormalizing S-parameters

This example demonstrates how to use skrf to renormalize a Network's s-parameters to new port impedances. Although trivial, this example creates a matched load in 50ohms and then re-normalizes to a 25ohm environment, producing a reflection coefficient of 1/3.

Ok lets do it

[1]:
```
import skrf as rf
from skrf.instances import wr10
```

(continues on next page)

(continued from previous page)

```
%matplotlib inline

rf.stylely()

# this is just for plotting junk
kw = dict(draw_labels=True, marker = 'o', markersize = 10)
```

Create a one-port ideal match Network, (using the premade media class wr10 as a dummy)

[2]:
```
match_at_50 = wr10.match()
```

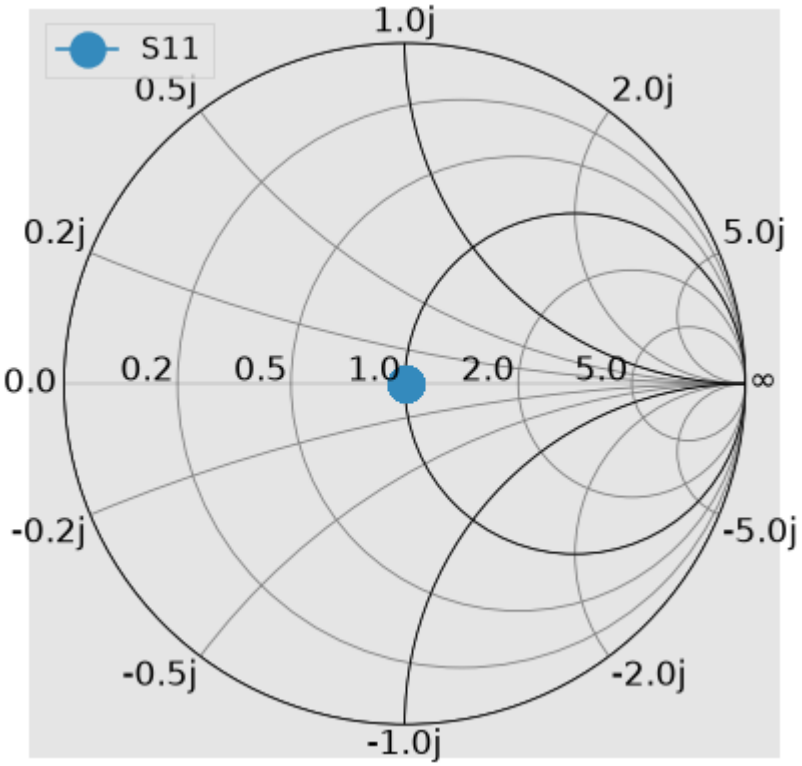
Note that the z0 for this Network defaults to a constant 50ohm

[3]:
```
match_at_50
```

[3]:
```
1-Port Network: '',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j]
```

Plotting its reflection coefficient on the smith chart, shows its a match

[4]:
```
match_at_50.plot_s_smith(**kw)
```

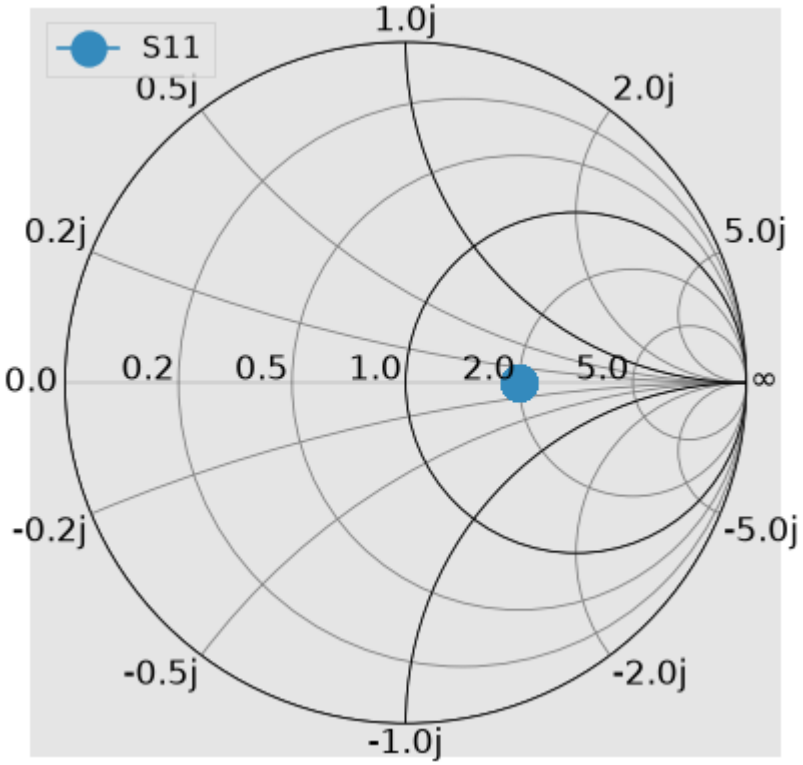


Now, renormalize the port impedance from 50 -> 25, thus the previous 50ohm load now produces a reflection coefficient of

$$\Gamma' = \frac{50-25}{50+25} = \frac{25}{75} = .333$$

Plotting the renormalized response on the Smith Chart

[5]:
```
match_at_50.renormalize(25)
match_at_50.plot_s_smith(**kw)
```



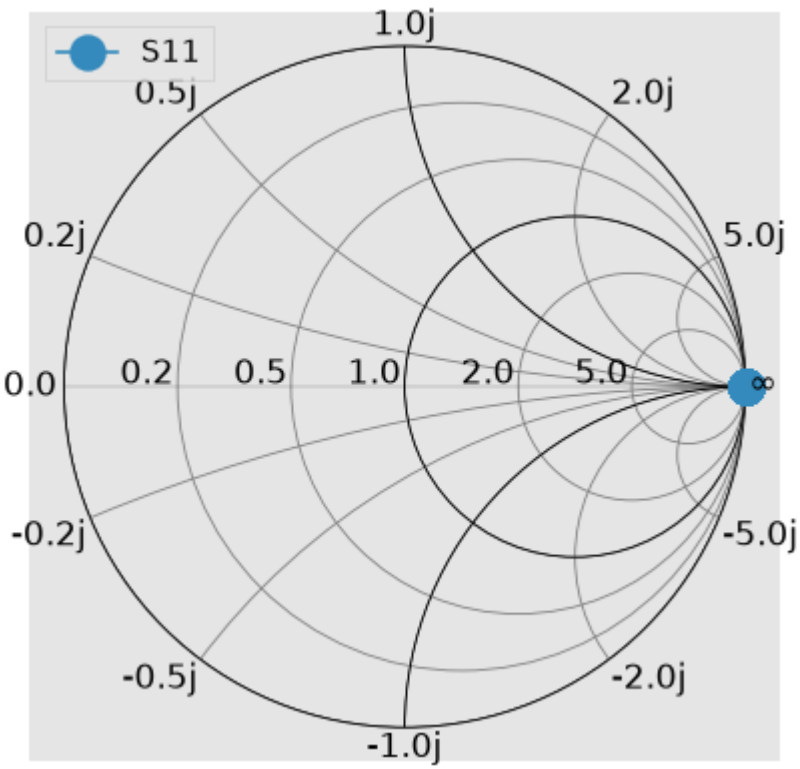
Complex Impedances

You could also renormalize to a complex port impedance if you're crazy. For example, renormalizing to 50j, one would expect:

$$\Gamma' = \frac{50 - 50j}{50 + 50j} = 50\frac{1 - j}{1 + j} = -50j$$

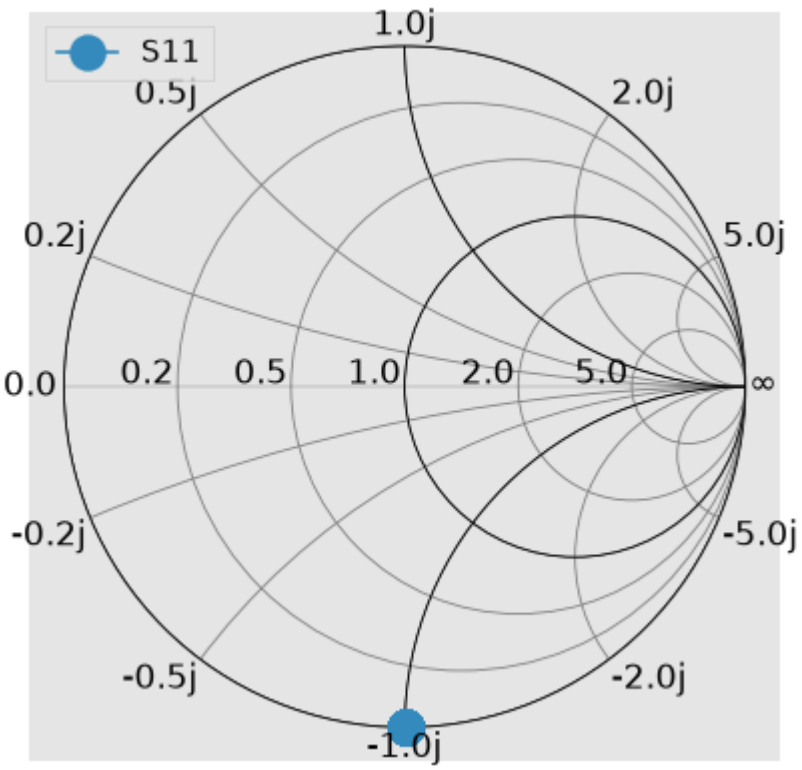
However, one finds an unexpected result when plotting the Smith chart:

[6]:
```
match_at_50 = wr10.match()
match_at_50.renormalize(50j)  # same as renormalize(50j, s_def='power')
match_at_50.plot_s_smith(**kw)  # expect -1j
```



This is because the default behaviour of scikit-rf is to use *power-waves* scattering parameter definition (since it is the most popular one is CAD software). But the *power-waves* definition is known to fail in such a case. This is why scikit-rf also implement the *pseudo-waves* scattering parameters definition, but you have to specify it using the `s_def` parameter:

[7]:
```
match_at_50 = wr10.match()
match_at_50.renormalize(50j, s_def='pseudo')
match_at_50.plot_s_smith(**kw)  # expect -1j
```



Which gives the expected result.

2.1.12 Time Domain and Gating

Intro

This notebooks demonstrates how to use scikit-rf for time-domain analysis and gating. A quick example is given first, followed by a more detailed explanation.

S-parameters are measured in the frequency domain, but can be analyzed in time domain if you like. In many cases, measurements are not made down to DC. This implies that the time-domain transform is not complete, but it can be very useful nonetheless. A major application of time-domain analysis is to use *gating* to isolate a single response in space. More information about the details of time domain analysis see [1].

References

- [1] Keysight - Time Domain Analysis Using a Network Analyzer - Application Note pdf

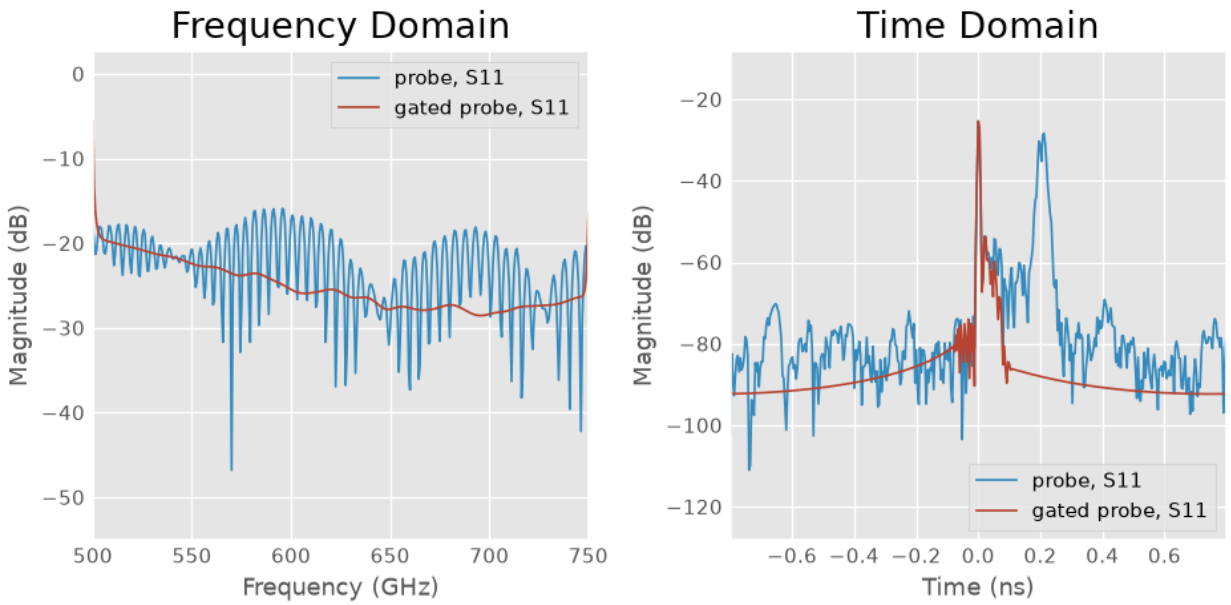
Quick Example

[1]:
```
import matplotlib.pyplot as plt

import skrf as rf

%matplotlib inline
rf.stylely()

# load data for the waveguide to CPW probe
```

(continues on next page)

(continued from previous page)

```
probe = rf.Network('../metrology/oneport_tiered_calibration/probe.s2p')

# we will focus on s11
s11 = probe.s11

#  time-gate the first largest reflection
s11_gated = s11.time_gate(center=0, span=.2, t_unit='ns')
s11_gated.name='gated probe'

# plot frequency and time-domain s-parameters
plt.figure(figsize=(8,4))
plt.subplot(121)
s11.plot_s_db()
s11_gated.plot_s_db()
plt.title('Frequency Domain')

plt.subplot(122)
s11.plot_s_db_time()
s11_gated.plot_s_db_time()
plt.title('Time Domain')
plt.tight_layout()
```



Interpreting Time Domain

Out DUT in this example is a waveguide-to-CPW probe, that was measured in *this other example*.

[2]:
```
# load data for the waveguide to CPW probe
probe = rf.Network('../metrology/oneport_tiered_calibration/probe.s2p')
probe
```

[2]:
```
2-Port Network: 'probe',  500.0-750.0 GHz, 401 pts, z0=[50.+0.j 50.+0.j]
```

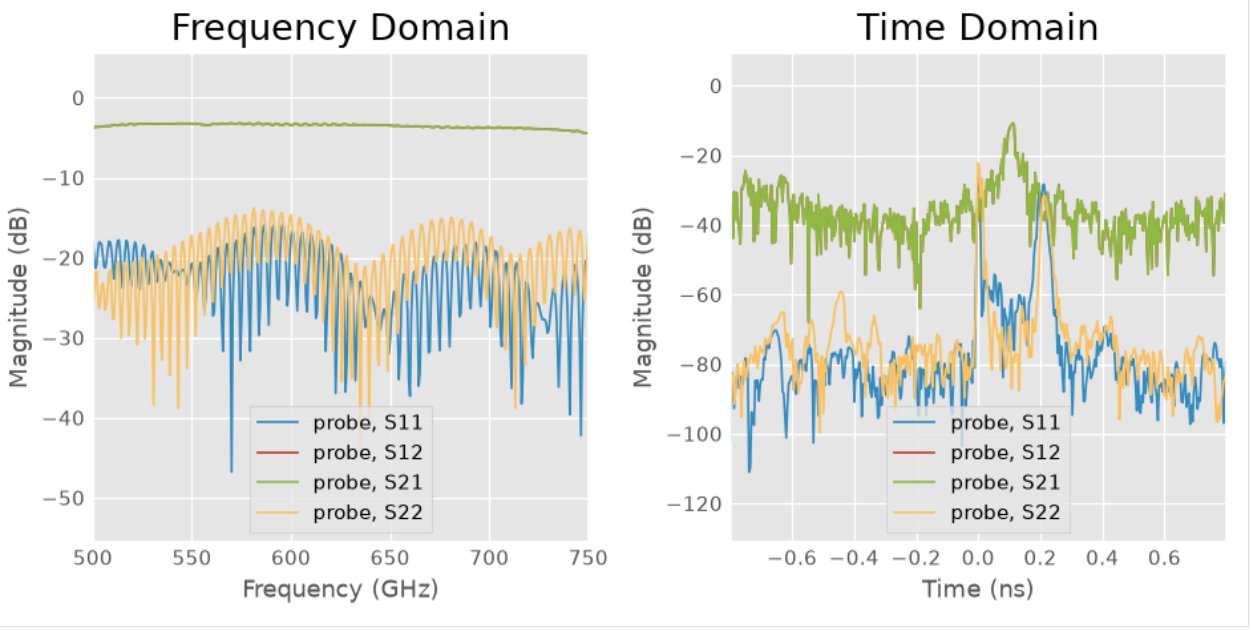
Note there are two time-domain plotting functions in scikit-rf:

- `Network.plot_s_db_time()`
- `Network.plot_s_time_db()`

The difference is that the former, `plot_s_db_time()`, employs windowing before plotting to enhance impulse resolution. Windowing will be discussed in a bit, but for now we just use `plot_s_db_time()`.

Plotting all four s-parameters of the probe in both frequency and time-domain.

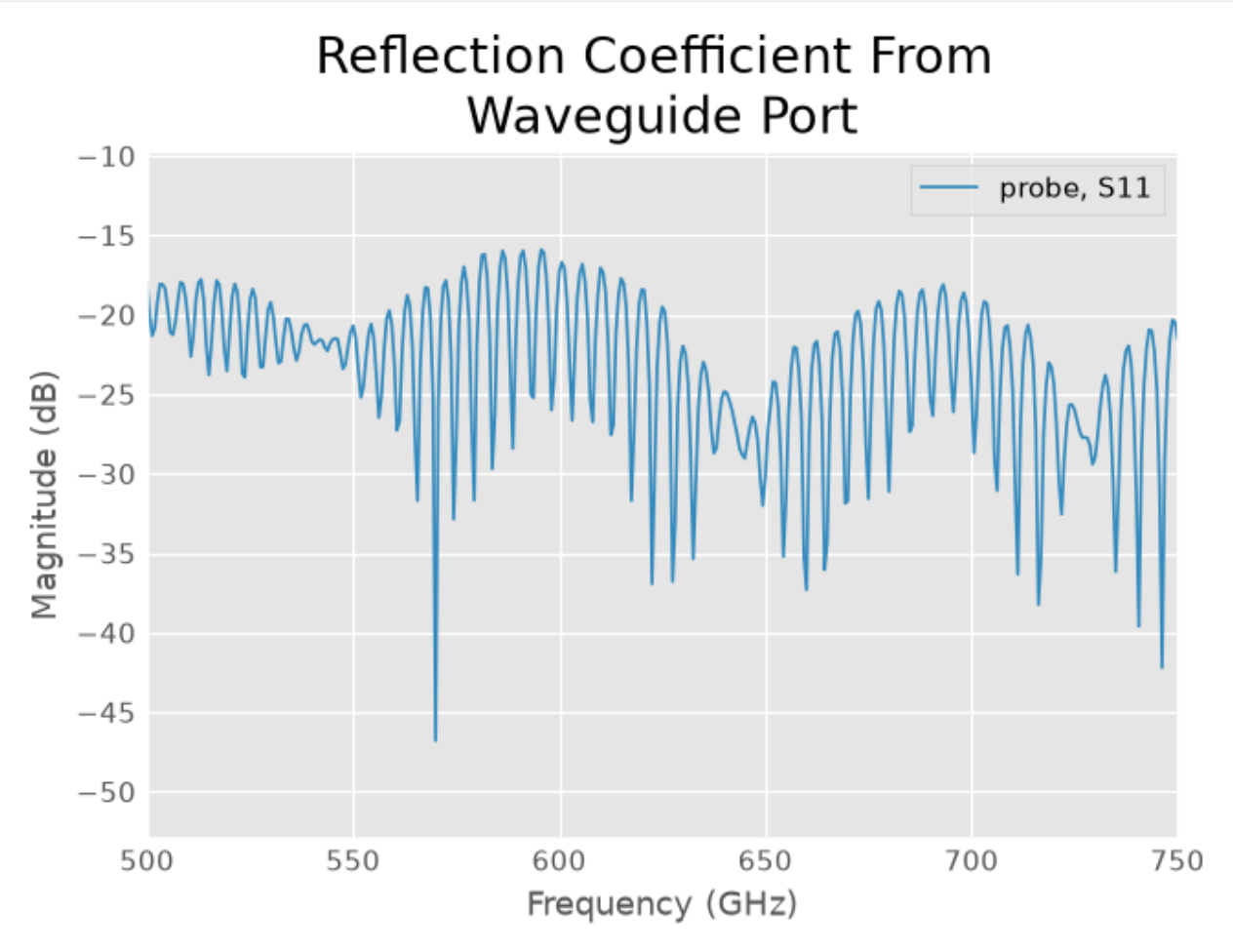
[3]:
```
# plot frequency and time-domain s-parameters
plt.figure(figsize=(8,4))
plt.subplot(121)
probe.plot_s_db()
plt.title('Frequency Domain')
plt.subplot(122)
probe.plot_s_db_time()
plt.title('Time Domain')
plt.tight_layout()
```



Focusing on the reflection coefficient from the waveguide port (s11), you can see there is an interference pattern present.

[4]:
```
probe.plot_s_db(0,0)
plt.title('Reflection Coefficient From \nWaveguide Port')
```

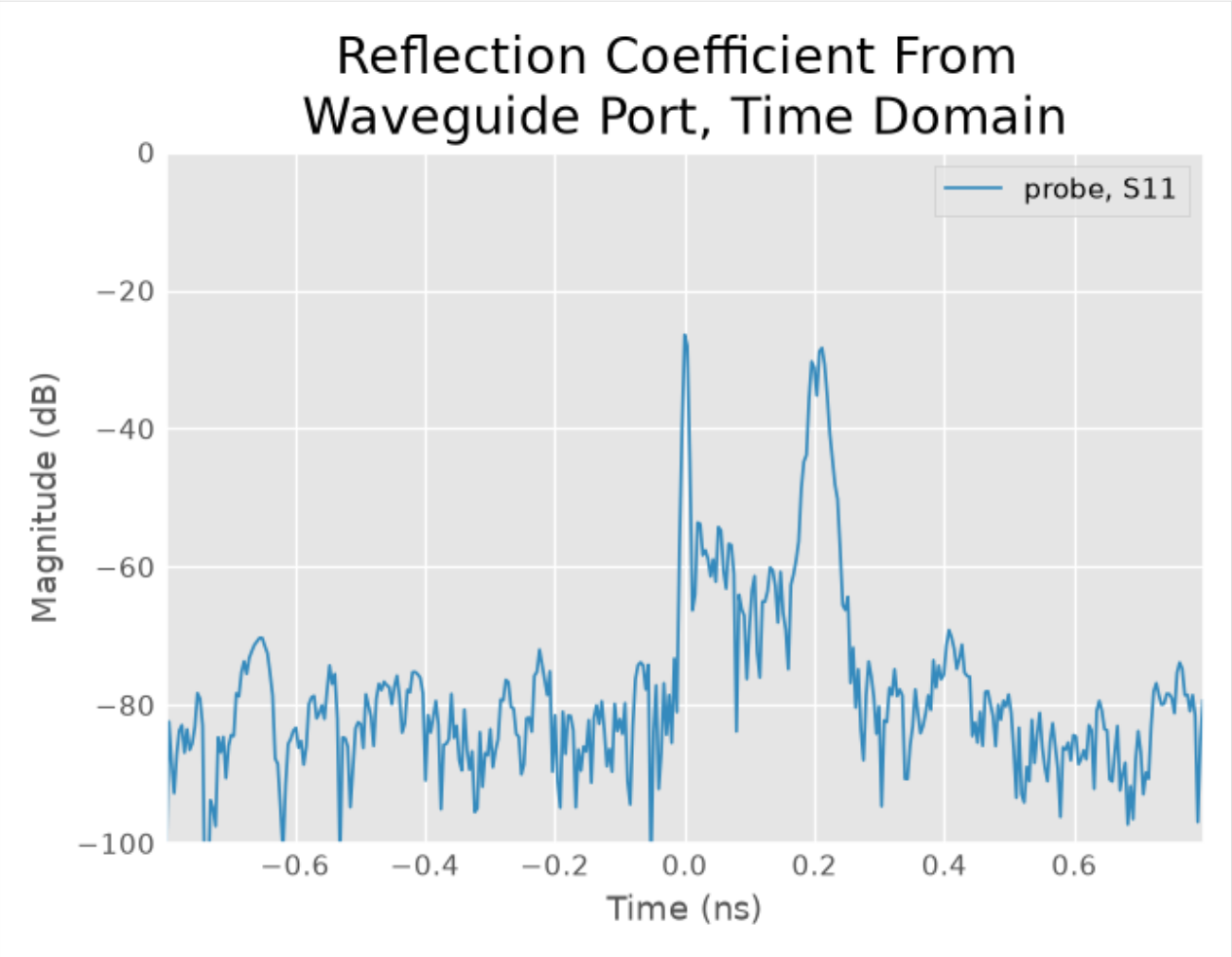
[4]:
```
Text(0.5, 1.0, 'Reflection Coefficient From \nWaveguide Port')
```



This ripple is evidence of several discrete reflections. Plotting s11 in the time-domain allows us to see where, or *when*, these reflections occur.

[5]:
```
probe_s11 = probe.s11
probe_s11.plot_s_db_time(0,0)
plt.title('Reflection Coefficient From \nWaveguide Port, Time Domain')
plt.ylim(-100,0)
```

[5]:
```
(-100.0, 0.0)
```



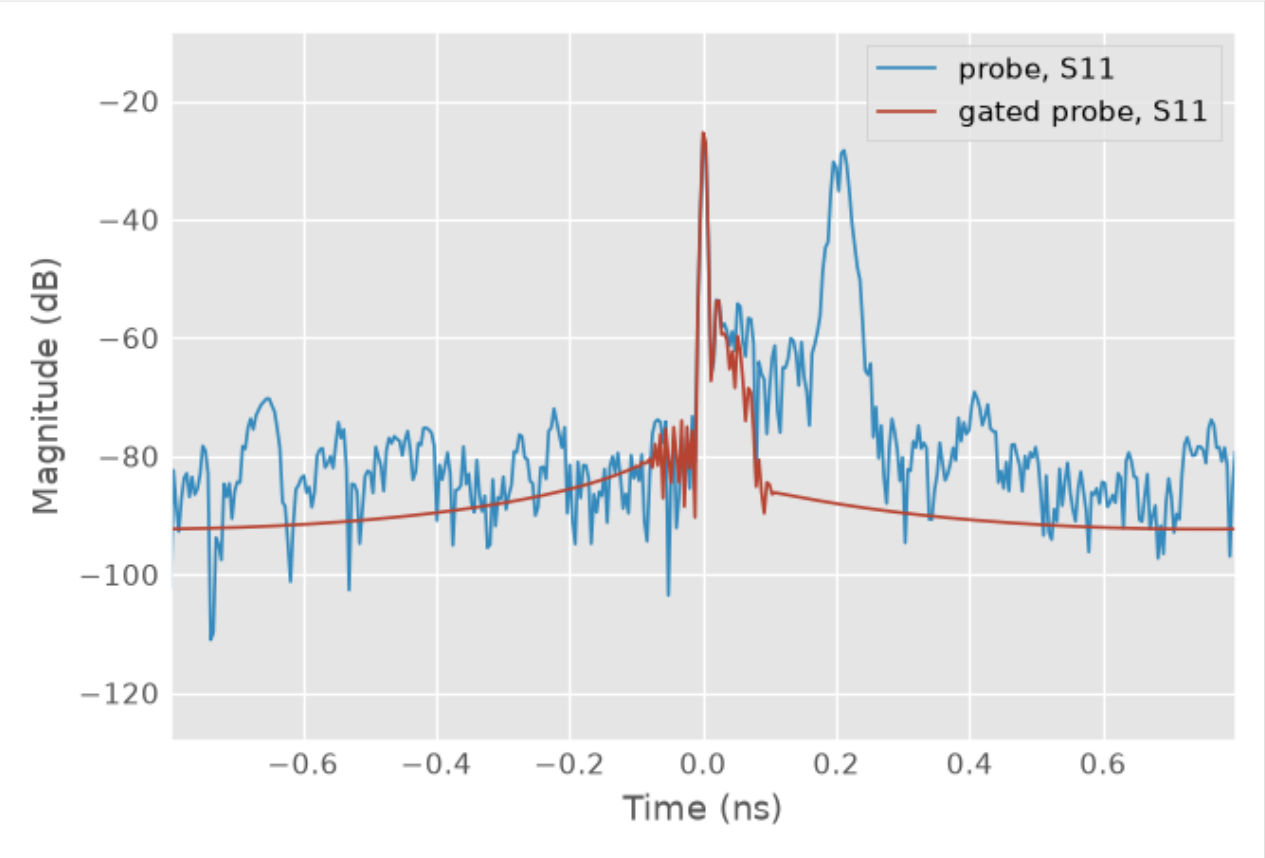
From this plot we can see two dominant reflections;

- one at $t = 0$ns (the test-port)
- and another at $t = .2$ ns (who knows?).

Gating The Reflection of Interest

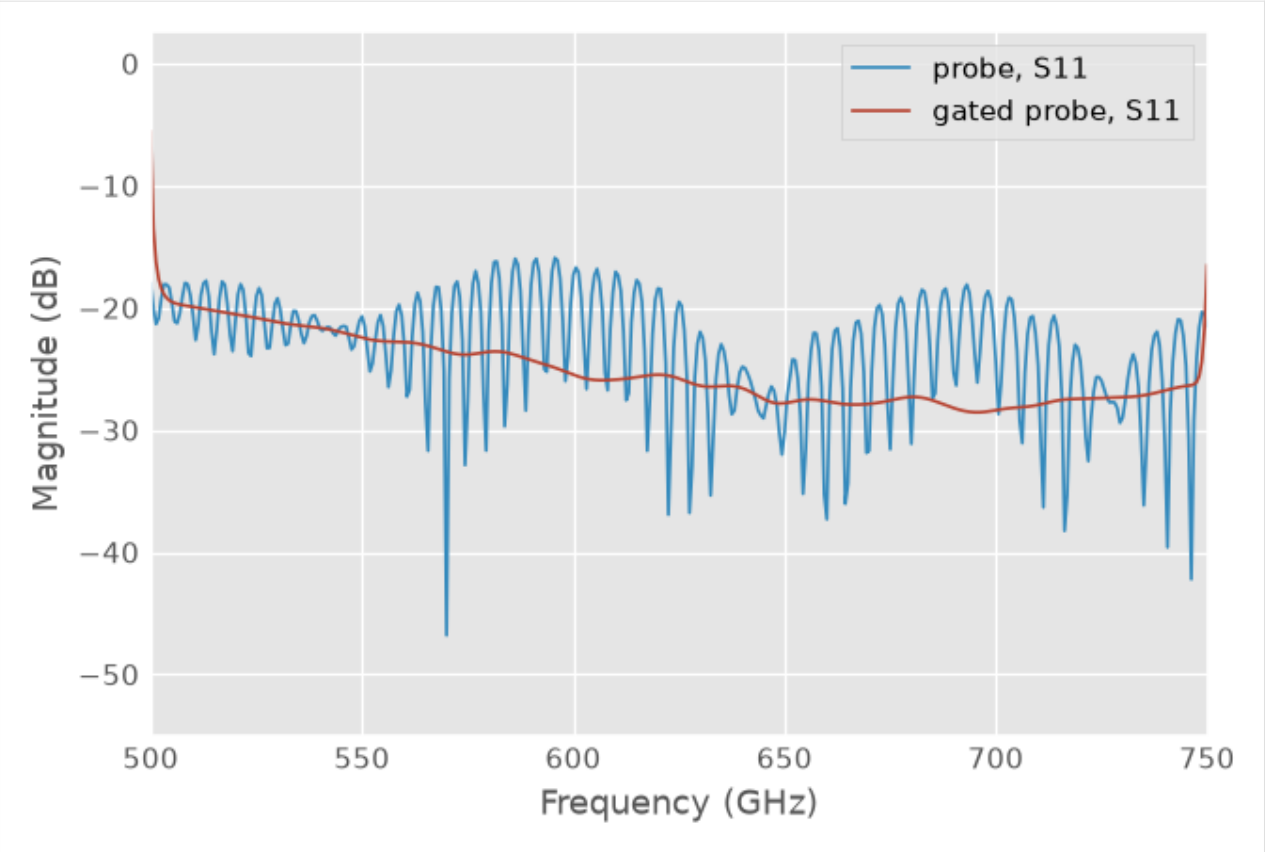
To isolate the reflection from the waveguide port, we can use time-gating. This can be done by using the method `Network.time_gate()`, and provide it an appropriate center and span (in ns). To see the effects of the gate, both the original and gated response are compared.

[6]:
```
probe_s11_gated = probe_s11.time_gate(center=0, span=.2, t_unit='ns')
probe_s11_gated.name='gated probe'

s11.plot_s_db_time()
s11_gated.plot_s_db_time()
```



Next, compare both responses in frequency domain to see the effect of the gate.

[7]:
```
s11.plot_s_db()
s11_gated.plot_s_db()
```



Auto-gate

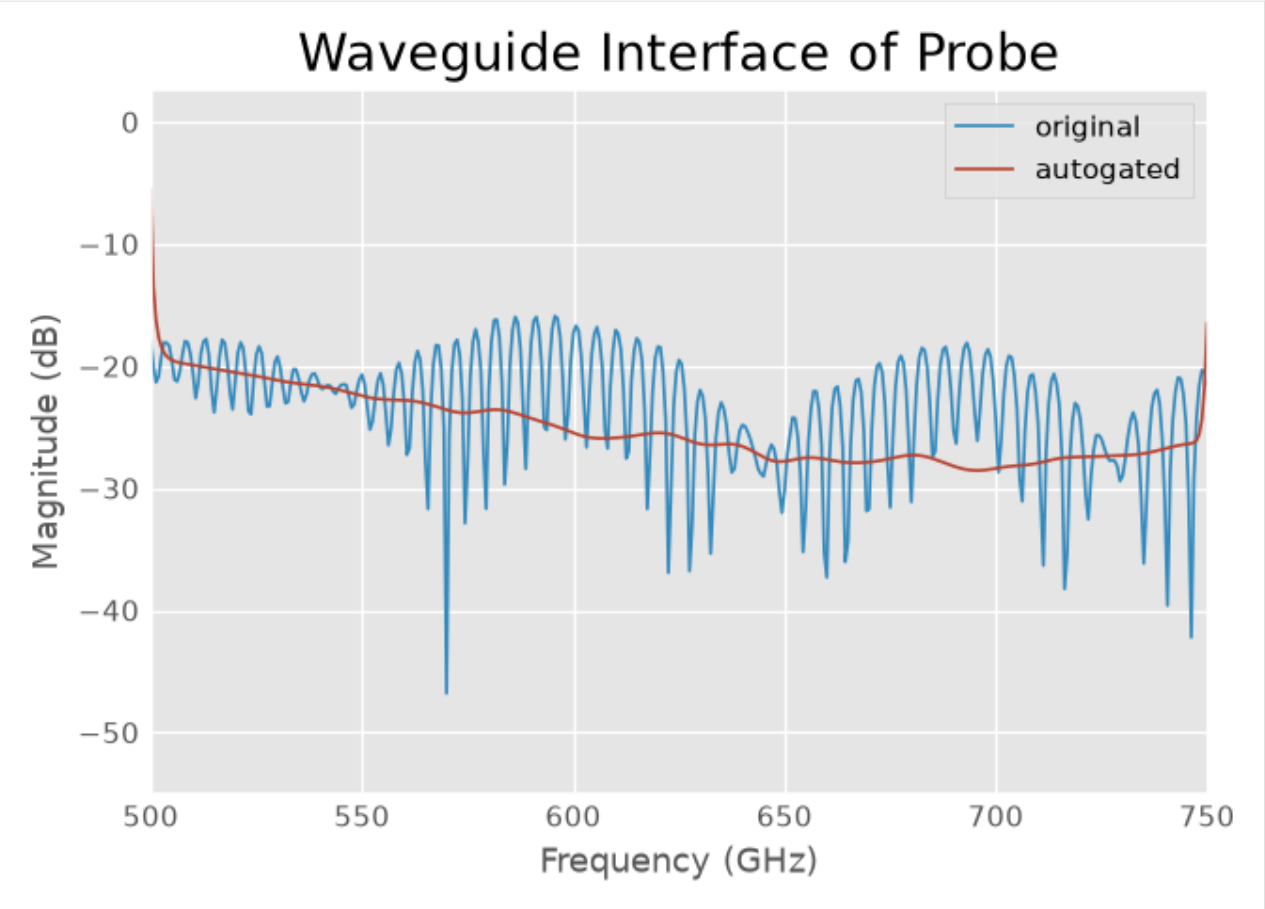
The time-gating method in `skrf` has an auto-gating feature which can also be used to gate the largest reflection. When no gate parameters are provided, `time_gate()` does the following:

1. find the two largest peaks

- center the gate on the tallest peak
- set span to distance between two tallest peaks

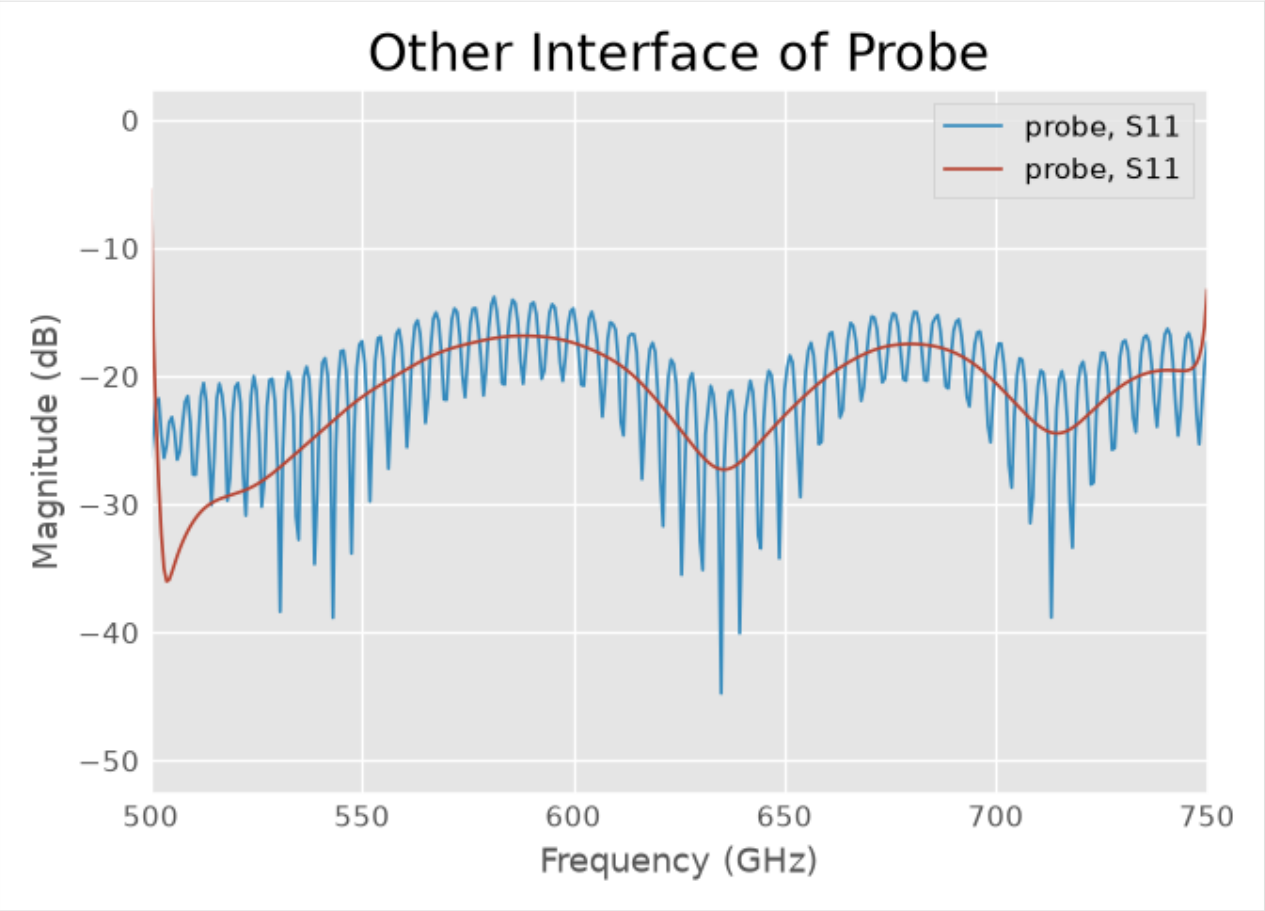
You may want to plot the gated network in time-domain to see what the determined gate shape looks like.

[8]:
```
plt.title('Waveguide Interface of Probe')
s11.plot_s_db(label='original')
s11.time_gate().plot_s_db(label='autogated') #autogate on the fly
```



Might see how the autogate does on the other probe interface,

[9]:
```
plt.title('Other Interface of Probe')
probe.s22.plot_s_db()
probe.s22.time_gate().plot_s_db()
```



Determining Distance

To make time-domain useful as a diagnostic tool, one would like to convert the x-axis to distance. This requires knowledge of the propagation velocity in the device. **skrf** provides some transmission-line models in the module *skrf.media*, which can be used for this.

However…

For dispersive media, such as rectangular waveguide, the phase velocity is a function of frequency, and transforming time to distance is not straightforward. As an approximation, you can normalize the x-axis to the speed of light.

Alternatively, you can simulate the a known device and compare the two time domain responses. This allows you to attribute quantitative meaning to the axes. For example, you could create an ideal delayed load as shown below. Note: the magnitude of a response *behind* a large impulse doesn not have meaningful units.

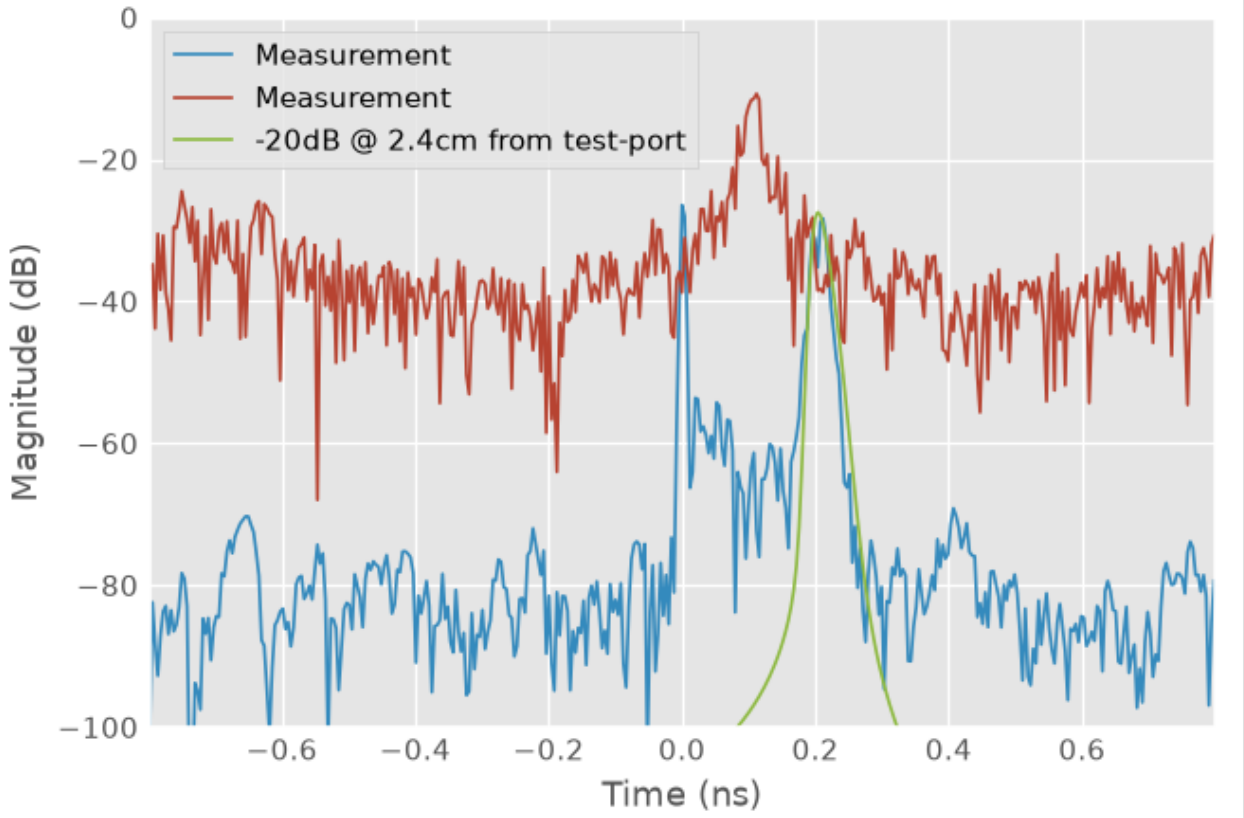
[10]:
```
from skrf.constants import mil
from skrf.media import RectangularWaveguide

# create a rectangular waveguide media to generate a theoretical network
wr1p5 = RectangularWaveguide(frequency=probe.frequency,
                             a=15*mil,z0_override=1)

# create an ideal delayed load, parameters are adjusted until the
# theoretical response agrees with the measurement
theory = wr1p5.delay_load(Gamma0=rf.mathFunctions.db_2_mag(-20),
```

(continues on next page)

(continued from previous page)

```
                          d=2.4, unit='cm')


probe.plot_s_db_time(0,0, label = 'Measurement')
theory.plot_s_db_time(label='-20dB @ 2.4cm from test-port')
plt.ylim(-100,0)
```

[10]:
```
(-100.0, 0.0)
```



This plot demonstrates a few important points:

- the theoretical delayed load is not a perfect impulse in time. This is due to the dispersion in waveguide.
- the peak of the magnitude in time domain is not identical to that specified, also due to dispersion (and windowing).

What the hell is Windowing?

The `'plot_s_db_time()'` function does a few things.

1. windows the s-parameters.

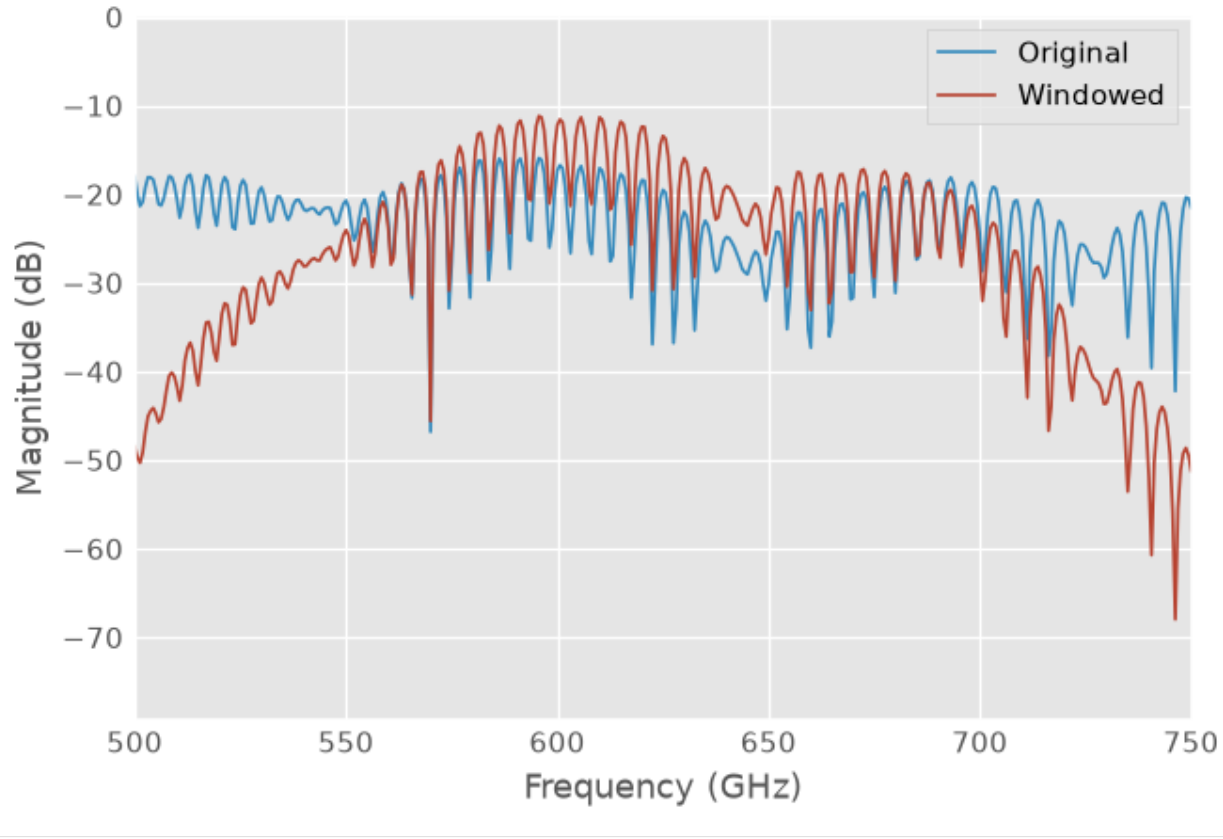
- converts to time domain
- takes magnitude component, convert to dB
- calculates time-axis s
- plots

A word about step 1: **windowing**. A FFT represents a signal with a basis of periodic signals (sinusoids). If your frequency response is not periodic, which in general it isn't, taking a FFT will introduces artifacts in the time-domain results. To minimize these effects, the frequency response is *windowed*. This makes the frequency response more periodic by tapering off the band-edges.

Windowing is just applied to improve the plot appearance,d it does not affect the original network.

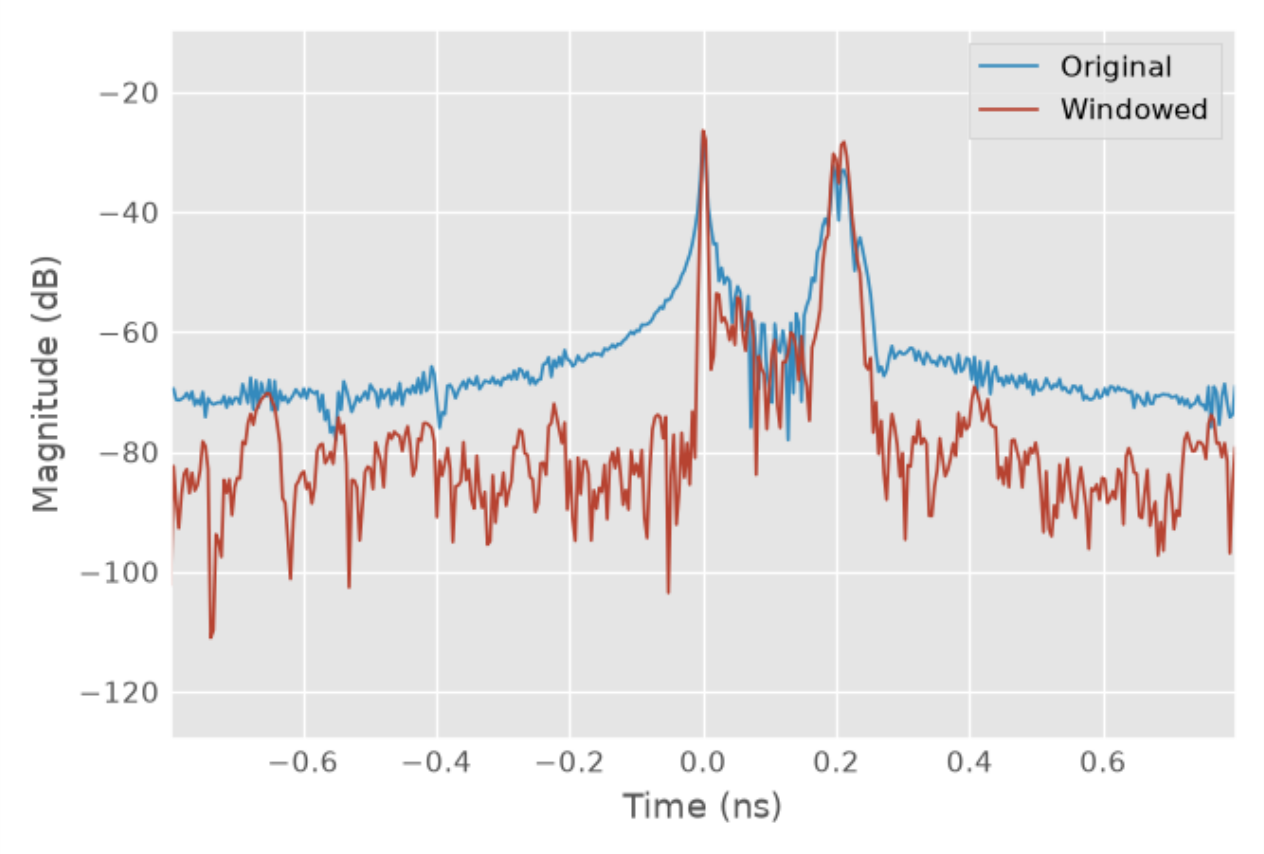
In skrf this can be done explicitly using the `'windowed()'` function. By default this function uses the hamming window, but can be adjusted through arguments. The result of windowing is show below.

[11]:
```
probe_w = probe.windowed()
probe.plot_s_db(0,0, label = 'Original')
probe_w.plot_s_db(0,0, label = 'Windowed')
```



Comparing the two time-domain plotting functions, we can see the difference between windowed and not.

[12]:
```
probe.plot_s_time_db(0,0, label = 'Original')
probe_w.plot_s_time_db(0,0, label = 'Windowed')
```



[ ]:

2.1.13 Time domain reflectometry, measurement vs simulation

This example demonstrate the use of frequency to time domain transformation by comparing measurements and simulations of a *microstripline* and a *microstripline with stepped impedance sections*.

The simulation data is generated by `skrf` using a simple transmission line model for connectors and each impedance section. To achieve a reasonable agreement between measured and simulated data, the dielectric permittivity as well as the connector impedance and delay are extracted by optimization. The code for the *simulation* and *parameters optimization* is given at the end of the example.

Data preparation

Setup

[1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import Image
from numpy import absolute, log, sum
from scipy.optimize import minimize

import skrf
from skrf.io.general import read_all_networks
```

(continues on next page)

(continued from previous page)

```
from skrf.media import DefinedAEpTandZ0, MLine

skrf.stylely()
```

Load data into skrf

The measurement was performed the 19th February 2018 on an Anritsu MS46524B 20GHz Vector Network Analyser. The setup is a linear frequency sweep from 1MHz to 10GHz with 1MHz step, 10000 points. If bandwidth 1kHz, 0dBm output power, no smoothing, no averaging. Full two-port calibration with eCal kit.

Considerations about *time resolution* and *range limitation to avoid alias response* are at the end of this document.

[2]:
```
#load all measurement and simulation data into a dictionary
meas = read_all_networks('tdr_measurement_vs_simulation/measurement/')
simu = read_all_networks('tdr_measurement_vs_simulation/simulation/')
```

DC point extrapolation

The measured and simulated data are available from 1 Mhz to 10 GHz with 1 MHz step (harmonic sweep), thus we need to extrapolate the dc point.

[3]:
```
names = ['P1-MSL_Stepped_140-P2',
         'P1-MSL_Thru_100-P2']

meas_dc_ext = meas.copy()
simu_dc_ext = simu.copy()
for n in names:
    meas_dc_ext[n] = meas_dc_ext[n].extrapolate_to_dc(kind='linear')
    simu_dc_ext[n] = simu_dc_ext[n].extrapolate_to_dc(kind='linear')
```

Microstripline

`MSL_Thru_100` is a $L1$ long, $W1$ wide, T thick copper microstripline on a H height substrate with bottom ground plane.

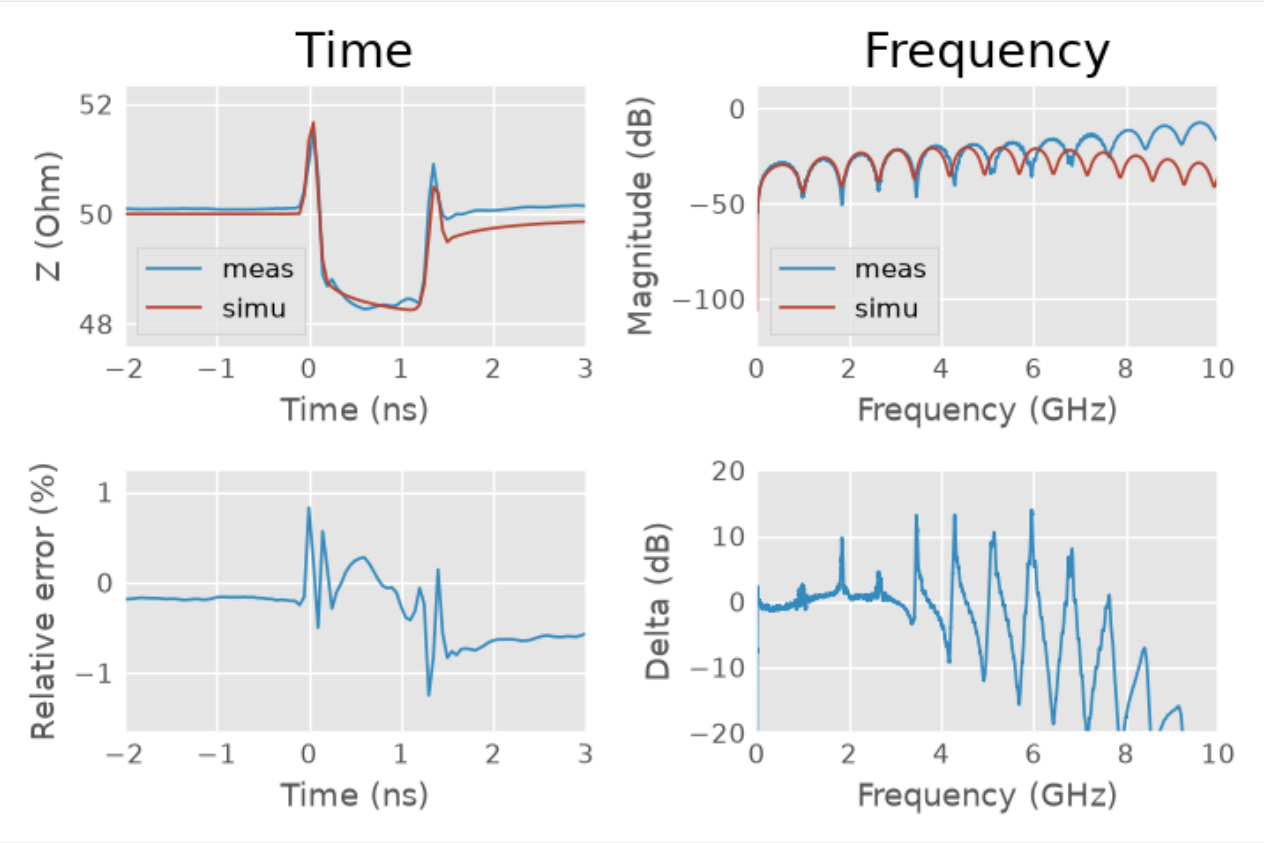
Parameter	Value
$L1$	100mm
$W1$	3.0mm
T	50umm
H	1.5mm
Connector	Cinch, 142-0701-851
Substrate	FR-4

[4]:
```
Image('tdr_measurement_vs_simulation/figures/MSL_100.jpg', width='50%')
```

[4]:

Measurement vs simulation comparison

[5]:
```
plt.figure()
plt.subplot(2,2,1)
plt.title('Time')
meas_dc_ext['P1-MSL_Thru_100-P2'].plot_z_time_step(0, 0, label='meas')
simu_dc_ext['P1-MSL_Thru_100-P2'].plot_z_time_step(0, 0, label='simu')
plt.xlim((-2, 3))

plt.subplot(2,2,2)
plt.title('Frequency')
meas_dc_ext['P1-MSL_Thru_100-P2'].plot_s_db(0, 0, label='meas')
simu_dc_ext['P1-MSL_Thru_100-P2'].plot_s_db(0, 0, label='simu')

plt.subplot(2,2,3)
z0 = 50
t, ymea = meas_dc_ext['P1-MSL_Thru_100-P2'].s11.step_response(window='hamming', pad=0)
ymea[ymea ==  1.] =  1. + 1e-12  # solve numerical singularity
ymea[ymea == -1.] = -1. + 1e-12  # solve numerical singularity
ymea = z0 * (1+ymea) / (1-ymea)
t, ysim = simu_dc_ext['P1-MSL_Thru_100-P2'].s11.step_response(window='hamming', pad=0)
ysim[ysim ==  1.] =  1. + 1e-12  # solve numerical singularity
ysim[ysim == -1.] = -1. + 1e-12  # solve numerical singularity
ysim = z0 * (1+ysim) / (1-ysim)
plt.xlabel('Time (ns)')
plt.ylabel('Relative error (%)')
plt.plot(t*1e9, 100*(ysim-ymea)/ymea)
plt.xlim((-2, 3))

plt.subplot(2,2,4)
delta = simu_dc_ext['P1-MSL_Thru_100-P2'].s_db[:,0,0] - meas_dc_ext['P1-MSL_Thru_100-P2
↪'].s_db[:,0,0]
f = simu_dc_ext['P1-MSL_Thru_100-P2'].f * 1e-9
plt.xlabel('Frequency (GHz)')
plt.ylabel('Delta (dB)')
plt.plot(f, delta)
plt.ylim((-20,20))

plt.tight_layout()
plt.show()
```



Surprisingly, the time domain results shows a very good agreement, within $\pm 2\%$, while the frequency domain results exhibit a reasonable agreement only on the lower half of the frequencies.

This is because the time domain base shape is mostly impacted by the low frequency data.

There is a small offset between the time domain data, sign that the DC point is a bit different. The inductive peaks caused by the connector-to-microstripline transition are clearly visible and their simulation model agree well with measurements.

To go further, it would be possible to measure up to a bigger frequency span, thus increasing the resolution and build the dut on a more frequency stable substrate, but this would be more costly. A cross section of the transmission line could also improve the knowledge of the actual geometry, including manufacturing tolerances.

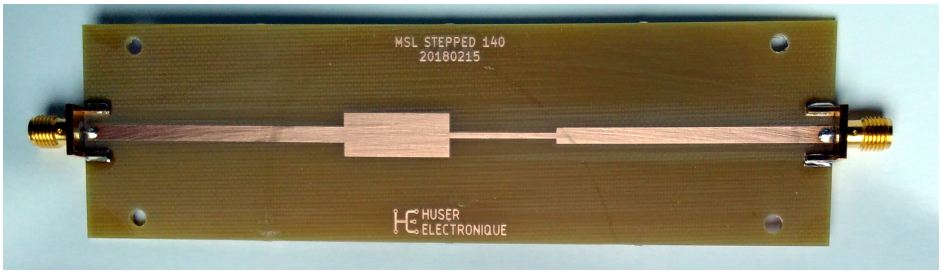
Stepped impedance microstripline

`MSL_Stepped_100` is a stepped microstripline made of T thick copper on a H height substrate with bottom ground plane. The section 1, 2, 3 and 4 from the left to the right, are described with length Lx and width Wx, where x is the section number.

Parameter	Value
$L1$	50mm
$W1$	3.0mm
$L2$	20mm
$W2$	8.0mm (capacitive)
$L3$	20mm
$W3$	1.0mm (inductive)
$L4$	50mm
$W4$	3.0mm
T	50umm
H	1.5mm
Connector	Cinch, 142-0701-851
Substrate	FR-4

[6]:
```
Image('tdr_measurement_vs_simulation/figures/MSL_Stepped_140.jpg', width='75%')
```

[6]:



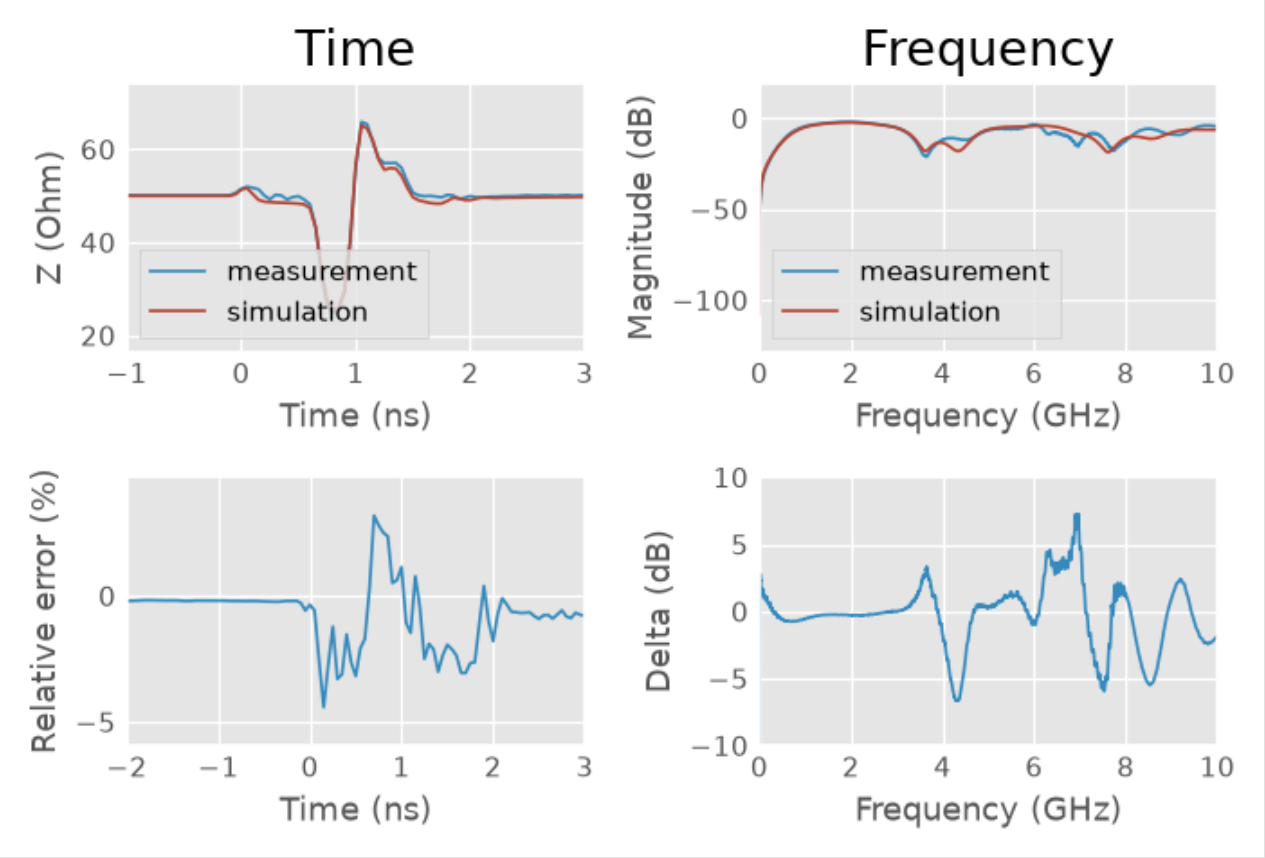
Measurement vs simulation comparison

[7]:
```
plt.figure()
plt.subplot(2,2,1)
plt.title('Time')
meas_dc_ext['P1-MSL_Stepped_140-P2'].plot_z_time_step(0, 0, label='measurement')
simu_dc_ext['P1-MSL_Stepped_140-P2'].plot_z_time_step(0, 0, label='simulation')
plt.xlim((-1, 3))

plt.subplot(2,2,2)
plt.title('Frequency')
meas_dc_ext['P1-MSL_Stepped_140-P2'].plot_s_db(0, 0, label='measurement')
simu_dc_ext['P1-MSL_Stepped_140-P2'].plot_s_db(0, 0, label='simulation')

plt.subplot(2,2,3)
z0 = 50
t, ymea = meas_dc_ext['P1-MSL_Stepped_140-P2'].s11.step_response(window='hamming', pad=0)
ymea[ymea ==  1.] =  1. + 1e-12  # solve numerical singularity
ymea[ymea == -1.] = -1. + 1e-12  # solve numerical singularity
ymea = z0 * (1+ymea) / (1-ymea)
t, ysim = simu_dc_ext['P1-MSL_Stepped_140-P2'].s11.step_response(window='hamming', pad=0)
ysim[ysim ==  1.] =  1. + 1e-12  # solve numerical singularity
ysim[ysim == -1.] = -1. + 1e-12  # solve numerical singularity
ysim = z0 * (1+ysim) / (1-ysim)
plt.xlabel('Time (ns)')
```

(continues on next page)

(continued from previous page)

```
plt.ylabel('Relative error (%)')
plt.plot(t*1e9, 100*(ysim-ymea)/ymea)
plt.xlim((-2, 3))

plt.subplot(2,2,4)
delta = simu_dc_ext['P1-MSL_Stepped_140-P2'].s_db[:,0,0] - meas_dc_ext['P1-MSL_Stepped_
↪140-P2'].s_db[:,0,0]
f = simu_dc_ext['P1-MSL_Stepped_140-P2'].f * 1e-9
plt.xlabel('Frequency (GHz)')
plt.ylabel('Delta (dB)')
plt.plot(f, delta)
plt.ylim((-10,10))

plt.tight_layout()
plt.show()
```



Both time domain and the frequency domain results shows a reasonable agreement, within ±5% for time domain. The frequency domain results exhibit a good agreement, within ±dB only on the lower half of the frequencies, and the upper frequencies shows better agreement than in the previous *microstripline* case.

An explanation to the better frequency domain agreement is that the impedance steps induce such a discontinuities that more energi is reflected back, leading to a shorter effective length and an increased accuracy (more signal).

The capacitive and inductive section of the stepped line are clearly visible and their simulation model agree well with measurements. The first connector effect is hidden by the plot scale, while the second is masked by the reflexions.

To go further, it would again be possible to measure up to a bigger frequency span, thus increasing the resolution and

build the dut on a more frequency stable substrate, but this would be more costly. A cross section of the transmission line could also improve the knowledge of the actual geometry, including manufacturing tolerances.

Eventually, the stepped discontinuities could be made smaller, to produce a smaller effect on the overall measurement (each discontinuity has an influence on the following time domain signal shape).

Parameters optimization

Dielectric effective relative permittivity and loss tangent characterisation based on multiline method

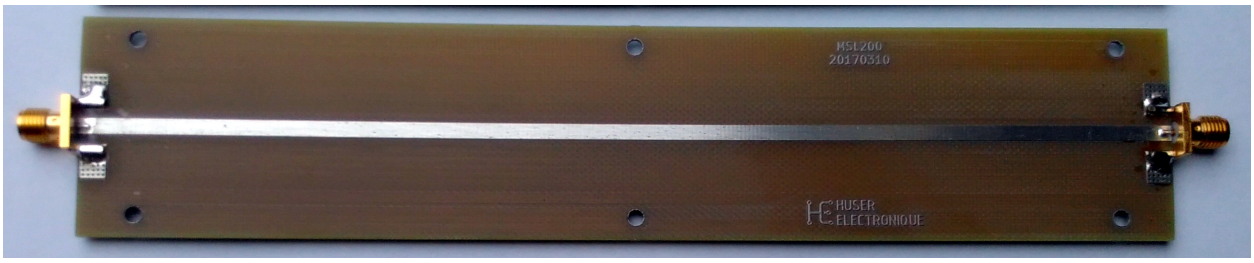
Only two lines with different lengths are required for dielectric permittivity and loss tangent characterisation. Since we have measured the reflects too, we will use them instead of fake reflects. We don't have the switch terms either, but as we only extract the dielectric permittivity and loss tangent rather that doing a real calibration, there is no problem with that.

Multiline calibration algorithm get rid of connectors effects by using multiple lengths of lines. The docstring explain that *At every frequency point there should be at least one line pair that has a phase difference that is not 0 degree or a multiple of 180 degree otherwise calibration equations are singular and accuracy is very poor.* These conditions will not be met with the chosen lines combination, but we will still be able to get a decent estimation of the dielectric permittivity and loss tangent.

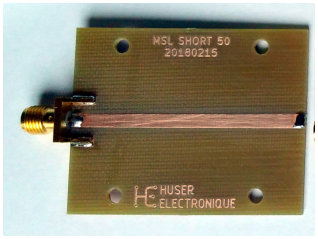
[8]:
```
Image('tdr_measurement_vs_simulation/figures/MSL_100.jpg', width='50%')
```

[8]:

[9]:
```
Image('tdr_measurement_vs_simulation/figures/MSL_200.jpg', width='100%')
```

[9]:


[10]:
```
Image('tdr_measurement_vs_simulation/figures/MSL_Short_50.jpg', width='25%')
```

[10]:


Perform NISTMultilineTRL algorithm

Discard the *No switch term provided* warning.

[11]:
```
# Perform NISTMultilineTRL algorithm
line100mm = meas['P1-MSL_Thru_100-P2']
line200mm = meas['P1-MSL_Thru_200-P2']
short50mm = skrf.network.two_port_reflect(meas['P1-MSL_Short_50'], meas['P2-MSL_Short_50
↪'])
measured  = [line100mm, short50mm, line200mm]
Grefls    = [-1]
lengths   = [100e-3, 200e-3] # in meter
offset    = [50e-3] # in meter
cal = skrf.calibration.NISTMultilineTRL(measured, Grefls, lengths, er_est=4.5, refl_
↪offset=offset)
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:2948: UserWarning: No switch terms provided
  EightTerm.__init__(self,
```

Relative dielectric permittivity and loss tangent

The `NISTMultilineTRL` calibration give the frequency-dependent effective relative dielectric of the geometry, which is a mix between air and substrate relative permittivity.

Unfortunately, a single value of relative dielectric permittivity of the substrate at a given frequency is required for the microstripline media simulation, instead of the frequency-dependent effective value of the geometry we get from the calibration.

To overcome this difficult situation, the microstripline model which will be used is fitted by optimization on the calibration results in such a way the difference between the frequency-dependent effective relative dielectric permittivity of both dataset is minimised.

Additionally, a weighted contribution of the dielectric loss tangent is inserted in the optimization to minimise the difference between the measured modelled attenuation.

The optimization results are dielectric ϵ_r and $\tan\delta$ at 1 GHz and will be used in the dielectric dispersion model.

[12]:
```
# frequency axis
freq  = line100mm.frequency
f     = line100mm.frequency.f
f_ghz = line100mm.frequency.f/1e9

# the physical dimensions of the lines are known by design (neglecting manufacturing␣
↪tolerances)
W   = 3.00e-3
H   = 1.55e-3
T   = 50e-6
L     = 0.1

# calibration results to compare against
ep_r_mea = cal.er_eff.real
A_mea    = 20/log(10)*cal.gamma.real

# starting values for the optimizer
A     = 0.0
```

(continues on next page)

(continued from previous page)

```
f_A   = 1e9
ep_r0 = 4.5
tanD0 = 0.02
f_epr_tand  = 1e9
x0 = [ep_r0, tanD0]

# function to be minimised
def model(x, freq, ep_r_mea, A_mea, f_ep):
    ep_r, tanD = x[0], x[1]
    m = MLine(frequency=freq, Z0=50, w=W, h=H, t=T,
            ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tanD, rough=0.15e-6,
             f_low=1e3, f_high=1e12, f_epr_tand=f_ep,
             diel='djordjevicsvensson', disp='kirschningjansen')
    ep_r_mod = m.ep_reff_f.real
    A_mod = m.alpha * 20/log(10)
    return sum((ep_r_mod - ep_r_mea)**2)  + 0.1*sum((A_mod - A_mea)**2)

# run optimizer
res = minimize(model, x0, args=(freq, ep_r_mea, A_mea, f_epr_tand),
               bounds=[(4.0, 5.0), (0.001, 0.1)])

# get the results and print the results
ep_r, tanD = res.x[0], res.x[1]
print(f'epr={ep_r:.3f}, tand={tanD:.4f} at {f_epr_tand * 1e-9:.1f} GHz.')

# build the corresponding media
m = MLine(frequency=freq, Z0=50, w=W, h=H, t=T,
         ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tanD, rough=0.15e-6,
         f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
         diel='djordjevicsvensson', disp='kirschningjansen')
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/media/
↪mline.py:276: RuntimeWarning: Conductor loss calculation invalid for lineheight t (5e-
↪05)  < 3 * skin depth (6.5852461309085 66e-05)
  self.alpha_conductor, self.alpha_dielectric = self.analyse_loss(
```

```
epr=4.450, tand=0.0174 at 1.0 GHz.
```

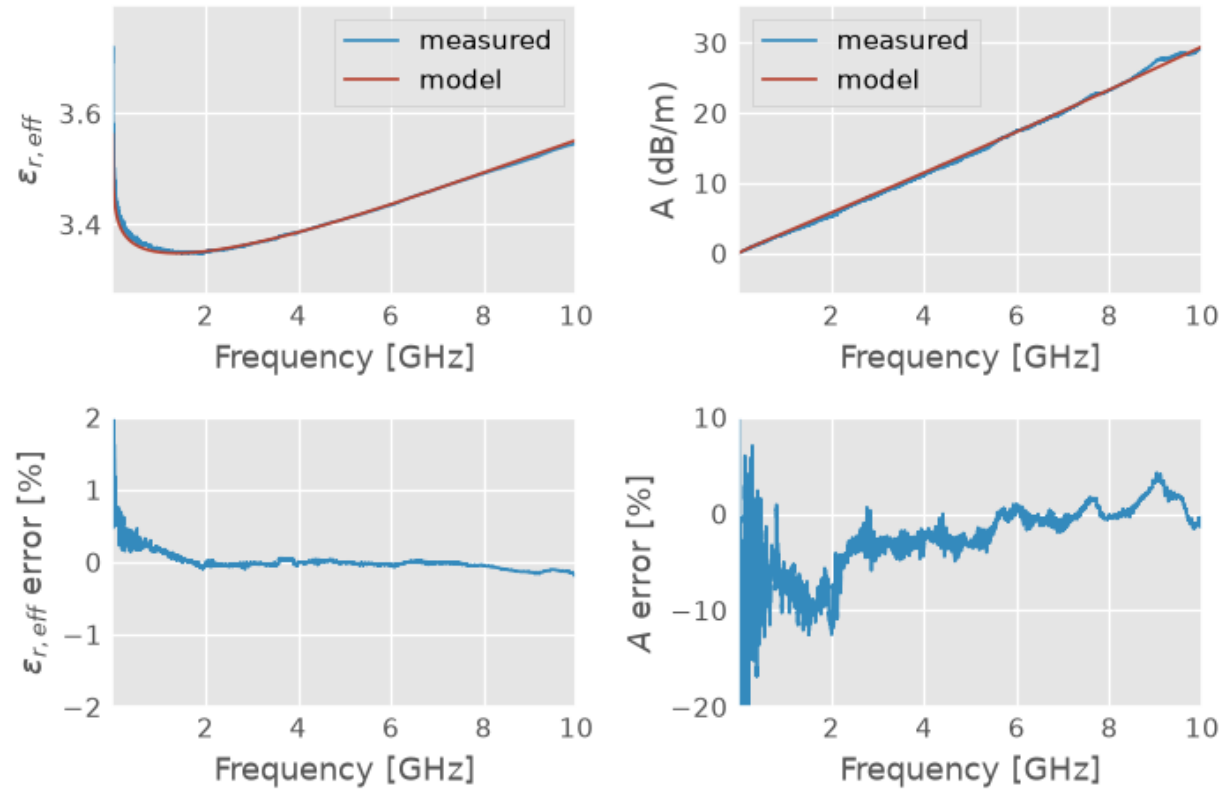
Calibration based values are plotted against modelled value as a sanity check.

[13]:
```
plt.figure()
plt.subplot(2,2,1)
plt.xlabel('Frequency [GHz]')
plt.ylabel(r'$\epsilon_{r,eff}$')
plt.plot(f_ghz, ep_r_mea, label='measured')
plt.plot(f_ghz, m.ep_reff_f.real, label='model')
plt.legend()

plt.subplot(2,2,2)
plt.xlabel('Frequency [GHz]')
plt.ylabel('A (dB/m)')
plt.plot(f_ghz, A_mea, label='measured')
A_mod = 20/log(10)*m.alpha
```

(continues on next page)

(continued from previous page)

```
plt.plot(f_ghz, A_mod, label='model')
plt.legend()

plt.subplot(2,2,3)
plt.xlabel('Frequency [GHz]')
plt.ylabel(r'$\epsilon_{r,eff}$ error [%]')
rel_err = 100 * ((ep_r_mea - m.ep_reff_f.real)/ep_r_mea)
plt.plot(f_ghz, rel_err)
plt.ylim((-2,2))

plt.subplot(2,2,4)
plt.xlabel('Frequency [GHz]')
plt.ylabel('$A$ error [%]')
rel_err = 100 * ((A_mea - A_mod)/A_mea)
plt.plot(f_ghz, rel_err)
plt.ylim((-20,10))

plt.tight_layout()
plt.show()
```



The agreement between measurements and the model seems very reasonable. Relative error of $\epsilon_{r,eff}$ stay within $\pm 1\%$ outside very low frequencies and relative error of A is kept between $\pm 10\%$ on most of the range. Considering the shape of A, it is not possible to do much better with this model.

Connector effect characterization

The `NISTMultilineTRL` calibration coefficients contain information about the connector characteristics, which is corrected by the calibration.

These coefficients can be used to fit a connector model based on a transmission line section.

Delay and attenuation

[14]:
```
# extract connector characteristic from port 1 error coefficients
conn = skrf.calibration.error_dict_2_network(cal.coefs, cal.frequency, is_
↪reciprocal=True)[0]
```

Estimate connector delay with linear regression on unwrapped phase.

[15]:
```
# connector delay estimation by linear regression on the unwrapped phase
xlim = 9000 # used to avoid phase jump if any
phi_conn = (np.angle(conn.s[:xlim,1,0]))
z = np.polyfit(f[:xlim], phi_conn, 1)
p = np.poly1d(z)
delay_conn = -z[0]/(2*np.pi)
print(f'Connector delay: {delay_conn * 1e12:.1f} ps')
```

```
Connector delay: 41.6 ps
```

Build connector model and compare it against calibration extracted data.

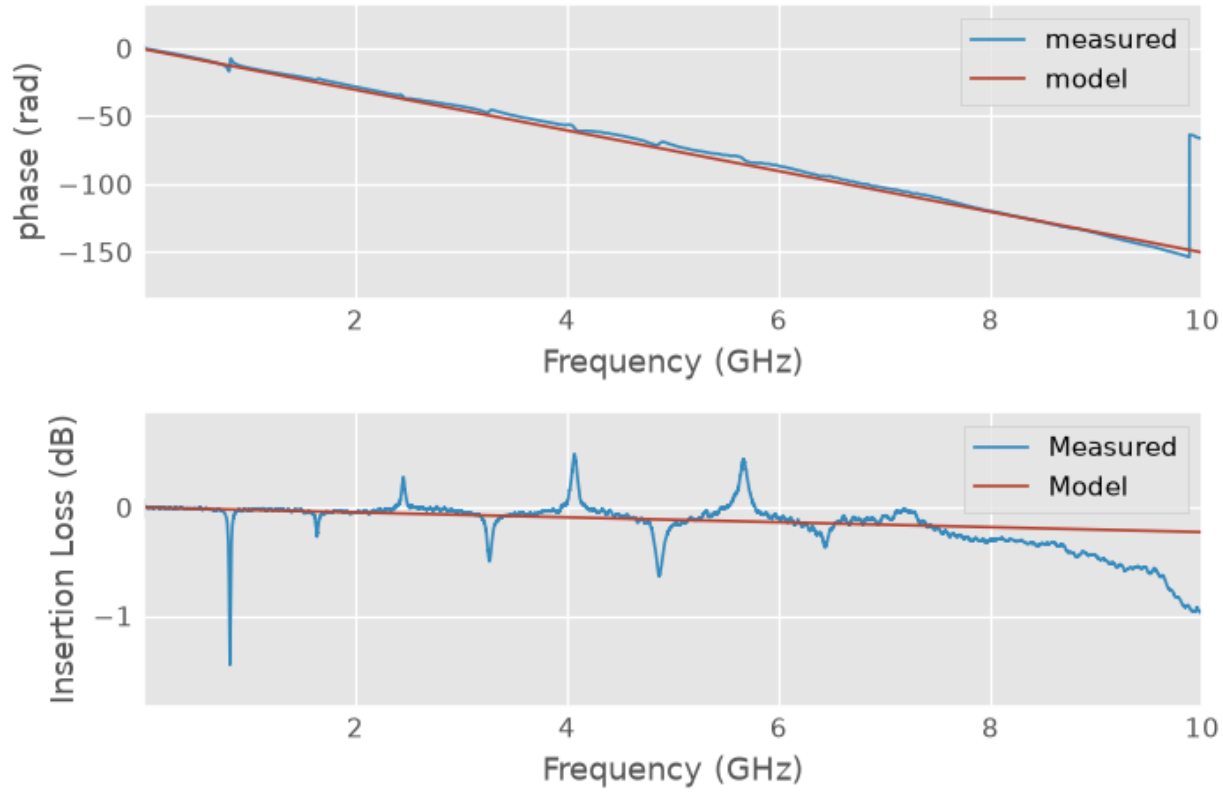
[16]:
```
mc = DefinedAEpTandZ0(m.frequency, ep_r=1, tanD=0.02, Z0=50,
                                f_low=1e3, f_high=1e18, f_ep=f_epr_tand, model=
↪'frequencyinvariant')

Z0_conn = 50.0 # the actual connector characteristic impedance will be tuned later
left = mc.line(delay_conn, 's', z0=Z0_conn)
check = mc.thru() ** left ** mc.thru()

plt.figure()
plt.subplot(2,1,1)
conn.plot_s_deg(1, 0, label='measured')
check.plot_s_deg(1, 0, label='model')
plt.ylabel('phase (rad)')
plt.legend()

plt.subplot(2,1,2)
conn.plot_s_db(1, 0, label='Measured')
check.plot_s_db(1, 0, label='Model')
plt.xlabel('Frequency (GHz)')
plt.ylabel('Insertion Loss (dB)')
plt.legend()

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_7048/1413003635.py:1: DeprecationWarning: Use of `Z0` in DefinedAEpTandZ0␣
↪initialization is deprecated.
`Z0` has no effect. Use `z0` instead
```

(continues on next page)

(continued from previous page)

```
`Z0` will be removed in version 1.0
  mc = DefinedAEpTandZ0(m.frequency, ep_r=1, tanD=0.02, Z0=50,
```



Comparison of connector model characteristics against calibration results shows a reasonable agreement. Calibration results exhibit some glitches that do not correspond to the expected physical behavior. They are caused by the calibration being close to singular due to the thru and line phase being a multiple of 180 degrees. Accuracy could be enhanced by feeding more distinct lines to the algorithm, but these are not manufactured yet.

Characteristic impedance

We now have estimated connector delay and attenuation, but what about the characteristic impedance? This value is required to properly parametrize the transmission line section model.

Optimization is used to find the characteristic impedance that minimize the difference between modelled and measured return loss.

[17]:
```
s11_ref = conn.s[:,0,0]
x0 = [Z0_conn]

# function to be minimised
def model2(x, mc, delay_conn, s11_ref):
    Z0_mod = x[0]
    conn_mod = mc.line(delay_conn, 's', z0=Z0_mod)
    check = mc.thru(z0 = 50.) ** conn_mod ** mc.thru(z0 = 50.)
    s11_mod = check.s[:,0,0]
```

(continues on next page)

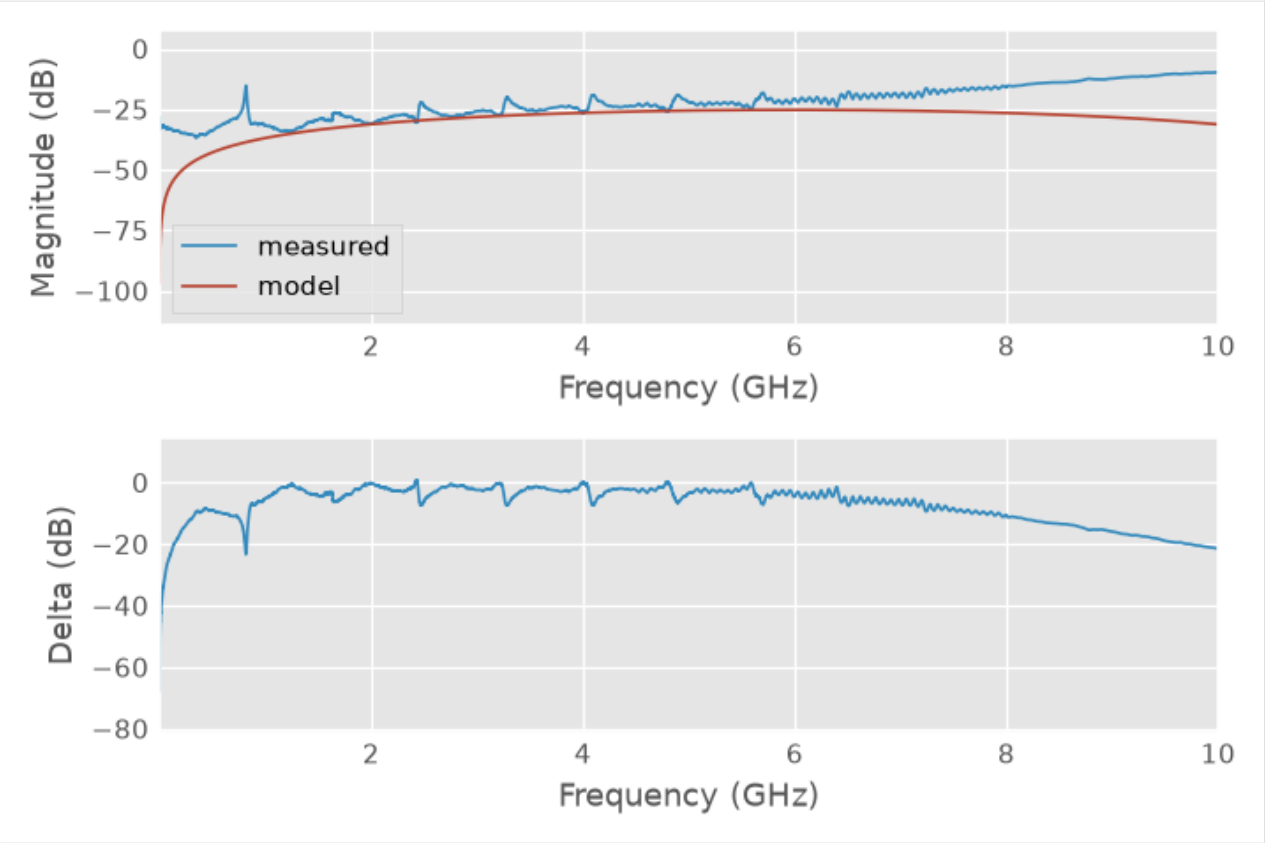
(continued from previous page)

```
    return sum(absolute(s11_ref-s11_mod))

# run optimizer
res = minimize(model2, x0, args=(mc, delay_conn, s11_ref),
               bounds=[(20, 100)])

# get the results and print the results
Z0_conn = res.x[0]
print(f'Z0_conn={Z0_conn:.1f} ohm.')
```

```
Z0_conn=52.9 ohm.
```

The modelled results are plotted against the calibration data, as a sanity check.

[18]:
```
conn_mod = mc.line(delay_conn, 's', z0=Z0_conn)
check = mc.thru(z0 = 50.) ** conn_mod ** mc.thru(z0 = 50.)

plt.figure()
plt.subplot(2,1,1)
conn.plot_s_db(0,0, label = 'measured')
check.plot_s_db(0,0, label = 'model')


plt.subplot(2,1,2)
plt.plot(check.f*1e-9, (check.s_db[:,0,0]-conn.s_db[:,0,0]))
plt.ylabel('Delta (dB)')
plt.xlabel('Frequency (GHz)')

plt.tight_layout()
plt.show()
```



The delta in dB is quite big in low and high frequencies, but we will see when comparing measurement and simulation that the time domain reflectometry results are very decent with this value. The connector effects inductive peaks are well rendered in the case of the *microstripline*.

Simulation

Frequency axis

[19]:
```
freq = skrf.Frequency(1,10e3,10000, 'mhz')
```

Media sections with different geometries

[20]:
```
# 50 ohm segment
MSL1 = MLine(frequency=freq, z0_port=50, w=W, h=H, t=T,
        ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tanD, rough=0.15e-6,
        f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
        diel='djordjevicsvensson', disp='kirschningjansen')

# Capacitive segment
MSL2 = MLine(frequency=freq, z0_port=50, w=8.0e-3, h=H, t=T,
        ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tanD, rough=0.15e-6,
        f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
        diel='djordjevicsvensson', disp='kirschningjansen')

# Inductive segment
```

(continues on next page)

(continued from previous page)

```
MSL3 = MLine(frequency=freq, z0_port=50, w=1.0e-3, h=H, t=T,
        ep_r=ep_r, mu_r=1, rho=1.712e-8, tand=tanD, rough=0.15e-6,
        f_low=1e3, f_high=1e12, f_epr_tand=f_epr_tand,
        diel='djordjevicsvensson', disp='kirschningjansen')

# Connector transmission line media with guessed loss
MCON = DefinedAEpTandZ0(m.frequency, z0_port=50, ep_r=1, tanD=0.025, z0=Z0_conn,
        f_low=1e3, f_high=1e18, f_ep=f_epr_tand, model='frequencyinvariant')
```

Simulated devices under test

[21]:
```
# SMA connector
conn_sma  = MCON.line(delay_conn, 's')


# microstripline
thru_simu = conn_sma ** MSL1.line(100e-3, 'm') ** conn_sma
thru_simu.name = 'P1-MSL_Thru_100-P2'

# stepped impedance microstripline
step_simu = conn_sma \
            ** MSL1.line(50e-3, 'm') \
            ** MSL2.line(20e-3, 'm') \
            ** MSL3.line(20e-3, 'm') \
            ** MSL1.line(50e-3, 'm') \
            ** conn_sma
step_simu.name = 'P1-MSL_Stepped_140-P2'

# write simulated data to .snp files
write_data = False
if write_data:
    step_simu.write_touchstone(dir='tdr_measurement_vs_simulation/simulation/')
    thru_simu.write_touchstone(dir='tdr_measurement_vs_simulation/simulation/')
```

Notes

Time resolution

After DC point extrapolation, and neglecting the effect of windowing, the time resolution of measurement is

$$Resolution = \frac{1}{f_{span}}$$

where $Resolution$ is the resolution in s and f_{span} is the frequency span in Hz.

In our case, with $f_{span} = 10\,[\text{GHz}]$

$$Resolution \approx \frac{1}{10^{10}} \approx 100 \quad [\text{ps}]$$

With an effective relative dielectric permittivity of about 3.5, this give following physical resolution:

$$Resolution_{meter} = \frac{Resolution c_0}{\sqrt{\epsilon_r}} \approx \frac{10^{-10} 3 10^8}{\sqrt{3.5}} \approx 16 \quad [\text{mm}]$$

We will use reflexion measurement, so the actual range is divided by two, because the signal goes back and forth. Thus, the approximate distance resolution on our device under test will be 8mm. The discontinuities on the stepped impedance microstripline and the connector effects shall be well visible.

Measurement range limitation to avoid alias response

The measurement range should be set in such a way the true response of the device is seen without repetitions (alias).

$$Range = \frac{c_0}{\sqrt{\epsilon_r}\Delta_f}$$

Where $Range$ is the range in m, c_0 the speed of light in m/s, Δ_f the frequency step in Hz and ϵ_r is the effective relative permittivity constant of the device under test.

With the given measurement setup and considering $\Delta_f = 1\,[\mathrm{MHz}]$, worst case $\epsilon_r \approx 5.0$, the range is

$$Range \approx \frac{3 10^8}{\sqrt{5} 10^6} \approx 134 \quad [\mathrm{m}]$$

Relative dielectric permittivity for FR-4 is approximately 5.0, the effective ϵ_r of microstripline geometry will be smaller because of the air on top, which give a safety margin, the range being underevaluated.

We will use reflexion measurement, so the actual range is divided by two, because the signal goes back and forth. However, our longest device being 200mm long, we can just quietly forget aliasing.

[ ]:

2.1.14 Transmission Line Losses on a Loaded Lossy Line

When dealing with RF power, for instance in radio, industry or scientific applications, a recurrent problem is to handle the inevitable RF losses correctly to avoid overheating of cables and components.

Matched Load

In this example, we will use `scikit-rf` to evaluate the losses in a 50 Ohm, 20 meters long, RG-8 cable (VF=0.84) charged with a resistive load $R_L = 50\Omega$ at 13.56 MHz. The cable losses are estimated to 1.483 dB/100 meters and the source power to 400W.

First, the usual Python imports:

[1]:
```
%matplotlib inline
```

[2]:
```
import numpy as np

import skrf as rf
from skrf.constants import c
from skrf.mathFunctions import db_2_np, db_per_100feet_2_db_per_100meter, feet_2_meter,␣
↪mag_2_db10
from skrf.tlineFunctions import (
    Gamma0_2_swr,
    input_impedance_at_theta,
    voltage_current_propagation,
    zl_2_Gamma0,
    zl_2_Gamma_in,
    zl_2_total_loss,
```

(continues on next page)

(continued from previous page)

```
    zl_2_zin,
)
```

[3]:
```
rf.stylely()
```

Let's define the problem constants:

[4]:
```
Pin = 400  # W
z0 = 50 # Ohm
freq = rf.Frequency(13.56, npoints=1, unit='MHz')
VF = 0.84
RL = 50  # Ohm
L = 20  # m
```

The propagation constant of the transmission line $\gamma = \alpha + j\beta$ is:

[5]:
```
alpha = db_2_np(1.483/100)  # Np/m
beta = freq.w/c/VF
gamma = alpha + 1j*beta
```

The matched line loss (or power attenuation), $a = e^{2\alpha L}$, is:

[6]:
```
a = np.exp(2*alpha*L)  # also simply 2.84/100*20
print('Matched line loss: a=', mag_2_db10(a), 'dB')
```

```
Matched line loss: a= 0.29660000000000003 dB
```

If the line is charged with a matched load, ie. $R_L = 50\Omega$, then the total line loss is a. The power lost in the cable will thus be:

[7]:
```
print('(Forward) Power delivered to the  load:', Pin/a, 'W')
print('Power lost in the cable:', Pin *( 1 - 1/a), 'W')
```

```
(Forward) Power delivered to the  load: 373.59408479883416 W
Power lost in the cable: 26.405915201165886 W
```

Which can also be checked by the `scikit-rf` transmission line function `zl_2_total_loss`:

[8]:
```
a_skrf = zl_2_total_loss(z0, zl=RL, theta=gamma*L)
print('Power lost in the cable:',
      Pin * (1 - 1/a_skrf), 'W')
```

```
Power lost in the cable: [26.4059152] W
```

Another way to evaluate the total power dissipated in the circuit is to evaluate the power expression:

$$P_{delivered} = \frac{1}{2}\Re\left[VI^*\right]$$

where V and I are the (peak) total voltage and current. They can be evaluate using the transmission line function `voltage_current_propagation`:

[9]:
```
# reflection coefficient and input impedance
Gamma_in = zl_2_Gamma_in(z0, RL, theta=gamma*L)
Z_in = zl_2_zin(z0, RL, theta=gamma*L)
```

(continues on next page)

(continued from previous page)

```
# voltage and current at the line input as a function of source power
V_in = np.sqrt(2*z0*Pin)*(1 + Gamma_in)
I_in = V_in/Z_in

# voltage and current at z=L
V,I = voltage_current_propagation(V_in, I_in, z0, gamma*L)
P_delivered = 1/2 * np.real(V * np.conj(I))
print('Power delivered to the load: ', P_delivered, 'W')
print('Power dissipated in the cable: ',Pin - P_delivered, 'W')
```

```
Power delivered to the load:  [373.5940848] W
Power dissipated in the cable:  [26.4059152] W
```

Non Matched load

However, if the load is not perfectly matched to the line characteristic impedance z_0, for example with $R_L = 200 + 30j\Omega$, additional losses are induced by the reflected wave. The reflection coefficient Γ_{load} induced by this load is:

[10]:
```
z0 = 50
ZL = 200 - 30j
Gamma_load = zl_2_Gamma0(z0, ZL)
print('|Gamma_load|=', np.abs(Gamma_load))
```

```
|Gamma_load|= [0.6075238]
```

while the reflection coefficient seen at the input of the transmission line Γ_{in} is:

[11]:
```
Gamma_in = zl_2_Gamma_in(z0, ZL, theta=gamma*L)
SWR = Gamma0_2_swr(zl_2_Gamma_in(z0, ZL, theta=gamma*L))
print('|Gamma_in|=', np.abs(Gamma_in), '(SWR=', SWR,')')
```

```
|Gamma_in|= [0.56741825] (SWR= [3.62340355] )
```

The total loss in dB due to SWR is often stated as:

$$a_{[dB]} + 10\log_{10}\frac{1-|\Gamma_{in}|^2}{1-|\Gamma_{load}|^2}$$

[12]:
```
10*np.log10(a) + 10*np.log10((1 - np.abs(Gamma_in)**2)/(1 - np.abs(Gamma_load)**2))
```

[12]:
```
array([0.60942359])
```

However, this expression is only correct if either properties are verified:

- (i) the characteristic impedance of the line is real (distorsionless line)
- (ii) reflection coefficient is real (ie real Z_L/Z_0) [1].

The 1st condition is met here, however it will not be the case in the next section.

Anyway, the `scikit-rf` transmission line function `zl_2_total_loss` is correct in all conditions:

[13]:
```
a = zl_2_total_loss(z0, zl=ZL, theta=gamma * L)
print('Total power loss: ', mag_2_db10(a), 'dB' )
print('Delivered power:', Pin/a, 'W')
print('The total power loss is the cable:', Pin*(1 - 1/a), 'W')
```

```
Total power loss:  [0.60942359] dB
Delivered power: [347.63030721] W
The total power loss is the cable: [52.36969279] W
```

[14]:
```
# reflection coefficient and input impedance
Gamma_in = zl_2_Gamma_in(z0, ZL, theta=gamma*L)
Z_in = zl_2_zin(z0, ZL, theta=gamma*L)

# voltage and current at the line input as a function of source power
V_in = np.sqrt(2*z0*Pin)*(1 + Gamma_in)
I_in = V_in/Z_in

# voltage and current at z=L
V,I = voltage_current_propagation(V_in, I_in, z0, gamma*L)
P_delivered = 1/2 * np.real(V * np.conj(I))
print('Power delivered to the load: ', P_delivered, 'W')
print('Power dissipated in the cable: ',Pin - P_delivered, 'W')
```

```
Power delivered to the load:  [235.70604719] W
Power dissipated in the cable:  [164.29395281] W
```

[15]:
```
Gamma0_2_swr(Gamma_in)
```

[15]:
```
array([3.62340355])
```

[16]:
```
10*np.log10(P_delivered/Pin)
```

[16]:
```
array([-2.29689267])
```

A more advanced example

This example reproduces the example presented in reference [1].

Let's assume a coaxial line (Wireman #551, 450-Ohm) loaded with a complex load $Z_L = R_L + jX_L$, with the following parameters:

- line length: 100 feet
- frequency: 1.83 MHz
- attenuation constant: $\alpha = 0.095$ dB/100 feet
- coaxial relative permittivity: $\epsilon_r = 1.194$ (Velocity Factor VF=0.915)
- real part of the characteristic impedance: $R_0 = \Re[Z_0]$=402.75 Ohm
- Load resistance: R_L = 4.5 Ohm
- Load reactance: X_L = -1673 Ohm

[17]:
```
Z_L = 4.5 - 1673j
R_0 = 402.75
freq = rf.Frequency(1.83, npoints=1, unit='MHz')
VF = 0.915
L = feet_2_meter(100)
```

First, we can derive the propagation constant $\gamma = \alpha + j\beta$ with $\beta = \frac{\omega}{c}\sqrt{\epsilon_r}$ from the problem parameters:

[18]:
```
alpha = db_2_np(db_per_100feet_2_db_per_100meter(0.095)/100)
beta = freq.w/c/VF
gamma = alpha + 1j*beta
print(gamma)
```

```
[0.00035883+0.0419169j]
```

However, the transmission line characteristic reactance is not given in the problem parameters and must be determined. It can be approximated from a high-frequency, low-loss approximation [1]:

$$Z_0 = R_0 + jX_0 \approx R_0 - j\frac{\alpha}{\beta}R_0$$

ie,

$$X_0 \approx -\frac{\alpha}{\beta}R_0$$

[19]:
```
X_0 = -alpha/beta*R_0
Z_0 = R_0 + 1j*X_0
print('X_0=', X_0)
```

```
X_0= [-3.44778935]
```

Now that we have both the characteristic impedance and the propagation constant of the line, the reflection coefficients, input impedance and total loss can be deduced:

[20]:
```
print('Gamma at load:', np.abs(zl_2_Gamma0(Z_0, Z_L)))
print('Gamma at input:', np.abs(zl_2_Gamma_in(Z_0, Z_L, theta=gamma*L)))
print('SWR at load:', Gamma0_2_swr(zl_2_Gamma0(Z_0, Z_L)))

print('SWR at input:', Gamma0_2_swr(zl_2_Gamma_in(Z_0, Z_L, theta=gamma*L)))
print('Input impedance:', input_impedance_at_theta(Z_0, Z_L, theta=gamma*L ), 'Ohm')

total_loss_db = mag_2_db10(np.abs(zl_2_total_loss(z0=Z_0, zl=Z_L, theta=gamma*L)))
print('Total loss:', total_loss_db, 'dB')
```

```
Gamma at load: [0.99489308]
Gamma at input: [0.97336654]
SWR at load: [390.62580446]
SWR at input: [74.09350335]
Input impedance: [5.25687961-22.99765972j] Ohm
Total loss: [13.27540882] dB
```

which match the results presented in reference [1].

References

- [1] Steve Stearns (K6OIK), A Transmission Line Power Paradox and Its Resolution, ARRL Pacificon Antenna Seminar, Santa Clara, CA October 10-12, 2014: https://www.fars.k6ya.org/docs/K6OIK-A_Transmission_Line_Power_Paradox_and_Its_Resolution.pdf

[ ]:

2.1.15 Modeling Transmission Line Properties

Table of Contents

- *Introduction*
- Propagation constant
- Interlude on attenuation units
- Modeling a loaded lossy transmission line using transmission line functions
- Input impedances, reflection coefficients and SWR
- Voltages and Currents
- Modeling a loaded lossy transmission line by cascading Networks
- Determination of the propagation constant from the input impedance

Introduction

In this tutorial, `scikit-rf` is used to work with some classical transmission line situations, such as calculating impedances, reflection coefficients, standing wave ratios or voltages and currents. There is at least two way of performing these calculations, one using transmission line functions or by creating and cascading Networks

Let's consider a lossy coaxial cable of characteristic impedance $Z_0 = 75\Omega$ of length $d = 12m$. The coaxial cable has an attenuation of 0.02 Neper/m and a velocity factor VF=0.67 (This corresponds roughly to a RG-6 coaxial). The cable is loaded with a $Z_L = 150\Omega$ impedance. The RF frequency of interest is 250 MHz.

Please note that in `scikit-rf`, the line length is defined from the load, ie $z = 0$ at the load and $z = d$ at the input of the transmission line:

First, let's make the necessary Python import statements:

[1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf
from skrf.constants import c
from skrf.mathFunctions import db_2_np, db_per_100feet_2_db_per_100meter, feet_2_meter,␣
↪mag_2_db10, np_2_db
from skrf.tlineFunctions import (
    reflection_coefficient_2_propagation_constant,
    voltage_current_propagation,
    zl_2_Gamma0,
    zl_2_Gamma_in,
    zl_2_swr,
    zl_2_total_loss,
    zl_2_zin,
```

(continues on next page)

(continued from previous page)

```
)
```

[2]:
```
# skrf figure styling
rf.stylely()
```

And the constants of the problem:

[3]:
```
freq = rf.Frequency(250, npoints=1, unit='MHz')
Z_0 = 75  # Ohm
Z_L = 150  # Ohm
d = 12  # m
VF = 0.67
att = 0.02 # Np/m. Equivalent to 0.1737 dB/m
```

Before going into impedance and reflection coefficient calculations, first we need to define the transmission line properties, in particular its propagation constant.

Propagation constant

In order to get the RF parameters of the transmission line, it is necessary to derive the propagation constant of the line. The propagation constant γ of the line is defined in `scikit-rf` as $\gamma = \alpha + j\beta$ where α is the attenuation (in Neper/m) and $\beta = \frac{2\pi}{\lambda} = \frac{\omega}{c}/\mathrm{VF} = \frac{\omega}{c}\sqrt{\epsilon_r}$ the phase constant.

First, the wavelength in the coaxial cable is

$$\lambda = \frac{c}{f\sqrt{\epsilon_r}} = \frac{c}{f}\mathrm{VF}$$

[4]:
```
lambd = c/freq.f * VF
print('VF=', VF, 'and Wavelength:', lambd, 'm')
```

```
VF= 0.67 and Wavelength: [0.80344379] m
```

As the attenuation is already given in Np/m, the propagation constant is:

[5]:
```
alpha = att  # Np/m !
beta = freq.w/c/VF
gamma = alpha + 1j*beta
print('Transmission line propagation constant: gamma = ', gamma, 'rad/m')
```

```
Transmission line propagation constant: gamma =  [0.02+7.82031725j] rad/m
```

If the attenuation would have been given in other units, `scikit-rf` provides the necessary tools to convert units, as described below.

Interlude: On Attenuation Units

Attenuation is generally provided (or expected) in various kind of units. `scikit-rf` provides convenience functions to manipulate line attenuation units.

For example, the cable attenuation given in Np/m, can be expressed in dB:

[6]:
```
print('Attenuation dB/m:', np_2_db(att))
```
```
Attenuation dB/m: 0.1737177927613007
```

Hence, the attenuation in dB/100m is:

[7]:
```
print('Line attenuation in dB/100m:', np_2_db(att)*100)
```
```
Line attenuation in dB/100m: 17.37177927613007
```

And in dB/100feet is:

[8]:
```
print('Line attenuation in dB/100ft:', np_2_db(att)*100*feet_2_meter())
```
```
Line attenuation in dB/100ft: 5.2949183233644455
```

If the attenuation would have been given in imperial units, such as dB/100ft, the opposite conversions would have been:

[9]:
```
db_per_100feet_2_db_per_100meter(5.2949)  # to dB/100m
```
[9]:
```
17.371719160104988
```

[10]:
```
db_2_np(5.2949)/feet_2_meter(100)  # to Np/m
```
[10]:
```
np.float64(0.019999930788868397)
```

Using transmission line functions

`scikit-rf` brings few convenient functions to deal with transmission lines. They are detailed in the *transmission line functions* documentation pages.

Input impedances, reflection coefficients and SWR

The reflection coefficient Γ_L induced by the load is given by `zl_2_Gamma0()`:

[11]:
```
Gamma0 = zl_2_Gamma0(Z_0, Z_L)
print('|Gamma0|=', abs(Gamma0))
```
```
|Gamma0|= [0.33333333]
```

and its associated Standing Wave Ratio (SWR) is obtained from `zl_2_swr()`:

[12]:
```
zl_2_swr(Z_0, Z_L)
```
[12]:
```
array([2.])
```

After propagating by a distance d in the transmission line of propagation constant γ (hence having travelled an electrical length $\theta = \gamma d$), the reflection coefficient at the line input is obtained from `zl_2_Gamma_in()`:

[13]:
```
Gamma_in = zl_2_Gamma_in(Z_0, Z_L, theta=gamma*d)
print('|Gamma_in|=', abs(Gamma_in), 'phase=', 180/np.pi*np.angle(Gamma_in))
```
```
|Gamma_in|= [0.20626113] phase= [46.2918563]
```

The input impedance Z_{in} from `zl_2_zin()`:

[14]:
```
Z_in = zl_2_zin(Z_0, Z_L, gamma * d)
print('Input impedance Z_in=', Z_in)
```

```
Input impedance Z_in= [94.79804694+29.52482706j]
```

Like previously, the SWR at the line input is:

[15]:
```
zl_2_swr(Z_0, Z_in)
```

[15]:
```
array([1.51972037])
```

The total line loss in dB is get from `zl_2_total_loss()`:

[16]:
```
mag_2_db10(zl_2_total_loss(Z_0, Z_L, gamma*d))
```

[16]:
```
array([2.40732856])
```

Voltages and Currents

Now assume that the previous circuit is excited by a source delivering a voltage $V = 1V$ associated to a source impedance $Z_s = 100\Omega$:

[17]:
```
Z_s = 100  # Ohm
V_s = 1  # V
```

At the input of the transmission line, the voltage is a voltage divider circuit:

$$V_{in} = V_s \frac{Z_{in}}{Z_s + Z_{in}}$$

[18]:
```
V_in = V_s * Z_in / (Z_s + Z_in)
print('Voltage at transmission line input : V_in = ', V_in, ' V')
```

```
Voltage at transmission line input : V_in =  [0.49817591+0.07605964j]  V
```

and the current at the input of the transmission line is:

$$I_{in} = \frac{V_s}{Z_s + Z_{in}}$$

[19]:
```
I_in = V_s / (Z_s + Z_in)
print('Current at transmission line input : I_in = ', I_in, ' A')
```

```
Current at transmission line input : I_in =  [0.00501824-0.0007606j]  A
```

which represent a power of

$$P_{in} = \frac{1}{2}\Re\left[V_{in} I_{in}^*\right]$$

[20]:
```
P_in = 1/2 * np.real(V_in * np.conj(I_in))
print('Input Power : P_in = ', P_in, 'W')
```

```
Input Power : P_in =  [0.00122106] W
```

The reflected power is:

$$P_r = |\Gamma_{in}|^2 P_{in}$$

[21]:
```
P_r = abs(Gamma_in)**2 * P_in
print('Reflected power : P_r = ', P_r, 'W')
```

```
Reflected power : P_r =  [5.19482699e-05] W
```

The voltage and current at the load can be deduced from the ABCD parameters of the line of length L :

[22]:
```
V_out, I_out = voltage_current_propagation(V_in, I_in, Z_0,theta= gamma*d)
print('Voltage at load: V_out = ', V_out, 'V')
print('Current at load: I_out = ', I_out, 'A')
```

```
Voltage at load: V_out =  [0.41779105+0.18944365j] V
Current at load: I_out =  [0.00278527+0.00126296j] A
```

Note that voltages and currents are expressed a peak values. RMS values are thus:

[23]:
```
print(abs(V_out)/np.sqrt(2), abs(I_out)/np.sqrt(2))
```

```
[0.32437498] [0.0021625]
```

The power delivered to the load is thus:

[24]:
```
P_out = 1/2 * np.real(V_out * np.conj(I_out))
print('Power delivered to the load : P_out = ', P_out, ' W')
```

```
Power delivered to the load : P_out =  [0.00070146]  W
```

Voltage and current are plotted below against the transmission line length (pay attention to the sign of d in the voltage and current propagation: as we go from source ($z = d$) to the load ($z = 0$), θ goes in the opposite direction and should be inversed)

[25]:
```
ds = np.linspace(0, d, num=1001)

thetas = - gamma*ds

v1 = np.full_like(ds, V_in)
i1 = np.full_like(ds, I_in)

v2, i2 = voltage_current_propagation(v1, i1, Z_0, thetas)
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/envs/latest/lib/python3.12/
↪site-packages/numpy/_core/numeric.py:475: ComplexWarning: Casting complex values to␣
↪real discards the imaginary part
  multiarray.copyto(res, fill_value, casting='unsafe')
```

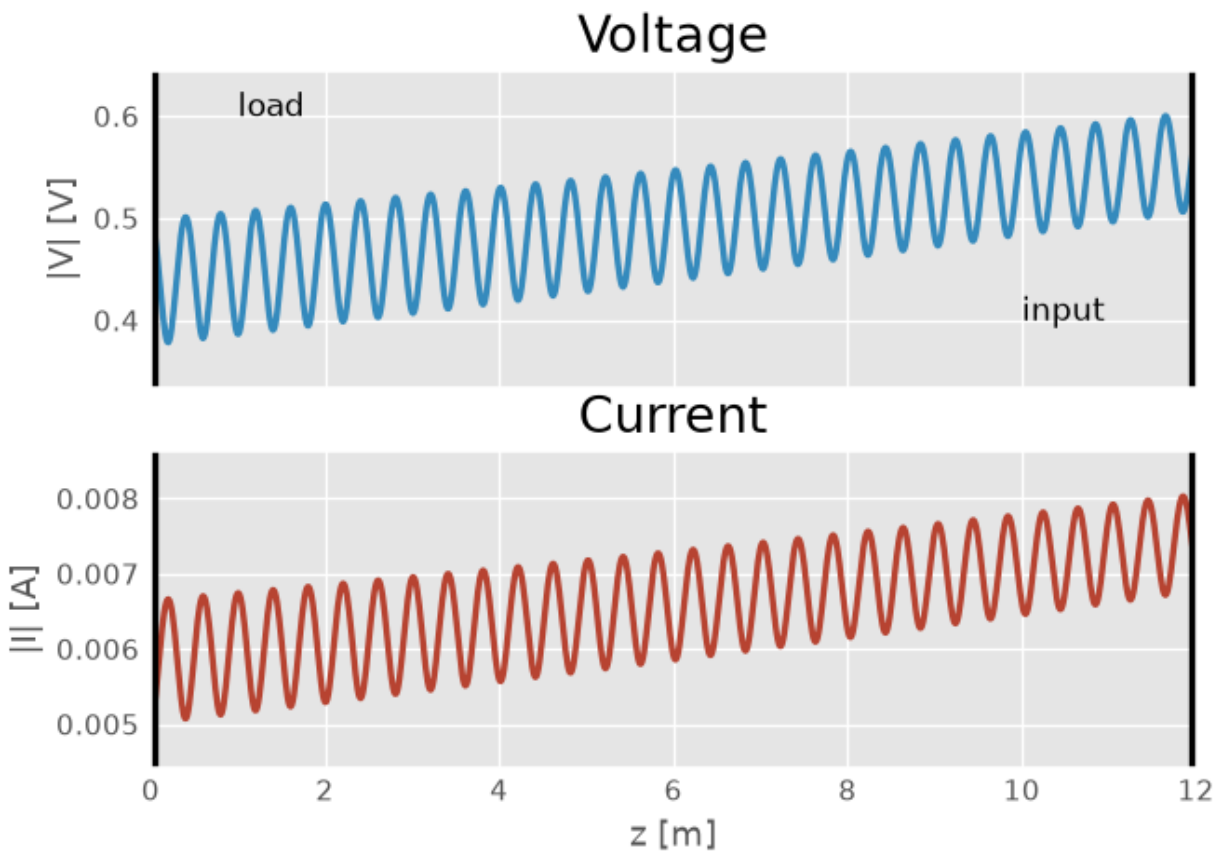
[26]:
```
fig, (ax_V, ax_I) = plt.subplots(2, 1, sharex=True)
ax_V.plot(ds, abs(v2), lw=2)
ax_I.plot(ds, abs(i2), lw=2, c='C1')
ax_I.set_xlabel('z [m]')
ax_V.set_ylabel('|V| [V]')
ax_I.set_ylabel('|I| [A]')
```

(continues on next page)

(continued from previous page)

```
ax_V.axvline(0, c='k', lw=5)
ax_I.axvline(0, c='k', lw=5)
ax_V.text(d-2, 0.4, 'input')
ax_V.text(1, 0.6, 'load')
ax_V.axvline(d, c='k', lw=5)
ax_I.axvline(d, c='k', lw=5)

ax_I.set_title('Current')
ax_V.set_title('Voltage')
```

[26]:
```
Text(0.5, 1.0, 'Voltage')
```



Using `media` objects for transmission line calculations

`scikit-rf` also provides objects representing transmission line mediums. The `Media` object provides generic methods to produce Network's for any transmission line medium, such as transmission line length (`line()`), lumped components (`resistor()`, `capacitor()`, `inductor()`, `shunt()`, etc.) or terminations (`open()`, `short()`, `load()`). For additional references, please see the *media documentation*.

Let's create a transmission line `media` object for our coaxial line of characteristic impedance Z_0 and propagation constant γ:

[27]:
```
# if not passing the gamma parameter, it would assume that gamma = alpha + j*beta = 0 +␣
↪j*1
coax_line = rf.media.DefinedGammaZ0(frequency=freq, z0=Z_0, gamma=gamma)
```

In order to build the circuit illustrated by the figure above, all the circuit's Networks are created and then cascaded with the ** operator:

- transmission line of length d (from the media created above),
- a resistor of impedance Z_L,
- then terminated by a short.

This results in a one-port network, which Z-parameter is then the input impedance:

[28]:
```
ntw = coax_line.line(d, unit='m') ** coax_line.resistor(Z_L) ** coax_line.short()
ntw.z
```

[28]:
```
array([[[94.79804694+29.52482706j]]])
```

Note that full Network can also be built with convenience functions load:

[29]:
```
ntw = coax_line.line(d, unit='m') ** coax_line.load(zl_2_Gamma0(Z_0, Z_L))
ntw.z
```

[29]:
```
array([[[94.79804694+29.52482706j]]])
```

or even more directly using or delay_load:

[30]:
```
ntw = coax_line.delay_load(zl_2_Gamma0(Z_0, Z_L), d, unit='m')
ntw.z
```

[30]:
```
array([[[94.79804694+29.52482706j]]])
```

Determination of the propagation constant from the input impedance

Let's assume the input impedance of a short-circuited lossy transmission line of length d=1.5 m and a characteristic impedance of $Z_0=$100 Ohm has been measured to $Z_{in} = 40 - 280j\Omega$.

The transmission line propagation constant γ is unknown and researched. Let see how to deduce its value using scikit-rf:

[31]:
```
# input data
z_in = 20 - 140j
z_0 = 75
d = 1.5
Gamma_load = -1 # short
```

Since we know the input impedance, we can deduce the reflection coefficient at the input of the transmission line. Since there is a direction relationship between the reflection coefficient at the load and at the input of the line:

$$\Gamma_{in} = \Gamma_L e^{-2\gamma d}$$

we can deduce the propagation constant value γ as:

$$\gamma = -\frac{1}{2d} \ln\left(\frac{\Gamma_{in}}{\Gamma_l}\right)$$

This is what the convenience function `reflection_coefficient_2_propagation_constant` is doing:

[32]:
```
# reflection coefficient at input
Gamma_in = zl_2_Gamma0(z_0, z_in)
# line propagation constant
gamma = reflection_coefficient_2_propagation_constant(Gamma_in, Gamma_load, d)
print('Line propagation constant, gamma =', gamma, 'rad/m')
```

```
Line propagation constant, gamma = [0.03920416+1.37070847j] rad/m
```

One can check the consistency of the result by making the reverse calculation: the input impedance at a distance d from the load Z_l:

[33]:
```
zl_2_zin(z_0, zl=0, theta=gamma * d)
```

[33]:
```
array([20.-140.j])
```

Which was indeed the value given as input of the example.

Now that the line propagation constant has been determined, one can replace the short by a load resistance:

[34]:
```
zl_2_zin(z_0, zl=50+50j, theta=gamma * d)
```

[34]:
```
array([37.90976384-20.46175538j])
```

2.1.16 How does the SWR vary along a line?

Let's assume a 10 MHz source powering an antenna (of load Z_L) through a transmission line of length L. Depending on the location of the SWR-meter, what does one would read?

This notebook is inspired from the reference: "Facts About SWR, Reflected Power, and Power Transfer on Real Transmission Lines with Loss" by Steve Stearns given at ARRL Pacificon Antenna Seminar in 2010.

Let solve this question using `scikit-rf`. But first, the traditional Python imports:

[1]:
```
%matplotlib inline
```

[2]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf
from skrf.constants import c
from skrf.tlineFunctions import voltage_current_propagation, zl_2_swr, zl_2_zin
```

[3]:
```
rf.stylely()
```

Lossless lines

Let's start with a lossless line of propagation constant $\gamma = j\beta$ and characteristic impedance $z_0 = 50\Omega$ (real).

[4]:
```
freq = rf.Frequency(10, unit='MHz', npoints=1)
```

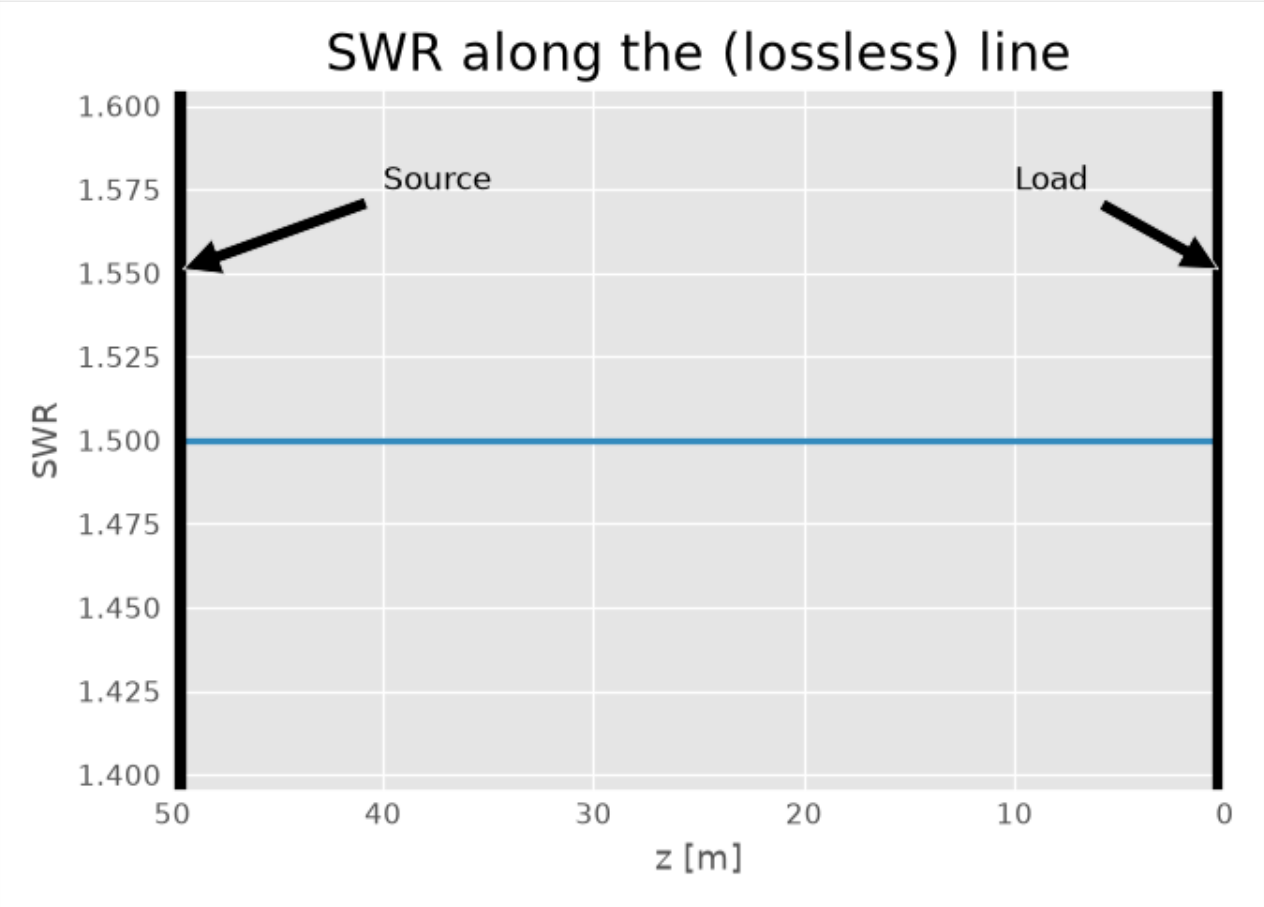
[5]:
```
# load and line properties
Z_L = 75  # Ohm
Z_0 = 50  # Ohm
L = 50  # m

# propagation constant
beta = freq.w/c
gamma = 1j*beta
```

Below we calculate the SWR of the line as a function of z the line length measured from the load ($z = 0$ at the load, $z = L$ at the source).

[6]:
```
z = np.linspace(start=L, stop=0, num=301)
SWRs = zl_2_swr(z0=Z_0, zl=zl_2_zin(Z_0, Z_L, gamma*z))
```

[7]:
```
fig, ax = plt.subplots()
ax.plot(z, SWRs, lw=2)
ax.set_xlabel('z [m]')
ax.set_ylabel('SWR')
ax.set_title('SWR along the (lossless) line')
ax.invert_xaxis()
ax.axvline(0, lw=8, color='k')
ax.axvline(L, lw=8, color='k')
ax.annotate('Load', xy=(0, 1.55), xytext=(10, 1.575),
            arrowprops=dict(facecolor='black', shrink=0.05),
            )
ax.annotate('Source', xy=(50, 1.55), xytext=(40, 1.575),
            arrowprops=dict(facecolor='black', shrink=0.05),
            )
```

[7]:
```
Text(40, 1.575, 'Source')
```



As expected, the SWR is the same everywhere along the line as the forward and backward wave amplitudes are also the same along the line.

Lossy Lines

Let's take the previous example but this time on a lossy line. The line is defined with a propagation constant $\gamma = \alpha + j\beta$:

[8]:
```
alpha = 0.01  # Np/m. Here a dummy value, just for the sake of the example
gamma = alpha + 1j*beta
```

[9]:
```
z = np.linspace(0, L, num=101)
SWRs = zl_2_swr(z0=Z_0, zl=zl_2_zin(Z_0, Z_L, gamma*z))
```

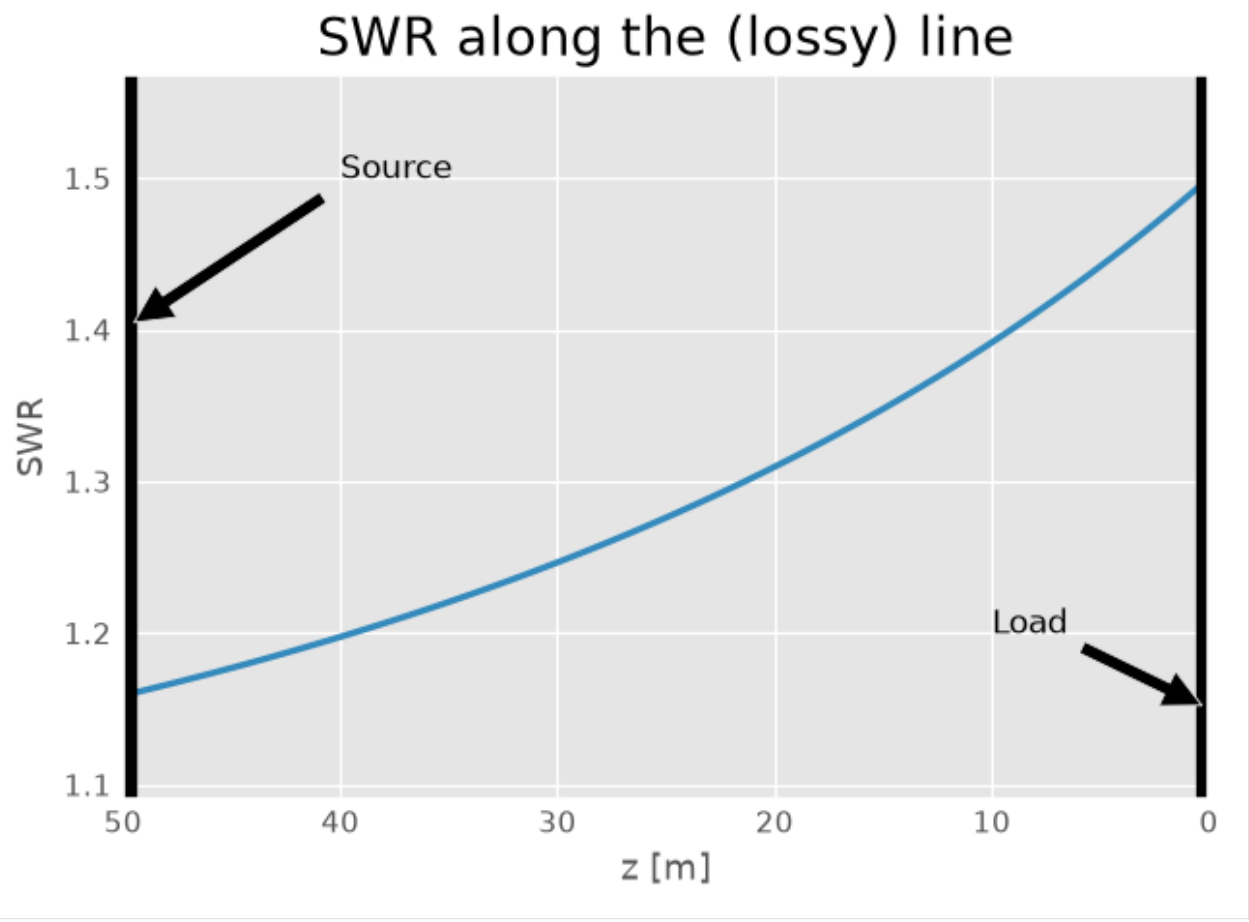
[10]:
```
fig, ax = plt.subplots()
ax.plot(z, SWRs, lw=2)
ax.set_xlabel('z [m]')
ax.set_ylabel('SWR')
ax.set_title('SWR along the (lossy) line')
ax.invert_xaxis()
ax.axvline(0, lw=8, color='k')
ax.axvline(L, lw=8, color='k')
ax.annotate('Load', xy=(0, 1.15), xytext=(10, 1.2),
            arrowprops=dict(facecolor='black', shrink=0.05),
```

(continues on next page)

(continued from previous page)

```
            )
ax.annotate('Source', xy=(50, 1.4), xytext=(40, 1.5),
            arrowprops=dict(facecolor='black', shrink=0.05),
            )
```

[10]:
```
Text(40, 1.5, 'Source')
```



For a lossy line, the SWR is maximum at the load and decreases to be minimum at the source side.

Let's see how the impedance varies along the line:

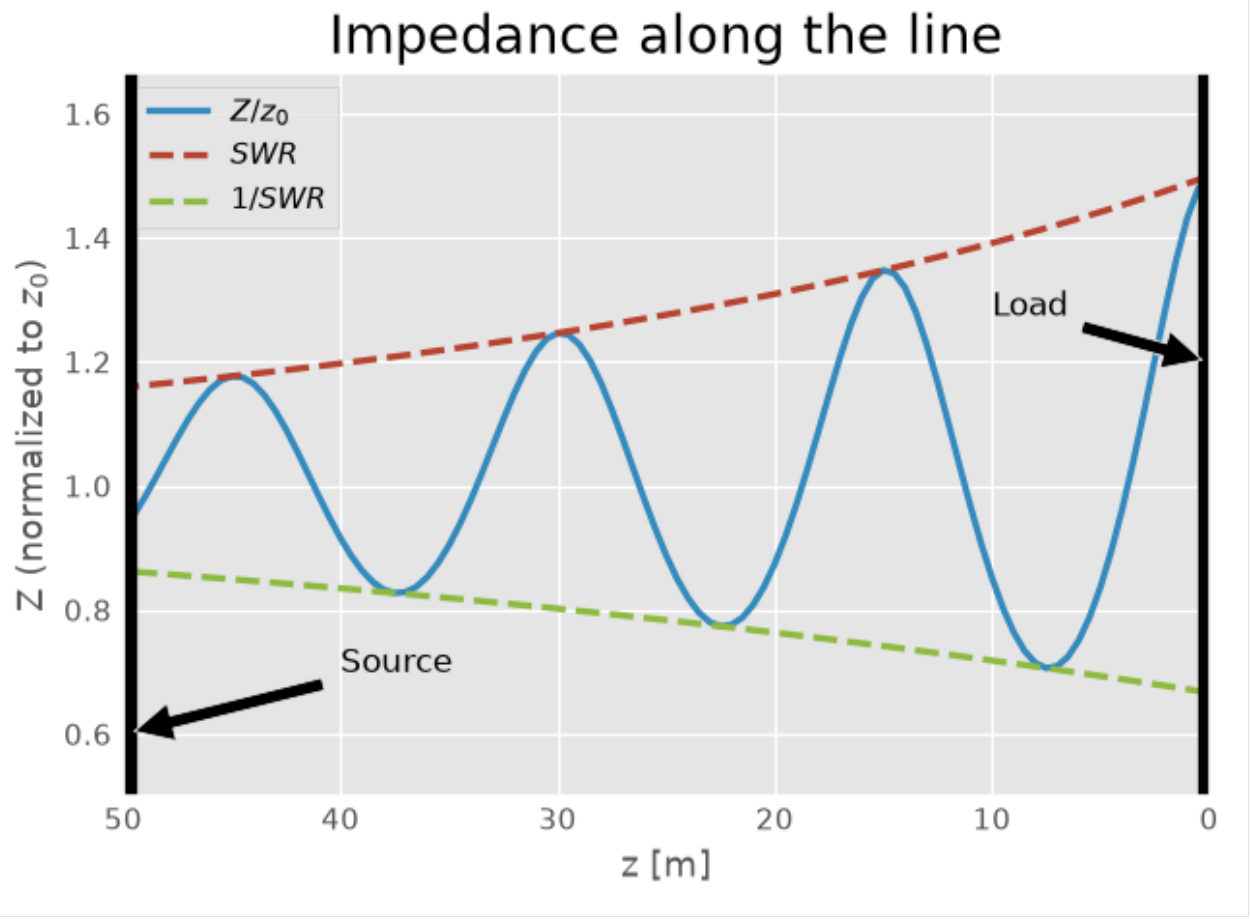
[11]:
```
Zins = zl_2_zin(Z_0, Z_L, gamma*z)

fig, ax = plt.subplots()
ax.plot(z, np.abs(Zins/Z_0), lw=2, label='$Z/z_0$')
ax.plot(z, SWRs, lw=2, ls='--', label=r'$SWR$')
ax.plot(z, 1/SWRs, lw=2, ls='--', label=r'$1/SWR$')

ax.set_xlabel('z [m]')
ax.set_ylabel('Z (normalized to $z_0$)')
ax.set_title('Impedance along the line')
ax.invert_xaxis()
ax.axvline(0, lw=8, color='k')
ax.axvline(L, lw=8, color='k')
```

(continues on next page)

(continued from previous page)

```
ax.annotate('Load', xy=(0, 1.2), xytext=(10, 1.275),
            arrowprops=dict(facecolor='black', shrink=0.05),
            )
ax.annotate('Source', xy=(50, 0.6), xytext=(40, 0.7),
            arrowprops=dict(facecolor='black', shrink=0.05),
            )
ax.legend()
```

[11]:
```
<matplotlib.legend.Legend at 0x723767168830>
```



The previous result is due to the fact that voltages and currents vary along the transmission line:

[12]:
```
V_s = 1
Z_in = zl_2_zin(Z_0, Z_L, gamma*z)
# Z_s = Z_0
V_in = V_s * Z_in/(Z_0 + Z_in)
I_in = V_in/(Z_0 + Z_in)
# note that here we are going from source to load
V, I = voltage_current_propagation(V_in, I_in, Z_0, gamma*z)
```

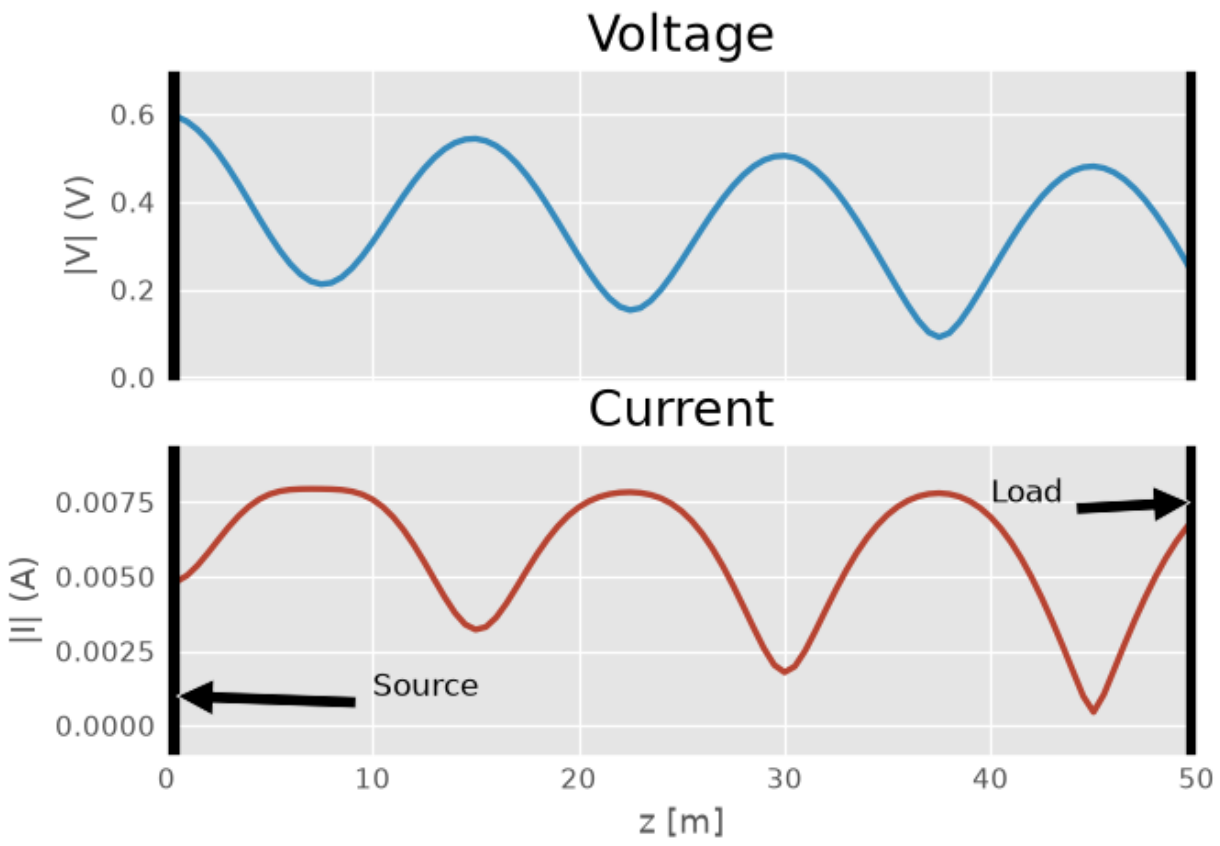
[13]:
```
fig, ax = plt.subplots(2,1,sharex=True)
ax[0].plot(z, np.abs(V), lw=2)
ax[1].plot(z, np.abs(I), lw=2, color='C1')
```

(continues on next page)

(continued from previous page)

```
ax[1].set_xlabel('z [m]')
ax[0].set_ylabel('|V| (V)')
ax[1].set_ylabel('|I| (A)')
ax[0].set_title('Voltage')
ax[1].set_title('Current')
[a.axvline(0, lw=8, color='k') for a in ax]
[a.axvline(L, lw=8, color='k') for a in ax]
ax[1].annotate('Load', xy=(50, 0.0075), xytext=(40, 0.0075),
            arrowprops=dict(facecolor='black', shrink=0.05))
ax[1].annotate('Source', xy=(0, 0.001), xytext=(10, 0.001),
            arrowprops=dict(facecolor='black', shrink=0.05))
```

[13]:
```
Text(10, 0.001, 'Source')
```



2.1.17 Working with Complex Characteristic Impedance

This section considers the propagation of electromagnetic waves on a transmission line with a possibly complex characteristic impedance.

TL;DR: `scikit-rf` use *power-waves* definition by default. Unless having complex-valued characteristic impedances, this s-parameter definition should not affect your results. However, pay attention that the power-waves definition may lead to incorrect results/interpretations when using complex characteristic impedances, for example for lossy transmission lines.

Characteristic Impedance

The *characteristic impedance* Z_0 associated to a transmission line (or any continuous media supporting the propagation of electromagnetic waves) is defined as the ratio of the (forward) voltage and current when the transmission line is infinite (i.e. SWR=1, meaning no reflection from a load and thus no backward voltage and current). It thus characterizes the property of the line to oppose a change in voltage for a change of current or vice-versa. Note that in most cases, it is *not* an impedance which can be measured using a DC Ohmmeter [1].

Voltage and current are derived quantities from the fundamental electric E and magnetic fields H (by mean of line integrals of those fields in the transverse section of a transmission line for given set of boundary conditions). Thus, voltage and current depends of the transmission line geometry and materials and in general their ratio may not be equal to the ratio of E/H. For transmission line supporting other modes than TEM, the definition of voltage and current may even be not unique (like for rectangular waveguides). For completeness, sometime the ratio of the electric field to the magnetic field is called the *intrinsic impedance* (like in free space).

Thus, Z_0 depends of the characteristics of the material and can be determined from the transmission line distributed circuit parameters (R, L, C, G). In lossless transmission line, Z_0 is real and positive. Most of the time, using a real reference impedance is also OK even for low-loss lines such as coaxial lines[7]. However for other cases, such as lossy lines, microstrip and coplanar waveguides, Z_0 can be complex. Matching problems directed at maximum power transfer can also use scattering parameters referenced to complex impedances [5].

Scattering Parameters Definitions

When it is necessary to work with a complex reference impedance there are 3 types of waves: *traveling-waves, pseudo-waves, and power-waves* [2-3]. All of these definitions give the same results for real-values characteristic impedances.

Traveling-waves

Traveling-waves are the "real" waves in the sense that they are linked directly to Maxwell's equations and can be measured. For example, "traveling-wave reflection coefficients can be measured by observing the peaks and valleys of the electric fields of the standing wave created by the beating of incident and reflected traveling waves in a slotted-line experiment" [2-3]. The through-reflect-line (TRL) vector-network-analyzer calibration also measures traveling waves [2].

With traveling-waves, the knowledge of the characteristic impedance Z_0 is mandatory to determine the equivalent circuit voltages and currents in microwave circuit theory. However, it is not always possible in practice to determine Z_0, in particular with printed transmission lines. This is the reason why pseudo-waves have been developed.

Pseudo-waves

Pseudo-waves have been introduced in [2-3] as a solution to the complexity of traveling-waves for lossy transmission lines. While being a mathematical tool, they obey the same rules and formulas that apply to the lossless transmission lines.

They are defined as:

$$a = \frac{\sqrt{\Re(Z_{ref})}}{2|Z_{ref}|}(V + Z_{ref}I)$$

$$b = \frac{\sqrt{\Re(Z_{ref})}}{2|Z_{ref}|}(V - Z_{ref}I)$$

where Z_{ref} is the reference impedance of the pseudo-wave.

Pseudo-waves are equals to traveling-waves when $Z_{ref} = Z_0$, where Z_0 is the characteristic impedance of the transmission line. However, if Z_{ref} is complex-valued, the power p transmitted by a pseudo-wave through the transmission

line is:

$$p = |a|^2 - |b|^2 + 2\Im(ab^*)\frac{\Im(Z_{ref})}{\Re(Z_{ref})}$$

which is the same result than for traveling waves [2]. Moreover, if Z_{ref} is complex-valued, the scattering matrix of a reciprocal device might not be symmetrical.

However, when Z_{ref} is chosen to be real, a and b behave just like traveling-wave amplitude in a lossless transmission line with a real characteristic impedance[2-3,5].

Power-waves

The scattering matrix as defined above has the fundamental problem that a zero-reflection coefficient does not guarantee maximal power transfer from the generator to the load [5-6]. To remedy this situation, the concept of *power-waves* has been introduced [4]. It is a mathematical tool which is compatible with the conjugate matching condition. It also gives intuitive total power expressions, as the sum of forward and backward waves, also for complex characteristic impedances.

The power-waves a and b are defined from frequency-dependent (complex) port impedance Z_0 [4]:

$$a = \frac{1}{2\sqrt{\Re(Z_0)}}(V + Z_0 I)$$

$$b = \frac{1}{2\sqrt{\Re(Z_0)}}(V - Z_0^* I)$$

where Z_0^* is the complex conjugate of Z_0.

At the contrary of pseudo-waves, power-waves satisfy the power relation:

$$p = |a|^2 - |b|^2$$

even when Z is complex. That is, power-waves have been developed such as zero power-wave reflection coefficient corresponds to maximum power transfer.

Most RF circuit solvers use the power-waves definition (such as ADS, ANSYS Circuit). `scikit-rf` also uses the power-waves definition by default.

Caveats

Reflection Coefficient and Smith Chart

The Smith chart is based on the reflection coefficient Γ, a ratio of the impedance of the load Z_{load} and the reference impedance Z_{ref} that defines the center of the chart:

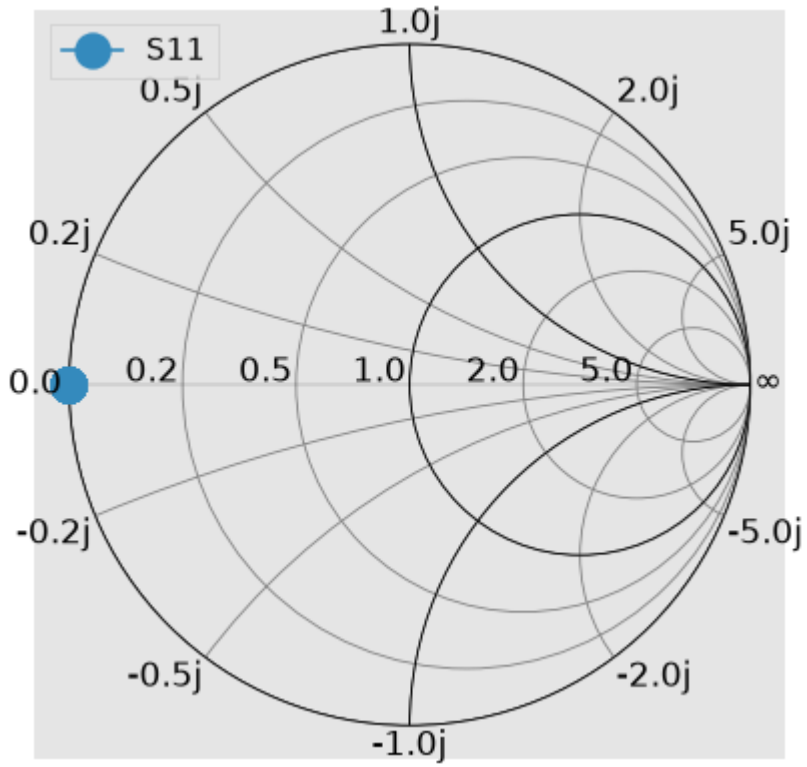
$$\Gamma(Z_{load}) = \frac{Z_{load} - Z_{ref}}{Z_{load} + Z_{ref}}$$

Let's consider the reflection coefficients of a short for both pseudo-waves and power-waves formulations with respect to a complex characteristic impedance:

[1]:
```
%matplotlib inline
import skrf as rf

rf.stylely()
# this is just for plotting junk
kw = dict(draw_labels=True, marker = 'o', markersize = 10)
```

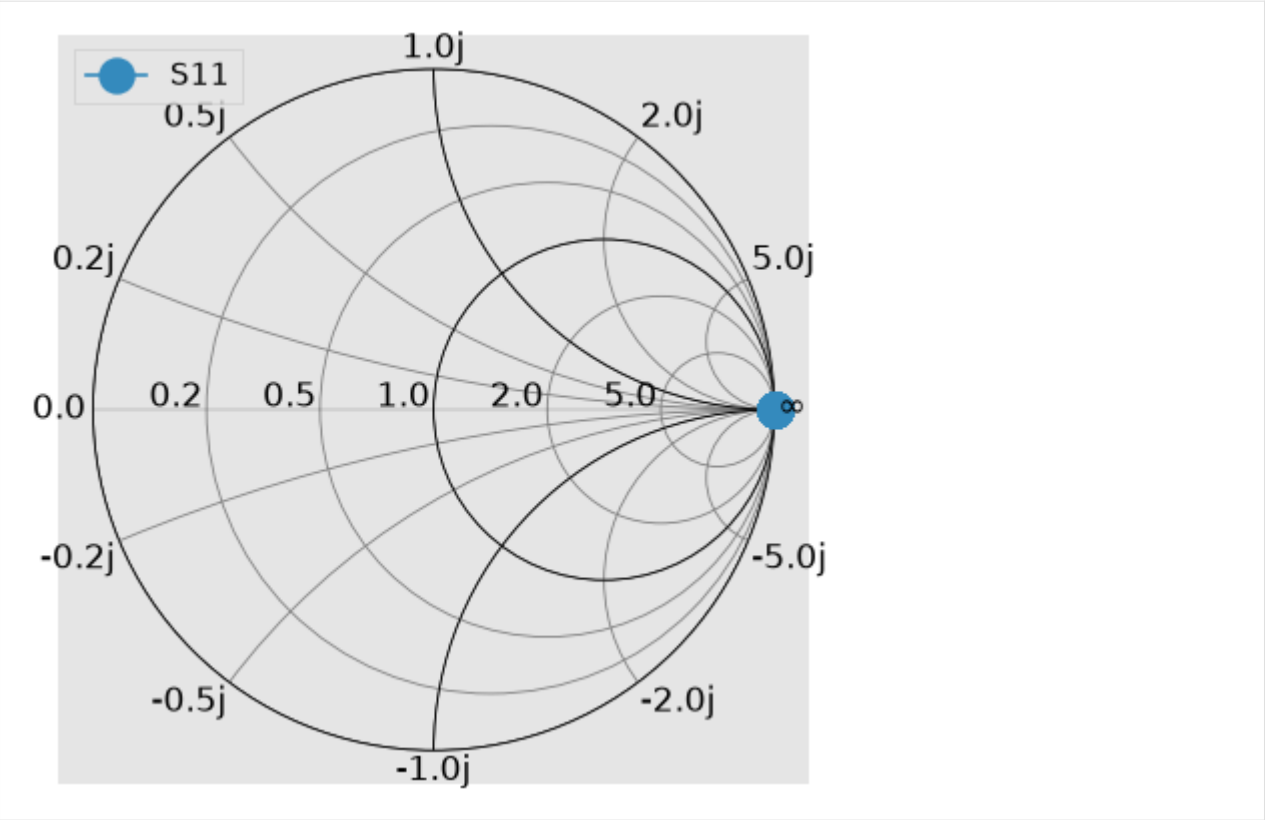
For the case of pseudo-waves, the ratio b/a gives the expected expression for Γ. Thus, whatever the reference impedance, the Smith chart for short is as one could expect:

[2]:
```
# same but paying attention to the s-param definition:
ntw_pseudo = rf.instances.wr10.short()
ntw_pseudo.renormalize(1j, s_def='pseudo')
ntw_pseudo.plot_s_smith(**kw)
```



However, for power-waves:

[3]:
```
# default: power wave formulation
ntw_power = rf.instances.wr10.short()
ntw_power.renormalize(1j)  # same as renormalize(1j, s_def='power')
ntw_power.plot_s_smith(**kw)  # gives an open!
```



This unexpected behavior is the result of the appearance of Z_0^* in the definition of the amplitude of the reverse *power-wave* b, hence changing the reflection coefficient formula:

$$\Gamma_{\text{powerwaves}}(Z_{load}) = \frac{b}{a} = \frac{Z_{load} - Z_0^*}{Z_{load} + Z_0}$$

Thus, in the case of complex characteristic impedance, the Smith chart does not apply with power-waves.

References

- [1] Microwaves 101: "Characteristic Impedance"

Reference Papers:

- [2] R. B. Marks et D. F. Williams, A general waveguide circuit theory, J. RES. NATL. INST. STAN., vol. 97, n 5, p. 533, sept. 1992, doi: 10/gf3wcs.
- [3] D. Williams, Traveling Waves and Power Waves: Building a Solid Foundation for Microwave Circuit Theory, IEEE Microwave, vol. 14, n. 7, p. 38-45, nov. 2013, doi: 10.1109/MMM.2013.2279494
- [4] K. Kurokawa, Power Waves and the Scattering Matrix, IEEE Trans. Microwave Theory Techn., vol. 13, n 2, p. 194-202, mars 1965, doi: 10/bdxqv3.
- [5] S. Llorente-Romano, A. Garca-Lamperez, T. K. Sarkar, et M. Salazar-Palma, An Exposition on the Choice of the Proper S Parameters in Characterizing Devices Including Transmission Lines with Complex Reference Impedances and a General Methodology for Computing Them, IEEE Antennas Propag. Mag., vol. 55, n 4, p. 94-112, août 2013, doi: 10/ggc28m.
- [6] J. Rahola, Power Waves and Conjugate Matching, IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 55, n 1, p. 92-96, janv. 2008, doi: 10/fgnf7j.

Some Books dealing about this subject:

- [7] T. C. Edwards et M. B. Steer, Foundations for microstrip circuit design, Fourth edition. Chichester, West Sussex, United Kingdom: John Wiley & Sons Inc, 2016. Section 4.5 on working with a complex characteristic impedance.
- [8] G. Gonzalez, Microwave transistor amplifiers: analysis and design, 2nd ed. Upper Saddle River, N.J: Prentice Hall, 1997. Section 1.4 for generalized scattering parameters and 1.7 for power waves.
- [9] P. Russer, Electromagnetics, microwave circuit and antenna design for communications engineering, 2nd ed. Boston, MA: Artech House, 2006. Section 10.2.3 for generalized scattering parameters and transformation tables (used in wikipedia pages).
- [10] J. Dobrowolski, Microwave network design using the scattering matrix. Boston: Artech House, 2010.
- [11] S. J. Orfanidis, Electromagnetic Waves and Antennas, Chap13: Impedance Matching, 2016.

2.2 Matching

2.2.1 Impedance Matching

Introduction

The general problem is illustrated by the figure below: a generator with an internal impedance Z_S delivers a power to a passive load Z_L, through a 2-ports matching network. This problem is commonly named as "the double matching problem". Impedance matching is important for the following reasons:

- maximizing the power transfer. Maximum power is delivered to the load when the generator *and* the load are matched to the line and power loss in the line minimized
- improving signal-to-noise ratio of the system
- reducing amplitude and phase errors
- reducing reflected power toward generator

As long as the load impedance Z_L has a real positive part, a matching network can always be found. Many choices are available and the examples below only describe a few. The examples are taken from the D.Pozar book "Microwave Engineering", 4th edition.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf

rf.stylely()
```

Matching with Lumped Elements

To begin, let's assume that the matching network is lossless and the feeding line characteristic impedance is Z_0:

The simplest type of matching network is the "L" network, which uses two reactive elements to match an arbitrary load impedance. Two possible configuration exist and are illustrated by the figures below. In either configurations, the reactive elements can be inductive of capacitive, depending on the load impedance.

Let's assume the load is $Z_L = 200 - 100j\Omega$ for a line $Z_0 = 100\Omega$ at the frequency of 500 MHz.

[2]:
```
Z_L = 200 - 100j
Z_0 = 100
f_0_str = '500MHz'
```

Let's define the `Frequency` and load `Network`:

[3]:
```
# frequency band centered on the frequency of interest
frequency = rf.Frequency(start=300, stop=700, npoints=401, unit='MHz')
# transmission line Media
line = rf.media.DefinedGammaZ0(frequency=frequency, z0=Z_0)
# load Network
load = line.load(rf.tlineFunctions.zl_2_Gamma0(Z_0, Z_L))
```

We are searching for a L-C Network corresponding to the first configuration above:

[4]:
```
def matching_network_LC_1(L, C):
    ' L and C in nH and pF'
    return line.inductor(L*1e-9)**line.shunt_capacitor(C*1e-12)**load

def matching_network_LC_2(L, C):
    ' L and C in nH and pF'
    return line.capacitor(C*1e-12)**line.shunt_inductor(L*1e-9)**load
```

Finding the set of inductance L and the capacitance C which matches the load is an optimization problem. The `scipy` package provides the necessary optimization function(s) for that:

[5]:
```
from scipy.optimize import minimize

# initial guess values
L0 = 10 # nH
C0 = 1 # pF
x0 = (L0, C0)
# bounds
L_minmax = (1, 100) #nH
C_minmax = (0.1, 10) # pF

# the objective functions minimize the return loss at the target frequency f_0
def optim_fun_1(x, f0=f_0_str):
    _ntw = matching_network_LC_1(*x)
    return np.abs(_ntw[f_0_str].s).ravel()

def optim_fun_2(x, f0=f_0_str):
    _ntw = matching_network_LC_2(*x)
    return np.abs(_ntw[f_0_str].s).ravel()
```

[6]:
```
res1 = minimize(optim_fun_1, x0, bounds=(L_minmax, C_minmax))
print(f'Optimum found for LC network 1: L={res1.x[0]} nH and C={res1.x[1]} pF')
```

```
Optimum found for LC network 1: L=38.98484015555227 nH and C=0.9227738237298321 pF
```

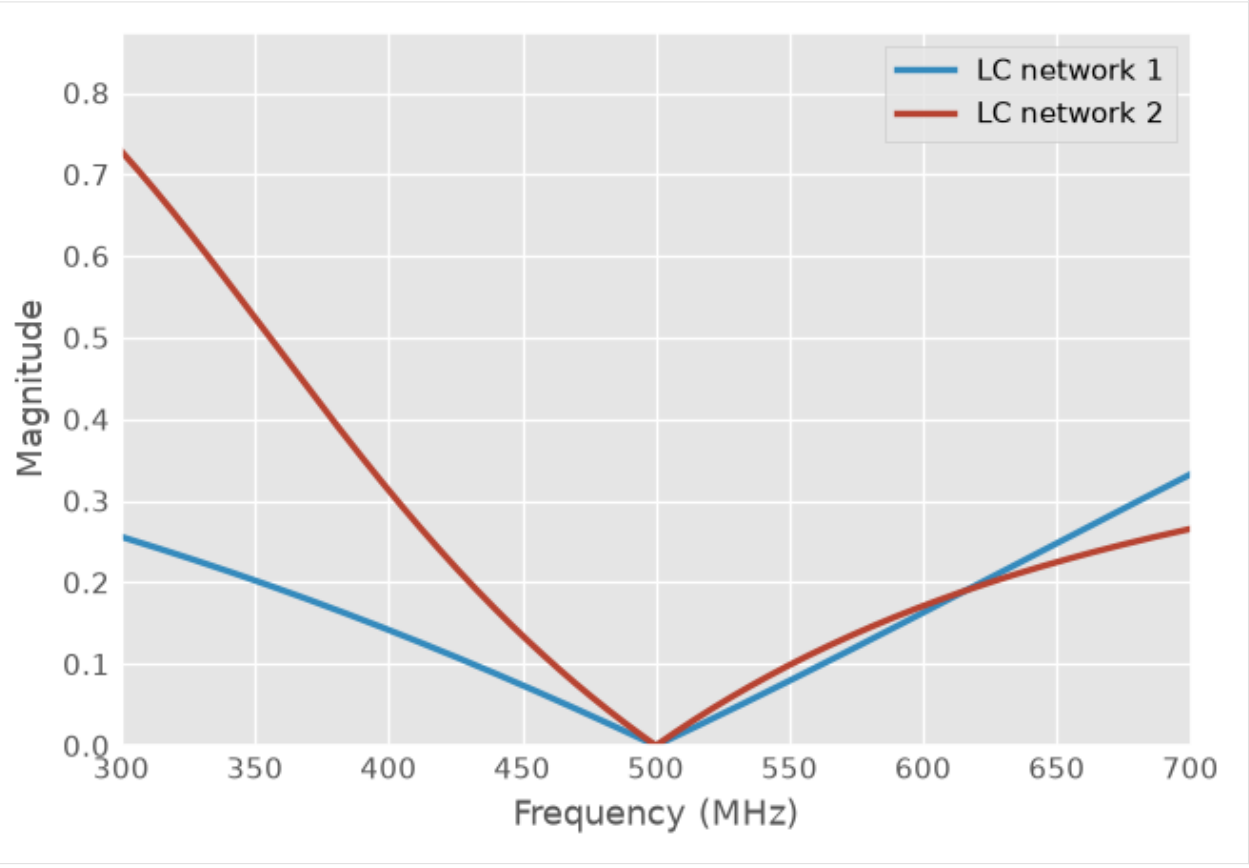
[7]:
```
res2 = minimize(optim_fun_2, x0, bounds=(L_minmax, C_minmax))
print(f'Optimum found for LC network 2: L={res2.x[0]} nH and C={res2.x[1]} pF')
```

```
Optimum found for LC network 2: L=46.13869161615916 nH and C=2.598989336748551 pF
```

[8]:
```
ntw1 = matching_network_LC_1(*res1.x)
ntw2 = matching_network_LC_2(*res2.x)
```

[9]:
```
ntw1.plot_s_mag(lw=2, label='LC network 1')
ntw2.plot_s_mag(lw=2, label='LC network 2')
plt.ylim(bottom=0)
```

[9]:
```
(0.0, 0.8742937659028248)
```



Single-Stub Matching

Matching can be made with a piece of open-ended or shorted transmission line (*stub*), connected either in parallel (*shunt*) or in series. In the example below, a matching network is realized from a shorted transmission line of length (θ_{stub}) connected in parallel, in association with a series transmission line (θ_{line}). Let's assume a load impedance $Z_L = 60 - 80j$ connected to a 50 Ohm transmission line.

Let's match this load at 2 GHz:

[10]:
```
Z_L = 60 - 80j
Z_0 = 50
f_0_str = '2GHz'
# Frequency, wavenumber and transmission line media
freq = rf.Frequency(start=1, stop=3, npoints=301, unit='GHz')
```

(continues on next page)

(continued from previous page)

```
beta = freq.w/rf.constants.c
line = rf.media.DefinedGammaZ0(freq, gamma=1j*beta, z0=Z_0)
```

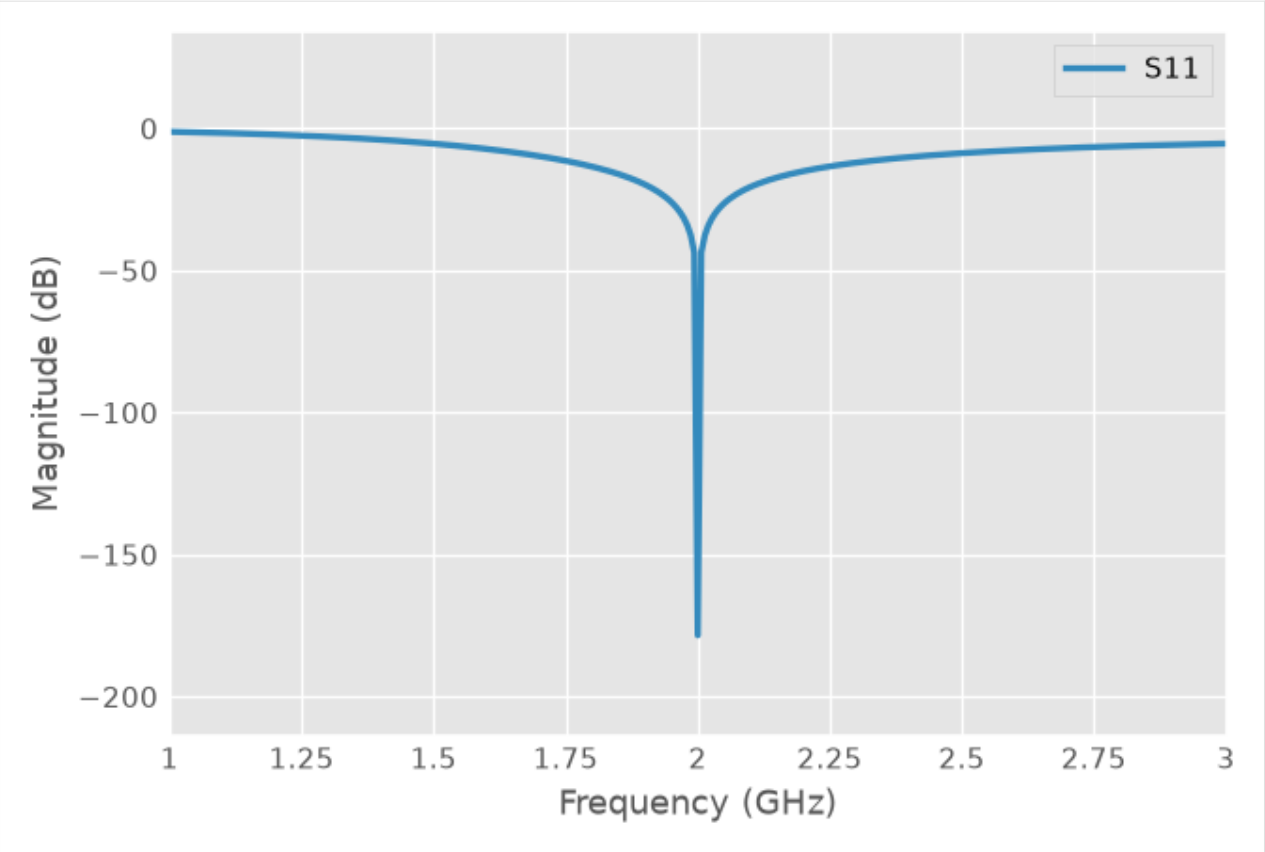
[11]:
```
def resulting_network(theta_delay, theta_stub):
    """
    Return a loaded single stub matching network

    NB: theta_delay and theta_stub lengths are in deg
    """
    delay_load = line.delay_load(rf.tlineFunctions.zl_2_Gamma0(Z_0, Z_L), theta_delay)
    shunted_stub = line.shunt_delay_short(theta_stub)
    return shunted_stub ** delay_load
```

Optimize the matching network variables `theta_delay` and `theta_stub` to match the resulting 1-port network ($|S| = 0$)

[12]:
```
from scipy.optimize import minimize


def optim_fun(x):
    return resulting_network(*x)[f_0_str].s_mag.ravel()

x0 = (50, 50)
bnd = (0, 180)
res = minimize(optim_fun, x0, bounds=(bnd, bnd))
print(f'Optimum found for: theta_delay={res.x[0]:.1f} deg and theta_stub={res.x[1]:.1f}␣
↪deg')
```

```
Optimum found for: theta_delay=39.8 deg and theta_stub=34.2 deg
```

[13]:
```
# Optimized network at f0
ntw = resulting_network(*res.x)
ntw.plot_s_db(lw=2)
```



[ ]:

2.3 Metrology and Calibration

2.3.1 Calibration With Three Receivers

Intro

It has long been known that full error correction is possible given a VNA with only three receivers and no internal coaxial switch. However, since no modern VNA employs such an architecture, the software required to make fully corrected measurements is not available on today's modern VNA's.

Recently, the application of Frequency Extender units containing only three receivers has become more common. Furthermore, low-cost VNAs are becoming useful tools for learning RF electronics. The representative example is NanoVNA, which uses a three-receiver design. Thus, there is a need for full error correction capability on systems with three receivers and no internal coaxial switch. This document describes how to use scikit-rf to fully correct two-port measurements made on such a system.

Theory

A circuit model for a switch-less three receiver system is shown below.

[1]:
```
from IPython.display import SVG, Image

SVG('three_receiver_cal/pics/vnaBlockDiagramForwardRotated.svg')
```

[1]:

To fully correct an arbitrary two-port, the device must be measured in two orientations, call these forward and reverse. Because there is no switch present, this requires the operator to physically flip the device, and save the pair of measurements. In on-wafer scenarios, one could fabricate two identical devices, one in each orientation. In either case, a pair of measurements are required for each DUT before correction can occur.

While in reality the device is being flipped, one can imaging that the device is static, and the entire VNA circuitry is flipped. This interpretation lends itself to implementation, as the existing 12-term correction can be reused by simply copying the forward error coefficients into the corresponding reverse error coefficients. This is what `scikit-rf` does internally.

Worked Example

This example demonstrates how to create a *TwoPortOnePath* and *EnhancedResponse* calibration from measurements taken on a Agilent PNAX with a set of VDI WR-12 TXRX-RX Frequency Extender heads. Comparisons between the two algorithms are made by correcting an asymmetric DUT.

Measure the Calibration Standands and DUT

First, one measures the S-parameters of the Open, Short, and Load standards on the VNA according to the standard SOLT calibration procedures. We assume the reader is familiar with this task in scikit-rf. If not, it's fully described in the *SOLT example*.

When calibrating a three-receiver VNA, the process is almost identical. But note that there are a few minor differences. First, the Open, Short, and Load standards are only measured at port 1, not port 2. The VNA cannot measure in the reverse orientation, so there's no reason to perform OSL calibration at port 2 (unless one needs the optional isolation calibration).

Nevetheless, scikit-rf still expects two-port networks as input, we therefore still save all results as two-port networks. In this case, only S_{11} and S_{21} are the actual measurements, S_{12} and S_{22} are unused, not meaningful, and can contain arbitrary data. One can use all-zero values as placeholders. Internally, scikit-rf will set $S_{22} = S_{11}$ and $S_{12} = S_{21}$ for the purpose of calculations.

Then, the DUT is measured as a two-port network in the forward direction, physically flipped, and then measured again in the reverse orientation.

Read in the Data

The measurements of the calibration standards and DUT's were downloaded from the VNA by saving touchstone files of the raw s-parameter data to disk.

[2]:
```
! ls three_receiver_cal/data/
```

```
'attenuator (forward).s2p'       simulation.s2p
'attenuator (reverse).s2p'       thru.s2p
 load.s2p                       'wr15 shim and swg (forward).s2p'
'quarter wave delay short.s2p'  'wr15 shim and swg (reverse).s2p'
 short.s2p
```

These files can be read by scikit-rf into `Networks` with the following.

[3]:
```
import matplotlib.pyplot as plt

import skrf as rf
```

(continues on next page)

(continued from previous page)

```
%matplotlib inline

rf.stylely()


raw = rf.io.general.read_all_networks('three_receiver_cal/data/')
# list the raw measurements
raw.keys()
```

[3]:
```
dict_keys(['attenuator (forward)', 'attenuator (reverse)', 'load', 'quarter wave delay␣
↪short', 'short', 'simulation', 'thru', 'wr15 shim and swg (forward)', 'wr15 shim and␣
↪swg (reverse)'])
```

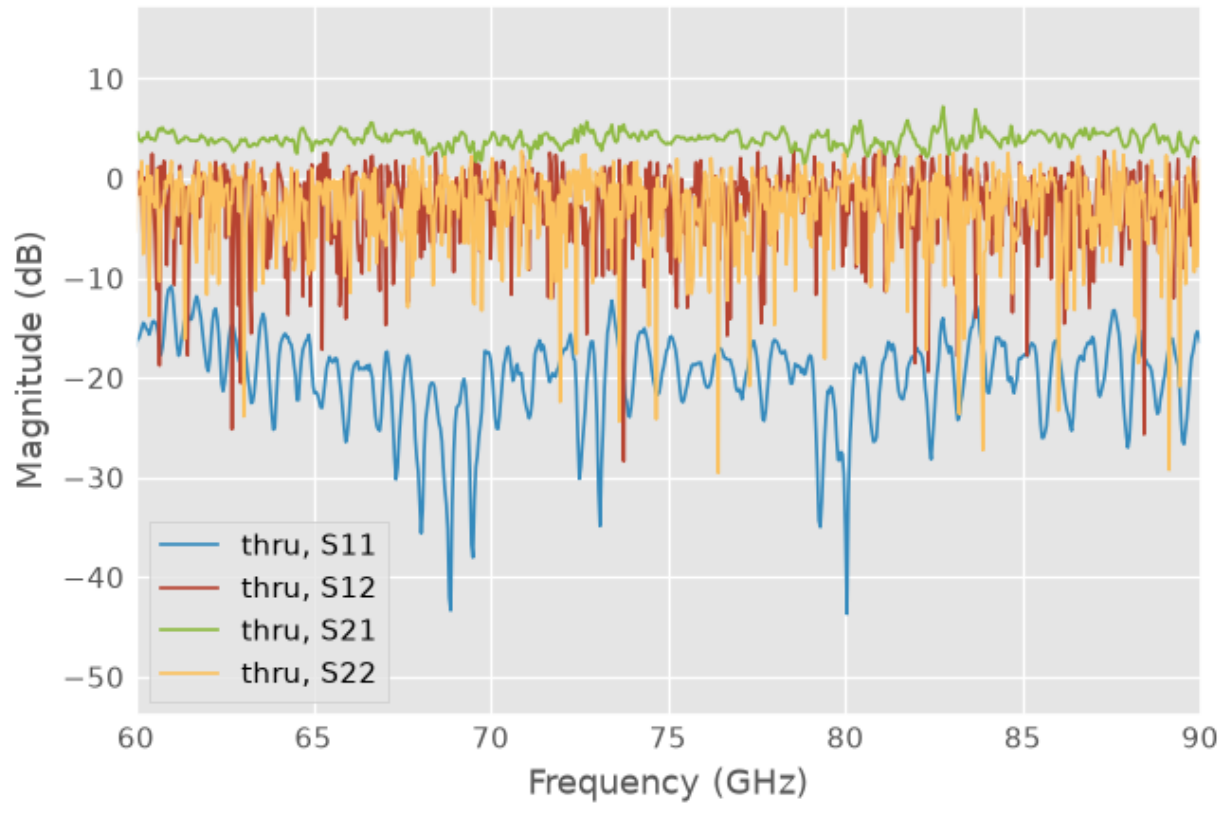
Each `Network` can be accessed through the dictionary `raw`.

[4]:
```
thru = raw['thru']
thru
```

[4]:
```
2-Port Network: 'thru',  60.0-90.0 GHz, 721 pts, z0=[50.+0.j 50.+0.j]
```

If we look at the *raw* measurement of the flush thru, it can be seen that only S_{11} and S_{21} contain meaningful data. The other s-parameters are noise.

[5]:
```
thru.plot_s_db()
```



Create Calibration

In the code that follows a TwoPortOnePath calibration is created from corresponding measured and ideal responses of the calibration standards. The measured networks are read from disk, while their corresponding ideal responses are generated using scikit-rf. More information about using scikit-rf to do offline calibrations can be found *here*.

[6]:
```
from skrf.calibration import TwoPortOnePath
from skrf.constants import mil
from skrf.media import RectangularWaveguide
from skrf.network import two_port_reflect as tpr

# pull frequency information from measurements
frequency = raw['short'].frequency

# the media object
wg = RectangularWaveguide(frequency=frequency, a=120*mil, z0_override=50)

# list of 'ideal' responses of the calibration standards
ideals = [wg.short(nports=2),
          tpr(wg.delay_short( 90,'deg'), wg.match()),
          wg.match(nports=2),
          wg.thru()]

# corresponding measurements to the 'ideals'
measured = [raw['short'],
            raw['quarter wave delay short'],
            raw['load'],
            raw['thru']]

# the Calibration object
cal = TwoPortOnePath(measured = measured, ideals = ideals)
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:2073: UserWarning: n_thrus is None, guessing which stds are␣
↪transmissive
  TwelveTerm.__init__(self,*args, **kwargs)
```

By default, `TwoPortOnePath` assumes port 1 is the active source in all calibrations and measurements (thus only S_{11} and S_{21} are valid data). If, for some reason, the valid measurements are presented as S_{12} and S_{22}, one can set the optional parameter `source_port=2` when creating the `TwoPortOnePath` object.

Apply Correction

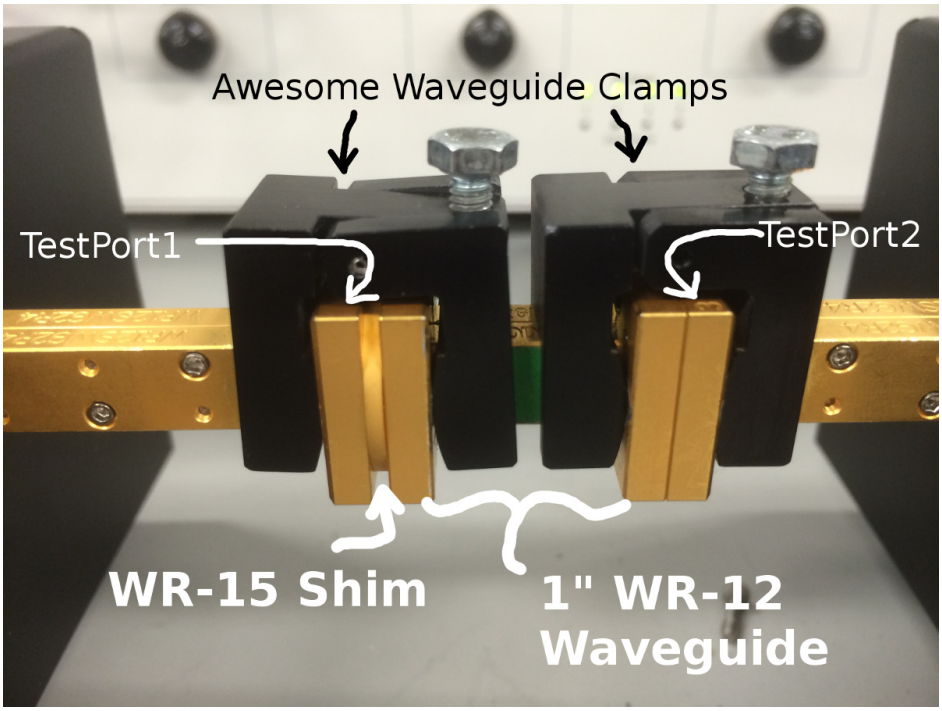
There are two types of correction possible with a 3-receiver system.

1. Full (TwoPortOnePath)
2. Partial (EnhancedResponse)

`scikit-rf` uses the same `Calibration` object for both, but employs different correction algorithms depending on the `type` of the DUT. The DUT used in this example is a WR-15 shim cascaded with a WR-12 1" straight waveguide, as shown in the picture below. Measurements of this DUT are corrected with both *full* and *partial* correction and the results are compared below.

[7]:
```
Image('three_receiver_cal/pics/asymmetic DUT.jpg', width='75%')
```

[7]:



Full Correction (TwoPortOnePath)

Full correction is achieved by measuring each device in both orientations, **forward** and **reverse**. To be clear, this means that the DUT must be physically removed, flipped, and re-inserted. The resulting pair of measurements are then passed to the `apply_cal()` function as a tuple. This returns a single corrected response.

Partial Correction (Enhanced Response)

If you pass a single measurement to the `apply_cal()` function, then the calibration will employ partial correction. This type of correction is known as `EnhancedResponse`. Depending on the measurement application, this type of correction may be *good enough*, and perhaps the only choice.

Comparison

Below are direct comparisons of the DUT shown above corrected with *full* and *partial* algorithms. It shows that the partial calibration produces a large ripple on the reflect measurements, and slightly larger ripple on the transmissive measurements.

[8]:
```
simulation = raw['simulation']

dutf = raw['wr15 shim and swg (forward)']
dutr = raw['wr15 shim and swg (reverse)']

corrected_full =     cal.apply_cal((dutf, dutr))
corrected_partial =  cal.apply_cal(dutf)
```

(continues on next page)

(continued from previous page)

```
# plot results

f, ax = plt.subplots(1,2, figsize=(8,4))

ax[0].set_title ('$S_{11}$')
ax[1].set_title ('$S_{21}$')

corrected_partial.plot_s_db(0,0, label='Partial Correction',ax=ax[0])
corrected_partial.plot_s_db(1,0, label='Partial Correction',ax=ax[1])

corrected_full.plot_s_db(0,0, label='Full Correction', ax = ax[0])
corrected_full.plot_s_db(1,0, label='Full Correction', ax = ax[1])

simulation.plot_s_db(0,0,label='Simulation', ax=ax[0], color='k')
simulation.plot_s_db(1,0,label='Simulation', ax=ax[1], color='k')

plt.tight_layout()
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.3333333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.875 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.4166666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.9583333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.5 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.0416666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.5833333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.125 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.6666666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.2083333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.75 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.2916666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.8333333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.375 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.9166666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.4583333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.0 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.5416666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.0833333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.625 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.1666666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.7083333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.25 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.7916666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.3333333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.875 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.4166666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.9583333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.5 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.0416666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.5833333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.125 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.6666666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.2083333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.75 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.2916666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.8333333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.375 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.9166666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.4583333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.0 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.5416666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.0833333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.625 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.1666666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.7083333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.25 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.7916666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.3333333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.875 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.4166666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.9583333333␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.5 GHz.␣
```

(continues on next page)

(continued from previous page)

```
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.0416666667␣
```

(continues on next page)

(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.5833333333␣
```

(continues on next page)

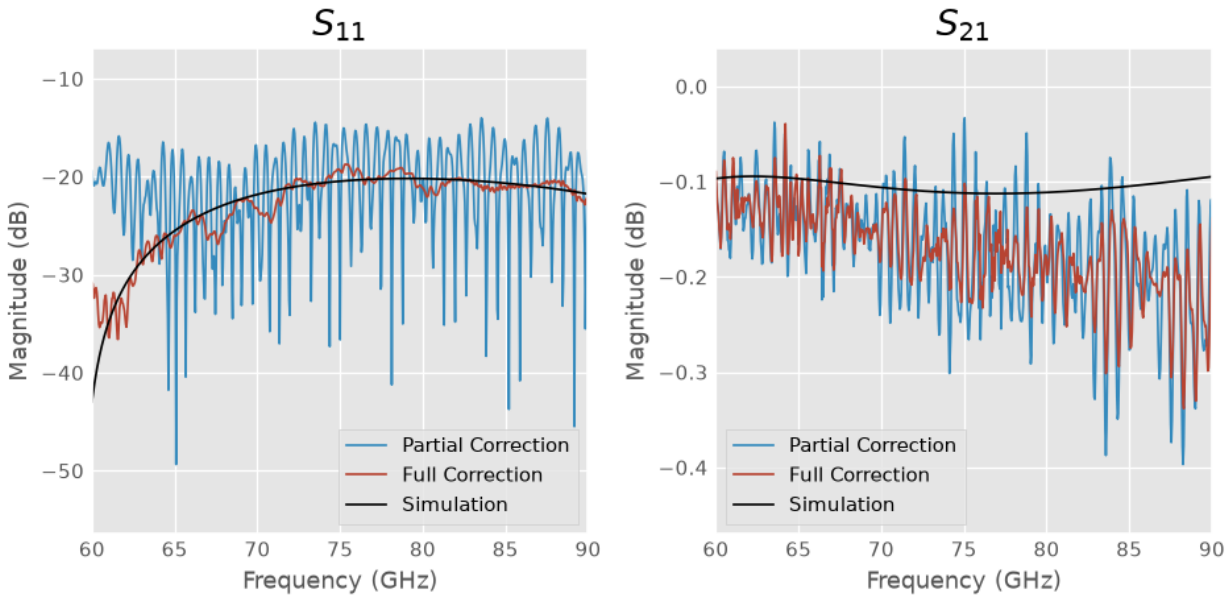
(continued from previous page)

```
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 90.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:1221: RuntimeWarning: divide by zero encountered in divide
  x = (vh.conj().swapaxes(-1, -2)/s[:, None, :])@(u.conj().swapaxes(-1, -2)@b)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:1221: RuntimeWarning: invalid value encountered in divide
  x = (vh.conj().swapaxes(-1, -2)/s[:, None, :])@(u.conj().swapaxes(-1, -2)@b)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:1221: RuntimeWarning: invalid value encountered in matmul
  x = (vh.conj().swapaxes(-1, -2)/s[:, None, :])@(u.conj().swapaxes(-1, -2)@b)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/network.
```

(continues on next page)

(continued from previous page)

```
↪py:7065: RuntimeWarning: invalid value encountered in divide
  tmp_a = Ael * (Alk / det) + Aek * (All / det)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/network.
↪py:7066: RuntimeWarning: invalid value encountered in divide
  tmp_b = Ael * (Akk / det) + Aek * (Akl / det)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:6753: RuntimeWarning: invalid value encountered in divide
  gamma_f = (Elf - Esr)/(Err + Edr*(Elf - Esr))
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:6754: RuntimeWarning: invalid value encountered in divide
  gamma_r = (Elr - Esf)/(Erf + Edf*(Elr - Esf))
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:6756: RuntimeWarning: invalid value encountered in divide
  k_first  =   Etf/(Err + Edr*(Elf  - Esr) )
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:6757: RuntimeWarning: invalid value encountered in divide
  k_second =1/(Etr/(Erf + Edf *(Elr - Esf)))
/tmp/ipykernel_2028/1667802585.py:7: UserWarning: only gave a single measurement␣
↪orientation, error correction is partial without a tuple
  corrected_partial =  cal.apply_cal(dutf)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪mathFunctions.py:303: RuntimeWarning: divide by zero encountered in log10
  out = 20 * np.log10(z)
```



What if my DUT is Symmetric??

If the DUT is known to be **reciprocal** ($S_{21} = S_{12}$) and **symmetric** ($S_{11} = S_{22}$), then its response should be the identical for both forward and reverse orientations. In this case, measuring the device twice is unnecessary, and can be circumvented. This is explored in the example: *TwoPortOnePath, EnhancedResponse, and FakeFlip*

2.3.2 LRRM

This example demonstrates how to use `skrf`'s LRRM calibration. LRRM stands for Line-Reflect-Reflect-Match which are the calibration standards needed for the calibration. There are few different implementations of the LRRM that use slightly different assumptions and match model. `skrf`'s LRRM calibration uses the following assumptions:

- Line standard needs to be known exactly.
- First reflect's phase needs to be known within 90 degrees. It can be lossy and it's assumed to be identical on both ports.
- Second reflect's |S11| is assumed to be known, it's phase also needs to be known within 90 degrees and it's assumed to be identical on both ports. The two reflects need to be different and their phase difference should be 180 degrees for the best accuracy.
- Match is assumed to be a known resistance in series with unknown inductance. Match only needs to be measured on the first port.

The calibration standards and measurements need to be given in the above order to the calibration routine. If the above assumptions are followed the calibration can solve the reflects, match and calibration parameters exactly.

LRRM example with synthetic data

Setup

[1]:
```
%matplotlib inline
import matplotlib.pyplot as plt

import skrf
from skrf.calibration import LRRM, TwelveTerm, terminate
from skrf.media import Coaxial
from skrf.network import two_port_reflect

skrf.stylely()
```

Generate example data

We first generate some synthetic error boxes and calibration standards. We will have two sets of the calibration standards. The real standards used for calibration that have parasitics and the approximate standards without parasitics that we will give to the calibration algorithm.

[2]:
```
freq = skrf.Frequency(1, 100, 100, 'GHz')

# 1.0 mm coaxial media for calibration error boxes
coax = Coaxial(freq, z0_port=50, Dint=0.44e-3, Dout=1.0e-3, sigma=1e8)

# Generate random error boxes
X = coax.random(n_ports=2, name='X')
Y = coax.random(n_ports=2, name='Y')

# Random switch terms
gamma_f = coax.random(n_ports=1, name='gamma_f')
gamma_r = coax.random(n_ports=1, name='gamma_r')

# Our guess of the standards. We assume they don't have any parasitics.
oo_i = coax.open(nports=2, name='open')
```

(continues on next page)

(continued from previous page)

```
ss_i = coax.short(nports=2, name='short')
# Match is only measured on one port. Resistance can be different from 50 ohms.
m_i = coax.resistor(R=50, name='r') ** coax.short(nports=1)
# Thru must be known exactly
thru = coax.line(d=100, unit='um', name='thru')

# Actual reflects with parasitics. They must be identical in both ports.
# Short is slightly lossy.
ss = coax.line(d=200, unit='um') ** coax.load(-0.98,nports=2, name='short') ** coax.
↪line(d=200, unit='um')
oo = coax.shunt_capacitor(10e-15) ** coax.open(nports=2, name='open') ** coax.shunt_
↪capacitor(10e-15)

# Match standard has inductance in series.
match_l = 40e-12
l = coax.inductor(L=match_l)
m = l**m_i

# Make two-port of the match with open on the second port.
mm = two_port_reflect(m, coax.open(nports=1))
# Make two-port for the match standard
mm_i = coax.match(nports=2, name='load')

# These are our guesses of the calibration standards.
approx_ideals = [
    thru,
    ss_i,
    oo_i,
    mm_i
    ]

# These are the actual standards with parasitics.
ideals = [
    thru,
    ss,
    oo,
    mm
    ]

# Make measurement of the standards using the random error boxes and switch terms.
measured = [terminate(X**k**Y, gamma_f, gamma_r) for k in ideals]
```

Visualize the standards

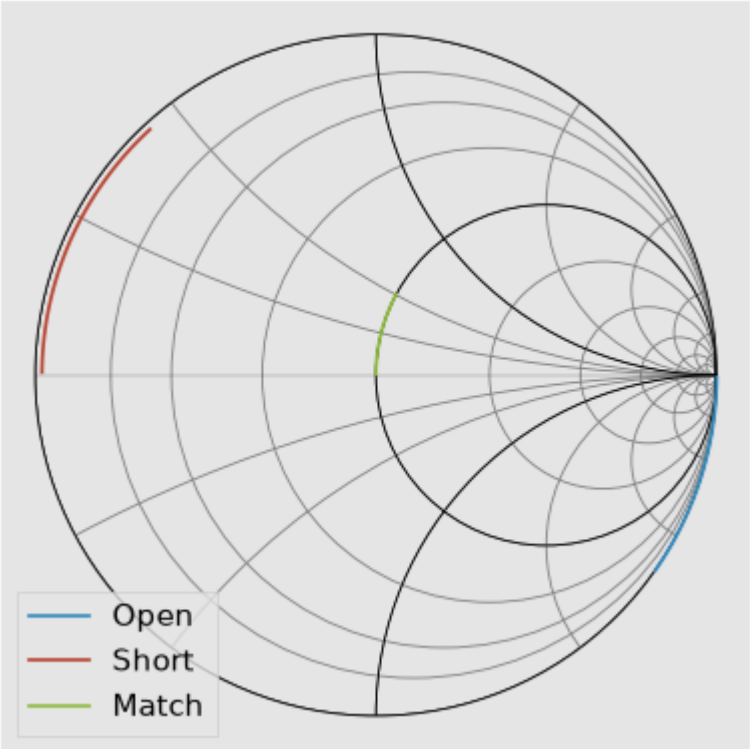
[3]:
```
plt.figure()
oo.plot_s_smith(m=0, n=0, label='Open')
ss.plot_s_smith(m=0, n=0, label='Short')
mm.plot_s_smith(m=0, n=0, label='Match')

plt.figure()
oo.plot_s_db(m=0, n=0, label='Open')
```

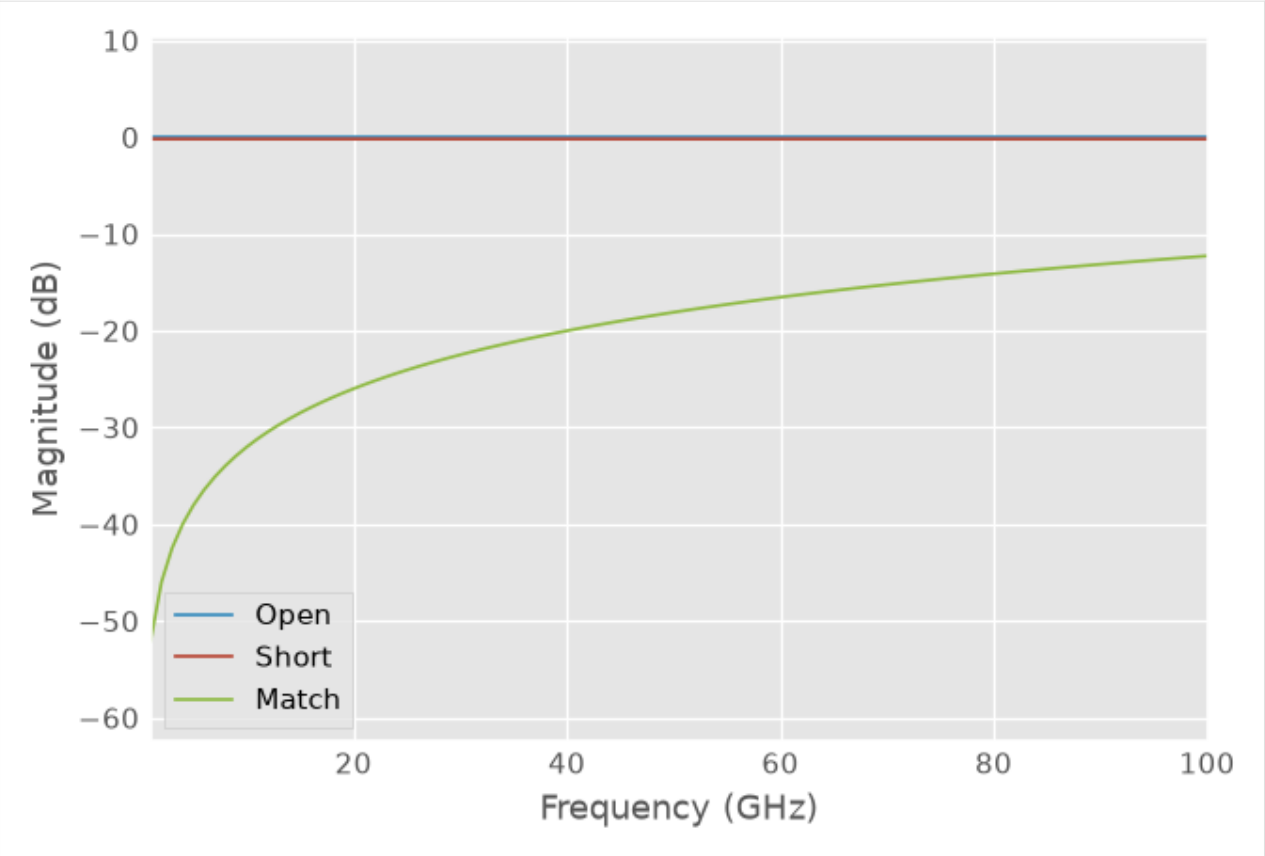
(continues on next page)

(continued from previous page)

```
ss.plot_s_db(m=0, n=0, label='Short')
mm.plot_s_db(m=0, n=0, label='Match')
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪mathFunctions.py:303: RuntimeWarning: divide by zero encountered in log10
  out = 20 * np.log10(z)
```





LRRM calibration

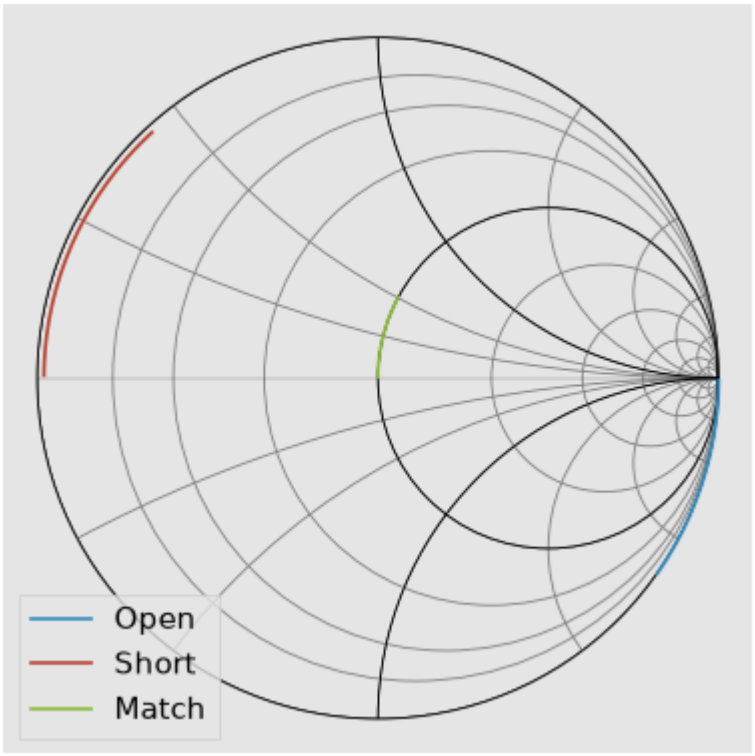
We pretend to not know the actual standards with parasitics and give only our approximations of the standards without parasitics and the measurements of the standard with parasitics.

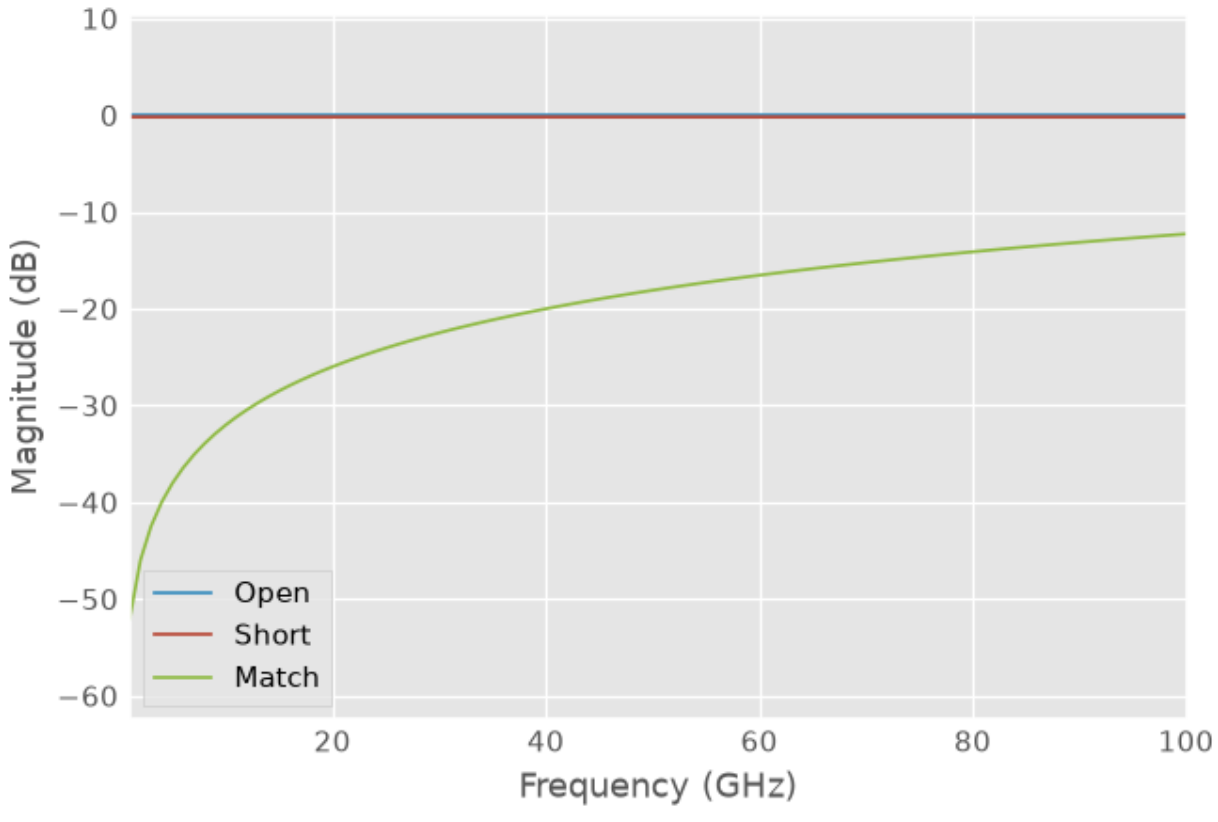
[4]:
```
cal = LRRM(
    ideals = approx_ideals,
    measured = measured,
    switch_terms = [gamma_f, gamma_r])
```

Visualizing the solved standards

LRRM calibration solves for the real standards. We can get the solved standards from the `cal` object. The solved standards should match the actual standards above.

[5]:
```
plt.figure()
cal.solved_r2.plot_s_smith(m=0, n=0, label='Open')
cal.solved_r1.plot_s_smith(m=0, n=0, label='Short')
cal.solved_m.plot_s_smith(m=0, n=0, label='Match')

plt.figure()
cal.solved_r2.plot_s_db(m=0, n=0, label='Open')
cal.solved_r1.plot_s_db(m=0, n=0, label='Short')
cal.solved_m.plot_s_db(m=0, n=0, label='Match')
```


Open
Short
Match


10
0
−10
−20
−30
−40
−50
−60
Magnitude (dB)
20
40
60
80
100
Frequency (GHz)
Open
Short
Match

The solved inductance of the match is also given as calibration output. It's given as an array but with the default options a single inductance is fitted over all frequencies.

[6]:
```
solved_match_l = cal.solved_l[0]
print(f'Solved inductance {1e12*solved_match_l:.1f} pH, actual inductance {1e12*match_l:.
↪1f} pH')
```

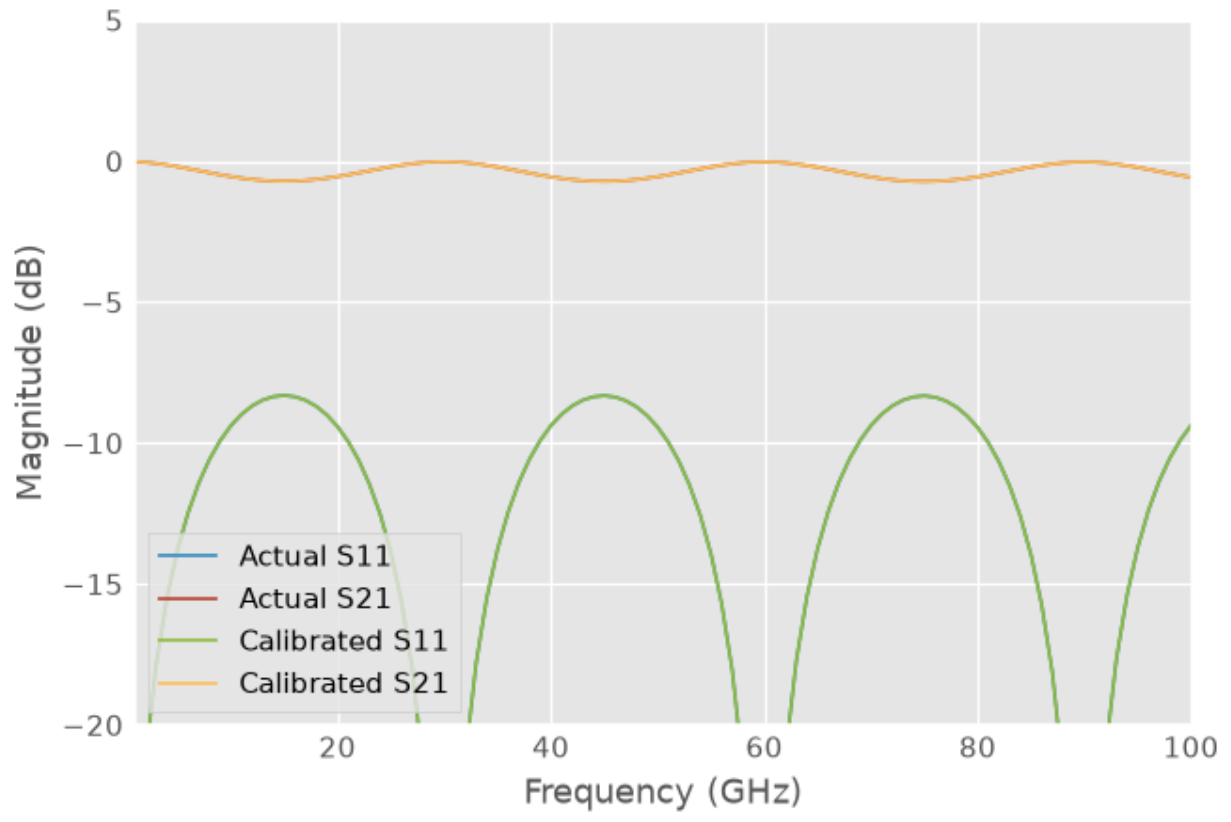
```
Solved inductance 40.0 pH, actual inductance 40.0 pH
```

Calibrating DUT

Measured DUT can be calibrated using the `apply_cal` method. The S-parameters should match exactly.

[7]:
```
# Let's generate a DUT: 5 mm long 75 ohm line.
dut = coax.line(d=5, unit='mm', z0=75)

dut_measured = terminate(X**dut**Y, gamma_f, gamma_r)
dut_cal = cal.apply_cal(dut_measured)

plt.figure()
dut.plot_s_db(m=0, n=0, label='Actual S11')
dut.plot_s_db(m=1, n=0, label='Actual S21')
dut_cal.plot_s_db(m=0, n=0, label='Calibrated S11')
dut_cal.plot_s_db(m=1, n=0, label='Calibrated S21')
plt.ylim([-20, 5])
```

[7]:
```
(-20.0, 5.0)
```



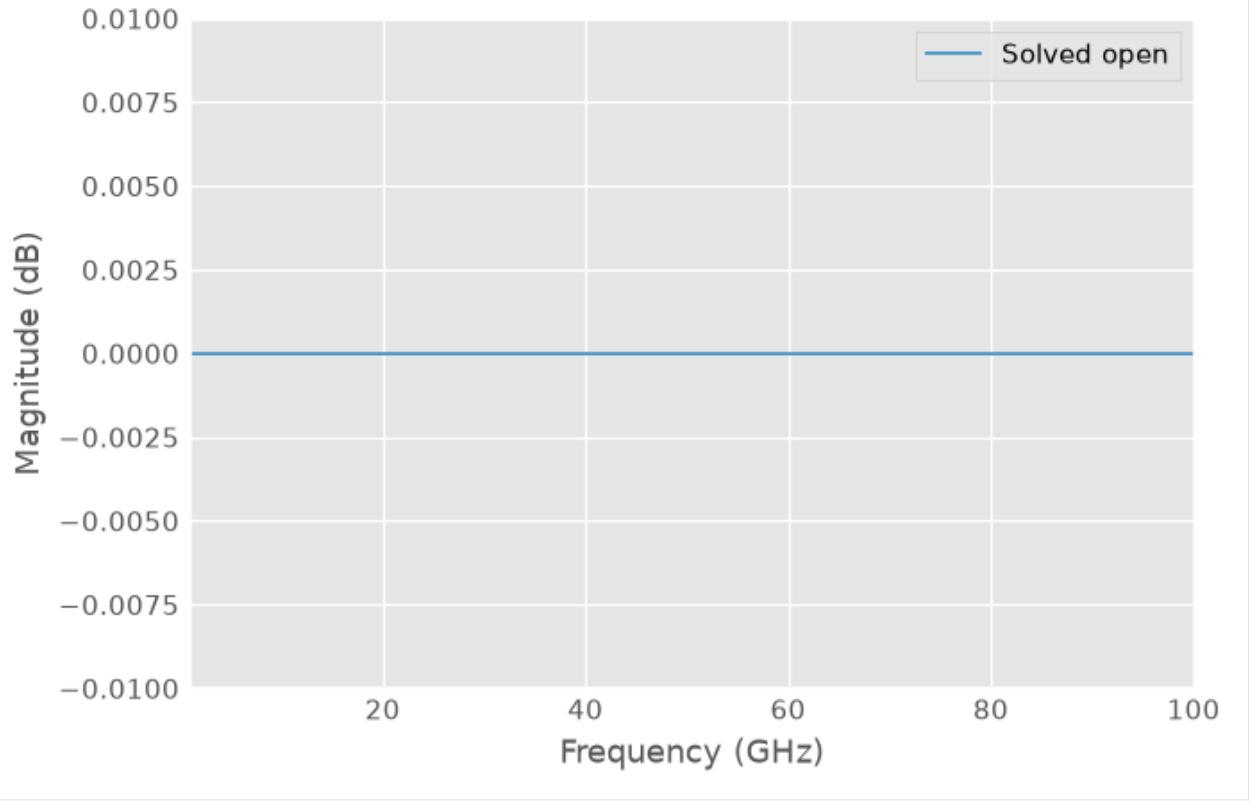
Calibration verification using reflect |S11|

During the calibration the second reflect |S11| is assumed to be known (|S11| = 1 in this case), but when a single inductance is fitted to the match standard this assumption can be broken. If the real match is not modeled well as known resistance in series with inductance it causes the reflect standard losslessness to be violated. By plotting the absolute value of the reflect we can get an idea on how good the calibration assumptions are.

Let's first plot the open |S11| in the previous calibration. It should be exactly 0 dB if everything worked correctly.

[8]:
```
plt.figure()
cal.solved_r2.plot_s_db(m=0, n=0, label='Solved open')
plt.ylim([-0.01, 0.01])
```

[8]:
```
(-0.01, 0.01)
```



Calibration with capacitive match

The LRRM calibration model of the match is a resistance in series with an inductor. If the match also has parallel capacitance it won't be solved correctly and there will be errors in the corrected measurements.

Let's define a new match standard and redo the calibration using it.

[9]:
```
# Match standard with series inductance and parallel capacitance.
match_c = 20e-15
match_l = 20e-12
l = coax.inductor(L=match_l)
c = coax.shunt_capacitor(match_c)
m = c**l**m_i
```

(continues on next page)

(continued from previous page)

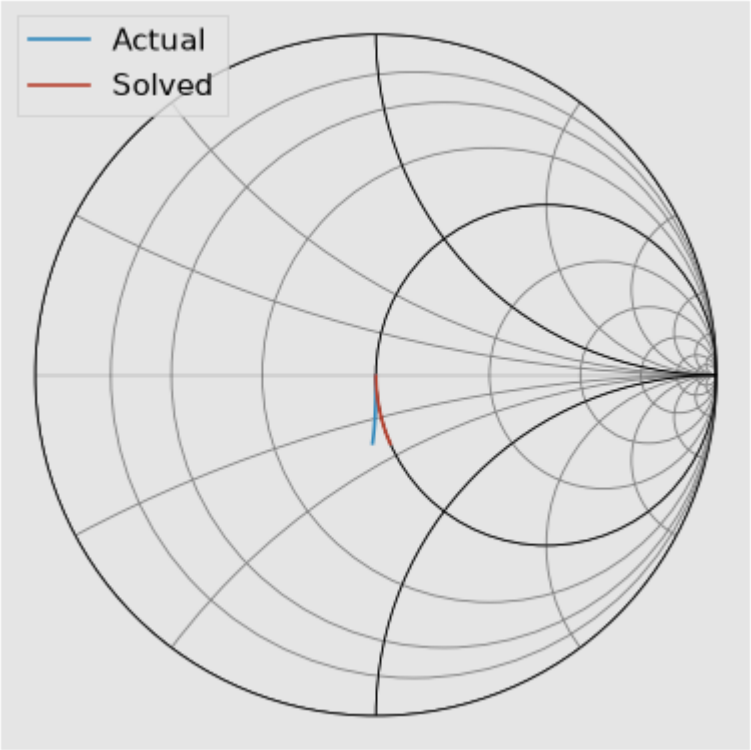
```
# Make two-port of the match with open on the second port.
mm = two_port_reflect(m, coax.open(nports=1))

# Redo the match measurement
ideals[3] = mm
measured[3] = terminate(X**mm**Y, gamma_f, gamma_r)

# Redo the calibration
cal = LRRM(
    ideals = approx_ideals,
    measured = measured,
    switch_terms = [gamma_f, gamma_r]
    )
```

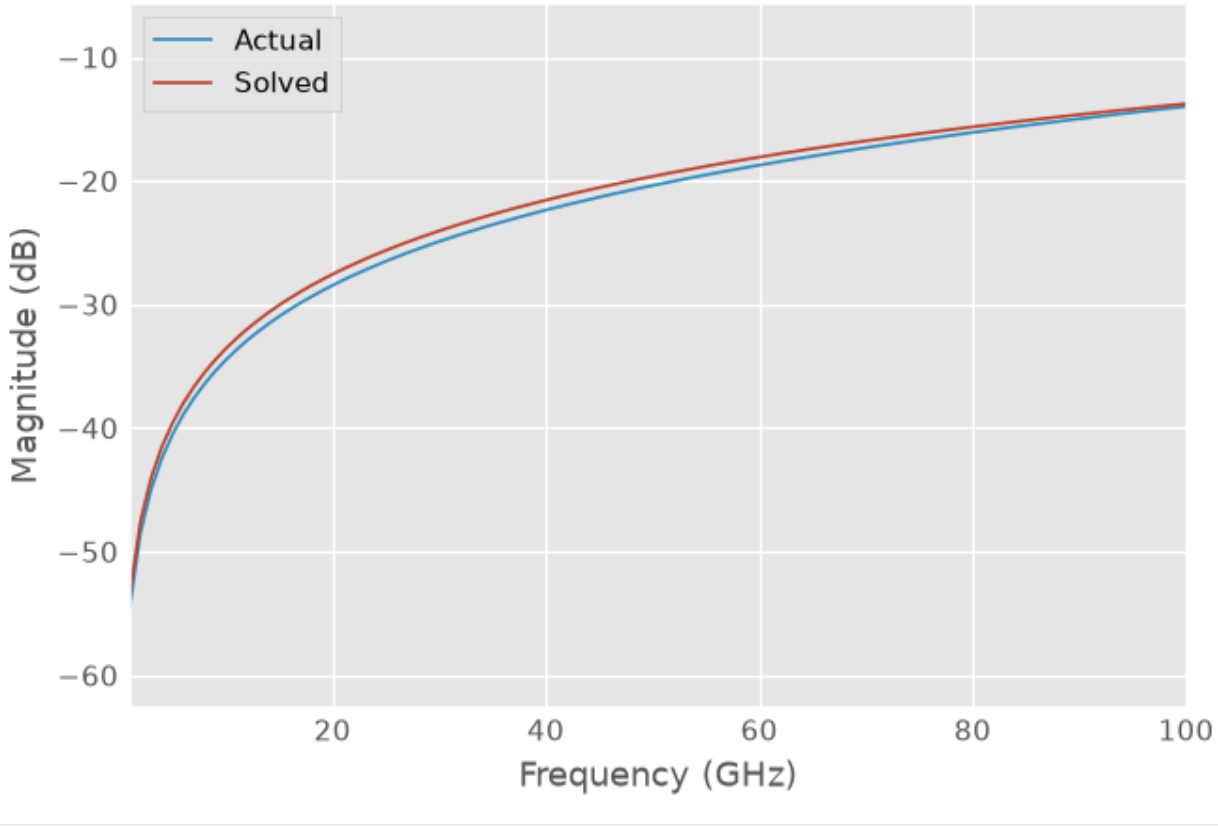
Visualize the capacitive match and the solved match

Calibration tries to fit the inductance to the match as best as it can but it can't model the match exactly with an inductor. The closest fit is a negative valued inductor.

[10]:
```
plt.figure()
mm.plot_s_smith(m=0, n=0, label='Actual')
cal.solved_m.plot_s_smith(m=0, n=0, label='Solved')

plt.figure()
mm.plot_s_db(m=0, n=0, label='Actual')
cal.solved_m.plot_s_db(m=0, n=0, label='Solved')
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪mathFunctions.py:303: RuntimeWarning: divide by zero encountered in log10
  out = 20 * np.log10(z)
```


Actual
Solved

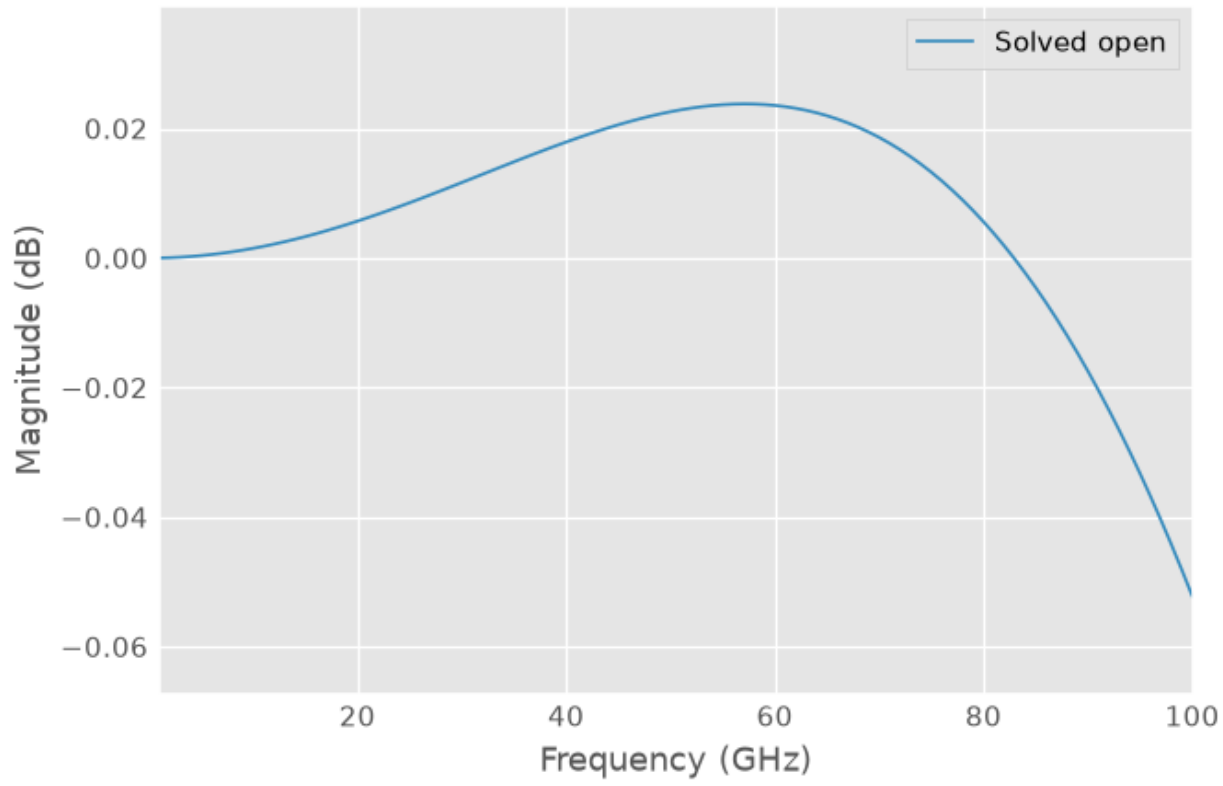

Actual
Solved
−10
−20
−30
−40
−50
−60
Magnitude (dB)
20
40
60
80
100
Frequency (GHz)

[11]:
```
solved_match_l = cal.solved_l[0]
print(f'Solved inductance {1e12*solved_match_l:.1f} pH')
```

```
Solved inductance -33.4 pH
```

Plotting the |S11| of the open reveals that the solved open is not lossless indicating that some of the calibration assumptions were violated.

[12]:
```
plt.figure()
cal.solved_r2.plot_s_db(m=0, n=0, label='Solved open')
```

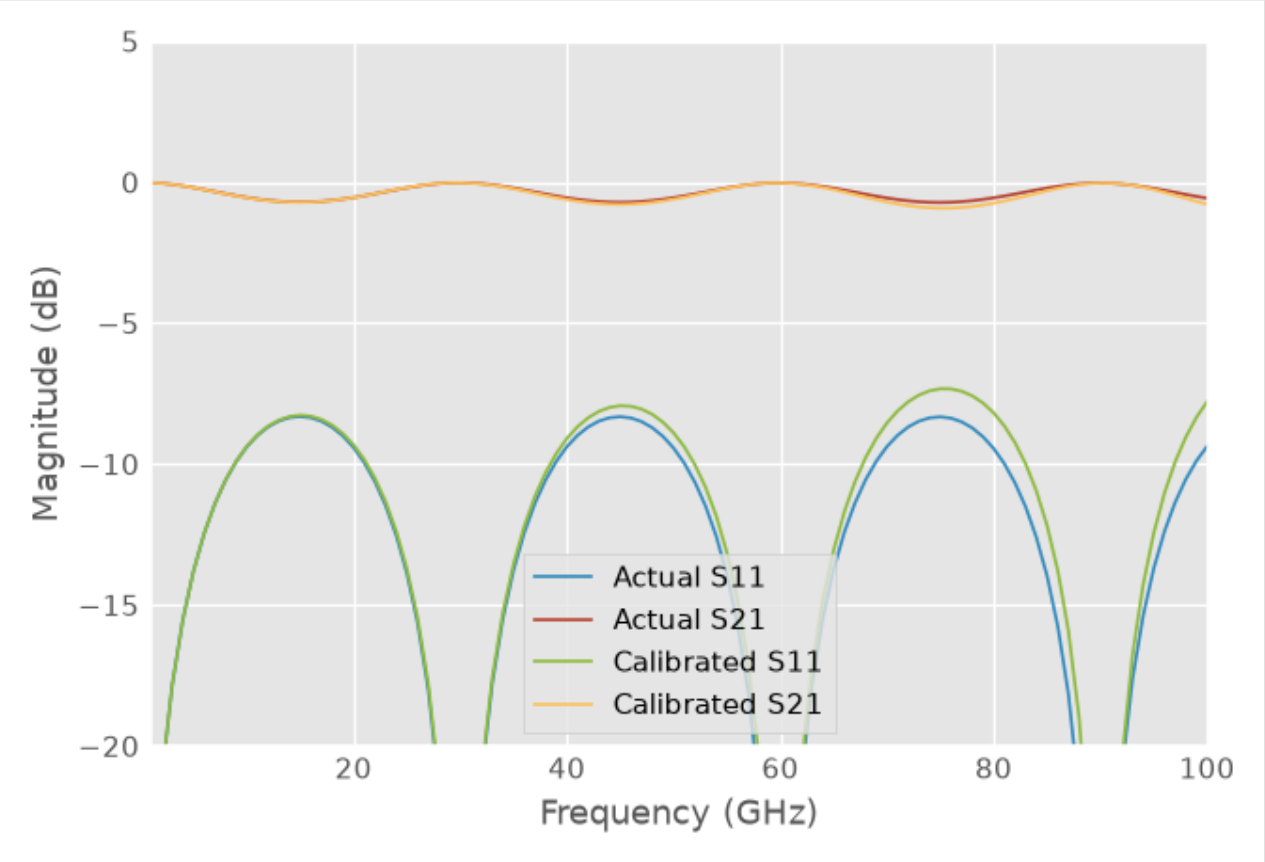


DUT measurement with incorrectly modeled match

The incorrect match causes errors in the calibration parameters. The error increases at higher frequencies where the match modeling error is bigger.

[13]:
```
dut_cal2 = cal.apply_cal(dut_measured)

plt.figure()
dut.plot_s_db(m=0, n=0, label='Actual S11')
dut.plot_s_db(m=1, n=0, label='Actual S21')
dut_cal2.plot_s_db(m=0, n=0, label='Calibrated S11')
dut_cal2.plot_s_db(m=1, n=0, label='Calibrated S21')
plt.ylim([-20, 5])
```

[13]:
```
(-20.0, 5.0)
```

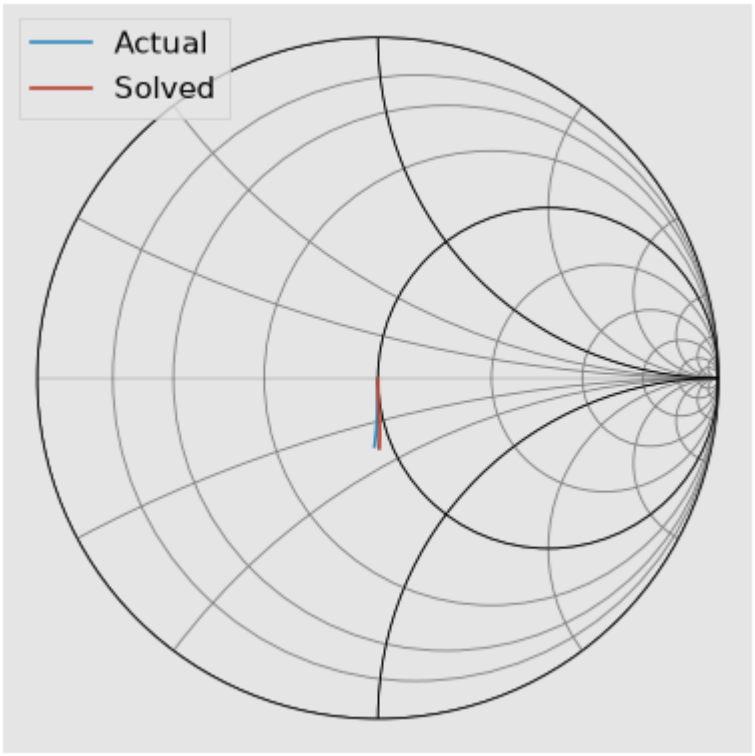


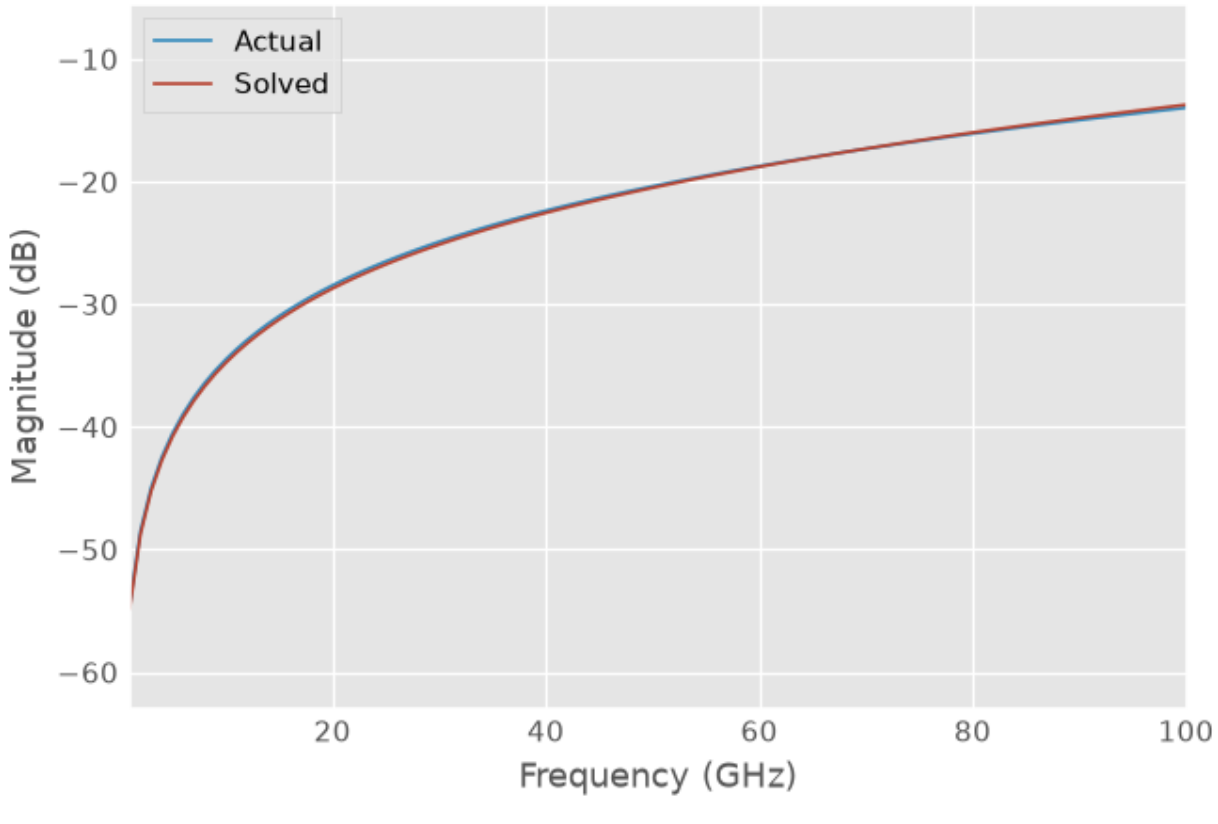
Match fit with inductance and capacitance

LRRM has an option to use a match model with parallel capacitance which allows fitting the above match. The additional requirement for this fitting method is that the second reflect is open with some unknown capacitance. The open capacitance is fitted first assuming match is perfectly resistive weighting low frequencies where the assumption is likely to hold better. When the open capacitance is known match capacitance and inductance are fitted. The open and match fitting is iterated few times to refine the open and match guesses. This fitting method can be used by passing `match_fit = 'lc'` to the calibration method.

[14]:
```
# Redo the calibration using LC match model
cal = LRRM(
    ideals = approx_ideals,
    measured = measured,
    match_fit = 'lc',
    switch_terms = [gamma_f, gamma_r]
    )
```

[15]:
```
plt.figure()
mm.plot_s_smith(m=0, n=0, label='Actual')
cal.solved_m.plot_s_smith(m=0, n=0, label='Solved')

plt.figure()
mm.plot_s_db(m=0, n=0, label='Actual')
cal.solved_m.plot_s_db(m=0, n=0, label='Solved')
```


Actual
Solved


Actual
Solved
−10
−20
−30
−40
−50
−60
Magnitude (dB)
20
40
60
80
100
Frequency (GHz)

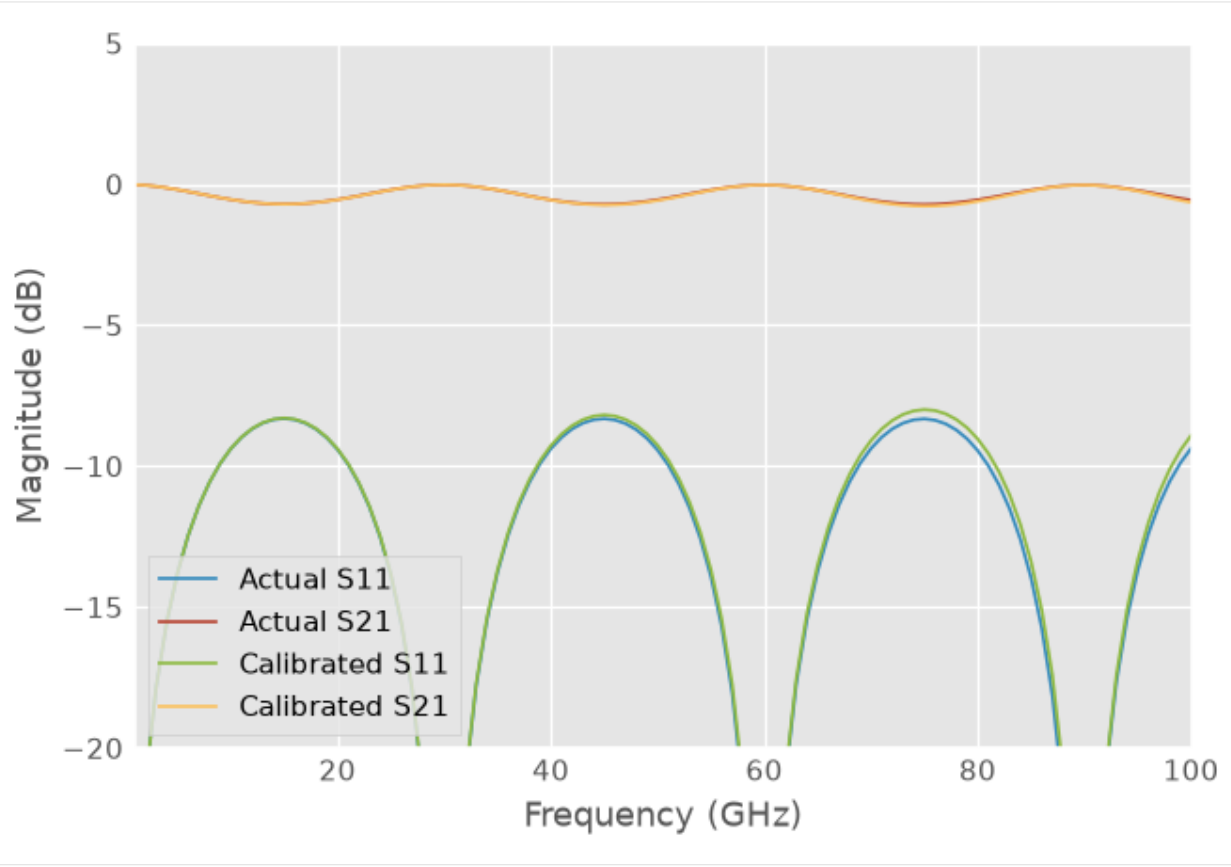
[16]:
```
solved_match_l = cal.solved_l[0]
solved_match_c = cal.solved_c[0]
print(f'Solved inductance {1e12*solved_match_l:.1f} pH, actual inductance {1e12*match_l:.
↪1f} pH')
print(f'Solved capacitance {1e15*solved_match_c:.1f} fF, actual capacitance {1e15*match_
↪c:.1f} fF')
```

```
Solved inductance 24.8 pH, actual inductance 20.0 pH
Solved capacitance 21.5 fF, actual capacitance 20.0 fF
```

Applying the calibration now to the measurements should give a close fit.

[17]:
```
dut_cal3 = cal.apply_cal(dut_measured)

plt.figure()
dut.plot_s_db(m=0, n=0, label='Actual S11')
dut.plot_s_db(m=1, n=0, label='Actual S21')
dut_cal3.plot_s_db(m=0, n=0, label='Calibrated S11')
dut_cal3.plot_s_db(m=1, n=0, label='Calibrated S21')
plt.ylim([-20, 5])
```

[17]:
```
(-20.0, 5.0)
```

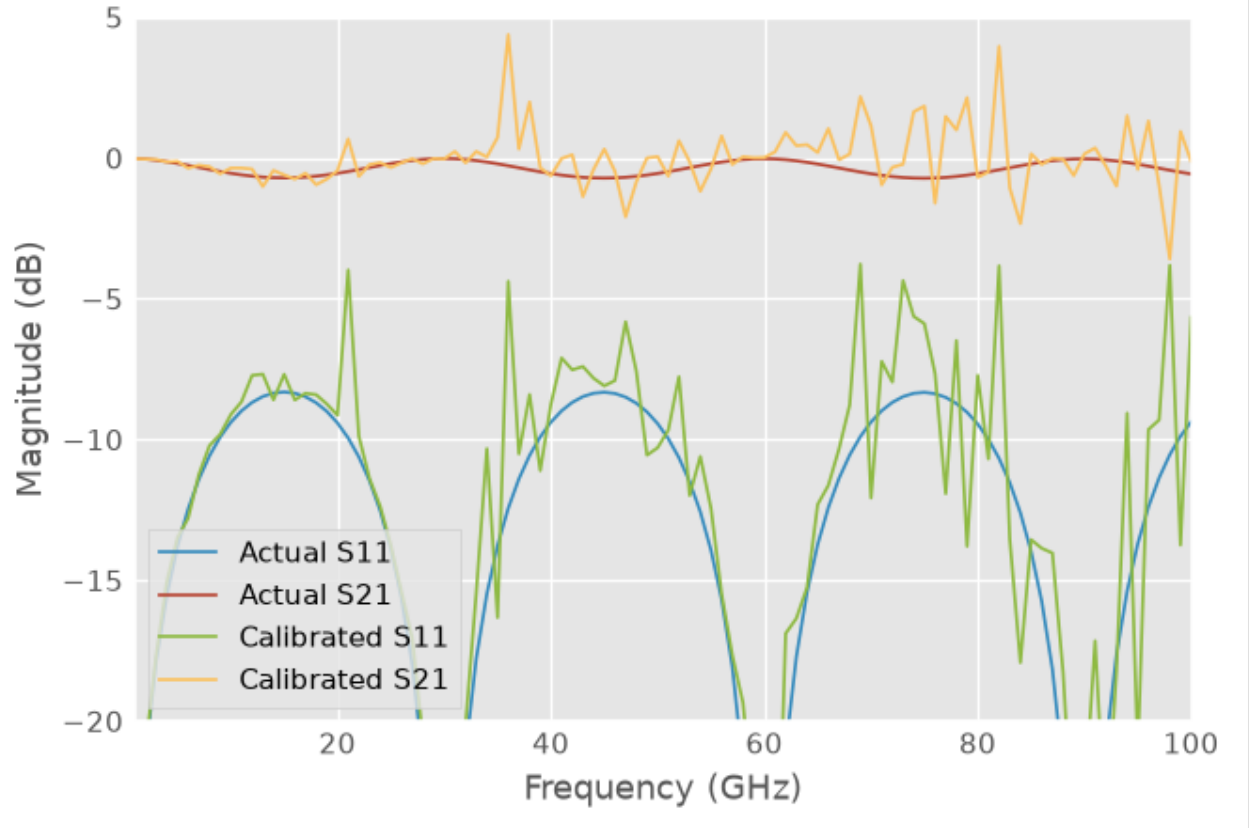


Comparison with SOLT calibration

The traditional two-port SOLT calibration assumes that all standards are known accurately. We can compare how it would perform with the same measurements with the same approximately known standards. Match needs to be also measured on the second port for SOLT, we assume it's identical to the first port. The randomly generated error boxes make the calibration especially difficult.

[18]:
```
# SOLT requires match measurement on both ports.
mm = two_port_reflect(m, m)
measured[3] = terminate(X**mm**Y, gamma_f, gamma_r)

# TwelveTerm assumes that thru is last.
cal12 = TwelveTerm(
    ideals = list(reversed(approx_ideals)),
    measured = list(reversed(measured)),
    n_thrus = 1,
    )

dut_cal12 = cal12.apply_cal(dut_measured)

plt.figure()
dut.plot_s_db(m=0, n=0, label='Actual S11')
dut.plot_s_db(m=1, n=0, label='Actual S21')
dut_cal12.plot_s_db(m=0, n=0, label='Calibrated S11')
dut_cal12.plot_s_db(m=1, n=0, label='Calibrated S21')
plt.ylim([-20, 5])
```

[18]:
```
(-20.0, 5.0)
```



[ ]:

2.3.3 Measuring a Multiport Device with a 2-Port Network Analyzer

Introduction

In microwave measurements, one commonly needs to measure a n-port device with a m-port network analyzer ($m < n$ of course).

This can be done by terminating each non-measured port with a matched load, and assuming the reflected power is negligible. With multiple measurements, it is then possible to reconstitute the original n-port. The first section of this example illustrates this method.

However, in some cases this may not provide the most accurate results, or even be possible in all measurement environments. Or, sometime it is not possible to have matched loads for all ports. The second part of this example presents an elegant solution to this problem, using impedance renormalization. We'll call it *Tippet's technique*, because it has a good ring to it.

[1]:
```
from itertools import combinations

import matplotlib.pyplot as plt
import numpy as np

import skrf as rf
from skrf.network import connect, subnetwork

%matplotlib inline

rf.stylely()
```

Matched Ports

Let's assume that you have a 2-ports VNA. In order to measure a n-port network, you will need at least $p = n(n-1)/2$ measurements between the different pair of ports (total number of unique pairs of a set of n).

For example, let's assume we wants to measure a 3-ports network with a 2-ports VNA. One needs to perform at least 3 measurements: between ports 1 & 2, between ports 2 & 3 and between ports 1 & 3. We will assume these measurements are then converted into three 2-ports `Network`. To build the full 3-ports `Network`, one needs to provide a list of these 3 (sub)networks to the scikit-rf builtin function `n_twoports_2_nport`. While the order of the measurements in the list is not important, pay attention to define the `Network.name` properties of these subnetworks to contain the port index, for example `p12` for the measurement between ports 1&2 or `p23` between 2&3, etc.

Let's suppose we want to measure a tee:

[2]:
```
tee = rf.data.tee
print(tee)
```

```
3-Port Network: 'tee',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j 50.+0.j]
```

For the sake of the demonstration, we will "fake" the 3 distinct measurements by extracting 3 subsets of the original Network, i.e., 3 subnetworks:

[3]:
```
# 2 port Networks as if one measures the tee with a 2 ports VNA
tee12 = subnetwork(tee, [0, 1])  # 2 port Network btw ports 1 & 2, port 3 being matched
```

(continues on next page)

(continued from previous page)

```
tee23 = subnetwork(tee, [1, 2])  # 2 port Network btw ports 2 & 3, port 1 being matched
tee13 = subnetwork(tee, [0, 2])  # 2 port Network btw ports 1 & 3, port 2 being matched
```

In reality of course, these three Networks comes from three measurements with distinct pair of ports, the non-used port being properly matched.

Before using the `n_twoports_2_nport` function, one must define the name of these subsets by setting the `Network.name` property, in order the function to know which corresponds to what:

[4]:
```
tee12.name = 'tee12'
tee23.name = 'tee23'
tee13.name = 'tee13'
```

Now we can build the 3-ports Network from these three 2-port subnetworks:

[5]:
```
ntw_list = [tee12, tee23, tee13]
tee_rebuilt = rf.network.n_twoports_2_nport(ntw_list, nports=3)
print(tee_rebuilt)
```

```
3-Port Network: '',  330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j 50.+0.j]
```

[6]:
```
# this is an ideal example, both Network are thus identical
print(tee == tee_rebuilt)
```

```
True
```

Tippet's Technique

This example demonstrates a numerical test of the technique described in "*A Rigorous Technique for Measuring the Scattering Matrix of a Multiport Device with a 2-Port Network Analyzer*" [1].

In *Tippets technique*, several sub-networks are measured in a similar way as before, but the port terminations are not assumed to be matched. Instead, the terminations just have to be known and no more than one can be completely reflective. So, in general $|\Gamma| \neq 1$.

During measurements, each port is terminated with a consistent termination. So port 1 is always terminated with Z_1 when not being measured. Once measured, each sub-network is re-normalized to these port impedances. Think about that. Finally, the composite network is constructed, and may then be re-normalized to the desired system impedance, say 50 ohm.

- [1] J. C. Tippet and R. A. Speciale, "A Rigorous Technique for Measuring the Scattering Matrix of a Multiport Device with a 2-Port Network Analyzer," IEEE Transactions on Microwave Theory and Techniques, vol. 30, no. 5, pp. 661–666, May 1982.

Outline of Tippet's Technique

Following the example given in [1], measuring a 4-port network with a 2-port network analyzer.

An outline of the technique:

1. Calibrate 2-port network analyzer
2. Get four known terminations (Z_1, Z_2, Z_3, Z_4). No more than one can have $|\Gamma| = 1$
3. Measure all combinations of 2-port subnetworks (there are 6). Each port not currently being measured must be terminated with its corresponding load.
4. Renormalize each subnetwork to the impedances of the loads used to terminate it when note being measured.

5. Build composite 4-port, renormalize to VNA impedance.

Implementation

First, we create a Media object, which is used to generate networks for testing. We will use WR-10 Rectangular waveguide.

[7]:
```
wg = rf.instances.wr10
```

Next, lets generate a random 4-port network which will be the DUT, that we are trying to measure with out 2-port network analyzer.

[8]:
```
dut = wg.random(n_ports  = 4,name= 'dut')
dut
```

[8]:
```
4-Port Network: 'dut',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j 50.+0.j 50.+0.j 50.+0.j]
```

Now, we need to define the loads used to terminate each port when it is not being measured, note as described in [1] not more than one can be have full reflection, $|\Gamma| = 1$

[9]:
```
loads = [wg.load(.1+.1j),
         wg.load(.2-.2j),
         wg.load(.3+.3j),
         wg.load(.5),
         ]
# construct the impedance array, of shape FXN
z_loads = np.array([k.z.flatten() for k in loads]).T
```

Create required measurement port combinations. There are 6 different measurements required to measure a 4-port with a 2-port VNA. In general, #measurements = $\binom{n}{2}$, for n-port DUT on a 2-port VNA.

[10]:
```
ports = np.arange(dut.nports)
port_combos = list(combinations(ports, 2))
port_combos
```

[10]:
```
[(np.int64(0), np.int64(1)),
 (np.int64(0), np.int64(2)),
 (np.int64(0), np.int64(3)),
 (np.int64(1), np.int64(2)),
 (np.int64(1), np.int64(3)),
 (np.int64(2), np.int64(3))]
```

Now to do it. Ok we loop over the port combo's and connect the loads to the right places, simulating actual measurements. Each raw subnetwork measurement is saved, along with the renormalized subnetwork. Finally, we stuff the result into the 4-port composite network.

[11]:
```
composite = wg.match(nports = 4)  # composite network, to be filled.
measured,measured_renorm = {},{}  # measured subnetworks and renormalized sub-networks

# ports  `a` and `b` are the ports we will connect the VNA too
for a,b in port_combos:
    # port `c` and `d` are the ports which we will connect the loads too
    c,d =ports[(ports!=a)& (ports!=b)]
```

(continues on next page)

(continued from previous page)

```
    # determine where `d` will be on four_port, after its reduced to a three_port
    e = np.where(ports[ports!=c]==d)[0][0]

    # connect loads
    three_port = connect(dut,c, loads[c],0)
    two_port =  connect(three_port,e, loads[d],0)

    # save raw and renormalized 2-port subnetworks
    measured['%i%i'%(a,b)] = two_port.copy()
    two_port.renormalize(np.c_[z_loads[:,a],z_loads[:,b]])
    measured_renorm['%i%i'%(a,b)] = two_port.copy()

    # stuff this 2-port into the composite 4-port
    for i,m in enumerate([a,b]):
        for j,n in enumerate([a,b]):
            composite.s[:,m,n] = two_port.s[:,i,j]

    # properly copy the port impedances
    composite.z0[:,a] = two_port.z0[:,0]
    composite.z0[:,b] = two_port.z0[:,1]

# finally renormalize from load z0 to 50 ohms
composite.renormalize(50)
```

Results

Self-Consistency

Note that 6-measurements of 2-port subnetworks works out to 24 s-parameters, and we only need 16. This is because each reflect, s-parameter is measured three-times. As, in [1], we will use this redundant measurement as a check of our accuracy.

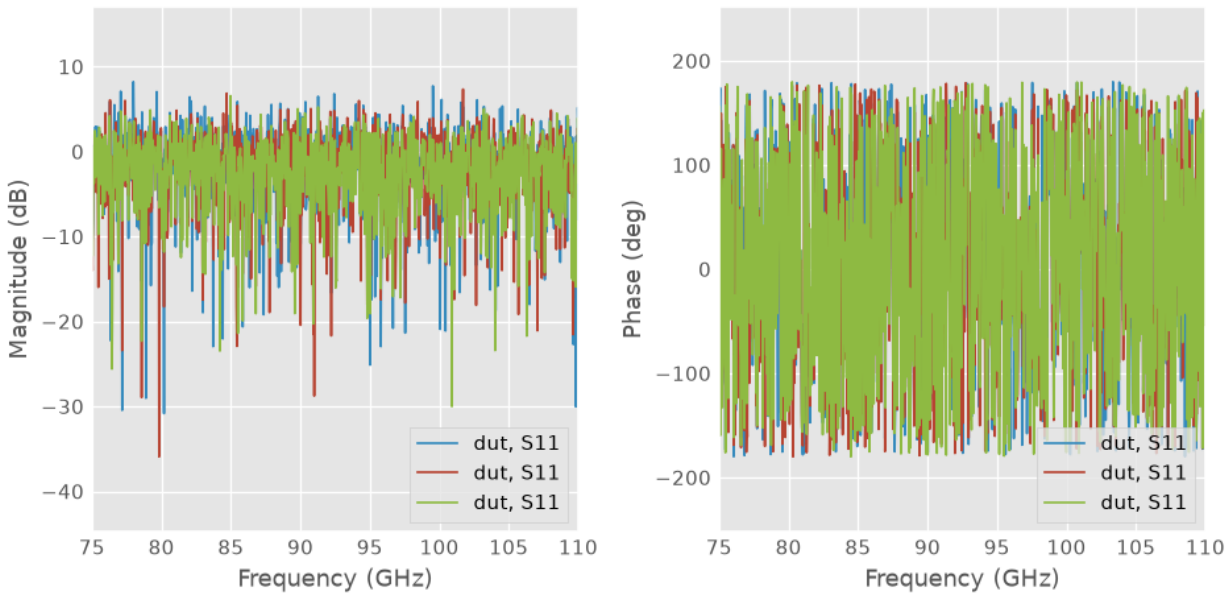
The renormalized networks are stored in a dictionary with names based on their port indices, from this you can see that each have been renormalized to the appropriate z0.

[12]:
```
measured_renorm
```

[12]:
```
{'01': 2-Port Network: 'dut',  75.0-110.0 GHz, 1001 pts, z0=[59.75609756+12.19512195j 67.
↪64705882-29.41176471j],
 '02': 2-Port Network: 'dut',  75.0-110.0 GHz, 1001 pts, z0=[59.75609756+12.19512195j 70.
↪68965517+51.72413793j],
 '03': 2-Port Network: 'dut',  75.0-110.0 GHz, 1001 pts, z0=[ 59.75609756+12.19512195j␣
↪150.         +0.j        ],
 '12': 2-Port Network: 'dut',  75.0-110.0 GHz, 1001 pts, z0=[67.64705882-29.41176471j 70.
↪68965517+51.72413793j],
 '13': 2-Port Network: 'dut',  75.0-110.0 GHz, 1001 pts, z0=[ 67.64705882-29.41176471j␣
↪150.         +0.j        ],
 '23': 2-Port Network: 'dut',  75.0-110.0 GHz, 1001 pts, z0=[ 70.68965517+51.72413793j␣
↪150.         +0.j        ]}
```

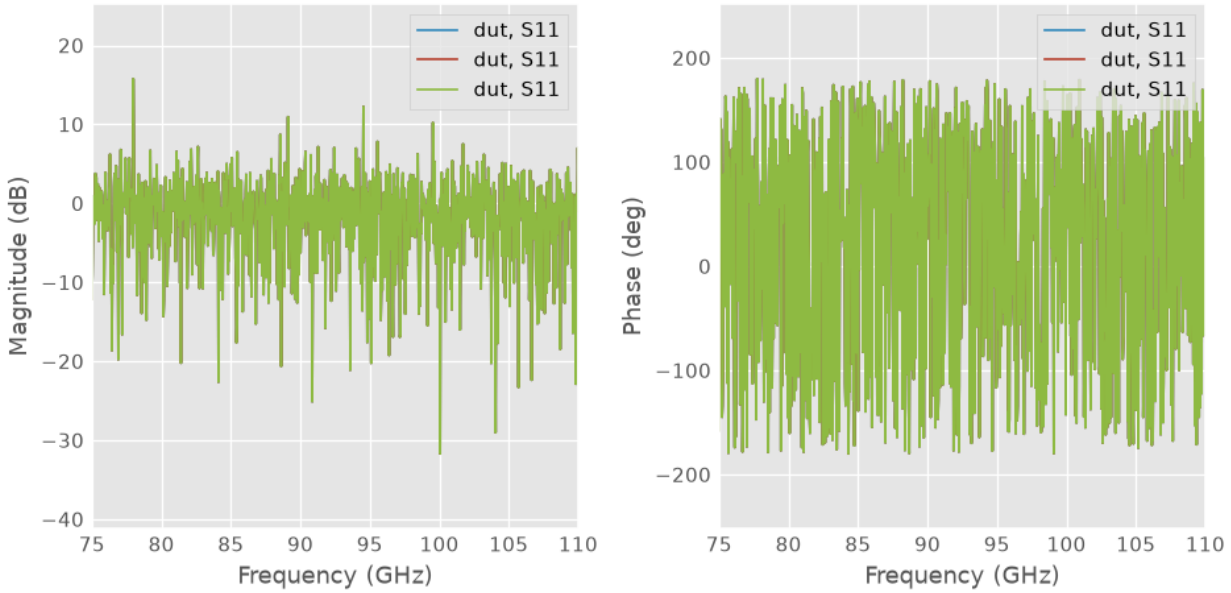
Plotting all three raw measurements of S_{11}, we can see that they are not in agreement. These plots answer to plots 5 and 7 of [1]

[13]:
```
s11_set = rf.networkSet.NetworkSet([measured[k] for k in measured if k[0]=='0'])

plt.figure(figsize = (8,4))
plt.subplot(121)
s11_set .plot_s_db(0,0)
plt.subplot(122)
s11_set .plot_s_deg(0,0)
plt.tight_layout()
```



However, the renormalized measurements agree perfectly. These plots answer to plots 6 and 8 of [1]

[14]:
```
s11_set = rf.networkSet.NetworkSet([measured_renorm[k] for k in measured_renorm if k[0]==
↪'0'])

plt.figure(figsize = (8,4))
plt.subplot(121)
s11_set .plot_s_db(0,0)
plt.subplot(122)
s11_set .plot_s_deg(0,0)
plt.tight_layout()
```



Test For Accuracy

Making sure our composite network is the same as our DUT

[15]:
```
composite == dut
```

[15]:
```
True
```

Nice!. How close ?

[16]:
```
sum((composite - dut).s_mag)
```

[16]:
```
array([[5.69405366e-13, 5.54028889e-13, 5.49784325e-13, 5.08556681e-13],
       [5.29531102e-13, 5.14282304e-13, 5.17591362e-13, 4.70236964e-13],
       [5.49259314e-13, 5.00233904e-13, 5.28772349e-13, 4.74618049e-13],
       [5.16778685e-13, 4.73998321e-13, 4.74856849e-13, 4.92654364e-13]])
```

Dang!

Practical Application

This could be used in many ways. In waveguide, one could just make a measurement of a radiating open after a standard two-port calibration (like TRL). Then using *Tippets technique*, you can leave each port wide open while not being measured. This way you dont have to buy a bunch of loads. How sweet would that be?

More Complex Simulations

[17]:
```
def tippits(dut, gamma, noise=None):
    """simulate tippits technique on a 4-port dut.
    """
    ports = np.arange(dut.nports)
    port_combos = list(combinations(ports, 2))
```

(continues on next page)

(continued from previous page)

```
    loads = [wg.load(gamma) for k in ports]

    # construct the impedance array, of shape FXN
    z_loads = np.array([k.z.flatten() for k in loads]).T
    composite = wg.match(nports = dut.nports)  # composite network, to be filled.

    # ports  `a` and `b` are the ports we will connect the VNA too
    for a,b in port_combos:
        # port `c` and `d` are the ports which we will connect the loads too
        c,d =ports[(ports!=a)& (ports!=b)]

        # determine where `d` will be on four_port, after its reduced to a three_port
        e = np.where(ports[ports!=c]==d)[0][0]

        # connect loads
        three_port = connect(dut,c, loads[c],0)
        two_port =  connect(three_port,e, loads[d],0)

        if noise is not None:
            two_port.add_noise_polar(*noise)
        # save raw and renormalized 2-port subnetworks
        measured['%i%i'%(a,b)] = two_port.copy()
        two_port.renormalize(np.c_[z_loads[:,a],z_loads[:,b]])
        measured_renorm['%i%i'%(a,b)] = two_port.copy()

        # stuff this 2-port into the composite 4-port
        for i,m in enumerate([a,b]):
            for j,n in enumerate([a,b]):
                composite.s[:,m,n] = two_port.s[:,i,j]

        # properly copy the port impedances
        composite.z0[:,a] = two_port.z0[:,0]
        composite.z0[:,b] = two_port.z0[:,1]

    # finally renormalize from load z0 to 50 ohms
    composite.renormalize(50)

    return composite
```

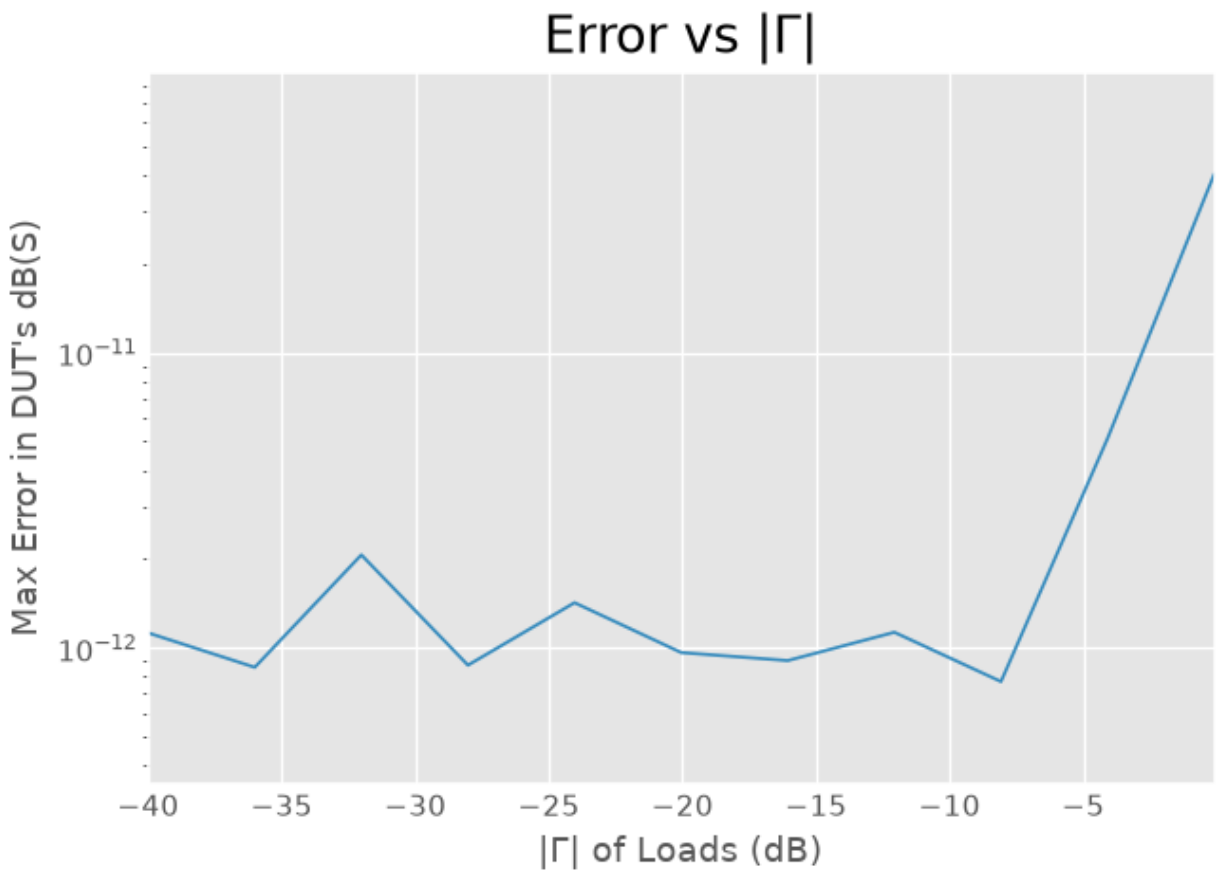
[18]:
```
dut = wg.random(4)

def er(gamma, *args):
    return max(abs(tippits(dut, rf.mathFunctions.db_2_mag(gamma),*args).s_db-dut.s_db).
↪flatten())

gammas = np.linspace(-40,-0.1,11)

plt.title(r'Error vs $|\Gamma|$')
plt.plot(gammas, [er(k) for k in gammas])
plt.semilogy()
plt.xlabel(r'$|\Gamma|$ of Loads (dB)')
plt.ylabel('Max Error in DUT\'s dB(S)')
```

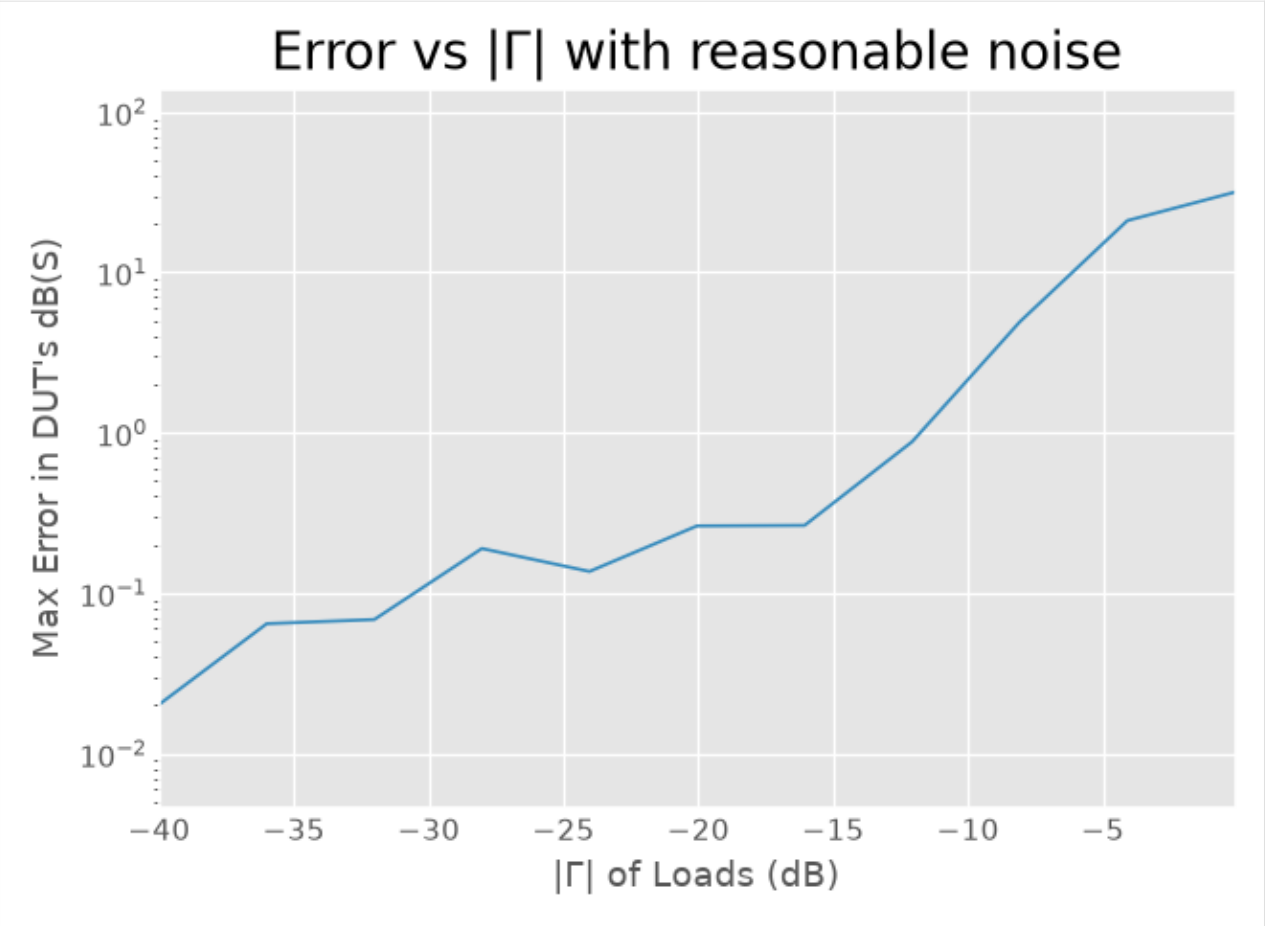
(continues on next page)

(continued from previous page)

```
plt.figure()
noise = (1e-5,.1)
plt.title(r'Error vs $|\Gamma|$ with reasonable noise')
plt.plot(gammas, [er(k, noise) for k in gammas])
plt.semilogy()
plt.xlabel(r'$|\Gamma|$ of Loads (dB)')

plt.ylabel('Max Error in DUT\'s dB(S)')
```

[18]:
```
Text(0, 0.5, "Max Error in DUT's dB(S)")
```





2.3.4 Multi-port calibration

This example demonstrates how `MultiportSOLT` calibration class can be used to calibrate multi-port S-parameter measurement with three or more ports.

Setup

[1]:
```
import numpy as np

import skrf
from skrf.calibration import SOLT, MultiportCal, MultiportSOLT, UnknownThru, terminate_
↪nport
from skrf.frequency import Frequency
from skrf.media import Coaxial
from skrf.network import connect, twoport_to_nport

skrf.stylely()
```

Creating synthetic data

First we create a synthetic error network.

[2]:
```
freq = Frequency(1,100,100, unit='GHz')
nports = 3
# 1.0 mm coaxial media for calibration error boxes
coax = Coaxial(freq, z0_port=50, Dint=0.44e-3, Dout=1.0e-3, sigma=1e8)

# Create random error network
# First 'nports' ports will connect to an ideal VNA
# and the last 'nports' ports will connect to the DUT.
Z = coax.random(n_ports = 2*nports, name = 'Z')

# Zero leakage terms in the error network.
port_type = lambda n: 'VNA' if n < nports else 'DUT'  # noqa: E731
port_number = lambda n: n if n < nports else n - nports  # noqa: E731
for i in range(2*nports):
    for j in range(i+1, 2*nports):
        # No connection between different VNA/DUT ports.
        # No connection between VNA and DUT ports with different number.
        if port_type(i) == port_type(j) or port_number(i) != port_number(j):
            Z.s[:,i,j] = 0
            Z.s[:,j,i] = 0

# VNA switch terms. This is the termination impedance of each port
# when the source is not connected to that port.
gammas = []
for i in range(nports):
    gammas.append(coax.random(n_ports=1, name=f'gamma_{i}'))
```

Function for measuring a network through the error network. Simply connects error network to DUT network and adds the termination impedances.

[3]:
```
def measure(ntwk, Z, gammas, nports):
    out = terminate_nport(connect(Z, nports, ntwk, 0, num=nports), gammas)
    out.name = ntwk.name
    return out
```

Next, create calibration standards. They should be multi-port networks. With real VNA measurement it might be necessary to manually combine two-port standard measurements to N-ports.

The required standard are open, short and match on each port and thru measurement from the first port to all the other ports.

[4]:
```
o = coax.open(nports=nports, name='open')
s = coax.short(nports=nports, name='short')
m = coax.match(nports=nports, name='load')

thru = coax.thru(name='thru')

ideals = []
# nports-1 thrus from port 0 to all other ports.
for i in range(1, nports):
    thru_i = twoport_to_nport(thru, 0, i, nports)
    ideals.append(thru_i)
```

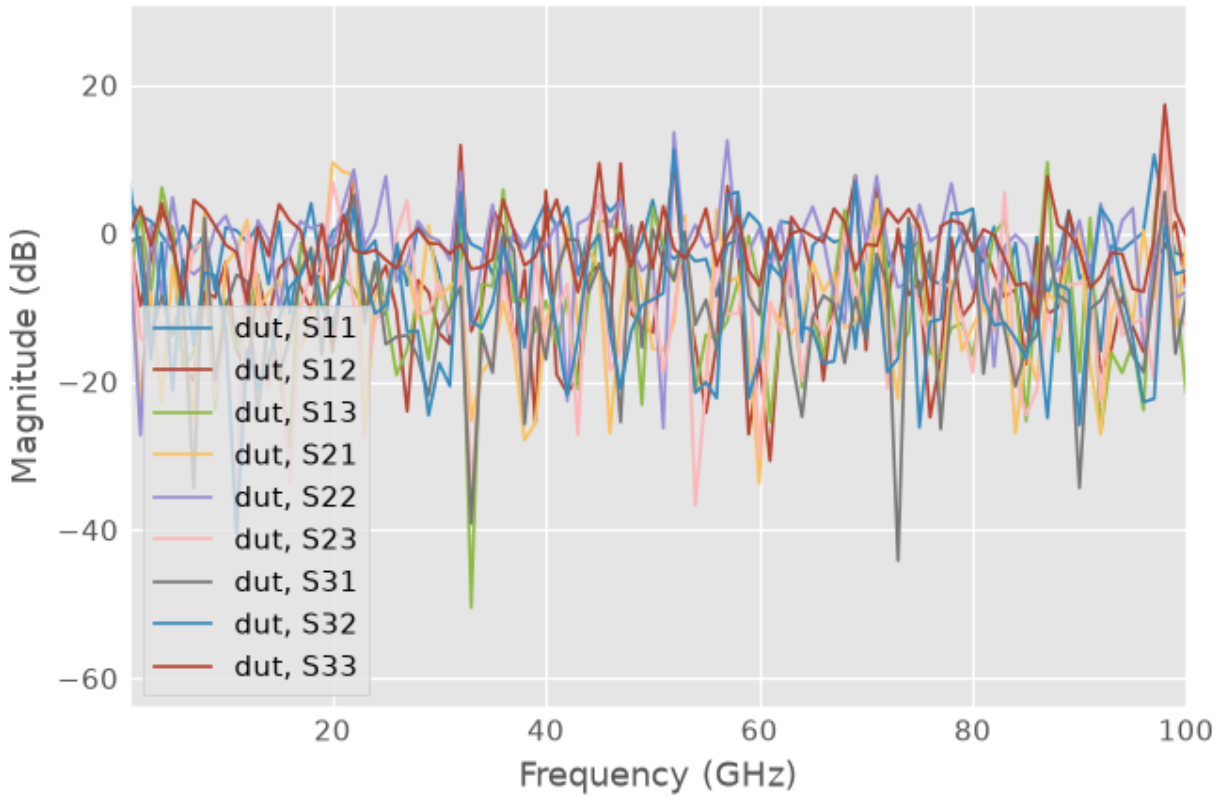
(continues on next page)

(continued from previous page)

```
ideals.extend([o,s,m])
measured = [measure(k, Z, gammas, nports) for k in ideals]
```

Device to test the calibration. A simple three-way junction. Because error network is randomly generated and have very high reflections, the plotted S-parameters of the uncalibrated network are unrecognizable.

[5]:
```
dut = coax.tee(name='dut')
dut_meas = measure(dut, Z, gammas, nports)
dut_meas.plot_s_db()
```



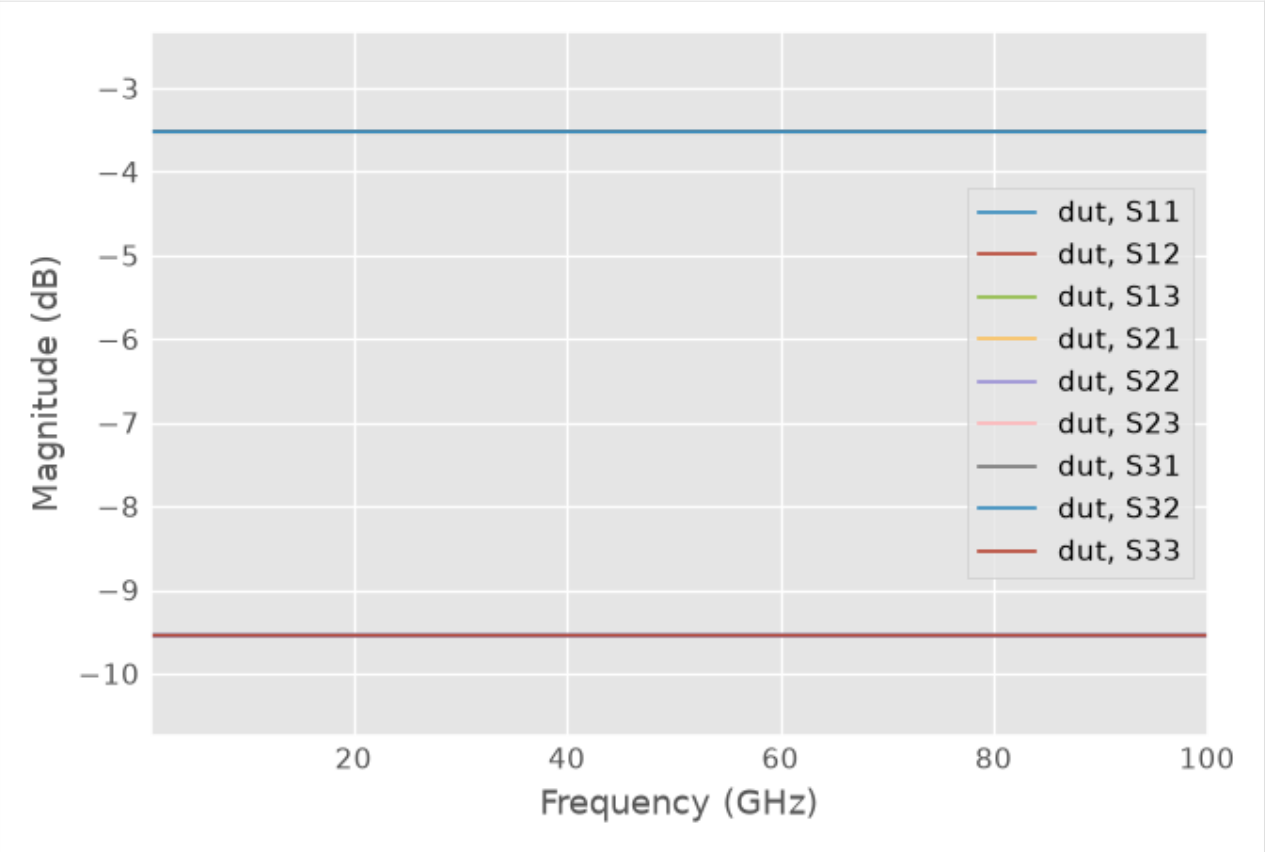
Calibration

Create `MultiportSOLT` calibration class. The first parameter is a two-port calibration method that the class uses for calibrating the multi-port. In this case we use simple SOLT calibration. Other possibilities are for example `UnknownThru` and `LRRM`. Note that different calibration algorithms might require standards to be given in specific order and might require additional inputs such as `switch_terms`. Refer to the two-port calibration documentation for the required inputs.

[6]:
```
cal = MultiportSOLT(method=SOLT, measured=measured, ideals=ideals)
```

Calibrate the DUT and plot the calibrated S-parameters.

[7]:
```
dut_cal = cal.apply_cal(dut_meas)
dut_cal.plot_s_db()
```



The difference between corrected and original dut S-parameters should be small.

[8]:
```
np.max(np.abs(dut_cal.s - dut.s))
```

[8]:
```
np.float64(1.0456303716445236e-06)
```

MultiportCal

The above `MultiportSOLT` class can only be used for SOLT style two-port calibrations that require only one transmissive standard between the two ports. Multi-port TRL requires thru and one or more lines between the ports which doesn't fit the above class interface. Also if for some reason different two-port calibration methods should be used for the port pairs the `MultiportSOLT` class is not able to do it. For those cases there is a lower level `MultiportCal`.

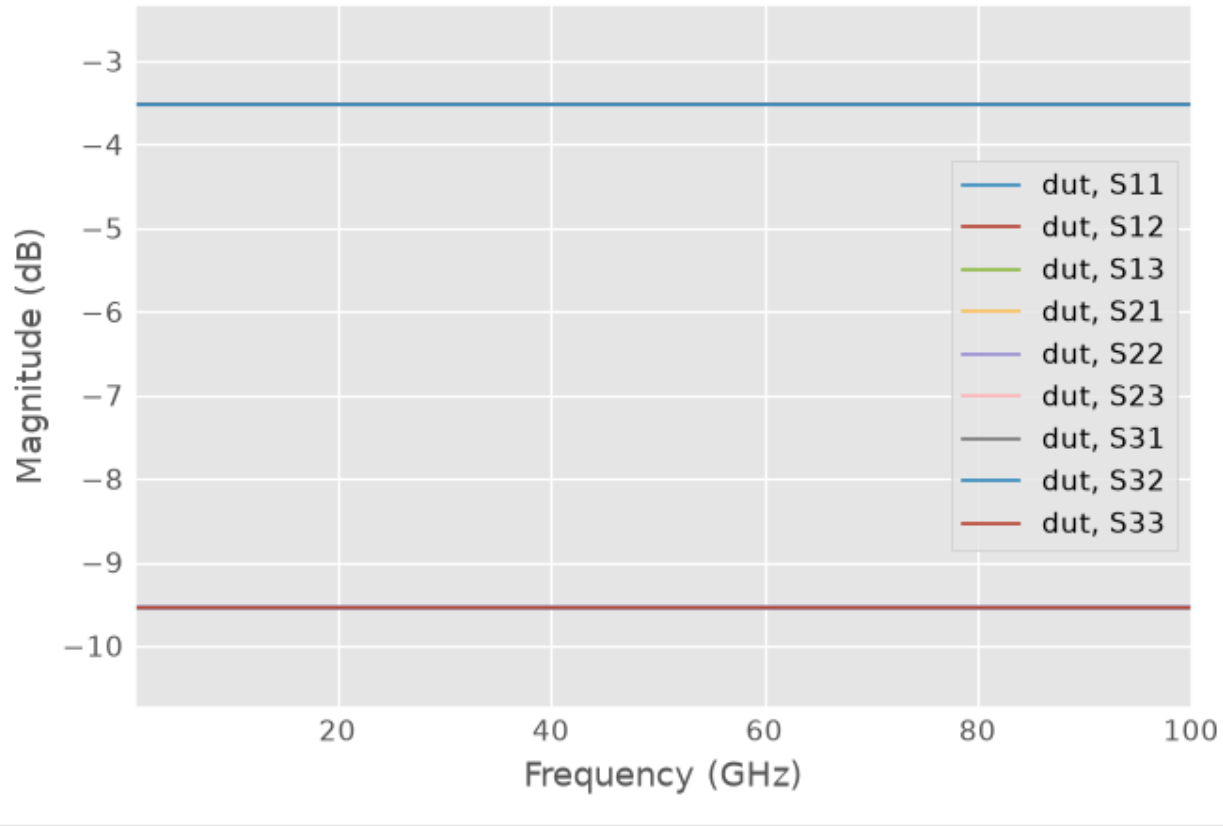
This calibration requires dictionary of port pairs as input. 'method' key is the two-port calibration class and 'measured' are the measured networks. They can be either two-ports or N-ports. Other keys are passed to the two-port calibration routine. In this case we are using `UnknownThru` class which requires additional `switch_term` arguments. Note that two-port switch terms are in the reverse order from multi-port switch terms. Two-port switch terms are listed in the order of where the source is connected, but this is unspecified in case of multi-port switch terms and they are listed in the order of which port they terminate. It's also important to list the ideals and measured networks in the correct order required by the two-port calibration. `UnknownThru` assumes that thru should be listed last and order of the other standards doesn't matter.

[9]:
```
ideals_01 = ideals[2:] + [ideals[0]]
ideals_02 = ideals[2:] + [ideals[1]]
meas_01 = measured[2:] + [measured[0]]
meas_02 = measured[2:] + [measured[1]]
```

(continues on next page)

(continued from previous page)

```
cal_dict = {(0, 1): {'method': UnknownThru, 'ideals': ideals_01,
                     'measured': meas_01, 'switch_terms':[gammas[1], gammas[0]]},
            (0, 2): {'method': UnknownThru, 'ideals': ideals_02,
                     'measured': meas_02, 'switch_terms':[gammas[2], gammas[0]]}}

cal2 = MultiportCal(cal_dict)

dut_cal2 = cal2.apply_cal(dut_meas)
dut_cal2.plot_s_db()
```



[ ]:

2.3.5 Multiline TRL

Multiline TRL is a two-port VNA calibration utilizing at least two transmission lines with different physical lengths and at least one reflective standard that is identical on both ports. The electrical parameters of the lines don't need to be known, but the transmission lines should have identical construction (same propagation constant and characteristic impedance). The reflect standard reflection coefficient doesn't need to be known exactly, phase needs to be known with 90 degree accuracy.

If the measured phase differences of the lines is a multiple of 180 degrees the calibration is singular. The calibration accuracy is worse the closer the line measurement phases are to the singularities, the best accuracy is obtained in the two lines case when the phase difference is 90 degrees. Multiple lines can be used to extend the frequency range where the calibration is accurate.

skrf has two implementations: the NIST-style multiline calibration (`NISTMultilineTRL`) and the TUG multiline cal-

ibration (TUGMultilineTRL). They use the same standards and give the same answer (most of the time), they differ in how the measurement noise impact the result, so this example runs the two side by side.

All data used in this example is generated with skrf, and the code for it is given at the start. The notebook then demonstrates how to use both methods and compares different combinations of lines to show how the calibration accuracy improves with the number of lines. Lastly, a Monte Carlo analysis compares the two methods.

Setup

[1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

import skrf
from skrf.calibration import NISTMultilineTRL, TUGMultilineTRL, terminate
from skrf.media import CPW, Coaxial
from skrf.network import two_port_reflect

skrf.stylely()
```

Simulation to generate the input data

Here is how we make the data used in the rest of the example.

Create frequency and Media

[2]:
```
freq = skrf.Frequency(1, 125, 249, 'GHz')

# CPW media used for DUT and the calibration standards
cpw = CPW(freq, w=49.1e-6, s=25.5e-6, ep_r=9.9,
                    t=4.9e-6, rho=1/4.11e7)
print(cpw.z0[0])

# 1.0 mm coaxial media for calibration error boxes
coax1mm = Coaxial(freq, z0_port=50, Dint=0.434e-3, Dout=1.0e-3, sigma=1e8)
print(coax1mm.z0[0])

f_ghz = cpw.frequency.f*1e-9
```

```
(45.65554776395502+0j)
(50.20564772916889-0.15748463048337036j)
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/media/
↪cpw.py:240: RuntimeWarning: Conductor loss calculation invalid for lineheight t (4.9e-
↪06)  < 3 * skin depth (2.4825569540602707e-06)
  self.alpha_conductor, self.alpha_dielectric = self.analyse_loss(
```

Make realistic looking error networks.

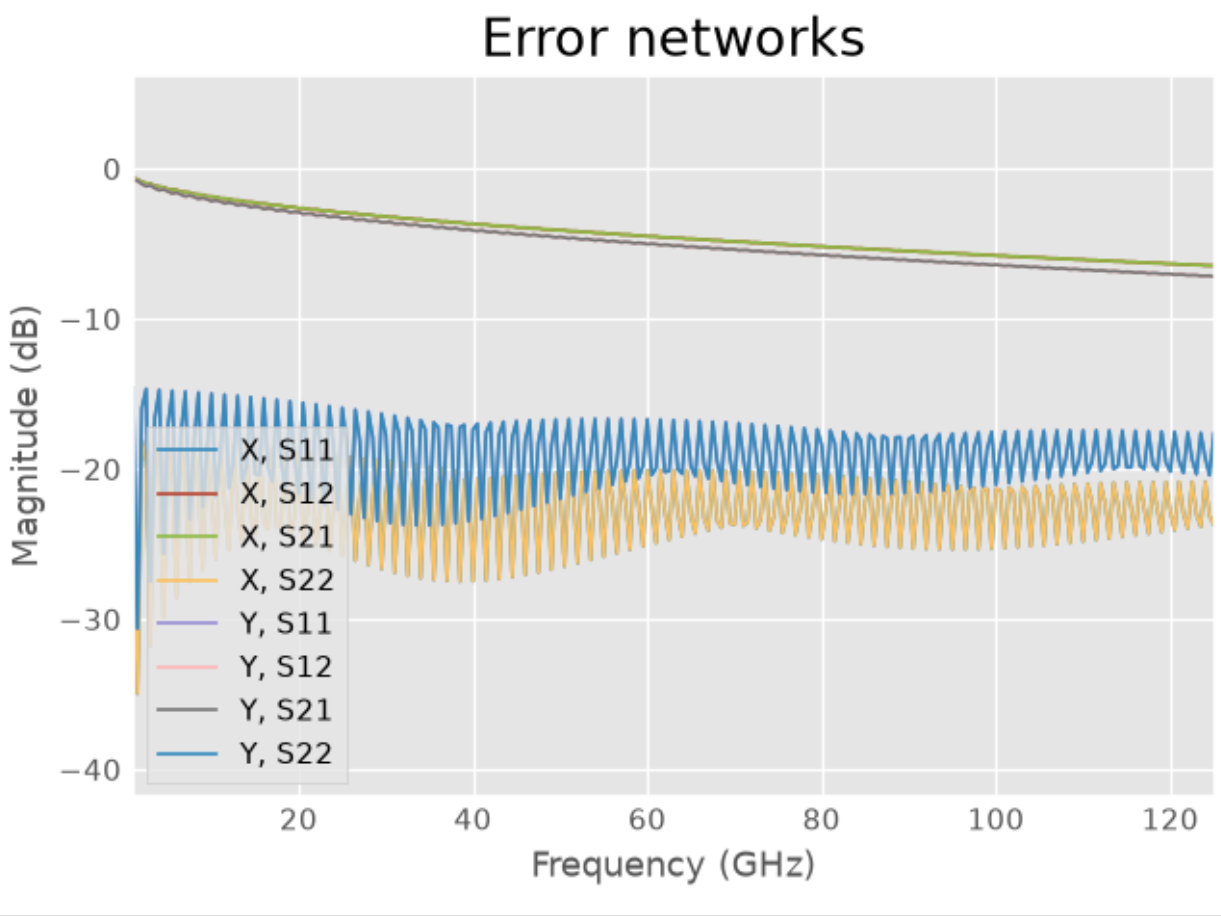
Propagation constant determination is iterative and doesn't work as well when the error networks are randomly generated

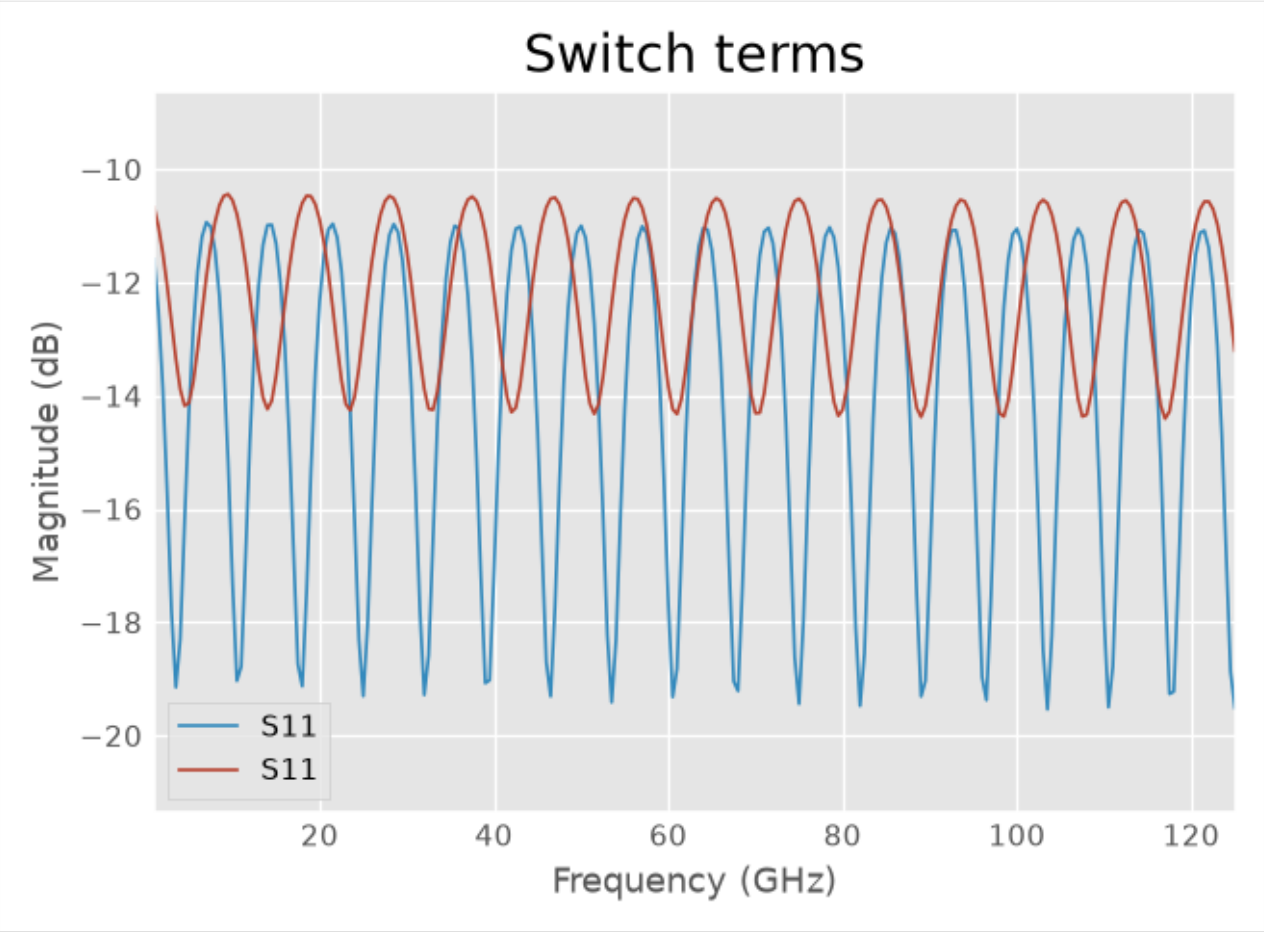
[3]:
```
X = coax1mm.line(1, 'm', z0=58, name='X')
Y = coax1mm.line(1.1, 'm', z0=40, name='Y')
```

(continues on next page)

(continued from previous page)

```
plt.figure()
plt.title('Error networks')
X.plot_s_db()
Y.plot_s_db()

# Realistic looking switch terms
gamma_f = coax1mm.delay_load(0.2, 21e-3, 'm', z0=60)
gamma_r = coax1mm.delay_load(0.25, 16e-3, 'm', z0=56)

plt.figure()
plt.title('Switch terms')
gamma_f.plot_s_db()
gamma_r.plot_s_db()
```





Generate Fictitious measurements

[4]:
```
# Lengths of the lines used in the calibration, units are in meters
line_len = [300e-6, 1200e-6, 2950e-6, 3550e-6, 5050e-6]
lines = [cpw.line(x, 'm') for x in line_len]

# Attenuator with mismatched feed lines
dut_feed = cpw.line(1.5e-3, 'm', z0=60)
dut = dut_feed**cpw.attenuator(-10)**dut_feed

res_50ohm = cpw.resistor(50) ** cpw.short(nports=2) ** cpw.resistor(50)

# Through and non-ideal short, 75 um behind the middle of the thru.
# Real reflection coefficient is solved during the calibration

short = cpw.delay_short(75e-6, 'm')

actuals = [
    cpw.thru(),
    two_port_reflect(short, short),
    ]
```

(continues on next page)

(continued from previous page)

```
actuals.extend(lines)

# Measured
measured = [X**k**Y for k in actuals]

# Switch termination
measured = [terminate(m, gamma_f, gamma_r) for m in measured]

# Add little noise to the measurements
for m in measured:
    m.add_noise_polar(0.001, 0.1)

names = ['thru', 'reflect', 'line300um', 'line1200um', 'line2950um',
         'line3550um', 'line5050um']
for k,name in enumerate(names):
    measured[k].name=name


# Noiseless DUT so that all the noise will be from the calibration
dut_meas = terminate(X**dut**Y, gamma_f, gamma_r)
dut_meas.name = 'DUT'

res_50ohm_meas = terminate(X**res_50ohm**Y, gamma_f, gamma_r)
res_50ohm_meas.name = 'res_50ohm'
```

Simple Multiline TRL

The two classes take the same information, but they group it differently. `NISTMultilineTRL` takes one flat list of `measured` standards, thru first, then the reflects, then the lines. `TUGMultilineTRL` keeps the lines and the reflects in separate arguments, and the first line is taken as the thru.

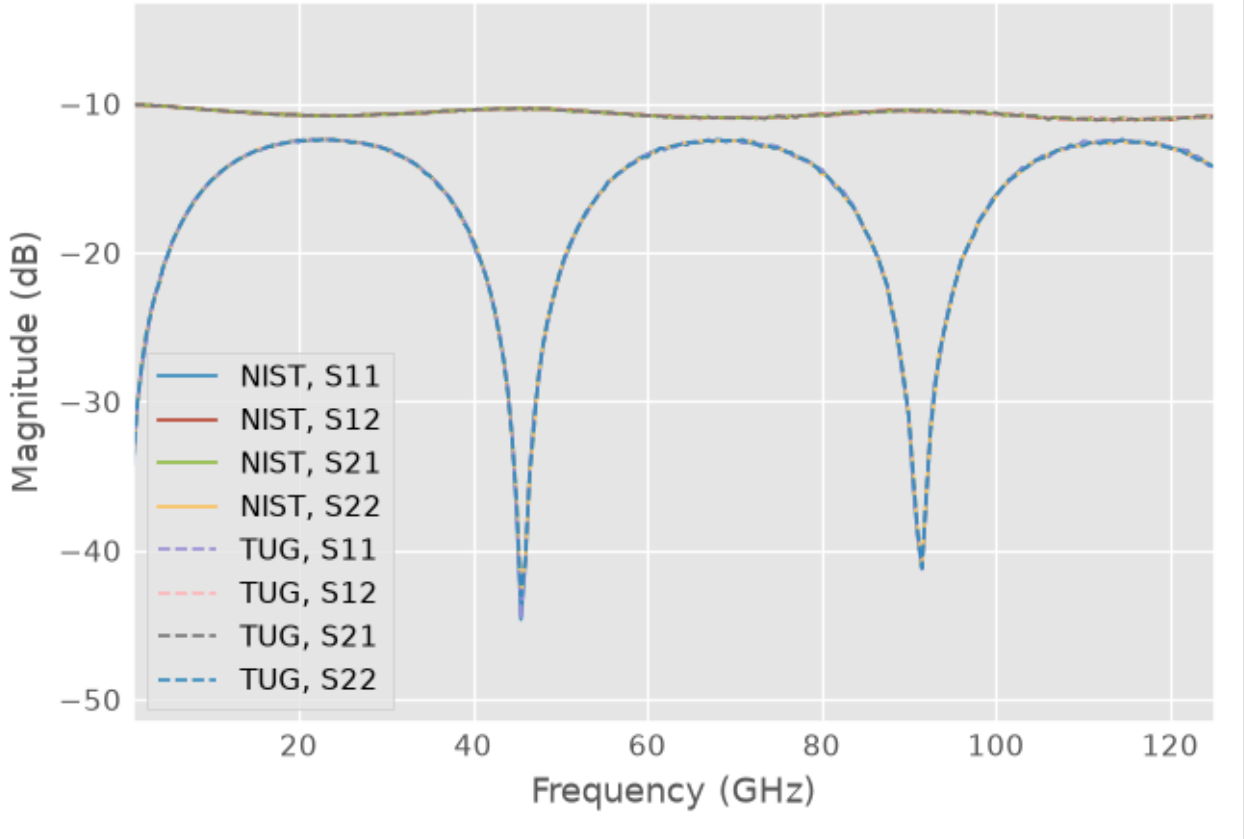
[5]:
```
# define the line lengths in meters (including thru)
l = [0, 300e-6, 1200e-6, 2950e-6, 3550e-6, 5050e-6]

# Do the calibration
cal = NISTMultilineTRL(
    measured = measured,  # Measured standards
    Grefls = [-1], # Reflection coefficient of the reflect, -1 for short
    l = l,         # Lengths of the lines
    refl_offset = 75e-6, # Reflect offset from the reference plane
    er_est = 5,    # Estimate of transmission line effective permittivity
    switch_terms = (gamma_f, gamma_r) # Switch terms
    )

# The same calibration with the TUG method
cal_tug = TUGMultilineTRL(
    line_meas = [measured[0]] + measured[2:], # Measured lines, thru first
    line_lengths = l,      # Lengths of the lines
    reflect_meas = [measured[1]], # Measured reflect
    reflect_est = [-1],    # Reflection coefficient of the reflect, -1 for short
    reflect_offset = 75e-6, # Reflect offset from the reference plane
    er_est = 5,            # Estimate of transmission line effective permittivity
```

(continues on next page)

(continued from previous page)

```
    switch_terms = (gamma_f, gamma_r) # Switch terms
    )

# Correct the DUT using the above calibrations
corrected = cal.apply_cal(dut_meas)
corrected.name = 'NIST'
corrected.plot_s_db()

corrected_tug = cal_tug.apply_cal(dut_meas)
corrected_tug.name = 'TUG'
corrected_tug.plot_s_db(linestyle='--')

print(f'Largest difference between the two: {abs(corrected.s - corrected_tug.s).max():.
↪2e}')
```

```
Largest difference between the two: 1.50e-04
```



The two methods sit on top of each other.

Compare calibrations with different combinations of lines

Here we loop through different line combinations to demonstrate the difference in the calibration accuracy.

[6]:
```
# Run multiline TRL calibration with different combinations of lines

cals = []
```

(continues on next page)

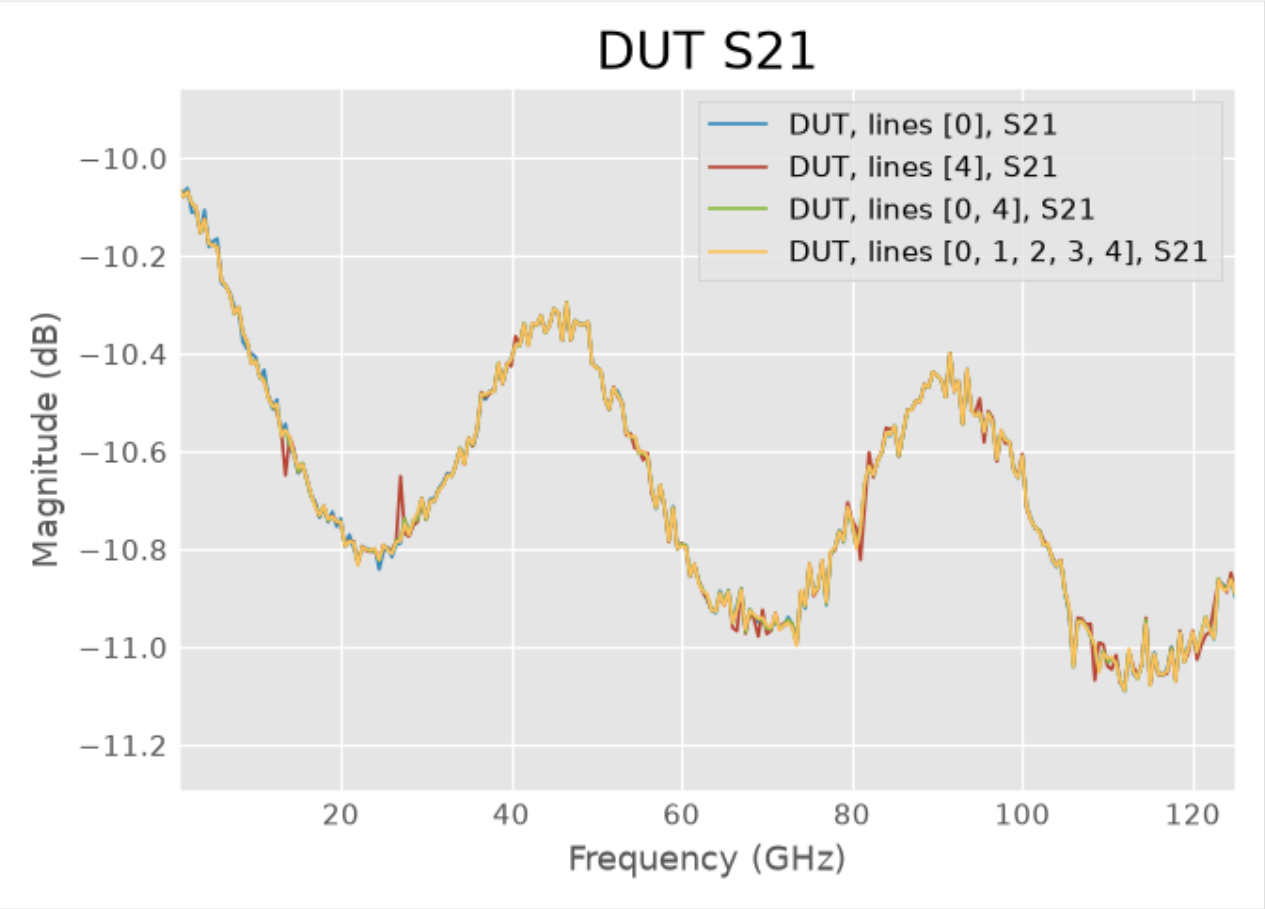
(continued from previous page)

```
cals_tug = []
duts = []

line_combinations = [[0], [4], [0,4], [0,1,2,3,4]]

for used_lines in line_combinations:

    # Thru and reflect, plus the lines being used
    m = measured[:2] + [measured[2:][i] for i in used_lines]

    # Add thru length to list of line lengths
    lc = [l[0]] + [l[1:][i] for i in used_lines]

    # Do the calibration
    cal = NISTMultilineTRL(
        measured = m,  # Measured standards
        Grefls = [-1], # Reflection coefficient of the reflect, -1 for short
        l = lc,        # Lengths of the lines
        refl_offset = 75e-6, # Reflect offset from the reference plane
        er_est = 5,    # Estimate of transmission line effective permittivity
        switch_terms = (gamma_f, gamma_r) # Switch terms
        )

    # The same lines with the TUG method
    cal_t = TUGMultilineTRL(
        line_meas = [m[0]] + m[2:],
        line_lengths = lc,
        reflect_meas = [m[1]],
        reflect_est = [-1],
        reflect_offset = 75e-6,
        er_est = 5,
        switch_terms = (gamma_f, gamma_r)
        )

    # Correct the DUT using the above calibration
    corrected = cal.apply_cal(dut_meas)
    corrected.name = f'DUT, lines {used_lines}'

    duts.append(corrected)
    cals.append(cal)
    cals_tug.append(cal_t)
```

Transmission of the corrected DUT

Plot the corrected DUT calibrated with different combination of calibration lines.

[7]:
```
plt.figure()
plt.title('DUT S21')
for dut in duts:
    dut.plot_s_db(m=1, n=0)
```

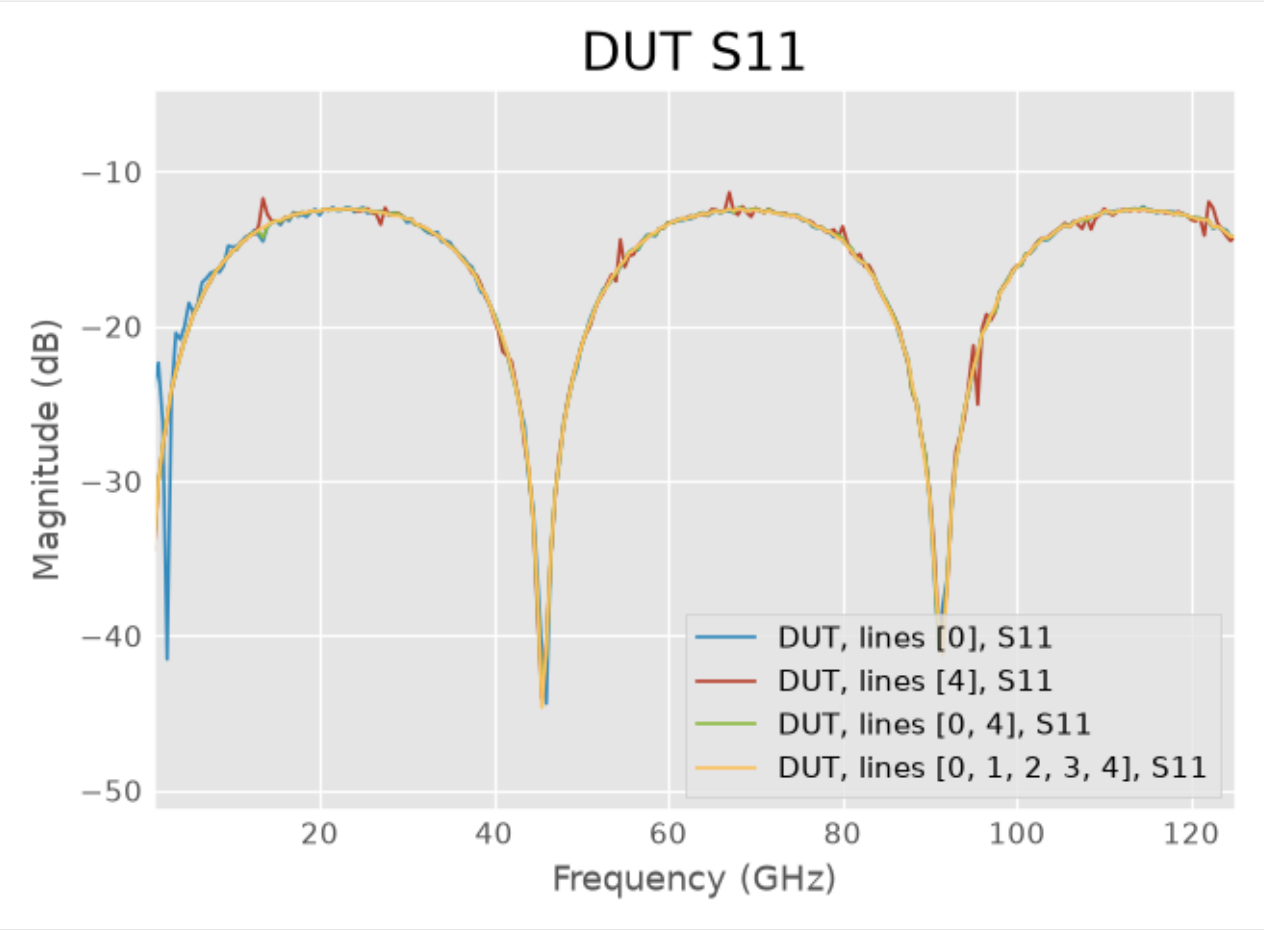


S11 of corrected DUT with different amount of calibration lines

S11 shows bigger changes.

- With one short line low frequencies are very noisy
- With only the long line the calibration is very inaccurate at frequencies where the phase difference of the thru and line is close to a multiple of 180 degrees
- With both lines calibration accuracy is good everywhere

[8]:
```
plt.figure()
plt.title('DUT S11')
for dut in duts:
    dut.plot_s_db(m=0, n=0)
```



Normalized standard deviation of different calibrations

Normalized standard deviation can be used to measure the accuracy of the calibration. Lower number means calibration is less sensitive to the measurement noise.

- TRL calibration with one 90 degrees long line has normalized standard deviation of 1.
- TRL calibration with one 180 degree long lossless line is singular and has infinite normalized standard deviation.
- With multiple lines normalized standard deviation less than one is possible.

Note that the nstd is normalized such that it doesn't consider the actual measurement noise. It's calculated only from the solved propagation constant and line lengths. The threshold of how large it can be depends on the DUT being measured, measurement noise and the required accuracy of the measurement. If there are large spikes, such as are visible in the long line case below, that's a sign that the calibration is very close to singular at that frequency and the measurement accuracy is going to be poor.

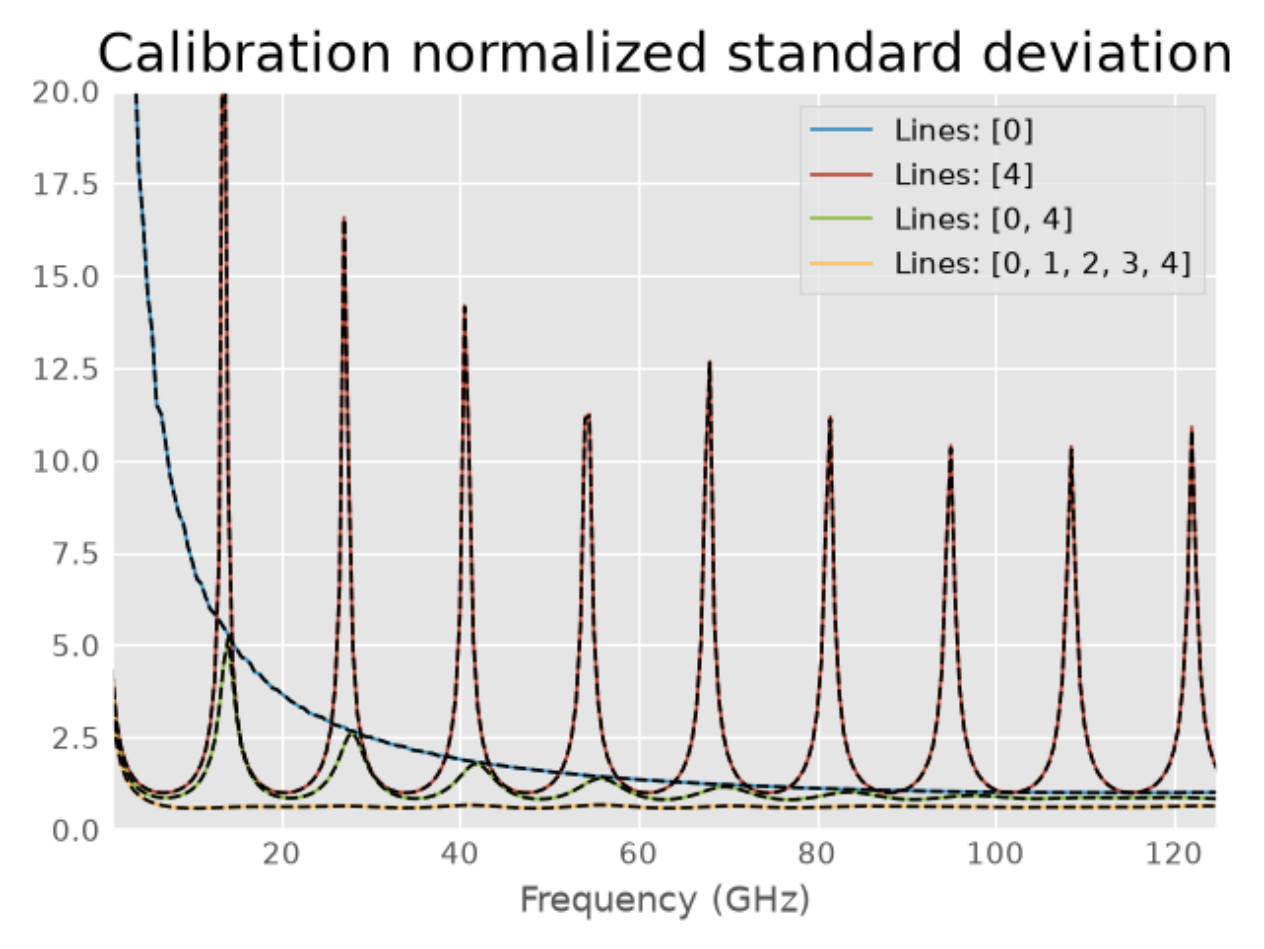
The TUG method reports the same thing through the eigenvalue λ of its weighted eigenvalue problem, where a small eigenvalue means a sensitive calibration. For N lines (thru included) the two are related by $\mathrm{nstd} \approx \sqrt{2N/\lambda}$. The dashed lines below are the TUG eigenvalue converted this way.

[9]:
```
f_ghz = dut.frequency.f_scaled

plt.figure()
plt.title('Calibration normalized standard deviation')
```

(continues on next page)

(continued from previous page)

```
for e, cal in enumerate(cals):
    plt.plot(f_ghz, cal.nstd, label=f'Lines: {line_combinations[e]}')

# The TUG eigenvalue converted to the same scale
for e, cal_t in enumerate(cals_tug):
    N = len(line_combinations[e]) + 1
    plt.plot(f_ghz, np.sqrt(2*N/cal_t.lambd), 'k--', lw=1)

plt.ylim([0,20])
plt.legend(loc='upper right')
dut.frequency.labelXAxis()
```



Calculate effective complex relative permittivity of transmission lines used in the calibration

Effective complex relative permittivity $\epsilon_{r,eff}$ of a transmission line is related to the propagation constant γ as:

$\gamma = \frac{2\pi f}{c}\sqrt{\epsilon_{r,eff}}$, where c equals the speed of light and f is frequency.

In general it's a complex value with the imaginary part indicating losses.

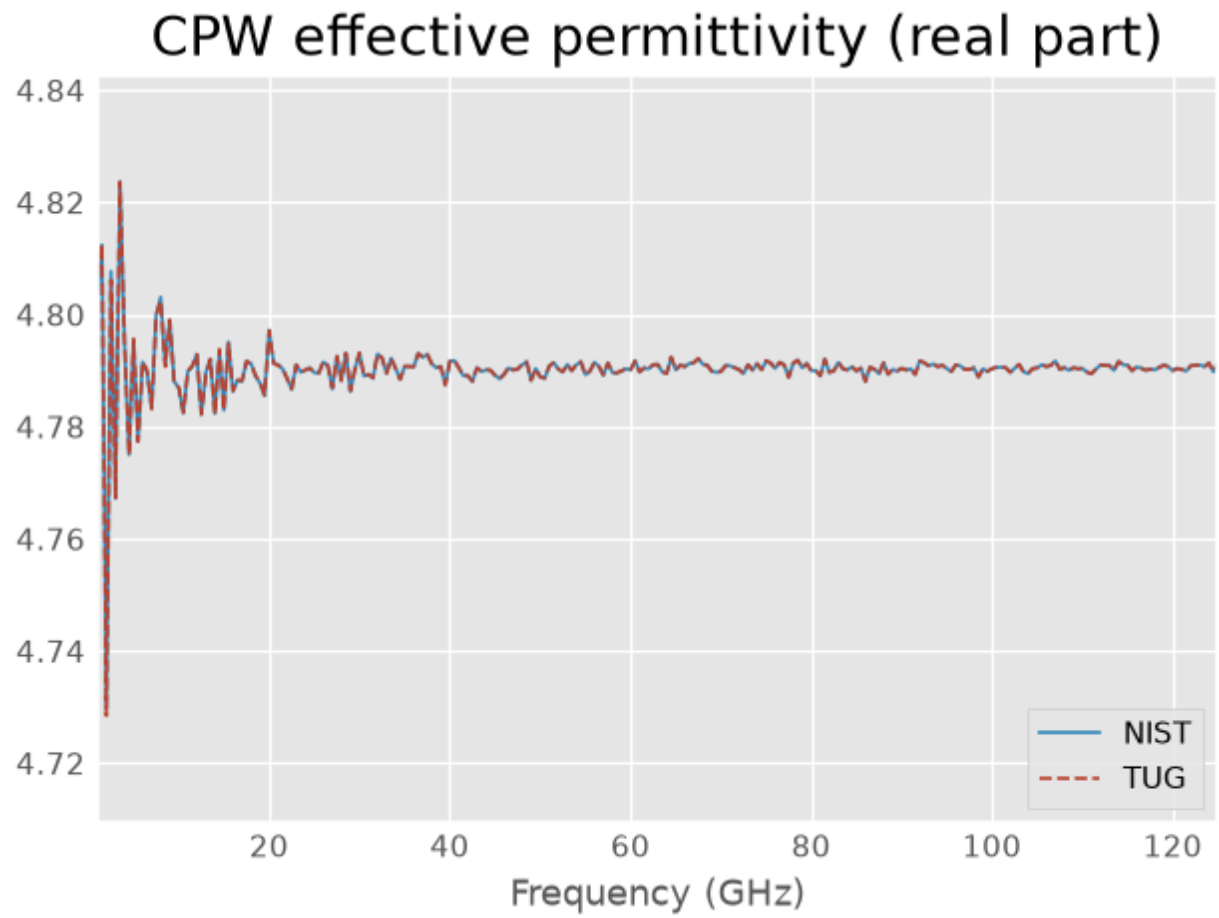
[10]:
```
# Get the cal with all the lines
cal = cals[-1]
cal_tug = cals_tug[-1]
```

(continues on next page)

(continued from previous page)

```
plt.figure()
plt.title('CPW effective permittivity (real part)')
plt.plot(f_ghz, cal.er_eff.real, label='NIST')
plt.plot(f_ghz, cal_tug.er_eff.real, '--', label='TUG')
plt.xlabel('Frequency (GHz)')
plt.legend(loc='lower right')
```

[10]:
```
<matplotlib.legend.Legend at 0x737e3e44bb90>
```

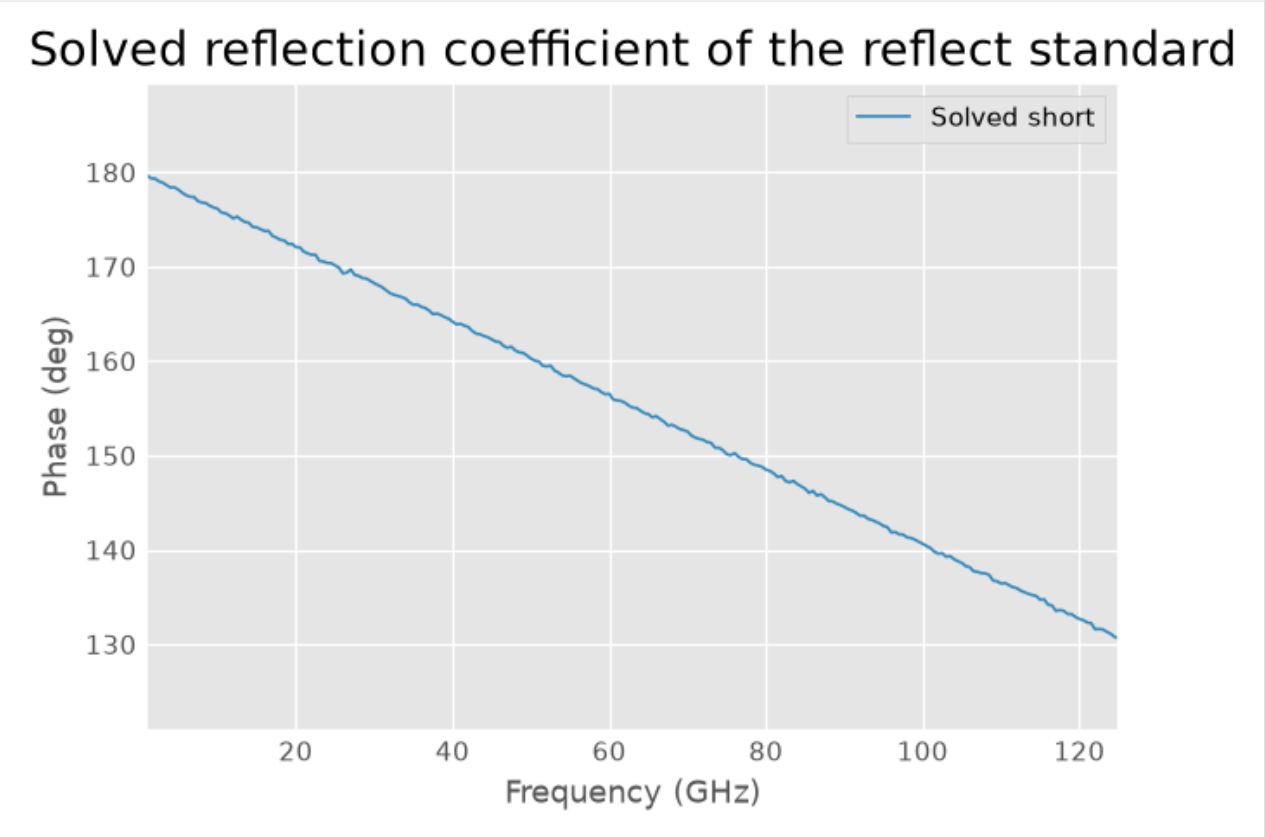


TRL calibration accuracy is the best when line length difference is 90 degrees. Solved propagation constant and effective permittivity however are more accurate the bigger the line length difference is. At low frequencies the estimate is noisier due to the line phase difference being small.

Plot the phase of the solved reflection coefficient

Applying the calibration to the measured reflect standard we can get the calibrated S-parameters of the unknown reflect.

[11]:
```
plt.figure()
plt.title('Solved reflection coefficient of the reflect standard')
cal.apply_cal(measured[1]).plot_s_deg(n=0, m=0, label='Solved short')
```

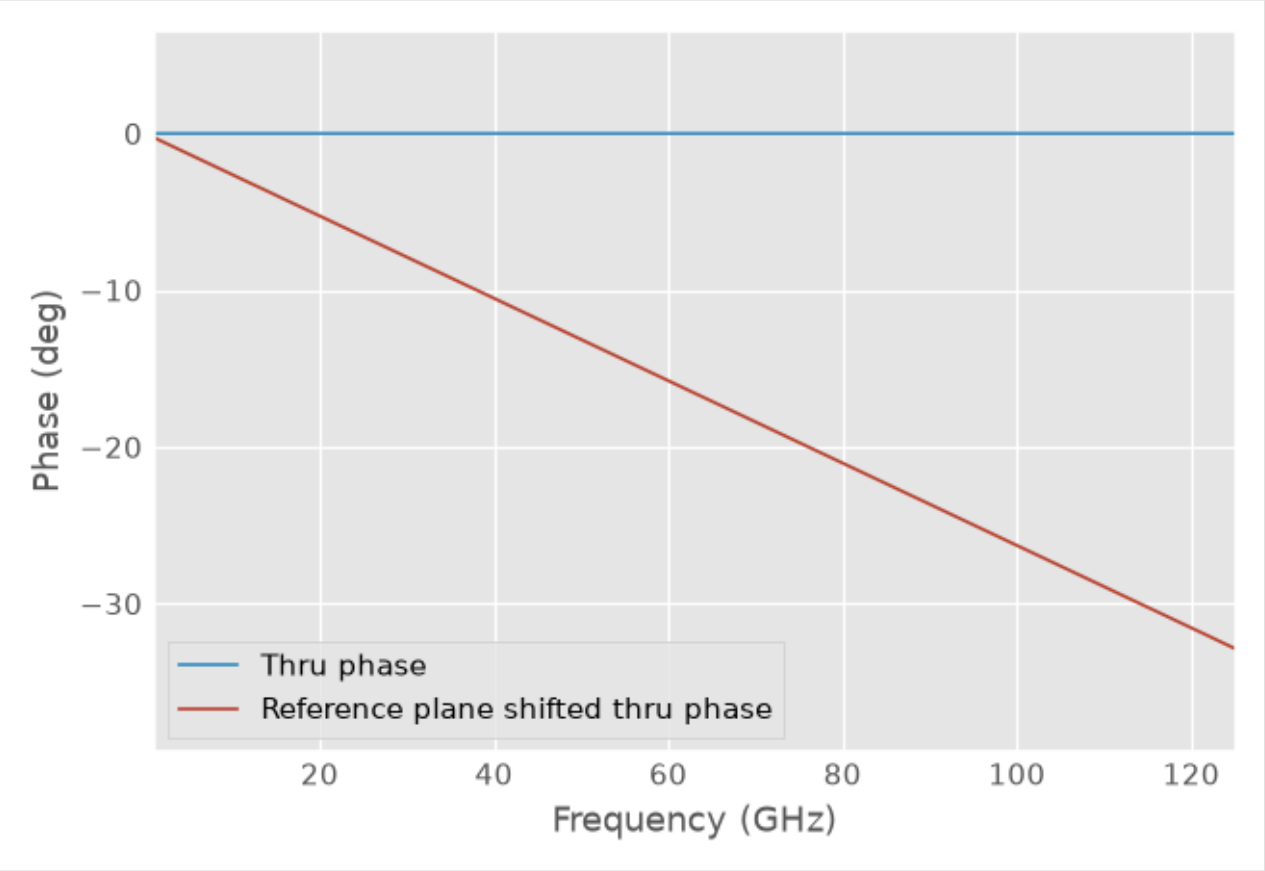


Reference plane shift

Because propagation constant of the media is solved during the calibration it's possible to shift the reference plane by a specified distance.

The reference plane shift can be specified with `ref_plane` argument. The shift should be specified in meters, negative lengths is towards the VNA. By default the same shift is applied to both ports. Unequal shift on the two ports is supported by passing a two element list. Both classes take the same argument.

[12]:
```
cal_shift = NISTMultilineTRL(
    measured = measured,  # Measured standards
    Grefls = [-1], # Reflection coefficient of the reflect, -1 for short
    l = l,         # Lengths of the lines
    refl_offset = 75e-6, # Reflect offset from the reference plane
    er_est = 5,    # Estimate of transmission line effective permittivity
    switch_terms = (gamma_f, gamma_r), # Switch terms
    # Shift reference planes towards VNA by this amount (in m) on both ports
    ref_plane = -50e-6
    )

# Correct the DUT using the above calibration
corrected_thru = cal.apply_cal(measured[0])
corrected_thru_shifted = cal_shift.apply_cal(measured[0])

corrected_thru.plot_s_deg(m=1, n=0, label='Thru phase')
corrected_thru_shifted.plot_s_deg(m=1, n=0, label='Reference plane shifted thru phase')
```



Calibration reference impedance renormalization

The reference impedance of the calibration is by default the transmission line characteristic impedance. If we know the actual characteristic impedance of the lines we can give it to the calibration routine with the `z0_line` argument to renormalize the measured S-parameters to a fixed reference `z0_ref`.

If the conductance per unit length (G) is much lower than the capacitive reactance per unit length ($j\omega C_0$), the characteristic impedance of the transmission line can be written in terms of the propagation constant γ and capacitance per unit length C_0:

$Z_0 = \gamma/(j2\pi f C_0)$

If C_0 is known it can be given to the calibration routine with `c0` parameter to renormalize the calibration reference impedance to `z0_ref` (defaults to 50 ohms) assuming G = 0. If the line is lossy the characteristic impedance is complex valued and giving a single `c0` instead of a fixed `z0_line` is usually more accurate.

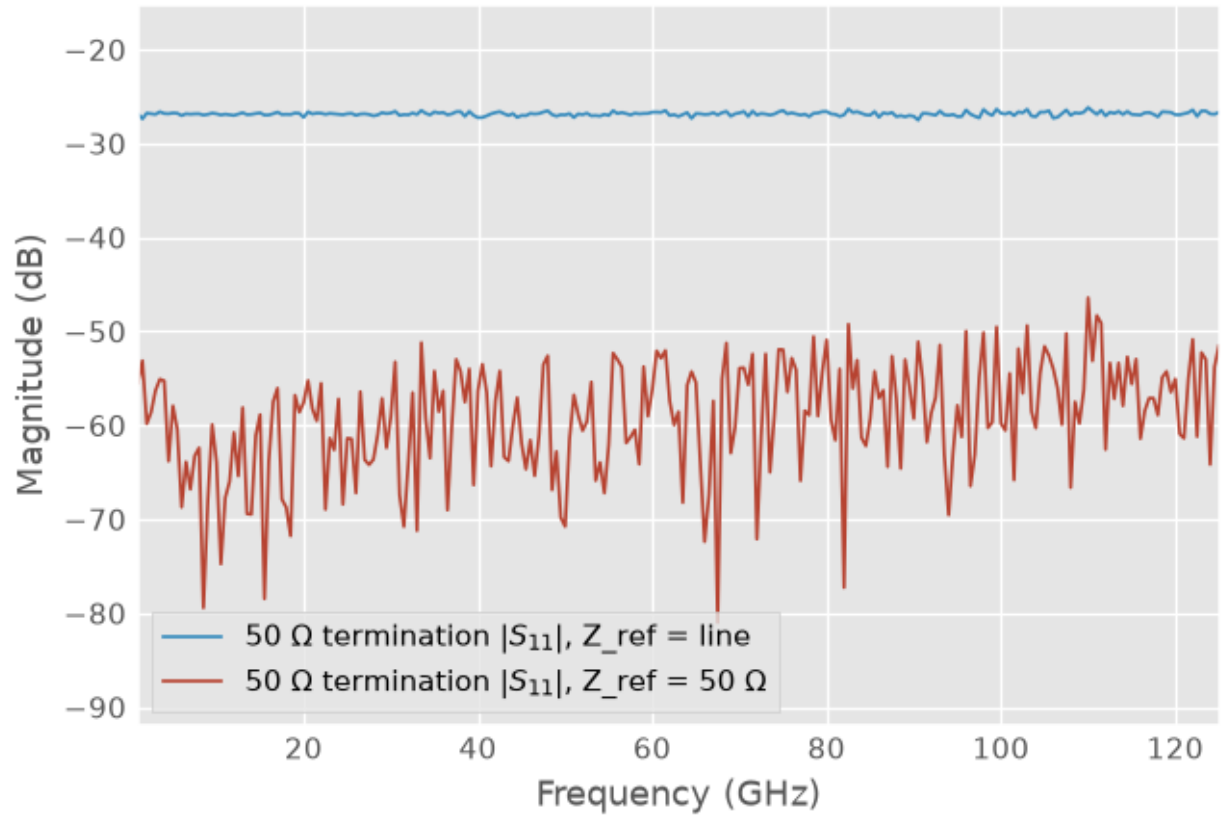
In this case we know that the line characteristic impedance is actually 45.7 ohms. To renormalize the calibration from 45.7 ohms to 50 ohms we need to give `z0_line=45.7` argument to the calibration routine. With `TUGMultilineTRL` the same is done afterwards with `cal.renormalize(45.7, 50)`.

[13]:
```
cal_ref = NISTMultilineTRL(
    measured = measured,  # Measured standards
    Grefls = [-1], # Reflection coefficient of the reflect, -1 for short
    l = l,         # Lengths of the lines
    refl_offset = 75e-6, # Reflect offset from the reference plane
    er_est = 5,    # Estimate of transmission line effective permittivity
```

(continues on next page)

(continued from previous page)

```
    switch_terms = (gamma_f, gamma_r), # Switch terms
    z0_line = 45.7, # Line actual characteristic impedance
    z0_ref = 50 # Calibration reference impedance
    )

cal.apply_cal(res_50ohm_meas).s11.plot_s_db(label=r'50 $\Omega$ termination |$S_{11}$|,␣
↪Z_ref = line')
cal_ref.apply_cal(res_50ohm_meas).s11.plot_s_db(label=r'50 $\Omega$ termination |$S_{11}
↪$|, Z_ref = 50 $\Omega$')
```



After renormalization the 50 ohm termination measurement shows good matching. It's not perfectly matched due to the noise in the measurements.

Calibration without reflect measurement

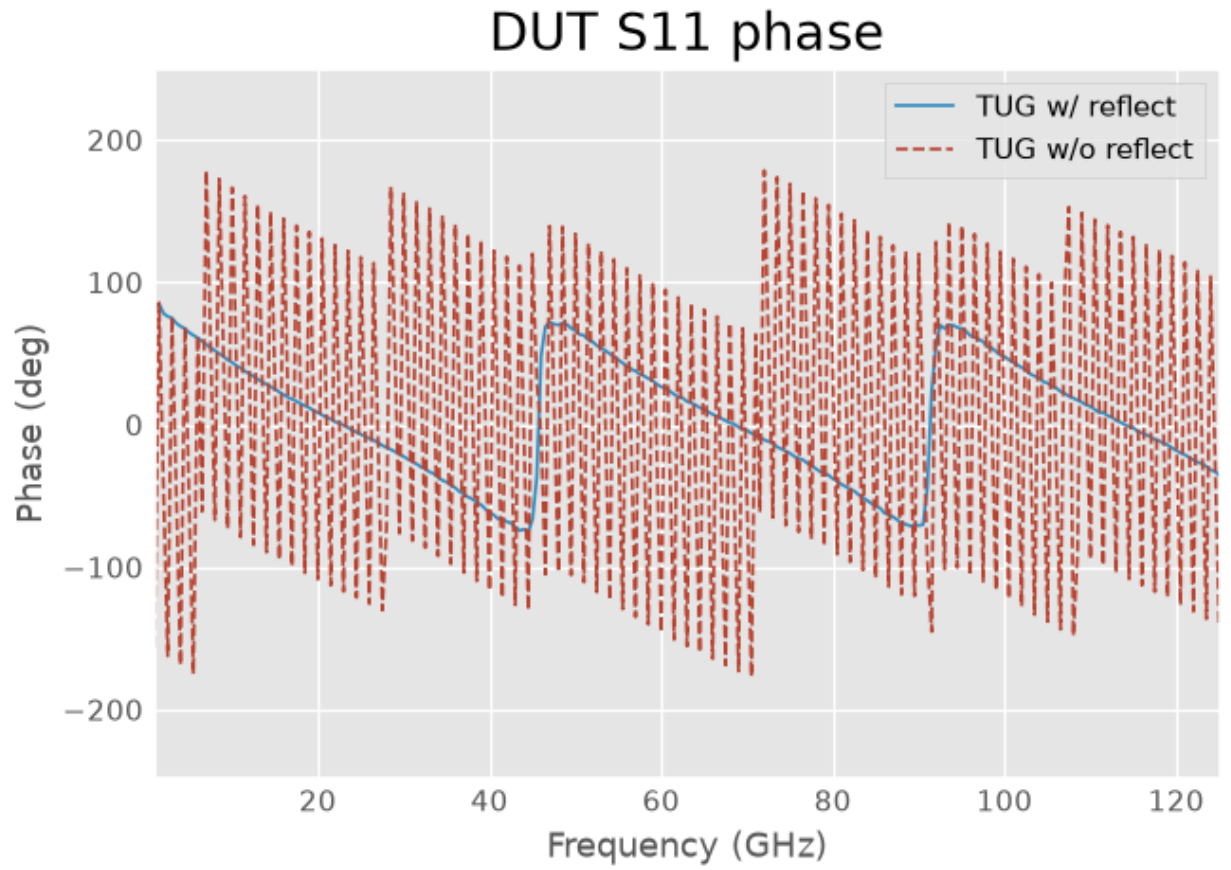
One nice feature of `TUGMultilineTRL` is that you can use only line standards if you only care about the propagation constant and S21 and S12 of the calibrated DUT. However, to obtain correct S11 and S22 values of the calibrated DUT, you also need to provide measurement of the reflect standard. This can be seen in the example below, where the phase of S11 does not overlap when using and not using the reflect standard. Note, if you do include the reflect standard, `TUGMultilineTRL` method also offers you the capability to provide multiple reflect measurements.

[14]:
```
# TUG multiline without reflect measurement
cal_no_reflect = TUGMultilineTRL(
        line_meas = [measured[0]] + measured[2:],
        line_lengths = l,
```

(continues on next page)

(continued from previous page)

```
        er_est = 5,
        switch_terms = (gamma_f, gamma_r)
        )

plt.figure()
plt.title('DUT S11 phase')
cal_tug.apply_cal(dut_meas).plot_s_deg(m=0, n=0, label='TUG w/ reflect')
cal_no_reflect.apply_cal(dut_meas).plot_s_deg(m=0, n=0, linestyle='--', label='TUG w/o␣
↪reflect')
```



Monte Carlo analysis - NIST vs TUG

Using MC analysis, we generate synthetic data of the standards with random noise and repeat the process multiple times, each time with a new noise sample. At each trial, we also compute the calibrated DUT, which we use to calculate the mean absolute error (MAE) of the calibration method. The general definition of MAE is as follows:

$$\mathrm{MAE} = \frac{1}{M}\sum_{i=1}^{M}|x_i - x_{\mathrm{true}}|$$

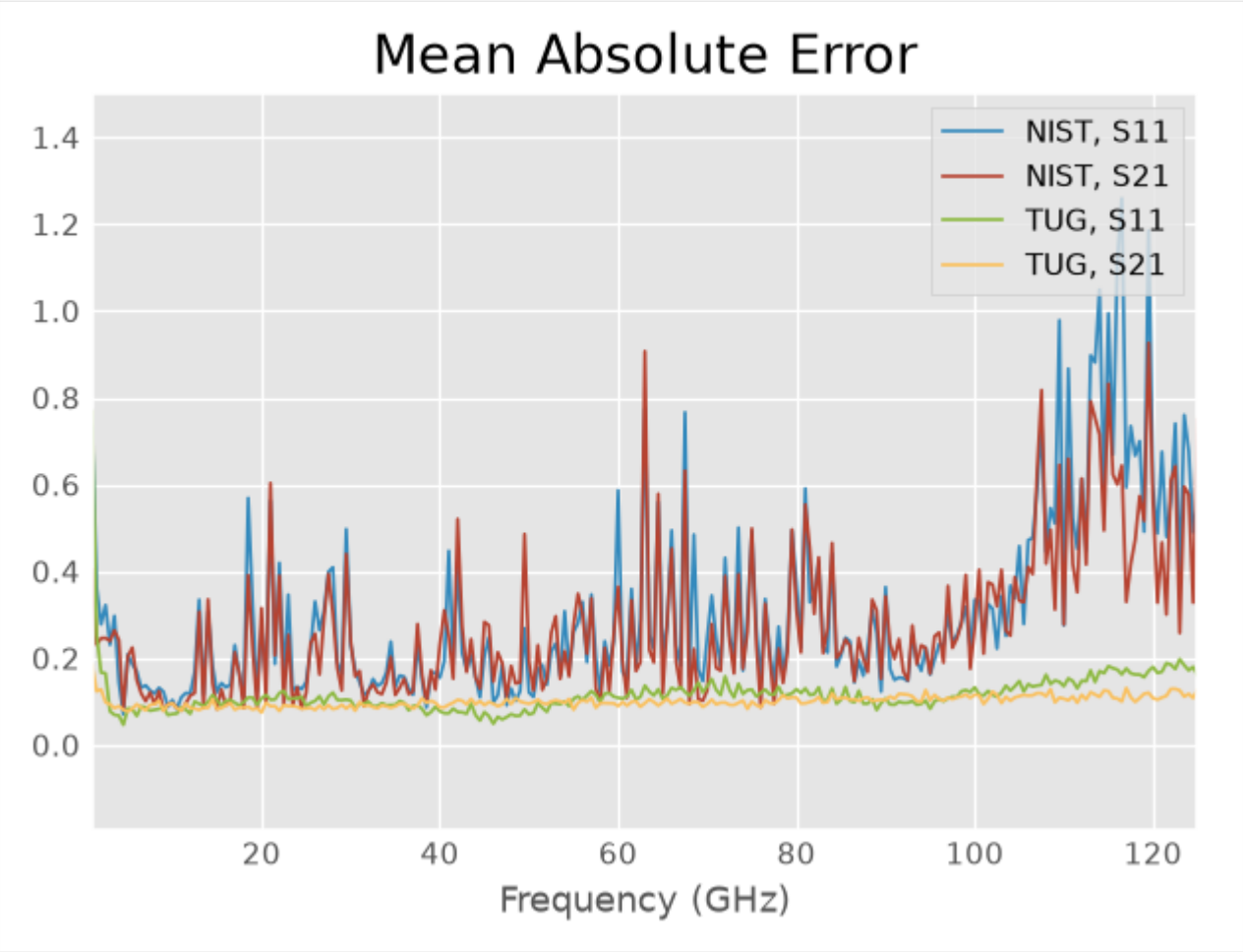
where x_i is the i-th MC trial and x_{true} is the true value. As the reference true value we use the DUT corrected with the measurements generated above, before the Monte Carlo noise is added. This runs both calibrations 100 times over the full frequency grid, so expect it to take about a minute.

[15]:
```
# Monte Carlo Analysis
print('MC starts: ')
M = 100 # number of trials
mag_std = 0.05 # std of magnitude (linear)
phase_std = 20 # std of phase (degrees)

# Reference value, both methods agree on the unmodified measurements
dut_ideal = cal_tug.apply_cal(dut_meas)

mae_nist = 0
mae_tug  = 0

for inx in range(M):
    # add noise (make sure to copy and not override the original data)
    measured_n = [NW.copy() for NW in measured]
    for NW in measured_n:
        NW.add_noise_polar(mag_std, phase_std)

    # NIST multiline TRL
    cal_n = NISTMultilineTRL(measured=measured_n, Grefls=[-1], l=l,
                refl_offset=75e-6, er_est=5, switch_terms=(gamma_f, gamma_r))
    mae_nist = mae_nist + abs(cal_n.apply_cal(dut_meas).s - dut_ideal.s)/M

    # TUG multiline TRL
    cal_t = TUGMultilineTRL(line_meas=[measured_n[0]]+measured_n[2:], line_lengths=l,
                reflect_meas=[measured_n[1]], reflect_est=[-1], reflect_offset=75e-6,
                er_est=5, switch_terms=(gamma_f, gamma_r))
    mae_tug = mae_tug + abs(cal_t.apply_cal(dut_meas).s - dut_ideal.s)/M

    print(f'MC Index: {inx+1}/{M} done.', end='\r', flush=True)
```

```
MC starts:
MC Index: 100/100 done.
```

Finally, we compute and plot the MAE. The results show that TUG method on average has a lower error compared to the NIST method.

[16]:
```
plt.figure()
plt.title('Mean Absolute Error')
plt.plot(f_ghz, mae_nist[:,0,0], label='NIST, S11')
plt.plot(f_ghz, mae_nist[:,1,0], label='NIST, S21')
plt.plot(f_ghz, mae_tug[:,0,0], label='TUG, S11')
plt.plot(f_ghz, mae_tug[:,1,0], label='TUG, S21')
plt.legend(loc='upper right')
dut_meas.frequency.labelXAxis()
```



2.3.6 Measuring a 4-Port With The 1.5-Port NanoVNA V2

The NanoVNA V2 is a 1.5-port VNA, meaning it can only measure the forward reflection and transmission coefficients S_{11} and S_{21} with its two ports. This is known as a three-receiver VNA architecture, still, fully-calibrated two-port measurements are possible despite this limitation. We recommend the readers to read *Calibration With Three Receivers* first to familiarize themselves with the basic background.

In short, to measure the reverse coefficients S_{12} and S_{22} of a 2-port, it needs to be measured a second time with the port orientation physically flipped (or fake-flipped), as explained in *this example*. The issue gets worse if the device under test (DUT) has even more ports, as described in *this example*.

In this example, a 4-port SMA power splitter is measured with a NanoVNA V2 using the Matched Port technique. It involves measuring the 4-port in all port combinations (12 measurements) with the unused ports terminated with a 50-Ohm match. For the VNA calibration, four more measurements with the SMA calibration standards are required (SHORT, OPEN, MATCH, THROUGH). The manufacturer of the DUT provides measured S-parameters on their website, which will later be used for comparison.

Data Acquisition

To use scikit-rf's 2-port calibration classes, such as `TwoPortOnePath`, the individual measurement results of the DUT and of the calibration standards need to be provided as 2-port networks holding the data as S_{11} and S_{21}. The following code snippet can be used to configure the NanoVNA, and acquire and save the data:

```
import skrf
from skrf.vi.vna.nanovna import NanoVNAv2

# connect to NanoVNA on /dev/ttyACM0 (Linux)
nanovna = NanoVNAv2('ASRL/dev/ttyACM0::INSTR')

# for Windows users: ASRL1 for COM1
# nanovna = NanoVNAv2('ASRL1::INSTR')

# configure frequency sweep (for example 1 MHz to 4.4 GHz in 1 MHz steps)
f_start = 1e6
f_stop = 4.4e9
f_step = 1e6
num = int(1 + (f_stop - f_start) / f_step)
nanovna.frequency = skrf.Frequency(start=f_start, stop=f_stop, npoints=num)

# measure all 12 combinations of the 4-port
n_ports = 4
for i_src in range(n_ports):
    for i_sink in range(n_ports):
        if i_sink != i_src:
            input('Connect vna_p1 -> dut_p{}, vna_p2 -> dut_p{} and press ENTER:'.
↪format(i_src + 1, i_sink + 1))
            nw_raw = nanovna.get_snp_network(ports=(1, 2))
            nw_raw.write_touchstone('./data_MiniCircuits_splitter/dut_raw_{}{}'.format(i_
↪sink + 1, i_src + 1))
```

The calibration standards should be measured with the same repeated calls of `get_snp_network(ports=(1, 2))` and `write_touchstone()`.

Offline Calibration Using `TwoPortOnePath`

The measured data transferred from the NanoVNA via USB is always raw (uncalibrated), regardless of any calibration performed on the NanoVNA itself. This requires the correction of the data using an offline calibration. With the measurements of the calibration standards stored as individual 2-ports, a TwoPortOnePath calibration is easily created using scikit-rf. In this example, the impedances and phase delays of the measured SHORT, OPEN, MATCH, and THROUGH are assumed to be ideal, i.e. without any loss or offset:

[1]:
```
import skrf
from skrf.calibration import TwoPortOnePath

# load networks of the raw calibration standard measurements
short_raw = skrf.Network('./data_MiniCircuits_splitter/cal_short_raw.s2p')
open_raw = skrf.Network('./data_MiniCircuits_splitter/cal_open_raw.s2p')
match_raw = skrf.Network('./data_MiniCircuits_splitter/cal_match_raw.s2p')
thru_raw = skrf.Network('./data_MiniCircuits_splitter/cal_thru_raw.s2p')

# create an ideal 50-Ohm line for the short, open, match and through reference responses␣
↪("ideals")
line = skrf.media.DefinedGammaZ0(frequency=short_raw.frequency, z0=50)

# create and run the calibration
cal = TwoPortOnePath(ideals=[line.short(nports=2), line.open(nports=2), line.
```

(continues on next page)

(continued from previous page)

```
↪match(nports=2), line.thru()],
                    measured=[short_raw, open_raw, match_raw, thru_raw],
                    n_thrus=1, source_port=1)
cal.run()
```

The 12 individual 2-port subnetworks can now be corrected with this calibration. For a full correction, the subnetworks with the forward and reverse measurements need to be provided in pairs, for example (S_{32}, S_{22}) paired with (S_{23}, S_{33}). A nested loop can take care of this. For the comparison with the measurements provided by the manufacturer, it is convenient to store the calibrated results in a single 4-port network, which can then easily be plotted:

[2]:
```
import numpy as np

# create an empty array (f x 4 x 4) for the 4-port to be filled
s = np.zeros((len(short_raw.frequency), 4, 4), dtype=complex)
splitter_cal = skrf.Network(frequency=short_raw.frequency, s=s)

# loop through all 12 measurements, apply the calibration and save it inside 4-port␣
↪network
for i_src in range(4):
    for i_recv in range(4):
        if i_src != i_recv:
            dut_raw_fwd = skrf.Network(f'./data_MiniCircuits_splitter/dut_raw_{i_recv +␣
↪1}{i_src + 1}.s2p')
            dut_raw_rev = skrf.Network(f'./data_MiniCircuits_splitter/dut_raw_{i_src + 1}
↪{i_recv + 1}.s2p')
            dut_cal = cal.apply_cal((dut_raw_fwd, dut_raw_rev))

            # dut_cal is now a fully populated and corrected 2-port; save it in splitter_
↪cal
            splitter_cal.s[:, i_src, i_src] = dut_cal.s[:, 0, 0]
            splitter_cal.s[:, i_recv, i_src] = dut_cal.s[:, 1, 0]
            splitter_cal.s[:, i_src, i_recv] = dut_cal.s[:, 0, 1]
            splitter_cal.s[:, i_recv, i_recv] = dut_cal.s[:, 1, 1]
```

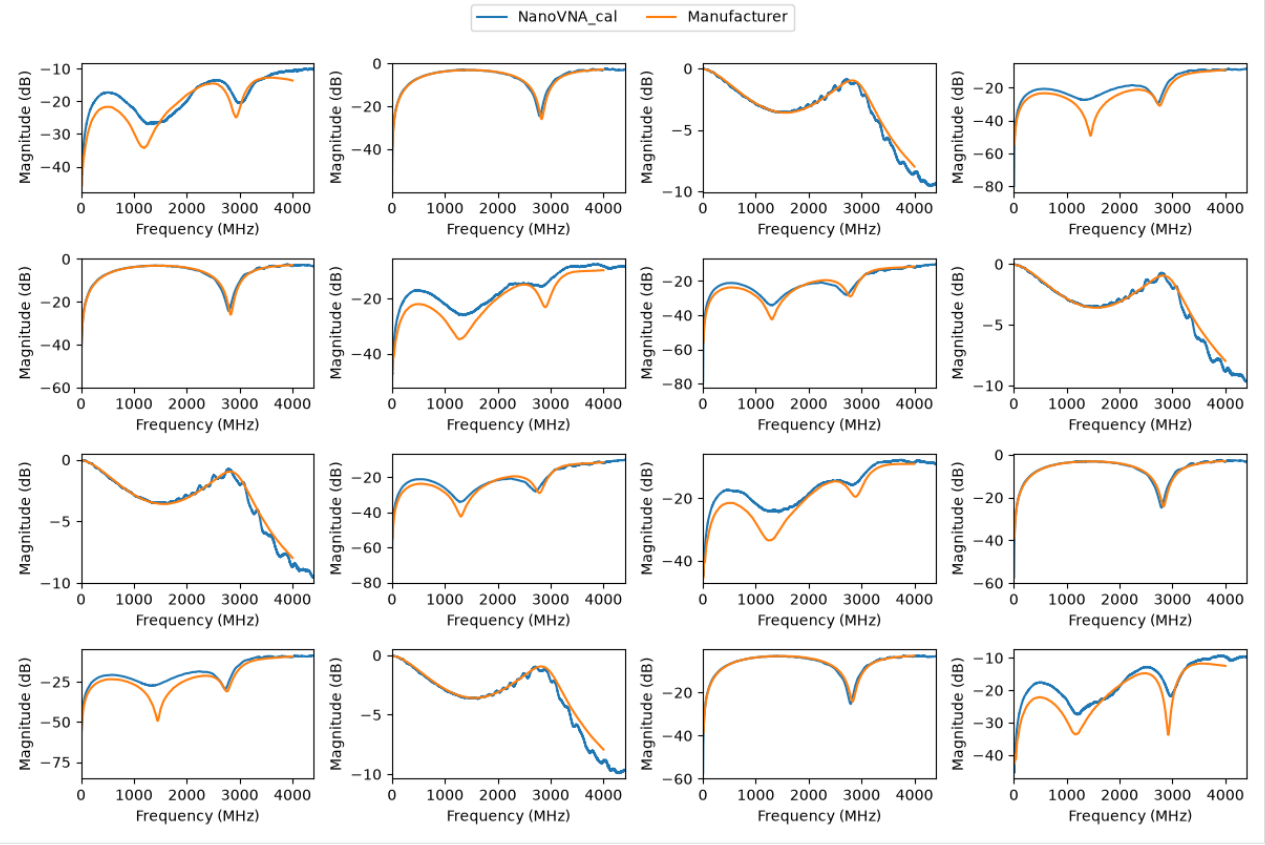
The results are now corrected and assembled as a single 4-port network. For comparison, the magnitudes are plotted together with the measurements provided by the manufacturer:

[3]:
```
import matplotlib.pyplot as mplt

# load reference results by MiniCircuits
splitter_mc = skrf.Network('./data_MiniCircuits_splitter/MiniCircuits_ZX10Q-2-19-S+___
↪Plus25degC.s4p')

# plot both results
fig, ax = mplt.subplots(4, 4)
fig.set_size_inches(12, 8)

for i in range(4):
    for j in range(4):
        splitter_cal.plot_s_db(i, j, ax=ax[i][j])
        splitter_mc.plot_s_db(i, j, ax=ax[i][j])
        ax[i][j].get_legend().remove()
```

(continues on next page)

(continued from previous page)

```
        ax[i][j].set_xlim(0, 4.4e9)
fig.legend(['NanoVNA_cal', 'Manufacturer'], loc='upper center', ncol=2)
fig.tight_layout(rect=(0, 0, 1, 0.95))
mplt.show()
```



The calibrated results are pretty close to those from the manufacturer. They apparently used a Keysight N5242A PNA-X (26.5 GHz 4-port VNA) for that measurement. Given the $200 NanoVNA V2 and the sketchy assumptions made for the Matched Port technique (the termination of the unused ports of the DUT have certainly not been entirely free of any reflections), these results are not too bad.

2.3.7 One Port Tiered Calibration

Intro

A one-port network analyzer can be used to measure a two-port device, provided that the device is reciprocal. This is accomplished by performing two calibrations, which is why its called a *tiered* calibration.

First, the VNA is calibrated at the test-port like normal. This is called the *first tier*. Next, the device is connected to the test-port, and a calibration is performed at the far end of the device, the *second tier*. A diagram is shown below,

This notebook will demonstrate how to use skrf to do a two-tiered one-port calibration. We'll use data that was taken to characterize a waveguide-to-CPW probe. So, for this specific example the diagram above looks like:

Some Data

The data available is the folders `'tier1/'` and `'tier2/'`.

[1]:
```
!ls {"oneport_tiered_calibration/"}
```

```
images  probe.s2p  tier1  tier2
```

(if you dont have the git repo for these examples, the data for this notebook can be found here)

In each folder you will find the two sub-folders, called `'ideals/'` and `'measured/'`. These contain touchstone files of the calibration standards ideal and measured responses, respectively.

[2]:
```
!ls {"oneport_tiered_calibration/tier1/"}
```

```
ideals  measured
```

The first tier is at waveguide interface, and consisted of the following set of standards

- short
- delay short
- load
- radiating open (literally an open waveguide)

[3]:
```
!ls {"oneport_tiered_calibration/tier1/measured/"}
```

```
ds.s1p  load.s1p  ro.s1p  short.s1p
```

Creating Calibrations

Tier 1

First defining the calibration for *Tier 1*

[4]:
```
import matplotlib.pyplot as plt

import skrf as rf
from skrf.calibration import OnePort
from skrf.io.general import read_all_networks

%matplotlib inline

rf.stylely()


tier1_ideals = read_all_networks('oneport_tiered_calibration/tier1/ideals/')
tier1_measured = read_all_networks('oneport_tiered_calibration/tier1/measured/')


tier1 = OnePort(measured = tier1_measured,
                ideals = tier1_ideals,
                name = 'tier1',
                sloppy_input=True)
tier1
```

[4]:
```
OnePort Calibration: 'tier1', 500.0-750.0 GHz, 401 pts, 4-standards
```

Because we saved corresponding *ideal* and *measured* standards with identical names, the Calibration will automatically align our standards upon initialization. (More info on creating Calibration objects this can be found in *the docs*.)

Similarly for the second tier 2,

Tier 2

[5]:
```
tier2_ideals = read_all_networks('oneport_tiered_calibration/tier2/ideals/')
tier2_measured = read_all_networks('oneport_tiered_calibration/tier2/measured/')


tier2 = OnePort(measured = tier2_measured,
                ideals = tier2_ideals,
                name = 'tier2',
                sloppy_input=True)
tier2
```

[5]:
```
OnePort Calibration: 'tier2', 500.0-750.0 GHz, 401 pts, 5-standards
```

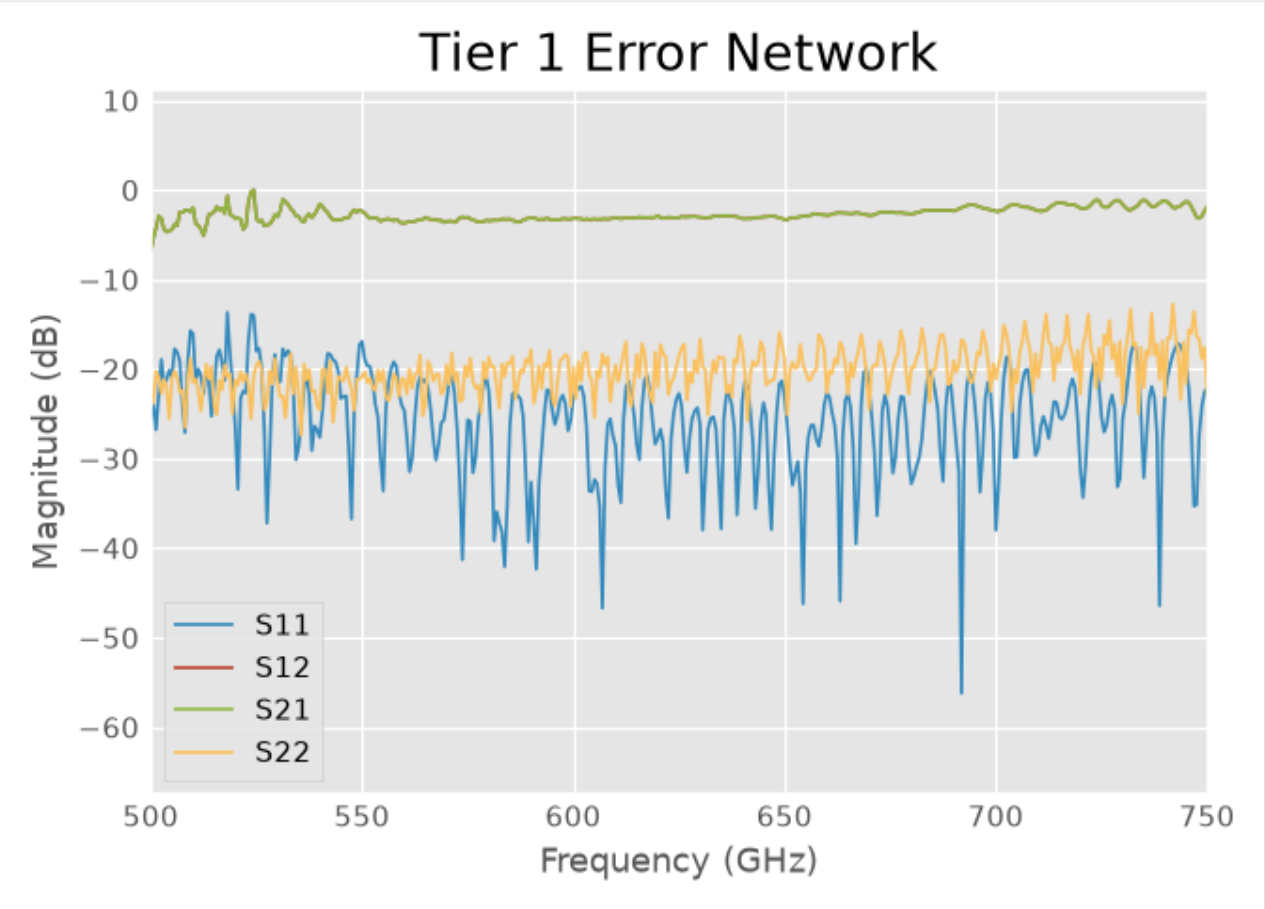
Error Networks

Each one-port Calibration contains a two-port error network, that is determined from the calculated error coefficients. The error network for *tier1* models the VNA, while the error network for *tier2* represents the VNA **and** the DUT. These can be visualized through the parameter `'error_ntwk'`.

For tier 1,

[6]:
```
tier1.error_ntwk.plot_s_db()
plt.title('Tier 1 Error Network')
```

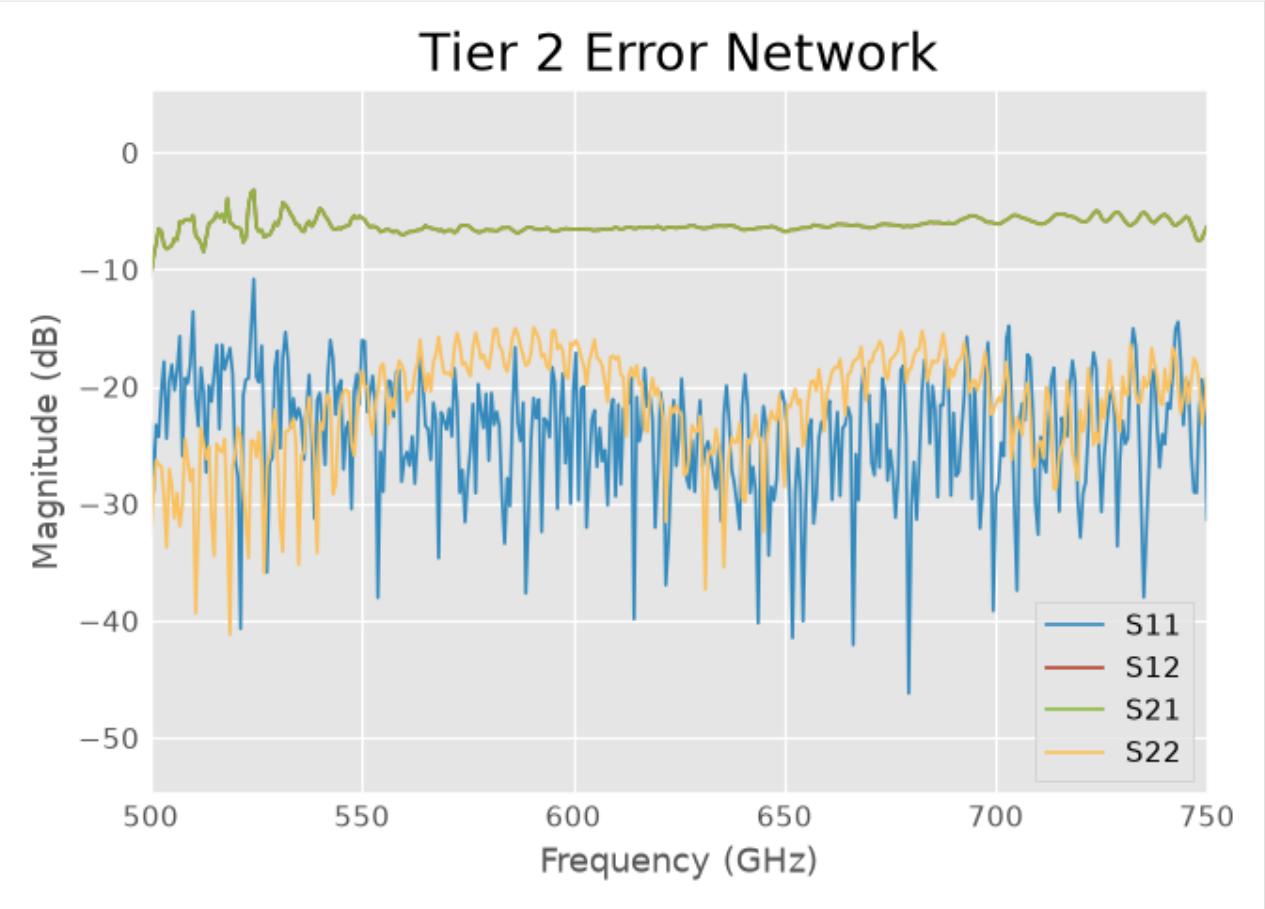
[6]:
```
Text(0.5, 1.0, 'Tier 1 Error Network')
```



Similarly for tier 2,

[7]:
```
tier2.error_ntwk.plot_s_db()
plt.title('Tier 2 Error Network')
```

[7]:
```
Text(0.5, 1.0, 'Tier 2 Error Network')
```



De-embedding the DUT

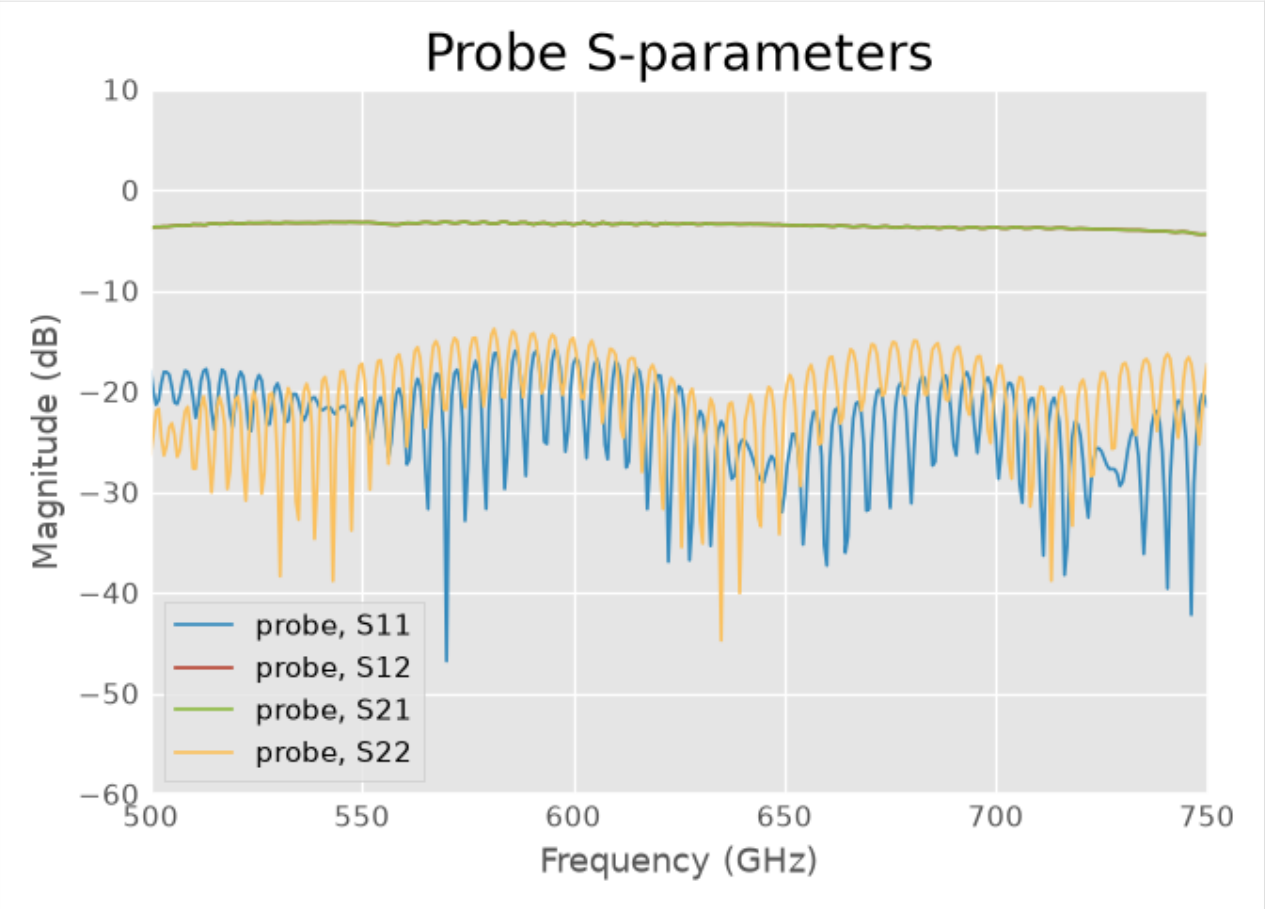
As previously stated, the error network for *tier1* models the VNA, and the error network for *tier2* represents the VNA+DUT. So to determine the DUT's response, we cascade the inverse S-parameters of the VNA with the VNA+DUT.

$$DUT = VNA^{-1} \cdot (VNA \cdot DUT)$$

In skrf, this is done as follows

[8]:
```
dut = tier1.error_ntwk.inv ** tier2.error_ntwk
dut.name = 'probe'
dut.plot_s_db()
plt.title('Probe S-parameters')
plt.ylim(-60,10)
```

[8]:
```
(-60.0, 10.0)
```



You may want to save this to disk, for future use,

```
dut.write_touchstone()
```

[9]:
```
!ls {"probe*"}
```

```
probe.s2p
```

[ ]:
```

```

2.3.8 SOLT

This little example demonstrates how to use scikit-rf for a basic SOLT calibration.

Basic Use

Imports

First we import `skrf`, the `SOLT` class, and setup matplotlib plotting.

[1]:
```
import skrf as rf
from skrf.network import two_port_reflect

rf.stylely()
```

Typical SOLT Procedure

A two-port calibration is accomplished in an identical way to one-port. However, scikit-rf only accepts measurement results as two-port networks, even for reflective and load standards.

Measure Two Standards Separately

If you don't have two sets of calibration standards, no worries! It's possible to measure reflective standards on each port separately. For example, connect a short to port-1, save a one-port measurement as `short.s1p`, then move the same short to port-2, save another one-port measurement as `short2.s1p` or similar.

Next, you can forge a two-port Network from two one-port Network using the function `skrf.network.two_port_reflect`:

```
short = rf.Network('ideals/short.s1p')  # a 1-port Network
shorts = two_port_reflect(short, short)  # a 2-port Network
```

The function `skrf.network.two_port_reflect` does this:

Measure Two Standards Simultaneously

Often, it's possible to measure two reflective standards simultaneously, then to directly store your data as a two-port network. For example, connect the first short standard to port-1 and a second short to port-2, and save everything as a two-port measurement, such as `short,short.s2p` or similar.

Isolation Calibration

For Open, Short, and Load calibrations, even though the inputs are supplied to scikit-rf as 2-port networks with four S-parameters in each, S_{21}, and S_{12} are ignored - by definition, both are equal to 0. Only S_{11} and S_{22} are used, this is true even for a Load-Load measurement. Thus, no isolation calibration takes place by default.

If isolation calibration is needed, one must explicitly tell scikit-rf to do so. Specifically, `SOLT()` class accepts an optional parameter `isolation` for this purpose. To calibrate isolation, set this parameter to a two-port network, representing the measurement result with loads on both ports.

Code

The typical workflow for a SOLT calibration is:

```
# a list of Network types, holding 'ideal' responses
my_ideals = [
    rf.Network('ideal/short, short.s2p'),
    rf.Network('ideal/open, open.s2p'),
    rf.Network('ideal/load, load.s2p'),
    rf.Network('ideal/thru.s2p'),
    ]

# a list of Network types, holding 'measured' responses
my_measured = [
    rf.Network('measured/short, short.s2p'),
    rf.Network('measured/open, open.s2p'),
    rf.Network('measured/load, load.s2p'),
```

(continues on next page)

(continued from previous page)

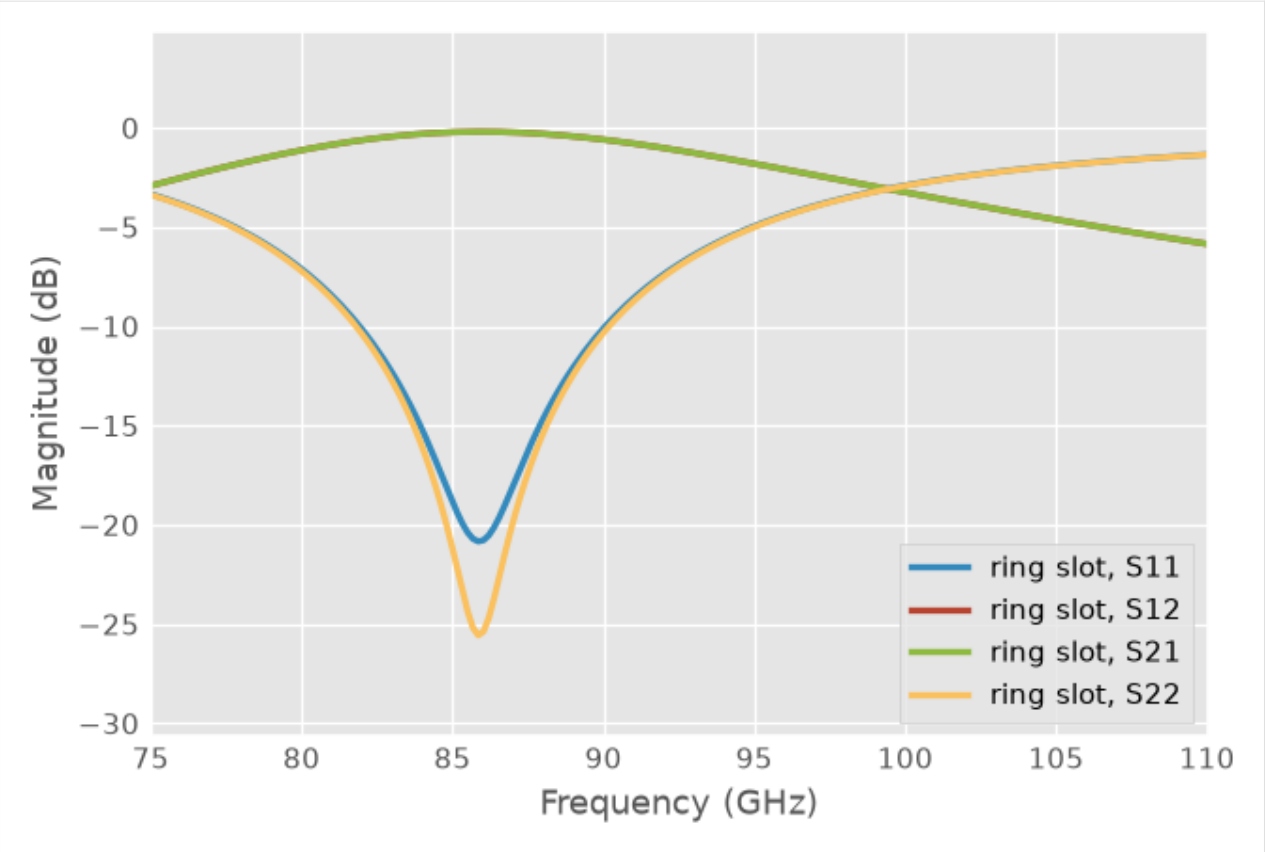
```
    rf.Network('measured/thru.s2p'),
    ]

## create a SOLT instance
cal = SOLT(
    ideals = my_ideals,
    measured = my_measured,
    isolation = rf.Network('measured/load, load.s2p'),
    # isolation calibration is optional, it can be removed.
    )

## run, and apply calibration to a DUT
# run calibration algorithm
cal.run()

# apply it to a dut
dut = rf.Network('my_dut.s2p')
dut_caled = cal.apply_cal(dut)

# plot results
dut_caled.plot_s_db()
# save results
dut_caled.write_touchstone()
```

Example

The following example illustrates a common situation: a DUT is connected to a VNA using two cables of different lengths. The purpose of the calibration is to move the reference plane do the DUT, that is to remove the effect of the cables from the measurement.

In the example below, the DUT is already known, just to be able to confirm that the calibration method is working at the end. Of course, in reality, the DUT is generally not known…

[2]:
```
dut = rf.data.ring_slot
dut.plot_s_db(lw=2)  # this is what we should find after the calibration
```



The ideal component Networks are obtained from your calibration kit manufacturers or from modelling.

In this example, we simulate ideal components from transmission line theory. We create a lossy and noisy transmission line (for the sake of the example).

[3]:
```
media = rf.media.DefinedGammaZ0(frequency=dut.frequency, gamma=0.5 + 1j)
```

Then we create the ideal components: Short, Open and Load, and Through. By default, the methods `media.short()`, `media.open()`, and `media.match()` return a one-port network, the SOLT class expects a list of two-port Network, so `two_port_reflect()` is needed to forge a two-port network from two one-port networks (`media.thru()` returns a two-port network and no adjustment is needed).

Alternatively, the argument `nports=2` can be used as a shorthand for this task.

[4]:
```
# ideal 1-port Networks
short_ideal = media.short()
open_ideal = media.open()
load_ideal = media.match()  # could also be: media.load(Gamma0=0)
thru_ideal = media.thru()

# forge a two-port network from two one-port networks
short_ideal_2p = two_port_reflect(short_ideal, short_ideal)
open_ideal_2p = two_port_reflect(open_ideal, open_ideal)
load_ideal_2p = two_port_reflect(load_ideal, load_ideal)

# alternatively, the "nport=2" argument can be used as a shorthand
# short_ideal_2p = media.short(nports=2)
```

(continues on next page)

(continued from previous page)

```
# open_ideal_2p = media.open(nports=2)
# load_ideal_2p = media.match(nports=2)
```

Now that we have our ideal elements, let's fake the measurements.

Note that the transmission lines are not symmetric in the example below, to make it as generic as possible. In such case, it is necessary to call the *flipped()* method to connect the ideal elements on the correct side of the `line2` object.

[5]:
```
# left and right piece of transmission lines
line1 = media.line(d=20, unit='cm')**media.impedance_mismatch(1,2)
line2 = media.line(d=30, unit='cm')**media.impedance_mismatch(1,3)

# add some noise to make it more realistic
line1.add_noise_polar(.01, .1)
line2.add_noise_polar(.01, .1)

# fake the measured setup
measured = line1 ** dut  ** line2

# fake the calibration measurements
# Note the use of flipped() on line2
open_measured = two_port_reflect(line1 ** media.open(), line2.flipped() ** media.open())
short_measured = two_port_reflect(line1 ** media.short(), line2.flipped() ** media.
↪short())
load_measured = two_port_reflect(line1 ** media.load(Gamma0=0), line2.flipped() ** media.
↪load(Gamma0=0))
thru_measured = line1 ** line2
```

We can now create the lists of Network that the `SOLT` class expects:

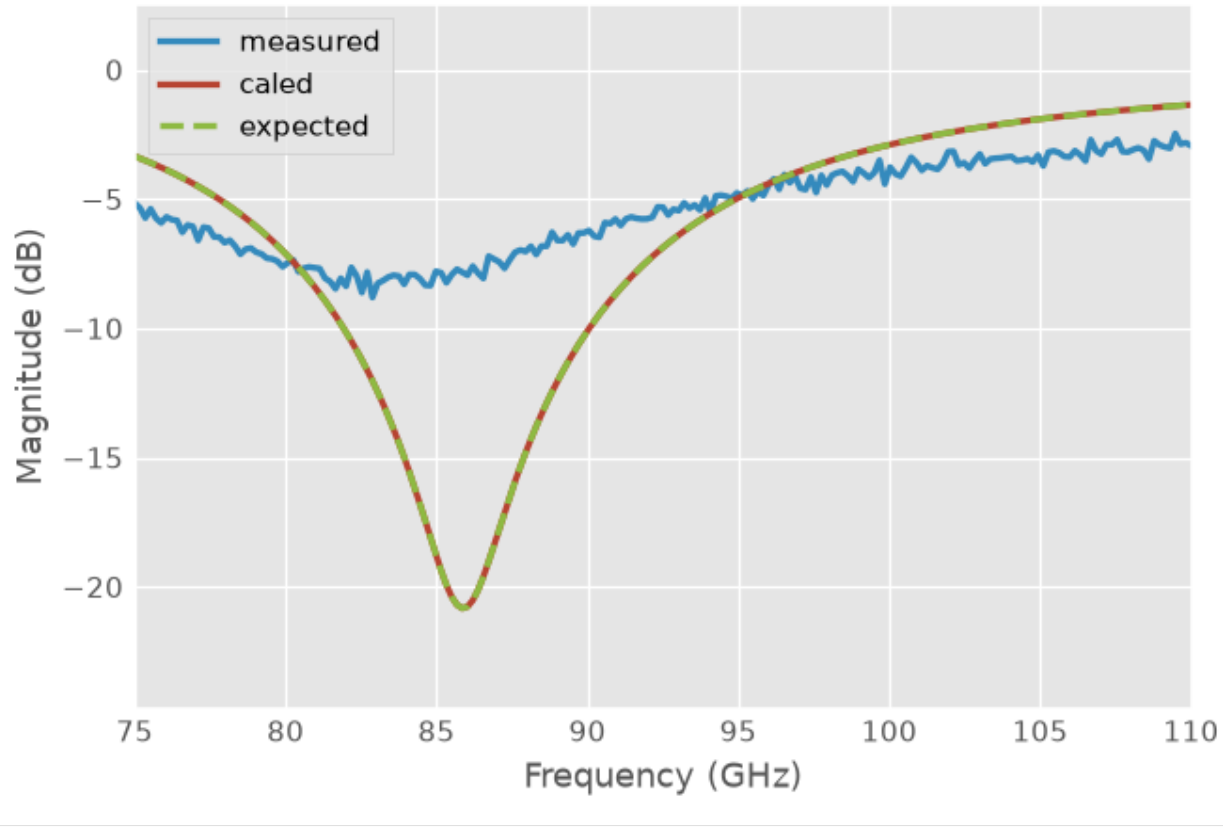
[6]:
```
# a list of Network types, holding 'ideal' responses
my_ideals = [
    short_ideal_2p,
    open_ideal_2p,
    load_ideal_2p,
    thru_ideal,   # Thru should be the last
    ]

# a list of Network types, holding 'measured' responses
my_measured = [
    short_measured,
    open_measured,
    load_measured,
    thru_measured,   # Thru should be the last
]

## create a SOLT instance
cal = rf.calibration.SOLT(
    ideals = my_ideals,
    measured = my_measured,
    )
```

And finally apply the calibration:

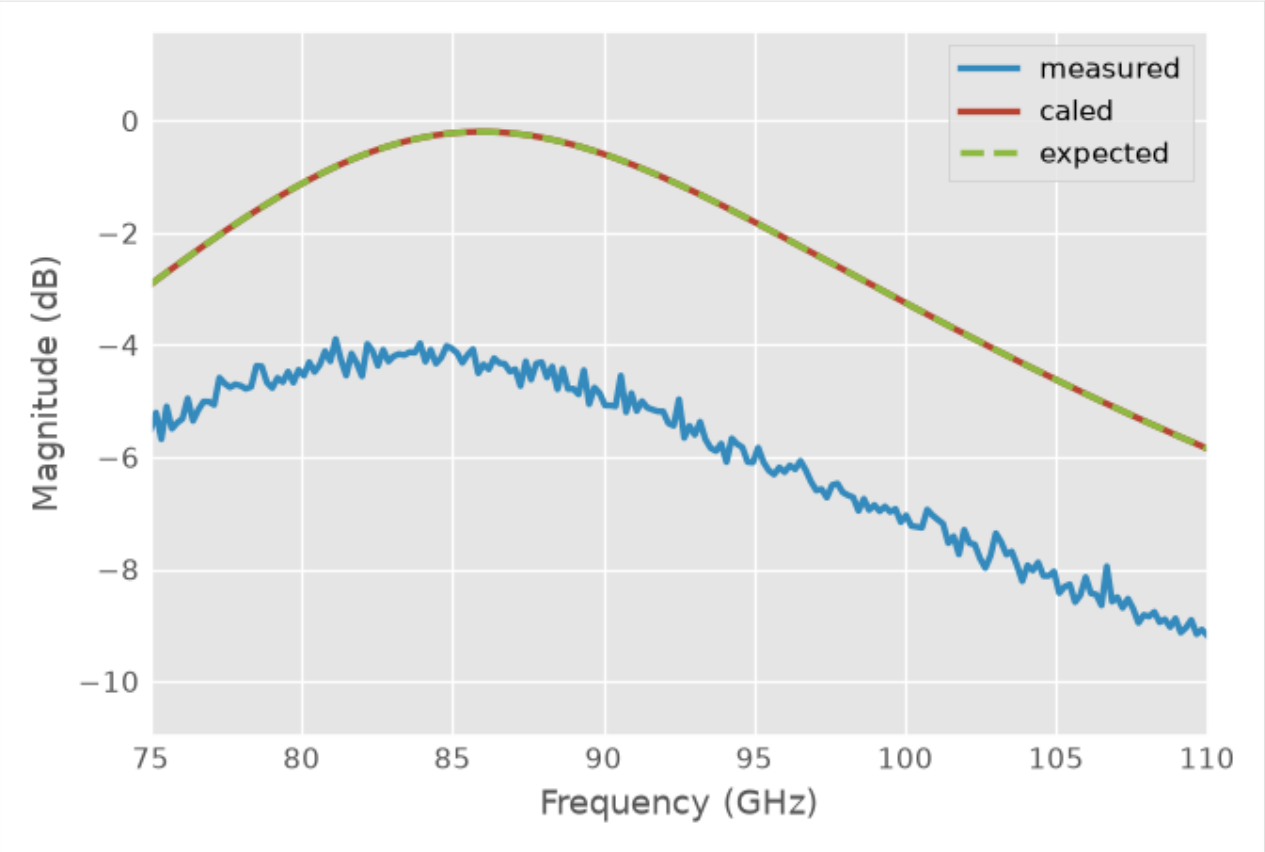
[7]:
```
# run calibration algorithm
cal.run()

# apply it to a dut
measured_caled = cal.apply_cal(measured)
```

Let's see the results for S11 and S21:

[8]:
```
measured.plot_s_db(m=0, n=0, lw=2, label='measured')
measured_caled.plot_s_db(m=0, n=0, lw=2, label='caled')
dut.plot_s_db(m=0, n=0, ls='--', lw=2, label='expected')
```



[9]:
```
measured.plot_s_db(m=1, n=0, lw=2, label='measured')
measured_caled.plot_s_db(m=1, n=0, lw=2, label='caled')
dut.plot_s_db(m=1, n=0, ls='--', lw=2, label='expected')
```



The caled Network is (mostly) equal the DUT as expected:

[10]:
```
dut == measured
```

[10]:
```
False
```

[11]:
```
dut == measured_caled  # within 1e-4 absolute tolerance
```

[11]:
```
True
```

2.3.9 SOLT Calibration Standards Creation

Introduction

In scikit-rf, a calibration standard is treated just as a regular one-port or two-port `skrf.Network`, defined by its full S-parameters. It can represent reflection, transmission, load, and even arbitrary-impedance standards. Since no additional errors are introduced in circuit modeling and fitting, this approach allows the highest calibration accuracy, and is known as a *data-based standard* in the terminology of VNA vendors.

However, VNA calibration standards are traditionally defined using a circuit model with fitted coefficients. Since data-based standards are often offered as premium products, many model-based calibration kits are still being used and manufactured (even at 50 GHz [1]). This necessitates the creation of their network models before they can be used in scikit-rf's calibration routines.

This example explains network creation from coefficients given in both the HP-Agilent-Keysight format and the Rohde & Schwarz / Anritsu format. Both are essentially the same circuit model, but the latter format uses different units of measurement for an offset transmission line.

Calculated responses for Keysight 85032F (Type-N plug), Keysight 85033E (3.5 mm plug), generic flush standards (SMA plugs), and Maury Microwave 8050CK10 (3.5 mm plug) are also plotted in this document as examples.

Note

Only coaxial standards are covered by this guide. The calculation is different for waveguides. In particular, the scaling by $\sqrt{\text{GHz}}$ for coaxial lines cannot be applied to waveguides because loss is also a function of their physical dimensions, with significantly more complicated formulas. Do you have waveguide experience? If so, you can help by *contributing* to the doc.

Alternatives to scikit-rf Modeling

Before we begin, it's worth pointing out some alternatives.

In scikit-rf, you are able to use any existing standard definition by its S-parameters. If you already have your standard defined as a network in other tools (e.g. in your favorite circuit simulator, or actual measurements results), you can simply export the S-parameters to Touchstone files for use in scikit-rf. Similarly, if you're already using a data-based calibration standard, it should be possible to use its data directly. The S-parameters may be stored in device-specific file formats, consult your vendor about whether they can be exported as a Touchstone file.

As a special case, if *flush* (zero-length) standards are used - meaning that the standards sit right at the connector's reference plane without extended bodies - sometimes one can assume the calibration standards are ideal for non-critical measurements. In scikit-rf, one can create ideal responses conveniently by defining an ideal transmission line and calling the `short()`, `open()`, `match()`, and `thru()` methods (explained in the *Preparation* section).

Important

Ideal assumptions are only approximately valid for *flush* standards. Most lab-grade standards are *offset* standards, always use these standards with the correct definitions entered. They have additional electrical delays from their extended bodies, phase errors would be unacceptably large under ideal assumptions. Using a *flush* standard behind a *Thru* adapter creates the same phase deviations. See below for the distinction between *flush* and *offset* standards.

HP-Agilent-Keysight Coefficient Format

After the necessary background is introduced, let's begin.

For the purpose of this guide, we're going to model the Keysight 85032F, Type-N, 50 Ω, DC to 9 GHz calibration kit (plug), with the following coefficients.

Parameter	Unit	Open	Short	Load	Thru
C_0	10^{-15} F	89.939			
C_1	10^{-27} F/Hz	2536.800			
C_2	10^{-36} F/Hz^2	-264.990			
C_3	10^{-45} F/Hz^3	13.400			
L_0	10^{-12} H		3.3998		
L_1	10^{-24} H/Hz		-496.4808		
L_2	10^{-33} H/Hz^2		34.8314		
L_3	10^{-42} H/Hz^3		-0.7847		
Resistance	Ω			50	
Offset Delay	ps	40.856	45.955	0	0
Offset Loss	GΩ / s	0.93	1.087	0	0
Offset Z_0	Ω	50	49.992	50	50
Reference Z_0	Ω	50	50	50	50

Circuit Model

Before we start creating their network definitions, we first need to know the underlying circuit model and the meaning of these coefficients. As this schematic shows, this is the HP-Agilent-Keysight model for a calibration standard.

Circuit Model of Calibration Standards

Termination Impedance

A shunt impedance is connected at the end of an *offset* transmission line, and models the distributed capacitance or inductance in the open or short standard. It's given as a third-degree polynomial with four coefficients, $y(f) = a_0 + a_1 f + a_2 f^2 + a_3 f^3$, where f is the frequency and a_i are the coefficients. For an open standard, they're C_0, C_1, C_2, C_3, the first constant term is in femtofarad. For a short standard, they're L_0, L_1, L_2, L_3, the first constant term is in picohenry.

Important

The reflection coefficient (S_{11}) of the termination impedance is calculated with respect to the system impedance (Z_{ref}), not the impedance of the offset transmission line (Z_{off} or Z_{c}). [15]

Offset Line

In front of the calibration standard, there exists an *offset* lossy transmission line. To understand its context, we need to distinguish two types of calibration standards: *flush* (zero-length) and *offset* standards.

In a *flush* standard, the physical standard is located directly at the reference plane of the port. This design is widely used in uncharacterized generic SMA calibration "standards" (if they can be called as such) in amateur radio. *Flush* standards are not always of low quality, they're also found in some lab-grade standards, such as Keysight 85031B for APC-7 connectors.

Flush standards sit flush against the connector, with no additional structure. Offset standards have extended bodies, introducing electrical delays.

However, most lab-grade standards are designed as *offset* standards, with a short length between the reference plane of the connector and the physical location of the standard. This can be done to improve the predictability of a standard's electrical characteristics [2], or merely a limitation of a connector's mechanical design (e.g. all Type-N connectors have a reference plane offset of 5.258-5.360 mm [3]). Both factors introduce an electrical length that must be accounted for, otherwise it would introduce a large phase deviation.

This *offset* length is modeled as a transmission line using three parameters:

1. **Lossless Offset Impedance** (Z_{off}): A real characteristic impedance, with line loss (R and G) neglected. This often matches the VNA's reference impedance, but not always. Sometimes a value slightly different from the

reference impedance is used to model physical imperfections, such as 50.209 or 49.992 . Also, waveguide standards use a special normalized value 1.

2. **Offset Delay** (t_{delay}) - One-way electrical delay, given in picoseconds (ps).
3. **Offset Loss** (A_{loss}). The nominal attenuation at 1 GHz per unit time, given in G/s (gigaohms per second). Scale this value by $\sqrt{f/(1\text{ GHz})}$ for other frequencies. The unit G/s is unconventional, but for a reason. To express a transmission line segment's length, both a distance (in meters) or a time delay (in seconds) can be used. Defining the line with respect to time allows calculating $R = A_{\text{loss}} t_{\text{delay}}$ in ohms, without needing to explicitly define the medium permittivity ϵ.

Important

These parameters are given in datasheets in scaled units, but unscaled SI units should be use in all calculations (without prefix): ohms (), seconds (s), ohms per second (/s), and hertz (Hz).

RLCG parameters

RLCG transmission line model

As these parameters are non-standard, we need to convert them to the familiar RLCG line parameters. After algebraic manipulation of [4], equation 2B.4, the corresponding frequency-dependent RLCG parameters are computed as follows.

$$
\begin{aligned}
R(f) \cdot l &= (A_{\text{loss}} t_{\text{delay}}) \cdot \sqrt{\frac{f}{10^9}} \\
L(f) &= L_0 + L_{\text{cond}} = t_{\text{delay}} Z_{\text{off}} + \frac{R}{2\pi f} \\
C &= \frac{t_{\text{delay}}}{Z_{\text{off}}} \\
G &= 0 \\
l &= 1 \text{ (normalized)}
\end{aligned}
$$

The term L_0 is the ideal line inductance, $L_{\text{cond}} = R/(2\pi f)$ is the frequency-dependent inductance of imperfect conductors due to skin effect. In [5], nearly the same equations were derived, except that the term L_{cond} has been omitted. It must be included here to be consistent with the original Keysight publication [4] (otherwise the phase error would be greater than 0.1 degrees at 9 GHz for the 85032F).

Using these parameters, one can calculate the *propagation constant* (γ) and the complex *characteristic impedance* (Z_c) of the lossy line. Note that in the frequency-dependent `RLCG(f)` model, each frequency has its own RLCG parameters and line constants.

Important

Our formulation is *not* exactly identical to the official Keysight calculations and other works. The latter model used a low-loss approximation when calculating line constants, while our formulation starts from the RLCG parameters to derive the exact line constants. Both formulations have negligible differences (Keysight 85056A's magnitude and phase angle are in agreement to 4 decimal places at 50 GHz), this difference is only academic.

Note

We show the RLCG approach due to its wide understanding and ease of use. If reproducing the exact numerical values down to the last digit is absolutely required, see *Appendix* for derivation and code for Keysight's official low-loss approximation.

Reference Impedance

All calibration standards have an implied *reference impedance* Z_{ref}. Some Keysight datasheets show it as the "System Z_0" [6], but it's not always given out explicitly. One may require some context to deduce it: if a standard is named *Keysight 85032F Type N (50) Calibration Kit*, Z_{ref} is 50 .

It's not to be confused with the offset transmission line's characteristic impedance Z_0 (lossless) or Z_c (lossy), which can have a value of 49.992 . Unlike the *line impedance*, the *reference impedance's* purpose is to renormalize the S-parameters to the standard form, since all S-parameters are relative. This is always set to the reference impedance of the VNA system: 50 or 75 .

Connector Discontinuity

Since the reference impedance appears to be an arbitrary choice, one may conclude that any calibration standard can be used to calibrate any VNA port (as long as they're mechanically compatible) regardless of its offset line impedance. For example, one may attempt to calibrate a 50 VNA port by connecting it directly to a 75 calibration standard, so that all calibrated measurements behave as if they were made using a 75 system. Likewise, if the reference impedance of a 75 calibration standard is renormalized, one can apparently predict its ideal S-parameters on a 50 VNA in order to abuse it as a 50 standard.

However, this assumption is not true. Renormalization cannot predict the parasitic effects at the physical test port due to discontinuity of the electromagnetic field, from a sudden change of the transmission line's physical dimensions. [7][9] It's approximately a capacitive discontinuity. [18] This effect exists even if both sides have the same characteristic impedance, as long as the coaxial line's physical dimensions have a step change (e.g. 3.5 mm to 2.92 mm port). [8]

In theory, the excitation of high-order modes at a discontinuity makes calibration unreliable, since it can be sensitive to perturbation and not repeatable. Indeed, pathological cases have been observed at 50 GHz even between a pair of "perfect" 1.85 mm connectors - the mated connector pair generates a resonance because its mechanical tolerance is *too precise*, the near-zero gap forms a resonator. [25][26]

Nevertheless, in most practical measurements unaffected by such pathological phenomena, it has been showed that, if the DUT and the calibration standards have the same connector design, both produce a similar discontinuity, this discontinuity can often be calibrated out, leaving only a tiny error negligible for all but the most demanding metrological applications. [24] Such is the case when measuring 75 devices after calibration by 75 standards using a 50 port. [9] However, if the DUT and the calibration standards have different connector designs (e.g. cross-connecting 3.5 mm, 2.92 mm and SMA), this discontinuity produces a slight measurement error. [8]

Thus, for satisfactory results, the calibration standards and the DUT must use the same connector. For the best result, the test port (or adapter) must be of high quality and matches the calibration standards as well.

Fitting Non-Uniqueness and Physical Interpretability

The phase shift of a small capacitance or inductance is approximately linear, it's often possible to model it equally well as a termination reactance, or as an electrical delay in an offset transmission line. [4][10]

For example, Keysight 85033D's datasheet show two models for the short standard, either as a pure time delay, or a time delay with inductance. [6]

Due to the non-uniqueness of standard definitions, caution is required when interpreting individual terms in standard definitions: the overall response is meaningful, the individual terms may or may not. Most lab-grade calibration standards attempt to ensure that the standard definitions are physically meaningful [11], but it's not always guaranteed.

When defining a custom calibration standard via curve fitting, separating the phase shift due to the offset transmission line and termination reactances is difficult. With prior knowledge, one can fix the offset length by mechanical measurements [12], or fixing the termination capacitance by electromagnetic theory or simulation. [10] Both require precise knowledge and modeling of the standard's design. For example, historical studies focused on hollow air-filled open standards (using the theory of circular waveguide below cutoff) [10], but it's no longer applicable today as many open standards use dielectric support. When there's a lack of information, a delay-only model may be the only practical choice.

Tip

In fact, a delay-only short standard is the only model supported by older VNA's firmware, such as the HP 8510 [13]. External calibration via scikit-rf allows you to bypass this limitation.

Neglected Terms

Both an open or short circuit can be modeled as an electrical delay only without reactance, or vice versa. The delay-only model is most common for the short standard, since it was used historically.

A matched load generates little reflection, thus it's often simply modeled as a Z_0 termination. Since its reflection coefficient is approximately 0, its phase angle is meaningless, its electrical delay is thus also neglected.

The Thru standard is sometimes modeled with an offset delay only, if loss is considered negligible.

Some calibration kits don't have any Thru delay data. These kits are meant for *Flush Thru*, *Swap Equal Adapters*, or *Unknown Thru* calibration, without using *Thru*'s data.

Flush Thru is the traditional SOLT calibration practice: port 1 and port 2 are connected directly, without using adapters. This *Thru* is ideal by definition and has zero length, no modeling is required. In a *Swap Equal Adapters* calibration, a pair of two delay-matched adapters are involved, making the *Thru*'s actual length invisible. The *Unknown Thru* calibration algorithm is relatively newer, which doesn't require a characterized Thru.

Note

Sometimes an offset loss still appears in the Load and Thru's definitions, but with a zero electrical length. These parameters should be ignored. The zero offset delay disables the offset transmission line.

Plug and Socket

Finally, it's worth clarifying Keysight's plug and socket notations. If a *plug* standard is used to calibrate a *socket*, it's denoted by datasheets as `-m-` with respect to itself, or `(f)` with respect to the test port. If a *socket* standard is used to calibrate a plug, it's denoted by datasheets as `-f-` with respect to itself, or `(m)` with respect to the test port. [14]

Preparation

Equipped with this circuit model, we can start to model the calibration standards.

First, we need to import some library definitions, specify the frequency range of our calculation. Here, we used 1 MHz to 9 GHz, with 1001 points. You may want to adjust it for your needs. We also define an `ideal_medium` with a 50Ω port impedance for the purpose of some future calculations.

[1]:
```
import numpy as np

import skrf
from skrf.media import DefinedGammaZ0, DistributedCircuit
```

(continues on next page)

(continued from previous page)

```
# reference impedance of the S-parameters (not the offset line)
z0_ref = 50
freq = skrf.Frequency(1, 9000, 1001, "MHz")
ideal_medium = DefinedGammaZ0(frequency=freq, z0=z0_ref)
```

Ideal Responses

It's useful to know the special case first: ideal calibration standards are easily created by calling the `open()`, `short()`, `match()`, and `thru()` methods in the `ideal_medium`, the first three return a 1-port network. The `thru()` method returns a two-port network.

[2]:
```
ideal_open  = ideal_medium.open()
ideal_short = ideal_medium.short()
ideal_load  = ideal_medium.match()
ideal_thru  = ideal_medium.thru()
```

Modeling the Offset Transmission Line

To correctly model the offset transmission line, one should use the offset delay, offset loss, and offset Z_0 to derive the RLCG parameters of the lossy line. Internally, `scikit-rf` automatically derives its *propagation constant* (γ) and the complex *characteristic impedance* (Z_c) of the lossy line.

The relationship between the offset line parameters and RLCG line parameters has already been given in the discussion above. Let's translate these formulas to code.

[3]:
```
def offset_rlcg(freq, offset_delay, offset_loss, offset_z0):
    r = offset_loss * offset_delay * np.sqrt(freq.f / 1e9)
    l = (offset_delay * offset_z0) + r / (2 * np.pi * freq.f)
    c = offset_delay / offset_z0
    g = 0
    return (r, l, c, g)
```

> **Tip**
>
> The broadcasting feature in `numpy` is used here. The quantities `r`, `l` are frequency-dependent, thus they're arrays, not scalars. But instead of looping over each frequency explicitly and adding them to an array, here, arrays are automatically created by the multiplication of a scalar and a `numpy.array`. We'll continue to use this technique.

With the function `offset_rlcg()` defined, we can now calculate the line constants for the open and short standards by calling it.

[4]:
```
rlcg_open = offset_rlcg(freq, 40.856e-12, 0.93e9, 50)
rlcg_short = offset_rlcg(freq, 45.955e-12, 1.087e9, 49.992)
```

At this point, we already have everything we need to know about this offset line. The other half of the task is straightforward: create a two-port network for this transmission line in scikit-rf using these RLCG parameters, scikit-rf will help us automatically calculating its propagation constant γl and the complex characteristic impedance Z_c.

It's easy to perform this task in scikit-rf:

1. First, create a `DistributedCircuit` medium with four arrays (RLCG) as inputs. The created `DistributedCircuit` represents a physical medium.

2. When creating the medium, we also need to specify the S-parameter reference impedance Z_{ref} as `z0_port=50` (or `z0_port=75`).
    - This reference impedance Z_{ref} is not to be confused with the offset line impedances Z_{off} and Z_c. Z_{ref} (real) is always the nominal system or port impedance of the VNA, while the line impedances Z_{off} (real) and Z_c (complex) may differ due to physical imperfection and losses.
3. Then, an actual line with a 1-meter length is derived by calling the medium's `line()` method.

[5]:
```
medium_open = DistributedCircuit(frequency=freq,
    R=rlcg_open[0], L=rlcg_open[1], C=rlcg_open[2], G=rlcg_open[3],
    z0_port=z0_ref
)
line_open = medium_open.line(
    d=1, unit='m'
)

medium_short = DistributedCircuit(frequency=freq,
    R=rlcg_short[0], L=rlcg_short[1], C=rlcg_short[2], G=rlcg_short[3],
    z0_port=z0_ref
)
line_short = medium_short.line(
    d=1, unit='m'
)
```

Modeling the Shunt Impedance

Then, we need to model the shunt impedance of the open and short standards. For the open standard, it's a capacitance. For the short standard, it's an inductance.

Both are modeled as third-degree polynomials, as functions of frequency. In `numpy`, one can quickly define such a function via `np.poly1d([x3, x2, x1, x0])`. This is a higher-order function which accepts a list of coefficients in descending order, and returns a callable polynomial function.

After the polynomial is evaluated, we can generate the frequency-dependent capacitors and inductors. The open circuit is modeled as a series `medium.capacitor()` followed by an ideal `medium.short()`. The short circuit is modeled as a series `medium.inductor()` followed by an ideal `medium.short()`.

According to [15] the S-parameters of the capacitor and inductor are defined with respect to the system's reference impedance, not the *offset* transmission line, so we use `ideal_medium` to avoid confusion. Nevertheless, `medium_open` or `medium_short` are also acceptable: they also correctly use the port impedance of `skrf.Media()`' as the reference impedance.

[6]:
```
# use ideal_medium, not medium_open and medium_short to avoid
# reference impedance confusions.

capacitor_poly = np.poly1d([
      13.400 * 1e-45,
    -264.990 * 1e-36,
    2536.800 * 1e-27,
      89.939 * 1e-15
])
capacitor_list = capacitor_poly(freq.f)
shunt_open = ideal_medium.capacitor(capacitor_list) ** ideal_medium.short()

inductor_poly = np.poly1d([
```

(continues on next page)

(continued from previous page)

```
      -0.7847 * 1e-42,
      34.8314 * 1e-33,
    -496.4808 * 1e-24,
       3.3998 * 1e-12
])
inductor_list = inductor_poly(freq.f)
shunt_short = ideal_medium.inductor(inductor_list) ** ideal_medium.short()
```

Note on Series and Shunt Impedance

To model a shunt impedance such as a capacitance for the open standard, one can use a series `medium.capacitor()` terminated by a `medium.short()`.

```
# [PORT] ---- [CAPACITOR] --- [PORT]
#                               |
#                               |
#                            [SHORT]
#                               |
#                               |
#                             GND
shunt_open = ideal_medium.capacitor(capacitor_list) ** ideal_medium.short()
```

Or a parallel `medium.shunt_capacitor()` terminated by a `medium.open()`.

```
# [PORT] -----v----- [PORT] ----- [OPEN]
#             |
#             |
#     [SHUNT CAPACITOR]
#             |
#             |
#           [GND]
shunt_open = ideal_medium.shunt_capacitor(capacitor_list) ** ideal_medium.open()
```

Both are equivalent. The `medium.open()` termination is important for `shunt_capacitor()`: it creates a two-port network with a capacitor to ground - this network is similar to an oscilloscope terminator, which is a 2-port device with a shunt 50 resistance - so the other "output" port needs to be open. Otherwise, a line terminated solely by a `shunt_capacitor()` produces incorrect S-parameters.

Completion

Finally, we connect these model components together, and add definitions for the ideal load and Thru, this completes our modeling.

[7]:
```
open_std = line_open ** shunt_open
short_std = line_short ** shunt_short
load_std = ideal_medium.match()
thru_std = ideal_medium.thru()
```

Now you can pass these standards into scikit-rf's calibration routines, or use the `write_touchstone()` method to save them on the disk for future use.

> **Note**
>
> Here, the `open_std`, `short_std` and `load_std` we generated are one-port networks, but most scikit-rf's calibration routines expect a two-port networks as standards since they're used in two-port calibrations. You can use the function `skrf.two_port_reflect()` to generate a two-port network from two one-port networks. For more information, be sure to read the *SOLT calibration* example in the doc.

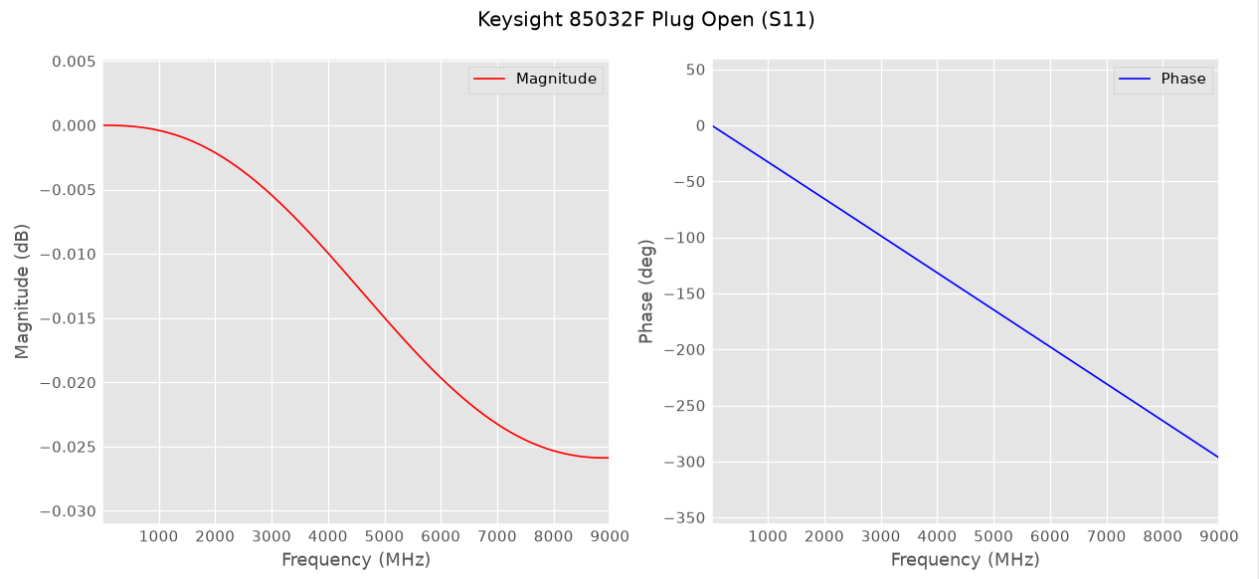
Plotting

[8]:
```
%matplotlib inline
import matplotlib.pyplot as plt

skrf.stylely()
```

Finally, let's take a look at the magnitudes and phase shifts of our standards.

Open

[9]:
```
fig, ax = plt.subplots(1, 2)
fig.set_size_inches(12, 5)

plt.suptitle("Keysight 85032F Plug Open (S11)")
open_std.plot_s_db(ax=ax[0], color='red', label="Magnitude")
open_std.plot_s_deg_unwrap(ax=ax[1], color='blue', label="Phase")
```



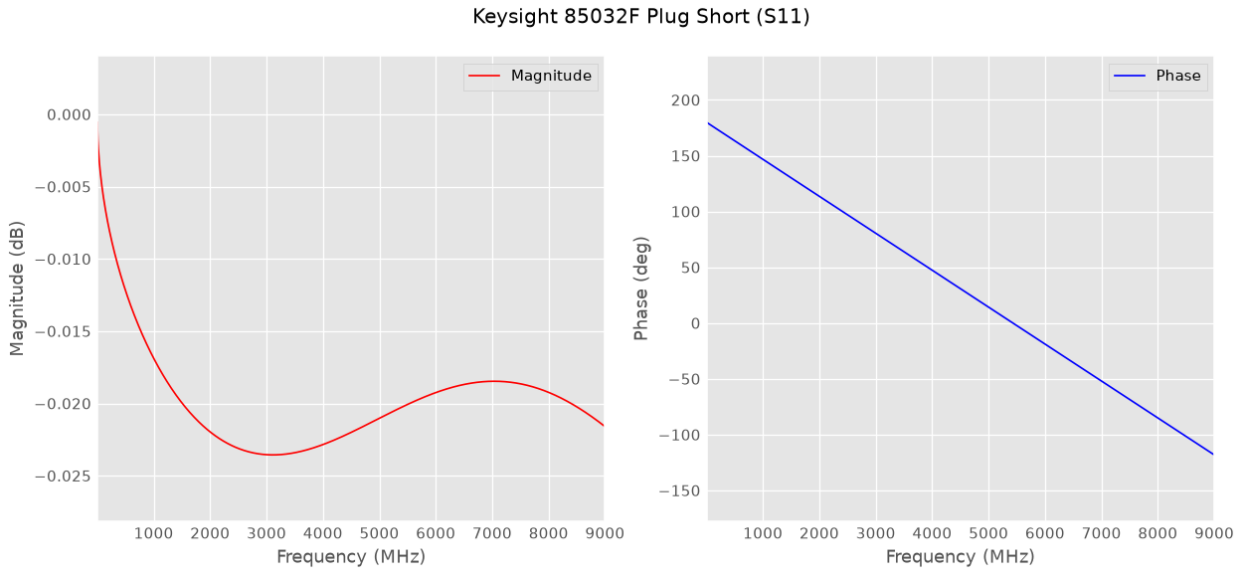
Short

[10]:
```
fig, ax = plt.subplots(1, 2)
fig.set_size_inches(12, 5)

plt.suptitle("Keysight 85032F Plug Short (S11)")
short_std.plot_s_db(ax=ax[0], color='red', label="Magnitude")
```

(continues on next page)

(continued from previous page)

```
short_std.plot_s_deg_unwrap(ax=ax[1], color='blue', label="Phase")
```



Conclusion

As shown in the graphs above, the losses in the standards are extremely low, on the order of 0.01 dB throughout the spectrum. Meanwhile, the phase shift is what really needs compensation for. At 1 GHz, the phase shift has already reached 25 degrees or so.

> **Important**
>
> Most lab-grade standards are offset standards, so large phase shifts are unavoidable. Always use these standards with the correct definitions entered.

Code Snippet

For convenience, you can reuse the following code snippets to generate calibration standard networks from coefficients in Keysight format.

[11]:
```
import numpy as np

import skrf
from skrf.media import DefinedGammaZ0, DistributedCircuit


def keysight_calkit_offset_line(freq, offset_delay, offset_loss, offset_z0, ref_z0):
    if offset_delay or offset_loss:
        r = offset_loss * offset_delay * np.sqrt(freq.f / 1e9)
        l = (offset_delay * offset_z0) + r / (2 * np.pi * freq.f)
        c = offset_delay / offset_z0
        g = 0

        medium = DistributedCircuit(frequency=freq,
```

(continues on next page)

(continued from previous page)

```
            R=r, L=l, C=c, G=g,
            z0_port=ref_z0
        )
        offset_line = medium.line(d=1, unit='m')
        return medium, offset_line
    else:
        medium = DefinedGammaZ0(frequency=freq, z0=ref_z0)
        line = medium.line(d=0)
        return medium, line


def keysight_calkit_open(freq, offset_delay, offset_loss, c0, c1, c2, c3, offset_z0, ref_
↪z0):
    # Capacitance is defined with respect to the system reference impedance ref_z0, not␣
↪the
    # lossy line impedance. In scikit-rf, the return values of `shunt_capacitor()` and
    # `medium.open()` methods are (correctly) referenced to z0_port, which has been set to
    # ref_z0.
    medium, line = keysight_calkit_offset_line(freq, offset_delay, offset_loss, offset_
↪z0, ref_z0)
    if c0 or c1 or c2 or c3:
        poly = np.poly1d([c3, c2, c1, c0])
        capacitance = medium.shunt_capacitor(poly(freq.f)) ** medium.open()
    else:
        capacitance = medium.open()
    return line ** capacitance


def keysight_calkit_short(freq, offset_delay, offset_loss, l0, l1, l2, l3, offset_z0,␣
↪ref_z0):
    # Inductance is defined with respect to the system reference impedance ref_z0, not␣
↪the
    # lossy line impedance. In scikit-rf, the return values of `shunt_inductance()` and
    # `medium.short()` methods are (correctly) referenced to z0_port, which has been set␣
↪to
    # ref_z0.
    medium, line = keysight_calkit_offset_line(freq, offset_delay, offset_loss, offset_
↪z0, ref_z0)
    if l0 or l1 or l2 or l3:
        poly = np.poly1d([l3, l2, l1, l0])
        inductance = medium.inductor(poly(freq.f)) ** medium.short()
    else:
        inductance = medium.short()
    return line ** inductance


def keysight_calkit_load(freq, offset_delay=0, offset_loss=0, offset_z0=50, ref_z0=50):
    medium, line = keysight_calkit_offset_line(freq, offset_delay, offset_loss, offset_
↪z0, ref_z0)
    ideal_medium = DefinedGammaZ0(frequency=freq, z0=ref_z0)
    load = ideal_medium.match()
    return line ** load
```

(continues on next page)

(continued from previous page)

```
def keysight_calkit_thru(freq, offset_delay=0, offset_loss=0, offset_z0=50, ref_z0=50):
    medium, line = keysight_calkit_offset_line(freq, offset_delay, offset_loss, offset_
↪z0, ref_z0)
    thru = medium.thru()
    return line ** thru
```

Example: Keysight 85033E

To show another usage example of the code snippet above, we're going to model the Keysight 85033E, 3.5 mm, 50 Ω, DC to 9 GHz calibration kit (plug), with the following coefficients. [22] This is one of the most widely used 3.5 mm calibration standards across labs.

Parameter	Unit	Open	Short	Load	Thru
C_0	10^{-15} F	49.433			
C_1	10^{-27} F/Hz	-310.13			
C_2	10^{-36} F/Hz^2	23.168			
C_3	10^{-45} F/Hz^3	-0.15966			
L_0	10^{-12} H		2.0765		
L_1	10^{-24} H/Hz		-108.54		
L_2	10^{-33} H/Hz^2		2.1705		
L_3	10^{-42} H/Hz^3		-0.01		
Resistance	Ω			50	
Offset Delay	ps	29.243	31.785	0	0
Offset Loss	GΩ / s	2.2	2.36	2.3	2.3
Offset Z_0	Ω	50	50	50	50
Reference Z_0	Ω	50	50	50	50

Note

Although Keysight specified offset losses for the Load and Thru standards, they have offset delays of 0, effectively removing the transmission line, hence these losses should be ignored. The offset Thru standard is meant to be an unknown or flush Thru.

Important

Keysight 85033E's plug and socket calibration standards have mostly identical parameters, but one difference: the Open standard (socket) has an offset loss of 2.3 G/s, but the Open standard (plug) has an offset loss of 2.2 G/s.

Creation

To create this calibration kit using the above code snippet:

[12]:
```
freq = skrf.Frequency(1, 9000, 1001, "MHz")
keysight_85033e_plug_open_std = keysight_calkit_open(
    freq,
```

(continues on next page)

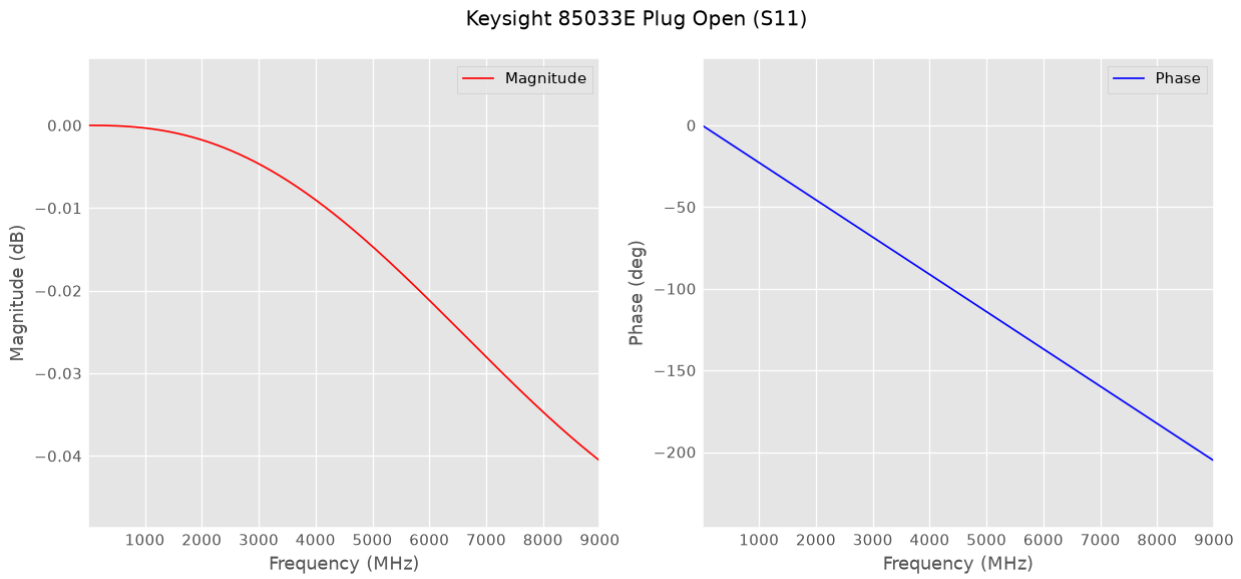
(continued from previous page)

```
    offset_delay=29.243e-12, offset_loss=2.2e9,
    c0=49.433e-15, c1=-310.13e-27, c2=23.168e-36, c3=-0.15966e-45,
    offset_z0=50, ref_z0=50
)
keysight_85033e_plug_short_std = keysight_calkit_short(
    freq,
    offset_delay=31.785e-12, offset_loss=2.36e9,
    l0=2.0765e-12, l1=-108.54e-24, l2=2.1705e-33, l3=-0.01e-42,
    offset_z0=50, ref_z0=50
)
load_std = keysight_calkit_load(freq)
thru_std = keysight_calkit_thru(freq)
```

Plotting

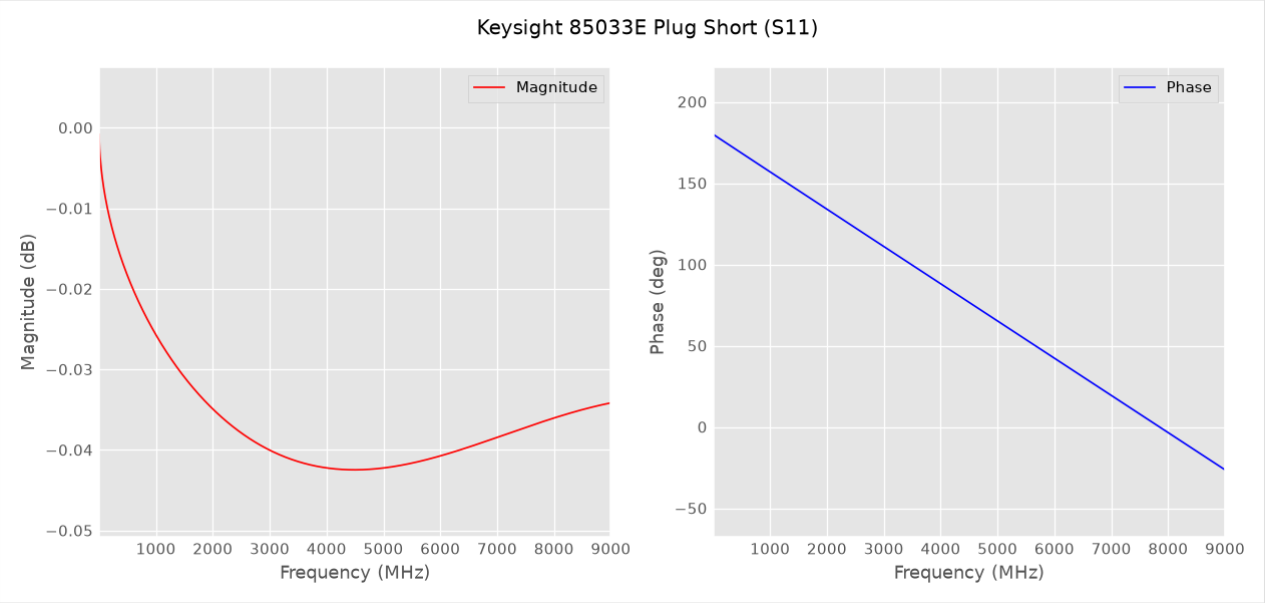
Open

[13]:
```
fig, ax = plt.subplots(1, 2)
fig.set_size_inches(12, 5)

plt.suptitle("Keysight 85033E Plug Open (S11)")
keysight_85033e_plug_open_std.plot_s_db(ax=ax[0], color='red', label="Magnitude")
keysight_85033e_plug_open_std.plot_s_deg_unwrap(ax=ax[1], color='blue', label="Phase")
```



Short

[14]:
```
fig, ax = plt.subplots(1, 2)
fig.set_size_inches(12, 5)

plt.suptitle("Keysight 85033E Plug Short (S11)")
keysight_85033e_plug_short_std.plot_s_db(ax=ax[0], color='red', label="Magnitude")
keysight_85033e_plug_short_std.plot_s_deg_unwrap(ax=ax[1], color='blue', label="Phase")
```



Conclusion

Again, entering the correct offset delay is critical when using an offset calibration standard.

Example: Generic SMA Plug Calkit

And now for something completely different. The author has measured the typical parameters of generic SMA standards with the following data using NanoRFE VNA6000 with respect to the Keysight 85033E calibration standard. The numerically fitted circuit parameters are shown below.

Parameter	Unit	Flush Open	Flush Open	Thru Open
VNA Connector		SMA Socket	3.5 mm Socket	SMA Plug
DUT Connector		SMA Plug	SMA Plug	SMA Thru+Plug
C_0	10^{-15} F	13.670	28.065	13.670
Offset Delay	ps	0	0	47.08
Offset Z_0	Ω	50	50	50
Reference Z_0	Ω	50	50	50

The generic *Short* is essentially a flush short, so only the generic *Open* standard is shown. Due to measurement noise, only a linear fit has been performed. All the above results are preliminary, and has not yet undergone rigorous verification.

Creation

[15]:
```
freq = skrf.Frequency(1, 9000, 1001, "MHz")
generic_sma_plug_on_sma_socket_open_std = keysight_calkit_open(
    freq,
    offset_delay=0, offset_loss=0,
    c0=13.670e-15, c1=0, c2=0, c3=0,
    offset_z0=50, ref_z0=50
)
```

(continues on next page)

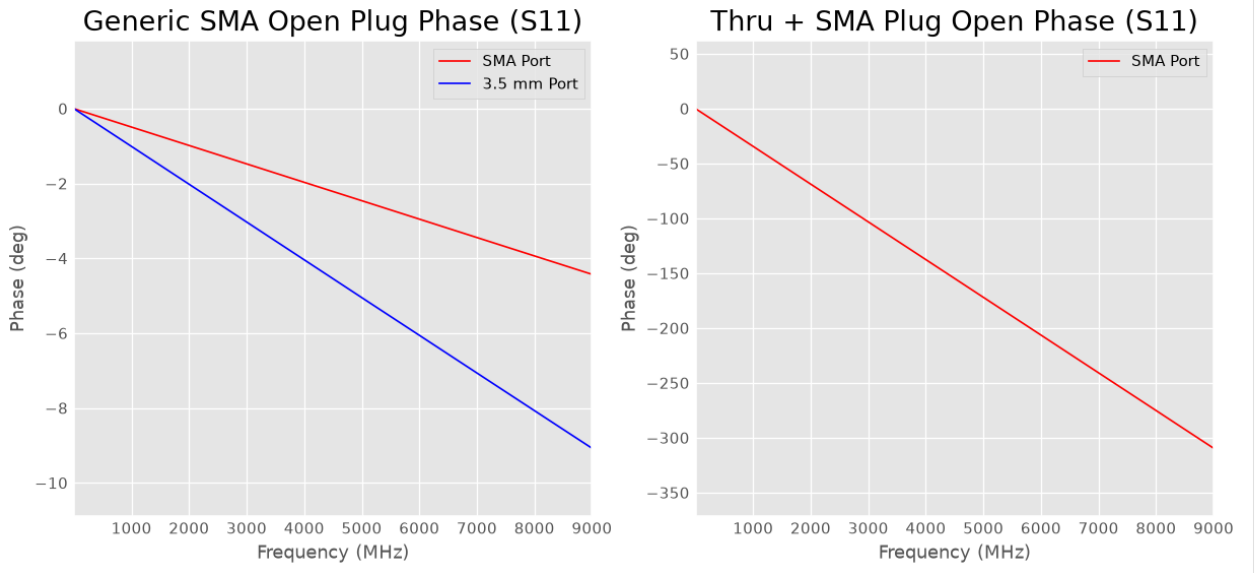
(continued from previous page)

```
generic_sma_plug_on_apc35_socket_open_std = keysight_calkit_open(
    freq,
    offset_delay=0, offset_loss=0,
    c0=28.065e-15, c1=0, c2=0, c3=0,
    offset_z0=50, ref_z0=50
)
generic_sma_plug_on_sma_socket_thru_open_std = keysight_calkit_open(
    freq,
    offset_delay=47.08e-12, offset_loss=0,
    c0=13.670e-15, c1=0, c2=0, c3=0,
    offset_z0=50, ref_z0=50
)
```

Plotting

Flush Open

[16]:
```
fig, ax = plt.subplots(1, 2)
fig.set_size_inches(12, 5)

ax[0].set_title("Generic SMA Open Plug Phase (S11)")
generic_sma_plug_on_sma_socket_open_std.plot_s_deg_unwrap(ax=ax[0], color='red', label=
↪"SMA Port")
generic_sma_plug_on_apc35_socket_open_std.plot_s_deg_unwrap(ax=ax[0], color='blue',␣
↪label="3.5 mm Port")

ax[1].set_title("Thru + SMA Plug Open Phase (S11)")
generic_sma_plug_on_sma_socket_thru_open_std.plot_s_deg_unwrap(ax=ax[1], color='red',␣
↪label="SMA Port")
```



Conclusion

When a generic SMA standard is used to calibrate an SMA socket, the phase error is less than 10 degrees in comparison to an ideal open standard. This justifies the ideal assumption for non-critical measurements.

In comparison, when the generic SMA open standard is cascaded with a generic Thru adapter to form an SMA socket, this effectively converts the standard to an *offset* standard, with a phase deviation in excess of 300 degrees. When calibrating SMA plugs and coaxial cables, the ideal assumption creates large measurement errors. This can be corrected by characterizing the *offset delay* of the Thru standard, equivalent to applying a port extension to all measurements.

When the same generic open standard is measured on SMA and 3.5 mm sockets, significant differences are observed. The author believes a true physical fringe capacitance difference is unlikely. It's more likely due to circuit parasitics, such as the shift of a connector's effective reference plane between its *connected* and *unconnected state* (the calibration is only valid for the *connected* port, not an unterminated one). [2] Connector parasitics such as pin depths [16][17] and step capacitances in a cross-mated 3.5 mm and SMA/2.92 mm pair are likely also involved. [8]

This is the possible reason that a pinless hollow open standard is no longer used in RF measurements (though they found historical metrology uses in APC-7 and Type-N connectors with acceptable results). The lack of a center conductor makes the electrical characteristic of the open standard unstable and port-dependent.

Tip

HP 85032B/E Type-N standard was a historical example with a pinless Open. Kurt Poulsen (OZ7OU) discovered that most generic Type-N standards (plug) circulating on the market are its clones [23], allowing one to use HP's definition [22] in non-critical calibration. However, this is not applicable to socket types due to the lack of matching center conductor extenders.

Rohde & Schwarz / Anritsu Coefficient Format

On Rohde & Schwarz and Anritsu VNAs, a slightly different format is used to define the coefficients. Here's an example of a Maury Microwave 8050CK10, a 3.5 mm, DC to 26.5 GHz calibration kit defined in Rohde & Schwarz's format. [19]

Parameter	Unit	Open	Short	Load	Thru
C_0	10^{-15} F	62.54			
C_1	10^{-15} F/GHz	1284.0			
C_2	10^{-15} F/GHz^2	107.6			
C_3	10^{-15} F/GHz^3	-1.886			
L_0	10^{-12} H		0		
L_1	10^{-12} H/GHz		0		
L_2	10^{-12} H/GHz^2		0		
L_3	10^{-12} H/GHz^3		0		
Resistance	Ω			50	
Offset Length	mm	4.344	5.0017	0	17.375
Offset Loss	dB / $\sqrt{\text{GHz}}$	0.0033	0.0038	0	0.0065

Modeling the Offset Transmission Line

As shown, it's essentially the same circuit model, the only difference is that the offset transmission line is defined in different units of measurements: offset delay is defined as a physical length instead of a time delay, offset loss is defined in decibel. The offset Z_0 is defined to be the system impedance Z_{ref} (50 or 75), thus unlisted.

> **Tip**
>
> Note that a signal travels in both directions in a S_{11} measurement, so the loss from S_{11} in decibel is doubled numerically in comparison to the stated offset loss.

We can reuse the same calculations in the Keysight model after a simple unit conversion using these equations. [15]

$$
\begin{aligned}
Z_{\text{off}} &= Z_{\text{ref}} \\
t_{\text{delay}} &= \frac{D \cdot \sqrt{\epsilon_r}}{c_0} \\
A_{\text{loss}} &= \frac{L \cdot Z_0}{t_{\text{delay}} \cdot 20 \log_{10}(\mathrm{e})}
\end{aligned}
$$

where D and L are the offset length (meter) and offset loss (dB / $\sqrt{\text{GHz}}$) in the R&S model, t_{delay} and A_{loss} are the offset delay (second) and offset loss (Ω / s) in Keysight's model, ϵ_r is the dielectric constant, it's air by definition, thus $\epsilon_r = 1$, and c_0 is the speed of light. The term $20 \log_{10}(\mathrm{e})$ is a conversion from decibel to neper.

[17]:
```
def rs_to_keysight(rs_offset_length, rs_offset_loss, offset_z0=50):
    offset_delay = rs_offset_length / skrf.constants.c
    offset_loss = skrf.mathFunctions.db_2_np(rs_offset_loss * offset_z0 / offset_delay)
    return offset_delay, offset_loss
```

After unit conversion, we can define standards just like how calibration standards in Keysight-style coefficients are defined.

[18]:
```
z0_ref = 50

offset_delay, offset_loss = rs_to_keysight(4.344e-3, 0.0033)
rlcg_open = offset_rlcg(freq, offset_delay, offset_loss, z0_ref)
medium_open = DistributedCircuit(frequency=freq,
    R=rlcg_open[0], L=rlcg_open[1], C=rlcg_open[2], G=rlcg_open[3],
    z0_port=z0_ref
)
line_open = medium_open.line(
    d=1, unit='m'
)

offset_delay, offset_loss = rs_to_keysight(5.0017e-3, 0.0038)
rlcg_short = offset_rlcg(freq, offset_delay, offset_loss, z0_ref)
medium_short = DistributedCircuit(frequency=freq,
    R=rlcg_short[0], L=rlcg_short[1], C=rlcg_short[2], G=rlcg_short[3],
    z0_port=z0_ref
)
line_short = medium_short.line(
    d=1, unit='m'
)

offset_delay, offset_loss = rs_to_keysight(17.375e-3, 0.0065)
rlcg_thru = offset_rlcg(freq, offset_delay, offset_loss, z0_ref)
medium_thru = DistributedCircuit(frequency=freq,
    R=rlcg_thru[0], L=rlcg_thru[1], C=rlcg_thru[2], G=rlcg_thru[3],
    z0_port=z0_ref
)
```

(continues on next page)

(continued from previous page)

```
line_thru = medium_thru.line(
    d=1, unit='m'
)
```

Modeling the Shunt Impedance

The definition of shunt impedance is identical to the Keysight format.

But, beware of the units used for the capacitance and inductance! In the given table, the capacitances are given in 10^{-15} F, 10^{-15} F/GHz, 10^{-15} F/GHz^2, and 10^{-15} F/GHz^3. For Keysight and Anritsu VNAs, they're given in 10^{-15} F, 10^{-27} F/Hz, 10^{-36} F/Hz^2 and 10^{-45} F/Hz^3. Inductance units have the same differences. Always double-check the units before start modeling. To convert the units from the first to the second format, multiply x_1, x_2 and x_3 by 1000 (don't change the constant term x_0). For consistency, we'll use the second format in the code.

Since the inductance in the short standard is neglected, only the capacitance in the open standard is modeled, the short is modeled as ideal.

[19]:
```
capacitor_poly = np.poly1d([
    -0.001886 * 1000e-45,
     0.1076   * 1000e-36,
    -1.284    * 1000e-27,
    62.54     * 1e-15
])
capacitor_open = capacitor_poly(freq.f)
shunt_open = ideal_medium.shunt_capacitor(capacitor_open) ** ideal_medium.open()
# or: shunt_open = ideal_medium.capacitor(capacitor_open) ** ideal_medium.short()
# see the Keysight example for explanation.

shunt_short = ideal_medium.short()
```

Completion

Finally, we connect these model components together.

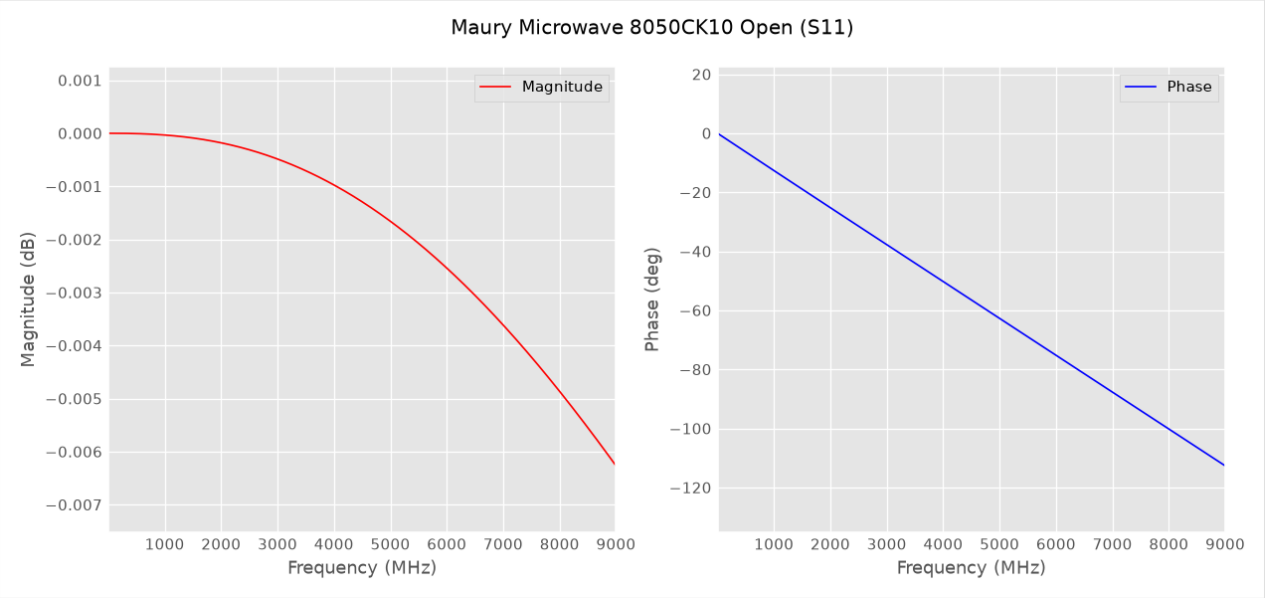
[20]:
```
open_std = line_open ** shunt_open
short_std = line_short ** shunt_short
load_std = ideal_medium.match()
thru_std = line_thru
```

Plotting

Again, let's examine the behaviors of the finished standards.

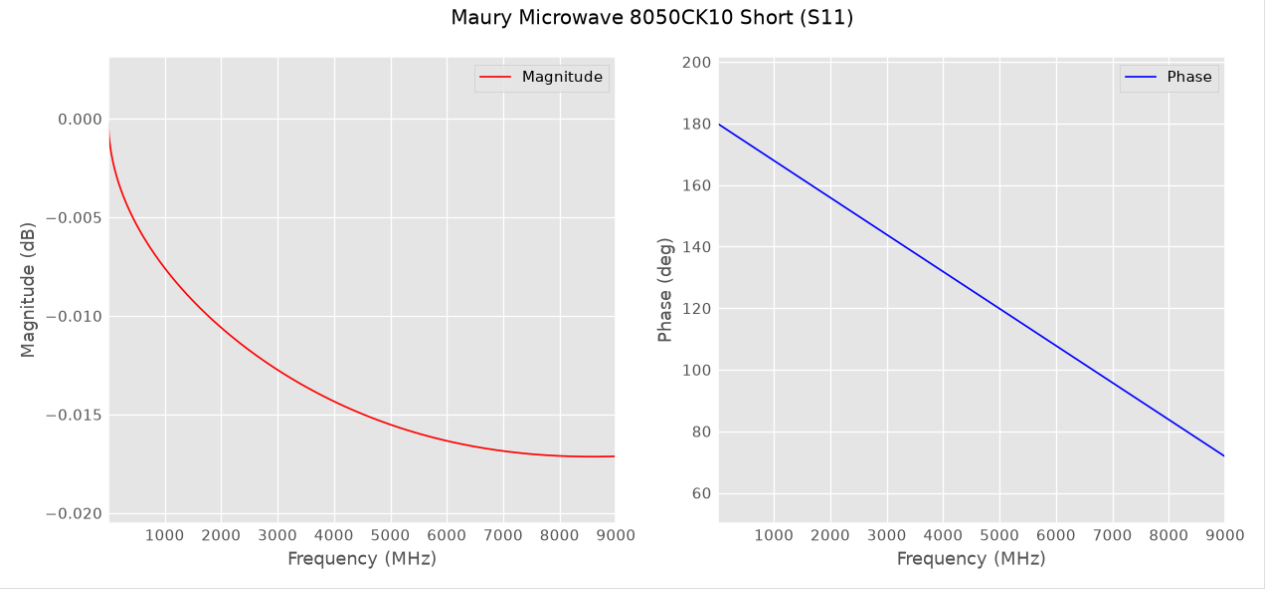
Open

[21]:
```
fig, ax = plt.subplots(1, 2)
fig.set_size_inches(12, 5)

plt.suptitle("Maury Microwave 8050CK10 Open (S11)")
open_std.plot_s_db(ax=ax[0], color='red', label="Magnitude")
open_std.plot_s_deg_unwrap(ax=ax[1], color='blue', label="Phase")
```



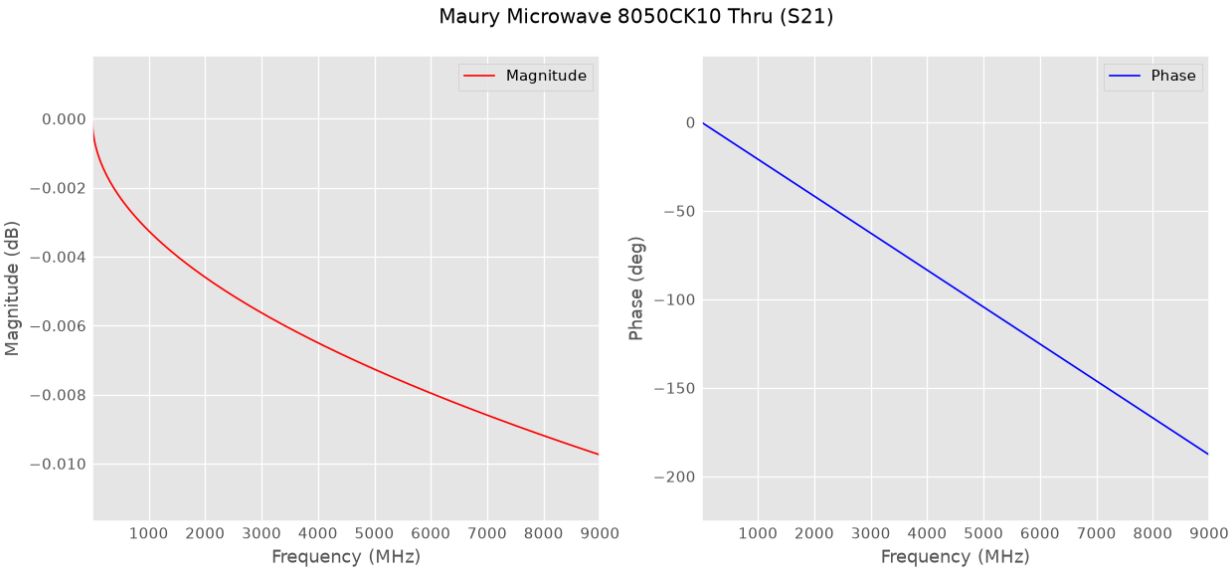
Short

[22]:
```
fig, ax = plt.subplots(1, 2)
fig.set_size_inches(12, 5)

plt.suptitle("Maury Microwave 8050CK10 Short (S11)")
short_std.plot_s_db(ax=ax[0], color='red', label="Magnitude")
short_std.plot_s_deg_unwrap(ax=ax[1], color='blue', label="Phase")
```



Thru

[23]:
```
fig, ax = plt.subplots(1, 2)
fig.set_size_inches(12, 5)

plt.suptitle("Maury Microwave 8050CK10 Thru (S21)")
thru_std.s21.plot_s_db(ax=ax[0], color='red', label="Magnitude")
thru_std.s21.plot_s_deg_unwrap(ax=ax[1], color='blue', label="Phase")
```



Conclusion

The results are similar to the Keysight calibration standards and the generic Thru adapter. The S21 graph for the Thru standard explains why adding an electrical delay sometimes can serve as a crude but usable calibration method ("port extension") for VNA measurements. Again, losses are extremely low, phase shift is the source of non-ideal properties in the *offset* standards.

Code Snippet

For convenience, you can reuse the following code snippet to generate calibration standard networks from coefficients in Rohde & Schwarz and Anritsu format.

```
import numpy as np

import skrf
from skrf.media import DefinedGammaZ0


def keysight_calkit_offset_line(freq, offset_delay, offset_loss, offset_z0, ref_z0):
    if offset_delay or offset_loss:
        r = offset_loss * offset_delay * np.sqrt(freq.f / 1e9)
        l = (offset_delay * offset_z0) + r / (2 * np.pi * freq.f)
        c = offset_delay / offset_z0
        g = 0
```

(continues on next page)

(continued from previous page)

```
        medium = DistributedCircuit(frequency=freq,
            R=r, L=l, C=c, G=g,
            z0_port=ref_z0
        )
        offset_line = medium.line(d=1, unit='m')
        return medium, offset_line
    else:
        medium = DefinedGammaZ0(frequency=freq, z0=ref_z0)
        line = medium.line(d=0)
        return medium, line


def rs_calkit_offset_line(freq, rs_offset_length, rs_offset_loss, offset_z0, ref_z0):
    offset_delay = rs_offset_length / skrf.constants.c
    if offset_delay != 0:
        offset_loss = skrf.mathFunctions.db_2_np(rs_offset_loss * offset_z0 / offset_
↪delay)
    else:
        offset_loss = 0
    return keysight_calkit_offset_line(freq, offset_delay, offset_loss, offset_z0, ref_
↪z0)


def rs_calkit_open(freq, offset_length, offset_loss, c0, c1, c2, c3, offset_z0, ref_z0):
    # Capacitance is defined with respect to the system reference impedance ref_z0, not␣
↪the
    # lossy line impedance. In scikit-rf, the return values of `shunt_capacitor()` and
    # `medium.open()` methods are (correctly) referenced to z0_port, which has been set to
    # ref_z0.
    medium, line = rs_calkit_offset_line(freq, offset_length, offset_loss, offset_z0,␣
↪ref_z0)
    if c0 or c1 or c2 or c3:
        poly = np.poly1d([c3, c2, c1, c0])
        capacitance = medium.shunt_capacitor(poly(freq.f)) ** medium.open()
    else:
        capacitance = medium.open()
    return line ** capacitance


def rs_calkit_short(freq, offset_length, offset_loss, l0, l1, l2, l3, offset_z0, ref_z0):
    # Inductance is defined with respect to the system reference impedance ref_z0, not␣
↪the
    # lossy line impedance. In scikit-rf, the return values of `shunt_inductance()` and
    # `medium.short()` methods are (correctly) referenced to z0_port, which has been set␣
↪to
    # ref_z0.
    medium, line = rs_calkit_offset_line(freq, offset_length, offset_loss, offset_z0,␣
↪ref_z0)
    if l0 or l1 or l2 or l3:
        poly = np.poly1d([l3, l2, l1, l0])
        inductance = medium.inductor(poly(freq.f)) ** medium.short()
    else:
```

(continues on next page)

(continued from previous page)

```
        inductance = medium.short()
    return line ** inductance


def rs_calkit_load(freq, offset_length, offset_loss, offset_z0, ref_z0):
    medium, line = rs_calkit_offset_line(freq, offset_length, offset_loss, ref_z0, ref_
↪z0)
    load = medium.match()
    return line ** load


def rs_calkit_thru(freq, offset_length, offset_loss, offset_z0, ref_z0):
    medium, line = rs_calkit_offset_line(freq, offset_length, offset_loss, offset_z0,␣
↪ref_z0)
    thru = medium.thru()
    return line ** thru


freq = skrf.Frequency(1, 9000, 1001, "MHz")
open_std = rs_calkit_open(
    freq,
    offset_length=4.344e-3, offset_loss=0.0033,
    offset_z0=50, ref_z0=50,
    # Due to unit differences, the numerical values of c1, c2 and c3
    # must be multiplied by 1000 from the R&S datasheet value. For
    # Anritsu, this is not needed. Check the units on your datasheet!
    c0=62.54     * 1e-15,
    c1=-1.284    * 1000e-27,
    c2=0.1076    * 1000e-36,
    c3=-0.001886 * 1000e-45,
)
short_std = rs_calkit_short(
    freq,
    offset_length=5.0017e-3, offset_loss=0.0038,
    offset_z0=50, ref_z0=50,
    l0=0, l1=0, l2=0, l3=0
)
load_std = rs_calkit_load(
    freq,
    offset_length=0, offset_loss=0,
    offset_z0=50, ref_z0=50,
)
thru_std = rs_calkit_thru(
    freq,
    offset_length=17.375e-3, offset_loss=0.0065,
    offset_z0=50, ref_z0=50,
)
```

Appendix: Derivation of Offset Transmission Line γ and Z_c

Derivation of RLCG Line Parameters

According to Keysight [4], equation 2B.3, the *offset* transmission line parameters are defined with respect to a physical coaxial line using the following relations:

$$
\begin{aligned}
A_{\text{loss}} &= \frac{R_0 v}{\sqrt{\epsilon_r \frac{f}{1\text{ GHz}}}} \\
t_{\text{delay}} &= \frac{l\sqrt{\epsilon_r}}{v} = \sqrt{L_0 C} \\
Z_{\text{off}} &= \frac{\mu_0 v}{2\pi\sqrt{\epsilon_r}} \ln(\frac{D}{d}) = \sqrt{\frac{L_0}{C}}
\end{aligned}
$$

where $v = c_0$ [13] is the speed of light, μ_0 is the vacuum permeability ϵ_r is the medium's relative permittivity, R_0 and L_0 are the distributed nominal resistance and perfect-conductor inductance, C is the distributed capacitance, D and d are the outer and inner diameters of the coaxial line.

By algebraic manipulation:

$$
\begin{aligned}
\left(R_0\sqrt{\frac{f}{10^9}}\right) l = A_{\text{loss}} t_{\text{delay}} &= \frac{R_0 v}{\sqrt{\epsilon_r \frac{f}{10^9}}} \frac{l\sqrt{\epsilon_r}}{v} \\
L_0 = t_{\text{delay}} Z_{\text{off}} &= \sqrt{L_0 C}\sqrt{\frac{L_0}{C}} \\
C = \frac{t_{\text{delay}}}{Z_{\text{off}}} &= \frac{\sqrt{L_0 C}}{\sqrt{\frac{L_0}{C}}}
\end{aligned}
$$

Finally, apply definitions in [4], equation 2B.2:

$$
\begin{aligned}
G &= 0 \\
L &= L_0 + \frac{R}{\omega}
\end{aligned}
$$

The full RLCG line parameters are derived as follows:

$$
\begin{aligned}
R(f)l &= (A_{\text{loss}} t_{\text{delay}}) \cdot \sqrt{\frac{f}{10^9}} \\
L(f) &= L_0 + L_{\text{cond}} = t_{\text{delay}} Z_{\text{off}} + \frac{R}{2\pi f} \\
C &= \frac{t_{\text{delay}}}{Z_{\text{off}}} \\
G &= 0 \\
l &= 1 \text{ (normalized)}
\end{aligned}
$$

Keysight's Low-Loss Approximation

One can calculate the line constant using the general formula from RLCG parameters.

$$
\begin{aligned}
Z_c &= \sqrt{\frac{R + j\omega L}{G + j\omega C}} \\
\gamma &= \sqrt{(R + j\omega L)(G + j\omega C)}
\end{aligned}
$$

This is the method used in scikit-rf's `DistributedCircuit` in general.

However, Keysight's official publication [4] and other works [15][20] used a low-loss approximation to calculate the line constants. We show both the formulas and code to manually calculate γ and Z_c using the same low-loss approximation. This can be useful if identical numerical values to the last digit are absolutely required.

Regroup the terms in γ's expression (see [21], equation 2.83):

$$\begin{aligned}\gamma &= \sqrt{(R+j\omega L)(G+j\omega C)} \\ &= j\omega\sqrt{LC}\sqrt{1-j\left(\frac{R}{\omega L}+\frac{G}{\omega C}\right)-\frac{RG}{\omega^2 LC}}\end{aligned}$$

When $G=0$:

$$\begin{aligned}Z_c &= \sqrt{\frac{R+j\omega L}{j\omega C}} \\ \gamma &= j\omega\sqrt{LC}\sqrt{1-j\left(\frac{R}{\omega L}\right)}\end{aligned}$$

Since $L = L_0 + \frac{R}{\omega}$:

$$\begin{aligned}Z_c &= \sqrt{\frac{R+j\omega\left(L_0+\frac{R}{\omega}\right)}{j\omega C}} \\ &= \sqrt{\frac{L_0}{C}}\sqrt{1+\frac{R}{\omega L_0}(1-j)} \\ \gamma &= j\omega\sqrt{\left(L_0+\frac{R}{\omega}\right)C}\sqrt{1-j\left(\frac{R}{\omega\left(L_0+\frac{R}{\omega}\right)}\right)} \\ &= j\omega\sqrt{L_0 C}\sqrt{1+\frac{R}{\omega L_0}(1-j)}\end{aligned}$$

Apply the first-order Taylor expansion $\sqrt{1+x} = 1 + x/2$ [20]:

$$\begin{aligned}\gamma &= j\omega\sqrt{L_0 C}\left[1+(1-j)\frac{R}{2\omega L_0}\right] \\ Z_c &= \sqrt{\frac{L_0}{C}}\left[1+(1-j)\frac{R}{2\omega L_0}\right]\end{aligned}$$

By plugging R, L_0, C into these equations, one can eventually obtain the official formulas in [4]:

$$\begin{aligned}\alpha l &= \frac{A_{\text{loss}} t_{\text{delay}}}{2Z_{\text{off}}}\sqrt{\frac{f}{10^9}} \\ \beta l &= \omega t_{\text{delay}} + \alpha l \\ \gamma l &= \alpha l + j\beta l \\ Z_c &= Z_{\text{off}} + (1-1j)\left(\frac{A_{\text{loss}}}{4\pi f}\right)\sqrt{\frac{f}{10^9}}\end{aligned}$$

Implementation

The following code implements Keysight's method for calculating line constants.

```
def offset_gamma_and_zc(offset_delay, offset_loss, offset_z0):
    alpha_l = (offset_loss * offset_delay) / (2 * offset_z0)
    alpha_l *= np.sqrt(freq.f / 1e9)
    beta_l = 2 * np.pi * freq.f * offset_delay + alpha_l
    gamma_l = alpha_l + 1j * beta_l
    zc = (offset_z0) + (1 - 1j) * (offset_loss / (4 * np.pi * freq.f)) * np.sqrt(freq.f /
↪ 1e9)
    return gamma_l, zc
```

Code Snippet

Replace `keysight_calkit_offset_line()` in the code snippet above with the following definition to replicate the exact Keysight data:

```
def keysight_calkit_offset_line(freq, offset_delay, offset_loss, offset_z0, ref_z0):
    if offset_delay or offset_loss:
        alpha_l = (offset_loss * offset_delay) / (2 * offset_z0)
        alpha_l *= np.sqrt(freq.f / 1e9)
        beta_l = 2 * np.pi * freq.f * offset_delay + alpha_l
        zc = offset_z0 + (1 - 1j) * (offset_loss / (4 * np.pi * freq.f)) * np.sqrt(freq.
↪f / 1e9)
        gamma_l = alpha_l + beta_l * 1j

        medium = DefinedGammaZ0(frequency=freq, z0=zc, gamma=gamma_l, z0_port=ref_z0)
        offset_line = medium.line(d=1, unit='m')
        return medium, offset_line
    else:
        medium = DefinedGammaZ0(frequency=freq, z0=ref_z0)
        line = medium.line(d=0)
        return medium, line
```

References

[1] "Agilent Technologies 85056A 2.4 mm precision calibration kit," Agilent Technologies, User's and Service Guide 85056–90020, Jan. 2002.

[2] N. M. Ridler, "Connectors, air lines and RF impedance," in 14th IEE Microwave Measurements Training Course, 2005, p. 9/1-9/18. "*the test port connector's mating mechanism affects the established measurement reference plane which limits accurate characterisation as a standard. [...] problem can be overcome either by depressing the mating mechanism using a dielectric plug or attaching a length of line to the centre conductor, terminated in an abrupt truncation. The dielectric plug technique is used as a standard with numerous VNA calibration kits.*"

[3] IEEE standard for precision coaxial connectors (DC to 110 GHz), IEEE 287-2007, 2007. "*The reference plane for the Type N connector is the junction surface of the outer conductors [...] Unlike many of the other pin and socket connectors, the junction surface of the inner conductor is offset from the reference plane [...] The offset specifications in this document for the inner conductor pin differ from other common standards documents. Those documents specify the offset of the inner conductor pin as 5.283 mm through 5.360 mm (0.208 in through 0.211 in). This standard specifies the offset as 5.258 mm through 5.360 mm (0.207 in through 0.211 in).*"

[4] "Specifying calibration standards and kits for Agilent vector network analyzers," Agilent Technologies, Application Note 1287–11, Aug. 2009. See Equation 36, 37 for the propagation constant formulas of the offset transmission line.

[5] P. J. Pupalaikis, S-parameters for Signal Integrity. Cambridge: Cambridge University Press, 2020. Page 481. Written by a former signal integrity expert at LeCroy, the author of this tutorial recommends everyone who simulates

or measures RF/microwave devices to get a copy. Each concept is accomplished by both formulas and executable code, making it an invaluable reference in this field.

[6] "Agilent Technologies 85033D 3.5 mm calibration kit," Agilent Technologies, User's and Service Guide 85033–90027, July 2002. "*The offset opens have inner conductors that are supported by a strong, low-dielectric constant plastic to minimize compensation values.*"

[7] R. B. Marks and D. F. Williams, "A general waveguide circuit theory," Journal of research of the National Institute of Standards and Technology, vol. 97, no. 5, pp. 533–562, 1992. "*This potentially useful result suggests that a particular line may be used as a calibration standard for any network analyzer with identical results. However, the assumption that v and i are continuous, which led to the result, is not generally valid. The example of a 50 Ω, 2.4 mm coaxial standard used on 50 Ω, 3.5 mm coaxial test ports makes this clear, for the standard must reflect the traveling waves even though its characteristic impedance is appropriate for a reflectionless standard. In general, the quality of the approximation depends in detail on the nature of the waveguide interface.*"

[8] R. D. Pollard, "Compensation technique improves measurements for a range of mechanically compatible connectors," Microwave Journal, vol. 37, no. 10, p. 91+, Oct. 1994.

[9]: J. R. Juroshek, C. A. Hoer, and R. F. Kaiser, "Calibrating network analyzers with imperfect test ports," IEEE Transactions on Instrumentation and Measurement, vol. 38, no. 4, pp. 898–901, Aug. 1989, doi: 10.1109/19.31010.

[10]: P. I. Somlo, "The discontinuity capacitance and the effective position of a shielded open circuit in a coaxial line," Proceedings of the Institution of Radio and Electrical Engineers Australia, vol. 28, no. 1, pp. 7–9, Jan. 1967, doi: 10.5281/zenodo.17015632, alt link.

[11] N. M. Ridler and M. Salter, "Measuring the capacitance coefficients of coaxial open-circuits with traceability to national standards," Microwave Journal, vol. 49, pp. 138–154, Oct. 2006.

[12] N. M. Ridler, J. C. Medley, A. J. B. Fuller, and M. Runham, "Computer generated equivalent circuit models for coaxial-line offset open circuits," IEE Proceedings A (Science, Measurement and Technology), vol. 139, no. 5, pp. 229–231, 1992.

[13] "Specifying calibration standards for the HP 8510 network analyzer," Hewlett-Packard, Product Note 8510–5, Mar. 1986. *Note the absence of any inductance parameters in both the text and the calibration standard datasheet on the last page.*

[14] "Agilent Technologies 85052D 3.5 mm economy calibration kit," Agilent Technologies, User's and Service Guide 85052–90079, Aug. 2010.

[15] M. Wollensack and J. Hoffmann, "METAS VNA Tools - Math Reference V2.9.0." Federal Institute of Metrology (METAS), Aug. 08, 2025. See Page 26, 27 for formulas of the Keysight and R&S coefficients. The Keysight formulas are equivalent to [4].

[16] K. Wong and J. Hoffmann, "Improving VNA measurement accuracy by including connector effects in the models of calibration standards," in 82nd ARFTG Microwave Measurement Conference, Nov. 2013, pp. 1–7.

[17] J. Ruefenacht and J. Hoffmann, "Coaxial connector effects," presented at the EURAMET RF&MW expert meeting, Torino, May 2011. doi: 10.13140/2.1.3666.7685.

[18] P. I. Somlo, "The computation of coaxial line step capacitances," IEEE Transactions on Microwave Theory and Techniques, vol. 15, no. 1, pp. 48–53, Jan. 1967.

[19] "3.5 mm Coaxial Calibration Kit - DC to 26.5 GHz - Models: 8050CK10/11 - 8050CK20/21," Maury Microwave, User Guide 8050–511, 2015. Source of the Maury Microwave 8050CK10 example. The coefficients of this calibration kit are given in multiple formats (Anritsu, Keysight, and R&S). You can compare their differences.

[20] R. Monsalve, "Effect of loss on VNA calibration standards," SESE, Arizona State University, Aug. 2013.

[21] D. M. Pozar, Microwave engineering, Fourth edition. Hoboken, NJ: John Wiley & Sons, Inc, 2012.

[22] "Keysight vector network analyzer calibration kit standards definitions." Keysight Technologies, 2024.

[23] K. Poulsen, "How to fabricate a N male and female calibration kit with arbitrary calibration data for VNWA 2 and VNWA3." Feb. 2014.

[24] J. Hoffmann, M. Wollensack, J. Ruefenacht, and M. Zeier, "Extended S-parameters for imperfect test ports," Metrologia, vol. 52, no. 1, p. 121, Jan. 2015, doi: 10.1088/0026-1394/52/1/121.

[25] J. Ruefenacht and J. Hoffmann, "Coaxial connector effects," presented at the EURAMET RF&MW expert meeting, Torino, May 2011. doi: 10.13140/2.1.3666.7685.

[26] K. Wong and J. Hoffmann, "Improving VNA measurement accuracy by including connector effects in the models of calibration standards," in 82nd ARFTG Microwave Measurement Conference, Nov. 2013, pp. 1–7. doi: 10.1109/ARFTG-2.2013.6737334.

2.3.10 TRL

This little example demonstrates how to use scikit-rf for a basic TRL calibration.

Basic Use

Imports

First we import `skrf`, the TRL class, and setup matplotlib plotting.

[1]:
```
import skrf as rf
from skrf.calibration import TRL

%matplotlib inline
from skrf.plotting import stylely

stylely()
```

Load the data

Load the calibration standard's measurements and the switch terms from touchstone files. (If you dont know what switch terms are, see *this section below*.)

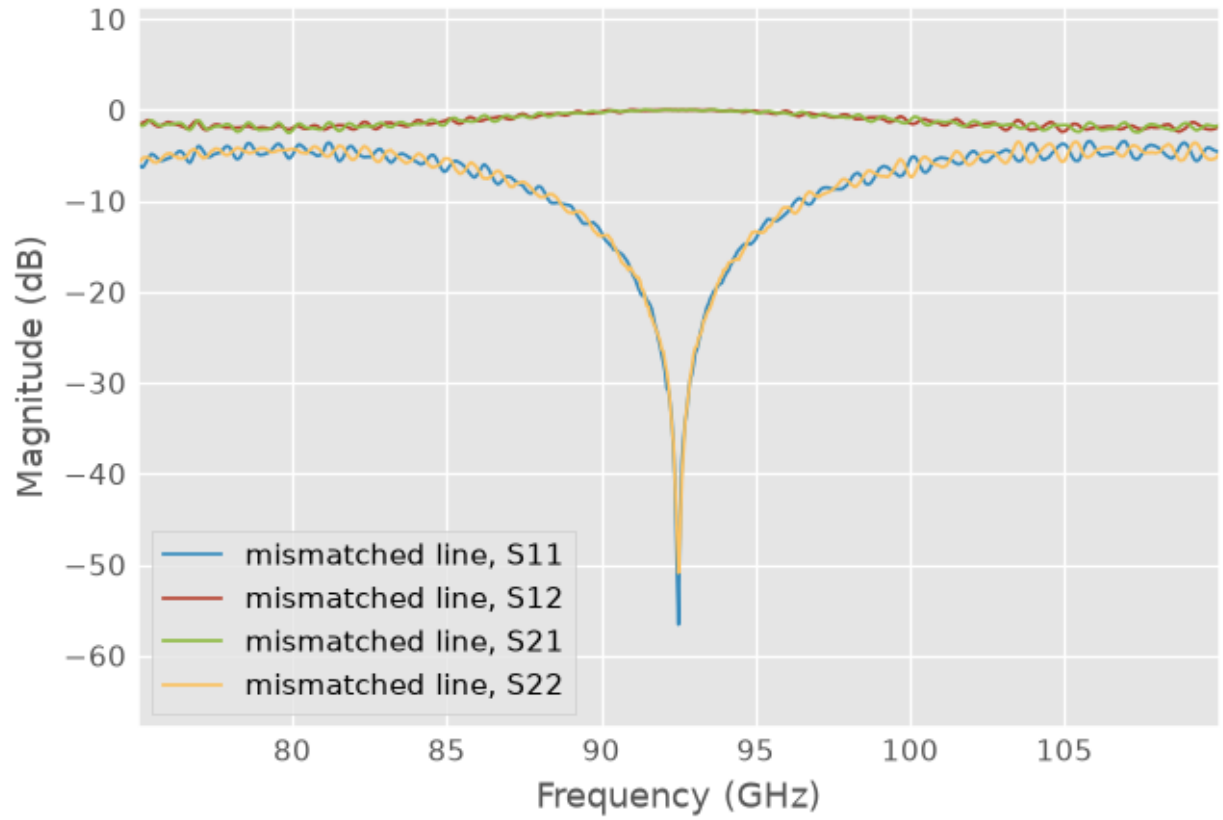
[2]:
```
T = rf.Network('trl_data/thru.s2p')
R = rf.Network('trl_data/reflect.s2p')
L = rf.Network('trl_data/line.s2p')

switch_terms = (rf.Network('trl_data/forward switch term.s1p'),
                rf.Network('trl_data/reverse switch term.s1p'))

measured = [T,R,L]
```

Create a TRL calibration Object

[3]:
```
trl = TRL(measured = measured,
          switch_terms = switch_terms)
```

Correct a DUT with it.

[4]:
```
dut_raw = rf.Network('trl_data/mismatched line.s2p')
dut_corrected = trl.apply_cal(dut_raw)
dut_corrected.plot_s_db()
```



Switch Terms

What are they?

Described in [1] by Roger Marks, switch-terms account for the fact that the 8-term (aka error-box) model is overly simplified. The two error networks change slightly depending on which port is being excited. This is due to the internal switch within the VNA. The change in reflection coefficient at the VNA's internal switch is known as a *switch term*, it can be directed measured and removed.

- [1] R.B. Marks, "Formulations of the Basic Vector Network Analyzer Error Model including Switch-Terms," in ARFTG Conference Digest-Fall, 50th, 1997, vol. 32, pp. 115-126

How do I get them ?

To measure the switch terms manually, you can define custom traces as ratio's of receivers and save the data to touchstone files. Assume ports are labeled 1 and 2, and incident waves are a's and reflected waves are b's. Then, the

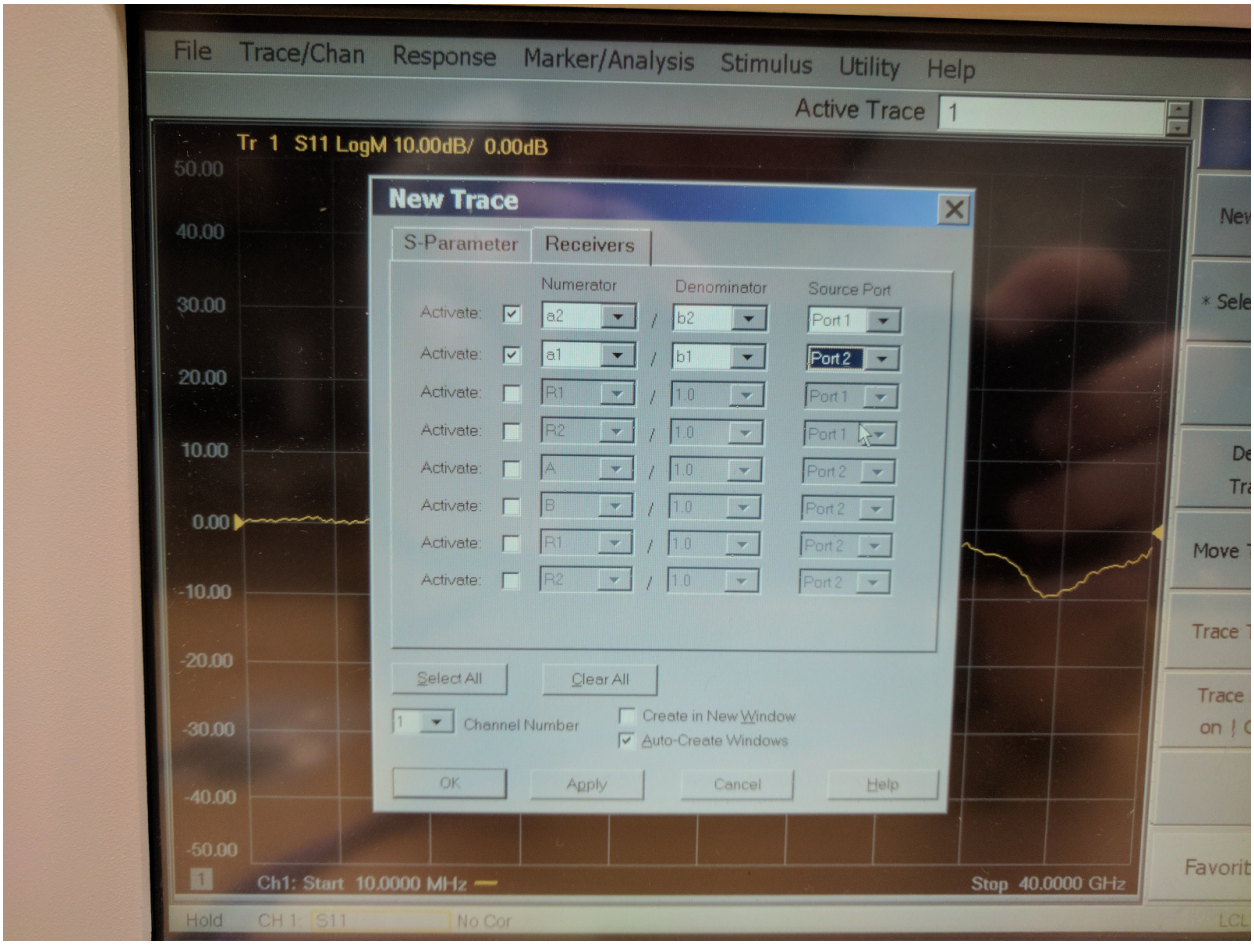
- **forward switch term** == `a2/b2` with `source port 1`
- **reverse switch term** == `a1/b1` with `source port 2`

Can I get them with skrf?

Yes. There is a PNA driver in the `skrf.vi.vna module`. It has code to pull the switch terms.

Can get them Onboard a PNA?

Yes. See the image below for an example of setting this up on a PNA.



2.3.11 TwoPortOnePath, EnhancedResponse, and FakeFlip

Intro

This example demonstrates a macgyver-ish shortcut you can take if you are measuring a device that is ** reciprocal** and **symmetric** on a switch-less three-receiver system. For more information about error correction this type of architecture, see *Calibration With Three Receivers*.

In general, full error correction of a 2-port network on a switchless three-receiver architecture requires each DUT to measured in two orientations. However, if the DUT is known to be reciprocal ($S_{21} = S_{12}$) and symmetric ($S_{11} = S_{22}$), then measurements in both orientations produce the same response, and therefore are unnecessary.

The following worked example compares the corrected response of a 10dB attenuator at WR-12 as corrected using full error correction and pseudo-full error correction using:

1. Full Correction
2. Pseudo-Full Correction (FakeFlip)

3. Partial (EnhancedResponse)

[1]:
```
from IPython.display import Image

Image('three_receiver_cal/pics/macgyver.jpg', width='50%')
```

[1]:

Example

These measurements where taken on a Agilent PNAX with a set of VDI WR-12 TXRX-RX Frequency Extender heads. The measurements of the calibration standards and DUT's were downloaded from the VNA by saving touchstone files of the raw s-parameter data to disk.

In the code that follows a TwoPortOnePath calibration is created from corresponding measured and ideal responses of the calibration standards. The measured networks are read from disk, while their corresponding ideal responses are generated using scikit-rf. More information about using scikit-rf to do offline calibrations can be found *here*.

[2]:
```
import matplotlib.pyplot as plt

import skrf as rf

%matplotlib inline

from skrf.calibration import TwoPortOnePath
from skrf.constants import mil
from skrf.media import RectangularWaveguide
from skrf.network import two_port_reflect as tpr

rf.stylely()

raw = rf.io.general.read_all_networks('three_receiver_cal/data/')



# pull frequency information from measurements
```

(continues on next page)

(continued from previous page)

```
frequency = raw['short'].frequency

# the media object
wg = RectangularWaveguide(frequency=frequency, a=120*mil, z0_override=50)

# list of 'ideal' responses of the calibration standards
ideals = [wg.short(nports=2),
          tpr(wg.delay_short( 90,'deg'), wg.match()),
          wg.match(nports=2),
          wg.thru()]

# corresponding measurements to the 'ideals'
measured = [raw['short'],
            raw['quarter wave delay short'],
            raw['load'],
            raw['thru']]

# the Calibration object
cal = TwoPortOnePath(measured = measured, ideals = ideals )
```

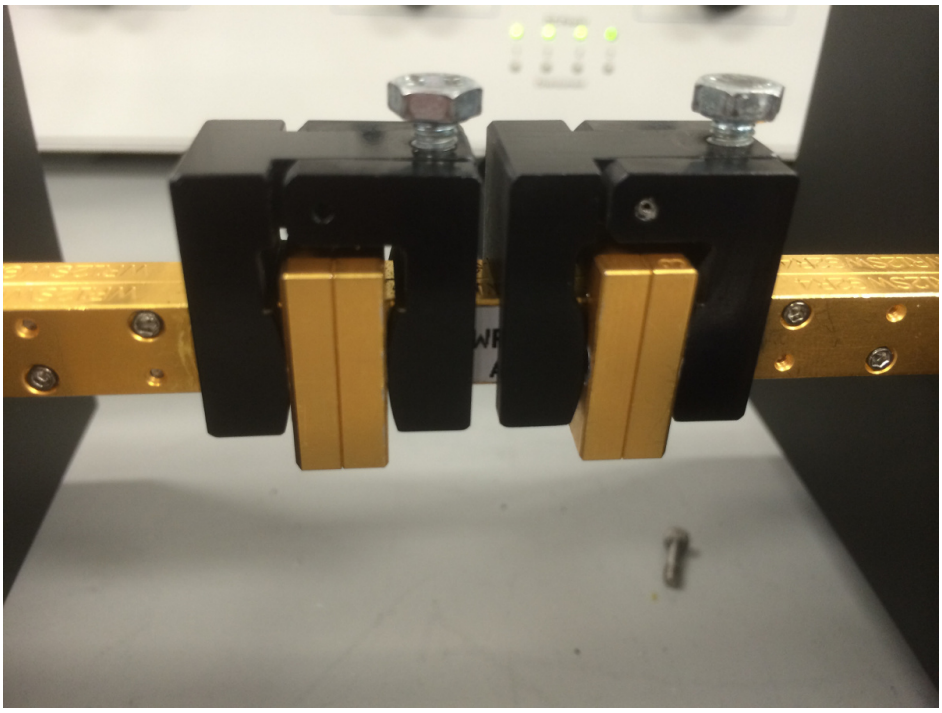
```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:2073: UserWarning: n_thrus is None, guessing which stds are␣
↪transmissive
  TwelveTerm.__init__(self,*args, **kwargs)
```

Correction Options

With the calibration created above, we compare the corrected response of WR-12 10dB attenuator using **Full**, **Pseudo-Full**, and **Partial** Correction. Each correction algorithm is described below.

[3]:
```
Image('three_receiver_cal/pics/symmetric DUT.jpg', width='75%')
```

[3]:



Full Correction (TwoPortOnePath)

Full correction on this type of architecture has been called *TwoPortOnePath*. In `scikit-rf` using this correction algorithm requires the device to be measured in both orientations, **forward** and **reverse**, and passing them both to the `apply_cal()` function as a `tuple`. Neglecting the connector uncertainty, this type of correction is identical to full two-port **SOLT** calibration.

Pseudo-full Correction (FakeFlip)

If we assume the DUT is **reciprocal** and **symmetric**, then measuring the device in both orientations will produce the same result. Therefore, the reverse orientation measurement may be replaced by a copy of the forward orientation measurement. We refer to this technique as the *Fake Flip*.

Warning: Be sure that you understand the assumptions of reciprocity and symmetry before using this macgyver technique, incorrect usage can lead to nonsense results.

Partial Correction (EnhancedResponse)

If you pass a single measurement to the `apply_cal()` function, then the calibration will employ partial correction. This type of correction is known as `EnhancedResponse`. While the *Fake Flip* technique assumes the device is reciprocal and symmetric, the `EnhancedResponse` algorithm *implicitly* assumes that the port 2 of the device is perfectly matched. The accuracy of the corrected result produced with either of these algorithms depends on accuracy of the assumptions.

Comparison

[4]:
```
dutf = raw['attenuator (forward)']
dutr = raw['attenuator (reverse)']


# note the correction algorithm is different depending on what is passed to
```

(continues on next page)

(continued from previous page)

```
# apply_cal
corrected_full =     cal.apply_cal((dutf, dutr))
corrected_fakeflip = cal.apply_cal((dutf,dutf))
corrected_partial =  cal.apply_cal(dutf)



f, ax = plt.subplots(2,2, figsize=(8,8))

for m in [0,1]:
    for n in [0,1]:
        ax_ = ax[m,n]
        ax_.set_title('$S_{%i%i}$'%(m+1,n+1))
        corrected_full.plot_s_db(m,n, label='Full Correction',ax=ax_ )
        corrected_fakeflip.plot_s_db(m,n, label='Pseudo-full Correction', ax=ax_)
        if n==0:
            corrected_partial.plot_s_db(m,n, label='Partial Correction', ax=ax_)


plt.tight_layout()
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 60.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 61.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 62.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 63.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 64.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 65.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 66.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 67.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 68.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 69.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 70.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 71.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 72.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 73.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 74.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 75.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 76.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 77.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 78.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 79.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 80.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 81.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 82.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 83.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 84.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 85.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 86.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 87.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 88.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.0416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.0833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.125 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.1666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.2083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.25 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.2916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.3333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.375 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.4166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.4583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.5 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.5416666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
```

(continues on next page)

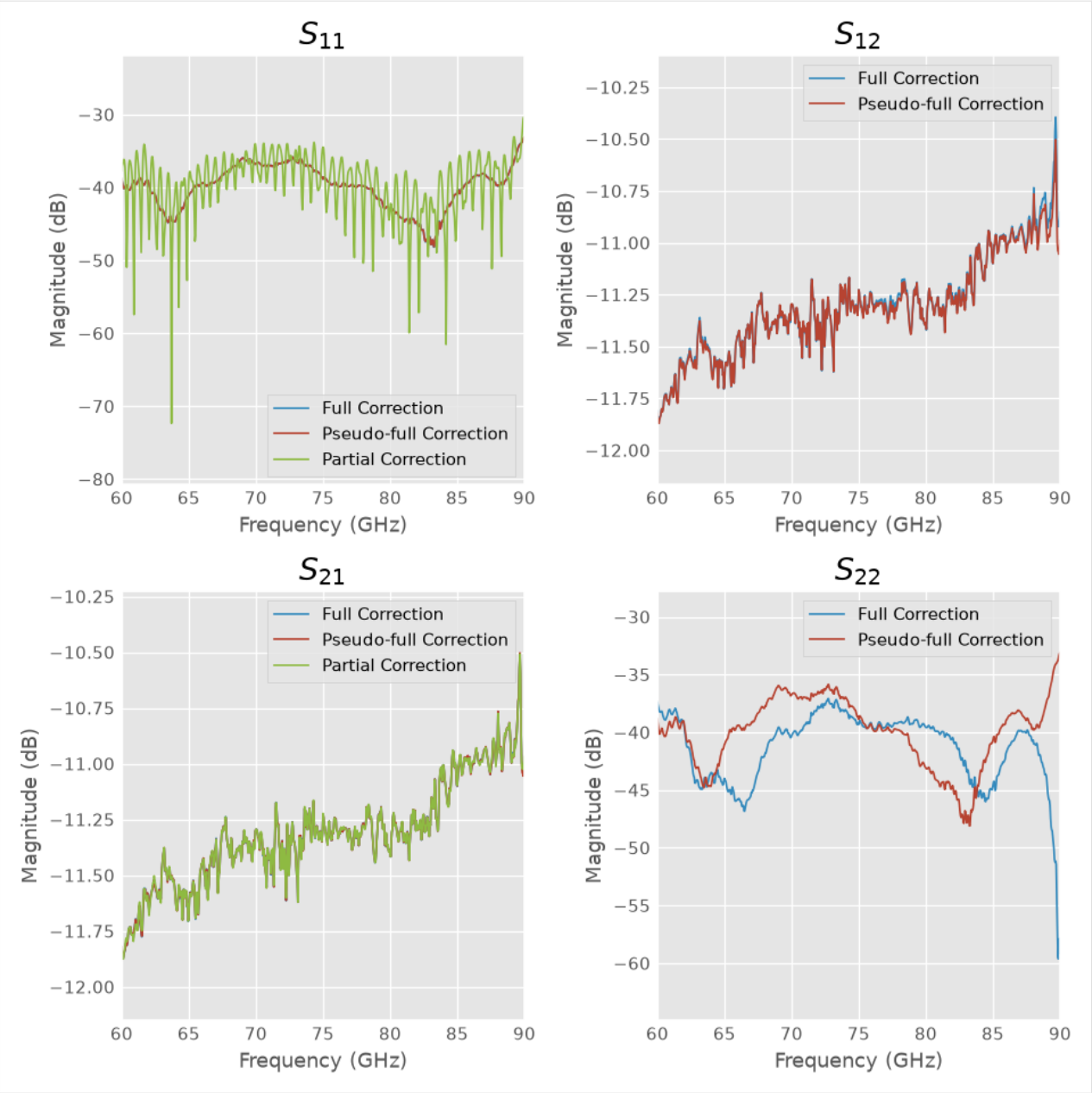
(continued from previous page)

```
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.5833333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.625 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.6666666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.7083333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.75 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.7916666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.8333333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.875 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.9166666667␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 89.9583333333␣
↪GHz. Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:575: UserWarning: The system is singular at 90.0 GHz.␣
↪Ensure that the standards are far enough away from each other on the Smith chart.
  self.run()
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:1221: RuntimeWarning: divide by zero encountered in divide
  x = (vh.conj().swapaxes(-1, -2)/s[:, None, :])@(u.conj().swapaxes(-1, -2)@b)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:1221: RuntimeWarning: invalid value encountered in divide
  x = (vh.conj().swapaxes(-1, -2)/s[:, None, :])@(u.conj().swapaxes(-1, -2)@b)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
```

(continues on next page)

(continued from previous page)

```
↪calibration/calibration.py:1221: RuntimeWarning: invalid value encountered in matmul
  x = (vh.conj().swapaxes(-1, -2)/s[:, None, :])@(u.conj().swapaxes(-1, -2)@b)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/network.
↪py:7065: RuntimeWarning: invalid value encountered in divide
  tmp_a = Ael * (Alk / det) + Aek * (All / det)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/network.
↪py:7066: RuntimeWarning: invalid value encountered in divide
  tmp_b = Ael * (Akk / det) + Aek * (Akl / det)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:6753: RuntimeWarning: invalid value encountered in divide
  gamma_f = (Elf - Esr)/(Err + Edr*(Elf - Esr))
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:6754: RuntimeWarning: invalid value encountered in divide
  gamma_r = (Elr - Esf)/(Erf + Edf*(Elr - Esf))
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:6756: RuntimeWarning: invalid value encountered in divide
  k_first  =   Etf/(Err + Edr*(Elf  - Esr) )
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪calibration/calibration.py:6757: RuntimeWarning: invalid value encountered in divide
  k_second =1/(Etr/(Erf + Edf *(Elr - Esf)))
/tmp/ipykernel_4394/1466338557.py:9: UserWarning: only gave a single measurement␣
↪orientation, error correction is partial without a tuple
  corrected_partial =  cal.apply_cal(dutf)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪mathFunctions.py:303: RuntimeWarning: divide by zero encountered in log10
  out = 20 * np.log10(z)
```



[ ]:

2.4 NetworkSets

2.4.1 Export Network Set as Generalized MDIF File

Frequently a set of `Networks` is recorded while changing some other parameters; like temperature, voltage, current, etc. Once this set of data acquired, it is sometime useful to combine all the networks into a single Generalized MDIF file for use in CAD tools like AWR Microwave Office.

[1]:
```
import pathlib
import tempfile
import zipfile

import numpy as np
import requests

import skrf as rf
```

Narda 3752 phase shifter

In this example, we are characterizing an old narda phase shifter 3752 at 1.5 GHz.

In order to deduce the phase shift that one can obtain at this specific frequency, we have measured scattering parameters in the 1-2 GHz band at 19 positions of the phase knob (from 0 to 180). These measurements are loaded into a *NetworkSets* object:

[2]:
```
# Array containing the 19 phase shift indicator values
indicators_mes = np.linspace(0, 180, num=19)  # from 0 to 180 per 10
```

[3]:
```
ntw_set = rf.networkSet.NetworkSet.from_zip('phase_shifter_measurements/phase_shifter_
↪measurements.zip')
print('ntw_set contains', len(ntw_set), 'networks')
```

```
ntw_set contains 19 networks
```

The content of the NetworkSet can be exported into a MDIF file:

[4]:
```
ntw_set.write_mdif("phase_shifter.mdif")
```

Note that is possible to tune the parameters, values and types defined in the MDIF file by passing the optional parameters `values` and `data_types`. Hence, making "indicator" an MDIF variable of type "double" and save the NetworkSet to "phase_shifter.mdif":

[5]:
```
values = {"indicator": indicators_mes}
data_types = {"indicator": "double"}
ntw_set.write_mdif("phase_shifter.mdif", values, data_types)
```

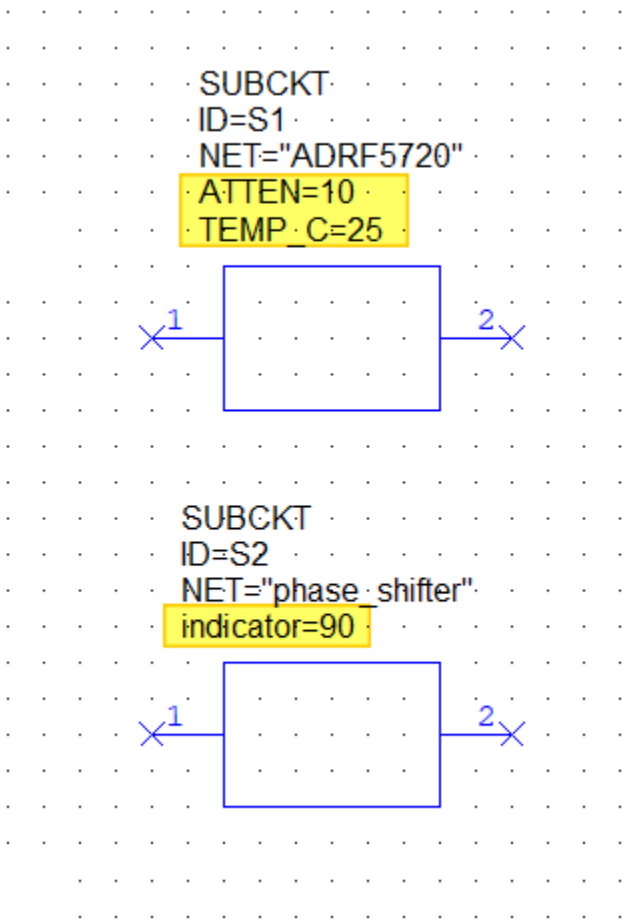
ADRF5720

ADRF5720 is 6-bit 9 kHz to 40 GHz digital step attenuator and is measured over temperature which results in many Touchstone files.

[6]:
```
# download the zip archive of ADRF5720 Touchstone files
url = "https://www.analog.com/media/en/simulation-models/s-parameters/ADRF5720_
↪Sparameters.zip"
try:
    response = requests.get(url, timeout=10)
    open("ADRF5720.zip", "wb").write(response.content)
    tmpdir = pathlib.Path(tempfile.mkdtemp())
    zf = zipfile.ZipFile("ADRF5720.zip")
    zf.extractall(path = tmpdir)
    zf.close()

    # filter out one file (which contains '5720_noDC')
    input_files = [file for file in tmpdir.rglob('*.s2p') if '5720_noDC' not in file.
↪stem]
    ns = rf.NetworkSet(rf.read_all(files=[str(file) for file in input_files]))
    print('ns contains', len(ns), 'networks')

    # extract the attenuation value from the filenames and store in list
    attn = []
    temp = []
    for f in input_files:
        _,_,_,_,_,a,t = f.stem.split('_')
        t = t.replace('M','-').replace('C','')
        attn.append(float(a))
        temp.append(int(t))

    # sort files
    v = list(zip(attn,temp,input_files))
    v.sort()
    (attn,temp,input_files) = list(zip(*v))

    values = {'ATTEN': attn, 'TEMP_C': temp }
    datatypes = {'ATTEN': 'double', 'TEMP_C': 'double'}

    # write to a generalized MDIF file
    ns.write_mdif("ADRF5720.mdif", values, datatypes)

except requests.ReadTimeout:
    print('Timeout... skipping this example')
```

```
Timeout... skipping this example
```

Finally, the parameterized MDIF files can be imported into AWR Microwave Office:



2.4.2 Interpolating Network Sets

Frequently a set of `Networks` is recorded while changing some other parameters; like temperature, voltage, current, etc. Once this set of data acquired, it is sometime useful to estimate the behaviour of the network for parameter values that lie in between those that have been measured. For that purpose, interpolating a network from a set of network is possible. This example demonstrates how to do this using *NetworkSets*.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf

rf.stylely()
```

Narda 3752 phase shifter

In this example, we are characterizing an old narda phase shifter 3752 at 1.5 GHz.

:

In order to deduce the phase shift that one can obtain at this specific frequency, we have measured scattering parameters in the 1-2 GHz band at 19 positions of the phase knob (from 0 to 180). These measurements are loaded into a *NetworkSets* object:

[2]:
```
# Array containing the 19 phase shift indicator values
indicators_mes = np.linspace(0, 180, num=19)  # from 0 to 180 per 10
```

[3]:
```
ntw_set = rf.networkSet.NetworkSet.from_zip('phase_shifter_measurements/phase_shifter_
↪measurements.zip')
print('ntw_set contains', len(ntw_set), 'networks')
```

```
ntw_set contains 19 networks
```

We extract from the network set the phase shift and S_{11} at the specific frequency of 1.5 GHz:

[4]:
```
f = '1.5 GHz'
phases_mes = np.squeeze([ntw[f].s21.s_deg for ntw in ntw_set])
s11_mes = np.squeeze([ntw[f].s11.s_db for ntw in ntw_set])
```

We would like however to get the phase shift values for intermediate settings of this phase shifter. To that purpose, let's create a network from the interpolation of the measured networks.

[5]:
```
# the indicator values for which we want to interpolate the network
indicators = np.linspace(0, 180, num=181)  # every degrees for 0 to 180

phases_interp = [ntw_set.interpolate_from_network(indicators_mes, phi)['1.5GHz'].s21.s_
```

(continues on next page)

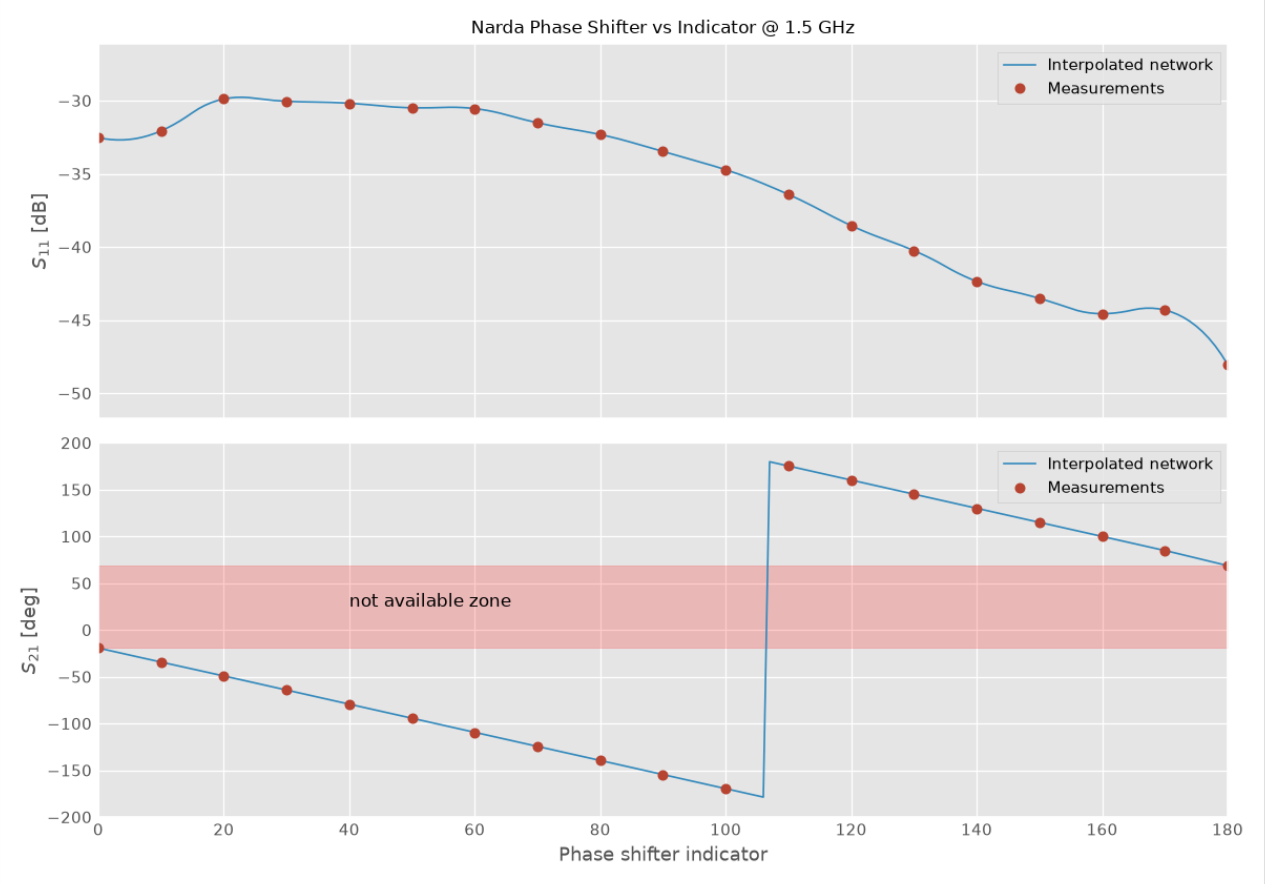
(continued from previous page)

```
↪deg for phi in indicators]
phases_interp = np.squeeze(phases_interp)

s11_interp = [ntw_set.interpolate_from_network(
    indicators_mes, phi, interp_kind='quadratic')['1.5GHz'].s11.s_db for phi in␣
↪indicators]
s11_interp = np.squeeze(s11_interp)

print('We have interpolated the network for', len(phases_interp), 'points')
```

```
We have interpolated the network for 181 points
```

Now let's plot everything

[6]:
```
fig, (ax1, ax2) = plt.subplots(2,1, figsize=(10,7), sharex=True)
ax1.set_title('Narda Phase Shifter vs Indicator @ 1.5 GHz', fontsize=10)
ax1.plot(indicators, s11_interp, label='Interpolated network')
ax1.plot(indicators_mes, s11_mes, '.', ms=10, label='Measurements')
ax1.legend()
ax1.set_ylabel(r'$S_{11}$ [dB]')

ax2.plot(indicators, phases_interp, label='Interpolated network')
ax2.plot(indicators_mes, phases_mes, '.', ms=10, label='Measurements')
ax2.set_xlabel('Phase shifter indicator')
ax2.set_ylabel('$S_{21}$ [deg]')

ax2.fill_between(indicators, phases_mes[0], phases_mes[-1], alpha=0.2, color='r')
ax2.text(40, 25, 'not available zone')
ax2.legend()
ax2.set_ylim(-200, 200)

fig.tight_layout()
```



[ ]:

2.4.3 Sorting Network Sets

Frequently a set of `Networks` is recorded while changing some other variable; like voltage, or current or time. So… now you have this set of data and you want to look at how some feature evolves, or calculate some representative statistics. This example demonstrates how to do this using *NetworkSets*.

Generate some Data

For the purpose of this example we use a predefined `skrf.media.Media` object to generate some networks, and save them as a series of touchstone files. Each file is named with a timestamp, generated with the convenience function `rf.util.now_string()`.

[1]:
```
from time import sleep

import matplotlib.pyplot as plt
import numpy as np

import skrf as rf
from skrf.util import now_string, now_string_2_dt

%matplotlib inline
rf.stylely()
```

(continues on next page)

(continued from previous page)

```
!rm -rf tmp
!mkdir tmp

wg = rf.instances.wr10 # just a dummy media object to generate data

for k in range(10):
    # timestamp generated with `now_string()`
    ntwk = wg.random(name=now_string()+'.s1p')
    ntwk.s = k*ntwk.s
    ntwk.write_touchstone(dir='tmp')
    sleep(.1)
```

Lets take a look at what we made

[2]:
```
!ls tmp
```

```
2026.11.02.19.12.19.615433.s1p  2026.11.02.19.12.20.136732.s1p
2026.11.02.19.12.19.718994.s1p  2026.11.02.19.12.20.241090.s1p
2026.11.02.19.12.19.823499.s1p  2026.11.02.19.12.20.345446.s1p
2026.11.02.19.12.19.927911.s1p  2026.11.02.19.12.20.449806.s1p
2026.11.02.19.12.20.032367.s1p  2026.11.02.19.12.20.554200.s1p
```

Not sorted (default)

When created using `NetworkSet.from_dir()`, the `NetworkSet`'s stores each `Network` randomly

[3]:
```
ns = rf.networkSet.NetworkSet.from_dir('tmp')
ns.ntwk_set
```

[3]:
```
[1-Port Network: '2026.11.02.19.12.19.615433',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.718994',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.823499',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.927911',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.032367',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.136732',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.241090',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.345446',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.449806',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.554200',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j]]
```

Sort it

[4]:
```
ns.sort()
ns.ntwk_set
```

[4]:
```
[1-Port Network: '2026.11.02.19.12.19.615433',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.718994',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.823499',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.927911',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.032367',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.136732',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
```

(continues on next page)

(continued from previous page)

```
 1-Port Network: '2026.11.02.19.12.20.241090',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.345446',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.449806',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.554200',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j]]
```

Sorting using `key` argument

You can also pass a function through the `key` argument, which allows you to sort on arbitrary properties. For example, we could sort based on the sub-second field of the name,

[5]:
```
ns = rf.networkSet.NetworkSet.from_dir('tmp')
ns.sort(key = lambda x: x.name.split('.')[0])
ns.ntwk_set
```

[5]:
```
[1-Port Network: '2026.11.02.19.12.19.615433',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.718994',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.823499',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.19.927911',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.032367',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.136732',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.241090',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.345446',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.449806',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j],
 1-Port Network: '2026.11.02.19.12.20.554200',  75.0-110.0 GHz, 1001 pts, z0=[50.+0.j]]
```

Extracting Datetimes

You can also convert the ntwk names to datetime objects, in case you want to plot something with pandas or do some other processing. There is a companion function to `rf.util.now_string()` which is `rf.util.now_string_2_dt()`. How creative..

[6]:
```
ns.sort()
dt_idx = [now_string_2_dt(k.name ) for k in ns]
dt_idx
```

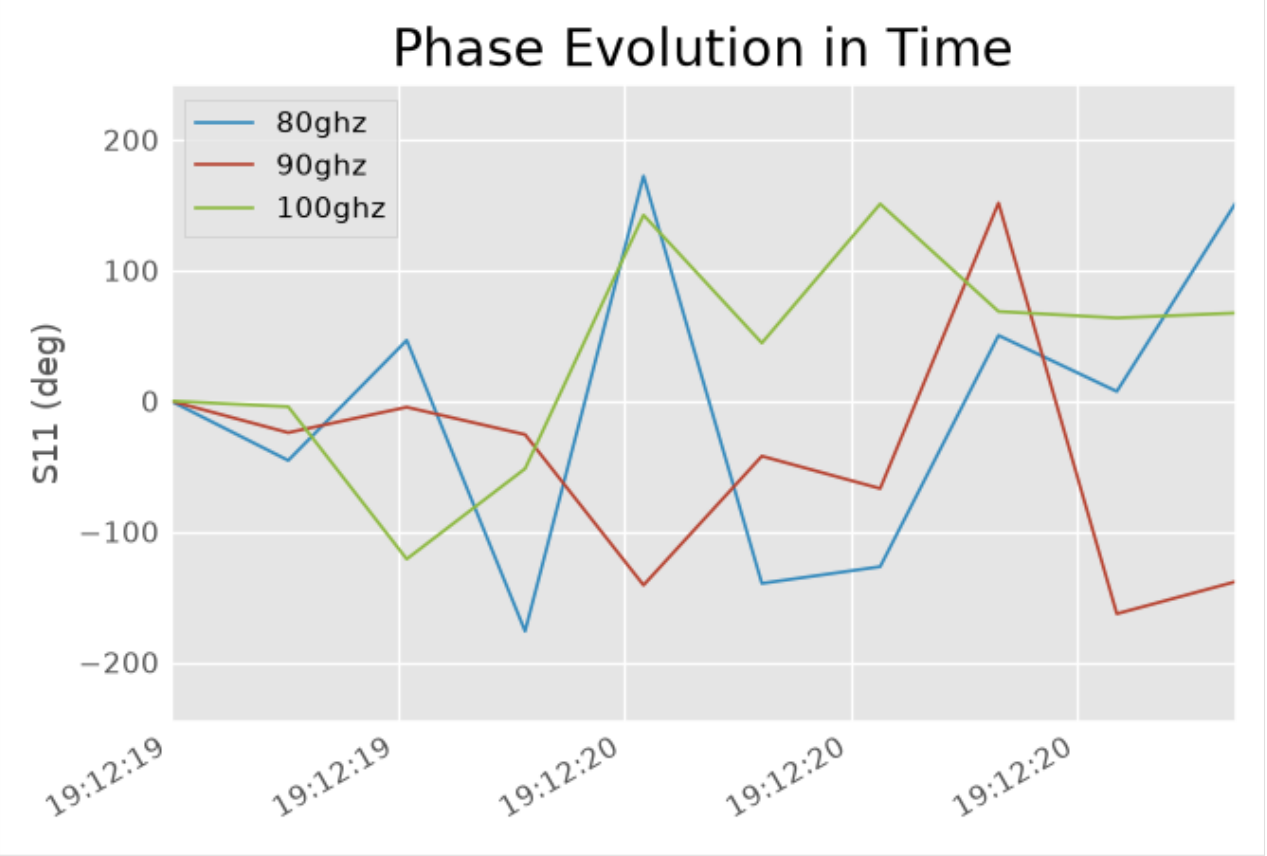
[6]:
```
[datetime.datetime(2026, 11, 2, 19, 12, 19, 615433),
 datetime.datetime(2026, 11, 2, 19, 12, 19, 718994),
 datetime.datetime(2026, 11, 2, 19, 12, 19, 823499),
 datetime.datetime(2026, 11, 2, 19, 12, 19, 927911),
 datetime.datetime(2026, 11, 2, 19, 12, 20, 32367),
 datetime.datetime(2026, 11, 2, 19, 12, 20, 136732),
 datetime.datetime(2026, 11, 2, 19, 12, 20, 241090),
 datetime.datetime(2026, 11, 2, 19, 12, 20, 345446),
 datetime.datetime(2026, 11, 2, 19, 12, 20, 449806),
 datetime.datetime(2026, 11, 2, 19, 12, 20, 554200)]
```

Put into a Pandas DataFrame and Plot

The next step is to slice the network set along the time axis. For example we may want to look at S11 phase, at a few different frequencies. This can be done with the following script. Note that NetworkSets can be sliced by frequency with human readable strings, just like Networks.

[7]:
```
import pandas as pd

dates = pd.DatetimeIndex(dt_idx)

# create a function to pull out S11 in degrees at a specific frequency

s_deg_at = lambda s:{s: [k[s].s_deg[0,0,0] for k in ns]}  # noqa: E731

for f in ['80ghz', '90ghz','100ghz']:
    df =pd.DataFrame(s_deg_at(f), index=dates)
    df.plot(ax=plt.gca())
plt.title('Phase Evolution in Time')
plt.ylabel('S11 (deg)')
```

[7]:
```
Text(0, 0.5, 'S11 (deg)')
```



Visualizing Behavior with `signature`

It may be of use to visualize the evolution of a scalar component of the network set over all frequencies. This can be done with a little bit of array manipulation and `imshow`. For example if we take the magnitude in dB for each network, and create 2D matrix from this,

[8]:
```
mat = np.array([k.s_db.flatten() for k in ns])
mat.shape
```

```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪mathFunctions.py:303: RuntimeWarning: divide by zero encountered in log10
```

(continues on next page)

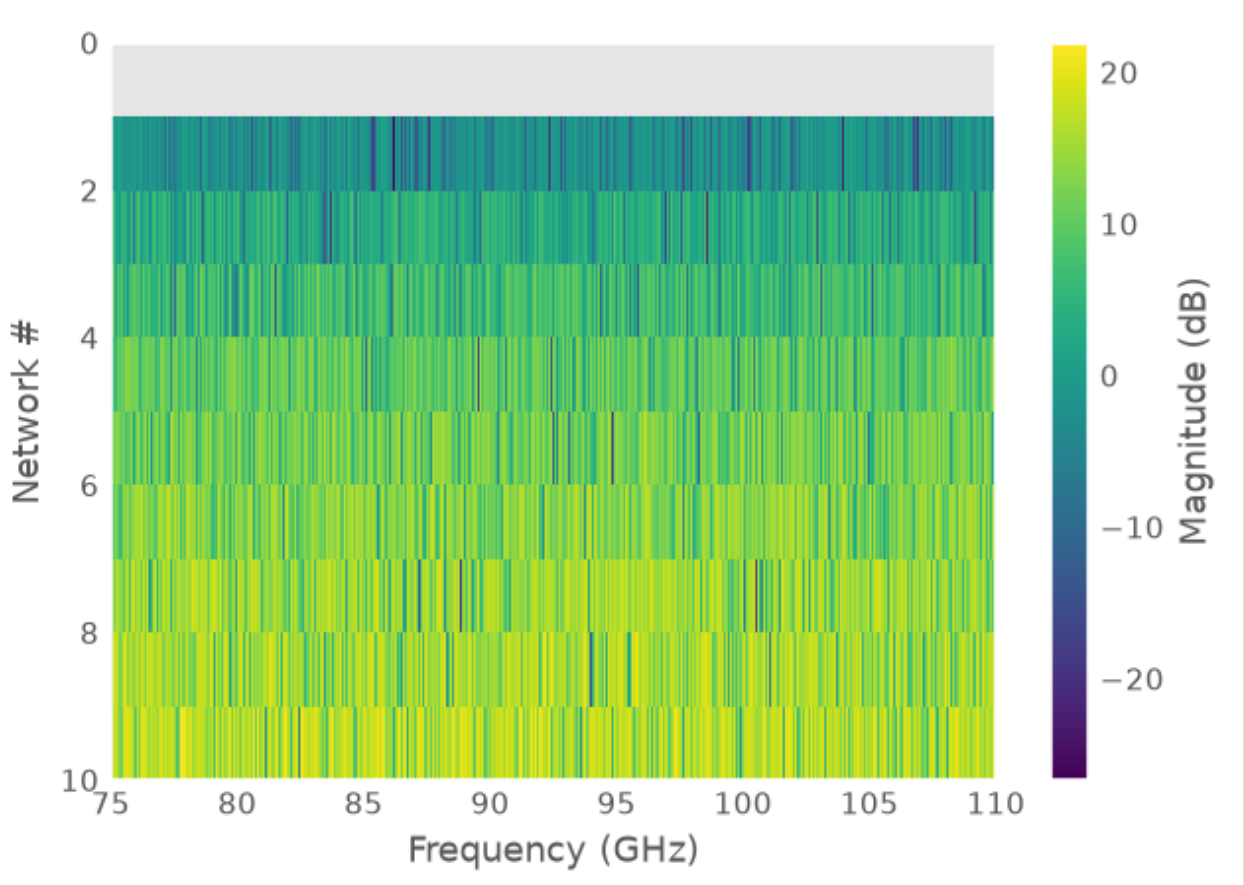
(continued from previous page)

```
  out = 20 * np.log10(z)
```

[8]:
```
(10, 1001)
```

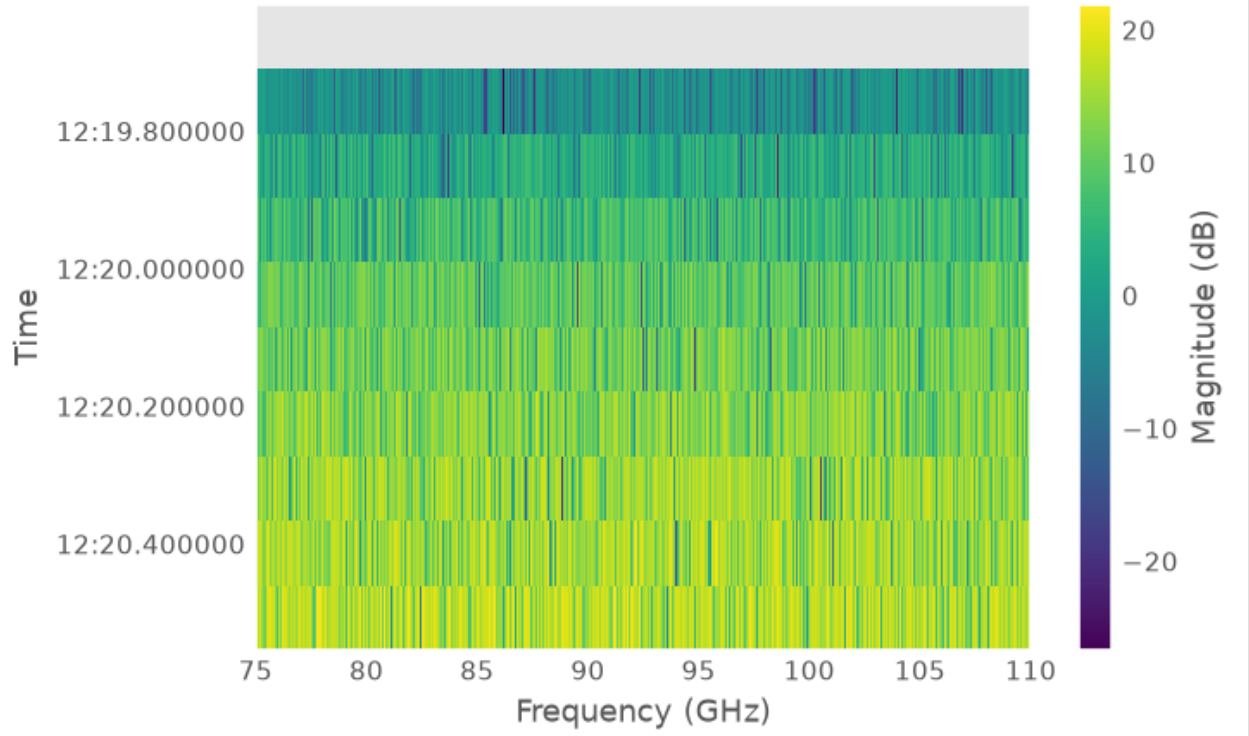
This array has shape ('Number of Networks' , 'Number frequency points'). This can be visualized with imshow. Most of the code below just adds labels, and axis-scales.

[9]:
```
freq = ns[0].frequency

# creates x and y scales
extent = [freq.f_scaled[0], freq.f_scaled[-1], len(ns) ,0]

#make the image
plt.imshow(mat, aspect='auto',extent=extent,interpolation='nearest')

# label things
plt.grid(0)
freq.labelXAxis()
plt.ylabel('Network #')
cbar = plt.colorbar()
cbar.set_label('Magnitude (dB)')
```



This process is automated with the method `NetworkSet.signature()`. It even has a `vs_time` parameter which will automatically create the DateTime index from the Network's names, if they were written by `rf.now_string()`

[10]:
```
ns.signature(component='s_db', vs_time=True,cbar_label='Magnitude (dB)')
```



2.5 Plotting

2.5.1 Properties of RG-58

This example demonstrates how to use skrf's to explore some properties of Coax (RG-58). This is done through the `media` package which provides basic transmission line models. Specifically, the coax is created using by `DistributedCircuit`.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf
from skrf.media import DistributedCircuit
from skrf.plotting import func_on_all_figs as foaf

%matplotlib inline

rf.stylely()



# define a frequency object from a vector
freq = rf.Frequency.from_f(np.logspace(0,6,1001), unit='hz')

# create a Media object for RG-58, based on distributed ckt values
```

(continues on next page)

(continued from previous page)

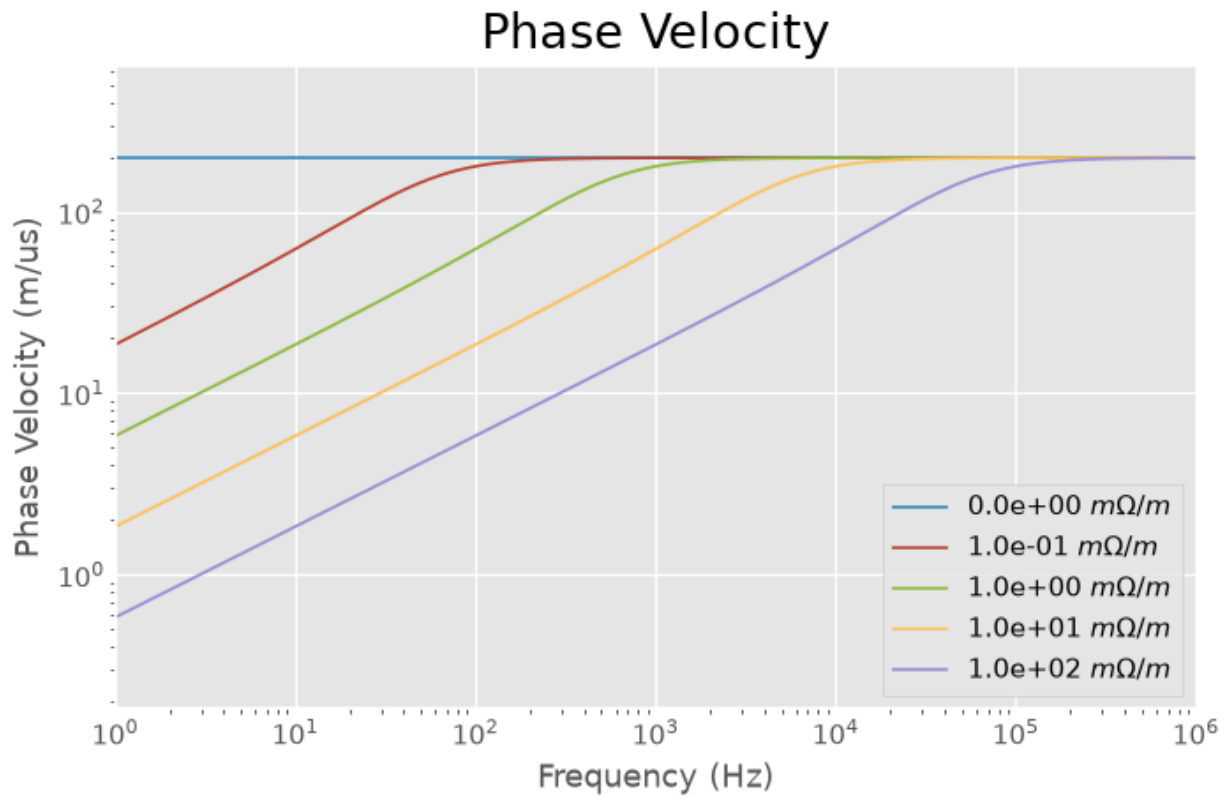
```
rg58 = DistributedCircuit(
    frequency = freq,
    C =93.5e-12,#F/m
    L =273e-9,  #H/m
    R =0,#53e-3,   #Ohm/m
    G =0,       #S/m
    )


# loop thru values of resistivity plot various quantities
for k in (0,.1,1,10,100):
    rg58.R = k*1e-3

    plt.figure(0)
    plt.ylabel('Phase Velocity (m/us)')
    plt.title('Phase Velocity')
    plt.loglog(freq.f_scaled, rg58.v_p*1e-6, label=rf'{k:.1e} $m \Omega/m $')

    plt.figure(1)
    plt.ylabel('Real($Z_0$)')
    plt.title('Characteristic Impedance (Real)')
    plt.loglog(freq.f_scaled, rg58.z0.real, label=rf'{k:.1e} $m \Omega/m $')

    plt.figure(2)
    plt.ylabel('-Imag($Z_0$)')
    plt.title('Characteristic Impedance (Imag)')
    plt.plot(freq.f_scaled, -1*rg58.z0.imag, label=rf'{k:.1e} $m \Omega/m $')

    plt.figure(3)
    plt.ylabel(r'Real($\gamma$)')
    plt.title('Propagation Constant (Real)')
    plt.plot(freq.f_scaled, rg58.alpha, label=rf'{k:.1e} $m \Omega/m $')

    plt.figure(4)
    plt.ylabel(r'Imag($\gamma$)')
    plt.title('Propagation Constant (Imag)')
    plt.plot(freq.f_scaled, rg58.beta, label=rf'{k:.1e} $m \Omega/m $')




foaf(freq.labelXAxis)
foaf(plt.tight_layout)
foaf(plt.legend)
foaf(plt.loglog)

plt.tight_layout()
```

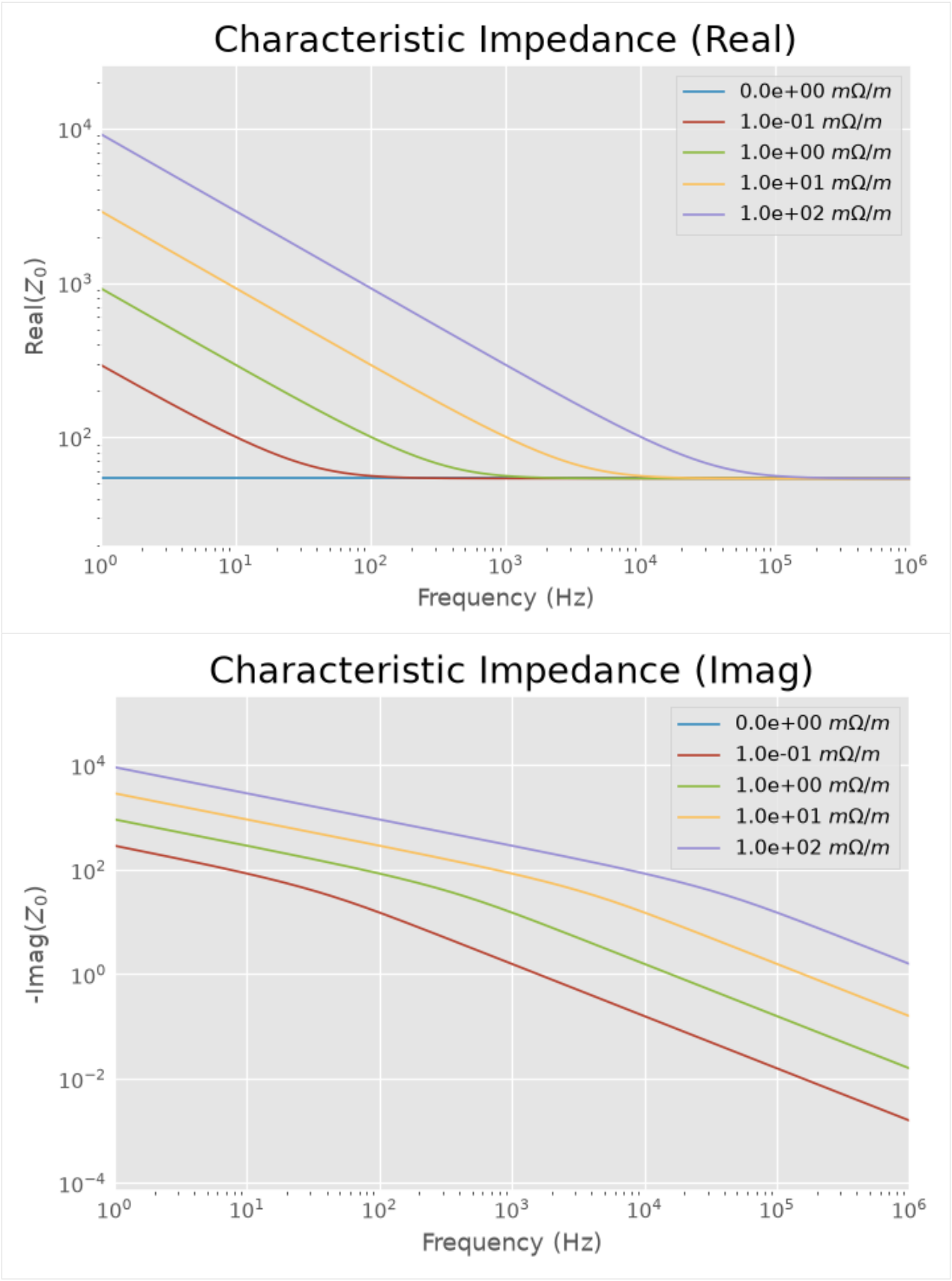
```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/envs/latest/lib/python3.12/
↪site-packages/matplotlib/cbook.py:1811: ComplexWarning: Casting complex values to real␣
```

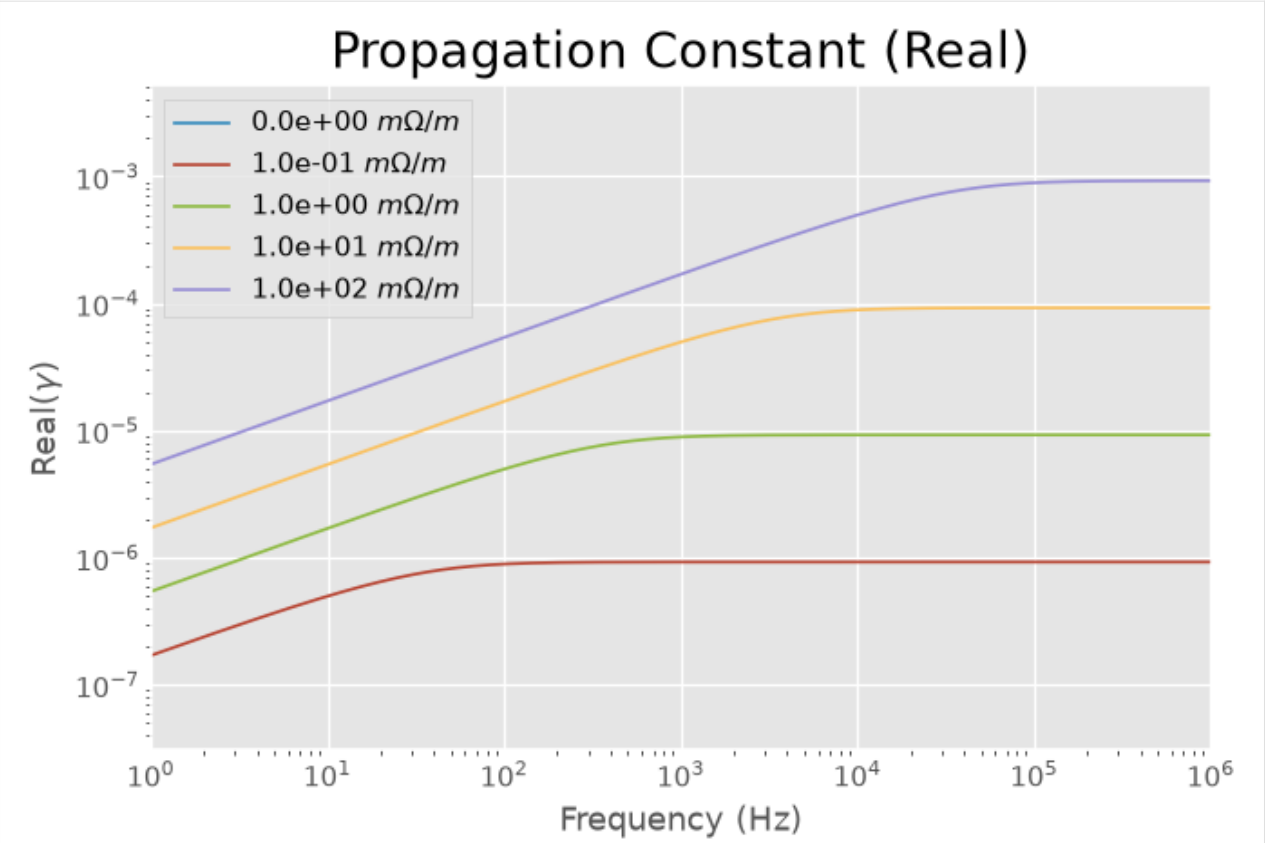
(continues on next page)

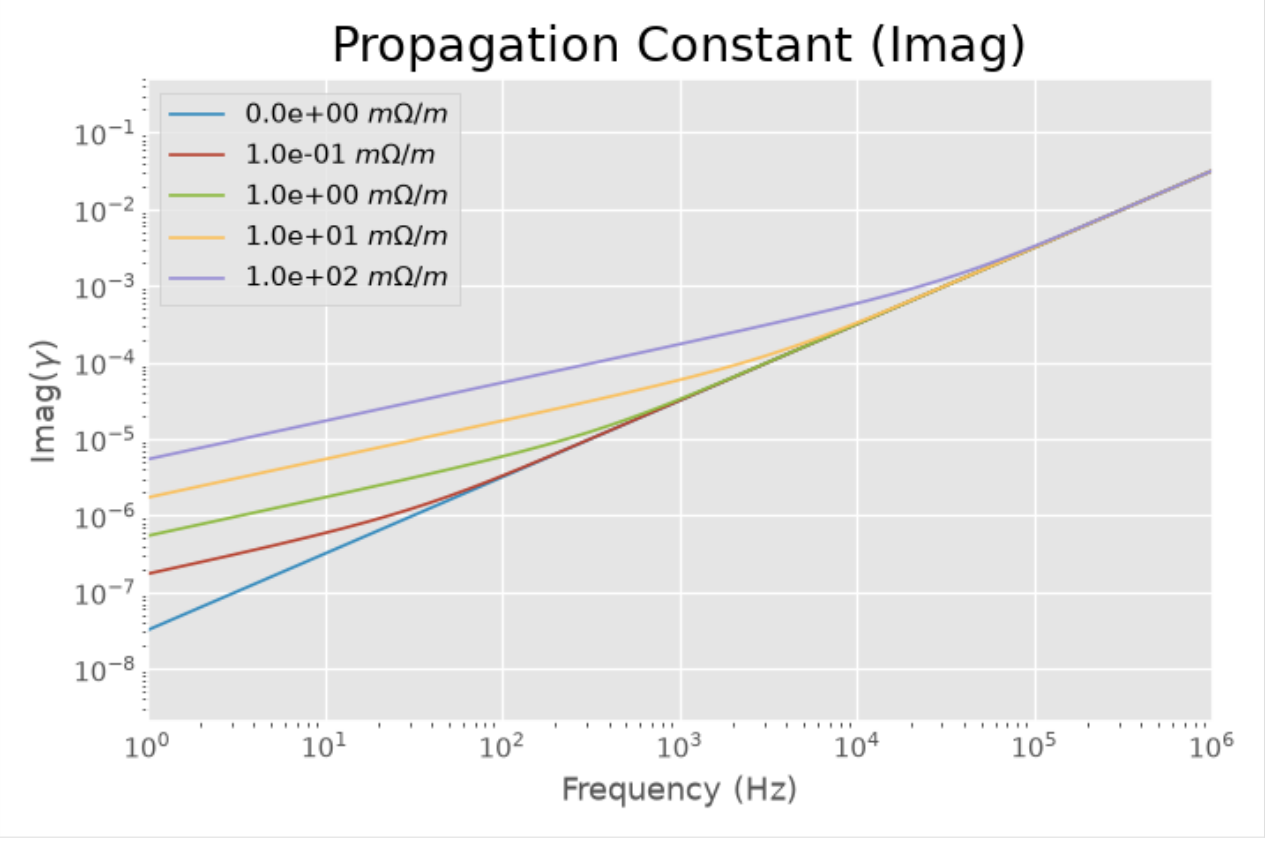
(continued from previous page)

```
↪discards the imaginary part
  return math.isfinite(val)
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/envs/latest/lib/python3.12/
↪site-packages/matplotlib/cbook.py:1408: ComplexWarning: Casting complex values to real␣
↪discards the imaginary part
  return np.asanyarray(x, float)
```




Characteristic Impedance (Real)
0.0e+00 mΩ/m
1.0e-01 mΩ/m
1.0e+00 mΩ/m
1.0e+01 mΩ/m
1.0e+02 mΩ/m
Real(Z_0)
10^4
10^3
10^2
10^0 10^1 10^2 10^3 10^4 10^5 10^6
Frequency (Hz)
Characteristic Impedance (Imag)
0.0e+00 mΩ/m
1.0e-01 mΩ/m
1.0e+00 mΩ/m
1.0e+01 mΩ/m
1.0e+02 mΩ/m
-Imag(Z_0)
10^4
10^2
10^0
10^{-2}
10^{-4}
10^0 10^1 10^2 10^3 10^4 10^5 10^6
Frequency (Hz)


Propagation Constant (Real)
0.0e+00 mΩ/m
1.0e-01 mΩ/m
1.0e+00 mΩ/m
1.0e+01 mΩ/m
1.0e+02 mΩ/m
Real(γ)
Frequency (Hz)


Propagation Constant (Imag)
0.0e+00 mΩ/m
1.0e-01 mΩ/m
1.0e+00 mΩ/m
1.0e+01 mΩ/m
1.0e+02 mΩ/m
Imag(γ)
Frequency (Hz)

[ ]:

2.5.2 XKCDify

This notebook serves no useful purpose. It just demonstrates a cool matplotlib function which makes a plot look like a hand drawn cartoon, similar to those on xkcd. Read more at jakes post here,

http://jakevdp.github.io/blog/2013/07/10/XKCD-plots-in-matplotlib/

[1]:
```
%matplotlib inline
import matplotlib.pyplot as plt

from skrf.data import ring_slot

plt.xkcd() # from matplotlib version
```

[1]:
```
<contextlib.ExitStack at 0x771678e7db20>
```

[2]:
```
plt.figure(figsize = (8,6))
ring_slot.plot_s_db(title='Important Results')
plt.ylim(-30,30)
```

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

[2]:
```
(-30.0, 30.0)
```

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

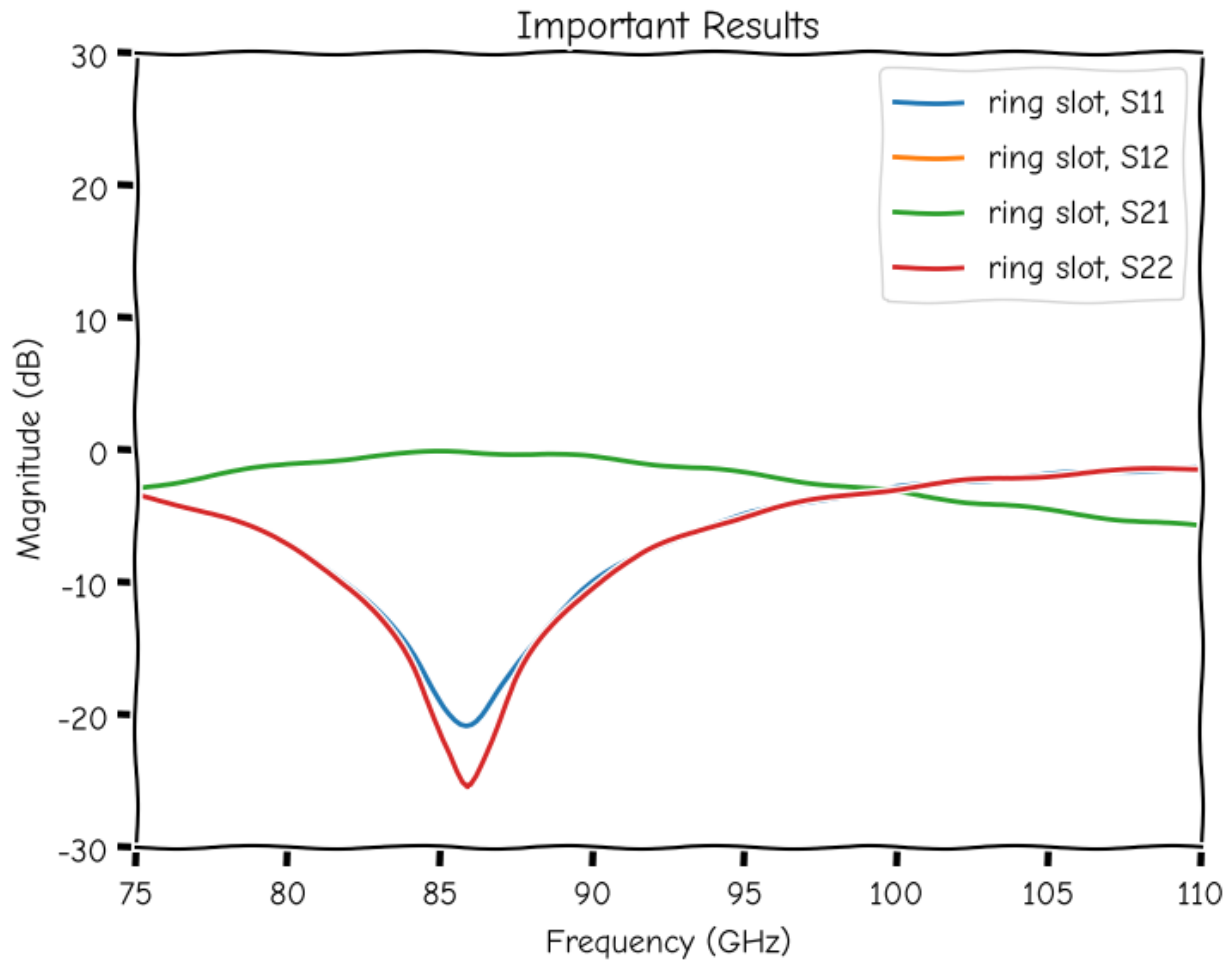
(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```



[3]:
```
plt.figure(figsize = (8,6))
ring_slot.plot_s_smith()
plt.title('Important Results')
```

[3]:
```
Text(0.5, 1.0, 'Important Results')
```

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```

(continues on next page)

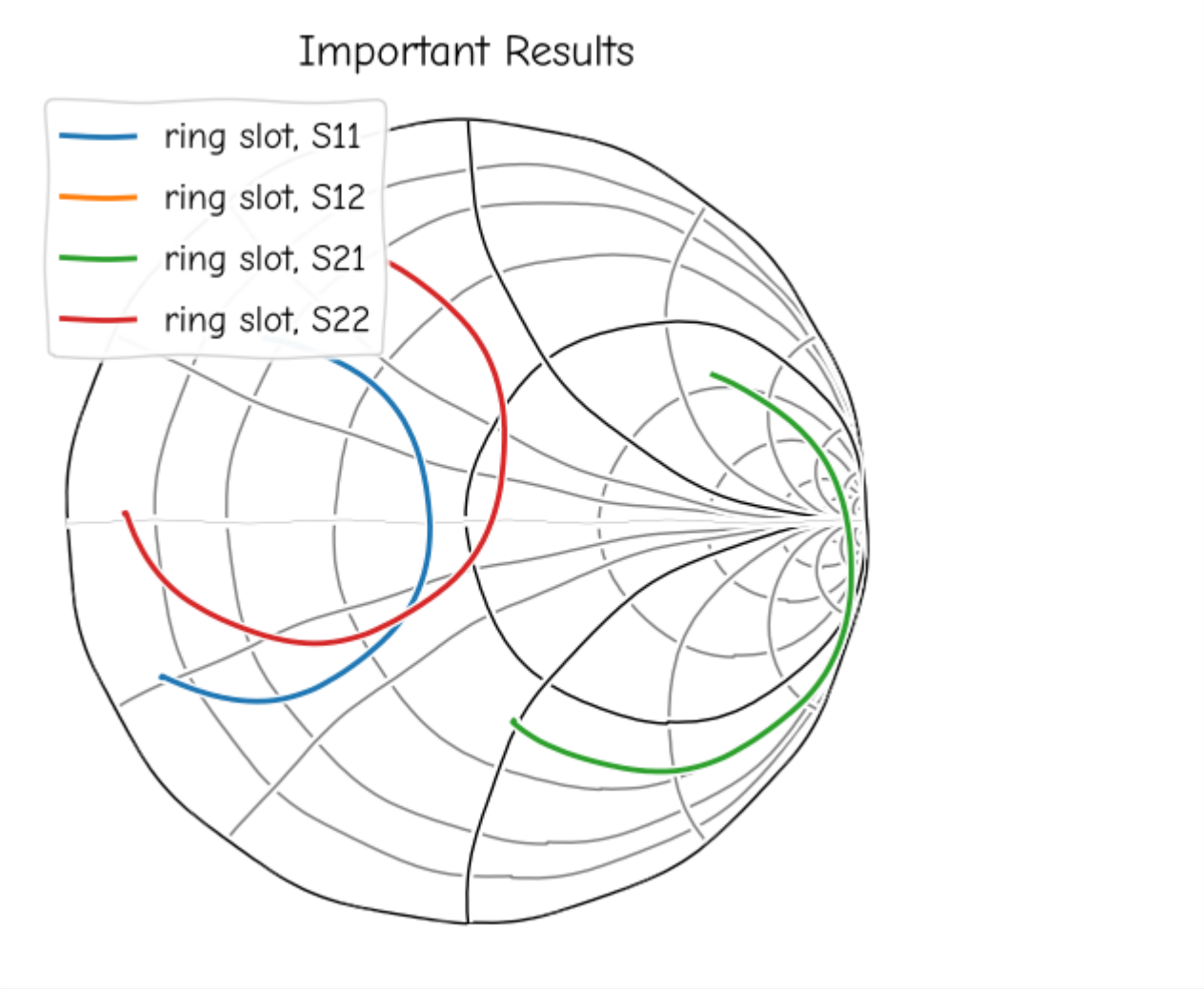
(continued from previous page)

```
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
```

(continues on next page)

(continued from previous page)

```
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
findfont: Font family 'xkcd' not found.
findfont: Font family 'xkcd Script' not found.
findfont: Font family 'Comic Sans MS' not found.
```



[ ]:

2.6 Circuit

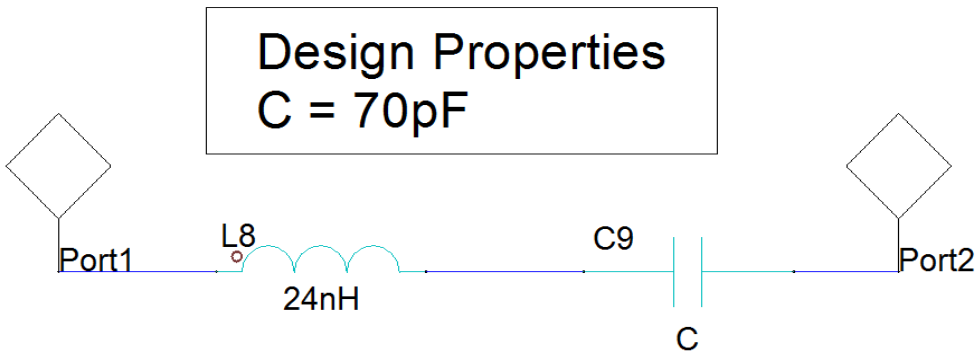
2.6.1 Lumped Elements Circuits

In this notebook, we construct various network from basic lumped elements (resistor, capacitor, inductor), with the 'classic' and the `Circuit` approach. Generally the `Circuit` approach is more verbose than the 'classic' way for building a circuit. However, as the circuit complexity increases, in particular when components are connected in parallel, the `Circuit` approach is interesting as it increases the readability of the code. Moreover, `Circuit` object can be plotted using its `plot_graph()` method, which is useful to rapidly control if the circuit is built as expected.

[1]:
```
import numpy as np  # for np.allclose() to check that S-params are similar

import skrf as rf
from skrf.circuit import Circuit
from skrf.network import connect, innerconnect

rf.stylely()
```

LC Series Circuit

In this section we reproduce a simple equivalent model of a capacitor C, as illustrated by the figure below:



[2]:
```
# reference LC circuit made in Designer
LC_designer = rf.Network('designer_capacitor_30_80MHz_simple.s2p')
```

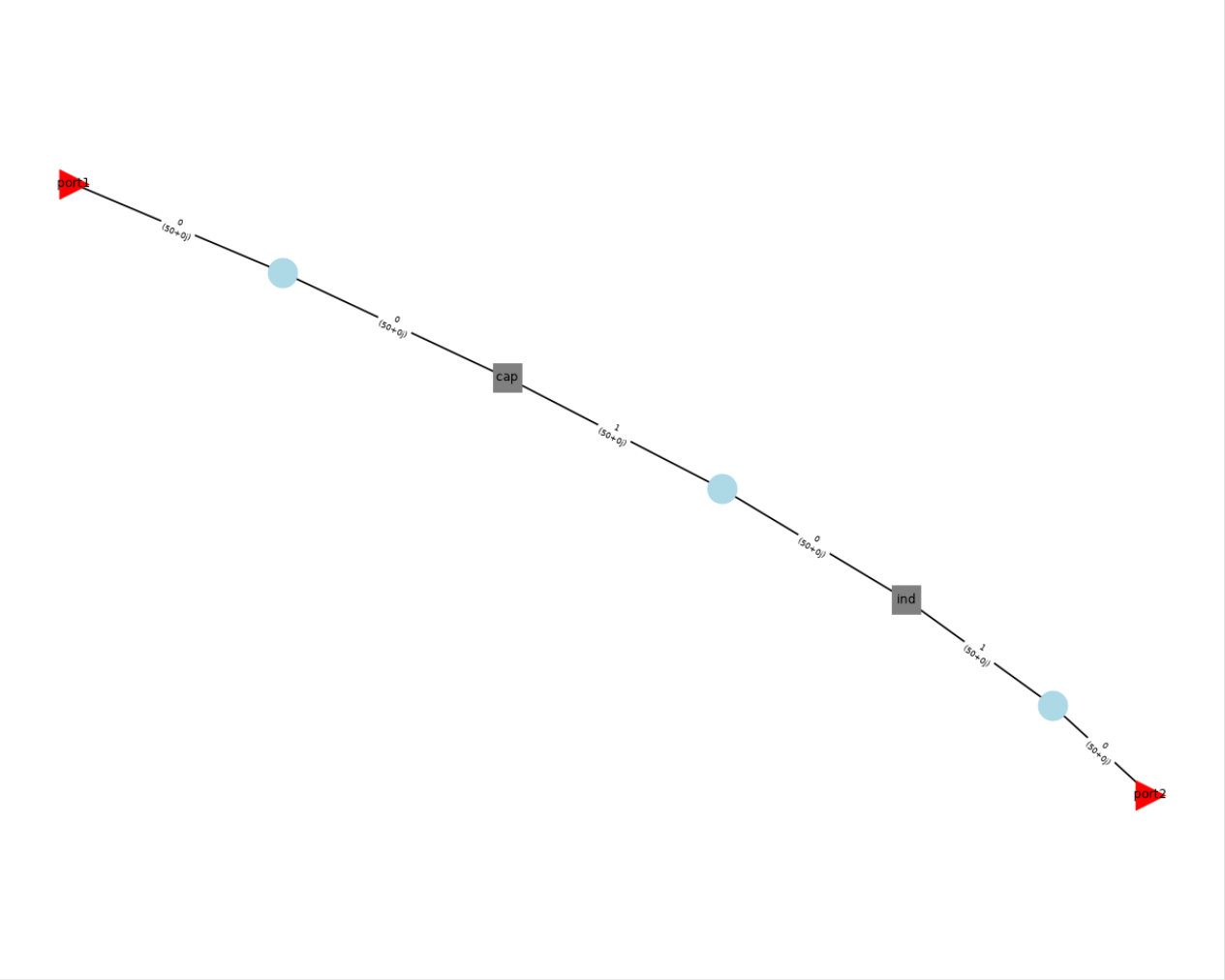
[3]:
```
# scikit-rf: manually connecting networks
line = rf.media.DefinedGammaZ0(frequency=LC_designer.frequency, z0=50)
LC_manual = line.inductor(24e-9) ** line.capacitor(70e-12)
```

[4]:
```
# scikit-rf: using Circuit builder
port1 = Circuit.Port(frequency=LC_designer.frequency, name='port1', z0=50)
port2 = Circuit.Port(frequency=LC_designer.frequency, name='port2', z0=50)
cap = Circuit.SeriesImpedance(frequency=LC_designer.frequency, name='cap', z0=50,
                              Z=1/(1j*LC_designer.frequency.w*70e-12))
ind = Circuit.SeriesImpedance(frequency=LC_designer.frequency, name='ind', z0=50,
                              Z=1j*LC_designer.frequency.w*24e-9)

# NB: it is also possible to create 2-port lumped elements like:
# line = rf.media.DefinedGammaZ0(frequency=LC_designer.frequency, z0=50)
# cap = line.capacitor(70e-12, name='cap')
# ind = line.inductor(24e-9, name='ind')

connections = [
    [(port1, 0), (cap, 0)],
    [(cap, 1), (ind, 0)],
    [(ind, 1), (port2, 0)]
]
circuit = Circuit(connections)
LC_from_circuit = circuit.network
```

[5]:
```
# testing the equivalence of the results
print(np.allclose(LC_designer.s, LC_manual.s))
print(np.allclose(LC_designer.s, LC_from_circuit.s))
```

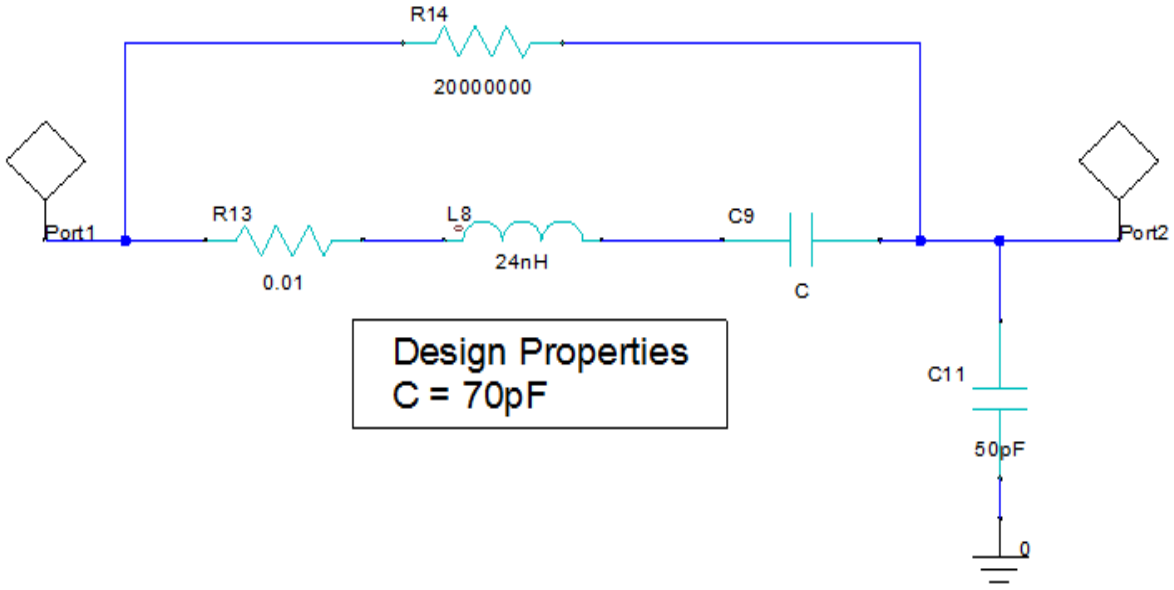
```
True
True
```

[6]:
```
circuit.plot_graph(network_labels=True, edge_labels=True, port_labels=True)
```



A More Advanced Equivalent Model

In this section we reproduce an equivalent model of a capacitor C, as illustrated by the figure below:



[7]:
```
# Reference results from ANSYS Designer
LCC_designer = rf.Network('designer_capacitor_30_80MHz_adv.s2p')
```

[8]:
```
# scikit-rf: usual way, but this time this is more tedious to deal with connection and␣
↪port number
freq = LCC_designer.frequency
line = rf.media.DefinedGammaZ0(frequency=freq, z0=50)
elements1 = line.resistor(1e-2) ** line.inductor(24e-9) ** line.capacitor(70e-12)
elements2 = line.resistor(20e6)
T_in = line.tee()
T_out = line.tee()
ntw = connect(T_in, 1, elements1, 0)
ntw = connect(ntw, 2, elements2, 0)
ntw = connect(ntw, 1, T_out, 1)
ntw = innerconnect(ntw, 1, 2)
LCC_manual = ntw ** line.shunt_capacitor(50e-12)
```

[9]:
```
# scikit-rf: using Circuit builder
freq = LCC_designer.frequency
port1 = Circuit.Port(frequency=freq, name='port1', z0=50)
port2 = Circuit.Port(frequency=freq, name='port2', z0=50)
line = rf.media.DefinedGammaZ0(frequency=freq, z0=50)
cap = line.capacitor(70e-12, name='cap')
ind = line.inductor(24e-9, name='ind')
res_series = line.resistor(1e-2, name='res_series')
res_parallel = line.resistor(20e6, name='res_parallel')
cap_shunt = line.capacitor(50e-12, name='cap_shunt')
ground = Circuit.Ground(frequency=freq, name='ground', z0=50)

connections = [
```

(continues on next page)

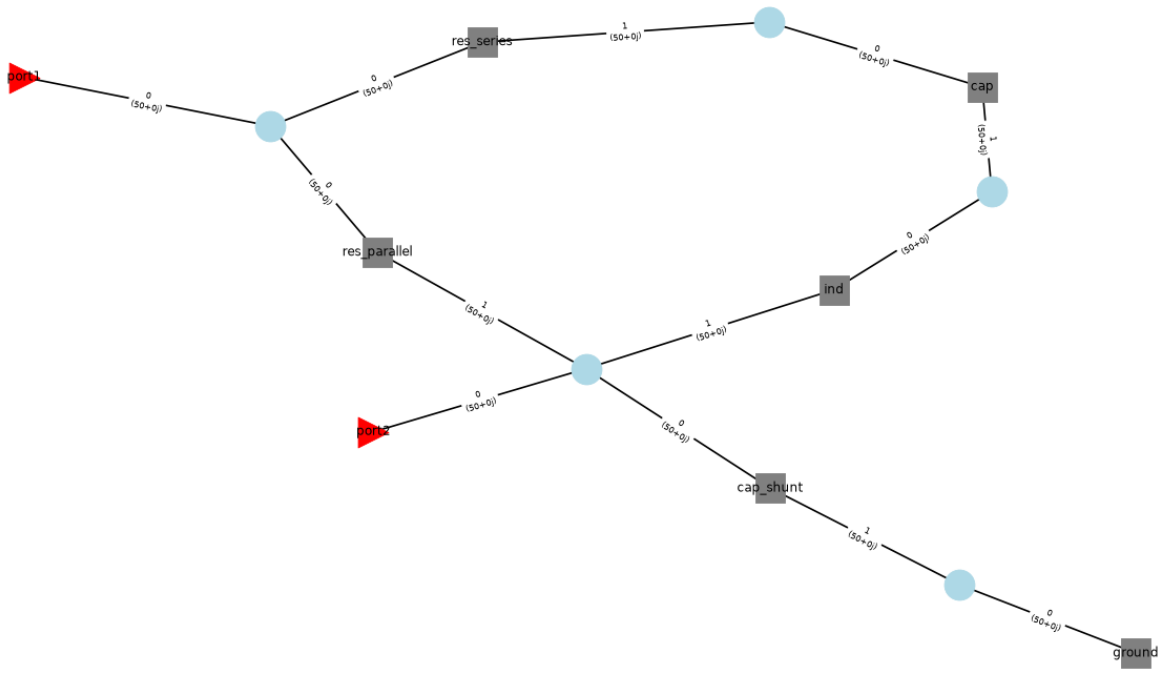
(continued from previous page)

```
    [(port1, 0), (res_series, 0), (res_parallel, 0)],
    [(res_series, 1), (cap, 0)],
    [(cap, 1), (ind, 0)],
    [(ind, 1), (cap_shunt, 0), (res_parallel, 1), (port2, 0)],
    [(cap_shunt, 1), (ground, 0)],
]
circuit = Circuit(connections)
LCC_from_circuit = circuit.network
```

[10]:
```
# testing the equivalence of the results
print(np.allclose(LCC_designer.s, LCC_manual.s))
print(np.allclose(LCC_designer.s, LCC_from_circuit.s))
```

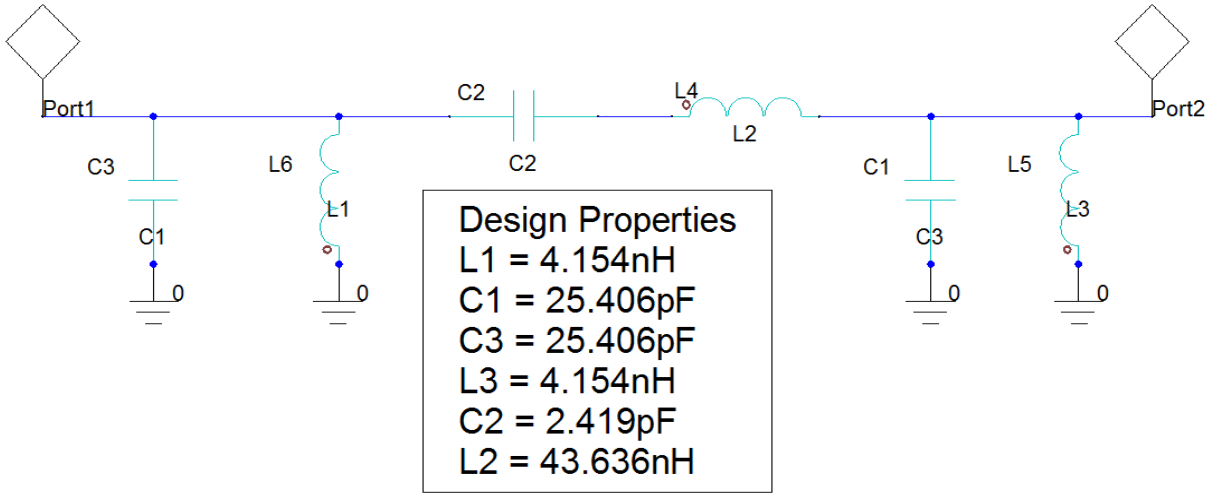
```
True
True
```

[11]:
```
circuit.plot_graph(network_labels=True, edge_labels=True, port_labels=True)
```



Pass band filter

Below we construct a pass-band filter, from an example given in Microwaves101:

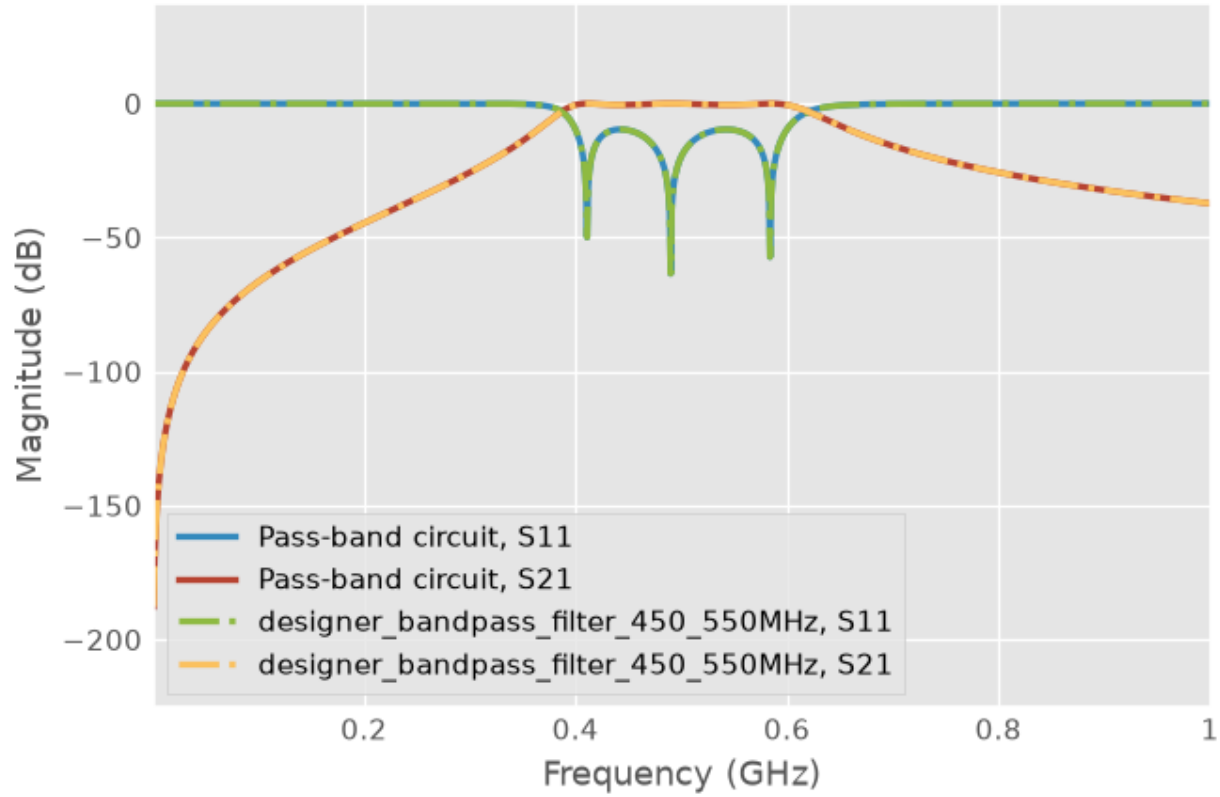


[12]:
```
# Reference result calculated from Designer
passband_designer = rf.Network('designer_bandpass_filter_450_550MHz.s2p')
```

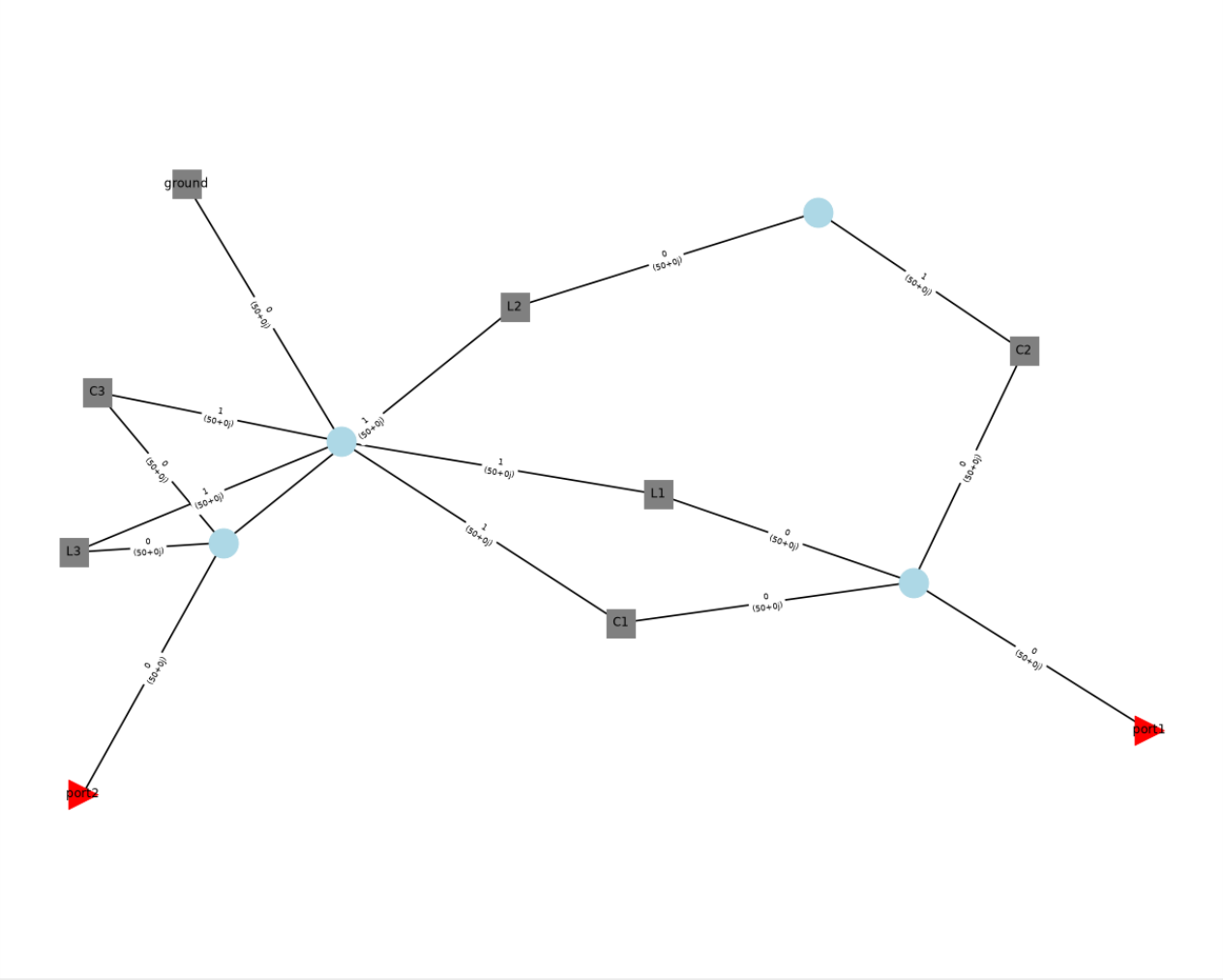
[13]:
```
# scikit-rf: the filter by cascading all lumped-elements
freq = passband_designer.frequency
passband_manual = line.shunt_capacitor(25.406e-12) ** line.shunt_inductor(4.154e-9) ** \
                  line.capacitor(2.419e-12) ** line.inductor(43.636e-9) ** \
                  line.shunt_capacitor(25.406e-12) ** line.shunt_inductor(4.154e-9)
```

[14]:
```
# scikit-rf: the filter with the Circuit builder
freq = passband_designer.frequency
line = rf.media.DefinedGammaZ0(frequency=freq)
C1 = line.capacitor(25.406e-12, name='C1')
C2 = line.capacitor(2.419e-12, name='C2')
C3 = line.capacitor(25.406e-12, name='C3')
L1 = line.inductor(4.154e-9, name='L1')
L2 = line.inductor(43.636e-9, name='L2')
L3 = line.inductor(4.154e-9, name='L3')
port1 = Circuit.Port(frequency=freq, name='port1', z0=50)
port2 = Circuit.Port(frequency=freq, name='port2', z0=50)
ground =  Circuit.Ground(frequency=freq, name='ground', z0=50)

connections = [
    [(port1, 0), (C1, 0), (L1, 0), (C2, 0)],
    [(C2, 1), (L2, 0)],
    [(L2, 1), (C3, 0), (L3, 0), (port2, 0)],
    [(C1, 1), (C3, 1), (L1, 1), (L3, 1), (ground, 0)],
]

circuit = Circuit(connections)
passband_circuit = circuit.network
passband_circuit.name = 'Pass-band circuit'
```

[15]:
```
passband_circuit.plot_s_db(m=0, n=0, lw=2)
passband_circuit.plot_s_db(m=1, n=0, lw=2)
passband_designer.plot_s_db(m=0, n=0, lw=2, ls='-.')
passband_designer.plot_s_db(m=1, n=0, lw=2, ls='-.')
```



[16]:
```
circuit.plot_graph(network_labels=True, port_labels=True, edge_labels=True)
```



[ ]:

2.6.2 Voltages and Currents in Circuits

The scikit-rf `Circuit` object allows one to deduce the voltages and currents at all the intersections of the circuit for a given power (and phase) excitation at circuit's ports. Here, few examples are examined, in order to clarify the conventions used for current directions.

[1]:
```
%matplotlib inline
```

[2]:
```
# standard imports
import numpy as np

import skrf as rf
from skrf.circuit import Circuit
```

[3]:
```
rf.stylely()
```

A simple transmission line

To start, let's assume a simple transmission line excited by a generator. Let's assume the generator is matched to the line ($Z_s = Z_0$) and the line connected to a matched load ($Z_0 = Z_L$), all 50 Ohm.

What is the RF currents and voltages at the input and output of this line for a given input power (and phase)?

[4]:
```
P_f = 1  # forward power in Watt
Z = 50  # source internal impedance, line characteristic impedance and load impedance
L = 10  # line length in [m]

freq = rf.Frequency(2, 2, 1, unit='GHz')
line_media = rf.media.DefinedGammaZ0(freq, z0=Z)  # lossless line medium
line = line_media.line(d=L, unit='m', name='line')  # transmission line Network
```

Assuming the source generates an input power of P_f with a phase ϕ, with such a line the voltage and current at the entrance of the line are:

$$V_{in} = \sqrt{2 Z_0 P_f} e^{j\phi}$$

$$I_{in} = \sqrt{2 \frac{P_f}{Z_0}} e^{j\phi}$$

[5]:
```
V_in = np.sqrt(2*Z*P_f)
I_in = np.sqrt(2*P_f/Z)
print(f'Input voltage and current: {V_in} V and {I_in} A')
```

```
Input voltage and current: 10.0 V and 0.2 A
```

The voltage and current evolve along the transmission line. The voltage and current at the output of the line can be calculated using the transmission line tools provided with scikit-rf:

[6]:
```
theta = rf.tlineFunctions.theta(line_media.gamma, freq.f, L)  # electrical length
V_out, I_out = rf.tlineFunctions.voltage_current_propagation(V_in, I_in, Z, theta)
print(f'Output voltage and current: {V_out} V and {I_out} A')
```

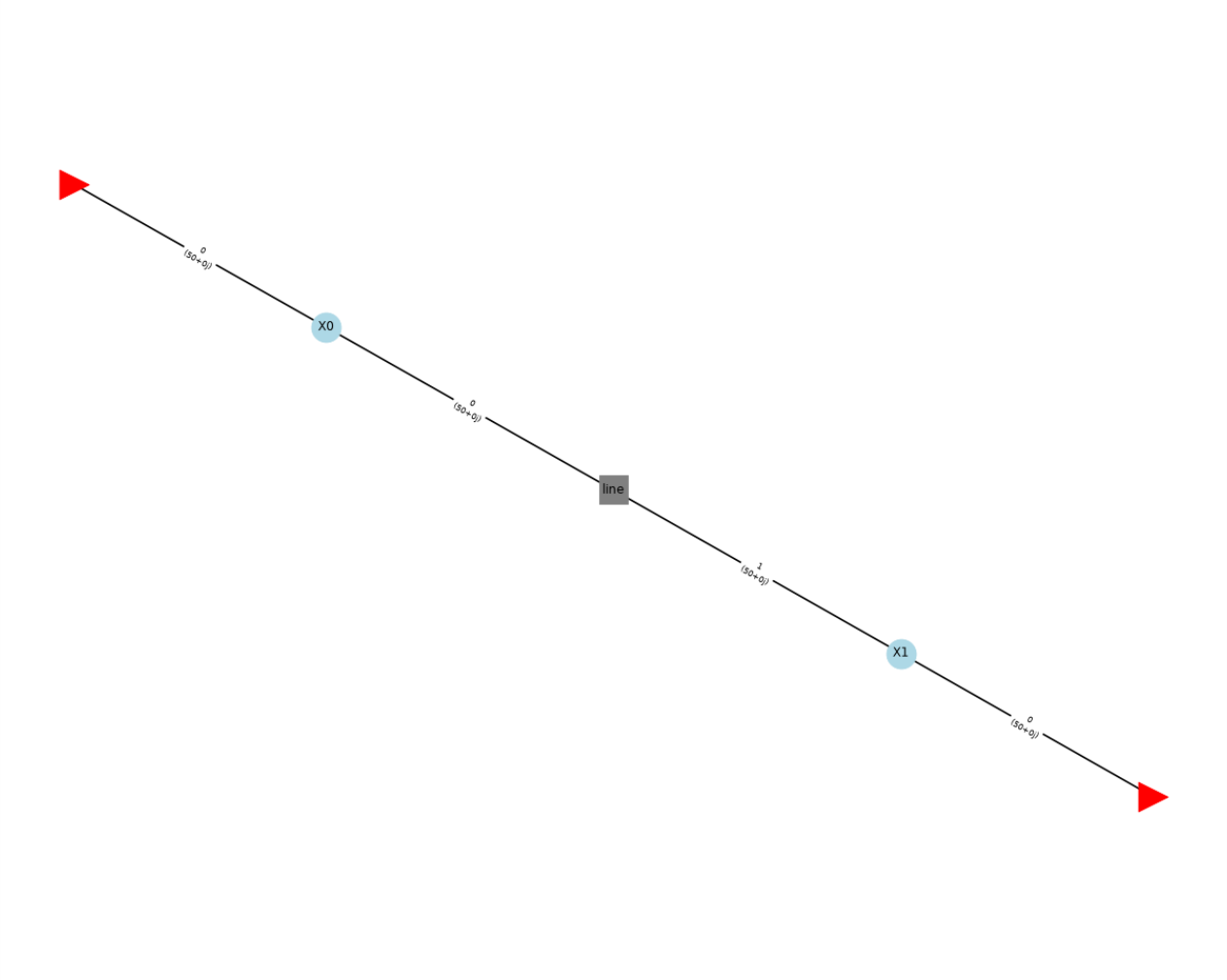
```
Output voltage and current: [-8.39071529+5.44021111j] V and [-0.16781431+0.10880422j] A
```

Let's perform the same calculation using `Circuit`. First, one needs to define the circuit, that is to create input/output ports and to connect these ports to the transmission line Network we've already created. Then, we can build the circuit:

[7]:
```
port1 = Circuit.Port(frequency=freq, name='port1', z0=50)
port2 = Circuit.Port(frequency=freq, name='port2', z0=50)
cnx = [
    [(port1, 0), (line, 0)],
    [(port2, 0), (line, 1)]
]
crt = Circuit(cnx)
```

It's always a good practice to check if the circuit's graph is as expected. In this case, the graph is pretty simple: 2 ports connected to a 2-ports Network.

[8]:
```
crt.plot_graph(network_labels=True, edge_labels=True, inter_labels=True)
```



`Circuit` provides two methods to determine voltages and currents at the circuit input/output ports (also known as "external ports"). These methods take as input the power and phase inputs at each ports:

[9]:
```
power = [1, 0]  # 1 Watt at port1 and 0 at port2
phase = [0, 0]  # 0 radians
```

[10]:
```
V_at_ports = crt.voltages_external(power, phase)
print(V_at_ports)
```

```
[[10.        +0.j         -8.39071529+5.44021111j]]
```

[11]:
```
I_at_ports = crt.currents_external(power, phase)
print(I_at_ports)
```

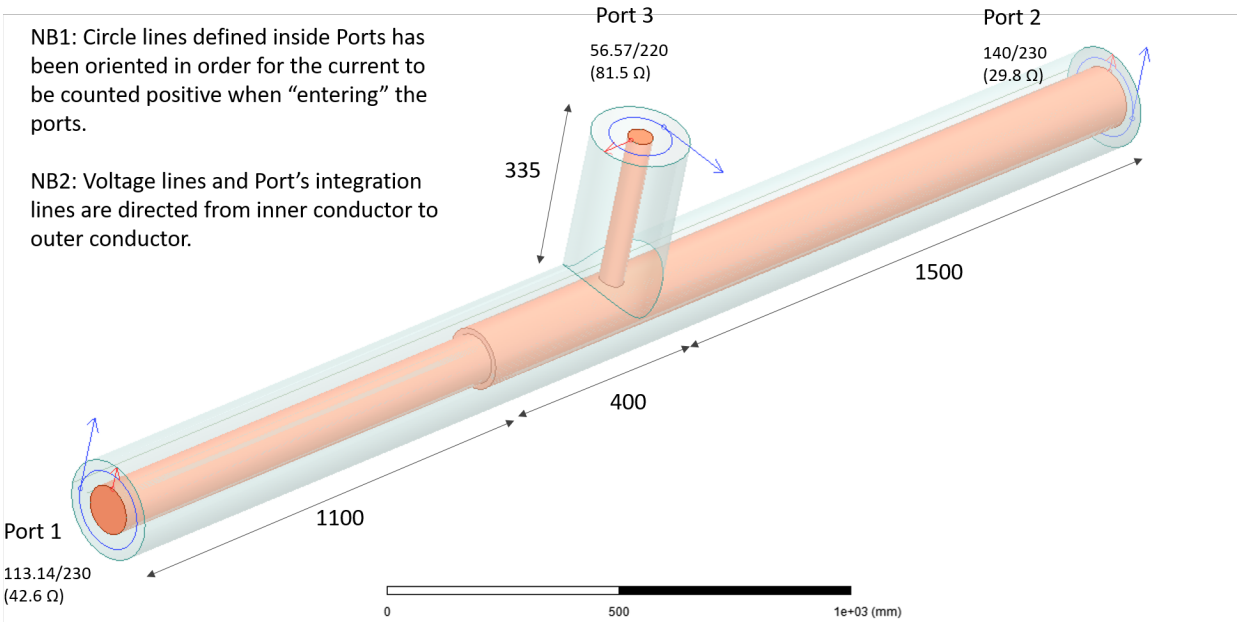
```
[[0.2       +0.j         0.16781431-0.10880422j]]
```

The results are similar to the previous, except the sign of the current at port2 which is reversed.

This is normal, as the `currents_external()` method defines the positive direction of a current as the direction which "enters" into the circuit. More details about this convention at the bottom of this example.

A coaxial T with different characteristic impedances

As a more advanced example, we've built in a full-wave software (here ANSYS HFSS, but other are fine too) the following structure: a coaxial T, defined with different coaxial cross-sections (and such different characteristic impedances).



All the three ports are excited with different power and phase to complicate a bit, as the following:

port	power [W]	phase [deg]
#1	1	-10
#2	2	-20
#3	3	+60

Full-wave reference solution

The voltages and currents are evaluated in the full-wave software directly. Voltage is obtained by integrating the E fields along a straight line going from the inner to the outer conductors in the port's cross-sections. Current is obtained by integrating the H fields along a circle enclosing the inner conductor in port's cross-sections. Note the circles are directed in order to define the positive current direction as the direction inward the ports (right-hand rule).

The full-wave solutions are given here for reference, at 3 frequencies:

[12]:
```
import pandas as pd  # convenient to read .csv files

pd.read_csv('circuit_vi_HFSS_Voltages.csv')
```

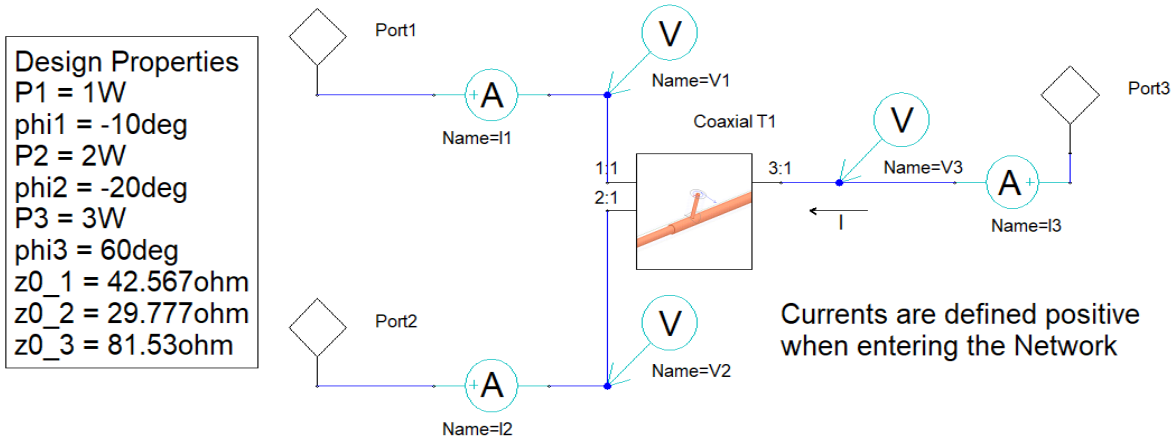
[12]:
```
   Phase [deg]  Freq [MHz]  mag(V1) [V]  mag(V2) [V]  mag(V3) [V]  \
0            0          10    19.333738    20.846031    15.272200
1            0          50     7.495888    12.520379    14.048313
2            0         100    13.378541     9.825921    41.581058

   ang_deg(V1) [deg]  ang_deg(V2) [deg]  ang_deg(V3) [deg]
0         -10.650177          -9.289474          -9.877413
1         -29.855289         -44.233339         168.419884
2         -38.869964         -35.275652         102.383086
```

[13]:
```
pd.read_csv('circuit_vi_HFSS_Currents.csv')
```

[13]:
```
   Phase [deg]  Freq [MHz]  mag(I1) [A]  mag(I2) [A]  mag(I3) [A]  \
0            0          10     0.021591     0.137606     0.512142
1            0          50     0.275976     0.389991     0.622469
2            0         100     0.220526     0.424061     0.384085

   ang_deg(I1) [deg]  ang_deg(I2) [deg]  ang_deg(I3) [deg]
0         156.244427         -90.890034          80.224971
1           2.596668           6.317826          44.672585
2          33.833593          -8.087971          -4.331087
```

In a Circuit simulator

The voltages and currents can also be derived using a Circuit simulator, like for example:



(where, again, the current probes are oriented in such a way that the current is positive when flowing in the Network). This gives essentially the same results:

[14]:
```
pd.read_csv('circuit_vi_Designer_Voltages.csv')
```

[14]:
```
   Freq [MHz]  mag(V(V1)) [V]  mag(V(V2)) [V]  mag(V(V3)) [V]  \
0          10       19.342985       20.849054       15.290036
1          50        7.499412       12.522681       14.064961
2         100       13.385809        9.828158       41.631139

   ang_deg(V(V1)) [deg]  ang_deg(V(V2)) [deg]  ang_deg(V(V3)) [deg]
0            -10.644150             -9.283262             -9.870625
1            -29.854300            -44.230084            168.423657
2            -38.866437            -35.272131            102.384693
```

[15]:
```
pd.read_csv('circuit_vi_Designer_Currents.csv')
```

[15]:
```
   Freq [MHz]  mag(I(I1)) [A]  mag(I(I2)) [A]  mag(I(I3)) [A]  \
0          10        0.021481        0.137779        0.509421
1          50        0.274448        0.389875        0.619155
2         100        0.219239        0.423718        0.381925
```

(continues on next page)

(continued from previous page)

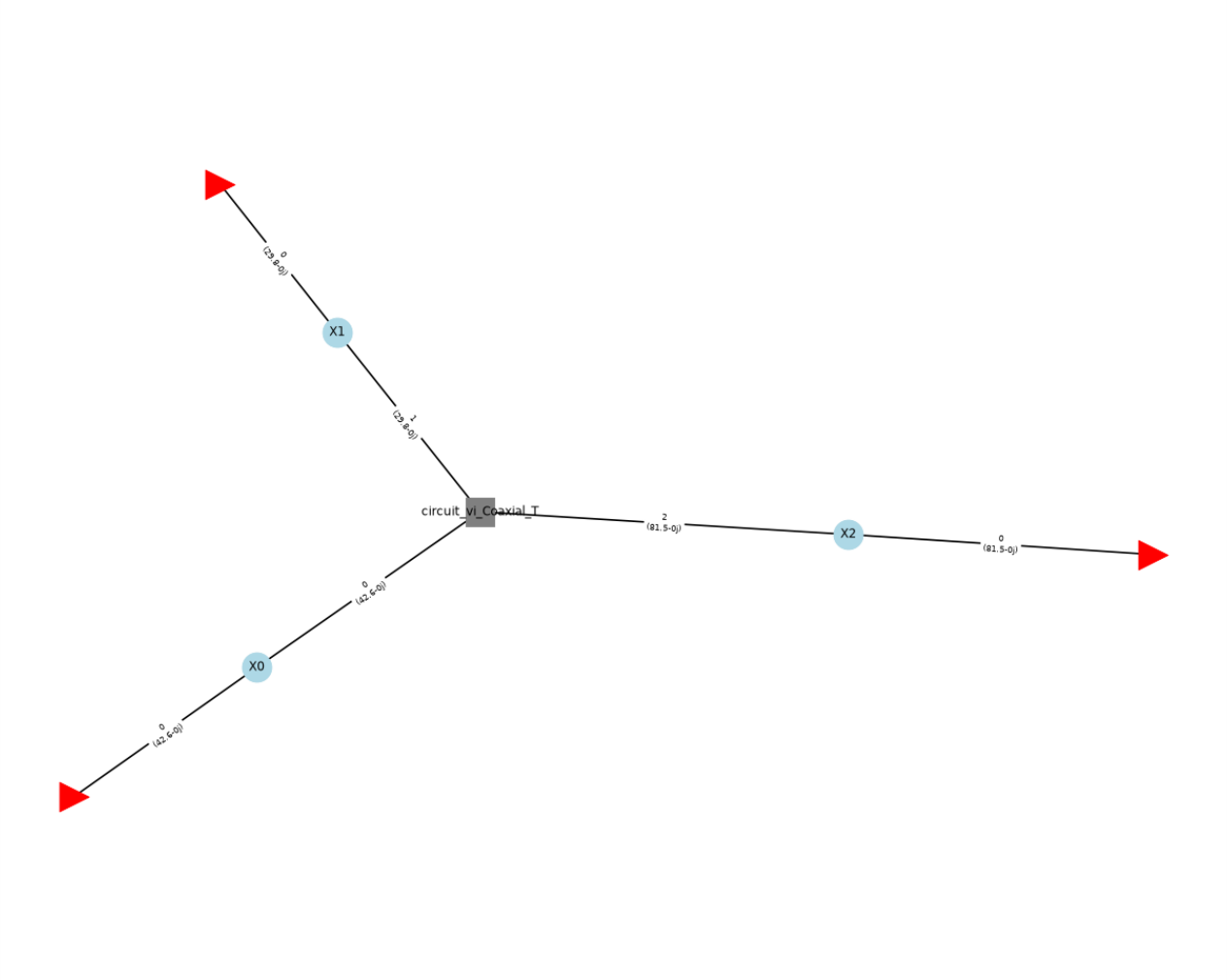
```
   ang_deg(I(I1)) [deg]  ang_deg(I(I2)) [deg]  ang_deg(I(I3)) [deg]
0            156.253941            -90.892977             80.229605
1              2.599727              6.286661             44.674195
2             33.836727             -8.152957             -4.333328
```

With `Circuit`

Now let's build the same circuit with scikit-rf `Circuit`:

[16]:
```
# Importing the 3-port .s3p file exported from full-wave simulation
coaxial_T = rf.Network('circuit_vi_Coaxial_T.s3p')
# pay attention to the port's characteristic impedance
# it should match the Network characteristic impedances otherwise this will generate␣
↪mismatches
port1 = Circuit.Port(coaxial_T.frequency, 'port1', coaxial_T.z0[:,0])
port2 = Circuit.Port(coaxial_T.frequency, 'port2', coaxial_T.z0[:,1])
port3 = Circuit.Port(coaxial_T.frequency, 'port3', coaxial_T.z0[:,2])
# connection list
cnx = [
    [(port1, 0), (coaxial_T, 0)],
    [(port2, 0), (coaxial_T, 1)],
    [(port3, 0), (coaxial_T, 2)]
]
# building the circuit
crt = Circuit(cnx)
```

[17]:
```
# let's check if our connection list is correctly defined
crt.plot_graph(network_labels=True, edge_labels=True, inter_labels=True)
```



The voltages and currents at the ports for the given excitation is:

[18]:
```
power = [1, 2, 3]  # input power in watts at ports 1, 2 and 3
phase = np.deg2rad([-10, -20, +60])  # input phase in rad at ports 1, 2 and 3

voltages = crt.voltages_external(power, phase)
currents = crt.currents_external(power, phase)
```

[19]:
```
# just for a better rendering in the notebook
pd.concat([
    pd.DataFrame(np.abs(voltages), columns=['mag V1', 'mag V2', 'mag V3'], index=crt.
↪frequency.f/1e6),
    pd.DataFrame(np.angle(voltages, deg=True), columns=['Phase V1', 'Phase V2', 'Phase V3
↪'], index=crt.frequency.f/1e6)
], axis=1)
```

[19]:

	mag V1	mag V2	mag V3	Phase V1	Phase V2	Phase V3
10.0	19.342535	20.848763	15.289548	-10.650167	-9.289472	-9.877400
50.0	7.499300	12.522020	14.064485	-29.855280	-44.233336	168.419396
100.0	13.384629	9.827209	41.628636	-38.869951	-35.275642	102.382983

[20]:
```
# just for a better rendering in the notebook
pd.concat([
    pd.DataFrame(np.abs(currents), columns=['mag I1', 'mag I2', 'mag I3'], index=crt.
↪frequency.f/1e6),
    pd.DataFrame(np.angle(currents, deg=True), columns=['Phase I1', 'Phase I2', 'Phase I3
↪'], index=crt.frequency.f/1e6)
], axis=1)
```

[20]:

	mag I1	mag I2	mag I3	Phase I1	Phase I2	Phase I3
10.0	0.021471	0.137796	0.509429	156.241420	-90.898041	80.225072
50.0	0.274422	0.389839	0.619125	2.597008	6.284260	44.671994
100.0	0.219233	0.423683	0.381901	33.834350	-8.154641	-4.333857

These results matches well the one given by the full-wave calculations, hourra.

external vs internal ports?

You have maybe noticed in the `Circuit` documentation that we often talk about "internal" or "inner" ports, and of "external" or "outer" ports. The external ports corresponds to the `Circuit.Port()` Networks defined when building a `Circuit`. The internal ports are all the other connexions inside the `Circuit`

The `Circuit` algorithm allows one to have access to the voltages and current at the internal connexions inside the circuit. In the previous examples, there is not too much internal ports as we've connected a N-port directly to external ports. However, in more complex usages we can have a lot (look to the other `Circuit` examples).

In `Circuit`, voltages and currents are peak values. While voltages are defined in a non-ambiguous manner, positive currents can be defined in a way or another, leading to a 180 degree ambiguity. To solve this, we have chosen the following definition: internal currents are defined such as they are measured positive when flowing into Networks.

Hence, you find that "external" current are sign opposite of "internal" ones at the corresponding ports, because the internal currents are actually flowing into the ports Networks.

[21]:
```
# internals currents (currents at all connexions)
# in this example, there are 6 internal connexions (3 pairs of connexions)
crt.currents(power, phase)
```

[21]:
```
array([[ 0.0196517 -0.00865047j, -0.0196517 +0.00865047j,
         0.00215969+0.13777892j, -0.00215969-0.13777892j,
        -0.08649001-0.50203344j,  0.08649001+0.50203344j],
       [-0.27414049-0.0124343j ,  0.27414049+0.0124343j ,
        -0.38749668-0.04267229j,  0.38749668+0.04267229j,
        -0.44028632-0.43527388j,  0.44028632+0.43527388j],
       [-0.18210634-0.12206774j,  0.18210634+0.12206774j,
        -0.41939939+0.0600975j ,  0.41939939-0.0600975j ,
        -0.38080855+0.02885945j,  0.38080855-0.02885945j]])
```

[22]:
```
# This gives the indices of the "external" ports
crt.port_indexes
```

[22]:
```
[0, 2, 4]
```

[23]:
```
# So we can keep only "external" ports
crt.currents(power, phase)[:, crt.port_indexes]
```

[23]:
```
array([[ 0.0196517 -0.00865047j,  0.00215969+0.13777892j,
        -0.08649001-0.50203344j],
```

(continues on next page)

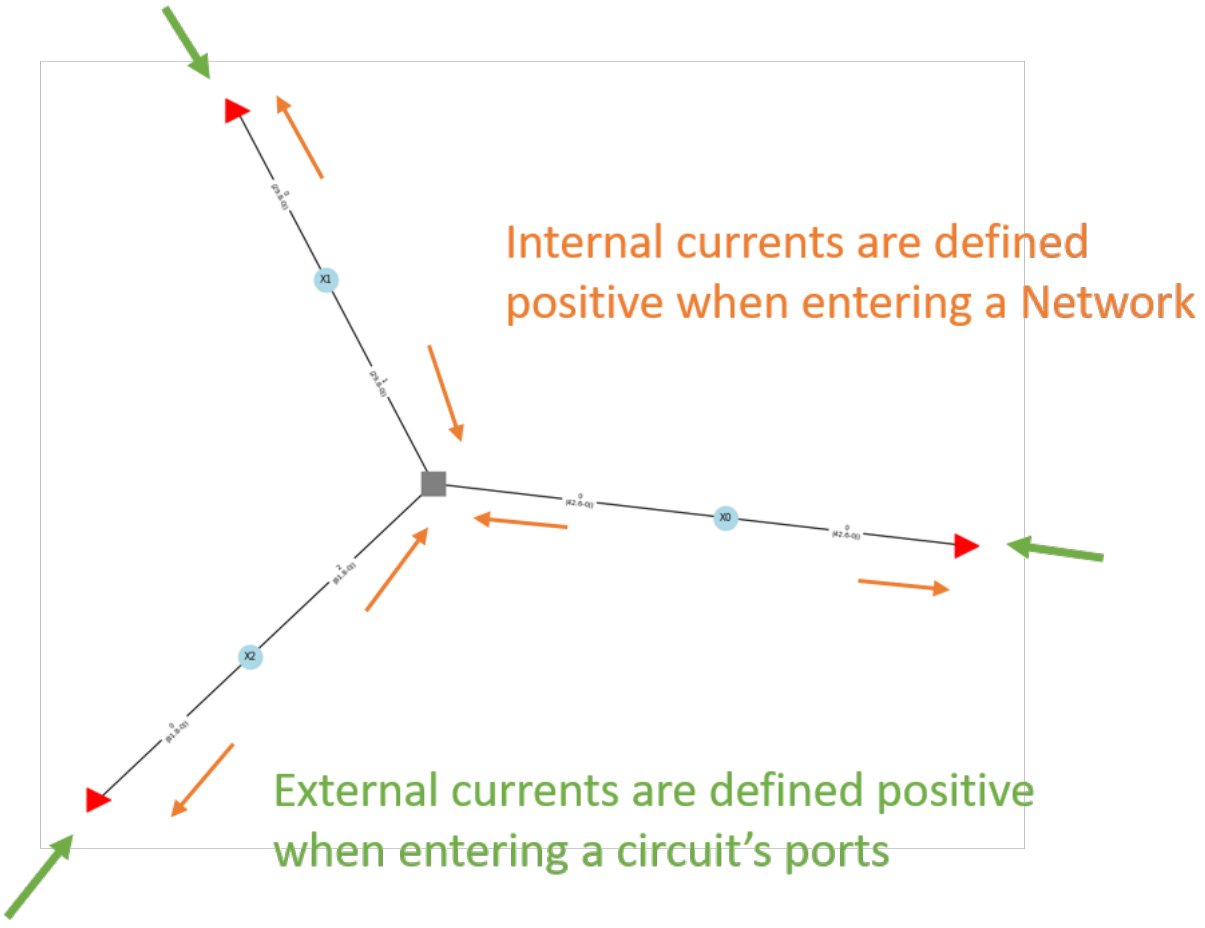
(continued from previous page)

```
       [-0.27414049-0.0124343j , -0.38749668-0.04267229j,
        -0.44028632-0.43527388j],
       [-0.18210634-0.12206774j, -0.41939939+0.0600975j ,
        -0.38080855+0.02885945j]])
```

[24]:
```
# note the sign difference due to the convention chosen for internal ports
crt.currents_external(power, phase)
```

[24]:
```
array([[-0.0196517 +0.00865047j, -0.00215969-0.13777892j,
         0.08649001+0.50203344j],
       [ 0.27414049+0.0124343j ,  0.38749668+0.04267229j,
         0.44028632+0.43527388j],
       [ 0.18210634+0.12206774j,  0.41939939-0.0600975j ,
         0.38080855-0.02885945j]])
```

The figure below illustrates these differences using the previous example:



Voltages and Currents in complex Circuits

power-waves and traveling-waves

In `scikit-rf`, the calculation of `Voltages` and `Currents` relies on the Scattering Parameters (`S-parameters`) of the `Circuit`. The default definition of `S-parameters` uses power-waves, expressed as:

$$s_{ij} = \frac{b_i}{a_j}$$

where b_i and a_j represent the output and input power waves at ports `i` and `j` in a given `Network`, respectively.

As an example, the methods `Circuit._b()` and `Circuit._a()` provide support for calculating these output and input power waves, although this is often unnecessary for typical use cases.

[25]:
```
# To obtain the input/output power waves under given phase and amplitude, you can use:
a = crt._a(crt._a_external(power, phase))
b = crt._b(a)
```

From the `S-parameters` definition, we can determine the forward (input) traveling wave V_i^+ and the backward (output) traveling wave V_j^- at ports `i` and `j` in a given `Network` as:

$$V_i^+ = a_i \sqrt{\Re(Z_{0i})}$$

$$V_j^- = b_j \sqrt{\Re(Z_{0j})}$$

Here, Z_{0i} and Z_{0j} are the characteristic impedances at port `i` and `j`, respectively.

Voltage and Traveling waves

In transmission line theory, the total voltage and total current are functions of the incident and reflected voltage wave amplitudes. Similarly, the `Voltage` and `Current` at each node in a `Circuit` can be calculated using the forward (input) and backward (output) traveling wave at the components' ports that form the node.

For simple series connections, such as when port `m` of `Network` A is connected to port `n` of `Network` B, the total `Voltage` at the node can be calculated using the same principles as for transmission lines:

$$V = V_m^- + V_n^-$$

Here, the V_m^- and V_n^- are the backward (output) traveling waves at ports `m` and `n`, respectively.

Voltage under impedance mismatch

When the port characteristic impedances of `Network` A's port `m` (Z_{0Am}) and `Network` B's port `n` (Z_{0Bn}) are different, impedance mismatch occurs. This requires the use of the transmission coefficient $_{nm}$ to correct the mismatch.

When a voltage wave is transmitted from `Network` A' port `m` to `Network` B's port `n`, `Network` A can be treated as the source and `Network` B as the load. The transmission coefficient $_{nm}$ from `m` to `n` can then be used to adjust for the mismatch:

$$_{nm} = \frac{2Z_{Load}}{Z_{Load} + Z_{Source}} = \frac{2Z_{0Am}}{Z_{0Am} + Z_{0Bn}}$$

Complex Circuits with Parallel Connections

In more complex `Circuits`, such as those with parallel connections, multiple ports may converge at a node, each potentially having a different characteristic impedance. Despite the added complexity, the approach of using the transmission coefficient remains effective.

By selecting one `Network`'s characteristic impedance Z_l as the source impedance and treating the combined characteristic impedance of the other k parallel `Networks` as the load impedance, the actual transmission coefficient for port 1 can be determined:

$$\frac{1}{Z_{Source}} = \sum_{i=0}^{k} \frac{1}{Z_i} - \frac{1}{Z_l}$$

This approach allows for the calculation of the effective impedance at the node, which can then be used to adjust for impedance mismatches and ensure accurate `Voltage` and `Current` calculations in complex parallel `Networks` configurations.

Example

The Wilkinson Power Divider provides an example of voltage and current calculations in complex circuits. It demonstrates the calculation for both purely series-connected `Circuit` using splitters and more complex `Circuit` that involve component parallelization, yielding consistent results in both scenarios.

[ ]:

2.6.3 Wilkinson Power Divider

In this notebook we create a Wilkinson power divider, which splits an input signal into two equals phase output signals. Theoretical results about this circuit are exposed in reference [1]. Here we will reproduce the ideal circuit illustrated below and discussed in reference [2]. In this example, the circuit is designed to operate at 1 GHz.

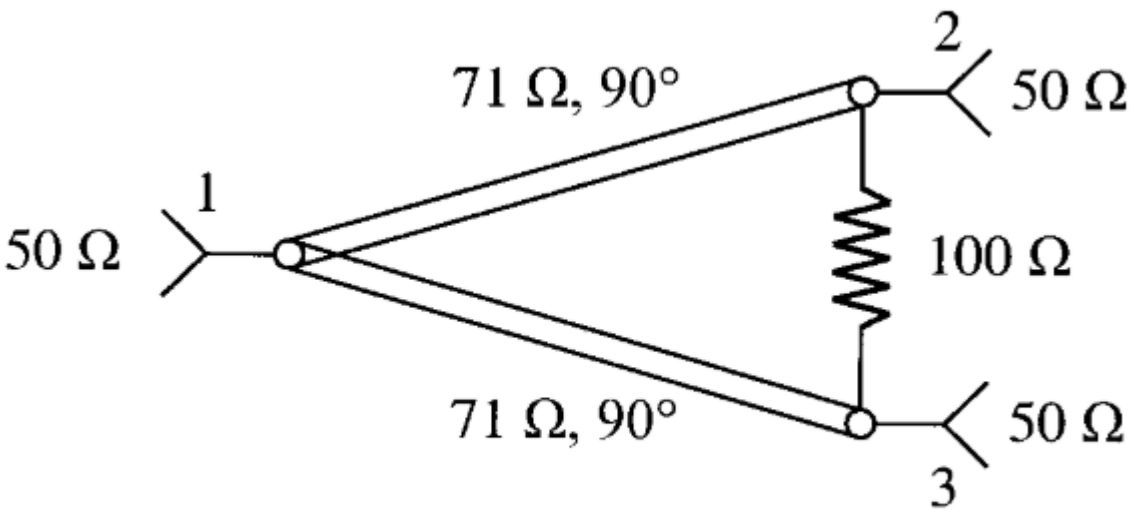


Figure 4 Wilkinson power divider

- [1] P. Hallbjörner, Microw. Opt. Technol. Lett. 38, 99 (2003).
- [2] Microwaves 101: "Wilkinson Power Splitters"

[1]:
```
# standard imports
import matplotlib.pyplot as plt
import numpy as np
```

(continues on next page)

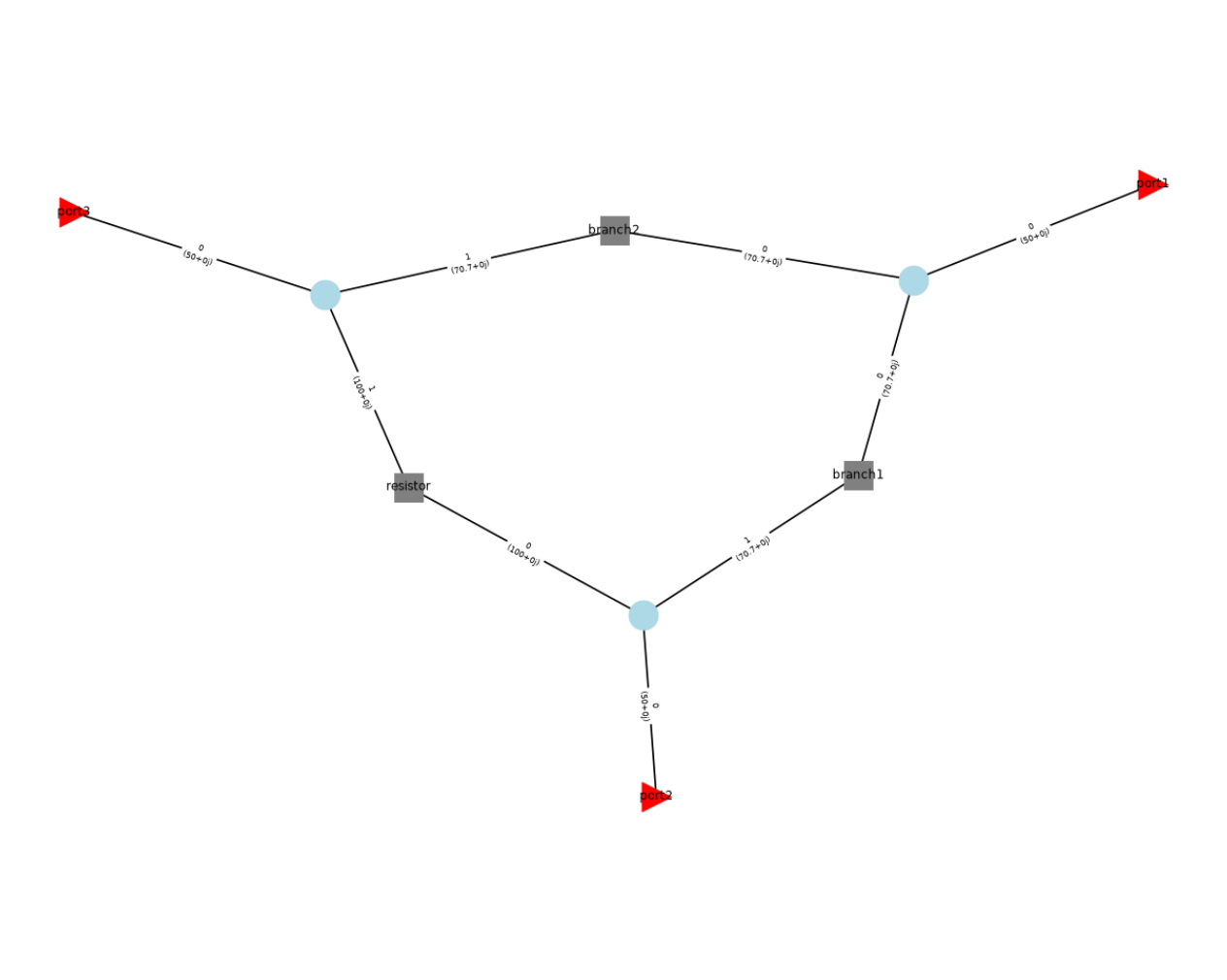
(continued from previous page)

```
import skrf as rf
from skrf.circuit import Circuit

rf.stylely()
```

[2]:
```
# frequency band
freq = rf.Frequency(start=0, stop=2, npoints=501, unit='GHz')

# characteristic impedance of the ports
z0_ports = 50

# resistor
R = 100
line_resistor = rf.media.DefinedGammaZ0(frequency=freq, z0=R)
resistor = line_resistor.resistor(R, name='resistor')

# branches
z0_branches = np.sqrt(2)*z0_ports
beta = freq.w/rf.constants.c
line_branches = rf.media.DefinedGammaZ0(frequency=freq, z0=z0_branches, gamma=0+beta*1j)

d = line_branches.theta_2_d(90, deg=True)  # @ 90°(lambda/4)@ 1 GHz is ~ 75 mm
branch1 = line_branches.line(d, unit='m', name='branch1')
branch2 = line_branches.line(d, unit='m', name='branch2')

# ports
port1 = Circuit.Port(freq, name='port1', z0=50)
port2 = Circuit.Port(freq, name='port2', z0=50)
port3 = Circuit.Port(freq, name='port3', z0=50)

# Connection setup
#Note that the order of appearance of the port in the setup is important
connections = [
           [(port1, 0), (branch1, 0), (branch2, 0)],
           [(port2, 0), (branch1, 1), (resistor, 0)],
           [(port3, 0), (branch2, 1), (resistor, 1)]
       ]

# Building the circuit
C = Circuit(connections)
```

The circuit setup can be checked by visualising the circuit graph (this requires the python package `networkx` to be available).

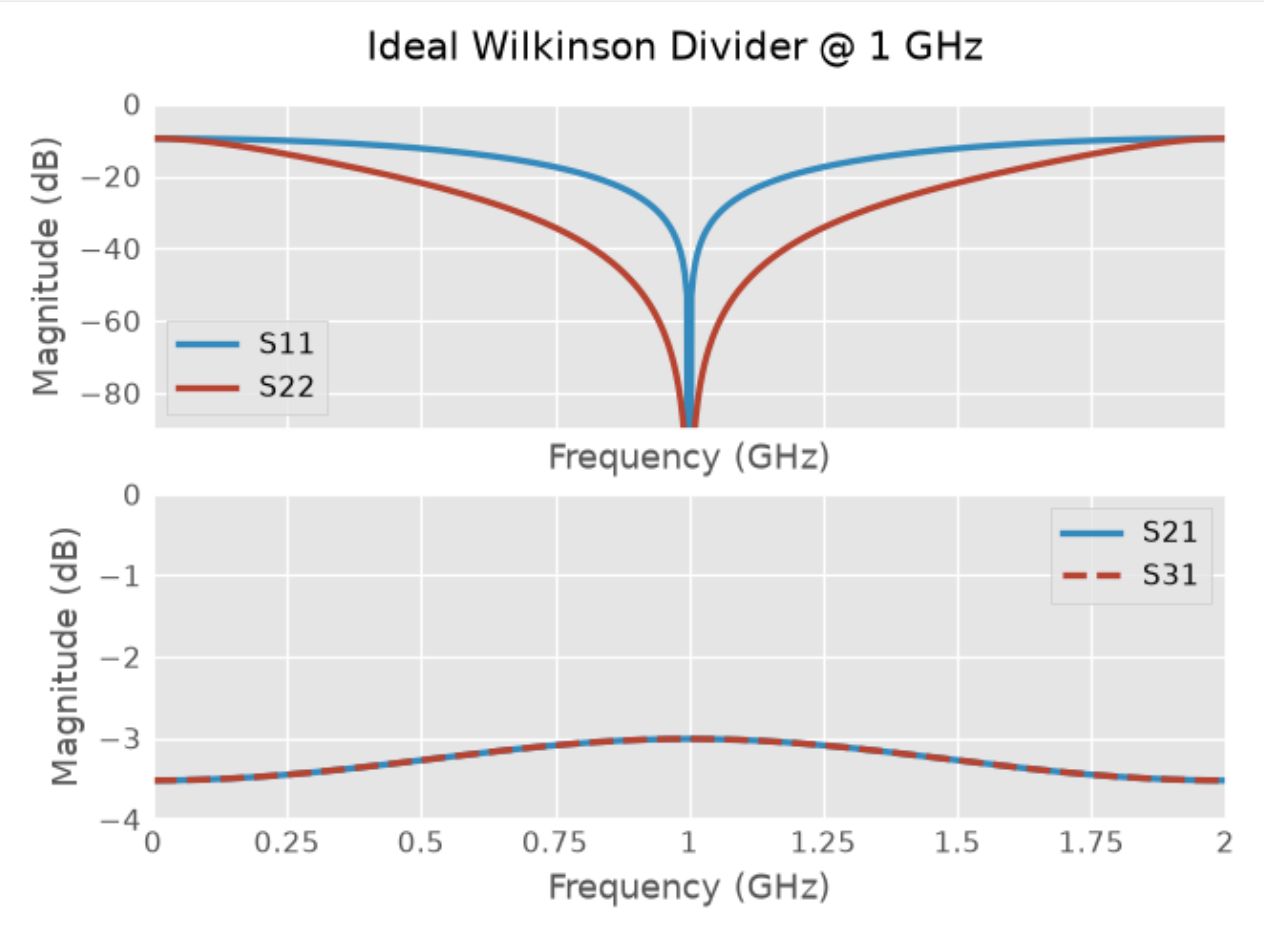
[3]:
```
C.plot_graph(network_labels=True, edge_labels=True, port_labels=True, port_fontize=2)
```



Let's look to the scattering parameters of the circuit:

[4]:
```
fig, (ax1,ax2) = plt.subplots(2, 1, sharex=True)
C.network.plot_s_db(ax=ax1, m=0, n=0,  lw=2)  # S11
C.network.plot_s_db(ax=ax1, m=1, n=1,  lw=2)  # S22
ax1.set_ylim(-90, 0)
C.network.plot_s_db(ax=ax2, m=1, n=0,  lw=2)  # S21
C.network.plot_s_db(ax=ax2, m=2, n=0,  ls='--', lw=2)  # S31
ax2.set_ylim(-4, 0)
fig.suptitle('Ideal Wilkinson Divider @ 1 GHz')
```

[4]:
```
Text(0.5, 0.98, 'Ideal Wilkinson Divider @ 1 GHz')
```



Currents and Voltages

In the previous examples, we didn't have many internal ports because we connected the N ports directly to the external ports. However, in more complex scenarios, circuits can have many internal ports.

Additionally, the earlier examples featured circuits with components connected in series. As circuit complexity increases, it becomes inevitable that some components will be connected in parallel, which introduces new challenges for calculating current and voltage.

Fortunately, scikit-rf is well-equipped to handle these complex circuit scenarios. It is possible to calculate the currents and voltages at the circuit's internal ports.

[5]:
```
power = [1,0,0]
phase = [0,0,0]
C.voltages(power, phase)  # or C2.currents(power, phase)
```

[5]:
```
array([[ 6.66666667+0.00000000e+00j,  6.66666667+0.00000000e+00j,
         6.66666667+0.00000000e+00j, ...,  6.66666667+0.00000000e+00j,
         6.66666667+0.00000000e+00j,  6.66666667+0.00000000e+00j],
       [ 6.66678364+1.97457133e-02j,  6.66678364+1.97457133e-02j,
         6.66678364+1.97457133e-02j, ...,  6.66656431-3.94922062e-02j,
         6.66656431-3.94922062e-02j,  6.66656431-3.94922062e-02j],
       [ 6.66713454+3.94888282e-02j,  6.66713454+3.94888282e-02j,
         6.66713454+3.94888282e-02j, ...,  6.66625725-7.89838926e-02j,
```

(continues on next page)

(continued from previous page)

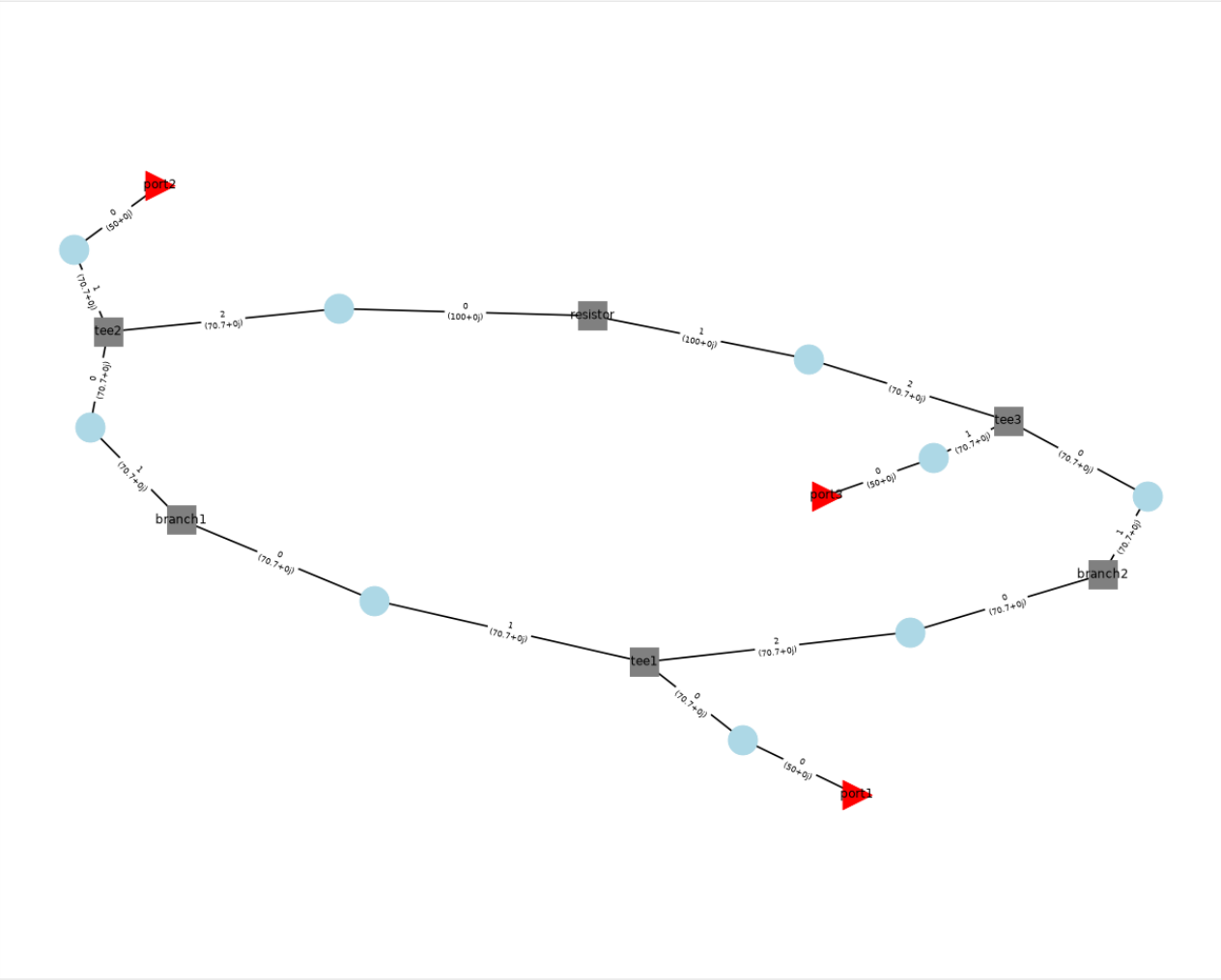
```
         6.66625725-7.89838926e-02j,  6.66625725-7.89838926e-02j],
       ...,
       [ 6.66713454-3.94888282e-02j,  6.66713454-3.94888282e-02j,
         6.66713454-3.94888282e-02j, ..., -6.66625725-7.89838926e-02j,
        -6.66625725-7.89838926e-02j, -6.66625725-7.89838926e-02j],
       [ 6.66678364-1.97457133e-02j,  6.66678364-1.97457133e-02j,
         6.66678364-1.97457133e-02j, ..., -6.66656431-3.94922062e-02j,
        -6.66656431-3.94922062e-02j, -6.66656431-3.94922062e-02j],
       [ 6.66666667+1.01076838e-15j,  6.66666667+1.01076838e-15j,
         6.66666667+1.01076838e-15j, ..., -6.66666667+2.02153677e-15j,
        -6.66666667+2.02153677e-15j, -6.66666667+2.02153677e-15j]],
      shape=(501, 9))
```

Alternatively, you can use splitting Networks, such as "T" Networks in our case, to create circuits with only pairs of connections and perform current and voltage calculations. While using splitting Networks aligns more closely with 'classic' way for building a circuit, it has disadvantages in terms of code readability, complexity, and computational performance.

[6]:
```
tee1 = line_branches.tee(name='tee1')
tee2 = line_branches.tee(name='tee2')
tee3 = line_branches.tee(name='tee3')

cnx = [
    [(port1, 0), (tee1, 0)],
    [(tee1, 1), (branch1, 0)],
    [(tee1, 2), (branch2, 0)],
    [(branch1, 1), (tee2, 0)],
    [(branch2, 1), (tee3, 0)],
    [(tee2, 2), (resistor, 0)],
    [(tee3, 2), (resistor, 1)],
    [(tee3, 1), (port3, 0)],
    [(tee2, 1), (port2, 0)],
]
C2 = Circuit(cnx)
```

The resulting graph is a bit more stuffed:

[7]:
```
C2.plot_graph(network_labels=True, edge_labels=True, port_labels=True, port_fontize=2)
```



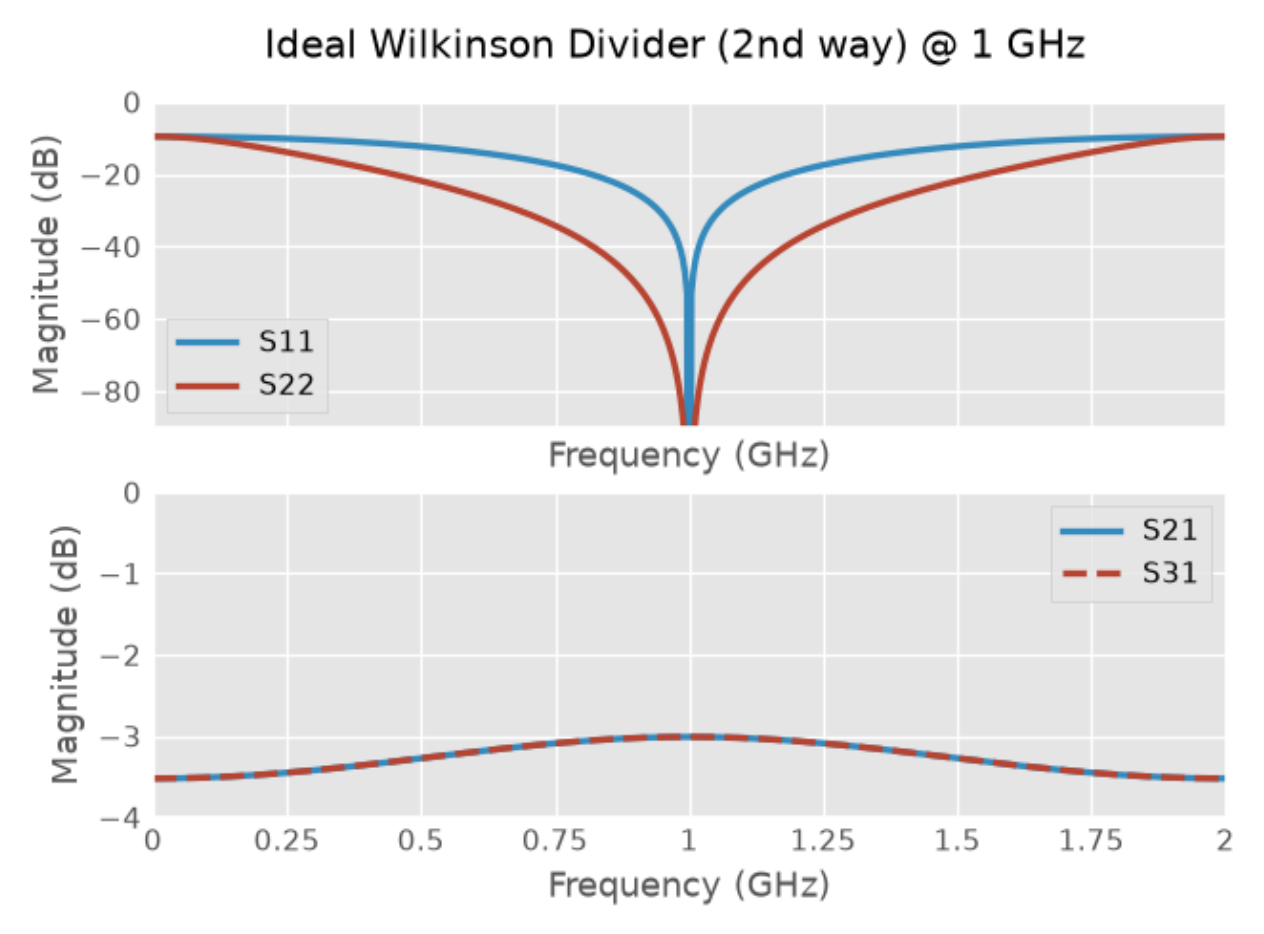
But the results are the same:

[8]:
```
C.network == C2.network
```

[8]:
```
True
```

[9]:
```
fig, (ax1,ax2) = plt.subplots(2, 1, sharex=True)
C2.network.plot_s_db(ax=ax1, m=0, n=0,  lw=2)  # S11
C2.network.plot_s_db(ax=ax1, m=1, n=1,  lw=2)  # S22
ax1.set_ylim(-90, 0)
C2.network.plot_s_db(ax=ax2, m=1, n=0,  lw=2)  # S21
C2.network.plot_s_db(ax=ax2, m=2, n=0,  ls='--', lw=2)  # S31
ax2.set_ylim(-4, 0)
fig.suptitle('Ideal Wilkinson Divider (2nd way) @ 1 GHz')
```

[9]:
```
Text(0.5, 0.98, 'Ideal Wilkinson Divider (2nd way) @ 1 GHz')
```



And the internal voltages and currents are also consistent.

[10]:
```
C2.voltages(power, phase)  # or C2.currents(power, phase)
```

[10]:
```
array([[ 6.66666667+0.00000000e+00j,  6.66666667+0.00000000e+00j,
         6.66666667+0.00000000e+00j, ...,  6.66666667+0.00000000e+00j,
         6.66666667+0.00000000e+00j,  6.66666667+0.00000000e+00j],
       [ 6.66678364+1.97457133e-02j,  6.66678364+1.97457133e-02j,
         6.66678364+1.97457133e-02j, ...,  6.66656431-3.94922062e-02j,
         6.66656431-3.94922062e-02j,  6.66656431-3.94922062e-02j],
       [ 6.66713454+3.94888282e-02j,  6.66713454+3.94888282e-02j,
         6.66713454+3.94888282e-02j, ...,  6.66625725-7.89838926e-02j,
         6.66625725-7.89838926e-02j,  6.66625725-7.89838926e-02j],
       ...,
       [ 6.66713454-3.94888282e-02j,  6.66713454-3.94888282e-02j,
         6.66713454-3.94888282e-02j, ..., -6.66625725-7.89838926e-02j,
        -6.66625725-7.89838926e-02j, -6.66625725-7.89838926e-02j],
       [ 6.66678364-1.97457133e-02j,  6.66678364-1.97457133e-02j,
         6.66678364-1.97457133e-02j, ..., -6.66656431-3.94922062e-02j,
        -6.66656431-3.94922062e-02j, -6.66656431-3.94922062e-02j],
       [ 6.66666667+1.01076838e-15j,  6.66666667+1.01076838e-15j,
         6.66666667+1.01076838e-15j, ..., -6.66666667+2.02153677e-15j,
        -6.66666667+2.02153677e-15j, -6.66666667+2.02153677e-15j]],
      shape=(501, 18))
```

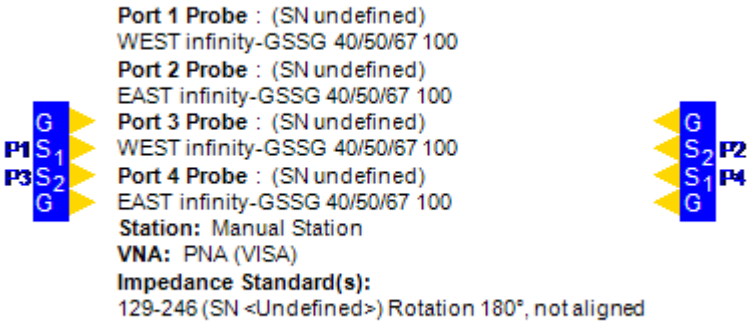
You will find more details on voltages and currents calculation on the dedicated example.

[ ]:

2.7 Mixed Mode

2.7.1 Mixed Mode S-Parameter Conversion

When analyzing differential devices, mixed-mode S-Parameters are typically used to look at differential and common mode characteristics. Scikit-rf provides functions to ease conversion between single ended and mixed mode S-parameters. Nevertheless, the user must be careful to set up ports correctly and take note of the form of the mixed-mode matrix to prevent confusion. This notebook will introduce you to the process of converting from single ended to mixed mode S parameters using a 50 ohm calibration load as an example. The experimental port setup consists of two differential probes as shown:



Experimental data for the 50 ohm load taken in normal "single ended" mode will be compared against data saved in a mixed mode format and taken using Keysight's integrated True Mode Stimulus, which applies true differential and common mode signals during the measurement process.

[1]:
```
import re

import matplotlib.pyplot as plt
import numpy as np

import skrf as rf

sedatafile = r'mixedmodebasics_files/load_se.s4p'
mmdatafile = r'mixedmodebasics_files/load_truemode_balbal.s4p'

for file in [sedatafile, mmdatafile]:
    with open(file, encoding='cp1252') as f:
        for line in f:
            print(line.rstrip())
            if re.search('#', line):
                break
            else:
                pass
    print('\n')
```

```
!  4-Port S-parameters saved by WinCal
! VAR MeasName=S-Parameters (CALIBRATED_DATA) read from VNA (N5225A)
! VAR MeasDate=7/7/2020 2:51:53 PM
! VAR NAME=load_se
! VAR FILENAME=load_se.S4P
! VAR DATE=7/7/2020 2:51:53 PM
! VAR PHYS_PORTS=1,2,3,4
!
# Hz S RI R 50


!Agilent Technologies,N5225A,MY51451011,A.09.90.21
!Agilent N5225A: A.09.90.21
!Date: Tuesday, July 07, 2020 17:08:48
!Correction: Sdd11(Full 4 Port(1,2,3,4))
!Sdc11(Full 4 Port(1,2,3,4))
!Sdd12(Full 4 Port(1,2,3,4))
!Sdc12(Full 4 Port(1,2,3,4))
!Scd11(Full 4 Port(1,2,3,4))
!Scc11(Full 4 Port(1,2,3,4))
!Scd12(Full 4 Port(1,2,3,4))
!Scc12(Full 4 Port(1,2,3,4))
!Sdd21(Full 4 Port(1,2,3,4))
!Sdc21(Full 4 Port(1,2,3,4))
!Sdd22(Full 4 Port(1,2,3,4))
!Sdc22(Full 4 Port(1,2,3,4))
!Scd21(Full 4 Port(1,2,3,4))
!Scc21(Full 4 Port(1,2,3,4))
!Scd22(Full 4 Port(1,2,3,4))
!Scc22(Full 4 Port(1,2,3,4))
!Balanced Topology: BBAL
!S4P File: Measurements: <Sdd11,Sdc11,Sdd12,Sdc12>,
!<Scd11,Scc11,Scd12,Scc12>,
!<Sdd21,Sdc21,Sdd22,Sdc22>,
!<Scd21,Scc21,Scd22,Scc22>:
# Hz S  RI   R 50
```

The header data for the mixed mode data indicates that it is saved in the following format:

$$\begin{bmatrix} \begin{bmatrix} S_{dd} & S_{dc} \\ S_{cd} & S_{cc} \end{bmatrix}_{11} & \begin{bmatrix} S_{dd} & S_{dc} \\ S_{cd} & S_{cc} \end{bmatrix}_{12} \\ \begin{bmatrix} S_{dd} & S_{dc} \\ S_{cd} & S_{cc} \end{bmatrix}_{21} & \begin{bmatrix} S_{dd} & S_{dc} \\ S_{cd} & S_{cc} \end{bmatrix}_{22} \end{bmatrix}$$

It is important to keep this in mind, as this format may vary between different software and hardware. For instance, skrf will transform singled ended data to the following form when two balanced ports are present:

$$\begin{bmatrix} \begin{bmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{bmatrix}_{dd} & \begin{bmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{bmatrix}_{dc} \\ \begin{bmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{bmatrix}_{cd} & \begin{bmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{bmatrix}_{cc} \end{bmatrix}$$

To transform our single ended data, we must first pair the ports as they existed during the experimental setup with ports 1 and 3 making up one balanced port, and 2 and 4 on the other probe. We can then use the `se2gmm()` method

of the skrf.Network class to transform to a mixed mode s-parameter matrix, with the p parameter used to specify the number of mixed mode ports. Skrf will transform the ports in pairs starting at the lowest number ports (1 and 3 after our renumbering) and continue until the matrix contains p mixed mode ports, leaving the remaining ports as single ended.

[2]:
```
sedata = rf.Network(sedatafile)
sedata.renumber([0, 1, 2, 3], [0, 2, 1, 3])  # pair ports as 1,3 and 2,4 to match␣
↪experimental setup
sedata.se2gmm(p=2)  # two balanced ports
# sedata now in form  Sdd  Sdc with each submatrix as S11 S12
#                     Scd  Scc                        S21 S22
```

The following function converts a two-port balanced-balanced network in the skrf format into the format used by Keysight to make data comparisons easier:

[3]:
```
def gmm_reorder(m):
    """
    Reorders data from form 11 12 with each submatrix as dd dc
                            21 22                        cd cc
    to form dd dc with each submatrix as 11 12
            cd cc                        21 22
    """
    b = np.array([1, 0, 0, 0,
                  0, 0, 1, 0,
                  0, 1, 0, 0,
                  0, 0, 0, 1]).reshape(4, 4)
    m = b.dot(m.dot(b))
    return m

mmdata = rf.Network(mmdatafile)
# raw data is mmdata in form S11 S12 with each submatrix as dd dc
#                            S21 S22                        cd cc
for i in range(mmdata.frequency.npoints):
    mmdata.s[i, :, :] = gmm_reorder(mmdata.s[i, :, :])
mmdata.port_modes = ['D', 'D', 'C', 'C']
```

Now that the two networks are in the same format, we can see that skrf does not consider them equal. This is because the networks are not identical, they are merely two measurements of the same device, both with experimental noise and variation. If the tolerances of the comparison are relaxed, the comparison of the networks returns True:

[4]:
```
print(sedata == mmdata) # uses np.allclose with tight tolerances
print(np.allclose(abs(sedata.s), abs(mmdata.s), rtol=1, atol=1e-3)) # relaxed tolerances
```

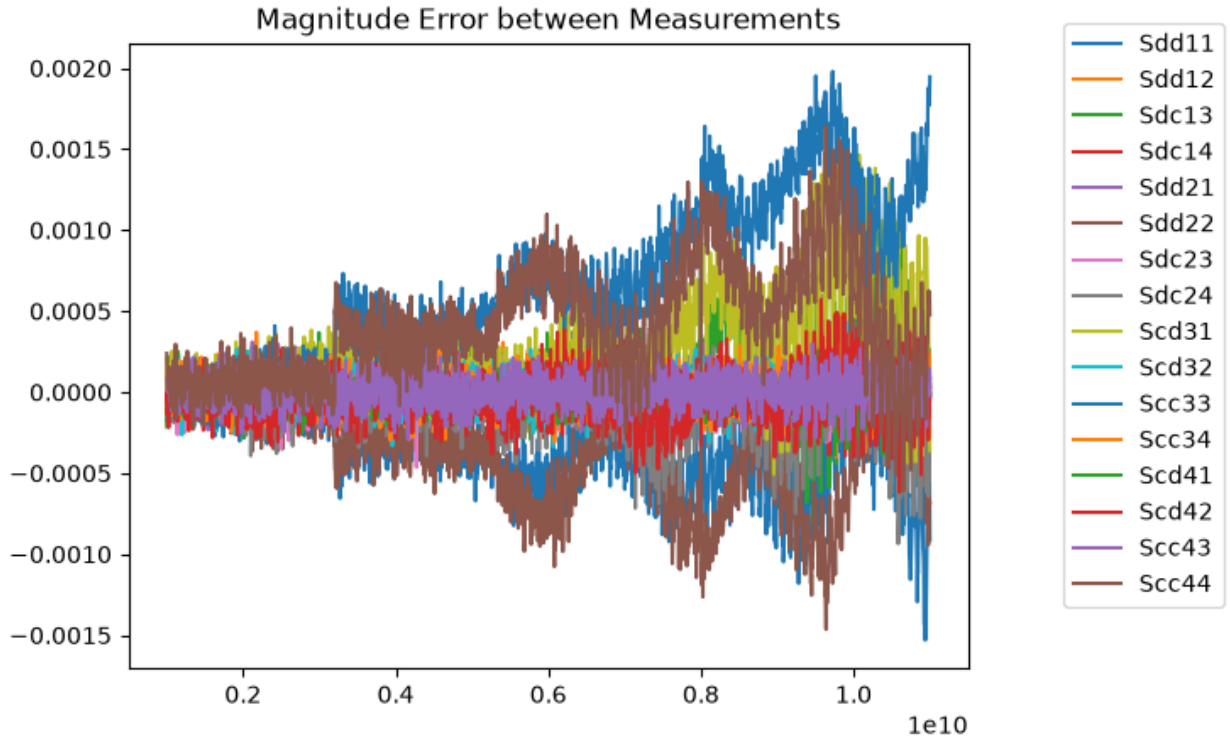
```
False
True
```

Plotted, the error looks like this:

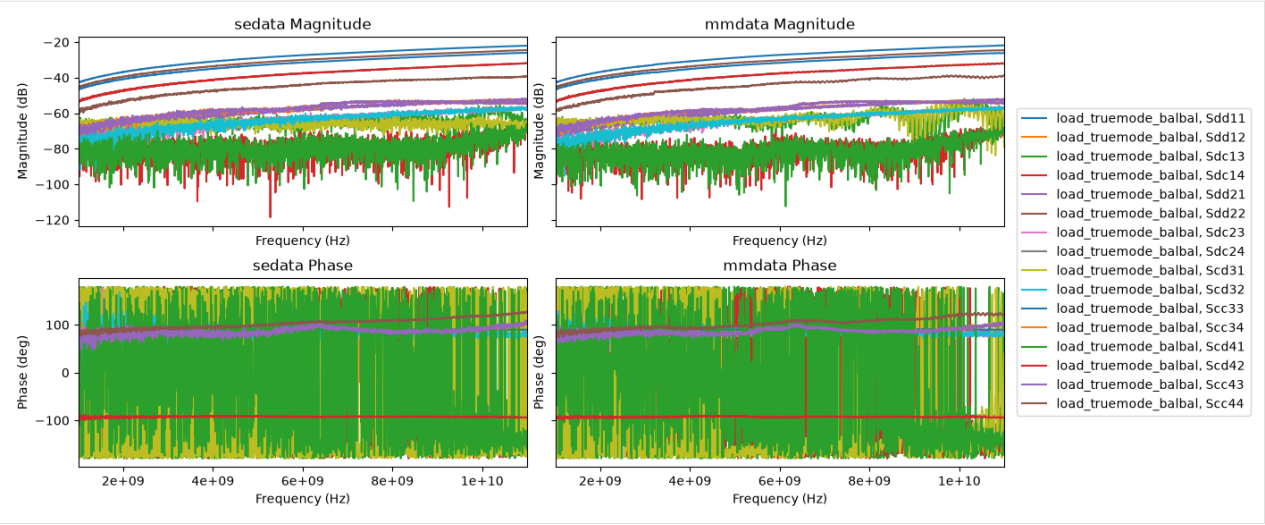
[5]:
```
for m in range(4):
    for n in range(4):
        plt.plot(sedata.f, abs(mmdata.s)[:,m,n]-abs(sedata.s)[:,m,n], label=f'S{sedata._
↪fmt_trace_name(m, n)}')
        plt.title('Magnitude Error between Measurements')
        plt.legend(bbox_to_anchor=(1.1, 1.05))
```

(continues on next page)

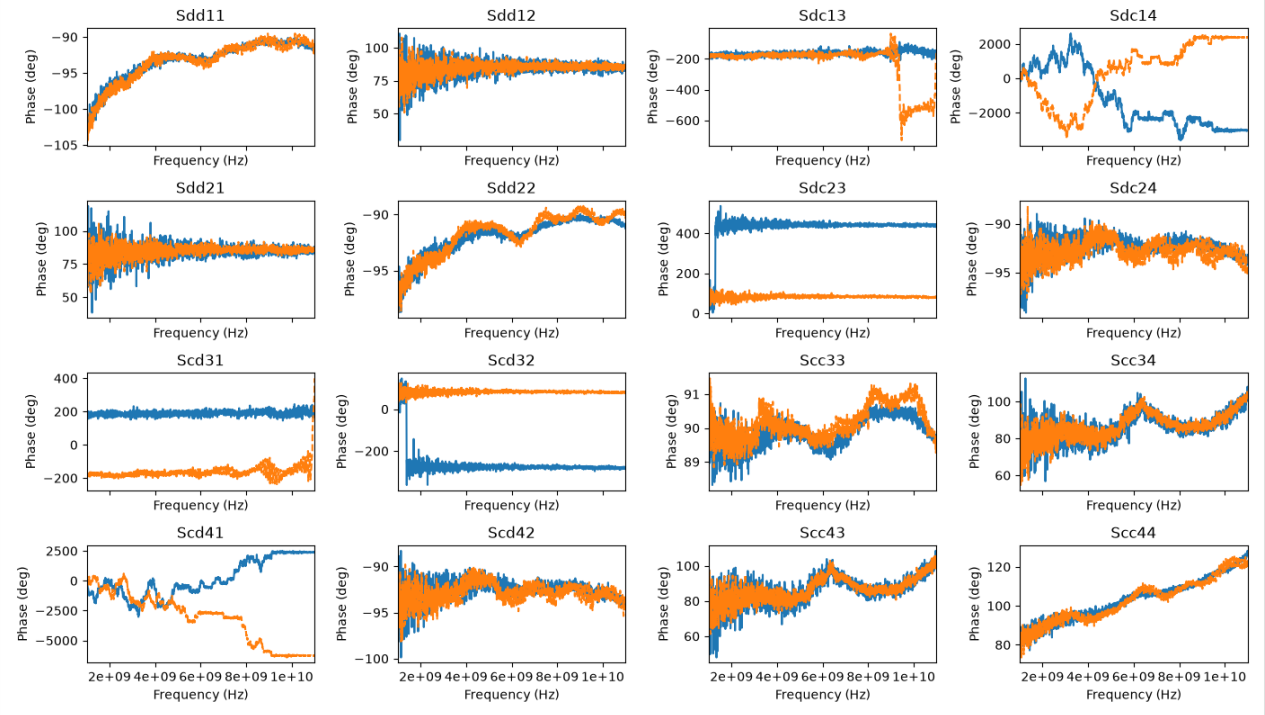
(continued from previous page)



[6]:
```
fig, axes = plt.subplots(2,2, sharex=True, sharey='row', figsize=(14,6))
sedata.plot_s_db(ax=axes[0][0])
mmdata.plot_s_db(ax=axes[0][1])
sedata.plot_s_deg(ax=axes[1][0])
mmdata.plot_s_deg(ax=axes[1][1])
axes[0][0].get_legend().remove()
axes[1][0].get_legend().remove()
axes[1][1].get_legend().remove()
axes[0][1].get_legend().remove()
axes[0][0].set_title('sedata Magnitude')
axes[0][1].set_title('mmdata Magnitude')
axes[1][0].set_title('sedata Phase')
axes[1][1].set_title('mmdata Phase')

fig.legend(*axes[0, 1].get_legend_handles_labels(), loc="center right")
fig.tight_layout()
plt.subplots_adjust(top=0.9, right=0.8)
```



[7]:
```
fig, axes = plt.subplots(4,4, sharex=True, figsize=(14,8))
for m in range(4):
    for n in range(4):
        sedata.plot_s_deg_unwrap(m=m, n=n, ax=axes[m][n])
        mmdata.plot_s_deg_unwrap(m=m, n=n, ax=axes[m][n], ls='--')
        axes[m][n].get_legend().remove()
        axes[m][n].set_title(f'S{sedata._fmt_trace_name(m, n)}')
fig.tight_layout()
```

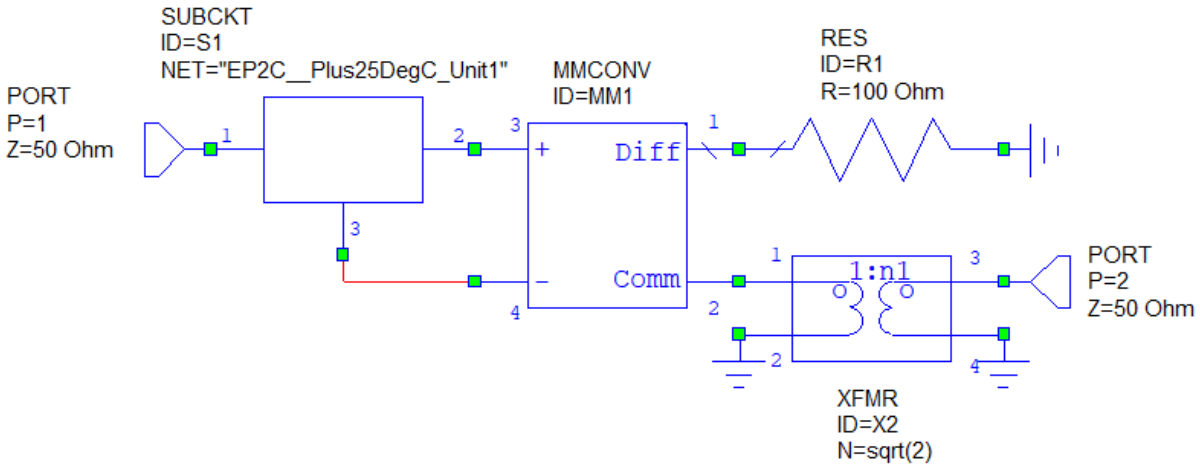


[ ]:

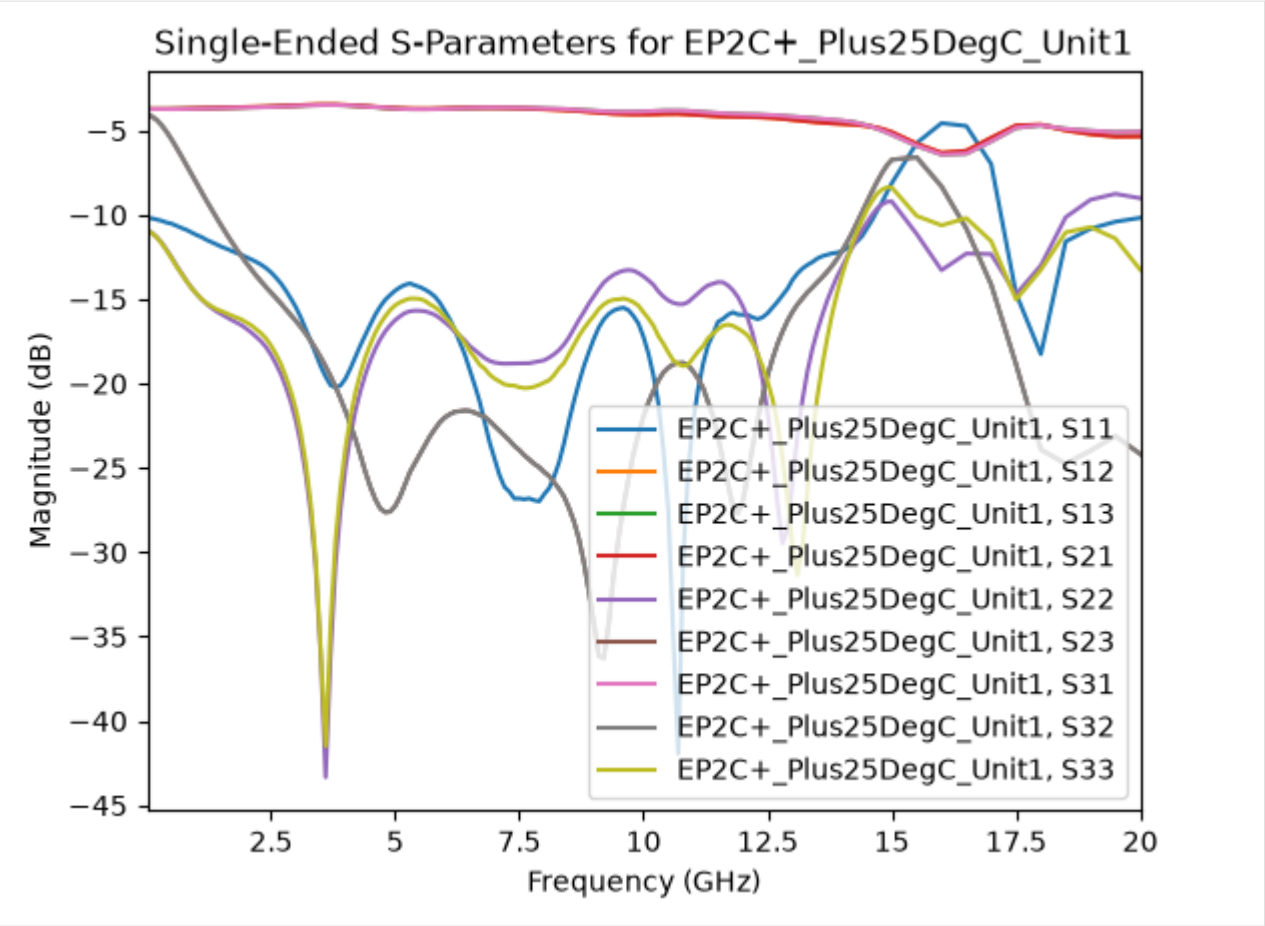
2.7.2 Mixed Mode S-Parameters & Impedance Transformation

Mini-circuits EP2C+ is a 1.8 to 12.5 GHz MMIC based splitter/combiner. The s-parameters provided by Mini-circuits are single-ended. For this example, the single-ended s-parameters will be converted to mixed mode s-parameters so that the common mode gain (the gain from the common port to the common mode terminated in 25) can be examined. Additionally, the differential mode gain (the gain from the common port to the differential mode terminated in 100) can be plotted. It is expected that the differential mode gain should be well below the common mode gain since this is a 0 degree splitter/combiner.

Lastly, since it is desired to use this network in a cascade analysis as a 2-port block in a 50 environment, the differential mode will be terminated in 100 and a 50 port transformed to 25 will be connected to the common mode port:

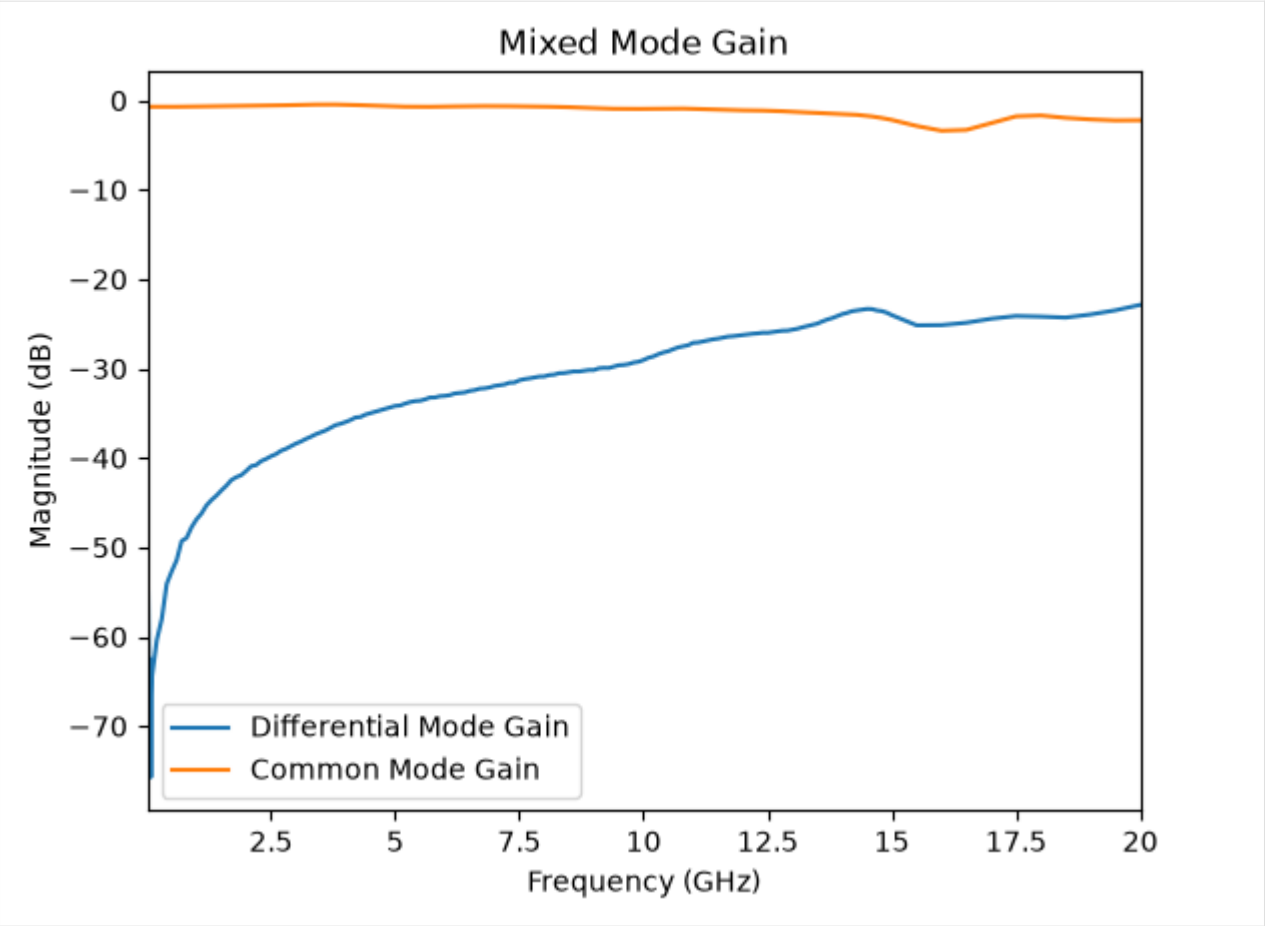


[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf
from skrf.network import a2s, connect

filename = r'mixedmodeSandZtransform_files/EP2C+_Plus25DegC_Unit1.S3P'
se_ntwk = skrf.Network(filename)
se_ntwk.frequency.unit = 'GHz'

# plot single-ended s-parameters
fig,ax0 = plt.subplots(1)
se_ntwk.plot_s_db(ax=ax0)
ax0.set_title(f'Single-Ended S-Parameters for {se_ntwk.name}')

# use the same frequency list for all networks
freq = se_ntwk.frequency
```



[2]:
```
# convert to mixed-mode s-parameters
mm_ntwk = se_ntwk.copy()

# for a 3-port, the common port has to be the last
mm_ntwk.renumber([0,1,2],[2,1,0])
mm_ntwk.se2gmm(p=1)

# make the differential mode port and common mode ports consistent with the figure above
mm_ntwk.renumber([2,1,0],[0,2,1])

fig,ax1 = plt.subplots(1)
mm_ntwk.plot_s_db(1,0,label='Differential Mode Gain',ax=ax1)
mm_ntwk.plot_s_db(2,0,label='Common Mode Gain',ax=ax1)
ax1.set_title('Mixed Mode Gain')
```

[2]:
```
Text(0.5, 1.0, 'Mixed Mode Gain')
```



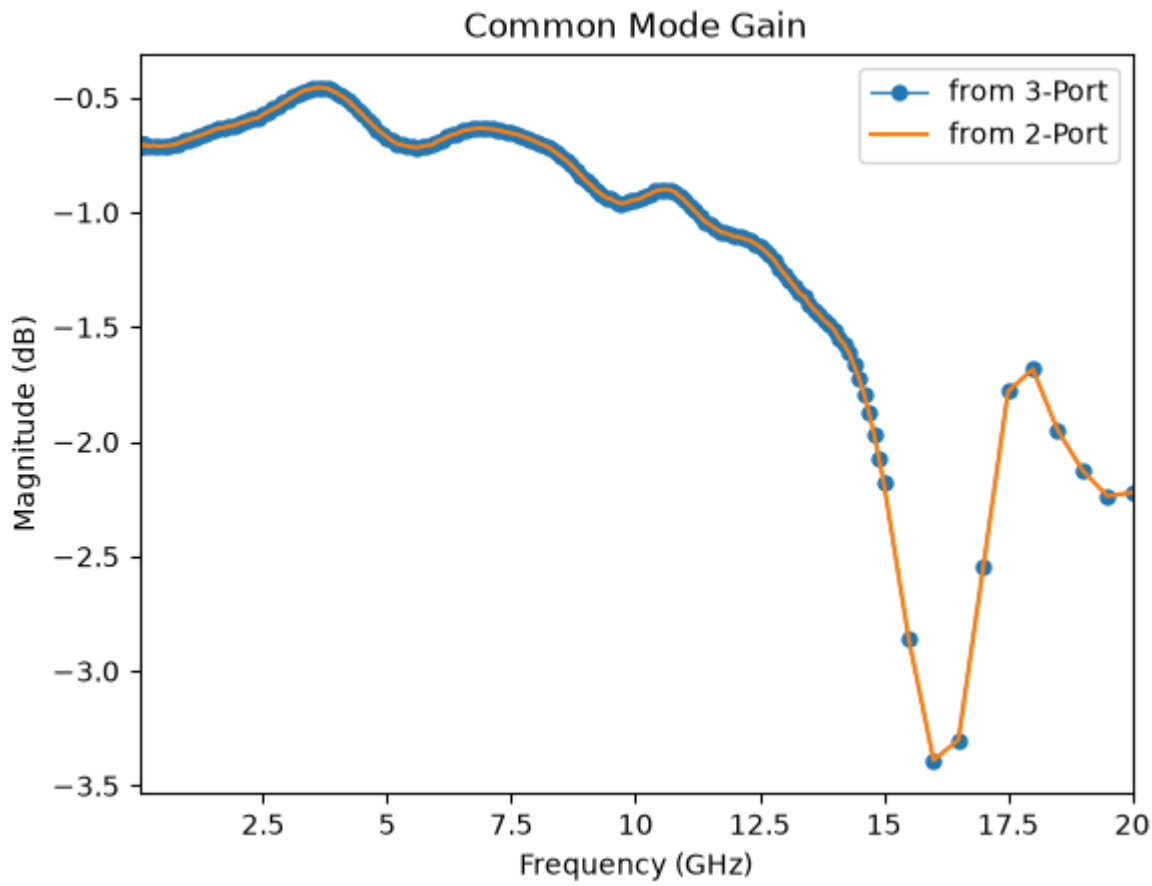
[3]:
```
# create a 25 to 50 ohm transformer

# turns ratio
n = np.sqrt(2)

# ABCD Parameters
a = np.array([[n, 0], [0, 1/n]])[np.newaxis, :, :]
abcd = np.tile(a, [freq.npoints,1,1])

transformer = skrf.Network(name='transformer',frequency=freq)
transformer.s = a2s(abcd, z0=[50, 50])

# connect the transformer to the common mode port
# common mode port impedance transformed from 50  to 25
mm_ntwk_t = connect(mm_ntwk,2,transformer,1)
```

[4]:
```
# create a 100  termination
term = skrf.Network(name='100 ohm termination',z0=100,
                    s=np.zeros([mm_ntwk.frequency.npoints]),
                    frequency=mm_ntwk.frequency)

# connect the 100  termination to the differential mode port
mm_ntwk_2port = connect(mm_ntwk_t,1,term,0)
```

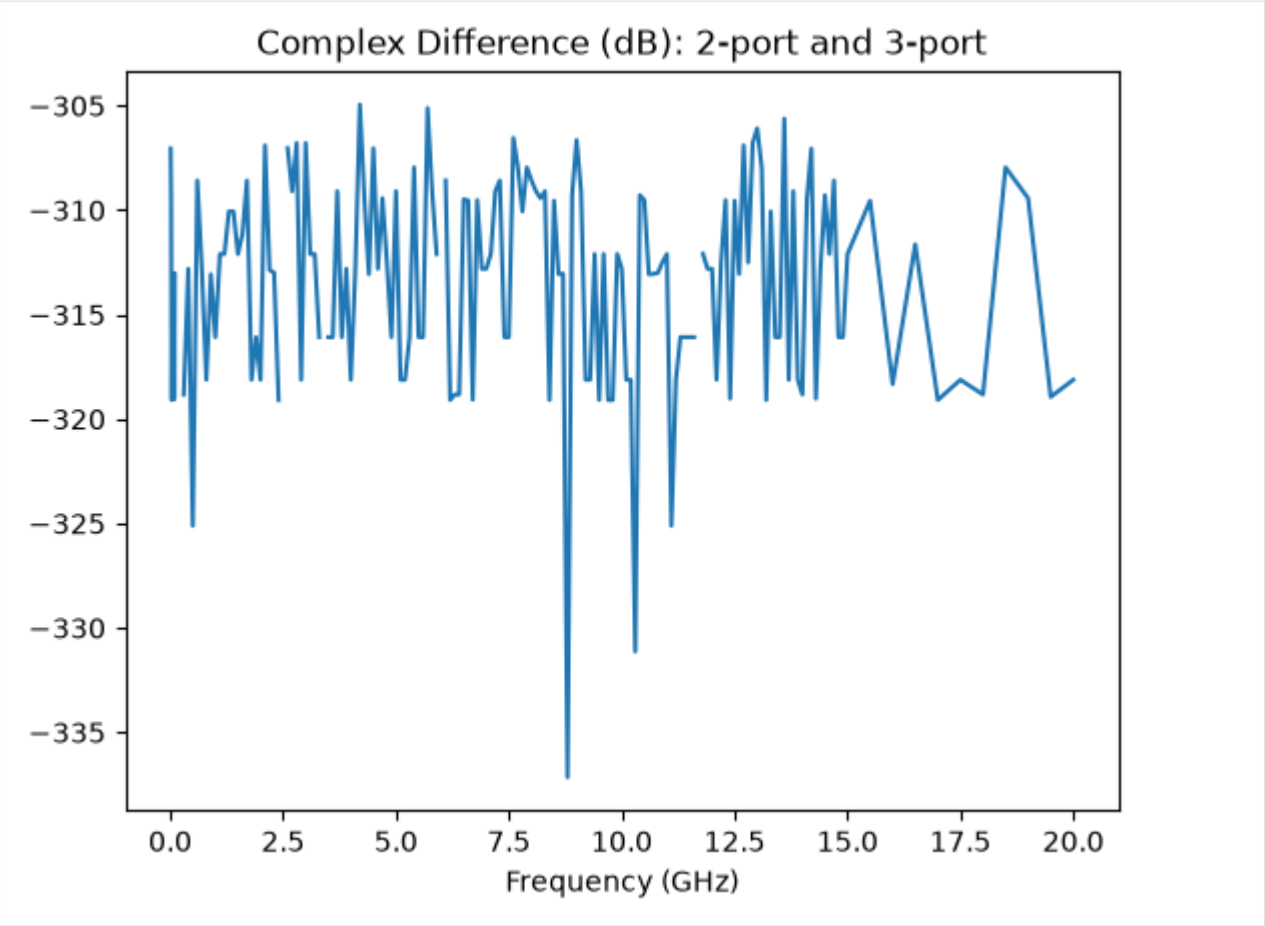
(continues on next page)

(continued from previous page)

```
fig,ax2 = plt.subplots(1)
mm_ntwk.plot_s_db(2,0,label='from 3-Port',ax=ax2,marker='o',lw=1,markersize=5)
mm_ntwk_2port.plot_s_db(1,0,label='from 2-Port',ax=ax2)

ax2.set_title('Common Mode Gain');
```



[5]:
```
# compare the common mode gain of the impedance transformed 2-port to the mixed-mode␣
↪untransformed 3-port
complex_diff = np.abs(mm_ntwk_2port.s[:,1,0] - mm_ntwk.s[:,2,0])

# don't give warning for -inf
complex_diff[complex_diff==0] = np.nan

fig,ax3 = plt.subplots(1)
plt.plot(freq.f_scaled,20*np.log10(complex_diff))
ax3.set_title('Complex Difference (dB): 2-port and 3-port')
ax3.set_xlabel('Frequency (GHz)');
```



2.8 Vector Fitting

2.8.1 Ex1: Ring Slot

The Vector Fitting feature is demonstrated using the *ring slot* example network from the scikit-rf `data` folder. Additional explanations and background information can be found in the *Vector Fitting tutorial*.

[1]:
```
import matplotlib.pyplot as mplt
import numpy as np

import skrf
from skrf.vectorFitting import VectorFitting
```

To create a `VectorFitting` instance, a `Network` containing the frequency responses of the N-port is passed. In this example the *ring slot* is used, which can be loaded directly as a `Network` from scikit-rf:

[2]:
```
nw = skrf.data.ring_slot
vf = VectorFitting(nw)
```

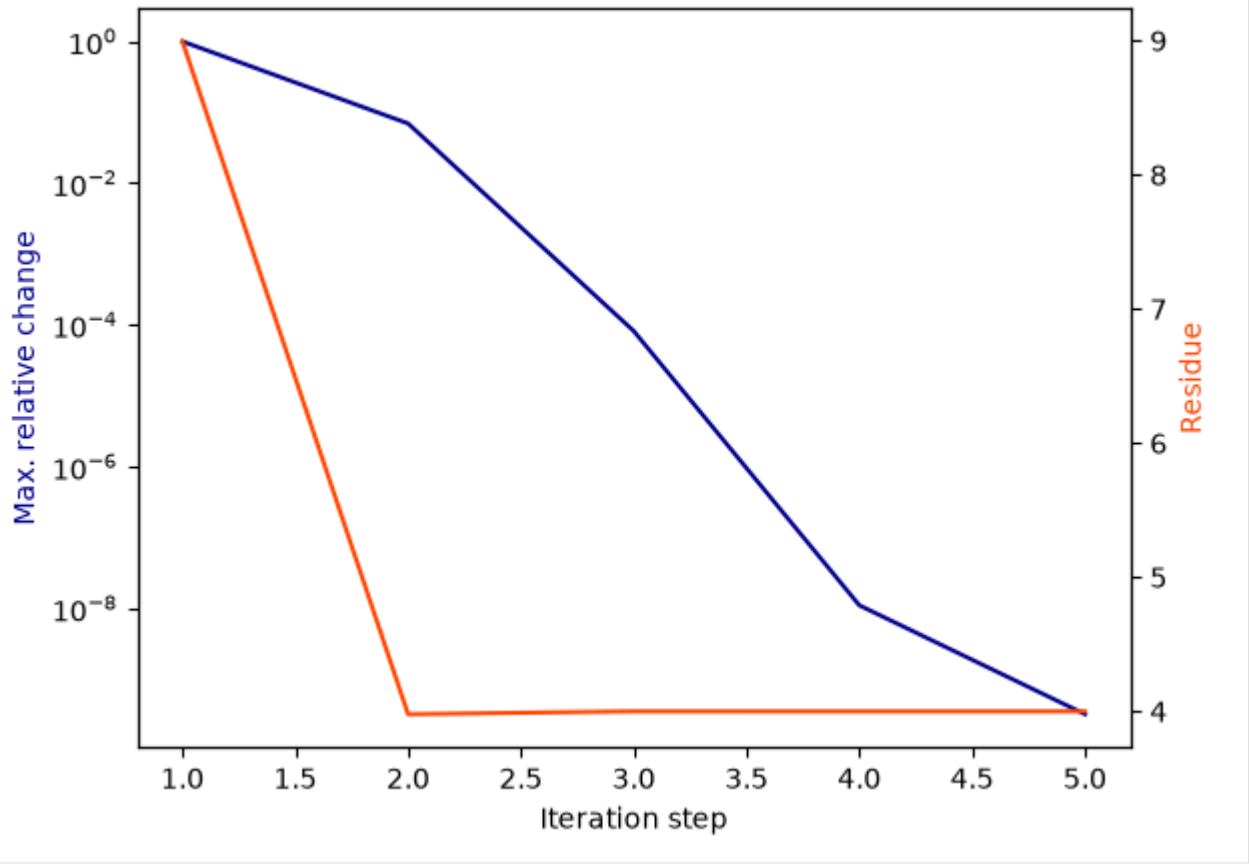
Now, the vector fit can be performed. The number of poles has to be specified, which depends on the *behaviour* of the responses. A smooth response would only require very few poles (2-5). In this case, 3 real poles are sufficient:

[3]:
```
vf.vector_fit(n_poles_real=3, n_poles_cmplx=0)
```

```
/tmp/ipykernel_8688/3090911508.py:1: UserWarning: The fitted network is passive, but the␣
↪vector fit is not passive. Consider running `passivity_enforce()` to enforce passivity␣
↪before using this model.
  vf.vector_fit(n_poles_real=3, n_poles_cmplx=0)
```

As printed in the logging output (not shown), the pole relocation process converged quickly after just 5 iteration steps. This can also be checked with the convergence plot:

[4]:
```
vf.plot_convergence()
```



The fitted model parameters are now stored in the class attributes `poles`, `residues`, `proportional_coeff` and `constant_coeff` for further use. To verify the result, the model response can be compared to the original network response. One option is to analyze the rms error magnitude, which should be smaller than 0.05 for a good fit:

[5]:
```
vf.get_rms_error()
```

[5]:
```
np.float64(0.003855803397460332)
```

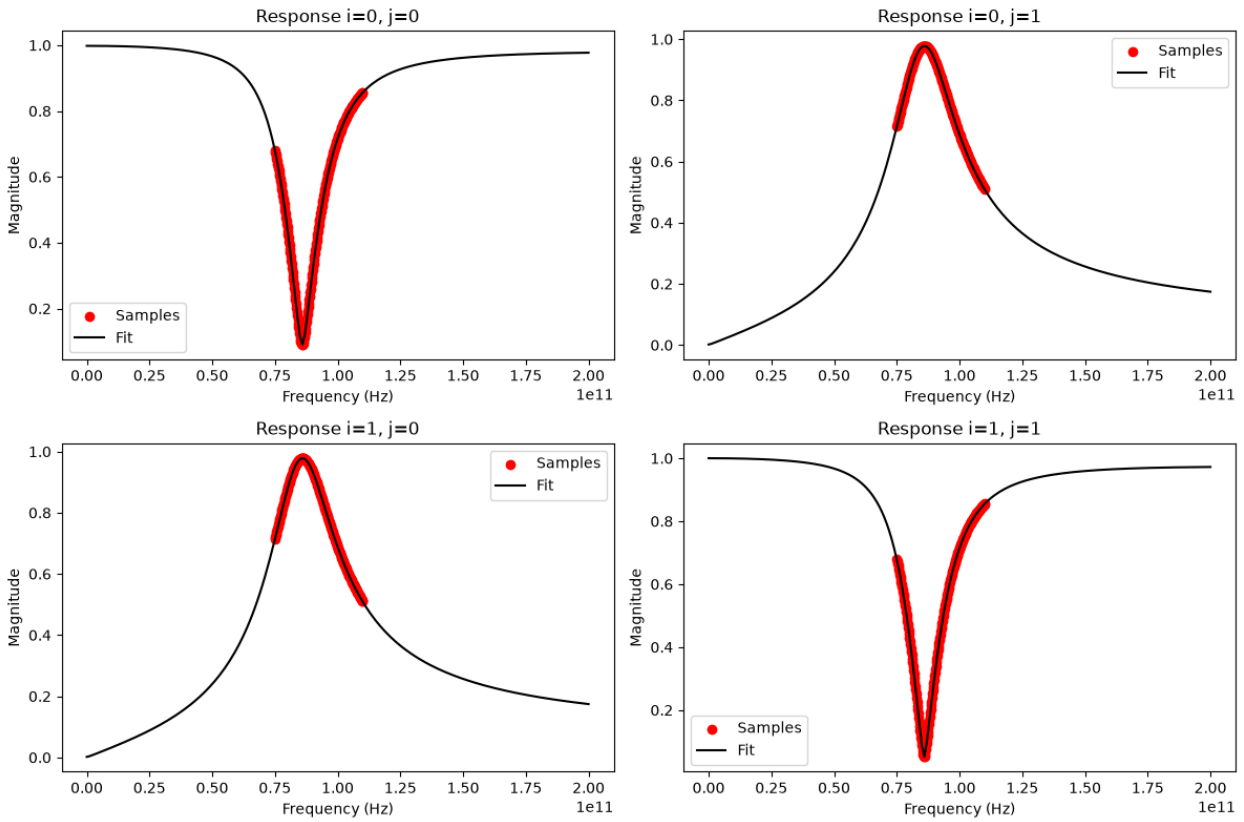
As the model will return a response at any given frequency, it makes sense to also manually check its response outside the frequency range of the original samples by plotting it at lower and higher frequencies:

[6]:
```
freqs1 = np.linspace(0, 200e9, 201)
fig, ax = mplt.subplots(2, 2)
fig.set_size_inches(12, 8)
vf.plot_s_mag(0, 0, freqs1, ax=ax[0][0]) # plot s11
```

(continues on next page)

(continued from previous page)

```
vf.plot_s_mag(1, 0, freqs1, ax=ax[1][0]) # plot s21
vf.plot_s_mag(0, 1, freqs1, ax=ax[0][1]) # plot s12
vf.plot_s_mag(1, 1, freqs1, ax=ax[1][1]) # plot s22
fig.tight_layout()
mplt.show()
```



To use the model in a circuit simulation, an equivalent circuit can be created based on the fitting parameters. This is currently only implemented for SPICE, but the structure of the equivalent circuit can be adopted to any kind of circuit simulator. Attention: A UserWarning about a non-passive fit was printed (see output of vector_fit() above). The assessment and enforcement of model passivity is described in more detail in *this example*, which is important for certain use cases in circuit simulators, i.e. transient simulations.

```
vf1.write_spice_subcircuit_s('/home/vinc/Desktop/ring_slot.sp')
```

For a quick test, the subcircuit is included in a schematic in QUCS-S for AC simulation and S-parameter calculation based on the port voltages and currents (see the equations):

The simulation outputs from ngspice compare well to the plots above:

2.8.2 Ex2: Measured 190 GHz Active 2-Port

The Vector Fitting feature is demonstrated using a 2-port S-matrix of an active circuit measured from 140 GHz to 220 GHz. Additional explanations and background information can be found in the *Vector Fitting tutorial*.

[1]:
```
import numpy as np
```

(continues on next page)

(continued from previous page)

```
import skrf
from skrf.vectorFitting import VectorFitting
```

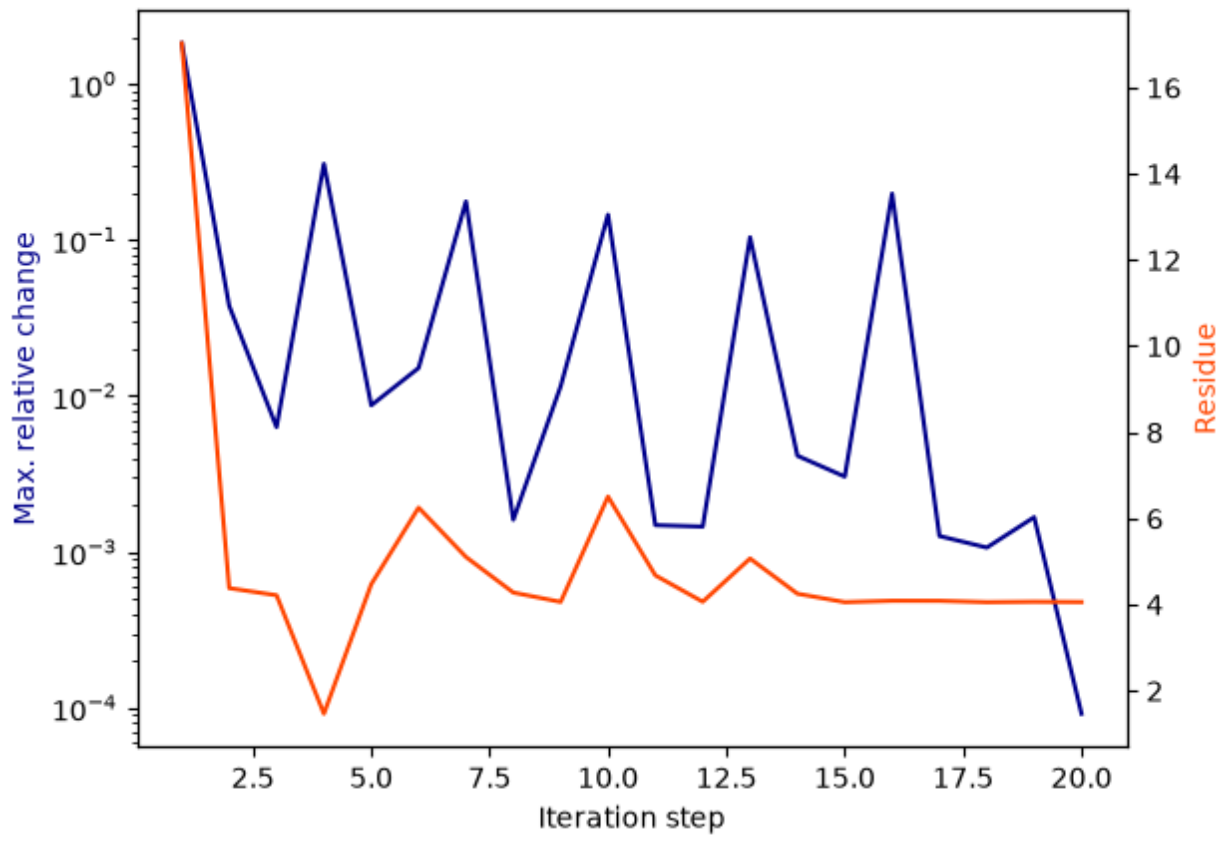
This example is a lot more tricky to fit, because the responses contain a few "bumps" and noise from the measurement. In such a case, finding a good number of initial poles can take a few iterations. A good alternative for such cases is the automatic vector fitting feature, `auto_fit()`, which automatically refines the number of poles until a good fit is achieved.

Load the Network from a Touchstone file and create the Vector Fitting instance and start the automatic fit:

[2]:
```
nw = skrf.network.Network('./190ghz_tx_measured.S2P')
vf = VectorFitting(nw)
vf.auto_fit()
```

The function `plot_convergence()` can be helpful to examine the convergence and see if something was going wrong. Important parameters to check are the number of iterations until convergence and the final model order.

[3]:
```
vf.plot_convergence()
```



[4]:
```
print(f'model order = {vf.get_model_order(vf.poles)}')
print(f'n_poles_real = {np.sum(vf.poles.imag == 0.0)}')
print(f'n_poles_complex = {np.sum(vf.poles.imag > 0.0)}')
```

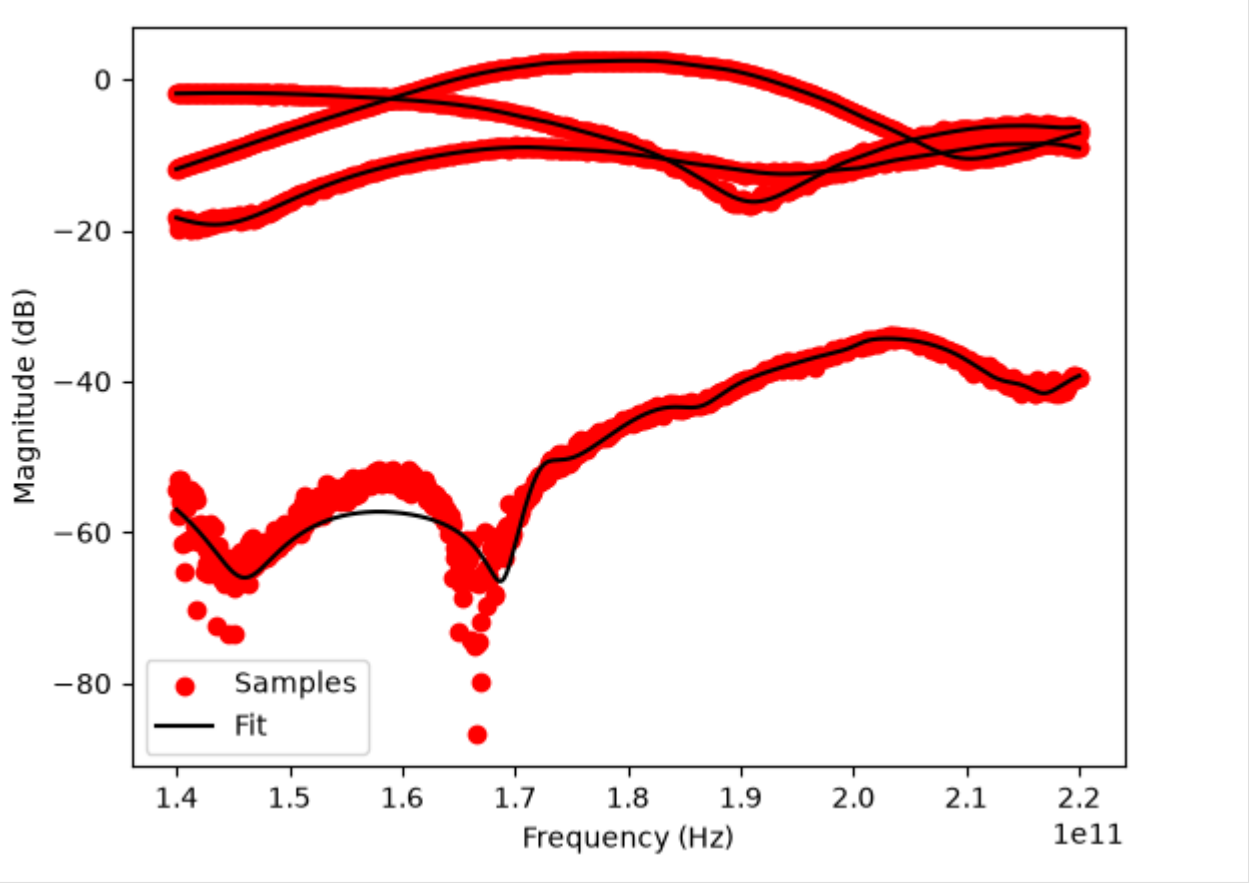
```
model order = 21
n_poles_real = 1
n_poles_complex = 10
```

Checking the results by comparing the model responses to the original sampled data indicates a successful fit, which is also indicated by a small rms error (less than 0.05):

[5]:
```
vf.get_rms_error()
```

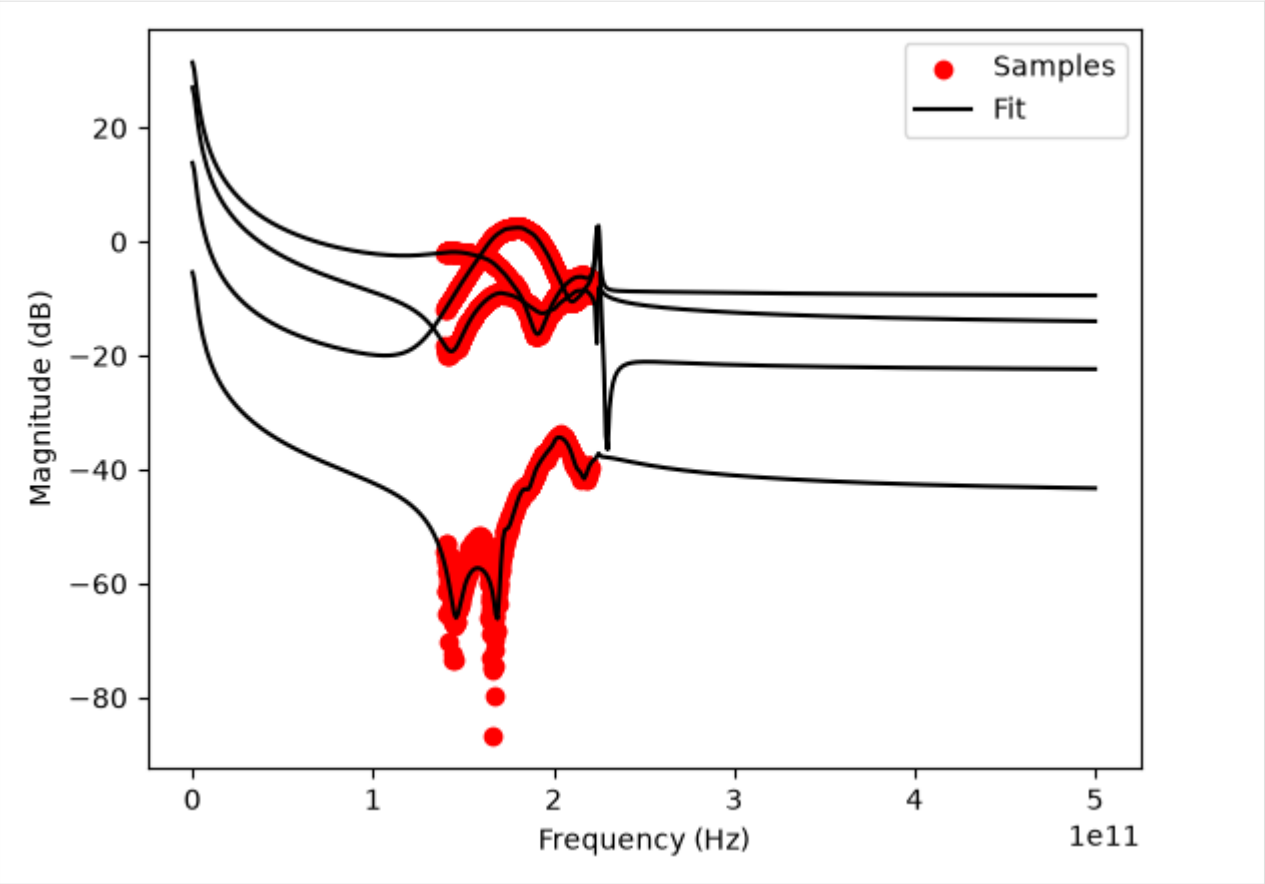
[5]:
```
np.float64(0.013618943572754429)
```

[6]:
```
# plot frequency responses
vf.plot_s_db()
```



It is a good idea to also check the model response well outside the original frequency range.

[7]:
```
freqs = np.linspace(0, 500e9, 501) # plot model response from dc to 500 GHz
vf.plot_s_db(freqs=freqs)
```



2.8.3 Ex3: Fitting spiky responses

The Vector Fitting feature is demonstrated using a 4-port example network copied from the scikit-rf `tests` folder. This network is a bit tricky to fit because of its many resonances in the individual response. Additional explanations and background information can be found in the *Vector Fitting tutorial*.

[1]:
```
import matplotlib.pyplot as mplt
import numpy as np

import skrf
from skrf.vectorFitting import VectorFitting
```

To create a `VectorFitting` instance, a `Network` containing the frequency responses of the N-port is passed. In this example a copy of `Agilent_E5071B.s4p` from the `skrf/tests` folder is used:

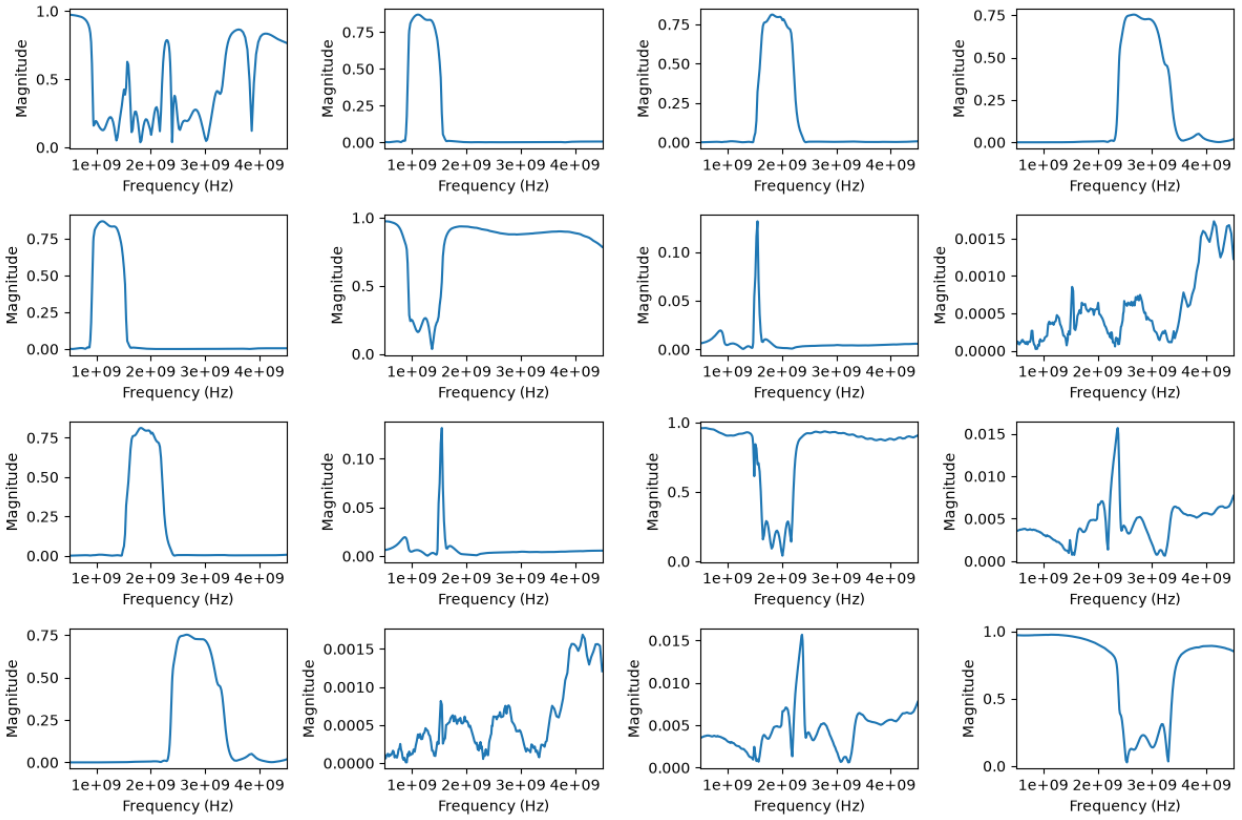
[2]:
```
nw = skrf.network.Network('./Agilent_E5071B.s4p')
vf = VectorFitting(nw)
```

For comparison, this network will be fitted using both the regular vector fitting routine `vector_fit()` as well as the automatic vector fitting routine `auto_fit()`.

Regular Vector Fitting

For regular vector fitting, the number and type of the initial poles have to be specified, which depend on the *behaviour* of the responses. As a rule of thumb for an initial guess, one can count the number of resonances or "bumps" in the individual responses. In this case, the 4-port network has 16 responses to be fitted. As shown in the magnitude plots below, S_{11} and some other responses are quite *spiky* and have roughly 15 local maxima each and about the same number of local minima in between. Other responses have only 5-6 local maxima, or they are very noisy with very small magnitudes (like S_{24} and S_{42}). Assuming that most of the 15 maxima of S_{11} occur at the same frequencies as the maxima of the other responses, one can expect to require 15 complex-conjugate poles for a fit. As this is probably not completely the case, trying with 20-30 poles should be a good start to fit all of the resonances in all of the responses.

[3]:
```
# plot magnitudes of all 16 responses in the 4-port network
fig, ax = mplt.subplots(4, 4)
fig.set_size_inches(12, 8)
for i in range(4):
    for j in range(4):
        nw.plot_s_mag(i, j, ax=ax[i][j])
        ax[i][j].get_legend().remove()
fig.tight_layout()
mplt.show()
```



After trying different numbers of real and complex-conjugate poles, the following setup was found to result in a very good fit. The initial model order is $1 + 2 * 26 = 53$, which will remain unchanged during the fitting process. Other setups were also found to work well for this network (e.g. 0-2 real poles and 25-26 cc poles):

[4]:
```
vf.vector_fit(n_poles_real=1, n_poles_cmplx=26)
```

```
/tmp/ipykernel_8903/3610353239.py:1: UserWarning: The fitted network is passive, but the␣
```

(continues on next page)

(continued from previous page)

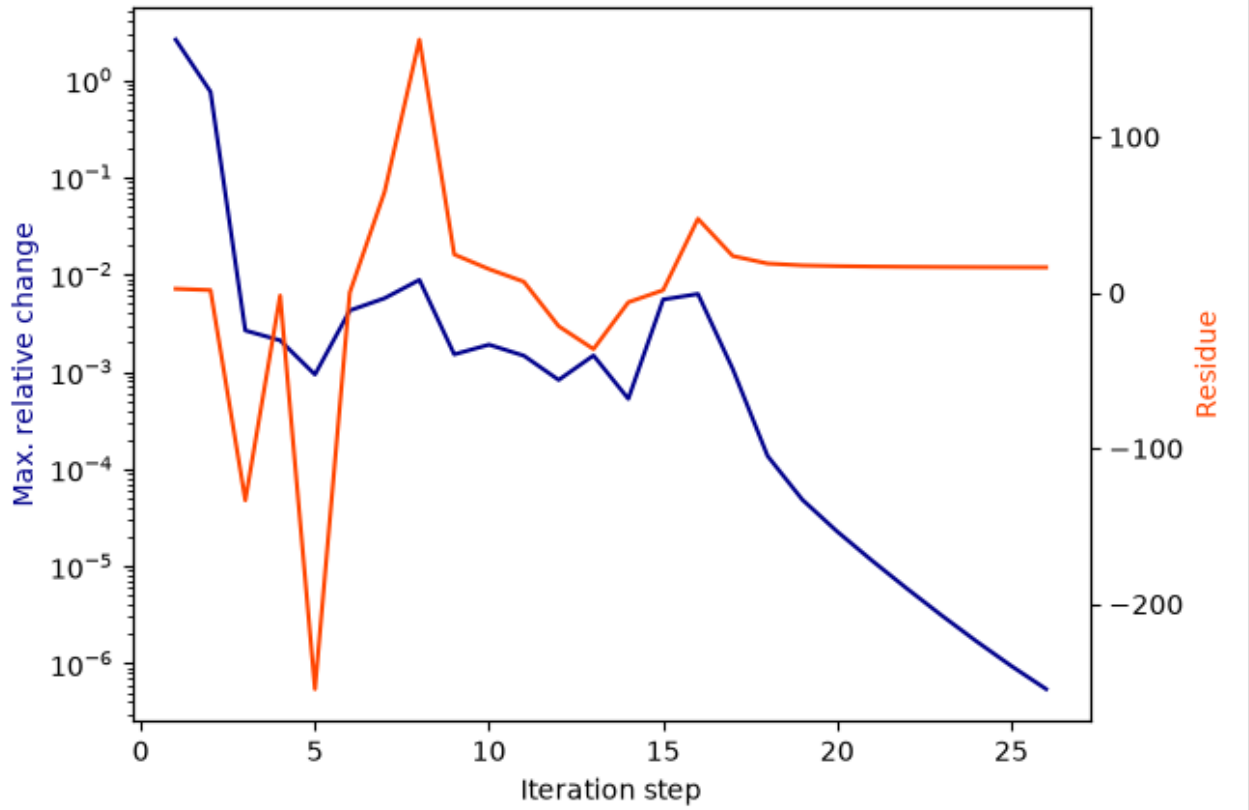
```
↪vector fit is not passive. Consider running `passivity_enforce()` to enforce passivity␣
↪before using this model.
  vf.vector_fit(n_poles_real=1, n_poles_cmplx=26)
```

[5]:
```
print(f'model order = {vf.get_model_order(vf.poles)}')
```

```
model order = 53
```

The convergence can also be checked with the convergence plot:

[6]:
```
vf.plot_convergence()
```



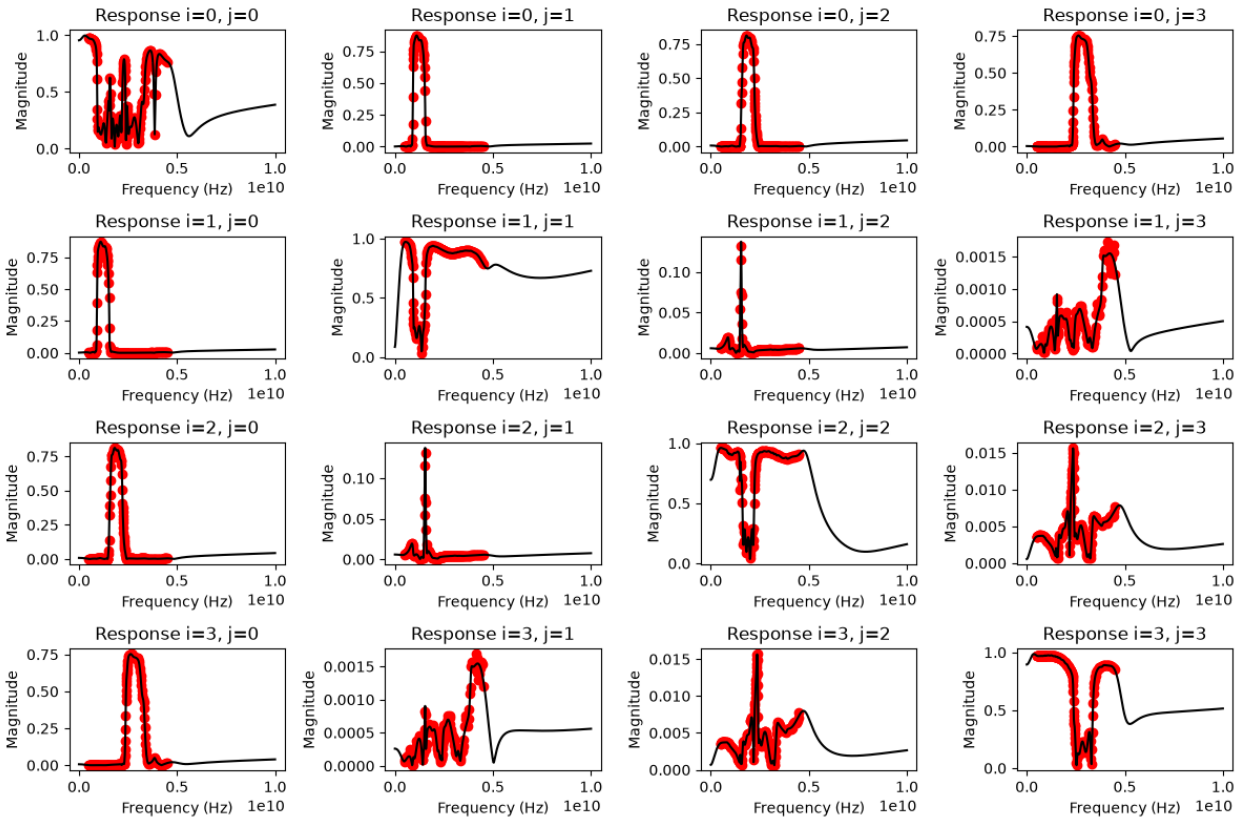
The fitted model parameters are now stored in the class attributes `poles`, `residues`, `proportional_coeff` and `constant_coeff` for further use. To verify the result, the fitted model responses can be compared to the original network responses. As the model will return a response at any given frequency, it makes sense to also check its response outside the frequency range of the original samples. In this case, the original network was measured from 0.5 GHz to 4.5 GHz, so we can plot the fit from dc to 10 GHz:

[7]:
```
freqs = np.linspace(0, 10e9, 501)
fig, ax = mplt.subplots(4, 4)
fig.set_size_inches(12, 8)
for i in range(4):
    for j in range(4):
        vf.plot_s_mag(i, j, freqs=freqs, ax=ax[i][j])
        ax[i][j].get_legend().remove()
fig.tight_layout()
```

(continues on next page)

(continued from previous page)

```
mplt.show()
```



[8]:
```
print(f'rms error = {vf.get_rms_error()}')
```

```
rms error = 0.008987793152488367
```

As shown in the plots, a very good fit was achieved. This is also indicated with the low rms error of less than 0.01. However, a UserWarning about a non-passive fit was printed (see output of `vector_fit()` above). The assessment and enforcement of model passivity is described in more detail in *this example*, which is important for certain use cases in circuit simulators, i.e. transient simulations. To use the model in a circuit simulation, an equivalent circuit can be created based on the fitting parameters. This is currently only implemented for SPICE, but the structure of the equivalent circuit can be adopted to any kind of circuit simulator.

```
vf.write_spice_subcircuit_s('/home/vinc/Desktop/4-port_model.sp')
```

The exported `.sp` file can then be imported into SPICE as a subcircuit. Have a look at the *Ring Slot Example*.

Automatic Vector Fitting

For the following automatic vector fitting routine, the user does not have to specify the starting poles. The default settings should be sufficient for a successful fit, as the required number of poles gets optimized during the process. Still, changes in the parameters for `auto_fit()` can further improve the convergence and reduce the model order.

[9]:
```
vf.auto_fit()
```

[10]:
```
print(f'model order = {vf.get_model_order(vf.poles)}')
print(f'n_poles_real = {np.sum(vf.poles.imag == 0.0)}')
```

(continues on next page)

(continued from previous page)

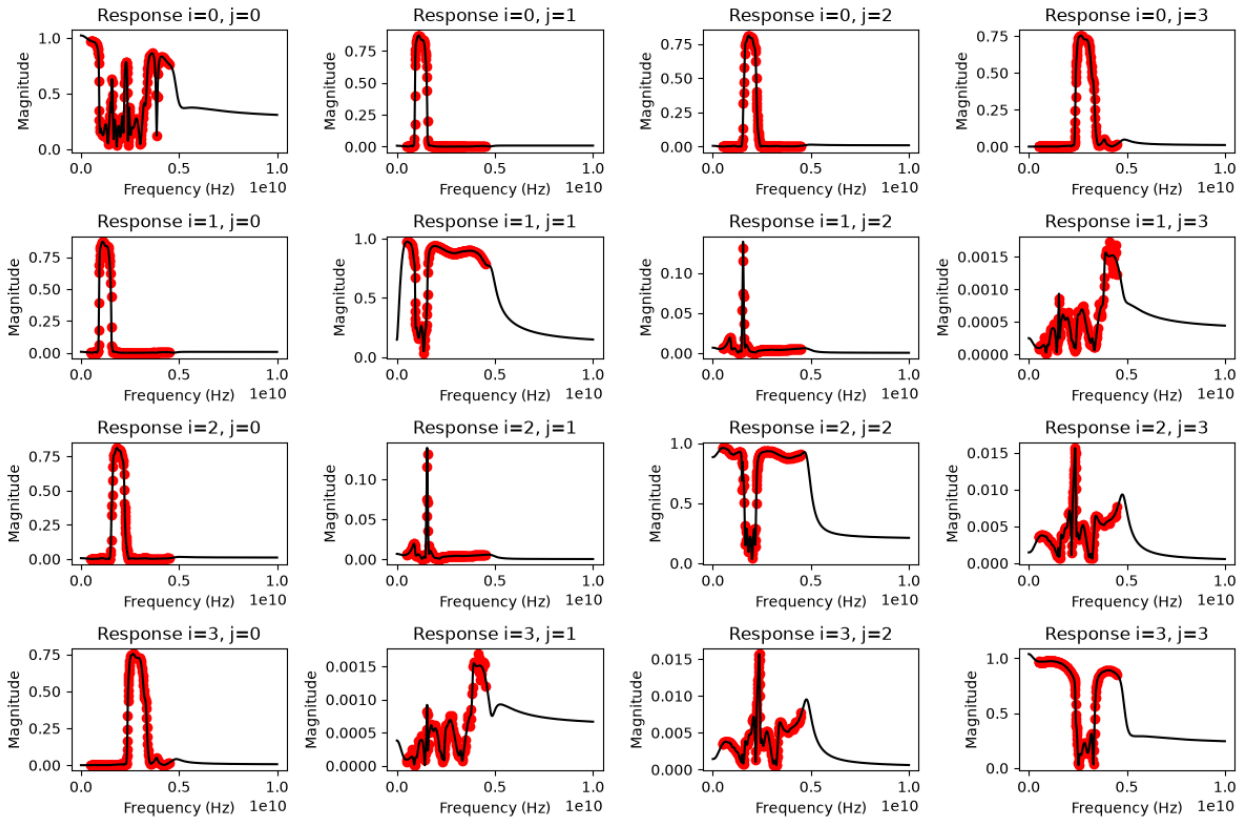
```
print(f'n_poles_complex = {np.sum(vf.poles.imag > 0.0)}')
```

```
model order = 57
n_poles_real = 1
n_poles_complex = 28
```

[11]:
```
print(f'rms error = {vf.get_rms_error()}')
```

```
rms error = 0.005893768165089838
```

[12]:
```
freqs = np.linspace(0, 10e9, 501)
fig, ax = mplt.subplots(4, 4)
fig.set_size_inches(12, 8)
for i in range(4):
    for j in range(4):
        vf.plot_s_mag(i, j, freqs=freqs, ax=ax[i][j])
        ax[i][j].get_legend().remove()
fig.tight_layout()
mplt.show()
```

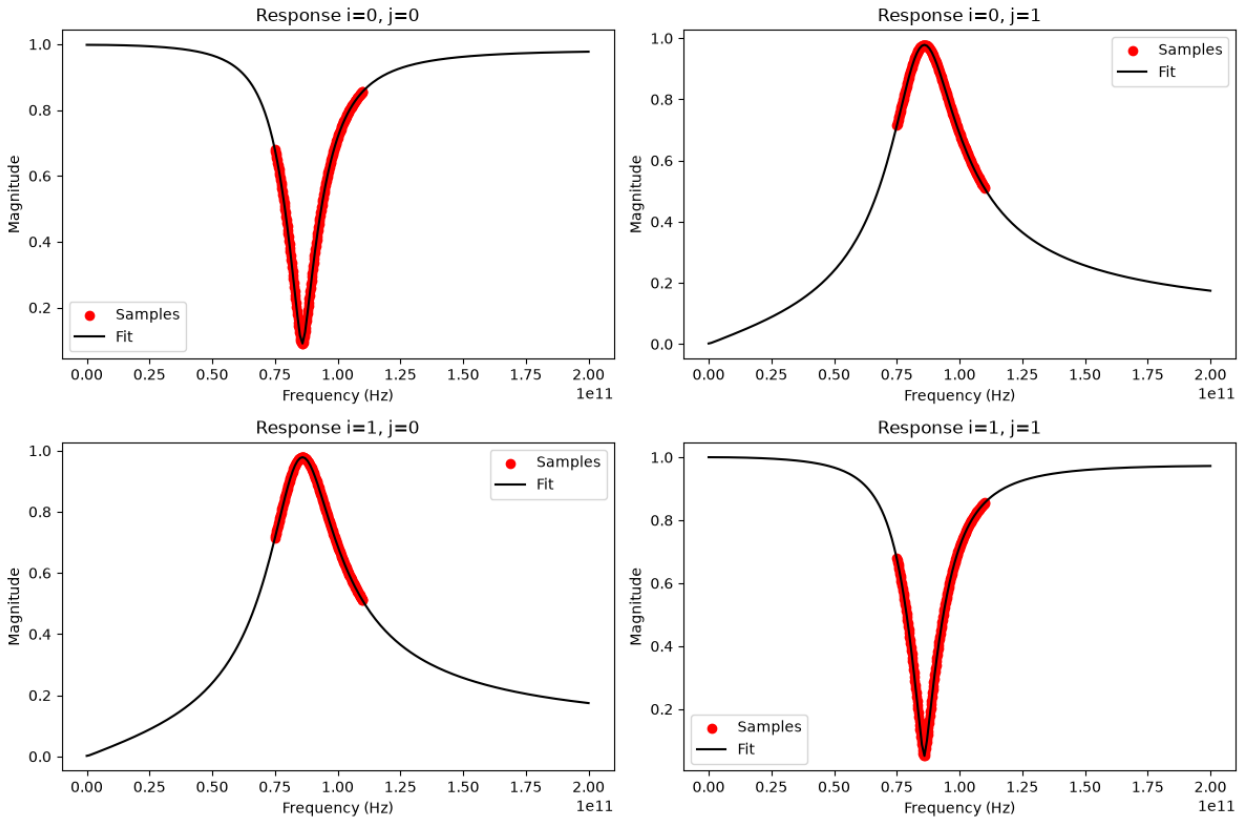


2.8.4 Ex4: Passivity Evaluation and Enforcement

To demonstrate the passivity evaluation and enforcement features of the vector fitting class, the ring slot example 2-port is used, once again. Have a look at the other vector fitting example notebooks for more general explanations of the fitting process.

[1]:
```
import matplotlib.pyplot as mplt
import numpy as np

import skrf
from skrf.vectorFitting import VectorFitting

# load and fit the ring slot network with 3 poles
nw = skrf.data.ring_slot
vf = VectorFitting(nw)
vf.vector_fit(n_poles_real=3, n_poles_cmplx=0)

# plot fitting results
freqs = np.linspace(0, 200e9, 201)
fig, ax = mplt.subplots(2, 2)
fig.set_size_inches(12, 8)
vf.plot_s_mag(0, 0, freqs=freqs, ax=ax[0][0]) # s11
vf.plot_s_mag(0, 1, freqs=freqs, ax=ax[0][1]) # s12
vf.plot_s_mag(1, 0, freqs=freqs, ax=ax[1][0]) # s21
vf.plot_s_mag(1, 1, freqs=freqs, ax=ax[1][1]) # s22
fig.tight_layout()
mplt.show()
```

```
/tmp/ipykernel_9066/968116361.py:10: UserWarning: The fitted network is passive, but the␣
↪vector fit is not passive. Consider running `passivity_enforce()` to enforce passivity␣
↪before using this model.
  vf.vector_fit(n_poles_real=3, n_poles_cmplx=0)
```



The fitting result looks fine, but a UserWarning about a non-passive vector fit was printed. Before investigating this issue, let's check the RMS error:

[2]:
```
vf.get_rms_error()
```

[2]:
```
np.float64(0.003855803397460332)
```

An RMS error of less than 0.05 usually indicates a good fit and confirms our optical inspection. But what about the passivity of the fitted model?

[3]:
```
vf.is_passive()
```

[3]:
```
False
```

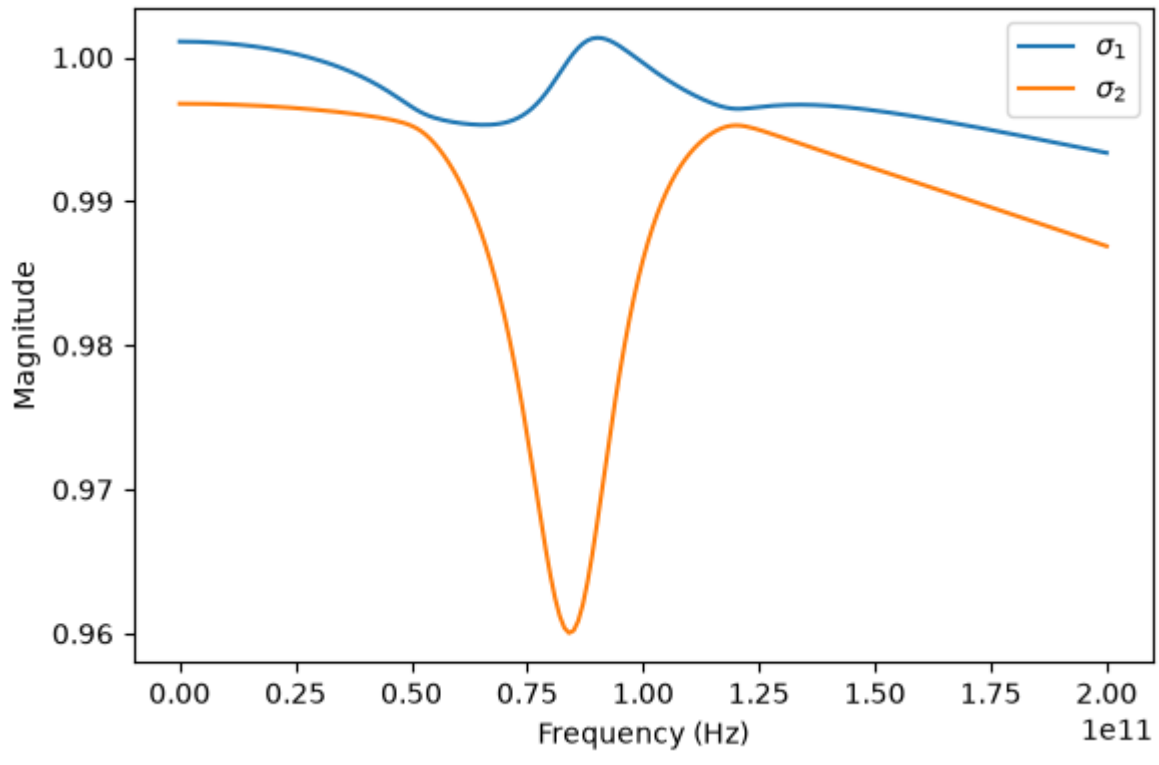
Why is the model not passive? Wasn't the original data of the ring slot representing a passive network?

[4]:
```
nw.is_passive()
```

[4]:
```
True
```

The network data was passive, but the vector fitted model is not. Let's investigate (and correct?) the problem some more.

[5]:
```
# plot singular values of vector fitted scattering matrix
freqs = np.linspace(0, 200e9, 201)
fig, ax = mplt.subplots(1, 1)
fig.set_size_inches(6, 4)
vf.plot_s_singular(freqs=freqs, ax=ax)
fig.tight_layout()
mplt.show()
```



One of the singular values of the fitted scattering matrix is greater than 1 at some frequencies. This indeed indicates a non-passive model. For further analysis, you can get a list of all frequency bands with passivity violations:

[6]:
```
vf.passivity_test()
```

[6]:
```
array([[0.00000000e+00, 2.78120033e+10],
       [8.43130648e+10, 9.83113388e+10]])
```

The network is not passive in two frequency bands: From dc to about 27.8 GHz, and from 84.3 GHz to 98.5 GHz. Luckily, passivity can be enforced to obtain passive vector fitted model:

[7]:
```
vf.passivity_enforce()
```

```
/tmp/ipykernel_9066/1584954637.py:1: UserWarning: Passivity enforcement: The dc point in␣
↪the model is not passive. Cannot preserve the dc point during passivity enforcement.
  vf.passivity_enforce()
```

After passivity enforcement, the network should be passive at all frequencies. Let's check ourselves:

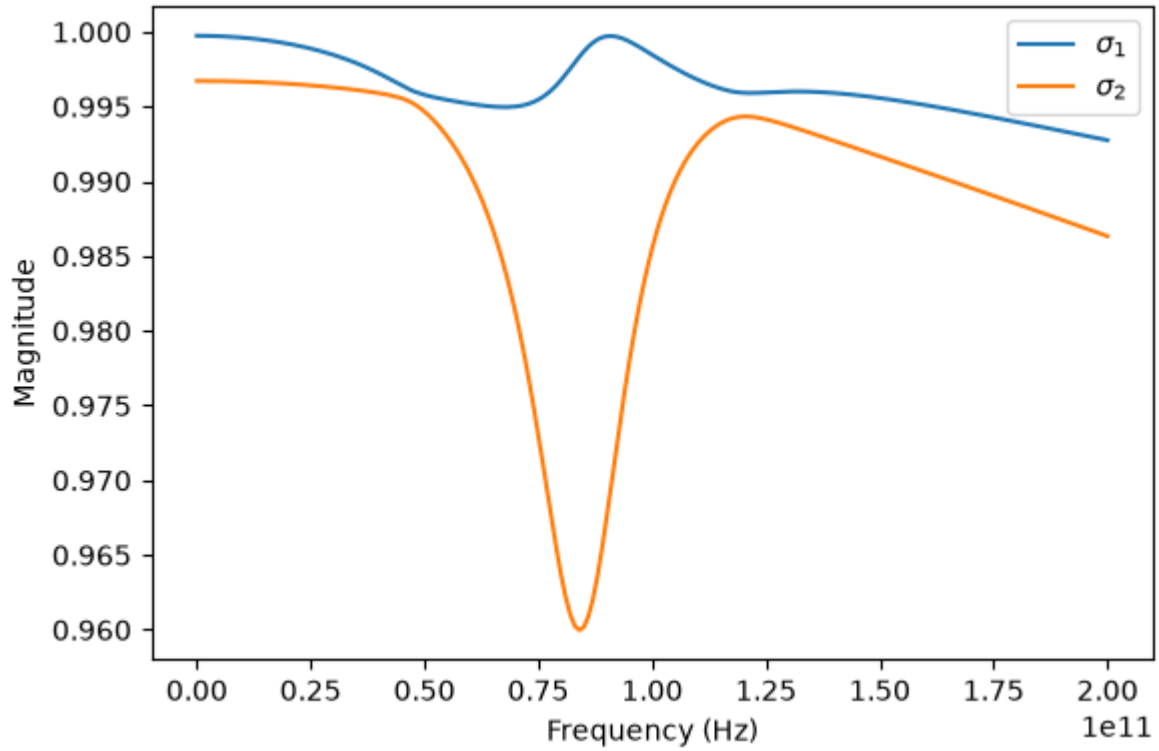
[8]:
```
vf.is_passive()
```

[8]:
```
True
```

[9]:
```
vf.passivity_test()
```

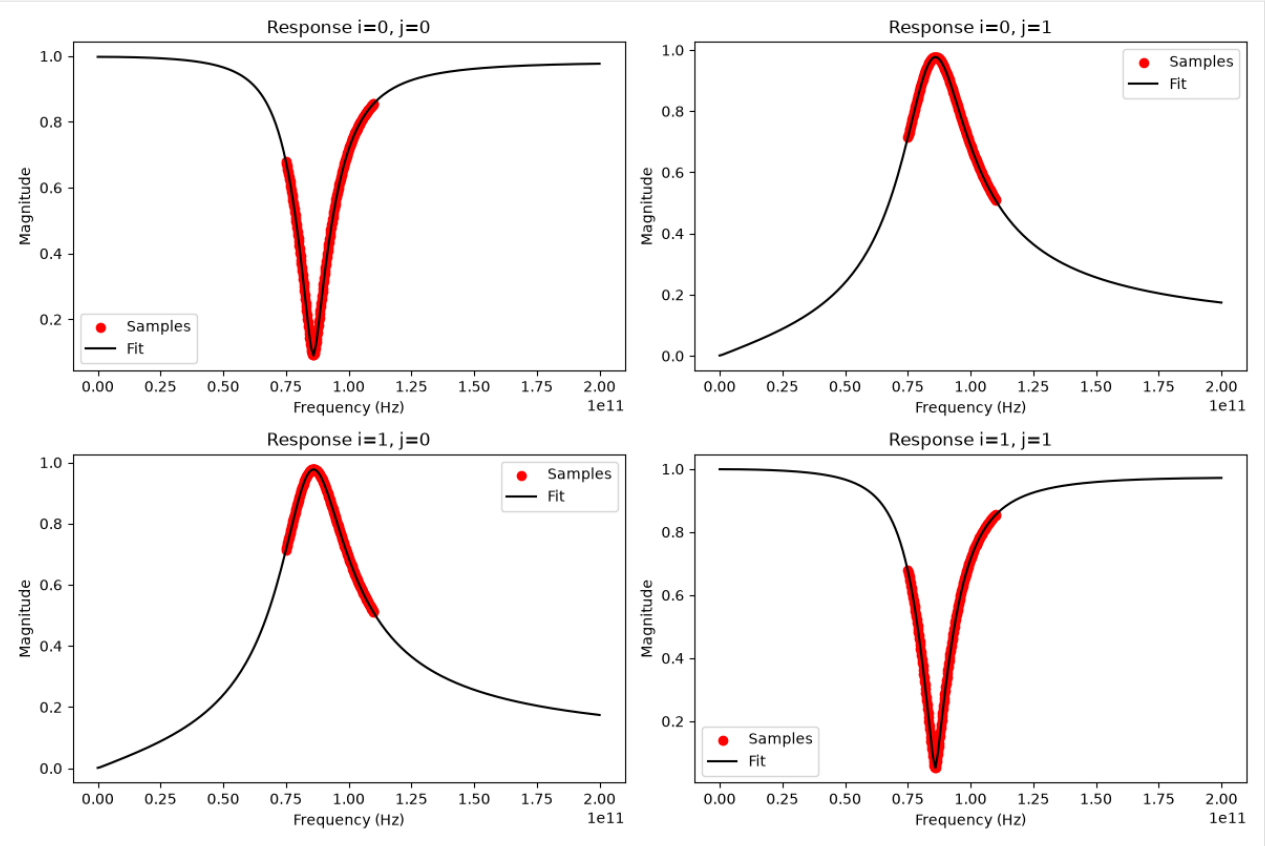
[9]:
```
array([], dtype=float64)
```

[10]:
```
# plot singular values of vector fitted scattering matrix
freqs = np.linspace(0, 200e9, 201)
fig, ax = mplt.subplots(1, 1)
fig.set_size_inches(6, 4)
vf.plot_s_singular(freqs=freqs, ax=ax)
fig.tight_layout()
mplt.show()
```



Alright, the model is finally passive. But does it still fit the original network data?

[11]:
```
# plot fitting results again after passivity enforcement
freqs = np.linspace(0, 200e9, 201)
fig, ax = mplt.subplots(2, 2)
fig.set_size_inches(12, 8)
vf.plot_s_mag(0, 0, freqs=freqs, ax=ax[0][0]) # s11
vf.plot_s_mag(0, 1, freqs=freqs, ax=ax[0][1]) # s12
vf.plot_s_mag(1, 0, freqs=freqs, ax=ax[1][0]) # s21
vf.plot_s_mag(1, 1, freqs=freqs, ax=ax[1][1]) # s22
fig.tight_layout()
mplt.show()
```



In addition to the visual inspection, let's check the RMS error again:

[12]:
```
vf.get_rms_error()
```

[12]:
```
np.float64(0.004160844096426696)
```

Yes, the model still fits the original data very well and the differences to the first non-passive fit from above are insignificant: the rms error is still very low.

2.8.5 Common problems and pitfalls

Regular Vector Fitting is susceptible to some general problems and to specific user errors. This section is intended to collect and address common issues and to help mitigating them. Additional explanations and background information can be found in the *Vector Fitting tutorial*.

Sources of problems

The following things can cause significant problems in the vector fitting process and may result in poor convergence and/or large errors of the fit:

1. not enough poles
2. too many poles
3. strong noise in the data

From points 1) and 2) it is obvious that the number of poles has to be chosen carefully, which often requires several attempts to get right. But even with the right number of poles, the fit quality and the required number of iterations can be disappointing in case of strong noise in the data, usually from measurements. In such cases, successful vector fits

might still be achieved with `auto_fit()`, an improved and automated version of the vector fitting algorithm utilizing iterative pole adding and skimming.

The 2-port measurement from example 2 is a tricky network to fit properly. It will serve as an example of the different issues. Also see *Ex2: Measured 190 GHz Active 2-Port* for a demonstration of `auto_fit()` on this network.

[1]:
```
import matplotlib.pyplot as mplt
import numpy as np

import skrf
from skrf.vectorFitting import VectorFitting

nw = skrf.network.Network('./190ghz_tx_measured.S2P')
vf = VectorFitting(nw)
freqs = np.linspace(100e9, 300e9, 401)
```

Example of noisy data with not enough poles

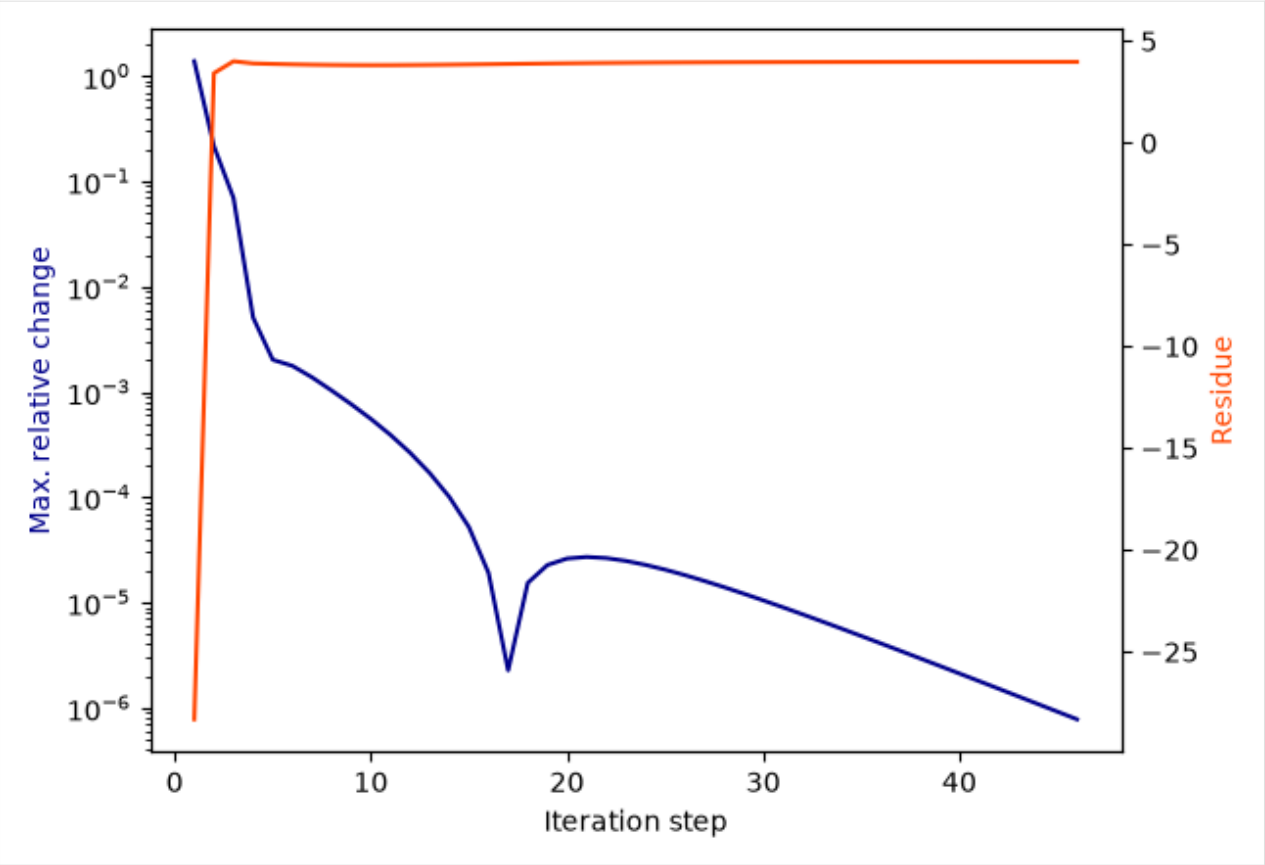
Three real poles and one complex conjugate pair is not enough for an acurrate fit of this network. It takes surprisingly many iterations and the resulting model error is still fairly large:

[2]:
```
vf.vector_fit(n_poles_real=3, n_poles_cmplx=1)
```

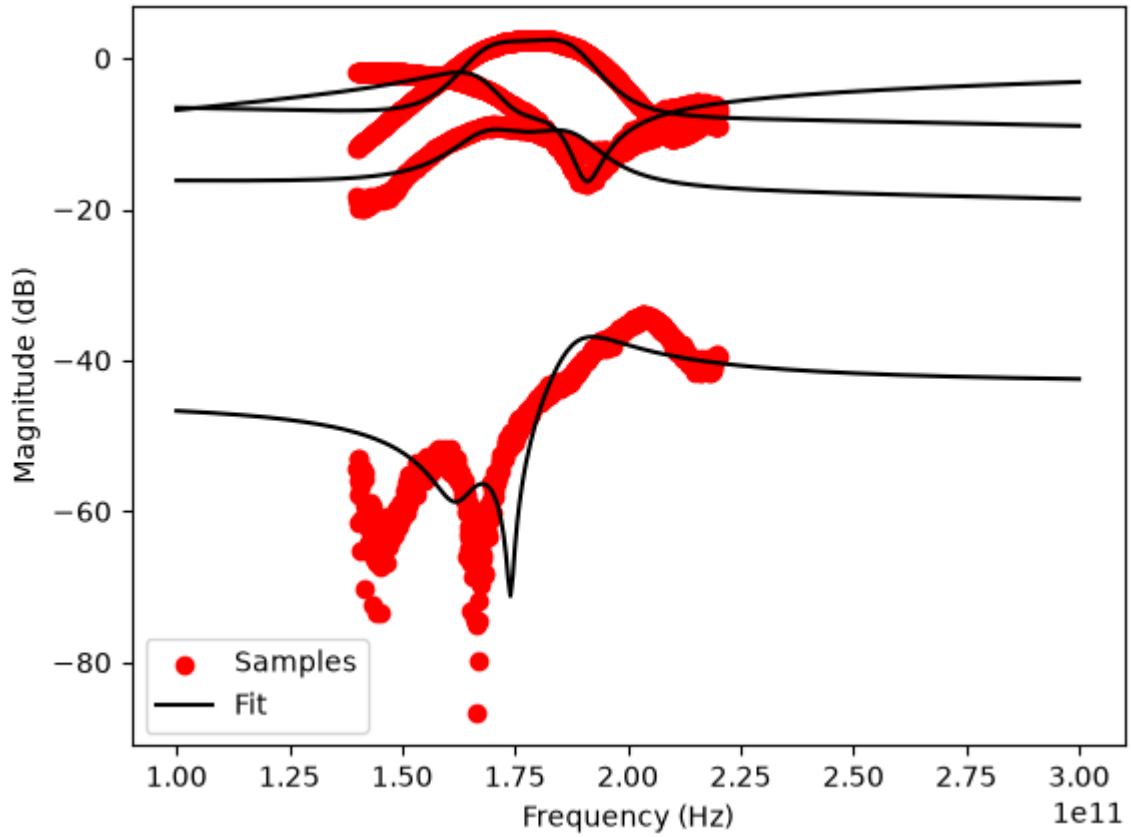
[3]:
```
print(f'RMS error = {vf.get_rms_error()}')
```

```
RMS error = 0.22701137743065555
```

[4]:
```
vf.plot_convergence()
```



[5]:
```
vf.plot_s_db(freqs=freqs)
```



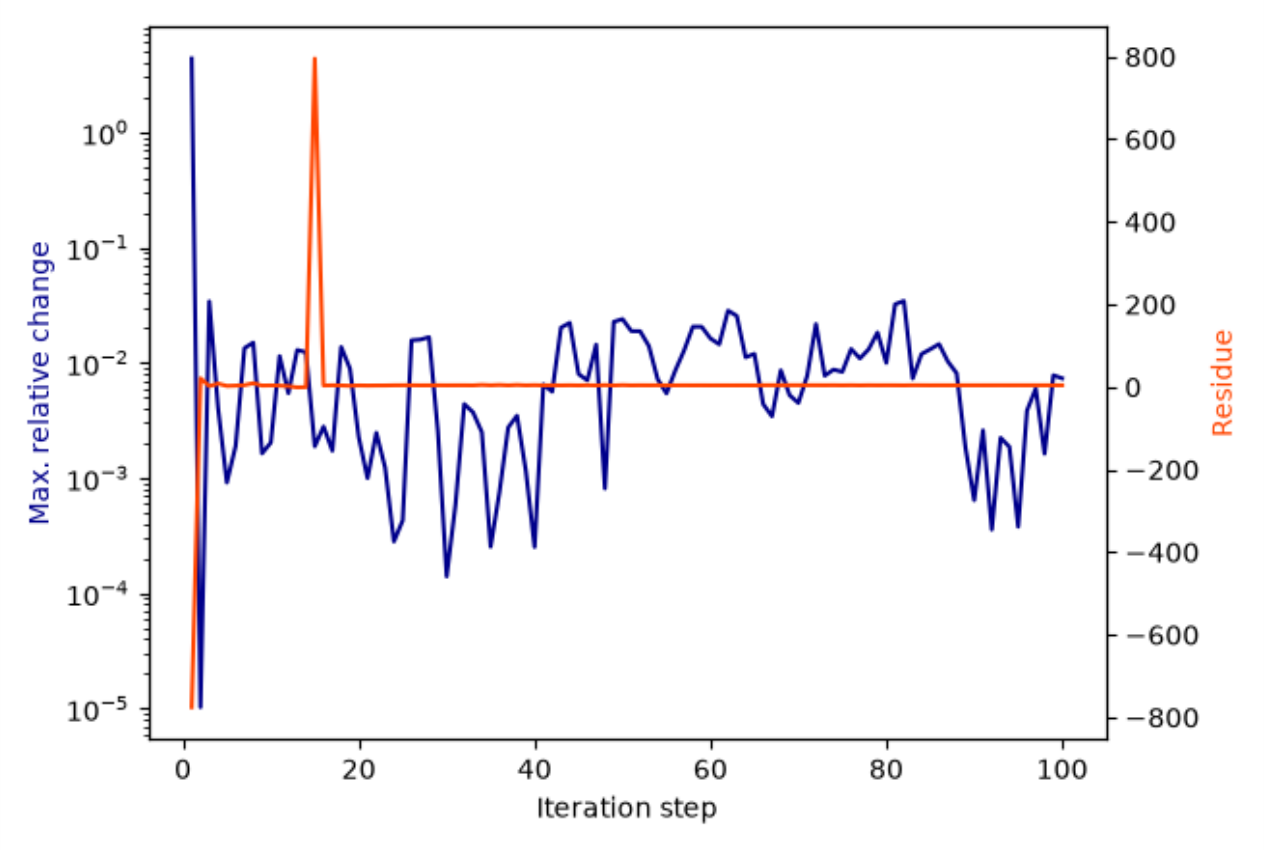
Example of noisy data with too many poles

Ten real poles and ten complex conjugate pairs are too much for this simple network. The unused poles get shifted toward higher frequencies outside the measured band and still spook around during the relocation process. Due to these spurious pole, the relocation process cannot converge properly and gets stopped after reaching the maximum number of iterations (default is 100). Interstingly, the residue settles rather quickly within the first 20 iterations, as shown in the convergence plot. Still, the fitting result within the measured frequency band is very good, but the spurious out-of-band poles are clearly visible.

[6]:
```
vf.vector_fit(n_poles_real=10, n_poles_cmplx=10)
```

```
/tmp/ipykernel_9198/2893947848.py:1: RuntimeWarning: Vector Fitting: The pole relocation␣
↪process stopped after reaching the maximum number of iterations (N_max = 100). The␣
↪results did not converge properly.
Hint: the linear system was ill-conditioned (max. condition number was 2.
↪2170670374393428e+16).
  vf.vector_fit(n_poles_real=10, n_poles_cmplx=10)
```

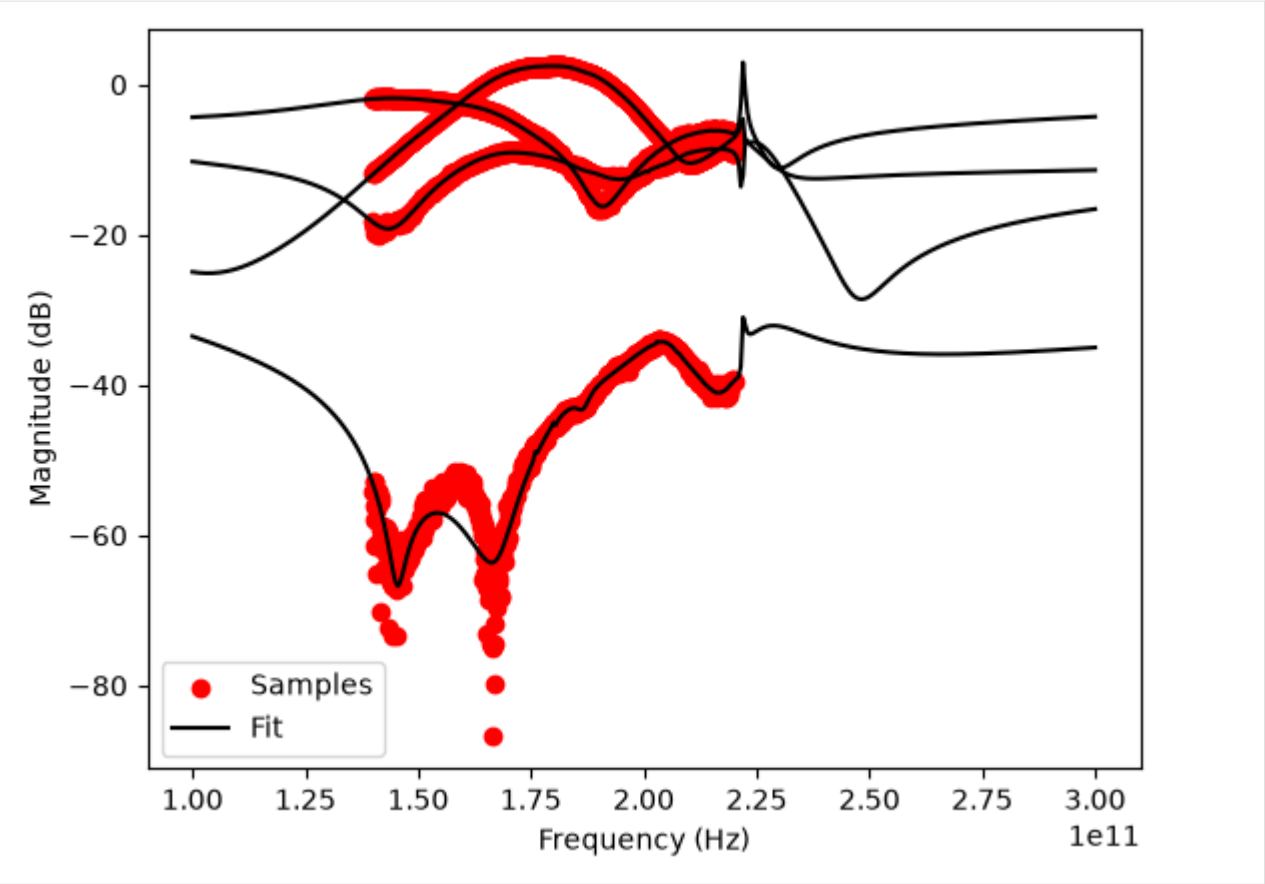
[7]:
```
vf.plot_convergence()
```



[8]:
```
print(f'RMS error = {vf.get_rms_error()}')
```

```
RMS error = 0.013207154679437493
```

[9]:
```
vf.plot_s_db(freqs=freqs)
```



Comment on starting poles

During the pole relocation process (first step in the fitting process), the starting poles are successively moved to frequencies where they can best match all target responses. Additionally, the type of poles can change from real to complex-conjugate: two real poles can become one complex-conjugate pole (and vice versa). As a result, there are multiple combinations of starting poles which can produce the same final set of poles. However, certain setups will converge faster than others, which also depends on the initial pole spacing. In extreme cases, the algorithm can even be "undecided" if two real poles behave exactly like one complex-conjugate pole and it gets "stuck" jumping back and forth without converging to a final solution.

Equivalent setups for a fitting attempt with `n_poles_real=3, n_poles_cmplx=4` (i.e. 3+4):

- 1+5
- **3+4**
- 5+3
- 7+2
- 9+1
- 11+0

Equivalent setups for another fitting attempt with `n_poles_real=4, n_poles_cmplx=4` (i.e. 4+4):

- 0+6
- 2+5

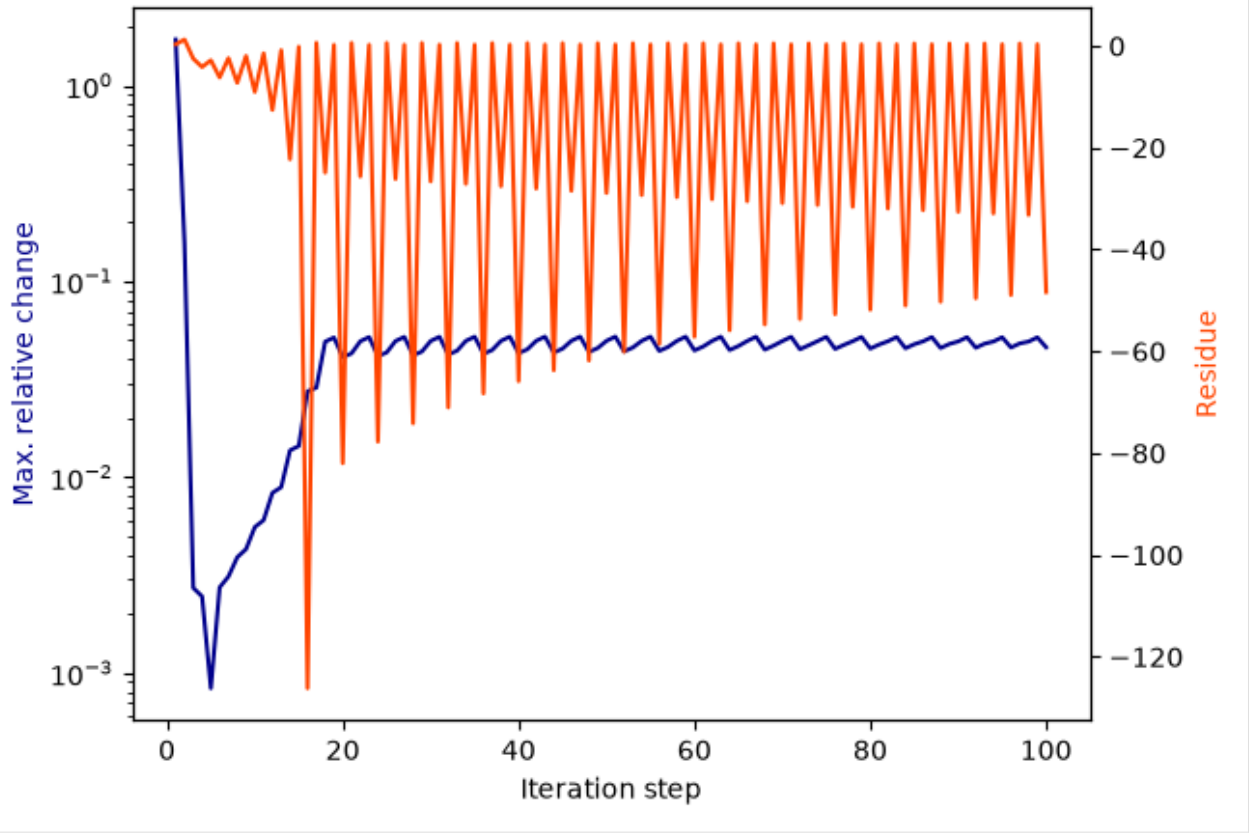
- **4+4**
- 6+3
- 8+2
- 10+1
- 12+0

Examples for problematic setups that do not converge properly due to an oscillation between two (equally good) solutions:

- 0+5 <–> 2+4 <–> …
- 0+7 <–> 2+5 <–> …

[10]:
```
vf.vector_fit(n_poles_real=0, n_poles_cmplx=5)
vf.plot_convergence()
```

```
/tmp/ipykernel_9198/2802061987.py:1: RuntimeWarning: Vector Fitting: The pole relocation␣
↪process stopped after reaching the maximum number of iterations (N_max = 100). The␣
↪results did not converge properly.
  vf.vector_fit(n_poles_real=0, n_poles_cmplx=5)
```

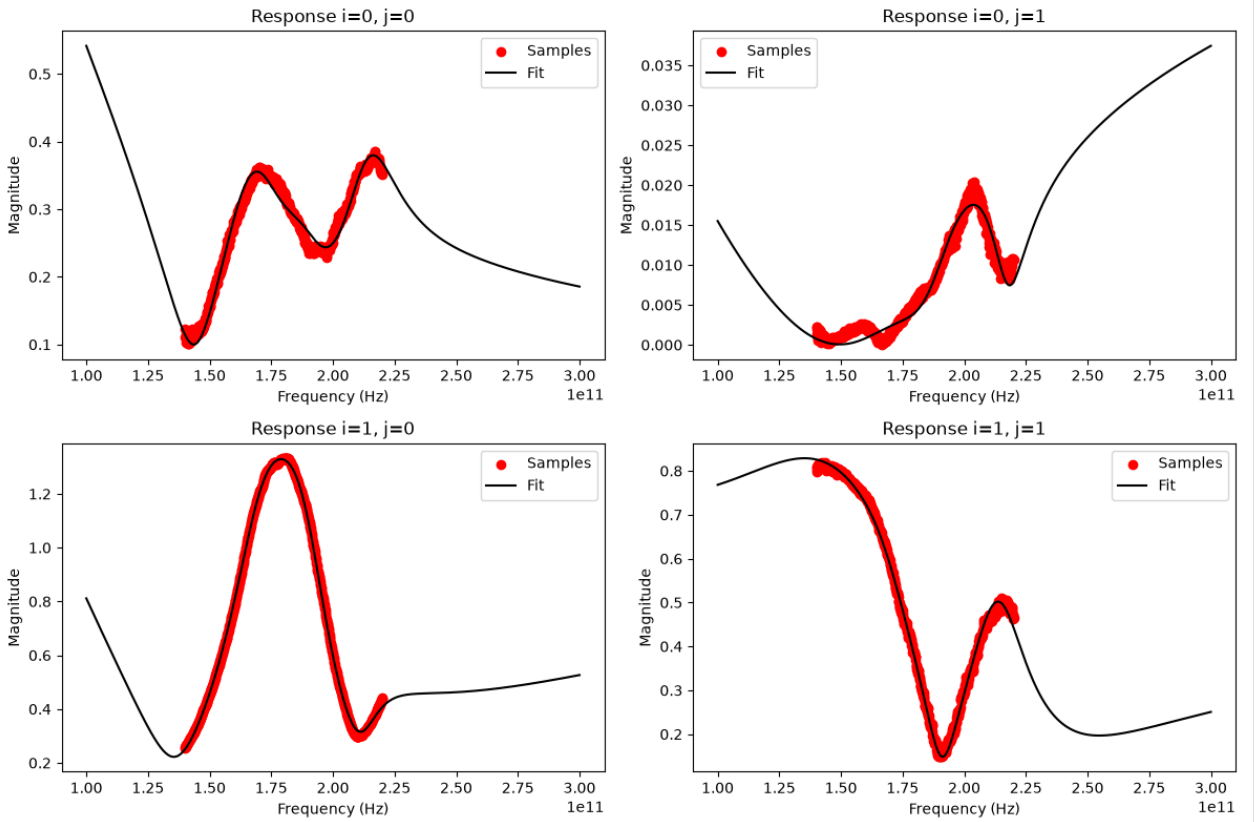


Even though the pole relocation process oscillated between two (or more?) solutions and did not converge, the fit was still successful, because the solutions themselves did converge:

[11]:
```
vf.get_rms_error()
```

[11]:
```
np.float64(0.027879745406753236)
```

[12]:
```
fig, ax = mplt.subplots(2, 2)
fig.set_size_inches(12, 8)
vf.plot_s_mag(0, 0, freqs=freqs, ax=ax[0][0]) # s11
vf.plot_s_mag(0, 1, freqs=freqs, ax=ax[0][1]) # s12
vf.plot_s_mag(1, 0, freqs=freqs, ax=ax[1][0]) # s21
vf.plot_s_mag(1, 1, freqs=freqs, ax=ax[1][1]) # s22
fig.tight_layout()
mplt.show()
```



2.9 Parametric Vector Fitting

2.9.1 Parametric Vector Fitting Of An Inductor

As an example for the application of parametric vector fitting, a set of electromagnetic simulations of an inductor is given. The inductors are simulated for different diameters $D = [80, 100, 120, 140, 160, 180, 200]$ µm. For this demonstration, the fitting process will be based on just a subset $D_{\text{fit}} = [80, 120, 160, 200]$ µm. The remaining datapoints will be used later to assess the fit accuracy at $D_{\text{check}} = [100, 140, 180]$ µm.

In the following, a typical workflow to create and use the parametric model is demonstrated.

Create a NetworkSet

The individual simulation files need to be loaded and grouped into a labelled NetworkSet.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np
```

(continues on next page)

(continued from previous page)

```
from skrf.network import Network
from skrf.networkSet import NetworkSet
from skrf.vectorFitting import VectorFittingParametric

networks = [Network('spiral_D80.s2p', params={'D': 80}),
            Network('spiral_D120.s2p', params={'D': 120}),
            Network('spiral_D160.s2p', params={'D': 160}),
            Network('spiral_D200.s2p', params={'D': 200})]
nwset = NetworkSet(networks)
```

Initialize and run the fitting process

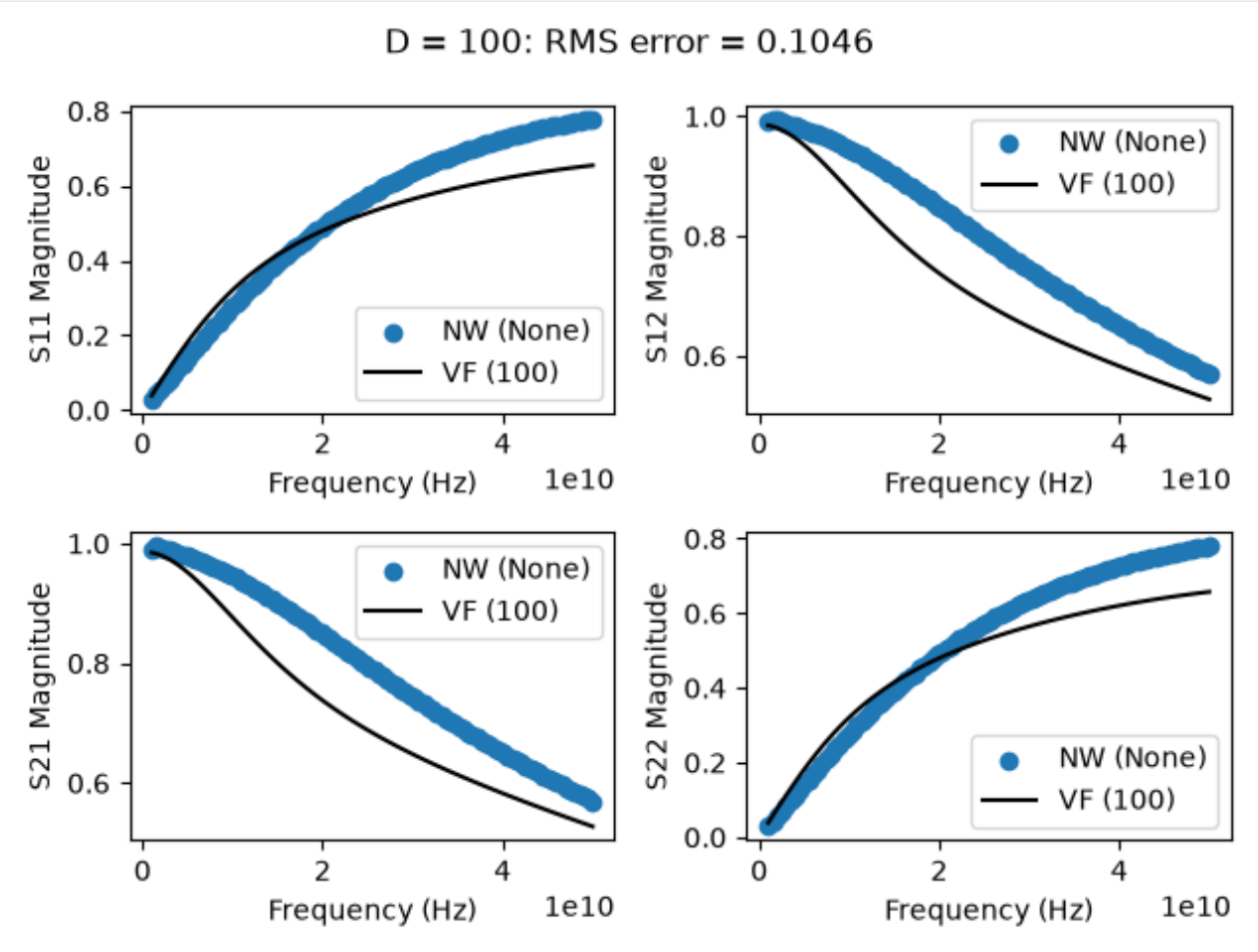
These particular networks turned out to be overly sensitive to the initial pole setup for the vector fitting process. For most networks, the automatic pole initialization (with `n_poles_real=-1, n_poles_cmplx=-1` will work well. Here, we have to specify `n_poles_real=3, n_poles_cmplx=0`:

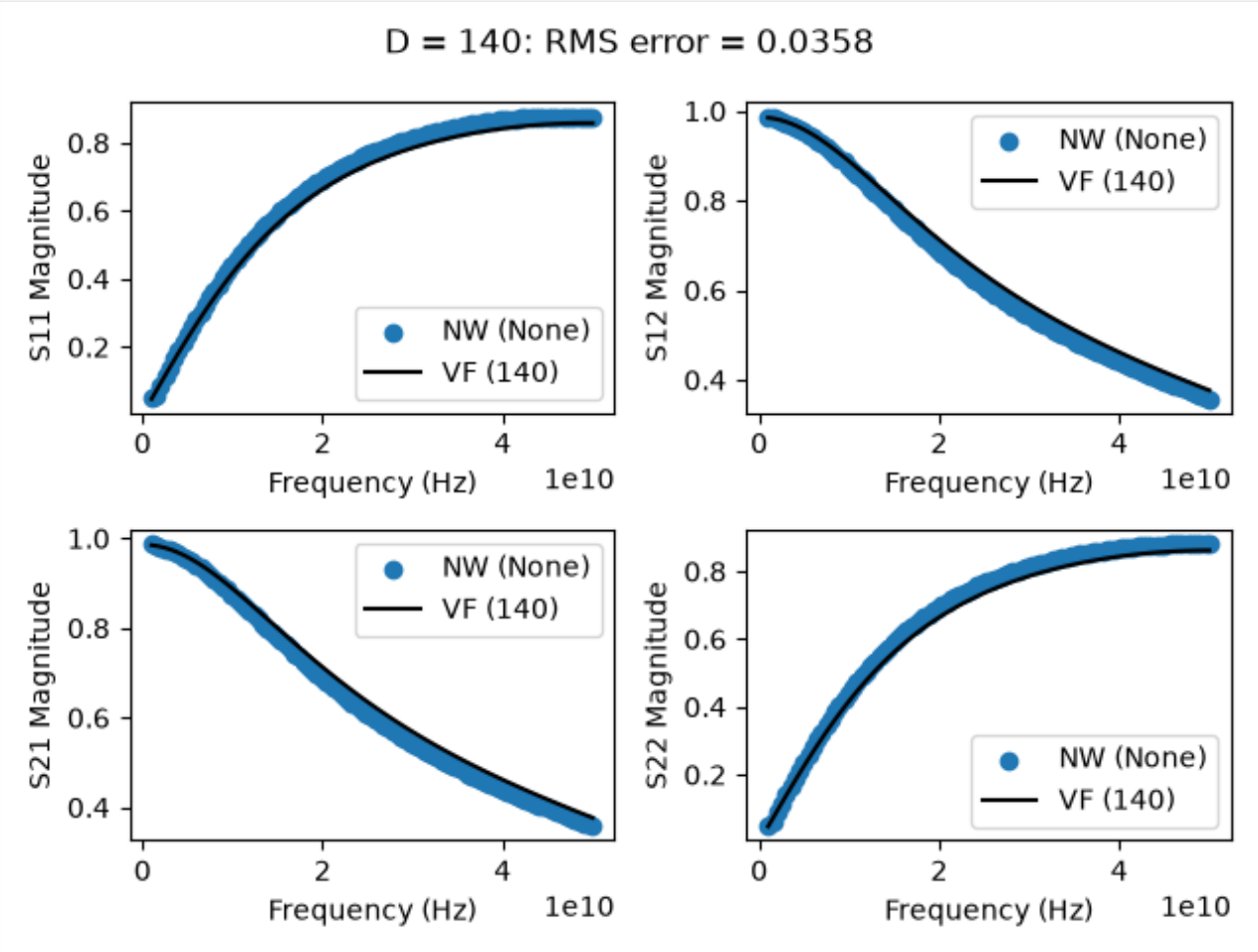
[2]:
```
vf = VectorFittingParametric(nwset, n_poles_real=3, n_poles_cmplx=0)
vf.auto_fit()
```

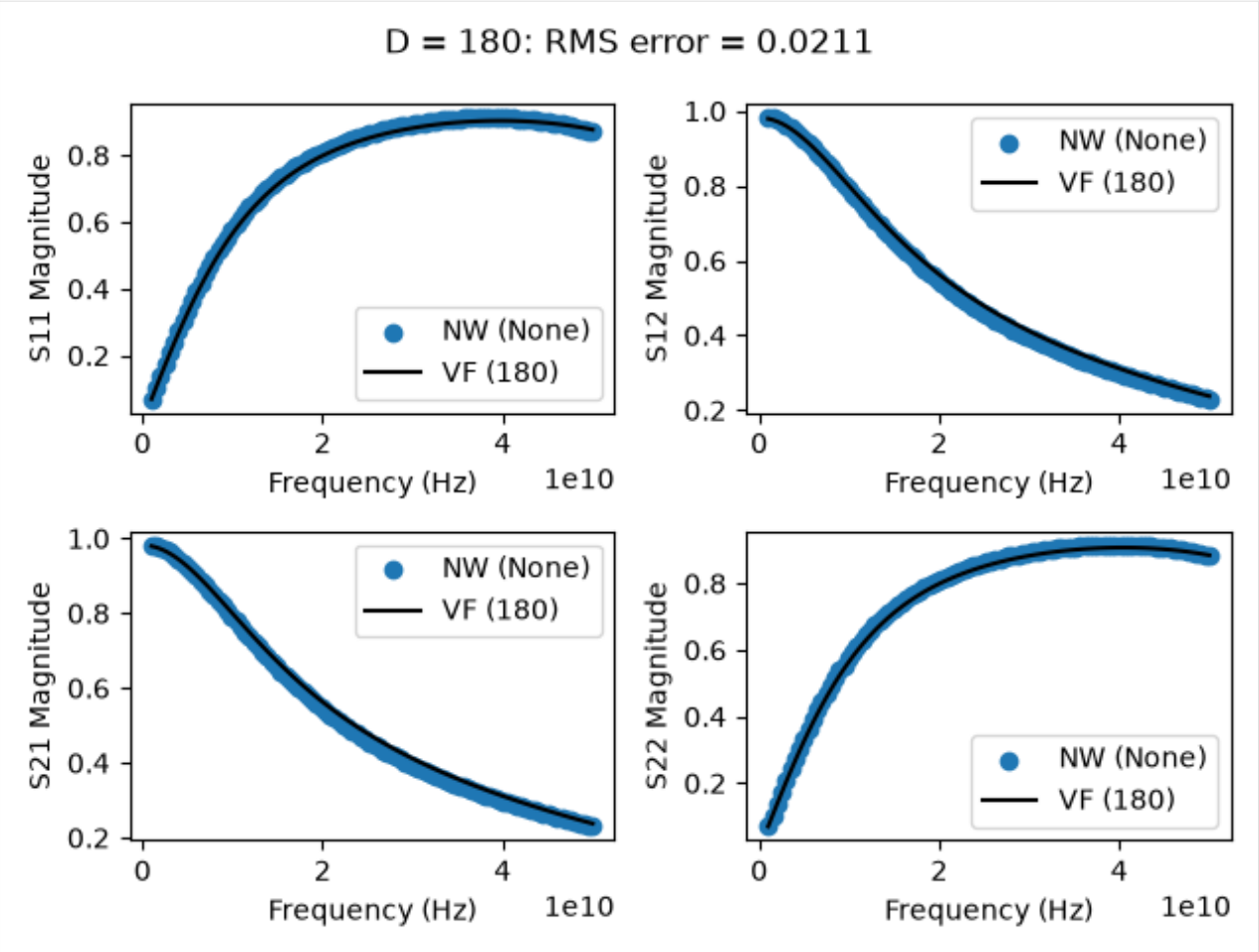
```
/home/docs/checkouts/readthedocs.org/user_builds/scikit-rf/checkouts/latest/skrf/
↪vectorFitting.py:2792: UserWarning: Passivity enforcement: The dc point in the model␣
↪is not passive. Cannot preserve the dc point during passivity enforcement.
  vf.passivity_enforce()
```

Use the model

[3]:
```
for d_check in [100, 140, 180]:
    nw = Network(f'spiral_D{d_check}.s2p')
    model = vf.get_model_response({'D': d_check}, nw.f)
    error = np.sqrt(np.mean(np.square(np.abs(model - nw.s))))

    n_ports = nw.nports
    fig, ax = plt.subplots(n_ports, n_ports)
    fig.suptitle(f'D = {d_check}: RMS error = {error:.4f}')
    for i in range(n_ports):
        for j in range(n_ports):
            ax[i][j].scatter(nw.f, np.abs(nw.s[:, i, j]), label=f'NW ({nw.params})')
            ax[i][j].plot(nw.f, np.abs(model[:, i, j]), color='k', label=f'VF ({d_check})
↪')
            ax[i][j].legend()
            ax[i][j].set_xlabel('Frequency (Hz)')
            ax[i][j].set_ylabel(f'S{i+1}{j+1} Magnitude')
    fig.tight_layout()
    plt.show()
```


D = 100: RMS error = 0.1046
S11 Magnitude
S12 Magnitude
S21 Magnitude
S22 Magnitude
Frequency (Hz)
1e10
NW (None)
VF (100)


D = 140: RMS error = 0.0358
S11 Magnitude
S12 Magnitude
S21 Magnitude
S22 Magnitude
Frequency (Hz)
1e10
NW (None)
VF (140)



2.10 Q-factor

2.10.1 Finding Dielectric Constant and Loss from Resonance Fitting

In this example, the Q-factors are fitted from S-parameters data measured to characterize PCB laminate materials. The data correspond to a 36, 72 and 144 mm long stripline resonator respectively. We will then deduce the Dielectric permittivity and loss constants.

Fitting Q-factors

[1]:
```
import matplotlib.pyplot as plt

import skrf as rf
from skrf.constants import c
from skrf.qfactor import Qfactor

rf.stylely()
```

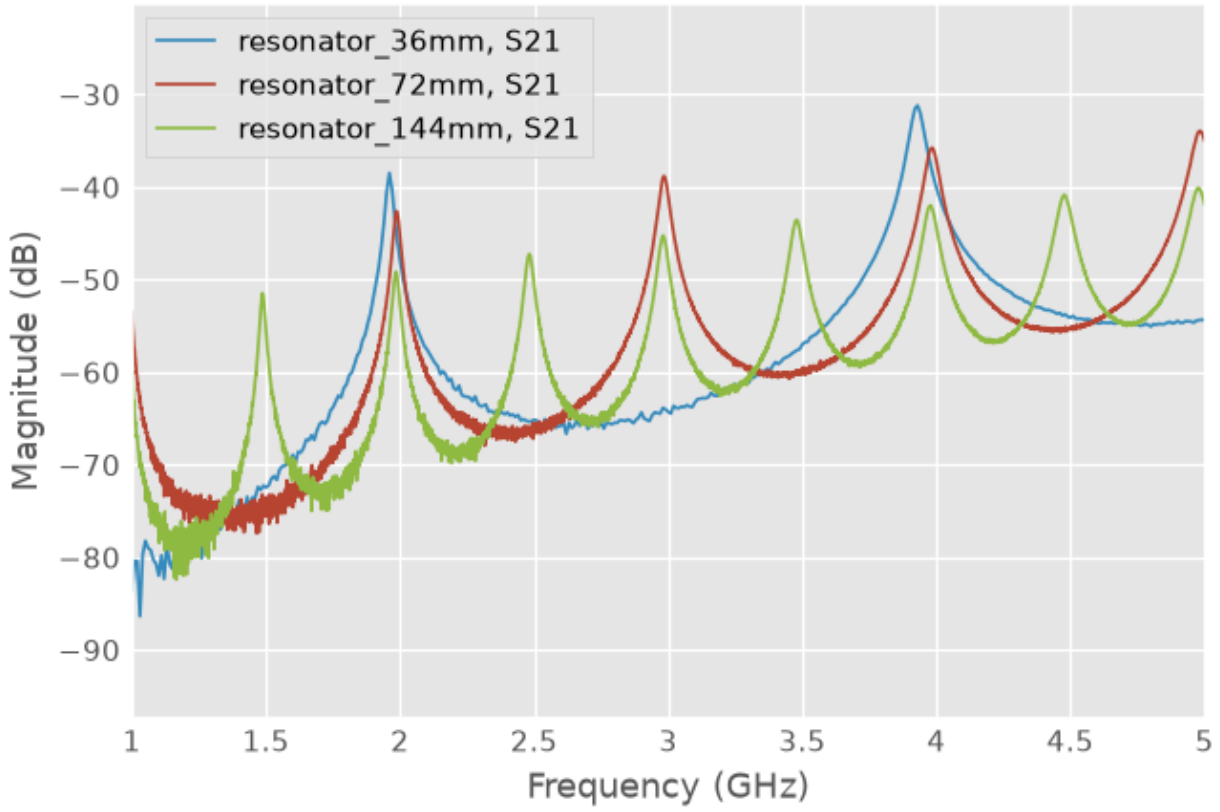
[2]:
```
# Loading the data.
# Setting the frequency unit is optional, just a convenience for displaying results
Reson36 = rf.Network('data/resonator_36mm.s2p', f_unit='GHz')
```

(continues on next page)

(continued from previous page)

```
Reson72 = rf.Network('data/resonator_72mm.s2p', f_unit='GHz')
Reson144 = rf.Network('data/resonator_144mm.s2p', f_unit='GHz')
```

[3]:
```
Reson36.plot_s_db(m=1, n=0)
Reson72.plot_s_db(m=1, n=0)
Reson144.plot_s_db(m=1, n=0)
```



To find the Q-factors of these resonators, we create a `Qfactor` object from the transmission S-parameters we want to fit.

Note that is there are many resonances close to each other, it is recommended to pass the frequency sub-Networks for the range of frequency of interest, to help the convergence of the fitting algorithm. Here, we are interested in the resonances occurring around 2 and 4 GHz, hence:

[4]:
```
# Around 2 GHz
QF_36_2GHz = Qfactor(Reson36['1.75-2.25GHz'].s21, res_type='transmission')
QF_72_2GHz = Qfactor(Reson72['1.75-2.25GHz'].s21, res_type='transmission')
QF_144_2GHz = Qfactor(Reson144['1.75-2.25GHz'].s21, res_type='transmission')
# Around 4 GHz
QF_36_4GHz = Qfactor(Reson36['3.75-4.25GHz'].s21, res_type='transmission')
QF_72_4GHz = Qfactor(Reson72['3.75-4.25GHz'].s21, res_type='transmission')
QF_144_4GHz = Qfactor(Reson144['3.75-4.25GHz'].s21, res_type='transmission')

all_QFs = [QF_36_2GHz, QF_72_2GHz, QF_144_2GHz, QF_36_4GHz, QF_72_4GHz, QF_144_4GHz]
```

[5]:
```
print(*all_QFs, sep='\n')
```

```
Q-factor of Network resonator_36mm. (not fitted)
Q-factor of Network resonator_72mm. (not fitted)
Q-factor of Network resonator_144mm. (not fitted)
Q-factor of Network resonator_36mm. (not fitted)
Q-factor of Network resonator_72mm. (not fitted)
Q-factor of Network resonator_144mm. (not fitted)
```

Then we fit the Q-factors:

[6]:
```
[Q.fit() for Q in all_QFs]
print(*all_QFs, sep='\n')
```

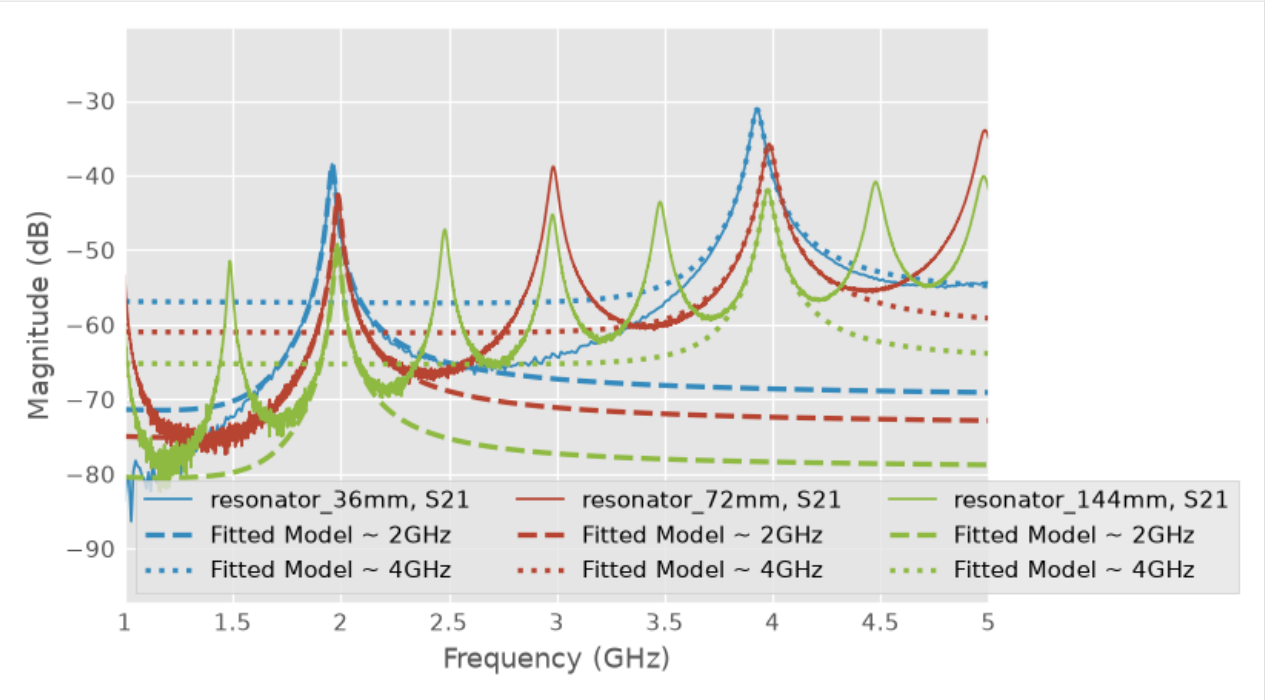
```
Q-factor of Network resonator_36mm. (fitted: f_L=1.960GHz, Q_L=72.475)
Q-factor of Network resonator_72mm. (fitted: f_L=1.987GHz, Q_L=74.283)
Q-factor of Network resonator_144mm. (fitted: f_L=1.985GHz, Q_L=73.545)
Q-factor of Network resonator_36mm. (fitted: f_L=3.927GHz, Q_L=74.018)
Q-factor of Network resonator_72mm. (fitted: f_L=3.983GHz, Q_L=75.717)
Q-factor of Network resonator_144mm. (fitted: f_L=3.977GHz, Q_L=74.711)
```

We can create the Networks corresponding to the fitted results to benchmark the model against the measurements:

[7]:
```
new_freq = rf.Frequency(1, 5, unit='GHz', npoints=1001)
fitted_ntwk_36_2GHz = QF_36_2GHz.fitted_network(frequency=new_freq)
fitted_ntwk_72_2GHz = QF_72_2GHz.fitted_network(frequency=new_freq)
fitted_ntwk_144_2GHz = QF_144_2GHz.fitted_network(frequency=new_freq)

fitted_ntwk_36_4GHz = QF_36_4GHz.fitted_network(frequency=new_freq)
fitted_ntwk_72_4GHz = QF_72_4GHz.fitted_network(frequency=new_freq)
fitted_ntwk_144_4GHz = QF_144_4GHz.fitted_network(frequency=new_freq)
```

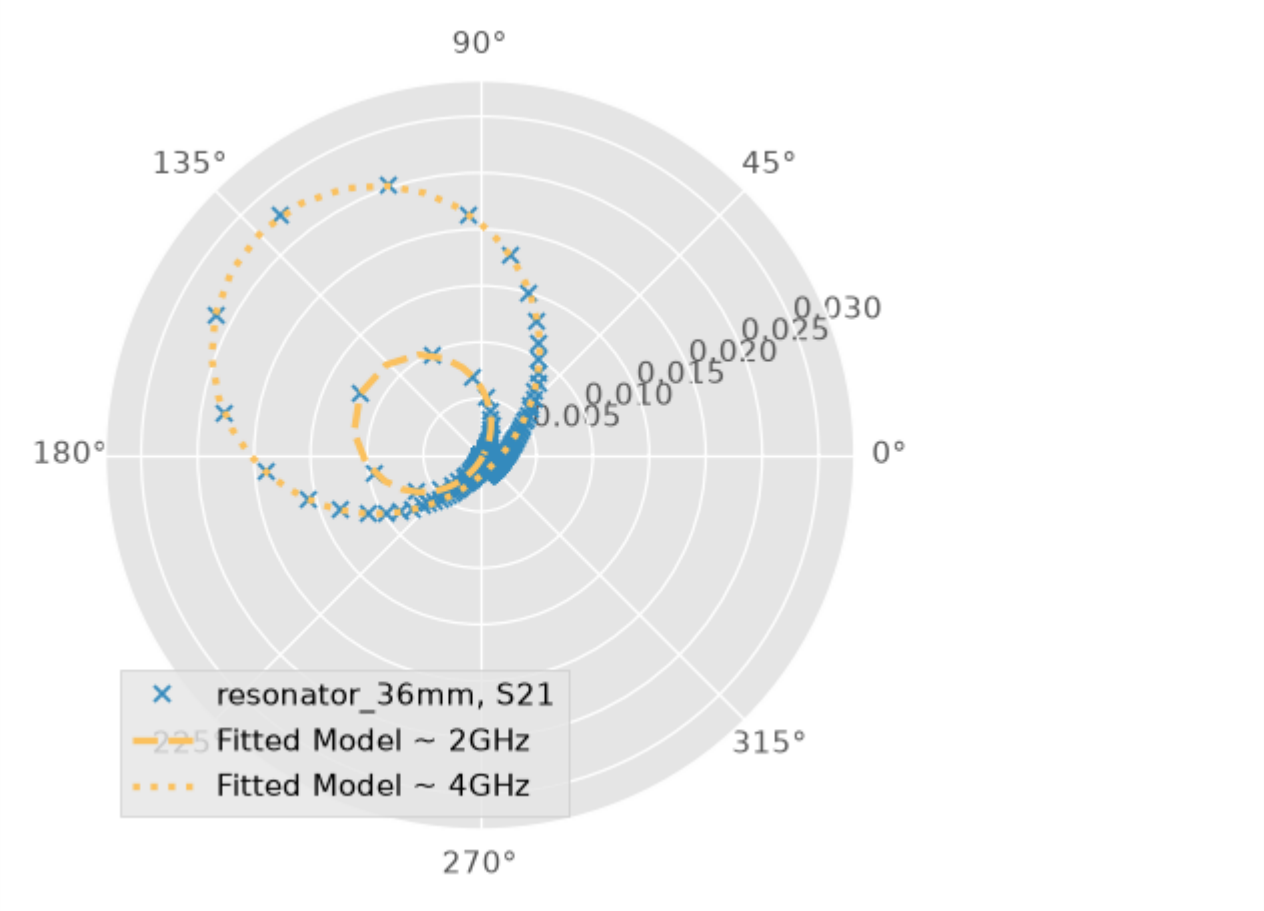
[8]:
```
Reson36.plot_s_db(m=1, n=0, color='C0')
fitted_ntwk_36_2GHz.plot_s_db(label='Fitted Model ~ 2GHz', lw=2, color='C0', ls='--')
fitted_ntwk_36_4GHz.plot_s_db(label='Fitted Model ~ 4GHz', lw=2, color='C0', ls=':')
Reson72.plot_s_db(m=1, n=0, color='C1')
fitted_ntwk_72_2GHz.plot_s_db(label='Fitted Model ~ 2GHz', lw=2, color='C1', ls='--')
fitted_ntwk_72_4GHz.plot_s_db(label='Fitted Model ~ 4GHz', lw=2, color='C1', ls=':')
Reson144.plot_s_db(m=1, n=0, color='C2')
fitted_ntwk_144_2GHz.plot_s_db(label='Fitted Model ~ 2GHz', lw=2, color='C2', ls='--')
fitted_ntwk_144_4GHz.plot_s_db(label='Fitted Model ~ 4GHz', lw=2, color='C2', ls=':')
plt.gca().legend(ncol=3)
```

[8]:
```
<matplotlib.legend.Legend at 0x7c696af8f0e0>
```

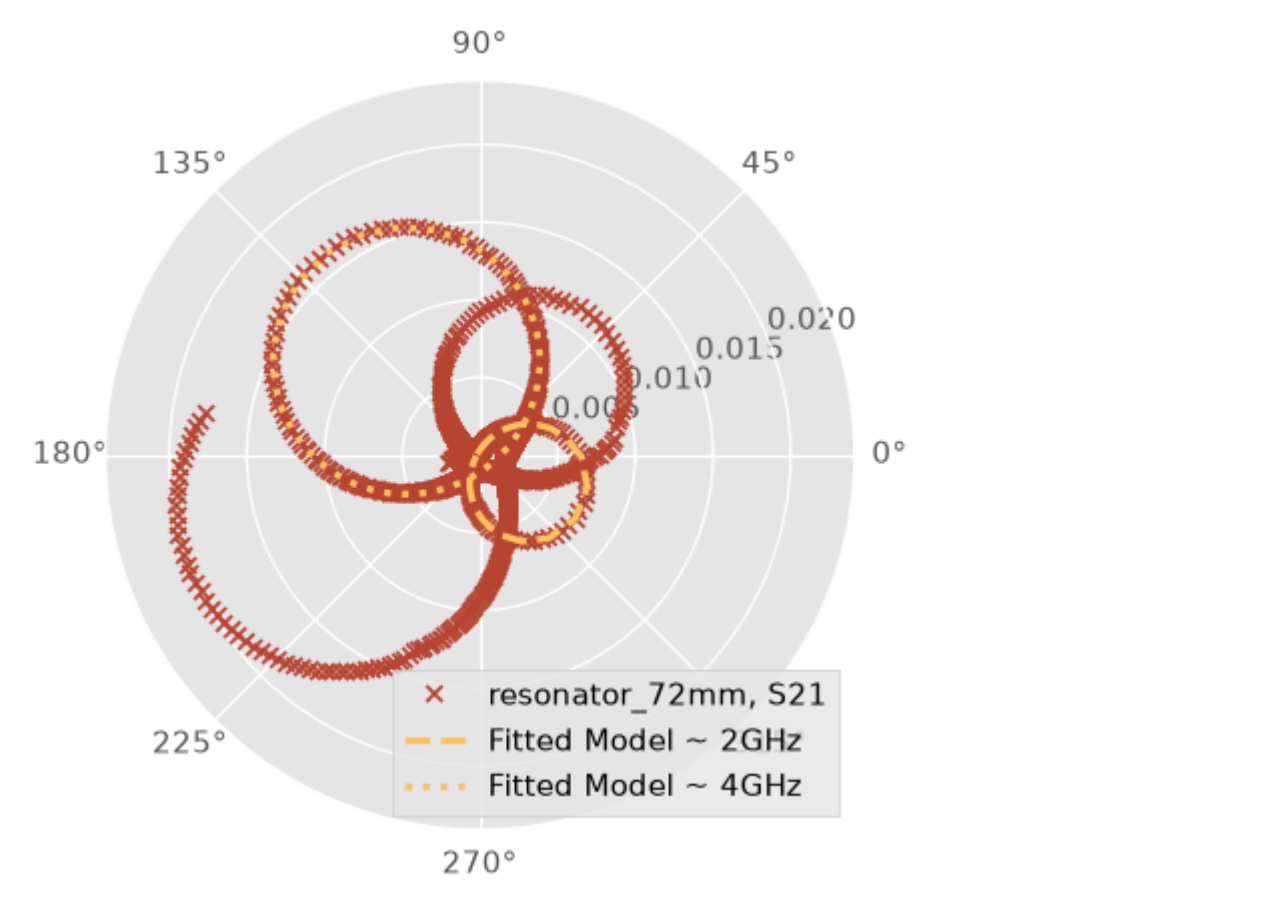


Another way to represent the results is to use polar planes:

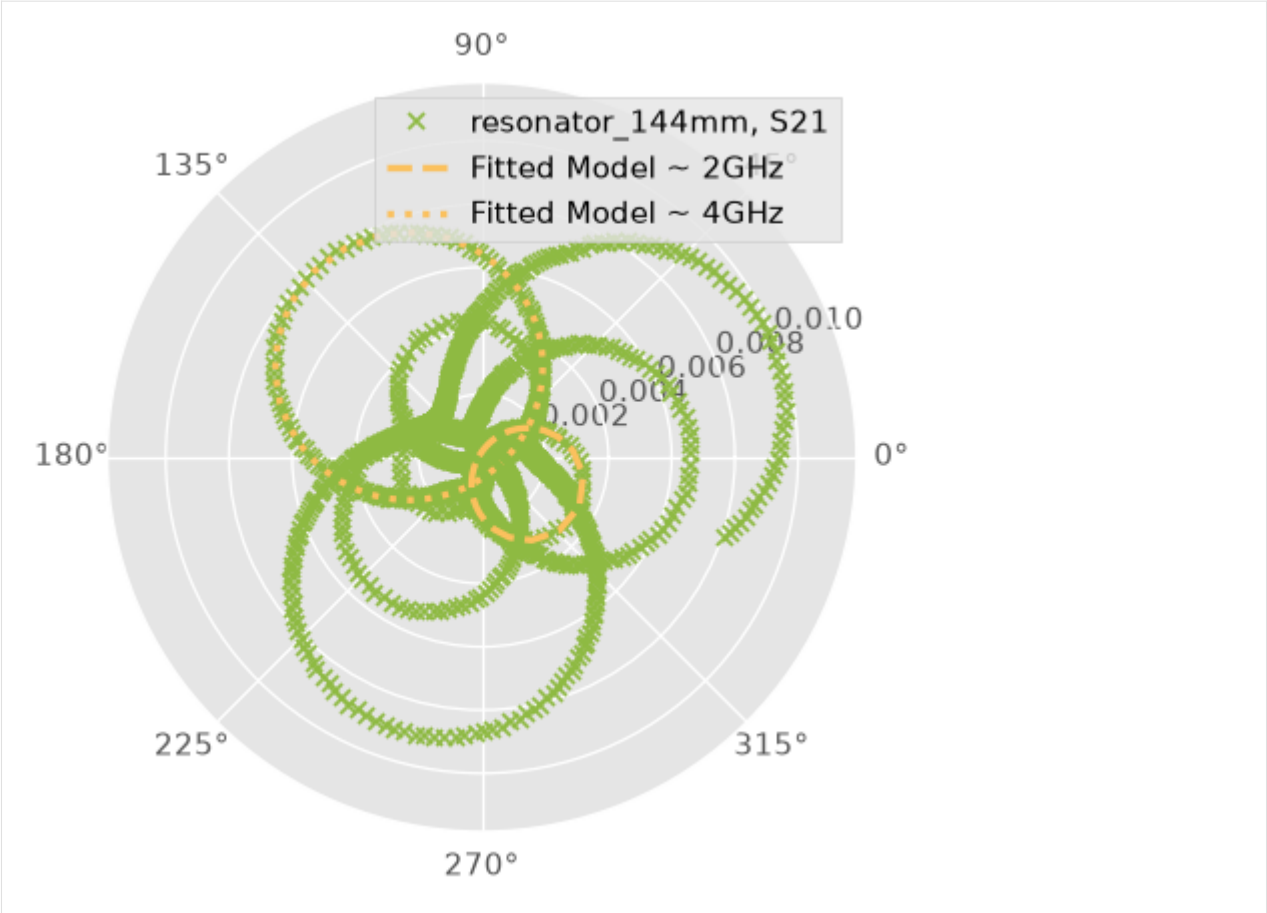
[9]:
```
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
Reson36.plot_s_polar(m=1, n=0, ax=ax, ls='', marker='x', ms=5)
fitted_ntwk_36_2GHz.plot_s_polar(ax=ax, label="Fitted Model ~ 2GHz", lw=2, ls='--',␣
↪color='C3')
fitted_ntwk_36_4GHz.plot_s_polar(ax=ax, label="Fitted Model ~ 4GHz", lw=2, ls=':', color=
↪'C3')
```



[10]:
```
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
Reson72.plot_s_polar(m=1, n=0, ax=ax, ls='', marker='x', ms=5, color='C1')
fitted_ntwk_72_2GHz.plot_s_polar(ax=ax, label="Fitted Model ~ 2GHz", lw=2, ls='--',␣
↪color='C3')
fitted_ntwk_72_4GHz.plot_s_polar(ax=ax, label="Fitted Model ~ 4GHz", lw=2, ls=':', color=
↪'C3')
```



[11]:
```
fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
Reson144.plot_s_polar(m=1, n=0, ax=ax, ls='', marker='x', ms=5, color='C2')
fitted_ntwk_144_2GHz.plot_s_polar(ax=ax, label="Fitted Model ~ 2GHz", lw=2, ls='--',␣
↪color='C3')
fitted_ntwk_144_4GHz.plot_s_polar(ax=ax, label="Fitted Model ~ 4GHz", lw=2, ls=':',␣
↪color='C3')
```



Increasing Accuracy

If you want to get the best accuracy in the calculation of the Q-factors, it's recommended to repeat the fitting a second time using S-parameters data having the resonance peak at the centre frequency and the span reduced to $\pm f_L/Q_L$ (cf. [1] page 21). This is justified in two ways:

- Q_L is defined at the resonant frequency f_L
- Limiting the span reduces the effects of other modes, mismatches, and differences between the actual resonator and the lumped-component model for the resonator. It also adds more points near the peak.

The advantage is greatest if the resonance has poor shape, or if there are other modes nearby. Thus, a recommended workflow for optimal accuracy would be to:

1. Set sweep parameters so the resonance is on the VNA screen
2. Get Measurements
3. Fit Q param to deduce f_L
4. Set the Centre Freq to f_L
5. Set Span to $2f_L/Q_L$
6. Get Measurements
7. Fit Q param
8. Calculate Unloaded Q

So, for example:

[12]:
```
# extracting the subfrequency range of interest from the data
f_L = QF_72_2GHz.f_L
Q_L = QF_72_2GHz.Q_L
span = f_L/Q_L
ntwk = Reson72[f'{f_L - span/2}-{f_L + span/2}Hz']
```

[13]:
```
QF_72_2GHz_2 = Qfactor(ntwk.s21, res_type='transmission')
```

[14]:
```
# After the second fit, the weighting ratio is close to 0.2 at the minimum and maximum␣
↪frequencies
# provided that the loop_plan contains 'w'. (cf. [1] Fig. 6(b) and equation (28))
QF_72_2GHz_2.fit()
```

[14]:
```
               Q_L: np.float64(73.9766246902494)
         RMS_Error: array([[3.08798804e-05]])
               f_L: np.float64(1986775772.7937503)
                m1: np.float64(-0.0002509776933233847)
                m2: np.float64(-3.0130265829522624e-05)
                m3: np.float64(0.006746948979003869)
                m4: np.float64(-0.0035023152339341804)
            method: 'NLQFIT6'
 number_iterations: 6
           success: array([[ True]])
   weighting_ratio: np.float64(1.9692726960194984)
```

[15]:
```
QF_72_2GHz_2.Q_L  # higher accury fit
```

[15]:
```
np.float64(73.9766246902494)
```

Deducing Permittivity and Loss

Equations for calculation Dk (aka ϵ_r) and Df (aka $\tan\delta$) from measured data for stripline resonators are given in [2]:

$$\epsilon_r = \left[\frac{cn}{2f_r(L+\Delta L)}\right]^2$$

where:

- L is the length of resonator,
- n is the number of half wavelengths at resonance in resonator,
- f_r is the resonance frequency of resonator,
- c is the speed of light in vacuum,
- ΔL is the total effective increase in length of the resonant strip due to the fringing field at the ends of the resonant strip (neglected below for the example)

and

$$\tan\delta = \frac{1}{Q} - \frac{1}{Q_c}$$

where:

- Q_c is the Q-value associated with the copper loss: 250 at 2 GHz and 360 at 4 GHz according to IPC.

Hence, for the permittivity:

[16]:
```
def Dk(f, L, n):
    "Calculates Dk from resonator data. Eq.(1) of ref [1]"
    return (n*c/(2*f*L))**2
```

[17]:
```
Dk(QF_72_2GHz.f_L, 72e-3, 2)
```

[17]:
```
np.float64(4.391664797501884)
```

[18]:
```
Dk(QF_72_2GHz.f_L, 144e-3, 4)
```

[18]:
```
np.float64(4.391664797501884)
```

And the loss tangent:

[19]:
```
Qc = 250  # from IPC ref [1]
print(1/QF_36_2GHz.Q_L - 1/Qc)
```

```
0.00979784715250747
```

[20]:
```
Qc = 250  # from IPC ref [1]
print(1/QF_144_2GHz.Q_L - 1/Qc)
```

```
0.009597091857433796
```

[21]:
```
Qc = 360  # from IPC
print(1/QF_36_4GHz.Q_L - 1/Qc)
```

```
0.01073250544039093
```

[22]:
```
Qc = 360  # from IPC
print(1/QF_144_4GHz.Q_L - 1/Qc)
```

```
0.010607133713033646
```

References

- [1] A. P. Gregory, "Q-factor Measurement by using a Vector Network Analyser", National Physical Laboratory Report MAT 58 (2021), https://eprintspublications.npl.co.uk/9304/
- [2] IPC-TM-650 TEST METHODS MANUAL, https://www.ipc.org/sites/default/files/test_methods_docs/2-5_2-5-5-5.pdf

[ ]:
```
```

2.11 Taper

2.11.1 Tapered Transmission Lines

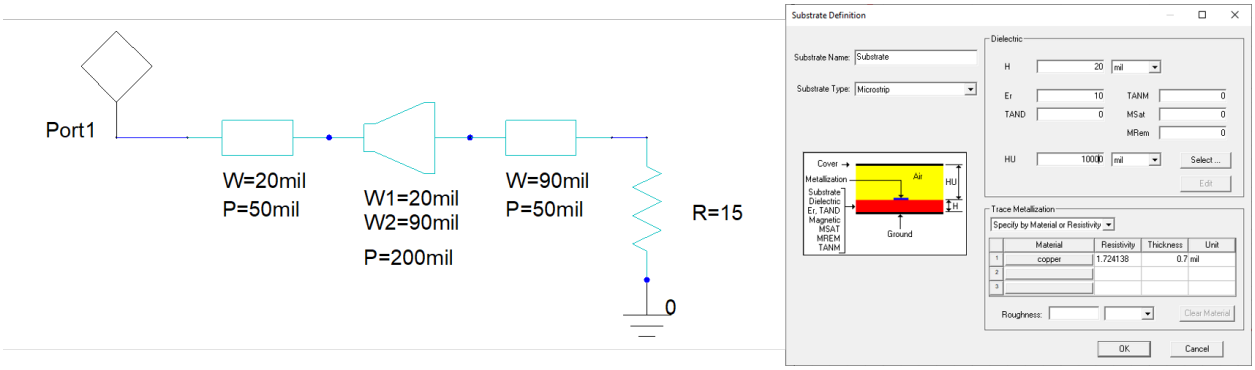
A tapered line (or simply a taper), is an impedance transformer that matches an impedance Z_1 to an impedance Z_2 using a gradually varying characteristic impedance $Z(z)$ along the line.

There is an infinite number of ways one can define a profile $Z(z)$ along the transmission line. scikit-rf implements a generic 1D Taper *Taper1D*, from which the following direct classes are derived: `Linear`, `Exponential` and `SmoothStep`.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf
from skrf.media import MLine
from skrf.network import cascade_list

rf.stylely()
```

Geometry

In this example, we compare a microstrip line which width W is tapered and loaded with a 15 Ohm resistor:



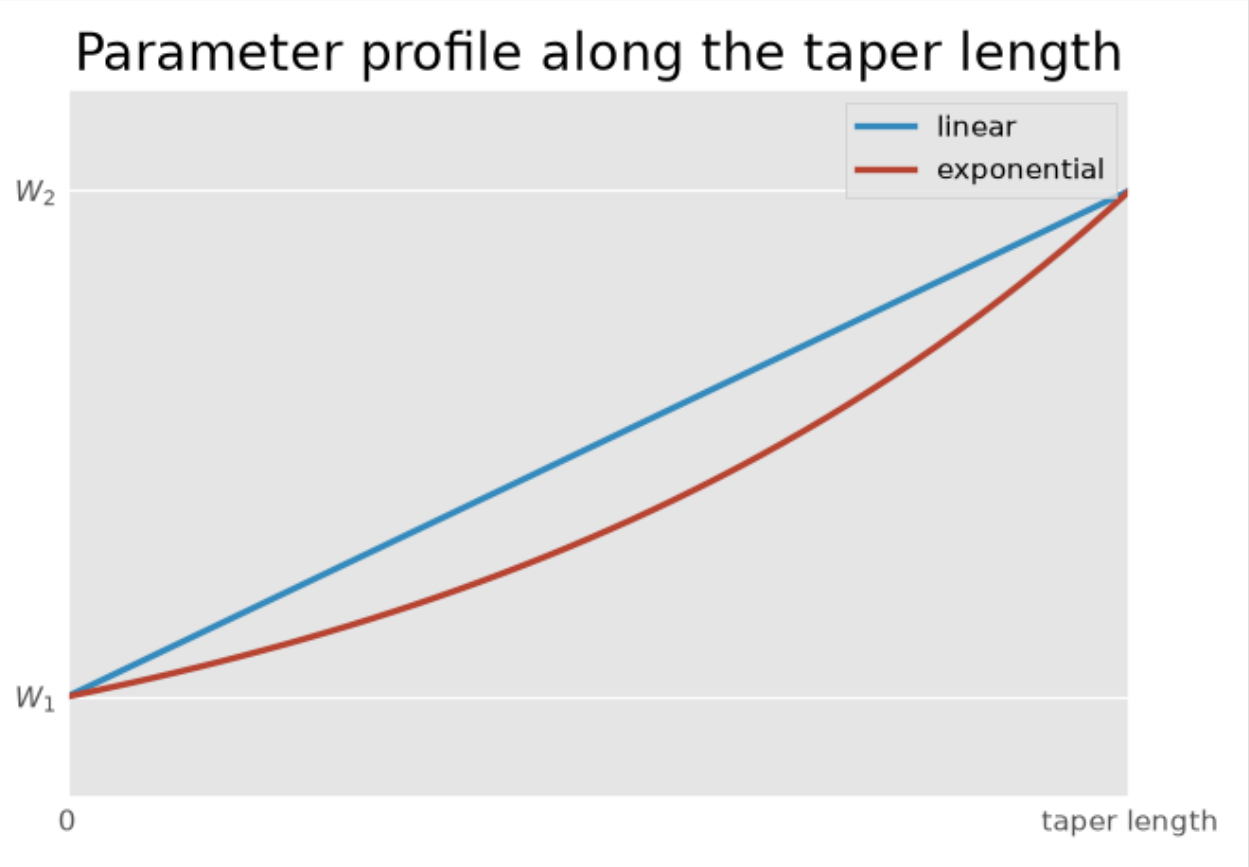
Before comparing the taper profiles, let's define the other elements of the circuit, that is the piece of transmission lines and resistor:

[2]:
```
# Model Parameters
from skrf.constants import mil

freq = rf.Frequency(1, 20, unit='GHz', npoints=191)
w1 = 20*mil  # conductor width [m]
w2 = 90*mil  # conductor width [m]
h = 20*mil  # dielectric thickness [m]
t = 0.7*mil  # conductor thickness [m]
rho = 1.724138e-8  # Copper resistivity [Ohm.m]
ep_r = 10  # dielectric relative permittivity
rough = 1e-6  # conductor RMS roughness [m]
taper_length = 200*mil  # [m]

# Media definitions
microstrip_w1 = MLine(freq, w=w1, h=h, t=t, rho=rho, ep_r=ep_r, rough=rough,
                      disp='kobayashi', z0_port=50)
microstrip_w2 = MLine(freq, w=w2, h=h, t=t, rho=rho, ep_r=ep_r, rough=rough,
                      disp='kobayashi', z0_port=50)

# piece of transmission lines connected to the taper
line1 = microstrip_w1.line(d=50, unit='mil', name='feeder')
line2 = microstrip_w2.line(d=50, unit='mil', name='feeder')

# loading resistor
resistor = microstrip_w2.resistor(R=15)
```

The idea is hence to forge a transmission line of variable characteristic impedance. In this example, the width of the metallization W is varied along the transmission line length. Most common profile are summarized in the figure below:

[3]:
```
fig, ax = plt.subplots()
z = np.linspace(0, taper_length)
ax.plot(z, (w2-w1)/taper_length*z + w1, lw=2, label='linear')
ax.plot(z, w1*np.exp(z/taper_length*(np.log(w2/w1))), lw=2, label='exponential')
ax.set_xticks([0, taper_length])
ax.set_xticklabels(['0', 'taper length'])
ax.set_yticks([w1, w2])
ax.set_yticklabels(['$W_1$', '$W_2$'])
ax.legend()
ax.set_title('Parameter profile along the taper length')
```

[3]:
```
Text(0.5, 1.0, 'Parameter profile along the taper length')
```



Linear Taper

[4]:
```
# create a 2-port Network
taper_linear = rf.taper.Linear(med=MLine, param='w', start=w1, stop=w2,
                        length=taper_length, n_sections=50,
                        med_kw={'frequency': freq, 'h': h, 't':t, 'ep_r': ep_r,
                                'rough': rough, 'rho': rho}).network
print(taper_linear)

# build the full circuit
```

(continues on next page)

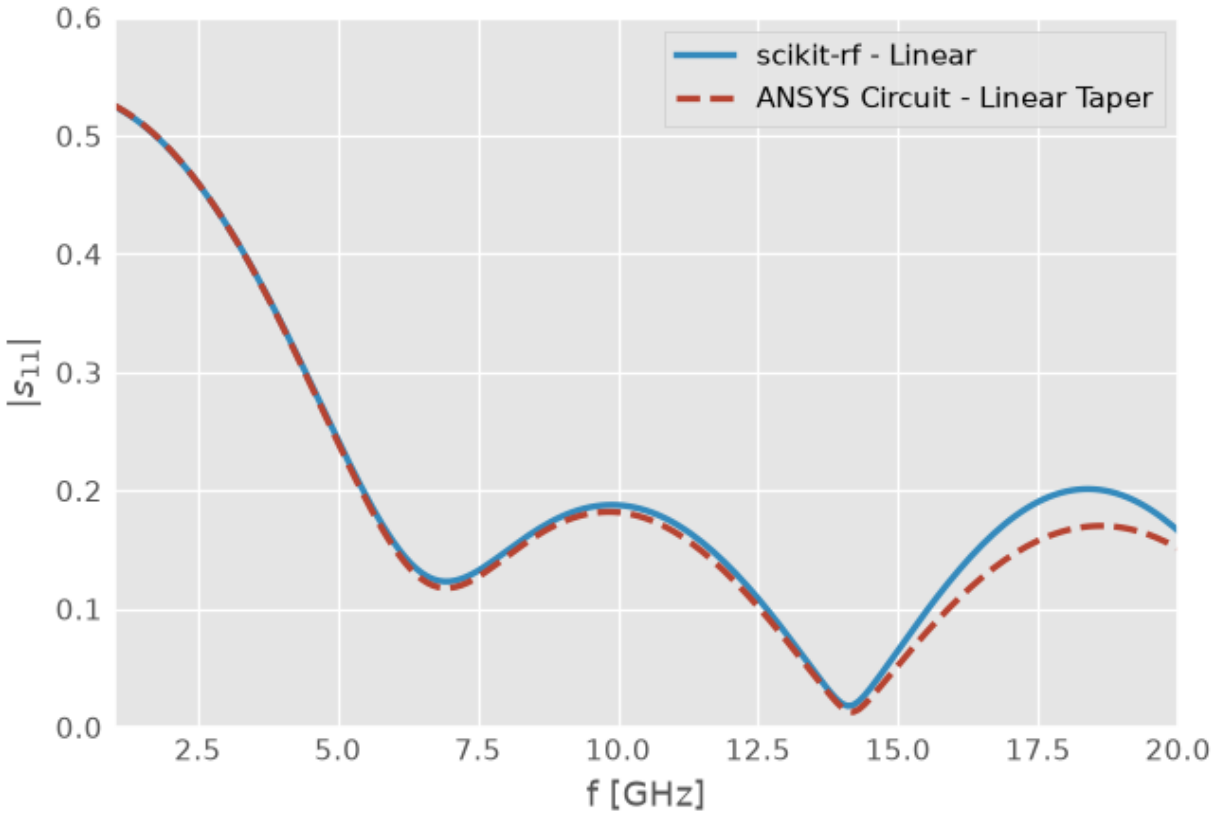
(continued from previous page)

```
# equivalent to ntwk = line1 ** taper_linear ** resistor ** line2 ** microstrip_w2.
↪short()
ntwk_linear = cascade_list([line1, taper_linear, line2, resistor, microstrip_w2.short()])
```

```
2-Port Network: '',  1.0-20.0 GHz, 191 pts, z0=[47.98835487+0.j 18.86744136+0.j]
```

[5]:
```
fig, ax = plt.subplots()
ax.plot(ntwk_linear.frequency.f_scaled, ntwk_linear.s_mag[:,0], lw=2, label='scikit-rf -␣
↪Linear')

f_ref, s_mag_ref = np.loadtxt('ANSYS_Circuit_taper_linear_s_mag.csv', delimiter=',',␣
↪skiprows=1, unpack=True)
ax.plot(f_ref, s_mag_ref, label='ANSYS Circuit - Linear Taper', lw=2, ls='--')

ax.set_xlabel('f [GHz]')
ax.set_ylabel('$|s_{11}|$')
ax.set_ylim(0, 0.6)
ax.set_xlim(1, 20)
ax.legend()
```

[5]:
```
<matplotlib.legend.Legend at 0x7591b18c80b0>
```

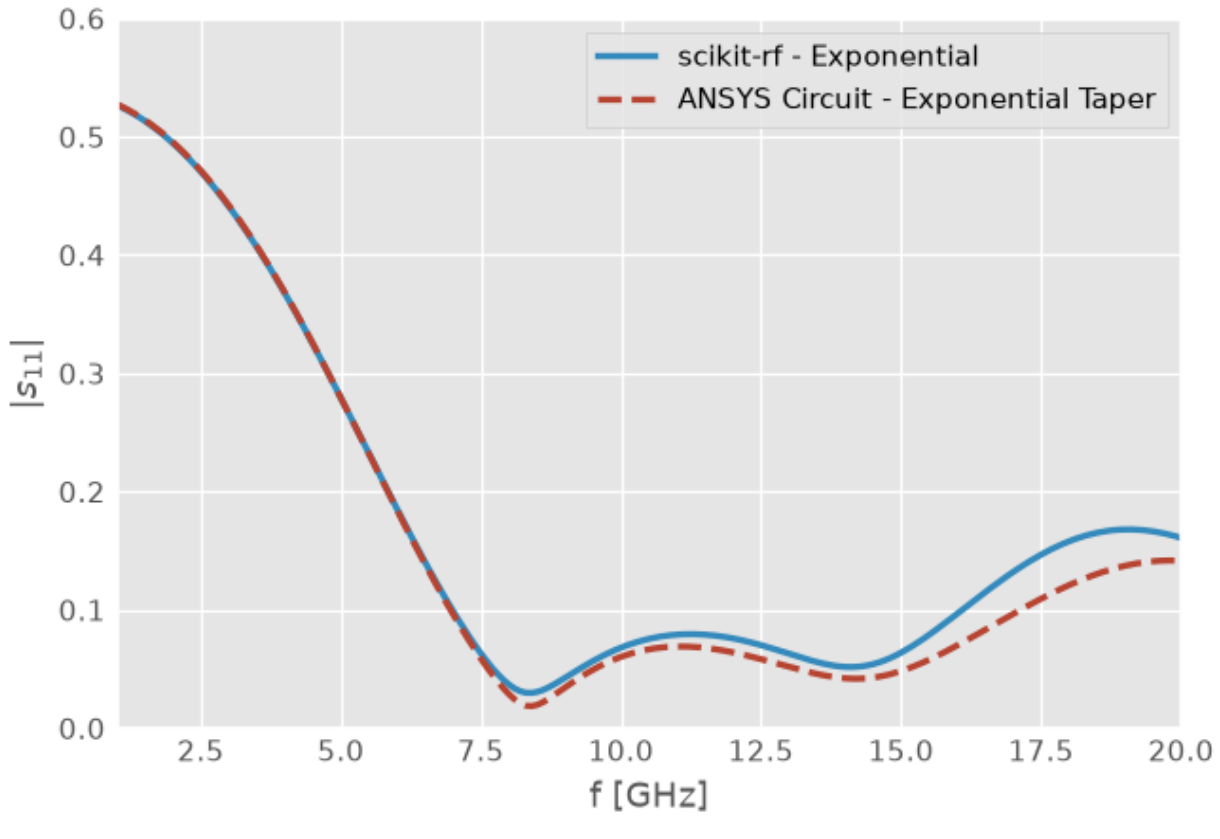


Exponential Taper

[6]:
```
taper_exp = rf.taper.Exponential(med=MLine, param='w', start=w1, stop=w2,
                         length=taper_length, n_sections=50,
                         med_kw={'frequency': freq, 'h': h, 't':t, 'ep_r': ep_r,
                                 'rough': rough, 'rho': rho}).network

ntwk_exp = line1 ** taper_exp ** line2 ** resistor ** microstrip_w2.short()
```

[7]:
```
fig, ax = plt.subplots()
ax.plot(ntwk_exp.frequency.f_scaled, ntwk_exp.s_mag[:,0], lw=2, label='scikit-rf -␣
↪Exponential')

f_ref, s_mag_ref = np.loadtxt('ANSYS_Circuit_taper_exponential_s_mag.csv', delimiter=',',
↪ skiprows=1, unpack=True)
ax.plot(f_ref, s_mag_ref, label='ANSYS Circuit - Exponential Taper', lw=2, ls='--')

ax.set_xlabel('f [GHz]')
ax.set_ylabel('$|s_{11}|$')
ax.set_ylim(0, 0.6)
ax.set_xlim(1, 20)
ax.legend()
```

[7]:
```
<matplotlib.legend.Legend at 0x7591af473710>
```



[ ]:

2.12 Instrument Control

2.12.1 VNA control

scikit-rf has a few virtual instrument classes, located in the `skrf.vi` module. This page demonstrates the basic control of a PNA

Connecting and querying instrument ID

```
from skrf.vi.vna import keysight

instr = keysight.PNA("TCPIP0::192.168.0.2::INSTR")
instr.id
```

Retrieving S-parameters as a scikit-rf Network

```
ntwk =  instr.get_snp_network()
```

[ ]:

2.13 Interactive

2.13.1 Interactive Matching System

This example illustrate a matching system made of an open stub in series with a delay load. The length of both transmission lines can be tuned, with a direct view of the resulting Smith chart.

[1]:
```
%matplotlib inline

import matplotlib.pyplot as plt
from ipywidgets import interact

import skrf as rf
from skrf.frequency import Frequency
from skrf.media import DistributedCircuit

rf.stylely()
# define a frequency object
freq = Frequency(0.01, 10, 201, 'GHz')

# create a Media object for RG-58, based on distributed ckt values
rg58 = DistributedCircuit(frequency = freq,
                          C =93.5e-12,#F/m
                          I =273e-9,  #H/m
                          R =0,#53e-3,   #Ohm/m
                          G =0,       #S/m
                          z0_port = 50
                          )
```

(continues on next page)

(continued from previous page)

```
load = rg58.load(.5)
def func(d1=180, d2=180):
    l1 = rg58.line(d=d1, unit='deg')
    l2 = rg58.shunt_delay_open(d=d2, unit='deg')
    ntwk = l2**l1**load
    ntwk.plot_s_smith(0,0)
    plt.draw()
    plt.show()

a=interact(func, d1= (0,360,10),d2= (0,360,10),embed=True);
```

```
interactive(children=(IntSlider(value=180, description='d1', max=360, step=10),␣
↪IntSlider(value=180, descripti...
```

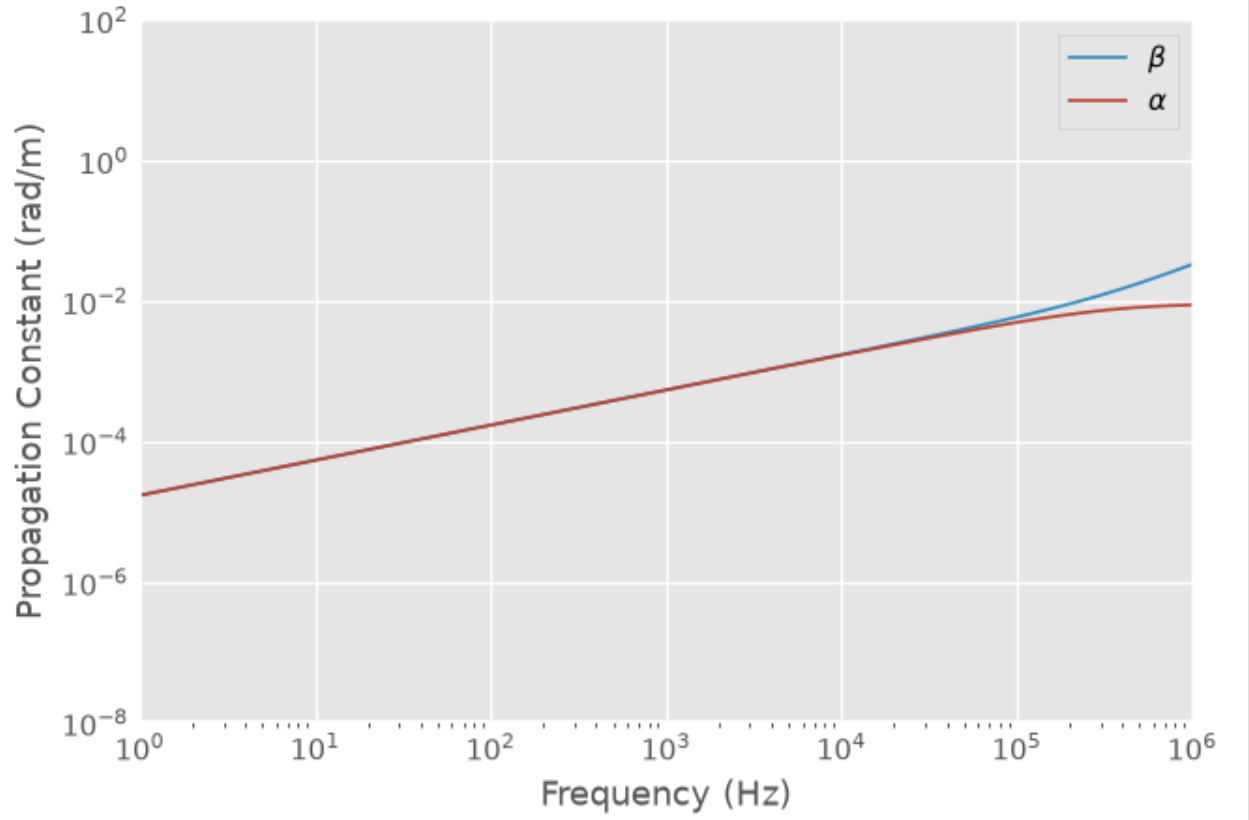
[ ]:

2.13.2 Transmission Line Properties

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

import skrf as rf

%matplotlib inline

from skrf.frequency import Frequency
from skrf.media import DistributedCircuit

rf.stylely()

# define a frequency object
freq = Frequency.from_f(np.logspace(0,6,101), unit='Hz')

def func(C=93,I=273, R=0, G=0):
    coax = DistributedCircuit(frequency = freq,
                              C =C*1e-12,#F/m
                              I =I*1e-9,  #H/m
                              R =10**(-R),#53e-3,   #Ohm/m
                              G =G,       #S/m
                              )
    freq.plot(coax.gamma.imag,label=r'$ \beta$')
    freq.plot(coax.gamma.real, label=r'$ \alpha$')
    plt.ylabel('Propagation Constant (rad/m)')
    plt.ylim(1e-8,1e2)
    plt.loglog()# impose log-log scaling
    plt.legend()
```

[2]:
```
from ipywidgets import interact

interact(func, C=(0,100,10), I=(100,400,10), R =(0,10),G=(0,10));
```

```
interactive(children=(IntSlider(value=93, description='C', step=10), IntSlider(value=273,
↪ description='I', max...
```



[ ]:

2.13.3 Interactive Mismatched Line

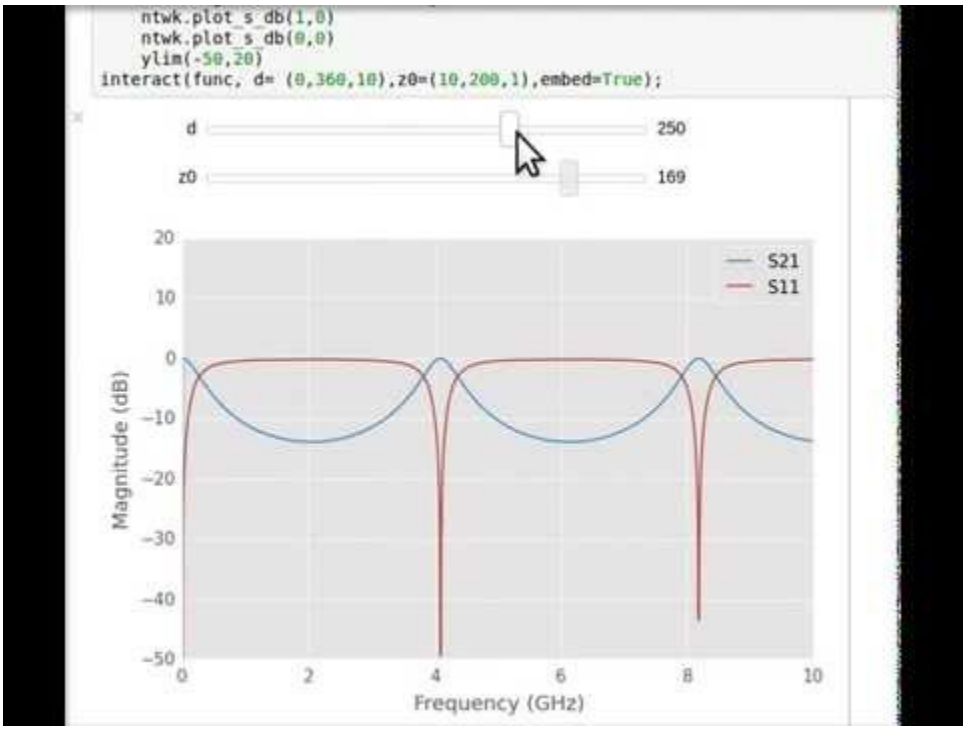
Intro

This is a demonstration of using IPythons `interact` function with some of `skrf`'s network creation methods.

As of IPython 2.0, the widgets in this notebook won't show up on http://nbviewer.ipython.org. To view the widgets and interact with them, you will need to download this notebook and run it with an IPython Notebook server.

[1]:
```
from IPython.display import YouTubeVideo

YouTubeVideo('JyYi_1SswXs',width=700, height=580)
```

[1]:



[2]:
```
import matplotlib.pyplot as plt
from ipywidgets import interact

%matplotlib inline

import skrf as rf
from skrf.frequency import Frequency
from skrf.media import DistributedCircuit

rf.stylely()


# define a frequency object
freq = Frequency(0,10,1001,'GHz')

# create a Media object for RG-58, based on distributed ckt values
rg58 = DistributedCircuit(frequency = freq,
                          C =93.5e-12,#F/m
                          L =273e-9,  #H/m
                          R =0,#53e-3,   #Ohm/m
                          G =0,       #S/m
                          )
```

Mismatched Line

This demonstrates the response of a mismatched transmission line of a given length. Specifically, a line of electrical length d with characteristic impedance z_0 is embedded in the coaxial environment (RG-58) with $z_0 \approx 54\Omega$. The resultant S-parameters are plotted in log mag and on the smith chart.

Log Mag

[3]:
```
def func(d=180,z0=54):
    ntwk =rg58.line(d=d,unit='deg',z0=z0)
    ntwk.renormalize(rg58.z0)
    ntwk.plot_s_db(1,0)
    ntwk.plot_s_db(0,0)
    plt.ylim(-50,20)
    plt.draw()
    plt.show()

interact(func, d= (0,360,10),z0=(10,200,1),embed=True);
```

```
interactive(children=(IntSlider(value=180, description='d', max=360, step=10),␣
↪IntSlider(value=54, description...
```

Smith Chart

[4]:
```

def func(d=180,z0=54):
    ntwk =rg58.line(d=d,unit='deg',z0=z0)
    ntwk.renormalize(rg58.z0)
    ntwk.plot_s_smith(1,0)
    ntwk.plot_s_smith(0,0)
    plt.draw()
    plt.show()


interact(func, d= (0,180,10),z0=(10,200,1),embed=True);
```

```
interactive(children=(IntSlider(value=180, description='d', max=180, step=10),␣
↪IntSlider(value=54, description...
```

[ ]:
```

```

CHAPTER

THREE

API

3.1 network (*skrf.network*)

Provide an n-port network class and associated functions.

Much of the functionality in this module is provided as methods and properties of the *Network* Class.

3.1.1 Network Class

Network([file, name, params, comments, ...])	An n-port electrical network.

skrf.network.Network

class skrf.network.**Network**(*file=None*, *name=None*, *params=None*, *comments=None*, *f_unit=None*, *s_def=None*, ***kwargs*)

An n-port electrical network.

For instructions on how to create Network see *__init__()*. An n-port network[1] may be defined by three quantities

- network parameter matrix (s, z, or y-matrix)
- port characteristic impedance matrix
- frequency information

The *Network* class stores these data structures internally in the form of complex numpy.ndarray's. These arrays are not interfaced directly but instead through the use of the properties:

Property	Meaning
s	Scattering parameter matrix.
z0	Characteristic impedance matrix.
f	Frequency vector.

Although these docs focus on s-parameters, other equivalent network representations such as *z* and *y* are available. Scalar projections of the complex network parameters are accessible through properties as well. These also return numpy.ndarray's.

[1] http://en.wikipedia.org/wiki/Two-port_network

Property	Meaning
s_re	Real part of the s-matrix.
s_im	Imaginary part of the s-matrix.
s_mag	Magnitude of the s-matrix.
s_db	Magnitude in log scale of the s-matrix.
s_deg	Phase of the s-matrix in degrees.

The following operations act on the networks s-matrix.

Operator	Function
+	Element-wise addition of the s-matrix.
-	Element-wise difference of the s-matrix.
*	Element-wise multiplication of the s-matrix.
/	Element-wise division of the s-matrix.
**	Cascading (only for 2-ports).
//	De-embedding (for 2-ports, see *inv*).

Different components of the *Network* can be visualized through various plotting methods. These methods can be used to plot individual elements of the s-matrix or all at once. For more info about plotting see the *Plotting* tutorial.

Method	Meaning
plot_s_smith()	Plot complex s-parameters on smith chart.
plot_s_re()	Plot real part of s-parameters vs frequency.
plot_s_im()	Plot imaginary part of s-parameters vs frequency.
plot_s_mag()	Plot magnitude of s-parameters vs frequency.
plot_s_db()	Plot magnitude (in dB) of s-parameters vs frequency.
plot_s_deg()	Plot phase of s-parameters (in degrees) vs frequency.
plot_s_deg_unwrap()	Plot phase of s-parameters (in unwrapped degrees) vs frequency.

Network objects can be created from a touchstone or pickle file (see *__init__()*), by a *Media* object, or manually by assigning the network properties directly. *Network* objects can be saved to disk in the form of touchstone files with the *write_touchstone()* method.

An exhaustive list of *Network* Methods and Properties (Attributes) are given below

References

Parameters

- **file** (*str*)
- **name** (*str*)
- **params** (*dict*)
- **comments** (*str*)
- **f_unit** (*FrequencyUnitT | None*)
- **s_def** (*SdefT | None*)

Attributes

COMPONENT_FUNC_DICT	Component functions like 're', 'im', 'mag', 'db', etc.
PRIMARY_PROPERTIES	Primary Network Properties list like 's', 'z', 'y', etc.
Y_LABEL_DICT	Y-axis labels to the plotting functions.
a	abcd parameter matrix.
a_arcl	The arcl component of the a-matrix.
a_arcl_unwrap	The arcl_unwrap component of the a-matrix.
a_db	The db component of the a-matrix.
a_db10	The db10 component of the a-matrix.
a_deg	The deg component of the a-matrix.
a_deg_unwrap	The deg_unwrap component of the a-matrix.
a_im	The im component of the a-matrix.
a_mag	The mag component of the a-matrix.
a_rad	The rad component of the a-matrix.
a_rad_unwrap	The rad_unwrap component of the a-matrix.
a_re	The re component of the a-matrix.
a_time	The time component of the a-matrix.
a_time_db	The time_db component of the a-matrix.
a_time_impulse	The time_impulse component of the a-matrix.
a_time_mag	The time_mag component of the a-matrix.
a_time_step	The time_step component of the a-matrix.
a_vswr	The vswr component of the a-matrix.
f	The frequency vector for the network, in Hz.
f_noise	The frequency vector for the noise of the network, in Hz.
frequency	Frequency information for the network.
g	Inverse hybrid parameter matrix.
g_arcl	The arcl component of the g-matrix.
g_arcl_unwrap	The arcl_unwrap component of the g-matrix.
g_db	The db component of the g-matrix.
g_db10	The db10 component of the g-matrix.
g_deg	The deg component of the g-matrix.
g_deg_unwrap	The deg_unwrap component of the g-matrix.
g_im	The im component of the g-matrix.
g_mag	The mag component of the g-matrix.
g_opt	The optimum source reflection coefficient to minimize noise.
g_rad	The rad component of the g-matrix.
g_rad_unwrap	The rad_unwrap component of the g-matrix.
g_re	The re component of the g-matrix.
g_time	The time component of the g-matrix.
g_time_db	The time_db component of the g-matrix.
g_time_impulse	The time_impulse component of the g-matrix.
g_time_mag	The time_mag component of the g-matrix.
g_time_step	The time_step component of the g-matrix.
g_vswr	The vswr component of the g-matrix.
group_delay	Group delay.
h	Hybrid parameter matrix.
h_arcl	The arcl component of the h-matrix.
h_arcl_unwrap	The arcl_unwrap component of the h-matrix.
h_db	The db component of the h-matrix.
h_db10	The db10 component of the h-matrix.

continues on next page

Table 2 – continued from previous page

h_deg	The deg component of the h-matrix.
h_deg_unwrap	The deg_unwrap component of the h-matrix.
h_im	The im component of the h-matrix.
h_mag	The mag component of the h-matrix.
h_rad	The rad component of the h-matrix.
h_rad_unwrap	The rad_unwrap component of the h-matrix.
h_re	The re component of the h-matrix.
h_time	The time component of the h-matrix.
h_time_db	The time_db component of the h-matrix.
h_time_impulse	The time_impulse component of the h-matrix.
h_time_mag	The time_mag component of the h-matrix.
h_time_step	The time_step component of the h-matrix.
h_vswr	The vswr component of the h-matrix.
inv	A *Network* object with 'inverse' s-parameters.
max_gain	Maximum available power gain for K > 1 and maximum stable power gain for K <= 1 (in linear).
max_stable_gain	Maximum stable power gain [1]_ (in linear).
n	The ABCD form of the noise correlation matrix for the network.
nfmin	The minimum noise figure for the network.
nfmin_db	The minimum noise figure for the network in dB.
noisy	Whether this network has noise.
nports	The number of ports the network has.
number_of_ports	The number of ports the network has.
passivity	Passivity metric for a network.
port_modes	Array of size nports with the mode of each port.
port_names	Names of the ports, or None if the ports are not named.
port_tuples	Returns a list of tuples, for each port index pair.
reciprocity	Reciprocity metric for a multi-port network.
reciprocity2	Reciprocity metric #2
rn	The equivalent noise resistance for the network.
s	Scattering parameter matrix.
s_arcl	The arcl component of the s-matrix.
s_arcl_unwrap	The arcl_unwrap component of the s-matrix.
s_db	The db component of the s-matrix.
s_db10	The db10 component of the s-matrix.
s_deg	The deg component of the s-matrix.
s_deg_unwrap	The deg_unwrap component of the s-matrix.
s_im	The im component of the s-matrix.
s_invert	Inverted scattering parameter matrix.
s_mag	The mag component of the s-matrix.
s_power	Scattering parameter matrix with s_def = 'power'.
s_pseudo	Scattering parameter matrix with s_def = 'pseudo'.
s_rad	The rad component of the s-matrix.
s_rad_unwrap	The rad_unwrap component of the s-matrix.
s_re	The re component of the s-matrix.
s_time	The time component of the s-matrix.
s_time_db	The time_db component of the s-matrix.
s_time_impulse	The time_impulse component of the s-matrix.
s_time_mag	The time_mag component of the s-matrix.
s_time_step	The time_step component of the s-matrix.
s_traveling	Scattering parameter matrix with s_def = 'traveling'.

continues on next page

Table 2 – continued from previous page

s_vswr	The vswr component of the s-matrix.
stability	Stability factor.
t	Scattering transfer parameter matrix.
t_arcl	The arcl component of the t-matrix.
t_arcl_unwrap	The arcl_unwrap component of the t-matrix.
t_db	The db component of the t-matrix.
t_db10	The db10 component of the t-matrix.
t_deg	The deg component of the t-matrix.
t_deg_unwrap	The deg_unwrap component of the t-matrix.
t_im	The im component of the t-matrix.
t_mag	The mag component of the t-matrix.
t_rad	The rad component of the t-matrix.
t_rad_unwrap	The rad_unwrap component of the t-matrix.
t_re	The re component of the t-matrix.
t_time	The time component of the t-matrix.
t_time_db	The time_db component of the t-matrix.
t_time_impulse	The time_impulse component of the t-matrix.
t_time_mag	The time_mag component of the t-matrix.
t_time_step	The time_step component of the t-matrix.
t_vswr	The vswr component of the t-matrix.
unilateral_gain	Mason's unilateral power gain [1]_ (in linear).
y	Admittance parameter matrix.
y_arcl	The arcl component of the y-matrix.
y_arcl_unwrap	The arcl_unwrap component of the y-matrix.
y_db	The db component of the y-matrix.
y_db10	The db10 component of the y-matrix.
y_deg	The deg component of the y-matrix.
y_deg_unwrap	The deg_unwrap component of the y-matrix.
y_im	The im component of the y-matrix.
y_mag	The mag component of the y-matrix.
y_opt	The optimum source admittance to minimize noise.
y_rad	The rad component of the y-matrix.
y_rad_unwrap	The rad_unwrap component of the y-matrix.
y_re	The re component of the y-matrix.
y_time	The time component of the y-matrix.
y_time_db	The time_db component of the y-matrix.
y_time_impulse	The time_impulse component of the y-matrix.
y_time_mag	The time_mag component of the y-matrix.
y_time_step	The time_step component of the y-matrix.
y_vswr	The vswr component of the y-matrix.
z	Impedance parameter matrix.
z0	Characteristic impedance[s] of the network ports.
z_arcl	The arcl component of the z-matrix.
z_arcl_unwrap	The arcl_unwrap component of the z-matrix.
z_db	The db component of the z-matrix.
z_db10	The db10 component of the z-matrix.
z_deg	The deg component of the z-matrix.
z_deg_unwrap	The deg_unwrap component of the z-matrix.
z_im	The im component of the z-matrix.
z_mag	The mag component of the z-matrix.
z_opt	The optimum source impedance to minimize noise.
z_rad	The rad component of the z-matrix.

continues on next page

Table 2 – continued from previous page

z_rad_unwrap	The rad_unwrap component of the z-matrix.
z_re	The re component of the z-matrix.
z_time	The time component of the z-matrix.
z_time_db	The time_db component of the z-matrix.
z_time_impulse	The time_impulse component of the z-matrix.
z_time_mag	The time_mag component of the z-matrix.
z_time_step	The time_step component of the z-matrix.
z_vswr	The vswr component of the z-matrix.

skrf.network.Network.COMPONENT_FUNC_DICT

Network.**COMPONENT_FUNC_DICT**: *dict*[ComponentFuncT, Callable | None] = {'arcl': <function Network.<lambda>>, 'arcl_unwrap': <function Network.<lambda>>, 'db': <function complex_2_db>, 'db10': <function complex_2_db10>, 'deg': <function Network.<lambda>>, 'deg_unwrap': <function Network.<lambda>>, 'im': <function imag>, 'mag': <ufunc 'absolute'>, 'rad': <function angle>, 'rad_unwrap': <function Network.<lambda>>, 're': <function real>, 'time': <function ifft>, 'time_db': <function Network.<lambda>>, 'time_impulse': None, 'time_mag': <function Network.<lambda>>, 'time_step': None, 'vswr': <function Network.<lambda>>}

Component functions like ‘re’, ‘im’, ‘mag’, ‘db’, etc.

skrf.network.Network.PRIMARY_PROPERTIES

Network.**PRIMARY_PROPERTIES**: tuple[PrimaryPropertiesT, ...] = ('s', 'z', 'y', 'a', 'g', 'h', 't')

Primary Network Properties list like ‘s’, ‘z’, ‘y’, etc.

skrf.network.Network.Y_LABEL_DICT

Network.**Y_LABEL_DICT**: *dict*[ComponentFuncT, str] = {'arcl': 'Arc Length', 'arcl_unwrap': 'Arc Length', 'db': 'Magnitude (dB)', 'db10': 'Magnitude (dB)', 'deg': 'Phase (deg)', 'deg_unwrap': 'Phase (deg)', 'im': 'Imag Part', 'mag': 'Magnitude', 'rad': 'Phase (rad)', 'rad_unwrap': 'Phase (rad)', 're': 'Real Part', 'time': 'Time (real)', 'time_db': 'Magnitude (dB)', 'time_impulse': 'Magnitude', 'time_mag': 'Magnitude', 'time_step': 'Magnitude', 'vswr': 'VSWR'}

Y-axis labels to the plotting functions.

skrf.network.Network.a

property Network.**a**: ndarray

abcd parameter matrix. Used to cascade two-ports.

The abcd-matrix[1] is a 3-dimensional numpy.ndarray which has shape *fxnxn*, where *f* is frequency axis and *n* is number of ports. Note that indexing starts at 0, so abcd11 can be accessed by taking the slice *abcd[:,0,0]*.

Returns
abcd – the Impedance parameter matrix.

Return type
complex numpy.ndarray of shape *fxnxn*

[1] http://en.wikipedia.org/wiki/impedance_parameters

See also

s, *y*, *z*, *t*, *a*, abcd

References

skrf.network.Network.a_arcl

property Network.**a_arcl**

The arcl component of the a-matrix.

See also

a

skrf.network.Network.a_arcl_unwrap

property Network.**a_arcl_unwrap**

The arcl_unwrap component of the a-matrix.

See also

a

skrf.network.Network.a_db

property Network.**a_db**

The db component of the a-matrix.

See also

a

skrf.network.Network.a_db10

property Network.**a_db10**

The db10 component of the a-matrix.

See also

a

skrf.network.Network.a_deg

property Network.**a_deg**

The deg component of the a-matrix.

See also

a

skrf.network.Network.a_deg_unwrap

property Network.**a_deg_unwrap**

The deg_unwrap component of the a-matrix.

See also

a

skrf.network.Network.a_im

property Network.**a_im**

The im component of the a-matrix.

See also

a

skrf.network.Network.a_mag

property Network.**a_mag**

The mag component of the a-matrix.

See also

a

skrf.network.Network.a_rad

property Network.**a_rad**

The rad component of the a-matrix.

See also

a

skrf.network.Network.a_rad_unwrap

property Network.**a_rad_unwrap**

The rad_unwrap component of the a-matrix.

See also

a

skrf.network.Network.a_re

property Network.**a_re**

The re component of the a-matrix.

See also

a

skrf.network.Network.a_time

property Network.**a_time**

The time component of the a-matrix.

See also

a

skrf.network.Network.a_time_db

property Network.**a_time_db**

The time_db component of the a-matrix.

See also

a

skrf.network.Network.a_time_impulse

property Network.**a_time_impulse**

The time_impulse component of the a-matrix.

See also

a

skrf.network.Network.a_time_mag

property Network.**a_time_mag**

The time_mag component of the a-matrix.

See also

a

skrf.network.Network.a_time_step

property Network.**a_time_step**

The time_step component of the a-matrix.

See also

a

skrf.network.Network.a_vswr

property Network.**a_vswr**

The vswr component of the a-matrix.

See also

a

skrf.network.Network.f

property Network.**f**: ndarray

The frequency vector for the network, in Hz.

Returns
f – frequency vector in Hz

Return type
numpy.ndarray

See also

frequency
frequency property that holds all frequency information

skrf.network.Network.f_noise

property Network.**f_noise**: *Frequency*

The frequency vector for the noise of the network, in Hz.

skrf.network.Network.frequency

property Network.**frequency**: *Frequency*

Frequency information for the network.

This property is a *Frequency* object. It holds the frequency vector, as well frequency unit, and provides other properties related to frequency information, such as start, stop, etc.

Returns
frequency – frequency information for the network.

Return type
Frequency object

See also

f
property holding frequency vector in Hz

change_frequency
updates frequency property, and interpolates s-parameters if needed

interpolate
interpolate function based on new frequency info

skrf.network.Network.g

property Network.**g**: ndarray

Inverse hybrid parameter matrix.

The g-matrix[1] is a 3-dimensional numpy.ndarray which has shape *fx2x2*, where *f* is frequency axis. Note that indexing starts at 0, so g11 can be accessed by taking the slice *g[:,0,0]*.

Returns
g – the inverse hybrid parameter matrix.

Return type
complex numpy.ndarray of shape *fx2x2*

See also

s, *y*, *z*, *t*, *a*, *h*

References

skrf.network.Network.g_arcl

property Network.**g_arcl**

The arcl component of the g-matrix.

See also

g

skrf.network.Network.g_arcl_unwrap

property Network.**g_arcl_unwrap**

The arcl_unwrap component of the g-matrix.

[1] https://en.wikipedia.org/wiki/Two-port_network#Inverse_hybrid_parameters_(g-parameters)

See also

g

skrf.network.Network.g_db

property Network.**g_db**

The db component of the g-matrix.

See also

g

skrf.network.Network.g_db10

property Network.**g_db10**

The db10 component of the g-matrix.

See also

g

skrf.network.Network.g_deg

property Network.**g_deg**

The deg component of the g-matrix.

See also

g

skrf.network.Network.g_deg_unwrap

property Network.**g_deg_unwrap**

The deg_unwrap component of the g-matrix.

See also

g

skrf.network.Network.g_im

property Network.**g_im**

The im component of the g-matrix.

See also

g

skrf.network.Network.g_mag

property Network.**g_mag**

The mag component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_opt

property Network.**g_opt: ndarray**

The optimum source reflection coefficient to minimize noise.

skrf.network.Network.g_rad

property Network.**g_rad**

The rad component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_rad_unwrap

property Network.**g_rad_unwrap**

The rad_unwrap component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_re

property Network.**g_re**

The re component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_time

property Network.**g_time**

The time component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_time_db

property Network.**g_time_db**

The time_db component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_time_impulse

property Network.**g_time_impulse**

The time_impulse component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_time_mag

property Network.**g_time_mag**

The time_mag component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_time_step

property Network.**g_time_step**

The time_step component of the g-matrix.

> **See also**
>
> *g*

skrf.network.Network.g_vswr

property Network.**g_vswr**

The vswr component of the g-matrix.

See also

g

skrf.network.Network.group_delay

property Network.**group_delay: ndarray**

Group delay.

Usually used as a measure of dispersion (or distortion).

Defined as the derivative of the unwrapped s-parameter phase (in rad) with respect to the frequency:

```
-d(self.s_rad_unwrap)/d(self.frequency.w)
```

Returns
gd

Return type
numpy.ndarray of shape *xnxn*

References

https://en.wikipedia.org/wiki/Group_delay_and_phase_delay

skrf.network.Network.h

property Network.**h: ndarray**

Hybrid parameter matrix.

The h-matrix[1] is a 3-dimensional numpy.ndarray which has shape *fx2x2*, where *f* is frequency axis. Note that indexing starts at 0, so h11 can be accessed by taking the slice *h[:,0,0]*.

Returns
h – the hybrid parameter matrix.

Return type
complex numpy.ndarray of shape *fx2x2*

See also

s, *y*, *z*, *t*, *a*, *h*

References

skrf.network.Network.h_arcl

property Network.**h_arcl**

The arcl component of the h-matrix.

See also

h

[1] http://en.wikipedia.org/wiki/Two-port_network#Hybrid_parameters_(h-parameters)

skrf.network.Network.h_arcl_unwrap

property Network.**h_arcl_unwrap**

The arcl_unwrap component of the h-matrix.

See also

h

skrf.network.Network.h_db

property Network.**h_db**

The db component of the h-matrix.

See also

h

skrf.network.Network.h_db10

property Network.**h_db10**

The db10 component of the h-matrix.

See also

h

skrf.network.Network.h_deg

property Network.**h_deg**

The deg component of the h-matrix.

See also

h

skrf.network.Network.h_deg_unwrap

property Network.**h_deg_unwrap**

The deg_unwrap component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_im

property Network.**h_im**

The im component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_mag

property Network.**h_mag**

The mag component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_rad

property Network.**h_rad**

The rad component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_rad_unwrap

property Network.**h_rad_unwrap**

The rad_unwrap component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_re

property Network.**h_re**

The re component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_time

property Network.**h_time**

The time component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_time_db

property Network.**h_time_db**

The time_db component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_time_impulse

property Network.**h_time_impulse**

The time_impulse component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_time_mag

property Network.**h_time_mag**

The time_mag component of the h-matrix.

> **See also**
>
> *h*

skrf.network.Network.h_time_step

property Network.**h_time_step**

The time_step component of the h-matrix.

See also

h

skrf.network.Network.h_vswr

property Network.**h_vswr**

The vswr component of the h-matrix.

See also

h

skrf.network.Network.inv

property Network.**inv: *Network***

A *Network* object with 'inverse' s-parameters.

This is used for de-embedding. It is defined such that the inverse of the s-matrix cascaded with itself is a unity scattering transfer parameter (T) matrix.

Returns

inv – a *Network* object with 'inverse' s-parameters.

Return type

a *Network* object

See also

inv

function which implements the inverse s-matrix

skrf.network.Network.max_gain

property Network.**max_gain: ndarray**

Maximum available power gain for K > 1 and maximum stable power gain for K <= 1 (in linear).

$$G_{max}|_{K>1} = \frac{|S_{21}|}{|S_{12}|} \times \frac{1}{K + \sqrt{K^2 - 1}}$$

$$G_{max}|_{K<=1} = \frac{|S_{21}|}{|S_{12}|}$$

Returns

gmax

Return type

numpy.ndarray of shape f

Note

The maximum available power gain is defined for a unconditionally stable network (K > 1). For K <= 1, this property returns the maximum stable gain instead. This behavior is similar to the max_gain() function in Keysight's Advanced Design System (but differs in decibel or linear)[1][2][3].

References

See also

max_stable_gain
: Maximum stable power gain

unilateral_gain
: Mason's unilateral power gain

stability
: Stability factor

skrf.network.Network.max_stable_gain

property Network.**max_stable_gain**: ndarray

Maximum stable power gain[1] (in linear).

$$G_{ms} = |S_{21}|/|S_{12}|$$

Returns
: **gms**

Return type
: numpy.ndarray of shape f

References

See also

max_gain
: Maximum available and stable power gain

unilateral_gain
: Mason's unilateral power gain

stability
: Stability factor

[1] M. S. Gupta, "Power gain in feedback amplifiers, a classic revisited," in IEEE Transactions on Microwave Theory and Techniques, vol. 40, no. 5, pp. 864-879, May 1992, doi: 10.1109/22.137392.

[2] https://www.microwaves101.com/encyclopedias/stability-factor

[3] https://edadocs.software.keysight.com/pages/viewpage.action?pageId=5920581

[1] M. S. Gupta, "Power gain in feedback amplifiers, a classic revisited," in IEEE Transactions on Microwave Theory and Techniques, vol. 40, no. 5, pp. 864-879, May 1992, doi: 10.1109/22.137392.

skrf.network.Network.n

property Network.**n**: ndarray

The ABCD form of the noise correlation matrix for the network.

skrf.network.Network.nfmin

property Network.**nfmin**: ndarray

The minimum noise figure for the network.

skrf.network.Network.nfmin_db

property Network.**nfmin_db**: ndarray

The minimum noise figure for the network in dB.

skrf.network.Network.noisy

property Network.**noisy**: bool

Whether this network has noise.

skrf.network.Network.nports

property Network.**nports**: int

The number of ports the network has.

Returns
number_of_ports – the number of ports the network has.

Return type
number

skrf.network.Network.number_of_ports

property Network.**number_of_ports**: int

The number of ports the network has.

Returns
number_of_ports – the number of ports the network has.

Return type
number

skrf.network.Network.passivity

property Network.**passivity**: ndarray

Passivity metric for a network.

This returns a matrix whose diagonals are equal to the total power received at all ports, normalized to the power at a single excitement port.

Mathematically, this is a test for unitary-ness of the s-parameter matrix[1].

For a one-port network this is

$$|S_{11}|$$

[1] http://en.wikipedia.org/wiki/Scattering_parameters#Lossless_networks

For a two-port network this is

$$(|S_{11}|^2 + |S_{21}|^2,\ |S_{22}|^2 + |S_{12}|^2)$$

in general it is

$$\sqrt{(S^H \cdot S)}$$

where H is conjugate transpose of S, and $\cdot$ is dot product.

Returns
passivity

Return type
numpy.ndarray of shape fxnxn

References

skrf.network.Network.port_modes

property Network.**port_modes: ndarray**

Array of size nports with the mode of each port.

This information is used to store mixed-modes networks in touchstone V2 format or to plot trace name with subscript like 'Sdd11'.

- 'C': common
- 'D': differential
- 'S': single-ended

Returns
port_modes – port modes

Return type
numpy.ndarray

skrf.network.Network.port_names

property Network.**port_names: list[str] | None**

Names of the ports, or None if the ports are not named.

Named ports can be indexed by name. The names are read from and written to touchstone files.

Length of the port_names list should match the number of ports in the network, this is checked on assignment. The names are dropped when an operation changes the number of ports without being able to tell what the new ports are.

Returns
port_names – names of the ports

Return type
list of str, or None

Examples

```
>>> ntwk = rf.Network(f=[1], s=np.zeros((1, 2, 2)), port_names=['in', 'out'])
>>> ntwk['in', 'out']  # same as ntwk.s21
```

skrf.network.Network.port_tuples

property Network.port_tuples: list[tuple[int, int]]

Returns a list of tuples, for each port index pair.

A convenience function for the common task for iterating over all s-parameters index pairs.

This just calls:

```
[(y,x) for x in range(self.nports) for y in range(self.nports)]
```

Returns
ports_ind – list of all port index tuples.

Return type
list of tuples

Examples

```
>>> ntwk = skrf.data.ring_slot
>>> for (idx_i, idx_j) in ntwk.port_tuples: print(idx_i, idx_j)
```

skrf.network.Network.reciprocity

property Network.reciprocity: ndarray

Reciprocity metric for a multi-port network.

This returns the difference between the s-parameter matrix and its transpose.

For two port this is

$$S - S^T$$

where T is transpose of S

Returns
reciprocity

Return type
numpy.ndarray of shape *fxnxn*

skrf.network.Network.reciprocity2

property Network.reciprocity2: ndarray

Reciprocity metric #2

$$abs(1 - S/S^T)$$

for the two port case, this evaluates to the distance of the determinant of the wave-cascading matrix from unity.

Returns
reciprocity

Return type
numpy.ndarray of shape *fxnxn*

skrf.network.Network.rn

property Network.**rn: ndarray**

The equivalent noise resistance for the network.

skrf.network.Network.s

property Network.**s: ndarray**

Scattering parameter matrix.

The s-matrix[1] is a 3-dimensional `numpy.ndarray` which has shape *fxnxn*, where *f* is frequency axis and *n* is number of ports. Note that indexing starts at 0, so s11 can be accessed by taking the slice s[:,0,0].

Returns
s – The scattering parameter matrix.

Return type
complex `numpy.ndarray` of shape *fxnxn*

See also

s, *y*, *z*, *t*, *a*

References

skrf.network.Network.s_arcl

property Network.**s_arcl**

The arcl component of the s-matrix.

See also

s

skrf.network.Network.s_arcl_unwrap

property Network.**s_arcl_unwrap**

The arcl_unwrap component of the s-matrix.

See also

s

skrf.network.Network.s_db

property Network.**s_db**

The db component of the s-matrix.

See also

s

[1] http://en.wikipedia.org/wiki/Scattering_parameters

skrf.network.Network.s_db10

property Network.**s_db10**

The db10 component of the s-matrix.

> **See also**
>
> *s*

skrf.network.Network.s_deg

property Network.**s_deg**

The deg component of the s-matrix.

> **See also**
>
> *s*

skrf.network.Network.s_deg_unwrap

property Network.**s_deg_unwrap**

The deg_unwrap component of the s-matrix.

> **See also**
>
> *s*

skrf.network.Network.s_im

property Network.**s_im**

The im component of the s-matrix.

> **See also**
>
> *s*

skrf.network.Network.s_invert

property Network.**s_invert: ndarray**

Inverted scattering parameter matrix.

Inverted scattering parameters are simply inverted s-parameters, defined as a = 1/s. Useful in analysis of active networks. The a-matrix is a 3-dimensional `numpy.ndarray` which has shape *fxnxn*, where *f* is frequency axis and *n* is number of ports. Note that indexing starts at 0, so a11 can be accessed by taking the slice a[:,0,0].

Returns
s_inv – the inverted scattering parameter matrix.

Return type
complex `numpy.ndarray` of shape *fxnxn*

See also

s, *y*, *z*, *t*, *a*

skrf.network.Network.s_mag

property Network.**s_mag**

The mag component of the s-matrix.

See also

s

skrf.network.Network.s_power

property Network.**s_power: ndarray**

Scattering parameter matrix with s_def = 'power'.

Returns

s – The scattering parameter[1] matrix.

Return type

complex numpy.ndarray of shape *fxnxn*

See also

s, *s_power*, *s_pseudo*, *s_traveling*

References

skrf.network.Network.s_pseudo

property Network.**s_pseudo: ndarray**

Scattering parameter matrix with s_def = 'pseudo'.

Returns

s – The scattering parameter[1] matrix.

Return type

complex numpy.ndarray of shape *fxnxn*

See also

s, *s_power*, *s_pseudo*, *s_traveling*

References

skrf.network.Network.s_rad

[1] http://en.wikipedia.org/wiki/Scattering_parameters

[1] http://en.wikipedia.org/wiki/Scattering_parameters

property Network.**s_rad**

The rad component of the s-matrix.

See also

s

skrf.network.Network.s_rad_unwrap

property Network.**s_rad_unwrap**

The rad_unwrap component of the s-matrix.

See also

s

skrf.network.Network.s_re

property Network.**s_re**

The re component of the s-matrix.

See also

s

skrf.network.Network.s_time

property Network.**s_time**

The time component of the s-matrix.

See also

s

skrf.network.Network.s_time_db

property Network.**s_time_db**

The time_db component of the s-matrix.

See also

s

skrf.network.Network.s_time_impulse

property Network.**s_time_impulse**

The time_impulse component of the s-matrix.

See also

s

skrf.network.Network.s_time_mag

property Network.**s_time_mag**

The time_mag component of the s-matrix.

See also

s

skrf.network.Network.s_time_step

property Network.**s_time_step**

The time_step component of the s-matrix.

See also

s

skrf.network.Network.s_traveling

property Network.**s_traveling: ndarray**

Scattering parameter matrix with s_def = 'traveling'.

Returns

s – The scattering parameter[1] matrix.

Return type

complex numpy.ndarray of shape *fxnxn*

See also

s, *s_power*, *s_pseudo*, *s_traveling*

References

skrf.network.Network.s_vswr

property Network.**s_vswr**

The vswr component of the s-matrix.

See also

s

[1] http://en.wikipedia.org/wiki/Scattering_parameters

skrf.network.Network.stability

property Network.**stability: ndarray**

Stability factor.

$$K = (1 - |S_{11}|^2 - |S_{22}|^2 + |D|^2)/(2 * |S_{12}| * |S_{21}|)$$
$$with$$
$$D = S_{11}S_{22} - S_{12}S_{21}$$

Returns
K

Return type
numpy.ndarray of shape *f*

See also

stability_circle

skrf.network.Network.t

property Network.**t: ndarray**

Scattering transfer parameter matrix.

The t-matrix[1] is a 3-dimensional numpy.ndarray which has shape *fx2x2*, where *f* is frequency axis. Note that indexing starts at 0, so t11 can be accessed by taking the slice *t[:,0,0]*.

The t-matrix, also known as the wave cascading matrix, is only defined for a 2-port Network.

Returns
t – t-parameters, aka scattering transfer parameters

Return type
complex np.ndarray of shape *fx2x2*

See also

s, *y*, *z*, *t*, *a*

References

skrf.network.Network.t_arcl

property Network.**t_arcl**

The arcl component of the t-matrix.

See also

t

[1] http://en.wikipedia.org/wiki/Scattering_parameters#Scattering_transfer_parameters

skrf.network.Network.t_arcl_unwrap

property Network.**t_arcl_unwrap**

The arcl_unwrap component of the t-matrix.

See also

t

skrf.network.Network.t_db

property Network.**t_db**

The db component of the t-matrix.

See also

t

skrf.network.Network.t_db10

property Network.**t_db10**

The db10 component of the t-matrix.

See also

t

skrf.network.Network.t_deg

property Network.**t_deg**

The deg component of the t-matrix.

See also

t

skrf.network.Network.t_deg_unwrap

property Network.**t_deg_unwrap**

The deg_unwrap component of the t-matrix.

See also

t

skrf.network.Network.t_im

property Network.**t_im**

The im component of the t-matrix.

See also

t

skrf.network.Network.t_mag

property Network.**t_mag**

The mag component of the t-matrix.

See also

t

skrf.network.Network.t_rad

property Network.**t_rad**

The rad component of the t-matrix.

See also

t

skrf.network.Network.t_rad_unwrap

property Network.**t_rad_unwrap**

The rad_unwrap component of the t-matrix.

See also

t

skrf.network.Network.t_re

property Network.**t_re**

The re component of the t-matrix.

See also

t

skrf.network.Network.t_time

property Network.**t_time**

The time component of the t-matrix.

See also

t

skrf.network.Network.t_time_db

property Network.**t_time_db**

The time_db component of the t-matrix.

See also

t

skrf.network.Network.t_time_impulse

property Network.**t_time_impulse**

The time_impulse component of the t-matrix.

See also

t

skrf.network.Network.t_time_mag

property Network.**t_time_mag**

The time_mag component of the t-matrix.

See also

t

skrf.network.Network.t_time_step

property Network.**t_time_step**

The time_step component of the t-matrix.

See also

t

skrf.network.Network.t_vswr

property Network.**t_vswr**

The vswr component of the t-matrix.

See also

t

skrf.network.Network.unilateral_gain

property Network.**unilateral_gain: ndarray**

Mason's unilateral power gain[1] (in linear).

$$U = \frac{|\frac{S_{21}}{S_{12}} - 1|^2}{2K\frac{|S_{21}|}{|S_{12}|} - 2Re(\frac{S_{21}}{S_{12}})}$$

Returns
U

Return type
numpy.ndarray of shape *f*

References

See also

max_stable_gain
Maximum stable power gain

max_gain
Maximum available and stable power gain

stability
Stability factor

skrf.network.Network.y

property Network.**y: ndarray**

Admittance parameter matrix.

The y-matrix[1] is a 3-dimensional numpy.ndarray which has shape *fxnxn*, where *f* is frequency axis and *n* is number of ports. Note that indexing starts at 0, so y11 can be accessed by taking the slice *y[:,0,0]*.

Returns
y – the admittance parameter matrix.

[1] M. S. Gupta, "Power gain in feedback amplifiers, a classic revisited," in IEEE Transactions on Microwave Theory and Techniques, vol. 40, no. 5, pp. 864-879, May 1992, doi: 10.1109/22.137392.
[1] http://en.wikipedia.org/wiki/Admittance_parameters

Return type
complex `numpy.ndarray` of shape *fxnxn*

See also

s, *y*, *z*, *t*, *a*

References

skrf.network.Network.y_arcl

property `Network.`**y_arcl**

The arcl component of the y-matrix.

See also

y

skrf.network.Network.y_arcl_unwrap

property `Network.`**y_arcl_unwrap**

The arcl_unwrap component of the y-matrix.

See also

y

skrf.network.Network.y_db

property `Network.`**y_db**

The db component of the y-matrix.

See also

y

skrf.network.Network.y_db10

property `Network.`**y_db10**

The db10 component of the y-matrix.

See also

y

skrf.network.Network.y_deg

property Network.**y_deg**

The deg component of the y-matrix.

See also

y

skrf.network.Network.y_deg_unwrap

property Network.**y_deg_unwrap**

The deg_unwrap component of the y-matrix.

See also

y

skrf.network.Network.y_im

property Network.**y_im**

The im component of the y-matrix.

See also

y

skrf.network.Network.y_mag

property Network.**y_mag**

The mag component of the y-matrix.

See also

y

skrf.network.Network.y_opt

property Network.**y_opt**: ndarray

The optimum source admittance to minimize noise.

skrf.network.Network.y_rad

property Network.**y_rad**

The rad component of the y-matrix.

See also

y

skrf.network.Network.y_rad_unwrap

property Network.**y_rad_unwrap**

The rad_unwrap component of the y-matrix.

See also

y

skrf.network.Network.y_re

property Network.**y_re**

The re component of the y-matrix.

See also

y

skrf.network.Network.y_time

property Network.**y_time**

The time component of the y-matrix.

See also

y

skrf.network.Network.y_time_db

property Network.**y_time_db**

The time_db component of the y-matrix.

See also

y

skrf.network.Network.y_time_impulse

property Network.**y_time_impulse**

The time_impulse component of the y-matrix.

See also

y

skrf.network.Network.y_time_mag

property Network.**y_time_mag**

The time_mag component of the y-matrix.

See also

y

skrf.network.Network.y_time_step

property Network.**y_time_step**

The time_step component of the y-matrix.

See also

y

skrf.network.Network.y_vswr

property Network.**y_vswr**

The vswr component of the y-matrix.

See also

y

skrf.network.Network.z

property Network.**z**: ndarray

Impedance parameter matrix.

The z-matrix[1] is a 3-dimensional numpy.ndarray which has shape *fxnxn*, where *f* is frequency axis and *n* is number of ports. Note that indexing starts at 0, so z11 can be accessed by taking the slice *z[:,0,0]*.

[1] http://en.wikipedia.org/wiki/impedance_parameters

Returns
z – the Impedance parameter matrix.

Return type
complex `numpy.ndarray` of shape *fxnxn*

See also

s, *y*, *z*, *t*, *a*

References

skrf.network.Network.z0

property Network.**z0: ndarray**

Characteristic impedance[s] of the network ports.

This property stores the characteristic impedance of each port of the network. Because it is possible that each port has a different characteristic impedance each varying with frequency, *z0* is stored internally as a *fxn* array.

However because *z0* is frequently simple (like 50ohm), it can be set with just number as well.

Returns
z0 – characteristic impedance for network

Return type
`numpy.ndarray` of shape fxn

skrf.network.Network.z_arcl

property Network.**z_arcl**

The arcl component of the z-matrix.

See also

z

skrf.network.Network.z_arcl_unwrap

property Network.**z_arcl_unwrap**

The arcl_unwrap component of the z-matrix.

See also

z

skrf.network.Network.z_db

property Network.**z_db**

The db component of the z-matrix.

See also

z

skrf.network.Network.z_db10

property Network.**z_db10**

The db10 component of the z-matrix.

See also

z

skrf.network.Network.z_deg

property Network.**z_deg**

The deg component of the z-matrix.

See also

z

skrf.network.Network.z_deg_unwrap

property Network.**z_deg_unwrap**

The deg_unwrap component of the z-matrix.

See also

z

skrf.network.Network.z_im

property Network.**z_im**

The im component of the z-matrix.

See also

z

skrf.network.Network.z_mag

property Network.**z_mag**

The mag component of the z-matrix.

See also

z

skrf.network.Network.z_opt

property Network.**z_opt: ndarray**

The optimum source impedance to minimize noise.

skrf.network.Network.z_rad

property Network.**z_rad**

The rad component of the z-matrix.

See also

z

skrf.network.Network.z_rad_unwrap

property Network.**z_rad_unwrap**

The rad_unwrap component of the z-matrix.

See also

z

skrf.network.Network.z_re

property Network.**z_re**

The re component of the z-matrix.

See also

z

skrf.network.Network.z_time

property Network.**z_time**

The time component of the z-matrix.

See also

z

skrf.network.Network.z_time_db

property Network.**z_time_db**

The time_db component of the z-matrix.

See also

z

skrf.network.Network.z_time_impulse

property Network.**z_time_impulse**

The time_impulse component of the z-matrix.

See also

z

skrf.network.Network.z_time_mag

property Network.**z_time_mag**

The time_mag component of the z-matrix.

See also

z

skrf.network.Network.z_time_step

property Network.**z_time_step**

The time_step component of the z-matrix.

See also

z

skrf.network.Network.z_vswr

property Network.**z_vswr**

The vswr component of the z-matrix.

See also

z

Methods

__init__	Network constructor.
add_noise_polar	Adds a complex zero-mean gaussian white-noise.
add_noise_polar_flatband	Add a flatband complex zero-mean gaussian white-noise signal of given standard deviations for magnitude and phase.
attribute	
copy	Return a copy of this Network.
copy_from	Copy the contents of another Network into self.
copy_subset	Return a copy of a frequency subset of this Network.
crop	Crop Network based on start and stop frequencies.
cropped	Returns a cropped network, leaves self alone.

continues on next page

Table 3 – continued from previous page

delay	Add phase delay to a given port.
drop_non_monotonic_increasing	Drop invalid values based on duplicate and non increasing frequency values.
extrapolate_to_dc	Extrapolate S-parameters down to 0 Hz and interpolate to uniform spacing.
flip	Swap the ports of a 2n-port Network (inplace).
flipped	Returns a flipped network, leaves self alone.
from_string	Create a Network object from a text string
from_z	Create a Network from its Z-parameters.
func_on_parameter	Apply a function parameter matrix, one frequency slice at a time.
gain_circle	Returns loci of gain circles for a given port (0 or 1) and a specified gain.
gmm2se	Transform network from generalized mixed mode parameters [#]_ to single ended parameters.
impulse_response	Calculate time-domain impulse response of one-port.
interpolate	Interpolate a Network along frequency axis.
interpolate_self	Interpolate the current Network along frequency axis (inplace).
is_lossless	Test for losslessness.
is_passive	Test for passivity.
is_reciprocal	Test for reciprocity.
is_symmetric	Return whether the 2N-port network has n-th order reflection symmetry by checking.
multiply_noise	Multiply a complex bivariate gaussian white-noise signal of given standard deviations for magnitude and phase.
nf	The noise figure for the network if the source impedance is z.
nf_circle	Returns loci of noise figure circles for a specified noise figure.
nfdb_gs	Return dB(NF) foreach gamma_source x noise_frequency.
nonreciprocity	Normalized non-reciprocity metric.
nudge	Perturb s-parameters by small amount.
plot	Plot something vs frequency
plot_a_arcl	plot the Network attribute `a_arcl` component vs frequency.
plot_a_arcl_unwrap	plot the Network attribute `a_arcl_unwrap` component vs frequency.
plot_a_complex	plot the Network attribute `{}` vs frequency.
plot_a_db	plot the Network attribute `a_db` component vs frequency.
plot_a_db10	plot the Network attribute `a_db10` component vs frequency.
plot_a_deg	plot the Network attribute `a_deg` component vs frequency.
plot_a_deg_unwrap	plot the Network attribute `a_deg_unwrap` component vs frequency.
plot_a_im	plot the Network attribute `a_im` component vs frequency.

continues on next page

Table 3 – continued from previous page

plot_a_mag	plot the Network attribute a_mag component vs frequency.
plot_a_polar	plot the Network attribute {} vs frequency.
plot_a_rad	plot the Network attribute a_rad component vs frequency.
plot_a_rad_unwrap	plot the Network attribute a_rad_unwrap component vs frequency.
plot_a_re	plot the Network attribute a_re component vs frequency.
plot_a_time	plot the Network attribute a_time component vs time.
plot_a_time_db	plot the Network attribute a_time_db component vs time.
plot_a_time_impulse	plot the Network attribute a_time_impulse component vs time.
plot_a_time_mag	plot the Network attribute a_time_mag component vs time.
plot_a_time_step	plot the Network attribute a_time_step component vs time.
plot_a_vswr	plot the Network attribute a_vswr component vs frequency.
plot_attribute	plot the Network attribute conversion_attribute component vs frequency or time.
plot_g_arcl	plot the Network attribute g_arcl component vs frequency.
plot_g_arcl_unwrap	plot the Network attribute g_arcl_unwrap component vs frequency.
plot_g_complex	plot the Network attribute {} vs frequency.
plot_g_db	plot the Network attribute g_db component vs frequency.
plot_g_db10	plot the Network attribute g_db10 component vs frequency.
plot_g_deg	plot the Network attribute g_deg component vs frequency.
plot_g_deg_unwrap	plot the Network attribute g_deg_unwrap component vs frequency.
plot_g_im	plot the Network attribute g_im component vs frequency.
plot_g_mag	plot the Network attribute g_mag component vs frequency.
plot_g_polar	plot the Network attribute {} vs frequency.
plot_g_rad	plot the Network attribute g_rad component vs frequency.
plot_g_rad_unwrap	plot the Network attribute g_rad_unwrap component vs frequency.
plot_g_re	plot the Network attribute g_re component vs frequency.
plot_g_time	plot the Network attribute g_time component vs time.
plot_g_time_db	plot the Network attribute g_time_db component vs time.
plot_g_time_impulse	plot the Network attribute g_time_impulse component vs time.

continues on next page

Table 3 – continued from previous page

plot_g_time_mag	plot the Network attribute g_time_mag component vs time.
plot_g_time_step	plot the Network attribute g_time_step component vs time.
plot_g_vswr	plot the Network attribute g_vswr component vs frequency.
plot_h_arcl	plot the Network attribute h_arcl component vs frequency.
plot_h_arcl_unwrap	plot the Network attribute h_arcl_unwrap component vs frequency.
plot_h_complex	plot the Network attribute {} vs frequency.
plot_h_db	plot the Network attribute h_db component vs frequency.
plot_h_db10	plot the Network attribute h_db10 component vs frequency.
plot_h_deg	plot the Network attribute h_deg component vs frequency.
plot_h_deg_unwrap	plot the Network attribute h_deg_unwrap component vs frequency.
plot_h_im	plot the Network attribute h_im component vs frequency.
plot_h_mag	plot the Network attribute h_mag component vs frequency.
plot_h_polar	plot the Network attribute {} vs frequency.
plot_h_rad	plot the Network attribute h_rad component vs frequency.
plot_h_rad_unwrap	plot the Network attribute h_rad_unwrap component vs frequency.
plot_h_re	plot the Network attribute h_re component vs frequency.
plot_h_time	plot the Network attribute h_time component vs time.
plot_h_time_db	plot the Network attribute h_time_db component vs time.
plot_h_time_impulse	plot the Network attribute h_time_impulse component vs time.
plot_h_time_mag	plot the Network attribute h_time_mag component vs time.
plot_h_time_step	plot the Network attribute h_time_step component vs time.
plot_h_vswr	plot the Network attribute h_vswr component vs frequency.
plot_it_all	Plot dB, deg, smith, and complex in subplots.
plot_passivity	Plot dB(diag(passivity metric)) vs frequency.
plot_prop_complex	plot the Network attribute {} vs frequency.
plot_prop_polar	plot the Network attribute {} vs frequency.
plot_reciprocity	Plot reciprocity metric.
plot_reciprocity2	Plot reciprocity metric #2.
plot_s_arcl	plot the Network attribute s_arcl component vs frequency.
plot_s_arcl_unwrap	plot the Network attribute s_arcl_unwrap component vs frequency.
plot_s_complex	plot the Network attribute {} vs frequency.

continues on next page

Table 3 – continued from previous page

plot_s_db	plot the Network attribute s_db component vs frequency.
plot_s_db10	plot the Network attribute s_db10 component vs frequency.
plot_s_db_time	
plot_s_deg	plot the Network attribute s_deg component vs frequency.
plot_s_deg_unwrap	plot the Network attribute s_deg_unwrap component vs frequency.
plot_s_error	Plot the s-parameter error between two networks vs frequency.
plot_s_im	plot the Network attribute s_im component vs frequency.
plot_s_mag	plot the Network attribute s_mag component vs frequency.
plot_s_polar	plot the Network attribute {} vs frequency.
plot_s_rad	plot the Network attribute s_rad component vs frequency.
plot_s_rad_unwrap	plot the Network attribute s_rad_unwrap component vs frequency.
plot_s_re	plot the Network attribute s_re component vs frequency.
plot_s_smith	Plots the scattering parameter on a smith chart.
plot_s_time	plot the Network attribute s_time component vs time.
plot_s_time_db	plot the Network attribute s_time_db component vs time.
plot_s_time_impulse	plot the Network attribute s_time_impulse component vs time.
plot_s_time_mag	plot the Network attribute s_time_mag component vs time.
plot_s_time_step	plot the Network attribute s_time_step component vs time.
plot_s_vswr	plot the Network attribute s_vswr component vs frequency.
plot_t_arcl	plot the Network attribute t_arcl component vs frequency.
plot_t_arcl_unwrap	plot the Network attribute t_arcl_unwrap component vs frequency.
plot_t_complex	plot the Network attribute {} vs frequency.
plot_t_db	plot the Network attribute t_db component vs frequency.
plot_t_db10	plot the Network attribute t_db10 component vs frequency.
plot_t_deg	plot the Network attribute t_deg component vs frequency.
plot_t_deg_unwrap	plot the Network attribute t_deg_unwrap component vs frequency.
plot_t_im	plot the Network attribute t_im component vs frequency.
plot_t_mag	plot the Network attribute t_mag component vs frequency.
plot_t_polar	plot the Network attribute {} vs frequency.

continues on next page

Table 3 – continued from previous page

plot_t_rad	plot the Network attribute `t_rad` component vs frequency.
plot_t_rad_unwrap	plot the Network attribute `t_rad_unwrap` component vs frequency.
plot_t_re	plot the Network attribute `t_re` component vs frequency.
plot_t_time	plot the Network attribute `t_time` component vs time.
plot_t_time_db	plot the Network attribute `t_time_db` component vs time.
plot_t_time_impulse	plot the Network attribute `t_time_impulse` component vs time.
plot_t_time_mag	plot the Network attribute `t_time_mag` component vs time.
plot_t_time_step	plot the Network attribute `t_time_step` component vs time.
plot_t_vswr	plot the Network attribute `t_vswr` component vs frequency.
plot_y_arcl	plot the Network attribute `y_arcl` component vs frequency.
plot_y_arcl_unwrap	plot the Network attribute `y_arcl_unwrap` component vs frequency.
plot_y_complex	plot the Network attribute `{}` vs frequency.
plot_y_db	plot the Network attribute `y_db` component vs frequency.
plot_y_db10	plot the Network attribute `y_db10` component vs frequency.
plot_y_deg	plot the Network attribute `y_deg` component vs frequency.
plot_y_deg_unwrap	plot the Network attribute `y_deg_unwrap` component vs frequency.
plot_y_im	plot the Network attribute `y_im` component vs frequency.
plot_y_mag	plot the Network attribute `y_mag` component vs frequency.
plot_y_polar	plot the Network attribute `{}` vs frequency.
plot_y_rad	plot the Network attribute `y_rad` component vs frequency.
plot_y_rad_unwrap	plot the Network attribute `y_rad_unwrap` component vs frequency.
plot_y_re	plot the Network attribute `y_re` component vs frequency.
plot_y_time	plot the Network attribute `y_time` component vs time.
plot_y_time_db	plot the Network attribute `y_time_db` component vs time.
plot_y_time_impulse	plot the Network attribute `y_time_impulse` component vs time.
plot_y_time_mag	plot the Network attribute `y_time_mag` component vs time.
plot_y_time_step	plot the Network attribute `y_time_step` component vs time.
plot_y_vswr	plot the Network attribute `y_vswr` component vs frequency.

continues on next page

Table 3 – continued from previous page

plot_z_arcl	plot the Network attribute `z_arcl` component vs frequency.
plot_z_arcl_unwrap	plot the Network attribute `z_arcl_unwrap` component vs frequency.
plot_z_complex	plot the Network attribute `{}` vs frequency.
plot_z_db	plot the Network attribute `z_db` component vs frequency.
plot_z_db10	plot the Network attribute `z_db10` component vs frequency.
plot_z_deg	plot the Network attribute `z_deg` component vs frequency.
plot_z_deg_unwrap	plot the Network attribute `z_deg_unwrap` component vs frequency.
plot_z_im	plot the Network attribute `z_im` component vs frequency.
plot_z_mag	plot the Network attribute `z_mag` component vs frequency.
plot_z_polar	plot the Network attribute `{}` vs frequency.
plot_z_rad	plot the Network attribute `z_rad` component vs frequency.
plot_z_rad_unwrap	plot the Network attribute `z_rad_unwrap` component vs frequency.
plot_z_re	plot the Network attribute `z_re` component vs frequency.
plot_z_time	plot the Network attribute `z_time` component vs time.
plot_z_time_db	plot the Network attribute `z_time_db` component vs time.
plot_z_time_impulse	plot the Network attribute `z_time_impulse` component vs time.
plot_z_time_mag	plot the Network attribute `z_time_mag` component vs time.
plot_z_time_step	plot the Network attribute `z_time_step` component vs time.
plot_z_vswr	plot the Network attribute `z_vswr` component vs frequency.
read	Read a Network from a 'ntwk' file.
read_touchstone	Load values from a touchstone file.
renormalize	Renormalize s-parameter matrix given a new port impedances.
renumber	Renumber ports of a Network (inplace).
renumbered	Return a renumbered Network, leave self alone.
resample	Interpolate the current Network along frequency axis (inplace).
rotate	Rotate S-parameters
s_active	Returns the active s-parameters of the network for a defined wave excitation a.
s_error	Compute the error between s-parameters of this network and another network *ntwk*.
se2gmm	Transform network from single ended parameters to generalized mixed mode parameters [#]_.

continues on next page

Table 3 – continued from previous page

set_noise_a	Set the "A" (ie cascade) representation of the correlation matrix, based on the noise frequency and input parameters.
stability_circle	Returns loci of stability circles for a given port (0 or 1).
step_response	Calculate time-domain step response of one-port.
subnetwork	Return a subnetwork of a the Network from a list of port numbers.
time_gate	Time gate this Network.
to_dataframe	Convert attributes of a Network to a pandas DataFrame.
vswr_active	Return the active VSWR of the network for a defined wave excitation a.
windowed	Return a windowed version of s-matrix.
write	Write the Network to disk using the `pickle` module.
write_spreadsheet	Write contents of network to a spreadsheet, for your boss to use.
write_to_json_string	Serialize and convert network to a JSON string.
write_touchstone	Write a contents of the *Network* to a touchstone file.
y_active	Return the active Y-parameters of the network for a defined wave excitation a.
z_active	Return the active Z-parameters of the network for a defined wave excitation a.
zipped_touchstone	Read a Network from a Touchstone file in a ziparchive.

skrf.network.Network.__init__

Network.**__init__**(*file=None*, *name=None*, *params=None*, *comments=None*, *f_unit=None*, *s_def=None*, ***kwargs*)

Network constructor.

Creates an n-port microwave network from a *file* or directly from data. If no file or data is given, then an empty Network is created.

Parameters

- **file** (*str, Path, or file-object*) –

  file to load information from. supported formats are:

  - touchstone file (.s?p) (or .ts)
  - io.StringIO object (with *.name* property which contains the file extension, such as *myfile.s4p*)
  - pickled Network (.ntwk, .p) see *write()*

- **name** (*str, optional*) – Name of this Network. if None will try to use file, if it is a str
- **params** (*dict, optional*) – Dictionary of parameters associated with the Network
- **comments** (*str, optional*) – Comments associated with the Network
- **f_unit** (*str, optional*) – Frequency unit for the resulting network, overriding *FREQ_UNIT_DEFAULT*. For raw frequency values passed as *f*, this also specifies the input unit (Hz if omitted). File values are always read using their file unit.
- **s_def** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition : 'power' for power-waves definition, 'pseudo' for pseudo-waves defi-

nition. 'traveling' corresponds to the initial implementation. Default is 'power'. NB: results are the same for real-valued characteristic impedances.

- ****kwargs** – key word arguments can be used to assign properties of the Network, such as *s*, *f* and *z0*. keyword *port_names* can be used to name the ports, list with one

  name per port.

  keyword *encoding* can be used to define the Touchstone file encoding. keyword *noise_interp_kind* used to change the default interpolation

  method for noisy networks. Options are 'linear', 'nearest', 'nearest-up', 'zero', 'slinear', 'quadratic', 'cubic', 'previous', or 'next'. Review *scipy.interpolate.interp_1d* for details on each interpolation style. Defaults to 'linear'.

  keyword *noise_fill_value* used to change the default interpolation
  fill value for noisy networks. Defaults to np.nan.

Return type
None

Examples

From a touchstone

```
>>> n = rf.Network('ntwk1.s2p')
```

From a pickle file

```
>>> n = rf.Network('ntwk1.ntwk')
```

Create a blank network, then fill in values

```
>>> n = rf.Network()
>>> freq = rf.Frequency(1, 3, 3, 'GHz')
>>> n.frequency, n.s, n.z0 = freq, [1,2,3], [1,2,3]
```

Directly from values

```
>>> n = rf.Network(f=[1,2,3], s=[1,2,3], z0=[1,2,3])
```

Define some parameters associated with the Network

```
>>> n = rf.Network('ntwk1.s2p', params={'temperature': 25, 'voltage':5})
```

See also

from_z
init from impedance values

read
read a network from a file

write
write a network to a file, using pickle

write_touchstone
write a network to a touchstone file

skrf.network.Network.add_noise_polar

Network.**add_noise_polar**(*mag_dev*, *phase_dev*, ***kwargs*)

Adds a complex zero-mean gaussian white-noise.

adds a complex zero-mean gaussian white-noise of a given standard deviation for magnitude and phase

Parameters

- **mag_dev** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase [in degrees]

Return type
None

skrf.network.Network.add_noise_polar_flatband

Network.**add_noise_polar_flatband**(*mag_dev*, *phase_dev*, ***kwargs*)

Add a flatband complex zero-mean gaussian white-noise signal of given standard deviations for magnitude and phase.

Parameters

- **mag_dev** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase [in degrees]

Return type
None

skrf.network.Network.attribute

Network.**attribute**(*prop_name*, *conversion*)

Parameters

- **prop_name** (*Literal['s', 'z', 'y', 'a', 'g', 'h', 't']*)
- **conversion** (*Literal['re', 'im', 'mag', 'db', 'db10', 'rad', 'deg', 'arcl', 'rad_unwrap', 'deg_unwrap', 'arcl_unwrap', 'vswr', 'time', 'time_db', 'time_mag', 'time_impulse', 'time_step']*)

Return type
ndarray

skrf.network.Network.copy

Network.**copy**(** (Keyword-only parameters separator (PEP 3102))*, *shallow_copy=False*)

Return a copy of this Network.

Needed to allow pass-by-value for a Network instead of pass-by-reference

Parameters
shallow_copy (*bool, optional*) – If True, the method creates a new Network object with empty s-parameters that share the same shape as the original Network, but without copying the

actual s-parameters data. This is useful when you plan to immediately modify the s-parameters after creating the Network, as a deep copy would be unnecessary and costly. Using *shallow_copy* improves performance by leveraging lazy initialization through *numpy's np.empty()*, which allocates virtual memory without immediate physical memory allocation, deferring actual memory initialization until first access. This approach can significantly enhance *copy()* performance when dealing with large *Network* objects, when you are intended for immediate modification after the Network's creation.

Return type
Network

> **Note**
>
> If you require a complete copy of the *Network* instance or need to perform operation on the s-parameters of the copied Network, it is essential not to use the *shallow_copy* parameter!

Returns
ntwk – Copy of the Network

Return type
Network

Parameters
shallow_copy (*bool*)

skrf.network.Network.copy_from

`Network.copy_from(other)`

Copy the contents of another Network into self.

Uses copy, so that the data is passed-by-value, not reference

Parameters
other (Network) – the network to copy the contents of

Return type
None

Examples

```
>>> a = rf.N()
>>> b = rf.N('my_file.s2p')
>>> a.copy_from (b)
```

skrf.network.Network.copy_subset

`Network.copy_subset(key)`

Return a copy of a frequency subset of this Network.

Needed to allow pass-by-value for a subset Network instead of pass-by-reference

Parameters
key (*numpy array*) – the array indices of the frequencies to take

Returns
ntwk – Copy of the frequency subset of the Network

Return type
Network

skrf.network.Network.crop

`Network.crop(f_start, f_stop, unit=None)`

Crop Network based on start and stop frequencies.

No interpolation is done.

Parameters

- **f_start** (*number*) – start frequency of crop range, in units of self.frequency.unit. If *f_start* is lower than the lowest frequency, no change to the network is made by the lower bound.
- **f_stop** (*number*) – stop frequency of crop range, in units of self.frequency.unit If *f_stop* is higher than the highest frequency, no change to the network is made by the higher bound.
- **unit** (*string*) – Units that *f_start* and *f_stop* are described in. This must be a string recognized by the Frequency class, e.g. 'Hz','MHz', etc. A value of *None* assumes units are same as *self*

Return type
None

See also

cropped

skrf.network.Network.cropped

`Network.cropped(f_start, f_stop, unit=None)`

Returns a cropped network, leaves self alone.

Parameters

- **f_start** (*number*) – start frequency of crop range, in units of self.frequency.unit. If *f_start* is lower than the lowest frequency, no change to the network is made by the lower bound.
- **f_stop** (*number*) – stop frequency of crop range, in units of self.frequency.unit If *f_stop* is higher than the highest frequency, no change to the network is made by the higher bound.
- **unit** (*string*) – Units that *f_start* and *f_stop* are described in. This must be a string recognized by the Frequency class, e.g. 'Hz','MHz', etc. A value of *None* assumes units are same as *self*

Returns
ntwk – Resulting cropped network

Return type
Network object

See also

crop

skrf.network.Network.delay

Network.**delay**(*d*, *unit='deg'*, *port=0*, *media=None*, ***kw*)

Add phase delay to a given port.

This will connect a transmission line of length *d/2* to the selected *port*. If no propagation properties are specified for the line (*media=None*), then freespace is assumed to convert a distance *d* into an electrical length. If a phase angle is specified for *d*, it will be evaluated at the center frequency of the network.

Parameters

- **d** (*float*) – The angle/length/delay of the transmission line (see *unit* argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – The units of d. See `Media.to_meters()`, for details
- **port** (*int or str*) – Port to add the delay to, given by index or by name if the Network has named ports (*Network.port_names*). The delayed port keeps its name.
- **media** (*skrf.media.Media*) – Media object to use for generating the delay. If None, this will default to freespace.

Returns

ntwk – A delayed copy of the *Network*.

Return type

Network object

skrf.network.Network.drop_non_monotonic_increasing

Network.**drop_non_monotonic_increasing**()

Drop invalid values based on duplicate and non increasing frequency values.

Example

The following example shows how to use the *drop_non_monotonic_increasing()* automatically, if invalid frequency data is detected and an *InvalidFrequencyWarning* is thrown.

```
>>> import warnings
>>> import skrf as rf
>>> from skrf.frequency import InvalidFrequencyWarning
>>> with warnings.catch_warnings(record=True) as warns:
>>>     net = rf.Network('corrupted_network.s2p')
>>>     w = [w for w in warns if issubclass(w.category, InvalidFrequencyWarning)]
>>>     if w:
>>>         net.drop_non_monotonic_increasing()
```

Return type

None

skrf.network.Network.extrapolate_to_dc

Network.**extrapolate_to_dc**(*points=None*, *dc_sparam=None*, *kind='cubic'*, *coords='cart'*, ***kwargs*)

Extrapolate S-parameters down to 0 Hz and interpolate to uniform spacing.

If frequency vector needs to be interpolated aliasing will occur in time-domain. For the best results first frequency point should be a multiple of the frequency step so that points from DC to the first measured point can be added without interpolating rest of the frequency points.

Parameters

- **points** (*int or None*) – Number of frequency points to be used in interpolation. If None number of points is calculated based on the frequency step size and spacing between 0 Hz and first measured frequency point.
- **dc_sparam** (class:*np.ndarray* or None) – NxN S-parameters matrix at 0 Hz. If None S-parameters at 0 Hz are determined by linear extrapolation.
- **kind** (*str or int, default is 'cubic'*) – Specifies the kind of interpolation as a string ('linear', 'nearest', 'zero', 'slinear', 'quadratic, 'cubic') or as an integer specifying the order of the spline interpolator to use for *scipy.interp1d*.

  kind = 'rational' uses interpolation by rational polynomials.

  d kwarg controls the degree of rational polynomials when *kind* is 'rational'. Defaults to 4.
- **coords** (*str*) – Coordinate system to use for interpolation: 'cart' or 'polar'. 'cart' is cartesian is Re/Im, 'polar' is unwrapped phase/mag. Passed to *Network.interpolate()*

Returns
result – Extrapolated Network

Return type
Network

See also

interpolate, *impulse_response*, *step_response*

skrf.network.Network.flip

`Network.flip()`

Swap the ports of a 2n-port Network (inplace).

In case the network is 2n-port and n > 1, 'second' numbering scheme is assumed to be consistent with the ** cascade operator:

```
    +--------+                  +--------+
  0-|0      n|-n              0-|n      0|-n
  1-|1    n+1|-n+1    flip    1-|n+1    1|-n+1
   ...      ...        =>       ...      ...
n-1-|n-1 2n-1|-2n-1         n-1-|2n-1 n-1|-2n-1
    +--------+                  +--------+
```

See also

flipped, *renumber*, *renumbered*

Return type
None

skrf.network.Network.flipped

Network.**flipped**()

Returns a flipped network, leaves self alone.

Returns
ntwk – Resulting flipped Network

Return type
Network object

See also

flip, *renumber*, *renumbered*

skrf.network.Network.from_string

classmethod Network.**from_string**(*data*, ***kwargs*)

Create a Network object from a text string

Parameters

- **data** (*str*) –

  Touchstone file contents. Supported formats are

  – touchstone file (.s?p) (or .ts)

- ****kwargs** – key word arguments can be used to assign properties of the Network, such as *s*, *f* and *z0*. keyword *name* can be used to name the Network. keyword *params* is used to assign parameters to the Network. keyword *comments* associates comments with the Network. keyword *s_def* sets the scattering parameter definition. Can be 'power',

  'pseudo', or 'traveling'. 'power' for power-wave definition, 'pseudo' for pseudo-waves definition. 'traveling' corresponds to the initial implementation. Default is 'power'. NB: results are the same for real-valued characteristic impedances.

  keyword *encoding* can be used to define the Touchstone file encoding. keyword *noise_interp_kind* used to change the default interpolation

  method for noisy networks. Options are 'linear', 'nearest', 'nearest-up', 'zero', 'slinear', 'quadratic', 'cubic', 'previous', or 'next'. Review *scipy.interpolate.interp_1d* for details on each interpolation style. Defaults to 'linear'.

  keyword *noise_fill_value* used to change the default interpolation
  fill value for noisy networks. Defaults to np.nan.

Return type
Network

Examples

From file contents

```
>>> filepath = 'ntwk1.s2p'
>>> with open(filepath, 'r') as f:
>>>     content = f.read()
>>> n = rf.Network.from_string(content)
```

skrf.network.Network.from_z

classmethod Network.**from_z**(*z*, **args*, ***kw*)

Create a Network from its Z-parameters.

Parameters

- **z** (*Numpy array*) – Impedance matrix. Should be of shape fxnxn, where f is frequency axis and n is number of ports
- ****kwargs** – key word arguments can be used to assign properties of the Network, *f* and *z0*.

Returns
ntw – Created Network

Return type
Network

Examples

```
>>> f = rf.Frequency(start=1, stop=2, npoints=4, unit="GHz")  # 4 frequency points
>>> z = np.random.rand(len(f),2,2) + np.random.rand(len(f),2,2)*1j  # 2-port z-
↪matrix: shape=(4,2,2)
>>> ntw = rf.Network.from_z(z, frequency=f)
```

skrf.network.Network.func_on_parameter

Network.**func_on_parameter**(*func*, *attr='s'*, **args*, ***kwargs*)

Apply a function parameter matrix, one frequency slice at a time.

This is useful for functions that can only operate on 2d arrays, like numpy.linalg.inv. This loops over f and calls *func(ntwkA.s[f,:,:], *args, **kwargs)*

Parameters

- **func** (*func*) – function to apply to s-parameters, on a single-frequency slice. (ie *func(ntwkA.s[0,:,:], *args, **kwargs)*
- **attr** (*string*) – Name of the parameter to operate on. Ex: 's', 'z', etc. Default is 's'.
- ***args** – passed to the func
- ****kwargs** – passed to the func

Returns
ntwk – Resulting renumbered Network

Return type
Network object

Examples

```
>>> from numpy.linalg import inv
>>> ntwk.func_on_parameter(inv)
```

skrf.network.Network.gain_circle

Network.**gain_circle**(*target_port*, *gain*, *npoints=181*)

Returns loci of gain circles for a given port (0 or 1) and a specified gain. The network must have two ports. The center and radius of the source (here target_port=0) gain circle are calculated by the following equations[1][2].

$$C_S = \frac{g_S S_{11}^*}{1-(1-g_S)|S_{11}|^2}$$
$$R_S = |\frac{\sqrt{(1-g_S)}(1-|S_{11}|^2)}{1-(1-g_S)|S_{11}|^2}$$

where g_S is obtained by normalizing the specified gain by the maximum gain of the source matching network G_{Smax}

$$g_S = \frac{gain}{G_{Smax}} = gain * (1-|S_{11}|^2)$$

Similarly, those of the load side (here target_port=1) are calculated by the following equations.

$$C_L = \frac{g_L S_{22}^*}{1-(1-g_L)|S_{22}|^2}$$
$$R_L = |\frac{\sqrt{(1-g_L)}(1-|S_{22}|^2)}{1-(1-g_L)|S_{22}|^2}$$
$$with$$
$$g_L = \frac{gain}{G_{Lmax}} = gain * (1-|S_{22}|^2)$$

Parameters

- **target_port** (*int*) – Specifies the port number (0 or 1) to calculate gain circles.
- **gain** (*float*) – Gain of source or load matching network in decibels.
- **npoints** (*int, optional*) – The number of points on the circumference of the circle. More points result in a smoother circle, but require more computation. Default is 181.

Returns

gc – Loci of gain circles in complex numbers

Return type

numpy.ndarray (shape is *npoints x f*)

Example

```
>>> import skrf as rf
>>> import matplotlib.pyplot as plt
```

Create a two-port network object

```
>>> ntwk = rf.Network('fet.s2p')
```

Calculate the source gain circles for all the frequencies at a gain of 2 dB

[1] David. M. Pozar, "Microwave Engineering, Fourth Edition," Wiley, p. 576, 2011.
[2] https://www.allaboutcircuits.com/technical-articles/designing-a-unilateral-rf-amplifier-for-a-specified-gain/

```
>>> sgc = ntwk.gain_circle(target_port=0, gain=2.0)
```

Plot the circles on the smith chart

```
>>> rf.plotting.plot_smith(s=sgc, smith_r=1, marker='o')
>>> plt.show()
```

Slicing the network allows you to specify a frequency

```
>>> sgc = ntwk['1GHz'].gain_circle(target_port=0, gain=2.0)
>>> rf.plotting.plot_smith(s=sgc, smith_r=1, marker='o')
>>> plt.show()
```

References

See also

stability_circle, *nf_circle*

max_gain
Maximum available and stable power gain

max_stable_gain
Maximum stable power gain

unilateral_gain
Mason's unilateral power gain

skrf.network.Network.gmm2se

`Network.`**`gmm2se`**(*p*, *z0_se=None*, *s_def=None*)

Transform network from generalized mixed mode parameters[1] to single ended parameters.

Parameters

- **p** (*int*) – number of differential ports
- **z0_se** (*Numpy array*) – *f x 2*p x 2*p* matrix of single ended impedances, optional if input is None, extract the reference impedance from the differential network calculated as 0.5 * (0.5 * z0_diff + 2 * z0_comm) for each differential port. Single-ended ports not converted to differential mode keep their z0.
- **s_def** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition: None to use the definition set in the network *s_def* attribute. 'power' for power-waves definition[2], 'pseudo' for pseudo-waves definition[3]. All the definitions give the same result if z0 is real valued.

Return type
None

[1] Ferrero and Pirola; Generalized Mixed-Mode S-Parameters; IEEE Transactions on Microwave Theory and Techniques; Vol. 54; No. 1; Jan 2006

[2] Kurokawa, Kaneyuki "Power waves and the scattering matrix", IEEE Transactions on Microwave Theory and Techniques, vol.13, iss.2, pp. 194–202, March 1965.

[3] Marks, R. B. and Williams, D. F. "A general waveguide circuit theory", Journal of Research of National Institute of Standard and Technology, vol.97, iss.5, pp. 533–562, 1992.

Examples

In following examples, sx is single-mode port x, dy is differential-mode port y, and cz is common-mode port z. The low insertion loss path of a transmission line is symbolized by ==.

2-Port diagram:

```
  +-----+             +-----+
0-|s0   |           0-|d0   |
  |     | <=gmm2se=   |     |
1-|s1   |           1-|c0   |
  +-----+             +-----+
```

3-Port diagram:

```
  +-----+             +-----+
0-|s0   |           0-|d0   |
1-|s1   | <=gmm2se= 1-|c0   |
2-|s2   |           2-|s2   |
  +-----+             +-----+
```

Note: The port s2 remain in single-mode.

4-Port diagram:

```
  +------+              +------+
0-|s0==s2|-2           0-|d0==d1|-1
  |      |   <=gmm2se=  |      |
1-|s1==s3|-3           2-|c0==c1|-3
  +------+              +------+
```

5-Port diagram:

```
  +------+              +------+
0-|s0==s2|-2           0-|d0==d1|-1
1-|s1==s3|-3 <=gmm2se= 2-|c0==c1|-3
  |    s4|-4             |    s4|-4
  +------+              +------+
```

Note: The port s4 remain in single-mode.

8-Port diagram:

```
  +------+              +------+
0-|s0==s2|-2           0-|d0==d1|-1
1-|s1==s3|-3           2-|d2==d3|-3
  |      |   <=gmm2se=  |      |
4-|s4==s6|-6           4-|c0==c1|-5
5-|s5==s7|-7           6-|c2==c3|-7
  +------+              +------+
```

2N-Port diagram:

```
    A                           B
    +-----------+               +-----------+
  0-|s0=======s2|-2           0-|d0======d1|-1
```

(continues on next page)

(continued from previous page)

```
   1-|s1========s3|-3                  2-|d2=======d3|-3
     ...          ...     <=gmm2se=      ...         ...
2N-4-|s2N-4==s2N-2|-2N-2             2N-4-|cN-4===cN-3|-2N-3
2N-3-|s2N-3==s2N-1|-2N-1             2N-2-|cN-2===cN-1|-2N-1
     +------------+                       +-----------+
```

Note: The network *A* is not cascadable with the ** operator along transmission path.

References

See also

se2gmm

skrf.network.Network.impulse_response

`Network.impulse_response`(*window='hamming'*, *n=None*, *pad=0*, *bandpass=None*, *squeeze=True*)

Calculate time-domain impulse response of one-port.

First frequency must be 0 Hz for the transformation to be accurate and the frequency step must be uniform. Positions of the reflections are accurate even if the frequency doesn't begin from 0, but shapes will be distorted.

Real measurements should be extrapolated to DC and interpolated to uniform frequency spacing.

Y-axis is the reflection coefficient.

Parameters

- **window** (*string*) – FFT windowing function. (Default is 'hamming')
- **n** (*int*) – Length of impulse response output. If n is not specified, 2m - 1 points are used in low-pass mode, or m points in band-pass mode, where m = len(self) + pad. (default is None).
- **pad** (*int*) – Number of zeros to add as padding for FFT. Adding more zeros improves accuracy of peaks. (Default is 0)
- **bandpass** (*bool or None*) – If False window function is center on 0 Hz. If True full window is used and low frequencies are attenuated. If None value is determined automatically based on if the frequency vector begins from 0. (Default is None)
- **squeeze** (*bool*) – Squeeze impulse response to one dimension, if a oneport gets transformed. Has no effect when transforming a multiport. (Default is True)

Returns

- **t** (class:*np.ndarray*) – Time vector
- **y** (class:*np.ndarray*) – Impulse response

Return type
tuple[*ndarray*, *ndarray*]

skrf.network.Network.interpolate

`Network.interpolate`(*freq_or_n*, *basis='s'*, *coords='cart'*, *f_kwargs=None*, *return_array=False*, *kind=None*, ***kwargs*)

Interpolate a Network along frequency axis.

The input 'freq_or_n` can be either a new *Frequency* or an *int*, or a new frequency vector (in Hz).

This interpolates a given *basis*, ie s, z, y, etc, in the coordinate system defined by *coord* like polar or cartesian. Different interpolation types ('linear', 'quadratic') can be used by passing appropriate ***kwargs*. This function returns an interpolated Network. Alternatively *interpolate_self()* will interpolate self.

Parameters

- **freq_or_n** (*Frequency* or int or list-like) – The new frequency over which to interpolate. this arg may be one of the following:
  - a new *Frequency* object
  - an int: the current frequency span is resampled linearly.
  - a list-like: create a new frequency using *from_f()*
- **basis** (*['s','z','y','a'], etc*) – The network parameter to interpolate
- **coords** (*string*) – Coordinate system to use for interpolation: 'cart' or 'polar': 'cart' is cartesian is Re/Im. 'polar' is unwrapped phase/mag
- **return_array** (*bool*) – return the interpolated array instead of re-assigning it to a given attribute
- ****kwargs** (*keyword arguments*) – passed to interpolate method. *freq_cropped* kwarg controls whether to use pre-cropped frequency points for interpolation. Defaults to True.

  passed to `scipy.interpolate.interp1d()` initializer. *kind* controls interpolation type.

  kind = *rational* uses interpolation by rational polynomials.

  d kwarg controls the degree of rational polynomials when *kind* == *rational*. Defaults to 4.
- **f_kwargs** (*dict*)
- **kind** (*Literal['linear', 'cubic', 'nearest', 'zero', 'slinear', 'quadratic', 'rational'] | None*)

Returns

result – an interpolated Network, or array

Return type

Network

Note

Frequency cropping is only supported with methods from *scipy.interpolate.interpolate.interp1d*. The 'rational' method does not support frequency cropping.

The interpolation coordinate system (*coords*) makes a big difference for large amounts of interpolation. polar works well for duts with slowly changing magnitude. try them all.

See `scipy.interpolate.interpolate.interp1d()` for useful kwargs. For example:

kind

[string or int] Specifies the kind of interpolation as a string ('linear', 'nearest', 'zero', 'slinear', 'quadratic, 'cubic') or as an integer specifying the order of the spline interpolator to use.

See also

resample, *interpolate_self*, interpolate_from_f

Examples

```
In [1]: n = rf.data.ring_slot

In [2]: n
Out[2]: 2-Port Network: 'ring slot',  75.0-110.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]

In [3]: new_freq = rf.Frequency(75,110,501,'ghz')

In [4]: n.interpolate(new_freq, kind = 'cubic')
Out[4]: 2-Port Network: 'ring slot',  75.0-110.0 GHz, 501 pts, z0=[50.+0.j 50.+0.j]
```

skrf.network.Network.interpolate_self

Network.**interpolate_self**(*freq_or_n*, ***kwargs*)

Interpolate the current Network along frequency axis (inplace).

The input 'freq_or_n` can be either a new *Frequency* or an *int*, or a new frequency vector (in Hz).

See *interpolate()* for more information.

Parameters

- **freq_or_n** (*Frequency* or int or list-like) – The new frequency over which to interpolate. this arg may be one of the following:
  - a new *Frequency* object
  - an int: the current frequency span is resampled linearly.
  - a list-like: create a new frequency using *from_f()*
- ****kwargs** (*keyword arguments*) – passed to `scipy.interpolate.interp1d()` initializer.

Returns

The interpolation is performed inplace.

Return type

None

See also

resample, *interpolate*, interpolate_from_f

skrf.network.Network.is_lossless

Network.**is_lossless**(*tol=1e-12*)

Test for losslessness.

[S] is lossless if [S] is unitary, i.e. if $([S][S]^* = [1])$

Parameters
tol (*float, optional*) – Numerical tolerance. The default is `skrf.mathFunctions.ALMOST_ZERO`

Returns
bool

Return type
boolean

See also

is_passive, *is_symmetric*, *is_reciprocal*

References

https://en.wikipedia.org/wiki/Unitary_matrix

skrf.network.Network.is_passive

`Network.is_passive`(*tol=1e-12*)

Test for passivity.

Parameters
tol (*float, optional*) – Numerical tolerance. The default is `skrf.mathFunctions.ALMOST_ZERO`

Returns
bool

Return type
boolean

skrf.network.Network.is_reciprocal

`Network.is_reciprocal`(*tol=1e-12*)

Test for reciprocity.

Parameters
tol (*float, optional*) – Numerical tolerance. The default is `skrf.mathFunctions.ALMOST_ZERO`.

Returns
bool

Return type
boolean

> **See also**
>
> *reciprocity*

skrf.network.Network.is_symmetric

`Network.is_symmetric`(*n=1*, *port_order=None*, *tol=1e-12*)

Return whether the 2N-port network has n-th order reflection symmetry by checking. $S_{i,i} == S_{j,j}$ for appropriate pair(s) of i and j.

Parameters

- **n** (*int*) – Order of line symmetry to test for
- **port_order** (*dict[int, int]*) – Renumbering of zero-indexed ports before testing
- **tol** (*float*) – Tolerance in numeric comparisons. Default is `skrf.mathFunctions.ALMOST_ZERO`.

Returns
bool

Return type
boolean

Raises
`ValueError` –

(1) If the network has an odd number of ports (2) If n is not in the range 1 to N (3) If n does not evenly divide 2N (4) If port_order is not a valid reindexing of ports e.g. specifying x->y but not y->z, specifying x->y twice, or using an index outside the range 0 to 2N-1

References

https://en.wikipedia.org/wiki/Two-port_network#Scattering_parameters_(S-parameters)

skrf.network.Network.multiply_noise

`Network.multiply_noise`(*mag_dev*, *phase_dev*, ***kwargs*)

Multiply a complex bivariate gaussian white-noise signal of given standard deviations for magnitude and phase. The mean of the magnitude is 1, and the mena of the phase is 0.

Parameters

- **mag_dev** (*float*) – standard deviation of magnitude
- **phase_dev** (*float*) – standard deviation of phase [in degrees]

Return type
None

skrf.network.Network.nf

`Network.nf`(*z*)

The noise figure for the network if the source impedance is z.

Parameters
z (*Number | Sequence[Number] | ndarray*)

Return type
ndarray

skrf.network.Network.nf_circle

`Network.nf_circle(nf, npoints=181)`

Returns loci of noise figure circles for a specified noise figure. The network must have two ports and noise data. The center and radius of the noise figure circle are calculated by the following equations[1].

$$C_F = \frac{\Gamma_{opt}}{N+1}$$

$$R_F = \frac{\sqrt{N(N+1-|\Gamma_{opt}|^2)}}{N+1}$$

where N is the noise figure parameter defined by

$$N = \frac{|\Gamma_s - \Gamma_{opt}|^2}{1-|\Gamma_s|^2} = \frac{F - F_{min}}{4R_N/Z_0}|1+\Gamma_{opt}|^2$$

Parameters

- **nf** (*float*) – Noise figure of network in decibels.
- **npoints** (*int, optional*) – The number of points on the circumference of the circle. More points result in a smoother circle, but require more computation. Default is 181.

Returns
nfc – Loci of noise figure circles in complex numbers

Return type
numpy.ndarray (shape is *npoints x f*)

Example

```
>>> import skrf as rf
>>> import matplotlib.pyplot as plt
```

Create a two-port network object

```
>>> ntwk = rf.Network('ntwk_noise.s2p')
```

Calculate the noise figure circles for all the frequencies at a noise figure of 1 dB

```
>>> nfc = ntwk.nf_circle(nf=1.0)
```

Plot the circles on the smith chart

```
>>> rf.plotting.plot_smith(s=nfc, smith_r=1, marker='o')
>>> plt.show()
```

Slicing the network allows you to specify a frequency

```
>>> nfc = ntwk['1GHz'].nf_circle(nf=1.0)
>>> rf.plotting.plot_smith(s=nfc, smith_r=1, marker='o')
>>> plt.show()
```

[1] David. M. Pozar, "Microwave Engineering, Fourth Edition," Wiley, p. 580, 2011.

References

See also

stability_circle, *gain_circle*

g_opt
: The optimum source reflection coefficient to minimize noise.

nfmin
: The minimum noise figure of the network.

nfmin_db
: The minimum noise figure for the network in dB.

rn
: The equivalent noise resistance of the network.

skrf.network.Network.nfdb_gs

Network.**nfdb_gs**(*gs*)

Return dB(NF) foreach gamma_source x noise_frequency.

Parameters
gs (*Number | Sequence[Number] | ndarray*)

Return type
ndarray

skrf.network.Network.nonreciprocity

Network.**nonreciprocity**(*m*, *n*, *normalize=False*)

Normalized non-reciprocity metric.

This is a port-by-port measure of how non-reciprocal an n-port network is. It is defined by,

$$(S_{mn} - S_{nm})/\sqrt{(S_{mn}S_{nm})}$$

Parameters

- **m** (*int or str*) – m index, or port name if the Network has named ports (*Network.port_names*). Note that the indices are one-based, like the *s* matrix element they name, and unlike the zero-based port indices taken by *connect()* or *Network.renumber()*. Port names are unaffected by this.
- **n** (*int or str*) – n index, one-based, or port name
- **normalize** (*bool*) – Normalize the result. Default is False.

Returns
ntwk – Resulting renumbered Network

Return type
Network object

Raises

- **ValueError** – If an index is out of range. Being one-based, 0 is not a valid index.
- **KeyError** – If a port name is unknown.

skrf.network.Network.nudge

Network.**nudge**(*amount=1e-12*)

Perturb s-parameters by small amount.

This is useful to work-around numerical bugs.

Parameters
amount (*float*) – amount to add to s parameters

Returns
ntwk – Resulting renumbered Network

Return type
Network object

Note

This function is:

```
self.s = self.s + amount
```

skrf.network.Network.plot

Network.**plot**(**args*, ***kwargs*)

Plot something vs frequency

skrf.network.Network.plot_a_arcl

Network.**plot_a_arcl**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_arcl* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_arcl(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_arcl_unwrap

Network.**plot_a_arcl_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_arcl_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_arcl_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_complex

Network.**plot_a_complex**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_db

`Network.`**`plot_a_db`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_db* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_db10

`Network.`**`plot_a_db10`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_db10* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_db10(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_deg

`Network.`**plot_a_deg**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_deg* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_deg(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_deg_unwrap

Network.**plot_a_deg_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_deg_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_deg_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_im

Network.**plot_a_im**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_im* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_im(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_mag

`Network.`**`plot_a_mag`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_mag* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_polar

`Network.`**`plot_a_polar`**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on

- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_rad

`Network.`**plot_a_rad**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_rad* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_rad(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_rad_unwrap

`Network.`**plot_a_rad_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_rad_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_rad_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_re

Network.**plot_a_re**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_re* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_re(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_time

Network.**plot_a_time**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_time* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_time(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_time_db

Network.**plot_a_time_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_time_db* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)

- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_time_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_time_impulse

Network.**plot_a_time_impulse**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_time_impulse* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_time_impulse(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_time_mag

Network.**plot_a_time_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_time_mag* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all

- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_time_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_time_step

Network.**plot_a_time_step**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_time_step* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_time_step(m=1,n=0,color='r')
```

skrf.network.Network.plot_a_vswr

Network.**plot_a_vswr**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *a_vswr* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_a_vswr(m=1,n=0,color='r')
```

skrf.network.Network.plot_attribute

Network.**plot_attribute**(*attribute*, *conversion*, *m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute conversion_attribute component vs frequency or time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_conversion_attribute(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_arcl

Network.**plot_g_arcl**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_arcl* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_arcl(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_arcl_unwrap

Network.**plot_g_arcl_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_arcl_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_arcl_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_complex

`Network.`**plot_g_complex**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_db

`Network.`**plot_g_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_db* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_db10

Network.**plot_g_db10**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_db10* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_db10(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_deg

Network.**plot_g_deg**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_deg* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_deg(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_deg_unwrap

Network.**plot_g_deg_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_deg_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)

- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_deg_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_im

Network.**plot_g_im**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_im* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_im(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_mag

Network.**plot_g_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_mag* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on

- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_polar

`Network.`**plot_g_polar**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_rad

Network.**plot_g_rad**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_rad* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_rad(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_rad_unwrap

Network.**plot_g_rad_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_rad_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_rad_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_re

Network.**plot_g_re**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_re* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_re(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_time

Network.**plot_g_time**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_time* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_time(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_time_db

`Network.`**plot_g_time_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_time_db* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_time_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_time_impulse

Network.**plot_g_time_impulse**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_time_impulse* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_time_impulse(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_time_mag

Network.**plot_g_time_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_time_mag* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_time_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_time_step

Network.**plot_g_time_step**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_time_step* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_time_step(m=1,n=0,color='r')
```

skrf.network.Network.plot_g_vswr

Network.**plot_g_vswr**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *g_vswr* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_g_vswr(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_arcl

`Network.`**plot_h_arcl**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_arcl* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_arcl(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_arcl_unwrap

Network.**plot_h_arcl_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_arcl_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_arcl_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_complex

Network.**plot_h_complex**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_db

`Network.`**`plot_h_db`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_db* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_db10

`Network.`**`plot_h_db10`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_db10* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_db10(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_deg

`Network.`**plot_h_deg**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_deg* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_deg(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_deg_unwrap

Network.**plot_h_deg_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_deg_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_deg_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_im

Network.**plot_h_im**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_im* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_im(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_mag

Network.**plot_h_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_mag* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_polar

Network.**plot_h_polar**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on

- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_rad

`Network.`**`plot_h_rad`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_rad* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_rad(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_rad_unwrap

`Network.`**`plot_h_rad_unwrap`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_rad_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_rad_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_re

Network.**plot_h_re**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_re* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_re(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_time

Network.**plot_h_time**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_time* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_time(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_time_db

Network.**plot_h_time_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_time_db* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)

- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_time_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_time_impulse

Network.**plot_h_time_impulse**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_time_impulse* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_time_impulse(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_time_mag

Network.**plot_h_time_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_time_mag* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all

- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_time_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_time_step

Network.**plot_h_time_step**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_time_step* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_time_step(m=1,n=0,color='r')
```

skrf.network.Network.plot_h_vswr

Network.**plot_h_vswr**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *h_vswr* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_h_vswr(m=1,n=0,color='r')
```

skrf.network.Network.plot_it_all

Network.**plot_it_all**(**args*, ***kwargs*)

Plot dB, deg, smith, and complex in subplots.

Plots the magnitude in dB in subplot 1, the phase in degrees in subplot 2, a smith chart in subplot 3, and a complex plot in subplot 4.

Parameters

- ***args** (*arguments, optional*) – passed to the matplotlib.plot command
- ****kwargs** (*keyword arguments, optional*) – passed to the matplotlib.plot command

See also

plot_s_db, *plot_s_deg*, *plot_s_smith*, *plot_s_complex*

Examples

```
>>> from skrf.data import ring_slot
>>> ring_slot.plot_it_all()
```

skrf.network.Network.plot_passivity

Network.**plot_passivity**(*port=None*, *label_prefix=None*, **args*, ***kwargs*)

Plot dB(diag(passivity metric)) vs frequency.

> **Note**
>
> This plot does not completely capture the passivity metric, which is a test for *unitary-ness* of the s-matrix. However, it may be used to display a measure of power dissipated in a network.

> **See also**
>
> *passivity*

skrf.network.Network.plot_prop_complex

Network.**plot_prop_complex**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling plot_vs_frequency_generic()

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_prop_polar

Network.**plot_prop_polar**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all

- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling plot_vs_frequency_generic()

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_reciprocity

Network.**plot_reciprocity**(*db=False*, **args*, ***kwargs*)

Plot reciprocity metric.

See also

reciprocity

skrf.network.Network.plot_reciprocity2

Network.**plot_reciprocity2**(*db=False*, **args*, ***kwargs*)

Plot reciprocity metric #2.

This is distance of the determinant of the wave-cascading matrix from unity.

$$abs(1 - S/S^T)$$

See also

reciprocity

skrf.network.Network.plot_s_arcl

Network.**plot_s_arcl**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_arcl* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all

- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_arcl(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_arcl_unwrap

Network.**plot_s_arcl_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_arcl_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_arcl_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_complex

Network.**plot_s_complex**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling plot_vs_frequency_generic()

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_db

Network.**plot_s_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_db* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_db10

Network.**plot_s_db10**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_db10* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_db10(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_db_time

Network.**plot_s_db_time**(*center_to_dc=None*, **args*, ***kwargs*)

skrf.network.Network.plot_s_deg

Network.**plot_s_deg**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_deg* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off

- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_deg(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_deg_unwrap

`Network.`**`plot_s_deg_unwrap`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_deg_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_deg_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_error

`Network.`**`plot_s_error`**(*other*, *error_function='average_l2_norm'*, *db=False*, **args*, ***kwargs*)

Plot the s-parameter error between two networks vs frequency.

Parameters

- **netw** (*Network*) – The reference network.
- **other** (*Network*) – The network to compare against `netw`.
- **error_function** (*str*) – Error function name passed to *s_error()*. One of `average_l1_norm`, `average_l2_norm`, `maximum_l1_norm`, or `average_normalized_l1_norm`. Default is `average_l2_norm`.
- **db** (*bool*) – If True, plot $20\log_{10}(\delta)$. Default is False.
- ***args** – Passed to *plot()*.
- ****kwargs** – Passed to *plot()*.

See also

skrf.network.s_error, *skrf.network.Network.s_error*

skrf.network.Network.plot_s_im

`Network.`**plot_s_im**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_im* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_im(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_mag

`Network.`**plot_s_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_mag* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_polar

Network.**plot_s_polar**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling plot_vs_frequency_generic()

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_rad

Network.**plot_s_rad**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_rad* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_rad(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_rad_unwrap

Network.**plot_s_rad_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_rad_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)

- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_rad_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_re

Network.**plot_s_re**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_re* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_re(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_smith

Network.**plot_s_smith**(*m=None*, *n=None*, *r=1*, *ax=None*, *show_legend=True*, *chart_type='z'*, *draw_labels=False*, *label_axes=False*, *draw_vswr=None*, **args*, ***kwargs*)

Plots the scattering parameter on a smith chart.

Plots indices *m*, *n*, where *m* and *n* can be integers or lists of integers.

Parameters

- **m** (*int, optional*) – first index

- **n** (*int, optional*) – second index
- **ax** (*matplotlib.Axes object, optional*) – axes to plot on. in case you want to update an existing plot.
- **show_legend** (*boolean, optional*) – to turn legend show legend of not, optional
- **chart_type** (*['z','y']*) – draw impedance or admittance contours
- **draw_labels** (*Boolean*) – annotate chart with impedance values
- **label_axes** (*Boolean*) – Label axis with titles *Real* and *Imaginary*
- **border** (*Boolean*) – draw rectangular border around image with ticks
- **draw_vswr** (*list of numbers, Boolean or None*) – draw VSWR circles. If True, default values are used.
- ***args** (*arguments, optional*) – passed to the matplotlib.plot command
- ****kwargs** (*keyword arguments, optional*) – passed to the matplotlib.plot command

See also

plot_vs_frequency_generic, smith

Examples

```
>>> myntwk.plot_s_smith()
>>> myntwk.plot_s_smith(m=0,n=1,color='b', marker='x')
```

skrf.network.Network.plot_s_time

Network.**plot_s_time**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_time* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_time(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_time_db

Network.**plot_s_time_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_time_db* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_time_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_time_impulse

Network.**plot_s_time_impulse**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_time_impulse* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_time_impulse(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_time_mag

Network.**plot_s_time_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_time_mag* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_time_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_time_step

Network.**plot_s_time_step**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_time_step* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all

- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_time_step(m=1,n=0,color='r')
```

skrf.network.Network.plot_s_vswr

Network.**plot_s_vswr**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *s_vswr* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_s_vswr(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_arcl

Network.**plot_t_arcl**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_arcl* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_arcl(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_arcl_unwrap

Network.**plot_t_arcl_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_arcl_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)

- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_arcl_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_complex

Network.**plot_t_complex**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling plot_vs_frequency_generic()

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_db

Network.**plot_t_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_db* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_db10

`Network.`**plot_t_db10**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_db10* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_db10(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_deg

Network.**plot_t_deg**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_deg* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_deg(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_deg_unwrap

Network.**plot_t_deg_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_deg_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_deg_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_im

Network.**plot_t_im**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_im* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_im(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_mag

Network.**plot_t_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_mag* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_polar

`Network.`**plot_t_polar**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_rad

`Network.`**plot_t_rad**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_rad* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_rad(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_rad_unwrap

Network.**plot_t_rad_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_rad_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_rad_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_re

Network.**plot_t_re**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_re* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_re(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_time

Network.**plot_t_time**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_time* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_time(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_time_db

Network.**plot_t_time_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_time_db* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_time_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_time_impulse

Network.**plot_t_time_impulse**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_time_impulse* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on

- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_time_impulse(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_time_mag

`Network.`**plot_t_time_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_time_mag* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_time_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_time_step

Network.**plot_t_time_step**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_time_step* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_time_step(m=1,n=0,color='r')
```

skrf.network.Network.plot_t_vswr

Network.**plot_t_vswr**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *t_vswr* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_t_vswr(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_arcl

Network.**plot_y_arcl**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_arcl* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_arcl(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_arcl_unwrap

Network.**plot_y_arcl_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_arcl_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_arcl_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_complex

`Network.`**plot_y_complex**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_db

`Network.`**plot_y_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_db* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_db10

Network.**plot_y_db10**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_db10* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_db10(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_deg

Network.**plot_y_deg**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_deg* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_deg(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_deg_unwrap

Network.**plot_y_deg_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_deg_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)

- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_deg_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_im

Network.**plot_y_im**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_im* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_im(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_mag

Network.**plot_y_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_mag* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on

- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_polar

`Network.`**plot_y_polar**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_rad

Network.**plot_y_rad**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_rad* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_rad(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_rad_unwrap

Network.**plot_y_rad_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_rad_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_rad_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_re

Network.**plot_y_re**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_re* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_re(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_time

Network.**plot_y_time**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_time* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_time(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_time_db

`Network.`**plot_y_time_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_time_db* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_time_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_time_impulse

Network.**plot_y_time_impulse**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_time_impulse* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_time_impulse(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_time_mag

Network.**plot_y_time_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_time_mag* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_time_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_time_step

Network.**plot_y_time_step**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_time_step* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_time_step(m=1,n=0,color='r')
```

skrf.network.Network.plot_y_vswr

Network.**plot_y_vswr**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *y_vswr* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_y_vswr(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_arcl

`Network.`**plot_z_arcl**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_arcl* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_arcl(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_arcl_unwrap

Network.**plot_z_arcl_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_arcl_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_arcl_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_complex

Network.**plot_z_complex**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_db

`Network.`**`plot_z_db`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_db* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_db10

`Network.`**`plot_z_db10`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_db10* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not

- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_db10(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_deg

`Network.`**plot_z_deg**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_deg* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_deg(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_deg_unwrap

Network.**plot_z_deg_unwrap**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_deg_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_deg_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_im

Network.**plot_z_im**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_im* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_im(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_mag

Network.**plot_z_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_mag* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_polar

Network.**plot_z_polar**(**args*, ***kwargs*)

plot the Network attribute {} vs frequency.

Parameters

- **attribute** (*string*) – Network attribute to plot
- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on

- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- ***args** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling `plot_vs_frequency_generic()`

Examples

```
>>> myntwk.plot_{}(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_rad

`Network.`**`plot_z_rad`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_rad* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (`matplotlib.Axes` object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to `matplotlib.plot()`
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_rad(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_rad_unwrap

`Network.`**`plot_z_rad_unwrap`**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_rad_unwrap* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_rad_unwrap(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_re

Network.**plot_z_re**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_re* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_re(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_time

Network.**plot_z_time**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_time* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_time(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_time_db

Network.**plot_z_time_db**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_time_db* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)

- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_time_db(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_time_impulse

Network.**plot_z_time_impulse**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_time_impulse* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_time_impulse(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_time_mag

Network.**plot_z_time_mag**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_time_mag* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all

- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_time_mag(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_time_step

Network.**plot_z_time_step**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_time_step* component vs time.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

> **Note**
>
> This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_time_step(m=1,n=0,color='r')
```

skrf.network.Network.plot_z_vswr

Network.**plot_z_vswr**(*m=None*, *n=None*, *ax=None*, *show_legend=True*, *y_label=None*, *logx=False*, ***kwargs*)

plot the Network attribute *z_vswr* component vs frequency.

Parameters

- **m** (*int, optional*) – first index of s-parameter matrix, if None will use all
- **n** (*int, optional*) – second index of the s-parameter matrix, if None will use all
- **ax** (matplotlib.Axes object, optional) – An existing Axes object to plot on
- **show_legend** (*Boolean*) – draw legend or not
- **y_label** (*string, optional*) – the y-axis label
- **logx** (*Boolean, optional*) – Enable logarithmic x-axis, default off
- ****kwargs** (*arguments, keyword arguments*) – passed to matplotlib.plot()
- **attribute** (*PrimaryPropertiesT*)
- **conversion** (*ComponentFuncT*)

Note

This function is dynamically generated upon Network initialization. This is accomplished by calling *Network.plot_attribute()*

Examples

```
>>> myntwk.plot_z_vswr(m=1,n=0,color='r')
```

skrf.network.Network.read

Network.**read**(**args*, ***kwargs*)

Read a Network from a 'ntwk' file.

A ntwk file is written with *write()*. It is just a pickled file.

Parameters

- ***args** (*args and kwargs*) – passed to *skrf.io.general.read()*
- ****kwargs** (*args and kwargs*) – passed to *skrf.io.general.read()*

Return type

None

Note

This function calls *skrf.io.general.read()*.

Examples

```
>>> rf.read('myfile.ntwk')
>>> rf.read('myfile.p')
```

See also

skrf.io.general.read, *write*, *skrf.io.general.write*

skrf.network.Network.read_touchstone

Network.**read_touchstone**(*filename*, *encoding=None*)

Load values from a touchstone file.

The work of this function is done through the *touchstone* class.

Parameters

- **filename** (*str, Path, or file-object*) – touchstone file name.
- **encoding** (*str, optional*) – define the file encoding to use. Default value is None, meaning the encoding is guessed.

Return type

None

Note

Only the scattering parameters format is supported at the moment.

skrf.network.Network.renormalize

Network.**renormalize**(*z_new*, *s_def=None*)

Renormalize s-parameter matrix given a new port impedances.

Parameters

- **z_new** (*complex array of shape FxN, F, N or a scalar*) – new port impedances
- **s_def** (*str -> s_def : can be: None, 'power', 'pseudo' or 'traveling'*) – None to use the definition set in the network *s_def* attribute. Scattering parameter definition : 'power' for power-waves definition, 'pseudo' for pseudo-waves definition. 'traveling' corresponds to the initial implementation. Default is 'power'. NB: results are the same for real-valued characteristic impedances.

Return type

None

See also

renormalize_s, *fix_z0_shape*

skrf.network.Network.renumber

Network.**renumber**(*from_ports*, *to_ports*, *only_z0=False*)

Renumber ports of a Network (inplace).

Parameters

- **from_ports** (*list-like*) – List of port indices to change. Size between 1 and N_ports. Ports can be given by name if the Network has named ports (*Network.port_names*).
- **to_ports** (*list-like*) – List of desired port indices. Size between 1 and N_ports. A port name refers to the port which currently has that name, so renumber([‘a’, ‘b’], [‘b’, ‘a’]) swaps the ports named ‘a’ and ‘b’.
- **only_z0** (*bool*) – If true only z0 is renumbered, s-parameters are not affected. This should only be used after executing the “connect_s” method which keeps the port index where you expect it to be.
- **NB** (*from_ports and to_ports must have same size.*)

Returns

- *None*
- *The reorganization of the Network’s port is performed inplace.*

Return type

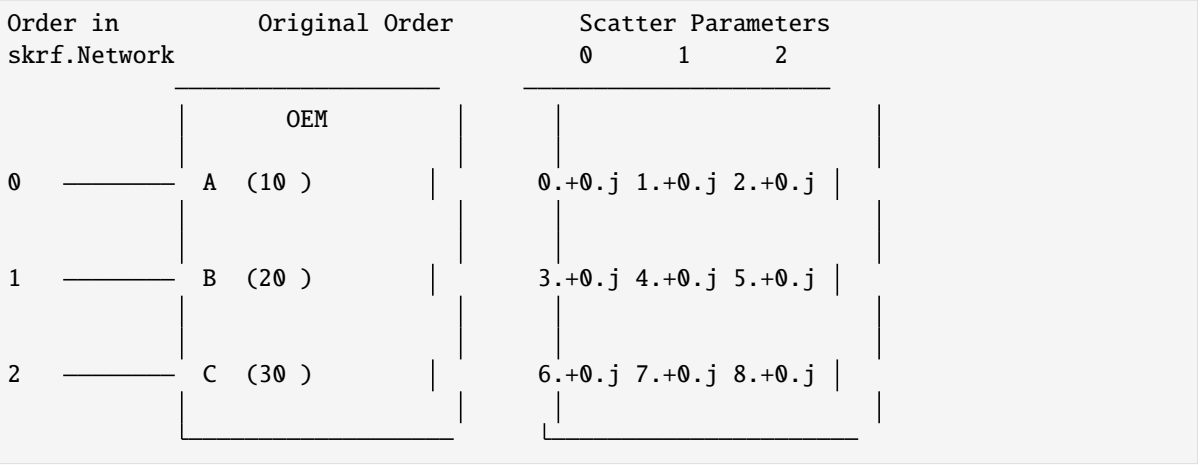
None

Examples

In the following example, the ports of a 3-ports Network are reorganized. Dummy reference impedances are set only to follow more easily the renumbering.

```
>>> f = rf.Frequency(1, 1, 1, 'hz')
>>> s = np.arange(9).reshape(1, 3, 3)
>>> z0 = [10, 20, 30]
>>> ntw = rf.Network(frequency=f, s=s, z0=z0)  # our OEM Network
```

In picture, we have:



While after renumbering

```
>>> ntw.renumber([0, 1, 2], [1, 2, 0])
```

we now have:

```
Order in                              Scatter Parameters
skrf.Network                          0      1      2
             ┌──────────────────┐     ┌──────────────────────┐
             │       OEM        │     │                      │
             │                  │     │                      │
0   ─────────  C  (30 )         │     │ 8.+0.j 6.+0.j 7.+0.j │
             │                  │     │                      │
             │                  │     │                      │
1   ─────────  A  (10 )         │     │ 2.+0.j 0.+0.j 1.+0.j │
             │                  │     │                      │
             │                  │     │                      │
2   ─────────  B  (20 )         │     │ 5.+0.j 3.+0.j 4.+0.j │
             │                  │     │                      │
             └──────────────────┘     └──────────────────────┘
```

Other examples:

Reorganized only the reference impedance of the ports, while keeping the order of the scattering parameters is also supported. This is beneficial in some special cases.

```
>>> ntw.renumber([1, 2, 0], [0, 1, 2], only_z0=True)
```

we now have:

```
Order in                              Scatter Parameters
skrf.Network                          0      1      2
             ┌──────────────────┐     ┌──────────────────────┐
             │       OEM        │     │                      │
             │                  │     │                      │
0   ─────────  A  (10 )         │     │ 8.+0.j 6.+0.j 7.+0.j │
             │                  │     │                      │
             │                  │     │                      │
1   ─────────  B  (20 )         │     │ 2.+0.j 0.+0.j 1.+0.j │
             │                  │     │                      │
             │                  │     │                      │
2   ─────────  C  (30 )         │     │ 5.+0.j 3.+0.j 4.+0.j │
             │                  │     │                      │
             └──────────────────┘     └──────────────────────┘
```

To flip the ports of a 2-port network ‘foo’:

```
>>> foo.renumber( [0,1], [1,0] )
```

To rotate the ports of a 3-port network ‘bar’ so that port 0 becomes port 1:

```
>>> bar.renumber( [0,1,2], [1,2,0] )
```

To swap the first and last ports of an N-port (N>=2) Network ‘duck’:

```
>>> duck.renumber( [0,-1], [-1,0] )
```

See also

renumbered, *flip*, *flipped*

skrf.network.Network.renumbered

`Network.`**`renumbered`**(*from_ports*, *to_ports*)

Return a renumbered Network, leave self alone.

Parameters

- **from_ports** (*list-like*) – List of port indices to change. Size between 1 and N_ports. Ports can be given by name if the Network has named ports (*Network.port_names*).
- **to_ports** (*list-like*) – List of desired port indices. Size between 1 and N_ports.
- **NB** (*from_ports and to_ports must have same size.*)

Returns

ntwk – Resulting renumbered Network

Return type

Network object

See also

renumber, *flip*, *flipped*

skrf.network.Network.resample

`Network.`**`resample`**(*freq_or_n*, ***kwargs*)

Interpolate the current Network along frequency axis (inplace).

The input 'freq_or_n` can be either a new *Frequency* or an *int*, or a new frequency vector (in Hz).

See *interpolate()* for more information.

Parameters

- **freq_or_n** (*Frequency* or int or list-like) – The new frequency over which to interpolate. this arg may be one of the following:
  - a new *Frequency* object
  - an int: the current frequency span is resampled linearly.
  - a list-like: create a new frequency using *from_f()*
- ****kwargs** (*keyword arguments*) – passed to `scipy.interpolate.interp1d()` initializer.

Returns

The interpolation is performed inplace.

Return type

None

See also

resample, *interpolate*, interpolate_from_f

skrf.network.Network.rotate

Network.**rotate**(*theta*, *unit='deg'*)

Rotate S-parameters

Parameters

- **theta** (*Number | Sequence[Number] | ndarray*)
- **unit** (*str*)

Return type
None

skrf.network.Network.s_active

Network.**s_active**(*a*)

Returns the active s-parameters of the network for a defined wave excitation a.

The active s-parameter at a port is the reflection coefficients when other ports are excited. It is an important quantity for active phased array antennas.

Active s-parameters are defined by[1]:

$$\mathrm{active(s)}_m = \sum_{i=1}^{N} s_{mi} \frac{a_i}{a_m}$$

where s are the scattering parameters and N the number of ports

Parameters
a (*complex array of shape (n_ports)*) – forward wave complex amplitude (pseudowave formulation[2])

Returns
s_act – active S-parameters for the excitation a

Return type
complex array of shape (n_freqs, n_ports)

See also

z_active
active Z-parameters

y_active
active Y-parameters

[1] D. M. Pozar, IEEE Trans. Antennas Propag. 42, 1176 (1994).

[2] D. Williams, IEEE Microw. Mag. 14, 38 (2013).

vswr_active
active VSWR

References

skrf.network.Network.s_error

Network.**s_error**(*ntwk*, *error_function='average_l2_norm'*)

Compute the error between s-parameters of this network and another network *ntwk*.

See *skrf.s_error()*.

Parameters

- **ntwk** (Network)
- **error_function** (*Literal['average_l1_norm', 'average_l2_norm', 'maximum_l1_norm', 'average_normalized_l1_norm']*)

Return type
ndarray

skrf.network.Network.se2gmm

Network.**se2gmm**(*p*, *z0_mm=None*, *s_def=None*)

Transform network from single ended parameters to generalized mixed mode parameters[1].

Parameters

- **p** (*int*) – number of differential ports
- **z0_mm** (*Numpy array*) – *f x 2*p x 2*p* matrix of mixed mode impedances, optional. If input is None, 2 * z0 Ohms differential and z0 / 2 Ohms common mode reference impedance is used, where z0 is average of the differential pair ports reference impedance. Single-ended ports not converted to differential mode keep their z0.
- **s_def** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition : None to use the definition set in the network *s_def* attribute. 'power' for power-waves definition[2], 'pseudo' for pseudo-waves definition[3]. All the definitions give the same result if z0 is real valued.
- **Ports** (*Note Odd Number of*) – In the case where there are an odd number of ports (such as a 3-port network with ports 0, 1, and 2), se2gmm() assumes that the last port (port 2) remains single-ended and ports 0 and 1 are converted to differential mode and common mode, respectively. For networks in which the port ordering is not suitable, port renumbering can be used.

Return type
None

[1] Ferrero and Pirola; Generalized Mixed-Mode S-Parameters; IEEE Transactions on Microwave Theory and Techniques; Vol. 54; No. 1; Jan 2006

[2] Kurokawa, Kaneyuki "Power waves and the scattering matrix", IEEE Transactions on Microwave Theory and Techniques, vol.13, iss.2, pp. 194–202, March 1965.

[3] Marks, R. B. and Williams, D. F. "A general waveguide circuit theory", Journal of Research of National Institute of Standard and Technology, vol.97, iss.5, pp. 533–562, 1992.

Examples

For example, a 3-port single-ended network is converted to mixed-mode parameters:

```
| Port 0 (single-ended, 50 ohms) --> Port 0 (single-ended, 50 ohms)
| Port 1 (single-ended, 50 ohms) --> Port 1 (differential mode, 100 ohms)
| Port 2 (single-ended, 50 ohms) --> Port 2 (common mode, 25 ohms)
```

```
>>> ntwk.renumber([0,1,2], [2,1,0])
>>> ntwk.se2gmm(p=1)
>>> ntwk.renumber([0,1,2], [1,2,0])
```

In the resulting network, port 0 is single-ended, port 1 is differential mode, and port 2 is common mode.

In following examples, sx is single-mode port x, dy is differential-mode port y, and cz is common-mode port z. The low insertion loss path of a transmission line is symbolized by ==.

2-Port diagram:

```
  +-----+             +-----+
0-|s0   |           0-|d0   |
  |     | =se2gmm=>   |     |
1-|s1   |           1-|c0   |
  +-----+             +-----+
```

3-Port diagram:

```
  +-----+             +-----+
0-|s0   |           0-|d0   |
1-|s1   | =se2gmm=> 1-|c0   |
2-|s2   |           2-|s2   |
  +-----+             +-----+
```

Note: The port s2 remain in single-mode.

4-Port diagram:

```
  +------+              +------+
0-|s0==s2|-2           0-|d0==d1|-1
  |      |   =se2gmm=>  |      |
1-|s1==s3|-3           2-|c0==c1|-3
  +------+              +------+
```

5-Port diagram:

```
  +------+              +------+
0-|s0==s2|-2           0-|d0==d1|-1
1-|s1==s3|-3 =se2gmm=> 2-|c0==c1|-3
  |    s4|-4            |    s4|-4
  +------+              +------+
```

Note: The port s4 remain in single-mode.

8-Port diagram:

```
  +------+               +------+
0-|s0==s2|-2           0-|d0==d1|-1
1-|s1==s3|-3           2-|d2==d3|-3
  |      |   =se2gmm=>   |      |
4-|s4==s6|-6           4-|c0==c1|-5
5-|s5==s7|-7           6-|c2==c3|-7
  +------+               +------+
```

2N-Port diagram:

```
    A                                B
    +------------+                   +-----------+
  0-|s0========s2|-2               0-|d0=======d1|-1
  1-|s1========s3|-3               2-|d2=======d3|-3
   ...          ...     =se2gmm=>    ...         ...
2N-4-|s2N-4==s2N-2|-2N-2          2N-4-|cN-4===cN-3|-2N-3
2N-3-|s2N-3==s2N-1|-2N-1          2N-2-|cN-2===cN-1|-2N-1
    +------------+                   +-----------+
```

Note: The network *A* is not cascadable with the ** operator along transmission path.

References

See also

gmm2se

skrf.network.Network.set_noise_a

`Network.`**`set_noise_a`**(*noise_freq=None*, *nfmin_db=0*, *gamma_opt=0*, *rn=1*)

Set the "A" (ie cascade) representation of the correlation matrix, based on the noise frequency and input parameters.

Parameters

- **noise_freq** (*Frequency*)
- **nfmin_db** (*float*)
- **gamma_opt** (*float*)
- **rn** (*Number | Sequence[Number] | ndarray*)

Return type

None

skrf.network.Network.stability_circle

`Network.`**`stability_circle`**(*target_port*, *npoints=181*)

Returns loci of stability circles for a given port (0 or 1). The network must have two ports. The center and radius

of the load (here target_port=1) stability circle are calculated by the following equation[1].

$$C_L = \frac{(S_{22} - DS_{11}^*)^*}{|S_{22}|^2 - |D|^2}$$
$$R_L = |\frac{S_{12}S_{21}}{|S_{22}|^2 - |D|^2}|$$
$$with$$
$$D = S_{11}S_{22} - S_{12}S_{21}$$

Similarly, those of the source side (here target_port=0) are calculated by the following equations.

$$C_S = \frac{(S_{11} - DS_{22}^*)^*}{|S_{11}|^2 - |D|^2}$$
$$R_S = |\frac{S_{12}S_{21}}{|S_{11}|^2 - |D|^2}|$$

Parameters

- **target_port** (*int*) – Specifies the port number (0 or 1) to calculate stability circles.
- **npoints** (*int, optional*) – The number of points on the circumference of the circle. More points result in a smoother circle, but require more computation. Default is 181.

Returns

sc – Loci of stability circles in complex numbers

Return type

numpy.ndarray (shape is *npoints x f*)

Example

```
>>> import skrf as rf
>>> import matplotlib.pyplot as plt
```

Create a two-port network object

```
>>> ntwk = rf.Network('fet.s2p')
```

Calculate the load stability circles for all the frequencies

```
>>> lsc = ntwk.stability_circle(target_port=1)
```

Plot the circles on the smith chart

```
>>> rf.plotting.plot_smith(s=lsc, smith_r=5, marker='o')
>>> plt.show()
```

Slicing the network allows you to specify a frequency

```
>>> lsc = ntwk['1GHz'].stability_circle(target_port=1)
>>> rf.plotting.plot_smith(s=lsc, smith_r=5, marker='o')
>>> plt.show()
```

[1] David. M. Pozar, "Microwave Engineering, Fourth Edition," Wiley, p. 566, 2011.

References

See also

gain_circle, *nf_circle*, *stability*

skrf.network.Network.step_response

`Network.step_response`(*window='hamming'*, *n=None*, *pad=0*, *squeeze=True*)

Calculate time-domain step response of one-port.

First frequency must be 0 Hz for the transformation to be accurate and the frequency step must be uniform.

Real measurements should be extrapolated to DC and interpolated to uniform frequency spacing.

Y-axis is the reflection coefficient. *step_resonse* is equal to the cumulative trapezoid integration of the *impulse_response* function.

Parameters

- **window** (*string*) – FFT windowing function. (Default is 'hamming')
- **n** (*int*) – Length of step response output. If n is not specified, 2m - 1 points are used in low-pass mode where m = len(self) + pad. (default is None).
- **pad** (*int*) – Number of zeros to add as padding for FFT. Adding more zeros improves accuracy of peaks. (Default is 0)
- **squeeze** (*bool*) – Squeeze step response to one dimension, if a oneport gets transformed. Has no effect when transforming a multiport. (Default is True)

Returns

- **t** (class:*np.ndarray*) – Time vector
- **y** (class:*np.ndarray*) – Step response

Raises

- **ValueError** – If used with an Network with more than one port
- **NotImplementedError** – If used with non equidistant sampled frequency vector

Return type

tuple[*ndarray*, *ndarray*]

skrf.network.Network.subnetwork

`Network.subnetwork`(*ports*, *offby=1*)

Return a subnetwork of a the Network from a list of port numbers.

A subnetwork is Network which S-parameters corresponds to selected ports, with all non-selected ports considered matched.

The resulting subNetwork is given a new Network.name property from the initial name and adding the kept ports indices (ex: 'device' -> 'device13'). Such name should make easier the use of functions such as n_twoports_2_nport.

Parameters

- **ports** (*list of int or str*) – List of ports to keep in the resultant Network. Ports can be given by name if the Network has named ports (*Network.port_names*). Indices are the Python indices (starts at 0)
- **offby** (*int*) – starting value for s-parameters indexes in the sub-Network name parameter. A value of *1*, assumes that a s21 = ntwk.s[:,1,0]. Default is 1.

Returns
subntw – Resulting subnetwork of the Network from the given ports

Return type
Network object

See also

subnetwork, *n_twoports_2_nport*

skrf.network.Network.time_gate

`Network.time_gate(*args, **kw)`

Time gate this Network.

Returns

- **ntwk** (*Network* object) – Resulting time-gated Network
- see *skrf.time_domain.time_gate*

Return type
Network

skrf.network.Network.to_dataframe

`Network.to_dataframe(attrs=None, ports=None, port_sep=None)`

Convert attributes of a Network to a pandas DataFrame.

Use the same parameters than *skrf.io.general.network_2_dataframe()*

Parameters

- **attrs** (*list of string*) – Network attributes to convert, like ['s_db','s_deg']
- **ports** (*list of tuples*) – list of port pairs to write. defaults to ntwk.port_tuples (like [[0,0]])
- **port_sep** (*string*) – defaults to None, which means a empty string "" is used for networks with lower than 10 ports. (s_db 11, s_db 21) For more than ten ports a "_" is used to avoid ambiguity. (s_db 1_1, s_db 2_1) For consistent behaviour it's recommended to specify "_" or "," explicitly.

Returns
df

Return type
pandas.DataFrame

See also

skrf.io.general.network_2_dataframe

skrf.network.Network.vswr_active

`Network.`**`vswr_active`**`(`a`)`

Return the active VSWR of the network for a defined wave excitation a.

The active VSWR is defined by :

$$\text{active}(vswr)_m = \frac{1 + |\text{active}(s)_m|}{1 - |\text{active}(s)_m|}$$

where $\text{active}(s)_m$ the active S-parameter of port m.

Parameters
a (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns
vswr_act – active VSWR for the excitation a

Return type
complex array of shape (nfreqs, *nports*)

See also

s_active
active S-parameters

z_active
active Z-parameters

y_active
active Y-parameters

skrf.network.Network.windowed

`Network.`**`windowed`**`(`*window=('kaiser', 6)*, *normalize=True*, *center_to_dc=None*`)`

Return a windowed version of s-matrix. Used in time-domain analysis.

When using time domain through *s_time_db*, or similar properties, the spectrum is usually windowed, before the IFFT is taken. This is done to compensate for the band-pass nature of a spectrum[1].

This function calls `scipy.signal.get_window()` which gives more details about the windowing or a custom window function with the required length as parameter.

Parameters

- **window** (*string, float, tuple or callable*) – The type of window to create. See `scipy.signal.get_window()` for details.
- **normalize** (*bool*) – Normalize the window to preserve power. ie sum(ntwk.s,axis=0) == sum(ntwk.windowed().s,axis=0)

[1] Agilent Time Domain Analysis Using a Network Analyzer Application Note 1287-12

- **center_to_dc** (*bool or None*) – If True only the positive half of the window is applied to the signal. This should be used if frequency vector begins from DC or from "close enough" to DC. If False full window is used which also attenuates low frequencies. If None then value is determined automatically based on if frequency vector begins from 0.

Returns
ntwk – Resulting windowed Network

Return type
Network object

Examples

```
>>> ntwk = rf.Network('myfile.s2p')
>>> ntwk_w = ntwk.windowed()
>>> ntwk_w.plot_s_time_db()
```

References

skrf.network.Network.write

Network.**write**(*file=None*, **args*, ***kwargs*)

Write the Network to disk using the `pickle` module.

The resultant file can be read either by using the Networks constructor, *__init__()* , the read method *read()*, or the general read function *skrf.io.general.read()*

Parameters

- **file** (*str, Path, or file-object*) – filename or a file-object. If left as None then the filename will be set to Network.name, if its not None. If both are None, ValueError is raised.
- ***args** – passed through to *write()*
- ****kwargs** – passed through to *write()*

Return type
None

Note

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> n = rf.N(f=[1,2,3],s=[1,1,1],z0=50, name = 'open')
>>> n.write()
>>> n2 = rf.read('open.ntwk')
```

See also

skrf.io.general.write
write any skrf object

skrf.io.general.read
read any skrf object

skrf.network.Network.write_spreadsheet

Network.**write_spreadsheet**(**args*, ***kwargs*)

Write contents of network to a spreadsheet, for your boss to use.

See also

skrf.io.general.network_2_spreadsheet

Return type
None

skrf.network.Network.write_to_json_string

Network.**write_to_json_string**()

Serialize and convert network to a JSON string.

This is ~3x faster than writing to and reading back from touchstone for a 4port 20,000 point device.

Returns
jsonstr – JSON string

Return type
string

See also

skrf.io.general.to_json_string

skrf.network.Network.write_touchstone

Network.**write_touchstone**(*filename=None*, *dir=None*, *write_z0=False*, *skrf_comment=True*, *return_string=False*, *to_archive=None*, *form='ri'*, *format_spec_A='{}'*, *format_spec_B='{}'*, *format_spec_freq='{}'*, *r_ref=None*, *format_spec_nf_freq='{}'*, *format_spec_nf_min='{}'*, *format_spec_g_opt_mag='{}'*, *format_spec_g_opt_phase='{}'*, *format_spec_rn='{}'*, *write_noise=True*, *parameter='S'*, *version='1.0'*, *encoding='ISO-8859-1'*)

Write a contents of the *Network* to a touchstone file.

Parameters

- **filename** (*a string or Path, optional*) – touchstone filename, without extension. if 'None', then will use the network's `name`.
- **dir** (*string or Path, optional*) – the directory to save the file in.
- **write_z0** (*boolean*) – write impedance information into touchstone as comments, like Ansoft HFSS does
- **skrf_comment** (*bool, optional*) – write *created by skrf <version>* comment

- **return_string** (*bool, optional*) – return the file_string rather than write to a file
- **to_archive** (*zipfile.Zipfile*) – opened ZipFile object to place touchstone file in
- **form** (*string*) – format to write data: 'db': db, deg. 'ma': mag, deg. 'ri': real, imag.
- **format_spec_A** (*string, optional*) – Any valid format specifying string as given by https://docs.python.org/3/library/string.html#format-string-syntax This specifies the formatting in the resulting touchstone file for the A part of the S parameter, (e.g. the dB magnitude for 'db' format, the linear magnitude for 'ma' format, or the real part for 'ri' format)
- **format_spec_B** (*string, optional*) – Any valid format specifying string as given by https://docs.python.org/3/library/string.html#format-string-syntax This specifies the formatting in the resulting touchstone file for the B part of the S parameter, (e.g. the angle in degrees for 'db' format, the angle in degrees for 'ma' format, or the imaginary part for 'ri' format)
- **format_spec_freq** (*string, optional*) – Any valid format specifying string as given by https://docs.python.org/3/library/string.html#format-string-syntax This specifies the formatting in the resulting touchstone file for the frequency.
- **r_ref** (*float*) – Reference impedance to renormalize the network. If None network port impedance is used if possible. If None and network port impedance is complex and not equal at all ports and frequency points raises ValueError.
- **format_spec_nf_freq** (*string, optional*) – Any valid format specifying string as given by https://docs.python.org/3/library/string.html#format-string-syntax This specifies the formatting in the resulting touchstone file for the noise data frequency.
- **format_spec_nf_min** (*string, optional*) – Any valid format specifying string as given by https://docs.python.org/3/library/string.html#format-string-syntax This specifies the formatting in the resulting touchstone file for the minimum NF.
- **format_spec_g_opt_mag** (*string, optional*) – Any valid format specifying string as given by https://docs.python.org/3/library/string.html#format-string-syntax This specifies the formatting in the resulting touchstone file for the GammaOpt magnitude.
- **format_spec_g_opt_phase** (*string, optional*) – Any valid format specifying string as given by https://docs.python.org/3/library/string.html#format-string-syntax This specifies the formatting in the resulting touchstone file for the GammaOpt phase.
- **format_spec_rn** (*string, optional*) – Any valid format specifying string as given by https://docs.python.org/3/library/string.html#format-string-syntax This specifies the formatting in the resulting touchstone file for the noise resistance.
- **write_noise** (*bool, optional*) – Write noise parameters.
- **parameter** (*string*) – Specify the network parameter ("S", "Y", "Z", "G", "H") to write, defaults to "S". "G" and "H" is only available for 2-port Networks.
- **version** (*string*) – Touchstone file format version. Supported versions are "1.0", "2.0", and "2.1". Version "1.0" preserves the legacy output format.
- **encoding** (*string*) – File encoding. Supported encodings are ISO-8859-1 and UTF-8.

Return type
str | None

Note

Format supported at the moment are [Hz/kHz/MHz/GHz] S [DB/MA/RI] Frequency unit can be changed by setting Network.frequency.unit property

Note

The functionality of this function should take place in the *Touchstone* class.

skrf.network.Network.y_active

`Network.y_active`(a)

Return the active Y-parameters of the network for a defined wave excitation a.

The active Y-parameters are defined by:

$$\text{active}(y)_m = y_{0,m} \frac{1 - \text{active}(s)_m}{1 + \text{active}(s)_m}$$

where $y_{0,m}$ is the characteristic admittance and $\text{active}(s)_m$ the active S-parameter of port m.

Parameters
a (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns
y_act – active Y-parameters for the excitation a

Return type
complex array of shape (nfreqs, *nports*)

See also

s_active
active S-parameters

z_active
active Z-parameters

vswr_active
active VSWR

skrf.network.Network.z_active

`Network.z_active`(a)

Return the active Z-parameters of the network for a defined wave excitation a.

The active Z-parameters are defined by:

$$\text{active}(z)_m = z_{0,m} \frac{1 + \text{active}(s)_m}{1 - \text{active}(s)_m}$$

where $z_{0,m}$ is the characteristic impedance and $\text{active}(s)_m$ the active S-parameter of port m.

Parameters
a (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns
z_act – active Z-parameters for the excitation a

Return type
complex array of shape (nfreqs, *nports*)

See also

s_active
active S-parameters

y_active
active Y-parameters

vswr_active
active VSWR

skrf.network.Network.zipped_touchstone

classmethod Network.**zipped_touchstone**(*filename*, *archive*, *encoding=None*)

Read a Network from a Touchstone file in a ziparchive.

Parameters

- **filename** (*str*) – the full path filename of the touchstone file
- **archive** (*zipfile.ZipFile*) – the opened zip archive
- **encoding** (*str, optional*) – File encoding. Supported encodings are ISO-8859-1 and UTF-8. If omitted, UTF-8 is attempted before ISO-8859-1.

Returns
ntwk – Network from the Touchstone file

Return type
Network

Building Network

Network.from_z(z, *args, **kw)	Create a Network from its Z-parameters.

3.1.2 Network Representations

Network.s	Scattering parameter matrix.
Network.z	Impedance parameter matrix.
Network.y	Admittance parameter matrix.
Network.a	abcd parameter matrix.
Network.t	Scattering transfer parameter matrix.

3.1.3 Connecting Networks

connect(ntwkA, k, ntwkB, l[, num])	Connect two n-port networks together.
innerconnect(ntwkA, k, l[, num])	Connect ports of a single n-port network.
cascade(ntwkA, ntwkB)	Cascade two 2, 2N-ports Networks together.
cascade_list(l)	Cascade a list of 2N-port networks.
de_embed(ntwkA, ntwkB)	De-embed *ntwkA* from *ntwkB*.
flip(a)	Invert the ports of a networks s-matrix, 'flipping' it over left and right.
inv(s)	Calculate 'inverse' s-parameter matrix, used for de-embedding.
parallelconnect(ntwks, ports[, name])	Connects a series of multi-port networks in parallel, ensuring that the specified port indices share the concatenated intersection.

skrf.network.connect

skrf.network.**connect**(*ntwkA*, *k*, *ntwkB*, *l*, *num=1*)

Connect two n-port networks together.

Connect ports *k* thru *k+num-1* on *ntwkA* to ports *l* thru *l+num-1* on *ntwkB*. The resultant network has (ntwkA.nports+ntwkB.nports-2*num) ports. The port indices ('k','l') start from 0. Ports of networks which have named ports (*Network.port_names*) can be given by name instead of by index. Port impedances **are** taken into account. When the two networks have overlapping frequencies, the resulting network will contain only the overlapping frequencies.

Note

The effect of mis-matched port impedances is handled by inserting a 2-port 'mismatch' network between the two connected ports. This mismatch Network is calculated with the *impedance_mismatch()* function.

Parameters

- **ntwkA** (*Network*) – network 'A'
- **k** (*int or str*) – starting port index on *ntwkA* (port indices start from 0), or port name if *ntwkA* has named ports
- **ntwkB** (*Network*) – network 'B'
- **l** (*int or str*) – starting port index on *ntwkB*, or port name if *ntwkB* has named ports
- **num** (*int*) – number of consecutive ports to connect (default 1)

Returns

ntwkC – new network of rank (ntwkA.nports + ntwkB.nports - 2*num)

Return type

Network

See also

connect_s
actual S-parameter connection algorithm.

innerconnect_s
actual S-parameter connection algorithm.

innerconnect_s_lstsq
actual S-parameter connection algorithm using lstsq.

Examples

To implement a *cascade* of two networks

```
>>> ntwkA = rf.Network('ntwkA.s2p')
>>> ntwkB = rf.Network('ntwkB.s2p')
>>> ntwkC = rf.connect(ntwkA, 1, ntwkB,0)
```

Ports of networks with named ports can be given by name

```
>>> ntwkA.port_names
['in', 'out']
>>> ntwkC = rf.connect(ntwkA, 'out', ntwkB, 0)
```

skrf.network.innerconnect

skrf.network.**innerconnect**(*ntwkA*, *k*, *l*, *num=1*)

Connect ports of a single n-port network.

this results in a (n-2)-port network. remember port indices start from 0. Ports of a network which has named ports (*Network.port_names*) can be given by name instead of by index.

Note

A 2-port 'mismatch' network is inserted between the connected ports if their impedances are not equal.

Parameters

- **ntwkA** (*Network*) – network 'A'
- **k** (*int or str*) – starting port indices on ntwkA (port indices start from 0), or port names if *ntwkA* has named ports
- **l** (*int or str*) – starting port indices on ntwkA (port indices start from 0), or port names if *ntwkA* has named ports
- **num** (*int*) – number of consecutive ports to connect

Returns
ntwkC – new network of rank (ntwkA.nports - 2*num)

Return type
Network

See also

connect_s
actual S-parameter connection algorithm.

innerconnect_s
actual S-parameter connection algorithm.

innerconnect_s_lstsq
actual S-parameter connection algorithm using lstsq.

Examples

To connect ports '0' and port '1' on ntwkA

```
>>> ntwkA = rf.Network('ntwkA.s3p')
>>> ntwkC = rf.innerconnect(ntwkA, 0,1)
```

skrf.network.cascade

skrf.network.**cascade**(*ntwkA*, *ntwkB*)

Cascade two 2, 2N-ports Networks together.

Connects ports N through 2N-1 on *ntwkA* to ports 0 through N of *ntwkB*. This calls *connect()*, which is a more general function. Use *Network.renumber* to change port order if needed.

Note

connection diagram:

```
    A                 B
 +----------+   +----------+
-|0       N |---|0       N |-
-|1      N+1|---|1      N+1|-
...        ... ...        ...
-|N-2   2N-2|---|N-2   2N-2|-
-|N-1   2N-1|---|N-1   2N-1|-
 +----------+   +----------+
```

Parameters

- **ntwkA** (*Network*) – network *ntwkA*
- **ntwkB** (*Network*) – network *ntwkB*

Returns
C – the resultant network of ntwkA cascaded with ntwkB

Return type
Network

See also

connect
connects two Networks together at arbitrary ports.

Network.renumber
changes the port order of a network

skrf.network.cascade_list

skrf.network.**cascade_list**(*l*)

Cascade a list of 2N-port networks.

all networks must have same frequency

Parameters
l (*list-like*) – (ordered) list of networks

Returns
out – the results of cascading all networks in the list *l*

Return type
2-port Network

skrf.network.de_embed

skrf.network.**de_embed**(*ntwkA*, *ntwkB*)

De-embed *ntwkA* from *ntwkB*.

This calls *ntwkA.inv ** ntwkB*. The syntax of cascading an inverse is more explicit, it is recommended that it be used instead of this function.

Parameters

- **ntwkA** (*Network*) – network *ntwkA*
- **ntwkB** (*Network*) – network *ntwkB*

Returns
C – the resultant network of ntwkB de-embedded from ntwkA

Return type
Network

See also

connect
connects two Networks together at arbitrary ports.

skrf.network.flip

skrf.network.**flip**(*a*)

Invert the ports of a networks s-matrix, 'flipping' it over left and right.

In case the network is 2n-port and n > 1, 'second' numbering scheme is assumed to be consistent with the ** cascade operator:

```
   +--------+                +--------+
 0-|0      n|-n            0-|n      0|-n
 1-|1    n+1|-n+1   flip    1-|n+1    1|-n+1
  ...      ...       =>      ...      ...
n-1-|n-1 2n-1|-2n-1        n-1-|2n-1 n-1|-2n-1
   +--------+                +--------+
```

Parameters
a (*numpy.ndarray*) – scattering parameter matrix. shape should be *2nx2n*, or *fx2nx2n*

Returns
c – flipped scattering parameter matrix

Return type
numpy.ndarray

See also

renumber

skrf.network.inv

skrf.network.**inv**(*s*)

Calculate 'inverse' s-parameter matrix, used for de-embedding.

This is not literally the inverse of the s-parameter matrix. Instead, it is defined such that the inverse of the s-matrix cascaded with itself is a unity scattering transfer parameter (T) matrix.

$$inv(s) = t2s(s2t(s)^{-1})$$

where x^{-1} is the matrix inverse. In words, this is the inverse of the scattering transfer parameters matrix transformed into a scattering parameters matrix.

Parameters
s (numpy.ndarray (shape fx2nx2n)) – scattering parameter matrix.

Returns
s' – inverse scattering parameter matrix.

Return type
numpy.ndarray

See also

t2s
converts scattering transfer parameters to scattering parameters

s2t
converts scattering parameters to scattering transfer parameters

skrf.network.parallelconnect

skrf.network.**parallelconnect**(*ntwks*, *ports*, *name=None*)

Connects a series of multi-port networks in parallel, ensuring that the specified port indices share the concatenated intersection.

Parameters

- **ntwks** (:Sequence[*Network*] | *Network*) – A sequence of multi-port networks or a single network to be connected in parallel.
- **ports** (*Sequence[int | str | Sequence[int | str]]*) – A sequence of port indices, where each entry can be an int or a sequence of ints corresponding to the ports of the respective network. The length of *ports* should match the length of *networks*. Each specified port index is connect to the concatenated intersection, implying they are electrically

common. Ports of networks which have named ports (*Network.port_names*) can be given by name instead of by index.

- **name** (*str, optional*) – define the connected network's name. Default is None.

Returns

connected_network – A new network created from the parallel connection of the input networks. The number of ports in the resulting network equals the sum of ports in *ntwks*, minus the number of ports specified in *ports* that were connected in parallel. The remaining ports follow the original port order of the input networks, after removing those involved in the parallel connection.

Return type

Network

Note

This function calculates the resulting scattering parameters after parallel connecting a set of networks. This algorithm[1], adapted from the *Circuit.s* method, constructs the concatenated intersection matrix [X] and the global scattering matrix [C] to perform the calculations.

See also

connect_s
actual S-parameter connection algorithm.

innerconnect_s
actual S-parameter connection algorithm.

innerconnect_s_lstsq
actual S-parameter connection algorithm using lstsq.

Examples

The following examples demonstrate how to use the *parallelconnect()* in different ways, such as open a port, connect two networks, innerconnect one network, and parallel multiple networks.

```
>>> # Prepare the example Networks
>>> ntwkA = rf.Network('ntwkA.s2p')
>>> ntwkB = rf.Network('ntwkB.s2p')
>>> ntwkC = rf.connect('ntwkC.s4p')
>>>
>>> # 1) Open port 1 of ntwkA
>>> #    +-----+
>>> #   [0] A [1]--Open
>>> #    +-----+
>>> #
>>> rf.parallelconnect(ntwkA, [1])
>>>
>>>
>>> # 2) Connect ntwkA's port 1 to ntwkB's port 0
```

(continues on next page)

[1] P. Hallbjörner, Microw. Opt. Technol. Lett. 38, 99 (2003).

(continued from previous page)

```
>>> #    +-----+     +-----+
>>> #   [0] A [1]---[0] B [1]
>>> #    +-----+     +-----+
>>> #
>>> rf.parallelconnect([ntwkA, ntwkB], [1, 0])
>>>
>>>
>>> # 3) Innerconnect the ntwkC's port 1 and 3
>>> #  +-----+
>>> #  |    [1]--+
>>> # [0] C [2]  |
>>> #  |    [3]--+
>>> #  +-----+
>>> #
>>> rf.parallelconnect(ntwkC, [[2, 3]])
>>>
>>>
>>> # 4) Parallel connect the ntwkA's port 1, ntwkB's port 1 and ntwkC's port 0
>>> #  +-----+
>>> # [0] A [1]---+    +-----+
>>> #  +-----+    |    |    [1]
>>> #             |---[0] C [2]
>>> #  +-----+    |    |    [3]
>>> # [0] B [1]---+    +-----+
>>> #  +-----+
>>> #
>>> rf.parallelconnect([ntwkA, ntwkB, ntwkC], [1, 1, 0])
>>>
>>>
>>> # 5) The port order of connected ntwk follows the order of ntwks and ports
>>> # as shown in example 4:
>>> #  +-----+
>>> # [0] A [1]---+    +-----+          +----------------+
>>> #  +-----+    |    |    [1]        [0]=A[0]   C[3]=[3]
>>> #             |---[0] C [2]  ===>  [1]=B[0]         |
>>> #  +-----+    |    |    [3]        [2]=C[1]   C[3]=[4]
>>> # [0] B [1]---+    +-----+          +----------------+
>>> #  +-----+
```

References

3.1.4 Interpolation and Concatenation Along Frequency Axis

stitch(ntwkA, ntwkB, **kwargs)	Stitch ntwkA and ntwkB together.
overlap(ntwkA, ntwkB)	Return the overlapping parts of two Networks, interpolating if needed.
overlap_multi(ntwk_list)	Return the overlapping parts of multiple Networks, interpolating if needed.
Network.resample(freq_or_n, **kwargs)	Interpolate the current Network along frequency axis (inplace).
Network.interpolate(freq_or_n[, basis, ...])	Interpolate a Network along frequency axis.

continues on next page

Table 7 – continued from previous page

Network.interpolate_self(freq_or_n, **kwargs)	Interpolate the current Network along frequency axis (inplace).

skrf.network.stitch

skrf.network.**stitch**(*ntwkA*, *ntwkB*, ***kwargs*)

Stitch ntwkA and ntwkB together.

Concatenates two networks' data. Given two networks that cover different frequency bands this can be used to combine their data into a single network.

Parameters

- **ntwkA** (*Network* objects) – Networks to stitch together
- **ntwkB** (*Network* objects) – Networks to stitch together
- ****kwargs** (*keyword args*) – passed to *Network* constructor, for output network

Returns

ntwkC – result of stitching the networks *ntwkA* and *ntwkB* together

Return type

Network

Examples

```
>>> from skrf.data import wr2p2_line, wr1p5_line
>>> rf.stitch(wr2p2_line, wr1p5_line)
2-Port Network: 'wr2p2,line',  330-750 GHz, 402 pts, z0=[ 50.+0.j  50.+0.j]
```

skrf.network.overlap

skrf.network.**overlap**(*ntwkA*, *ntwkB*)

Return the overlapping parts of two Networks, interpolating if needed.

If frequency vectors for each ntwk don't perfectly overlap, then ntwkB is interpolated so that the resultant networks have identical frequencies.

Parameters

- **ntwkA** (*Network*) – a ntwk which overlaps *ntwkB*. (the *dominant* network)
- **ntwkB** (*Network*) – a ntwk which overlaps *ntwkA*.

Returns

- **ntwkA_new** (*Network*) – part of *ntwkA* that overlapped *ntwkB*
- **ntwkB_new** (*Network*) – part of *ntwkB* that overlapped *ntwkA*, possibly interpolated

Return type

tuple[Network, Network]

See also

skrf.frequency.overlap_freq(), *skrf.network.overlap_multi()*

skrf.network.overlap_multi

skrf.network.**overlap_multi**(*ntwk_list*)

Return the overlapping parts of multiple Networks, interpolating if needed.

If frequency vectors for each ntwk don't perfectly overlap, then all networks after the first are interpolated so that the resultant networks have identical frequencies.

Parameters
ntwk_list (*list of skrf.Networks*) – a list of networks with some overlap

Returns
overlap_list – a list of networks that mutually overlap

Return type
list of skrf.Networks

See also

skrf.frequency.overlap_freq(), *skrf.network.overlap()*

3.1.5 Combining and Splitting Networks

subnetwork(ntwk, ports[, offby])	Return a subnetwork of a given Network from a list of port numbers.
one_port_2_two_port(ntwk)	Calculate the 2-port network given a symmetric, reciprocal and lossless 1-port network.
twoport_to_nport(ntwk, port1, port2, nports, ...)	Add ports to two-port.
four_oneports_2_twoport(s11, s12, s21, s22, ...)	Build a 2-port Network from list of four 1-ports.
n_oneports_2_nport(ntwk_list, *args, **kwargs)	Build an N-port Network from list of N one-ports.
n_twoports_2_nport(ntwk_list, nports[, ...])	Build an N-port Network from list of two-ports.
concat_ports(ntwk_list[, port_order])	Concatenate networks along the port axis.

skrf.network.subnetwork

skrf.network.**subnetwork**(*ntwk*, *ports*, *offby=1*)

Return a subnetwork of a given Network from a list of port numbers.

A subnetwork is Network which S-parameters corresponds to selected ports, with all non-selected ports considered matched.

The resulting subNetwork is given a new Network.name property from the initial name and adding the kept ports indices (ex: 'device' -> 'device13'). Such name should make easier the use of functions such as n_twoports_2_nport.

Parameters

- **ntwk** (*Network* object) – Network to split into a subnetwork
- **ports** (*list of int or str*) – List of ports to keep in the resultant Network. Ports can be given by name if the Network has named ports (*Network.port_names*). Indices are the Python indices (starts at 0)
- **offby** (*int*) – starting value for s-parameters indexes in the sub-Network name parameter. A value of *1*, assumes that a s21 = ntwk.s[:,1,0]. Default is 1.

Returns
subntwk – Resulting subnetwork from the given ports

Return type
Network object

See also

Network.subnetwork, *n_twoports_2_nport*

Examples

```
>>> tee = rf.data.tee  # 3 port Network
>>> tee12 = rf.subnetwork(tee, [0, 1])  # 2 port Network from ports 1 & 2, port 3␣
↪matched
>>> tee23 = rf.subnetwork(tee, [1, 2])  # 2 port Network from ports 2 & 3, port 1␣
↪matched
>>> tee13 = rf.subnetwork(tee, [0, 2])  # 2 port Network from ports 1 & 3, port 2␣
↪matched
```

skrf.network.one_port_2_two_port

skrf.network.**one_port_2_two_port**(*ntwk*)

Calculate the 2-port network given a symmetric, reciprocal and lossless 1-port network.

Parameters
ntwk (*Network*) – a symmetric, reciprocal and lossless one-port network.

Returns
ntwk – the resultant two-port Network

Return type
Network

skrf.network.twoport_to_nport

skrf.network.**twoport_to_nport**(*ntwk*, *port1*, *port2*, *nports*, ***kwargs*)

Add ports to two-port. S-parameters of added ports are all zeros.

Parameters

- **ntwk** (*Two-port Network object*)
- **port1** (*int*) – First port of the two-port in the resulting N-port.
- **port2** (*int*) – Second port of the two-port in the resulting N-port.
- **nports** (*int*) – Number of ports in the N-port network.
- ****kwargs** – Passed to *Network.__init__()* for resultant network.

Returns
nport

Return type
N-port Network object

skrf.network.four_oneports_2_twoport

skrf.network.**four_oneports_2_twoport**(*s11*, *s12*, *s21*, *s22*, **args*, ***kwargs*)

Build a 2-port Network from list of four 1-ports.

Parameters

- **s11** (one-port *Network*) – s11
- **s12** (one-port *Network*) – s12
- **s21** (one-port *Network*) – s21
- **s22** (one-port *Network*) – s22
- ***args** – passed to *Network.__init__()* for the twoport
- ****kwargs** – passed to *Network.__init__()* for the twoport

Returns

twoport – result

Return type

two-port *Network*

See also

n_oneports_2_nport

skrf.network.n_oneports_2_nport

skrf.network.**n_oneports_2_nport**(*ntwk_list*, **args*, ***kwargs*)

Build an N-port Network from list of N one-ports.

Parameters

- **ntwk_list** (list of *Network* objects) – must follow left-right, top-bottom order, ie, s11,s12,s21,s22
- ***args** – passed to *Network.__init__()* for the N-port
- ****kwargs** – passed to *Network.__init__()* for the N-port

Returns

nport – result

Return type

n-port *Network*

skrf.network.n_twoports_2_nport

skrf.network.**n_twoports_2_nport**(*ntwk_list*, *nports*, *offby=1*, *port_sep=''*, ***kwargs*)

Build an N-port Network from list of two-ports.

This method was made to reconstruct an n-port network from 2-port subnetworks as measured by a 2-port VNA. So, for example, given a 3-port DUT, you might measure the set p12.s2p, p23.s2p, p13.s2p. From these measurements, you can construct p.s3p.

By default all entries of result.s are filled with 0's, in case you don't fully specify the entire s-matrix of the resultant ntwk.

Parameters

- **ntwk_list** (list of *Network* objects) – the names must contain the port index, ie 'p12' or 'p43', ie. define the Network.name property of the *Network* object.
- **nports** (*int*) – Number of ports to expect by the parser.
- **offby** (*int*) – starting value for s-parameters indices. ie a value of *1*, assumes that a s21 = ntwk.s[:,1,0]
- **port_sep** (*str, default ""*) – string separating port 1 connection from port 2 for the vna connected to the DUT. If constructing nport network with a maximum of 10 ports, it can be left as "". To avoid ambiguity for more than 10 ports, port_sep is required to format the trace names like S{}
- ***args** – passed to *Network.__init__()* for the N-port
- ****kwargs** – passed to *Network.__init__()* for the N-port

Returns
nport – result

Return type
n-port *Network*

See also

concat_ports
concatenate ntwks along their ports

skrf.network.concat_ports

`skrf.network.`**concat_ports**(*ntwk_list*, *port_order='second'*, **args*, ***kw*)

Concatenate networks along the port axis.

Note

The *port_order* ='first', means front-to-back, while *port_order`='second' means left-to-right. So, for example, when concatenating two 2-networks, `A* and *B*, the ports are ordered as follows:

'first'
a0 o—o a1 -> 0 o—o 1 b0 o—o b1 -> 2 o—o 3

'second'
a0 o—o a1 -> 0 o—o 2 b0 o—o b1 -> 1 o—o 3

use *Network.renumber* to change port ordering.

Parameters

- **ntwk_list** (*list of skrf.Networks*) – ntwks to concatenate
- **port_order** (*['first', 'second']*)

Return type
Network

Examples

```
>>> concat([ntwkA,ntwkB])
>>> concat([ntwkA,ntwkB,ntwkC,ntwkD], port_order='second')
```

To put for lines in parallel

```
>>> from skrf import air
>>> l1 = air.line(100, z0=[0,1])
>>> l2 = air.line(300, z0=[2,3])
>>> l3 = air.line(400, z0=[4,5])
>>> l4 = air.line(400, z0=[6,7])
>>> concat_ports([l1,l2,l3,l4], port_order='second')
```

See also

stitch
: concatenate two networks along the frequency axis

renumber
: renumber ports

3.1.6 IO

skrf.io.general.read(file, *args, **kwargs)	Read skrf object[s] from a pickle file.
skrf.io.general.write(file, obj[, overwrite])	Write skrf object[s] to a file.
skrf.io.general.network_2_spreadsheet(ntwk)	Write a Network object to a spreadsheet, for your boss.
Network.write([file])	Write the Network to disk using the `pickle` module.
Network.write_touchstone([filename, dir, ...])	Write a contents of the *Network* to a touchstone file.
Network.read(*args, **kwargs)	Read a Network from a 'ntwk' file.
Network.write_spreadsheet(*args, **kwargs)	Write contents of network to a spreadsheet, for your boss to use.

3.1.7 Noise

Network.add_noise_polar(mag_dev, phase_dev, ...)	Adds a complex zero-mean gaussian white-noise.
Network.add_noise_polar_flatband(mag_dev, ...)	Add a flatband complex zero-mean gaussian white-noise signal of given standard deviations for magnitude and phase.
Network.multiply_noise(mag_dev, phase_dev, ...)	Multiply a complex bivariate gaussian white-noise signal of given standard deviations for magnitude and phase.

3.1.8 Supporting Functions

inv(s)	Calculate 'inverse' s-parameter matrix, used for de-embedding.
connect_s(A, k, B, l[, num])	Connect two n-port networks' s-matrices together.

continues on next page

Table 11 – continued from previous page

innerconnect_s(A, k, l)	Connect two ports of a single n-port network's s-matrix.
innerconnect_s_lstsq(A, k, l)	Connect two ports of a single n-port network's s-matrix using a least-squares solution.
s2s(s, z0, s_def_new, s_def_old)	Convert scattering parameters to scattering parameters with different definition.
s2z(s[, z0, s_def])	Convert scattering parameters [#]_ to impedance parameters [#]_.
s2y(s[, z0, s_def])	Convert scattering parameters [#]_ to admittance parameters [#]_.
s2t(s)	Convert scattering parameters [#]_ to scattering transfer parameters [#]_.
s2a(s[, z0])	Convert scattering parameters to abcd parameters [#]_.
s2g(s[, z0])	Convert inverse hybrid parameters to s parameters.
s2h(s[, z0])	Convert scattering parameters [#]_ to hybrid parameters [#]_.
z2s(z[, z0, s_def])	Convert impedance parameters [#]_ to scattering parameters [#]_.
z2y(z)	Convert impedance parameters [#]_ to admittance parameters [#]_.
z2t(z)	Not Implemented yet.
z2a(z)	Converts impedance parameters to abcd parameters [#]_.
y2s(y[, z0, s_def])	Convert admittance parameters [#]_ to scattering parameters [#]_.
y2z(y)	Convert admittance parameters [#]_ to impedance parameters [#]_.
y2t(y)	Not Implemented Yet.
t2s(t)	Converts scattering transfer parameters [1]_ to scattering parameters [2]_.
t2z(t)	Not Implemented Yet.
t2y(t)	Not Implemented Yet.
h2s(h[, z0])	Convert hybrid parameters to s parameters.
h2z(h)	Convert hybrid parameters to z parameters [#]_.
s2s_active(s, a)	Return active s-parameters for a defined wave excitation a.
s2z_active(s, z0, a)	Return the active Z-parameters for a defined wave excitation a.
s2y_active(s, z0, a)	Return the active Y-parameters for a defined wave excitation a.
s2vswr_active(s, a)	Return the active VSWR for a defined wave excitation a..
fix_z0_shape(z0, nfreqs, nports)	Make a port impedance of correct shape for a given network's matrix.
renormalize_s(s, z_old, z_new[, s_def, ...])	Renormalize a s-parameter matrix given old and new port impedances.
passivity(s)	Passivity metric for a network.
reciprocity(s)	Reciprocity metric for a multi-port network.

skrf.network.connect_s

skrf.network.**connect_s**(*A*, *k*, *B*, *l*, *num=1*)

Connect two n-port networks' s-matrices together.

Specifically, connect port *k* on network *A* to port *l* on network *B*. The resultant network has nports = (A.rank + B.rank-2). This function operates on, and returns s-matrices. The function *connect()* operates on *Network*

types.

Parameters

- **A** (numpy.ndarray) – S-parameter matrix of A, shape is fxnxn
- **k** (*int*) – port index on A (port indices start from 0)
- **B** (numpy.ndarray) – S-parameter matrix of B, shape is fxnxn
- **l** (*int*) – port index on B
- **num** (*int*) – number of consecutive ports to connect (default 1)

Returns

C – new S-parameter matrix

Return type

numpy.ndarray

Note

Internally, this function creates a larger composite network and calls the *innerconnect_s()* function. see that function for more details about the implementation

See also

connect
operates on *Network* types

innerconnect_s
function which implements the connection algorithm

innerconnect_s_lstsq
actual S-parameter connection algorithm using lstsq.

skrf.network.innerconnect_s

skrf.network.**innerconnect_s**(*A*, *k*, *l*)

Connect two ports of a single n-port network's s-matrix.

Specifically, connect port k to port l on A. This results in a (n-2)-port network. This function operates on, and returns s-matrices. The function *innerconnect()* operates on *Network* types.

Note

The algorithm used to calculate the resultant network is called a 'sub-network growth', can be found in[1]. The original paper describing the algorithm is given in[2].

Parameters

[1] Compton, R.C.; , "Perspectives in microwave circuit analysis," Circuits and Systems, 1989., Proceedings of the 32nd Midwest Symposium on , vol., no., pp.716-718 vol.2, 14-16 Aug 1989. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=101955&isnumber=3167

[2] Filipsson, Gunnar; , "A New General Computer Algorithm for S-Matrix Calculation of Interconnected Multiports" ,Microwave Conference, 1981. 11th European , vol., no., pp.700-704, 7-11 Sept. 1981. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4131699&isnumber=4131585

- **A** (numpy.ndarray) – S-parameter matrix of *A*, shape is fxnxn
- **k** (*int*) – port index on *A* (port indices start from 0)
- **l** (*int*) – port index on *A*

Returns
C – new S-parameter matrix

Return type
numpy.ndarray

See also

innerconnect_s_lstsq
actual S-parameter connection algorithm using lstsq.

References

skrf.network.innerconnect_s_lstsq

skrf.network.**innerconnect_s_lstsq**(*A*, *k*, *l*)

Connect two ports of a single n-port network's s-matrix using a least-squares solution. It uses a least-squares approach to handle cases where the determinant of the sub-matrix is close to zero, which can lead to numerical instability in the direct formula.

Note

If the determinant of the sub-matrix is not close to zero, it is recommended to use the *innerconnect_s* function instead, which uses a direct formula for better numerical stability.

Parameters

- **A** (numpy.ndarray) – S-parameter matrix of *A*, shape is fxnxn
- **k** (*int*) – port index on *A* (port indices start from 0)
- **l** (*int*) – port index on *A*

Returns
AEE – new S-parameter matrix

Return type
numpy.ndarray

See also

connect_s
actual S-parameter connection algorithm.

innerconnect_s
actual S-parameter connection algorithm.

skrf.network.s2s

skrf.network.**s2s**(*s*, *z0*, *s_def_new*, *s_def_old*)

Convert scattering parameters to scattering parameters with different definition.

Calculates port voltages and currents using the old definition and then calculates the incoming and reflected waves from the voltages using the new S-parameter definition.

Only has effect if z0 has at least one complex impedance port.

Parameters

- **s** (*complex array-like*) – impedance parameters
- **z0** (*complex array-like or number*) – port impedances
- **s_def_new** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition of the output network. 'power' for power-waves definition[1], 'pseudo' for pseudo-waves definition[2]. 'traveling' corresponds to the initial implementation.
- **s_def_old** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition of the input network. 'power' for power-waves definition[1], 'pseudo' for pseudo-waves definition[2]. 'traveling' corresponds to the initial implementation.

Returns

s – scattering parameters

Return type

complex array-like

References

skrf.network.s2z

skrf.network.**s2z**(*s*, *z0=50*, *s_def='power'*)

Convert scattering parameters[1] to impedance parameters[2].

For power-waves, Eq.(19) from[3]:

$$Z = F^{-1}(1-S)^{-1}(SG+G^{*})F$$

where $G = diag([Z_0])$ and $F = diag([1/2\sqrt{|Re(Z_0)|}])$

For pseudo-waves, Eq.(74) from[4]:

$$Z = (1-U^{-1}SU)^{-1}(1+U^{-1}SU)G$$

where $U = \sqrt{Re(Z_0)}/|Z_0|$

Parameters

- **s** (*complex array-like*) – scattering parameters
- **z0** (*complex array-like or number*) – port impedances.

[1] Kurokawa, Kaneyuki "Power waves and the scattering matrix", IEEE Transactions on Microwave Theory and Techniques, vol.13, iss.2, pp. 194–202, March 1965.

[2] Marks, R. B. and Williams, D. F. "A general waveguide circuit theory", Journal of Research of National Institute of Standard and Technology, vol.97, iss.5, pp. 533–562, 1992.

[1] http://en.wikipedia.org/wiki/S-parameters

[2] http://en.wikipedia.org/wiki/impedance_parameters

[3] Kurokawa, Kaneyuki "Power waves and the scattering matrix", IEEE Transactions on Microwave Theory and Techniques, vol.13, iss.2, pp. 194–202, March 1965.

[4] Marks, R. B. and Williams, D. F. "A general waveguide circuit theory", Journal of Research of National Institute of Standard and Technology, vol.97, iss.5, pp. 533–562, 1992.

- **s_def** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition : 'power' for power-waves definition[Page 815, 3], 'pseudo' for pseudo-waves definition[Page 815, 4]. 'traveling' corresponds to the initial implementation. Default is 'power'.

Returns
z – impedance parameters

Return type
complex array-like

References

skrf.network.s2y

`skrf.network.s2y(s, z0=50, s_def='power')`

Convert scattering parameters[1] to admittance parameters[2].

Equations are the inverse of *s2z()*.

Parameters

- **s** (*complex array-like*) – scattering parameters
- **z0** (*complex array-like or number*) – port impedances
- **s_def** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition : 'power' for power-waves definition[3], 'pseudo' for pseudo-waves definition[4]. 'traveling' corresponds to the initial implementation. Default is 'power'.

Returns
y – admittance parameters

Return type
complex array-like

See also

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.s2t

`skrf.network.s2t(s)`

Convert scattering parameters[1] to scattering transfer parameters[2].

transfer parameters are also referred to as 'wave cascading matrix'[3], this function only operates on 2N-ports networks with same number of input and output ports, also known as 'balanced networks'.

[1] http://en.wikipedia.org/wiki/S-parameters
[2] http://en.wikipedia.org/wiki/Admittance_parameters
[3] Kurokawa, Kaneyuki "Power waves and the scattering matrix", IEEE Transactions on Microwave Theory and Techniques, vol.13, iss.2, pp. 194–202, March 1965.
[4] Marks, R. B. and Williams, D. F. "A general waveguide circuit theory", Journal of Research of National Institute of Standard and Technology, vol.97, iss.5, pp. 533–562, 1992.
[1] http://en.wikipedia.org/wiki/S-parameters
[2] http://en.wikipedia.org/wiki/Scattering_transfer_parameters#Scattering_transfer_parameters
[3] Janusz A. Dobrowolski, "Scattering Parameter in RF and Microwave Circuit Analysis and Design", Artech House, 2016, pp. 65-68

Parameters
s (numpy.ndarray (shape fx2nx2n)) – scattering parameter matrix

Returns
t – scattering transfer parameters (aka wave cascading matrix)

Return type
np.ndarray

See also

inv
calculates inverse s-parameters

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.s2a

skrf.network.**s2a**(*s*, *z0=50*)

Convert scattering parameters to abcd parameters[1].

Parameters
- **s** (numpy.ndarray (shape *fx2x2*)) – impedance parameter matrix
- **z0** (number or, numpy.ndarray (shape *fx2*)) – port impedance

Returns
abcd – scattering transfer parameters (aka wave cascading matrix)

Return type
np.ndarray

References

skrf.network.s2g

skrf.network.**s2g**(*s*, *z0=50*)

Convert inverse hybrid parameters to s parameters.

Parameters
- **s** (*complex array-like*) – scattering parameters
- **z0** (*complex array-like or number*) – port impedances

Returns
g – inverse hybrid parameters

Return type
complex array-like

[1] https://en.wikipedia.org/wiki/Two-port_network

skrf.network.s2h

skrf.network.**s2h**(*s*, *z0=50*)

Convert scattering parameters[1] to hybrid parameters[2].

Parameters

- **s** (*complex array-like*) – scattering parameters
- **z0** (*complex array-like or number*) – port impedances.

Returns
h – hybrid parameters

Return type
complex array-like

References

skrf.network.z2s

skrf.network.**z2s**(*z*, *z0=50*, *s_def='power'*)

Convert impedance parameters[1] to scattering parameters[2].

For power-waves, Eq.(18) from[3]:

$$S = F(Z^{\cdot}G^{*})(Z+G)^{-1}F^{-1}$$

where $G = diag([Z_0])$ and $F = diag([1/2\sqrt{|Re(Z_0)|}])$

For pseudo-waves, Eq.(73) from[4]:

$$S = U(Z-G)(Z+G)^{-1}U^{-1}$$

where $U = \sqrt{Re(Z_0)}/|Z_0|$

Parameters

- **z** (*complex array-like*) – impedance parameters
- **z0** (*complex array-like or number*) – port impedances
- **s_def** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition : 'power' for power-waves definition[3], 'pseudo' for pseudo-waves definition[4]. 'traveling' corresponds to the initial implementation. Default is 'power'.

Returns
s – scattering parameters

Return type
complex array-like

[1] http://en.wikipedia.org/wiki/S-parameters
[2] http://en.wikipedia.org/wiki/Two-port_network#Hybrid_parameters_(h-parameters)
[1] http://en.wikipedia.org/wiki/impedance_parameters
[2] http://en.wikipedia.org/wiki/S-parameters
[3] Kurokawa, Kaneyuki "Power waves and the scattering matrix", IEEE Transactions on Microwave Theory and Techniques, vol.13, iss.2, pp. 194–202, March 1965.
[4] Marks, R. B. and Williams, D. F. "A general waveguide circuit theory", Journal of Research of National Institute of Standard and Technology, vol.97, iss.5, pp. 533–562, 1992.

References

skrf.network.z2y

skrf.network.**z2y**(*z*)

Convert impedance parameters[1] to admittance parameters[2].

$$y = z^{-1}$$

Parameters
z (*complex array-like*) – impedance parameters

Returns
y – admittance parameters

Return type
complex array-like

See also

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.z2t

skrf.network.**z2t**(*z*)

Not Implemented yet.

Convert impedance parameters[1] to scattering transfer parameters[2].

Parameters
z (*complex array-like or number*) – impedance parameters

Returns
s – scattering parameters

Return type
complex array-like or number

See also

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.z2a

skrf.network.**z2a**(*z*)

Converts impedance parameters to abcd parameters[1].

[1] http://en.wikipedia.org/wiki/impedance_parameters
[2] http://en.wikipedia.org/wiki/Admittance_parameters
[1] http://en.wikipedia.org/wiki/impedance_parameters
[2] http://en.wikipedia.org/wiki/Scattering_transfer_parameters#Scattering_transfer_parameters
[1] https://en.wikipedia.org/wiki/Two-port_network

Parameters
z (numpy.ndarray (shape fx2x2)) – impedance parameter matrix

Returns
abcd – scattering transfer parameters (aka wave cascading matrix)

Return type
np.ndarray

See also

inv
calculates inverse s-parameters

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.y2s

skrf.network.**y2s**(*y*, *z0=50*, *s_def='power'*)

Convert admittance parameters[1] to scattering parameters[2].

For power-waves, from[3]:

$$S = F(1^{\backsim}GY)(1+GY)^{-1}F^{-1}$$

where $G = diag([Z_0])$ and $F = diag([1/2\sqrt{|Re(Z_0)|}])$

For pseudo-waves, Eq.(73) from[4]:

$$S = U(Y^{-1}-G)(Y^{-1}+G)^{-1}U^{-1}$$

where $U = \sqrt{Re(Z_0)}/|Z_0|$

Parameters

- **y** (*complex array-like*) – admittance parameters
- **z0** (*complex array-like or number*) – port impedances
- **s_def** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition : 'power' for power-waves definition[3], 'pseudo' for pseudo-waves definition[4]. 'traveling' corresponds to the initial implementation. Default is 'power'.

Returns
s – scattering parameters

Return type
complex array-like or number

[1] http://en.wikipedia.org/wiki/Admittance_parameters
[2] http://en.wikipedia.org/wiki/S-parameters
[3] Kurokawa, Kaneyuki "Power waves and the scattering matrix", IEEE Transactions on Microwave Theory and Techniques, vol.13, iss.2, pp. 194–202, March 1965.
[4] Marks, R. B. and Williams, D. F. "A general waveguide circuit theory", Journal of Research of National Institute of Standard and Technology, vol.97, iss.5, pp. 533–562, 1992.

See also

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.y2z

skrf.network.**y2z**(*y*)

Convert admittance parameters[1] to impedance parameters[2].

$$z = y^{-1}$$

Parameters
y (*complex array-like*) – admittance parameters

Returns
z – impedance parameters

Return type
complex array-like

See also

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.y2t

skrf.network.**y2t**(*y*)

Not Implemented Yet.

Convert admittance parameters[1] to scattering-transfer parameters[2].

Parameters
y (*complex array-like or number*) – impedance parameters

Returns
t – scattering parameters

Return type
complex array-like or number

See also

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

[1] http://en.wikipedia.org/wiki/Admittance_parameters
[2] http://en.wikipedia.org/wiki/impedance_parameters
[1] http://en.wikipedia.org/wiki/Admittance_parameters
[2] http://en.wikipedia.org/wiki/Scattering_transfer_parameters#Scattering_transfer_parameters

References

skrf.network.t2s

skrf.network.**t2s**(*t*)

Converts scattering transfer parameters[1] to scattering parameters[2].

transfer parameters are also referred to as 'wave cascading matrix', this function only operates on 2N-ports networks with same number of input and output ports, also known as 'balanced networks'[3].

Parameters
t (numpy.ndarray (shape fx2nx2n)) – scattering transfer parameters

Returns
s – scattering parameter matrix.

Return type
numpy.ndarray

See also

inv
calculates inverse s-parameters

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.t2z

skrf.network.**t2z**(*t*)

Not Implemented Yet.

Convert scattering transfer parameters[1] to impedance parameters[2].

Parameters
t (*complex array-like or number*) – impedance parameters

Returns
z – scattering parameters

Return type
complex array-like or number

See also

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

[1] http://en.wikipedia.org/wiki/Scattering_transfer_parameters#Scattering_transfer_parameters
[2] http://en.wikipedia.org/wiki/S-parameters
[3] Janusz A. Dobrowolski, "Scattering Parameter in RF and Microwave Circuit Analysis and Design", Artech House, 2016, pp. 65-68
[1] http://en.wikipedia.org/wiki/Scattering_transfer_parameters#Scattering_transfer_parameters
[2] http://en.wikipedia.org/wiki/impedance_parameters

References

skrf.network.t2y

skrf.network.**t2y**(*t*)

Not Implemented Yet.

Convert scattering transfer parameters to admittance parameters[1].

Parameters
t (*complex array-like or number*) – t-parameters

Returns
y – admittance parameters

Return type
complex array-like or number

See also

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.h2s

skrf.network.**h2s**(*h*, *z0=50*)

Convert hybrid parameters to s parameters.

Parameters

- **h** (*complex array-like*) – hybrid parameters
- **z0** (*complex array-like or number*) – port impedances

Returns
s – scattering parameters

Return type
complex array-like

skrf.network.h2z

skrf.network.**h2z**(*h*)

Convert hybrid parameters to z parameters[1].

Parameters
h (numpy.ndarray (shape fx2x2)) – hybrid parameter matrix

Returns
z – impedance parameters

Return type
np.ndarray

[1] http://en.wikipedia.org/wiki/Scattering_transfer_parameters#Scattering_transfer_parameters
[1] https://en.wikipedia.org/wiki/Two-port_network

See also

inv
calculates inverse s-parameters

s2z, *s2y*, *s2t*, *z2s*, *z2y*, *z2t*, *y2s*, *y2z*, *y2t*, *t2s*, *t2z*, *t2y*, *Network.s*, *Network.y*, *Network.z*, *Network.t*

References

skrf.network.s2s_active

skrf.network.**s2s_active**(*s*, *a*)

Return active s-parameters for a defined wave excitation a.

The active s-parameter at a port is the reflection coefficients when other ports are excited. It is an important quantity for active phased array antennas.

Active s-parameters are defined by[1]:

$$\text{active}(s)_m = \sum_{i=1}^{N} \left(s_{mi} a_i\right) / a_m$$

where s are the scattering parameters and N the number of ports

Parameters

- **s** (*complex array*) – scattering parameters (nfreqs, nports, nports)
- **a** (*complex array of shape (n_ports)*) – forward wave complex amplitude (pseudowave formulation[2])

Returns
s_act – active S-parameters for the excitation a

Return type
complex array of shape (n_freqs, n_ports)

See also

s2z_active
active Z-parameters

s2y_active
active Y-parameters

s2vswr_active
active VSWR

[1] D. M. Pozar, IEEE Trans. Antennas Propag. 42, 1176 (1994).

[2] D. Williams, IEEE Microw. Mag. 14, 38 (2013).

References

skrf.network.s2z_active

skrf.network.**s2z_active**(*s*, *z0*, *a*)

Return the active Z-parameters for a defined wave excitation a.

The active Z-parameters are defined by:

$$\text{active}(z)_m = z_{0,m} \frac{1 + \text{active}(s)_m}{1 - \text{active}(s)_m}$$

where $z_{0,m}$ is the characteristic impedance and $\text{active}(s)_m$ the active S-parameter of port m.

Parameters

- **s** (*complex array*) – scattering parameters (nfreqs, nports, nports)
- **z0** (*complex array-like or number*) – port impedances.
- **a** (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns

z_act – active Z-parameters for the excitation a

Return type

complex array of shape (nfreqs, *nports*)

See also

s2s_active
active S-parameters

s2y_active
active Y-parameters

s2vswr_active
active VSWR

skrf.network.s2y_active

skrf.network.**s2y_active**(*s*, *z0*, *a*)

Return the active Y-parameters for a defined wave excitation a.

The active Y-parameters are defined by:

$$\text{active}(y)_m = y_{0,m} \frac{1 - \text{active}(s)_m}{1 + \text{active}(s)_m}$$

where $y_{0,m}$ is the characteristic admittance and $\text{active}(s)_m$ the active S-parameter of port m.

Parameters

- **s** (*complex array*) – scattering parameters (nfreqs, nports, nports)
- **z0** (*complex array-like or number*) – port impedances.
- **a** (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns

y_act – active Y-parameters for the excitation a

Return type
complex array of shape (nfreqs, *nports*)

See also

s2s_active
active S-parameters

s2z_active
active Z-parameters

s2vswr_active
active VSWR

skrf.network.s2vswr_active

skrf.network.**s2vswr_active**(s, a)

Return the active VSWR for a defined wave excitation a..

The active VSWR is defined by :

$$\text{active}(vswr)_m = \frac{1 + |\text{active}(s)_m|}{1 - |\text{active}(s)_m|}$$

where $\text{active}(s)_m$ the active S-parameter of port m.

Parameters

- **s** (*complex array*) – scattering parameters (nfreqs, nports, nports)
- **a** (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns
vswr_act – active VSWR for the excitation a

Return type
complex array of shape (nfreqs, *nports*)

See also

s2s_active
active S-parameters

s2z_active
active Z-parameters

s2y_active
active Y-parameters

skrf.network.fix_z0_shape

skrf.network.**fix_z0_shape**(*z0*, *nfreqs*, *nports*)

Make a port impedance of correct shape for a given network's matrix.

This attempts to broadcast z0 to satisfy
np.shape(z0) == (nfreqs,nports)

Parameters

- **z0** (*number, array-like*) – z0 can be: * a number (same at all ports and frequencies) * an array-like of length == number ports. * an array-like of length == number frequency points. * the correct shape ==(nfreqs,nports)
- **nfreqs** (*int*) – number of frequency points
- **nports** (*int*) – number of ports

Returns
z0 – z0 with the right shape for a nport Network

Return type
array of shape ==(nfreqs,*nports*)

Examples

For a two-port network with 201 frequency points, possible uses may be

```
>>> z0 = rf.fix_z0_shape(50 , 201,2)
>>> z0 = rf.fix_z0_shape([50,25] , 201,2)
>>> z0 = rf.fix_z0_shape(range(201) , 201,2)
```

skrf.network.renormalize_s

skrf.network.**renormalize_s**(*s*, *z_old*, *z_new*, *s_def='power'*, *s_def_old=None*)

Renormalize a s-parameter matrix given old and new port impedances. Can be also used to convert between different S-parameter definitions.

Note

This re-normalization assumes power-wave formulation per default. To use the pseudo-wave formulation, use s_def='pseudo'. However, results should be the same for real-valued characteristic impedances. See the[1] and[2] for more details.

Note

This just calls

```
z2s(s2z(s, z0=z_old, s_def=s_def_old), z0=z_new, s_def=s_def)
```

Parameters

- **s** (complex array of shape *fxnxn*) – s-parameter matrix
- **z_old** (complex array of shape *fxnxn* or a scalar) – old (original) port impedances
- **z_new** (complex array of shape *fxnxn* or a scalar) – new port impedances
- **s_def** (*str -> s_def : can be: 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition of the output network: 'power' for power-waves definition, 'pseudo'

[1] R. B. Marks and D. F. Williams, "A general waveguide circuit theory," Journal of Research of the National Institute of Standards and Technology, vol. 97, no. 5, pp. 533-561, 1992.

[2] Anritsu Application Note: Arbitrary Impedance, https://web.archive.org/web/20200111134414/https://archive.eetasia.com/www.eetasia.com/ARTICLES/2002MAY/2002MAY02_AMD_ID_NTES_AN.PDF?SOURCES=DOWNLOAD

for pseudo-waves definition. 'traveling' corresponds to the initial implementation. Default is 'power'.

- **s_def_old** (*str -> s_def : can be: None, 'power', 'pseudo' or 'traveling'*) – Scattering parameter definition of the input network: None to copy s_def. 'power' for power-waves definition, 'pseudo' for pseudo-waves definition. 'traveling' corresponds to the initial implementation.

Returns
renormalized s-parameter matrix (shape *fxnxn*)

Return type
numpy.ndarray

See also

Network.renormalize
method of Network to renormalize s

fix_z0_shape, *s2z*, *z2s*

References

Examples

```
>>> s = zeros(shape=(101,2,2))
>>> renormalize_s(s, 50,25)
```

skrf.network.passivity

skrf.network.**passivity**(*s*)

Passivity metric for a network.

A metric which is proportional to the amount of power lost in a network, depending on the excitation port. Specifically, this returns a matrix whose diagonals are equal to the total power received at all ports, normalized to the power at a single excitement port.

Mathematically, this is a test for unitary-ness of the s-parameter matrix[1].

For a one-port network this is

$$|S_{11}|$$

For a two-port network this is

$$\sqrt{(|S_{11}|^2 + |S_{21}|^2\,,\, |S_{22}|^2 + |S_{12}|^2)}$$

in general it is

$$\sqrt{(S^H \cdot S)}$$

where H is conjugate transpose of S, and $\cdot$ is dot product.

[1] http://en.wikipedia.org/wiki/Scattering_parameters#Lossless_networks

> **Note**
>
> The total amount of power dissipated in a network depends on the port matches. For example, given a matched attenuator, this metric will yield the attenuation value. However, if the attenuator is cascaded with a mismatch, the power dissipated will not be equivalent to the attenuator value, nor equal for each excitation port.

Parameters
s (numpy.ndarray of shape *fxnxn*) – s-parameter matrix

Returns
passivity

Return type
numpy.ndarray of shape fxnxn

References

skrf.network.reciprocity

skrf.network.**reciprocity**(*s*)

Reciprocity metric for a multi-port network.

This returns the magnitude of the difference between the s-parameter matrix and its transpose.

for two port this is

$$|S - S^T|$$

where T is transpose of S

Parameters
s (numpy.ndarray of shape *fxnxn*) – s-parameter matrix

Returns
reciprocity

Return type
numpy.ndarray of shape *fxnxn*

3.1.9 Misc Functions

average(list_of_networks[, polar])	Calculate the average network from a list of Networks.
stdev(list_of_networks[, attr])	Calculate the standard deviation of a network attribute from a list of Networks.
s_error(ntwkA, ntwkB[, error_function])	Compute the error between s-parameters of ntwkA and ntwkB.
impedance_mismatch(z1, z2[, s_def])	Create a two-port s-matrix for a impedance mismatch.
two_port_reflect(ntwk1[, ntwk2, name])	Generate a two-port reflective two-port, from two one-ports.
chopinhalf(ntwk, *args, **kwargs)	Chop a sandwich of identical, reciprocal 2-ports in half.
Network.nudge([amount])	Perturb s-parameters by small amount.
Network.renormalize(z_new[, s_def])	Renormalize s-parameter matrix given a new port impedances.
Network.drop_non_monotonic_increasing()	Drop invalid values based on duplicate and non increasing frequency values.

continues on next page

Table 12 – continued from previous page

evenodd2delta(n[, z0, renormalize, doublehalf])	Convert ntwk's s-matrix from even/odd mode into a delta (normal) s-matrix.

skrf.network.average

skrf.network.**average**(*list_of_networks*, *polar=False*)

Calculate the average network from a list of Networks.

This is complex average of the s-parameters for a list of Networks.

Parameters

- **list_of_networks** (list of *Network* objects) – the list of networks to average
- **polar** (*boolean, optional*) – Average the mag/phase components individually. Default is False.

Returns

ntwk – the resultant averaged Network

Return type

Network

Note

This same function can be accomplished with properties of a `NetworkSet` class.

Examples

```
>>> ntwk_list = [rf.Network('myntwk.s1p'), rf.Network('myntwk2.s1p')]
>>> mean_ntwk = rf.average(ntwk_list)
```

skrf.network.stdev

skrf.network.**stdev**(*list_of_networks*, *attr='s'*)

Calculate the standard deviation of a network attribute from a list of Networks.

This is the standard deviation for complex values of the s-parameters and other related attributes for a list of Networks.

Parameters

- **list_of_networks** (list of *Network* objects) – the list of networks to average
- **attr** (*str, optional*) – name of attribute to average

Returns

- **stdev_array** (*ndarray*)
- *An array of standard deviation values computed after combining the s-parameter values of the given networks.*

Return type

np.ndarray

Examples

```
>>> ntwk_list = [rf.Network('myntwk.s1p'), rf.Network('myntwk2.s1p')]
>>> ntwk_stdev = rf.stdev(ntwk_list)
```

skrf.network.s_error

skrf.network.**s_error**(*ntwkA*, *ntwkB*, *error_function='average_l2_norm'*)

Compute the error between s-parameters of ntwkA and ntwkB.

Parameters

- **ntwkA** (*Network* object) – The first network.
- **ntwkB** (*Network* object) – A second network used to compute the error with ntwkA
- **error_function** (*str*) – average_l1_norm, average_l2_norm, maximum_l1_norm, or average_normalized_l1_norm.

Returns

- **error** (numpy.ndarray) – The error between ntwkA and ntwkB
- *Description*
- ———
- *Average L1 Norm. The weighted difference is the average magnitude of the difference between each*
- *element of the S-parameter matrix.*
- *.. math::* – delta = frac{1}{N^2} sum_{i=1}^{N} sum_{j=1}^{N} **|S_{ij}^A - S_{ij}^B|**
- *Average L2 Norm. The weighted difference is the average squared magnitude of the difference*
- *between each element of the S-parameter matrix.*
- *.. math::* – delta = frac{1}{N^2} sum_{i=1}^{N} sum_{j=1}^{N} |S_{ij}^A - S_{ij}^B|^2
- *Maximum L1 Norm. The maximum difference is the magnitude of the maximum difference between each*
- *element of the S-parameter matrix (the magnitude of the largest difference between any pair of*
- *entries in the S-parameter matrices).*
- *.. math::* – delta = max(**|S_{ij}^A - S_{ij}^B|**)
- *Average Normalized L1 Norm. The magnitude of the difference between each element of the*
- *S-parameter matrix is calculated. Each difference is then normalized by the average magnitude of*
- *the two matrix elements (one from each set).*
- *.. math::* – delta = frac{2}{N^2} sum_{i=1}^{N} sum_{j=1}^{N} frac{**|S_{ij}^A - S_{ij}^B|**}{**|S_{ij}^A|** + **|S_{ij}^B|**}
- *To express these in decibels, use*
- *.. math::* – 20log_{10} delta

Return type

ndarray

skrf.network.impedance_mismatch

skrf.network.**impedance_mismatch**(*z1*, *z2*, *s_def='traveling'*)

Create a two-port s-matrix for a impedance mismatch.

Parameters

- **z1** (*number or array-like*) – complex impedance of port 1
- **z2** (*number or array-like*) – complex impedance of port 2
- **s_def** (*str, optional. Default is 'traveling'.*) – Scattering parameter definition: 'power' for power-waves definition, 'pseudo' for pseudo-waves definition. 'traveling' corresponds to the initial implementation. NB: results are the same for real-valued characteristic impedances.

Returns

s

Return type

2-port s-matrix for the impedance mismatch

References

skrf.network.two_port_reflect

skrf.network.**two_port_reflect**(*ntwk1*, *ntwk2=None*, *name=None*)

Generate a two-port reflective two-port, from two one-ports.

Parameters

- **ntwk1** (*one-port Network object*) – network seen from port 1
- **ntwk2** (*one-port Network object, or None*) – network seen from port 2. if None then will use ntwk1.
- **name** (*Name for the combined network. If None, then construct the name*) – from the names of the input networks

Returns

result – two-port reflective network

Return type

Network object

Note

The resultant Network is copied from *ntwk1*, so its various properties(name, frequency, etc.) are inherited from that Network.

Examples

```
>>> short,open = rf.Network('short.s1p', rf.Network('open.s1p')
>>> rf.two_port_reflect(short,open)
```

skrf.network.chopinhalf

skrf.network.**chopinhalf**(*ntwk*, **args*, ***kwargs*)

Chop a sandwich of identical, reciprocal 2-ports in half.

Given two identical, reciprocal 2-ports measured in series, this returns one.

Note

In other words, given

$$B = A \cdot A$$

Return A, where A port2 is connected to A port1. The result may be found through signal flow graph analysis and is,

$$a_{11} = \frac{b_{11}}{1 + b_{12}}$$
$$a_{22} = \frac{b_{22}}{1 + b_{12}}$$
$$a_{12}^2 = b_{21}(1 - \frac{b_{11}b_{22}}{(1 + b_{12})^2}$$

Parameters

ntwk (*Network*) – a 2-port that is equal to two identical two-ports in cascade

Return type

Network

skrf.network.evenodd2delta

skrf.network.**evenodd2delta**(*n*, *z0=50*, *renormalize=True*, *doublehalf=True*)

Convert ntwk's s-matrix from even/odd mode into a delta (normal) s-matrix.

This assumes even/odd ports are ordered [1e,1o,2e,2o].

This is useful for handling coupler sims. Only 4-ports supported for now.

Parameters

- **n** (*skrf.Network*) – Network with an even/odd mode s-matrix
- **z0** (*number, list of numbers*) – the characteristic impedance to set output networks port impedance to , and used to renormalize s-matrix before conversio if `renormalize`=True.
- **renormalize** (*Bool*) – if impedances are in even/odd then they must be renormalized to get correct transformation
- **doublehalf** (*Bool*) – convert even/odd impedances to double/half their values. this is required if data comes from hfss waveports .

Returns

out – same network as *n* but with s-matrix in normal delta basis

Return type

skrf.Network

See also

Network.se2gmm, *Network.gmm2se*

3.1.10 Network utilities

fix_param_shape(p)	Attempt to broadcast p to satisfy.
fix_z0_shape(z0, nfreqs, nports)	Make a port impedance of correct shape for a given network's matrix.
check_frequency_exist(ntwk)	Check if a Network has a non-zero Frequency.
check_nports_equal(ntwkA, ntwkB)	Check if two Networks have same number of ports.
check_frequency_equal(ntwkA, ntwkB)	Check if two Networks have same frequency.
check_z0_equal(ntwkA, ntwkB)	Check if two Networks have same port impedances.
assert_frequency_equal(ntwkA, ntwkB)	
assert_nports_equal(ntwkA, ntwkB)	
assert_frequency_exist(ntwk)	Test if the Network Frequency is defined.
assert_z0_equal(ntwkA, ntwkB)	
assert_z0_at_ports_equal(ntwkA, k, ntwkB, l)	

skrf.network.fix_param_shape

skrf.network.**fix_param_shape**(*p*)

Attempt to broadcast p to satisfy.

np.shape(p) == (nfreqs, nports, nports)

Parameters

p (*number, array-like*) – p can be: * a number (one frequency, one port) * 1D array-like (many frequencies, one port) * 2D array-like (one frequency, many ports) * 3D array-like (many frequencies, many ports)

Returns

p – p with the right shape for a nport Network

Return type

array of shape == (nfreqs, *nports*, *nports*)

skrf.network.check_frequency_exist

skrf.network.**check_frequency_exist**(*ntwk*)

Check if a Network has a non-zero Frequency.

Return type

None

skrf.network.check_nports_equal

skrf.network.**check_nports_equal**(*ntwkA*, *ntwkB*)

Check if two Networks have same number of ports.

Parameters

- **ntwkA** (Network)
- **ntwkB** (Network)

Return type
None

skrf.network.check_frequency_equal

skrf.network.**check_frequency_equal**(*ntwkA*, *ntwkB*)

Check if two Networks have same frequency.

Parameters

- **ntwkA** (Network)
- **ntwkB** (Network)

Return type
None

skrf.network.check_z0_equal

skrf.network.**check_z0_equal**(*ntwkA*, *ntwkB*)

Check if two Networks have same port impedances.

Parameters

- **ntwkA** (Network)
- **ntwkB** (Network)

Return type
None

skrf.network.assert_frequency_equal

skrf.network.**assert_frequency_equal**(*ntwkA*, *ntwkB*)

Parameters

- **ntwkA** (Network)
- **ntwkB** (Network)

Return type
bool

skrf.network.assert_nports_equal

skrf.network.**assert_nports_equal**(*ntwkA*, *ntwkB*)

Parameters

- **ntwkA** (Network)
- **ntwkB** (Network)

Return type
bool

skrf.network.assert_frequency_exist

skrf.network.**assert_frequency_exist**(*ntwk*)

Test if the Network Frequency is defined.

Returns
bool

Return type
boolean

Parameters
ntwk (Network)

skrf.network.assert_z0_equal

skrf.network.**assert_z0_equal**(*ntwkA*, *ntwkB*)

Parameters

- **ntwkA** (Network)
- **ntwkB** (Network)

Return type
bool

skrf.network.assert_z0_at_ports_equal

skrf.network.**assert_z0_at_ports_equal**(*ntwkA*, *k*, *ntwkB*, *l*)

Parameters

- **ntwkA** (Network)
- **k** (*int*)
- **ntwkB** (Network)
- **l** (*int*)

Return type
bool

3.2 frequency (*skrf.frequency*)

Provides a frequency object and related functions.

Most of the functionality is provided as methods and properties of the *Frequency* Class.

3.2.1 Frequency Class

Frequency([start, stop, npoints, unit, ...])	A frequency band.

skrf.frequency.Frequency

class skrf.frequency.**Frequency**(*start=0*, *stop=0*, *npoints=0*, *unit=None*, *sweep_type='lin'*)

A frequency band.

The frequency object provides a convenient way to work with and access a frequency band. It contains a frequency vector as well as a frequency unit. This allows a frequency vector in a given unit to be available

(*f_scaled*), as well as an absolute frequency axis in 'Hz' (*f*).

A Frequency object can be created from either (start, stop, npoints) using the default constructor, *__init__()*. Or, it can be created from an arbitrary frequency vector by using the class method *from_f()*.

Internally, the frequency information is stored in the *f* property combined with the *unit* property. All other properties, *start stop*, etc are generated from these.

Parameters

- **start** (*float*)
- **stop** (*float*)
- **npoints** (*int*)
- **unit** (FrequencyUnitT | *None*)
- **sweep_type** (SweepTypeT)

Attributes

center	Center frequency in Hz.
center_idx	Closes idx of `f` to the center frequency.
center_scaled	Center frequency in `unit`'s.
df	The gradient of the frequency vector.
df_scaled	The gradient of the frequency vector (in unit of `unit`).
dw	The gradient of the frequency vector (in radians).
f	Frequency vector in Hz.
f_scaled	Frequency vector in units of `unit`.
multiplier	Multiplier for formatting axis.
multiplier_dict	Frequency unit multipliers.
npoints	Number of points in the frequency.
span	The frequency span.
span_scaled	The frequency span.
start	Starting frequency in Hz.
start_scaled	Starting frequency in `unit`'s.
step	The inter-frequency step size (in Hz) for evenly-spaced frequency sweeps
step_scaled	The inter-frequency step size (in `unit`) for evenly-spaced frequency sweeps.
stop	Stop frequency in Hz.
stop_scaled	Stop frequency in `unit`'s.
sweep_type	Frequency sweep type.
t	Time vector in s.
t_ns	Time vector in ns.
unit	Unit of this frequency band.
unit_dict	Dictionary to convert unit string with correct capitalization for display.
w	Angular frequency in radians/s.

skrf.frequency.Frequency.center

property Frequency.**center: float**

Center frequency in Hz.

Returns

center – the exact center frequency in units of Hz

Return type
number

skrf.frequency.Frequency.center_idx

property Frequency.**center_idx**: int

Closes idx of *f* to the center frequency.

skrf.frequency.Frequency.center_scaled

property Frequency.**center_scaled**: float

Center frequency in *unit*'s.

Returns
center – the exact center frequency in units of *unit*'s

Return type
number

skrf.frequency.Frequency.df

property Frequency.**df**: ndarray

The gradient of the frequency vector.

Note

The gradient is calculated using:

```
`gradient(self.f)`
```

skrf.frequency.Frequency.df_scaled

property Frequency.**df_scaled**: ndarray

The gradient of the frequency vector (in unit of *unit*).

Note

The gradient is calculated using:

```
`gradient(self.f_scaled)`
```

skrf.frequency.Frequency.dw

property Frequency.**dw**: ndarray

The gradient of the frequency vector (in radians).

Note

The gradient is calculated using:

```
`gradient(self.w)`
```

skrf.frequency.Frequency.f

property Frequency.**f: ndarray**

Frequency vector in Hz.

Returns
f – The frequency vector in Hz

Return type
numpy.ndarray

See also

f_scaled
frequency vector in units of *unit*

w
angular frequency vector in rad/s

skrf.frequency.Frequency.f_scaled

property Frequency.**f_scaled: ndarray**

Frequency vector in units of *unit*.

Returns
f_scaled – A frequency vector in units of *unit*

Return type
numpy.ndarray

See also

f
frequency vector in Hz

w
frequency vector in rad/s

skrf.frequency.Frequency.multiplier

property Frequency.**multiplier: float**

Multiplier for formatting axis.

This accesses the internal dictionary *multiplier_dict* using the value of *unit*

Returns
multiplier – multiplier for this Frequencies unit

Return type
number

skrf.frequency.Frequency.multiplier_dict

Frequency.**multiplier_dict = {'ghz': 1000000000.0, 'hz': 1.0, 'khz': 1000.0, 'mhz': 1000000.0, 'thz': 1000000000000.0}**

Frequency unit multipliers.

skrf.frequency.Frequency.npoints

property Frequency.**npoints: int**

Number of points in the frequency.

skrf.frequency.Frequency.span

property Frequency.**span: float**

The frequency span.

skrf.frequency.Frequency.span_scaled

property Frequency.**span_scaled: float**

The frequency span.

skrf.frequency.Frequency.start

property Frequency.**start: float**

Starting frequency in Hz.

skrf.frequency.Frequency.start_scaled

property Frequency.**start_scaled: float**

Starting frequency in *unit*'s.

skrf.frequency.Frequency.step

property Frequency.**step: float**

The inter-frequency step size (in Hz) for evenly-spaced frequency sweeps

See also

df
: for general case

skrf.frequency.Frequency.step_scaled

property Frequency.**step_scaled: float**

The inter-frequency step size (in *unit*) for evenly-spaced frequency sweeps.

See also

df
: for general case

skrf.frequency.Frequency.stop

property Frequency.**stop**: float

Stop frequency in Hz.

skrf.frequency.Frequency.stop_scaled

property Frequency.**stop_scaled**: float

Stop frequency in *unit*'s.

skrf.frequency.Frequency.sweep_type

property Frequency.**sweep_type**: str

Frequency sweep type.

Returns
sweep_type – 'lin' if linearly increasing, 'log' or 'unknown'.

Return type
str

skrf.frequency.Frequency.t

property Frequency.**t**: ndarray

Time vector in s.

t_period = 2*(n-1)/f_step

skrf.frequency.Frequency.t_ns

property Frequency.**t_ns**: ndarray

Time vector in ns.

t_period = 2*(n-1)/f_step

skrf.frequency.Frequency.unit

property Frequency.**unit**: *FrequencyUnitT*

Unit of this frequency band.

Possible strings for this attribute are: 'Hz', 'kHz', 'MHz', 'GHz', 'THz'

Setting this attribute is not case sensitive.

Returns
unit – String representing the frequency unit

Return type
string

skrf.frequency.Frequency.unit_dict

Frequency.**unit_dict = {'ghz': 'GHz', 'hz': 'Hz', 'khz': 'kHz', 'mhz': 'MHz', 'thz': 'THz'}**

Dictionary to convert unit string with correct capitalization for display.

skrf.frequency.Frequency.w

property Frequency.**w**: ndarray

Angular frequency in radians/s.

Angular frequency is defined as $\omega = 2\pi f$[1]

Returns
w – Angular frequency in rad/s

Return type
numpy.ndarray

References

See also

f_scaled
frequency vector in units of *unit*

f
frequency vector in Hz

Methods

__init__	Frequency initializer.
check_monotonic_increasing	Validate the frequency values
copy	Returns a new copy of this frequency.
drop_non_monotonic_increasing	Drop duplicate and invalid frequency values and return the dropped indices
from_f	Construct Frequency object from a frequency vector.
labelXAxis	Label the x-axis of a plot.
overlap	Calculates overlapping frequency between self and f2.
plot	Plot something vs this frequency.
round_to	Round off frequency values to a specified precision.

skrf.frequency.Frequency.__init__

Frequency.**__init__**(*start=0*, *stop=0*, *npoints=0*, *unit=None*, *sweep_type='lin'*)

Frequency initializer.

Creates a Frequency object from start/stop/npoints and a unit. Alternatively, the class method *from_f()* can be used to create a Frequency object from a frequency vector instead.

Parameters

- **start** (*number, optional*) – start frequency in units of *unit*. Default is 0.
- **stop** (*number, optional*) – stop frequency in units of *unit*. Default is 0.
- **npoints** (*int, optional*) – number of points in the band. Default is 0.
- **unit** (*string, optional*) – Unit of the input start/stop values: 'Hz', 'kHz', 'MHz', 'GHz', 'THz'. If omitted, use *FREQ_UNIT_DEFAULT*, or Hz when that setting is None. A configured

[1] https://en.wikipedia.org/wiki/Angular_frequency

global unit determines the resulting *unit* and *f_scaled*, even when an explicit input unit is supplied. With None, retain the input unit.

- **sweep_type** (*string, optional*) – Type of the sweep: 'lin' or 'log'. 'lin' for linear and 'log' for logarithmic. Default is 'lin'.

Return type
None

Note

The attribute *unit* sets the frequency multiplier, which is used to scale the frequency when *f_scaled* is referenced.

Note

The attribute *unit* is not case sensitive. Hence, for example, 'GHz' or 'ghz' is the same.

See also

from_f
constructs a Frequency object from a frequency vector instead of start/stop/npoints.

unit
frequency unit of the band

Examples

```
>>> wr1p5band = Frequency(start=500, stop=750, npoints=401, unit='ghz')
>>> logband = Frequency(1, 1e9, 301, sweep_type='log')
```

skrf.frequency.Frequency.check_monotonic_increasing

Frequency.**check_monotonic_increasing**()

Validate the frequency values

Raises
InvalidFrequencyWarning: – If frequency points are not monotonously increasing

Return type
None

skrf.frequency.Frequency.copy

Frequency.**copy**()

Returns a new copy of this frequency.

Return type
Frequency

skrf.frequency.Frequency.drop_non_monotonic_increasing

Frequency.**drop_non_monotonic_increasing**()

Drop duplicate and invalid frequency values and return the dropped indices

Returns
The dropped indices

Return type
list[int]

skrf.frequency.Frequency.from_f

classmethod Frequency.**from_f**(*f*, *unit=None*)

Construct Frequency object from a frequency vector.

Interpret the vector using its input unit, then apply the global unit.

Parameters

- **f** (*scalar or array-like*) – frequency vector
- **unit** (*string, optional*) – Unit of the input vector: 'Hz', 'kHz', 'MHz', 'GHz', 'THz'. If omitted, use *FREQ_UNIT_DEFAULT*, or Hz when that setting is None. A configured global unit controls the result's unit without changing its absolute frequencies.

Returns
myfrequency – the Frequency object

Return type
Frequency object

Raises
InvalidFrequencyWarning: – If frequency points are not monotonously increasing

Examples

```
>>> f = np.linspace(75,100,101)
>>> rf.Frequency.from_f(f, unit='GHz')
```

skrf.frequency.Frequency.labelXAxis

Frequency.**labelXAxis**(*ax=None*)

Label the x-axis of a plot.

Sets the labels of a plot using `matplotlib.x_label()` with string containing the frequency unit.

Parameters
ax (`matplotlib.Axes` or None, optional) – Axes on which to label the plot. Defaults is None, for the current axe returned by `matplotlib.gca()`

skrf.frequency.Frequency.overlap

Frequency.**overlap**(*f2*)

Calculates overlapping frequency between self and f2.

See also

overlap_freq

Parameters
f2 (Frequency)

Return type
Frequency

skrf.frequency.Frequency.plot

Frequency.**plot**(*y*, **args*, *ax=None*, ***kwargs*)

Plot something vs this frequency.

This plots whatever is given vs. *self.f_scaled* and then calls *labelXAxis*.

Parameters

- **y** (*NumberLike*)
- **ax** (*Axes*)

skrf.frequency.Frequency.round_to

Frequency.**round_to**(*val='Hz'*)

Round off frequency values to a specified precision.

This is useful for dealing with finite precision limitations of VNA's and/or other software

Parameters
val (*string or number*) – if val is a string it should be a frequency *unit* (ie 'Hz', 'MHz',etc). if its a number, then this returns f = f-f%val

Return type
None

Examples

```
>>> f = skrf.Frequency.from_f([.1,1.2,3.5],unit='Hz')
>>> f.round_to('Hz')
```

3.2.2 Functions

overlap_freq(f1, f2)	Calculates overlapping frequency between f1 and f2.

skrf.frequency.overlap_freq

skrf.frequency.**overlap_freq**(*f1*, *f2*)

Calculates overlapping frequency between f1 and f2.

Or, put more accurately, this returns a Frequency that is the part of f1 that is overlapped by f2. The resultant start frequency is the smallest f1.f that is greater than f2.f.start, and the stop frequency is the largest f1.f that is smaller than f2.f.stop.

This way the new frequency overlays onto f1.

Parameters

- **f1** (*Frequency*) – a frequency object
- **f2** (*Frequency*) – a frequency object

Returns

f3 – part of f1 that is overlapped by f2

Return type

Frequency

3.2.3 Misc

InvalidFrequencyWarning	Thrown if frequency values aren't monotonously increasing

skrf.frequency.InvalidFrequencyWarning

exception skrf.frequency.**InvalidFrequencyWarning**

Thrown if frequency values aren't monotonously increasing

3.3 networkSet (*skrf.networkSet*)

Provides a class representing an un-ordered set of n-port microwave networks.

Frequently one needs to make calculations, such as mean or standard deviation, on an entire set of n-port networks. To facilitate these calculations the *NetworkSet* class provides convenient ways to make such calculations.

Another usage is to interpolate a set of Networks which depend of an parameter (like a knob, or a geometrical parameter).

The results are returned in *Network* objects, so they can be plotted and saved in the same way one would do with a *Network*.

The functionality in this module is provided as methods and properties of the *NetworkSet* Class.

3.3.1 NetworkSet Class

NetworkSet([ntwk_set, name])	A set of Networks.

skrf.networkSet.NetworkSet

class skrf.networkSet.**NetworkSet**(*ntwk_set=None*, *name=None*)

A set of Networks.

This class allows functions on sets of Networks, such as mean or standard deviation, to be calculated conveniently. The results are returned in *Network* objects, so that they may be plotted and saved in like *Network* objects.

This class also provides methods which can be used to plot uncertainty bounds for a set of *Network*.

The names of the *NetworkSet* properties are generated dynamically upon initialization, and thus documentation for individual properties and methods is not available. However, the properties do follow the convention:

```
>>> my_network_set.function_name_network_property_name
```

For example, the complex average (mean) *Network* for a *NetworkSet* is:

```
>>> my_network_set.mean_s
```

This accesses the property 's', for each element in the set, and **then** calculates the 'mean' of the resultant set. The order of operations is important.

Results are returned as *Network* objects, so they may be plotted or saved in the same way as for *Network* objects:

```
>>> my_network_set.mean_s.plot_s_mag()
>>> my_network_set.mean_s.write_touchstone('mean_response')
```

If you are calculating functions that return scalar variables, then the result is accessible through the Network property .s_re. For example:

```
>>> std_s_deg = my_network_set.std_s_deg
```

This result would be plotted by:

```
>>> std_s_deg.plot_s_re()
```

The operators, properties, and methods of NetworkSet object are dynamically generated by private methods

- `__add_a_operator()`
- `__add_a_func_on_property()`
- `__add_a_element_wise_method()`
- `__add_a_plot_uncertainty()`

thus, documentation on the individual methods and properties are not available.

Parameters

- **ntwk_set** (*list | dict*)
- **name** (*str*)

Attributes

inv	Return the NetworkSet of inverted Networks (Network.inv()).
mean_s_db	Return Network of mean magnitude in dB.
params	Return the list of parameter names stored in the Network of the NetworkSet.
params_types	Return a dictionary describing the data type of each parameters.
params_values	Return a dictionary containing all parameters and their values.
std_s_db	Return the Network of the standard deviation magnitude in dB.

skrf.networkSet.NetworkSet.inv

property NetworkSet.**inv**: *NetworkSet*

Return the NetworkSet of inverted Networks (Network.inv()).

Returns
ntwkSet – NetworkSet of inverted Networks

Return type

class
~skrf.networkSet.NetworkSet

skrf.networkSet.NetworkSet.mean_s_db

property NetworkSet.**mean_s_db**: *Network*

Return Network of mean magnitude in dB.

Returns
ntwk – Network of the mean magnitude in dB

Return type

class
~skrf.network.Network

Note

The mean is taken on the magnitude before converted to db, so

magnitude_2_db(mean(s_mag))

which is NOT the same as

mean(s_db)

skrf.networkSet.NetworkSet.params

property NetworkSet.**params**: list

Return the list of parameter names stored in the Network of the NetworkSet.

Similar to the *dims* property, except it returns a list instead of a view.

Returns
list – list of the parameter names if any. Empty list if no parameter found.

Return type
list

skrf.networkSet.NetworkSet.params_types

property NetworkSet.**params_types**: *dict* | None

Return a dictionary describing the data type of each parameters.

Returns
data_types – Dictionary of the (guessed) type of each parameters. Return None if no parameters are defined in the NetworkSet.

Return type
dict or None.

skrf.networkSet.NetworkSet.params_values

property NetworkSet.**params_values**: *dict* | None

Return a dictionary containing all parameters and their values.

Returns
values – Dictionary of all parameters names and their values (into a list). Return None if no parameters are defined in the NetworkSet.

Return type
dict or None.

skrf.networkSet.NetworkSet.std_s_db

property NetworkSet.**std_s_db**: *Network*

Return the Network of the standard deviation magnitude in dB.

Returns
ntwk – Network of the mean magnitude in dB

Return type

class
~skrf.network.Network

Note

The standard deviation is taken on the magnitude before converted to db, so

magnitude_2_db(std(s_mag))

which is NOT the same as

std(s_db)

Methods

__init__	Initialize for NetworkSet.
add_polar_noise	
animate	Animate a property of the networkset.
copy	Copy each network of the network set.
cov	Covariance matrix.
datetime_index	Create a datetime index from networks names.
element_wise_method	Call a given method of each element and returns the result as a new NetworkSet if the output is a Network.
filter	Filter NetworkSet based on a string in *Network.name*.
from_citi	Create a NetworkSet from a CITI file.
from_dir	Create a NetworkSet from a directory containing Networks.
from_mdif	Create a NetworkSet from a MDIF file.
from_s_dict	Create a NetworkSet from a dictionary of s-parameters
from_zip	Create a NetworkSet from a zipfile of touchstones.
has_params	Check is all Networks in the NetworkSet have a similar params dictionary.

continues on next page

Table 21 – continued from previous page

interpolate_frequency	Interpolates each network in the set by frequency by calling `Network.interpolate()`.
interpolate_from_network	Interpolate a Network from a NetworkSet, as a multi-file N-port network.
interpolate_from_params	Interpolate a Network from given parameters of NetworkSet's Networks.
ntwk_attr_2_df	Converts an attributes of the Networks within a NetworkSet to a Pandas DataFrame.
plot_logsigma	Plot the uncertainty for the set in units of log-sigma.
plot_minmax_bounds_component	Plots mean value of the NetworkSet with minimum and maximum bounds in an Network's attribute.
plot_minmax_bounds_s_db	Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.
plot_minmax_bounds_s_db10	Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.
plot_minmax_bounds_s_time_db	Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.
plot_uncertainty_bounds_component	Plot mean value of a NetworkSet with +/- uncertainty bounds in an Network's attribute.
plot_uncertainty_bounds_s_db	Call `plot_uncertainty_bounds(attribute='s_mag', 'ppf':mf.magnitude_2_db*args,**kwargs)`.
plot_uncertainty_bounds_s_time_db	Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.
plot_uncertainty_decomposition	Plot the total and component-wise uncertainty.
plot_violin	Plots the violin plot of the network set for the desired attribute.
rand	Return *n* random samples from this NetworkSet.
scalar_mat	Return a scalar ndarray representing *param* data vs freq and element idx.
sel	Select Network(s) in the NetworkSet from a given value of a parameter.
set_wise_function	Calls a function on a specific property of the Networks in this NetworkSet.
signature	Visualization of a NetworkSet.
sort	Sort this network set.
to_dataframe	Convert attributes of a NetworkSet to a pandas DataFrame.
to_dict	Return a dictionary representation of the NetworkSet.
to_s_dict	Converts a NetworkSet to a dictionary of s-parameters.
uncertainty_ntwk_triplet	Return a 3-tuple of Network objects which contain the mean, upper_bound, and lower_bound for the given Network attribute.
write	Write the NetworkSet to disk using *write()*
write_mdif	Convert a scikit-rf NetworkSet object to a Generalized MDIF file.
write_spreadsheet	Write contents of network to a spreadsheet, for your boss to use.

skrf.networkSet.NetworkSet.__init__

NetworkSet.**__init__**(*ntwk_set=None*, *name=None*)

Initialize for NetworkSet.

Parameters

- **ntwk_set** (list of *Network* objects) – the set of *Network* objects
- **name** (*string*) – the name of the NetworkSet, given to the Networks returned from properties of this class.

skrf.networkSet.NetworkSet.add_polar_noise

NetworkSet.**add_polar_noise**(*ntwk*)

Parameters
ntwk

Returns
ntwk

Return type

class
~skrf.network.Network

skrf.networkSet.NetworkSet.animate

NetworkSet.**animate**(**args*, ***kwargs*)

Animate a property of the networkset.

This loops through all elements in the NetworkSet and calls a plotting attribute (ie Network.plot_`attr`), with given *args and **kwargs.

Parameters

- **attr** (*str, optional*) – plotting property of a Network (ie 's_db', 's_deg', etc) Default is 's_deg'
- **ylims** (*tuple, optional*) – passed to ylim. needed to have consistent y-limits across frames. Default is (-5 ,5).
- **xlims** (*tuple or None, optional.*) – passed to xlim. Default is None.
- **show** (*bool, optional*) – show each frame as its animated. Default is True.
- **savefigs** (*bool, optional*) – save each frame as a png. Default is False.
- ***args** – passed to the Network plotting function
- ****kwargs** – passed to the Network plotting function

Note

using *label=None* will speed up animation significantly, because it prevents the legend from drawing

to create video paste this:

!avconv -r 10 -i out_%5d.png -vcodec huffyuv out.avi

or (depending on your ffmpeg version)

```
!ffmpeg -r 10 -i out_%5d.png -vcodec huffyuv out.avi
```

Examples

```
>>> ns.animate('s_deg', ylims=(-5,5), label=None)
```

skrf.networkSet.NetworkSet.copy

NetworkSet.**copy**()

Copy each network of the network set.

Returns
ns

Return type
class
~skrf.networkSet.NetworkSet

skrf.networkSet.NetworkSet.cov

NetworkSet.**cov**(***kw*)

Covariance matrix.

shape of output will be (nfreq, 2*nports**2, 2*nports**2)

Return type
ndarray

skrf.networkSet.NetworkSet.datetime_index

NetworkSet.**datetime_index**()

Create a datetime index from networks names.

this is just:

[rf.now_string_2_dt(k.name) for k in self]

Return type
list

skrf.networkSet.NetworkSet.element_wise_method

NetworkSet.**element_wise_method**(*network_method_name*, **args*, ***kwargs*)

Call a given method of each element and returns the result as a new NetworkSet if the output is a Network.

Parameter

network_property_name: str
A property of the Network class, which must have a matrix output of shape (f, n, n)

returns
ns

rtype

class
~skrf.networkSet.NetworkSet

Parameters
network_method_name (*str*)

Return type
NetworkSet

skrf.networkSet.NetworkSet.filter

`NetworkSet.`**`filter`**`(`*`s`*`)`

Filter NetworkSet based on a string in *Network.name*.

Notes

This is just

NetworkSet([k for k in self if s in k.name])

Parameters
s (*str*) – string contained in network elements to be filtered

Returns
ns

Return type

class
skrf.networkSet.NetworkSet

Examples

```
>>> ns.filter('monday')
```

skrf.networkSet.NetworkSet.from_citi

`classmethod` `NetworkSet.`**`from_citi`**`(`*`file`*`)`

Create a NetworkSet from a CITI file.

Parameters
file (*str, Path, or file-object*) – CITI file to load

Returns
ns

Return type

class
~skrf.networkSet.NetworkSet

See also

`Citi`

skrf.networkSet.NetworkSet.from_dir

classmethod NetworkSet.**from_dir**(*dir='.'*, **args*, ***kwargs*)

Create a NetworkSet from a directory containing Networks.

This just calls

```
rf.networkSet.NetworkSet(rf.read_all_networks(dir), *args, **kwargs)
```

Parameters

- **dir** (*str or Path*) – directory containing Network files.
- ***args** – passed to NetworkSet constructor
- ****kwargs** – passed to NetworkSet constructor

Examples

```
>>> my_set = rf.networkSet.NetworkSet.from_dir('./data/')
```

skrf.networkSet.NetworkSet.from_mdif

classmethod NetworkSet.**from_mdif**(*file*)

Create a NetworkSet from a MDIF file.

Parameters
file (*str, Path, file-object*) – MDIF file to load

Returns
ns

Return type
class
~skrf.networkSet.NetworkSet

See also

Mdif
MDIF Object

write_mdif
Convert a NetworkSet to a Generalized MDIF file.

skrf.networkSet.NetworkSet.from_s_dict

classmethod NetworkSet.**from_s_dict**(*d*, *frequency*, **args*, ***kwargs*)

Create a NetworkSet from a dictionary of s-parameters

The resultant elements of the NetworkSet are named by the keys of the dictionary.

Parameters

- **d** (*dict*) – dictionary of s-parameters data. values of this should be *numpy.ndarray* assignable to *skrf.network.Network.s*
- **frequency** (*Frequency* object) – frequency assigned to each network

- ***args** – passed to Network.__init__ for each key/value pair of d
- ****kwargs** – passed to Network.__init__ for each key/value pair of d

Returns
ns

Return type
NetworkSet

See also

NetworkSet.to_s_dict

skrf.networkSet.NetworkSet.from_zip

classmethod NetworkSet.**from_zip**(*zip_file_name*, *sort_filenames=True*, **args*, ***kwargs*)

Create a NetworkSet from a zipfile of touchstones.

Parameters

- **zip_file_name** (*string or Path*) – name of zipfile
- **sort_filenames** (*Boolean*) – sort the filenames in the zip file before constructing the NetworkSet
- ***args** (*arguments*) – passed to NetworkSet constructor
- ****kwargs** (*arguments*) – passed to NetworkSet constructor

Examples

```
>>> import skrf as rf
>>> my_set = rf.networkSet.NetworkSet.from_zip('myzip.zip')
```

skrf.networkSet.NetworkSet.has_params

NetworkSet.**has_params**()

Check is all Networks in the NetworkSet have a similar params dictionary.

Returns
True is all Networks have a .params dictionary (of same size), False otherwise

Return type
bool

skrf.networkSet.NetworkSet.interpolate_frequency

NetworkSet.**interpolate_frequency**(*freq_or_n*, *basis='s'*, *coords='cart'*, *f_kwargs=None*, ***kwargs*)

Interpolates each network in the set by frequency by calling `Network.interpolate()`.

Parameters

- **freq_or_n** (*Frequency* or int or list-like) – The new frequency over which to interpolate. this arg may be one of the following:
  - a new *Frequency* object
  - an int: the current frequency span is resampled linearly.

– a list-like: create a new frequency using *from_f()*

- **basis** (*['s','z','y','a'], etc*) – The network parameter to interpolate
- **coords** (*string*) – Coordinate system to use for interpolation: 'cart' or 'polar': 'cart' is cartesian is Re/Im. 'polar' is unwrapped phase/mag
- **f_kwargs** (dict) – Key word arguments that are passed to the new Frequency object
- ****kwargs** (*keyword arguments*) – passed to scipy.interpolate.interp1d() initializer. *kind* controls interpolation type.

  kind = *rational* uses interpolation by rational polynomials.

  d kwarg controls the degree of rational polynomials when *kind* == *rational*. Defaults to 4.

Returns
NetworkSet – New NetworkSet with interpolated frequencies

Return type
NetworkSet

skrf.networkSet.NetworkSet.interpolate_from_network

NetworkSet.**interpolate_from_network**(*ntw_param*, *x*, *interp_kind='linear'*)

Interpolate a Network from a NetworkSet, as a multi-file N-port network.

Assumes that the NetworkSet contains N-port networks with same number of ports N and same number of frequency points.

These networks differ from an given array parameter *interp_param*, which is used to interpolate the returned Network. Length of *interp_param* should be equal to the length of the NetworkSet.

Parameters

- **ntw_param** (*(N,) array_like*) – A 1-D array of real values. The length of ntw_param must be equal to the length of the NetworkSet
- **x** (*real*) – Point to evaluate the interpolated network at
- **interp_kind** (*str*) – Specifies the kind of interpolation as a string: 'linear', 'nearest', 'zero', 'slinear', 'quadratic', 'cubic'. See scipy.interpolate.interp1d for detailed description. Default is 'linear'.

Returns
ntw – Network interpolated at x

Return type
class:~*skrf.network.Network*

Example

Assuming that *ns* is a NetworkSet containing 3 Networks (length=3) :

```
>>> param_x = [1, 2, 3]  # a parameter associated to each Network
>>> x0 = 1.5  # parameter value to interpolate for
>>> interp_ntwk = ns.interpolate_from_network(param_x, x0)
```

skrf.networkSet.NetworkSet.interpolate_from_params

NetworkSet.**interpolate_from_params**(*param*, *x*, *sub_params=None*, *interp_kind='linear'*)

Interpolate a Network from given parameters of NetworkSet's Networks.

Parameters

- **param** (*string*) – Name of the parameter to interpolate the NetworkSet with
- **x** (*float*) – Point to evaluate the interpolated network at
- **sub_params** (*dict, optional*) – Dictionary of parameter/values to filter the NetworkSet, if necessary to avoid an ambiguity. Default is empty dict.
- **interp_kind** (*str*) – Specifies the kind of interpolation as a string: 'linear', 'nearest', 'zero', 'slinear', 'quadratic', 'cubic'. Cf `scipy.interpolate.interp1d` for detailed description. Default is 'linear'.

Returns

ntw – Network interpolated at x

Return type

class:*~skrf.network.Network*

:raises ValueError : if the interpolating param/value are incorrect or ambiguous:

Example

Creating a dummy example:

```
>>> params = [
        {'a':0, 'X':10, 'c':'A'},
        {'a':1, 'X':10, 'c':'A'},
        {'a':2, 'X':10, 'c':'A'},
        {'a':1, 'X':20, 'c':'A'},
        {'a':0, 'X':20, 'c':'A'},
        ]
>>> freq1 = rf.Frequency(75, 110, 101, 'ghz')
>>> ntwks_params = [rf.Network(frequency=freq1,
                               s=np.random.rand(len(freq1),2,2),
                               name=f'ntwk_{m}',
                               comment=f'ntwk_{m}',
                               params=params)
 ↪for (m, params) in enumerate(params) ]
>>> ns = rf.networkSet.NetworkSet(ntwks_params)
```

Interpolated Network for a=1.2 within X=10 Networks:

```
>>> ns.interpolate_from_params('a', 1.2, {'X': 10})
```

skrf.networkSet.NetworkSet.ntwk_attr_2_df

NetworkSet.**ntwk_attr_2_df**(*attr='s_db'*, *m=0*, *n=0*, **args*, ***kwargs*)

Converts an attributes of the Networks within a NetworkSet to a Pandas DataFrame.

Examples

```
>>> df = ns.ntwk_attr_2_df('s_db', m=1, n=0)
>>> df.to_excel('output.xls')  # see Pandas docs for more info
```

skrf.networkSet.NetworkSet.plot_logsigma

NetworkSet.**plot_logsigma**(**args*, ***kwargs*)

Plot the uncertainty for the set in units of log-sigma.

Log-sigma is the complex standard deviation, plotted in units of dB's.

Parameters

- **label_axis** (*bool, optional*) – Default is True.
- ***args** (*arguments*) – passed to self.std_s.plot_s_db()
- ****kwargs** (*arguments*) – passed to self.std_s.plot_s_db()

skrf.networkSet.NetworkSet.plot_minmax_bounds_component

NetworkSet.**plot_minmax_bounds_component**(**args*, ***kwargs*)

Plots mean value of the NetworkSet with minimum and maximum bounds in an Network's attribute.

This is designed to represent min/max in a scalar component of the s-parameter. For example plotting the min/max in the magnitude would be expressed by

$$min(|s|)$$
$$mean(|s|)$$
$$max(|s|)$$

The order of mean and abs is important.

Parameters

- **attribute** (*str*) – attribute of Network type to analyze
- **m** (*int*) – first index of attribute matrix
- **n** (*int*) – second index of attribute matrix
- **type** (*str*) – ['shade' | 'bar'], type of plot to draw
- **alpha** (*float*) – passed to matplotlib.fill_between() command. [number, 0-1]
- **color_error** (*str*) – color of the min/max fill shading. Default is None.
- **markevery_error** (*float*) – tbd
- **type** – if type=='bar', this controls frequency of error bars
- **ax** (*matplotlib axes object*) – Axes to plot on. Default is None.
- **ppf** (*function*) –

  post processing function. a function applied to the
  upper and lower bounds. Default is None

  kwargs_error
  [dict] dictionary of kwargs to pass to the fill_between or errorbar plot command depending on value of type.

- ****kwargs** – passed to Network.plot_s_re command used to plot mean response

> **Note**
>
> For phase uncertainty you probably want s_deg_unwrap, or similar. Uncertainty for wrapped phase blows up at +-pi.

skrf.networkSet.NetworkSet.plot_minmax_bounds_s_db

NetworkSet.**plot_minmax_bounds_s_db**(**args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute= 's_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

See plot_uncertainty_bounds for help.

skrf.networkSet.NetworkSet.plot_minmax_bounds_s_db10

NetworkSet.**plot_minmax_bounds_s_db10**(**args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute= 's_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

see plot_uncertainty_bounds for help

skrf.networkSet.NetworkSet.plot_minmax_bounds_s_time_db

NetworkSet.**plot_minmax_bounds_s_time_db**(**args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute= 's_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

See plot_uncertainty_bounds for help.

skrf.networkSet.NetworkSet.plot_uncertainty_bounds_component

NetworkSet.**plot_uncertainty_bounds_component**(**args*, ***kwargs*)

Plot mean value of a NetworkSet with +/- uncertainty bounds in an Network's attribute.

This is designed to represent uncertainty in a scalar component of the s-parameter. for example plotting the uncertainty in the magnitude would be expressed by,

$$mean(|s|) \pm std(|s|)$$

The order of mean and abs is important.

Parameters

- **attribute** (*str*) – attribute of Network type to analyze
- **m** (*int or None*) – first index of attribute matrix. Default is None (all)
- **n** (*int or None*) – second index of attribute matrix. Default is None (all)
- **type** (*str*) – ['shade' | 'bar'], type of plot to draw
- **n_deviations** (*int*) – number of std deviations to plot as bounds
- **alpha** (*float*) – passed to matplotlib.fill_between() command. [number, 0-1]
- **color_error** (*str*) – color of the +- std dev fill shading. Default is None.
- **markevery_error** (*float*) – tbd

- **type** – if type=='bar', this controls frequency of error bars
- **ax** (*matplotlib axes object*) – Axes to plot on. Default is None.
- **ppf** (*function*) –

  post processing function. a function applied to the
  upper and lower bounds. Default is None

  kwargs_error
  [dict] dictionary of kwargs to pass to the fill_between or errorbar plot command depending on value of type.
- ****kwargs** – passed to Network.plot_s_re command used to plot mean response

> **Note**
>
> For phase uncertainty you probably want s_deg_unwrap, or similar. uncertainty for wrapped phase blows up at +-pi.

skrf.networkSet.NetworkSet.plot_uncertainty_bounds_s_db

NetworkSet.**plot_uncertainty_bounds_s_db**(**args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args,**kwargs)`.

See plot_uncertainty_bounds for help.

skrf.networkSet.NetworkSet.plot_uncertainty_bounds_s_time_db

NetworkSet.**plot_uncertainty_bounds_s_time_db**(**args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute= 's_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

See plot_uncertainty_bounds for help.

skrf.networkSet.NetworkSet.plot_uncertainty_decomposition

NetworkSet.**plot_uncertainty_decomposition**(**args*, ***kwargs*)

Plot the total and component-wise uncertainty.

Parameters

- **m** (*int*) – first s-parameters index
- **n** – second s-parameter index

skrf.networkSet.NetworkSet.plot_violin

NetworkSet.**plot_violin**(*attribute*, **args*, ***kwargs*)

Plots the violin plot of the network set for the desired attribute.

A violin plot provides the distribution of the attribute at each frequency point, and optionally the extrema, mean, and median. The plot becomes cluttered quickly with many frequencies, so reducing the number with *NetworkSet.interpolate_frequency()* is recommended.

Parameters

- **attribute** (*str*) – attribute of Network type to analyze

- **m** (*int*) – first index of attribute matrix
- **n** (*int*) – second index of attribute matrix
- **widths** (*float*) – The maximum width of each violin in units of the positions axis. The default is 0.75 of the distance between the first two frequencies.
- **showmeans** (*bool*) – Whether to show the mean with a line.
- **showextrema** (*bool*) – Whether to show the extrema with a line.
- **showmedians** (*bool*) – Whether to show the median with a line.
- **quantiles** (*ArrayLike*) – If not None, set a list of floats in interval [0, 1] for each violin, which stands for the quantiles that will be rendered for that violin.
- **points** (*int*) – The number of points to evaluate each of the gaussian kernel density estimations at.
- **bw_method** (*{'scott', 'silverman'} or float or callable, default: 'scott'*) – _description_. Defaults to None.
- **ax** (*matplotlib axes object*) – Axes to plot on. Default is None.
- ****kwargs** – passed to `matplotlib.pyplot.violinplot()`

Note

For phase plots you probably want s_deg_unwrap, or similar. Uncertainty for wrapped phase blows up at +-pi.

skrf.networkSet.NetworkSet.rand

`NetworkSet.`**rand**(*n=1*, *rng=None*)

Return *n* random samples from this NetworkSet.

Parameters

- **n** (*int*) – number of samples to return (default is 1)
- **rng** (`numpy.random.Generator` or None) – override the global `numpy` random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.

skrf.networkSet.NetworkSet.scalar_mat

`NetworkSet.`**scalar_mat**(*param='s'*)

Return a scalar ndarray representing *param* data vs freq and element idx.

Output is a 3d array with axes (freq, ns_index, port/ri). ports is a flattened re/im components of port index (*len = 2*nports**2*).

Parameter

param
[str] name of the parameter to export. Default is 's'.

returns
x

rtype

class
np.ndarray

Parameters
param (*str*)

Return type
ndarray

skrf.networkSet.NetworkSet.sel

`NetworkSet.sel(indexers=None)`

Select Network(s) in the NetworkSet from a given value of a parameter.

Parameters
indexers (dict, *optional*) – A dict with keys matching dimensions and values given by scalars, or arrays of parameters. Default is None, which returns the entire NetworkSet

Returns
ns – NetworkSet containing the selected Networks or an empty NetworkSet if no match is found

Return type
NetworkSet

Example

Creating a dummy example:

```
>>> params = [
        {'a':0, 'X':10, 'c':'A'},
        {'a':1, 'X':10, 'c':'A'},
        {'a':2, 'X':10, 'c':'A'},
        {'a':1, 'X':20, 'c':'A'},
        {'a':0, 'X':20, 'c':'A'},
        ]
>>> freq1 = rf.Frequency(75, 110, 101, 'ghz')
>>> ntwks_params = [rf.Network(frequency=freq1,
                               s=np.random.rand(len(freq1),2,2),
                               name=f'ntwk_{m}',
                               comment=f'ntwk_{m}',
                               params=params)
 ↪for (m, params) in enumerate(params) ]
>>> ns = rf.networkSet.NetworkSet(ntwks_params)
```

Selecting the sub-NetworkSet matching scalar parameters:

```
>>> ns.sel({'a': 1})  # len == 2
>>> ns.sel({'a': 0, 'X': 10})  # len == 1
```

Selectong the sub-NetworkSet matching a range of parameters:

```
>>> ns.sel({'a': 0, 'X': [10,20]})  # len == 2
>>> ns.sel({'a': [0,1], 'X': [10,20]}) # len == 4
```

If using a parameter name of value that does not exist, returns empty NetworkSet:

```
>>> ns.sel({'a': -1})  # len == 0
>>> ns.sel({'duh': 0})  # len == 0
```

skrf.networkSet.NetworkSet.set_wise_function

NetworkSet.**set_wise_function**(*func*, *a_property*, **args*, ***kwargs*)

Calls a function on a specific property of the Networks in this NetworkSet.

Parameters

- **func** (*callable*)
- **a_property** (*str*)

Example

```
>>> my_ntwk_set.set_wise_func(mean,'s')
```

skrf.networkSet.NetworkSet.signature

NetworkSet.**signature**(**args*, ***kwargs*)

Visualization of a NetworkSet.

Creates a colored image representing the some component of each Network in the NetworkSet, vs frequency.

Parameters

- **m** (*int, optional*) – first s-parameters index. Default is 0.
- **n** (*int, optional*) – second s-parameter index. Default is 0.
- **component** (*['s_mag','s_db','s_deg' ..]*) – scalar component of Network to visualize. should be a property of the Network object.
- **vmax** (*number or None.*) – sets upper limit of colorbar, if None, will be set to 3*mean of the magnitude of the complex difference. Default is None.
- **vs_time** (*Boolean, optional.*) – if True, then we assume each Network.name was made with rf.now_string, and we make the y-axis a datetime axis. Default is False.
- **cbar_label** (*String or None, optional*) – label for the colorbar. Default is None
- ***args** (*arguments, keyword arguments*) – passed to `imshow()`
- ****kw** (*arguments, keyword arguments*) – passed to `imshow()`

skrf.networkSet.NetworkSet.sort

NetworkSet.**sort**(*key=<function NetworkSet.<lambda>>*, *inplace=True*, ***kwargs*)

Sort this network set.

Parameters

- **key**
- **inplace** (*bool*) – Sort the NetworkSet object directly if True, return the sorted NetworkSet if False. Default is True.
- ****kwargs** (*dict*) – keyword args passed to builtin sorted acting on self.ntwk_set

Returns
ns

Return type
None if inplace=True, NetworkSet if False

Examples

```
>>> ns = rf.networkSet.NetworkSet.from_dir('mydir')
>>> ns.sort()
```

Sort by other property:

```
>>> ns.sort(key= lambda x: x.voltage)
```

Returns a new NetworkSet:

```
>>> sorted_ns = ns.sort(inplace=False)
```

skrf.networkSet.NetworkSet.to_dataframe

NetworkSet.**to_dataframe**(*attrs=None*, *ports=None*, *port_sep=None*)

Convert attributes of a NetworkSet to a pandas DataFrame.

Use the same parameters than *skrf.io.general.network_2_dataframe()*

Parameters

- **attrs** (*list of string*) – Network attributes to convert, like ['s_db','s_deg']
- **ports** (*list of tuples*) – list of port pairs to write. defaults to ntwk.port_tuples (like [[0,0]])
- **port_sep** (*string*) – defaults to None, which means a empty string "" is used for networks with lower than 10 ports. (s_db 11, s_db 21) For more than ten ports a "_" is used to avoid ambiguity. (s_db 1_1, s_db 2_1) For consistent behaviour it's recommended to specify "_" or "," explicitly.

Returns
df

Return type
pandas.DataFrame

:raises ValueError : if the networkset doesn't have parameters:

See also

skrf.io.general.network_2_dataframe

skrf.networkSet.NetworkSet.to_dict

NetworkSet.**to_dict**()

Return a dictionary representation of the NetworkSet.

Returns
d – The returned dictionary has the Network names for keys, and the Networks as values.

Return type
dict

skrf.networkSet.NetworkSet.to_s_dict

`NetworkSet.to_s_dict()`

Converts a NetworkSet to a dictionary of s-parameters.

The resultant keys of the dictionary are the names of the Networks in NetworkSet

Returns
s_dict – contains s-parameters in the form of complex numpy arrays

Return type
dictionary

See also

NetworkSet.from_s_dict

skrf.networkSet.NetworkSet.uncertainty_ntwk_triplet

`NetworkSet.uncertainty_ntwk_triplet(attribute, n_deviations=3)`

Return a 3-tuple of Network objects which contain the mean, upper_bound, and lower_bound for the given Network attribute.

Used to save and plot uncertainty information data.

Note that providing 's' and 's_mag' as attributes will provide different results. For those who want to directly find uncertainty on dB performance, use 's_mag'.

Parameters

- **attribute** (*str*) – Attribute to operate on.
- **n_deviations** (*int, optional*) – Number of standard deviation. The default is 3.

Returns

- **ntwk_mean** (:class: *~skrf.network.Network*) – Network of the averaged attribute
- **lower_bound** (:class: *~skrf.network.Network*) – Network of the lower bound of N*sigma deviation.
- **upper_bound** (:class: *~skrf.network.Network*) – Network of the upper bound of N*sigma deviation.

Return type
tuple(*Network*, *Network*, *Network*)

Example

```
>>> (ntwk_mean, ntwk_lb, ntwk_ub) = my_ntwk_set.uncertainty_ntwk_triplet('s')
>>> (ntwk_mean, ntwk_lb, ntwk_ub) = my_ntwk_set.uncertainty_ntwk_triplet('s_mag')
```

skrf.networkSet.NetworkSet.write

NetworkSet.**write**(*file=None*, **args*, ***kwargs*)

Write the NetworkSet to disk using *write()*

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ns* extension appended to the filename.

Examples

```
>>> ns.name = 'my_ns'
>>> ns.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.networkSet.NetworkSet.write_mdif

NetworkSet.**write_mdif**(*filename*, *values=None*, *data_types=None*, *comments=None*, ***kwargs*)

Convert a scikit-rf NetworkSet object to a Generalized MDIF file.

Parameters

- **filename** (*string*) – Output MDIF file name.
- **values** (*dictionary or None. Default is None.*) – The keys of the dictionary are MDIF variables and its values are a list of the parameter values. If None, then the values will be set to the NetworkSet names and the datatypes will be set to "string".
- **data_types** (*dictionary or None. Default is None.*) – The keys are MDIF variables and the value are datatypes specified by the following strings: "int", "double", and "string"
- **comments** (*list of strings*) – Comments to add to output_file. Each list items is a separate comment line
- ****kwargs** (*dictionary with extra arguments to pass through to the*) – underlying Mdif.write and Network.write_touchstone methods

See also

from_mdif
Create a NetworkSet from a MDIF file.

params_values
parameters values

params_types
parameters types

skrf.networkSet.NetworkSet.write_spreadsheet

NetworkSet.**write_spreadsheet**(**args*, ***kwargs*)

Write contents of network to a spreadsheet, for your boss to use.

Example

```
>>> ns.write_spreadsheet()  # the ns.name attribute must exist
>>> ns.write_spreadsheet(file_name='testing.xlsx')
```

See also

skrf.io.general.network_2_spreadsheet

3.3.2 NetworkSet Utilities

func_on_networks(ntwk_list, func[, ...])	Applies a function to some attribute of a list of networks.
getset(ntwk_dict, s, *args, **kwargs)	Creates a *NetworkSet*, of all `Network`s objects in a dictionary that contain `s` in its key.

skrf.networkSet.func_on_networks

skrf.networkSet.**func_on_networks**(*ntwk_list*, *func*, *attribute='s'*, *name=None*, **args*, ***kwargs*)

Applies a function to some attribute of a list of networks.

Returns the result in the form of a Network. This means information that may not be s-parameters is stored in the s-matrix of the returned Network.

Parameters

- **ntwk_list** (list of *Network* objects) – list of Networks on which to apply *func* to
- **func** (*function*) – function to operate on *ntwk_list* s-matrices
- **attribute** (*string*) – attribute of Network's in ntwk_list for func to act on
- ***args** (*arguments and keyword arguments*) – passed to func
- ****kwargs** (*arguments and keyword arguments*) – passed to func

Returns
ntwk – Network with s-matrix the result of func, operating on ntwk_list's s-matrices

Return type
Network

Examples

averaging can be implemented with func_on_networks by

```
>>> func_on_networks(ntwk_list, mean)
```

skrf.networkSet.getset

skrf.networkSet.**getset**(*ntwk_dict*, *s*, **args*, ***kwargs*)

Creates a *NetworkSet*, of all `Network`s objects in a dictionary that contain `s` in its key. This is useful for dealing with the output of `load_all_touchstones()`, which contains Networks grouped by some kind of naming convention.

Parameters

- **ntwk_dict** (*dictionary of Network objects*) – network dictionary that contains a set of keys *s*
- **s** (*string*) – string contained in the keys of ntwk_dict that are to be in the NetworkSet that is returned
- ***args** (*passed to NetworkSet()*)
- ****kwargs** (*passed to NetworkSet()*)

Returns

ntwk_set – A NetworkSet that made from values of ntwk_dict with *s* in their key

Return type

NetworkSet object

Examples

>>>ntwk_dict = rf.load_all_touchstone('my_dir') >>>set5v = getset(ntwk_dict,'5v') >>>set10v = getset(ntwk_dict,'10v')

3.4 plotting (*skrf.plotting*)

This module provides general plotting functions.

3.4.1 Plots and Charts

smith([smithR, chart_type, draw_labels, ...])	Plot the Smith chart of a given radius.
plot_smith(s[, smith_r, chart_type, ...])	Plot complex data on smith chart.
plot_rectangular(x, y[, x_label, y_label, ...])	Plot rectangular data and optionally label axes.
plot_polar(theta, r[, x_label, y_label, ...])	Plot polar data on a polar plot and optionally label axes.
plot_complex_rectangular(z[, x_label, ...])	Plot complex data on the complex plane.
plot_complex_polar(z[, x_label, y_label, ...])	Plot complex data in polar format.
plot_it_all(netw, *args, **kwargs)	Plot dB, deg, smith, and complex in subplots.
plot_minmax_bounds_component(self, attribute)	Plots mean value of the NetworkSet with minimum and maximum bounds in an Network's attribute.
plot_minmax_bounds_s_db(self, *args, **kwargs)	Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

continues on next page

Table 23 – continued from previous page

plot_minmax_bounds_s_db10(self, *args, **kwargs)	Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.
plot_minmax_bounds_s_time_db(self, *args, ...)	Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.
plot_uncertainty_bounds_component(self, ...)	Plot mean value of a NetworkSet with +/- uncertainty bounds in an Network's attribute.
plot_uncertainty_bounds_s_db(self, *args, ...)	Call `plot_uncertainty_bounds(attribute='s_mag', 'ppf':mf.magnitude_2_db*args,**kwargs)`.
plot_uncertainty_bounds_s_time_db(self, ...)	Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.
plot_passivity(netw[, port, label_prefix])	Plot dB(diag(passivity metric)) vs frequency.
plot_logsigma(self[, label_axis])	Plot the uncertainty for the set in units of log-sigma.
plot_contour(freq, x, y, z, min0max1[, ...])	Create a contour plot.

skrf.plotting.smith

skrf.plotting.**smith**(*smithR=1*, *chart_type='z'*, *draw_labels=False*, *border=False*, *ax=None*, *ref_imm=1.0*, *draw_vswr=None*)

Plot the Smith chart of a given radius.

The Smith chart is used to assist in solving problems with transmission lines and matching circuits. It can be used to simultaneously display multiple parameters including impedances, admittances, reflection coefficients, scattering parameters, noise figure circles, etc.[1]

Parameters

- **smithR** (*number, optional*) – radius of smith chart. Default is 1.
- **chart_type** (*str, optional*) – Contour type. Default is 'z'. Possible values are:
  - *'z'* : lines of constant impedance
  - *'y'* : lines of constant admittance
  - *'zy'* : lines of constant impedance stronger than admittance
  - *'yz'* : lines of constant admittance stronger than impedance
- **draw_labels** (*Boolean, optional*) – annotate real and imaginary parts of impedance on the chart (only if smithR=1). Default is False.
- **border** (*Boolean, optional.*) – draw a rectangular border with axis ticks, around the perimeter of the figure. Not used if draw_labels = True. Default is False.
- **ax** (`matplotlib.pyplot.Axes` or None, optional) – existing axes to draw smith chart on. Default is None (creates a new figure)
- **ref_imm** (*number, optional*) – Reference immittance for center of Smith chart. Only changes labels, if printed. Default is 1.0.
- **draw_vswr** (*list of numbers, Boolean or None, optional*) – draw VSWR circles. If True, default values are used. Default is None.

[1] https://en.wikipedia.org/wiki/Smith_chart

References

skrf.plotting.plot_smith

skrf.plotting.**plot_smith**(*s*, *smith_r=1*, *chart_type='z'*, *x_label='Real'*, *y_label='Imaginary'*, *title='Complex Plane'*, *show_legend=True*, *axis='equal'*, *ax=None*, *force_chart=False*, *draw_vswr=None*, *draw_labels=False*, ***kwargs*)

Plot complex data on smith chart.

Parameters

- **s** (*complex array-like*) – reflection-coefficient-like data to plot
- **smith_r** (*number*) – radius of smith chart
- **chart_type** (*str in ['z','y']*) – Contour type for chart. * 'z' : lines of constant impedance * 'y' : lines of constant admittance
- **x_label** (*string, optional.*) – x-axis label. Default is 'Real'.
- **y_label** (*string, optional.*) – y-axis label. Default is 'Imaginary'
- **title** (*string, optional.*) – plot title, Default is 'Complex Plane'.
- **show_legend** (*Boolean, optional.*) – controls the drawing of the legend. Default is True.
- **axis_equal** (*Boolean, optional.*) – sets axis to be equal increments. Default is 'equal'.
- **ax** (matplotlib.axes.AxesSubplot object or None.) – axes to draw on. Default is None (creates a new figure).
- **force_chart** (*Boolean, optional.*) – forces the re-drawing of smith chart. Default is False.
- **draw_vswr** (*list of numbers, Boolean or None, optional*) – draw VSWR circles. If True, default values are used. Default is None.
- **draw_labels** (*Boolean*) – annotate chart with impedance values
- ****kwargs** (*passed to pylab.plot*)
- **axis** (*str*)

See also

plot_rectangular
: plots rectangular data

plot_complex_rectangular
: plot complex data on complex plane

plot_polar
: plot polar data

plot_complex_polar
: plot complex data on polar plane

plot_smith
: plot complex data on smith chart

skrf.plotting.plot_rectangular

skrf.plotting.**plot_rectangular**(*x*, *y*, *x_label=None*, *y_label=None*, *title=None*, *show_legend=True*, *axis='tight'*, *ax=None*, **args*, ***kwargs*)

Plot rectangular data and optionally label axes.

Parameters

- **x** (*array-like, of complex data*) – data to plot
- **y** (*array-like, of complex data*) – data to plot
- **x_label** (*string or None, optional.*) – x-axis label. Default is None.
- **y_label** (*string or None, optional.*) – y-axis label. Default is None.
- **title** (*string or None, optional.*) – plot title. Default is None.
- **show_legend** (*Boolean, optional.*) – controls the drawing of the legend. Default is True.
- **axis** (*str, optional*) – whether or not to autoscale the axis. Default is 'tight'
- **ax** (matplotlib.axes.AxesSubplot object or None, optional.) – axes to draw on. Default is None (creates a new figure)
- ***args** (*passed to pylab.plot*)
- ****kwargs** (*passed to pylab.plot*)

skrf.plotting.plot_polar

skrf.plotting.**plot_polar**(*theta*, *r*, *x_label=None*, *y_label=None*, *title=None*, *show_legend=True*, *axis_equal=False*, *ax=None*, **args*, ***kwargs*)

Plot polar data on a polar plot and optionally label axes.

Parameters

- **theta** (*array-like*) – angular data to plot
- **r** (*array-like*) – radial data to plot
- **x_label** (*string or None, optional*) – x-axis label. Default is None.
- **y_label** (*string or None, optional.*) – y-axis label. Default is None
- **title** (*string or None, optional.*) – plot title. Default is None.
- **show_legend** (*Boolean, optional.*) – controls the drawing of the legend. Default is True.
- **ax** (matplotlib.axes.AxesSubplot object or None.) – axes to draw on. Default is None (creates a new figure).
- ***args** (*passed to pylab.plot*)
- ****kwargs** (*passed to pylab.plot*)
- **axis_equal** (*bool*)

See also

plot_rectangular
plots rectangular data

plot_complex_rectangular
plot complex data on complex plane

plot_polar
plot polar data

plot_complex_polar
plot complex data on polar plane

plot_smith
plot complex data on smith chart

skrf.plotting.plot_complex_rectangular

skrf.plotting.**plot_complex_rectangular**(*z*, *x_label='Real'*, *y_label='Imag'*, *title='Complex Plane'*, *show_legend=True*, *axis='equal'*, *ax=None*, ***kwargs*)

Plot complex data on the complex plane.

Parameters

- **z** (*array-like, of complex data*) – data to plot
- **x_label** (*string, optional.*) – x-axis label. Default is 'Real'.
- **y_label** (*string, optional.*) – y-axis label. Default is 'Imag'.
- **title** (*string, optional.*) – plot title. Default is 'Complex Plane'
- **show_legend** (*Boolean, optional.*) – controls the drawing of the legend. Default is True.
- **ax** (matplotlib.axes.AxesSubplot object or None.) – axes to draw on. Default is None (creates a new figure)
- ****kwargs** (*passed to pylab.plot*)
- **axis** (*str*)

See also

plot_rectangular
plots rectangular data

plot_complex_rectangular
plot complex data on complex plane

plot_polar
plot polar data

plot_complex_polar
plot complex data on polar plane

plot_smith
plot complex data on smith chart

skrf.plotting.plot_complex_polar

skrf.plotting.**plot_complex_polar**(*z*, *x_label=None*, *y_label=None*, *title=None*, *show_legend=True*, *axis_equal=False*, *ax=None*, ***kwargs*)

Plot complex data in polar format.

Parameters

- **z** (*array-like, of complex data*) – data to plot
- **x_label** (*string or None, optional*) – x-axis label. Default is None.
- **y_label** (*string or None, optional.*) – y-axis label. Default is None
- **title** (*string or None, optional.*) – plot title. Default is None.
- **show_legend** (*Boolean, optional.*) – controls the drawing of the legend. Default is True.
- **ax** (matplotlib.axes.AxesSubplot object or None.) – axes to draw on. Default is None (creates a new figure).
- ****kwargs** (*passed to pylab.plot*)
- **axis_equal** (*bool*)

See also

plot_rectangular
plots rectangular data

plot_complex_rectangular
plot complex data on complex plane

plot_polar
plot polar data

plot_complex_polar
plot complex data on polar plane

plot_smith
plot complex data on smith chart

skrf.plotting.plot_it_all

skrf.plotting.**plot_it_all**(*netw*, **args*, ***kwargs*)

Plot dB, deg, smith, and complex in subplots.

Plots the magnitude in dB in subplot 1, the phase in degrees in subplot 2, a smith chart in subplot 3, and a complex plot in subplot 4.

Parameters

- ***args** (*arguments, optional*) – passed to the matplotlib.plot command
- ****kwargs** (*keyword arguments, optional*) – passed to the matplotlib.plot command
- **netw** (Network)

See also

plot_s_db, plot_s_deg, plot_s_smith, plot_s_complex

Examples

```
>>> from skrf.data import ring_slot
>>> ring_slot.plot_it_all()
```

skrf.plotting.plot_minmax_bounds_component

skrf.plotting.**plot_minmax_bounds_component**(*self*, *attribute*, *m=0*, *n=0*, ***, *type='shade'*, *alpha=0.3*, *color_error=None*, *markevery_error=20*, *ax=None*, *ppf=None*, *kwargs_error=None*, ***kwargs*)

Plots mean value of the NetworkSet with minimum and maximum bounds in an Network's attribute.

This is designed to represent min/max in a scalar component of the s-parameter. For example plotting the min/max in the magnitude would be expressed by

$$min(|s|)$$
$$mean(|s|)$$
$$max(|s|)$$

The order of mean and abs is important.

Parameters

- **attribute** (*str*) – attribute of Network type to analyze
- **m** (*int*) – first index of attribute matrix
- **n** (*int*) – second index of attribute matrix
- **type** (*str*) – ['shade' | 'bar'], type of plot to draw
- **alpha** (*float*) – passed to matplotlib.fill_between() command. [number, 0-1]
- **color_error** (*str*) – color of the min/max fill shading. Default is None.
- **markevery_error** (*float*) – tbd
- **type** – if type=='bar', this controls frequency of error bars
- **ax** (*matplotlib axes object*) – Axes to plot on. Default is None.
- **ppf** (*function*) –

  post processing function. a function applied to the
  upper and lower bounds. Default is None

  kwargs_error
  [dict] dictionary of kwargs to pass to the fill_between or errorbar plot command depending on value of type.

- ****kwargs** – passed to Network.plot_s_re command used to plot mean response
- **self** (NetworkSet)
- **kwargs_error** (dict)

> **Note**
>
> For phase uncertainty you probably want s_deg_unwrap, or similar. Uncertainty for wrapped phase blows up at +-pi.

skrf.plotting.plot_minmax_bounds_s_db

skrf.plotting.**plot_minmax_bounds_s_db**(*self*, **args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute= 's_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

See plot_uncertainty_bounds for help.

Parameters
self (NetworkSet)

skrf.plotting.plot_minmax_bounds_s_db10

skrf.plotting.**plot_minmax_bounds_s_db10**(*self*, **args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute= 's_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

see plot_uncertainty_bounds for help

Parameters
self (NetworkSet)

skrf.plotting.plot_minmax_bounds_s_time_db

skrf.plotting.**plot_minmax_bounds_s_time_db**(*self*, **args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute= 's_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

See plot_uncertainty_bounds for help.

Parameters
self (NetworkSet)

skrf.plotting.plot_uncertainty_bounds_component

skrf.plotting.**plot_uncertainty_bounds_component**(*self*, *attribute*, *m=None*, *n=None*, ***, *type='shade'*, *n_deviations=3*, *alpha=0.3*, *color_error=None*, *markevery_error=20*, *ax=None*, *ppf=None*, *kwargs_error=None*, ***kwargs*)

Plot mean value of a NetworkSet with +/- uncertainty bounds in an Network's attribute.

This is designed to represent uncertainty in a scalar component of the s-parameter. for example plotting the uncertainty in the magnitude would be expressed by,

$$mean(|s|) \pm std(|s|)$$

The order of mean and abs is important.

Parameters

- **attribute** (*str*) – attribute of Network type to analyze
- **m** (*int or None*) – first index of attribute matrix. Default is None (all)

- **n** (*int or None*) – second index of attribute matrix. Default is None (all)
- **type** (*str*) – [‘shade’ | ‘bar’], type of plot to draw
- **n_deviations** (*int*) – number of std deviations to plot as bounds
- **alpha** (*float*) – passed to matplotlib.fill_between() command. [number, 0-1]
- **color_error** (*str*) – color of the +- std dev fill shading. Default is None.
- **markevery_error** (*float*) – tbd
- **type** – if type==’bar’, this controls frequency of error bars
- **ax** (*matplotlib axes object*) – Axes to plot on. Default is None.
- **ppf** (*function*) –

  post processing function. a function applied to the
  upper and lower bounds. Default is None

  kwargs_error
  [dict] dictionary of kwargs to pass to the fill_between or errorbar plot command depending on value of type.
- ****kwargs** – passed to Network.plot_s_re command used to plot mean response
- **self** (NetworkSet)
- **kwargs_error** (dict)

Note

For phase uncertainty you probably want s_deg_unwrap, or similar. uncertainty for wrapped phase blows up at +-pi.

skrf.plotting.plot_uncertainty_bounds_s_db

skrf.plotting.**plot_uncertainty_bounds_s_db**(*self*, **args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute='s_mag','ppf':mf.magnitude_2_db*args,**kwargs)`.

See plot_uncertainty_bounds for help.

Parameters
self (NetworkSet)

skrf.plotting.plot_uncertainty_bounds_s_time_db

skrf.plotting.**plot_uncertainty_bounds_s_time_db**(*self*, **args*, ***kwargs*)

Call `plot_uncertainty_bounds(attribute= 's_mag','ppf':mf.magnitude_2_db*args, **kwargs)`.

See plot_uncertainty_bounds for help.

Parameters
self (NetworkSet)

skrf.plotting.plot_passivity

skrf.plotting.**plot_passivity**(*netw*, *port=None*, *label_prefix=None*, ***kwargs*)

Plot dB(diag(passivity metric)) vs frequency.

> **Note**
>
> This plot does not completely capture the passivity metric, which is a test for *unitary-ness* of the s-matrix. However, it may be used to display a measure of power dissipated in a network.

> **See also**
>
> `passivity`

Parameters

netw (Network)

skrf.plotting.plot_logsigma

skrf.plotting.**plot_logsigma**(*self*, *label_axis=True*, **args*, ***kwargs*)

Plot the uncertainty for the set in units of log-sigma.

Log-sigma is the complex standard deviation, plotted in units of dB's.

Parameters

- **label_axis** (*bool, optional*) – Default is True.
- ***args** (*arguments*) – passed to self.std_s.plot_s_db()
- ****kwargs** (*arguments*) – passed to self.std_s.plot_s_db()
- **self** (NetworkSet)

skrf.plotting.plot_contour

skrf.plotting.**plot_contour**(*freq*, *x*, *y*, *z*, *min0max1*, *graph=True*, *cmap='plasma_r'*, *title=''*, ***kwargs*)

Create a contour plot.

Parameters

- **freq** – Frequency object.
- **x** (*array*) – x points
- **y** (*array*) – y points.
- **z** (*array*) – z points.
- **min0max1** (*int*) – 0 for min, 1 for max.
- **graph** (*bool, optional*) – plot graph if True. The default is True.
- **cmap** (*str, optional*) – Colormap label. The default is 'plasma_r'.
- **title** (*str, optional*) – Figure title. The default is ''.
- ****kwargs** (dict) – Other parameters passed to *matplotlib.plot()*.

Returns

- **GAMopt** (*:skrf.Network:*) – Network
- **VALopt** (*float*) – min or max.

3.4.2 Convenience plotting functions

stylely([rc_dict, style_file])	Loads the rc-params from the specified file (file must be located in skrf/data).
subplot_params(ntwk[, param, proj, ...])	Plot all networks parameters individually on subplots.
shade_bands(edges[, y_range, cmap])	Shades frequency bands.
save_all_figs([dir, format, replace_spaces, ...])	Save all open Figures to disk.
scale_frequency_ticks(ax, funit)	Scale frequency axis ticks.
add_markers_to_lines([ax, marker_list, ...])	Add markers to existing lings on a plot.
legend_off([ax])	Turn off the legend for a given axes.
func_on_all_figs(func, *args, **kwargs)	Run a function after making all open figures current.
scrape_legend([n, ax])	Scrape a legend with redundant labels.
signature(self[, m, n, component, vmax, ...])	Visualization of a NetworkSet.

skrf.plotting.stylely

skrf.plotting.**stylely**(*rc_dict=None*, *style_file='skrf.mplstyle'*)

Loads the rc-params from the specified file (file must be located in skrf/data).

Parameters

- **rc_dict** (*dict, optional*) – rc dict passed to `matplotlib.rc()`, by default {}
- **style_file** (*str, optional*) – style file, by default 'skrf.mplstyle'

skrf.plotting.subplot_params

skrf.plotting.**subplot_params**(*ntwk*, *param='s'*, *proj='db'*, *size_per_port=4*, *newfig=True*, *add_titles=True*, *keep_it_tight=True*, *subplot_kw=None*, ***kwargs*)

Plot all networks parameters individually on subplots.

Parameters

- **ntwk** (*Network*) – Network to get data from.
- **param** (*str, optional*) – Parameter to plot, by default 's'
- **proj** (*str, optional*) – Projection type, by default 'db'
- **size_per_port** (*int, optional*) – by default 4
- **newfig** (*bool, optional*) – by default True
- **add_titles** (*bool, optional*) – by default True
- **keep_it_tight** (*bool, optional*) – by default True
- **subplot_kw** (*dict, optional*) – by default {}

Returns

- **f** (`matplotlib.pyplot.Figure`) – Matplotlib Figure
- **ax** (`matplotlib.pyplot.Axes`) – Matplotlib Axes

skrf.plotting.shade_bands

skrf.plotting.**shade_bands**(*edges*, *y_range=None*, *cmap='prism'*, ***kwargs*)

Shades frequency bands.

When plotting data over a set of frequency bands it is nice to have each band visually separated from the other. The kwarg *alpha* is useful.

Parameters

- **edges** (*array-like*) – x-values separating regions of a given shade
- **y_range** (*tuple or None, optional.*) – y-values to shade in. Default is None.
- **cmap** (*str, optional.*) – see matplotlib.cm or matplotlib.colormaps for acceptable values. Default is 'prism'.
- ****kwargs** (*key word arguments*) – passed to *matplotlib.fill_between*

Examples

```
>>> rf.shade_bands([325,500,750,1100], alpha=.2)
```

skrf.plotting.save_all_figs

skrf.plotting.**save_all_figs**(*dir='./'*, *format=None*, *replace_spaces=True*, *echo=None*)

Save all open Figures to disk.

Parameters

- **dir** (*string, optional.*) – path to save figures into. Default is './'
- **format** (*None or list of strings, optional.*) – the types of formats to save figures as. The elements of this list are passed to `matplotlib.pyplot.savefig()`. This is a list so that you can save each figure in multiple formats. Default is None.
- **replace_spaces** (*bool, optional*) – default is True.
- **echo** (*bool, optional.*) – True prints filenames as they are saved. Default is True.

skrf.plotting.scale_frequency_ticks

skrf.plotting.**scale_frequency_ticks**(*ax*, *funit*)

Scale frequency axis ticks.

Parameters

- **ax** (*plt.Axes*) – Matplotlib figure axe
- **funit** (*str*) – frequency unit string as in *unit*

Raises

ValueError – if invalid unit is passed

skrf.plotting.add_markers_to_lines

skrf.plotting.**add_markers_to_lines**(*ax=None*, *marker_list=None*, *markevery=10*)

Add markers to existing lings on a plot.

Convenient if you have already have a plot made, but then need to add markers afterwards, so that it can be interpreted in black and white. The markevery argument makes the markers less frequent than the data, which is generally what you want.

Parameters

- **ax** (*matplotlib.Axes or None, optional*) – axis which to add markers to. Default is current axe gca()
- **marker_list** (*list of string, optional*) – list of marker characters. Default is ['o', 'D', 's', '+', 'x']. see matplotlib.plot help for possible marker characters
- **markevery** (*int, optional.*) – markevery number of points with a marker. Default is 10.

skrf.plotting.legend_off

skrf.plotting.**legend_off**(*ax=None*)

Turn off the legend for a given axes.

If no axes is given then it will use current axes.

Parameters

ax (*matplotlib.Axes or None, optional*) – axis to operate on. Default is None for current axe gca()

skrf.plotting.func_on_all_figs

skrf.plotting.**func_on_all_figs**(*func*, **args*, ***kwargs*)

Run a function after making all open figures current.

Useful if you need to change the properties of many open figures at once, like turn off the grid.

Parameters

- **func** (*function*) – function to call
- ***args** (*passed to func*)
- ****kwargs** (*passed to func*)

Examples

```
>>> rf.func_on_all_figs(grid, alpha=.3)
```

skrf.plotting.scrape_legend

skrf.plotting.**scrape_legend**(*n=None*, *ax=None*)

Scrape a legend with redundant labels.

Given a legend of m entries of n groups, this will remove all but every m/nth entry. This is used when you plot many lines representing the same thing, and only want one label entry in the legend for the whole ensemble of lines.

Parameters

- **n** (*int or None, optional.*) – Default is None.
- **ax** (*matplotlib.Axes or None, optional*) – axis to operate on. Default is None for current axe gca()

skrf.plotting.signature

skrf.plotting.**signature**(*self*, *m=0*, *n=0*, *component='s_mag'*, *vmax=None*, *vs_time=False*, *cbar_label=None*, **args*, ***kwargs*)

Visualization of a NetworkSet.

Creates a colored image representing the some component of each Network in the NetworkSet, vs frequency.

Parameters

- **m** (*int, optional*) – first s-parameters index. Default is 0.
- **n** (*int, optional*) – second s-parameter index. Default is 0.
- **component** (*['s_mag','s_db','s_deg' ..]*) – scalar component of Network to visualize. should be a property of the Network object.
- **vmax** (*number or None.*) – sets upper limit of colorbar, if None, will be set to 3*mean of the magnitude of the complex difference. Default is None.
- **vs_time** (*Boolean, optional.*) – if True, then we assume each Network.name was made with rf.now_string, and we make the y-axis a datetime axis. Default is False.
- **cbar_label** (*String or None, optional*) – label for the colorbar. Default is None
- ***args** (*arguments, keyword arguments*) – passed to imshow()
- ****kw** (*arguments, keyword arguments*) – passed to imshow()
- **self** (NetworkSet)

3.5 Media

3.5.1 media (*skrf.media*)

This package provides objects representing transmission line mediums.

The *Media* object is the base-class that is inherited by specific transmission line instances, such as *Freespace*, or *RectangularWaveguide*. The *Media* object provides generic methods to produce *Network*'s for any transmission line medium, such as *line()* and *delay_short()*. These methods are inherited by the specific transmission line classes, which internally define relevant quantities such as propagation constant (*gamma*), and characteristic impedance (*Z0*). This allows the specific transmission line mediums to produce networks without re-implementing methods for each specific media instance.

Media base-class

Media	Abstract Base Class for a single mode on a transmission line media.

skrf.media.media.Media

class skrf.media.media.**Media**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*)

Abstract Base Class for a single mode on a transmission line media.

This class init's with *frequency* and *z0_port* (the port impedance); attributes shared by all media. Methods defined here make use of the properties :

- *gamma* - (complex) media propagation constant
- *z0* - (complex) media characteristic impedance

Which define the properties of a specific media. Any sub-class of Media must implement these properties. *gamma* and *z0* should return complex arrays of the same length as *frequency*. *gamma* must follow the convention:

- positive real(gamma) = attenuation
- positive imag(gamma) = forward propagation

Parameters

- **frequency** (*Frequency* object or None) – frequency band of this transmission line medium. Defaults to None, which produces a 1-10ghz band with 101 points.
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0_override** (*number, array-like, or None*) – *z0_override* override the characteristic impedance for the media. If *z0_override* is not None, the networks generated by the media have their characteristic impedance *z0* overridden by *z0_override*. (Default is None)
- **z0** (*number, array-like, or None*) – deprecated parameter, alias to *z0_override* if *z0_override* is None. Emit a deprecation warning.

Notes

Full-wave simulations results can be networks with complex and frequency-dependant *z0* port impedances. However, the regular touchstone (s-parameters) files generated by most VNA measurements are normalized to a single real value, e. g. 1. or 50 ohm. Manually setting the *z0_port* to a single real value enforce the second behaviour. The default (*z0_port* = *None*) is the first behaviour.

Attributes

Z0	Characteristic Impedance
alpha	Real (attenuation) component of gamma.
beta	Imaginary (propagating) component of gamma.
gamma	Propagation constant.
npoints	Number of points of the frequency axis.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	Characteristic Impedance, z_0.
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.media.Media.Z0

property Media.**Z0**

Characteristic Impedance

Returns

z0 – Characteristic Impedance in units of ohms

Return type

np.ndarray

skrf.media.media.Media.alpha

property Media.**alpha**: ndarray

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.media.Media.beta

property Media.**beta**: ndarray

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.media.Media.gamma

abstract property Media.**gamma**

Propagation constant.

In skrf, defined as $\gamma = \alpha + j\beta$.

Returns
gamma – complex propagation constant for this media

Return type
numpy.ndarray

Notes

gamma must adhere to the following convention:

- positive real(gamma) = attenuation
- positive imag(gamma) = forward propagation

skrf.media.media.Media.npoints

property Media.**npoints**: int

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.media.Media.v_g

property Media.v_g

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the *j* is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.media.Media.v_p

property Media.v_p: ndarray

Complex phase velocity (in m/s).

$$j \cdot \omega/\gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.media.Media.z0

property Media.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.media.Media.z0_characteristic

abstract property Media.**z0_characteristic**

Characteristic Impedance, z_0. This abstract method has to be defined in the Media Class.

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.media.Media.z0_override

property Media.**z0_override**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.media.Media.z0_port

property Media.**z0_port**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.

continues on next page

Table 27 – continued from previous page

extract_distance	Determines physical distance from a transmission or reflection Network.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.media.Media.__init__

Media.**__init__**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*Number | Sequence[Number] | ndarray | None*)
- **z0_override** (*Number | Sequence[Number] | ndarray | None*)
- **z0** (*Number | Sequence[Number] | ndarray | None*)

skrf.media.media.Media.attenuator

Media.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns
ntwk – 2-port attenuator

Return type
Network object

skrf.media.media.Media.capacitor

Media.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
capacitor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.media.Media.capacitor_q

Media.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.media.Media.copy

Media.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.media.Media.delay_load

Media.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.media.Media.delay_open

Media.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns

delay_open – a delayed open

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.media.Media.delay_short

Media.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns

delay_short – a delayed short

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.media.Media.electrical_length

Media.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns

theta – complex electrical length in radians or degrees, depending on value of deg.

Return type

number or array-like

skrf.media.media.Media.extract_distance

Media.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters

ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns

d – physical distance

Return type

number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.media.Media.get_array_of

Media.**get_array_of**(*x*)

skrf.media.media.Media.impedance_mismatch

Media.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

mismatch – a 2-port network representing the impedance mismatch

Return type

Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.media.Media.inductor

Media.**inductor**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

inductor

Return type

a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.media.Media.inductor_q

Media.**inductor_q**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.media.Media.isolator

Media.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.media.Media.line

Media.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line – matched transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.media.Media.line_floating

Media.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

line_floating – matched, floating transmission line of given length

Return type

Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.media.Media.load

Media.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an

optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native **z0**.

- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
load – n-port load, where S = Gamma0*eye(...)

Return type
Network object

See also

match, *open*, *short*

skrf.media.media.Media.lossless_mismatch

Media.**lossless_mismatch**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns
ntwk – 2-port lossless mismatch

Return type
Network object

skrf.media.media.Media.match

Media.**match**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's z0_port value is used. If z0_port is also None (by default), in which case the Media's native **z0** is used. A Media's z0_port is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native **z0**.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns
match – a n-port match

Return type
Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.media.Media.mode

Media.**mode**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.media.Media.open

Media.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.media.Media.plot

Media.**plot**(**args*, ***kw*)

skrf.media.media.Media.random

Media.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero
- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns

result – the network

Return type

Network object

skrf.media.media.Media.resistor

Media.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

resistor

Return type

a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.media.Media.short

Media.**short**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns

match – a n-port short circuit

Return type

Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.media.Media.shunt

`Media.`**shunt**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns

shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type

Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.media.Media.shunt_capacitor

Media.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns

shunt_capacitor – shunted capacitor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.media.Media.shunt_delay_load

Media.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.media.Media.shunt_delay_open

Media.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.media.Media.shunt_delay_short

Media.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.media.Media.shunt_inductor

`Media.`**`shunt_inductor`**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.media.Media.shunt_resistor

`Media.`**`shunt_resistor`**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.media.Media.splitter

`Media.splitter`(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

tee – a n-port splitter

Return type

Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.media.Media.tee

`Media.tee`(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters

****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.media.Media.theta_2_d

Media.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)
- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.media.Media.thru

Media.**thru**(*z0=None*, ***kwargs*)

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.media.Media.to_meters

`Media.to_meters(d, unit='deg')`

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.media.Media.white_gaussian_polar

`Media.white_gaussian_polar(phase_dev, mag_dev, n_ports=1, **kwargs)`

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.media.Media.write_csv

Media.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters

filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

3.5.2 Media Classes

DefinedGammaZ0([frequency, z0_port, z0, Z0, ...])	A media directly defined by its propagation constant and characteristic impedance.
DistributedCircuit([frequency, z0_port, ...])	A transmission line mode defined in terms of distributed impedance and admittance values.
RectangularWaveguide([frequency, z0_port, ...])	A single mode of a homogeneously filled rectangular waveguide.
CircularWaveguide([frequency, z0_port, ...])	A single mode of a homogeneously filled Circular Waveguide
CPW([frequency, z0_port, z0_override, z0, ...])	Coplanar waveguide.
Coaxial([frequency, z0_port, z0_override, ...])	A coaxial transmission line defined in terms of its inner/outer diameters and permittivity.
MLine([frequency, z0_port, z0_override, z0, ...])	A microstripline transmission line defined in terms of width, thickness and height on a given relative permittivity substrate.
Freespace([frequency, z0_port, z0_override, ...])	A plane-wave (TEM Mode) in Freespace.
DefinedAEpTandZ0([frequency, A, f_A, ep_r, ...])	Transmission line medium defined by A, Ep, Tand and Z0.

skrf.media.media.DefinedGammaZ0

class skrf.media.media.**DefinedGammaZ0**(*frequency=None*, *z0_port=None*, *z0=50*, *Z0=None*, *gamma=1j*)

A media directly defined by its propagation constant and characteristic impedance.

Parameters

- **frequency** (*Frequency* object or None) – frequency band of this transmission line medium. Default is None, which produces a 1-10ghz band with 101 points.
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance Z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance Z0 of the media. (Default is None)
- **gamma** (*number or array-like, optional*) – complex propagation constant. *gamma* must adhere to the following convention:
  - positive real(gamma) = attenuation

– positive imag(gamma) = forward propagation

Default is 1j (lossless).

- **z0** (*number or array-like, optional.*) – complex characteristic impedance of the media. Default is 50 ohm.
- **Z0** (*number, array-like, or None*) – deprecated parameter, only emit a deprecation warning.

Attributes

Z0	Characteristic Impedance
alpha	Real (attenuation) component of gamma.
beta	Imaginary (propagating) component of gamma.
frequency	
gamma	Propagation constant.
npoints	Number of points of the frequency axis.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	Characteristic Impedance, z_0.
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.media.DefinedGammaZ0.Z0

property DefinedGammaZ0.**Z0**

Characteristic Impedance

Returns
z0 – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.media.DefinedGammaZ0.alpha

property DefinedGammaZ0.**alpha**: ndarray

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.media.DefinedGammaZ0.beta

property DefinedGammaZ0.**beta**: ndarray

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.media.DefinedGammaZ0.frequency

property DefinedGammaZ0.**frequency**

skrf.media.media.DefinedGammaZ0.gamma

property DefinedGammaZ0.**gamma**

Propagation constant.

Returns
gamma – complex propagation constant for this media

Return type
numpy.ndarray

Notes

gamma must adhere to the following convention:

- positive real(gamma) = attenuation
- positive imag(gamma) = forward propagation

skrf.media.media.DefinedGammaZ0.npoints

property DefinedGammaZ0.**npoints**

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.media.DefinedGammaZ0.v_g

property DefinedGammaZ0.**v_g**

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the *j* is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.media.DefinedGammaZ0.v_p

property DefinedGammaZ0.**v_p**: ndarray

Complex phase velocity (in m/s).

$$j \cdot \omega / \gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.media.DefinedGammaZ0.z0

property DefinedGammaZ0.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.media.DefinedGammaZ0.z0_characteristic

property DefinedGammaZ0.**z0_characteristic**

Characteristic Impedance, z_0.

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.media.DefinedGammaZ0.z0_override

property DefinedGammaZ0.**z0_override**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.media.DefinedGammaZ0.z0_port

property DefinedGammaZ0.**z0_port**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.
extract_distance	Determines physical distance from a transmission or reflection Network.
from_csv	Create a Media from numerical values stored in a csv file.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.

continues on next page

Table 30 – continued from previous page

short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.media.DefinedGammaZ0.__init__

DefinedGammaZ0.**__init__**(*frequency=None*, *z0_port=None*, *z0=50*, *Z0=None*, *gamma=1j*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*Number | Sequence[Number] | ndarray | None*)
- **z0** (*Number | Sequence[Number] | ndarray*)
- **Z0** (*Number | Sequence[Number] | ndarray | None*)
- **gamma** (*Number | Sequence[Number] | ndarray*)

skrf.media.media.DefinedGammaZ0.attenuator

DefinedGammaZ0.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns

ntwk – 2-port attenuator

Return type

Network object

skrf.media.media.DefinedGammaZ0.capacitor

DefinedGammaZ0.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
capacitor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.media.DefinedGammaZ0.capacitor_q

DefinedGammaZ0.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.media.DefinedGammaZ0.copy

DefinedGammaZ0.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.media.DefinedGammaZ0.delay_load

DefinedGammaZ0.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns

delay_load – a delayed load

Return type

Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.media.DefinedGammaZ0.delay_open

DefinedGammaZ0.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns
delay_open – a delayed open

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.media.DefinedGammaZ0.delay_short

DefinedGammaZ0.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns
delay_short – a delayed short

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.media.DefinedGammaZ0.electrical_length

DefinedGammaZ0.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns
theta – complex electrical length in radians or degrees, depending on value of deg.

Return type
number or array-like

skrf.media.media.DefinedGammaZ0.extract_distance

DefinedGammaZ0.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters
ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns
d – physical distance

Return type
number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.media.DefinedGammaZ0.from_csv

classmethod DefinedGammaZ0.**from_csv**(*filename*, **args*, ***kwargs*)

Create a Media from numerical values stored in a csv file.

The csv file format must be written by the function *write_csv()*, or similar method which produces the following format:

```
f[$unit], Re(Z0), Im(Z0), Re(gamma), Im(gamma), Re(port Z0), Im(port Z0)
1, 1, 1, 1, 1, 1, 1
2, 1, 1, 1, 1, 1, 1
.....
```

See also

write_csv

Parameters
filename (*str*)

Return type
Media

skrf.media.media.DefinedGammaZ0.get_array_of

DefinedGammaZ0.**get_array_of**(*x*)

skrf.media.media.DefinedGammaZ0.impedance_mismatch

DefinedGammaZ0.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
mismatch – a 2-port network representing the impedance mismatch

Return type
Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.media.DefinedGammaZ0.inductor

DefinedGammaZ0.**inductor**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.media.DefinedGammaZ0.inductor_q

DefinedGammaZ0.**inductor_q**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.media.DefinedGammaZ0.isolator

DefinedGammaZ0.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.media.DefinedGammaZ0.line

DefinedGammaZ0.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)

- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line – matched transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.media.DefinedGammaZ0.line_floating

DefinedGammaZ0.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line_floating – matched, floating transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.media.DefinedGammaZ0.load

DefinedGammaZ0.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

load – n-port load, where S = Gamma0*eye(...)

Return type

Network object

See also

match, *open*, *short*

skrf.media.media.DefinedGammaZ0.lossless_mismatch

DefinedGammaZ0.**lossless_mismatch**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns

ntwk – 2-port lossless mismatch

Return type

Network object

skrf.media.media.DefinedGammaZ0.match

DefinedGammaZ0.**match**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns
match – a n-port match

Return type
Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.media.DefinedGammaZ0.mode

DefinedGammaZ0.**mode**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.media.DefinedGammaZ0.open

DefinedGammaZ0.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.media.DefinedGammaZ0.plot

DefinedGammaZ0.**plot**(**args*, ***kw*)

skrf.media.media.DefinedGammaZ0.random

DefinedGammaZ0.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero
- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns

result – the network

Return type

Network object

skrf.media.media.DefinedGammaZ0.resistor

DefinedGammaZ0.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

resistor

Return type

a 2-port *Network*

> **See also**
>
> *match*, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.media.DefinedGammaZ0.short

`DefinedGammaZ0.`**`short`**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns

match – a n-port short circuit

Return type

Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

> **See also**
>
> *match*, *open*, *load*

skrf.media.media.DefinedGammaZ0.shunt

`DefinedGammaZ0.`**`shunt`**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns

shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type

Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.media.DefinedGammaZ0.shunt_capacitor

DefinedGammaZ0.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns

shunt_capacitor – shunted capacitor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.media.DefinedGammaZ0.shunt_delay_load

DefinedGammaZ0.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.media.DefinedGammaZ0.shunt_delay_open

DefinedGammaZ0.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.media.DefinedGammaZ0.shunt_delay_short

DefinedGammaZ0.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.media.DefinedGammaZ0.shunt_inductor

DefinedGammaZ0.**shunt_inductor**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.media.DefinedGammaZ0.shunt_resistor

DefinedGammaZ0.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.media.DefinedGammaZ0.splitter

`DefinedGammaZ0.splitter`(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.media.DefinedGammaZ0.tee

`DefinedGammaZ0.tee`(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.media.DefinedGammaZ0.theta_2_d

DefinedGammaZ0.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)
- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.media.DefinedGammaZ0.thru

DefinedGammaZ0.**thru**(*z0=None*, ***kwargs*)

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.media.DefinedGammaZ0.to_meters

DefinedGammaZ0.**to_meters**(*d*, *unit='deg'*)

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.media.DefinedGammaZ0.white_gaussian_polar

DefinedGammaZ0.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.media.DefinedGammaZ0.write_csv

DefinedGammaZ0.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters

filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

skrf.media.distributedCircuit.DistributedCircuit

class skrf.media.distributedCircuit.**DistributedCircuit**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *C=9e-11*, *L=2.8e-07*, *R=0*, *G=0*, **args*, ***kwargs*)

A transmission line mode defined in terms of distributed impedance and admittance values.

Parameters

- **frequency** (*Frequency* object) – frequency band of the media
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0_override** (*number, array-like, or None*) – *z0_override* override the characteristic impedance for the media. If *z0_override* is not None, the networks generated by the media have their characteristic impedance *z0* overridden by *z0_override*. (Default is None)
- **z0** (*number, array-like, or None*) – deprecated parameter, alias to *z0_override* if *z0_override* is None. Emit a deprecation warning.
- **C** (*number, or array-like*) – distributed capacitance, in F/m
- **L** (*number, or array-like*) – distributed inductance, in H/m
- **R** (*number, or array-like*) – distributed resistance, in Ohm/m
- **G** (*number, or array-like*) – distributed conductance, in S/m

Notes

if C,L,R,G are vectors they should be the same length

DistributedCircuit is *Media* object representing a transmission line mode defined in terms of distributed impedance and admittance values.

A *DistributedCircuit* may be defined in terms of the following attributes:

Quantity	Symbol	Property
Distributed Capacitance	$C^{'}$	`C`
Distributed Inductance	$L^{'}$	`L`
Distributed Resistance	$R^{'}$	`R`
Distributed Conductance	$G^{'}$	`G`

The following quantities may be calculated, which are functions of angular frequency (ω):

Quantity	Symbol	Property
Distributed Impedance	$Z^{'} = R^{'} + j\omega L^{'}$	*Z*
Distributed Admittance	$Y^{'} = G^{'} + j\omega C^{'}$	*Y*

The properties which define their wave behavior:

Quantity	Symbol	Method
Characteristic Impedance	$Z_0 = \sqrt{\frac{Z^{'}}{Y^{'}}}$	*Z0()*
Propagation Constant	$\gamma = \sqrt{Z^{'}Y^{'}}$	*gamma()*

Given the following definitions, the components of propagation constant are interpreted as follows:

$$+\Re e\{\gamma\} = \text{attenuation}$$
$$-\Im m\{\gamma\} = \text{forward propagation}$$

See also

from_media

Attributes

Y	Distributed Admittance, $Y^{'}$.
Z	Distributed Impedance, $Z^{'}$.
Z0	Characteristic Impedance
alpha	Real (attenuation) component of gamma.
beta	Imaginary (propagating) component of gamma.
gamma	Propagation Constant, γ.
npoints	Number of points of the frequency axis.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	Characteristic Impedance, z_0
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.distributedCircuit.DistributedCircuit.Y

property DistributedCircuit.**Y**: Number | Sequence[Number] | ndarray

Distributed Admittance, $Y^{'}$.

Defined as

$$Y^{'} = G^{'} + j\omega C^{'}$$

Returns
Y – Distributed Admittance in units of S/m

Return type
np.ndarray

skrf.media.distributedCircuit.DistributedCircuit.Z

property DistributedCircuit.**Z**: Number | Sequence[Number] | ndarray

Distributed Impedance, $Z^{'}$.

Defined as

$$Z^{'} = R^{'} + j\omega L^{'}$$

Returns
Z – Distributed impedance in units of ohm/m

Return type
np.ndarray

skrf.media.distributedCircuit.DistributedCircuit.Z0

property DistributedCircuit.**Z0**

Characteristic Impedance

Returns
z0 – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.distributedCircuit.DistributedCircuit.alpha

property DistributedCircuit.**alpha**: ndarray

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.distributedCircuit.DistributedCircuit.beta

property DistributedCircuit.**beta**: ndarray

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.distributedCircuit.DistributedCircuit.gamma

property DistributedCircuit.**gamma**: Number | Sequence[Number] | ndarray

Propagation Constant, γ.

Defined as,

$$\gamma = \sqrt{Z'Y'}$$

Returns
gamma – Propagation Constant,

Return type
np.ndarray

> **Note**
>
> The components of propagation constant are interpreted as follows:
>
> - positive real(gamma) = attenuation
> - positive imag(gamma) = forward propagation

skrf.media.distributedCircuit.DistributedCircuit.npoints

property DistributedCircuit.**npoints**: int

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.distributedCircuit.DistributedCircuit.v_g

property DistributedCircuit.**v_g**

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the j is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.distributedCircuit.DistributedCircuit.v_p

property DistributedCircuit.**v_p: ndarray**

Complex phase velocity (in m/s).

$$j \cdot \omega / \gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.distributedCircuit.DistributedCircuit.z0

property DistributedCircuit.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.distributedCircuit.DistributedCircuit.z0_characteristic

property DistributedCircuit.**z0_characteristic**: Number | Sequence[Number] | ndarray

Characteristic Impedance, z_0

$$Z_0 = \sqrt{\frac{Z'}{Y'}}$$

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.distributedCircuit.DistributedCircuit.z0_override

property DistributedCircuit.**z0_override**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.distributedCircuit.DistributedCircuit.z0_port

property DistributedCircuit.**z0_port**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.
extract_distance	Determines physical distance from a transmission or reflection Network.
from_csv	Create a *DistributedCircuit* from numerical values stored in a csv file.
from_media	Initializes a *DistributedCircuit* from an existing *Media* instance.
get_array_of	

continues on next page

Table 32 – continued from previous page

impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.distributedCircuit.DistributedCircuit.__init__

DistributedCircuit.**__init__**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *C=9e-11*, *L=2.8e-07*, *R=0*, *G=0*, **args*, ***kwargs*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*NumberLike | None*)
- **z0_override** (*NumberLike | None*)
- **z0** (*NumberLike | None*)
- **C** (*NumberLike*)
- **L** (*NumberLike*)
- **R** (*NumberLike*)
- **G** (*NumberLike*)

skrf.media.distributedCircuit.DistributedCircuit.attenuator

DistributedCircuit.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns

ntwk – 2-port attenuator

Return type

Network object

skrf.media.distributedCircuit.DistributedCircuit.capacitor

DistributedCircuit.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

capacitor

Return type

a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.distributedCircuit.DistributedCircuit.capacitor_q

DistributedCircuit.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor

- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.distributedCircuit.DistributedCircuit.copy

DistributedCircuit.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.distributedCircuit.DistributedCircuit.delay_load

DistributedCircuit.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.distributedCircuit.DistributedCircuit.delay_open

DistributedCircuit.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns
delay_open – a delayed open

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.distributedCircuit.DistributedCircuit.delay_short

DistributedCircuit.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns
delay_short – a delayed short

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.distributedCircuit.DistributedCircuit.electrical_length

DistributedCircuit.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns

theta – complex electrical length in radians or degrees, depending on value of deg.

Return type

number or array-like

skrf.media.distributedCircuit.DistributedCircuit.extract_distance

DistributedCircuit.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters

ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns

d – physical distance

Return type

number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.distributedCircuit.DistributedCircuit.from_csv

classmethod DistributedCircuit.**from_csv**(**args*, ***kw*)

Create a *DistributedCircuit* from numerical values stored in a csv file.

The csv file format must be written by the function *write_csv()*, or similar method which produces the following format:

```
f[$unit], Re(z0), Im(z0), Re(gamma), Im(gamma), Re(z0_port), Im(z0_port)
1, 1, 1, 1, 1, 1, 1
2, 1, 1, 1, 1, 1, 1
.....
```

See also

write_csv

skrf.media.distributedCircuit.DistributedCircuit.from_media

classmethod DistributedCircuit.**from_media**(*my_media*, **args*, ***kwargs*)

Initializes a *DistributedCircuit* from an existing *Media* instance.

Parameters

my_media (*Media* instance.) – the media object

Return type

Media

See also

Media

skrf.media.distributedCircuit.DistributedCircuit.get_array_of

DistributedCircuit.**get_array_of**(*x*)

skrf.media.distributedCircuit.DistributedCircuit.impedance_mismatch

DistributedCircuit.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

mismatch – a 2-port network representing the impedance mismatch

Return type

Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.distributedCircuit.DistributedCircuit.inductor

`DistributedCircuit.`**`inductor`**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.distributedCircuit.DistributedCircuit.inductor_q

`DistributedCircuit.`**`inductor_q`**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.distributedCircuit.DistributedCircuit.isolator

DistributedCircuit.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.distributedCircuit.DistributedCircuit.line

DistributedCircuit.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line – matched transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.distributedCircuit.DistributedCircuit.line_floating

DistributedCircuit.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

line_floating – matched, floating transmission line of given length

Return type

Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.distributedCircuit.DistributedCircuit.load

DistributedCircuit.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

load – n-port load, where S = Gamma0*eye(...)

Return type

Network object

See also

match, *open*, *short*

skrf.media.distributedCircuit.DistributedCircuit.lossless_mismatch

DistributedCircuit.**lossless_mismatch**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns

ntwk – 2-port lossless mismatch

Return type

Network object

skrf.media.distributedCircuit.DistributedCircuit.match

DistributedCircuit.**match**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns

match – a n-port match

Return type

Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.distributedCircuit.DistributedCircuit.mode

DistributedCircuit.**mode**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.distributedCircuit.DistributedCircuit.open

DistributedCircuit.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.distributedCircuit.DistributedCircuit.plot

DistributedCircuit.**plot**(**args*, ***kw*)

skrf.media.distributedCircuit.DistributedCircuit.random

DistributedCircuit.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero

- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns
result – the network

Return type
Network object

skrf.media.distributedCircuit.DistributedCircuit.resistor

DistributedCircuit.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
resistor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.distributedCircuit.DistributedCircuit.short

DistributedCircuit.**short**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns
match – a n-port short circuit

Return type
Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.distributedCircuit.DistributedCircuit.shunt

DistributedCircuit.**shunt**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns
shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type
Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.distributedCircuit.DistributedCircuit.shunt_capacitor

DistributedCircuit.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
shunt_capacitor – shunted capacitor (2-port)

Return type
Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.distributedCircuit.DistributedCircuit.shunt_delay_load

DistributedCircuit.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.distributedCircuit.DistributedCircuit.shunt_delay_open

DistributedCircuit.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.distributedCircuit.DistributedCircuit.shunt_delay_short

DistributedCircuit.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.distributedCircuit.DistributedCircuit.shunt_inductor

DistributedCircuit.**shunt_inductor**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.distributedCircuit.DistributedCircuit.shunt_resistor

DistributedCircuit.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.distributedCircuit.DistributedCircuit.splitter

DistributedCircuit.**splitter**(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.distributedCircuit.DistributedCircuit.tee

DistributedCircuit.**tee**(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.distributedCircuit.DistributedCircuit.theta_2_d

DistributedCircuit.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)

- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.distributedCircuit.DistributedCircuit.thru

DistributedCircuit.**thru**(*z0=None*, ***kwargs*)

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.distributedCircuit.DistributedCircuit.to_meters

DistributedCircuit.**to_meters**(*d*, *unit='deg'*)

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.distributedCircuit.DistributedCircuit.white_gaussian_polar

DistributedCircuit.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.distributedCircuit.DistributedCircuit.write_csv

DistributedCircuit.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters
filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

skrf.media.rectangularWaveguide.RectangularWaveguide

class skrf.media.rectangularWaveguide.**RectangularWaveguide**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *a=1*, *b=None*, *mode_type='te'*, *m=1*, *n=0*, *ep_r=1*, *mu_r=1*, *rho=1.724137931034483e-08*, *roughness=None*, *model='lomakin'*, *dielectric=None*, *wall_a=None*, *wall_b=None*, **args*, ***kwargs*)

A single mode of a homogeneously filled rectangular waveguide.

Parameters

- **frequency** (*Frequency* object) – frequency band of this transmission line medium
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0_override** (*number, array-like, or None*) – *z0_override* override the characteristic impedance for the media. If *z0_override* is not None, the networks generated by the media have their characteristic impedance *z0* overridden by *z0_override*. (Default is None)
- **z0** (*number, array-like, or None*) – deprecated parameter, alias to *z0_override* if *z0_override* is None. Emit a deprecation warning.
- **a** (*number, optional*) – width of waveguide, in meters. Default is 1.
- **b** (*number or None, optional*) – height of waveguide, in meters. If *None* defaults to a/2. Default is None
- **mode_type** (*['te','tm']*) – mode type, transverse electric (te) or transverse magnetic (tm) to-z. where z is direction of propagation
- **m** (*int*) – mode index in 'a'-direction
- **n** (*int*) – mode index in 'b'-direction
- **ep_r** (*number, array-like,*) – filling material's relative permittivity
- **mu_r** (*number, array-like*) – filling material's relative permeability
- **rho** (*number, array-like, string, or None*) – resistivity (ohm-m) of the conductor walls. If array-like must be same length as frequency. if str, it must be a key in *skrf.data.materials*. Default is 1/58e6, the resistivity of annealed copper at 20 °C (100% IACS). Use 0, or None, for perfectly conducting walls.
- **roughness** (*number, or array-like*) – rms roughness of the conductor walls, in meter. It is handed to *surface_impedance()*, and it applies to any wall whose material does not carry an `'rms_roughness'` of its own.
- **model** (*str, optional*) – Model of the loss. 'lomakin' (default) is the two-wire model of[1], where the loss of the walls shapes the phase constant as well as the attenuation (agrees better with EM simulations). 'marcuvitz' is the power loss method of[2], which is also given by the IEC 60153-2 and IEEE 1785.1 standards and discussed in[3]; it leaves the phase constant at its lossless value. A TM mode, or a mode with both indices nonzero, is not covered by 'lomakin' and follows 'marcuvitz' instead, which is warned about.
- **dielectric** (*dict or None, optional*) – Material filling the waveguide, as a dict taking the keys `'ep_r'` (complex relative permittivity) and `'mu_r'` (complex relative permeability), both defaulting to 1. If None (default), the filling is described by parameters *ep_r* and *mu_r*.

[1] K. Lomakin, G. Gold and K. Helmreich, "Analytical Waveguide Model Precisely Predicting Loss and Delay Including Surface Roughness," IEEE Trans. Microw. Theory Techn., vol. 66, no. 6, pp. 2649-2662, June 2018. https://doi.org/10.1109/TMTT.2018.2827383

[2] N. Marcuvitz, "Transmission-line modes," in Waveguide Handbook, 2nd ed. New York: McGraw-Hill, 1951, Chapter 2.

[3] J. Skinner, D. Koller, H.-U. Nickel, N. M. Ridler and S. Lucyszyn, "Derivation of Rectangular Metallic Waveguide Attenuation Constant for IEC 60153-2 and IEEE 1785.1 International Standards," IEEE Trans. THz Sci. Technol., vol. 15, no. 4, pp. 734-737, July 2025. https://doi.org/10.1109/TTHZ.2025.3573847

- **wall_a** (*dict, list of dict, or None, optional*) – Material of the pair of walls of width *a*. It describes both walls of the pair. A single dict describes a bulk conductor, a list of dict a stack of coatings ordered from the filling inwards, of which the deepest layer is the bulk. Each dict takes the keys `'sigma'`, `'mu_r'` and `'ep_r'` of the layer itself, and `'rms_roughness'`, `'boundary_loc'` and `'distribution'` of the boundary on top of it, which are handed to *surface_impedance()*. If None (default), the pair is described by *rho* and *roughness*.
- **wall_b** (*dict, list of dict, or None, optional*) – Material of the pair of walls of width *b*, described in similar way as *wall_a*. The two pairs are usually of the same material, but would have different effective conductivities, e.g., different roughness due to machining or 3D printed waveguides[4],[5].
- ***args** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*
- ****kwargs** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*

Note

The two-wire model is derived for the TE10 mode. It carries over to any TE_m0 and TE_0n mode. It does not carry over to a mode with both indices nonzero.

References

Examples

Most common usage is standard aspect ratio (2:1) dominant mode, TE10 mode of wr10 waveguide can be constructed by

```
>>> import numpy as np
>>> import skrf as rf
>>> from skrf.constants import mil
>>> from skrf.media import RectangularWaveguide
>>> freq = rf.Frequency(75, 110, 101, unit='ghz')
>>> RectangularWaveguide(freq, a=100*mil)
Rectangular Waveguide Media.  75.0-110.0 GHz.  101 points
 a= 2.54e-03m, b= 1.27e-03m
```

A WR-12 guide of brass, with the *a* walls rougher than the *b* ones, and the attenuation of a 100 mm length of it at 90 GHz:

```
>>> freq = rf.Frequency(60, 90, 61, unit='GHz')
>>> wr12 = RectangularWaveguide(freq, a=3.0988e-3, b=1.5494e-3,
...     wall_a={'sigma': 0.28*58e6, 'rms_roughness': 1e-6},
...     wall_b={'sigma': 0.28*58e6, 'rms_roughness': 0.4e-6})
>>> print(f"{wr12.line(100, 'mm').s_db[-1, 1, 0]:.3f} dB")
-1.002 dB
```

A WR-6.5 waveguide (D-band) whose walls carry an ENIG finish, 0.05 um of gold over 4 um of nickel over copper, roughened by 50 nm rms at the outside and by 0.2 um rms at the copper underneath:

[4] L. Zhu et al., "3-D Printed THz Waveguide Components," IEEE Access, vol. 11, pp. 79073-79086, 2023. https://doi.org/10.1109/ACCESS.2023.3297271

[5] K. Lomakin et al., "SLA-Printed 3-D Waveguide Paths for E-Band Using Electroless Silver Plating," IEEE Trans. Compon. Packag. Manuf. Technol., vol. 9, no. 12, pp. 2476-2481, Dec. 2019. https://doi.org/10.1109/TCPMT.2019.2927671

```
>>> enig = [{'sigma': 41.1e6, 'rms_roughness': 50e-9, 'boundary_loc': 0},
...         {'sigma': 14.5e6, 'mu_r': 20, 'rms_roughness': 50e-9, 'boundary_loc': 0.
↪05e-6},
...         {'sigma': 58e6, 'rms_roughness': 0.2e-6, 'boundary_loc': 4.05e-6}]
>>> freq = rf.Frequency(110, 170, 61, unit='GHz')
>>> wr65 = RectangularWaveguide(freq, a=1.6510e-3, b=0.8255e-3,
...                             wall_a=enig, wall_b=enig)
>>> smooth_copper = RectangularWaveguide(freq, a=1.6510e-3, b=0.8255e-3, rho=1/58e6)
>>> np2db = 20*np.log10(np.e)
>>> print(f"{np2db*wr65.gamma[-1].real:.1f} dB/m, against "
...       f"{np2db*smooth_copper.gamma[-1].real:.1f} dB/m for smooth copper")
20.3 dB/m, against 4.6 dB/m for smooth copper
```

Attributes

Z0	Characteristic Impedance
alpha	Real (attenuation) component of gamma.
alpha_c	Loss of the walls by the power loss method, in Np/m.
beta	Imaginary (propagating) component of gamma.
ep	The permittivity of the filling material.
f_cutoff	cutoff frequency for this mode.
f_norm	Frequency vector normalized to cutoff.
gamma	The propagation constant (aka Longitudinal wave number).
k0	Characteristic wave number.
kc	Cut-off wave number.
kx	Eigenvalue in the 'a' direction.
ky	Eigenvalue in the *b* direction.
lambda_cutoff	Cutoff wavelength.
lambda_guide	Guide wavelength.
mu	The permeability of the filling material.
npoints	Number of points of the frequency axis.
rho	Resistivity of all four walls in ohm*m.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	The characteristic impedance, z_0.
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.rectangularWaveguide.RectangularWaveguide.Z0

property RectangularWaveguide.**Z0**

Characteristic Impedance

Returns
z0 – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.rectangularWaveguide.RectangularWaveguide.alpha

property RectangularWaveguide.**alpha: ndarray**

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.rectangularWaveguide.RectangularWaveguide.alpha_c

property RectangularWaveguide.**alpha_c: NumberLike**

Loss of the walls by the power loss method, in Np/m. This is ch. 2, eq. (14a) of Marcuvitz, which the IEC 60153-2 and IEEE 1785.1 standards give for the attenuation constant. See the references of *RectangularWaveguide*.

$$\alpha = \frac{R_{s,a} + \frac{2b}{a}\left(\frac{f_c}{f}\right)^2 R_{s,b}}{\eta b \sqrt{1 - \left(\frac{f_c}{f}\right)^2}}$$

where η is the intrinsic impedance of the filling and R_s the surface resistance of a pair of walls. The two pairs are carried separately, unlike ch. 2, eq. (14a) as published, which the same material on both recovers.

It is derived for TE10 and holds for any TE_m0 and TE_0n mode, the mode entering only through f_c. Any other mode falls back to it and is only estimated, its *b* walls also carrying longitudinal current. It diverges at cutoff and is zero below it, where the evanescent decay of the mode dwarfs the loss of the walls.

See also

rho

skrf.media.rectangularWaveguide.RectangularWaveguide.beta

property RectangularWaveguide.**beta: ndarray**

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.rectangularWaveguide.RectangularWaveguide.ep

property RectangularWaveguide.**ep: NumberLike**

The permittivity of the filling material.

$$\varepsilon = \varepsilon_r \varepsilon_0$$

Returns
ep – filling material's permittivity in F/m.

Return type
number

skrf.media.rectangularWaveguide.RectangularWaveguide.f_cutoff

property RectangularWaveguide.**f_cutoff: NumberLike**

cutoff frequency for this mode.

$$f_c = \frac{v}{2\pi}\sqrt{m\frac{\pi}{a}^2 + n\frac{\pi}{b}^2}$$

where $v = 1/\sqrt{\varepsilon\mu}$.

skrf.media.rectangularWaveguide.RectangularWaveguide.f_norm

property RectangularWaveguide.**f_norm: NumberLike**

Frequency vector normalized to cutoff.

skrf.media.rectangularWaveguide.RectangularWaveguide.gamma

property RectangularWaveguide.**gamma: NumberLike**

The propagation constant (aka Longitudinal wave number).

Defined as

$$k_z = \sqrt{k_c^2 - k_0^2}$$

With a lossless filling this is purely imaginary above cutoff, where the mode propagates, and purely real below it, where the mode is evanescent. A lossy filling or lossy walls make it complex throughout.

Returns
gamma – The propagation constant

Return type
number

skrf.media.rectangularWaveguide.RectangularWaveguide.k0

property RectangularWaveguide.**k0: NumberLike**

Characteristic wave number.

$$k_0 = \frac{\omega}{v} = \omega\sqrt{\varepsilon\mu}$$

Returns
k0 – characteristic wave number

Return type
number

skrf.media.rectangularWaveguide.RectangularWaveguide.kc

property RectangularWaveguide.**kc: NumberLike**

Cut-off wave number.

Defined as

$$k_c = \sqrt{k_x^2 + k_y^2} = \sqrt{m\frac{\pi}{a}^2 + n\frac{\pi}{b}^2}$$

Returns
kc – cut-off wavenumber

Return type
number

skrf.media.rectangularWaveguide.RectangularWaveguide.kx

property RectangularWaveguide.**kx: NumberLike**

Eigenvalue in the 'a' direction.

Defined as

$$k_x = m\frac{\pi}{a}$$

Returns
kx – eigenvalue in a direction

Return type
number

skrf.media.rectangularWaveguide.RectangularWaveguide.ky

property RectangularWaveguide.**ky: NumberLike**

Eigenvalue in the b direction.

Defined as

$$k_y = n\frac{\pi}{b}$$

Returns
ky – eigenvalue in b direction

Return type
number

skrf.media.rectangularWaveguide.RectangularWaveguide.lambda_cutoff

property RectangularWaveguide.**lambda_cutoff: NumberLike**

Cutoff wavelength.

$$\lambda_c = v/f_c$$

where $v = 1/\sqrt{\varepsilon\mu}$ and f_c the cut-off frequency.

See also

f_cutoff

skrf.media.rectangularWaveguide.RectangularWaveguide.lambda_guide

property RectangularWaveguide.**lambda_guide: NumberLike**

Guide wavelength.

$$\lambda_g = \frac{2\pi}{\beta}$$

The distance in which the phase of the field increases by 2 pi.

See also

k0

skrf.media.rectangularWaveguide.RectangularWaveguide.mu

property RectangularWaveguide.**mu: NumberLike**

The permeability of the filling material.

$$\mu = \mu_r \mu_0$$

Returns
mu – filling material's permeability in H/m.

Return type
number

skrf.media.rectangularWaveguide.RectangularWaveguide.npoints

property RectangularWaveguide.**npoints: int**

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.rectangularWaveguide.RectangularWaveguide.rho

property RectangularWaveguide.**rho: None | NumberLike**

Resistivity of all four walls in ohm*m.

This is the bulk resistivity of the metal, and it describes any wall pair that *wall_a* or *wall_b* does not describe itself. Roughness is not folded into it. Zero, or None, is a perfect conductor.

Parameters
val (*float, array-like, str or None*) – the resistivity in ohm*m. If array-like must be same length as self.frequency. if str, it must be a key in *skrf.data.materials*.

Examples

```
>>> import numpy as np
>>> import skrf as rf
>>> from skrf.media import RectangularWaveguide
>>> wg = RectangularWaveguide(rf.Frequency(75, 110, 101, unit='GHz'), a=2.54e-3)
>>> wg.rho = 2.8e-8
>>> wg.rho = 2.8e-8*np.ones(len(wg.frequency))
>>> wg.rho = 'al'
>>> wg.rho = 'aluminum'
>>> wg.rho = 0
```

skrf.media.rectangularWaveguide.RectangularWaveguide.v_g

property RectangularWaveguide.**v_g**

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the *j* is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.rectangularWaveguide.RectangularWaveguide.v_p

property RectangularWaveguide.**v_p: ndarray**

Complex phase velocity (in m/s).

$$j \cdot \omega/\gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.rectangularWaveguide.RectangularWaveguide.z0

property RectangularWaveguide.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.rectangularWaveguide.RectangularWaveguide.z0_characteristic

property RectangularWaveguide.**z0_characteristic: NumberLike**

The characteristic impedance, z_0.

The characteristic impedance depends of the mode ('te' or 'tm').

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.rectangularWaveguide.RectangularWaveguide.z0_override

property RectangularWaveguide.**z0_override: ndarray**

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.rectangularWaveguide.RectangularWaveguide.z0_port

property RectangularWaveguide.**z0_port: ndarray**

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.

continues on next page

Table 34 – continued from previous page

electrical_length	Calculate the complex electrical length for a given distance.
extract_distance	Determines physical distance from a transmission or reflection Network.
from_z0	Initialize from specified impedance at a given frequency, assuming the fundamental TE10 mode.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.rectangularWaveguide.RectangularWaveguide.__init__

RectangularWaveguide.**__init__**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *a=1*, *b=None*, *mode_type='te'*, *m=1*, *n=0*, *ep_r=1*, *mu_r=1*, *rho=1.724137931034483e-08*, *roughness=None*, *model='lomakin'*, *dielectric=None*, *wall_a=None*, *wall_b=None*, **args*, ***kwargs*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*NumberLike | None*)
- **z0_override** (*NumberLike | None*)
- **z0** (*NumberLike | None*)

- **a** (*float*)
- **b** (*float | None*)
- **mode_type** (*str*)
- **m** (*int*)
- **n** (*int*)
- **ep_r** (*None | NumberLike*)
- **mu_r** (*None | NumberLike*)
- **rho** (*None | NumberLike | str*)
- **roughness** (*None | NumberLike*)
- **model** (*str*)
- **dielectric** (*dict | None*)
- **wall_a** (*dict | list[dict] | None*)
- **wall_b** (*dict | list[dict] | None*)

skrf.media.rectangularWaveguide.RectangularWaveguide.attenuator

RectangularWaveguide.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns

ntwk – 2-port attenuator

Return type

Network object

skrf.media.rectangularWaveguide.RectangularWaveguide.capacitor

RectangularWaveguide.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
capacitor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.rectangularWaveguide.RectangularWaveguide.capacitor_q

RectangularWaveguide.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.rectangularWaveguide.RectangularWaveguide.copy

RectangularWaveguide.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.rectangularWaveguide.RectangularWaveguide.delay_load

RectangularWaveguide.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.rectangularWaveguide.RectangularWaveguide.delay_open

RectangularWaveguide.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns
delay_open – a delayed open

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.rectangularWaveguide.RectangularWaveguide.delay_short

RectangularWaveguide.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns
delay_short – a delayed short

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.rectangularWaveguide.RectangularWaveguide.electrical_length

RectangularWaveguide.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns
theta – complex electrical length in radians or degrees, depending on value of deg.

Return type
number or array-like

skrf.media.rectangularWaveguide.RectangularWaveguide.extract_distance

RectangularWaveguide.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters
ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns
d – physical distance

Return type
number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.rectangularWaveguide.RectangularWaveguide.from_z0

classmethod RectangularWaveguide.**from_z0**(*frequency*, *z0*, *f*, *ep_r=1*, *mu_r=1*, ***kw*)

Initialize from specified impedance at a given frequency, assuming the fundamental TE10 mode.

Parameters

- **frequency** (*Frequency Object*)
- **z0** (*number /array*) – characteristic impedance to create at *f*
- **f** (*number*) – frequency (in Hz) at which the resultant waveguide has the characteristic impedance z0
- **ep_r** (*number, array-like,*) – filling material's relative permittivity
- **mu_r** (*number, array-like*) – filling material's relative permeability

Return type
Media

skrf.media.rectangularWaveguide.RectangularWaveguide.get_array_of

RectangularWaveguide.**get_array_of**(*x*)

skrf.media.rectangularWaveguide.RectangularWaveguide.impedance_mismatch

RectangularWaveguide.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
mismatch – a 2-port network representing the impedance mismatch

Return type
Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.rectangularWaveguide.RectangularWaveguide.inductor

`RectangularWaveguide.`**`inductor`**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.rectangularWaveguide.RectangularWaveguide.inductor_q

`RectangularWaveguide.`**`inductor_q`**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.rectangularWaveguide.RectangularWaveguide.isolator

RectangularWaveguide.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.rectangularWaveguide.RectangularWaveguide.line

RectangularWaveguide.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line – matched transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.rectangularWaveguide.RectangularWaveguide.line_floating

RectangularWaveguide.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

line_floating – matched, floating transmission line of given length

Return type

Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.rectangularWaveguide.RectangularWaveguide.load

RectangularWaveguide.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

load – n-port load, where S = Gamma0*eye(...)

Return type

Network object

See also

match, *open*, *short*

skrf.media.rectangularWaveguide.RectangularWaveguide.lossless_mismatch

`RectangularWaveguide.`**`lossless_mismatch`**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns

ntwk – 2-port lossless mismatch

Return type

Network object

skrf.media.rectangularWaveguide.RectangularWaveguide.match

`RectangularWaveguide.`**`match`**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns

match – a n-port match

Return type

Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.rectangularWaveguide.RectangularWaveguide.mode

`RectangularWaveguide.`**`mode`**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.rectangularWaveguide.RectangularWaveguide.open

RectangularWaveguide.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.rectangularWaveguide.RectangularWaveguide.plot

RectangularWaveguide.**plot**(**args*, ***kw*)

skrf.media.rectangularWaveguide.RectangularWaveguide.random

RectangularWaveguide.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero

- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns
result – the network

Return type
Network object

skrf.media.rectangularWaveguide.RectangularWaveguide.resistor

RectangularWaveguide.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
resistor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.rectangularWaveguide.RectangularWaveguide.short

RectangularWaveguide.**short**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns
match – a n-port short circuit

Return type
Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.rectangularWaveguide.RectangularWaveguide.shunt

RectangularWaveguide.**shunt**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns
shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type
Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.rectangularWaveguide.RectangularWaveguide.shunt_capacitor

RectangularWaveguide.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
shunt_capacitor – shunted capacitor (2-port)

Return type
Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.rectangularWaveguide.RectangularWaveguide.shunt_delay_load

RectangularWaveguide.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.rectangularWaveguide.RectangularWaveguide.shunt_delay_open

RectangularWaveguide.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.rectangularWaveguide.RectangularWaveguide.shunt_delay_short

RectangularWaveguide.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.rectangularWaveguide.RectangularWaveguide.shunt_inductor

RectangularWaveguide.**shunt_inductor**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.rectangularWaveguide.RectangularWaveguide.shunt_resistor

RectangularWaveguide.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.rectangularWaveguide.RectangularWaveguide.splitter

RectangularWaveguide.**splitter**(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.rectangularWaveguide.RectangularWaveguide.tee

RectangularWaveguide.**tee**(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.rectangularWaveguide.RectangularWaveguide.theta_2_d

RectangularWaveguide.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)

- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.rectangularWaveguide.RectangularWaveguide.thru

RectangularWaveguide.**thru**(*z0=None*, ***kwargs*)

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.rectangularWaveguide.RectangularWaveguide.to_meters

RectangularWaveguide.**to_meters**(*d*, *unit='deg'*)

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.rectangularWaveguide.RectangularWaveguide.white_gaussian_polar

RectangularWaveguide.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.rectangularWaveguide.RectangularWaveguide.write_csv

RectangularWaveguide.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters
filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

skrf.media.circularWaveguide.CircularWaveguide

class skrf.media.circularWaveguide.**CircularWaveguide**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *r=1*, *mode_type='te'*, *m=1*, *n=1*, *ep_r=1*, *mu_r=1*, *rho=None*, **args*, ***kwargs*)

A single mode of a homogeneously filled Circular Waveguide

Represents a single mode of a homogeneously filled circular waveguide of cross-section *r^2 pi*. The mode is determined by *mode-type* (*'te'* or *'tm'*) and mode indices (*m* and *n*). Corrugated circular waveguides, which also support HE modes, are not supported.

Quantity	Symbol	Variable
Characteristic Wave Number	k_0	*k0*
Cut-off Wave Number	k_c	*kc*
Longitudinal Wave Number	k_z	*gamma*
Transverse Wave Number (a)	k_x	kx
Transverse Wave Number (b)	k_y	ky
Characteristic Impedance	z_0	*z0*

Parameters

- **frequency** (*Frequency* object) – frequency band of this transmission line medium
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0_override** (*number, array-like, or None*) – *z0_override* override the characteristic impedance for the media. If *z0_override* is not None, the networks generated by the media have their characteristic impedance *z0* overridden by *z0_override*. (Default is None)
- **z0** (*number, array-like, or None*) – deprecated parameter, alias to *z0_override* if *z0_override* is None. Emit a deprecation warning.
- **r** (*number*) – radius of the waveguide, in meters.
- **mode_type** (*['te','tm']*) – mode type, transverse electric (te) or transverse magnetic (tm) to-z. where z is direction of propagation
- **m** (*int*) – mode index in 'phi'-direction, the azimuthal index
- **n** (*int*) – mode index in 'r'-direction, the radial index
- **ep_r** (*number, array-like,*) – filling material's relative permittivity
- **mu_r** (*number, array-like*) – filling material's relative permeability
- **rho** (*number, array-like, string*) – resistivity (ohm-m) of the conductor walls. If array-like must be same length as frequency. if str, it must be a key in *skrf.data.materials*.
- ***args** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*
- ****kwargs** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*

Examples

In the following example an ideal waveguide of 2.39 mm diameter is constructed for the high W band, operated in the fundamental TE11 mode. If no conductivity is provided the walls are treated as perfect electric conductors.

```
>>> freq = rf.Frequency(88, 110, 101, 'ghz')
>>> rf.CircularWaveguide(freq, r=0.5 * 2.39e-3)
```

Attributes

Z0	Characteristic Impedance
alpha	Real (attenuation) component of gamma.
alpha_c	Loss due to finite conductivity of the sidewalls.
beta	Imaginary (propagating) component of gamma.
ep	The permativity of the filling material.
f_cutoff	cutoff frequency for this mode
f_norm	frequency vector normalized to cutoff
gamma	The propagation constant (aka Longitudinal wave number)
k0	Characteristic wave number.
kc	Cut-off wave number.
lambda_cutoff	Cutoff wavelength.
lambda_guide	Guide wavelength.
mu	The permeability of the filling material.
npoints	Number of points of the frequency axis.
rho	conductivity of sidewalls in ohm*m
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	The characteristic impedance, z_0.
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.circularWaveguide.CircularWaveguide.Z0

property CircularWaveguide.**Z0**

Characteristic Impedance

Returns
z0 – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.circularWaveguide.CircularWaveguide.alpha

property CircularWaveguide.**alpha: ndarray**

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.circularWaveguide.CircularWaveguide.alpha_c

property CircularWaveguide.**alpha_c: NumberLike**

Loss due to finite conductivity of the sidewalls. Effects due to surface roughness are not implemented.

In units of Np/m

See property *rho* for setting conductivity.

Effects of finite conductivity are taken from [#].

References

See also

rho

skrf.media.circularWaveguide.CircularWaveguide.beta

property CircularWaveguide.**beta: ndarray**

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.circularWaveguide.CircularWaveguide.ep

property CircularWaveguide.**ep: NumberLike**

The permativity of the filling material.

Returns
ep – filling material's relative permittivity

Return type
number

skrf.media.circularWaveguide.CircularWaveguide.f_cutoff

property CircularWaveguide.**f_cutoff: NumberLike**

cutoff frequency for this mode

$$f_c = \frac{v}{2\pi}\frac{u_{mn}}{R}$$

where R is the radius of the waveguide, and u_mn is:

- the n-th root of the m-th Bessel function for 'tm' mode
- the n-th root of the Derivative of the m-th Bessel function for 'te' mode.

and v= 1/sqrt(ep*mu) is the bulk velocity inside the filling material.

skrf.media.circularWaveguide.CircularWaveguide.f_norm

property CircularWaveguide.**f_norm: NumberLike**

frequency vector normalized to cutoff

skrf.media.circularWaveguide.CircularWaveguide.gamma

property CircularWaveguide.**gamma: NumberLike**

The propagation constant (aka Longitudinal wave number)

Defined as

$$k_z = \pm j\sqrt{k_0^2 - k_c^2}$$

This is:

- IMAGINARY for propagating modes
- REAL for non-propagating modes,

Returns
gamma – The propagation constant

Return type
number

skrf.media.circularWaveguide.CircularWaveguide.k0

property CircularWaveguide.**k0: NumberLike**

Characteristic wave number.

$$k_0 = \omega\sqrt{\varepsilon\mu}$$

Returns
k0 – characteristic wave number

Return type
number

skrf.media.circularWaveguide.CircularWaveguide.kc

property CircularWaveguide.**kc: NumberLike**

Cut-off wave number.

Defined as

$$k_c = \frac{u_{mn}}{R}$$

where R is the radius of the waveguide, and u_mn is:

- the n-th root of the m-th Bessel function for 'tm' mode
- the n-th root of the Derivative of the m-th Bessel function for 'te' mode.

Returns
kc – cut-off wavenumber

Return type
number

skrf.media.circularWaveguide.CircularWaveguide.lambda_cutoff

property CircularWaveguide.**lambda_cutoff: NumberLike**

Cutoff wavelength.

$$\lambda_c = v/f_c$$

where v= 1/sqrt(ep*mu)

skrf.media.circularWaveguide.CircularWaveguide.lambda_guide

property CircularWaveguide.**lambda_guide: NumberLike**

Guide wavelength.

$$\lambda_g = 2\pi/\beta$$

the distance in which the phase of the field increases by 2 pi.

skrf.media.circularWaveguide.CircularWaveguide.mu

property CircularWaveguide.**mu: NumberLike**

The permeability of the filling material.

Returns
mu – filling material's relative permeability

Return type
number

skrf.media.circularWaveguide.CircularWaveguide.npoints

property CircularWaveguide.**npoints: int**

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.circularWaveguide.CircularWaveguide.rho

property CircularWaveguide.**rho: NumberLike**

conductivity of sidewalls in ohm*m

Parameters
val (*float, array-like or str*) – the conductivity in ohm*m. If array-like must be same length as self.frequency. if str, it must be a key in *skrf.data.materials*.

Examples

```
>>> wg.rho = 2.8e-8
>>> wg.rho = 2.8e-8 * ones(len(wg.frequency))
>>> wg.rho = 'al'
>>> wg.rho = 'aluminum'
```

skrf.media.circularWaveguide.CircularWaveguide.v_g

property CircularWaveguide.**v_g**

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the *j* is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.circularWaveguide.CircularWaveguide.v_p

property CircularWaveguide.**v_p: ndarray**

Complex phase velocity (in m/s).

$$j \cdot \omega / \gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.circularWaveguide.CircularWaveguide.z0

property CircularWaveguide.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.circularWaveguide.CircularWaveguide.z0_characteristic

property CircularWaveguide.**z0_characteristic: NumberLike**

The characteristic impedance, z_0.

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.circularWaveguide.CircularWaveguide.z0_override

property CircularWaveguide.**z0_override: ndarray**

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.circularWaveguide.CircularWaveguide.z0_port

property CircularWaveguide.**z0_port: ndarray**

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.
extract_distance	Determines physical distance from a transmission or reflection Network.
from_z0	Initialize from specified impedance at a given frequency, assuming the fundamental TE11 mode.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.

continues on next page

Table 36 – continued from previous page

inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.circularWaveguide.CircularWaveguide.__init__

CircularWaveguide.**__init__**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *r=1*, *mode_type='te'*, *m=1*, *n=1*, *ep_r=1*, *mu_r=1*, *rho=None*, **args*, ***kwargs*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*NumberLike | None*)
- **z0_override** (*NumberLike | None*)
- **z0** (*NumberLike | None*)
- **r** (*NumberLike*)
- **mode_type** (*str*)
- **m** (*int*)
- **n** (*int*)
- **ep_r** (*NumberLike*)
- **mu_r** (*NumberLike*)
- **rho** (*NumberLike | str | None*)

skrf.media.circularWaveguide.CircularWaveguide.attenuator

CircularWaveguide.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns

ntwk – 2-port attenuator

Return type

Network object

skrf.media.circularWaveguide.CircularWaveguide.capacitor

CircularWaveguide.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

capacitor

Return type

a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.circularWaveguide.CircularWaveguide.capacitor_q

CircularWaveguide.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor

- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.circularWaveguide.CircularWaveguide.copy

`CircularWaveguide.`**`copy`**`()`

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
`Media`

skrf.media.circularWaveguide.CircularWaveguide.delay_load

`CircularWaveguide.`**`delay_load`**`(`*Gamma0*, *d*, *unit='deg'*, ***kwargs*`)`

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.circularWaveguide.CircularWaveguide.delay_open

CircularWaveguide.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns
delay_open – a delayed open

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.circularWaveguide.CircularWaveguide.delay_short

CircularWaveguide.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns
delay_short – a delayed short

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.circularWaveguide.CircularWaveguide.electrical_length

CircularWaveguide.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns

theta – complex electrical length in radians or degrees, depending on value of deg.

Return type

number or array-like

skrf.media.circularWaveguide.CircularWaveguide.extract_distance

CircularWaveguide.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters

ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns

d – physical distance

Return type

number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.circularWaveguide.CircularWaveguide.from_z0

classmethod CircularWaveguide.**from_z0**(*frequency*, *z0*, *f*, *ep_r=1*, *mu_r=1*, ***kwargs*)

Initialize from specified impedance at a given frequency, assuming the fundamental TE11 mode.

Parameters

- **frequency** (*Frequency Object*)
- **z0** (*number /array*) – characteristic impedance to create at *f*
- **f** (*number*) – frequency (in Hz) at which the resultant waveguide has the characteristic impedance Z0
- **ep_r** (*number, array-like,*) – filling material's relative permittivity
- **mu_r** (*number, array-like*) – filling material's relative permeability
- ****kwargs** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*

skrf.media.circularWaveguide.CircularWaveguide.get_array_of

CircularWaveguide.**get_array_of**(*x*)

skrf.media.circularWaveguide.CircularWaveguide.impedance_mismatch

CircularWaveguide.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

mismatch – a 2-port network representing the impedance mismatch

Return type

Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.circularWaveguide.CircularWaveguide.inductor

CircularWaveguide.**inductor**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.circularWaveguide.CircularWaveguide.inductor_q

CircularWaveguide.**inductor_q**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.circularWaveguide.CircularWaveguide.isolator

CircularWaveguide.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.circularWaveguide.CircularWaveguide.line

CircularWaveguide.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

line – matched transmission line of given length

Return type

Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.circularWaveguide.CircularWaveguide.line_floating

CircularWaveguide.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line_floating – matched, floating transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.circularWaveguide.CircularWaveguide.load

CircularWaveguide.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
load – n-port load, where S = Gamma0*eye(...)

Return type
Network object

See also

match, *open*, *short*

skrf.media.circularWaveguide.CircularWaveguide.lossless_mismatch

CircularWaveguide.**lossless_mismatch**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns
ntwk – 2-port lossless mismatch

Return type
Network object

skrf.media.circularWaveguide.CircularWaveguide.match

CircularWaveguide.**match**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns
match – a n-port match

Return type
Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.circularWaveguide.CircularWaveguide.mode

CircularWaveguide.**mode**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.circularWaveguide.CircularWaveguide.open

CircularWaveguide.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.circularWaveguide.CircularWaveguide.plot

CircularWaveguide.**plot**(**args*, ***kw*)

skrf.media.circularWaveguide.CircularWaveguide.random

CircularWaveguide.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero
- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns
result – the network

Return type
Network object

skrf.media.circularWaveguide.CircularWaveguide.resistor

CircularWaveguide.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
resistor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.circularWaveguide.CircularWaveguide.short

`CircularWaveguide.`**`short`**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns
match – a n-port short circuit

Return type
Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.circularWaveguide.CircularWaveguide.shunt

CircularWaveguide.**shunt**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns

shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type

Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.circularWaveguide.CircularWaveguide.shunt_capacitor

CircularWaveguide.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns

shunt_capacitor – shunted capacitor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.circularWaveguide.CircularWaveguide.shunt_delay_load

CircularWaveguide.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.circularWaveguide.CircularWaveguide.shunt_delay_open

CircularWaveguide.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.circularWaveguide.CircularWaveguide.shunt_delay_short

`CircularWaveguide.`**`shunt_delay_short`**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.circularWaveguide.CircularWaveguide.shunt_inductor

`CircularWaveguide.`**`shunt_inductor`**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.circularWaveguide.CircularWaveguide.shunt_resistor

CircularWaveguide.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.circularWaveguide.CircularWaveguide.splitter

CircularWaveguide.**splitter**(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports

- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.circularWaveguide.CircularWaveguide.tee

CircularWaveguide.**tee**(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.circularWaveguide.CircularWaveguide.theta_2_d

CircularWaveguide.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)
- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)

- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.circularWaveguide.CircularWaveguide.thru

`CircularWaveguide.thru(z0=None, **kwargs)`

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.circularWaveguide.CircularWaveguide.to_meters

`CircularWaveguide.to_meters(d, unit='deg')`

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.circularWaveguide.CircularWaveguide.white_gaussian_polar

CircularWaveguide.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns

result – a noise network

Return type

Network object

skrf.media.circularWaveguide.CircularWaveguide.write_csv

CircularWaveguide.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters

filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

skrf.media.cpw.CPW

class skrf.media.cpw.**CPW**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *w=0.003*, *s=0.0003*, *h=1.55*, *ep_r=4.5*, *t=None*, *diel='djordjevicsvensson'*, *rho=1.68e-08*, *tand=0*, *f_low=1000.0*, *f_high=1000000000000.0*, *f_epr_tand=1000000000.0*, *has_metal_backside=False*, *compatibility_mode=None*, **args*, ***kwargs*)

Coplanar waveguide.

A coplanar waveguide transmission line is defined in terms of width, spacing, and thickness on a given relative permittivity substrate of a certain height. The line has a conductor resistivity and a tangential loss factor. The backside of the strip can be made of air or metal (grounded coplanar waveguide).

This class is highly inspired by the technical documentation [QUCSa] and sources provided by the qucs project [QUCSb] .

In addition, Djordjevic [DBLS01] /Svensson [SvDe01] wideband debye dielectric model is considered to provide a more realistic modelling of broadband microstrip with causal time domain response.

A compatibility mode is provided to mimic the behaviour of QUCS or of Keysight ADS. There is known differences in the output of these simulators.

The quasi-static models of characteristic impedance and effective permittivity give the value at zero frequency. The dispersion models compute frequency-dependant values of these variables.

- Quasi-static characteristic impedance and effective permittivity model use [GhNa84] and [GhNa83]. The models are corrected to account for strip thickness using a first-order approach described in [GGBB96]. A comparison shows that ADS simulator uses another thickness correction method that is according to ADS doc based on [Cohn60]. This second method is not implemented in skrf.
- Frequency dispersion of impedance and effective permittivity model use [FGVM91] and [GMDK97].
- Loss model is computed using Wheeler's incremental inductance rule [Whee42] applied to coplanar waveguide by [OwWu58] and [Ghio93].

Parameters

- **frequency** (*Frequency* object, optional) – frequency band of the media. The default is None.
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0_override** (*number, array-like, or None*) – *z0_override* override the characteristic impedance for the media. If *z0_override* is not None, the networks generated by the media have their characteristic impedance *z0* overridden by *z0_override*. (Default is None)
- **z0** (*number, array-like, or None*) – deprecated parameter, alias to *z0_override* if *z0_override* is None. Emit a deprecation warning.
- **w** (*number, or array-like*) – width of the center conductor, in m. Default is 3e-3 m.
- **s** (*number, or array-like*) – spacing (width of the gap), in m. Default is 0.3e-3 m.
- **h** (*number, or array-like*) – height of the substrate between backside and conductor, in m. Default is 1.55 m (equivalent to infinite height for default w and s).
- **t** (*number, or array-like, optional*) – conductor thickness, in m. Default is None (no width correction to account for strip thickness).
- **has_metal_backside** (*bool, default False*) – If the backside is air (False) or metal (True)
- **ep_r** (*number, or array-like, optional*) – relative permittivity of the substrate at frequency f_epr_tand, no unit. Default is 4.5.
- **diel** (*str*) – dielectric frequency dispersion model in:
  - 'djordjevicsvensson' (default)
  - 'frequencyinvariant'
- **rho** (*number, or array-like, or None*) – resistivity of conductor, ohm / m. Default is 1.68e-8 ohm /m (copper).

- **tand** (*number, or array-like*) – dielectric loss factor at frequency f_epr_tand. Default is 0.
- **f_low** (*number, or array-like*) – lower frequency for wideband Debye Djordjevic/Svensson dielectric model, in Hz. Default is 1 kHz.
- **f_high** (*number, or array-like*) – higher frequency for wideband Debye Djordjevic/Svensson dielectric model, in Hz. Default is 1 THz.
- **f_epr_tand** (*number, or array-like*) – measurement frequency for ep_r and tand of dielectric, in Hz. Default is 1 GHz.
- **compatibility_mode** (*str or None (default)*) – If set to 'qucs', following behaviour happens :
  - Characteristic impedance will be real (no imaginary part due to tand)
- ***args** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*
- ****kwargs** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*

Note

When the thickness of the strip is smaller than 3 skin depth, the losses model gives over-optimistic results and the media will issue a warning. At DC, the losses of the line could be smaller than its conductor resistance, which is not physical.

References

Attributes

Z0	Characteristic Impedance
alpha	Real (attenuation) component of gamma.
beta	Imaginary (propagating) component of gamma.
gamma	Propagation constant.
npoints	Number of points of the frequency axis.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	Characteristic Impedance, z_0.
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.cpw.CPW.Z0

property CPW.**Z0**

Characteristic Impedance

Returns

z0 – Characteristic Impedance in units of ohms

Return type

np.ndarray

skrf.media.cpw.CPW.alpha

property CPW.alpha: ndarray

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.cpw.CPW.beta

property CPW.beta: ndarray

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.cpw.CPW.gamma

property CPW.gamma: Number | Sequence[Number] | ndarray

Propagation constant.

Returns
gamma

Return type
numpy.ndarray

skrf.media.cpw.CPW.npoints

property CPW.npoints: int

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.cpw.CPW.v_g

property CPW.v_g

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the *j* is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.cpw.CPW.v_p

property CPW.**v_p**: **ndarray**

Complex phase velocity (in m/s).

$$j \cdot \omega / \gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.cpw.CPW.z0

property CPW.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.cpw.CPW.z0_characteristic

property CPW.**z0_characteristic**: Number | Sequence[Number] | ndarray

Characteristic Impedance, z_0.

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.cpw.CPW.z0_override

property CPW.**z0_override**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.cpw.CPW.z0_port

property CPW.**z0_port**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
analyse_dielectric	This function calculate the frequency-dependent relative permittivity of dielectric and tangential loss factor.
analyse_dispersion	This function computes the frequency-dependent characteristic impedance and effective permittivity accounting for coplanar waveguide frequency dispersion.
analyse_loss	Calculates conductor and dielectric losses of a coplanar waveguide line.
analyse_quasi_static	Calculates quasi-static impedance of a coplanar waveguide line.
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.

continues on next page

Table 38 – continued from previous page

extract_distance	Determines physical distance from a transmission or reflection Network.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.cpw.CPW.__init__

CPW.**__init__**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *w=0.003*, *s=0.0003*, *h=1.55*, *ep_r=4.5*, *t=None*, *diel='djordjevicsvensson'*, *rho=1.68e-08*, *tand=0*, *f_low=1000.0*, *f_high=1000000000000.0*, *f_epr_tand=1000000000.0*, *has_metal_backside=False*, *compatibility_mode=None*, **args*, ***kwargs*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*NumberLike | None*)
- **z0_override** (*NumberLike | None*)
- **z0** (*NumberLike | None*)
- **w** (*NumberLike*)
- **s** (*NumberLike*)

- **h** (*NumberLike*)
- **ep_r** (*NumberLike*)
- **t** (*NumberLike | None*)
- **diel** (*str*)
- **rho** (*NumberLike | None*)
- **tand** (*NumberLike*)
- **f_low** (*NumberLike*)
- **f_high** (*NumberLike*)
- **f_epr_tand** (*NumberLike*)
- **has_metal_backside** (*bool*)
- **compatibility_mode** (*str | None*)

skrf.media.cpw.CPW.analyse_dielectric

CPW.**analyse_dielectric**(*ep_r*, *tand*, *f_low*, *f_high*, *f_epr_tand*, *f*, *diel*)

This function calculate the frequency-dependent relative permittivity of dielectric and tangential loss factor.

References

Returns

- **ep_r_f** (numpy.ndarray)
- **tand_f** (numpy.ndarray)

Parameters

- **ep_r** (*Number | Sequence[Number] | ndarray*)
- **tand** (*Number | Sequence[Number] | ndarray*)
- **f_low** (*Number | Sequence[Number] | ndarray*)
- **f_high** (*Number | Sequence[Number] | ndarray*)
- **f_epr_tand** (*Number | Sequence[Number] | ndarray*)
- **f** (*Number | Sequence[Number] | ndarray*)
- **diel** (*str*)

skrf.media.cpw.CPW.analyse_dispersion

CPW.**analyse_dispersion**(*zl_eff*, *ep_reff*, *ep_r*, *w*, *s*, *h*, *f*)

This function computes the frequency-dependent characteristic impedance and effective permittivity accounting for coplanar waveguide frequency dispersion.

References

Returns

- **z** (numpy.ndarray)
- **e** (numpy.ndarray)

Parameters

- **zl_eff** (*Number | Sequence[Number] | ndarray*)
- **ep_reff** (*Number | Sequence[Number] | ndarray*)
- **ep_r** (*Number | Sequence[Number] | ndarray*)
- **w** (*Number | Sequence[Number] | ndarray*)
- **s** (*Number | Sequence[Number] | ndarray*)
- **h** (*Number | Sequence[Number] | ndarray*)
- **f** (*Number | Sequence[Number] | ndarray*)

skrf.media.cpw.CPW.analyse_loss

CPW.**analyse_loss**(*ep_r*, *ep_reff*, *tand*, *rho*, *mu_r*, *f*, *w*, *t*, *s*, *k1*, *kk1*, *kpk1*)

Calculates conductor and dielectric losses of a coplanar waveguide line.

Use wheeler's incremental inductance rule[1],[2],[3].

References

Returns

- **a_conductor** (numpy.ndarray)
- **a_dielectric** (numpy.ndarray)

Parameters

- **ep_r** (*Number | Sequence[Number] | ndarray*)
- **ep_reff** (*Number | Sequence[Number] | ndarray*)
- **tand** (*Number | Sequence[Number] | ndarray*)
- **rho** (*Number | Sequence[Number] | ndarray*)
- **mu_r** (*Number | Sequence[Number] | ndarray*)
- **f** (*Number | Sequence[Number] | ndarray*)
- **w** (*Number | Sequence[Number] | ndarray*)
- **t** (*Number | Sequence[Number] | ndarray*)
- **s** (*Number | Sequence[Number] | ndarray*)
- **k1** (*Number | Sequence[Number] | ndarray*)
- **kk1** (*Number | Sequence[Number] | ndarray*)
- **kpk1** (*Number | Sequence[Number] | ndarray*)

skrf.media.cpw.CPW.analyse_quasi_static

[1] H. A. Wheeler, "Formulas for the Skin Effect," Proceedings of the IRE, vol. 30, no. 9, pp. 412-424, Sept. 1942.

[2] G. H. Owyang and T. T. Wu, "The Approximate Parameters of Slot Lines and Their Complement" IRE Transactions on Antennas and Propagation, pp. 49-55, Jan. 1958.

[3] G. Ghione, "A CAD-Oriented Analytical Model for the Losses of General Asymmetric Coplanar Lines in Hybrid and Monolithic MICs" IEEE Trans. on Microwave Theory and Techniques, vol. 41, no. 9, pp. 1499-1510, Sept. 1993.

CPW.**analyse_quasi_static**(*ep_r*, *w*, *s*, *h*, *t*, *has_metal_backside*)

Calculates quasi-static impedance of a coplanar waveguide line.

The value of the effective permittivity as per filling factor, and the effective width due to the finite conductor thickness for the given coplanar waveguide line and substrate properties. Model from [GhNa84] with air backside and [GhNa83] with a metal backside. The models are corrected to account for strip thickness using a first-order approach described in [GGBB96]. ADS simulator report to use a custom correction based on [Cohn60]. This second method is not implemented in skrf.

References

Returns

- **zl_eff** (numpy.ndarray)
- **ep_reff** (numpy.ndarray)

Parameters

- **ep_r** (*Number | Sequence[Number] | ndarray*)
- **w** (*Number | Sequence[Number] | ndarray*)
- **s** (*Number | Sequence[Number] | ndarray*)
- **h** (*Number | Sequence[Number] | ndarray*)
- **t** (*Number | Sequence[Number] | ndarray*)
- **has_metal_backside** (*bool*)

skrf.media.cpw.CPW.attenuator

CPW.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns
ntwk – 2-port attenuator

Return type
Network object

skrf.media.cpw.CPW.capacitor

CPW.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
capacitor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.cpw.CPW.capacitor_q

CPW.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.cpw.CPW.copy

CPW.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.cpw.CPW.delay_load

CPW.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)

- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.cpw.CPW.delay_open

CPW.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns
delay_open – a delayed open

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.cpw.CPW.delay_short

CPW.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns

delay_short – a delayed short

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.cpw.CPW.electrical_length

CPW.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns

theta – complex electrical length in radians or degrees, depending on value of deg.

Return type

number or array-like

skrf.media.cpw.CPW.extract_distance

CPW.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters
ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns
d – physical distance

Return type
number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.cpw.CPW.get_array_of

CPW.**get_array_of**(*x*)

skrf.media.cpw.CPW.impedance_mismatch

CPW.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
mismatch – a 2-port network representing the impedance mismatch

Return type
Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.cpw.CPW.inductor

CPW.**inductor**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.cpw.CPW.inductor_q

CPW.**inductor_q**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.cpw.CPW.isolator

CPW.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.

- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.cpw.CPW.line

CPW.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line – matched transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.cpw.CPW.line_floating

CPW.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)

- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line_floating – matched, floating transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.cpw.CPW.load

CPW.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
load – n-port load, where S = Gamma0*eye(...)

Return type
Network object

See also

match, *open*, *short*

skrf.media.cpw.CPW.lossless_mismatch

CPW.**lossless_mismatch**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns

ntwk – 2-port lossless mismatch

Return type

Network object

skrf.media.cpw.CPW.match

CPW.**match**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns

match – a n-port match

Return type

Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.cpw.CPW.mode

CPW.**mode**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.cpw.CPW.open

CPW.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.cpw.CPW.plot

CPW.**plot**(**args*, ***kw*)

skrf.media.cpw.CPW.random

CPW.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero

- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns
result – the network

Return type
Network object

skrf.media.cpw.CPW.resistor

CPW.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
resistor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.cpw.CPW.short

CPW.**short**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns
match – a n-port short circuit

Return type
Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.cpw.CPW.shunt

CPW.**shunt**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns
shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type
Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.cpw.CPW.shunt_capacitor

CPW.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
shunt_capacitor – shunted capacitor (2-port)

Return type
Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.cpw.CPW.shunt_delay_load

CPW.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.cpw.CPW.shunt_delay_open

CPW.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.cpw.CPW.shunt_delay_short

CPW.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.cpw.CPW.shunt_inductor

CPW.**shunt_inductor**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.cpw.CPW.shunt_resistor

CPW.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.cpw.CPW.splitter

CPW.**splitter**(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.cpw.CPW.tee

CPW.**tee**(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.cpw.CPW.theta_2_d

CPW.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)

- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.cpw.CPW.thru

CPW.**thru**(*z0=None*, ***kwargs*)

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.cpw.CPW.to_meters

CPW.**to_meters**(*d*, *unit='deg'*)

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.cpw.CPW.white_gaussian_polar

CPW.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.cpw.CPW.write_csv

CPW.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters
filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

skrf.media.coaxial.Coaxial

class skrf.media.coaxial.**Coaxial**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *Dint=0.00081*, *Dout=0.005*, *epsilon_r=1*, *tan_delta=0*, *sigma=58000000.0*, *tout=None*, *model='schelkunoff'*, *dielectric=None*, *inner_conductor=None*, *outer_conductor=None*, **args*, ***kwargs*)

A coaxial transmission line defined in terms of its inner/outer diameters and permittivity.

Parameters

- **frequency** (*Frequency* object) – Frequency band.
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0_override** (*number, array-like, or None*) – *z0_override* override the characteristic impedance for the media. If *z0_override* is not None, the networks generated by the media have their characteristic impedance *z0* overridden by *z0_override*. (Default is None)
- **z0** (*number, array-like, or None*) – deprecated parameter, alias to *z0_override* if *z0_override* is None. Emit a deprecation warning.
- **Dint** (*number, or array-like*) – inner conductor diameter, in m
- **Dout** (*number, or array-like*) – outer conductor diameter, in m
- **epsilon_r** (*number, or array-like, optional*) – relative permittivity of the dielectric medium. Default is 1.
- **tan_delta** (*number, or array-like, optional*) – loss tangent of the dielectric medium. Default is 0.
- **sigma** (*number, or array-like, optional.*) – conductors electrical conductivity, in S/m. Default is 58e6, the conductivity of annealed copper at 20 °C (100% IACS).
- **tout** (*number, array-like, or None, optional*) – wall thickness of the outer conductor, in m. If None (default), the shield is infinitely thick.
- **model** (*str, optional*) – Model of the conductors. 'schelkunoff' (default) is the Bessel function solution of[1], 'tesche' the equivalent circuit approximation of[2].
- **dielectric** (*dict or None, optional*) – Material of the dielectric filling, as a dict taking the keys `'ep_r'` (complex relative permittivity $\epsilon_r(1 - j \tan\delta)$) and `'mu_r'` (complex relative permeability), both defaulting to 1. If None (default), the filling is described by *epsilon_r* and *tan_delta*.
- **inner_conductor** (*dict, list of dict, or None, optional*) – Material of the inner conductor. A single dict describes a bulk conductor, a list of dict a stack of coatings ordered from the dielectric inwards, of which the deepest layer is the bulk that carries the dc current. Each dict takes the keys `'sigma'`, `'mu_r'` and `'ep_r'` of the layer itself, and `'rms_roughness'`, `'boundary_loc'` and `'distribution'` of the boundary on top of it, which are handed to *surface_impedance()*. If None (default), the conductor is smooth and made of bulk *sigma*.
- **outer_conductor** (*dict, list of dict, or None, optional*) – Material of the outer conductor, described as *inner_conductor*.

Note

Dint, Dout, tout, epsilon_r, tan_delta, sigma can all be vectors as long as they are the same length

[1] S. A. Schelkunoff, "The electromagnetic theory of coaxial transmission lines and cylindrical shields," Bell Syst. Tech. J., vol. 13, no. 4, pp. 532-579, Oct. 1934. https://doi.org/10.1002/j.1538-7305.1934.tb00679.x

[2] F. M. Tesche, "A Simple Model for the Line Parameters of a Lossy Coaxial Cable Filled With a Nondispersive Dielectric," IEEE Trans. Electromagn. Compat., vol. 49, no. 1, pp. 12-17, Feb. 2007. https://doi.org/10.1109/TEMC.2006.888185

References

Examples

An RG58C/U flexible cable with copper conductors and a lossy polyethylene filling. Its characteristic impedance and the insertion loss of a 100 mm length of it at 5 GHz:

```
>>> import skrf as rf
>>> from skrf.media import Coaxial
>>> freq = rf.Frequency(1, 5, 101, unit='GHz')
>>> rg58 = Coaxial(freq, Dint=0.91e-3, Dout=2.95e-3, epsilon_r=2.3,
...                tan_delta=2e-4, sigma=58e6, z0_port=50)
>>> print(f"{rg58.z0[0]:.2f} ohm, {rg58.line(100, 'mm').s_db[-1, 1, 0]:.3f} dB")
46.56-0.05j ohm, -0.094 dB
```

The same cable roughened by 0.6 um rms at the inner conductor and by 1 um rms at the braided shield, which is described by giving each conductor a material of its own instead of the bulk *sigma*:

```
>>> rg58 = Coaxial(freq, Dint=0.91e-3, Dout=2.95e-3, epsilon_r=2.3,
...                tan_delta=2e-4, z0_port=50,
...                inner_conductor={'sigma': 58e6, 'rms_roughness': 0.6e-6},
...                outer_conductor={'sigma': 58e6, 'rms_roughness': 1e-6})
>>> print(f"{rg58.line(100, 'mm').s_db[-1, 1, 0]:.3f} dB")
-0.140 dB
```

An APC-7 air line whose two conductors carry an ENIG finish, 0.05 um of gold over 4 um of nickel over copper, roughened by 50 nm rms at the outside and by 0.2 um rms at the copper underneath:

```
>>> enig = [{'sigma': 41.1e6, 'rms_roughness': 50e-9, 'boundary_loc': 0},
...         {'sigma': 14.5e6, 'mu_r': 20, 'rms_roughness': 50e-9, 'boundary_loc': 0.
↪05e-6},
...         {'sigma': 58e6, 'rms_roughness': 0.2e-6, 'boundary_loc': 4.05e-6}]
>>> freq = rf.Frequency(1, 18, 171, unit='GHz')
>>> apc7 = Coaxial(freq, Dint=3.04e-3, Dout=7.00e-3,
...                inner_conductor=enig, outer_conductor=enig)
>>> print(f"{apc7.z0[0]:.2f} ohm, {apc7.line(100, 'mm').s_db[-1, 1, 0]:.3f} dB")
50.22-0.25j ohm, -0.301 dB
```

Attributes

C	Distributed capacitance C, in F/m
G	Distributed conductance G, in S/m
L	Distributed inductance L, in H/m
R	Distributed resistance R, in Ohm/m.
Rs	Surface resistivity in Ohm/area.
Y	Distributed Admittance, $Y^{'}$.
Z	Distributed Impedance, $Z^{'}$.
Z0	Characteristic Impedance
a	Inner radius of the coaxial line
alpha	Real (attenuation) component of gamma.
b	Outer radius of the coaxial line
beta	Imaginary (propagating) component of gamma.
gamma	Propagation Constant, γ.

continues on next page

Table 39 – continued from previous page

npoints	Number of points of the frequency axis.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	Characteristic Impedance, z_0
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.coaxial.Coaxial.C

property Coaxial.C: Number | Sequence[Number] | ndarray

Distributed capacitance C, in F/m

Returns
C – distributed capacitance, in F/m

Return type
number, or array-like

skrf.media.coaxial.Coaxial.G

property Coaxial.G: Number | Sequence[Number] | ndarray

Distributed conductance G, in S/m

Returns
G – distributed conductance, in S/m

Return type
number, or array-like

skrf.media.coaxial.Coaxial.L

property Coaxial.L: Number | Sequence[Number] | ndarray

Distributed inductance L, in H/m

This is the external inductance of the field between the two conductors, plus the internal inductance of the conductors themselves. The latter is dropped at dc, where it is not defined for an infinitely thick shield.

Returns
L – distributed inductance, in H/m

Return type
number, or array-like

skrf.media.coaxial.Coaxial.R

property Coaxial.R: Number | Sequence[Number] | ndarray

Distributed resistance R, in Ohm/m.

This is the loss of the two conductors, from their dc resistance up to the skin effect, plus the magnetic loss of the filling if there is any. See the references of *Coaxial*.

Returns
R – distributed resistance, in Ohm/m

Return type
number, or array-like

skrf.media.coaxial.Coaxial.Rs

property Coaxial.**Rs**: Number | Sequence[Number] | ndarray

Surface resistivity in Ohm/area.

Returns
Rs – surface resistivity

Return type
number or array

> **Note**
>
> Not used in code. Kept for backward-compatibility.

skrf.media.coaxial.Coaxial.Y

property Coaxial.**Y**: Number | Sequence[Number] | ndarray

Distributed Admittance, $Y^{'}$.

Defined as

$$Y^{'} = G^{'} + j\omega C^{'}$$

Returns
Y – Distributed Admittance in units of S/m

Return type
np.ndarray

skrf.media.coaxial.Coaxial.Z

property Coaxial.**Z**: Number | Sequence[Number] | ndarray

Distributed Impedance, $Z^{'}$.

Defined as

$$Z^{'} = R^{'} + j\omega L^{'}$$

Returns
Z – Distributed impedance in units of ohm/m

Return type
np.ndarray

skrf.media.coaxial.Coaxial.Z0

property Coaxial.**Z0**

Characteristic Impedance

Returns
z0 – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.coaxial.Coaxial.a

property Coaxial.a: Number | Sequence[Number] | ndarray

Inner radius of the coaxial line

Returns
a – Inner radius

Return type
float

skrf.media.coaxial.Coaxial.alpha

property Coaxial.alpha: ndarray

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.coaxial.Coaxial.b

property Coaxial.b: Number | Sequence[Number] | ndarray

Outer radius of the coaxial line

Returns
b – Outer radius

Return type
float

skrf.media.coaxial.Coaxial.beta

property Coaxial.beta: ndarray

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.coaxial.Coaxial.gamma

property Coaxial.gamma: Number | Sequence[Number] | ndarray

Propagation Constant, γ.

Defined as,

$$\gamma = \sqrt{Z'Y'}$$

Returns
gamma – Propagation Constant,

Return type
np.ndarray

Note

The components of propagation constant are interpreted as follows:

- positive real(gamma) = attenuation
- positive imag(gamma) = forward propagation

skrf.media.coaxial.Coaxial.npoints

property Coaxial.**npoints**: int

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.coaxial.Coaxial.v_g

property Coaxial.**v_g**

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the j is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.coaxial.Coaxial.v_p

property Coaxial.**v_p**: ndarray

Complex phase velocity (in m/s).

$$j \cdot \omega/\gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.coaxial.Coaxial.z0

property Coaxial.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.coaxial.Coaxial.z0_characteristic

property Coaxial.**z0_characteristic**: Number | Sequence[Number] | ndarray

Characteristic Impedance, z_0

$$Z_0 = \sqrt{\frac{Z'}{Y'}}$$

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.coaxial.Coaxial.z0_override

property Coaxial.**z0_override**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.coaxial.Coaxial.z0_port

property Coaxial.**z0_port**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.
extract_distance	Determines physical distance from a transmission or reflection Network.
from_Z0_Dout	Init from characteristic impedance and outer diameter.
from_attenuation_VF	Init from electrical properties of the line: attenuation and velocity factor.
from_csv	Create a *DistributedCircuit* from numerical values stored in a csv file.
from_media	Initializes a *DistributedCircuit* from an existing *Media* instance.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.

continues on next page

Table 40 – continued from previous page

shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.coaxial.Coaxial.__init__

Coaxial.**__init__**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *Dint=0.00081*, *Dout=0.005*, *epsilon_r=1*, *tan_delta=0*, *sigma=58000000.0*, *tout=None*, *model='schelkunoff'*, *dielectric=None*, *inner_conductor=None*, *outer_conductor=None*, **args*, ***kwargs*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*NumberLike | None*)
- **z0_override** (*NumberLike | None*)
- **z0** (*NumberLike | None*)
- **Dint** (*NumberLike*)
- **Dout** (*NumberLike*)
- **epsilon_r** (*NumberLike*)
- **tan_delta** (*NumberLike*)
- **sigma** (*NumberLike*)
- **tout** (*NumberLike | None*)
- **model** (*str*)
- **dielectric** (*dict | None*)
- **inner_conductor** (*dict | list[dict] | None*)
- **outer_conductor** (*dict | list[dict] | None*)

skrf.media.coaxial.Coaxial.attenuator

Coaxial.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.

- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns
ntwk – 2-port attenuator

Return type
Network object

skrf.media.coaxial.Coaxial.capacitor

Coaxial.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
capacitor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.coaxial.Coaxial.capacitor_q

Coaxial.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.coaxial.Coaxial.copy

Coaxial.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.coaxial.Coaxial.delay_load

Coaxial.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.coaxial.Coaxial.delay_open

Coaxial.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns

delay_open – a delayed open

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.coaxial.Coaxial.delay_short

Coaxial.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns

delay_short – a delayed short

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.coaxial.Coaxial.electrical_length

Coaxial.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns

theta – complex electrical length in radians or degrees, depending on value of deg.

Return type

number or array-like

skrf.media.coaxial.Coaxial.extract_distance

Coaxial.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters

ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns

d – physical distance

Return type

number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.coaxial.Coaxial.from_Z0_Dout

classmethod Coaxial.**from_Z0_Dout**(*frequency=None*, *z0_port=None*, *z0=50*, *epsilon_r=1*, *Dout=0.005*, ***kw*)

Init from characteristic impedance and outer diameter.

Parameters

- **frequency** (*Frequency* object)
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0** (*number*) – desired characteristic impedance
- **Dout** (*number, or array-like*) – outer conductor diameter, in m
- **epsilon_r=1** (*number, or array-like*) – relative permittivity of the dielectric medium
- ****kw** – passed to __init__
- **epsilon_r** (*NumberLike*)

Returns
media

Return type
Media

skrf.media.coaxial.Coaxial.from_attenuation_VF

classmethod Coaxial.**from_attenuation_VF**(*frequency=None*, *z0_port=None*, *z0=50*, *att=0*, *unit='dB/m'*, *VF=1*)

Init from electrical properties of the line: attenuation and velocity factor.

Attenuation can be expressed in dB/m, dB/100m, dB/ft, dB/100ft, Neper/m or Neper/ft. Default unit is dB/m. A different unit is set by the *unit* parameter.

Parameters

- **frequency** (*Frequency* object)
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0** (*number*) – desired characteristic impedance
- **att** (*number, or array-like. optional*) – Attenuation of the coaxial line. The default is 0. If passed as an array, should be of same size than the frequency.
- **unit** (*string, optional*) – Unit of the attenuation. Can be: 'dB/m', dB/100m", 'dB/ft', 'dB/100ft', 'Neper/m' or 'Neper/ft' (or 'Np/m', 'Np/ft'). The default is 'dB/m'.
- **VF** (*number, or array-like. optional*) – Velocity Factor VF [VF]. The default is 1. If passed as an array, should be of same size than the frequency.

Returns
media

Return type
Media

References

skrf.media.coaxial.Coaxial.from_csv

classmethod Coaxial.**from_csv**(**args*, ***kw*)

Create a *DistributedCircuit* from numerical values stored in a csv file.

The csv file format must be written by the function *write_csv()*, or similar method which produces the following format:

```
f[$unit], Re(z0), Im(z0), Re(gamma), Im(gamma), Re(z0_port), Im(z0_port)
1, 1, 1, 1, 1, 1, 1
2, 1, 1, 1, 1, 1, 1
.....
```

See also

write_csv

skrf.media.coaxial.Coaxial.from_media

classmethod Coaxial.**from_media**(*my_media*, **args*, ***kwargs*)

Initializes a *DistributedCircuit* from an existing *Media* instance.

Parameters

my_media (*Media* instance.) – the media object

Return type

Media

See also

Media

skrf.media.coaxial.Coaxial.get_array_of

Coaxial.**get_array_of**(*x*)

skrf.media.coaxial.Coaxial.impedance_mismatch

Coaxial.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

mismatch – a 2-port network representing the impedance mismatch

Return type

Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.coaxial.Coaxial.inductor

`Coaxial.`**`inductor`**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.coaxial.Coaxial.inductor_q

`Coaxial.`**`inductor_q`**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.coaxial.Coaxial.isolator

Coaxial.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns

ntwk – 2-port isolator

Return type

Network object

skrf.media.coaxial.Coaxial.line

Coaxial.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

line – matched transmission line of given length

Return type

Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.coaxial.Coaxial.line_floating

Coaxial.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

line_floating – matched, floating transmission line of given length

Return type

Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.coaxial.Coaxial.load

Coaxial.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

load – n-port load, where S = Gamma0*eye(...)

Return type

Network object

See also

match, *open*, *short*

skrf.media.coaxial.Coaxial.lossless_mismatch

Coaxial.**lossless_mismatch**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns

ntwk – 2-port lossless mismatch

Return type

Network object

skrf.media.coaxial.Coaxial.match

Coaxial.**match**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns

match – a n-port match

Return type

Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.coaxial.Coaxial.mode

Coaxial.**mode**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.coaxial.Coaxial.open

Coaxial.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.coaxial.Coaxial.plot

Coaxial.**plot**(**args*, ***kw*)

skrf.media.coaxial.Coaxial.random

Coaxial.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero

- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns
result – the network

Return type
Network object

skrf.media.coaxial.Coaxial.resistor

Coaxial.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
resistor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.coaxial.Coaxial.short

Coaxial.**short**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns
match – a n-port short circuit

Return type
Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.coaxial.Coaxial.shunt

Coaxial.**shunt**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns
shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type
Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.coaxial.Coaxial.shunt_capacitor

Coaxial.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
shunt_capacitor – shunted capacitor (2-port)

Return type
Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.coaxial.Coaxial.shunt_delay_load

Coaxial.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.coaxial.Coaxial.shunt_delay_open

Coaxial.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.coaxial.Coaxial.shunt_delay_short

Coaxial.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.coaxial.Coaxial.shunt_inductor

Coaxial.**shunt_inductor**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.coaxial.Coaxial.shunt_resistor

Coaxial.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.coaxial.Coaxial.splitter

Coaxial.**splitter**(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.coaxial.Coaxial.tee

Coaxial.**tee**(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.coaxial.Coaxial.theta_2_d

Coaxial.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)

- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.coaxial.Coaxial.thru

Coaxial.**thru**(*z0=None*, ***kwargs*)

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.coaxial.Coaxial.to_meters

Coaxial.**to_meters**(*d*, *unit='deg'*)

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.coaxial.Coaxial.white_gaussian_polar

Coaxial.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.coaxial.Coaxial.write_csv

Coaxial.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters
filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

skrf.media.mline.MLine

class skrf.media.mline.**MLine**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *w=3*, *h=1.6*, *t=None*, *ep_r=4.5*, *mu_r=1.0*, *model='hammerstadjensen'*, *disp='kirschningjansen'*, *diel='djordjevicsvensson'*, *rho=1.68e-08*, *tand=0*, *rough=1.5e-07*, *f_low=1000.0*, *f_high=1000000000000.0*, *f_epr_tand=1000000000.0*, *compatibility_mode=None*, **args*, ***kwargs*)

A microstripline transmission line defined in terms of width, thickness and height on a given relative permittivity substrate. The line has a conductor resistivity and a tangential loss factor.

This class is highly inspired from the technical documentation [QUCS_tech] and sources provided by the qucs project [QUCS_repo] .

In addition, Djordjevic [Djordjevic] /Svensson [Svensson] wideband debye dielectric model is considered to provide more realistic modelling of broadband microstrip with as causal time domain response.

A compatibility mode is provided to mimic the behaviour of QUCS or of Keysight ADS. There is known differences in the output of these simulators.

The quasi-static models of characteristic impedance and effective permittivity give the value at zero frequency. The dispersion models compute a frequency-dependant values of these variables.

- Quasi-static characteristic impedance and effective permittivity models:
  - Hammerstad and Jensen (default) [Hammerstad]
  - Schneider [Schneider1969]
  - Wheeler [Wheeler1942], [Wheeler1977]
- Frequency dispersion of impedance and effective permittivity models:
  - Hammerstad and Jensen [Hammerstad]
  - Kirschning and Jansen (default) [Kirschning]
  - Kobayashi [Kobayashi]
  - Schneider [Schneider1972]
  - Yamashita [Yamashita]
  - (No dispersion)
- Strip thickness correction model:
  - all quasi-static models add a certain amount to W to account for non-zero thickness of the strip. Computation with zero thickness is possible.

Parameters

- **frequency** (*Frequency* object) – frequency band of the media
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0_override** (*number, array-like, or None*) – *z0_override* override the characteristic impedance for the media. If *z0_override* is not None, the networks generated by the media have their characteristic impedance *z0* overridden by *z0_override*. (Default is None)
- **z0** (*number, array-like, or None*) – deprecated parameter, alias to *z0_override* if *z0_override* is None. Emit a deprecation warning.
- **w** (*number, or array-like*) – width of conductor, in m
- **h** (*number, or array-like*) – height of substrate between ground plane and conductor, in m
- **t** (*number, or array-like or None, optional*) – conductor thickness, in m. Default is None (no width correction to account strip thickness).
- **ep_r** (*number, or array-like*) – relative permittivity of dielectric at frequency f_epr_tand, no unit

- **mu_r** (*number, array-like*) – relative permeability mof dielectric, no unit
- **model** (*str*) – microstripline quasi-static impedance and dielectric model in:
  - 'hammerstadjensen' (default)
  - 'schneider'
  - 'wheeler'
- **disp** (*str*) – microstripline impedance and dielectric frequency dispersion model in:
  - 'hammerstadjensen'
  - 'kirschningjansen' (default)
  - 'kobayashi'
  - 'schneider'
  - 'yamashita'
  - 'none'
- **diel** (*str*) – dielectric frequency dispersion model in:
  - 'djordjevicsvensson' (default)
  - 'frequencyinvariant'
- **rho** (*number, or array-like, optional*) – resistivity of conductor, ohm / m
- **tand** (*number, or array-like*) – dielectric loss factor at frequency f_epr_tand
- **rough** (*number, or array-like*) – RMS roughness of conductor in m
- **f_low** (*number, or array-like*) – lower frequency for wideband Debye Djordjevic/Svensson dielectric model, in Hz
- **f_high** (*number, or array-like*) – higher frequency for wideband Debye Djordjevic/Svensson dielectric model, in Hz
- **f_epr_tand** (*number, or array-like*) – measurement frequency for ep_r and tand of dielectric, in Hz
- **compatibility_mode** (*str or None (default)*) – If set to 'qucs', following behaviour happens :
  - Characteristic impedance will be real (no imaginary part due to tand)
  - Quasi-static relative permittivity and impedance will by used for loss computation instead of frequency-dispersed values
  - Kobayashi and Yamashita models will disperse permittivity but keep quasi-static impedance values
- ***args** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*
- ****kwargs** (*arguments, keyword arguments*) – passed to *Media*'s constructor (*__init__()*

> **Note**
>
> In the case dispersion model only include effective permittivity, no dispersion is used for impedance in QUCS mode and Kirschning Jansen is used in ADS mode. QUCS mode is the default.
>
> When the thickness of the strip is smaller than 3 skin depth, the losses model gives over-optimistic results and the media will issue a warning. At DC, the losses of the line could be smaller than its conductor resistance, which is not physical.

References

Attributes

Z0	Characteristic Impedance
Z0_f	Alias fos Characteristic Impedance for backward compatibility.
alpha	Real (attenuation) component of gamma.
beta	Imaginary (propagating) component of gamma.
gamma	Propagation constant.
npoints	Number of points of the frequency axis.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	Characteristic Impedance, z_0
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.mline.MLine.Z0

property MLine.**Z0**

Characteristic Impedance

Returns
z0 – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.mline.MLine.Z0_f

property MLine.**Z0_f**: ndarray

Alias fos Characteristic Impedance for backward compatibility. Deprecated, do not use.

Returns
z0

Return type
numpy.ndarray

skrf.media.mline.MLine.alpha

property MLine.**alpha**: ndarray

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.mline.MLine.beta

property MLine.**beta**: ndarray

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.mline.MLine.gamma

property MLine.**gamma**

Propagation constant.

Returns
gamma

Return type
numpy.ndarray

skrf.media.mline.MLine.npoints

property MLine.**npoints**: int

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.mline.MLine.v_g

property MLine.**v_g**

Complex group velocity (in m/s).

$$j \cdot d\omega / d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the *j* is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.mline.MLine.v_p

property MLine.**v_p: ndarray**

Complex phase velocity (in m/s).

$$j \cdot \omega / \gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.mline.MLine.z0

property MLine.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.mline.MLine.z0_characteristic

property MLine.**z0_characteristic**: ndarray

Characteristic Impedance, z_0

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.mline.MLine.z0_override

property MLine.**z0_override**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.mline.MLine.z0_port

property MLine.**z0_port**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
analyse_dielectric	This function calculate the frequency dependent relative permittivity of dielectric and and tangential loss factor.
analyse_dispersion	This function compute the frequency dependent characteristic impedance and effective permittivity accounting for microstripline frequency dispersion.
analyse_loss	The function calculates the conductor and dielectric losses of a single microstrip line using wheeler's incremental inductance rule.
analyse_quasi_static	This function calculates the quasi-static impedance of a microstrip line, the value of the effective permittivity as per filling factor and the effective width due to the finite conductor thickness for the given microstrip line and substrate properties.
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.

continues on next page

Table 42 – continued from previous page

delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.
extract_distance	Determines physical distance from a transmission or reflection Network.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.mline.MLine.__init__

MLine.**__init__**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *w=3*, *h=1.6*, *t=None*, *ep_r=4.5*, *mu_r=1.0*, *model='hammerstadjensen'*, *disp='kirschningjansen'*, *diel='djordjevicsvensson'*, *rho=1.68e-08*, *tand=0*, *rough=1.5e-07*, *f_low=1000.0*, *f_high=1000000000000.0*, *f_epr_tand=1000000000.0*, *compatibility_mode=None*, **args*, ***kwargs*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*NumberLike | None*)
- **z0_override** (*NumberLike | None*)
- **z0** (*NumberLike | None*)

- **w** (*NumberLike*)
- **h** (*NumberLike*)
- **t** (*NumberLike | None*)
- **ep_r** (*NumberLike*)
- **mu_r** (*NumberLike*)
- **model** (*str*)
- **disp** (*str*)
- **diel** (*str*)
- **rho** (*NumberLike*)
- **tand** (*NumberLike*)
- **rough** (*NumberLike*)
- **f_low** (*NumberLike*)
- **f_high** (*NumberLike*)
- **f_epr_tand** (*NumberLike*)
- **compatibility_mode** (*str | None*)

skrf.media.mline.MLine.analyse_dielectric

MLine.**analyse_dielectric**(*ep_r*, *tand*, *f_low*, *f_high*, *f_epr_tand*, *f*, *diel*)

This function calculate the frequency dependent relative permittivity of dielectric and and tangential loss factor.

References

Returns

- **ep_r_f** (numpy.ndarray)
- **tand_f** (numpy.ndarray)

Parameters

- **ep_r** (*NumberLike*)
- **tand** (*NumberLike*)
- **f_low** (*NumberLike*)
- **f_high** (*NumberLike*)
- **f_epr_tand** (*NumberLike*)
- **f** (*NumberLike*)
- **diel** (*str*)

skrf.media.mline.MLine.analyse_dispersion

MLine.**analyse_dispersion**(*zl_eff*, *ep_reff*, *ep_r*, *wr*, *w_eff*, *h*, *t*, *f*, *disp*)

This function compute the frequency dependent characteristic impedance and effective permittivity accounting for microstripline frequency dispersion.

References

Returns

- **z** (numpy.ndarray)
- **e** (numpy.ndarray)

Parameters

- **zl_eff** (*NumberLike*)
- **ep_reff** (*NumberLike*)
- **ep_r** (*NumberLike*)
- **wr** (*NumberLike*)
- **w_eff** (*NumberLike*)
- **h** (*NumberLike*)
- **t** (*NumberLike*)
- **f** (*NumberLike*)
- **disp** (*str*)

skrf.media.mline.MLine.analyse_loss

MLine.**analyse_loss**(*ep_r*, *ep_reff*, *tand*, *rho*, *mu_r*, *zl_eff_f1*, *zl_eff_f2*, *f*, *w*, *t*, *D*)

The function calculates the conductor and dielectric losses of a single microstrip line using wheeler's incremental inductance rule.

References

Returns

- **a_conductor** (numpy.ndarray)
- **a_dielectric** (numpy.ndarray)

Parameters

- **ep_r** (*NumberLike*)
- **ep_reff** (*NumberLike*)
- **tand** (*NumberLike*)
- **rho** (*NumberLike*)
- **mu_r** (*NumberLike*)
- **zl_eff_f1** (*NumberLike*)
- **zl_eff_f2** (*NumberLike*)
- **f** (*NumberLike*)
- **w** (*NumberLike*)
- **t** (*NumberLike*)
- **D** (*NumberLike*)

skrf.media.mline.MLine.analyse_quasi_static

MLine.**analyse_quasi_static**(*ep_r*, *w*, *h*, *t*, *model*)

This function calculates the quasi-static impedance of a microstrip line, the value of the effective permittivity as per filling factor and the effective width due to the finite conductor thickness for the given microstrip line and substrate properties.

References

Returns

- **zl_eff** (numpy.ndarray)
- **ep_reff** (numpy.ndarray)

Parameters

- **ep_r** (*NumberLike*)
- **w** (*NumberLike*)
- **h** (*NumberLike*)
- **t** (*NumberLike*)
- **model** (*str*)

skrf.media.mline.MLine.attenuator

MLine.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns
ntwk – 2-port attenuator

Return type
Network object

skrf.media.mline.MLine.capacitor

MLine.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.

- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
capacitor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.mline.MLine.capacitor_q

MLine.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.mline.MLine.copy

MLine.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.mline.MLine.delay_load

MLine.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details

- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.mline.MLine.delay_open

MLine.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns
delay_open – a delayed open

Return type
Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.mline.MLine.delay_short

MLine.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns

delay_short – a delayed short

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.mline.MLine.electrical_length

MLine.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns

theta – complex electrical length in radians or degrees, depending on value of deg.

Return type

number or array-like

skrf.media.mline.MLine.extract_distance

MLine.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters
ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns
d – physical distance

Return type
number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.mline.MLine.get_array_of

MLine.**get_array_of**(*x*)

skrf.media.mline.MLine.impedance_mismatch

MLine.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
mismatch – a 2-port network representing the impedance mismatch

Return type
Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.mline.MLine.inductor

MLine.**inductor**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.mline.MLine.inductor_q

MLine.**inductor_q**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.mline.MLine.isolator

MLine.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.

- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.mline.MLine.line

MLine.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line – matched transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.mline.MLine.line_floating

MLine.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)

- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line_floating – matched, floating transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.mline.MLine.load

MLine.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
load – n-port load, where S = Gamma0*eye(...)

Return type
Network object

See also

match, *open*, *short*

skrf.media.mline.MLine.lossless_mismatch

MLine.**lossless_mismatch**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns

ntwk – 2-port lossless mismatch

Return type

Network object

skrf.media.mline.MLine.match

MLine.**match**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns

match – a n-port match

Return type

Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.mline.MLine.mode

MLine.**mode**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.mline.MLine.open

MLine.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.mline.MLine.plot

MLine.**plot**(**args*, ***kw*)

skrf.media.mline.MLine.random

MLine.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero

- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns
result – the network

Return type
Network object

skrf.media.mline.MLine.resistor

MLine.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
resistor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.mline.MLine.short

MLine.**short**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns
match – a n-port short circuit

Return type
Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.mline.MLine.shunt

MLine.**shunt**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns
shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type
Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.mline.MLine.shunt_capacitor

MLine.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
shunt_capacitor – shunted capacitor (2-port)

Return type
Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.mline.MLine.shunt_delay_load

MLine.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.mline.MLine.shunt_delay_open

MLine.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.mline.MLine.shunt_delay_short

MLine.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.mline.MLine.shunt_inductor

MLine.**shunt_inductor**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.mline.MLine.shunt_resistor

MLine.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.mline.MLine.splitter

MLine.**splitter**(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.mline.MLine.tee

MLine.**tee**(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.mline.MLine.theta_2_d

MLine.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)

- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.mline.MLine.thru

MLine.**thru**(*z0=None*, ***kwargs*)

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.mline.MLine.to_meters

MLine.**to_meters**(*d*, *unit='deg'*)

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.mline.MLine.white_gaussian_polar

MLine.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.mline.MLine.write_csv

MLine.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters
filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

skrf.media.freespace.Freespace

class skrf.media.freespace.**Freespace**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *ep_r=1 + 0j*, *mu_r=1 + 0j*, *ep_loss_tan=None*, *mu_loss_tan=None*, *rho=None*, **args*, ***kwargs*)

A plane-wave (TEM Mode) in Freespace.

A Freespace media can be constructed in two ways:

- from complex, relative permativity and permeability OR

- from real relative permativity and permeability with loss tangents.

There is also a method to initialize from a existing distributed circuit, appropriately named *Freespace.from_distributed_circuit()*

Parameters

- **frequency** (*Frequency* object) – frequency band of this transmission line medium
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **z0_override** (*number, array-like, or None*) – *z0_override* override the characteristic impedance for the media. If *z0_override* is not None, the networks generated by the media have their characteristic impedance *z0* overridden by *z0_override*. (Default is None)
- **z0** (*number, array-like, or None*) – deprecated parameter, alias to *z0_override* if *z0_override* is None. Emit a deprecation warning.
- **ep_r** (*number, array-like*) – complex relative permittivity. negative imaginary is lossy.
- **mu_r** (*number, array-like*) – complex relative permeability. negative imaginary is lossy.
- **ep_loss_tan** (*None, number, array-like*) – electric loss tangent (of the permativity). If not None, imag(ep_r) is ignored.
- **mu_loss_tan** (*None, number, array-like*) – magnetic loss tangent (of the permeability). If not None, imag(mu_r) is ignored.
- **rho** (*number, array-like, string or None*) – resistivity (ohm-m) of the conductor walls. If array-like must be same length as frequency. if str, it must be a key in *skrf.data.materials*. Default is None (lossless).
- ***args** (*arguments and keyword arguments*)
- ****kwargs** (*arguments and keyword arguments*)

Examples

```
>>> from skrf.media.freespace import Freespace
>>> from skrf.frequency import Frequency
>>> f = Frequency(75,110,101,'ghz')
>>> Freespace(frequency=f, ep_r=11.9)
>>> Freespace(frequency=f, ep_r=11.9-1.1j)
>>> Freespace(frequency=f, ep_r=11.9, ep_loss_tan=.1)
>>> Freespace(frequency=f, ep_r=11.9-1.1j, mu_r = 1.1-.1j)
```

Attributes

Z0	Characteristic Impedance
alpha	Real (attenuation) component of gamma.
beta	Imaginary (propagating) component of gamma.
ep	Complex dielectric permittivity.
ep_with_rho	Complex permittivity with resistivity absorbed into its imaginary component.

continues on next page

Table 43 – continued from previous page

gamma	Propagation Constant, γ.
mu	Complex dielectric permeability.
npoints	Number of points of the frequency axis.
rho	Conductivity in ohm*m.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).
z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	Characteristic Impedance, z_0.
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.freespace.Freespace.Z0

property Freespace.**Z0**

Characteristic Impedance

Returns
z0 – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.freespace.Freespace.alpha

property Freespace.**alpha: ndarray**

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.freespace.Freespace.beta

property Freespace.**beta: ndarray**

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.freespace.Freespace.ep

property Freespace.**ep: Number | Sequence[Number] | ndarray**

Complex dielectric permittivity.

If $\tan\delta_e$ is not defined:

$$\varepsilon = \varepsilon_0 \varepsilon_r$$

otherwise,

$$\varepsilon = \varepsilon_0 \Re[\varepsilon_r](1 - j\tan\delta_e)$$

where $\tan\delta_e$ is the electric loss tangent.

Returns
ep – Complex dielectric permittivity in F/m.

Return type
number or array-like

skrf.media.freespace.Freespace.ep_with_rho

property Freespace.**ep_with_rho**: Number | Sequence[Number] | ndarray

Complex permittivity with resistivity absorbed into its imaginary component.

$$\varepsilon - j\frac{1}{\rho\omega}$$

See also

rho, *ep*

skrf.media.freespace.Freespace.gamma

property Freespace.**gamma**: Number | Sequence[Number] | ndarray

Propagation Constant, γ.

Defined as,

$$\gamma = \sqrt{Z'Y'}$$

Returns
gamma – Propagation Constant,

Return type
np.ndarray

Note

The components of propagation constant are interpreted as follows:

- positive real(gamma) = attenuation
- positive imag(gamma) = forward propagation

skrf.media.freespace.Freespace.mu

property Freespace.**mu**: Number | Sequence[Number] | ndarray

Complex dielectric permeability.

If $\tan\delta_m$ is not defined:

$$\mu = \mu_0\mu_r$$

otherwise,

$$\mu = \mu_0\Re[\mu_r](1 - j\tan\delta_m)$$

where $\tan\delta_m$ is the magnetic loss tangent.

Returns
mu – Complex permeability in H/m.

Return type
number

skrf.media.freespace.Freespace.npoints

property Freespace.**npoints**: int

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.freespace.Freespace.rho

property Freespace.**rho**: Number | Sequence[Number] | ndarray

Conductivity in ohm*m.

Parameters
val (*float, array-like or str*) – the resistivity in ohm*m. If array-like must be same length as self.frequency. if str, it must be a key in *materials*.

Examples

```
>>> wg.rho = 2.8e-8
>>> wg.rho = 2.8e-8 * ones(len(wg.frequency))
>>> wg.rho = 'al'
>>> wg.rho = 'aluminum'
```

skrf.media.freespace.Freespace.v_g

property Freespace.**v_g**

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the j is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.freespace.Freespace.v_p

property Freespace.**v_p: ndarray**

Complex phase velocity (in m/s).

$$j \cdot \omega / \gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.freespace.Freespace.z0

property Freespace.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.freespace.Freespace.z0_characteristic

property Freespace.**z0_characteristic: Number | Sequence[Number] | ndarray**

Characteristic Impedance, z_0.

$$Z_0 = \sqrt{\frac{Z'}{Y'}}$$

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.freespace.Freespace.z0_override

property Freespace.**z0_override: ndarray**

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.freespace.Freespace.z0_port

property Freespace.**z0_port: ndarray**

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.
extract_distance	Determines physical distance from a transmission or reflection Network.
from_distributed_circuit	Initialize a freespace from *DistributedCircuit*.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.
open	Open ($\Gamma_0 = 1$).
plot	

continues on next page

Table 44 – continued from previous page

plot_ep	Plot the real and imaginary part of the complex permittivity.
plot_ep_mu	Plot the real and imaginary part of the complex permittivity with resistivity.
plot_mu	Plot the real and imaginary part of the complex permeability.
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.freespace.Freespace.__init__

Freespace.**__init__**(*frequency=None*, *z0_port=None*, *z0_override=None*, *z0=None*, *ep_r=1 + 0j*, *mu_r=1 + 0j*, *ep_loss_tan=None*, *mu_loss_tan=None*, *rho=None*, **args*, ***kwargs*)

Parameters

- **frequency** (*Frequency | None*)
- **z0_port** (*NumberLike | None*)
- **z0_override** (*NumberLike | None*)
- **z0** (*NumberLike | None*)
- **ep_r** (*NumberLike*)
- **mu_r** (*NumberLike*)
- **ep_loss_tan** (*NumberLike | None*)
- **mu_loss_tan** (*NumberLike | None*)
- **rho** (*NumberLike | str | None*)

skrf.media.freespace.Freespace.attenuator

Freespace.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation

- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns
ntwk – 2-port attenuator

Return type
Network object

skrf.media.freespace.Freespace.capacitor

Freespace.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
capacitor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.freespace.Freespace.capacitor_q

Freespace.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.freespace.Freespace.copy

Freespace.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.freespace.Freespace.delay_load

Freespace.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.freespace.Freespace.delay_open

Freespace.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns

delay_open – a delayed open

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.freespace.Freespace.delay_short

Freespace.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns

delay_short – a delayed short

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_open*

skrf.media.freespace.Freespace.electrical_length

`Freespace.`**`electrical_length`**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns
theta – complex electrical length in radians or degrees, depending on value of deg.

Return type
number or array-like

skrf.media.freespace.Freespace.extract_distance

`Freespace.`**`extract_distance`**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters
ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns
d – physical distance

Return type
number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.freespace.Freespace.from_distributed_circuit

`classmethod` `Freespace.`**`from_distributed_circuit`**(*dc*, **args*, ***kwargs*)

Initialize a freespace from *DistributedCircuit*.

Parameters

- **dc** (*DistributedCircuit*) – a DistributedCircuit object
- ***args** – passed to `Freespace.__init__
- ****kwargs** – passed to `Freespace.__init__

Return type
Media

Notes

Here are the details:

```
w = dc.frequency.w
z= dc.Z/(w*mu_0)
y= dc.Y/(w*epsilon_0)
ep_r = -1j*y
mu_r = -1j*z
```

See also

skrf.media.distributedCircuit.DistributedCircuit

skrf.media.freespace.Freespace.get_array_of

Freespace.**get_array_of**(*x*)

skrf.media.freespace.Freespace.impedance_mismatch

Freespace.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
mismatch – a 2-port network representing the impedance mismatch

Return type
Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.freespace.Freespace.inductor

Freespace.**inductor**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.freespace.Freespace.inductor_q

Freespace.**inductor_q**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.freespace.Freespace.isolator

Freespace.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.freespace.Freespace.line

Freespace.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line – matched transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.freespace.Freespace.line_floating

Freespace.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*

- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line_floating – matched, floating transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.freespace.Freespace.load

`Freespace.load`(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
load – n-port load, where S = Gamma0*eye(...)

Return type
Network object

See also

match, *open*, *short*

skrf.media.freespace.Freespace.lossless_mismatch

`Freespace.lossless_mismatch`(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns
ntwk – 2-port lossless mismatch

Return type
Network object

skrf.media.freespace.Freespace.match

`Freespace.match(nports=1, z0=None, z0_norm=False, **kwargs)`

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns
match – a n-port match

Return type
Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.freespace.Freespace.mode

`Freespace.mode(**kw)`

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
`Media`

skrf.media.freespace.Freespace.open

`Freespace.open(nports=1, **kwargs)`

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports

- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.freespace.Freespace.plot

Freespace.**plot**(**args*, ***kw*)

skrf.media.freespace.Freespace.plot_ep

Freespace.**plot_ep**()

Plot the real and imaginary part of the complex permittivity.

skrf.media.freespace.Freespace.plot_ep_mu

Freespace.**plot_ep_mu**()

Plot the real and imaginary part of the complex permittivity with resistivity.

skrf.media.freespace.Freespace.plot_mu

Freespace.**plot_mu**()

Plot the real and imaginary part of the complex permeability.

skrf.media.freespace.Freespace.random

Freespace.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero

- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns
result – the network

Return type
Network object

skrf.media.freespace.Freespace.resistor

Freespace.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
resistor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.freespace.Freespace.short

Freespace.**short**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z^*_{ref}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns
match – a n-port short circuit

Return type
Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.freespace.Freespace.shunt

Freespace.**shunt**(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns
shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type
Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.freespace.Freespace.shunt_capacitor

Freespace.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
shunt_capacitor – shunted capacitor (2-port)

Return type
Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.freespace.Freespace.shunt_delay_load

Freespace.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.freespace.Freespace.shunt_delay_open

Freespace.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.freespace.Freespace.shunt_delay_short

Freespace.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.freespace.Freespace.shunt_inductor

Freespace.**shunt_inductor**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.freespace.Freespace.shunt_resistor

Freespace.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.freespace.Freespace.splitter

Freespace.**splitter**(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.freespace.Freespace.tee

Freespace.**tee**(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.freespace.Freespace.theta_2_d

Freespace.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)

- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.freespace.Freespace.thru

`Freespace.thru(z0=None, **kwargs)`

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.freespace.Freespace.to_meters

`Freespace.to_meters(d, unit='deg')`

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.freespace.Freespace.white_gaussian_polar

Freespace.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.freespace.Freespace.write_csv

Freespace.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters
filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0

class skrf.media.definedAEpTandZ0.**DefinedAEpTandZ0**(*frequency=None*, *A=0.0*, *f_A=1.0*, *ep_r=1.0*, *tanD=0.0*, *z0_port=None*, *z0=50.0*, *Z0=None*, *f_low=1000.0*, *f_high=1000000000000.0*, *f_ep=1000000000.0*, *model='frequencyinvariant'*, **args*, ***kwargs*)

Transmission line medium defined by A, Ep, Tand and Z0.

This medium is defined by attenuation *A*, relative permittivity *Ep_r*, loss angle *tand* and nominal impedance *Z0*.

Djirdjevic[2] / Svennson[1] dispersion model is provided for dielectric. Default behaviour is frequency invariant.

Parameters

- **frequency** (*Frequency* object) – Frequency band of this transmission line medium
- **z0_port** (*number, array-like, or None*) – *z0_port* is the port impedance for networks generated by the media. If *z0_port* is not None, the networks generated by the media are renormalized (or in other words embedded) from the characteristic impedance z0 of the media to *z0_port*. Else if *z0_port* is None, the networks port impedances will be the raw characteristic impedance z0 of the media. (Default is None)
- **A** (*number, array-like, default 0.0*) – Attenuation due to conductor loss in dB/m/sqrt(Hz) The attenuation $A(f)$ at frequency f is:

$$A(f) = A\sqrt{\frac{f}{f_A}}$$

- **f_A** (*number, default 1.0*) – Frequency scaling in Hz for the attenuation. See A.
- **ep_r** (*number, array-like, default 1.0*) – Real part of the relative permittivity of the dielectric: $\epsilon_r' = \Re[\epsilon]$.

  If *model='frequencyinvariant'*, the complex relative permittivity is:

$$\epsilon_r(f) = \epsilon_r' + j \cdot \epsilon_r' \cdot \tan\delta$$

  if *model='djordjevicsvensson'*, the complex relative permittivity is:

$$\epsilon_r(f) = \epsilon_{\inf} + m \cdot \ln \frac{f_{high} + j f_\epsilon}{f_{low} + j f_\epsilon}$$

  where $\epsilon_{\inf}$ is the permittivity value when frequency approaches infinity. In this case, the value of ϵ_r and $\tan\delta$ (tanD) are given for frequency f_ϵ:

$$\epsilon_r(f_\epsilon) = \epsilon_r' + j \cdot \epsilon_r' \cdot \tan\delta$$

- **tanD** (*number, array-like, default 0.0*) – Dielectric relative permittivity loss tangent $\tan\delta$. See *ep_r*.
- **z0** (*number, array-like, default 50.0*) – Quasi-static nominal impedance of the medium. Because of the dispersion introduced by the conductor and dielectric losses, the characteristic impedance can be frequency-dependent with an imaginary part. The characteristic impedance is computed with an RLGC model. If the impedance parameter is array-like, the characteristic impedance is assigned to this value without modification.

$$R = 2 Z_n \alpha_{conductor}$$

$$L = \frac{Z_n \sqrt{\epsilon_r}}{C_0}$$

$$G = \frac{2}{Z_n} \alpha_{dielectric}$$

[2] A. R. Djordjevic, R. M. Biljic, V. D. Likar-Smiljanic, and T. K. Sarkar, "Wideband Frequency-Domain Characterization of FR-4 and Time-Domain Causality," IEEE Trans. Electromagnetic Compatibility, Vol. 43, No. 4, November 2001. https://doi.org/10.1109/15.974647

[1] C. Svensson and G. E. Dermer, "Time Domain Modeling of Lossy Interconnects," IEEE Trans. Advanced Packaging, Vol. 24, No. 2, May 2001. https://doi.org/10.1109/6040.928754

$$C = \frac{\sqrt{\epsilon_r}}{C_0 Z_n}$$

$$Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}}$$

where Z_n is the nominal impedance and Z_0 is the characteristic impedance.

- **Z0** (*number, array-like, or None*) – deprecated parameter, only emit a deprecation warning.
- **f_low** (*number, default 1e3, optional*) – Low frequency in Hz for for Djordjevic/Svennson dispersion model. See *ep_r*.
- **f_high** (*number, default 1e12, optional*) – High frequency in Hz for for Djordjevic/Svennson dispersion model. See *ep_r*.
- **f_ep** (*number, default 1e9, , optional*) – Specification frequency in Hz for for Djirdjevic/Svennson dispersion model. *ep_r* and *tanD* parameters are specified for this frequency. See *ep_r*.
- **model** (*string, 'frequencyinvariant' or 'djordjevicsvensson', optional*) – Use Djirdjevic/Svennson wideband Debye dispersion model or not. Default is frequency invariant behaviour.
- ***args** (*arguments and keyword arguments*)
- ****kwargs** (*arguments and keyword arguments*)

Examples

```
>>> from skrf.media.definedAEpTandZ0 import DefinedAEpTandZ0
>>> from skrf.frequency import Frequency
>>> f = Frequency(75,110,101,'ghz')
>>> DefinedAEpTandZ0(frequency=f, A=1, f_A=1e9, ep_r=3, z0=50)
>>> DefinedAEpTandZ0(frequency=f, A=1, f_A=1e9, ep_r=3, tand=0.02, z0=50)
>>> DefinedAEpTandZ0(frequency=f, A=1, f_A=1e9, ep_r=3, tand=0.02, z0=50,
                     f_low=1e3, f_high=1e12, f_Ep=1e9,
                     model='djordjevicsvensson')
```

References

Attributes

Z0	Characteristic Impedance
alpha	Real (attenuation) component of gamma.
alpha_conductor	Losses due to conductor resistivity.
alpha_dielectric	Losses due to dielectric
beta	Imaginary (propagating) component of gamma.
beta_phase	Phase parameter
ep_r_f	Frequency dependent complex relative permittivity of dielectric.
gamma	Propagation constant.
npoints	Number of points of the frequency axis.
tand_f	Frequency dependent dielectric loss factor.
v_g	Complex group velocity (in m/s).
v_p	Complex phase velocity (in m/s).

continues on next page

Table 45 – continued from previous page

z0	Return Characteristic Impedance *z0_characteristic*.
z0_characteristic	Characteristic Impedance, z_0
z0_override	Port Impedance.
z0_port	Port Impedance.

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.Z0

property DefinedAEpTandZ0.**Z0**

Characteristic Impedance

Returns
z0 – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.alpha

property DefinedAEpTandZ0.**alpha**: ndarray

Real (attenuation) component of gamma.

Returns
alpha

Return type
numpy.ndarray

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.alpha_conductor

property DefinedAEpTandZ0.**alpha_conductor**: Number | Sequence[Number] | ndarray

Losses due to conductor resistivity.

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.alpha_dielectric

property DefinedAEpTandZ0.**alpha_dielectric**: Number | Sequence[Number] | ndarray

Losses due to dielectric

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.beta

property DefinedAEpTandZ0.**beta**: ndarray

Imaginary (propagating) component of gamma.

Returns
beta

Return type
numpy.ndarray

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.beta_phase

property DefinedAEpTandZ0.**beta_phase**: Number | Sequence[Number] | ndarray

Phase parameter

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.ep_r_f

property DefinedAEpTandZ0.**ep_r_f**: Number | Sequence[Number] | ndarray

Frequency dependent complex relative permittivity of dielectric.

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.gamma

property DefinedAEpTandZ0.**gamma**: Number | Sequence[Number] | ndarray

Propagation constant.

See also

alpha_conductor
: calculates conductor losses

alpha_dielectric
: calculates dielectric losses

beta_phase
: calculates phase parameter

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.npoints

property DefinedAEpTandZ0.**npoints**: int

Number of points of the frequency axis.

Returns
npoints – Number of points of the frequency axis.

Return type
int

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.tand_f

property DefinedAEpTandZ0.**tand_f**: Number | Sequence[Number] | ndarray

Frequency dependent dielectric loss factor.

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.v_g

property DefinedAEpTandZ0.**v_g**

Complex group velocity (in m/s).

$$j \cdot d\omega/d\gamma$$

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Notes

the *j* is used to make propagation real, this is needed because skrf defined the gamma as $\gamma = \alpha + j\beta$.

Returns
v_g

Return type
numpy.ndarray

References

https://en.wikipedia.org/wiki/Group_velocity

See also

propagation_constant, *v_p*, *gamma*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.v_p

property DefinedAEpTandZ0.**v_p**: ndarray

Complex phase velocity (in m/s).

$$j \cdot \omega / \gamma$$

Notes

The *j* is used so that real phase velocity corresponds to propagation

where:

- ω is angular frequency (rad/s),
- γ is complex propagation constant (rad/m)

Returns
v_p

Return type
numpy.ndarray

See also

propagation_constant, *gamma*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.z0

property DefinedAEpTandZ0.**z0**

Return Characteristic Impedance *z0_characteristic*. If *z0_override* is not None, it is returned instead.

Returns
z0 – z0_characteristic or z0_override in units of ohms

Return type
np.ndarray

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.z0_characteristic

property DefinedAEpTandZ0.**z0_characteristic**: Number | Sequence[Number] | ndarray

Characteristic Impedance, z_0

Returns
z0_characteristic – Characteristic Impedance in units of ohms

Return type
np.ndarray

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.z0_override

property DefinedAEpTandZ0.**z0_override**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.z0_port

property DefinedAEpTandZ0.**z0_port**: ndarray

Port Impedance.

Returns
z0_port

Return type
numpy.ndarray

Methods

__init__	
attenuator	Ideal matched attenuator of a given length.
capacitor	Capacitor.
capacitor_q	Capacitor with Q factor.
copy	Copy of this Media object.
delay_load	Delayed load.
delay_open	Delayed open transmission line.
delay_short	Delayed Short.
electrical_length	Calculate the complex electrical length for a given distance.
extract_distance	Determines physical distance from a transmission or reflection Network.
get_array_of	
impedance_mismatch	Two-port network for an impedance mismatch.
inductor	Inductor.
inductor_q	Inductor with Q factor.
isolator	Two-port isolator.
line	Transmission line of a given length and impedance.
line_floating	Floating transmission line of a given length and impedance.
load	Load of given reflection coefficient.
lossless_mismatch	Lossless, symmetric mismatch defined by its return loss.
match	Perfect matched load ($\Gamma_0 = 0$).
mode	Create another mode in this medium.

continues on next page

Table 46 – continued from previous page

open	Open ($\Gamma_0 = 1$).
plot	
random	Complex random network.
resistor	Resistor.
short	Short ($\Gamma_0 = -1$)
shunt	Shunts a *Network*.
shunt_capacitor	Shunted capacitor.
shunt_delay_load	Shunted delayed load.
shunt_delay_open	Shunted delayed open.
shunt_delay_short	Shunted delayed short.
shunt_inductor	Shunted inductor.
shunt_resistor	Shunted resistor.
splitter	Ideal, lossless n-way splitter.
tee	Ideal, lossless tee.
theta_2_d	Convert electrical length to physical distance.
thru	Matched transmission line of length 0.
to_meters	Translate various units of distance into meters.
white_gaussian_polar	Complex zero-mean gaussian white-noise network.
write_csv	write this media's frequency, gamma, Z0, and z0 to a csv file.

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.__init__

DefinedAEpTandZ0.**__init__**(*frequency=None*, *A=0.0*, *f_A=1.0*, *ep_r=1.0*, *tanD=0.0*, *z0_port=None*, *z0=50.0*, *Z0=None*, *f_low=1000.0*, *f_high=1000000000000.0*, *f_ep=1000000000.0*, *model='frequencyinvariant'*, **args*, ***kwargs*)

Parameters

- **frequency** (*Frequency | None*)
- **A** (*float*)
- **f_A** (*float*)
- **ep_r** (*Number | Sequence[Number] | ndarray*)
- **tanD** (*Number | Sequence[Number] | ndarray*)
- **z0_port** (*Number | Sequence[Number] | ndarray | None*)
- **z0** (*float*)
- **Z0** (*Number | Sequence[Number] | ndarray | None*)
- **f_low** (*float*)
- **f_high** (*float*)
- **f_ep** (*float*)
- **model** (*str*)

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.attenuator

DefinedAEpTandZ0.**attenuator**(*s21*, *db=True*, *d=0*, *unit='deg'*, *name=''*, ***kwargs*)

Ideal matched attenuator of a given length.

Parameters

- **s21** (*number, array-like*) – the attenuation
- **db** (*bool, optional*) – is s21 in dB? otherwise assumes linear. Default is True (dB).
- **d** (*number, optional*) – length of attenuator. Default is 0.
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details. Default is 'deg'
- **name** (*str*) – Name for the returned attenuator Network
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*line*

Returns
ntwk – 2-port attenuator

Return type
Network object

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.capacitor

DefinedAEpTandZ0.**capacitor**(*C*, ***kwargs*)

Capacitor.

Parameters

- **C** (*number, array*) – Capacitance, in Farads. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
capacitor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *resistor*, *inductor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.capacitor_q

DefinedAEpTandZ0.**capacitor_q**(*C*, *f_0*, *q_factor*, ***kwargs*)

Capacitor with Q factor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of capacitor
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns
capacitor_q – capacitor_q (2-port)

Return type
Network object

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.copy

DefinedAEpTandZ0.**copy**()

Copy of this Media object.

Returns
copy – A copy of this Media object

Return type
Media

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.delay_load

DefinedAEpTandZ0.**delay_load**(*Gamma0*, *d*, *unit='deg'*, ***kwargs*)

Delayed load.

A load with reflection coefficient *Gamma0* at the end of a matched line of length *d*.

Parameters

- **Gamma0** (*number, array-like*) – reflection coefficient of load (not in dB)
- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (*key word arguments*) – passed to *line()* and *load()*

Returns
delay_load – a delayed load

Return type
Network object

Examples

```
>>> my_media.delay_load(-.5, 90, 'deg', Z0=50)
```

Notes

This calls

```
line(d, unit, **kwargs) ** load(Gamma0, **kwargs)
```

See also

line
creates the network for line

load
creates the network for the load

delay_short, *delay_open*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.delay_open

DefinedAEpTandZ0.**delay_open**(*d*, *unit='deg'*, ***kwargs*)

Delayed open transmission line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*)

Returns

delay_open – a delayed open

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=1.0, d=d, unit=unit, **kwargs)
```

See also

delay_load, *delay_short*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.delay_short

DefinedAEpTandZ0.**delay_short**(*d*, *unit='deg'*, ***kwargs*)

Delayed Short.

A transmission line of given length terminated with a short.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- ****kwargs** (key word arguments passed to *delay_load()*.)

Returns

delay_short – a delayed short

Return type

Network object

Notes

This calls:

```
delay_load(Gamma0=-1.0, d=d, unit=unit, **kwargs)
```

> **See also**
>
> *delay_load*, *delay_open*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.electrical_length

DefinedAEpTandZ0.**electrical_length**(*d*, *deg=False*)

Calculate the complex electrical length for a given distance.

Electrical length is given by $\theta = \gamma d$.

Parameters

- **d** (*number or array-like*) – delay distance, in meters
- **deg** (*Boolean, optional*) – return electrical length in deg? Default is False (returns electrical length in radians)

Returns
theta – complex electrical length in radians or degrees, depending on value of deg.

Return type
number or array-like

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.extract_distance

DefinedAEpTandZ0.**extract_distance**(*ntwk*)

Determines physical distance from a transmission or reflection Network.

Given a matched transmission or reflection measurement the physical distance is estimated at each frequency point based on the scattering parameter phase of the ntwk and propagation constant.

Notes

If the Network is a reflect measurement, the returned distance will be twice the physical distance.

Parameters
ntwk (*Network*) – A one-port network of either the reflection or the transmission.

Returns
d – physical distance

Return type
number or array_like

Examples

```
>>> air = rf.air50
>>> l = air.line(1, 'cm')
>>> d_found = air.extract_distance(l.s21)
>>> d_found
```

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.get_array_of

DefinedAEpTandZ0.**get_array_of**(*x*)

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.impedance_mismatch

DefinedAEpTandZ0.**impedance_mismatch**(*z1*, *z2*, ***kwargs*)

Two-port network for an impedance mismatch.

Parameters

- **z1** (*number, or array-like*) – complex impedance of port 1
- **z2** (*number, or array-like*) – complex impedance of port 2
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
mismatch – a 2-port network representing the impedance mismatch

Return type
Network object

Notes

If z1 and z2 are arrays, they must be of same length as the `Media.frequency.npoints`

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*, *resistor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.inductor

DefinedAEpTandZ0.**inductor**(*L*, ***kwargs*)

Inductor.

Parameters

- **L** (*number, array*) – Inductance, in Henries. If this is an array, must be of same length as frequency vector.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
inductor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *resistor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.inductor_q

DefinedAEpTandZ0.**inductor_q**(*L*, *f_0*, *q_factor*, *rdc=0.0*, ***kwargs*)

Inductor with Q factor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- **f_0** (*number*) – Frequency at which Q is defined, in Hz.
- **q_factor** (*number*) – Q-factor of inductor
- **rdc** (*number, optional*) – DC resistance, in Ohms. Default is 0 Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns
inductor_q – inductor_q (2-port)

Return type
Network object

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.isolator

DefinedAEpTandZ0.**isolator**(*source_port=0*, ***kwargs*)

Two-port isolator.

Parameters

- **source_port** (*int in [0,1], optional*) – port at which power can flow from. Default is 0.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*thru*

Returns
ntwk – 2-port isolator

Return type
Network object

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.line

DefinedAEpTandZ0.**line**(*d*, *unit='deg'*, *z0=None*, *embed=False*, ***kwargs*)

Transmission line of a given length and impedance.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced. if *embed* is also True, then the line is embedded in this media's z0 environment, creating a mismatched line.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from self.z0. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- **embed** (*bool*) – deprecated parameter that only emit a warning if set to true.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
line – matched transmission line of given length

Return type
Network object

Examples

```
>>> my_media.line(1, 'mm', z0=100)
>>> my_media.line(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.line_floating

DefinedAEpTandZ0.**line_floating**(*d*, *unit='deg'*, *z0=None*, ***kwargs*)

Floating transmission line of a given length and impedance.

This method returns a transmission line with floating shields. This is a four-port network, as opposed to the two-port _line_ method. Ports 1 and 3 are one side of the line, and electrically connect to ports 2 and 4 on the other side, respectively.

The units of *length* are interpreted according to the value of *unit*. If *z0* is not None, then a line specified impedance is produced.

Parameters

- **d** (*number*) – the length of transmission line (see unit argument)
- **unit** (*['deg','rad','m','cm','um','in','mil','s','us','ns','ps']*) – the units of d. See *to_meters()*, for details
- **z0** (*number, string, or array-like or None*) – the characteristic impedance of the line, if different from *media.z0*. To set z0 in terms of normalized impedance, pass a string, like *z0='1+.2j'*
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns

line_floating – matched, floating transmission line of given length

Return type

Network object

Examples

```
>>> my_media.line_floating(1, 'mm', z0=100)
>>> my_media.line_floating(90, 'deg', z0='2') # set z0 as normalized impedance
```

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.load

DefinedAEpTandZ0.**load**(*Gamma0*, *nports=1*, *z0=None*, ***kwargs*)

Load of given reflection coefficient.

Parameters

- **Gamma0** (*number, array-like*) – Reflection coefficient of load (linear, not in db). If its an array it must be of shape: *kxnxn*, where k is number of frequency points in media, and n is *nports*
- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an

optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native **z0**.

- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
load – n-port load, where S = Gamma0*eye(...)

Return type
Network object

See also

match, *open*, *short*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.lossless_mismatch

DefinedAEpTandZ0.**lossless_mismatch**(*s11*, *db=True*, ***kwargs*)

Lossless, symmetric mismatch defined by its return loss.

Parameters

- **s11** (*complex number, number, or array-like*) – the reflection coefficient. if db==True, then phase is ignored
- **db** (*bool, optional*) – is s11 in db? otherwise assumes linear. Default is True (dB)
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*match*

Returns
ntwk – 2-port lossless mismatch

Return type
Network object

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.match

DefinedAEpTandZ0.**match**(*nports=1*, *z0=None*, *z0_norm=False*, ***kwargs*)

Perfect matched load ($\Gamma_0 = 0$).

Parameters

- **nports** (*int*) – number of ports
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's **z0_port** value is used. If **z0_port** is also None (by default), in which case the Media's native **z0** is used. A Media's **z0_port** is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native **z0**.
- **z0_norm** (*bool*) – is z0 normalized to this media's *z0*?
- ****kwargs** (*key word arguments*) – passed to *Network* initializer

Returns
match – a n-port match

Return type
Network object

Examples

```
>>> my_match = my_media.match(2,z0 = 50, name='Super Awesome Match')
```

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.mode

DefinedAEpTandZ0.**mode**(***kw*)

Create another mode in this medium.

Convenient way to return a copy this Media object with eventually different properties.

Parameters
****kwargs** (*keyword arguments passed to the copy*)

Returns
copy – A copy of this Media object with **kwargs attribute

Return type
Media

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.open

DefinedAEpTandZ0.**open**(*nports=1*, ***kwargs*)

Open ($\Gamma_0 = 1$).

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*)

Returns
match – a n-port open circuit

Return type
Network object

Notes

This calls

```
load(1.0, nports, **kwargs)
```

See also

match, *load*, *short*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.plot

DefinedAEpTandZ0.**plot**(**args*, ***kw*)

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.random

DefinedAEpTandZ0.**random**(*n_ports=1*, *reciprocal=False*, *matched=False*, *symmetric=False*, *rng=None*, ***kwargs*)

Complex random network.

Creates a n-port network whose s-matrix is filled with random complex numbers. Optionally, result can be matched or reciprocal.

Parameters

- **n_ports** (*int*) – number of ports.
- **reciprocal** (*bool*) – makes s-matrix symmetric ($S_{mn} = S_{nm}$)
- **symmetric** (*bool*) – makes s-matrix diagonal have single value ($S_{mm}=S_{nn}$)
- **matched** (*bool*) – makes diagonals of s-matrix zero
- **rng** (numpy.random.Generator or None) – override the global numpy random number generator, useful for multi-threaded programs since *skrf.mathFunctions.set_rand_rng()* is not thread-safe.
- ****kwargs** (passed to *Network*) – initializer

Returns
result – the network

Return type
Network object

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.resistor

DefinedAEpTandZ0.**resistor**(*R*, **args*, ***kwargs*)

Resistor.

Parameters

- **R** (*number, array*) – Resistance , in Ohms. If this is an array, must be of same length as frequency vector.
- ***args** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.
- ****kwargs** (*arguments, key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
resistor

Return type
a 2-port *Network*

See also

match, *short*, *open*, *load*, *capacitor*, *inductor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.short

DefinedAEpTandZ0.**short**(*nports=1*, *z0=None*, ***kwargs*)

Short ($\Gamma_0 = -1$)

For s_def = 'power' ($-Z_{ref}^{*}/Z_{ref}$)

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (key word arguments passed to *load()*.)
- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's `z0_port` value is used. If `z0_port` is also None (by default), in which case the Media's native `z0` is used. A Media's `z0_port` is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native `z0`.

Returns

match – a n-port short circuit

Return type

Network object

Notes

This calls

```
load(-1.0, nports, **kwargs)
```

See also

match, *open*, *load*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.shunt

`DefinedAEpTandZ0.shunt`(*ntwk*, ***kwargs*)

Shunts a *Network*.

This creates a *tee()* and connects *ntwk* to port 1, and returns the result.

Parameters

- **ntwk** (*Network* object)
- ****kwargs** (keyword arguments passed to *tee()*)

Returns

shunted_ntwk – a shunted a ntwk. The resultant shunted_ntwk will have (2 + ntwk.number_of_ports -1) ports.

Return type

Network object

See also

shunt_delay_load, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.shunt_capacitor

DefinedAEpTandZ0.**shunt_capacitor**(*C*, ***kwargs*)

Shunted capacitor.

Parameters

- **C** (*number, array-like*) – Capacitance in Farads.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*capacitor*

Returns

shunt_capacitor – shunted capacitor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(capacitor(C, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_inductor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.shunt_delay_load

DefinedAEpTandZ0.**shunt_delay_load**(**args*, ***kwargs*)

Shunted delayed load.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_load*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_load*

Returns

shunt_delay_load – a shunted delayed load (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_load(*args, **kwargs))
```

See also

shunt, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.shunt_delay_open

DefinedAEpTandZ0.**shunt_delay_open**(**args*, ***kwargs*)

Shunted delayed open.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_open*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_open*

Returns

shunt_delay_open – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_open(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*, *shunt_inductor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.shunt_delay_short

DefinedAEpTandZ0.**shunt_delay_short**(**args*, ***kwargs*)

Shunted delayed short.

Parameters

- ***args** (*arguments, keyword arguments*) – passed to func:*delay_short*
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*delay_short*

Returns

shunt_delay_load – shunted delayed open (2-port)

Return type

Network object

Notes

This calls:

```
shunt(delay_short(*args, **kwargs))
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_capacitor*, *shunt_inductor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.shunt_inductor

DefinedAEpTandZ0.**shunt_inductor**(*L*, ***kwargs*)

Shunted inductor.

Parameters

- **L** (*number, array-like*) – Inductance in Henries.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*inductor*

Returns

shunt_inductor – shunted inductor(2-port)

Return type

Network object

Notes

This calls:

```
shunt(inductor(L, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_resistor*, *shunt_capacitor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.shunt_resistor

DefinedAEpTandZ0.**shunt_resistor**(*R*, ***kwargs*)

Shunted resistor.

Parameters

- **R** (*number, array-like*) – Resistor in Ohm.
- ****kwargs** (*arguments, keyword arguments*) – passed to func:*resistor*

Returns

shunt_resistor – shunted resistor (2-port)

Return type

Network object

Notes

This calls:

```
shunt(resistor(R, **kwargs) ** short())
```

See also

shunt, *shunt_delay_load*, *shunt_delay_open*, *shunt_delay_short*, *shunt_inductor*, *shunt_capacitor*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.splitter

DefinedAEpTandZ0.**splitter**(*nports*, ***kwargs*)

Ideal, lossless n-way splitter.

The port impedances can be mismatched and the power is split accordingly.

For n > 2, the splitter is not matched because the power wave entering one port meet the equivalent impedance of the other ports in parallel.

$$s_{ii} = \frac{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} - Z_i^*}{\frac{1}{\sum_{j \neq i} \frac{1}{Z_j}} + Z_i}$$

The remaining power is split between the other ports depending their impedances.

$$S_{ij} = \frac{2\sqrt{\Re(Z_i) \cdot \Re(Z_j)}}{Z_i \cdot Z_j \cdot \sum_{k=1 \ldots n} \frac{1}{Z_j}}$$

Parameters

- **nports** (*int*) – number of ports
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a n-port splitter

Return type
Network object

See also

match
called to create a 'blank' network

splitter_s
actual S-parameter matrix generator algorithm.

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.tee

DefinedAEpTandZ0.**tee**(***kwargs*)

Ideal, lossless tee. (3-port splitter).

Parameters
****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
tee – a 3-port splitter

Return type
Network object

See also

splitter
this just calls splitter(3)

match
called to create a 'blank' network

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.theta_2_d

DefinedAEpTandZ0.**theta_2_d**(*theta*, *deg=True*, *bc=True*)

Convert electrical length to physical distance.

The electrical length is given by $d = \theta/\beta$.

The given electrical length can be given either at the center frequency or on the entire band depending of the parameter *bc*.

Parameters

- **theta** (*number*) – electrical length, at band center (see deg for unit)
- **deg** (*Boolean, optional*) – is theta in degrees? Default is True (theta is assumed in degrees)
- **bc** (*bool, optional.*) – evaluate only at band center, or across the entire band? Default is True (evaluation assumed at band center)

Returns
d – physical distance in meters

Return type
number, array-like

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.thru

DefinedAEpTandZ0.**thru**(*z0=None*, ***kwargs*)

Matched transmission line of length 0.

Parameters

- **z0** (*number, or array-like or None*) – port impedance. This sets the resultant *Network*'s *z0*. Default is None, the Media's z0_port value is used. If z0_port is also None (by default), in which case the Media's native z0 is used. A Media's z0_port is an optional attribute for renormalizing resultant *Network* to a new reference impedance that differs from a Media's native z0.
- ****kwargs** (*key word arguments*) – passed to *match()*, which is called initially to create a 'blank' network.

Returns
thru – matched transmission line of 0 length

Return type
Network object

See also

line
this just calls line(0)

open, *short*, *match*

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.to_meters

DefinedAEpTandZ0.**to_meters**(*d*, *unit='deg'*)

Translate various units of distance into meters.

This is a method of media to allow for electrical lengths as inputs. For dispersive media, mean group velocity is used to translate time-based units to distance.

Parameters

- **d** (*number or array-like*) – the value
- **unit** (*str*) – the unit to that x is in: ['deg','rad','m','cm','um','in','mil','s','us','ns','ps']

Returns
d_m – d in meters

Return type
number, array-like

See also

skrf.constants.to_meters

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.white_gaussian_polar

DefinedAEpTandZ0.**white_gaussian_polar**(*phase_dev*, *mag_dev*, *n_ports=1*, ***kwargs*)

Complex zero-mean gaussian white-noise network.

Creates a network whose s-matrix is complex zero-mean gaussian white-noise, of given standard deviations for phase and magnitude components. This 'noise' network can be added to networks to simulate additive noise.

Parameters

- **phase_mag** (*number*) – standard deviation of magnitude
- **phase_dev** (*number*) – standard deviation of phase
- **n_ports** (*int*) – number of ports.
- ****kwargs** (passed to *Network*) – initializer
- **mag_dev** (*Number*)

Returns
result – a noise network

Return type
Network object

skrf.media.definedAEpTandZ0.DefinedAEpTandZ0.write_csv

DefinedAEpTandZ0.**write_csv**(*filename='f,gamma,Z0,z0.csv'*)

write this media's frequency, gamma, Z0, and z0 to a csv file.

Parameters

filename (*string, optional*) – file name to write out data to. Default is 'f,gamma,Z0,z0.csv', so you probably want to specify it.

See also

from_csv
class method to initialize Media object from a csv file written from this function

3.6 taper (*skrf.taper*)

Taper Objects

Tapered transformers, or tapers in short, are used to match one impedance to another (from Z1 to Z2)[1].

By tapering a transmission line, a very broadband impedance match (low VSWR) can be realized over a wide bandwidth, the longer the taper, the wider the frequency band.

References

Taper1D(med, start, stop, n_sections, f, length)	Generic 1D Taper Object
Linear(**kw)	A linear Taper.
Exponential(**kw)	An Exponential Taper.
SmoothStep(**kw)	A smoothstep Taper.
Klopfenstein(**kw)	Klopfenstein Taper.

3.6.1 skrf.taper.Taper1D

class skrf.taper.**Taper1D**(*med*, *start*, *stop*, *n_sections*, *f*, *length*, *length_unit='m'*, *param='z0'*, *f_is_normed=True*, *med_kw=None*, *f_kw=None*)

Generic 1D Taper Object

Parameters

- **start** (*Number*)
- **stop** (*Number*)
- **n_sections** (*int*)
- **f** (*Callable*)
- **length** (*Number*)
- **length_unit** (*str*)
- **param** (*str*)
- **f_is_normed** (*bool*)

[1] https://www.microwaves101.com/encyclopedias/tapered-transformers

- **med_kw** (dict)
- **f_kw** (dict)

Attributes

medias	List of medias.
network	Resulting Network
section_length	Section length.
sections	List of sections.
value_vector	

skrf.taper.Taper1D.medias

property Taper1D.**medias**: list

List of medias.

Returns
medias

Return type
list of *Media*

skrf.taper.Taper1D.network

property Taper1D.**network**

Resulting Network

Returns
ntwk

Return type
Network

skrf.taper.Taper1D.section_length

property Taper1D.**section_length**: Number

Section length.

Returns
l

Return type
number

skrf.taper.Taper1D.sections

property Taper1D.**sections**: list

List of sections.

Returns
sections

Return type
list of *Network*

skrf.taper.Taper1D.value_vector

property Taper1D.**value_vector**

Methods

__init__	Generic 1D Taper Constructor.
media_at	Create a media instance for the taper with parameter value *val*.
section_at	Create a single section of the taper with parameter value *val*.

skrf.taper.Taper1D.__init__

Taper1D.**__init__**(*med*, *start*, *stop*, *n_sections*, *f*, *length*, *length_unit='m'*, *param='z0'*, *f_is_normed=True*, *med_kw=None*, *f_kw=None*)

Generic 1D Taper Constructor.

Parameters

- **med** (*Media*) – A *Media* object or a *@classmethod __init__*, used to generate the transmission line. See *med_kw* for arguments. Examples:
  - *skrf.media.RectangularWaveguide* # a class
  - *skrf.media.RectangularWaveguide.from_z0* # an init
- **start** (*number*) – starting value for *param*
- **stop** (*number*) – stop value for *param*
- **n_sections** (*int*) – number of sections in taper
- **f** (*Callable*) – function defining the taper transition. must take either no arguments or take *(x,length, start, stop)*. see *f_is_normed* arguments
- **length** (*number*) – physical length of the taper (in *length_unit*)
- **length_unit** (*str*) – unit of length variable. see *skrf.to_meters*
- **param** (*str*) – name of the parameter of *med* that varies along the taper
- **f_is_normed** (*bool*) – is *f* scalable and normalized. ie can f just be scaled to fit different start/stop conditions? if so then f is called with no arguments, and must have domain and raings of [0,1], and [0,1]
- **f_kw** (*dict*) – passed to *f()* when called
- **med_kw** (*dict*) – passed to *med.__init__* when an instance is created

Note

The default behaviour should is to taper based on impedance. To do this we inspect the *med* class for a *from_z0* init method, and if it exists, we assign it to *med* attribute, in *__init__*. Admittedly having *med* be a class or a method is abuse, it makes for a intuitive operation.

Examples

Create a linear taper from 100 to 1000mil

```
>>> from skrf import Frequency, RectangularWaveguide, Taper1D, mil, inch
>>> taper = Taper1D(med=RectangularWaveguide,
                    param='a',
                    start=100*mil,
                    stop=1000*mil,
                    length=1*inch,
                    n_sections=20,
                    f=lambda x: x,
                    f_is_normed=True,
                    med_kw={'frequency':Frequency(75,110,101,'ghz')})
```

skrf.taper.Taper1D.media_at

Taper1D.**media_at**(*val*)

Create a media instance for the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – media instance for the taper for the given parameter value

Return type
Media

skrf.taper.Taper1D.section_at

Taper1D.**section_at**(*val*)

Create a single section of the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – Network instance for the section of the taper for the given parameter value

Return type
Network

3.6.2 skrf.taper.Linear

class skrf.taper.**Linear**(**kw*)

A linear Taper.

Defined by $f(x) = x$

Attributes

medias	List of medias.
network	Resulting Network
section_length	Section length.
sections	List of sections.

continues on next page

Table 50 – continued from previous page

value_vector

skrf.taper.Linear.medias

property Linear.**medias**: list

List of medias.

Returns
medias

Return type
list of *Media*

skrf.taper.Linear.network

property Linear.**network**

Resulting Network

Returns
ntwk

Return type
Network

skrf.taper.Linear.section_length

property Linear.**section_length**: Number

Section length.

Returns
l

Return type
number

skrf.taper.Linear.sections

property Linear.**sections**: list

List of sections.

Returns
sections

Return type
list of *Network*

skrf.taper.Linear.value_vector

property Linear.**value_vector**

Methods

__init__	Generic 1D Taper Constructor.
media_at	Create a media instance for the taper with parameter value *val*.

continues on next page

Table 51 – continued from previous page

section_at	Create a single section of the taper with parameter value *val*.

skrf.taper.Linear.__init__

Linear.**__init__**(***kw*)

Generic 1D Taper Constructor.

Parameters

- **med** (*Media*) – A *Media* object or a *@classmethod __init__*, used to generate the transmission line. See *med_kw* for arguments. Examples:
  - *skrf.media.RectangularWaveguide* # a class
  - *skrf.media.RectangularWaveguide.from_z0* # an init
- **start** (*number*) – starting value for *param*
- **stop** (*number*) – stop value for *param*
- **n_sections** (*int*) – number of sections in taper
- **f** (*Callable*) – function defining the taper transition. must take either no arguments or take *(x,length, start, stop)*. see *f_is_normed* arguments
- **length** (*number*) – physical length of the taper (in *length_unit*)
- **length_unit** (*str*) – unit of length variable. see *skrf.to_meters*
- **param** (*str*) – name of the parameter of *med* that varies along the taper
- **f_is_normed** (*bool*) – is *f* scalable and normalized. ie can f just be scaled to fit different start/stop conditions? if so then f is called with no arguments, and must have domain and raings of [0,1], and [0,1]
- **f_kw** (*dict*) – passed to *f()* when called
- **med_kw** (*dict*) – passed to *med.__init__* when an instance is created

Note

The default behaviour should is to taper based on impedance. To do this we inspect the *med* class for a *from_z0* init method, and if it exists, we assign it to *med* attribute, in *__init__*. Admittedly having *med* be a class or a method is abuse, it makes for a intuitive operation.

Examples

Create a linear taper from 100 to 1000mil

```
>>> from skrf import Frequency, RectangularWaveguide, Taper1D, mil, inch
>>> taper = Taper1D(med=RectangularWaveguide,
                    param='a',
                    start=100*mil,
                    stop=1000*mil,
                    length=1*inch,
                    n_sections=20,
                    f=lambda x: x,
```

(continues on next page)

(continued from previous page)

```
                    f_is_normed=True,
                    med_kw={'frequency':Frequency(75,110,101,'ghz')})
```

skrf.taper.Linear.media_at

Linear.**media_at**(*val*)

Create a media instance for the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – media instance for the taper for the given parameter value

Return type
Media

skrf.taper.Linear.section_at

Linear.**section_at**(*val*)

Create a single section of the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – Network instance for the section of the taper for the given parameter value

Return type
Network

3.6.3 skrf.taper.Exponential

class skrf.taper.**Exponential**(***kw*)

An Exponential Taper.

Defined by $f(x) = f_0 \exp\left[\frac{x}{x_1} \ln\left(\frac{f_1}{f_0}\right)\right]$

where:

- f_0: start param value
- f_1: stop param value
- x: independent variable (position along taper)
- x_1: length of taper

Attributes

medias	List of medias.
network	Resulting Network
section_length	Section length.
sections	List of sections.
value_vector	

skrf.taper.Exponential.medias

property Exponential.**medias**: list

List of medias.

Returns
medias

Return type
list of *Media*

skrf.taper.Exponential.network

property Exponential.**network**

Resulting Network

Returns
ntwk

Return type
Network

skrf.taper.Exponential.section_length

property Exponential.**section_length**: Number

Section length.

Returns
l

Return type
number

skrf.taper.Exponential.sections

property Exponential.**sections**: list

List of sections.

Returns
sections

Return type
list of *Network*

skrf.taper.Exponential.value_vector

property Exponential.**value_vector**

Methods

__init__	Exponential Taper Constructor
media_at	Create a media instance for the taper with parameter value *val*.
section_at	Create a single section of the taper with parameter value *val*.

skrf.taper.Exponential.__init__

Exponential.**__init__**(***kw*)

Exponential Taper Constructor

skrf.taper.Exponential.media_at

Exponential.**media_at**(*val*)

Create a media instance for the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – media instance for the taper for the given parameter value

Return type
Media

skrf.taper.Exponential.section_at

Exponential.**section_at**(*val*)

Create a single section of the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – Network instance for the section of the taper for the given parameter value

Return type
Network

3.6.4 skrf.taper.SmoothStep

class skrf.taper.**SmoothStep**(***kw*)

A smoothstep Taper.

There is no analytical basis for this in the EE world that I know of. it is just a reasonable smooth curve, that is easy to implement.

$f(x) = (3x^2 - 2x^3)$

References

https://en.wikipedia.org/wiki/Smoothstep

Attributes

medias	List of medias.
network	Resulting Network
section_length	Section length.
sections	List of sections.
value_vector	

skrf.taper.SmoothStep.medias

property SmoothStep.**medias: list**

List of medias.

Returns
medias

Return type
list of *Media*

skrf.taper.SmoothStep.network

property SmoothStep.**network**

Resulting Network

Returns
ntwk

Return type
Network

skrf.taper.SmoothStep.section_length

property SmoothStep.**section_length: Number**

Section length.

Returns
l

Return type
number

skrf.taper.SmoothStep.sections

property SmoothStep.**sections: list**

List of sections.

Returns
sections

Return type
list of *Network*

skrf.taper.SmoothStep.value_vector

property SmoothStep.**value_vector**

Methods

__init__	Smoothstep Taper Constructor.
media_at	Create a media instance for the taper with parameter value *val*.
section_at	Create a single section of the taper with parameter value *val*.

skrf.taper.SmoothStep.__init__

SmoothStep.**__init__**(***kw*)

Smoothstep Taper Constructor.

skrf.taper.SmoothStep.media_at

SmoothStep.**media_at**(*val*)

Create a media instance for the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – media instance for the taper for the given parameter value

Return type
Media

skrf.taper.SmoothStep.section_at

SmoothStep.**section_at**(*val*)

Create a single section of the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – Network instance for the section of the taper for the given parameter value

Return type
Network

3.6.5 skrf.taper.Klopfenstein

class skrf.taper.**Klopfenstein**(***kw*)

Klopfenstein Taper.

This impedance taper was first described by R. W. Klopfenstein[1] in a paper titled "A Transmission Line Taper of Improved Design", published in 1956[2] . A correction to Klopfenstein's math was published by in May 1973 by Darko Kajfez and Jame Prewit[3] .

- rmax: maximum reflection coefficient in the pass band in percentage of :math:`

ho_0`

Attributes

medias	List of medias.
network	Resulting Network
section_length	Section length.

continues on next page

[1] https://www.microwaves101.com/encyclopedias/klopfenstein-taper
[2] R. W. Klopfenstein Proc. IRE. vol. 44 pp. 31-35 Jan. 1956. doi: 10.1109/JRPROC.1956.274847 https://ieeexplore.ieee.org/document/4051841
[3] D. Kajfez and J. O. Prewitt, "Correction to "A Transmission Line Taper of Improved Design" (Letters)," in IEEE Transactions on Microwave Theory and Techniques, vol. 21, no. 5, pp. 364-364, May 1973, doi: 10.1109/TMTT.1973.1128003. https://ieeexplore.ieee.org/document/1128003

Table 56 – continued from previous page

sections	List of sections.
value_vector	

skrf.taper.Klopfenstein.medias

property Klopfenstein.**medias**: list

List of medias.

Returns
medias

Return type
list of *Media*

skrf.taper.Klopfenstein.network

property Klopfenstein.**network**

Resulting Network

Returns
ntwk

Return type
Network

skrf.taper.Klopfenstein.section_length

property Klopfenstein.**section_length**: Number

Section length.

Returns
l

Return type
number

skrf.taper.Klopfenstein.sections

property Klopfenstein.**sections**: list

List of sections.

Returns
sections

Return type
list of *Network*

skrf.taper.Klopfenstein.value_vector

property Klopfenstein.**value_vector**

Methods

__init__	Klopfenstein Taper Constructor.

continues on next page

Table 57 – continued from previous page

media_at	Create a media instance for the taper with parameter value *val*.
section_at	Create a single section of the taper with parameter value *val*.

skrf.taper.Klopfenstein.__init__

Klopfenstein.**__init__**(***kw*)

Klopfenstein Taper Constructor.

skrf.taper.Klopfenstein.media_at

Klopfenstein.**media_at**(*val*)

Create a media instance for the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – media instance for the taper for the given parameter value

Return type
Media

skrf.taper.Klopfenstein.section_at

Klopfenstein.**section_at**(*val*)

Create a single section of the taper with parameter value *val*.

Parameters
val (*number*) – parameter value

Returns
media – Network instance for the section of the taper for the given parameter value

Return type
Network

3.7 calibration (*skrf.calibration.calibration*)

This module provides objects for VNA calibration. Specific algorithms inherit from the common base class *Calibration*.

3.7.1 Base Class

Calibration(measured, ideals[, ...])	Base class for all Calibration objects.

skrf.calibration.calibration.Calibration

class skrf.calibration.calibration.**Calibration**(*measured*, *ideals*, *sloppy_input=False*, *is_reciprocal=True*, *name=None*, *self_calibration=False*, **args*, ***kwargs*)

Base class for all Calibration objects.

This class implements the common mechanisms for all calibration algorithms. Specific calibration algorithms should inherit this class and override the methods: * *Calibration.run()* * *Calibration.apply_cal()* * *Calibration.embed()* (optional)

The family of properties prefixed *coefs* and *coefs..ntwks* returns error coefficients. If the property coefs is accessed and empty, then *Calibration.run()* is called.

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.Calibration.biased_error

property Calibration.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns

biased_error – Network with s_mag is proportional to the biased error

Return type

skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.Calibration.caled_ntwk_sets

property Calibration.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.Calibration.caled_ntwks

property Calibration.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.Calibration.coefs

property Calibration.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.Calibration.coefs_12term

property Calibration.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity

- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.Calibration.coefs_12term_ntwks

property Calibration.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.Calibration.coefs_3term

property Calibration.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.Calibration.coefs_3term_ntwks

property Calibration.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.Calibration.coefs_8term

property Calibration.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.Calibration.coefs_8term_ntwks

property Calibration.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.Calibration.coefs_ntwks

property Calibration.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.Calibration.error_ntwk

property Calibration.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.Calibration.family

Calibration.**family = ''**

skrf.calibration.calibration.Calibration.frequency

property Calibration.**frequency**

Frequency object of the calibration.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.Calibration.normalized_directivity

property Calibration.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.Calibration.nstandards

property Calibration.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.Calibration.output_from_run

property Calibration.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.Calibration.residual_ntwk_sets

property Calibration.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.Calibration.residual_ntwks

property Calibration.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.Calibration.total_error

property Calibration.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.Calibration.unbiased_error

property Calibration.**unbiased_error**: *NetworkSet*

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.Calibration.verify_12term

property Calibration.**verify_12term**

skrf.calibration.calibration.Calibration.verify_12term_ntwk

property Calibration.**verify_12term_ntwk**

Methods

__init__	Calibration initializer.
apply_cal	Apply correction to a Network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	Run the calibration algorithm.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.Calibration.__init__

Calibration.**__init__**(*measured*, *ideals*, *sloppy_input=False*, *is_reciprocal=True*, *name=None*, *self_calibration=False*, **args*, ***kwargs*)

Calibration initializer.

Notes

About the order of supplied standards,

If the measured and ideals parameters are lists of Networks and *sloppy_input=False*, then their elements must align. However, if the measured and ideals are dictionaries, or *sloppy_input=True*, then we will try to align them for you based on the names of the networks (see *func: `align_measured_ideals*).

You do not want to use this *sloppy_input* feature if the calibration depends on the standard order (like TRL).

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use *sloppy_input*)
- **sloppy_input** (*Boolean.*) – Allows ideals and measured lists to be 'aligned' based on the network names.
- **is_reciprocal** (*Boolean*) – enables the reciprocity assumption on the calculation of the error_network, which is only relevant for one-port calibrations.
- **name** (*string*) – the name of this calibration instance, like 'waveguide cal' this is just for convenience [None].
- **self_calibration** (*Boolean*) – True if there are less ideals than measurements. Used in self-calibration such as LMR, LRRM, where some of the standards can be unknown.
- ***args** (*key-word arguments*) – stored in self.kwargs, which may be used by sub-classes most likely in *run*.
- ****kwargs** (*key-word arguments*) – stored in self.kwargs, which may be used by sub-classes most likely in *run*.

skrf.calibration.calibration.Calibration.apply_cal

Calibration.**apply_cal**(*ntwk*)

Apply correction to a Network.

skrf.calibration.calibration.Calibration.apply_cal_to_all_in_dir

Calibration.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.Calibration.apply_cal_to_list

Calibration.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.Calibration.apply_cal_to_network_set

Calibration.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.Calibration.embed

Calibration.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.Calibration.from_coefs

classmethod Calibration.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.Calibration.from_coefs_ntwks

classmethod Calibration.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.Calibration.plot_caled_ntwks

Calibration.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.Calibration.plot_calibration_errors

Calibration.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.Calibration.plot_residuals

Calibration.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

> **See also**
>
> Calibration.residual_networks

skrf.calibration.calibration.Calibration.pop

Calibration.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.Calibration.remove_and_cal

Calibration.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.Calibration.run

`Calibration.`**`run`**`()`

Run the calibration algorithm.

skrf.calibration.calibration.Calibration.update_coefs

`Calibration.`**`update_coefs`**`(`*d*`)`

Update current dict of error coefficients.

skrf.calibration.calibration.Calibration.write

`Calibration.`**`write`**`(`*file=None*, **args*, ***kwargs*`)`

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

3.7.2 One-port

OnePort(measured, ideals, *args, **kwargs)	Standard algorithm for a one-port calibration.
SDDL(measured, ideals, *args, **kwargs)	Short-Delay-Delay-Load (Oneport Calibration), formulation by Arsenovic et, al.
SDDLWeikle(measured, ideals, *args, **kwargs)	Short-Delay-Delay-Load (Oneport Calibration), Liu-Weikle formulation.
PHN(measured, ideals, *args, **kwargs)	Pair of Half Knowns (Oneport Calibration).

skrf.calibration.calibration.OnePort

class skrf.calibration.calibration.**OnePort**(*measured*, *ideals*, **args*, ***kwargs*)

Standard algorithm for a one-port calibration.

The measured reflection coefficient of each standard relates to its definition through the bilinear (Moebius) transform of the error box[1],

$$\Gamma_{\text{meas}}^{(k)} = \frac{t_{11}\Gamma_{\text{def}}^{(k)} + t_{12}}{t_{21}\Gamma_{\text{def}}^{(k)} + 1}$$

where k is the index of the standard and the t's are the T-parameters of the error box, normalized to its last element. They relate to the classical error terms as[2]

$$\begin{bmatrix} t_{11} & t_{12} \\ t_{21} & 1 \end{bmatrix} = \begin{bmatrix} E_{\text{R}} - E_{\text{D}}E_{\text{S}} & E_{\text{D}} \\ -E_{\text{S}} & 1 \end{bmatrix}$$

with E_{D} the directivity, E_{S} the source match and E_{R} the reflection tracking, also sometimes written as `e00`, `e11` and `e01e10`, respectively.

Reordering the equation makes the problem linear in the T-parameters, so that K standards form the system $\mathbf{Ax} = \mathbf{b}$, as follows:

$$\begin{bmatrix} \Gamma_{\text{def}}^{(1)} & 1 & -\Gamma_{\text{def}}^{(1)}\Gamma_{\text{meas}}^{(1)} \\ \vdots & \vdots & \vdots \\ \Gamma_{\text{def}}^{(K)} & 1 & -\Gamma_{\text{def}}^{(K)}\Gamma_{\text{meas}}^{(K)} \end{bmatrix} \begin{bmatrix} t_{11} \\ t_{12} \\ t_{21} \end{bmatrix} = \begin{bmatrix} \Gamma_{\text{meas}}^{(1)} \\ \vdots \\ \Gamma_{\text{meas}}^{(K)} \end{bmatrix}$$

Three standards give a unique solution. With more standards the system is overdetermined and is solved in a least squares sense.

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*).
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with the *measured* list (or use *sloppy_input*).
- **args** – passed to *Calibration.__init__()*
- **kwargs** – passed to *Calibration.__init__()*

Notes

See *Calibration.__init__()* for details about automatic standards alignment (aka *sloppy_input*).

Examples

```
>>> import skrf as rf
>>> ideals = [rf.Network('ideal_short.s1p'),
...           rf.Network('ideal_open.s1p'),
...           rf.Network('ideal_load.s1p')]
>>> measured = [rf.Network('measured_short.s1p'),
```

(continues on next page)

[1] R. A. Speciale, "Projective Matrix Transformations in Microwave Network Theory," 1981 IEEE MTT-S International Microwave Symposium Digest, Los Angeles, CA, USA, 1981, pp. 510-512, doi: https://doi.org/10.1109/MWSYM.1981.1129979

[2] R. B. Marks, "Formulations of the Basic Vector Network Analyzer Error Model including Switch-Terms," 50th ARFTG Conference Digest, Portland, OR, USA, 1997, pp. 115-126, doi: https://doi.org/10.1109/ARFTG.1997.327265

(continued from previous page)

```
...             rf.Network('measured_open.s1p'),
...             rf.Network('measured_load.s1p')]
>>> cal = rf.OnePort(measured=measured, ideals=ideals)
>>> dut = cal.apply_cal(rf.Network('measured_dut.s1p'))
```

A fourth standard makes the problem overdetermined, and it is then solved by least squares:

```
>>> ideals.append(rf.Network('ideal_offset_short.s1p'))
>>> measured.append(rf.Network('measured_offset_short.s1p'))
>>> cal = rf.OnePort(measured=measured, ideals=ideals)
>>> dut = cal.apply_cal(rf.Network('measured_dut.s1p'))
```

References

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.OnePort.biased_error

property OnePort.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.OnePort.caled_ntwk_sets

property OnePort.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.OnePort.caled_ntwks

property OnePort.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.OnePort.coefs

property OnePort.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.OnePort.coefs_12term

property OnePort.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.OnePort.coefs_12term_ntwks

property OnePort.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.OnePort.coefs_3term

property OnePort.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.OnePort.coefs_3term_ntwks

property OnePort.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.OnePort.coefs_8term

property OnePort.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.OnePort.coefs_8term_ntwks

property OnePort.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.OnePort.coefs_ntwks

property OnePort.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.OnePort.error_ntwk

property OnePort.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.OnePort.family

OnePort.**family** = **'OnePort'**

skrf.calibration.calibration.OnePort.frequency

property OnePort.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.OnePort.normalized_directivity

property OnePort.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.OnePort.nstandards

property OnePort.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.OnePort.output_from_run

property OnePort.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.OnePort.residual_ntwk_sets

property OnePort.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.OnePort.residual_ntwks

property OnePort.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.OnePort.total_error

property OnePort.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.OnePort.unbiased_error

property OnePort.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.OnePort.verify_12term

property OnePort.**verify_12term**

skrf.calibration.calibration.OnePort.verify_12term_ntwk

property OnePort.**verify_12term_ntwk**

Methods

__init__	One Port initializer.
apply_cal	Apply correction to a Network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]

continues on next page

Table 63 – continued from previous page

from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	Run the calibration algorithm.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.OnePort.__init__

OnePort.**__init__**(*measured*, *ideals*, **args*, ***kwargs*)

One Port initializer.

> **See also**
>
> *Calibration.__init__*

skrf.calibration.calibration.OnePort.apply_cal

OnePort.**apply_cal**(*ntwk*)

Apply correction to a Network.

skrf.calibration.calibration.OnePort.apply_cal_to_all_in_dir

OnePort.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.OnePort.apply_cal_to_list

OnePort.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.OnePort.apply_cal_to_network_set

OnePort.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.OnePort.embed

OnePort.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.OnePort.from_coefs

classmethod OnePort.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.OnePort.from_coefs_ntwks

classmethod OnePort.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.OnePort.plot_caled_ntwks

OnePort.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.OnePort.plot_calibration_errors

OnePort.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.OnePort.plot_residuals

OnePort.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.OnePort.pop

OnePort.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.OnePort.remove_and_cal

OnePort.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.OnePort.run

OnePort.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.OnePort.update_coefs

OnePort.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.OnePort.write

OnePort.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.SDDL

class skrf.calibration.calibration.**SDDL**(*measured*, *ideals*, **args*, ***kwargs*)

Short-Delay-Delay-Load (Oneport Calibration), formulation by Arsenovic et, al.

One-port self-calibration, which contains a known short, a known arbitrary-impedance load, and two partially-unknown delay shorts. The short can be an offset short with known phase, or a zero-length (flush) short. The load can be perfectly matched or an arbitrary impedance standard, as long as it's known exactly. This is especially useful when an ideal load is impractical to fabricate. The delay shorts have unity reflection magnitudes, but unknown phases (unknown electrical lengths).

See *__init__()* for the correct parameters order.

This class uses the Arsenovic et, al. formulation[1], which allows either a mismatched or a matched load as the calibration standard. It's a generalization of both Sigg-Simon's original SDDM[2] and Liu-Weikle's modified SDDL methods[3], which are limited to matching or reflective loads only. The latter was originally designed for waveguide calibration with resistance to flange misalignment.

[1] A. Arsenovic, R. M. Weikle, and J. L. Hesler, "Reflectometer calibration with a pair of partially known standards," European Microwave Conference, pp. 327–330, Dec. 2015, doi: 10.1109/EUMC.2015.7345766.

[2] Z. Liu and R. M. Weikle, "A reflectometer calibration method resistant to waveguide flange misalignment," Microwave Theory and Techniques, IEEE Transactions on, vol. 54, no. 6, pp. 2447-2452, Jun. 2006.

[3] W. Sigg and J. Simon, "Reflectometer calibration using load, short and offset shorts with unknown phase," Electronics Letters, vol. 27, no. 18, pp. 1650–1651, 1991.

Note

This algorithm is bandwidth-limited due to phase wrapping, conceptually similar to TRL's bandwidth limit. Some over-determined multi-line variants allow wideband calibration, but this is not implemented in scikit-rf. If a wideband calibration is required (common for coaxial systems), it's necessary to manually split measurements to multiple subbands and calibrate each band separately.

Band edges may suffer from increased errors. It's recommended to test line lengths experimentally via simulations first. Choosing an optimal selection of line lengths that covers the widest bandwidth is a non-trivial optimization problem, with several proposed solutions[4].

References

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	

continues on next page

[4] A. Lewandowski, W. Wiatr, L. J. Opalski and R. Biedrzycki, "Accuracy and Bandwidth Optimization of the Over-Determined Offset-Short Reflectometer Calibration," in IEEE Transactions on Microwave Theory and Techniques, vol. 63, no. 3, pp. 1076-1089, March 2015, doi: 10.1109/TMTT.2015.2396496.

Table 64 – continued from previous page

verify_12term_ntwk

skrf.calibration.calibration.SDDL.biased_error

property SDDL.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SDDL.caled_ntwk_sets

property SDDL.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.SDDL.caled_ntwks

property SDDL.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.SDDL.coefs

property SDDL.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

> **See also**
>
> *coefs_3term*, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.SDDL.coefs_12term

property SDDL.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.SDDL.coefs_12term_ntwks

property SDDL.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.SDDL.coefs_3term

property SDDL.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.SDDL.coefs_3term_ntwks

property SDDL.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.SDDL.coefs_8term

property SDDL.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.SDDL.coefs_8term_ntwks

property SDDL.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.SDDL.coefs_ntwks

property SDDL.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.SDDL.error_ntwk

property SDDL.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.SDDL.family

SDDL.**family = 'SDDL'**

skrf.calibration.calibration.SDDL.frequency

property SDDL.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.SDDL.normalized_directivity

property SDDL.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.SDDL.nstandards

property SDDL.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.SDDL.output_from_run

property SDDL.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.SDDL.residual_ntwk_sets

property SDDL.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.SDDL.residual_ntwks

property SDDL.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.SDDL.total_error

property SDDL.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SDDL.unbiased_error

property SDDL.**unbiased_error**: *NetworkSet*

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SDDL.verify_12term

property SDDL.**verify_12term**

skrf.calibration.calibration.SDDL.verify_12term_ntwk

property SDDL.**verify_12term_ntwk**

Methods

__init__	Short-Delay-Delay-Load initializer.
apply_cal	Apply correction to a Network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	Run the calibration algorithm.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.SDDL.__init__

SDDL.**__init__**(*measured*, *ideals*, **args*, ***kwargs*)

Short-Delay-Delay-Load initializer.

Measured and ideal networks must be in the order:

[Short, Delay short1, Delay short2, Load]

The ideal delay shorts can be set to *None*, as they are determined during the calibration.

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use sloppy_input
- **args** – passed to func:*Calibration.__init__*
- **kwargs** – passed to func:*Calibration.__init__*

See also

Calibration.__init__

skrf.calibration.calibration.SDDL.apply_cal

SDDL.**apply_cal**(*ntwk*)

Apply correction to a Network.

skrf.calibration.calibration.SDDL.apply_cal_to_all_in_dir

SDDL.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.SDDL.apply_cal_to_list

SDDL.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.SDDL.apply_cal_to_network_set

SDDL.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.SDDL.embed

SDDL.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.SDDL.from_coefs

classmethod SDDL.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.SDDL.from_coefs_ntwks

classmethod SDDL.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

> **See also**
>
> *Calibration.from_coefs*

skrf.calibration.calibration.SDDL.plot_caled_ntwks

SDDL.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.SDDL.plot_calibration_errors

SDDL.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.SDDL.plot_residuals

SDDL.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

> **See also**
>
> Calibration.residual_networks

skrf.calibration.calibration.SDDL.pop

SDDL.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.SDDL.remove_and_cal

SDDL.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.SDDL.run

SDDL.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.SDDL.update_coefs

SDDL.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.SDDL.write

SDDL.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.SDDLWeikle

class skrf.calibration.calibration.**SDDLWeikle**(*measured*, *ideals*, **args*, ***kwargs*)

Short-Delay-Delay-Load (Oneport Calibration), Liu-Weikle formulation.

One-port self-calibration, which contains a known short, a known mismatched load, and two partially-unknown delay shorts. The short can be an offset short with known phase, or a zero-length (flush) short. The load can have an arbitrary impedance, as long as it's reflective (not perfectly matched) and known exactly. This is especially useful when an ideal load is impractical to fabricate. The delay shorts have unity reflection magnitudes, but unknown phases (unknown electrical lengths).

See *__init__()* for the correct parameters order.

This class uses the Liu-Weikle formulation, which requires a mismatched load (i.e. non-zero reflection coefficient). Calibration fails if the load is perfectly matched. In the paper, a fully-known radiating open is used as the load standard.

It was originally designed for waveguide calibration with resistance to flange misalignment by Liu-Weikle[1], which was itself a modification of the earlier Sigg-Simon SDDM calibration[2]. In Sigg-Simon's SDDM, the load must be matched, while Liu-Weikle SDDL only allows a mismatched load.

Note

This algorithm is bandwidth-limited due to phase wrapping, conceptually similar to TRL's bandwidth limit. Some over-determined multi-line variants allow wideband calibration, but this is not implemented in scikit-rf. If a wideband calibration is required (common for coaxial systems), it's necessary to manually split measurements to multiple subbands and calibrate each band separately.

Band edges may suffer from increased errors. It's recommended to test line lengths experimentally via simulations first. Choosing an optimal selection of line lengths that covers the widest bandwidth is a non-trivial optimization problem, with several proposed solutions[3].

References

Attributes

[1] Z. Liu and R. M. Weikle, "A reflectometer calibration method resistant to waveguide flange misalignment," Microwave Theory and Techniques, IEEE Transactions on, vol. 54, no. 6, pp. 2447-2452, Jun. 2006.

[2] W. Sigg and J. Simon, "Reflectometer calibration using load, short and offset shorts with unknown phase," Electronics Letters, vol. 27, no. 18, pp. 1650–1651, 1991.

[3] A. Lewandowski, W. Wiatr, L. J. Opalski and R. Biedrzycki, "Accuracy and Bandwidth Optimization of the Over-Determined Offset-Short Reflectometer Calibration," in IEEE Transactions on Microwave Theory and Techniques, vol. 63, no. 3, pp. 1076-1089, March 2015, doi: 10.1109/TMTT.2015.2396496.

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.SDDLWeikle.biased_error

property SDDLWeikle.**biased_error**: *NetworkSet*

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SDDLWeikle.caled_ntwk_sets

property SDDLWeikle.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.SDDLWeikle.caled_ntwks

property SDDLWeikle.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.SDDLWeikle.coefs

property SDDLWeikle.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.SDDLWeikle.coefs_12term

property SDDLWeikle.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity

- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.SDDLWeikle.coefs_12term_ntwks

property SDDLWeikle.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.SDDLWeikle.coefs_3term

property SDDLWeikle.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.SDDLWeikle.coefs_3term_ntwks

property SDDLWeikle.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.SDDLWeikle.coefs_8term

property SDDLWeikle.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.SDDLWeikle.coefs_8term_ntwks

property SDDLWeikle.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.SDDLWeikle.coefs_ntwks

property SDDLWeikle.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.SDDLWeikle.error_ntwk

property SDDLWeikle.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.SDDLWeikle.family

SDDLWeikle.**family = 'SDDL'**

skrf.calibration.calibration.SDDLWeikle.frequency

property SDDLWeikle.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.SDDLWeikle.normalized_directivity

property SDDLWeikle.**normalized_directivity**

Directivity normalized to the reflection tracking.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.SDDLWeikle.nstandards

property SDDLWeikle.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.SDDLWeikle.output_from_run

property SDDLWeikle.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.SDDLWeikle.residual_ntwk_sets

property SDDLWeikle.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.SDDLWeikle.residual_ntwks

property SDDLWeikle.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.SDDLWeikle.total_error

property SDDLWeikle.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SDDLWeikle.unbiased_error

property SDDLWeikle.**unbiased_error**: *NetworkSet*

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SDDLWeikle.verify_12term

property SDDLWeikle.**verify_12term**

skrf.calibration.calibration.SDDLWeikle.verify_12term_ntwk

property SDDLWeikle.**verify_12term_ntwk**

Methods

__init__	Short-Delay-Delay-Load initializer.
apply_cal	Apply correction to a Network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	Run the calibration algorithm.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.SDDLWeikle.__init__

SDDLWeikle.**__init__**(*measured*, *ideals*, **args*, ***kwargs*)

Short-Delay-Delay-Load initializer.

Measured and ideal networks must be in the order:

- short
- delay short1
- delay short2
- load

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use sloppy_input
- **args** – passed to func:*Calibration.__init__*
- **kwargs** – passed to func:*Calibration.__init__*

See also

Calibration.__init__

skrf.calibration.calibration.SDDLWeikle.apply_cal

SDDLWeikle.**apply_cal**(*ntwk*)

Apply correction to a Network.

skrf.calibration.calibration.SDDLWeikle.apply_cal_to_all_in_dir

SDDLWeikle.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.SDDLWeikle.apply_cal_to_list

SDDLWeikle.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.SDDLWeikle.apply_cal_to_network_set

SDDLWeikle.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.SDDLWeikle.embed

SDDLWeikle.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.SDDLWeikle.from_coefs

classmethod SDDLWeikle.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.SDDLWeikle.from_coefs_ntwks

classmethod SDDLWeikle.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.SDDLWeikle.plot_caled_ntwks

SDDLWeikle.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.SDDLWeikle.plot_calibration_errors

SDDLWeikle.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters
ax (*Axes*)

skrf.calibration.calibration.SDDLWeikle.plot_residuals

SDDLWeikle.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.SDDLWeikle.pop

SDDLWeikle.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.SDDLWeikle.remove_and_cal

SDDLWeikle.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.SDDLWeikle.run

SDDLWeikle.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.SDDLWeikle.update_coefs

SDDLWeikle.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.SDDLWeikle.write

SDDLWeikle.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.PHN

class skrf.calibration.calibration.**PHN**(*measured*, *ideals*, **args*, ***kwargs*)

Pair of Half Knowns (Oneport Calibration).

One-port self-calibration, which contains two fully-known arbitrary impedance standards, and two partially-known standards (such as open or short circuits with unknown phase). See *__init__()* for the correct parameters order.

This is a generalization of the SDDL calibration algorithm, see[1].

[1] A. Arsenovic, R. M. Weikle, and J. L. Hesler, "Reflectometer calibration with a pair of partially known standards," European Microwave Conference, pp. 327–330, Dec. 2015, doi: 10.1109/EUMC.2015.7345766.

Important

Unfortunately, due to a square-root sign ambiguity in the solution of a quadratic equation, this algorithm can be unstable. While it has proven to be reliable in rectangular waveguide calibration scenarios, it often fails for arbitrary-impedance standards with random reflection coefficients. This is a major drawback in the current solution, but the authors believe this problem may be overcome analytically by future research.

References

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.PHN.biased_error

property PHN.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.PHN.caled_ntwk_sets

property PHN.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.PHN.caled_ntwks

property PHN.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.PHN.coefs

property PHN.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.PHN.coefs_12term

property PHN.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.PHN.coefs_12term_ntwks

property PHN.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.PHN.coefs_3term

property PHN.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.PHN.coefs_3term_ntwks

property PHN.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.PHN.coefs_8term

property PHN.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.PHN.coefs_8term_ntwks

property PHN.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.PHN.coefs_ntwks

property PHN.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.PHN.error_ntwk

property PHN.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.PHN.family

PHN.**family** = **'PHN'**

skrf.calibration.calibration.PHN.frequency

property PHN.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.PHN.normalized_directivity

property PHN.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.PHN.nstandards

property PHN.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.PHN.output_from_run

property PHN.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.PHN.residual_ntwk_sets

property PHN.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.PHN.residual_ntwks

property PHN.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.PHN.total_error

property PHN.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.PHN.unbiased_error

property PHN.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.PHN.verify_12term

property PHN.**verify_12term**

skrf.calibration.calibration.PHN.verify_12term_ntwk

property PHN.**verify_12term_ntwk**

Methods

__init__	Half-Half-Full-Full initializer.
apply_cal	Apply correction to a Network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]

continues on next page

Table 69 – continued from previous page

from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	Run the calibration algorithm.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.PHN.__init__

PHN.**__init__**(*measured*, *ideals*, **args*, ***kwargs*)

Half-Half-Full-Full initializer.

Measured and ideal networks must be in the order:

[Half-Known 1, Half-Known 2, Fully-Known 1, Fully-Known 2]

The phases of half-known standards can be set to an approximate value, as they are determined during the calibration.

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter.
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list.
- **args** – passed to func:*Calibration.__init__*
- **kwargs** – passed to func:*Calibration.__init__*

See also

Calibration.__init__

skrf.calibration.calibration.PHN.apply_cal

PHN.**apply_cal**(*ntwk*)

Apply correction to a Network.

skrf.calibration.calibration.PHN.apply_cal_to_all_in_dir

PHN.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.PHN.apply_cal_to_list

PHN.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.PHN.apply_cal_to_network_set

PHN.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.PHN.embed

PHN.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.PHN.from_coefs

classmethod PHN.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.PHN.from_coefs_ntwks

classmethod PHN.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.PHN.plot_caled_ntwks

PHN.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.PHN.plot_calibration_errors

PHN.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.PHN.plot_residuals

PHN.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

> **See also**
>
> Calibration.residual_networks

skrf.calibration.calibration.PHN.pop

PHN.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.PHN.remove_and_cal

PHN.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.PHN.run

PHN.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.PHN.update_coefs

PHN.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.PHN.write

PHN.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

3.7.3 Two-port

TwelveTerm(measured, ideals[, n_thrus, ...])	12-term, full two-port calibration.
SOLT(measured, ideals[, n_thrus])	Short-Open-Load-Thru, Full two-port calibration.
EightTerm(measured, ideals[, switch_terms, ...])	General EightTerm (aka Error-box) Two-port calibration.
UnknownThru(measured, ideals, *args, **kwargs)	Two-Port Self-Calibration allowing the *thru* standard to be unknown.
LRM(measured, ideals[, switch_terms, isolation])	Line-Reflect-Match self-calibration.

continues on next page

Table 70 – continued from previous page

LRRM(measured, ideals[, switch_terms, ...])	Line-Reflect-Reflect-Match self-calibration.
MRC(measured, ideals, *args, **kwargs)	Misalignment Resistance Calibration.
TRL(measured[, ideals, estimate_line, ...])	Thru-Reflect-Line (also called LRL for Line-Reflect-Line).
MultilineTRL	
NISTMultilineTRL(measured, Grefls, l[, ...])	NIST Multiline TRL calibration.
TUGMultilineTRL(line_meas, line_lengths[, ...])	TUG Multiline TRL calibration.
SixteenTerm(measured, ideals[, switch_terms])	General SixteenTerm (aka Error-box) Two-port calibration.
LMR16(measured, ideals[, ideal_is_reflect, ...])	SixteenTerm Load-Match-Reflect self-calibration.
Normalization(measured, ideals[, ...])	Simple Thru Normalization.

skrf.calibration.calibration.TwelveTerm

class skrf.calibration.calibration.**TwelveTerm**(*measured*, *ideals*, *n_thrus=None*, *trans_thres=-40*, **args*, ***kwargs*)

12-term, full two-port calibration.

TwelveTerm is the traditional, fully determined, two-port calibration originally developed in[1].

TwelveTerm can accept any number of reflect and transmissive standards, as well as arbitrary (non-flush) transmissive standards.

- **If more than 3 reflect standards are provided, a least-squares**
  solution is implemented for the one-port stage of the calibration.
- **If more than 1 transmissive standard is given the *load match*,**
  and *transmission tracking* terms are calculated multiple times and averaged.

References

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.

continues on next page

[1] "Calibration Process of Automatic Network Analyzer Systems" by Stig Rehnmark

Table 71 – continued from previous page

error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.TwelveTerm.biased_error

property TwelveTerm.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TwelveTerm.caled_ntwk_sets

property TwelveTerm.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.TwelveTerm.caled_ntwks

property TwelveTerm.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.TwelveTerm.coefs

property TwelveTerm.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.TwelveTerm.coefs_12term

property TwelveTerm.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

References

skrf.calibration.calibration.TwelveTerm.coefs_12term_ntwks

property TwelveTerm.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.TwelveTerm.coefs_3term

property TwelveTerm.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.TwelveTerm.coefs_3term_ntwks

property TwelveTerm.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.TwelveTerm.coefs_8term

property TwelveTerm.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

References

skrf.calibration.calibration.TwelveTerm.coefs_8term_ntwks

property TwelveTerm.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.TwelveTerm.coefs_ntwks

property TwelveTerm.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.TwelveTerm.error_ntwk

property TwelveTerm.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.TwelveTerm.family

TwelveTerm.**family = 'TwelveTerm'**

skrf.calibration.calibration.TwelveTerm.frequency

property TwelveTerm.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.TwelveTerm.normalized_directivity

property TwelveTerm.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.TwelveTerm.nstandards

property TwelveTerm.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.TwelveTerm.output_from_run

property TwelveTerm.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.TwelveTerm.residual_ntwk_sets

property TwelveTerm.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.TwelveTerm.residual_ntwks

property TwelveTerm.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.TwelveTerm.total_error

property TwelveTerm.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TwelveTerm.unbiased_error

property TwelveTerm.**unbiased_error**: *NetworkSet*

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

skrf.calibration.calibration.TwelveTerm.verify_12term

property TwelveTerm.**verify_12term**

skrf.calibration.calibration.TwelveTerm.verify_12term_ntwk

property TwelveTerm.**verify_12term_ntwk**

Methods

__init__	TwelveTerm initializer.
apply_cal	
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.TwelveTerm.__init__

TwelveTerm.**__init__**(*measured*, *ideals*, *n_thrus=None*, *trans_thres=-40*, **args*, ***kwargs*)

TwelveTerm initializer.

Use the *n_thrus* argument to explicitly define the number of transmissive standards. Otherwise, if *n_thrus=None*, then we will try and guess which are transmissive, by comparing the mean $|s21|$ and $|s12|$ responses (in dB) to *trans_thres*.

Notes

See func:*Calibration.__init__* for details about automatic standards alignment (aka *sloppy_input*).

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)

- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use sloppy_input
- **n_thrus** (*int*) – Number of transmissive standards. If None, we will try and guess for you by comparing measure transmission to trans_thres,
- **trans_thres** (*float*) – The minimum transmission magnitude (in dB) that is the threshold for categorizing a transmissive standard. Compared to the measured s21,s12 averaged over frequency Only use if n_thrus=None.
- **isolation** (*Network* object) – Measurement with loads on both ports with a perfect isolation between the ports. Used for determining the isolation error terms. If no measurement is given leakage is assumed to be zero.

  Loads don't need to be same as the one used as a match standard or even have similar reflection coefficients. Reflects can be also used, but accuracy might not be as good.

See also

Calibration.__init__

skrf.calibration.calibration.TwelveTerm.apply_cal

TwelveTerm.**apply_cal**(*ntwk*)

skrf.calibration.calibration.TwelveTerm.apply_cal_to_all_in_dir

TwelveTerm.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.TwelveTerm.apply_cal_to_list

TwelveTerm.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.TwelveTerm.apply_cal_to_network_set

TwelveTerm.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.TwelveTerm.embed

TwelveTerm.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.TwelveTerm.from_coefs

classmethod TwelveTerm.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)

- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.TwelveTerm.from_coefs_ntwks

classmethod TwelveTerm.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.TwelveTerm.plot_caled_ntwks

TwelveTerm.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.TwelveTerm.plot_calibration_errors

TwelveTerm.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.TwelveTerm.plot_residuals

TwelveTerm.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.TwelveTerm.pop

`TwelveTerm.pop(std=-1)`

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.TwelveTerm.remove_and_cal

`TwelveTerm.remove_and_cal(std)`

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.TwelveTerm.run

`TwelveTerm.run()`

skrf.calibration.calibration.TwelveTerm.update_coefs

`TwelveTerm.update_coefs(d)`

Update current dict of error coefficients.

skrf.calibration.calibration.TwelveTerm.write

`TwelveTerm.write(file=None, *args, **kwargs)`

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.SOLT

class skrf.calibration.calibration.**SOLT**(*measured*, *ideals*, *n_thrus=1*, **args*, ***kwargs*)

Short-Open-Load-Thru, Full two-port calibration.

SOLT is the traditional, fully determined, two-port calibration originally developed in[1]. Although the acronym SOLT implies the use of 4 standards, skrf's algorithm can accept any number of reflect standards, If more than 3 reflect standards are provided a least-squares solution is implemented for the one-port stage of the calibration.

Redundant thru measurements can also be used, through the *n_thrus* parameter. See *__init__()*

References

See also

TwelveTerm

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.

continues on next page

[1] W. Kruppa and K. F. Sodomsky, "An Explicit Solution for the Scattering Parameters of a Linear Two-Port Measured with an Imperfect Test Set (Correspondence)," IEEE Transactions on Microwave Theory and Techniques, vol. 19, no. 1, pp. 122-123, Jan. 1971.

Table 73 – continued from previous page

coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.SOLT.biased_error

property SOLT.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SOLT.caled_ntwk_sets

property SOLT.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.SOLT.caled_ntwks

property SOLT.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.SOLT.coefs

property SOLT.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.SOLT.coefs_12term

property SOLT.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.SOLT.coefs_12term_ntwks

property SOLT.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.SOLT.coefs_3term

property SOLT.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.SOLT.coefs_3term_ntwks

property SOLT.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.SOLT.coefs_8term

property SOLT.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.SOLT.coefs_8term_ntwks

property SOLT.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.SOLT.coefs_ntwks

property SOLT.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.SOLT.error_ntwk

property SOLT.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.SOLT.family

SOLT.**family = 'SOLT'**

skrf.calibration.calibration.SOLT.frequency

property SOLT.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.SOLT.normalized_directivity

property SOLT.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.SOLT.nstandards

property SOLT.**nstandards**

Number of ideal/measurement pairs in calibration.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.SOLT.output_from_run

property SOLT.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.SOLT.residual_ntwk_sets

property SOLT.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.SOLT.residual_ntwks

property SOLT.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.SOLT.total_error

property SOLT.**total_error: *NetworkSet***

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SOLT.unbiased_error

property SOLT.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SOLT.verify_12term

property SOLT.**verify_12term**

skrf.calibration.calibration.SOLT.verify_12term_ntwk

property SOLT.**verify_12term_ntwk**

Methods

__init__	SOLT initializer.
apply_cal	
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.SOLT.__init__

SOLT.**__init__**(*measured*, *ideals*, *n_thrus=1*, **args*, ***kwargs*)

SOLT initializer.

If you aren't using *sloppy_input*, then the order of the standards must align.

If *n_thrus!=None*, then the thru standard[s] must be last in the list. The *n_thrus* argument can be used to allow multiple measurements of the thru standard.

If the ideal element for the thru is set to None, a flush thru is assumed.

Notes

See func:*Calibration.__init__* for details about automatic standards alignment (aka *sloppy_input*).

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use *sloppy_input*) The thru standard can be None
- **n_thrus** (*int*) – number of thru measurements
- **isolation** (*Network* object) – Measurement with loads on both ports. Used for determining the isolation error terms. If no measurement is given isolation is assumed to be zero.

See also

TwelveTerm.__init__

skrf.calibration.calibration.SOLT.apply_cal

SOLT.**apply_cal**(*ntwk*)

skrf.calibration.calibration.SOLT.apply_cal_to_all_in_dir

SOLT.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.SOLT.apply_cal_to_list

SOLT.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.SOLT.apply_cal_to_network_set

SOLT.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.SOLT.embed

SOLT.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.SOLT.from_coefs

classmethod SOLT.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

> **See also**
>
> *Calibration.from_coefs_ntwks*

skrf.calibration.calibration.SOLT.from_coefs_ntwks

classmethod SOLT.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

> **See also**
>
> *Calibration.from_coefs*

skrf.calibration.calibration.SOLT.plot_caled_ntwks

SOLT.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.SOLT.plot_calibration_errors

SOLT.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.SOLT.plot_residuals

SOLT.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'

- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.SOLT.pop

SOLT.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.SOLT.remove_and_cal

SOLT.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.SOLT.run

SOLT.**run**()

skrf.calibration.calibration.SOLT.update_coefs

SOLT.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.SOLT.write

SOLT.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.

- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.EightTerm

class skrf.calibration.calibration.**EightTerm**(*measured*, *ideals*, *switch_terms=None*, *isolation=None*, *ut_hook=None*, **args*, ***kwargs*)

General EightTerm (aka Error-box) Two-port calibration.

This is basically an extension of the one-port algorithm to two-port measurements, A least squares estimator is used to determine the error coefficients. No self-calibration takes place. The concept is presented in[1] , but implementation follows that of[2] .

See *__init__()*

Notes

An important detail of implementing the error-box model is that the internal switch must be correctly accounted for. This is done through the measurement of *switch terms*.

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.

continues on next page

[1] Speciale, R.A.; , "A Generalization of the TSD Network-Analyzer Calibration Procedure, Covering n-Port Scattering-Parameter Measurements, Affected by Leakage Errors," Microwave Theory and Techniques, IEEE Transactions on, vol.25, no.12, pp. 1100- 1115, Dec 1977. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1129282&isnumber=25047

[2] Rytting, D. (1996) Network Analyzer Error Models and Calibration Methods. RF 8: Microwave Measurements for Wireless Applications (ARFTG/NIST Short Course Notes)

Table 75 – continued from previous page

coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unisolated	
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.EightTerm.T_matrices

property EightTerm.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.EightTerm.biased_error

property EightTerm.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.EightTerm.caled_ntwk_sets

property EightTerm.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.EightTerm.caled_ntwks

property EightTerm.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.EightTerm.coefs

property EightTerm.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.EightTerm.coefs_12term

property EightTerm.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match

- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.EightTerm.coefs_12term_ntwks

property EightTerm.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.EightTerm.coefs_3term

property EightTerm.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.EightTerm.coefs_3term_ntwks

property EightTerm.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.EightTerm.coefs_8term

property EightTerm.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.EightTerm.coefs_8term_ntwks

property EightTerm.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.EightTerm.coefs_ntwks

property EightTerm.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.EightTerm.error_ntwk

property EightTerm.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.EightTerm.family

EightTerm.**family = 'EightTerm'**

skrf.calibration.calibration.EightTerm.frequency

property EightTerm.**frequency**

Frequency object of the calibration.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.EightTerm.measured_unisolated

property EightTerm.**measured_unisolated**

skrf.calibration.calibration.EightTerm.measured_unterminated

property EightTerm.**measured_unterminated**

skrf.calibration.calibration.EightTerm.normalized_directivity

property EightTerm.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.EightTerm.nstandards

property EightTerm.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.EightTerm.output_from_run

property EightTerm.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.EightTerm.residual_ntwk_sets

property EightTerm.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.EightTerm.residual_ntwks

property EightTerm.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.EightTerm.total_error

property EightTerm.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns

total_error – Network with s_mag is proportional to the total error

Return type

skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.EightTerm.unbiased_error

property EightTerm.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.EightTerm.verify_12term

property EightTerm.**verify_12term**

skrf.calibration.calibration.EightTerm.verify_12term_ntwk

property EightTerm.**verify_12term_ntwk**

Methods

__init__	EightTerm Initializer.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.

continues on next page

Table 76 – continued from previous page

from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.EightTerm.__init__

EightTerm.**__init__**(*measured*, *ideals*, *switch_terms=None*, *isolation=None*, *ut_hook=None*, **args*, ***kwargs*)

EightTerm Initializer.

Notes

See func:*Calibration.__init__* for details about automatic standards alignment (aka *sloppy_input*).

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list
- **switch_terms** (tuple of *Network* objects) – the pair of switch terms in the order (forward, reverse)
- **isolation** (*Network* object) – Measurement with loads on both ports with a perfect isolation between the ports. Used for determining the isolation error terms. If no measurement is given leakage is assumed to be zero.

  Loads don't need to be same as the one used as a match standard or even have similar reflection coefficients. Reflects can be also used, but accuracy might not be as good.

skrf.calibration.calibration.EightTerm.apply_cal

EightTerm.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters
ntwk (*Network*) – Uncalibrated input network.

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.EightTerm.apply_cal_to_all_in_dir

EightTerm.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.EightTerm.apply_cal_to_list

EightTerm.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.EightTerm.apply_cal_to_network_set

EightTerm.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.EightTerm.embed

EightTerm.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters
ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.EightTerm.from_coefs

classmethod EightTerm.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.EightTerm.from_coefs_ntwks

classmethod EightTerm.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

> **See also**
>
> *Calibration.from_coefs*

skrf.calibration.calibration.EightTerm.plot_caled_ntwks

EightTerm.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.EightTerm.plot_calibration_errors

EightTerm.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.EightTerm.plot_residuals

EightTerm.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

> **See also**
>
> Calibration.residual_networks

skrf.calibration.calibration.EightTerm.pop

EightTerm.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.EightTerm.remove_and_cal

EightTerm.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.EightTerm.renormalize

EightTerm.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.EightTerm.run

EightTerm.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.EightTerm.terminate

EightTerm.**terminate**(*ntwk*)

Terminate a network with switch terms.

> **See also**
>
> calibration.terminate

skrf.calibration.calibration.EightTerm.unterminate

EightTerm.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

> **See also**
>
> calibration.unterminate

skrf.calibration.calibration.EightTerm.update_coefs

EightTerm.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.EightTerm.write

EightTerm.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

> **See also**
>
> *skrf.io.general.write*, *skrf.io.general.read*

skrf.calibration.calibration.UnknownThru

class skrf.calibration.calibration.**UnknownThru**(*measured*, *ideals*, **args*, ***kwargs*)

Two-Port Self-Calibration allowing the *thru* standard to be unknown.

This algorithm was originally developed in[1], and is based on the 8-term error model (*EightTerm*). It allows the *thru* to be unknown, other than it must be reciprocal. This is useful when when a well-known thru is not realizable.

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unisolated	
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

[1] A. Ferrero and U. Pisani, "Two-port network analyzer calibration using an unknown *thru*," IEEE Microwave and Guided Wave Letters, vol. 2, no. 12, pp. 505-507, 1992.

skrf.calibration.calibration.UnknownThru.T_matrices

property UnknownThru.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.UnknownThru.biased_error

property UnknownThru.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.UnknownThru.caled_ntwk_sets

property UnknownThru.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.UnknownThru.caled_ntwks

property UnknownThru.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.UnknownThru.coefs

property UnknownThru.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.UnknownThru.coefs_12term

property UnknownThru.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.UnknownThru.coefs_12term_ntwks

property UnknownThru.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.UnknownThru.coefs_3term

property UnknownThru.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

- source match
- reflection tracking'

skrf.calibration.calibration.UnknownThru.coefs_3term_ntwks

property UnknownThru.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.UnknownThru.coefs_8term

property UnknownThru.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.UnknownThru.coefs_8term_ntwks

property UnknownThru.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.UnknownThru.coefs_ntwks

property UnknownThru.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.UnknownThru.error_ntwk

property UnknownThru.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.UnknownThru.family

UnknownThru.**family = 'UnknownThru'**

skrf.calibration.calibration.UnknownThru.frequency

property UnknownThru.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.UnknownThru.measured_unisolated

property UnknownThru.**measured_unisolated**

skrf.calibration.calibration.UnknownThru.measured_unterminated

property UnknownThru.**measured_unterminated**

skrf.calibration.calibration.UnknownThru.normalized_directivity

property UnknownThru.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.UnknownThru.nstandards

property UnknownThru.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.UnknownThru.output_from_run

property UnknownThru.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.UnknownThru.residual_ntwk_sets

property UnknownThru.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.UnknownThru.residual_ntwks

property UnknownThru.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.UnknownThru.total_error

property UnknownThru.**total_error: *NetworkSet***

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.UnknownThru.unbiased_error

property UnknownThru.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.UnknownThru.verify_12term

property UnknownThru.**verify_12term**

skrf.calibration.calibration.UnknownThru.verify_12term_ntwk

property UnknownThru.**verify_12term_ntwk**

Methods

__init__	UnknownThru Initializer.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.UnknownThru.__init__

UnknownThru.**__init__**(*measured*, *ideals*, **args*, ***kwargs*)

UnknownThru Initializer.

Note that the *thru* standard must be last in both measured, and ideal lists. The ideal for the *thru* is only used to choose the sign of a square root. Thus, it only has to be have s21, s12 known within π phase.

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use *sloppy_input*)
- **switch_terms** (tuple of *Network* objects) – the pair of switch terms in the order (forward, reverse)

skrf.calibration.calibration.UnknownThru.apply_cal

UnknownThru.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters
ntwk (*Network*) – Uncalibrated input network.

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.UnknownThru.apply_cal_to_all_in_dir

UnknownThru.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.UnknownThru.apply_cal_to_list

UnknownThru.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.UnknownThru.apply_cal_to_network_set

UnknownThru.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.UnknownThru.embed

UnknownThru.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters
ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.UnknownThru.from_coefs

classmethod UnknownThru.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

> **See also**
>
> *Calibration.from_coefs_ntwks*

skrf.calibration.calibration.UnknownThru.from_coefs_ntwks

classmethod UnknownThru.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

> **See also**
>
> *Calibration.from_coefs*

skrf.calibration.calibration.UnknownThru.plot_caled_ntwks

UnknownThru.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.UnknownThru.plot_calibration_errors

UnknownThru.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.UnknownThru.plot_residuals

UnknownThru.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'

- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.UnknownThru.pop

UnknownThru.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.UnknownThru.remove_and_cal

UnknownThru.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.UnknownThru.renormalize

UnknownThru.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.UnknownThru.run

UnknownThru.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.UnknownThru.terminate

UnknownThru.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.UnknownThru.unterminate

UnknownThru.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

See also

calibration.unterminate

skrf.calibration.calibration.UnknownThru.update_coefs

UnknownThru.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.UnknownThru.write

UnknownThru.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.LRM

class skrf.calibration.calibration.**LRM**(*measured*, *ideals*, *switch_terms=None*, *isolation=None*, **args*, ***kwargs*)

Line-Reflect-Match self-calibration.

The required calibration standards are:

- Line: Fully known.
- Reflect: Unknown reflect, phase needs to be known within 90 degrees.
- Match: Fully known.

Reflect and match are assumed to be identical on both ports. The measured and ideals lists must be given in LRM order.

Implementation is based on[1].

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unisolated	
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

continues on next page

[1] Zhao, W.; Liu, S.; Wang, H.; Liu, Y.; Zhang, S.; Cheng, C.; Feng, K.; Ocket, I.; Schreurs, D.; Nauwelaers, B.; Qin, H.; Yang, X. A Unified Approach for Reformulations of LRM/LRMM/LRRM Calibration Algorithms Based on the T-Matrix Representation. Appl. Sci. 2017, 7, 866.

Table 79 – continued from previous page

residual_ntwks	Dictionary of residual Networks.
solved_r	Solved reflect
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.LRM.T_matrices

property LRM.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.LRM.biased_error

property LRM.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LRM.caled_ntwk_sets

property LRM.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.LRM.caled_ntwks

property LRM.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.LRM.coefs

property LRM.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.LRM.coefs_12term

property LRM.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

References

skrf.calibration.calibration.LRM.coefs_12term_ntwks

property LRM.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.LRM.coefs_3term

property LRM.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.LRM.coefs_3term_ntwks

property LRM.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.LRM.coefs_8term

property LRM.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

References

skrf.calibration.calibration.LRM.coefs_8term_ntwks

property LRM.coefs_8term_ntwks

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.LRM.coefs_ntwks

property LRM.coefs_ntwks

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.LRM.error_ntwk

property LRM.error_ntwk

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.LRM.family

LRM.family = 'LRM'

skrf.calibration.calibration.LRM.frequency

property LRM.frequency

Frequency object of the calibration.

skrf.calibration.calibration.LRM.measured_unisolated

property LRM.measured_unisolated

skrf.calibration.calibration.LRM.measured_unterminated

property LRM.measured_unterminated

skrf.calibration.calibration.LRM.normalized_directivity

property LRM.normalized_directivity

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.LRM.nstandards

property LRM.nstandards

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.LRM.output_from_run

property LRM.output_from_run

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.LRM.residual_ntwk_sets

property LRM.residual_ntwk_sets

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.LRM.residual_ntwks

property LRM.residual_ntwks

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.LRM.solved_r

property LRM.solved_r

Solved reflect

skrf.calibration.calibration.LRM.total_error

property LRM.total_error: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LRM.unbiased_error

property LRM.unbiased_error: *NetworkSet*

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LRM.verify_12term

property LRM.**verify_12term**

skrf.calibration.calibration.LRM.verify_12term_ntwk

property LRM.**verify_12term_ntwk**

Methods

__init__	LRM Initializer.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.LRM.__init__

LRM.**__init__**(*measured*, *ideals*, *switch_terms=None*, *isolation=None*, **args*, ***kwargs*)

LRM Initializer.

Parameters

- **measured** (list of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter
- **ideals** (list of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *measured* list
- **switch_terms** (tuple of *Network* objects) – the pair of switch terms in the order (forward, reverse)
- **isolation** (*Network* object) – Measurement with loads on both ports with a perfect isolation between the ports. Used for determining the isolation error terms. If no measurement is given leakage is assumed to be zero.

skrf.calibration.calibration.LRM.apply_cal

LRM.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters

ntwk (*Network*) – Uncalibrated input network.

Returns

caled – Calibrated network.

Return type

Network

skrf.calibration.calibration.LRM.apply_cal_to_all_in_dir

LRM.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.LRM.apply_cal_to_list

LRM.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.LRM.apply_cal_to_network_set

LRM.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.LRM.embed

LRM.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters

ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.LRM.from_coefs

classmethod LRM.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.LRM.from_coefs_ntwks

classmethod LRM.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.LRM.plot_caled_ntwks

LRM.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.LRM.plot_calibration_errors

LRM.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters
ax (*Axes*)

skrf.calibration.calibration.LRM.plot_residuals

LRM.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

Calibration.residual_networks

skrf.calibration.calibration.LRM.pop

LRM.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.LRM.remove_and_cal

LRM.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.LRM.renormalize

LRM.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.LRM.run

LRM.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.LRM.terminate

LRM.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.LRM.unterminate

LRM.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

See also

calibration.unterminate

skrf.calibration.calibration.LRM.update_coefs

LRM.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.LRM.write

LRM.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.LRRM

class skrf.calibration.calibration.**LRRM**(*measured*, *ideals*, *switch_terms=None*, *isolation=None*, *z0=50*, *match_fit='l'*, **args*, ***kwargs*)

Line-Reflect-Reflect-Match self-calibration.

The required calibration standards are:

- Line: Fully known.
- Reflect: Unknown reflect, phase needs to be known within 90 degrees.
- Reflect: Reflect with known absolute value of the reflection coefficient, phase needs to be known within 90 degrees. Different from the other reflect.
- Match: Match with known resistance in series with unknown inductance.

Reflects are assumed to be identical on both ports. Note that the first reflect's magnitude of the reflection coefficient can be unknown, but the second reflect's magnitude of the reflection coefficient needs to be known. Match needs to be only measured on the first port, the second port of match measurement is not used during the calibration.

If match_fit == 'lc' then the second reflect is assumed to be a lossless capacitor. Measurements should then include low frequencies for accurate open capacitance determination. 'lc_fit_c_freq' argument can be given to specify the maximum frequency in Hz where open looks like an ideal capacitor, above this frequency open is assumed to only have known absolute value similar to 'l' fit. Default is infinity.

Implementation is based on papers[1] and[2]. 'lc' match_fit based on[3].

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.

continues on next page

[1] Zhao, W.; Liu, S.; Wang, H.; Liu, Y.; Zhang, S.; Cheng, C.; Feng, K.; Ocket, I.; Schreurs, D.; Nauwelaers, B.; Qin, H.; Yang, X. A Unified Approach for Reformulations of LRM/LRMM/LRRM Calibration Algorithms Based on the T-Matrix Representation. Appl. Sci. 2017, 7, 866.

[2] F. Purroy and L. Pradell, "New theoretical analysis of the LRRM calibration technique for vector network analyzers," in IEEE Transactions on Instrumentation and Measurement, vol. 50, no. 5, pp. 1307-1314, Oct. 2001.

[3] S. Liu, I. Ocket, A. Lewandowski, D. Schreurs and B. Nauwelaers, "An Improved Line-Reflect-Reflect-Match Calibration With an Enhanced Load Model," in IEEE Microwave and Wireless Components Letters, vol. 27, no. 1, pp. 97-99, Jan. 2017.

Table 81 – continued from previous page

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unisolated	
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
solved_c	Solved capacitance of the load.
solved_l	Solved inductance of the load
solved_m	Solved match
solved_r1	Solved reflect1
solved_r2	Solved reflect2
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.LRRM.T_matrices

property LRRM.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.LRRM.biased_error

property LRRM.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LRRM.caled_ntwk_sets

property LRRM.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.LRRM.caled_ntwks

property LRRM.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.LRRM.coefs

property LRRM.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.LRRM.coefs_12term

property LRRM.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.LRRM.coefs_12term_ntwks

property LRRM.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.LRRM.coefs_3term

property LRRM.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.LRRM.coefs_3term_ntwks

property LRRM.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.LRRM.coefs_8term

property LRRM.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.LRRM.coefs_8term_ntwks

property LRRM.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.LRRM.coefs_ntwks

property LRRM.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.LRRM.error_ntwk

property LRRM.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.LRRM.family

LRRM.**family = 'LRRM'**

skrf.calibration.calibration.LRRM.frequency

property LRRM.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.LRRM.measured_unisolated

property LRRM.**measured_unisolated**

skrf.calibration.calibration.LRRM.measured_unterminated

property LRRM.**measured_unterminated**

skrf.calibration.calibration.LRRM.normalized_directivity

property LRRM.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.LRRM.nstandards

property LRRM.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.LRRM.output_from_run

property LRRM.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.LRRM.residual_ntwk_sets

property LRRM.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.LRRM.residual_ntwks

property LRRM.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.LRRM.solved_c

property LRRM.**solved_c**

Solved capacitance of the load. Zero if match_fit != 'lc'.

skrf.calibration.calibration.LRRM.solved_l

property LRRM.solved_l
Solved inductance of the load

skrf.calibration.calibration.LRRM.solved_m

property LRRM.solved_m
Solved match

skrf.calibration.calibration.LRRM.solved_r1

property LRRM.solved_r1
Solved reflect1

skrf.calibration.calibration.LRRM.solved_r2

property LRRM.solved_r2
Solved reflect2

skrf.calibration.calibration.LRRM.total_error

property LRRM.total_error: *NetworkSet*
Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LRRM.unbiased_error

property LRRM.unbiased_error: *NetworkSet*
Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LRRM.verify_12term

property LRRM.**verify_12term**

skrf.calibration.calibration.LRRM.verify_12term_ntwk

property LRRM.**verify_12term_ntwk**

Methods

__init__	LRRM Initializer.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.LRRM.__init__

LRRM.**__init__**(*measured*, *ideals*, *switch_terms=None*, *isolation=None*, *z0=50*, *match_fit='l'*, **args*, ***kwargs*)

LRRM Initializer.

Parameters

- **measured** (list of *Network* objects) – Raw measurements of the calibration standards. The order must be line, reflect, reflect, match and must align with the *ideals* parameter
- **ideals** (list of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *measured* list
- **switch_terms** (tuple of *Network* objects) – the pair of switch terms in the order (forward, reverse)
- **isolation** (*Network* object) – Measurement with loads on both ports with a perfect isolation between the ports. Used for determining the isolation error terms. If no measurement is given leakage is assumed to be zero.
- **z0** (*int*) – Calibration reference impedance. Only affects the solved match inductance. Has no effect on the solved calibration parameters.
- **match_fit** (*string or None*) – Match model. Valid choices are: 'l' to fit a single inductance over all frequencies. 'lc' to fit match with series inductor and parallel capacitor and assuming that the second reflect is open with unknown capacitance. 'none' to not fit inductance and let it be different for each frequency point.

  'l' or 'lc' is recommended for normal use.

skrf.calibration.calibration.LRRM.apply_cal

LRRM.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters
ntwk (*Network*) – Uncalibrated input network.

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.LRRM.apply_cal_to_all_in_dir

LRRM.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.LRRM.apply_cal_to_list

LRRM.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.LRRM.apply_cal_to_network_set

LRRM.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.LRRM.embed

LRRM.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters
ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.LRRM.from_coefs

classmethod LRRM.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.LRRM.from_coefs_ntwks

classmethod LRRM.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.LRRM.plot_caled_ntwks

LRRM.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.LRRM.plot_calibration_errors

LRRM.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.LRRM.plot_residuals

LRRM.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

Calibration.residual_networks

skrf.calibration.calibration.LRRM.pop

LRRM.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.LRRM.remove_and_cal

LRRM.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.LRRM.renormalize

LRRM.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.LRRM.run

LRRM.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.LRRM.terminate

LRRM.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.LRRM.unterminate

LRRM.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

See also

calibration.unterminate

skrf.calibration.calibration.LRRM.update_coefs

LRRM.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.LRRM.write

LRRM.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.

- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.MRC

class skrf.calibration.calibration.**MRC**(*measured*, *ideals*, **args*, ***kwargs*)

Misalignment Resistance Calibration.

This is an error-box based calibration that is a combination of the SDDL[1]_ and the UnknownThru[2]_, algorithms. The self-calibration aspects of these two algorithms alleviate the need to know the phase of the delay shorts, as well as the exact response of the thru. Thus the calibration is resistant to waveguide flange misalignment.

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.

continues on next page

Table 83 – continued from previous page

error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unisolated	
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.MRC.T_matrices

property MRC.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.MRC.biased_error

property MRC.**biased_error**: *NetworkSet*

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

skrf.calibration.calibration.MRC.caled_ntwk_sets

property MRC.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.MRC.caled_ntwks

property MRC.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.MRC.coefs

property MRC.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

> **See also**
>
> *coefs_3term*, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.MRC.coefs_12term

property MRC.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking

- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.MRC.coefs_12term_ntwks

property MRC.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.MRC.coefs_3term

property MRC.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.MRC.coefs_3term_ntwks

property MRC.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.MRC.coefs_8term

property MRC.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.MRC.coefs_8term_ntwks

property MRC.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.MRC.coefs_ntwks

property MRC.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.MRC.error_ntwk

property MRC.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.MRC.family

MRC.**family = 'MRC'**

skrf.calibration.calibration.MRC.frequency

property MRC.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.MRC.measured_unisolated

property MRC.**measured_unisolated**

skrf.calibration.calibration.MRC.measured_unterminated

property MRC.**measured_unterminated**

skrf.calibration.calibration.MRC.normalized_directivity

property MRC.**normalized_directivity**

Directivity normalized to the reflection tracking.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.MRC.nstandards

property MRC.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.MRC.output_from_run

property MRC.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.MRC.residual_ntwk_sets

property MRC.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.MRC.residual_ntwks

property MRC.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.MRC.total_error

property MRC.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.MRC.unbiased_error

property MRC.**unbiased_error**: *NetworkSet*

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.MRC.verify_12term

property MRC.**verify_12term**

skrf.calibration.calibration.MRC.verify_12term_ntwk

property MRC.**verify_12term_ntwk**

Methods

__init__	MRC Initializer
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.MRC.__init__

MRC.**__init__**(*measured*, *ideals*, **args*, ***kwargs*)

MRC Initializer

This calibration takes exactly 5 standards, which must be in the order:

[Short, DelayShort1, DelayShort1, Load, Thru]

The ideals for the delay shorts are not used and the ideal for the *thru* is only used to choose the sign of a square root. Thus, it only has to be have s21, s12 known within π phase.

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list
- **switch_terms** (tuple of *Network* objects) – the pair of switch terms in the order (forward, reverse)

skrf.calibration.calibration.MRC.apply_cal

MRC.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters
ntwk (*Network*) – Uncalibrated input network.

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.MRC.apply_cal_to_all_in_dir

MRC.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.MRC.apply_cal_to_list

MRC.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.MRC.apply_cal_to_network_set

MRC.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.MRC.embed

MRC.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters
ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.MRC.from_coefs

classmethod MRC.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.MRC.from_coefs_ntwks

classmethod MRC.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.MRC.plot_caled_ntwks

MRC.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.MRC.plot_calibration_errors

MRC.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.MRC.plot_residuals

MRC.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

> **See also**
>
> Calibration.residual_networks

skrf.calibration.calibration.MRC.pop

MRC.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.MRC.remove_and_cal

MRC.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.MRC.renormalize

MRC.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.MRC.run

MRC.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.MRC.terminate

MRC.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also

`calibration.terminate`

skrf.calibration.calibration.MRC.unterminate

MRC.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

See also

`calibration.unterminate`

skrf.calibration.calibration.MRC.update_coefs

MRC.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.MRC.write

MRC.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.

- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.TRL

class skrf.calibration.calibration.**TRL**(*measured*, *ideals=None*, *estimate_line=False*, *n_reflects=1*, *solve_reflect=True*, **args*, ***kwargs*)

Thru-Reflect-Line (also called LRL for Line-Reflect-Line).

A Similar self-calibration algorithm as developed by Engen and Hoer[1], more closely following into a more matrix form in[2].

Reference impedance of the calibration is characteristic impedance of the transmission lines. If the characteristic impedance is known, reference impedance can be renormalized using *EightTerm.renormalize* function.

See also

determine_line

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.

continues on next page

[1] G. F. Engen and C. A. Hoer, "Thru-Reflect-Line: An Improved Technique for Calibrating the Dual Six-Port Automatic Network Analyzer," IEEE Transactions on Microwave Theory and Techniques, vol. 27, no. 12, pp. 987-993, 1979.

[2] H.-J. Eul and B. Schiek, "A generalized theory and new calibration procedures for network analyzer self-calibration," IEEE Transactions on Microwave Theory and Techniques, vol. 39, no. 4, pp. 724-731, 1991.

Table 85 – continued from previous page

coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unisolated	
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.TRL.T_matrices

property TRL.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.TRL.biased_error

property TRL.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TRL.caled_ntwk_sets

property TRL.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.TRL.caled_ntwks

property TRL.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.TRL.coefs

property TRL.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.TRL.coefs_12term

property TRL.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match

- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.TRL.coefs_12term_ntwks

property TRL.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.TRL.coefs_3term

property TRL.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.TRL.coefs_3term_ntwks

property TRL.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.TRL.coefs_8term

property TRL.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.TRL.coefs_8term_ntwks

property TRL.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.TRL.coefs_ntwks

property TRL.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.TRL.error_ntwk

property TRL.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.TRL.family

TRL.**family = 'TRL'**

skrf.calibration.calibration.TRL.frequency

property TRL.**frequency**

Frequency object of the calibration.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.TRL.measured_unisolated

property TRL.measured_unisolated

skrf.calibration.calibration.TRL.measured_unterminated

property TRL.measured_unterminated

skrf.calibration.calibration.TRL.normalized_directivity

property TRL.normalized_directivity

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.TRL.nstandards

property TRL.nstandards

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.TRL.output_from_run

property TRL.output_from_run

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.TRL.residual_ntwk_sets

property TRL.residual_ntwk_sets

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.TRL.residual_ntwks

property TRL.residual_ntwks

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.TRL.total_error

property TRL.total_error: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TRL.unbiased_error

property TRL.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TRL.verify_12term

property TRL.**verify_12term**

skrf.calibration.calibration.TRL.verify_12term_ntwk

property TRL.**verify_12term_ntwk**

Methods

__init__	Initialize a TRL calibration.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.

continues on next page

Table 86 – continued from previous page

from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.TRL.__init__

TRL.**__init__**(*measured*, *ideals=None*, *estimate_line=False*, *n_reflects=1*, *solve_reflect=True*, **args*, ***kwargs*)

Initialize a TRL calibration.

Note that the order of *measured* and *ideals* is strict. It must be [Thru, Reflect, Line]. A multiline algorithms is used if more than one line is passed. A multi-reflect algorithm is used if multiple reflects are passed, see *n_reflects* argument.

All of the *ideals* can be individually set to *None*, or the entire list set to *None* (*ideals=None*). For each ideal set to *None*, the following assumptions are made:

- thru : flush thru
- reflect : flush shorts
- line : an approximately 90deg matched line (can be lossy)

The reflect ideals can also be given as a +-1.

If thru is non-zero length, the calibration is done with zero length thru and the ideal thru length is subtracted from the ideal lines. The resulting calibration reference plane is at the center of the thru. Ideal reflect standard reference plane is at the center of the thru even if the ideal thru length is non-zero.

Note you can also use the *estimate_line* option to automatically estimate the initial guess for the line length from measurements . This is sensible if you have no idea what the line length is, but your **error networks** are well matched (E_ij >>E_ii).

Notes

This implementation inherits from *EightTerm*. dont forget to pass switch_terms.

Parameters

- **measured** (list of *Network*) – must be in order [Thru, Reflect[s], Line[s]]. if the number of reflects is >1 then use *n_reflects* argument.
- **ideals** (list of *Network*, [+1,-1] , None) – must be in order [Thru, Reflect, Line]. Each element in the list may be None, or equivalently, the list may be None. Also the reflects can be simply given as +1 or -1.

- **estimate_line** (*bool*) – should we estimates the length of the line standard from raw measurements, if not we assume its about 90 deg.
- **solve_reflect** (*bool*) – Solve for the reflect or not.
- **n_reflects** (*1*) – number of reflective standards
- ***args** (*passed to EightTerm.__init__*) – dont forget the *switch_terms* argument is important
- ****kwargs** (*passed to EightTerm.__init__*) – dont forget the *switch_terms* argument is important

Examples

```
>>> thru = rf.Network('thru.s2p')
>>> reflect = rf.Network('reflect.s2p')
>>> line = rf.Network('line.s2p')
```

Ideals is None, so we assume it's close to a flush short:

```
>>> trl = TRL(measured=[thru,reflect,line], ideals=None)
```

Reflect is given as close to a flush short:

```
>>> trl = TRL(measured=[thru,reflect,line], ideals=[None,-1,None])
```

Reflect is given as close to a flush open:

```
>>> trl = TRL(measured=[thru,reflect,line], ideals=[None,+1,None])
```

See also

determine_line, *determine_reflect*, *NISTMultilineTRL*, *TUGMultilineTRL*

skrf.calibration.calibration.TRL.apply_cal

TRL.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters
ntwk (*Network*) – Uncalibrated input network.

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.TRL.apply_cal_to_all_in_dir

TRL.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.TRL.apply_cal_to_list

TRL.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.TRL.apply_cal_to_network_set

TRL.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.TRL.embed

TRL.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters
ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.TRL.from_coefs

classmethod TRL.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.TRL.from_coefs_ntwks

classmethod TRL.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.TRL.plot_caled_ntwks

TRL.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.TRL.plot_calibration_errors

TRL.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.TRL.plot_residuals

TRL.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.TRL.pop

TRL.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.TRL.remove_and_cal

TRL.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.TRL.renormalize

TRL.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.TRL.run

TRL.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.TRL.terminate

TRL.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.TRL.unterminate

TRL.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

See also

calibration.unterminate

skrf.calibration.calibration.TRL.update_coefs

TRL.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.TRL.write

TRL.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.MultilineTRL

skrf.calibration.calibration.**MultilineTRL**

alias of *TRL*

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.

continues on next page

Table 87 – continued from previous page

coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unisolated	
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.MultilineTRL.T_matrices

property MultilineTRL.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.MultilineTRL.biased_error

property MultilineTRL.**biased_error**: *NetworkSet*

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.MultilineTRL.caled_ntwk_sets

property MultilineTRL.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.MultilineTRL.caled_ntwks

property MultilineTRL.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.MultilineTRL.coefs

property MultilineTRL.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.MultilineTRL.coefs_12term

property MultilineTRL.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match

- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.MultilineTRL.coefs_12term_ntwks

property MultilineTRL.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.MultilineTRL.coefs_3term

property MultilineTRL.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.MultilineTRL.coefs_3term_ntwks

property MultilineTRL.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.MultilineTRL.coefs_8term

property MultilineTRL.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.MultilineTRL.coefs_8term_ntwks

property MultilineTRL.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.MultilineTRL.coefs_ntwks

property MultilineTRL.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.MultilineTRL.error_ntwk

property MultilineTRL.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.MultilineTRL.family

MultilineTRL.**family = 'TRL'**

skrf.calibration.calibration.MultilineTRL.frequency

property MultilineTRL.**frequency**

Frequency object of the calibration.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.MultilineTRL.measured_unisolated

property MultilineTRL.**measured_unisolated**

skrf.calibration.calibration.MultilineTRL.measured_unterminated

property MultilineTRL.**measured_unterminated**

skrf.calibration.calibration.MultilineTRL.normalized_directivity

property MultilineTRL.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.MultilineTRL.nstandards

property MultilineTRL.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.MultilineTRL.output_from_run

property MultilineTRL.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.MultilineTRL.residual_ntwk_sets

property MultilineTRL.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.MultilineTRL.residual_ntwks

property MultilineTRL.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.MultilineTRL.total_error

property MultilineTRL.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.MultilineTRL.unbiased_error

property MultilineTRL.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.MultilineTRL.verify_12term

property MultilineTRL.**verify_12term**

skrf.calibration.calibration.MultilineTRL.verify_12term_ntwk

property MultilineTRL.**verify_12term_ntwk**

Methods

__init__	Initialize a TRL calibration.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.

continues on next page

Table 88 – continued from previous page

from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.MultilineTRL.__init__

MultilineTRL.**__init__**(*measured*, *ideals=None*, *estimate_line=False*, *n_reflects=1*, *solve_reflect=True*, **args*, ***kwargs*)

Initialize a TRL calibration.

Note that the order of *measured* and *ideals* is strict. It must be [Thru, Reflect, Line]. A multiline algorithms is used if more than one line is passed. A multi-reflect algorithm is used if multiple reflects are passed, see *n_reflects* argument.

All of the *ideals* can be individually set to *None*, or the entire list set to *None* (*ideals=None*). For each ideal set to *None*, the following assumptions are made:

- thru : flush thru
- reflect : flush shorts
- line : an approximately 90deg matched line (can be lossy)

The reflect ideals can also be given as a +-1.

If thru is non-zero length, the calibration is done with zero length thru and the ideal thru length is subtracted from the ideal lines. The resulting calibration reference plane is at the center of the thru. Ideal reflect standard reference plane is at the center of the thru even if the ideal thru length is non-zero.

Note you can also use the *estimate_line* option to automatically estimate the initial guess for the line length from measurements . This is sensible if you have no idea what the line length is, but your **error networks** are well matched (E_ij >>E_ii).

Notes

This implementation inherits from *EightTerm*. dont forget to pass switch_terms.

Parameters

- **measured** (list of *Network*) – must be in order [Thru, Reflect[s], Line[s]]. if the number of reflects is >1 then use *n_reflects* argument.
- **ideals** (list of *Network*, [+1,-1] , None) – must be in order [Thru, Reflect, Line]. Each element in the list may be None, or equivalently, the list may be None. Also the reflects can be simply given as +1 or -1.

- **estimate_line** (*bool*) – should we estimates the length of the line standard from raw measurements, if not we assume its about 90 deg.
- **solve_reflect** (*bool*) – Solve for the reflect or not.
- **n_reflects** (*1*) – number of reflective standards
- ***args** (*passed to EightTerm.__init__*) – dont forget the *switch_terms* argument is important
- ****kwargs** (*passed to EightTerm.__init__*) – dont forget the *switch_terms* argument is important

Examples

```
>>> thru = rf.Network('thru.s2p')
>>> reflect = rf.Network('reflect.s2p')
>>> line = rf.Network('line.s2p')
```

Ideals is None, so we assume it's close to a flush short:

```
>>> trl = TRL(measured=[thru,reflect,line], ideals=None)
```

Reflect is given as close to a flush short:

```
>>> trl = TRL(measured=[thru,reflect,line], ideals=[None,-1,None])
```

Reflect is given as close to a flush open:

```
>>> trl = TRL(measured=[thru,reflect,line], ideals=[None,+1,None])
```

See also

determine_line, *determine_reflect*, *NISTMultilineTRL*, *TUGMultilineTRL*

skrf.calibration.calibration.MultilineTRL.apply_cal

MultilineTRL.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters
ntwk (*Network*) – Uncalibrated input network.

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.MultilineTRL.apply_cal_to_all_in_dir

MultilineTRL.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.MultilineTRL.apply_cal_to_list

MultilineTRL.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.MultilineTRL.apply_cal_to_network_set

MultilineTRL.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.MultilineTRL.embed

MultilineTRL.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters
ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.MultilineTRL.from_coefs

classmethod MultilineTRL.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.MultilineTRL.from_coefs_ntwks

classmethod MultilineTRL.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.MultilineTRL.plot_caled_ntwks

MultilineTRL.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.MultilineTRL.plot_calibration_errors

MultilineTRL.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.MultilineTRL.plot_residuals

MultilineTRL.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

Calibration.residual_networks

skrf.calibration.calibration.MultilineTRL.pop

MultilineTRL.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.MultilineTRL.remove_and_cal

MultilineTRL.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.MultilineTRL.renormalize

MultilineTRL.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.MultilineTRL.run

MultilineTRL.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.MultilineTRL.terminate

MultilineTRL.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also
calibration.terminate

skrf.calibration.calibration.MultilineTRL.unterminate

MultilineTRL.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

See also
calibration.unterminate

skrf.calibration.calibration.MultilineTRL.update_coefs

MultilineTRL.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.MultilineTRL.write

MultilineTRL.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.NISTMultilineTRL

class skrf.calibration.calibration.**NISTMultilineTRL**(*measured*, *Grefls*, *l*, *er_est=1*, *refl_offset=None*, *ref_plane=0*, *gamma_root_choice='auto'*, *k_method='multical'*, *c0=None*, *z0_ref=50*, *z0_line=None*, **args*, ***kwargs*)

NIST Multiline TRL calibration.

Multiline TRL can use multiple lines to extend bandwidth and accuracy of the calibration. Different line measurements are combined in a way that minimizes the error in calibration.

Calibration reference plane is at the edges of the lines.

At every frequency point there should be at least one line pair that has phase difference that is not 0 degrees or a multiple of 180 degrees otherwise calibration equations are singular and accuracy is very poor.

By default the reference impedance of the calibration is the characteristic impedance of the transmission lines. If the characteristic impedance is known reference impedance can be renormalized by giving *z0_ref* and *z0_line*. Alternatively if capacitance/length of the transmission line is given with *c0* argument, characteristic impedance can be solved assuming that conductance/length is zero.

Algorithm is the one published in[0], but implementation is based on[1].

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
er_eff	Effective permittivity of the solved line.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
gamma	Propagation constant of the solved line.
measured_unisolated	
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
nstd	Normalized standard deviation of the calibration error.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

continues on next page

[0] D. C. DeGroot, J. A. Jargon and R. B. Marks, "Multiline TRL revealed," 60th ARFTG Conference Digest, Fall 2002., Washington, DC, USA, 2002, pp. 131-155.

[1] K. Yau "On the metrology of nanoscale Silicon transistors above 100 GHz" Ph.D. dissertation, Dept. Elec. Eng. and Comp. Eng., University of Toronto, Toronto, Canada, 2011.

Table 89 – continued from previous page

z0	Solved characteristic impedance of the transmission lines.

skrf.calibration.calibration.NISTMultilineTRL.T_matrices

property NISTMultilineTRL.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.NISTMultilineTRL.biased_error

property NISTMultilineTRL.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.NISTMultilineTRL.caled_ntwk_sets

property NISTMultilineTRL.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.NISTMultilineTRL.caled_ntwks

property NISTMultilineTRL.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.NISTMultilineTRL.coefs

property NISTMultilineTRL.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.NISTMultilineTRL.coefs_12term

property NISTMultilineTRL.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.NISTMultilineTRL.coefs_12term_ntwks

property NISTMultilineTRL.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.NISTMultilineTRL.coefs_3term

property NISTMultilineTRL.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.NISTMultilineTRL.coefs_3term_ntwks

property NISTMultilineTRL.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.NISTMultilineTRL.coefs_8term

property NISTMultilineTRL.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.NISTMultilineTRL.coefs_8term_ntwks

property NISTMultilineTRL.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.NISTMultilineTRL.coefs_ntwks

property NISTMultilineTRL.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

> **See also**
>
> *coefs_3term_ntwks*, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.NISTMultilineTRL.er_eff

property NISTMultilineTRL.**er_eff**

Effective permittivity of the solved line.

Defined in terms of the propagation constant:

$$\gamma = \alpha + j\beta = \frac{2\pi f}{c}\sqrt{\epsilon_{r,eff}}$$

skrf.calibration.calibration.NISTMultilineTRL.error_ntwk

property NISTMultilineTRL.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.NISTMultilineTRL.family

NISTMultilineTRL.**family = 'TRL'**

skrf.calibration.calibration.NISTMultilineTRL.frequency

property NISTMultilineTRL.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.NISTMultilineTRL.gamma

property NISTMultilineTRL.**gamma**

Propagation constant of the solved line.

skrf.calibration.calibration.NISTMultilineTRL.measured_unisolated

property NISTMultilineTRL.**measured_unisolated**

skrf.calibration.calibration.NISTMultilineTRL.measured_unterminated

property NISTMultilineTRL.**measured_unterminated**

skrf.calibration.calibration.NISTMultilineTRL.normalized_directivity

property NISTMultilineTRL.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.NISTMultilineTRL.nstandards

property NISTMultilineTRL.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.NISTMultilineTRL.nstd

property NISTMultilineTRL.**nstd**

Normalized standard deviation of the calibration error.

Normalization is done such that 90 degree long single line gives a standard deviation of 1.

skrf.calibration.calibration.NISTMultilineTRL.output_from_run

property NISTMultilineTRL.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.NISTMultilineTRL.residual_ntwk_sets

property NISTMultilineTRL.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.NISTMultilineTRL.residual_ntwks

property NISTMultilineTRL.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.NISTMultilineTRL.total_error

property NISTMultilineTRL.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.NISTMultilineTRL.unbiased_error

property NISTMultilineTRL.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.NISTMultilineTRL.verify_12term

property NISTMultilineTRL.**verify_12term**

skrf.calibration.calibration.NISTMultilineTRL.verify_12term_ntwk

property NISTMultilineTRL.**verify_12term_ntwk**

skrf.calibration.calibration.NISTMultilineTRL.z0

property NISTMultilineTRL.**z0**

Solved characteristic impedance of the transmission lines.

This is only solved if C0 parameter (Capacitance/length in units F/m) is given.

Solved Z0 assumes that conductance/length is zero and line supports TEM mode.

Methods

__init__	NISTMultilineTRL initializer.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.

continues on next page

Table 90 – continued from previous page

embed	Applies the error boxes to the calibrated input network.
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
load_calibration_archive	
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
save_calibration	save calibration as an archive containing the standards, parameters and calibration results
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.NISTMultilineTRL.__init__

NISTMultilineTRL.**__init__**(*measured*, *Grefls*, *l*, *er_est=1*, *refl_offset=None*, *ref_plane=0*, *gamma_root_choice='auto'*, *k_method='multical'*, *c0=None*, *z0_ref=50*, *z0_line=None*, **args*, ***kwargs*)

NISTMultilineTRL initializer.

Note that the order of *measured* is strict. It must be [Thru, Reflects, Lines]. Multiple reflects can also be used.

Notes

This implementation inherits from *EightTerm*. Don't forget to pass switch_terms.

Parameters

- **measured** (list of *Network*) – must be in order [Thru, Reflects, Lines]
- **Grefls** (*complex or list of complex*) – Estimated reflection coefficients of reflect standards. Usually -1 for short or +1 for open.
- **l** (*list of float*) – Lengths of through and lines. If through is non-zero length its length is subtracted from the line lengths for the calibration and afterwards the calibration reference planes are shifted back by half thru on both ports using the solved propagation constant.
- **er_est** (*complex*) – Estimated effective permittivity of the lines. Imaginary part is the loss at 1 GHz. Negative imaginary part indicates losses.
- **refl_offset** (*float or list of float*) – Estimated offsets of the reflect standards from the reference plane at the end of the lines. Negative length is towards the VNA. Units are in meters.
- **ref_plane** (*float or list of float*) – Reference plane shift after the calibration. Negative length is towards the VNA. Units are in meters.

  Different shifts can be given to different ports by giving a two element list. First element is shift of port 1 and second is shift of port 2.

- **gamma_root_choice** (*string*) – Method to use for choosing the correct eigenvalue for propagation constant.

  'estimate' : Choose the root that is closer to the estimated propagation constant. Best choice when data is of good quality. To improve the performance in phatological cases it's possible to give estimate of the propagation constant as a keyword argument with 'gamma_est'.

  'real' : Force the real part of the gamma to be positive corresponding to a lossy line. Imaginary part can be negative. This is the suggested method when lines have moderate loss and the measurement noise is low. This is the default method.

  'auto' : Use heuristics to choose the eigenvalue.

  'imag' : Force the imaginary part of the gamma to be positive, corresponding to a positive length line. Real part can be positive. May choose incorrectly when the line is long due to phase wrapping.

- **k_method** (*string*) – Method to use for determining the 'k' error coefficient. Currently valid choices are 'marks' or 'multical'. The default method is 'multical'.

- **c0** (*None, float or list of float*) – Capacitance/length of the transmission line in units F/m used for reference impedance renormalization.

  Characteristic impedance of the transmission lines can be determined from the solved propagation constant if capacitance per length is known. After the characteristic impedance is solved reference impedance of the calibration is changed to *z0_ref*. Solved characteristic impedance can be accessed with *cal.z0*.

  Assumes TEM mode and conductance/length to be zero.

  If *c0* == *z0_line* is None, the characteristic impedance is not renormalized. In this case reference impedance of the calibration is characteristic impedance of the transmission lines.

- **z0_ref** (*None, complex or list of complex*) – New reference impedance for the characteristic impedance renormalizarion.

  No effect if *c0* is None and *z0_line* is None.

  If *z0_ref* is None, no renormalization is done.

- **z0_line** (*None, complex or list of complex*) – Characteristic impedance of the transmission lines. Used for reference impedance renormalization.

  If *z0* == *z0_line* is None, the characteristic impedance is not renormalized. In this case reference impedance of the calibration is characteristic impedance of the transmission lines.

- ***args** (*passed to EightTerm.__init__*) – dont forget the *switch_terms* argument is important

- ****kwargs** (*passed to EightTerm.__init__*) – dont forget the *switch_terms* argument is important

See also

TUGMultilineTRL

skrf.calibration.calibration.NISTMultilineTRL.apply_cal

NISTMultilineTRL.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters
ntwk (*Network*) – Uncalibrated input network.

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.NISTMultilineTRL.apply_cal_to_all_in_dir

NISTMultilineTRL.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.NISTMultilineTRL.apply_cal_to_list

NISTMultilineTRL.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.NISTMultilineTRL.apply_cal_to_network_set

NISTMultilineTRL.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.NISTMultilineTRL.embed

NISTMultilineTRL.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters
ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.NISTMultilineTRL.from_coefs

classmethod NISTMultilineTRL.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.NISTMultilineTRL.from_coefs_ntwks

classmethod NISTMultilineTRL.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.NISTMultilineTRL.load_calibration_archive

classmethod NISTMultilineTRL.**load_calibration_archive**(*filename*)

skrf.calibration.calibration.NISTMultilineTRL.plot_caled_ntwks

NISTMultilineTRL.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.NISTMultilineTRL.plot_calibration_errors

NISTMultilineTRL.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.NISTMultilineTRL.plot_residuals

NISTMultilineTRL.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.NISTMultilineTRL.pop

NISTMultilineTRL.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.NISTMultilineTRL.remove_and_cal

NISTMultilineTRL.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.NISTMultilineTRL.renormalize

NISTMultilineTRL.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.NISTMultilineTRL.run

NISTMultilineTRL.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.NISTMultilineTRL.save_calibration

NISTMultilineTRL.**save_calibration**(*filename*)

save calibration as an archive containing the standards, parameters and calibration results

Parameters

filename (*str*) – the path of the zip archive to save

skrf.calibration.calibration.NISTMultilineTRL.terminate

NISTMultilineTRL.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.NISTMultilineTRL.unterminate

NISTMultilineTRL.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

See also

calibration.unterminate

skrf.calibration.calibration.NISTMultilineTRL.update_coefs

NISTMultilineTRL.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.NISTMultilineTRL.write

NISTMultilineTRL.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.TUGMultilineTRL

class skrf.calibration.calibration.**TUGMultilineTRL**(*line_meas*, *line_lengths*, *er_est=1 + 0j*, *reflect_meas=None*, *reflect_est=None*, *reflect_offset=0*, *ref_plane=0*, *compensate_repeated_lines=False*, *lnorm=1*, **args*, ***kwargs*)

TUG Multiline TRL calibration.

An improved multiline TRL calibration procedure that generalizes the calibration process by solving a single 4x4 weighted eigenvalue problem.

The overall algorithm is based on[1], but the weighting matrix calculation is based on[2]. You can read the mathematical details online at[3].

The thru normalization uses the S-parameter formulation from[4]. Optionally, the weighting matrix can be scaled to compensate for repeated line lengths and to change the L-norm weighting of the eigenvalue problem, as described in[5] (see `compensate_repeated_lines` and `lnorm`).

The calibration reference plane is at the edges of the first line. By default, the reference impedance of the calibration is the characteristic impedance of the transmission lines. If the characteristic impedance is known, the reference impedance can be renormalized afterwards by running the method *renormalize()*.

Examples

```
>>> line1 = rf.Network('line1.s2p')
>>> line2 = rf.Network('line2.s2p')
>>> line3 = rf.Network('line3.s2p')
>>> short = rf.Network('short.s2p')
>>> dut   = rf.Network('dut.s2p')
```

Normal multiline TRL calibration:

[1] Z. Hatab, M. Gadringer and W. Bösch, "Improving The Reliability of The Multiline TRL Calibration Algorithm," 2022 98th ARFTG Microwave Measurement Conference (ARFTG), Las Vegas, NV, USA, 2022, pp. 1-5, doi: https://doi.org/10.1109/ARFTG52954.2022.9844064

[2] Z. Hatab, M. Gadringer and W. Bösch, "Propagation of Linear Uncertainties Through Multiline Thru-Reflect-Line Calibration," in IEEE Transactions on Instrumentation and Measurement, vol. 72, pp. 1-9, 2023, doi: https://doi.org/10.1109/TIM.2023.3296123

[3] https://ziadhatab.github.io/posts/multiline-trl-calibration/

[4] Z. Hatab, M. E. Gadringer and W. Bösch, "A Thru-Free Multiline Calibration," in IEEE Transactions on Instrumentation and Measurement, vol. 72, pp. 1-9, 2023, Art no. 1008709, doi: https://doi.org/10.1109/TIM.2023.3308226

[5] Z. Hatab, M. E. Gadringer, and W. Bösch, "The Choice of Line Lengths in Multiline Thru-Reflect-Line Calibration," arXiv: https://arxiv.org/abs/2512.18641

```
>>> cal = rf.TUGMultilineTRL(line_meas=[line1,line2,line3], line_lengths=[0, 1e-3,␣
↪5e-3], er_est=4-.0j,
>>>        reflect_meas=short, reflect_est=-1, reflect_offset=0)
>>> dut_cal = cal.apply_cal(dut)
```

Case of not using reflect measurements:

```
>>> cal = rf.TUGMultilineTRL(line_meas=[line1,line2,line3], line_lengths=[0, 1e-3,␣
↪5e-3], er_est=4-.0j)
>>> dut_cal = cal.apply_cal(dut)  # only S21 and S12 are correct
```

References

See also

NISTMultilineTRL

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
effective_phase_deg	Effective phase of the calibration, `arcsin(kappa/2)`, in degrees.
er_eff	Relative effective permittivity of the solved line.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
gamma	Propagation constant of the solved line.
kappa	Normalized eigenvalue of the weighted eigendecomposition.
lambd	Eigenvalue of the weighted eigendecomposition.
measured_unisolated	

continues on next page

Table 91 – continued from previous page

measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.TUGMultilineTRL.T_matrices

property TUGMultilineTRL.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.TUGMultilineTRL.biased_error

property TUGMultilineTRL.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TUGMultilineTRL.caled_ntwk_sets

property TUGMultilineTRL.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.TUGMultilineTRL.caled_ntwks

property TUGMultilineTRL.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.TUGMultilineTRL.coefs

property TUGMultilineTRL.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.TUGMultilineTRL.coefs_12term

property TUGMultilineTRL.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.TUGMultilineTRL.coefs_12term_ntwks

property TUGMultilineTRL.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.TUGMultilineTRL.coefs_3term

property TUGMultilineTRL.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.TUGMultilineTRL.coefs_3term_ntwks

property TUGMultilineTRL.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.TUGMultilineTRL.coefs_8term

property TUGMultilineTRL.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.TUGMultilineTRL.coefs_8term_ntwks

property TUGMultilineTRL.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.TUGMultilineTRL.coefs_ntwks

property TUGMultilineTRL.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.TUGMultilineTRL.effective_phase_deg

property TUGMultilineTRL.**effective_phase_deg**

Effective phase of the calibration, `arcsin(kappa/2)`, in degrees.

A quality metric derived from the normalized eigenvalue *kappa*: values approaching 90 degrees indicate a well-conditioned calibration. `kappa/2` is clipped to 1, so the result is capped at 90 degrees. See [5]_.

skrf.calibration.calibration.TUGMultilineTRL.er_eff

property TUGMultilineTRL.**er_eff**

Relative effective permittivity of the solved line.

skrf.calibration.calibration.TUGMultilineTRL.error_ntwk

property TUGMultilineTRL.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.TUGMultilineTRL.family

TUGMultilineTRL.**family = 'TRL'**

skrf.calibration.calibration.TUGMultilineTRL.frequency

property TUGMultilineTRL.**frequency**

Frequency object of the calibration.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.TUGMultilineTRL.gamma

property TUGMultilineTRL.**gamma**

Propagation constant of the solved line.

skrf.calibration.calibration.TUGMultilineTRL.kappa

property TUGMultilineTRL.**kappa**

Normalized eigenvalue of the weighted eigendecomposition.

It is the eigenvalue divided by the mean magnitude of the weighting matrix, providing a scale-independent measure of how well-conditioned the calibration is. See [5]_.

skrf.calibration.calibration.TUGMultilineTRL.lambd

property TUGMultilineTRL.**lambd**

Eigenvalue of the weighted eigendecomposition. The closer the eigenvalue to zero, the more sensitive the calibration to error. Inversely proportional to the normalized standard deviation of NIST multiline TRL.

skrf.calibration.calibration.TUGMultilineTRL.measured_unisolated

property TUGMultilineTRL.**measured_unisolated**

skrf.calibration.calibration.TUGMultilineTRL.measured_unterminated

property TUGMultilineTRL.**measured_unterminated**

skrf.calibration.calibration.TUGMultilineTRL.normalized_directivity

property TUGMultilineTRL.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.TUGMultilineTRL.nstandards

property TUGMultilineTRL.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.TUGMultilineTRL.output_from_run

property TUGMultilineTRL.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.TUGMultilineTRL.residual_ntwk_sets

property TUGMultilineTRL.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.TUGMultilineTRL.residual_ntwks

property TUGMultilineTRL.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.TUGMultilineTRL.total_error

property TUGMultilineTRL.**total_error: *NetworkSet***

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TUGMultilineTRL.unbiased_error

property TUGMultilineTRL.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TUGMultilineTRL.verify_12term

property TUGMultilineTRL.**verify_12term**

skrf.calibration.calibration.TUGMultilineTRL.verify_12term_ntwk

property TUGMultilineTRL.**verify_12term_ntwk**

Methods

__init__	TUGMultilineTRL initializer.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminates switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.TUGMultilineTRL.__init__

TUGMultilineTRL.**__init__**(*line_meas*, *line_lengths*, *er_est=1 + 0j*, *reflect_meas=None*, *reflect_est=None*, *reflect_offset=0*, *ref_plane=0*, *compensate_repeated_lines=False*, *lnorm=1*, **args*, ***kwargs*)

TUGMultilineTRL initializer.

The order of the lines in *line_meas* and *line_lengths* should be the same. Also, the first line is defined as thru. If non-zero, the calibration plane is shifted by half of its length using the extracted propagation constant.

You can perform calibration without reflect measurements, but this will only provide you with the propagation constant and relative effective permittivity. Without reflect measurements, you can accurately calibrate S21 and S12 of a DUT. However, calibrating S11 and S22 requires a symmetric reflect as part of the calibration process.

Notes

This implementation inherits from *EightTerm*. Don't forget to pass switch_terms.

Parameters

- **line_meas** (list of two-port *Network*) – measurement of the lines. First line is defined as thru.

- **line_lengths** (*list of float*) – Lengths of the lines. If thru is non-zero length, the calibration plane is shifted by half of its length using the solved propagation constant. Units are in meters.
- **er_est** (*complex*) – Estimated permittivity of the lines at first frequency point of the measurement. Negative imaginary part indicates losses.
- **reflect_meas** (a two-port *Network* or a list of two-port *Network*) – measurement of symmetric reflect. Multiple symmetric reflect can be passed in a list, which is used to compute to average solution of the error terms.
- **reflect_est** (*complex or list of complex*) – Estimated reflection coefficients of reflect standards at first frequency point of the measurement. Usually -1 for short or +1 for open.
- **reflect_offset** (*float or list of float*) – Offset of the reflect standards from the reference plane. Units are in meters.
- **ref_plane** (*float or list of float*) – Reference plane shift after the calibration. Negative length is towards the VNA. Units are in meters.

  Different shifts can be given to different ports by giving a two element list. First element is shift of port 1 and second is shift of port 2.
- **compensate_repeated_lines** (*bool*) – If True, scale the weighting matrix to compensate for line measurements that share the same length (repeated lines). Default is False.
- **lnorm** (*int*) – Norm-weighting used in the eigenvalue problem (e.g. 1 for L1, 2 for L2). Default is 1.
- ***args** (*passed to EightTerm.__init__*) – dont forget the *switch_terms* argument is important
- ****kwargs** (*passed to EightTerm.__init__*) – dont forget the *switch_terms* argument is important

skrf.calibration.calibration.TUGMultilineTRL.apply_cal

TUGMultilineTRL.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*.

Parameters
ntwk (*Network*) – Uncalibrated input network.

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.TUGMultilineTRL.apply_cal_to_all_in_dir

TUGMultilineTRL.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.TUGMultilineTRL.apply_cal_to_list

TUGMultilineTRL.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.TUGMultilineTRL.apply_cal_to_network_set

TUGMultilineTRL.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.TUGMultilineTRL.embed

TUGMultilineTRL.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*.

Parameters
ntwk (*Network*) – Calibrated input network.

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.TUGMultilineTRL.from_coefs

classmethod TUGMultilineTRL.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.TUGMultilineTRL.from_coefs_ntwks

classmethod TUGMultilineTRL.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.TUGMultilineTRL.plot_caled_ntwks

TUGMultilineTRL.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.TUGMultilineTRL.plot_calibration_errors

TUGMultilineTRL.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.TUGMultilineTRL.plot_residuals

TUGMultilineTRL.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

Calibration.residual_networks

skrf.calibration.calibration.TUGMultilineTRL.pop

TUGMultilineTRL.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.TUGMultilineTRL.remove_and_cal

TUGMultilineTRL.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.TUGMultilineTRL.renormalize

TUGMultilineTRL.**renormalize**(*z0_old*, *z0_new*, *powerwave=False*)

Renormalizes the calibration error boxes to a new reference impedance.

Useful for example after doing a TRL calibration with non-50 ohm transmission lines. After the TRL calibration reference impedance is the line characteristic impedance. If the transmission line characteristic impedance is known this function can be used to renormalize the reference impedance to 50 ohms or any other impedance.

skrf.calibration.calibration.TUGMultilineTRL.run

TUGMultilineTRL.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.TUGMultilineTRL.terminate

TUGMultilineTRL.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also
`calibration.terminate`

skrf.calibration.calibration.TUGMultilineTRL.unterminate

TUGMultilineTRL.**unterminate**(*ntwk*)

Unterminates switch terms from a raw measurement.

See also
`calibration.unterminate`

skrf.calibration.calibration.TUGMultilineTRL.update_coefs

TUGMultilineTRL.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.TUGMultilineTRL.write

TUGMultilineTRL.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.SixteenTerm

class skrf.calibration.calibration.**SixteenTerm**(*measured*, *ideals*, *switch_terms=None*, **args*, ***kwargs*)

General SixteenTerm (aka Error-box) Two-port calibration.

16-term error model is a complete error model that can solve for leakages between the different VNA receivers.

There are several different combinations of calibration standards that can be used. At least five two port measurements are needed. Using through, open, short, and load standards some combinations result in singular matrix. See[1] for list of non-singular combinations.

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.

continues on next page

[1] K. J. Silvonen, "Calibration of 16-term error model (microwave measurement)," in Electronics Letters, vol. 29, no. 17, pp. 1544-1545, 19 Aug. 1993.

Table 93 – continued from previous page

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.SixteenTerm.T_matrices

property SixteenTerm.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.SixteenTerm.biased_error

property SixteenTerm.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SixteenTerm.caled_ntwk_sets

property SixteenTerm.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.SixteenTerm.caled_ntwks

property SixteenTerm.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.SixteenTerm.coefs

property SixteenTerm.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.SixteenTerm.coefs_12term

property SixteenTerm.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.SixteenTerm.coefs_12term_ntwks

property SixteenTerm.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.SixteenTerm.coefs_3term

property SixteenTerm.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.SixteenTerm.coefs_3term_ntwks

property SixteenTerm.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.SixteenTerm.coefs_8term

property SixteenTerm.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.SixteenTerm.coefs_8term_ntwks

property SixteenTerm.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.SixteenTerm.coefs_ntwks

property SixteenTerm.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.SixteenTerm.error_ntwk

property SixteenTerm.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.SixteenTerm.family

SixteenTerm.**family** = 'SixteenTerm'

skrf.calibration.calibration.SixteenTerm.frequency

property SixteenTerm.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.SixteenTerm.measured_unterminated

property SixteenTerm.**measured_unterminated**

skrf.calibration.calibration.SixteenTerm.normalized_directivity

property SixteenTerm.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.SixteenTerm.nstandards

property SixteenTerm.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.SixteenTerm.output_from_run

property SixteenTerm.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.SixteenTerm.residual_ntwk_sets

property SixteenTerm.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.SixteenTerm.residual_ntwks

property SixteenTerm.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.SixteenTerm.total_error

property SixteenTerm.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns

total_error – Network with s_mag is proportional to the total error

Return type

skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SixteenTerm.unbiased_error

property SixteenTerm.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.SixteenTerm.verify_12term

property SixteenTerm.**verify_12term**

skrf.calibration.calibration.SixteenTerm.verify_12term_ntwk

property SixteenTerm.**verify_12term_ntwk**

Methods

E_matrices	Convert solved calibration T matrices to S-parameters.
__init__	SixteenTerm Initializer.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.

continues on next page

Table 94 – continued from previous page

embed	Applies the error boxes to the calibrated input network.
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.
terminate	Terminate a network with switch terms.
unterminate	Unterminate switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.SixteenTerm.E_matrices

SixteenTerm.**E_matrices**(*T1*, *T2*, *T3*, *T4*)

Convert solved calibration T matrices to S-parameters.

Returns
E1,E2,E3,E4

Return type
numpy ndarray

skrf.calibration.calibration.SixteenTerm.__init__

SixteenTerm.**__init__**(*measured*, *ideals*, *switch_terms=None*, **args*, ***kwargs*)

SixteenTerm Initializer.

Notes

Switch terms are already assumed to be corrected since the ordinary correction equations are not valid if the crosstalk is significant.

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list
- **switch_terms** (tuple of *Network* objects) – the pair of switch terms in the order (forward, reverse)

skrf.calibration.calibration.SixteenTerm.apply_cal

SixteenTerm.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*. :param ntwk: Uncalibrated input network. :type ntwk: *Network*

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.SixteenTerm.apply_cal_to_all_in_dir

SixteenTerm.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.SixteenTerm.apply_cal_to_list

SixteenTerm.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.SixteenTerm.apply_cal_to_network_set

SixteenTerm.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.SixteenTerm.embed

SixteenTerm.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*. :param ntwk: Calibrated input network. :type ntwk: *Network*

Returns
embedded – Network with error boxes applied.

Return type
Network

skrf.calibration.calibration.SixteenTerm.from_coefs

classmethod SixteenTerm.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.SixteenTerm.from_coefs_ntwks

classmethod SixteenTerm.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.SixteenTerm.plot_caled_ntwks

SixteenTerm.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.SixteenTerm.plot_calibration_errors

SixteenTerm.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters
ax (*Axes*)

skrf.calibration.calibration.SixteenTerm.plot_residuals

SixteenTerm.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

Calibration.residual_networks

skrf.calibration.calibration.SixteenTerm.pop

SixteenTerm.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.SixteenTerm.remove_and_cal

SixteenTerm.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.SixteenTerm.renormalize

SixteenTerm.**renormalize**(*z0_old*, *z0_new*)

Renormalizes the calibration error boxes to a new reference impedance.

skrf.calibration.calibration.SixteenTerm.run

SixteenTerm.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.SixteenTerm.terminate

SixteenTerm.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.SixteenTerm.unterminate

SixteenTerm.**unterminate**(*ntwk*)

Unterminate switch terms from a raw measurement.

See also

calibration.unterminate

skrf.calibration.calibration.SixteenTerm.update_coefs

SixteenTerm.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.SixteenTerm.write

SixteenTerm.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.LMR16

class skrf.calibration.calibration.**LMR16**(*measured*, *ideals*, *ideal_is_reflect=True*, *sign=None*, *switch_terms=None*, **args*, ***kwargs*)

SixteenTerm Load-Match-Reflect self-calibration.

16-Term self calibration for leaky VNA. Implementation is based on[1].

Needs five standards to be measured and given in this order:

- Through

[1] K. Silvonen, "LMR 16-a self-calibration procedure for a leaky network analyzer," in IEEE Transactions on Microwave Theory and Techniques, vol. 45, no. 7, pp. 1041-1049, Jul 1997

- Match-match
- Reflect-reflect
- Reflect-match
- Match-reflect

Reflect standard needs to be very reflective and same in all measurements. Matching of through and match standards is assumed to be perfect. Loss of the through is assumed to be zero, but its length can be non-zero.

Only reflect or through standard needs to be known and the other one will be solved during the calibration. Solved S-parameters of the standards can be accessed with LMR16.solved_through and LMR16.solved_reflect.

Switch termination is already assumed to be done either by the previous calibration or during the measurements. Regular switch correction equations used with EightTerm calibration can't be used if leakage is significant.

References

Attributes

T_matrices	Intermediate matrices used for embedding and de-embedding.
biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
measured_unterminated	
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
solved_reflect	Return the solved reflect or the ideal reflect if through was solved.
solved_through	Return the solved through or the ideal through if reflect was solved.

continues on next page

Table 95 – continued from previous page

total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.LMR16.T_matrices

property LMR16.**T_matrices**

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.LMR16.biased_error

property LMR16.**biased_error**: *NetworkSet*

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LMR16.caled_ntwk_sets

property LMR16.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.LMR16.caled_ntwks

property LMR16.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.LMR16.coefs

property LMR16.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.LMR16.coefs_12term

property LMR16.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

References

skrf.calibration.calibration.LMR16.coefs_12term_ntwks

property LMR16.coefs_12term_ntwks

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.LMR16.coefs_3term

property LMR16.coefs_3term

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.LMR16.coefs_3term_ntwks

property LMR16.coefs_3term_ntwks

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.LMR16.coefs_8term

property LMR16.coefs_8term

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

References

skrf.calibration.calibration.LMR16.coefs_8term_ntwks

property LMR16.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.LMR16.coefs_ntwks

property LMR16.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.LMR16.error_ntwk

property LMR16.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.LMR16.family

LMR16.**family = 'SixteenTerm'**

skrf.calibration.calibration.LMR16.frequency

property LMR16.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.LMR16.measured_unterminated

property LMR16.**measured_unterminated**

skrf.calibration.calibration.LMR16.normalized_directivity

property LMR16.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.LMR16.nstandards

property LMR16.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.LMR16.output_from_run

property LMR16.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.LMR16.residual_ntwk_sets

property LMR16.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.LMR16.residual_ntwks

property LMR16.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.LMR16.solved_reflect

property LMR16.**solved_reflect**

Return the solved reflect or the ideal reflect if through was solved.

skrf.calibration.calibration.LMR16.solved_through

property LMR16.**solved_through**

Return the solved through or the ideal through if reflect was solved.

skrf.calibration.calibration.LMR16.total_error

property LMR16.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LMR16.unbiased_error

property LMR16.**unbiased_error**: *NetworkSet*

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.LMR16.verify_12term

property LMR16.**verify_12term**

skrf.calibration.calibration.LMR16.verify_12term_ntwk

property LMR16.**verify_12term_ntwk**

Methods

E_matrices	Convert solved calibration T matrices to S-parameters.
__init__	LMR16 initializer.
apply_cal	Applies the calibration to the input network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Applies the error boxes to the calibrated input network.
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
renormalize	Renormalizes the calibration error boxes to a new reference impedance.
run	Run the calibration algorithm.

continues on next page

Table 96 – continued from previous page

terminate	Terminate a network with switch terms.
unterminate	Unterminate switch terms from a raw measurement.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.LMR16.E_matrices

LMR16.**E_matrices**(*T1*, *T2*, *T3*, *T4*)

Convert solved calibration T matrices to S-parameters.

Returns
E1,E2,E3,E4

Return type
numpy ndarray

skrf.calibration.calibration.LMR16.__init__

LMR16.**__init__**(*measured*, *ideals*, *ideal_is_reflect=True*, *sign=None*, *switch_terms=None*, **args*, ***kwargs*)

LMR16 initializer.

Due to needing to solve a second order equation during the calibration a choice must be taken on the correct root. Sign argument, +1 or -1, can be given to make the root choice.

If sign argument is not given it is tried to be solved automatically by choosing the sign that makes $k = \frac{t_{15}}{t_{12}}$ closer to +1, which holds if test fixture is symmetric.

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards.
- **ideals** (list/dict of *Network* objects) – Estimated response of the reflect or through calibration standard.
- **ideal_is_reflect** (*Boolean*) – True if given ideal is reflect and False if ideal is through
- **sign** (*+1,-1 or None*) – Sign to be used for the root choice.

skrf.calibration.calibration.LMR16.apply_cal

LMR16.**apply_cal**(*ntwk*)

Applies the calibration to the input network. Inverse of *embed*. :param ntwk: Uncalibrated input network. :type ntwk: *Network*

Returns
caled – Calibrated network.

Return type
Network

skrf.calibration.calibration.LMR16.apply_cal_to_all_in_dir

LMR16.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.LMR16.apply_cal_to_list

LMR16.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.LMR16.apply_cal_to_network_set

LMR16.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.LMR16.embed

LMR16.**embed**(*ntwk*)

Applies the error boxes to the calibrated input network. Inverse of *apply_cal*. :param ntwk: Calibrated input network. :type ntwk: *Network*

Returns

embedded – Network with error boxes applied.

Return type

Network

skrf.calibration.calibration.LMR16.from_coefs

classmethod LMR16.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.LMR16.from_coefs_ntwks

classmethod LMR16.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.LMR16.plot_caled_ntwks

LMR16.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.LMR16.plot_calibration_errors

LMR16.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.LMR16.plot_residuals

LMR16.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.LMR16.pop

LMR16.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.LMR16.remove_and_cal

LMR16.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.LMR16.renormalize

LMR16.**renormalize**(*z0_old*, *z0_new*)

Renormalizes the calibration error boxes to a new reference impedance.

skrf.calibration.calibration.LMR16.run

LMR16.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.LMR16.terminate

LMR16.**terminate**(*ntwk*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.LMR16.unterminate

LMR16.**unterminate**(*ntwk*)

Unterminate switch terms from a raw measurement.

See also

calibration.unterminate

skrf.calibration.calibration.LMR16.update_coefs

LMR16.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.LMR16.write

LMR16.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.Normalization

class skrf.calibration.calibration.**Normalization**(*measured*, *ideals*, *sloppy_input=False*, *is_reciprocal=True*, *name=None*, *self_calibration=False*, **args*, ***kwargs*)

Simple Thru Normalization.

For calibration the S parameters of the network are divided by average of the measured networks. The ideal networks are not used in the calibration.

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.

continues on next page

Table 97 – continued from previous page

coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.Normalization.biased_error

property Normalization.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.Normalization.caled_ntwk_sets

property Normalization.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.Normalization.caled_ntwks

property Normalization.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.Normalization.coefs

property Normalization.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.Normalization.coefs_12term

property Normalization.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.Normalization.coefs_12term_ntwks

property Normalization.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.Normalization.coefs_3term

property Normalization.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.Normalization.coefs_3term_ntwks

property Normalization.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.Normalization.coefs_8term

property Normalization.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.Normalization.coefs_8term_ntwks

property Normalization.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.Normalization.coefs_ntwks

property Normalization.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.Normalization.error_ntwk

property Normalization.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.Normalization.family

Normalization.**family = ''**

skrf.calibration.calibration.Normalization.frequency

property Normalization.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.Normalization.normalized_directivity

property Normalization.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.Normalization.nstandards

property Normalization.**nstandards**

Number of ideal/measurement pairs in calibration.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.Normalization.output_from_run

property Normalization.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.Normalization.residual_ntwk_sets

property Normalization.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.Normalization.residual_ntwks

property Normalization.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.Normalization.total_error

property Normalization.**total_error: *NetworkSet***

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.Normalization.unbiased_error

property Normalization.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.Normalization.verify_12term

property Normalization.**verify_12term**

skrf.calibration.calibration.Normalization.verify_12term_ntwk

property Normalization.**verify_12term_ntwk**

Methods

__init__	Calibration initializer.
apply_cal	Apply correction to a Network.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	Run the calibration algorithm.
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.Normalization.__init__

Normalization.**__init__**(*measured*, *ideals*, *sloppy_input=False*, *is_reciprocal=True*, *name=None*, *self_calibration=False*, **args*, ***kwargs*)

Calibration initializer.

Notes

About the order of supplied standards,

If the measured and ideals parameters are lists of Networks and *sloppy_input=False*, then their elements must align. However, if the measured and ideals are dictionaries, or *sloppy_input=True*, then we will try to align them for you based on the names of the networks (see *func: `align_measured_ideals*).

You do not want to use this *sloppy_input* feature if the calibration depends on the standard order (like TRL).

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use *sloppy_input*)
- **sloppy_input** (*Boolean.*) – Allows ideals and measured lists to be 'aligned' based on the network names.
- **is_reciprocal** (*Boolean*) – enables the reciprocity assumption on the calculation of the error_network, which is only relevant for one-port calibrations.
- **name** (*string*) – the name of this calibration instance, like 'waveguide cal' this is just for convenience [None].
- **self_calibration** (*Boolean*) – True if there are less ideals than measurements. Used in self-calibration such as LMR, LRRM, where some of the standards can be unknown.
- ***args** (*key-word arguments*) – stored in self.kwargs, which may be used by sub-classes most likely in *run*.
- ****kwargs** (*key-word arguments*) – stored in self.kwargs, which may be used by sub-classes most likely in *run*.

skrf.calibration.calibration.Normalization.apply_cal

Normalization.**apply_cal**(*input_ntwk*)

Apply correction to a Network.

skrf.calibration.calibration.Normalization.apply_cal_to_all_in_dir

Normalization.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.Normalization.apply_cal_to_list

Normalization.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.Normalization.apply_cal_to_network_set

Normalization.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.Normalization.embed

Normalization.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.Normalization.from_coefs

classmethod Normalization.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also

Calibration.from_coefs_ntwks

skrf.calibration.calibration.Normalization.from_coefs_ntwks

classmethod Normalization.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also

Calibration.from_coefs

skrf.calibration.calibration.Normalization.plot_caled_ntwks

Normalization.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.Normalization.plot_calibration_errors

Normalization.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters
ax (*Axes*)

skrf.calibration.calibration.Normalization.plot_residuals

Normalization.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

Calibration.residual_networks

skrf.calibration.calibration.Normalization.pop

Normalization.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.Normalization.remove_and_cal

Normalization.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.Normalization.run

Normalization.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.Normalization.update_coefs

Normalization.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.Normalization.write

Normalization.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

3.7.4 Multi-port

MultiportCal(cal_dict[, isolation])	Multi-port VNA calibration using two-port calibration method.
MultiportSOLT(method, measured, ideals[, ...])	Multi-port VNA calibration using two-port calibration method with one transmissive standard for each two-port pair.

skrf.calibration.calibration.MultiportCal

class skrf.calibration.calibration.**MultiportCal**(*cal_dict*, *isolation=None*)

Multi-port VNA calibration using two-port calibration method.

cal_dict should be a dictionary with key being two-tuples of port numbers, such as (0, 1), (0, 2), ...

For each key it should have another dictionary as value. The two required keys are 'method' that should be the

two-port calibration class that is used to calibrate that two-port combination (for example SOLT, EightTerm, TRL) and 'measured' which is a list of measured Networks. Other inputs required by the two-port calibration should be given as additional keys (for example 'ideals', 'switch_terms', ...).

There should be one common port in all measurements. For example in case of 4-port calibration one possible choice would be to make three measurements between ports: 0-1, 0-2, and 0-3.

The list of measured networks can be given as either two-ports, in which case it's assumed that the two ports corresponds to the ports in the key, or multi-ports in which case a subnetwork is taken for calibration according to the key.

Example cal_dict for three port measurement:

```
{(0,1): {'method':SOLT, 'measured': [list of measured networks], 'ideals': [list of␣
↪ideal networks]},
 (0,2): {'method':SOLT, 'measured': [list of measured networks], 'ideals': [list of␣
↪ideal networks]}
}
```

isolation is optional N-port network of all ports matched for isolation calibration. If None, no isolation calibration is performed.

Parameters

- **cal_dict** (*dictionary*) – Dictionary of port pair keys as specified above.
- **isolation** (*Network* or None) – N-port measurement of all matched ports for isolation calibration. None to skip isolation calibration.

See also

calibration.MultiportSOLT

Attributes

coefs
family

skrf.calibration.calibration.MultiportCal.coefs

property MultiportCal.**coefs**

skrf.calibration.calibration.MultiportCal.family

MultiportCal.**family** = **'Multiport'**

Methods

T_matrices	Intermediate matrices used for embedding and de-embedding.
__init__	
apply_cal	Apply correction to a Network.
coefs_to_ntwks	Two-port 8-term error coefficients to Networks.

continues on next page

Table 101 – continued from previous page

dut_termination	Impedance looking from DUT to VNA terminated with switch term.
embed	Embed an ideal response in the estimated error network[s]
run	Run the calibration algorithm.
run_2port	Call the two-port calibration algorithm and populate multi-port error coefficients for this port pair.
terminate_2port	Terminate a network with switch terms.
unterminate_2port	Unterminates switch terms from a raw measurement.

skrf.calibration.calibration.MultiportCal.T_matrices

MultiportCal.**T_matrices**(*p1*, *p2*)

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.MultiportCal.__init__

MultiportCal.**__init__**(*cal_dict*, *isolation=None*)

skrf.calibration.calibration.MultiportCal.apply_cal

MultiportCal.**apply_cal**(*ntwk*)

Apply correction to a Network.

skrf.calibration.calibration.MultiportCal.coefs_to_ntwks

MultiportCal.**coefs_to_ntwks**(*coefs*, *k_side=0*)

Two-port 8-term error coefficients to Networks.

Parameters
k_side (*int, 0 or 1*) – Port to put 'k' coefficient.

skrf.calibration.calibration.MultiportCal.dut_termination

MultiportCal.**dut_termination**(*S*, *gamma*)

Impedance looking from DUT to VNA terminated with switch term.

skrf.calibration.calibration.MultiportCal.embed

MultiportCal.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.MultiportCal.run

MultiportCal.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.MultiportCal.run_2port

MultiportCal.**run_2port**(*method*, *p*, *k_side*, ***kwargs*)

Call the two-port calibration algorithm and populate multi-port error coefficients for this port pair.

Parameters

- **method** (*Two-port calibration class*) – Use to calibrate the port pair.
- **p** (*tuple of int*) – Length two tuple of port indices in the pair.
- **k_side** (*int*) – Port to put 'k' coefficient in the pair.
- ****kwargs** (*Keyword arguments*) – Passed to the calibration method.

skrf.calibration.calibration.MultiportCal.terminate_2port

MultiportCal.**terminate_2port**(*ntwk*, *p1*, *p2*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.MultiportCal.unterminate_2port

MultiportCal.**unterminate_2port**(*ntwk*, *p1*, *p2*)

Unterminates switch terms from a raw measurement.

See also

calibration.unterminate

skrf.calibration.calibration.MultiportSOLT

class skrf.calibration.calibration.**MultiportSOLT**(*method*, *measured*, *ideals*, *isolation=None*, *switch_terms=None*, *thru_pos='auto'*, *cal_args=None*)

Multi-port VNA calibration using two-port calibration method with one transmissive standard for each two-port pair.

method should be a two-port calibration method such as *EightTerm*, *UnknownThru* or *SOLT*. This class calibrates a multi-port network using the given two-port calibration method.

There should be *Nports - 1* thru standards and the number of other standards depending on the number of standards required by the chosen calibration method. Standards should be always given thrus first. Thru standards are used to find the port connections during the calibration and all standards should be N-ports.

There should be one common port in all thru measurements. For example in case of 4-port calibration one possible choice would be to make three thru measurements between ports: 0-1, 0-2, and 0-3.

If isolation calibration is used it should be not passed as an argument to the two-port calibration and N-port isolation measurement should be instead given as an argument for this class.

switch_terms should be a list of switch terms with nth entry being the switch term an/bn with source from any other port. Note that in case of two-port this is the reversed order than what two-port calibration would require. If calibration is a 12-term calibration such as *SOLT* or *TwelveTerm* no switch terms are required.

thru_pos argument can be used to change the order of thru given to the calibration method. *auto* will try to determine it automatically from the calibration method. Other options are *first* and *last*. Other standards are given in the same order they are given to this class.

cal_args can be used to give addition arguments to the calibration method. If no arguments are given it should be None, otherwise a dictionary of arguments and values.

See also

calibration.MultiportCal
Lower level multi-port calibration class. This needs to be used for TRL calibration as it has also lines between ports which doesn't fit the interface of this class.

Parameters

- **measured** (Network)
- **ideals** (*list*[Network])
- **thru_pos** (*Literal['first', 'last', 'auto']*)
- **cal_args** (dict | *None*)

Attributes

coefs
family

skrf.calibration.calibration.MultiportSOLT.coefs

property MultiportSOLT.**coefs**

skrf.calibration.calibration.MultiportSOLT.family

MultiportSOLT.**family** = **'Multiport'**

Methods

T_matrices	Intermediate matrices used for embedding and de-embedding.
__init__	
apply_cal	Apply correction to a Network.
coefs_to_ntwks	Two-port 8-term error coefficients to Networks.
dut_termination	Impedance looking from DUT to VNA terminated with switch term.

continues on next page

Table 103 – continued from previous page

embed	Embed an ideal response in the estimated error network[s]
run	Run the calibration algorithm.
run_2port	Call the two-port calibration algorithm and populate multi-port error coefficients for this port pair.
terminate_2port	Terminate a network with switch terms.
unterminate_2port	Unterminates switch terms from a raw measurement.

skrf.calibration.calibration.MultiportSOLT.T_matrices

MultiportSOLT.**T_matrices**(*p1*, *p2*)

Intermediate matrices used for embedding and de-embedding.

Returns
T1,T2,T3,T4

Return type
numpy ndarray

skrf.calibration.calibration.MultiportSOLT.__init__

MultiportSOLT.**__init__**(*method*, *measured*, *ideals*, *isolation=None*, *switch_terms=None*, *thru_pos='auto'*, *cal_args=None*)

Parameters

- **measured** (Network)
- **ideals** (*list*[Network])
- **thru_pos** (*Literal*['first', 'last', 'auto'])
- **cal_args** (dict | None)

skrf.calibration.calibration.MultiportSOLT.apply_cal

MultiportSOLT.**apply_cal**(*ntwk*)

Apply correction to a Network.

skrf.calibration.calibration.MultiportSOLT.coefs_to_ntwks

MultiportSOLT.**coefs_to_ntwks**(*coefs*, *k_side=0*)

Two-port 8-term error coefficients to Networks.

Parameters
k_side (*int, 0 or 1*) – Port to put 'k' coefficient.

skrf.calibration.calibration.MultiportSOLT.dut_termination

MultiportSOLT.**dut_termination**(*S*, *gamma*)

Impedance looking from DUT to VNA terminated with switch term.

skrf.calibration.calibration.MultiportSOLT.embed

MultiportSOLT.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.MultiportSOLT.run

MultiportSOLT.**run**()

Run the calibration algorithm.

skrf.calibration.calibration.MultiportSOLT.run_2port

MultiportSOLT.**run_2port**(*method*, *p*, *k_side*, ***kwargs*)

Call the two-port calibration algorithm and populate multi-port error coefficients for this port pair.

Parameters

- **method** (*Two-port calibration class*) – Use to calibrate the port pair.
- **p** (*tuple of int*) – Length two tuple of port indices in the pair.
- **k_side** (*int*) – Port to put 'k' coefficient in the pair.
- ****kwargs** (*Keyword arguments*) – Passed to the calibration method.

skrf.calibration.calibration.MultiportSOLT.terminate_2port

MultiportSOLT.**terminate_2port**(*ntwk*, *p1*, *p2*)

Terminate a network with switch terms.

See also

calibration.terminate

skrf.calibration.calibration.MultiportSOLT.unterminate_2port

MultiportSOLT.**unterminate_2port**(*ntwk*, *p1*, *p2*)

Unterminates switch terms from a raw measurement.

See also

calibration.unterminate

3.7.5 Three Receiver (1.5 port)

TwoPortOnePath(measured, ideals[, n_thrus, ...])	Two Port One Path Calibration (aka poor man's TwelveTerm).
EnhancedResponse(measured, ideals[, ...])	Enhanced Response Partial Calibration.

skrf.calibration.calibration.TwoPortOnePath

class skrf.calibration.calibration.**TwoPortOnePath**(*measured*, *ideals*, *n_thrus=None*, *source_port=1*, **args*, ***kwargs*)

Two Port One Path Calibration (aka poor man's TwelveTerm).

Provides full error correction on a switchless three receiver system, i.e. you can only measure the waves a1,b1,and b2. Given this architecture, the DUT must be flipped and measured twice to be fully corrected.

To allow for this, the *apply_cal* method takes a tuple of measurements in the order (forward,reverse), and creates a composite measurement that is correctable.

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.
verify_12term	
verify_12term_ntwk	

skrf.calibration.calibration.TwoPortOnePath.biased_error

property TwoPortOnePath.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TwoPortOnePath.caled_ntwk_sets

property TwoPortOnePath.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.TwoPortOnePath.caled_ntwks

property TwoPortOnePath.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.TwoPortOnePath.coefs

property TwoPortOnePath.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.TwoPortOnePath.coefs_12term

property TwoPortOnePath.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.TwoPortOnePath.coefs_12term_ntwks

property TwoPortOnePath.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.TwoPortOnePath.coefs_3term

property TwoPortOnePath.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity
- source match
- reflection tracking'

skrf.calibration.calibration.TwoPortOnePath.coefs_3term_ntwks

property TwoPortOnePath.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.TwoPortOnePath.coefs_8term

property TwoPortOnePath.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.TwoPortOnePath.coefs_8term_ntwks

property TwoPortOnePath.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.TwoPortOnePath.coefs_ntwks

property TwoPortOnePath.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

skrf.calibration.calibration.TwoPortOnePath.error_ntwk

property TwoPortOnePath.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.TwoPortOnePath.family

TwoPortOnePath.**family** = **'TwoPortOnePath'**

skrf.calibration.calibration.TwoPortOnePath.frequency

property TwoPortOnePath.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.TwoPortOnePath.normalized_directivity

property TwoPortOnePath.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.TwoPortOnePath.nstandards

property TwoPortOnePath.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.TwoPortOnePath.output_from_run

property TwoPortOnePath.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.TwoPortOnePath.residual_ntwk_sets

property TwoPortOnePath.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.TwoPortOnePath.residual_ntwks

property TwoPortOnePath.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.TwoPortOnePath.total_error

property TwoPortOnePath.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns

total_error – Network with s_mag is proportional to the total error

Return type

skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TwoPortOnePath.unbiased_error

property TwoPortOnePath.**unbiased_error: *NetworkSet***

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.TwoPortOnePath.verify_12term

property TwoPortOnePath.**verify_12term**

skrf.calibration.calibration.TwoPortOnePath.verify_12term_ntwk

property TwoPortOnePath.**verify_12term_ntwk**

Methods

__init__	Two Port One Path Calibration initializer.
apply_cal	apply the calibration to a measurement.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]

continues on next page

Table 106 – continued from previous page

from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.TwoPortOnePath.__init__

TwoPortOnePath.**__init__**(*measured*, *ideals*, *n_thrus=None*, *source_port=1*, **args*, ***kwargs*)

Two Port One Path Calibration initializer.

Use the *n_thrus* argument to explicitly define the number of transmissive standards. Otherwise, if *n_thrus=None*, then we will try and guess which are transmissive, by comparing the mean :math: *|s21|* and :math: *|s12|* responses (in dB) to *trans_thres*.

Notes

See func:*Calibration.__init__* for details about automatic standards alignment (aka *sloppy_input*).

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use sloppy_input
- **n_thrus** (*int*) – number of thru measurements
- **source_port** (*[1,2]*) – The port on which the source is active. should be 1 or 2

See also

TwelveTerm.__init__

skrf.calibration.calibration.TwoPortOnePath.apply_cal

TwoPortOnePath.**apply_cal**(*ntwk_tuple*)

apply the calibration to a measurement.

Notes

Full correction is possible given you have measured your DUT in both orientations. Meaning, you have measured the device, then physically flipped the device and made a second measurement.

This tuple of 2-port Networks is what is meant by (forward,reverse), in the docstring below

If you pass a single 2-port Network, then the measurement will only be partially corrected using what is known as the *EnhancedResponse* calibration.

Parameters
network_tuple (*tuple, or Network*) – tuple of 2-port Networks in order (forward, reverse) OR a single 2-port Network.

skrf.calibration.calibration.TwoPortOnePath.apply_cal_to_all_in_dir

TwoPortOnePath.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.TwoPortOnePath.apply_cal_to_list

TwoPortOnePath.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.TwoPortOnePath.apply_cal_to_network_set

TwoPortOnePath.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.TwoPortOnePath.embed

TwoPortOnePath.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.TwoPortOnePath.from_coefs

classmethod TwoPortOnePath.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

See also
Calibration.from_coefs_ntwks

skrf.calibration.calibration.TwoPortOnePath.from_coefs_ntwks

classmethod TwoPortOnePath.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters
coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

See also
Calibration.from_coefs

skrf.calibration.calibration.TwoPortOnePath.plot_caled_ntwks

TwoPortOnePath.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.TwoPortOnePath.plot_calibration_errors

TwoPortOnePath.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

See also

biased_error, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.TwoPortOnePath.plot_residuals

TwoPortOnePath.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'
- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

Calibration.residual_networks

skrf.calibration.calibration.TwoPortOnePath.pop

TwoPortOnePath.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.TwoPortOnePath.remove_and_cal

TwoPortOnePath.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters
std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns
ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type
tuple of skrf.Networks

skrf.calibration.calibration.TwoPortOnePath.run

TwoPortOnePath.**run**()

skrf.calibration.calibration.TwoPortOnePath.update_coefs

TwoPortOnePath.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.TwoPortOnePath.write

TwoPortOnePath.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.
- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

skrf.calibration.calibration.EnhancedResponse

class skrf.calibration.calibration.**EnhancedResponse**(*measured*, *ideals*, *n_thrus=None*, *source_port=1*, **args*, ***kwargs*)

Enhanced Response Partial Calibration.

Why are you using this? For full error you correction, you can measure the DUT in both orientations and instead use TwoPortOnePath

Accuracy of correct measurements will rely on having a good match at the passive side of the DUT.

For code-structuring reasons, this is a dummy placeholder class. Its just TwoPortOnePath, which defaults to enhancedresponse correction when you apply the calibration to a single network, and not a tuple of networks.

Attributes

biased_error	Estimate of biased error for overdetermined calibration with multiple connections of each standard.
caled_ntwk_sets	Return a NetworkSet for each *caled_ntwk*, grouped by their names.
caled_ntwks	List of the corrected calibration standards.
coefs	Dictionary or error coefficients in form of numpy arrays.
coefs_12term	Dictionary of error coefficients for 12-term Model.
coefs_12term_ntwks	Dictionary or error coefficients in form of Network objects.
coefs_3term	Dictionary of error coefficients for One-port Error model.
coefs_3term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_8term	Dictionary of error coefficients for 8-term (Error-box) Model.
coefs_8term_ntwks	Dictionary of error coefficients in form of Network objects.
coefs_ntwks	Dictionary of error coefficients in form of Network objects.
error_ntwk	The calculated error Network or Network[s].
family	
frequency	*Frequency* object of the calibration.
normalized_directivity	Directivity normalized to the reflection tracking.
nstandards	Number of ideal/measurement pairs in calibration.
output_from_run	Return any output from the `run()`.
residual_ntwk_sets	Returns a NetworkSet for each *residual_ntwk*, grouped by their names.
residual_ntwks	Dictionary of residual Networks.
total_error	Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.
unbiased_error	Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

continues on next page

Table 107 – continued from previous page

verify_12term
verify_12term_ntwk

skrf.calibration.calibration.EnhancedResponse.biased_error

property EnhancedResponse.**biased_error: *NetworkSet***

Estimate of biased error for overdetermined calibration with multiple connections of each standard.

Returns
biased_error – Network with s_mag is proportional to the biased error

Return type
skrf.Network

Notes

Mathematically, this is

$$mean_s(|mean_c(r)|)$$

Where:

- r: complex residual errors
- mean_c: complex mean taken across connection
- mean_s: complex mean taken across standard

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.EnhancedResponse.caled_ntwk_sets

property EnhancedResponse.**caled_ntwk_sets**

Return a NetworkSet for each *caled_ntwk*, grouped by their names.

skrf.calibration.calibration.EnhancedResponse.caled_ntwks

property EnhancedResponse.**caled_ntwks**

List of the corrected calibration standards.

skrf.calibration.calibration.EnhancedResponse.coefs

property EnhancedResponse.**coefs**

Dictionary or error coefficients in form of numpy arrays.

The keys of this will be different depending on the Calibration Model. This dictionary should be populated when the *run()* function is called.

Notes

when setting this, property, the numpy arrays are flattened. this makes accessing the coefs more concise in the code.

See also

coefs_3term, *coefs_8term*, *coefs_12term*, *coefs_ntwks*

skrf.calibration.calibration.EnhancedResponse.coefs_12term

property EnhancedResponse.**coefs_12term**

Dictionary of error coefficients for 12-term Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- forward transmission tracking
- forward load match
- forward isolation
- reverse directivity
- reverse load match
- reverse reflection tracking
- reverse transmission tracking
- reverse source match
- reverse isolation

Notes

If this calibration uses the 8-term model, then *convert_8term_2_12term()* is called. See[1]

References

skrf.calibration.calibration.EnhancedResponse.coefs_12term_ntwks

property EnhancedResponse.**coefs_12term_ntwks**

Dictionary or error coefficients in form of Network objects.

skrf.calibration.calibration.EnhancedResponse.coefs_3term

property EnhancedResponse.**coefs_3term**

Dictionary of error coefficients for One-port Error model.

Contains the keys:

- directivity

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

- source match
- reflection tracking'

skrf.calibration.calibration.EnhancedResponse.coefs_3term_ntwks

property EnhancedResponse.**coefs_3term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.EnhancedResponse.coefs_8term

property EnhancedResponse.**coefs_8term**

Dictionary of error coefficients for 8-term (Error-box) Model.

Contains the keys:

- forward directivity
- forward source match
- forward reflection tracking
- reverse directivity
- reverse source match
- reverse reflection tracking
- forward switch term
- reverse switch term
- k
- forward isolation
- reverse isolation

Notes

If this calibration uses the 12-term model, then *convert_12term_2_8term()* is called. See[1]

References

skrf.calibration.calibration.EnhancedResponse.coefs_8term_ntwks

property EnhancedResponse.**coefs_8term_ntwks**

Dictionary of error coefficients in form of Network objects.

skrf.calibration.calibration.EnhancedResponse.coefs_ntwks

property EnhancedResponse.**coefs_ntwks**

Dictionary of error coefficients in form of Network objects.

See also

coefs_3term_ntwks, *coefs_12term_ntwks*, *coefs_8term_ntwks*

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

skrf.calibration.calibration.EnhancedResponse.error_ntwk

property EnhancedResponse.**error_ntwk**

The calculated error Network or Network[s].

This will return a single two-port network for a one-port cal. For a 2-port calibration this will return networks for forward and reverse excitation. However, these are not sufficient to use for embedding, see the *embed()* function for that.

skrf.calibration.calibration.EnhancedResponse.family

EnhancedResponse.**family = 'EnhancedResponse'**

skrf.calibration.calibration.EnhancedResponse.frequency

property EnhancedResponse.**frequency**

Frequency object of the calibration.

skrf.calibration.calibration.EnhancedResponse.normalized_directivity

property EnhancedResponse.**normalized_directivity**

Directivity normalized to the reflection tracking.

skrf.calibration.calibration.EnhancedResponse.nstandards

property EnhancedResponse.**nstandards**

Number of ideal/measurement pairs in calibration.

skrf.calibration.calibration.EnhancedResponse.output_from_run

property EnhancedResponse.**output_from_run**

Return any output from the *run()*.

This just returns what's in _output_from_run, and calls *run()* if that attribute is non-existent. finally, returns None if run() is called, and nothing is in _output_from_run.

skrf.calibration.calibration.EnhancedResponse.residual_ntwk_sets

property EnhancedResponse.**residual_ntwk_sets**

Returns a NetworkSet for each *residual_ntwk*, grouped by their names.

skrf.calibration.calibration.EnhancedResponse.residual_ntwks

property EnhancedResponse.**residual_ntwks**

Dictionary of residual Networks.

These residuals are complex differences between the ideal standards and their corresponding corrected measurements.

skrf.calibration.calibration.EnhancedResponse.total_error

property EnhancedResponse.**total_error**: *NetworkSet*

Estimate of total error for overdetermined calibration with multiple connections of each standard.This is the combined effects of both biased and un-biased errors.

Returns
total_error – Network with s_mag is proportional to the total error

Return type
skrf.Network

Notes

Mathematically, this is

std_cs(r)

where: * r : complex residual errors * std_cs : standard deviation taken across connections and standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.EnhancedResponse.unbiased_error

property EnhancedResponse.**unbiased_error**: *NetworkSet*

Estimate of unbiased error for overdetermined calibration with multiple connections of each standard.

Returns
unbiased_error – Network with s_mag is proportional to the unbiased error

Return type
skrf.Network

Notes

Mathematically, this is

mean_s(std_c(r))

where: * r : complex residual errors * std_c : standard deviation taken across connections * mean_s : complex mean taken across standards

See also

biased_error, *unbiased_error*, *total_error*

skrf.calibration.calibration.EnhancedResponse.verify_12term

property EnhancedResponse.**verify_12term**

skrf.calibration.calibration.EnhancedResponse.verify_12term_ntwk

property EnhancedResponse.**verify_12term_ntwk**

Methods

__init__	Two Port One Path Calibration initializer.
apply_cal	apply the calibration to a measurement.
apply_cal_to_all_in_dir	Apply correction to all touchstone files in a given directory.
apply_cal_to_list	Apply correction to list or dict of Networks.
apply_cal_to_network_set	Apply correction to a NetworkSet.
embed	Embed an ideal response in the estimated error network[s]
from_coefs	Create a calibration from its error coefficients.
from_coefs_ntwks	Create a calibration from its error coefficients.
plot_caled_ntwks	Plot corrected calibration standards.
plot_calibration_errors	Plot biased, unbiased and total error in dB scaled.
plot_residuals	Plot residual networks.
pop	Remove and return tuple of (ideal, measured) at index.
remove_and_cal	Remove a cal standard and correct it, returning correct and ideal.
run	
update_coefs	Update current dict of error coefficients.
write	Write the Calibration to disk using *write()*.

skrf.calibration.calibration.EnhancedResponse.__init__

EnhancedResponse.**__init__**(*measured*, *ideals*, *n_thrus=None*, *source_port=1*, **args*, ***kwargs*)

Two Port One Path Calibration initializer.

Use the *n_thrus* argument to explicitly define the number of transmissive standards. Otherwise, if *n_thrus=None*, then we will try and guess which are transmissive, by comparing the mean :math: *|s21|* and :math: *|s12|* responses (in dB) to *trans_thres*.

Notes

See func:*Calibration.__init__* for details about automatic standards alignment (aka *sloppy_input*).

Parameters

- **measured** (list/dict of *Network* objects) – Raw measurements of the calibration standards. The order must align with the *ideals* parameter (or use *sloppy_input*)
- **ideals** (list/dict of *Network* objects) – Predicted ideal response of the calibration standards. The order must align with *ideals* list (or use sloppy_input
- **n_thrus** (*int*) – number of thru measurements
- **source_port** (*[1,2]*) – The port on which the source is active. should be 1 or 2

See also

TwelveTerm.__init__

skrf.calibration.calibration.EnhancedResponse.apply_cal

EnhancedResponse.**apply_cal**(*ntwk_tuple*)

apply the calibration to a measurement.

Notes

Full correction is possible given you have measured your DUT in both orientations. Meaning, you have measured the device, then physically flipped the device and made a second measurement.

This tuple of 2-port Networks is what is meant by (forward,reverse), in the docstring below

If you pass a single 2-port Network, then the measurement will only be partially corrected using what is known as the *EnhancedResponse* calibration.

Parameters

network_tuple (*tuple, or Network*) – tuple of 2-port Networks in order (forward, reverse) OR a single 2-port Network.

skrf.calibration.calibration.EnhancedResponse.apply_cal_to_all_in_dir

EnhancedResponse.**apply_cal_to_all_in_dir**(**args*, ***kwargs*)

Apply correction to all touchstone files in a given directory.

See *skrf.io.general.read_all_networks*.

skrf.calibration.calibration.EnhancedResponse.apply_cal_to_list

EnhancedResponse.**apply_cal_to_list**(*ntwk_list*)

Apply correction to list or dict of Networks.

skrf.calibration.calibration.EnhancedResponse.apply_cal_to_network_set

EnhancedResponse.**apply_cal_to_network_set**(*ntwk_set*)

Apply correction to a NetworkSet.

skrf.calibration.calibration.EnhancedResponse.embed

EnhancedResponse.**embed**(*ntwk*)

Embed an ideal response in the estimated error network[s]

skrf.calibration.calibration.EnhancedResponse.from_coefs

classmethod EnhancedResponse.**from_coefs**(*frequency*, *coefs*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

- **frequency** (*Frequency*) – frequency info, (duh)
- **coefs** (*dict of numpy arrays*) – error coefficients for the calibration

> **See also**
>
> *Calibration.from_coefs_ntwks*

skrf.calibration.calibration.EnhancedResponse.from_coefs_ntwks

classmethod EnhancedResponse.**from_coefs_ntwks**(*coefs_ntwks*, ***kwargs*)

Create a calibration from its error coefficients.

Parameters

coefs_ntwks (*dict of Networks objects*) – error coefficients for the calibration

> **See also**
>
> *Calibration.from_coefs*

skrf.calibration.calibration.EnhancedResponse.plot_caled_ntwks

EnhancedResponse.**plot_caled_ntwks**(*attr='s_smith'*, *show_legend=False*, ***kwargs*)

Plot corrected calibration standards.

Given that the calibration is overdetermined, this may be used as a heuristic verification of calibration quality.

Parameters

- **attr** (*str*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_smith'
- **show_legend** (*bool, optional*) – draw a legend or not. Default is False.
- ****kwargs** (*kwargs*) – passed to the plot method of Network

skrf.calibration.calibration.EnhancedResponse.plot_calibration_errors

EnhancedResponse.**plot_calibration_errors**(**args*, *ax=None*, ***kwargs*)

Plot biased, unbiased and total error in dB scaled.

> **See also**
>
> *biased_error*, *unbiased_error*, *total_error*

Parameters

ax (*Axes*)

skrf.calibration.calibration.EnhancedResponse.plot_residuals

EnhancedResponse.**plot_residuals**(*attr='s_db'*, ***kwargs*)

Plot residual networks.

Given that the calibration is overdetermined, this may be used as a metric of the calibration's *goodness of fit*

Parameters

- **attr** (*str, optional.*) – Network property to plot, ie 's_db', 's_smith', etc. Default is 's_db'

- ****kwargs** (*kwargs*) – passed to the plot method of Network

See also

`Calibration.residual_networks`

skrf.calibration.calibration.EnhancedResponse.pop

EnhancedResponse.**pop**(*std=-1*)

Remove and return tuple of (ideal, measured) at index.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,measured – the ideal and measured networks which were popped out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.EnhancedResponse.remove_and_cal

EnhancedResponse.**remove_and_cal**(*std*)

Remove a cal standard and correct it, returning correct and ideal.

This requires requires overdetermination. Useful in troubleshooting a calibration in which one standard is junk, but you dont know which.

Parameters

std (*int or str*) – the integer of calibration standard to remove, or the name of the ideal or measured calibration standard to remove.

Returns

ideal,corrected – the ideal and corrected networks which were removed out of the calibration

Return type

tuple of skrf.Networks

skrf.calibration.calibration.EnhancedResponse.run

EnhancedResponse.**run**()

skrf.calibration.calibration.EnhancedResponse.update_coefs

EnhancedResponse.**update_coefs**(*d*)

Update current dict of error coefficients.

skrf.calibration.calibration.EnhancedResponse.write

EnhancedResponse.**write**(*file=None*, **args*, ***kwargs*)

Write the Calibration to disk using *write()*.

Parameters

- **file** (*str or file-object*) – filename or a file-object. If left as None then the filename will be set to Calibration.name, if its not None. If both are None, ValueError is raised.

- ***args** (*arguments and keyword arguments*) – passed through to *write()*
- ****kwargs** (*arguments and keyword arguments*) – passed through to *write()*

Notes

If the self.name is not None and file is can left as None and the resultant file will have the *.ntwk* extension appended to the filename.

Examples

```
>>> cal.name = 'my_cal'
>>> cal.write()
```

See also

skrf.io.general.write, *skrf.io.general.read*

3.7.6 Switch Terms

compute_switch_terms(ntwks)	A method for indirectly computing the switch terms of a VNA using measurements of at least three transmissive reciprocal devices.

skrf.calibration.calibration.compute_switch_terms

skrf.calibration.calibration.**compute_switch_terms**(*ntwks*)

A method for indirectly computing the switch terms of a VNA using measurements of at least three transmissive reciprocal devices. The VNA does not need to be calibrated, and more than three reciprocal devices can be used. However, the accuracy of the computed switch terms depends on the uniqueness of the measured reciprocal devices. Devices with asymmetric structure and semi-reflective properties can help ensure the conditioning of the system matrix, which solves the switch terms.

See[1] and[2]

Parameters
ntwks (*List of networks*) – measured reciprocal devices. At least 3 required.

Returns
Gammas – The order is [Gamma21, Gamma12]. Gamma21 is forward and Gamma12 is reverse.

Return type
List of one-port networks of the switch terms.

References

with Reciprocal Devices," 2023, e-print: https://arxiv.org/abs/2306.07066

[1] 26. Hatab, M. E. Gadringer, and W. Bösch, "Indirect Measurement of Switch Terms of a Vector Network Analyzer

[2] https://ziadhatab.github.io/posts/vna-switch-terms/

See also

terminate, *unterminate*

3.7.7 Generic Methods

ideal_coefs_12term(frequency)	An ideal set of 12term calibration coefficients.
terminate(ntwk, gamma_f, gamma_r)	Terminate a network with switch terms.
terminate_nport(ntwk, gammas)	Terminate N-port network with switch terms.
unterminate(ntwk, gamma_f, gamma_r)	Unterminate switch terms from a raw measurement.
determine_line(thru_m, line_m[, line_approx])	Determine S21 of a matched line.
determine_reflect(thru_m, reflect_m, line_m)	Determine reflect from a thru, reflect, line measurements.
convert_12term_2_8term(coefs_12term[, ...])	Convert the 12-term and 8-term error coefficients.
convert_8term_2_12term(coefs_8term)	
align_measured_ideals(measured, ideals)	Aligns two lists of networks based on the intersection of their names.
two_port_error_vector_2_Ts(error_coefficients)	
error_dict_2_network(coefs, frequency[, ...])	Create a Network from a dictionary of standard error terms.

skrf.calibration.calibration.ideal_coefs_12term

skrf.calibration.calibration.**ideal_coefs_12term**(*frequency*)

An ideal set of 12term calibration coefficients.

Produces a set of error coefficients, that would result if the error networks were matched thrus

skrf.calibration.calibration.terminate

skrf.calibration.calibration.**terminate**(*ntwk*, *gamma_f*, *gamma_r*)

Terminate a network with switch terms.

see[1]

Parameters

- **ntwk** (*2-port Network*) – an unterminated network
- **gamma_f** (*1-port Network*) – measured forward switch term. gamma_f = a2/b2 sourced by port1
- **gamma_r** (*1-port Network*) – measured reverse switch term gamma_r = a1/b1 sourced by port2

Returns
ntwk

Return type
Network object

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

See also

`unterminate_switch_terms`

References

skrf.calibration.calibration.terminate_nport

skrf.calibration.calibration.**terminate_nport**(*ntwk*, *gammas*)

Terminate N-port network with switch terms.

Note that for 2-port the order *gammas* is opposite of terminate. Correct order for 2-port is [gamma_r, gamma_f].

See[1]

Parameters

- **ntwk** (Network) – an unterminated network.
- **gammas** (*list of 1-port Network*) – measured switch term. gammas[i] = ai/bi sourced by any other port.

Returns
ntwk

Return type
Network object

See also

terminate

References

skrf.calibration.calibration.unterminate

skrf.calibration.calibration.**unterminate**(*ntwk*, *gamma_f*, *gamma_r*)

Unterminate switch terms from a raw measurement.

In order to use the 8-term error model on a VNA which employs a switched source, the effects of the switch must be accounted for. This is done through *switch terms* as described in[1] . The two switch terms are defined as,

$$\Gamma_f = \frac{a2}{b2}, \qquad \text{sourced by port 1 } \Gamma_r = \frac{a1}{b1}, \qquad \text{sourced by port 2}$$

These can be measured by four-sampler VNA's by setting up user-defined traces onboard the VNA. If the VNA doesn't have 4-samplers, then you can measure switch terms indirectly by using a two-tier two-port calibration. First do a SOLT, then convert the 12-term error coefs to 8-term, and pull out the switch terms.

Parameters

- **ntwk** (*2-port Network*) – the raw measurement
- **gamma_f** (*1-port Network*) – the measured forward switch term. gamma_f = a2/b2 sourced by port1
- **gamma_r** (*1-port Network*) – the measured reverse switch term gamma_r = a1/b1 sourced by port2

[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks
[1] "Formulations of the Basic Vector Network Analyzer Error Model including Switch Terms" by Roger B. Marks

Returns
ntwk

Return type
Network object

References

skrf.calibration.calibration.determine_line

skrf.calibration.calibration.**determine_line**(*thru_m*, *line_m*, *line_approx=None*)

Determine S21 of a matched line.

Given raw measurements of a *thru* and a matched *line* with unknown s21, this will calculate the response of the line. This works for lossy lines, and attenuators. The *line_approx* is an approximation to line, this used to choose the correct root sign. If left as *None*, it will be estimated from raw measurements, which requires your error networks to be well matched (S_ij >>S_ii).

This is possible because two measurements can be combined to create a relationship of similar matrices, as shown below. Equating the eigenvalues between these measurements allows one to solve for S21 of the line.

$$M_t = X \cdot A_t \cdot Y$$
$$M_l = X \cdot A_l \cdot Y$$

$$M_t \cdot M_l^{-1} = X \cdot A_t \cdot A_l^{-1} \cdot X^{-1}$$

$$eig(M_t \cdot M_l^{-1}) = eig(A_t \cdot A_l^{-1})$$

which can be solved to yield S21 of the line

Notes

This relies on the 8-term error model, which requires that switch terms are accounted for. specifically, thru and line have their switch terms unterminated.

Parameters

- **thru_m** (*Network*) – raw measurement of a flush thru
- **line_m** (*Network*) – raw measurement of a matched transmissive standard
- **line_approx** (*Network*) – approximate network the ideal line response. if None, then the response is approximated by line_approx = line/thru. This makes the assumption that the error networks have much larger transmission than reflection

References

skrf.calibration.calibration.determine_reflect

skrf.calibration.calibration.**determine_reflect**(*thru_m*, *reflect_m*, *line_m*, *reflect_approx=None*, *line_approx=None*, *return_all=False*)

Determine reflect from a thru, reflect, line measurements.

This is used in the TRL algorithm, but is made modular for multi-line, multi-reflect options.

Parameters

- **thru_m** (*Network*) – raw measurement of a thru

- **reflect_m** (*Network*) – raw measurement of a reflect standard
- **line_m** (*Network*) – raw measurement of a matched transmissive standard
- **reflect_approx** (*Network*) – approximate One-port network for the reflect. if None, then we assume its a flush short (gamma=-1)
- **return_all** (*bool*) – return all possible values for reflect, one for each root-choice. useful for troubleshooting.

Returns

- **reflect** (*Network*) – a One-port network for the found reflect.
- **The equations are from "Thru-Reflect-Line** (*An Improved Technique for Calibrating the Dual Six-Port*)
- *Automatic Network Analyzer", G.F. Engen et al., 1979*

skrf.calibration.calibration.convert_12term_2_8term

skrf.calibration.calibration.**convert_12term_2_8term**(*coefs_12term*, *redundant_k=False*)

Convert the 12-term and 8-term error coefficients.

Derivation of this conversion can be found in[1] .

References

skrf.calibration.calibration.convert_8term_2_12term

skrf.calibration.calibration.**convert_8term_2_12term**(*coefs_8term*)

skrf.calibration.calibration.align_measured_ideals

skrf.calibration.calibration.**align_measured_ideals**(*measured*, *ideals*)

Aligns two lists of networks based on the intersection of their names.

skrf.calibration.calibration.two_port_error_vector_2_Ts

skrf.calibration.calibration.**two_port_error_vector_2_Ts**(*error_coefficients*)

skrf.calibration.calibration.error_dict_2_network

skrf.calibration.calibration.**error_dict_2_network**(*coefs*, *frequency*, *is_reciprocal=False*, ***kwargs*)

Create a Network from a dictionary of standard error terms.

3.7.8 PNA interaction

convert_skrfcoefs_2_pna(coefs[, ports])	Convert skrf error coefficients to pna error coefficients
convert_pnacoefs_2_skrf(coefs)	Convert PNA error coefficients to skrf error coefficients.

[1] Marks, Roger B.; , "Formulations of the Basic Vector Network Analyzer Error Model including Switch-Terms," ARFTG Conference Digest-Fall, 50th , vol.32, no., pp.115-126, Dec. 1997. URL: http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4119948&isnumber=4119931

skrf.calibration.calibration.convert_skrfcoefs_2_pna

skrf.calibration.calibration.**convert_skrfcoefs_2_pna**(*coefs*, *ports=(1, 2)*)

Convert skrf error coefficients to pna error coefficients

Notes

The skrf calibration terms can be found in variables

- skrf.calibration.coefs_list_3term
- skrf.calibration.coefs_list_12term

Parameters

- **coefs** (dict) – complex ndarrays for the cal coefficients as defined by skrf
- **ports** (*tuple*) – port indices. in order (forward, reverse)

Returns

pna_coefs – same error coefficients but with keys matching skrf's convention

Return type

dict

skrf.calibration.calibration.convert_pnacoefs_2_skrf

skrf.calibration.calibration.**convert_pnacoefs_2_skrf**(*coefs*)

Convert PNA error coefficients to skrf error coefficients.

Parameters

- **coefs** (dict) – coefficients as retrieved from PNA
- **ports** (*tuple*) – port indices. in order (forward, reverse)

Returns

skrf_coefs – same error coefficients but with keys matching skrf's convention

Return type

dict

3.8 deembedding (*skrf.calibration.deembedding*)

De-embedding is the procedure of removing effects of the test fixture that is often present in the measurement of a device or circuit. It is based on a lumped element approximation of the test fixture which needs removal from the raw data, and its equivalent circuit often needs to be known a-priori. This is often required since implementation of calibration methods such as Thru-Reflect-Line (TRL) becomes too expensive for implementation in on-wafer measurement environments where space is limited, or insufficiently accurate as in the case of Short-Open-Load-Thru (SOLT) calibration where the load cannot be manufactured accurately. De-embedding is often performed as a second step, after a SOLT, TRL or similar calibration to the end of a known reference plane, like the probe-tips in on-wafer measurements.

This module provides objects to implement commonly used de-embedding method in on-wafer applications. Each de-embedding method inherits from the common abstract base class *Deembedding*.

3.8.1 Base Class

Deembedding(dummies[, name])	Abstract Base Class for all de-embedding objects.

skrf.calibration.deembedding.Deembedding

class skrf.calibration.deembedding.**Deembedding**(*dummies*, *name=None*, **args*, ***kwargs*)

Abstract Base Class for all de-embedding objects.

This class implements the common mechanisms for all de-embedding algorithms. Specific calibration algorithms should inherit this class and over-ride the methods: * *Deembedding.deembed()*

Methods

__init__	De-embedding Initializer
deembed	Apply de-embedding correction to a Network

skrf.calibration.deembedding.Deembedding.__init__

Deembedding.**__init__**(*dummies*, *name=None*, **args*, ***kwargs*)

De-embedding Initializer

Notes

Each de-embedding algorithm may use a different number of dummy networks. We check that each of these dummy networks have matching frequencies to perform de-embedding.

It should be known a-priori what the equivalent circuit of the parasitic network looks like. The proper de-embedding method should then be chosen accordingly.

Parameters

- **dummies** (list of *Network* objects) – Network info of all the dummy structures used in a given de-embedding algorithm.
- **name** (*string*) – Name of this de-embedding instance, like 'open-short-set1' This is for convenience of identification.
- ***args** (*keyword arguments*) – stored in self.args and self.kwargs, which may be used by sub-classes if needed.
- ****kwargs** (*keyword arguments*) – stored in self.args and self.kwargs, which may be used by sub-classes if needed.

skrf.calibration.deembedding.Deembedding.deembed

abstractmethod Deembedding.**deembed**(*ntwk*)

Apply de-embedding correction to a Network

3.8.2 De-embedding Methods

OpenShort(dummy_open, dummy_short[, name])	Remove open parasitics followed by short parasitics.
Open(dummy_open[, name])	Remove open parasitics only.

continues on next page

Table 114 – continued from previous page

ShortOpen(dummy_short, dummy_open[, name])	Remove short parasitics followed by open parasitics.
Short(dummy_short[, name])	Remove short parasitics only.
SplitPi(dummy_thru[, name])	Remove shunt and series parasitics assuming pi-type embedding network.
SplitTee(dummy_thru[, name])	Remove series and shunt parasitics assuming tee-type embedding network.
AdmittanceCancel(dummy_thru[, name])	Cancel shunt admittance by swapping (a.k.a Mangan's method).
ImpedanceCancel(dummy_thru[, name])	Cancel series impedance by swapping.
IEEEP370(dummies[, name])	Abstract Base Class for all IEEEP370 de-embedding class.
IEEEP370_SE_NZC_2xThru(dummy_2xthru[, name, ...])	Creates error boxes from a test fixture 2xThru network.
IEEEP370_MM_NZC_2xThru(dummy_2xthru[, name, ...])	Creates error boxes from a 4-port test fixture 2xThru.
IEEEP370_SE_ZC_2xThru(dummy_2xthru, ...[, ...])	Creates error boxes from 2x-Thru and FIX-DUT-FIX networks.
IEEEP370_MM_ZC_2xThru(dummy_2xthru, ...[, ...])	Creates error boxes from a 4-port from 2x-Thru and FIX-DUT-FIX networks.

skrf.calibration.deembedding.OpenShort

class skrf.calibration.deembedding.**OpenShort**(*dummy_open*, *dummy_short*, *name=None*, **args*, ***kwargs*)

Remove open parasitics followed by short parasitics.

This is a commonly used de-embedding method for on-wafer applications.

A deembedding object is created with two dummy measurements: *dummy_open* and *dummy_short*. When *Deembedding.deembed()* is applied, Open de-embedding is applied to the short dummy because the measurement results for the short dummy contains parallel parasitics. Then the Y-parameters of the dummy_open are subtracted from the DUT measurement, followed by subtraction of Z-parameters of dummy-short which is previously de-embedded.

This method is applicable only when there is a-priori knowledge of the equivalent circuit model of the parasitic network to be de-embedded, where the series parasitics are closest to device under test, followed by the parallel parasitics. For more information, see[1]

References

Example

```
>>> import skrf as rf
>>> from skrf.calibration import OpenShort
```

Create network objects for dummy structures and dut

```
>>> op = rf.Network('open_ckt.s2p')
>>> sh = rf.Network('short_ckt.s2p')
>>> dut = rf.Network('full_ckt.s2p')
```

Create de-embedding object

[1] M. C. A. M. Koolen, J. A. M. Geelen and M. P. J. G. Versleijen, "An improved de-embedding technique for on-wafer high frequency characterization", IEEE 1991 Bipolar Circuits and Technology Meeting, pp. 188-191, Sep. 1991.

```
>>> dm = OpenShort(dummy_open = op, dummy_short = sh, name = 'test_openshort')
```

Remove parasitics to get the actual device network

```
>>> realdut = dm.deembed(dut)
```

Methods

__init__	Open-Short De-embedding Initializer
deembed	Perform the de-embedding calculation

skrf.calibration.deembedding.OpenShort.__init__

OpenShort.**__init__**(*dummy_open*, *dummy_short*, *name=None*, **args*, ***kwargs*)

Open-Short De-embedding Initializer

Parameters

- **dummy_open** (*Network* object) – Measurement of the dummy open structure
- **dummy_short** (*Network* object) – Measurement of the dummy short structure
- **name** (*string*) – Optional name of de-embedding object
- **args** – Passed to *Deembedding.__init__()*
- **kwargs** – Passed to *Deembedding.__init__()*

See also

Deembedding.__init__()

skrf.calibration.deembedding.OpenShort.deembed

OpenShort.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters

ntwk (*Network* object) – Network data of device measurement from which parasitics needs to be removed via de-embedding

Returns

caled – Network data of the device after de-embedding

Return type

Network object

skrf.calibration.deembedding.Open

class skrf.calibration.deembedding.**Open**(*dummy_open*, *name=None*, **args*, ***kwargs*)

Remove open parasitics only.

A deembedding object is created with just one open dummy measurement, *dummy_open*. When *Deembedding.deembed()* is applied, the Y-parameters of the open dummy are subtracted from the DUT measurement,

This method is applicable only when there is a-priori knowledge of the equivalent circuit model of the parasitic network to be de-embedded, where the series parasitics are assumed to be negligible, but parallel parasitics are unwanted.

Example

```
>>> import skrf as rf
>>> from skrf.calibration import Open
```

Create network objects for dummy structure and dut

```
>>> op = rf.Network('open_ckt.s2p')
>>> dut = rf.Network('full_ckt.s2p')
```

Create de-embedding object

```
>>> dm = Open(dummy_open = op, name = 'test_open')
```

Remove parasitics to get the actual device network

```
>>> realdut = dm.deembed(dut)
```

Methods

__init__	Open De-embedding Initializer
deembed	Perform the de-embedding calculation

skrf.calibration.deembedding.Open.__init__

Open.**__init__**(*dummy_open*, *name=None*, **args*, ***kwargs*)

Open De-embedding Initializer

Parameters

- **dummy_open** (*Network* object) – Measurement of the dummy open structure
- **name** (*string*) – Optional name of de-embedding object
- **args** – Passed to *Deembedding.__init__()*
- **kwargs** – Passed to *Deembedding.__init__()*

See also

Deembedding.__init__()

skrf.calibration.deembedding.Open.deembed

Open.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters

ntwk (*Network* object) – Network data of device measurement from which parasitics needs to be removed via de-embedding

Returns
caled – Network data of the device after de-embedding

Return type
Network object

skrf.calibration.deembedding.ShortOpen

class skrf.calibration.deembedding.**ShortOpen**(*dummy_short*, *dummy_open*, *name=None*, **args*, ***kwargs*)

Remove short parasitics followed by open parasitics.

A deembedding object is created with two dummy measurements: *dummy_open* and *dummy_short*. When *Deembedding.deembed()* is applied, short de-embedding is applied to the open dummy because the measurement results for the open dummy contains series parasitics. the Z-parameters of the dummy_short are subtracted from the DUT measurement, followed by subtraction of Y-parameters of dummy_open which is previously de-embedded.

This method is applicable only when there is a-priori knowledge of the equivalent circuit model of the parasitic network to be de-embedded, where the parallel parasitics are closest to device under test, followed by the series parasitics.

Example

```
>>> import skrf as rf
>>> from skrf.calibration import ShortOpen
```

Create network objects for dummy structures and dut

```
>>> op = rf.Network('open_ckt.s2p')
>>> sh = rf.Network('short_ckt.s2p')
>>> dut = rf.Network('full_ckt.s2p')
```

Create de-embedding object

```
>>> dm = ShortOpen(dummy_short = sh, dummy_open = op, name = 'test_shortopen')
```

Remove parasitics to get the actual device network

```
>>> realdut = dm.deembed(dut)
```

Methods

__init__	Short-Open De-embedding Initializer
deembed	Perform the de-embedding calculation

skrf.calibration.deembedding.ShortOpen.__init__

ShortOpen.**__init__**(*dummy_short*, *dummy_open*, *name=None*, **args*, ***kwargs*)

Short-Open De-embedding Initializer

Parameters

- **dummy_short** (*Network* object) – Measurement of the dummy short structure

- **dummy_open** (*Network* object) – Measurement of the dummy open structure
- **name** (*string*) – Optional name of de-embedding object
- **args** – Passed to *Deembedding.__init__()*
- **kwargs** – Passed to *Deembedding.__init__()*

See also

Deembedding.__init__()

skrf.calibration.deembedding.ShortOpen.deembed

ShortOpen.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters
ntwk (*Network* object) – Network data of device measurement from which parasitics needs to be removed via de-embedding

Returns
caled – Network data of the device after de-embedding

Return type
Network object

skrf.calibration.deembedding.Short

class skrf.calibration.deembedding.**Short**(*dummy_short*, *name=None*, **args*, ***kwargs*)

Remove short parasitics only.

This is a useful method to remove pad contact resistances from measurement.

A deembedding object is created with just one dummy measurement: *dummy_short*. When *Deembedding.deembed()* is applied, the Z-parameters of the dummy_short are subtracted from the DUT measurement,

This method is applicable only when there is a-priori knowledge of the equivalent circuit model of the parasitic network to be de-embedded, where only series parasitics are to be removed while retaining all others.

Example

```
>>> import skrf as rf
>>> from skrf.calibration import Short
```

Create network objects for dummy structures and dut

```
>>> sh = rf.Network('short_ckt.s2p')
>>> dut = rf.Network('full_ckt.s2p')
```

Create de-embedding object

```
>>> dm = Short(dummy_short = sh, name = 'test_short')
```

Remove parasitics to get the actual device network

```
>>> realdut = dm.deembed(dut)
```

Methods

__init__	Short De-embedding Initializer
deembed	Perform the de-embedding calculation

skrf.calibration.deembedding.Short.__init__

Short.**__init__**(*dummy_short*, *name=None*, **args*, ***kwargs*)

Short De-embedding Initializer

Parameters

- **dummy_short** (*Network* object) – Measurement of the dummy short structure
- **name** (*string*) – Optional name of de-embedding object
- **args** – Passed to *Deembedding.__init__()*
- **kwargs** – Passed to *Deembedding.__init__()*

See also

Deembedding.__init__()

skrf.calibration.deembedding.Short.deembed

Short.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters

ntwk (*Network* object) – Network data of device measurement from which parasitics needs to be removed via de-embedding

Returns

caled – Network data of the device after de-embedding

Return type

Network object

skrf.calibration.deembedding.SplitPi

class skrf.calibration.deembedding.**SplitPi**(*dummy_thru*, *name=None*, **args*, ***kwargs*)

Remove shunt and series parasitics assuming pi-type embedding network.

A deembedding object is created with just one thru dummy measurement *dummy_thru*. The thru dummy is, for example, a direct cascaded connection of the left and right test pads.

When *Deembedding.deembed()* is applied, the shunt admittance and series impedance of the thru dummy are removed.

This method is applicable only when there is a-priori knowledge of the equivalent circuit model of the parasitic network to be de-embedded, where the series parasitics are closest to device under test, followed by the shunt parasitics. For more information, see[2]

[2] L. Nan, K. Mouthaan, Y.-Z. Xiong, J. Shi, S. C. Rustagi, and B.-L. Ooi, "Experimental Characterization of the Effect of Metal Dummy Fills on Spiral Inductors," in 2007 IEEE Radio Frequency Integrated Circuits (RFIC) Symposium, Jun. 2007, pp. 307–310.

References

Example

```
>>> import skrf as rf
>>> from skrf.calibration import SplitPi
```

Create network objects for dummy structure and dut

```
>>> th = rf.Network('thru_ckt.s2p')
>>> dut = rf.Network('full_ckt.s2p')
```

Create de-embedding object

```
>>> dm = SplitPi(dummy_thru = th, name = 'test_thru')
```

Remove parasitics to get the actual device network

```
>>> realdut = dm.deembed(dut)
```

Methods

__init__	SplitPi De-embedding Initializer
deembed	Perform the de-embedding calculation

skrf.calibration.deembedding.SplitPi.__init__

SplitPi.**__init__**(*dummy_thru*, *name=None*, **args*, ***kwargs*)

SplitPi De-embedding Initializer

Parameters

- **dummy_thru** (*Network* object) – Measurement of the dummy thru structure
- **name** (*string*) – Optional name of de-embedding object
- **args** – Passed to *Deembedding.__init__()*
- **kwargs** – Passed to *Deembedding.__init__()*

See also

Deembedding.__init__()

skrf.calibration.deembedding.SplitPi.deembed

SplitPi.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters

ntwk (*Network* object) – Network data of device measurement from which parasitics needs to be removed via de-embedding

Returns

caled – Network data of the device after de-embedding

Return type
Network object

skrf.calibration.deembedding.SplitTee

class skrf.calibration.deembedding.**SplitTee**(*dummy_thru*, *name=None*, **args*, ***kwargs*)

Remove series and shunt parasitics assuming tee-type embedding network.

A deembedding object is created with just one thru dummy measurement *dummy_thru*. The thru dummy is, for example, a direct cascaded connection of the left and right test pads.

When *Deembedding.deembed()* is applied, the shunt admittance and series impedance of the thru dummy are removed.

This method is applicable only when there is a-priori knowledge of the equivalent circuit model of the parasitic network to be de-embedded, where the shunt parasitics are closest to device under test, followed by the series parasitics. For more information, see[3]

References

Example

```
>>> import skrf as rf
>>> from skrf.calibration import SplitTee
```

Create network objects for dummy structure and dut

```
>>> th = rf.Network('thru_ckt.s2p')
>>> dut = rf.Network('full_ckt.s2p')
```

Create de-embedding object

```
>>> dm = SplitTee(dummy_thru = th, name = 'test_thru')
```

Remove parasitics to get the actual device network

```
>>> realdut = dm.deembed(dut)
```

Methods

__init__	SplitTee De-embedding Initializer
deembed	Perform the de-embedding calculation

skrf.calibration.deembedding.SplitTee.__init__

SplitTee.**__init__**(*dummy_thru*, *name=None*, **args*, ***kwargs*)

SplitTee De-embedding Initializer

Parameters

- **dummy_thru** (*Network* object) – Measurement of the dummy thru structure
- **name** (*string*) – Optional name of de-embedding object

[3] M. J. Kobrinsky, S. Chakravarty, D. Jiao, M. C. Harmes, S. List, and M. Mazumder, "Experimental validation of crosstalk simulations for on-chip interconnects using S-parameters," IEEE Transactions on Advanced Packaging, vol. 28, no. 1, pp. 57–62, Feb. 2005.

- **args** – Passed to *Deembedding.__init__()*
- **kwargs** – Passed to *Deembedding.__init__()*

See also

Deembedding.__init__()

skrf.calibration.deembedding.SplitTee.deembed

SplitTee.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters

ntwk (*Network* object) – Network data of device measurement from which parasitics needs to be removed via de-embedding

Returns

caled – Network data of the device after de-embedding

Return type

Network object

skrf.calibration.deembedding.AdmittanceCancel

class skrf.calibration.deembedding.**AdmittanceCancel**(*dummy_thru*, *name=None*, **args*, ***kwargs*)

Cancel shunt admittance by swapping (a.k.a Mangan's method). A deembedding object is created with just one thru dummy measurement *dummy_thru*. The thru dummy is, for example, a direct cascaded connection of the left and right test pads.

When *Deembedding.deembed()* is applied, the shunt admittance of the thru dummy are canceled, from the DUT measurement by left-right mirroring operation.

This method is applicable to only symmetric (i.e. S11=S22 and S12=S21) 2-port DUTs, but suitable for the characterization of transmission lines at mmW frequencies. For more information, see[4]

References

Example

```
>>> import skrf as rf
>>> from skrf.calibration import AdmittanceCancel
```

Create network objects for dummy structure and dut

```
>>> th = rf.Network('thru_ckt.s2p')
>>> dut = rf.Network('full_ckt.s2p')
```

Create de-embedding object

```
>>> dm = AdmittanceCancel(dummy_thru = th, name = 'test_thru')
```

Remove parasitics to get the actual device network

[4] A. M. Mangan, S. P. Voinigescu, Ming-Ta Yang, and M. Tazlauanu, "De-embedding transmission line measurements for accurate modeling of IC designs," IEEE Trans. Electron Devices, vol. 53, no. 2, pp. 235–241, Feb. 2006.

```
>>> realdut = dm.deembed(dut)
```

Methods

__init__	AdmittanceCancel De-embedding Initializer
deembed	Perform the de-embedding calculation

skrf.calibration.deembedding.AdmittanceCancel.__init__

AdmittanceCancel.**__init__**(*dummy_thru*, *name=None*, **args*, ***kwargs*)

AdmittanceCancel De-embedding Initializer

Parameters

- **dummy_thru** (*Network* object) – Measurement of the dummy thru structure
- **name** (*string*) – Optional name of de-embedding object
- **args** – Passed to *Deembedding.__init__()*
- **kwargs** – Passed to *Deembedding.__init__()*

See also

Deembedding.__init__()

skrf.calibration.deembedding.AdmittanceCancel.deembed

AdmittanceCancel.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters

ntwk (*Network* object) – Network data of device measurement from which parasitics needs to be removed via de-embedding

Returns

caled – Network data of the device after de-embedding

Return type

Network object

skrf.calibration.deembedding.ImpedanceCancel

class skrf.calibration.deembedding.**ImpedanceCancel**(*dummy_thru*, *name=None*, **args*, ***kwargs*)

Cancel series impedance by swapping.

A deembedding object is created with just one thru dummy measurement *dummy_thru*. The thru dummy is, for example, a direct cascaded connection of the left and right test pads.

When *Deembedding.deembed()* is applied, the series impedance of the thru dummy are canceled, from the DUT measurement by left-right mirroring operation.

This method is applicable to only symmetric (i.e. S11=S22 and S12=S21) 2-port DUTs, but suitable for the characterization of transmission lines at mmW frequencies. For more information, see[5]

[5] S. Amakawa, K. Katayama, K. Takano, T. Yoshida, and M. Fujishima, "Comparative analysis of on-chip transmission line de-embedding techniques," in 2015 IEEE International Symposium on Radio-Frequency Integration Technology, Sendai, Japan, Aug. 2015, pp. 91–93.

References

Example

```
>>> import skrf as rf
>>> from skrf.calibration import ImpedanceCancel
```

Create network objects for dummy structure and dut

```
>>> th = rf.Network('thru_ckt.s2p')
>>> dut = rf.Network('full_ckt.s2p')
```

Create de-embedding object

```
>>> dm = ImpedanceCancel(dummy_thru = th, name = 'test_thru')
```

Remove parasitics to get the actual device network

```
>>> realdut = dm.deembed(dut)
```

Methods

__init__	ImpedanceCancel De-embedding Initializer
deembed	Perform the de-embedding calculation

skrf.calibration.deembedding.ImpedanceCancel.__init__

ImpedanceCancel.**__init__**(*dummy_thru*, *name=None*, **args*, ***kwargs*)

ImpedanceCancel De-embedding Initializer

Parameters

- **dummy_thru** (*Network* object) – Measurement of the dummy thru structure
- **name** (*string*) – Optional name of de-embedding object
- **args** – Passed to *Deembedding.__init__()*
- **kwargs** – Passed to *Deembedding.__init__()*

See also

Deembedding.__init__()

skrf.calibration.deembedding.ImpedanceCancel.deembed

ImpedanceCancel.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters

ntwk (*Network* object) – Network data of device measurement from which parasitics needs to be removed via de-embedding

Returns

caled – Network data of the device after de-embedding

Return type
Network object

skrf.calibration.deembedding.IEEEP370

class skrf.calibration.deembedding.**IEEEP370**(*dummies*, *name=None*, **args*, ***kwargs*)

Abstract Base Class for all IEEEP370 de-embedding class.

This class implements the common mechanisms for all IEEEP370 de-embedding algorithms. Specific algorithms should inherit this class and override the methods: * *IEEEP370.deembed()* * *IEEEP370.split2xthru()*

Based on [ElSA20], [I3E3701], [I3E3702], [I3E3703], [I3E3704], and [I3E3705].

References

Parameters

- **dummies** (*Sequence[Network]*)
- **name** (*str*)

Methods

COM_receiver_noise_filter	receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A
DC	Advanced reflective DC point extrapolation.
NRP	Enforce the Nyquist Rate Point.
__init__	IEEEP370 de-embedding Initializer.
add_dc	Extrapolate a network to DC using interpolation for all S-parameters.
dc_interp	enforces symmetric upon the first 10 points and interpolates the DC point.
deembed	Apply de-embedding correction to a Network
extrapolate_to_dc	Extrapolate the network to DC using IEEE370 NZC algorithm.
getz	Compute step response to get the time-domain impedance from S-parameters.
makeStep	Make a time-domain step response from an impulse response.
makeTL	Compute the S-parameters of a transmission line.
peelNPointsLossless	Peel N points of the network on both side and return the corresponding error boxes.
shiftNPoints	Shift the whole network of N samples in time-domain.
shiftOnePort	Shift one port of the network of N samples in time-domain.
split2xthru	Determine fixtures models
thru	Create a perfect thru

skrf.calibration.deembedding.IEEEP370.COM_receiver_noise_filter

static IEEEP370.**COM_receiver_noise_filter**(*f*, *fr*)

receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A

Parameters

- **f** (*ndarray*)

- **fr** (*float*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370.DC

static IEEEP370.**DC**(*s*, *f*, *allowedError=1e-12*)

Advanced reflective DC point extrapolation.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)
- **allowedError** (*float*)

Return type
float

skrf.calibration.deembedding.IEEEP370.NRP

static IEEEP370.**NRP**(*ntwk*, *TD=None*, *port=None*)

Enforce the Nyquist Rate Point. Force the length of the transmissive network to be an integer multiple of the wavelength at the highest frequency. If required, a proper delay is added to meet this condition. The function can also be used to remove the delay.

Parameters

- **ntwk** (*Network* object) – Network to be extrapolated to DC
- **TD** (*:array-like*) – If None, the delay will be computed and added. Else, the delay will be removed (to reset the original length). (default: None)
- **port** (*:Number*) – Specify to apply NRP only on a single port of the network (default: None)

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370.__init__

IEEEP370.**__init__**(*dummies*, *name=None*, **args*, ***kwargs*)

IEEEP370 de-embedding Initializer.

Parameters

- **dummies** (list of *Network* objects) – Network info of all the dummy structures used in a given de-embedding algorithm.
- **name** (*string*) – Name of this de-embedding instance, like 'open-short-set1' This is for convenience of identification.
- ***args** (*keyword arguments*) – stored in self.args and self.kwargs, which may be used by sub-classes if needed.
- ****kwargs** (*keyword arguments*) – stored in self.args and self.kwargs, which may be used by sub-classes if needed.

Return type
None

skrf.calibration.deembedding.IEEEP370.add_dc

`static IEEEP370.add_dc(ntwk)`

Extrapolate a network to DC using interpolation for all S-parameters.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
out – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370.dc_interp

`static IEEEP370.dc_interp(s, f)`

enforces symmetric upon the first 10 points and interpolates the DC point.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)

Return type
float

skrf.calibration.deembedding.IEEEP370.deembed

`abstractmethod IEEEP370.deembed(ntwk)`

Apply de-embedding correction to a Network

Parameters
ntwk (Network)

Return type
Network

skrf.calibration.deembedding.IEEEP370.extrapolate_to_dc

`static IEEEP370.extrapolate_to_dc(ntwk)`

Extrapolate the network to DC using IEEE370 NZC algorithm. This is useful to compare the fixtures and deembedded networks to the input data in the same conditions used by NZC algorithm. If the network already have a DC point, it will be replaced.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
ntwk_dc – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370.getz

static IEEEP370.**getz**(*s*, *f*, *z0*)

Compute step response to get the time-domain impedance from S-parameters. The S-parameters are DC extrapolated first.

Parameters

- **s** (*:array-like*) – 1-Port S-parameters array
- **f** (*:array-like*) – Frequency array for DC extrapolation
- **z0** (*:array-like*) – Reference impedance

Returns
z – Time-domain impedance step response

Return type
:array-like

skrf.calibration.deembedding.IEEEP370.makeStep

static IEEEP370.**makeStep**(*impulse*)

Make a time-domain step response from an impulse response.

Parameters
impulse (*ndarray*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370.makeTL

static IEEEP370.**makeTL**(*zline*, *z0*, *gamma*, *l*)

Compute the S-parameters of a transmission line.

Parameters

- **zline** (*:number*) – Characteristic impedance
- **z0** (*:number*) – Port impedance to renormalize into
- **gamma** (*:array-like*) – Frequency-dependent propagation constant
- **l** (*:number*) – Length in the same length unit as gamma

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370.peelNPointsLossless

static IEEEP370.**peelNPointsLossless**(*ntwk*, *N*, *z0*)

Peel N points of the network on both side and return the corresponding error boxes. This is done in a lossless way without determination of the propagation constant gamma.

Parameters

- **ntwk** (*Network* object) – Network to be peeled

- **N** (*:number*) – Number of points to peel
- **z0** (*:number*) – Reference impedance
- **gamma** (*:array-like*) – Frequency-dependent propagation constant

Returns

- **out** (*Network* object) – Peeled network
- **out** (*Network* object) – Error box side port 1
- **out** (*Network* object) – Error box side port 2

Return type
Network

skrf.calibration.deembedding.IEEEP370.shiftNPoints

static IEEEP370.**shiftNPoints**(*ntwk*, *N*)

Shift the whole network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370.shiftOnePort

static IEEEP370.**shiftOnePort**(*ntwk*, *N*, *port*)

Shift one port of the network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift
- **port** (*:Number*) – Port to be shifted

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370.split2xthru

abstractmethod IEEEP370.**split2xthru**()

Determine fixtures models

skrf.calibration.deembedding.IEEEP370.thru

static IEEEP370.**thru**(*ntwk*)

Create a perfect thru

Parameters
ntwk (*Network* object) – Network from which copy frequency, z0 and other parameters. The S-parameters will be replaced by zero-length matched lossless thru.

Returns
out – Network of the perfect thru

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru

class skrf.calibration.deembedding.**IEEEP370_SE_NZC_2xThru**(*dummy_2xthru*, *name=None*, *z0=50*, *use_z_instead_ifft=False*, *verbose=False*, *forced_z0_line=None*, **args*, ***kwargs*)

Creates error boxes from a test fixture 2xThru network.

Based on [ElSA20] and [I3E370].

A deembedding object is created with a single 2xThru (FIX-FIX) network, which is split into left (FIX-1) and right (FIX-2) fixtures with IEEEP370 2xThru method.

When *Deembedding.deembed()* is applied, the s-parameters of FIX-1 and FIX-2 are deembedded from the FIX_DUT_FIX network.

This method is applicable only when there is a 2x-Thru network.

The S-parameters bisection is done by time gating S11 and S22, taking the proper square root of the S21 corrected by return loss, and remixing the parameters according to the fixture signal flow graph. This method gives crude results but is robust.

Example

```
>>> import skrf as rf
>>> from skrf.calibration import IEEEP370_SE_NZC_2xThru
```

Create network objects for 2x-Thru and FIX_DUT_FIX

```
>>> s2xthru = rf.Network('2xthru.s2p')
>>> fdf = rf.Network('f-dut-f.s2p')
```

Create de-embedding object

```
>>> dm = IEEEP370_SE_NZC_2xThru(dummy_2xthru = s2xthru, name = '2xthru')
```

Apply deembedding to get the actual DUT network

```
>>> dut = dm.deembed(fdf)
```

Note

numbering diagram:

```
 FIX-1     DUT       FIX-2
 +----+    +----+    +----+
-|1  2|---|1  2|---|2  1|-
 +----+    +----+    +----+
```

Warning

There are two differences compared to the original matlab implementation [I3E370]:

- FIX-2 is flipped (see diagram above)
- A more robust root choice solution is used that avoids the apparition of 180° phase jumps in the fixtures in certain circumstances

References

Parameters

- **dummy_2xthru** (*Network*)
- **name** (*str*)
- **z0** (*float*)
- **use_z_instead_ifft** (*bool*)
- **verbose** (*bool*)
- **forced_z0_line** (*float*)

Methods

COM_receiver_noise_filter	receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A
DC	Advanced reflective DC point extrapolation.
NRP	Enforce the Nyquist Rate Point.
__init__	IEEEP370_SE_NZC_2xThru De-embedding Initializer
add_dc	Extrapolate a network to DC using interpolation for all S-parameters.
dc_interp	enforces symmetric upon the first 10 points and interpolates the DC point.
deembed	Perform the de-embedding calculation
extrapolate_to_dc	Extrapolate the network to DC using IEEE370 NZC algorithm.
getz	Compute step response to get the time-domain impedance from S-parameters.
makeStep	Make a time-domain step response from an impulse response.
makeTL	Compute the S-parameters of a transmission line.

continues on next page

Table 124 – continued from previous page

peelNPointsLossless	Peel N points of the network on both side and return the corresponding error boxes.
plot_check_impedance	
plot_check_residuals	
shiftNPoints	Shift the whole network of N samples in time-domain.
shiftOnePort	Shift one port of the network of N samples in time-domain.
split2xthru	Perform the fixtures extraction.
thru	Create a perfect thru

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.COM_receiver_noise_filter

static IEEEP370_SE_NZC_2xThru.**COM_receiver_noise_filter**(*f*, *fr*)

receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A

Parameters

- **f** (*ndarray*)
- **fr** (*float*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.DC

static IEEEP370_SE_NZC_2xThru.**DC**(*s*, *f*, *allowedError=1e-12*)

Advanced reflective DC point extrapolation.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)
- **allowedError** (*float*)

Return type
float

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.NRP

static IEEEP370_SE_NZC_2xThru.**NRP**(*ntwk*, *TD=None*, *port=None*)

Enforce the Nyquist Rate Point. Force the length of the transmissive network to be an integer multiple of the wavelength at the highest frequency. If required, a proper delay is added to meet this condition. The function can also be used to remove the delay.

Parameters

- **ntwk** (*Network* object) – Network to be extrapolated to DC
- **TD** (*:array-like*) – If None, the delay will be computed and added. Else, the delay will be removed (to reset the original length). (default: None)
- **port** (*:Number*) – Specify to apply NRP only on a single port of the network (default: None)

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.__init__

IEEEP370_SE_NZC_2xThru.**__init__**(*dummy_2xthru*, *name=None*, *z0=50*, *use_z_instead_ifft=False*, *verbose=False*, *forced_z0_line=None*, **args*, ***kwargs*)

IEEEP370_SE_NZC_2xThru De-embedding Initializer

Parameters

- **dummy_2xthru** (*Network* object) – 2xThru (FIX-FIX) network.
- **z0** (*float*) – reference impedance of the S-parameters (default: 50)
- **name** (*string*) – optional name of de-embedding object
- **use_z_instead_ifft** (*bool*) – use z-transform instead ifft. This method is not documented in the paper but exists in the IEEE repo. It could be used if the 2x-Thru is so short that there is not enough points in time domain to determine the length of half fixtures from the s21 impulse response and the the impedance at split plane from the s11 step response. Parameter *verbose* could be used for diagnostic in ifft mode. (default: False)
- **forced_z0_line** (*float*) – If specified, the value for the split plane impedance is forced to *forced_z0_line*. The IEEEP370 standard recommends the 2x-Thru being at least three wavelengths at the highest measured frequency. This ensures that the split plane impedance measured in the S11 step response is free of reflections from the launches. If the 2x-Thru is too short, any point in the s11 step response contain reflections from the lanches and split plane impedance cannot be determined accurately by this method. In this case, setting the impedance manually can improve the results. However, it should be noted that each fixture model will still include some reflections from the opposite side launch because there is not enough time resolution to separate them. (Default: None)
- **verbose** (*bool*) – view the process (default: False)
- **args** – passed to *Deembedding.__init__()*
- **kwargs** – passed to *Deembedding.__init__()*

Return type
None

See also

Deembedding.__init__()

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.add_dc

static IEEEP370_SE_NZC_2xThru.**add_dc**(*ntwk*)

Extrapolate a network to DC using interpolation for all S-parameters.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
out – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.dc_interp

static IEEEP370_SE_NZC_2xThru.**dc_interp**(*s*, *f*)

enforces symmetric upon the first 10 points and interpolates the DC point.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)

Return type
float

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.deembed

IEEEP370_SE_NZC_2xThru.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters
ntwk (*Network* object) – FIX-DUT-FIX network from which FIX-1 AND FIX-2 fixtures needs to be removed via de-embedding

Returns
caled – Network data of the device after de-embedding

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.extrapolate_to_dc

static IEEEP370_SE_NZC_2xThru.**extrapolate_to_dc**(*ntwk*)

Extrapolate the network to DC using IEEE370 NZC algorithm. This is useful to compare the fixtures and deembedded networks to the input data in the same conditions used by NZC algorithm. If the network already have a DC point, it will be replaced.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
ntwk_dc – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.getz

static IEEEP370_SE_NZC_2xThru.**getz**(*s*, *f*, *z0*)

Compute step response to get the time-domain impedance from S-parameters. The S-parameters are DC extrapolated first.

Parameters

- **s** (*:array-like*) – 1-Port S-parameters array
- **f** (*:array-like*) – Frequency array for DC extrapolation
- **z0** (*:array-like*) – Reference impedance

Returns
z – Time-domain impedance step response

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.makeStep

`static` IEEEP370_SE_NZC_2xThru.**makeStep**(*impulse*)

Make a time-domain step response from an impulse response.

Parameters
impulse (*ndarray*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.makeTL

`static` IEEEP370_SE_NZC_2xThru.**makeTL**(*zline*, *z0*, *gamma*, *l*)

Compute the S-parameters of a transmission line.

Parameters

- **zline** (*:number*) – Characteristic impedance
- **z0** (*:number*) – Port impedance to renormalize into
- **gamma** (*:array-like*) – Frequency-dependent propagation constant
- **l** (*:number*) – Length in the same length unit as gamma

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.peelNPointsLossless

`static` IEEEP370_SE_NZC_2xThru.**peelNPointsLossless**(*ntwk*, *N*, *z0*)

Peel N points of the network on both side and return the corresponding error boxes. This is done in a lossless way without determination of the propagation constant gamma.

Parameters

- **ntwk** (*Network* object) – Network to be peeled
- **N** (*:number*) – Number of points to peel
- **z0** (*:number*) – Reference impedance
- **gamma** (*:array-like*) – Frequency-dependent propagation constant

Returns

- **out** (*Network* object) – Peeled network
- **out** (*Network* object) – Error box side port 1
- **out** (*Network* object) – Error box side port 2

Return type
Network

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.plot_check_impedance

IEEEP370_SE_NZC_2xThru.**plot_check_impedance**(*fix_dut_fix=None*, *ax=None*, *window='hamming'*)

Parameters

- **fix_dut_fix** (Network)
- **ax** (*Axes*)
- **window** (*str*)

Return type
(Figure, Axes)

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.plot_check_residuals

IEEEP370_SE_NZC_2xThru.**plot_check_residuals**(*ax=None*)

Parameters
ax (*Axes*)

Return type
(Figure, Axes)

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.shiftNPoints

static IEEEP370_SE_NZC_2xThru.**shiftNPoints**(*ntwk*, *N*)

Shift the whole network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.shiftOnePort

static IEEEP370_SE_NZC_2xThru.**shiftOnePort**(*ntwk*, *N*, *port*)

Shift one port of the network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift
- **port** (*:Number*) – Port to be shifted

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.split2xthru

IEEEP370_SE_NZC_2xThru.**split2xthru**(*s2xthru*)

Perform the fixtures extraction.

Parameters
s2xthru (Network)

Return type
(*Network*, *Network*)

skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru.thru

static IEEEP370_SE_NZC_2xThru.**thru**(*ntwk*)

Create a perfect thru

Parameters
ntwk (*Network* object) – Network from which copy frequency, z0 and other parameters. The S-parameters will be replaced by zero-length matched lossless thru.

Returns
out – Network of the perfect thru

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru

class skrf.calibration.deembedding.**IEEEP370_MM_NZC_2xThru**(*dummy_2xthru*, *name=None*, *z0=50*, *port_order='second'*, *use_z_instead_ifft=False*, *verbose=False*, *forced_z0_line_dd=None*, *forced_z0_line_cc=None*, **args*, ***kwargs*)

Creates error boxes from a 4-port test fixture 2xThru.

Based on [ElSA20] and [I3E370].

A deembedding object is created with a single 2xThru (FIX-FIX) network, which is split into left (FIX-1_2) and right (FIX-3_4) fixtures with IEEEP370 2xThru method.

When *Deembedding.deembed()* is applied, the s-parameters of FIX-1 and FIX-2 are deembedded from the FIX_DUT_FIX network.

This method is applicable only when there is a 2x-Thru measurement.

The S-parameters bisection is done by time gating S11 and S22, taking the proper square root of the S21 corrected by return loss, and remixing the parameters according to the fixture signal flow graph. This method gives crude results but is robust.

Note

The *port_order* =’first’, means front-to-back also known as odd/even, while `port_order`=’second’ means left-to-right also known as sequential. `port_order`=’third’ means yet another numbering method. Next figure show example of port numbering with 4-port networks.

The *scikit-rf* cascade ** 2N-port operator use second scheme. This is very convenient to write compact deembedding and other expressions.

numbering diagram:

```
port_order = 'first'
     +---------+
    -|0       1|-
    -|2       3|-
     +---------+

port_order = 'second'
     +---------+
    -|0       2|-
    -|1       3|-
     +---------+

port_order = 'third'
     +---------+
    -|0       3|-
    -|1       2|-
     +---------+
```

use *Network.renumber* to change port ordering.

Example

```
>>> import skrf as rf
>>> from skrf.calibration import IEEEP370_MM_NZC_2xThru
```

Create network objects for 2xThru and FIX-DUT-FIX

```
>>> s2xthru = rf.Network('2xthru.s4p')
>>> fdf = rf.Network('f-dut-f.s4p')
```

Create de-embedding object

```
>>> dm = IEEEP370_MM_NZC_2xThru(dummy_2xthru = s2xthru, name = '2xthru')
```

Apply deembedding to get the actual DUT network

```
>>> dut = dm.deembed(fdf)
```

Note

numbering diagram:

```
 FIX-1_2  DUT      FIX-3_4
 +----+   +----+   +----+
-|1  3|---|1  3|---|3  1|-
-|2  4|---|2  4|---|4  2|-
 +----+   +----+   +----+
```

> **Warning**
>
> There are two differences compared to the original matlab implementation [I3E370]: - FIX-2 is flipped (see diagram above) - A more robust root choice solution is used that avoids the apparition of 180° phase jumps in the fixtures in certain circumstances

References

Parameters

- **dummy_2xthru** (*Network*)
- **name** (*str*)
- **z0** (*float*)
- **port_order** (*PortOrderT*)
- **use_z_instead_ifft** (*bool*)
- **verbose** (*bool*)
- **forced_z0_line_dd** (*float*)
- **forced_z0_line_cc** (*float*)

Methods

COM_receiver_noise_filter	receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A
DC	Advanced reflective DC point extrapolation.
NRP	Enforce the Nyquist Rate Point.
__init__	IEEEP370_MM_NZC_2xThru De-embedding Initializer
add_dc	Extrapolate a network to DC using interpolation for all S-parameters.
dc_interp	enforces symmetric upon the first 10 points and interpolates the DC point.
deembed	Perform the de-embedding calculation
extrapolate_to_dc	Extrapolate the network to DC using IEEE370 NZC algorithm.
getz	Compute step response to get the time-domain impedance from S-parameters.
makeStep	Make a time-domain step response from an impulse response.
makeTL	Compute the S-parameters of a transmission line.
peelNPointsLossless	Peel N points of the network on both side and return the corresponding error boxes.
plot_check_impedance	
plot_check_residuals	
shiftNPoints	Shift the whole network of N samples in time-domain.
shiftOnePort	Shift one port of the network of N samples in time-domain.
split2xthru	Perform the fixtures extraction.
thru	Create a perfect thru

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.COM_receiver_noise_filter

static IEEEP370_MM_NZC_2xThru.**COM_receiver_noise_filter**(*f*, *fr*)

receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A

Parameters

- **f** (*ndarray*)
- **fr** (*float*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.DC

static IEEEP370_MM_NZC_2xThru.**DC**(*s*, *f*, *allowedError=1e-12*)

Advanced reflective DC point extrapolation.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)
- **allowedError** (*float*)

Return type
float

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.NRP

static IEEEP370_MM_NZC_2xThru.**NRP**(*ntwk*, *TD=None*, *port=None*)

Enforce the Nyquist Rate Point. Force the length of the transmissive network to be an integer multiple of the wavelength at the highest frequency. If required, a proper delay is added to meet this condition. The function can also be used to remove the delay.

Parameters

- **ntwk** (*Network* object) – Network to be extrapolated to DC
- **TD** (*:array-like*) – If None, the delay will be computed and added. Else, the delay will be removed (to reset the original length). (default: None)
- **port** (*:Number*) – Specify to apply NRP only on a single port of the network (default: None)

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.__init__

IEEEP370_MM_NZC_2xThru.**__init__**(*dummy_2xthru*, *name=None*, *z0=50*, *port_order='second'*, *use_z_instead_ifft=False*, *verbose=False*, *forced_z0_line_dd=None*, *forced_z0_line_cc=None*, **args*, ***kwargs*)

IEEEP370_MM_NZC_2xThru De-embedding Initializer

Parameters

- **dummy_2xthru** (*Network* object) – 2xThru (FIX-FIX) network.

- **z0** (*float*) – reference impedance of the S-parameters (default: 50)
- **port_order** (*['first', 'second', 'third']*) – specify what numbering scheme to use. See above. (default: second)
- **name** (*string*) – optional name of de-embedding object
- **use_z_instead_ifft** (*bool*) – use z-transform instead ifft. This method is not documented in the paper but exists in the IEEE repo. It could be used if the 2x-Thru is so short that there is not enough points in time domain to determine the length of half fixtures from the s21 impulse response and the the impedance at split plane from the s11 step response. Parameter *verbose* could be used for diagnostic in ifft mode. (default: False)
- **forced_z0_line_dd** (*float*) – If specified, the value for the split plane impedance is forced to *forced_z0_line* for differential-mode. The IEEEP370 standard recommends the 2x-Thru being at least three wavelengths at the highest measured frequency. This ensures that the split plane impedance measured in the S11 step response is free of reflections from the launches. If the 2x-Thru is too short, any point in the s11 step response contain reflections from the lanches and split plane impedance cannot be determined accurately by this method. In this case, setting the impedance manually can improve the results. However, it should be noted that each fixture model will still include some reflections from the opposite side launch because there is not enough time resolution to separate them. (Default: None)
- **forced_z0_line_cc** (*float*) – Same behaviour as *forced_z0_line_dd*, but for the common-mode split plane impedance. (Default: None)
- **verbose** (*bool*) – view the process (default: False)
- **args** – passed to *Deembedding.__init__()*
- **kwargs** – passed to *Deembedding.__init__()*

Return type
None

See also

Deembedding.__init__()

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.add_dc

static IEEEP370_MM_NZC_2xThru.**add_dc**(*ntwk*)

Extrapolate a network to DC using interpolation for all S-parameters.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
out – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.dc_interp

static IEEEP370_MM_NZC_2xThru.**dc_interp**(*s*, *f*)

enforces symmetric upon the first 10 points and interpolates the DC point.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)

Return type
float

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.deembed

IEEEP370_MM_NZC_2xThru.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters
ntwk (*Network* object) – FIX-DUT-FIX network from which FIX-1_2 and FIX-3_4 fixtures needs to be removed via de-embedding

Returns
caled – Network data of the device after de-embedding

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.extrapolate_to_dc

static IEEEP370_MM_NZC_2xThru.**extrapolate_to_dc**(*ntwk*)

Extrapolate the network to DC using IEEE370 NZC algorithm. This is useful to compare the fixtures and deembedded networks to the input data in the same conditions used by NZC algorithm. If the network already have a DC point, it will be replaced.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
ntwk_dc – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.getz

static IEEEP370_MM_NZC_2xThru.**getz**(*s*, *f*, *z0*)

Compute step response to get the time-domain impedance from S-parameters. The S-parameters are DC extrapolated first.

Parameters

- **s** (*:array-like*) – 1-Port S-parameters array
- **f** (*:array-like*) – Frequency array for DC extrapolation
- **z0** (*:array-like*) – Reference impedance

Returns
z – Time-domain impedance step response

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.makeStep

static IEEEP370_MM_NZC_2xThru.**makeStep**(*impulse*)

Make a time-domain step response from an impulse response.

Parameters
impulse (*ndarray*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.makeTL

static IEEEP370_MM_NZC_2xThru.**makeTL**(*zline*, *z0*, *gamma*, *l*)

Compute the S-parameters of a transmission line.

Parameters

- **zline** (*:number*) – Characteristic impedance
- **z0** (*:number*) – Port impedance to renormalize into
- **gamma** (*:array-like*) – Frequency-dependent propagation constant
- **l** (*:number*) – Length in the same length unit as gamma

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.peelNPointsLossless

static IEEEP370_MM_NZC_2xThru.**peelNPointsLossless**(*ntwk*, *N*, *z0*)

Peel N points of the network on both side and return the corresponding error boxes. This is done in a lossless way without determination of the propagation constant gamma.

Parameters

- **ntwk** (*Network* object) – Network to be peeled
- **N** (*:number*) – Number of points to peel
- **z0** (*:number*) – Reference impedance
- **gamma** (*:array-like*) – Frequency-dependent propagation constant

Returns

- **out** (*Network* object) – Peeled network
- **out** (*Network* object) – Error box side port 1
- **out** (*Network* object) – Error box side port 2

Return type
Network

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.plot_check_impedance

IEEEP370_MM_NZC_2xThru.**plot_check_impedance**(*fix_dut_fix=None*, *ax=None*, *window='hamming'*)

Parameters

- **fix_dut_fix** (Network)
- **ax** (*Axes*)
- **window** (*str*)

Return type
(Figure, Axes)

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.plot_check_residuals

IEEEP370_MM_NZC_2xThru.**plot_check_residuals**(*ax=None*)

Parameters
ax (*Axes*)

Return type
(Figure, Axes)

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.shiftNPoints

static IEEEP370_MM_NZC_2xThru.**shiftNPoints**(*ntwk*, *N*)

Shift the whole network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.shiftOnePort

static IEEEP370_MM_NZC_2xThru.**shiftOnePort**(*ntwk*, *N*, *port*)

Shift one port of the network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift
- **port** (*:Number*) – Port to be shifted

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.split2xthru

IEEEP370_MM_NZC_2xThru.**split2xthru**(*se_2xthru*)

Perform the fixtures extraction.

Parameters
se_2xthru (Network)

Return type
(*Network*, *Network*)

skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru.thru

static IEEEP370_MM_NZC_2xThru.**thru**(*ntwk*)

Create a perfect thru

Parameters
ntwk (*Network* object) – Network from which copy frequency, z0 and other parameters. The S-parameters will be replaced by zero-length matched lossless thru.

Returns
out – Network of the perfect thru

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru

class skrf.calibration.deembedding.**IEEEP370_SE_ZC_2xThru**(*dummy_2xthru*, *dummy_fix_dut_fix*, *name=None*, *z0=50*, *bandwidth_limit=0*, *pullback1=0*, *pullback2=0*, *side1=True*, *side2=True*, *NRP_enable=True*, *leadin=1*, *verbose=False*, **args*, ***kwargs*)

Creates error boxes from 2x-Thru and FIX-DUT-FIX networks.

Based on [I3E370].

A deembedding object is created with 2x-Thru (FIX-FIX) and FIX-DUT-FIX measurements, which are split into left (FIX-1) and right (FIX-2) fixtures with IEEEP370 Zc2xThru method.

When *Deembedding.deembed()* is applied, the s-parameters of FIX-1 and FIX-2 are deembedded from FIX_DUT_FIX network.

This method is applicable only when there is 2xThru and FIX_DUT_FIX networks.

The possible difference of impedance between 2x-Thru and FIX-DUT-FIX is corrected.

The algorithm computes the length of the fixtures by halving the delay of 2x-Thru in time domain transmission. The propagation constant gamma is also determined from the 2xThru. It then peels the FIX-DUT-FIX time domain impedance profile iteratively in cycles of determining start impedance and deembedding a single time sample long transmission line.

Example

```
>>> import skrf as rf
>>> from skrf.calibration import IEEEP370_SE_ZC_2xThru
```

Create network objects for 2xThru and FIX-DUT-FIX

```
>>> s2xthru = rf.Network('2xthru.s2p')
>>> fdf = rf.Network('f-dut-f.s2p')
```

Create de-embedding object

```
>>> dm = IEEEP370_SE_ZC_2xThru(dummy_2xthru = s2xthru, dummy_fix_dut_fix = fdf,
                           bandwidth_limit = 10e9,
                           pullback1 = 0, pullback2 = 0,
                           leadin = 0,
                           name = 'zc2xthru')
```

Apply deembedding to get the DUT

```
>>> dut = dm.deembed(fdf)
```

Note

numbering diagram:

```
 FIX-1     DUT       FIX-2
 +----+    +----+    +----+
-|1  2|---|1  2|---|2  1|-
 +----+    +----+    +----+
```

Warning

There is one difference compared to the original matlab implementation [I3E370]: - FIX-2 is flipped (see diagram above)

References

Parameters

- **dummy_2xthru** (Network)
- **dummy_fix_dut_fix** (Network)
- **name** (*str*)
- **z0** (*float*)
- **bandwidth_limit** (*float*)
- **pullback1** (*int*)
- **pullback2** (*int*)
- **side1** (*bool*)
- **side2** (*bool*)
- **NRP_enable** (*bool*)
- **leadin** (*int*)
- **verbose** (*bool*)

Methods

COM_receiver_noise_filter	receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A
DC	Advanced reflective DC point extrapolation.
NRP	Enforce the Nyquist Rate Point.
__init__	IEEEP370_SE_ZC_2xThru De-embedding Initializer
add_dc	Extrapolate a network to DC using interpolation for all S-parameters.
dc_interp	enforces symmetric upon the first 10 points and interpolates the DC point.
deembed	Perform the de-embedding calculation
extrapolate_to_dc	Extrapolate the network to DC using IEEE370 NZC algorithm.
getz	Compute step response to get the time-domain impedance from S-parameters.
makeErrorBox_v7	Extract the fixtures on both sides.
makeErrorBox_v8	Extract the fixture only on a single side.
makeStep	Make a time-domain step response from an impulse response.
makeTL	Compute the S-parameters of a transmission line.
peelNPointsLossless	Peel N points of the network on both side and return the corresponding error boxes.
plot_check_impedance	
plot_check_residuals	
shiftNPoints	Shift the whole network of N samples in time-domain.
shiftOnePort	Shift one port of the network of N samples in time-domain.
split2xthru	Perform the fixtures extraction.
thru	Create a perfect thru

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.COM_receiver_noise_filter

static IEEEP370_SE_ZC_2xThru.**COM_receiver_noise_filter**(*f*, *fr*)

receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A

Parameters

- **f** (*ndarray*)
- **fr** (*float*)

Return type

ndarray

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.DC

static IEEEP370_SE_ZC_2xThru.**DC**(*s*, *f*, *allowedError=1e-12*)

Advanced reflective DC point extrapolation.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)

- **allowedError** (*float*)

Return type
float

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.NRP

static IEEEP370_SE_ZC_2xThru.**NRP**(*ntwk*, *TD=None*, *port=None*)

Enforce the Nyquist Rate Point. Force the length of the transmissive network to be an integer multiple of the wavelength at the highest frequency. If required, a proper delay is added to meet this condition. The function can also be used to remove the delay.

Parameters

- **ntwk** (*Network* object) – Network to be extrapolated to DC
- **TD** (*:array-like*) – If None, the delay will be computed and added. Else, the delay will be removed (to reset the original length). (default: None)
- **port** (*:Number*) – Specify to apply NRP only on a single port of the network (default: None)

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.__init__

IEEEP370_SE_ZC_2xThru.**__init__**(*dummy_2xthru*, *dummy_fix_dut_fix*, *name=None*, *z0=50*, *bandwidth_limit=0*, *pullback1=0*, *pullback2=0*, *side1=True*, *side2=True*, *NRP_enable=True*, *leadin=1*, *verbose=False*, **args*, ***kwargs*)

IEEEP370_SE_ZC_2xThru De-embedding Initializer

Parameters

- **dummy_2xthru** (*Network* object) – 2xThru (FIX-FIX) network.
- **name** (*string*) – optional name of de-embedding object
- **z0** (*float*) – reference impedance of the S-parameters (default: 50)
- **bandwidth_limit** (*float*) – max frequency for a fitting function (default: 0, use all s-parameters without fit)
- **pullback1** (*int*) – a number of discrete points to leave in the fixture on side 1 respectively on side 2 (default: 0 leave all)
- **pullback2** (*int*) – a number of discrete points to leave in the fixture on side 1 respectively on side 2 (default: 0 leave all)
- **side1** (*bool*) – set to de-embed the side1 resp. side2 errorbox (default: True)
- **side2** (*bool*) – set to de-embed the side1 resp. side2 errorbox (default: True)
- **NRP_enable** (*bool*) – set to enforce the Nyquist Rate Point during de-embedding and to add the appropriote delay to the errorboxes (default: True)
- **leadin** (*int*) – a number of discrete points before t = 0 that are non-zero from calibration error (default: 1)
- **verbose** (*bool*) – view the process (default: False)

- **args** – passed to *Deembedding.__init__()*
- **kwargs** – passed to *Deembedding.__init__()*
- **dummy_fix_dut_fix** (Network)

Return type
None

See also

Deembedding.__init__()

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.add_dc

static IEEEP370_SE_ZC_2xThru.**add_dc**(*ntwk*)

Extrapolate a network to DC using interpolation for all S-parameters.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
out – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.dc_interp

static IEEEP370_SE_ZC_2xThru.**dc_interp**(*s*, *f*)

enforces symmetric upon the first 10 points and interpolates the DC point.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)

Return type
float

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.deembed

IEEEP370_SE_ZC_2xThru.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters
ntwk (*Network* object) – FIX-DUT-FIX network from which FIX-1 and FIX-2 fixtures needs to be removed via de-embedding

Returns
caled – Network data of the device after de-embedding

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.extrapolate_to_dc

static IEEEP370_SE_ZC_2xThru.**extrapolate_to_dc**(*ntwk*)

Extrapolate the network to DC using IEEE370 NZC algorithm. This is useful to compare the fixtures and deembedded networks to the input data in the same conditions used by NZC algorithm. If the network already have a DC point, it will be replaced.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
ntwk_dc – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.getz

static IEEEP370_SE_ZC_2xThru.**getz**(*s*, *f*, *z0*)

Compute step response to get the time-domain impedance from S-parameters. The S-parameters are DC extrapolated first.

Parameters

- **s** (*:array-like*) – 1-Port S-parameters array
- **f** (*:array-like*) – Frequency array for DC extrapolation
- **z0** (*:array-like*) – Reference impedance

Returns
z – Time-domain impedance step response

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.makeErrorBox_v7

IEEEP370_SE_ZC_2xThru.**makeErrorBox_v7**(*s_dut*, *s2x*, *gamma*, *z0*, *pullback*)

Extract the fixtures on both sides.

Parameters

- **s_dut** (Network)
- **s2x** (Network)
- **gamma** (*ndarray*)
- **z0** (*float*)
- **pullback** (*int*)

Return type
(*Network*, *Network*)

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.makeErrorBox_v8

IEEEP370_SE_ZC_2xThru.**makeErrorBox_v8**(*s_dut*, *s2x*, *gamma*, *z0*, *pullback*)

Extract the fixture only on a single side.

Parameters

- **s_dut** (Network)
- **s2x** (Network)
- **gamma** (*ndarray*)
- **z0** (*float*)
- **pullback** (*int*)

Return type
Network

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.makeStep

static IEEEP370_SE_ZC_2xThru.**makeStep**(*impulse*)

Make a time-domain step response from an impulse response.

Parameters
impulse (*ndarray*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.makeTL

static IEEEP370_SE_ZC_2xThru.**makeTL**(*zline*, *z0*, *gamma*, *l*)

Compute the S-parameters of a transmission line.

Parameters

- **zline** (*:number*) – Characteristic impedance
- **z0** (*:number*) – Port impedance to renormalize into
- **gamma** (*:array-like*) – Frequency-dependent propagation constant
- **l** (*:number*) – Length in the same length unit as gamma

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.peelNPointsLossless

static IEEEP370_SE_ZC_2xThru.**peelNPointsLossless**(*ntwk*, *N*, *z0*)

Peel N points of the network on both side and return the corresponding error boxes. This is done in a lossless way without determination of the propagation constant gamma.

Parameters

- **ntwk** (*Network* object) – Network to be peeled
- **N** (*:number*) – Number of points to peel
- **z0** (*:number*) – Reference impedance
- **gamma** (*:array-like*) – Frequency-dependent propagation constant

Returns

- **out** (*Network* object) – Peeled network

- **out** (*Network* object) – Error box side port 1
- **out** (*Network* object) – Error box side port 2

Return type
Network

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.plot_check_impedance

IEEEP370_SE_ZC_2xThru.**plot_check_impedance**(*fix_dut_fix=None*, *ax=None*, *window='hamming'*)

Parameters

- **fix_dut_fix** (Network)
- **ax** (*Axes*)
- **window** (*str*)

Return type
(Figure, Axes)

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.plot_check_residuals

IEEEP370_SE_ZC_2xThru.**plot_check_residuals**(*ax=None*)

Parameters
ax (*Axes*)

Return type
(Figure, Axes)

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.shiftNPoints

static IEEEP370_SE_ZC_2xThru.**shiftNPoints**(*ntwk*, *N*)

Shift the whole network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.shiftOnePort

static IEEEP370_SE_ZC_2xThru.**shiftOnePort**(*ntwk*, *N*, *port*)

Shift one port of the network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift
- **port** (*:Number*) – Port to be shifted

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.split2xthru

IEEEP370_SE_ZC_2xThru.**split2xthru**(*s2xthru*, *sfix_dut_fix*)

Perform the fixtures extraction.

Parameters

- **s2xthru** (Network)
- **sfix_dut_fix** (Network)

Return type
(*Network*, *Network*)

skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru.thru

static IEEEP370_SE_ZC_2xThru.**thru**(*ntwk*)

Create a perfect thru

Parameters
ntwk (*Network* object) – Network from which copy frequency, z0 and other parameters. The S-parameters will be replaced by zero-length matched lossless thru.

Returns
out – Network of the perfect thru

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru

class skrf.calibration.deembedding.**IEEEP370_MM_ZC_2xThru**(*dummy_2xthru*, *dummy_fix_dut_fix*, *name=None*, *z0=50*, *port_order='second'*, *bandwidth_limit=0*, *pullback1=0*, *pullback2=0*, *side1=True*, *side2=True*, *NRP_enable=True*, *leadin=1*, *verbose=False*, **args*, ***kwargs*)

Creates error boxes from a 4-port from 2x-Thru and FIX-DUT-FIX networks.

Based on [I3E370].

A deembedding object is created with 2x-Thru (FIX-FIX) and FIX-DUT-FIX measurements, which are split into left (FIX-1_2) and right (FIX-3_4) fixtures with IEEEP370 Zc2xThru method.

When *Deembedding.deembed()* is applied, the s-parameters of FIX-1_2 and FIX-3_4 are deembedded from FIX_DUT_FIX network.

This method is applicable only when there is 2xThru and FIX_DUT_FIX networks.

The possible difference of impedance between 2x-Thru and FIX-DUT-FIX is corrected.

The algorithm computes the length of the fixtures by halving the delay of 2x-Thru in time domain transmission. The propagation constant gamma is also determined from the 2xThru. It then peels the FIX-DUT-FIX time domain impedance profile iteratively in cycles of determining start impedance and deembedding a single time sample long transmission line.

Note

The *port_order* ='first', means front-to-back also known as odd/even, while `port_order`='second' means left-to-right also known as sequential. `port_order`='third' means yet another numbering method. Next figure show example of port numbering with 4-port networks.

The *scikit-rf* cascade ** 2N-port operator use second scheme. This is very convenient to write compact deembedding and other expressions.

numbering diagram:

```
port_order = 'first'
     +---------+
    -|0       1|-
    -|2       3|-
     +---------+

port_order = 'second'
     +---------+
    -|0       2|-
    -|1       3|-
     +---------+

port_order = 'third'
     +---------+
    -|0       3|-
    -|1       2|-
     +---------+
```

use *Network.renumber* to change port ordering.

Example

```
>>> import skrf as rf
>>> from skrf.calibration import IEEEP370_MM_ZC_2xThru
```

Create network objects for 2xThru and FIX-DUT-FIX

```
>>> s2xthru = rf.Network('2xthru.s4p')
>>> fdf = rf.Network('f-dut-f.s4p')
```

Create de-embedding object

```
>>> dm = IEEEP370_MM_ZC_2xThru(dummy_2xthru = s2xthru,
                           dummy_fix_dut_fix = fdf,
                           bandwidth_limit = 10e9,
                           pullback1 = 0, pullback2 = 0,
                           leadin = 0,
                           name = 'zc2xthru')
```

Apply deembedding to get the DUT

```
>>> dut = dm.deembed(fdf)
```

Note

numbering diagram:

```
 FIX-1_2  DUT      FIX-3_4
  +----+   +----+   +----+
-|1  3|---|1  3|---|3  1|-
-|2  4|---|2  4|---|4  2|-
  +----+   +----+   +----+
```

Warning

There is one difference compared to the original matlab implementation [I3E370]:

- FIX-2 is flipped (see diagram above)

References

Parameters

- **dummy_2xthru** (Network)
- **dummy_fix_dut_fix** (Network)
- **name** (*str*)
- **z0** (*float*)
- **port_order** (*PortOrderT*)
- **bandwidth_limit** (*float*)
- **pullback1** (*int*)
- **pullback2** (*int*)
- **side1** (*bool*)
- **side2** (*bool*)
- **NRP_enable** (*bool*)
- **leadin** (*int*)
- **verbose** (*bool*)

Methods

COM_receiver_noise_filter	receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A
DC	Advanced reflective DC point extrapolation.
NRP	Enforce the Nyquist Rate Point.
__init__	IEEEP370_MM_ZC_2xThru De-embedding Initializer
add_dc	Extrapolate a network to DC using interpolation for all S-parameters.
dc_interp	enforces symmetric upon the first 10 points and interpolates the DC point.

continues on next page

Table 127 – continued from previous page

deembed	Perform the de-embedding calculation
extrapolate_to_dc	Extrapolate the network to DC using IEEE370 NZC algorithm.
getz	Compute step response to get the time-domain impedance from S-parameters.
makeStep	Make a time-domain step response from an impulse response.
makeTL	Compute the S-parameters of a transmission line.
peelNPointsLossless	Peel N points of the network on both side and return the corresponding error boxes.
plot_check_impedance	
plot_check_residuals	
shiftNPoints	Shift the whole network of N samples in time-domain.
shiftOnePort	Shift one port of the network of N samples in time-domain.
split2xthru	Perform the fixtures extraction.
thru	Create a perfect thru

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.COM_receiver_noise_filter

static IEEEP370_MM_ZC_2xThru.**COM_receiver_noise_filter**(*f*, *fr*)

receiver filter in COM defined by eq 93A-20 As defined in 802.3-2022 - IEEE Standard for Ethernet annex 93A

Parameters

- **f** (*ndarray*)
- **fr** (*float*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.DC

static IEEEP370_MM_ZC_2xThru.**DC**(*s*, *f*, *allowedError=1e-12*)

Advanced reflective DC point extrapolation.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)
- **allowedError** (*float*)

Return type
float

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.NRP

static IEEEP370_MM_ZC_2xThru.**NRP**(*ntwk*, *TD=None*, *port=None*)

Enforce the Nyquist Rate Point. Force the length of the transmissive network to be an integer multiple of the wavelength at the highest frequency. If required, a proper delay is added to meet this condition. The function can also be used to remove the delay.

Parameters

- **ntwk** (*Network* object) – Network to be extrapolated to DC
- **TD** (*:array-like*) – If None, the delay will be computed and added. Else, the delay will be removed (to reset the original length). (default: None)
- **port** (*:Number*) – Specify to apply NRP only on a single port of the network (default: None)

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.__init__

IEEEP370_MM_ZC_2xThru.**__init__**(*dummy_2xthru*, *dummy_fix_dut_fix*, *name=None*, *z0=50*, *port_order='second'*, *bandwidth_limit=0*, *pullback1=0*, *pullback2=0*, *side1=True*, *side2=True*, *NRP_enable=True*, *leadin=1*, *verbose=False*, **args*, ***kwargs*)

IEEEP370_MM_ZC_2xThru De-embedding Initializer

Parameters

- **dummy_2xthru** (*Network* object) – 2xThru (FIX-FIX) network.
- **name** (*string*) – optional name of de-embedding object
- **z0** (*float*) – reference impedance of the S-parameters (default: 50)
- **port_order** (*['first', 'second', 'third']*) – specify what numbering scheme to use. See above. (default: second)
- **bandwidth_limit** (*float*) – max frequency for a fitting function (default: 0, use all s-parameters without fit)
- **pullback1** (*int*) – a number of discrete points to leave in the fixture on side 1 respectively on side 2 (default: 0 leave all)
- **pullback2** (*int*) – a number of discrete points to leave in the fixture on side 1 respectively on side 2 (default: 0 leave all)
- **side1** (*bool*) – set to de-embed the side1 resp. side2 errorbox (default: True)
- **side2** (*bool*) – set to de-embed the side1 resp. side2 errorbox (default: True)
- **NRP_enable** (*bool*) – set to enforce the Nyquist Rate Point during de-embedding and to add the appropriote delay to the errorboxes (default: True)
- **leadin** (*int*) – a number of discrete points before t = 0 that are non-zero from calibration error (default: 1)
- **verbose** (*bool*) – view the process (default: False)
- **args** – passed to *Deembedding.__init__()*
- **kwargs** – passed to *Deembedding.__init__()*
- **dummy_fix_dut_fix** (Network)

Return type
None

> **See also**
>
> *Deembedding.__init__()*

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.add_dc

static IEEEP370_MM_ZC_2xThru.**add_dc**(*ntwk*)

Extrapolate a network to DC using interpolation for all S-parameters.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
out – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.dc_interp

static IEEEP370_MM_ZC_2xThru.**dc_interp**(*s*, *f*)

enforces symmetric upon the first 10 points and interpolates the DC point.

Parameters

- **s** (*ndarray*)
- **f** (*ndarray*)

Return type
float

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.deembed

IEEEP370_MM_ZC_2xThru.**deembed**(*ntwk*)

Perform the de-embedding calculation

Parameters
ntwk (*Network* object) – FIX-DUT-FIX network from which FIX-1_2 and FIX-3_4 fixtures needs to be removed via de-embedding

Returns
caled – Network data of the device after de-embedding

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.extrapolate_to_dc

static IEEEP370_MM_ZC_2xThru.**extrapolate_to_dc**(*ntwk*)

Extrapolate the network to DC using IEEE370 NZC algorithm. This is useful to compare the fixtures and deembedded networks to the input data in the same conditions used by NZC algorithm. If the network already have a DC point, it will be replaced.

Parameters
ntwk (*Network* object) – Network to be extrapolated to DC

Returns
ntwk_dc – Network with DC point

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.getz

static IEEEP370_MM_ZC_2xThru.**getz**(*s*, *f*, *z0*)

Compute step response to get the time-domain impedance from S-parameters. The S-parameters are DC extrapolated first.

Parameters

- **s** (*:array-like*) – 1-Port S-parameters array
- **f** (*:array-like*) – Frequency array for DC extrapolation
- **z0** (*:array-like*) – Reference impedance

Returns
z – Time-domain impedance step response

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.makeStep

static IEEEP370_MM_ZC_2xThru.**makeStep**(*impulse*)

Make a time-domain step response from an impulse response.

Parameters
impulse (*ndarray*)

Return type
ndarray

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.makeTL

static IEEEP370_MM_ZC_2xThru.**makeTL**(*zline*, *z0*, *gamma*, *l*)

Compute the S-parameters of a transmission line.

Parameters

- **zline** (*:number*) – Characteristic impedance
- **z0** (*:number*) – Port impedance to renormalize into
- **gamma** (*:array-like*) – Frequency-dependent propagation constant
- **l** (*:number*) – Length in the same length unit as gamma

Returns
TL – S_Parameters of the transmission line

Return type
:array-like

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.peelNPointsLossless

static IEEEP370_MM_ZC_2xThru.**peelNPointsLossless**(*ntwk*, *N*, *z0*)

Peel N points of the network on both side and return the corresponding error boxes. This is done in a lossless way without determination of the propagation constant gamma.

Parameters

- **ntwk** (*Network* object) – Network to be peeled
- **N** (*:number*) – Number of points to peel
- **z0** (*:number*) – Reference impedance
- **gamma** (*:array-like*) – Frequency-dependent propagation constant

Returns

- **out** (*Network* object) – Peeled network
- **out** (*Network* object) – Error box side port 1
- **out** (*Network* object) – Error box side port 2

Return type
Network

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.plot_check_impedance

IEEEP370_MM_ZC_2xThru.**plot_check_impedance**(*fix_dut_fix=None*, *ax=None*, *window='hamming'*)

Parameters

- **fix_dut_fix** (Network)
- **ax** (*Axes*)
- **window** (*str*)

Return type
(Figure, Axes)

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.plot_check_residuals

IEEEP370_MM_ZC_2xThru.**plot_check_residuals**(*ax=None*)

Parameters
ax (*Axes*)

Return type
(Figure, Axes)

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.shiftNPoints

static IEEEP370_MM_ZC_2xThru.**shiftNPoints**(*ntwk*, *N*)

Shift the whole network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.shiftOnePort

static IEEEP370_MM_ZC_2xThru.**shiftOnePort**(*ntwk*, *N*, *port*)

Shift one port of the network of N samples in time-domain. This is achieved by cascading a delay.

Parameters

- **ntwk** (*Network* object) – Network to be shifted
- **N** (*:number*) – Number of point to shift
- **port** (*:Number*) – Port to be shifted

Returns
out – Shifted network

Return type
Network object

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.split2xthru

IEEEP370_MM_ZC_2xThru.**split2xthru**(*se_2xthru*, *se_fdf*)

Perform the fixtures extraction.

Parameters

- **se_2xthru** (Network)
- **se_fdf** (Network)

Return type
(*Network*, *Network*)

skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru.thru

static IEEEP370_MM_ZC_2xThru.**thru**(*ntwk*)

Create a perfect thru

Parameters
ntwk (*Network* object) – Network from which copy frequency, z0 and other parameters. The S-parameters will be replaced by zero-length matched lossless thru.

Returns
out – Network of the perfect thru

Return type
Network object

3.9 virtual instruments (*skrf.vi*)

This module defines "virtual instruments" or interfaces to a number of different types of instruments used in RF measurements and experiments.

3.9.1 Creating A Driver

Drivers are classes inherited from the respective instrument type's abstract base class and define methods specific to each instrument. This syntax can be complex and vary wildly between devices and manufacturers. The vi module attempts to present a standard interface for each.

Basically, creating a driver consists of the following:

1. Creating an instrument class that inherits from its abstract base class
2. Consulting the device's datasheet to find the command for each method in the abstract base class

Those wishing to use an instrument that's not supported are encouraged to consult existing classes to understand the architecture, develop their own class, and contribute it to scikit-rf.

If you choose to do so, please try to use the following standard terms for the associated meaning

Conventions

Name	Meaning
frequency	Frequency settings [`skrf.Frequency`]
freq_start	Start frequency [Hz]
freq_stop	Stop frequency [Hz]
freq_step	Frequency step [Hz]
freq_center	Center frequency [Hz]
freq_span	Frequency span [Hz]
npoints	Number of frequency points
sweep_time	Time taken for a frequency sweep [s]
sweep_type	How frequency points are distributed (linear,log,etc.)
sweep_mode	How the instrument is triggered (continuous,single,etc)
averaging_on	Whether averaging is on or not
averaging_count	The number of measurements combined for an average
averaging_mode	How averages are taken (per point, each sweep, etc)
if_bandwidth	IF bandwidth [Hz]

Validators

validators.IntValidator([min, max])	
validators.FloatValidator([min, max, ...])	
validators.FreqValidator()	
validators.EnumValidator(enum)	
validators.SetValidator(valid)	
validators.DictValidator(arg_string, ...)	
validators.DelimitedStrValidator([dtype, sep])	
validators.BooleanValidator([true_response, ...])	

skrf.vi.validators.IntValidator

class skrf.vi.validators.**IntValidator**(*min=None*, *max=None*)

Parameters

- **min** (*int | None*)

- **max** (*int | None*)

Methods

__init__	
check_bounds	
validate_input	Try to convert arg to a valid value and return.
validate_output	Try to convert arg to a valid value and return.

skrf.vi.validators.IntValidator.__init__

IntValidator.**__init__**(*min=None*, *max=None*)

Parameters

- **min** (*int | None*)
- **max** (*int | None*)

Return type
None

skrf.vi.validators.IntValidator.check_bounds

IntValidator.**check_bounds**(*arg*)

skrf.vi.validators.IntValidator.validate_input

IntValidator.**validate_input**(*arg*)

Try to convert arg to a valid value and return. Used to ensure commands sent to the instrument are the proper format.

Return type
int

skrf.vi.validators.IntValidator.validate_output

IntValidator.**validate_output**(*arg*)

Try to convert arg to a valid value and return. Used to convert responses from the instrument to something useful. By default, this just calls validate_input, but can be overwritten in subclasses. (See EnumValidator)

Return type
Any

skrf.vi.validators.FloatValidator

class skrf.vi.validators.**FloatValidator**(*min=None*, *max=None*, *decimal_places=50*)

Parameters

- **min** (*float | None*)
- **max** (*float | None*)
- **decimal_places** (*int*)

Methods

__init__	
check_bounds	
validate_input	Try to convert arg to a valid value and return.
validate_output	Try to convert arg to a valid value and return.

skrf.vi.validators.FloatValidator.__init__

FloatValidator.**__init__**(*min=None*, *max=None*, *decimal_places=50*)

Parameters

- **min** (*float | None*)
- **max** (*float | None*)
- **decimal_places** (*int*)

Return type
None

skrf.vi.validators.FloatValidator.check_bounds

FloatValidator.**check_bounds**(*arg*)

skrf.vi.validators.FloatValidator.validate_input

FloatValidator.**validate_input**(*arg*)

Try to convert arg to a valid value and return. Used to ensure commands sent to the instrument are the proper format.

Return type
float

skrf.vi.validators.FloatValidator.validate_output

FloatValidator.**validate_output**(*arg*)

Try to convert arg to a valid value and return. Used to convert responses from the instrument to something useful. By default, this just calls validate_input, but can be overwritten in subclasses. (See EnumValidator)

Return type
Any

skrf.vi.validators.FreqValidator

class skrf.vi.validators.**FreqValidator**

Attributes

freq_re
si

skrf.vi.validators.FreqValidator.freq_re

FreqValidator.**freq_re** =
re.compile('(?P<val>\\d+\\.?\\d*)\\s*(?P<si_prefix>[kMG])?(?:[hH][zZ])?')

skrf.vi.validators.FreqValidator.si

FreqValidator.**si = {'G': 1000000000.0, 'M': 1000000.0, 'k': 1000.0}**

Methods

__init__	
validate_input	Try to convert arg to a valid value and return.
validate_output	Try to convert arg to a valid value and return.

skrf.vi.validators.FreqValidator.__init__

FreqValidator.**__init__**()

skrf.vi.validators.FreqValidator.validate_input

FreqValidator.**validate_input**(*arg*)

Try to convert arg to a valid value and return. Used to ensure commands sent to the instrument are the proper format.

Return type
int

skrf.vi.validators.FreqValidator.validate_output

FreqValidator.**validate_output**(*arg*)

Try to convert arg to a valid value and return. Used to convert responses from the instrument to something useful. By default, this just calls validate_input, but can be overwritten in subclasses. (See EnumValidator)

Return type
int

skrf.vi.validators.EnumValidator

class skrf.vi.validators.**EnumValidator**(*enum*)

Parameters
enum (*Enum*)

Methods

__init__	
validate_input	Try to convert arg to a valid value and return.
validate_output	Try to convert arg to a valid value and return.

skrf.vi.validators.EnumValidator.__init__

EnumValidator.**__init__**(*enum*)

Parameters
enum (*Enum*)

Return type
None

skrf.vi.validators.EnumValidator.validate_input

EnumValidator.**validate_input**(*arg*)

Try to convert arg to a valid value and return. Used to ensure commands sent to the instrument are the proper format.

Return type
Any

skrf.vi.validators.EnumValidator.validate_output

EnumValidator.**validate_output**(*arg*)

Try to convert arg to a valid value and return. Used to convert responses from the instrument to something useful. By default, this just calls validate_input, but can be overwritten in subclasses. (See EnumValidator)

Return type
Any

skrf.vi.validators.SetValidator

class skrf.vi.validators.**SetValidator**(*valid*)

Parameters
valid (*Sequence*)

Methods

__init__	
validate_input	Try to convert arg to a valid value and return.
validate_output	Try to convert arg to a valid value and return.

skrf.vi.validators.SetValidator.__init__

SetValidator.**__init__**(*valid*)

Parameters
valid (*Sequence*)

Return type
None

skrf.vi.validators.SetValidator.validate_input

SetValidator.**validate_input**(*arg*)

Try to convert arg to a valid value and return. Used to ensure commands sent to the instrument are the proper format.

Return type
Any

skrf.vi.validators.SetValidator.validate_output

SetValidator.**validate_output**(*arg*)

Try to convert arg to a valid value and return. Used to convert responses from the instrument to something useful. By default, this just calls validate_input, but can be overwritten in subclasses. (See EnumValidator)

Return type
Any

skrf.vi.validators.DictValidator

class skrf.vi.validators.**DictValidator**(*arg_string*, *response_pattern*)

Parameters

- **arg_string** (*str*)
- **response_pattern** (*re.Pattern | str*)

Methods

__init__	
validate_input	Try to convert arg to a valid value and return.
validate_output	Try to convert arg to a valid value and return.

skrf.vi.validators.DictValidator.__init__

DictValidator.**__init__**(*arg_string*, *response_pattern*)

Parameters

- **arg_string** (*str*)
- **response_pattern** (*Pattern | str*)

Return type
None

skrf.vi.validators.DictValidator.validate_input

DictValidator.**validate_input**(*arg*)

Try to convert arg to a valid value and return. Used to ensure commands sent to the instrument are the proper format.

Return type
str

skrf.vi.validators.DictValidator.validate_output

DictValidator.**validate_output**(*arg*)

Try to convert arg to a valid value and return. Used to convert responses from the instrument to something useful. By default, this just calls validate_input, but can be overwritten in subclasses. (See EnumValidator)

Return type
dict

skrf.vi.validators.DelimitedStrValidator

class skrf.vi.validators.**DelimitedStrValidator**(*dtype=<class 'str'>*, *sep=','*)

Parameters

- **dtype** (*type*)
- **sep** (*str*)

Methods

__init__	
validate_input	Try to convert arg to a valid value and return.
validate_output	Try to convert arg to a valid value and return.

skrf.vi.validators.DelimitedStrValidator.__init__

DelimitedStrValidator.**__init__**(*dtype=<class 'str'>*, *sep=','*)

Parameters

- **dtype** (*type*)
- **sep** (*str*)

Return type
None

skrf.vi.validators.DelimitedStrValidator.validate_input

DelimitedStrValidator.**validate_input**(*arg*)

Try to convert arg to a valid value and return. Used to ensure commands sent to the instrument are the proper format.

Parameters
arg (*Sequence*)

Return type
str

skrf.vi.validators.DelimitedStrValidator.validate_output

DelimitedStrValidator.**validate_output**(*arg*)

Try to convert arg to a valid value and return. Used to convert responses from the instrument to something useful. By default, this just calls validate_input, but can be overwritten in subclasses. (See EnumValidator)

Parameters
arg (*str*)

Return type
list

skrf.vi.validators.BooleanValidator

class skrf.vi.validators.**BooleanValidator**(*true_response=None*, *false_response=None*, *true_setting='1'*, *false_setting='0'*)

Parameters

- **true_response** (*str | None*)
- **false_response** (*str | None*)
- **true_setting** (*str*)
- **false_setting** (*str*)

Attributes

falsey	
truthy	

skrf.vi.validators.BooleanValidator.falsey

BooleanValidator.**falsey = ['0', 'off', 'false']**

skrf.vi.validators.BooleanValidator.truthy

BooleanValidator.**truthy = ['1', 'on', 'true']**

Methods

__init__	
validate_input	Try to convert arg to a valid value and return.
validate_output	Try to convert arg to a valid value and return.

skrf.vi.validators.BooleanValidator.__init__

BooleanValidator.**__init__**(*true_response=None*, *false_response=None*, *true_setting='1'*, *false_setting='0'*)

Parameters

- **true_response** (*str | None*)
- **false_response** (*str | None*)
- **true_setting** (*str*)
- **false_setting** (*str*)

skrf.vi.validators.BooleanValidator.validate_input

BooleanValidator.**validate_input**(*arg*)

Try to convert arg to a valid value and return. Used to ensure commands sent to the instrument are the proper format.

Return type
str

skrf.vi.validators.BooleanValidator.validate_output

BooleanValidator.**validate_output**(*arg*)

Try to convert arg to a valid value and return. Used to convert responses from the instrument to something useful. By default, this just calls validate_input, but can be overwritten in subclasses. (See EnumValidator)

Return type
bool

3.10 vna (*skrf.vi.vna*)

Provides classes to interact with Vector Network Analyzers (VNAs) from numerous manufacturers.

3.10.1 VNA Classes

hp.HP8510C(address[, backend])	HP 8510 driver that is capable of compound sweeps, segmented sweeps, and fast binary transfers.
keysight.FieldFox(address[, backend])	Keysight FieldFox.
keysight.PNA(address[, backend])	Keysight PNA, PNA-L, PNA-X.
nanovna.NanoVNAv2(address[, backend])	NanoVNAv2.
rohde_schwarz.ZVA(address[, backend])	Rohde & Schwarz ZVA.

3.10.2 The Base Class and Writing an Instrument Driver

All VNAs are derived from the VNA class. This class should never be instantiated directly, but instead serves as a means to run certain setup code depending on the capabilities of the instrument.

- *SCPI*
- *Channels*
- *Property Creator*
- *Validators*

When writing a new instrument driver, the following minimum questions should be considered:

- Does the instrument use SCPI?
- Does the instrument support multiple channels?

SCPI

For those instruments that use SCPI, default universal SCPI methods are included by default. However, for those instruments that do not support SPCI, do the following:

```
class VNAWithoutSCPI(VNA):
    _scpi = False
    # ...
```

Channels

Some instruments support channels and can have multiple measurements with different frequency/power/if bandwidth/etc settings.

For instruments without channel support, the typical properties (freq_start, freq_stop, etc) should be defined on the class. For instruments **with** channel support, the class should include a subclass of `Channel` that defines properties specific to channel settings in the channel definition while other instrument properties should be defined in the instrument class. To make this more clear, consider the following example from the Keysight PNA implementation (the property creator *VNA.Command* is explained later)

```
class PNA(VNA):
    class Channel(vna.Channel):
        freq_start = VNA.Command(
            get_cmd="SENS<self:cnum>:FREQ:STAR?",
            set_cmd="SENS<self:cnum>:FREQ:STAR <arg>",
            doc="""Start frequency [Hz]""",
            validator=FreqValidator()
        )

    def __init__(self, address, backend='@py'):
        # ...

    nports = VNA.Command(
        get_cmd="SYST:CAP:HARD:PORT:COUN?",
        set_cmd=None,
        doc="""Number of ports""",
        validator=IntValidator()
    )
```

Here, the start frequency is set **per channel** whereas the number of ports is related to the instrument itself. Instruments with channel support should create a single channel in *__init__()* using *create_channel*

Channel properties can also be read and set through the instrument, using the currently active channel. For example, these assignments are equivalent:

```
instr.freq_start = 1e9
instr.active_channel.freq_start = 1e9
```

Properties defined on the instrument itself take precedence over channel properties.

Property Creator

Inspired by PyMeasure, skrf has a property creator to simplify creating properties that are queried and set with commands.

static VNA.**command**(*get_cmd=None*, *set_cmd=None*, *doc=None*, *validator=None*, *values=False*, *values_container=<built-in function array>*, *complex_values=False*)

Create a property for the instrument.

This method is used to add a property to an instrument. These properties can be read-only, write-only, read-write, and can validate values before sending to the instrument as well as validate responses from the instrument

to return proper types.

Parameters

- **get_cmd** (*str | None*) – Command sent to the instrument to request data
- **set_cmd** (*str | None*) – Command sent to the instrument to set data
- **doc** (*str | None*) – The docstring for the property
- **validator** (*Validator | None*) – The `Validator` that will be used to transform data to the proper format before sending and after querying
- **values** (*bool*) – Whether or not this command is using a *Sequence* to set data, or expects a *Sequence* in response.
- **values_container** (*type | None*) – If values is true, you set set this to the type of container the values should be returned in. For example, this is np.array by default, meaning instead of return a *list*, you will get a numpy array.
- **complex_values** (*bool*) – If the values expected from the instrument are complex. If so, the values will be converted from [real[0], imag[0], real[1], imag[1], ...] to [complex(real[0], imag[0]), complex(real[1], imag[1]), ...]

Returns

The property constructed from the parameters passed. Should be set to a class variable

Return type

property

For *get_cmd* and *set_cmd*, you can substitute delimiters to be replaced when the call is made. *self* is always passed, so you can reference any part of "self" when making the call. Additionally, the *set_cmd* receives an additional parameter *arg* which is the argument passed to the setter. These can be used in the get/set strings by using angle bracket delimiters and referring to the name. Here are some examples:

Here, we are assuming we are writing a command for an instrument with channels, and writing a command **for** the internal *Channel* class. In *get_cmd*, *<self:cnum>* gets the *self.cnum* property of the Channel class **at runtime**. In *set_cmd*, *<arg>* is replaced by the argument passed to the setter.

```
freq_start = VNA.command(
    get_cmd="CALC<self:cnum>:FREQ:STAR?",
    set_cmd="CALC<self:cnum>:FREQ:STAR <arg>",
)
```

And here's an example of calling this function and what the resultant command strings would be after replacement.

```
_ = instr.ch1.freq_start
# Sends the command CALC1:FREQ:STAR? to the instrument
instr.ch1.freq_start = 100
# Sends the command CALC1:FREQ:STAR 100 to the instrument
```

Validators

Validators are (optionally, but almost always) passed to *VNA.command*. When a property is get or set, the appropriate validate command is called to convert input to the proper format expected by the instrument, or convert responses from the instrument to the proper type.

The current validators are:

- *skrf.vi.validators.BooleanValidator*
- *skrf.vi.validators.DelimitedStrValidator*

- *skrf.vi.validators.DictValidator*
- *skrf.vi.validators.EnumValidator*
- *skrf.vi.validators.IntValidator*
- *skrf.vi.validators.FloatValidator*
- *skrf.vi.validators.FreqValidator*
- *skrf.vi.validators.SetValidator*

The documentation for each of these explains more about their functionality, but in essence when writing a *VNA.command*, consider how the command must be formatted to be sent to the instrument and what the expected response from the instrument will be and how that can be transformed to something more useful than, say, a string.

Here's an example of using a validator:

```
freq_start = VNA.command(
    get_cmd="CALC:FREQ:STAR?",
    set_cmd="CALC:FREQ:STAR <arg>",
    validator=FreqValidator()
)

# ...

# This will call FreqValidator.validate_input('100 kHz') to
# transform the string '100 kHz' to '100000'. The full command
# then becomes:
# CALC:FREQ:STAR 100000
instr.ch1.freq_start = '100 Hz'

# This will send the command CALC:FREQ:STAR? to the instrument
# then send the response from the instrument to
# FreqValidator.validate_output() to attempt to transform, as an
# example, the string '100000' to the int 100_000
_ = instr.ch1.freq_start
```

3.11 vectorFitting (skrf.vectorFitting)

Module for vector fitting of networks.

VectorFitting	This class provides a Python implementation of the Vector Fitting algorithm and various functions for the fit analysis, passivity evaluation and enforcement, and export of SPICE equivalent circuits.
VectorFittingParametric	This class provides a Python implementation of the parametric vector fitting algorithm **[#Triverio_parametric]_**.

3.11.1 skrf.vectorFitting.VectorFitting

class skrf.vectorFitting.**VectorFitting**(*network*)

This class provides a Python implementation of the Vector Fitting algorithm and various functions for the fit analysis, passivity evaluation and enforcement, and export of SPICE equivalent circuits.

Parameters

network (*skrf.network.Network*) – Network instance of the N-port holding the frequency responses to be fitted, for example a scattering, impedance or admittance matrix.

Examples

Load the *Network*, create a *VectorFitting* instance, perform the fit with a given number of real and complex-conjugate starting poles:

```
>>> nw_3port = skrf.Network('my3port.s3p')
>>> vf = skrf.VectorFitting(nw_3port)
>>> vf.vector_fit(n_poles_real=1, n_poles_cmplx=4)
```

Notes

The fitting code is based on the original algorithm[1] and on two improvements for relaxed pole relocation[2] and efficient (fast) solving[3]. See also the Vector Fitting website[4] for further information and download of the papers listed below. A Matlab implementation is also available there for reference.

References

Attributes

network	Instance variable holding the Network to be fitted.
poles	Instance variable holding the list of fitted poles.
residues	Instance variable holding the list of fitted residues.
proportional_coeff	Instance variable holding the list of fitted proportional coefficients.
constant_coeff	Instance variable holding the list of fitted constants.
max_iterations	Instance variable specifying the maximum number of iterations for the fitting process and for the passivity enforcement.
max_tol	Instance variable specifying the convergence criterion in terms of relative tolerance.
wall_clock_time	Instance variable holding the wall-clock time (in seconds) consumed by the most recent fitting process with `vector_fit()`.

skrf.vectorFitting.VectorFitting.network

`VectorFitting.`**`network`**

Instance variable holding the Network to be fitted. This is the Network passed during initialization, which may be changed or set to *None*.

[1] B. Gustavsen, A. Semlyen, "Rational Approximation of Frequency Domain Responses by Vector Fitting", IEEE Transactions on Power Delivery, vol. 14, no. 3, pp. 1052-1061, July 1999, DOI: https://doi.org/10.1109/61.772353

[2] B. Gustavsen, "Improving the Pole Relocating Properties of Vector Fitting", IEEE Transactions on Power Delivery, vol. 21, no. 3, pp. 1587-1592, July 2006, DOI: https://doi.org/10.1109/TPWRD.2005.860281

[3] D. Deschrijver, M. Mrozowski, T. Dhaene, D. De Zutter, "Marcomodeling of Multiport Systems Using a Fast Implementation of the Vector Fitting Method", IEEE Microwave and Wireless Components Letters, vol. 18, no. 6, pp. 383-385, June 2008, DOI: https://doi.org/10.1109/LMWC.2008.922585

[4] Vector Fitting website: https://www.sintef.no/projectweb/vectorfitting/

skrf.vectorFitting.VectorFitting.poles

VectorFitting.**poles**

Instance variable holding the list of fitted poles. It will be initialized by *vector_fit()* or *auto_fit()*. This array is 1-dimensional with shape (K), where $K_{real} + K_{complex}$ is the number of real and complex-conjugate poles. This implies that only one part of a complex-conjugate pair is stored in this array (the one with the positive imaginary part). When calculating the model response in pole-residue form, the missing part of each complex-conjugate pair has to be added manually. See the source code of *get_model_response()* as an example.

skrf.vectorFitting.VectorFitting.residues

VectorFitting.**residues**

Instance variable holding the list of fitted residues. It will be initialized by *vector_fit()* or *auto_fit()*. This array is 2-dimensional with shape (M, K), where $M = N_{ports}^2$ is the number of frequency responses of the fitted network, and K is the number of real and complex-conjugate poles. See the documentation of *poles* for details about complex-conjugate pairs. The residues of the individual frequency responses are stacked in row-major format (C-like), for example *[[S11_1, S11_2, ...], [S12_1, S12_2, ...], [S21_1, S21_2, ...], [S22_1, S22_2, ...]]* for a 2-port network.

skrf.vectorFitting.VectorFitting.proportional_coeff

VectorFitting.**proportional_coeff**

Instance variable holding the list of fitted proportional coefficients. It will be initialized by *vector_fit()* or *auto_fit()*. This array is 1-dimensional with shape M, where $M = N_{ports}^2$ is the number of frequency responses of the fitted network, and K is the number of real and complex-conjugate poles. See the documentation of *poles* for details about complex-conjugate pairs. The proportional coefficient of the individual frequency responses are stacked in row-major format (C-like), for example *[S11, S12, S21, S22]* for a 2-port network.

skrf.vectorFitting.VectorFitting.constant_coeff

VectorFitting.**constant_coeff**

Instance variable holding the list of fitted constants. It will be initialized by *vector_fit()* or *auto_fit()*. This array is 1-dimensional with shape M, where $M = N_{ports}^2$ is the number of frequency responses of the fitted network, and K is the number of real and complex-conjugate poles. See the documentation of *poles* for details about complex-conjugate pairs. The constants of the individual frequency responses are stacked in row-major format (C-like), for example *[S11, S12, S21, S22]* for a 2-port network.

skrf.vectorFitting.VectorFitting.max_iterations

VectorFitting.**max_iterations**

Instance variable specifying the maximum number of iterations for the fitting process and for the passivity enforcement. To be changed by the user before calling *vector_fit()* and/or *passivity_enforce()*.

skrf.vectorFitting.VectorFitting.max_tol

VectorFitting.**max_tol**

Instance variable specifying the convergence criterion in terms of relative tolerance. To be changed by the user before calling *vector_fit()*.

skrf.vectorFitting.VectorFitting.wall_clock_time

VectorFitting.**wall_clock_time**

Instance variable holding the wall-clock time (in seconds) consumed by the most recent fitting process with *vector_fit()*. Subsequent calls of *vector_fit()* will overwrite this value.

Methods

__init__	
auto_fit	Automatic fitting routine implementing the "vector fitting with adding and skimming" algorithm as proposed in **[#Grivet-Talocia]_**.
get_model_order	Returns the model order calculated with $N_{real} + 2N_{complex}$ for a given set of poles.
get_model_response	Returns one of the frequency responses $H_{i+1,j+1}$ of the fitted model H.
get_rms_error	Returns the root-mean-square (rms) error magnitude of the fit, i.e. $\sqrt{\mathrm{mean}(\lvert S - S_{\mathrm{fit}}\rvert^2)}$, either for an individual response $S_{i+1,j+1}$ or for larger slices of the network.
get_spurious	Classifies fitted pole-residue pairs as spurious or not spurious.
is_passive	Returns the passivity status of the model as a boolean value.
passivity_enforce	Enforces the passivity of the vector fitted model, if required.
passivity_test	Evaluates the passivity of reciprocal vector fitted models by means of a half-size test matrix **[#]_**.
plot	Plots the specified component of the parameter $H_{i+1,j+1}$ in the fit, where H is either the scattering (S), the impedance (Z), or the admittance (H) response specified in *parameter*.
plot_convergence	Plots the history of the model residue parameter **d_res** during the iterative pole relocation process of the vector fitting, which should eventually converge to a fixed value.
plot_passivation	Plots the history of the greatest singular value during the iterative passivity enforcement process, which should eventually converge to a value slightly lower than 1.0 or stop after reaching the maximum number of iterations specified in the class variable `max_iterations`.
plot_s_db	Plots the magnitude in dB of the scattering parameter response(s) in the fit.
plot_s_deg	Plots the phase in degrees of the scattering parameter response(s) in the fit.
plot_s_deg_unwrap	Plots the unwrapped phase in degrees of the scattering parameter response(s) in the fit.
plot_s_im	Plots the imaginary part of the scattering parameter response(s) in the fit.
plot_s_mag	Plots the magnitude in linear scale of the scattering parameter response(s) in the fit.
plot_s_re	Plots the real part of the scattering parameter response(s) in the fit.
plot_s_singular	Plots the singular values of the vector fitted S-matrix in linear scale.
read_npz	Reads all model parameters `poles`, `residues`, `proportional_coeff` and `constant_coeff` from a labeled NumPy .npz file.
vector_fit	Main work routine performing the vector fit.

continues on next page

Table 142 – continued from previous page

write_npz	Writes the model parameters in `poles`, `residues`, `proportional_coeff` and `constant_coeff` to a labeled NumPy .npz file.
write_spice_subcircuit_s	Creates an equivalent N-port subcircuit based on its vector fitted scattering (S) parameter responses in spice simulator netlist syntax (compatible with LTspice, ngspice, Xyce, ...).

skrf.vectorFitting.VectorFitting.__init__

VectorFitting.**__init__**(*network*)

Parameters
network (Network)

skrf.vectorFitting.VectorFitting.auto_fit

VectorFitting.**auto_fit**(*n_poles_init_real=3*, *n_poles_init_cmplx=3*, *n_poles_add=3*, *model_order_max=100*, *iters_start=3*, *iters_inter=3*, *iters_final=5*, *target_error=0.01*, *alpha=0.03*, *gamma=0.03*, *nu_samples=1.0*, *parameter_type='s'*, *enforce_dc=True*)

Automatic fitting routine implementing the "vector fitting with adding and skimming" algorithm as proposed in[1]. This algorithm is able to provide high quality macromodels with automatic model order optimization, while improving both the rate of convergence and the fit quality in case of noisy data. The resulting model parameters will be stored in the class variables *poles*, *residues*, *proportional_coeff* and *constant_coeff*.

Parameters

- **n_poles_init_real** (*int, optional*) – Number of real poles in the initial model.
- **n_poles_init_cmplx** (*int, optional*) – Number of complex conjugate poles in the initial model.
- **n_poles_add** (*int, optional*) – Number of new poles allowed to be added in each refinement iteration, if possible. This controls how fast the model order is allowed to grow. Unnecessary poles will have to be skimmed and removed later. This parameter has a strong effect on the convergence.
- **model_order_max** (*int, optional*) – Maximum model order as calculated with $N_{real} + 2N_{complex}$. This parameter provides a stopping criterion in case the refinement process is not converging.
- **iters_start** (*int, optional*) – Number of initial iterations for pole relocations as in regular vector fitting.
- **iters_inter** (*int, optional*) – Number of intermediate iterations for pole relocations during each iteration of the refinement process.
- **iters_final** (*int, optional*) – Number of final iterations for pole relocations after the refinement process.
- **target_error** (*float, optional*) – Target for the model error to be reached during the refinement process. The actual achievable error is bound by the noise in the data. If specified with a number greater than the noise floor, this parameter provides another stopping criterion

[1] S. Grivet-Talocia and M. Bandinu, "Improving the convergence of vector fitting for equivalent circuit extraction from noisy frequency responses," in IEEE Transactions on Electromagnetic Compatibility, vol. 48, no. 1, pp. 104-120, Feb. 2006, DOI: https://doi.org/10.1109/TEMC.2006.870814

for the refinement process. It therefore affects both the convergence, the final error, and the final model order (number of poles used in the model).

- **alpha** (*float, optional*) – Threshold for the error decay to stop the refinement loop in case of error stagnation. This parameter provides another stopping criterion for cases where the model already has enough poles but the target error still cannot be reached because of excess noise (target error too small for noise level in the data).
- **gamma** (*float, optional*) – Threshold for the detection of spurious poles.
- **nu_samples** (*float, optional*) – Required and enforced (relative) spacing in terms of frequency samples between existing poles and relocated or added poles. The number can be a float, it does not have to be an integer.
- **parameter_type** (*str, optional*) – Representation type of the frequency responses to be fitted. Either *scattering* (*'s'* or *'S'*), *impedance* (*'z'* or *'Z'*) or *admittance* (*'y'* or *'Y'*). It's recommended to perform the fit on the original S parameters. Otherwise, scikit-rf will convert the responses from S to Z or Y, which might work for the fit but can cause other issues.
- **enforce_dc** (*bool, optional*) – Enforces/Preserves the dc point given in the original network data in the vector fitted model. If the constant term **d** is enabled with *fit_constant=True*, the dc point will be enforced on **d** (preferred method). Otherwise, it will be enforced on the first residue in *self.residues*.

Returns
No return value.

Return type
None

See also

vector_fit
Regular vector fitting routine.

References

skrf.vectorFitting.VectorFitting.get_model_order

static VectorFitting.**get_model_order**(*poles*)

Returns the model order calculated with $N_{real} + 2N_{complex}$ for a given set of poles.

Parameters
poles (*ndarray*) – The poles of the model as a list or NumPy array.

Returns
order

Return type
int

skrf.vectorFitting.VectorFitting.get_model_response

VectorFitting.**get_model_response**(*i*, *j*, *freqs=None*)

Returns one of the frequency responses $H_{i+1,j+1}$ of the fitted model H.

Parameters

- **i** (*int*) – Row index of the response in the response matrix.
- **j** (*int*) – Column index of the response in the response matrix.
- **freqs** (*list of float or ndarray or None, optional*) – List of frequencies for the response plot. If None, the sample frequencies of the fitted network in *network* are used.

Returns
response – Model response $H_{i+1,j+1}$ at the frequencies specified in *freqs* (complex-valued Numpy array).

Return type
ndarray

Examples

Get fitted S11 at 101 frequencies from 0 Hz to 10 GHz:

```
>>> import skrf
>>> vf = skrf.VectorFitting(skrf.data.ring_slot)
>>> vf.vector_fit(3, 0)
>>> s11_fit = vf.get_model_response(0, 0, numpy.linspace(0, 10e9, 101))
```

skrf.vectorFitting.VectorFitting.get_rms_error

`VectorFitting.get_rms_error`(*i=-1*, *j=-1*, *parameter_type='s'*)

Returns the root-mean-square (rms) error magnitude of the fit, i.e. $\sqrt{\mathrm{mean}(|S - S_{\mathrm{fit}}|^2)}$, either for an individual response $S_{i+1,j+1}$ or for larger slices of the network.

Parameters

- **i** (*int, optional*) – Row indices of the responses to be evaluated. Either a single row selected by an integer $i \in [0, N_{\mathrm{ports}} - 1]$, or multiple rows selected by a list of integers, or all rows selected by $i = -1$ (*default*).
- **j** (*int, optional*) – Column indices of the responses to be evaluated. Either a single column selected by an integer $j \in [0, N_{\mathrm{ports}} - 1]$, or multiple columns selected by a list of integers, or all columns selected by $j = -1$ (*default*).
- **parameter_type** (*str, optional*) – Representation type of the fitted frequency responses. Either *scattering* (`s` or `S`), *impedance* (`z` or `Z`) or *admittance* (`y` or `Y`).

Returns
rms_error – The rms error magnitude between the vector fitted model and the original network data.

Return type
ndarray

Raises
`ValueError` – If the specified parameter representation type is not `s`, `z`, nor `y`.

skrf.vectorFitting.VectorFitting.get_spurious

static VectorFitting.**get_spurious**(*poles*, *residues*, *n_freqs=101*, *gamma=0.03*)

Classifies fitted pole-residue pairs as spurious or not spurious. The implementation is based on the evaluation of band-limited energy norms (p=2) of the resonance curves of individual pole-residue pairs, as proposed in[1].

Parameters

- **poles** (*ndarray, shape (N)*) – Array of fitted poles
- **residues** (*ndarray, shape (M, N)*) – Array of fitted residues
- **n_freqs** (*int, optional*) – Number of frequencies for the evaluation. The frequency range is chosen automatically and the default 101 frequencies should be appropriate in most cases.
- **gamma** (*float, optional*) – Sensitivity threshold for the classification. Typical values range from 0.01 to 0.05.

Returns

Boolean array having the same shape as *poles*. *True* marks the respective pole as spurious.

Return type

ndarray, bool, shape (N)

References

skrf.vectorFitting.VectorFitting.is_passive

VectorFitting.**is_passive**(*parameter_type='s'*)

Returns the passivity status of the model as a boolean value.

Parameters

parameter_type (*str, optional*) – Representation type of the fitted frequency responses. Either *scattering* (`s` or `S`), *impedance* (`z` or `Z`) or *admittance* (`y` or `Y`). Currently, only scattering parameters are supported for passivity evaluation.

Returns

passivity – `True` if model is passive, else `False`.

Return type

bool

See also

passivity_test
Verbose passivity evaluation routine.

passivity_enforce
Enforces the passivity of the vector fitted model, if required.

Examples

Load and fit the *Network*, then check whether or not the model is passive:

[1] S. Grivet-Talocia and M. Bandinu, "Improving the convergence of vector fitting for equivalent circuit extraction from noisy frequency responses," in IEEE Transactions on Electromagnetic Compatibility, vol. 48, no. 1, pp. 104-120, Feb. 2006, DOI: https://doi.org/10.1109/TEMC.2006.870814

```
>>> nw_3port = skrf.Network('my3port.s3p')
>>> vf = skrf.VectorFitting(nw_3port)
>>> vf.vector_fit(n_poles_real=1, n_poles_cmplx=4)
>>> vf.is_passive() # returns True or False
```

skrf.vectorFitting.VectorFitting.passivity_enforce

VectorFitting.**passivity_enforce**(*n_samples=200*, *f_max=None*, *parameter_type='s'*, *preserve_dc=True*)

Enforces the passivity of the vector fitted model, if required. This is an implementation of the methods presented in[1] and[2] using singular value perturbation. To preserve the dc point in the model during passivity enforcement, only the residues are perturbed, not the constant term.

Parameters

- **n_samples** (*int, optional*) – Number of linearly spaced frequency samples at which passivity will be evaluated and enforced. (Default: 200). If there are very narrow frequency bands of passivity violations, a sufficiently large number of frequency samples is required.
- **f_max** (*float or None, optional*) – Highest frequency of interest for the passivity enforcement (in Hz, not rad/s). This limit usually equals the highest sample frequency of the fitted Network. If None, the highest frequency in `self.network` is used, which must not be None is this case. If *f_max* is not None, it overrides the highest frequency in `self.network`.
- **parameter_type** (*str, optional*) – Representation type of the fitted frequency responses. Either *scattering* (`s` or `S`), *impedance* (`z` or `Z`) or *admittance* (`y` or `Y`). Currently, only scattering parameters are supported for passivity evaluation.
- **preserve_dc** (*bool, optional*) – Enables dc point preservation during passivity enforcement. This only works if the fitted model is already passive at the dc point, which is not always the case. If it is not passive, dc point preservation is disabled and passivity is also enforced on the dc point.

Return type
None

Raises

- **NotImplementedError** – If the function is called for *parameter_type* different than *S* (scattering).
- **ValueError** – If the function is used with a model containing nonzero proportional coefficients. Or if both *f_max* and `self.network` are None.

See also

is_passive
Returns the passivity status of the model as a boolean value.

passivity_test
Verbose passivity evaluation routine.

plot_passivation
Convergence plot for passivity enforcement iterations.

[1] T. Dhaene, D. Deschrijver and N. Stevens, "Efficient Algorithm for Passivity Enforcement of S-Parameter- Based Macromodels," in IEEE Transactions on Microwave Theory and Techniques, vol. 57, no. 2, pp. 415-420, Feb. 2009, DOI: 10.1109/TMTT.2008.2011201

[2] D. Deschrijver and T. Dhaene, "DC-Preserving Passivity Enforcement for S-Parameter Based Macromodels," in IEEE Transactions on Microwave Theory and Techniques, vol. 58, no. 4, pp. 923-928, April 2010, DOI: 10.1109/TMTT.2010.2042556

Examples

Load and fit the *Network*, then enforce the passivity of the model:

```
>>> nw_3port = skrf.Network('my3port.s3p')
>>> vf = skrf.VectorFitting(nw_3port)
>>> vf.vector_fit(n_poles_real=1, n_poles_cmplx=4)
>>> vf.passivity_enforce()  # won't do anything if model is already passive
```

References

skrf.vectorFitting.VectorFitting.passivity_test

VectorFitting.**passivity_test**(*parameter_type='s'*)

Evaluates the passivity of reciprocal vector fitted models by means of a half-size test matrix[1]. Any existing frequency bands of passivity violations will be returned as a sorted list.

Parameters

parameter_type (*str, optional*) – Representation type of the fitted frequency responses. Either *scattering* (`s` or `S`), *impedance* (`z` or `Z`) or *admittance* (`y` or `Y`). Currently, only scattering parameters are supported for passivity evaluation.

Raises

- `NotImplementedError` – If the function is called for *parameter_type* different than *S* (scattering).
- `ValueError` – If the function is used with a model containing nonzero proportional coefficients.

Returns

violation_bands – NumPy array with frequency bands of passivity violation: *[[f_start_1, f_stop_1], [f_start_2, f_stop_2], ...]*.

Return type

ndarray

See also

is_passive
Query the model passivity as a boolean value.

passivity_enforce
Enforces the passivity of the vector fitted model, if required.

Examples

Load and fit the *Network*, then evaluate the model passivity:

```
>>> nw_3port = skrf.Network('my3port.s3p')
>>> vf = skrf.VectorFitting(nw_3port)
>>> vf.vector_fit(n_poles_real=1, n_poles_cmplx=4)
>>> violations = vf.passivity_test()
```

[1] B. Gustavsen and A. Semlyen, "Fast Passivity Assessment for S-Parameter Rational Models Via a Half-Size Test Matrix," in IEEE Transactions on Microwave Theory and Techniques, vol. 56, no. 12, pp. 2701-2708, Dec. 2008, DOI: 10.1109/TMTT.2008.2007319.

References

skrf.vectorFitting.VectorFitting.plot

VectorFitting.**plot**(*component*, *i=-1*, *j=-1*, *freqs=None*, *parameter='s'*, ***, *ax=None*)

Plots the specified component of the parameter $H_{i+1,j+1}$ in the fit, where H is either the scattering (S), the impedance (Z), or the admittance (H) response specified in *parameter*.

Parameters

- **component** (*str*) – The component to be plotted. Must be one of the following items: ['db', 'mag', 'deg', 'deg_unwrap', 're', 'im']. *db* for magnitude in decibels, *mag* for magnitude in linear scale, *deg* for phase in degrees (wrapped), *deg_unwrap* for phase in degrees (unwrapped/continuous), *re* for real part in linear scale, *im* for imaginary part in linear scale.
- **i** (*int, optional*) – Row index of the response. *-1* to plot all rows.
- **j** (*int, optional*) – Column index of the response. *-1* to plot all columns.
- **freqs** (*list of float or ndarray or None, optional*) – List of frequencies for the response plot. If None, the sample frequencies of the fitted network in *network* are used. This only works if *network* is not *None*.
- **parameter** (*str, optional*) – The network representation to be used. This is only relevant for the plot of the original sampled response in *network* that is used for comparison with the fit. Must be one of the following items unless *network* is *None*: ['s', 'z', 'y'] for *scattering* (default), *impedance*, or *admittance*.
- **ax** (`matplotlib.Axes` object or None) – matplotlib axes to draw on. If None, the current axes is fetched with `gca()`.

Returns

matplotlib axes used for drawing. Either the passed `ax` argument or the one fetch from the current figure.

Return type

`matplotlib.Axes`

Raises

ValueError – If the *freqs* parameter is not specified while the Network in *network* is *None*. Also if *component* and/or *parameter* are not valid.

skrf.vectorFitting.VectorFitting.plot_convergence

VectorFitting.**plot_convergence**(*ax=None*)

Plots the history of the model residue parameter **d_res** during the iterative pole relocation process of the vector fitting, which should eventually converge to a fixed value. Additionally, the relative change of the maximum singular value of the coefficient matrix **A** are plotted, which serve as a convergence indicator.

Parameters

ax (`matplotlib.Axes` object or None) – matplotlib axes to draw on. If None, the current axes is fetched with `gca()`.

Returns

matplotlib axes used for drawing. Either the passed `ax` argument or the one fetch from the current figure.

Return type

`matplotlib.Axes`

skrf.vectorFitting.VectorFitting.plot_passivation

VectorFitting.**plot_passivation**(*ax=None*)

Plots the history of the greatest singular value during the iterative passivity enforcement process, which should eventually converge to a value slightly lower than 1.0 or stop after reaching the maximum number of iterations specified in the class variable *max_iterations*.

Parameters
ax (matplotlib.Axes object or None) – matplotlib axes to draw on. If None, the current axes is fetched with gca().

Returns
matplotlib axes used for drawing. Either the passed ax argument or the one fetch from the current figure.

Return type
matplotlib.Axes

skrf.vectorFitting.VectorFitting.plot_s_db

VectorFitting.plot_s_db(

Plots the magnitude in dB of the scattering parameter response(s) in the fit.

Parameters

- ***args** (*any, optional*) – Additional arguments to be passed to plot().
- ****kwargs** (dict, *optional*) – Additional keyword arguments to be passed to plot().

Returns
matplotlib axes used for drawing. Either the passed ax argument or the one fetch from the current figure.

Return type
matplotlib.Axes

Notes

This simply calls plot('db', *args, **kwargs).

skrf.vectorFitting.VectorFitting.plot_s_deg

VectorFitting.plot_s_deg(

Plots the phase in degrees of the scattering parameter response(s) in the fit.

Parameters

- ***args** (*any, optional*) – Additional arguments to be passed to plot().
- ****kwargs** (dict, *optional*) – Additional keyword arguments to be passed to plot().

Returns
matplotlib axes used for drawing. Either the passed ax argument or the one fetch from the current figure.

Return type
matplotlib.Axes

Notes

This simply calls `plot('deg', *args, **kwargs)`.

skrf.vectorFitting.VectorFitting.plot_s_deg_unwrap

`VectorFitting.plot_s_deg_unwrap(`

Plots the unwrapped phase in degrees of the scattering parameter response(s) in the fit.

Parameters

- ***args** (*any, optional*) – Additional arguments to be passed to `plot()`.
- ****kwargs** (`dict`, *optional*) – Additional keyword arguments to be passed to `plot()`.

Returns

matplotlib axes used for drawing. Either the passed `ax` argument or the one fetch from the current figure.

Return type

`matplotlib.Axes`

Notes

This simply calls `plot('deg_unwrap', *args, **kwargs)`.

skrf.vectorFitting.VectorFitting.plot_s_im

`VectorFitting.plot_s_im(`

Plots the imaginary part of the scattering parameter response(s) in the fit.

Parameters

- ***args** (*any, optional*) – Additional arguments to be passed to `plot()`.
- ****kwargs** (`dict`, *optional*) – Additional keyword arguments to be passed to `plot()`.

Returns

matplotlib axes used for drawing. Either the passed `ax` argument or the one fetch from the current figure.

Return type

`matplotlib.Axes`

Notes

This simply calls `plot('im', *args, **kwargs)`.

skrf.vectorFitting.VectorFitting.plot_s_mag

`VectorFitting.plot_s_mag(`

Plots the magnitude in linear scale of the scattering parameter response(s) in the fit.

Parameters

- ***args** (*any, optional*) – Additional arguments to be passed to `plot()`.
- ****kwargs** (`dict`, *optional*) – Additional keyword arguments to be passed to `plot()`.

Returns

matplotlib axes used for drawing. Either the passed `ax` argument or the one fetch from the current figure.

Return type
matplotlib.Axes

Notes

This simply calls `plot('mag', *args, **kwargs)`.

skrf.vectorFitting.VectorFitting.plot_s_re

`VectorFitting.plot_s_re(`

Plots the real part of the scattering parameter response(s) in the fit.

Parameters

- ***args** (*any, optional*) – Additional arguments to be passed to `plot()`.
- ****kwargs** (*dict, optional*) – Additional keyword arguments to be passed to `plot()`.

Returns
matplotlib axes used for drawing. Either the passed `ax` argument or the one fetch from the current figure.

Return type
matplotlib.Axes

Notes

This simply calls `plot('re', *args, **kwargs)`.

skrf.vectorFitting.VectorFitting.plot_s_singular

`VectorFitting.plot_s_singular`(*freqs=None*, *, *ax=None*)

Plots the singular values of the vector fitted S-matrix in linear scale.

Parameters

- **freqs** (*list of float or ndarray or None, optional*) – List of frequencies for the response plot. If None, the sample frequencies of the fitted network in *network* are used. This only works if *network* is not *None*.
- **ax** (`matplotlib.Axes` object or None) – matplotlib axes to draw on. If None, the current axes is fetched with `gca()`.

Returns
matplotlib axes used for drawing. Either the passed `ax` argument or the one fetch from the current figure.

Return type
matplotlib.Axes

Raises
ValueError – If the *freqs* parameter is not specified while the Network in *network* is *None*.

skrf.vectorFitting.VectorFitting.read_npz

`VectorFitting.read_npz`(*file*)

Reads all model parameters *poles*, *residues*, *proportional_coeff* and *constant_coeff* from a labeled NumPy .npz file.

Parameters
file (*str*) – NumPy .npz file containing the parameters. See notes.

Return type
None

Raises
ValueError – If the shapes of the coefficient arrays in the provided file are not compatible.

Notes

The .npz file needs to include the model parameters as individual NumPy arrays (ndarray) labeled '*poles*', '*residues*', '*proportionals*' and '*constants*'. The shapes of those arrays need to match the network properties in *network* (correct number of ports). Preferably, the .npz file was created by *write_npz()*.

See also

write_npz
Writes all model parameters to a .npz file

Examples

Create an empty *VectorFitting* instance (with or without the fitted *Network*) and load the model parameters:

```
>>> vf = skrf.VectorFitting(None)
>>> vf.read_npz('./data/coefficients_my3port.npz')
```

This can be useful to analyze or process a previous vector fit instead of fitting it again, which sometimes takes a long time. For example, the model passivity can be evaluated and enforced:

```
>>> vf.passivity_enforce()
```

skrf.vectorFitting.VectorFitting.vector_fit

VectorFitting.**vector_fit**(*n_poles_real=2*, *n_poles_cmplx=2*, *init_pole_spacing='lin'*, *parameter_type='s'*, *fit_constant=True*, *fit_proportional=False*, *enforce_dc=True*)

Main work routine performing the vector fit. The results will be stored in the class variables *poles*, *residues*, *proportional_coeff* and *constant_coeff*.

Parameters

- **n_poles_real** (*int, optional*) – Number of initial real poles. See notes.
- **n_poles_cmplx** (*int, optional*) – Number of initial complex conjugate poles. See notes.
- **init_pole_spacing** (*str, optional*) – Type of initial pole spacing across the frequency interval of the S-matrix. Either *linear* ('*lin*'), *logarithmic* ('*log*'), or *custom*. In case of *custom*, the initial poles must be stored in *poles* as a NumPy array before calling this method. They will be overwritten by the final poles. The initialization parameters *n_poles_real* and *n_poles_cmplx* will be ignored in case of '*custom*'.
- **parameter_type** (*str, optional*) – Representation type of the frequency responses to be fitted. Either *scattering* ('*s*' or '*S*'), *impedance* ('*z*' or '*Z*') or *admittance* ('*y*' or '*Y*'). It's recommended to perform the fit on the original S parameters. Otherwise, scikit-rf will convert the responses from S to Z or Y, which might work for the fit but can cause other issues.
- **fit_constant** (*bool, optional*) – Include a constant term **d** in the fit.

- **fit_proportional** (*bool, optional*) – Include a proportional term **e** in the fit.
- **enforce_dc** (*bool, optional*) – Enforces/Preserves the dc point given in the original network data in the vector fitted model. If the constant term **d** is enabled with *fit_constant=True*, the dc point will be enforced on **d** (preferred method). Otherwise, it will be enforced on the first residue in *self.residues*.

Returns
No return value.

Return type
None

Notes

The required number of real or complex conjugate starting poles depends on the behaviour of the frequency responses. To fit a smooth response such as a low-pass characteristic, 1-3 real poles and no complex conjugate poles is usually sufficient. If resonances or other types of peaks are present in some or all of the responses, a similar number of complex conjugate poles is required. Be careful not to use too many poles, as excessive poles will not only increase the computation workload during the fitting and the subsequent use of the model, but they can also introduce unwanted resonances at frequencies well outside the fit interval.

See also

auto_fit
Automatic vector fitting routine with pole adding and skimming.

skrf.vectorFitting.VectorFitting.write_npz

VectorFitting.**write_npz**(*path*)

Writes the model parameters in *poles*, *residues*, *proportional_coeff* and *constant_coeff* to a labeled NumPy .npz file.

Parameters
path (*str*) – Target path without filename for the export. The filename will be added automatically based on the network name in *network*

Return type
None

See also

read_npz
Reads all model parameters from a .npz file

Examples

Load and fit the *Network*, then export the model parameters to a .npz file:

```
>>> nw_3port = skrf.Network('my3port.s3p')
>>> vf = skrf.VectorFitting(nw_3port)
>>> vf.vector_fit(n_poles_real=1, n_poles_cmplx=4)
>>> vf.write_npz('./data/')
```

The filename depends on the network name stored in *nw_3port.name* and will have the prefix *coefficients_*, for example *coefficients_my3port.npz*. The coefficients can then be read using NumPy's load() function:

```
>>> coeffs = numpy.load('./data/coefficients_my3port.npz')
>>> poles = coeffs['poles']
>>> residues = coeffs['residues']
>>> prop_coeffs = coeffs['proportionals']
>>> constants = coeffs['constants']
```

Alternatively, the coefficients can be read directly into a new instance of *VectorFitting*, see *read_npz()*.

skrf.vectorFitting.VectorFitting.write_spice_subcircuit_s

VectorFitting.**write_spice_subcircuit_s**(*file*, *fitted_model_name='s_equivalent'*, *create_reference_pins=False*)

Creates an equivalent N-port subcircuit based on its vector fitted scattering (S) parameter responses in spice simulator netlist syntax (compatible with LTspice, ngspice, Xyce, ...). The circuit synthesis is based on a direct implementation of the state-space representation of the vector fitted model[1].

Parameters

- **file** (*str*) – Path and filename including file extension (usually .sp) for the subcircuit file.
- **fitted_model_name** (*str*) – Name of the resulting subcircuit, default "s_equivalent"
- **create_reference_pins** (*bool*) – If set to True, the synthesized subcircuit will have N pin-pairs: p1 p1_ref p2 p2_ref ... pN pN_ref

  If set to False, the synthesized subcircuit will have N pins p1 p2 ... pN In this case, the reference nodes will be internally connected to the global ground net 0.

  The default is False

Return type
None

Examples

Load and fit the *Network*, then export the equivalent subcircuit:

```
>>> nw_3port = skrf.Network('my3port.s3p')
>>> vf = skrf.VectorFitting(nw_3port)
>>> vf.auto_fit()
>>> vf.write_spice_subcircuit_s('/my3port_model.sp')
```

References

3.11.2 skrf.vectorFitting.VectorFittingParametric

class skrf.vectorFitting.**VectorFittingParametric**(*networkset=None*, *n_poles_real=-1*, *n_poles_cmplx=-1*)

This class provides a Python implementation of the parametric vector fitting algorithm[1].

Parameters

[1] S. Grivet-Talocia and B. Gustavsen, "Passive Macromodeling", Wiley, 2016, doi: https://doi.org/10.1002/9781119140931

[1] P. Triverio, S. Grivet-Talocia and M. S. Nakhla, "A Parameterized Macromodeling Strategy With Uniform Stability Test", IEEE Transactions on Advanced Packaging, vol. 32, no. 1, pp. 205-215, Feb. 2009, DOI: https://doi.org/10.1109/TADVP.2008.2007913

- **networkset** (*skrf.networkSet.NetworkSet*) – NetworkSet instance of multiple N-port networks holding the frequency responses to be fitted for some discrete values of one or multiple design parameter.
- **n_poles_real** (*int, optional*) – The number of real starting poles for the parametric fitting process. In case of *n_poles_real=-1*, a suitable number will be determined automatically by running *VectorFitting.auto_fit()* on the first network in `networkset`. This does not always succeed, so manual adjustments might be required.
- **n_poles_cmplx** (*int, optional*) – The number of complex-conjugate starting poles for the parametric fitting process. In case of *n_poles_cmplx=-1*, a suitable number will be determined automatically by running *VectorFitting.auto_fit()* on the first network in `networkset`. This does not always succeed, so manual adjustments might be required.

Examples

Load a bunch of parametric *Network* and bundle them in a parametric *NetworkSet*. Then create a *VectorFittingParametric* instance and perform the fit. Afterward, the parametric model can be evaluated (interpolated) at arbitrary values for each parameter:

```
>>> nwset = NetworkSet([Network('file1_2.s4p', params={'a': 1.2}),
>>>                     Network('file1_4.s4p', params={'a': 1.4}),
>>>                     Network('file1_6.s4p', params={'a': 1.6})])
>>> vfparam = VectorFittingParametric(nwset)
>>> vfparam.auto_fit()
>>> model = vfparam.get_model_response(params={'a': 1.27}, freqs=np.linspace(0,␣
↪10e9, 101))
```

References

Methods

__init__	
auto_fit	Perform the parametric vector fitting on all networks in `networkset` using *VectorFitting.auto_fit()*.
get_local_vectorfit	Returns a local point of the parametric model given by the parameters *params*.
get_model_response	Returns the network response matrix for a given list of frequencies and a given set of values for the design parameters used in `networkset`.
read_npz	Reads the model parameters `q`, `r`, `poles_global`, and the parameters in `parameters` from a labelled NumPy .npz file.
write_npz	Writes the model parameters in `q`, `r`, `poles_global`, and parameters in `parameters` to a NumPy .npz file.

skrf.vectorFitting.VectorFittingParametric.__init__

`VectorFittingParametric.__init__`(*networkset=None*, *n_poles_real=-1*, *n_poles_cmplx=-1*)

Parameters

- **networkset** (NetworkSet)
- **n_poles_real** (*int*)

- **n_poles_cmplx** (*int*)

skrf.vectorFitting.VectorFittingParametric.auto_fit

`VectorFittingParametric.`**auto_fit**(*enforce_passivity=True*)

Perform the parametric vector fitting on all networks in `networkset` using *VectorFitting.auto_fit()*.

Parameters

enforce_passivity (*bool*)

skrf.vectorFitting.VectorFittingParametric.get_local_vectorfit

`VectorFittingParametric.`**get_local_vectorfit**(*params*)

Returns a local point of the parametric model given by the parameters *params*. The returned object is an instance of the class *VectorFitting*, which can be used to post-process or export the local model.

Parameters

params (dict) – A dictionary with the design parameters to be used. For example: *params={'param1': 1.1, 'param2': -2.2}*.

Return type

VectorFitting

Examples

```
>>> vf_param = VectorFittingParametric(nwset)
>>> vf_param.auto_fit()
>>> vf_local = vf_param.get_local_vectorfit({'myparam1': -12, 'myparam2': 5})
>>> vf_local.passivity_enforce()
>>> vf_local.write_spice_subcircuit_s('model_-12_5.sp')
```

skrf.vectorFitting.VectorFittingParametric.get_model_response

`VectorFittingParametric.`**get_model_response**(*params*, *freqs*)

Returns the network response matrix for a given list of frequencies and a given set of values for the design parameters used in `networkset`. This also works for parameter values outside of the original sample interval (i.e. extrapolation instead of interpolation), but the results are likely going to be invalid.

Parameters

- **params** (dict) – A dictionary with the design parameters to be used. For example: *params={'param1': 1.1, 'param2': -2.2}*.
- **freqs** (*list or numpy.ndarray*) – A list of frequencies to evaluate the parametric model at.

Returns

model – The returned array has the shape [n_freqs, n_ports, n_ports], similar to *Network.s*.

Return type

numpy.ndarray

skrf.vectorFitting.VectorFittingParametric.read_npz

`VectorFittingParametric.`**read_npz**(*file*)

Reads the model parameters `q`, `r`, `poles_global`, and the parameters in `parameters` from a labelled NumPy .npz file.

Parameters
file (*str*) – NumPy .npz file containing the parameters. See notes.

Return type
None

Raises
ValueError – If the shapes of the coefficient arrays in the provided file are not compatible.

Notes

The .npz file contain include the model parameters Q, R, A, and the corresponding parameters as individual NumPy arrays (ndarray). The shapes of those arrays must be compatible. Preferably, the .npz file was created with *write_npz()*.

See also

write_npz
Writes all model parameters to a .npz file

Examples

Create an empty *VectorFittingParametric* instance and load the model parameters:

```
>>> vf = VectorFittingParametric(None)
>>> vf.read_npz('./data/coefficients_my3port.npz')
```

skrf.vectorFitting.VectorFittingParametric.write_npz

VectorFittingParametric.**write_npz**(*path*, *filename=None*)

Writes the model parameters in `q`, `r`, `poles_global`, and parameters in `parameters` to a NumPy .npz file.

Parameters

- **path** (*str*) – Target path without filename for the export.
- **filename** (*str*) – Name of the exported npz-file. If the filename is *None* (default), it will be taken from the name of the networkset in `networkset`. If that name is *None*, the file will be saved as *model.npz*.

Return type
None

See also

read_npz
Reads all model parameters from a .npz file

3.12 Qfactor (*skrf.qfactor*)

Module for estimating the Quality (Q) factor(s) from S-parameters.

This class implements methods for determining *loaded* and *unloaded* Q-factor from frequency-domain S-parameters, that can be applied to measurements of transmission or reflection.

Documentation and implementation are adapted from [MAT58]

3.12.1 Q-factor

Qfactor	Q-factor calculation class.

skrf.qfactor.Qfactor

class skrf.qfactor.**Qfactor**(*ntwk*, *res_type*, *Q_L0=None*, *f_L0=None*, *verbose=None*)

Q-factor calculation class.

This class implements methods for determining *loaded* and *unloaded* Q-factor from frequency-domain S-parameters, that can be applied to measurements of transmission or reflection.

Parameters

- **ntwk** (*Network* object) – A 1-port scikit-rf Network. If your device is a N-port, pass the desired sub-S-parameters to fit the data from, like *ntwk.s21*.
- **res_type** (*str*) – Specifies the resonance type: 'reflection', 'transmission', 'reflection_method2' or 'absorption': 'reflection' is generally suited for undercoupled resonators, while 'reflection_method2' is favoured for coupling with large loop.
- **Q_L0** (*float, optional. Default is None.*) – Estimated loaded Q-factor, used to improve fitting.
- **f_L0** (*float, optional. Default is None.*) – Estimated loaded resonant frequency, used to improve fitting [Hz] If None, automatically search for the min or max, depending on the resonance type defined by *res_type*.
- **verbose** (*bool, optional. Default is False.*) – Boolean flag controlling output of information to the console.

Raises

ValueError – If the passed Network is not 1-port or if the resonance type is unknown.

Notes

Uncalibrated line should be de-embedded (if it has significant length) from the S-parameter data before calling the functions in this module to get best results.

Attributes

BW	3-dB Bandwidth.
BW_scaled	3-dB Bandwidth scaled to the frequency unit.
f_L_scaled	Resonant Frequency in the frequency unit..

skrf.qfactor.Qfactor.BW

property Qfactor.**BW: float**

3-dB Bandwidth.

Return the half-power fractional bandwidth (aka 3-dB bandwidth) defined as:

$$BW = \frac{f_L}{Q_L}$$

Returns
3-dB Bandwidth in Hz.

Return type
float

See also

BW_scaled
3-dB Bandwidth scaled to the frequency unit.

skrf.qfactor.Qfactor.BW_scaled

property Qfactor.**BW_scaled: float**

3-dB Bandwidth scaled to the frequency unit.

Returns
3-dB Bandwidth in the frequency unit.

Return type
float

See also

BW
3-dB Bandwidth in Hz.

skrf.qfactor.Qfactor.f_L_scaled

property Qfactor.**f_L_scaled: float**

Resonant Frequency in the frequency unit..

Returns
Resonant frequency in the frequency unit.

Return type
float

See also

f_L
Resonant Frequency in Hz.

Methods

Q_circle	Q-circle diameter.
Q_unloaded	Unloaded Q-factor Q0.
__init__	Q-factor initializer.
angular_weights	Diagonal elements W_i of weights matrix.
fit	Fit Q-factor from S-parameter data.
fitted_network	Fitted Network.
fitted_s	S-parameter response of an equivalent circuit model resonator.

skrf.qfactor.Qfactor.Q_circle

Qfactor.**Q_circle**(*opt_res=None*, *A=None*)

Q-circle diameter.

The diameter of the Q-circle (as displayed in a VNA) provides a visual indication of whether the coupling is strong or weak [MAT58].

Parameters

- **opt_res** (None or OptimizedResult. Default is None.) – Solution produced by the *fit()* method. If None, uses the solution previously calculated, if performed.
- **A** (*None of float. Default is None.*) – Scaling factor as defined in MAT 58 [MAT58]. For *reflection* resonance type, can be set as None to use the magnitude of the fitted detuned reflection coefficient S_V

Returns

- **diam** (*float*) – Q-circle diameter d.
- **S_V** (*complex*) – Off-resonance Reflection Coefficient.
- **S_T** (*complex*) – Tuned Reflection Coefficient.

Return type
list

References

skrf.qfactor.Qfactor.Q_unloaded

Qfactor.**Q_unloaded**(*opt_res=None*, *A=None*)

Unloaded Q-factor Q0.

The value of the unloaded Q-factor Q0 cannot be measured directly but can be estimated from the measurement of the loaded Q-factor Q_L [MAT58] .

Parameters

- **opt_res** (None or OptimizedResult. Default is None.) – Solution produced by the *fit()* method. If None, uses the solution previously calculated, if performed.
- **A** (*float or None. Default is None.*) – Scaling factor as defined in MAT 58 [MAT58]. For *reflection* resonance type, can be set as None to use the magnitude of the fitted detuned reflection coefficient S_V

Returns
Q0 – Unloaded Q-factor.

Return type
float

References

skrf.qfactor.Qfactor.__init__

Qfactor.**__init__**(*ntwk*, *res_type*, *Q_L0=None*, *f_L0=None*, *verbose=None*)

Q-factor initializer.

Parameters

- **ntwk** (Network)
- **res_type** (*str*)
- **Q_L0** (*None | float*)
- **f_L0** (*None | float*)
- **verbose** (*bool | None*)

skrf.qfactor.Qfactor.angular_weights

static Qfactor.**angular_weights**(*f*, *f_L*, *Q_L*)

Diagonal elements W_i of weights matrix.

$$W_i = \frac{1}{\left[\frac{2Q_L(f_i - f_L)}{f_L}\right]^2 + 1}$$

The weights are needed when frequencies are equally-spaced (rather than points equally spaced around the Q-circle), and help reducing systematic error [MAT58].

Parameters

- **f** (*np.ndarray*) – Frequency values array.
- **f_L** (*float*) – Loaded resonant frequency.
- **Q_L** (*float*) – Loaded Q-factor.

Returns
W_i – Weighting factors in proportion to the rate of change of angle with frequency relative to the centre of the Q-circle.

Return type
np.ndarray

References

skrf.qfactor.Qfactor.fit

Qfactor.**fit**(*method='NLQFIT6'*, *loop_plan='fwfwc'*)

Fit Q-factor from S-parameter data.

Fitting overwrites the parameters *Q_L* and *f_L*.

Parameters

- **method** (*str, optional*) – Fitting method : 'NLQFIT6' (default), 'NLQFIT7', 'NLQFIT8':
  - 'NLQFIT6': Least Square Minimum of Eq.21 [MAT58] with 6 coefficients.
  - 'NLQFIT7': Least Square Minimum of Eq.26 [MAT58] with 7 coefficients, including one that characterize the trans. line length.
  - 'NLQFIT8': Least Square Minimum of Eq.43 [MAT58] with 8 coefficients, A model for frequency-dependent leakage.
- **loop_plan** (*str, optional*) – Defines order of steps used by the fitting process. The convergence algorithm uses a number of steps set by loop_plan, a string of characters as follows:
  - 'f': fit once without testing for convergence
  - 'c': repeated fit, iterating until convergence is obtained
  - 'w': re-calculate weighting factors on basis of previous fit
  - Initially the weighting factors are all unity.
  - The first character in *loop_plan* must not be 'w'.

  e.g.: 'fwfwc' (default).

Returns
result

Return type
OptimizedResult

References

skrf.qfactor.Qfactor.fitted_network

Qfactor.**fitted_network**(*opt_res=None*, *frequency=None*)

Fitted Network.

Return the Network corresponding to the fitted response.

Parameters

- **opt_res** (None or OptimizedResult. Default is None.) – Solution produced by the *fit()* method. If None, uses the solution previously calculated, if performed.
- **frequency** (None or *Frequency*. Default is None.) – Frequency for the fitted Network. If None, use the same Frequency than the one used to create the QFactor.

Returns
ntwk – Fitted Network for the passed Frequency.

Return type
Network

skrf.qfactor.Qfactor.fitted_s

`Qfactor.fitted_s(opt_res=None, f=None)`

S-parameter response of an equivalent circuit model resonator.

The approximate solution and estimated is obtained from [MAT58]:

$$S = m_1 + jm_2 + \frac{m_3 + jm_4}{1 + jQ_L t}$$

where the m coefficients come from the fitted solution and

$$t = \frac{f}{f_L} - \frac{f_L}{f} \approx 2\frac{f f_L}{f_L}$$

Parameters

- **opt_res** (None or `OptimizedResult`. Default is None.) – Solution produced by the *fit()* method. If None, uses the solution previously calculated, if performed.
- **f** (*None or np.ndarray. Default is None.*) – frequency array [Hz]. If None, use the self frequencies.

Returns
s – S-parameter response.

Return type
np.ndarray

References

3.12.2 Loaded and Unloaded Q-factor

The Quality factor (Q-factor) of a resonator is defined by [Pozar]:

$$Q = \frac{2\pi U}{\Delta U}$$

where U is the average energy stored by the resonator and ΔU is the decrease in the average stored energy per wave cycle at the resonant frequency, that is, the average power loss.

The *loaded* Q-factor, Q_L, describes energy dissipation within the entire resonant system comprising of the resonator itself and the instrument used for observing resonances. The term loading refers to the effect that the external circuit has on measured quantities.

The external circuit consists of the measuring instrument and uncalibrated lines, but not the couplings of microwave resonators. Loading by an instrument that has 50 Ohm impedance, such as a VNA, causes Q_L to be reduced substantially if strong coupling is used.

For many applications the quantity that is desired is the *unloaded* Q-factor Q_0, which is determined by energy dissipation associated with the resonator only and therefore gives the best description of the resonant mode(s).

In other words, Q_0 is the Q-factor of the uncoupled resonator. The value of Q_0 can be estimated from measurements of Q_L, but cannot be measured directly. Q_0 is largely governed by ohmic loss arising from surface currents in the metal conductors (walls and loop couplings), and from dielectric loss in any insulating materials that may be present.

3.12.3 Relationships between Loaded and Unloaded Q-factors

Energy dissipation in the external circuit is characterised by the *external* Q-factor, Q_e. For both series and parallel equivalent circuits, the three Q-factors are related by [Pozar]:

$$\frac{1}{Q_L} = \frac{1}{Q_0} + \frac{1}{Q_e}$$

The coupling factor β is defined for each port as:

$$\beta = \frac{Q_0}{Q_e}$$

Finding the unloaded Q from measured S-parameters consists in first finding the coupling factor, then measure Q_L from the 3 dB bandwidth and using the relationships above.

Fortunately, scikit-rf implements methods for determining loaded and unloaded Q-factors from frequency-domain S-parameters. The implemented methods are described in detail in [MAT58], and can be applied to measurements of transmission or reflection.

3.12.4 Q-factor determination through equivalent-circuit models

Characterisation of resonances from measurements in the frequency-domain can be achieved through equivalent-circuit models [MAT58]. Resonators can be modelled as an ideal RLC resonator connected to an external circuit, incorporating elements to account for a lossy coupling and coupling reactances.

For high Q-factor resonators (in practice, Q_L > 100), the S-parameter response of a resonator measured in a calibrated system with reference planes at the resonator couplings can be expressed like [MAT58], [Galwas] :

$$S = S_D + d\frac{e^{2j\delta}}{1 + jQ_L t}$$

where S_D is the detuned S-parameter measured at frequencies far above or below resonance, d is The diameter of the Q-circle, δ is a real-valued constant that defines the orientation of the Q-circle, and t is the fractional offset frequency given by:

$$t = \frac{f}{f_L} - \frac{f_L}{f} \approx 2\frac{f f_L}{f_L}$$

where f_L is the loaded resonant frequency and f the frequency at which S is measured. This equation can be applied to measurements by transmission (S21 or S12) or reflection (S11 or S22).

The S-parameters are fitted against a modified expression of the above equations to deduce the resonant frequency, loaded and unloaded Q-factors and other properties.

References

3.13 time (*skrf.time*)

Time domain functions

time_gate(ntwk[, start, stop, center, span, ...])	Time-domain gating of one-port s-parameters with a window function from scipy.signal.windows.
detect_span(ntwk[, t_unit])	Detect the correct time-span between two largest peaks.
find_n_peaks(x, n[, thres])	Find a given number of peaks in a signal.
indexes(y[, thres, min_dist])	Peak detection routine.
get_window(window, Nx[, fftbins])	Calls a custom window function or *scipy.signal.get_window()* depending on the window argument.

3.13.1 skrf.time.time_gate

skrf.time.**time_gate**(*ntwk*, *start=None*, *stop=None*, *center=None*, *span=None*, *mode='bandpass'*, *window=('kaiser', 6)*, *method='fft'*, *fft_window='cosine'*, *conv_mode='wrap'*, *t_unit=''*)

Time-domain gating of one-port s-parameters with a window function from scipy.signal.windows.

The gate can be defined with start/stop times, or by center/span. All times are in units of seconds but can be changed using the *t_unit* parameter. Common windows are:

- ('kaiser', 6)
- 6 # integers are interpreted as kaiser beta-values
- 'hamming'
- 'boxcar' # a straight up rect
- Callable function which accepts integer with window size as argument

If no parameters are passed this will try to auto-gate the largest peak.

Parameters

- **ntwk** (*Network*) – network to operate on
- **start** (*number, or None*) – start of time gate in t_unit.
- **stop** (*number, or None*) – stop of time gate in t_unit.
- **center** (*number, or None*) – center of time gate, in t_unit. If None, and span is given, the gate will be centered on the peak.
- **span** (*number, or None*) – span of time gate, in t_unit. If None span will be half of the distance to the second tallest peak
- **mode** (*['bandpass', 'bandstop']*) – mode of gate
- **window** (*string, float, or tuple*) – passed to *window* arg of *scipy.signal.get_window()* or callable function. String and tuple specifications are requested with *fftbins=False*, so that the time-domain gate is symmetric about its center sample. Callable windows are called as *window(window_width)* and are themselves responsible for returning a symmetric window.
- **method** (*str*) – Gating method. There are 3 option: 'convolution', 'fft', 'rfft'.

  With *'convolution'*, the time-domain gate gets transformed into frequency-domain using inverse FFT and the gating is then achieved by convolution with the frequency-domain data.

  With *'fft'* (default), the data gets transformed into time-domain using inverse FFT and the gating is achieved by multiplication with the time-domain gate. The gated time-domain signal is then transformed back into frequency-domain using FFT. As only positive signal frequencies are considered for the inverse FFT (with or without a dc component), the resulting time-domain signal has the same number of samples as in the frequency-domain, but is complex-valued. This method is also know as *time-domain band-pass mode*.

  With *'rfft'*, the procedure is the same as with *'fft'*, but the inverse FFT uses a complex-conjugate copy of the positive signal frequencies for the negative frequencies (Hermitian frequency response). A dc sample is also required. The resulting time-domain signal is real-valued and has twice the number of samples, which gives an improved time resolution. This method is also known as *time-domain low-pass mode*.
- **fft_window** (*str or tuple or None*) – Frequency-domain window applied before the inverse FFT in case of the (R)FFT method. This parameter takes the same values as the

window parameter. Example: *window='hann* (default), or *window=('kaiser', 5)*, or *window=None*. The window helps to remove artefacts such as time-domain sidelobes of the pulses, but it is a trade-off with the achievable pulse width. The window is removed when the gated time-domain signals is transformed back into frequency-domain.

- **conv_mode** (*str*) – Extension mode for the convolution (if selected) determining how the frequency-domain gate is extended beyond the boundaries. This has a large effect on the generation of gating artefacts due to boundary effects. The optimal mode depends on the data. See the parameter description of *scipy.ndimage.convolve1d* for the available options.
- **t_unit** (*str*) – Time unit for start, stop, center and span arguments, defaults to seconds (s).

  Possible values:

  - 's': seconds (default)
  - 'ms': milliseconds
  - 'µs' or 'us': microseconds
  - 'ns': nanoseconds
  - 'ps': picoseconds

Return type
Network

Note

You can't gate things that are 'behind' strong reflections. This is due to the multiple reflections that occur.

If you need to time-gate an N-port network, then you should gate each s-parameter independently.

Returns

- **ntwk** (*Network*) – copy of ntwk with time-gated s-parameters
- .. *warning::* – Depending on sharpness of the gate, the band edges may be inaccurate, due to properties of FFT. We do not re-normalize anything.

Parameters

- **ntwk** (Network)
- **start** (*float*)
- **stop** (*float*)
- **center** (*float*)
- **span** (*float*)
- **mode** (*str*)
- **method** (*str*)
- **fft_window** (*str*)
- **conv_mode** (*str*)
- **t_unit** (*str*)

Return type
Network

3.13.2 skrf.time.detect_span

skrf.time.**detect_span**(*ntwk*, *t_unit=''*)

Detect the correct time-span between two largest peaks.

Parameters

- **ntwk** (*Network*) – network to get data from
- **t_unit** (*str*) – Time unit for start, stop, center and span arguments, defaults to seconds (s).

  Possible values:

  - 's': seconds
  - 'ms': milliseconds
  - 'μs' or 'us': microseconds
  - 'ns': nanoseconds (default)
  - 'ps': picoseconds

Returns
span

Return type
float in unit t_unit

3.13.3 skrf.time.find_n_peaks

skrf.time.**find_n_peaks**(*x*, *n*, *thres=0.9*, ***kwargs*)

Find a given number of peaks in a signal.

Parameters

- **x** (*np.ndarray*) – signal
- **n** (*int*) – number of peaks to search for
- **thres** (*float, optional*) – threshold, default is 0.9
- ****kwargs** (optional keyword arguments passed to *indexes()*)

Returns
peak_idxs – List containing the numeric indexes of the peaks that were detected

Return type
list of int

Raises
ValueError – If no peaks are found.

3.13.4 skrf.time.indexes

skrf.time.**indexes**(*y*, *thres=0.3*, *min_dist=1*)

Peak detection routine.

Finds the numeric index of the peaks in *y* by taking its first order difference. By using *thres* and *min_dist* parameters, it is possible to reduce the number of detected peaks. *y* must be signed.

Parameters

- **y** (*ndarray (signed)*) – 1D amplitude data to search for peaks.

- **thres** (*float between [0., 1.], optional*) – Normalized threshold. Only the peaks with amplitude higher than the threshold will be detected. Default is 0.3
- **min_dist** (*int, optional*) – Minimum distance between each detected peak. The peak with the highest amplitude is preferred to satisfy this constraint. Default is 1

Returns
Array containing the numeric indexes of the peaks that were detected

Return type
ndarray

Notes

This function was taken from peakutils-1.1.0 http://pythonhosted.org/PeakUtils/index.html

3.13.5 skrf.time.get_window

skrf.time.**get_window**(*window*, *Nx*, *fftbins=True*, ***kwargs*)

Calls a custom window function or *scipy.signal.get_window()* depending on the window argument.

Parameters

- **window** (*str, tuple or Callable*) – The callable window function or a valid window string for *scipy.signal.get_window()*.
- **Nx** (*int*) – Number of samples
- **fftbins** (*bool*) – If True (default), create a periodic window, which is symmetric about sample *Nx / 2*. If False, create a symmetric window, which is symmetric about sample *(Nx - 1) / 2*. Only used for string or tuple window specifications, which are passed on to *scipy.signal.get_window()*. Callable windows are called as *window(Nx)* and do not receive this argument.

Returns
window – Window samples.

Return type
np.ndarray

3.14 circuit (*skrf.circuit*)

The Circuit class represents a circuit of arbitrary topology, consisting of an arbitrary number of N-ports networks.

Like in an electronic circuit simulator, the circuit must have one or more ports connected to the circuit. The Circuit object allows one retrieving the M-ports network, where M is the number of ports defined.

The results are returned in *Circuit* object.

3.14.1 Building a Circuit

Circuit(connections[, name, auto_reduce])	Creates a circuit made of a set of N-ports networks.
Circuit.Port(frequency, name[, z0])	Return a 1-port Network to be used as a Circuit port.
Circuit.SeriesImpedance(frequency, Z, name)	Return a 2-port network of a series impedance.
Circuit.ShuntAdmittance(frequency, Y, name)	Return a 2-port network of a shunt admittance.
Circuit.Ground(frequency, name[, z0])	Return a 1-port network of a grounded link.
Circuit.Open(frequency, name[, z0])	Return a 1-port network of an open link.

continues on next page

Table 148 – continued from previous page

Circuit.update_networks(networks[, name, ...])	Update the circuit connections with a new set of networks.

skrf.circuit.Circuit

class skrf.circuit.**Circuit**(*connections*, *name=None*, ***, *auto_reduce=False*, ***kwargs*)

Creates a circuit made of a set of N-ports networks.

For instructions on how to create Circuit see *__init__()*.

A Circuit object is representation a circuit assembly of an arbitrary number of N-ports networks connected together via an arbitrary topology.

The algorithm used to calculate the resultant network can be found in[1].

References

Parameters

- **connections** (*list[list[tuple[Network, int]]]*)
- **name** (*str | None*)
- **auto_reduce** (*bool*)
- **kwargs** (*Unpack[_REDUCE_OPTIONS]*)

Attributes

C	Return the global scattering matrix [C] of the networks in C-order.
CACHEDPROPERTIES	
C_F	Return the global scattering matrix [C] of the networks in F-order.
G	Generate the graph of the circuit.
T	Return the matrix of multiplication of the global scattering matrix [C] and concatenated intersection matrix [X] of the networks, that is [T] = - [C] @ [X].
X	Return the concatenated intersection matrix [X] of the circuit in C-order.
X_F	Return the concatenated intersection matrix [X] of the circuit in F-order.
connections	Circuit connections.
connections_list	Return the full list of connections, including intersections.
connections_nb	Return the number of intersections in the circuit.
connections_pair	List the connections by pair.
dim	Return the dimension of the C, X and global S matrices.
edge_labels	Return a dictionary describing the port and z0 of all graph edges.
edges	Return the list of all circuit connections

continues on next page

[1] P. Hallbjörner, Microw. Opt. Technol. Lett. 38, 99 (2003).

Table 149 – continued from previous page

intersections_dict	Return a dictionary of all intersections with associated ports and z0:
network	Return the Network associated to external ports.
networks_nb	Return the number of connected networks (port excluded).
nodes_nb	Return the number of nodes in the circuit.
port_indexes	Return the indexes of the "external" ports.
port_names	Return the names of the "external" ports.
port_z0	Return the external port impedances.
s	Return the global scattering parameters of the circuit.
s_external	Return the scattering parameters for the external ports.
z0	Characteristic impedances of "internal" ports.

skrf.circuit.Circuit.C

property Circuit.**C**: ndarray

Return the global scattering matrix [C] of the networks in C-order.

Returns
S – Global scattering matrix of the networks. Shape *f x (nb_inter*nb_n) x (nb_inter*nb_n)*

Return type
numpy.ndarray

skrf.circuit.Circuit.CACHEDPROPERTIES

Circuit.**CACHEDPROPERTIES** = **('s', 'X', 'X_F', 'C', 'C_F', 'T')**

skrf.circuit.Circuit.C_F

property Circuit.**C_F**: ndarray

Return the global scattering matrix [C] of the networks in F-order. The results of this function are the same as *C()* but in F-order.

Returns
S – Global scattering matrix of the networks. Shape *f x (nb_inter*nb_n) x (nb_inter*nb_n)*

Return type
numpy.ndarray

Note

F-order has a numerical bottleneck in matrix operations, but the assignment is more efficient.

skrf.circuit.Circuit.G

property Circuit.**G**

Generate the graph of the circuit. Convenience shortname for *graph()*.

skrf.circuit.Circuit.T

property Circuit.**T**: ndarray

Return the matrix of multiplication of the global scattering matrix [C] and concatenated intersection matrix [X] of the networks, that is [T] = - [C] @ [X].

Returns
T – Multiplication of the global scattering matrix [C] and concatenated intersection matrix [X] of the networks. In practice, F-contiguous [C_F] and [X_F] are used. Shape *f x (nb_inter*nb_n) x (nb_inter*nb_n)*

Return type
numpy.ndarray

> **Note**
>
> This is an auxiliary matrix used to break the numerical bottleneck of [C_F] @ [X_F] using the mathematical feature of block diagonal matrice [X].

skrf.circuit.Circuit.X

property Circuit.**X**: ndarray

Return the concatenated intersection matrix [X] of the circuit in C-order.

It is composed of the individual intersection matrices [X]_k assembled by block diagonal.

Returns
X

Return type
numpy.ndarray

> **Note**
>
> There is a numerical bottleneck in this function, when creating the block diagonal matrice [X] from the [X]_k matrices.

skrf.circuit.Circuit.X_F

property Circuit.**X_F**: ndarray

Return the concatenated intersection matrix [X] of the circuit in F-order. It is composed of the individual intersection matrices [X]_k assembled by block diagonal. The results of this function are the same as *X()* but in F-order.

Returns
X

Return type
numpy.ndarray

> **Note**
>
> F-order has a numerical bottleneck in matrix operations, but the assignment is more efficient.

skrf.circuit.Circuit.connections

property Circuit.**connections**: list[list[tuple[*Network*, int]]]

Circuit connections.

Description of circuit connections. Each connection is a described by a list of tuple. Each tuple contains (network, network_port_nb). Port number indexing starts from zero.

Returns
connections – The circuit connections.

Return type
list[list[tuple[Network, int]]]

skrf.circuit.Circuit.connections_list

property Circuit.**connections_list**: list[tuple[int, tuple[*Network*, int]]]

Return the full list of connections, including intersections.

The resulting list if of the form:

```
[
 [connexion_number, connexion],
 [connexion_number, connexion],
 ...
]
```

skrf.circuit.Circuit.connections_nb

property Circuit.**connections_nb**: int

Return the number of intersections in the circuit.

skrf.circuit.Circuit.connections_pair

property Circuit.**connections_pair**: list

List the connections by pair.

Each connection in the circuit is between a specific pair of two (networks, port, z0).

Returns
connections_pair – list of pair of connections

Return type
list

skrf.circuit.Circuit.dim

property Circuit.**dim**: int

Return the dimension of the C, X and global S matrices.

It correspond to the sum of all connections.

skrf.circuit.Circuit.edge_labels

property Circuit.**edge_labels**: *dict*

Return a dictionary describing the port and z0 of all graph edges.

Dictionary is in the form:

```
{('ntw1_name', 'X0'): '3 (50+0j)',
 ('ntw2_name', 'X0'): '0 (50+0j)',
 ('ntw2_name', 'X1'): '2 (50+0j)', ... }
```

which can be used in *networkx.draw_networkx_edge_labels*

skrf.circuit.Circuit.edges

property Circuit.**edges**: list

Return the list of all circuit connections

skrf.circuit.Circuit.intersections_dict

property Circuit.**intersections_dict**: *dict*

Return a dictionary of all intersections with associated ports and z0:

```
{ k: [(ntw1_name, ntw1_port), (ntw1_z0, ntw2_name, ntw2_port), ntw2_z0], ... }
```

skrf.circuit.Circuit.network

property Circuit.**network**: *Network*

Return the Network associated to external ports.

Returns
ntw – Network associated to external ports

Return type
Network

See also

port_names

skrf.circuit.Circuit.networks_nb

property Circuit.**networks_nb**: int

Return the number of connected networks (port excluded).

skrf.circuit.Circuit.nodes_nb

property Circuit.**nodes_nb**: int

Return the number of nodes in the circuit.

skrf.circuit.Circuit.port_indexes

property Circuit.port_indexes: list[int]

Return the indexes of the "external" ports.

Returns
port_indexes

Return type
list

skrf.circuit.Circuit.port_names

property Circuit.port_names: list[str]

Return the names of the "external" ports.

The names are given in the same order as the ports of the Network returned by *network()*, ie. in the order the port Networks appear in the connection list.

Returns
port_names

Return type
list of str

See also

port_indexes, *network*

skrf.circuit.Circuit.port_z0

property Circuit.port_z0: ndarray

Return the external port impedances.

Returns
z0s – shape *f x nb_ports*

Return type
numpy.ndarray

skrf.circuit.Circuit.s

property Circuit.s: ndarray

Return the global scattering parameters of the circuit.

Return the scattering parameters of both "inner" and "outer" ports.

Note

The original implementation of this method calculated the S-parameters [S] using the global scattering matrix [C] and the concatenated intersection matrix [X] by solving the equation: [S] = [X] @ ([E] - [C] @ [X])^-1

where [E] is the identity matrix. However, this approach is computationally inefficient for large networks due to its numerical bottlenecks.

The current implementation optimizes the calculation by leveraging matrix algebra to simplify the relationship between the incident {a} and reflected {b} power waves. The key steps are as follows:

1. **Define the relationship between {b} and {a} using [X] and [C]:**
   {b} = [X] @ ([E] - [C] @ [X])^-1 @ {a}
2. **Simplify the relationship between {a} and {b}:**
   {a} = ([X] @ ([E] - [C] @ [X])^-1)^-1 @ {b}
   = ([E] - [C] @ [X]) @ [X]^-1 @ {b} = ([X]^-1 - [C] @ [X] @ [X]^-1) @ {b} = ([X]^-1 - [C]) @ {b}
3. **Derive the S-parameters matrix [S] from the simplified relationship:**
   [S] = ([X]^-1 - [C])^-1
4. **Leverage the property of the concatenated intersection matrix [X]:**
   - **[X] is a block diagonal matrix composed of individual intersection matrices [X]_k:**
     [X] = diag([X_1], [X_2], ..., [X_n]).
   - **Each [X]_k represents an ideal lossless power splitter, which is unitary:**
     [X]_k^H @ [X]_k = I, where [X]_k^H is the conjugate transpose of [X]_k.
   - **Due to reciprocity, the inverse of [X]_k is its conjugate:**
     [X]_k^-1 = [X]_k^*.
5. **Substitute the property into the S-parameters equation:**
   [S] = ([X^T]^* - [C])^-1

This approach avoids the explicit computation of [X]^-1, significantly improving computational efficiency for large-scale networks.

Returns
S – global scattering parameters of the circuit.

Return type
numpy.ndarray

skrf.circuit.Circuit.s_external

property Circuit.**s_external: ndarray**

Return the scattering parameters for the external ports.

Returns
S – Scattering parameters of the circuit for the external ports. Shape *f x nb_ports x nb_ports*

Return type
numpy.ndarray

skrf.circuit.Circuit.z0

property Circuit.**z0: ndarray**

Characteristic impedances of "internal" ports.

Returns
z0 – Characteristic impedances of both "inner" and "outer" ports

Return type
complex array of shape (nfreqs, *nports*)

Methods

Ground	Return a 1-port network of a grounded link.
Open	Return a 1-port network of an open link.
Port	Return a 1-port Network to be used as a Circuit port.
SeriesImpedance	Return a 2-port network of a series impedance.
ShuntAdmittance	Return a 2-port network of a shunt admittance.
__init__	Circuit constructor.
check_duplicate_names	Check that a (ntwk, port) combination appears only once in the connection map
currents	Currents at internal ports.
currents_external	Currents at external ports.
graph	Generate the graph of the circuit.
is_connected	Check if the circuit's graph is connected.
networks_dict	Return the dictionary of Networks from the connection setup X.
networks_list	Return a list of unique networks (sorted by appearing order in connections).
plot_graph	Plot the graph of the circuit using networkx drawing capabilities.
s_active	Return "active" s-parameters of the circuit's network for a defined wave excitation *a*.
update_networks	Update the circuit connections with a new set of networks.
voltages	Voltages at internal ports.
voltages_external	Voltages at external ports
vswr_active	Return the "active" VSWR of the circuit's network for a defined wave excitation a.
y_active	Return the "active" Y-parameters of the circuit's network for a defined wave excitation a.
z_active	Return the "active" Z-parameters of the circuit's network for a defined wave excitation a.

skrf.circuit.Circuit.Ground

classmethod Circuit.**Ground**(*frequency*, *name*, *z0=50*)

Return a 1-port network of a grounded link.

The network topology is:

```
(Port)-----Gnd
```

Passing the frequency and a name is mandatory.

The ground link is implemented by media.short object.

Parameters

- **frequency** (*Frequency*) – Frequency common to all other networks in the circuit
- **name** (*string*) – Name of the ground.
- **z0** (*real, optional*) – Characteristic impedance of the port. Default is 50 Ohm.

Returns

ground – 1-port network

Return type
Network object

Examples

```
In [1]: freq = rf.Frequency(start=1, stop=2, npoints=101)

In [2]: ground = rf.circuit.Circuit.Ground(freq, name='GND')
```

skrf.circuit.Circuit.Open

classmethod Circuit.**Open**(*frequency*, *name*, *z0=50*)

Return a 1-port network of an open link.

The network topology is:

```
(Port)-----Open
```

Passing the frequency and name is mandatory.

The open link is implemented by media.open object.

Parameters

- **frequency** (*Frequency*) – Frequency common to all other networks in the circuit
- **name** (*string*) – Name of the open.
- **z0** (*real, optional*) – Characteristic impedance of the port. Default is 50 Ohm.

Returns
open – 1-port network

Return type
Network object

Examples

```
In [1]: freq = rf.Frequency(start=1, stop=2, npoints=101)

In [2]: open = rf.circuit.Circuit.Open(freq, name='open')
```

skrf.circuit.Circuit.Port

classmethod Circuit.**Port**(*frequency*, *name*, *z0=50*)

Return a 1-port Network to be used as a Circuit port.

The network topology is:

```
(Port)
```

Parameters

- **frequency** (*Frequency*) – Frequency common to all other networks in the circuit
- **name** (*string*) – Name of the port.
- **z0** (*real, optional*) – Characteristic impedance of the port. Default is 50 Ohm.

Returns
port – 1-port network

Return type
Network object

Examples

```
In [1]: freq = rf.Frequency(start=1, stop=2, npoints=101)

In [2]: port1 = rf.circuit.Circuit.Port(freq, name='Port1')
```

skrf.circuit.Circuit.SeriesImpedance

classmethod Circuit.**SeriesImpedance**(*frequency*, *Z*, *name*, *z0=50*)

Return a 2-port network of a series impedance.

The network topology is:

```
(Port 1)-----[Z]-----(Port 2)
```

Passing the frequency and name is mandatory.

Parameters

- **frequency** (*Frequency*) – Frequency common to all other networks in the circuit
- **Z** (*complex array of shape n_freqs or complex*) – Impedance
- **name** (*string*) – Name of the series impedance
- **z0** (*real, optional*) – Characteristic impedance of the port. Default is 50 Ohm.

Returns
serie_impedance – 2-port network

Return type
Network object

Examples

```
In [1]: freq = rf.Frequency(start=1, stop=2, npoints=101)

In [2]: open = rf.circuit.Circuit.SeriesImpedance(freq, rf.constants.INF, name=
↪'series_impedance')
```

skrf.circuit.Circuit.ShuntAdmittance

classmethod Circuit.**ShuntAdmittance**(*frequency*, *Y*, *name*, *z0=50*)

Return a 2-port network of a shunt admittance.

The network topology is:

```
(Port 1)-----(Port 2)
          |
         [Y]
          |
         Gnd
```

Passing the frequency and name is mandatory.

Parameters

- **frequency** (*Frequency*) – Frequency common to all other networks in the circuit
- **Y** (*complex array of shape n_freqs or complex*) – Admittance
- **name** (*string*) – Name of the shunt admittance
- **z0** (*real, optional*) – Characteristic impedance of the port. Default is 50 Ohm.

Returns

shunt_admittance – 2-port network

Return type

Network object

Examples

```
In [1]: freq = rf.Frequency(start=1, stop=2, npoints=101)

In [2]: short = rf.circuit.Circuit.ShuntAdmittance(freq, rf.constants.INF, name=
↪'shunt_admittance')
```

skrf.circuit.Circuit.__init__

Circuit.__init__(*connections*, *name=None*, *, *auto_reduce=False*, ***kwargs*)

Circuit constructor. Creates a circuit made of a set of N-ports networks.

Parameters

- **connections** (*list of list of tuples*) – Description of circuit connections. Each connection is a described by a list of tuple. Each tuple contains (network, network_port_nb). Port number indexing starts from zero. Network's port not explicitly listed will be treated as matched. Network's port listed individually will be treated as open.
- **name** (*string, optional*) – Name assigned to the circuit (Network). Default is None.
- **auto_reduce** (*bool, optional*) – If True, the circuit will be automatically reduced using *reduce_circuit()*. This will change the circuit connections description, affecting inner current and voltage distributions. Suitable for cases where only the S-parameters of the final circuit ports are of interest. Default is False. If *check_duplication*, *split_ground* or *max_nports* are provided as kwargs, *auto_reduce* will be automatically set to True, as this indicates an intent to use the *reduce_circuit* method.
- ****kwargs** – keyword arguments passed to *reduce_circuit* method.

  check_duplication
  [boolean. Default is True.] Controls whether to check the connections have duplicate names.

  split_ground
  [boolean. Default is False.] Controls whether to split the global ground connection to independent.

  split_multi
  [boolean, optional.] If True, use a splitter to handle connections involving more than two components. This approach increases the computational load for individual computations. However, it proves advantageous for batch processing by enabling a more comprehensive reduction of circuits, leading to more efficiency in batch computations. Default is False.

max_nports
[int. Default is 20.] Controls the maximum number of ports of a Network that can be reduced in circuit. If a Network in the circuit has a number of ports (nports), using the Network.connect() method to reduce the circuit's dimensions becomes less efficient compared to directly calculating it with Circuit.s_external. This value depends on the performance of the computer and the scale of the circuit.

dynamic_networks
[tuple. Default is an empty tuple.] Sequence of Networks to be skipped during the reduction process.

Return type
None

Examples

Example of connections between two 1-port networks:

```
connections = [
    [(network1, 0), (network2, 0)],
]
```

Example of a connection between three 1-port networks connected to a single node:

```
connections = [
    [(network1, 0), (network2, 0), (network3, 0)]
]
```

Example of a connection between two 1-port networks (port1 and port2) and two 2-ports networks (ntw1 and ntw2):

```
connections = [
    [(port1, 0), (ntw1, 0)],
    [(ntw1, 1), (ntw2, 0)],
    [(ntw2, 1), (port2, 0)]
]
```

Example of a connection between three 1-port networks (port1, port2 and port3) and a 3-ports network (ntw):

```
connections = [
    [(port1, 0), (ntw, 0)],
    [(port2, 0), (ntw, 1)],
    [(port3, 0), (ntw, 2)]
]
```

Example of a connection between three 1-port networks (port1, port2 and open) and a 3-ports network (ntw):

```
connections = [
    [(port1, 0), (ntw, 0)],
    [(open, 0), (ntw, 1)],
    [(port2, 0), (ntw, 2)]
]
# or equivalently
connections = [
    [(port1, 0), (ntw, 0)],
```

(continues on next page)

(continued from previous page)

```
    [(ntw, 1)],
    [(port2, 0), (ntw, 2)]
]
```

Example of a connection between three 1-port networks (port1, port2 and match) and a 3-ports network (ntw):

```
connections = [
    [(port1, 0), (ntw, 0)],
    [(match, 0), (ntw, 1)],
    [(port2, 0), (ntw, 2)]
]
# or equivalently
connections = [
    [(port1, 0), (ntw, 0)],
    [(port2, 0), (ntw, 2)]
]
```

NB1: Creating 1-port network to be used as a port should be made with *Port()*

NB2: The external ports indexing is defined by the order of appearance of the ports in the connections list. Thus, the first network identified as a port will be the 1st port of the resulting network (index 0), the second network identified as a port will be the second port (index 1), etc.

NB3: When a port of a network is listed individually in the connections, the circuit will treat this port with a reflection coefficient of 1, equivalent to an Open. Conversely, if a port is not explicitly included in the connections,the circuit will treat it as matched.

skrf.circuit.Circuit.check_duplicate_names

classmethod Circuit.**check_duplicate_names**(*connections_list*)

Check that a (ntwk, port) combination appears only once in the connection map

Parameters
connections_list (*list[tuple[int, tuple[Network, int]]]*)

skrf.circuit.Circuit.currents

Circuit.**currents**(*power*, *phase*)

Currents at internal ports.

NB: current direction is defined as positive when entering a node.

NB: external current sign are opposite than corresponding internal ones,
as the internal currents are actually flowing into the "port" networks

Parameters

- **power** (*list or array*) – Input power at external ports in Watts [W]
- **phase** (*list or array*) – Input phase at external ports in radian [rad]

Returns
I – Currents in Amperes [A] (peak) at internal ports.

Return type
complex array of shape (nfreqs, *nports*)

skrf.circuit.Circuit.currents_external

Circuit.**currents_external**(*power*, *phase*)

Currents at external ports.

NB: current direction is defined positive when "entering" into port.

Parameters

- **power** (*list or array*) – Input power at external ports in Watts [W]
- **phase** (*list or array*) – Input phase at external ports in radian [rad]

Returns

I – Currents in Amperes [A] (peak) at external ports.

Return type

complex array of shape (nfreqs, *nports*)

skrf.circuit.Circuit.graph

Circuit.**graph**()

Generate the graph of the circuit.

Returns

G – graph object[1] .

Return type

networkx.Graph

References

skrf.circuit.Circuit.is_connected

Circuit.**is_connected**()

Check if the circuit's graph is connected.

Check if every pair of vertices in the graph is connected.

Return type

bool

skrf.circuit.Circuit.networks_dict

Circuit.**networks_dict**(*connections=None*, *min_nports=1*)

Return the dictionary of Networks from the connection setup X.

Parameters

- **connections** (*List, optional*) – connections list, by default None (then uses the *self.connections*)
- **min_nports** (*int, optional*) – min number of ports, by default 1

Returns

Dictionary of Networks

Return type

dict

[1] https://networkx.github.io/

skrf.circuit.Circuit.networks_list

Circuit.**networks_list**(*connections=None*, *min_nports=1*)

Return a list of unique networks (sorted by appearing order in connections).

Parameters

- **connections** (*List, optional*) – connections list, by default None (then uses the *self.connections*)
- **min_nports** (*int, optional*) – min number of ports, by default 1

Returns

List of unique networks

Return type

list

skrf.circuit.Circuit.plot_graph

Circuit.**plot_graph**(***kwargs*)

Plot the graph of the circuit using networkx drawing capabilities.

Customisation options with default values:

```
'network_shape': 's'
'network_color': 'gray'
'network_size', 300
'network_fontsize': 7
'inter_shape': 'o'
'inter_color': 'lightblue'
'inter_size', 300
'port_shape': '>'
'port_color': 'red'
'port_size', 300
'port_fontsize': 7
'edges_fontsize': 5
'network_labels': False
'edge_labels': False
'inter_labels': False
'port_labels': False
'label_shift_x': 0
'label_shift_y': 0
```

skrf.circuit.Circuit.s_active

Circuit.**s_active**(*a*)

Return "active" s-parameters of the circuit's network for a defined wave excitation *a*.

The "active" s-parameter at a port is the reflection coefficients when other ports are excited. It is an important quantity for active phased array antennas.

Active s-parameters are defined by[1]:

$$\text{active}(s)_{mn} = \sum_i s_{mi} \frac{a_i}{a_n}$$

Parameters
a (*complex array of shape (n_ports)*) – forward wave complex amplitude (power-wave formulation[2])

Returns
s_act – active s-parameters for the excitation a

Return type
complex array of shape (n_freqs, n_ports)

References

skrf.circuit.Circuit.update_networks

`Circuit.update_networks`(*networks*, *name=None*, *, *inplace=False*, *auto_reduce=False*, ***kwargs*)

Update the circuit connections with a new set of networks.

Parameters

- **networks** (*tuple[Network]*) – A tuple of Networks to be updated in the circuit.
- **name** (*string, optional*) – Name assigned to the circuit (Network). Default is None.
- **inplace** (*bool, optional*) – If True, the circuit connections will be updated inplace. Default is False.
- **auto_reduce** (*bool, optional*) – If True, the circuit will be automatically reduced using *reduce_circuit()*. This will change the circuit connections description, affecting inner current and voltage distributions. Suitable for cases where only the S-parameters of the final circuit ports are of interest. Default is False. If *check_duplication*, *split_ground* or *max_nports* are provided as kwargs, *auto_reduce* will be automatically set to True, as this indicates an intent to use the *reduce_circuit* method.
- ****kwargs** – keyword arguments passed to *reduce_circuit* method.

  check_duplication kwarg controls whether to check the connections have duplicate names. Default is True.

  split_ground kwarg controls whether to split the global ground connection to independent. Default is False.

  max_nports kwarg controls the maximum number of ports of a Network that can be reduced in circuit. If a Network in the circuit has a number of ports (nports), using the Network.connect() method to reduce the circuit's dimensions becomes less efficient compared to directly calculating it with Circuit.s_external. This value depends on the performance of the computer and the scale of the circuit. Default is 20.

  dynamic_networks kwarg is a sequence of Networks to be skipped during the reduction process. Default is an empty tuple.

[1] D. M. Pozar, IEEE Trans. Antennas Propag. 42, 1176 (1994).

[2] D. Williams, IEEE Microw. Mag. 14, 38 (2013).

Returns
The updated *Circuit* with the specified networks.

Return type
Circuit or None if inplace=True

See also

Circuit.__init__
Circuit constructor method.

Examples

```
>>> import skrf as rf
>>> import numpy as np
>>>
>>> # Create a series of networks and build a circuit
>>> connections = [
...     ...
...     [(ntwk1, 0), (ntwk2, 0)],
...     [(ntwk2, 1), (ntwk3, 0)],
...     ...
... ]
>>> circuit = rf.circuit.Circuit(connections, dynamic_networks=[ntwk2])
>>>
>>> # Update the networks' S-parameters
>>> ntwk2.s = ntwk2.s @ ntwk2.s
>>>
>>> # Update the circuit in traditional way (ntwk2 has been updated)
>>> connections_updated = [
...     ...
...     [(ntwk1, 0), (ntwk2, 0)],
...     [(ntwk2, 1), (ntwk3, 0)],
...     ...
... ]
>>>
>>> circuit_updated_a = rf.circuit.Circuit(connections_updated)
>>>
>>> # Update the circuit by dynamic networks
>>> circuit_updated_b = circuit.update_networks(networks=[ntwk2])
>>>
>>> np.allclose(circuit_updated_b.s_external, circuit_updated_b.s_external)
True
```

skrf.circuit.Circuit.voltages

`Circuit.`**`voltages`**(*power*, *phase*)

Voltages at internal ports.

Parameters

- **power** (*list or array*) – Input power at external ports in Watts [W]
- **phase** (*list or array*) – Input phase at external ports in radian [rad]

Returns
V – Voltages in Volt [V] (peak) at internal ports.

Return type
complex array of shape (nfreqs, *nports*)

skrf.circuit.Circuit.voltages_external

`Circuit.`**`voltages_external`**(*power*, *phase*)

Voltages at external ports

Parameters

- **power** (*list or array*) – Input power at external ports in Watts [W]
- **phase** (*list or array*) – Input phase at external ports in radian [rad]

Returns
V – Voltages in Volt [V] (peak) at ports

Return type
complex array of shape (nfreqs, *nports*)

skrf.circuit.Circuit.vswr_active

`Circuit.`**`vswr_active`**(a)

Return the "active" VSWR of the circuit's network for a defined wave excitation a.

The "active" VSWR is defined by :

$$\text{active}(vswr)_m = \frac{1 + |\text{active}(s)_m|}{1 - |\text{active}(s)_m|}$$

where $\text{active}(s)_m$ the active S-parameter of port m.

Parameters
a (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns
vswr_act – active VSWR for the excitation a

Return type
complex array of shape (nfreqs, *nports*)

See also

s_active
active S-parameters

z_active
active Z-parameters

y_active
active Y-parameters

skrf.circuit.Circuit.y_active

`Circuit.y_active(`a`)`

Return the "active" Y-parameters of the circuit's network for a defined wave excitation a.

The "active" Y-parameters are defined by:

$$\text{active}(y)_m = y_{0,m}\frac{1-\text{active}(s)_m}{1+\text{active}(s)_m}$$

where $y_{0,m}$ is the characteristic admittance and $\text{active}(s)_m$ the active S-parameter of port m.

Parameters
a (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns
y_act – active Y-parameters for the excitation a

Return type
complex array of shape (nfreqs, *nports*)

See also

s_active
active S-parameters

z_active
active Z-parameters

vswr_active
active VSWR

skrf.circuit.Circuit.z_active

`Circuit.z_active(`a`)`

Return the "active" Z-parameters of the circuit's network for a defined wave excitation a.

The "active" Z-parameters are defined by:

$$\text{active}(z)_m = z_{0,m}\frac{1+\text{active}(s)_m}{1-\text{active}(s)_m}$$

where $z_{0,m}$ is the characteristic impedance and $\text{active}(s)_m$ the active S-parameter of port m.

Parameters
a (*complex array of shape (n_ports)*) – forward wave complex amplitude

Returns
z_act – active Z-parameters for the excitation a

Return type
complex array of shape (nfreqs, *nports*)

See also

s_active
active S-parameters

y_active
active Y-parameters

vswr_active
active VSWR

3.14.2 Representing a Circuit

Circuit.plot_graph(**kwargs)	Plot the graph of the circuit using networkx drawing capabilities.

3.14.3 Network Representations

Circuit.network	Return the Network associated to external ports.
Circuit.s	Return the global scattering parameters of the circuit.
Circuit.s_external	Return the scattering parameters for the external ports.
Circuit.s_active(a)	Return "active" s-parameters of the circuit's network for a defined wave excitation a.
Circuit.z_active(a)	Return the "active" Z-parameters of the circuit's network for a defined wave excitation a.
Circuit.y_active(a)	Return the "active" Y-parameters of the circuit's network for a defined wave excitation a.
Circuit.vswr_active(a)	Return the "active" VSWR of the circuit's network for a defined wave excitation a.
Circuit.port_z0	Return the external port impedances.

3.14.4 Voltages and Currents

Circuit.voltages(power, phase)	Voltages at internal ports.
Circuit.voltages_external(power, phase)	Voltages at external ports
Circuit.currents(power, phase)	Currents at internal ports.
Circuit.currents_external(power, phase)	Currents at external ports.

3.14.5 Circuit internals

Circuit.networks_dict([connections, min_nports])	Return the dictionary of Networks from the connection setup X.
Circuit.networks_list([connections, min_nports])	Return a list of unique networks (sorted by appearing order in connections).
Circuit.connections_nb	Return the number of intersections in the circuit.
Circuit.connections_list	Return the full list of connections, including intersections.
Circuit.nodes_nb	Return the number of nodes in the circuit.
Circuit.dim	Return the dimension of the C, X and global S matrices.
Circuit.intersections_dict	Return a dictionary of all intersections with associated ports and z0:
Circuit.port_indexes	Return the indexes of the "external" ports.

continues on next page

Table 154 – continued from previous page

Circuit.C	Return the global scattering matrix [C] of the networks in C-order.
Circuit.X	Return the concatenated intersection matrix [X] of the circuit in C-order.

3.14.6 Circuit reduction

reduce_circuit(connections[, ...])	Return a reduced equivalent circuit connections with fewer components.
Circuit._REDUCE_OPTIONS	Optional parameters passed to *reduce_circuit*.

skrf.circuit.reduce_circuit

skrf.circuit.**reduce_circuit**(*connections*, *check_duplication=True*, *split_ground=True*, *split_multi=False*, *max_nports=20*, *dynamic_networks=()*)

Return a reduced equivalent circuit connections with fewer components.

The reduced equivalent circuit connections allows faster calculation of the circuit network.

Parameters

- **connections** (*list[list[tuple[Network, int]]]*.) – The connection list to reduce.
- **check_duplication** (*bool, optional*.) – If True, check if the connections have duplicate names. Default is True.
- **split_ground** (*bool, optional*.) – If True, split the global ground connection to independent ground connections. Default is True.
- **split_multi** (*bool, optional*.) – If True, use a splitter to handle connections involving more than two components. This approach increases the computational load for individual computations. However, it proves advantageous for batch processing by enabling a more comprehensive reduction of circuits, leading to more efficiency in batch computations. Default is False.
- **max_nports** (*int, optional*.) – The maximum number of ports of a Network that can be reduced in circuit. If a Network in the circuit has a number of ports (nports), using the Network.connect() method to reduce the circuit's dimensions becomes less efficient compared to directly calculating it with Circuit.s_external. This value depends on the performance of the computer and the scale of the circuit. Default is 20.
- **dynamic_networks** (*Sequence[Network], optional*.) – A sequence of Networks to ignore in the reduction process. Default is an empty tuple.

Returns

reduced_cnxs – The reduced connections.

Return type

list.

Examples

```
>>> import skrf as rf
>>> import numpy as np
>>> circuit = rf.circuit.Circuit(connections)
```

(continues on next page)

(continued from previous page)

```
>>> reduced_cnxs = rf.circuit.reduce_circuit(connections)
>>> reduced_circuit = rf.circuit.Circuit(reduced_cnxs)
>>> ntwkA = circuit.network
>>> ntwkB = reduced_circuit.network
>>> np.allclose(ntwkA.s, ntwkB.s)
True
```

skrf.circuit.Circuit._REDUCE_OPTIONS

Attributes

check_duplication	
split_ground	
split_multi	
max_nports	
dynamic_networks	

skrf.circuit.Circuit._REDUCE_OPTIONS.check_duplication

skrf.circuit.Circuit._REDUCE_OPTIONS.split_ground

skrf.circuit.Circuit._REDUCE_OPTIONS.split_multi

skrf.circuit.Circuit._REDUCE_OPTIONS.max_nports

skrf.circuit.Circuit._REDUCE_OPTIONS.dynamic_networks

Methods

__init__	
clear	
copy	
fromkeys	Create a new dictionary with keys from iterable and values set to value.
get	Return the value for key if key is in the dictionary, else default.
items	
keys	
pop	If the key is not found, return the default if given; otherwise, raise a KeyError.
popitem	Remove and return a (key, value) pair as a 2-tuple.
setdefault	Insert key with a value of default if key is not in the dictionary.
update	If E is present and has a .keys() method, then does: for k in E.keys(): D[k] = E[k] If E is present and lacks a .keys() method, then does: for k, v in E: D[k] = v In either case, this is followed by: for k in F: D[k] = F[k]
values	

skrf.circuit.Circuit._REDUCE_OPTIONS.__init__

skrf.circuit.Circuit._REDUCE_OPTIONS.clear

skrf.circuit.Circuit._REDUCE_OPTIONS.copy

skrf.circuit.Circuit._REDUCE_OPTIONS.fromkeys

skrf.circuit.Circuit._REDUCE_OPTIONS.get

skrf.circuit.Circuit._REDUCE_OPTIONS.items

skrf.circuit.Circuit._REDUCE_OPTIONS.keys

skrf.circuit.Circuit._REDUCE_OPTIONS.pop

skrf.circuit.Circuit._REDUCE_OPTIONS.popitem

skrf.circuit.Circuit._REDUCE_OPTIONS.setdefault

skrf.circuit.Circuit._REDUCE_OPTIONS.update

skrf.circuit.Circuit._REDUCE_OPTIONS.values

3.14.7 Graph representation

Circuit.graph()	Generate the graph of the circuit.
Circuit.G	Generate the graph of the circuit.
Circuit.edges	Return the list of all circuit connections
Circuit.edge_labels	Return a dictionary describing the port and z0 of all graph edges.

3.15 device (*skrf.media.device*)

Device is a generic n-port microwave device class to create common Network.

3.15.1 Device Class

Device(media)	A n-port microwave device

skrf.media.device.Device

class skrf.media.device.**Device**(*media*)

A n-port microwave device

Parameters

media (*skrf.media.Media*)

Attributes

ntwk	the network representation of a given device

skrf.media.device.Device.ntwk

abstract property Device.**ntwk**

the network representation of a given device

Methods

__init__

skrf.media.device.Device.__init__

Device.**__init__**(*media*)

3.15.2 Example Devices

MatchedSymmetricCoupler(media[, c, t, ...])	A Matched Symmetric Coupler
Hybrid(media[, t_phase, phase_diff])	A 3dB Coupler of given phase difference
QuadratureHybrid(media[, t_phase])	A 3dB Coupler with 90deg phase diff between transmit and coupled arms
Hybrid180(media[, nports])	180degree hybrid
DualCoupler(media[, c1, c2, c1kw, c2kw])	Pair of back-to-back directional couplers

skrf.media.device.MatchedSymmetricCoupler

class skrf.media.device.**MatchedSymmetricCoupler**(*media*, *c=None*, *t=None*, *t_phase=0*, *phase_diff=0*, *nports=4*, **args*, ***kw*)

A Matched Symmetric Coupler

The resultant ntwk port assignment is as follows:

- 0 - insertion
- 1 - transmit
- 2 - coupled
- 3 - isolated

Attributes

c	
ntwk	the network representation of a given device
t	

skrf.media.device.MatchedSymmetricCoupler.c

property MatchedSymmetricCoupler.**c**

skrf.media.device.MatchedSymmetricCoupler.ntwk

property MatchedSymmetricCoupler.**ntwk**

the network representation of a given device

skrf.media.device.MatchedSymmetricCoupler.t

property MatchedSymmetricCoupler.**t**

Methods

__init__	
from_dbdeg	Create a coupler in terms of couping(dB) and phase offset(deg)

skrf.media.device.MatchedSymmetricCoupler.__init__

MatchedSymmetricCoupler.**__init__**(*media*, *c=None*, *t=None*, *t_phase=0*, *phase_diff=0*, *nports=4*, **args*, ***kw*)

skrf.media.device.MatchedSymmetricCoupler.from_dbdeg

classmethod MatchedSymmetricCoupler.**from_dbdeg**(*media*, *db*, *deg=0*, **args*, ***kw*)

Create a coupler in terms of couping(dB) and phase offset(deg)

Parameters

- **media** (*skrf.media.Media Object*)
- **db** (*number or array-like*) – the magnitude of the coupling value (in dB), sign dont matter
- **deg** (*number or array-like*) – phase offset between the transmit and coupled arms. defined as : coupled arm = transmit arm +phase offset
- ***args** (*passed to self.__init__()*)
- ****kw** (*passed to self.__init__()*)

skrf.media.device.Hybrid

class skrf.media.device.**Hybrid**(*media*, *t_phase=180*, *phase_diff=0*, **args*, ***kw*)

A 3dB Coupler of given phase difference

Attributes

c	
ntwk	the network representation of a given device
t	

skrf.media.device.Hybrid.c

property Hybrid.**c**

skrf.media.device.Hybrid.ntwk

property Hybrid.**ntwk**

the network representation of a given device

skrf.media.device.Hybrid.t

property Hybrid.**t**

Methods

__init__	
from_dbdeg	Create a coupler in terms of couping(dB) and phase offset(deg)

skrf.media.device.Hybrid.__init__

Hybrid.**__init__**(*media*, *t_phase=180*, *phase_diff=0*, **args*, ***kw*)

skrf.media.device.Hybrid.from_dbdeg

classmethod Hybrid.**from_dbdeg**(*media*, *db*, *deg=0*, **args*, ***kw*)

Create a coupler in terms of couping(dB) and phase offset(deg)

Parameters

- **media** (*skrf.media.Media Object*)
- **db** (*number or array-like*) – the magnitude of the coupling value (in dB), sign dont matter
- **deg** (*number or array-like*) – phase offset between the transmit and coupled arms. defined as : coupled arm = transmit arm +phase offset
- ***args** (*passed to self.__init__()*)
- ****kw** (*passed to self.__init__()*)

skrf.media.device.QuadratureHybrid

class skrf.media.device.**QuadratureHybrid**(*media*, *t_phase=0*, **args*, ***kw*)

A 3dB Coupler with 90deg phase diff between transmit and coupled arms

Attributes

c	
ntwk	the network representation of a given device
t	

skrf.media.device.QuadratureHybrid.c

property QuadratureHybrid.**c**

skrf.media.device.QuadratureHybrid.ntwk

property QuadratureHybrid.**ntwk**

the network representation of a given device

skrf.media.device.QuadratureHybrid.t

property QuadratureHybrid.**t**

Methods

__init__	
from_dbdeg	Create a coupler in terms of couping(dB) and phase offset(deg)

skrf.media.device.QuadratureHybrid.__init__

QuadratureHybrid.**__init__**(*media*, *t_phase=0*, **args*, ***kw*)

skrf.media.device.QuadratureHybrid.from_dbdeg

classmethod QuadratureHybrid.**from_dbdeg**(*media*, *db*, *deg=0*, **args*, ***kw*)

Create a coupler in terms of couping(dB) and phase offset(deg)

Parameters

- **media** (*skrf.media.Media Object*)
- **db** (*number or array-like*) – the magnitude of the coupling value (in dB), sign dont matter
- **deg** (*number or array-like*) – phase offset between the transmit and coupled arms. defined as : coupled arm = transmit arm +phase offset
- ***args** (*passed to self.__init__()*)
- ****kw** (*passed to self.__init__()*)

skrf.media.device.Hybrid180

class skrf.media.device.**Hybrid180**(*media*, *nports=4*, **args*, ***kw*)

180degree hybrid

This device can be used to combine two signals in and out of phase, or as a divider, with outputs in or out of phase.

The resultant ntwk port assignment is as follows:

- 0 - sum (A+B)
- 1 - input A
- 2 - input B

- 3 - delta (A-B)

http://www.microwaves101.com/encyclopedias/hybrid-couplers

Attributes

ntwk	the network representation of a given device

skrf.media.device.Hybrid180.ntwk

property Hybrid180.**ntwk**

the network representation of a given device

Methods

__init__

skrf.media.device.Hybrid180.__init__

Hybrid180.**__init__**(*media*, *nports=4*, **args*, ***kw*)

skrf.media.device.DualCoupler

class skrf.media.device.**DualCoupler**(*media*, *c1=np.float64(0.7071067811865475)*, *c2=None*, *c1kw=None*, *c2kw=None*)

Pair of back-to-back directional couplers

Ports are as follows:

- 0 : insertion of coupler 1
- 1 : insertion on coupler 2
- 2 : coupled on coupler 1
- 3 : coupled on coupler 2

Attributes

ntwk	the network representation of a given device

skrf.media.device.DualCoupler.ntwk

property DualCoupler.**ntwk**

the network representation of a given device

Methods

__init__

skrf.media.device.DualCoupler.__init__

DualCoupler.**__init__**(*media*, *c1=np.float64(0.7071067811865475)*, *c2=None*, *c1kw=None*, *c2kw=None*)

3.16 mathFunctions (*skrf.mathFunctions*)

Provides commonly used mathematical functions.

3.16.1 Mathematical Constants

Some convenient constants are defined in the *skrf.constants* module.

3.16.2 Complex Component Conversion

complex_2_reim(z)	Return real and imaginary parts of a complex number.
complex_2_magnitude(z)	Return the magnitude of the complex argument.
complex_2_db(z)	Return the magnitude in dB of a complex number (as $20\log_{10}(\|z\|)$)..
complex_2_db10(z)	Return the magnitude in dB of a complex number (as $10\log_{10}(\|z\|)$).
complex_2_radian(z)	Return the angle complex argument in radian.
complex_2_degree(z)	Returns the angle complex argument in degree.
complex_2_magnitude(z)	Return the magnitude of the complex argument.
complex_2_quadrature(z)	Take a complex number and returns quadrature, which is (length, arc-length from real axis)
complex_components(z)	Break up a complex array into all possible scalar components.

skrf.mathFunctions.complex_2_reim

skrf.mathFunctions.**complex_2_reim**(*z*)

Return real and imaginary parts of a complex number.

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns

- **real** (*array like or scalar*) – real part of input
- **imag** (*array like or scalar*) – imaginary part of input

skrf.mathFunctions.complex_2_magnitude

skrf.mathFunctions.**complex_2_magnitude**(*z*)

Return the magnitude of the complex argument.

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns
mag

Return type
ndarray or scalar

skrf.mathFunctions.complex_2_db

skrf.mathFunctions.**complex_2_db**(*z*)

Return the magnitude in dB of a complex number (as $20\log_{10}(|z|)$)..

The magnitude in dB is defined as $20\log_{10}(|z|)$ where z is a complex number.

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns
mag20dB

Return type
ndarray or scalar

skrf.mathFunctions.complex_2_db10

skrf.mathFunctions.**complex_2_db10**(*z*)

Return the magnitude in dB of a complex number (as $10\log_{10}(|z|)$).

The magnitude in dB is defined as $10\log_{10}(|z|)$ where z is a complex number.

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns
mag10dB

Return type
ndarray or scalar

skrf.mathFunctions.complex_2_radian

skrf.mathFunctions.**complex_2_radian**(*z*)

Return the angle complex argument in radian.

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns
ang_rad – The counterclockwise angle from the positive real axis on the complex plane in the range `(-pi, pi]`, with dtype as numpy.float64.

Return type
ndarray or scalar

skrf.mathFunctions.complex_2_degree

skrf.mathFunctions.**complex_2_degree**(*z*)

Returns the angle complex argument in degree.

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns
ang_deg

Return type
ndarray or scalar

skrf.mathFunctions.complex_2_quadrature

skrf.mathFunctions.**complex_2_quadrature**(*z*)

Take a complex number and returns quadrature, which is (length, arc-length from real axis)

Arc-length is calculated as $|z| \arg(z)$.

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns

- **mag** (*array like or scalar*) – magnitude (length)
- **arc_length** (*array like or scalar*) – arc-length from real axis: angle*magnitude

skrf.mathFunctions.complex_components

skrf.mathFunctions.**complex_components**(*z*)

Break up a complex array into all possible scalar components.

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns

- **c_real** (*array like or scalar*) – real part
- **c_imag** (*array like or scalar*) – imaginary part
- **c_angle** (*array like or scalar*) – angle in degrees
- **c_mag** (*array like or scalar*) – magnitude
- **c_arc** (*array like or scalar*) – arclength from real axis, angle*magnitude

3.16.3 Magnitude and decibel

magnitude_2_db(z[, zero_nan])	Convert linear magnitude to dB.
mag_2_db(z[, zero_nan])	Convert linear magnitude to dB.
mag_2_db10(z[, zero_nan])	Convert linear magnitude to dB.
db_2_magnitude(z)	Convert dB to linear magnitude.
db_2_mag(z)	Convert dB to linear magnitude.
db10_2_mag(z)	Convert dB to linear magnitude.

skrf.mathFunctions.magnitude_2_db

skrf.mathFunctions.**magnitude_2_db**(*z*, *zero_nan=True*)

Convert linear magnitude to dB.

Parameters

- **z** (*number or array_like*) – A complex number or sequence of complex numbers
- **zero_nan** (*bool, optional*) – Replace NaN with zero. The default is True.

Returns
z – Magnitude in dB given by 20*log10(|z|)

Return type
number or array_like

skrf.mathFunctions.mag_2_db

skrf.mathFunctions.**mag_2_db**(*z*, *zero_nan=True*)

Convert linear magnitude to dB.

Parameters

- **z** (*number or array_like*) – A complex number or sequence of complex numbers
- **zero_nan** (*bool, optional*) – Replace NaN with zero. The default is True.

Returns

z – Magnitude in dB given by 20*log10(|z|)

Return type

number or array_like

skrf.mathFunctions.mag_2_db10

skrf.mathFunctions.**mag_2_db10**(*z*, *zero_nan=True*)

Convert linear magnitude to dB.

Parameters

- **z** (*array_like*) – A complex number or sequence of complex numbers
- **zero_nan** (*bool, optional*) – Replace NaN with zero. The default is True.

Returns

z – Magnitude in dB given by 10*log10(|z|)

Return type

array_like

skrf.mathFunctions.db_2_magnitude

skrf.mathFunctions.**db_2_magnitude**(*z*)

Convert dB to linear magnitude.

Parameters

z (*number or array_like*) – A complex number or sequence of complex numbers

Returns

z – 10**((z)/20) where z is a complex number

Return type

number or array_like

skrf.mathFunctions.db_2_mag

skrf.mathFunctions.**db_2_mag**(*z*)

Convert dB to linear magnitude.

Parameters

z (*number or array_like*) – A complex number or sequence of complex numbers

Returns

z – 10**((z)/20) where z is a complex number

Return type

number or array_like

skrf.mathFunctions.db10_2_mag

skrf.mathFunctions.**db10_2_mag**(*z*)

Convert dB to linear magnitude.

Parameters
z (*array_like*) – A complex number or sequence of complex numbers

Returns
z – 10**((z)/10) where z is a complex number

Return type
array_like

3.16.4 Phase Unwrapping

unwrap_rad(phi)	Unwraps a phase given in radians.
sqrt_phase_unwrap(z)	Take the square root of a complex number with unwrapped phase.

skrf.mathFunctions.unwrap_rad

skrf.mathFunctions.**unwrap_rad**(*phi*)

Unwraps a phase given in radians.

Parameters
phi (*number or array_like*) – phase in radians

Returns
phi – unwrapped phase in radians

Return type
number of array_like

skrf.mathFunctions.sqrt_phase_unwrap

skrf.mathFunctions.**sqrt_phase_unwrap**(*z*)

Take the square root of a complex number with unwrapped phase.

This idea came from Lihan Chen.

$$\sqrt{|z|}\exp(\arg_{unwrap}(z)/2)$$

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns
z – A complex number or sequence of complex numbers

Return type
number of array_like

3.16.5 Unit Conversion

radian_2_degree(rad)	Convert angles from radians to degrees.
degree_2_radian(deg)	Convert angles from degrees to radians.

continues on next page

Table 176 – continued from previous page

np_2_db(x)	Converts a value in Nepers (Np) to decibel (dB).
db_2_np(db)	Converts a value in decibel (dB) to neper (Np).
feet_2_meter([feet])	Convert length in feet to meter.
meter_2_feet([meter])	Convert length in meter to feet.
db_per_100feet_2_db_per_100meter([...])	Convert attenuation values given in dB/100ft to dB/100m.

skrf.mathFunctions.radian_2_degree

skrf.mathFunctions.**radian_2_degree**(*rad*)

Convert angles from radians to degrees.

Parameters
rad (*number or array_like*) – Angle in radian

Returns
deg – Angle in degree

Return type
number or array_like

skrf.mathFunctions.degree_2_radian

skrf.mathFunctions.**degree_2_radian**(*deg*)

Convert angles from degrees to radians.

Parameters
deg (*number or array_like*) – Angle in radian

Returns
rad – Angle in degree

Return type
number or array_like

skrf.mathFunctions.np_2_db

skrf.mathFunctions.**np_2_db**(*x*)

Converts a value in Nepers (Np) to decibel (dB).

Parameters

- **np** (*number or array_like*) – A real number or sequence of real numbers
- **x** (*Number | Sequence[Number] | ndarray*)

Returns
db – A real number of sequence of real numbers

Return type
number or array_like

skrf.mathFunctions.db_2_np

skrf.mathFunctions.**db_2_np**(*db*)

Converts a value in decibel (dB) to neper (Np).

Parameters
db (*number or array_like*) – A real number or sequence of real numbers

Returns
np – A real number of sequence of real numbers

Return type
number or array_like

skrf.mathFunctions.feet_2_meter

skrf.mathFunctions.**feet_2_meter**(*feet=1*)

Convert length in feet to meter.

1 foot is equal to 0.3048 meters.

Parameters
feet (*number or array-like, optional*) – length in feet. Default is 1.

Returns
meter – length in meter

Return type
number or array-like

See also

meter_2_feet

skrf.mathFunctions.meter_2_feet

skrf.mathFunctions.**meter_2_feet**(*meter=1*)

Convert length in meter to feet.

1 meter is equal to 0.3.28084 feet.

Parameters
meter (*number or array-like, optional*) – length in meter. Default is 1.

Returns
feet – length in feet

Return type
number or array-like

See also

feet_2_meter

skrf.mathFunctions.db_per_100feet_2_db_per_100meter

skrf.mathFunctions.**db_per_100feet_2_db_per_100meter**(*db_per_100feet=1*)

Convert attenuation values given in dB/100ft to dB/100m.

db_per_100meter = db_per_100feet * rf.meter_2_feet()

Parameters
db_per_100feet (*number or array-like, optional*) – Attenuation in dB/ 100 ft. Default is 1.

Returns
db_per_100meter – Attenuation in dB/ 100 m

Return type
number or array-like

See also

meter_2_feet, *feet_2_meter*, *np_2_db*, *db_2_np*

3.16.6 Scalar-Complex Conversion

These conversions are useful for wrapping other functions that don't support complex numbers.

complex2Scalar(z)	Serialize a list/array of complex numbers
scalar2Complex(s)	Unserialize a list/array of real and imag numbers into a complex array.

skrf.mathFunctions.complex2Scalar

skrf.mathFunctions.**complex2Scalar**(*z*)

Serialize a list/array of complex numbers

Parameters
z (*number or array_like*) – A complex number or sequence of complex numbers

Returns
re_im – produce the following array for an input z: z[0].real, z[0].imag, z[1].real, z[1].imag, etc.

Return type
array_like

See also

scalar2Complex

skrf.mathFunctions.scalar2Complex

skrf.mathFunctions.**scalar2Complex**(*s*)

Unserialize a list/array of real and imag numbers into a complex array.

Inverse of *complex2Scalar()*.

Parameters
s (*array_like*) – an array with real and imaginary parts ordered as: z[0].real, z[0].imag, z[1].real, z[1].imag, etc.

Returns
z – A complex number or sequence of complex number

Return type
Number or array_like

See also

complex2Scalar

3.16.7 Special Functions

dirac_delta(x)	Calculate Dirac function.
neuman(x)	Calculate Neumans number.
null(A[, eps])	Calculate the null space of matrix A.
cross_ratio(a, b, c, d)	Calculate the cross ratio of a quadruple of distinct points on the real line.

skrf.mathFunctions.dirac_delta

skrf.mathFunctions.**dirac_delta**(x)

Calculate Dirac function.

Dirac function $\delta(x)$ defined as $\delta(x) = 1$ if x=0, 0 otherwise.

Parameters
x (*number of array_like*) – A real number or sequence of real numbers

Returns
delta – 1 or 0

Return type
number of array_like

References

https://en.wikipedia.org/wiki/Dirac_delta_function

skrf.mathFunctions.neuman

skrf.mathFunctions.**neuman**(x)

Calculate Neumans number.

It is defined as:

$$2 - \delta(x)$$

where δ is the Dirac function.

Parameters
x (*number or array_like*) – A real number or sequence of real numbers

Returns
y – A real number or sequence of real numbers

Return type
number or array_like

See also

dirac_delta

skrf.mathFunctions.null

skrf.mathFunctions.**null**(*A*, *eps=1e-15*)

Calculate the null space of matrix A.

Parameters

- **A** (*array_like*)
- **eps** (*float*)

Returns
null_space

Return type
array_like

References

https://scipy-cookbook.readthedocs.io/items/RankNullspace.html https://stackoverflow.com/questions/5889142/python-numpy-scipy-finding-the-null-space-of-a-matrix

skrf.mathFunctions.cross_ratio

skrf.mathFunctions.**cross_ratio**(*a*, *b*, *c*, *d*)

Calculate the cross ratio of a quadruple of distinct points on the real line.

The cross ratio is defined as:

$$r = \frac{(a-b)(c-d)}{(a-d)(c-b)}$$

Parameters

- **a** (*array-like or number*)
- **b** (*array-like or number*)
- **c** (*array-like or number*)
- **d** (*array-like or number*)

Returns
r

Return type
array-like or number

References

https://en.wikipedia.org/wiki/Cross-ratio

3.16.8 Random Number Generation

set_rand_rng(rng)	Set the global numpy random number generator instance for generating random numbers in scikit-rf.
rand_rng()	Obtain the global numpy random number generator instance.
rand_c(*size[, rng])	Creates a complex random array of shape s.

skrf.mathFunctions.set_rand_rng

skrf.mathFunctions.**set_rand_rng**(*rng*)

Set the global numpy random number generator instance for generating random numbers in scikit-rf. This is useful for fixing a random seed for reproducible Monte Carlo analysis.

This function is expected to be called before using any scikit-rf features, since it's a global variable and thread-unsafe. To temporarily change the random number generator of a particular method (e.g. `skrf.media.Media.random()`), use the `rng` argument instead.

Parameters
rng (numpy.random.Generator) – Any random number generator accepted by numpy.

Return type
None

Examples

```
>>> set_rand_rng(np.random.default_rng(seed=42))
```

skrf.mathFunctions.rand_rng

skrf.mathFunctions.**rand_rng**()

Obtain the global numpy random number generator instance. By default, it is numpy.random.default_rng().

Return type
Generator

skrf.mathFunctions.rand_c

skrf.mathFunctions.**rand_c**(**size*, *rng=None*)

Creates a complex random array of shape s.

The bounds on real and imaginary values are (-1,1)

Parameters

- **s** (*list-like*) – shape of array
- **rng** (numpy.random.Generator or None) – Any random number generator accepted by numpy.

Return type
ndarray

Examples

```
>>> x1 = rf.rand_c(2, 2)
>>> x2 = rf.rand_c(2, 2, np.random.default_rng(seed=42))
```

3.16.9 Various Utility Functions

psd2TimeDomain(f, y[, windowType])	Convert a one sided complex spectrum into a real time-signal.
rational_interp(x, y[, d, epsilon, axis, ...])	Interpolates function using rational polynomials of degree *d*.

continues on next page

Table 180 – continued from previous page

ifft(x)	Transforms S-parameters to time-domain bandpass.
irfft(x[, n])	Transforms S-parameters to time-domain, assuming complex conjugates for values corresponding to negative frequencies.
is_square(mat)	Tests whether mat is a square matrix.
is_symmetric(mat[, tol])	Tests mat for symmetry.
is_Hermitian(mat[, tol])	Tests whether mat is Hermitian.
is_positive_definite(mat[, tol])	Tests mat for positive definiteness.
is_positive_semidefinite(mat[, tol])	Tests mat for positive semidefiniteness.
get_Hermitian_transpose(mat)	Returns the conjugate transpose of mat.
sqrt_known_sign(z_squared, z_approx)	Return the square root of a complex number, with sign chosen to match *z_approx*.
find_correct_sign(z1, z2, z_approx)	Create new vector from z1, z2 choosing elements with sign matching z_approx.
find_closest(z1, z2, z_approx)	Return z1 or z2 depending on which is closer to z_approx.
inf_to_num(x)	Convert inf and -inf's to large numbers.
rsolve(A, B)	Solves x @ A = B.
nudge_eig(mat[, cond, min_eig])	Nudge eigenvalues with absolute value smaller than max(cond * max(eigenvalue), min_eig) to that value.

skrf.mathFunctions.psd2TimeDomain

skrf.mathFunctions.**psd2TimeDomain**(*f*, *y*, *windowType='hamming'*)

Convert a one sided complex spectrum into a real time-signal.

Parameters

- **f** (*list or np.ndarray*) – frequency array
- **y** (*list of np.ndarray*) – complex PSD array
- **windowType** (*string*) – windowing function, defaults to 'hamming"

Returns

- **timeVector** (*array_like*) – inverse units of the input variable f,
- **signalVector** (*array_like*)

Note

If spectrum is not baseband then, *timeSignal* is modulated by *exp(t*2*pi*f[0])*. So keep in mind units. Also, due to this, f must be increasing left to right.

skrf.mathFunctions.rational_interp

skrf.mathFunctions.**rational_interp**(*x*, *y*, *d=4*, *epsilon=1e-09*, *axis=0*, *assume_sorted=False*)

Interpolates function using rational polynomials of degree *d*.

Interpolating function is singular when xi is exactly one of the original x points. If xi is closer than epsilon to one of the original points, then the value at that points is returned instead.

Implementation is based on[1].

[1]

Parameters

- **x** (*np.ndarray*)
- **y** (*np.ndarray*)
- **d** (*int, optional*) – order of the polynomial, by default 4
- **epsilon** (*float, optional*) – numerical tolerance, by default 1e-9
- **axis** (*int, optional*) – axis to operate on, by default 0
- **assume_sorted** (*bool, optional*) – If False, values of x can be in any order and they are sorted first. If True, x has to be an array of monotonically increasing values.

Returns
fx – Interpolate function

Return type
Callable

Raises
NotImplementedError – if axis != 0.

References

approximation," Numer. Math., vol. 107, no. 2, pp. 315-331, Aug. 2007

skrf.mathFunctions.ifft

skrf.mathFunctions.**ifft**(*x*)

Transforms S-parameters to time-domain bandpass.

Parameters
x (*array_like*) – S-parameters vs frequency array.

Returns
X – Fourier transformed array

Return type
array_like

See also

irfft

skrf.mathFunctions.irfft

skrf.mathFunctions.**irfft**(*x*, *n=None*)

Transforms S-parameters to time-domain, assuming complex conjugates for values corresponding to negative frequencies.

Parameters

- **x** (*array_like*) – S-parameters vs frequency array.
- **n** (*int, optional*) – n parameter passed to numpy.fft.irfft(). Defaults to None.

M. S. Floater and K. Hormann, "Barycentric rational interpolation with no poles and high rates of

Returns
X – Fourier transformed array

Return type
array_like

See also

ifft

skrf.mathFunctions.is_square

skrf.mathFunctions.**is_square**(*mat*)

Tests whether mat is a square matrix.

Parameters
mat (*np.ndarray*) – Matrix to test for being square

Returns
res

Return type
boolean

See also

is_unitary, *is_symmetric*

skrf.mathFunctions.is_symmetric

skrf.mathFunctions.**is_symmetric**(*mat*, *tol=1e-12*)

Tests mat for symmetry.

Parameters

- **mat** (*np.ndarray*) – Matrix to test for symmetry
- **tol** (*float, optional*) – Absolute tolerance. Defaults to ALMOST_ZERO

Returns
res

Return type
boolean or array of boolean

See also

is_square, is_unitary

skrf.mathFunctions.is_Hermitian

skrf.mathFunctions.**is_Hermitian**(*mat*, *tol=1e-12*)

Tests whether mat is Hermitian.

Parameters

- **mat** (*np.ndarray*) – Matrix to test for being Hermitian

- **tol** (*float*) – Absolute tolerance

Returns
res

Return type
boolean

See also

is_positive_definite, *is_positive_semidefinite*

skrf.mathFunctions.is_positive_definite

skrf.mathFunctions.**is_positive_definite**(*mat*, *tol=1e-12*)

Tests mat for positive definiteness.

Verifying that (1) mat is symmetric (2) it's possible to compute the Cholesky decomposition of mat.

Parameters

- **mat** (*np.ndarray*) – Matrix to test for positive definiteness
- **tol** (*float, optional*) – Absolute tolerance. Defaults to ALMOST_ZERO

Returns
res

Return type
bool or array of bool

See also

is_Hermitian, *is_positive_semidefinite*

skrf.mathFunctions.is_positive_semidefinite

skrf.mathFunctions.**is_positive_semidefinite**(*mat*, *tol=1e-12*)

Tests mat for positive semidefiniteness.

Checking whether all eigenvalues of mat are nonnegative within a certain tolerance

Parameters

- **mat** (*np.ndarray*) – Matrix to test for positive semidefiniteness
- **tol** (*float, optional*) – Absolute tolerance in determining nonnegativity due to loss of precision when computing the eigenvalues of mat. Defaults to ALMOST_ZERO

Returns
res

Return type
bool or array of bool

See also

is_Hermitian, *is_positive_definite*

skrf.mathFunctions.get_Hermitian_transpose

skrf.mathFunctions.**get_Hermitian_transpose**(*mat*)

Returns the conjugate transpose of mat.

Parameters
mat (*np.ndarray*) – Matrix to compute the conjugate transpose of

Returns
mat

Return type
np.ndarray

skrf.mathFunctions.sqrt_known_sign

skrf.mathFunctions.**sqrt_known_sign**(*z_squared*, *z_approx*)

Return the square root of a complex number, with sign chosen to match *z_approx*.

Parameters

- **z_squared** (*number or array-like*) – the complex to be square-rooted
- **z_approx** (*number or array-like*) – the approximate value of z. sign of z is chosen to match that of z_approx

Returns
z – square root of z_squared.

Return type
number, array-like (same type as z_squared)

skrf.mathFunctions.find_correct_sign

skrf.mathFunctions.**find_correct_sign**(*z1*, *z2*, *z_approx*)

Create new vector from z1, z2 choosing elements with sign matching z_approx.

This is used when you have to make a root choice on a complex number. and you know the approximate value of the root.

$$z1, z2 = \pm\sqrt{(z^2)}$$

Parameters

- **z1** (*array-like*) – root 1
- **z2** (*array-like*) – root 2
- **z_approx** (*array-like*) – approximate answer of z

Returns
z3 – array built from z1 and z2 by z1 where sign(z1) == sign(z_approx), z2 else

Return type
np.array

skrf.mathFunctions.find_closest

skrf.mathFunctions.**find_closest**(*z1*, *z2*, *z_approx*)

Return z1 or z2 depending on which is closer to z_approx.

Parameters

- **z1** (*array-like*) – root 1
- **z2** (*array-like*) – root 2
- **z_approx** (*array-like*) – approximate answer of z

Returns
z3 – array built from z1 and z2

Return type
np.array

skrf.mathFunctions.inf_to_num

skrf.mathFunctions.**inf_to_num**(*x*)

Convert inf and -inf's to large numbers.

Parameters
x (*array-like or number*) – the input array or number

Returns
x – Input without with +/- inf replaced by large numbers

Return type
Number of array_like

skrf.mathFunctions.rsolve

skrf.mathFunctions.**rsolve**(*A*, *B*)

Solves x @ A = B.

Calls numpy.linalg.solve with transposed matrices.

Same as B @ np.linalg.inv(A) but avoids calculating the inverse and should be numerically slightly more accurate.

Input should have dimension of similar to (nfreqs, nports, nports).

Parameters

- **A** (*np.ndarray*)
- **B** (*np.ndarray*)

Returns
x

Return type
np.ndarray

skrf.mathFunctions.nudge_eig

skrf.mathFunctions.**nudge_eig**(*mat*, *cond=None*, *min_eig=None*)

Nudge eigenvalues with absolute value smaller than max(cond * max(eigenvalue), min_eig) to that value. Can be used to avoid singularities in solving matrix equations.

Input should have dimension of similar to (nfreqs, nports, nports).

Parameters

- **mat** (*np.ndarray*) – Matrices to nudge
- **cond** (*float, optional*) – Minimum eigenvalue ratio compared to the maximum eigenvalue. Default value is set by *skrf.constants.EIG_COND*.

- **min_eig** (*float, optional*) – Minimum eigenvalue. Default value is set by *skrf.constants.EIG_MIN*.

Returns
res – Nudged matrices

Return type
np.ndarray

3.17 tlineFunctions (*skrf.tlineFunctions*)

This module provides functions related to transmission line theory.

3.17.1 Impedance and Reflection Coefficient

These functions relate basic transmission line quantities such as characteristic impedance, input impedance, reflection coefficient, etc. Each function has two names. One is a long-winded but readable name and the other is a short-hand variable-like names. Below is a table relating these two names with each other as well as common mathematical symbols.

Symbol	Variable Name	Long Name
Z_l	z_l	load_impedance
Z_{in}	z_in	input_impedance
Γ_0	Gamma_0	reflection_coefficient
Γ_{in}	Gamma_in	reflection_coefficient_at_theta
θ	theta	electrical_length

There may be a bit of confusion about the difference between the load impedance the input impedance. This is because the load impedance **is** the input impedance at the load. An illustration may provide some useful reference.

Below is a (bad) illustration of a section of uniform transmission line of characteristic impedance Z_0, and electrical length θ. The line is terminated on the right with some load impedance, Z_l. The input impedance Z_{in} and input reflection coefficient Γ_{in} are looking in towards the load from the distance θ from the load.

$$
\begin{array}{rr}
 & Z_0, \theta \\
\text{o==========o=[}Z_l\text{]} & \\
\rightarrow \qquad\qquad\qquad \rightarrow & \\
Z_{in}, \Gamma_{in} \qquad\qquad\qquad & Z_l, \Gamma_0
\end{array}
$$

So, to clarify the confusion,

$$
Z_{in} = Z_l, \qquad \Gamma_{in} = \Gamma_l \text{ at } \theta = 0
$$

Short names

theta(gamma, f, d[, deg])	Electrical length of a section of transmission line.
zl_2_Gamma0(z0, zl)	Reflection coefficient from a load impedance.
zl_2_zin(z0, zl, theta)	Input impedance from load impedance at a given electrical length.
zl_2_Gamma_in(z0, zl, theta)	Reflection coefficient of load at a given electrical length.
zl_2_swr(z0, zl)	Standing Wave Ratio (SWR) for a given load impedance.

continues on next page

Table 181 – continued from previous page

zl_2_total_loss(z0, zl, theta)	Total loss of a terminated transmission line (in natural unit).
Gamma0_2_zl(z0, Gamma)	Input impedance from a load reflection coefficient.
Gamma0_2_Gamma_in(Gamma0, theta)	Reflection coefficient at a given electrical length.
Gamma0_2_zin(z0, Gamma0, theta)	Input impedance from load reflection coefficient at a given electrical length.
Gamma0_2_swr(Gamma0)	Standing Wave Ratio (SWR) for a given reflection coefficient.

skrf.tlineFunctions.theta

skrf.tlineFunctions.**theta**(*gamma*, *f*, *d*, *deg=False*)

Electrical length of a section of transmission line.

$$\theta = \gamma(f) \cdot d$$

Parameters

- **gamma** (*number, array-like or function*) – propagation constant. See Notes. If passed as a function, takes frequency in Hz as a sole argument.
- **f** (*number or array-like*) – frequency at which to calculate
- **d** (*number or array-like*) – length of line, in meters
- **deg** (*Boolean*) – return in degrees or not.

Returns

theta – electrical length in radians or degrees, depending on value of deg.

Return type

number or array-like

See also

electrical_length_2_distance
opposite conversion

Note

The convention has been chosen that forward propagation is represented by the positive imaginary part of the value returned by the gamma function.

skrf.tlineFunctions.zl_2_Gamma0

skrf.tlineFunctions.**zl_2_Gamma0**(*z0*, *zl*)

Reflection coefficient from a load impedance.

Return the reflection coefficient for a given load impedance, and characteristic impedance.

For a transmission line of characteristic impedance Z_0 terminated with load impedance Z_l, the complex reflection coefficient is given by,

$$\Gamma = \frac{Z_l - Z_0}{Z_l + Z_0}$$

Parameters

- **z0** (*number or array-like*) – characteristic impedance
- **zl** (*number or array-like*) – load impedance (aka input impedance)

Returns
gamma – reflection coefficient

Return type
number or array-like

See also

Gamma0_2_zl
reflection coefficient to load impedance

Note

Inputs are typecasted to 1D complex array.

skrf.tlineFunctions.zl_2_zin

skrf.tlineFunctions.**zl_2_zin**(*z0*, *zl*, *theta*)

Input impedance from load impedance at a given electrical length.

Input impedance of load impedance zl at a given electrical length, given characteristic impedance z0.

Parameters

- **z0** (*number or array-like*) – characteristic impedance
- **zl** (*number or array-like*) – load impedance
- **theta** (*number or array-like*) – electrical length of the line (may be complex)

Returns
zin – input impedance at theta

Return type
number or array-like

skrf.tlineFunctions.zl_2_Gamma_in

skrf.tlineFunctions.**zl_2_Gamma_in**(*z0*, *zl*, *theta*)

Reflection coefficient of load at a given electrical length.

Reflection coefficient of load impedance zl at a given electrical length, given characteristic impedance z0.

Parameters

- **z0** (*number or array-like*) – characteristic impedance.
- **zl** (*number or array-like*) – load impedance
- **theta** (*number or array-like*) – electrical length of the line (may be complex).

Returns
Gamma_in – input reflection coefficient at theta

Return type
number or array-like

skrf.tlineFunctions.zl_2_swr

skrf.tlineFunctions.**zl_2_swr**(*z0*, *zl*)

Standing Wave Ratio (SWR) for a given load impedance.

Standing Wave Ratio value is defined by:

$$VSWR = \frac{1+|\Gamma|}{1-|\Gamma|}$$

where

$$\Gamma = \frac{Z_L - Z_0}{Z_L + Z_0}$$

Parameters

- **z0** (*number or array-like*) – line characteristic impedance [Ohm]
- **zl** (*number or array-like*) – load impedance [Ohm]

Returns
swr – Standing Wave Ratio.

Return type
number or array-like

skrf.tlineFunctions.zl_2_total_loss

skrf.tlineFunctions.**zl_2_total_loss**(*z0*, *zl*, *theta*)

Total loss of a terminated transmission line (in natural unit).

The total loss expressed in terms of the load impedance is[1] :

$$TL = \frac{R_{in}}{R_L}\left|\cosh\theta + \frac{Z_L}{Z_0}\sinh\theta\right|^2$$

Parameters

- **z0** (*number or array-like*) – characteristic impedance.
- **zl** (*number or array-like*) – load impedance
- **theta** (*number or array-like*) – electrical length of the line (may be complex).

Returns
total_loss – total loss in natural unit

Return type
number or array-like

References

skrf.tlineFunctions.Gamma0_2_zl

[1] Steve Stearns (K6OIK), Transmission Line Power Paradox and Its Resolution. ARRL PacificonAntenna Seminar, Santa Clara, CA, October 10-12, 2014. https://www.fars.k6ya.org/docs/K6OIK-A_Transmission_Line_Power_Paradox_and_Its_Resolution.pdf

skrf.tlineFunctions.**Gamma0_2_zl**(*z0*, *Gamma*)

Input impedance from a load reflection coefficient.

Calculate the input impedance given a reflection coefficient and characteristic impedance.

$$Z_0 \left(\frac{1+\Gamma}{1-\Gamma} \right)$$

Parameters

- **Gamma** (*number or array-like*) – complex reflection coefficient
- **z0** (*number or array-like*) – characteristic impedance

Returns

zin – input impedance

Return type

number or array-like

skrf.tlineFunctions.Gamma0_2_Gamma_in

skrf.tlineFunctions.**Gamma0_2_Gamma_in**(*Gamma0*, *theta*)

Reflection coefficient at a given electrical length.

$$\Gamma_{in} = \Gamma_0 e^{-2\theta}$$

Parameters

- **Gamma0** (*number or array-like*) – reflection coefficient at theta=0
- **theta** (*number or array-like*) – electrical length (may be complex)

Returns

Gamma_in – input reflection coefficient

Return type

number or array-like

skrf.tlineFunctions.Gamma0_2_zin

skrf.tlineFunctions.**Gamma0_2_zin**(*z0*, *Gamma0*, *theta*)

Input impedance from load reflection coefficient at a given electrical length.

Calculate the input impedance at electrical length theta, given a reflection coefficient and characteristic impedance of the medium.

Parameters

- **z0** (*number or array-like*) – characteristic impedance.
- **Gamma** (*number or array-like*) – reflection coefficient
- **theta** (*number or array-like*) – electrical length of the line, (may be complex)
- **Gamma0** (*Number | Sequence[Number] | ndarray*)

Returns

zin – input impedance at theta

Return type

number or array-like

skrf.tlineFunctions.Gamma0_2_swr

skrf.tlineFunctions.**Gamma0_2_swr**(*Gamma0*)

Standing Wave Ratio (SWR) for a given reflection coefficient.

Standing Wave Ratio value is defined by:

$$VSWR = \frac{1 + |\Gamma_0|}{1 - |\Gamma_0|}$$

Parameters

Gamma0 (*number or array-like*) – Reflection coefficient

Returns

swr – Standing Wave Ratio.

Return type

number or array-like

Long-names

electrical_length(gamma, f, d[, deg])	Electrical length of a section of transmission line.
distance_2_electrical_length(gamma, f, d[, deg])	Electrical length of a section of transmission line.
electrical_length_2_distance(theta, gamma, f0)	Convert electrical length to a physical distance.
reflection_coefficient_at_theta(Gamma0, theta)	Reflection coefficient at a given electrical length.
reflection_coefficient_2_input_impedance(z0, ...)	Input impedance from a load reflection coefficient.
*reflection_coefficient_2_input_impedance_at_* ...)	Input impedance from load reflection coefficient at a given electrical length.
reflection_coefficient_2_propagation_constar	Propagation constant from line input and load reflection coefficients.
input_impedance_at_theta(z0, zl, theta)	Input impedance from load impedance at a given electrical length.
load_impedance_2_reflection_coefficient(z0, zl)	Reflection coefficient from a load impedance.
load_impedance_2_reflection_coefficient_at_t ...)	Reflection coefficient of load at a given electrical length.
voltage_current_propagation(v1, i1, z0, theta)	Voltages and currents calculated on electrical length theta of a transmission line.

skrf.tlineFunctions.electrical_length

skrf.tlineFunctions.**electrical_length**(*gamma*, *f*, *d*, *deg=False*)

Electrical length of a section of transmission line.

$$\theta = \gamma(f) \cdot d$$

Parameters

- **gamma** (*number, array-like or function*) – propagation constant. See Notes. If passed as a function, takes frequency in Hz as a sole argument.
- **f** (*number or array-like*) – frequency at which to calculate

- **d** (*number or array-like*) – length of line, in meters
- **deg** (*Boolean*) – return in degrees or not.

Returns
theta – electrical length in radians or degrees, depending on value of deg.

Return type
number or array-like

See also

electrical_length_2_distance
opposite conversion

Note

The convention has been chosen that forward propagation is represented by the positive imaginary part of the value returned by the gamma function.

skrf.tlineFunctions.distance_2_electrical_length

skrf.tlineFunctions.**distance_2_electrical_length**(*gamma*, *f*, *d*, *deg=False*)
Electrical length of a section of transmission line.

$$\theta = \gamma(f) \cdot d$$

Parameters

- **gamma** (*number, array-like or function*) – propagation constant. See Notes. If passed as a function, takes frequency in Hz as a sole argument.
- **f** (*number or array-like*) – frequency at which to calculate
- **d** (*number or array-like*) – length of line, in meters
- **deg** (*Boolean*) – return in degrees or not.

Returns
theta – electrical length in radians or degrees, depending on value of deg.

Return type
number or array-like

See also

electrical_length_2_distance
opposite conversion

Note

The convention has been chosen that forward propagation is represented by the positive imaginary part of the value returned by the gamma function.

skrf.tlineFunctions.electrical_length_2_distance

skrf.tlineFunctions.**electrical_length_2_distance**(*theta*, *gamma*, *f0*, *deg=True*)

Convert electrical length to a physical distance.

$$d = \frac{\theta}{\gamma(f_0)}$$

Parameters

- **theta** (*number or array-like*) – electrical length. units depend on *deg* option
- **gamma** (*number, array-like or function*) – propagation constant. See Notes. If passed as a function, takes frequency in Hz as a sole argument.
- **f0** (*number or array-like*) – frequency at which to calculate gamma
- **deg** (*Boolean*) – return in degrees or not.

Returns

d – physical distance in m

Return type

number or array-like (real)

Note

The convention has been chosen that forward propagation is represented by the positive imaginary part of the value returned by the gamma function.

See also

distance_2_electrical_length
opposite conversion

skrf.tlineFunctions.reflection_coefficient_at_theta

skrf.tlineFunctions.**reflection_coefficient_at_theta**(*Gamma0*, *theta*)

Reflection coefficient at a given electrical length.

$$\Gamma_{in} = \Gamma_0 e^{-2\theta}$$

Parameters

- **Gamma0** (*number or array-like*) – reflection coefficient at theta=0
- **theta** (*number or array-like*) – electrical length (may be complex)

Returns

Gamma_in – input reflection coefficient

Return type

number or array-like

skrf.tlineFunctions.reflection_coefficient_2_input_impedance

skrf.tlineFunctions.**reflection_coefficient_2_input_impedance**(*z0*, *Gamma*)

Input impedance from a load reflection coefficient.

Calculate the input impedance given a reflection coefficient and characteristic impedance.

$$Z_0 \left(\frac{1+\Gamma}{1-\Gamma} \right)$$

Parameters

- **Gamma** (*number or array-like*) – complex reflection coefficient
- **z0** (*number or array-like*) – characteristic impedance

Returns

zin – input impedance

Return type

number or array-like

skrf.tlineFunctions.reflection_coefficient_2_input_impedance_at_theta

skrf.tlineFunctions.**reflection_coefficient_2_input_impedance_at_theta**(*z0*, *Gamma0*, *theta*)

Input impedance from load reflection coefficient at a given electrical length.

Calculate the input impedance at electrical length theta, given a reflection coefficient and characteristic impedance of the medium.

Parameters

- **z0** (*number or array-like*) – characteristic impedance.
- **Gamma** (*number or array-like*) – reflection coefficient
- **theta** (*number or array-like*) – electrical length of the line, (may be complex)
- **Gamma0** (*Number | Sequence[Number] | ndarray*)

Returns

zin – input impedance at theta

Return type

number or array-like

skrf.tlineFunctions.reflection_coefficient_2_propagation_constant

skrf.tlineFunctions.**reflection_coefficient_2_propagation_constant**(*Gamma_in*, *Gamma_l*, *d*)

Propagation constant from line input and load reflection coefficients.

Calculate the propagation constant of a line of length d, given the reflection coefficient and characteristic impedance of the medium.

$$\Gamma_{in} = \Gamma_l e^{-2j\gamma \cdot d} \rightarrow \gamma = -\frac{1}{2d} \ln \left(\frac{\Gamma_{in}}{\Gamma_l} \right)$$

Parameters

- **Gamma_in** (*number or array-like*) – input reflection coefficient
- **Gamma_l** (*number or array-like*) – load reflection coefficient

- **d** (*number or array-like*) – length of line, in meters

Returns
gamma – propagation constant (see notes)

Return type
number (complex) or array-like

> **Note**
>
> The convention has been chosen that forward propagation is represented by the positive imaginary part of gamma.

skrf.tlineFunctions.input_impedance_at_theta

skrf.tlineFunctions.**input_impedance_at_theta**(*z0*, *zl*, *theta*)

Input impedance from load impedance at a given electrical length.

Input impedance of load impedance zl at a given electrical length, given characteristic impedance z0.

Parameters

- **z0** (*number or array-like*) – characteristic impedance
- **zl** (*number or array-like*) – load impedance
- **theta** (*number or array-like*) – electrical length of the line (may be complex)

Returns
zin – input impedance at theta

Return type
number or array-like

skrf.tlineFunctions.load_impedance_2_reflection_coefficient

skrf.tlineFunctions.**load_impedance_2_reflection_coefficient**(*z0*, *zl*)

Reflection coefficient from a load impedance.

Return the reflection coefficient for a given load impedance, and characteristic impedance.

For a transmission line of characteristic impedance Z_0 terminated with load impedance Z_l, the complex reflection coefficient is given by,

$$\Gamma = \frac{Z_l - Z_0}{Z_l + Z_0}$$

Parameters

- **z0** (*number or array-like*) – characteristic impedance
- **zl** (*number or array-like*) – load impedance (aka input impedance)

Returns
gamma – reflection coefficient

Return type
number or array-like

See also

Gamma0_2_zl
reflection coefficient to load impedance

Note

Inputs are typecasted to 1D complex array.

skrf.tlineFunctions.load_impedance_2_reflection_coefficient_at_theta

skrf.tlineFunctions.**load_impedance_2_reflection_coefficient_at_theta**(*z0*, *zl*, *theta*)

Reflection coefficient of load at a given electrical length.

Reflection coefficient of load impedance zl at a given electrical length, given characteristic impedance z0.

Parameters

- **z0** (*number or array-like*) – characteristic impedance.
- **zl** (*number or array-like*) – load impedance
- **theta** (*number or array-like*) – electrical length of the line (may be complex).

Returns
Gamma_in – input reflection coefficient at theta

Return type
number or array-like

skrf.tlineFunctions.voltage_current_propagation

skrf.tlineFunctions.**voltage_current_propagation**(*v1*, *i1*, *z0*, *theta*)

Voltages and currents calculated on electrical length theta of a transmission line.

Give voltage v2 and current i1 at theta, given voltage v1 and current i1 at theta=0 and given characteristic parameters gamma and z0.

```
i1                          i2
-->----------------------->--

v1         gamma,z0         v2

----------------------------

<------------ d ------------>

theta=0                   theta
```

Uses (inverse) ABCD parameters of a transmission line.

Parameters

- **v1** (*array-like (nfreqs,)*) – total voltage at z=0
- **i1** (*array-like (nfreqs,)*) – total current at z=0, directed toward the transmission line

- **z0** (*array-like (nfreqs,)*) – characteristic impedance
- **theta** (*number or array-like (nfreq, ntheta)*) – electrical length of the line (may be complex).

Returns

- **v2** (*array-like (nfreqs, ntheta)*) – total voltage at z=d
- **i2** (*array-like (nfreqs, ndtheta*) – total current at z=d, directed outward the transmission line

3.17.2 Distributed Circuit and Wave Quantities

distributed_circuit_2_propagation_impedance(	Convert distributed circuit values to wave quantities.
propagation_impedance_2_distributed_circuit(	Convert wave quantities to distributed circuit values.

skrf.tlineFunctions.distributed_circuit_2_propagation_impedance

skrf.tlineFunctions.**distributed_circuit_2_propagation_impedance**(*distributed_admittance*, *distributed_impedance*)

Convert distributed circuit values to wave quantities.

This converts complex distributed impedance and admittance to propagation constant and characteristic impedance. The relation is

$$Z_0 = \sqrt{\frac{Z'}{Y'}} \qquad \gamma = \sqrt{Z'Y'}$$

Parameters

- **distributed_admittance** (*number, array-like*) – distributed admittance
- **distributed_impedance** (*number, array-like*) – distributed impedance

Returns

- **propagation_constant** (*number, array-like*) – distributed impedance
- **characteristic_impedance** (*number, array-like*) – distributed impedance

See also

propagation_impedance_2_distributed_circuit
opposite conversion

skrf.tlineFunctions.propagation_impedance_2_distributed_circuit

skrf.tlineFunctions.**propagation_impedance_2_distributed_circuit**(*propagation_constant*, *characteristic_impedance*)

Convert wave quantities to distributed circuit values.

Convert complex propagation constant and characteristic impedance to distributed impedance and admittance. The relation is,

$$Z^{'} = \gamma Z_0 \qquad Y^{'} = \frac{\gamma}{Z_0}$$

Parameters

- **propagation_constant** (*number, array-like*) – distributed impedance
- **characteristic_impedance** (*number, array-like*) – distributed impedance

Returns

- **distributed_admittance** (*number, array-like*) – distributed admittance
- **distributed_impedance** (*number, array-like*) – distributed impedance

See also

distributed_circuit_2_propagation_impedance
opposite conversion

3.17.3 Transmission Line Physics

skin_depth(f, rho, mu_r)	Skin depth for a material.
surface_resistivity(f, rho, mu_r)	Surface resistivity.
surface_impedance(f, material_properties[, ...])	Surface impedance of an arbitrary conductor boundary that could compromise roughness and stack of materials.

skrf.tlineFunctions.skin_depth

skrf.tlineFunctions.**skin_depth**(*f*, *rho*, *mu_r*)

Skin depth for a material.

The skin depth is calculated as:

$$\delta = \sqrt{\frac{\rho}{\pi f \mu_r \mu_0}}$$

See www.microwaves101.com[1] or wikipedia[2] for more info.

Parameters

- **f** (*number or array-like*) – frequency, in Hz
- **rho** (*number of array-like*) – bulk resistivity of material, in ohm*m
- **mu_r** (*number or array-like*) – relative permeability of material

Returns
skin depth – the skin depth, in meter

Return type
number or array-like

References

[1] https://www.microwaves101.com/encyclopedias/skin-depth
[2] http://en.wikipedia.org/wiki/Skin_effect

See also

surface_resistivity, *surface_impedance*

skrf.tlineFunctions.surface_resistivity

skrf.tlineFunctions.**surface_resistivity**(*f*, *rho*, *mu_r*)

Surface resistivity.

The surface resistivity is calculated as:

$$\frac{\rho}{\delta}$$

where δ is the skin depth from *skin_depth()*.

See www.microwaves101.com[1] or wikipedia[2] for more info.

Parameters

- **f** (*number or array-like*) – frequency, in Hz
- **rho** (*number or array-like*) – bulk resistivity of material, in ohm*m
- **mu_r** (*number or array-like*) – relative permeability of material

Returns

surface resistivity – Surface resistivity in ohms/square

Return type

number of array-like

References

See also

skin_depth, *surface_impedance*

skrf.tlineFunctions.surface_impedance

skrf.tlineFunctions.**surface_impedance**(*f*, *material_properties*, *rms_roughness=0*, *boundary_loc=0*, *distribution='norm'*, *rtol=1e-06*)

Surface impedance of an arbitrary conductor boundary that could compromise roughness and stack of materials.

The rough boundary between two materials, and, more generally, each boundary of a stack of coated conductors, is described by the cumulative distribution function (CDF) of its surface height, which grades the material properties across the transition[1][2]:

$$\mu(x) = \mu_1 + \sum_k (\mu_{k+1} - \mu_k)\,\mathrm{CDF}_k(x)$$

and likewise for $\epsilon(x)$, with x the depth into the stack. The graded profile is discretized into short transmission line segments with propagation constant $\gamma_i = j\omega\sqrt{\mu_i \epsilon_i}$ and intrinsic impedance $\eta_i = \sqrt{\mu_i/\epsilon_i}$, and the input

[1] https://www.microwaves101.com/encyclopedias/sheet-resistance

[2] https://en.wikipedia.org/wiki/Sheet_resistance

[1] B. Tegowski, T. Jaschke, A. Sieganschin and A. F. Jacob, "A Transmission Line Approach for Rough Conductor Surface Impedance Analysis," IEEE Trans. Microw. Theory Techn., vol. 71, no. 2, pp. 471-479, 2023. https://doi.org/10.1109/TMTT.2022.3206440

[2] G. Gold and K. Helmreich, "Modeling of transmission lines with multiple coated conductors," 46th European Microwave Conference (EuMC), 2016. https://doi.org/10.1109/EuMC.2016.7824423

impedance is transformed from the deepest material up to the surface:

$$Z_i = \eta_i \frac{Z_{i+1} + \eta_i \tanh(\gamma_i \Delta x_i)}{\eta_i + Z_{i+1} \tanh(\gamma_i \Delta x_i)}$$

The result is the surface impedance seen by a wave impinging on the stack.

Parameters

- **f** (*number or array-like*) – frequency, in Hz. Must be non-zero.
- **material_properties** (*list of dict or dict*) – Material stack, ordered from the outside medium the wave impinges from (vacuum, dielectric) down to the deepest conductor. Each dict takes any of the keys `'sigma'` (conductivity in S/m), `'ep_r'` (relative permittivity) and `'mu_r'` (relative permeability), with defaults 0, 1 and 1; values are scalars or arrays matching `f`. Describe a conductor by `'sigma'`, where `'ep_r'`, if given, must be real-valued. On the other hand, a lossy dielectric described by complex `'ep_r'`. Passing a single dict is a shorthand for that material below vacuum, i.e. `[{}, {...}]`. Consecutive materials are separated by one rough boundary each.
- **rms_roughness** (*number or array-like, optional*) – rms roughness (i.e., Rq or Rrms) of each boundary, in meter. One value per boundary; a scalar applies to all. Default is 0.
- **boundary_loc** (*number or array-like, optional*) – Mean location of each boundary, in meter, in increasing order. Default is 0 for a single boundary. For multiple boundaries, the first is at 0 and the last is at the total thickness of the stack, e.g. `[0, 0.05e-6, 4.05e-6]` for a 50 nm gold layer on top of 4 um nickel layer.
- **distribution** (*str, frozen scipy.stats distribution, callable, or list thereof, optional*) – Height distribution of each boundary: the name of a scipy.stats continuous distribution without shape parameters (`'norm'` (default), `'rayleigh'`, `'uniform'`, …), a frozen distribution for those with shape parameters (e.g. `scipy.stats.gamma(2)`), or a function `distribution(x, rms_roughness, boundary_loc)` returning the CDF at depths `x`. Named and frozen distributions are shifted and scaled to the given mean and standard deviation.
- **rtol** (*float, optional*) – Relative tolerance to which the discretization is refined. Default is 1e-6.

Returns

Zs – surface impedance, in ohm (per square), at each frequency.

Return type

complex number or array-like

Notes

The material variation inside each segment is handled by a 4th-order Magnus scheme (Eqs. (252)-(254) in[3]), and the segment count over the rough transitions is doubled, combining the last refinement levels by Richardson extrapolation, until `Zs` changes by less than `rtol`; a `RuntimeWarning` is issued if the refinement does not converge.

A layer thinner than the roughness of its boundaries lets their CDFs cross, which mixes the materials with negative weights and can flip the sign of the loss. The CDFs are therefore put back in order before they are mixed, which leaves a well formed stack as it is, and a `RuntimeWarning` is issued if negative weight is found.

With the default `rms_roughness=0` a single smooth conductor recovers the classic $(1+j)\sqrt{\pi f \mu \rho}$ known from *surface_resistivity()*.

[3] S. Blanes, F. Casas, J. A. Oteo and J. Ros, "The Magnus expansion and some of its applications," Physics Reports, vol. 470, no. 5-6, pp. 151-238, 2009. https://doi.org/10.1016/j.physrep.2008.11.001

See also

skin_depth, *surface_resistivity*

References

Examples

Smooth copper at 10 GHz, reproducing $(1+j)R_s$:

```
>>> import skrf as rf
>>> Zs = rf.tlineFunctions.surface_impedance(10e9, {'sigma': 58e6})
>>> Rs = rf.tlineFunctions.surface_resistivity(10e9, rho=1/58e6, mu_r=1)
>>> print(f"{Zs:.4f} ohm, {(1 + 1j)*Rs:.4f} ohm")
0.0261+0.0261j ohm, 0.0261+0.0261j ohm
```

The same conductor with 1 um rms roughness:

```
>>> Zs = rf.tlineFunctions.surface_impedance(10e9, {'sigma': 58e6}, rms_
↪roughness=1e-6)
>>> print(f"{Zs:.4f} ohm")
0.0630+0.3356j ohm
```

The top side of a microstrip trace with an ENIG surface finish; a nearly flat (5 nm rms) stack of 0.05 um gold and 4 um nickel on copper:

```
>>> mats = [{}, {'sigma': 41.1e6}, {'sigma': 14.5e6, 'mu_r': 20}, {'sigma': 58e6}]
>>> Zs = rf.tlineFunctions.surface_impedance(10e9, mats, rms_roughness=5e-9,
...                                          boundary_loc=[0, 0.05e-6, 4.05e-6])
>>> print(f"{Zs:.4f} ohm")
0.1895+0.1012j ohm
```

The bottom side of the trace; copper against the substrate ($\epsilon_r = 3.2$), rough at the copper-substrate boundary, from 1 to 100 GHz:

```
>>> import numpy as np
>>> f = np.logspace(9, 11, 201)
>>> Zs = rf.tlineFunctions.surface_impedance(f, [{'ep_r': 3.2}, {'sigma': 58e6}],
...                                          rms_roughness=0.5e-6)
>>> print(f"{Zs[-1]:.4f} ohm")  # at 100 GHz
0.2743+1.5154j ohm
```

3.18 instances (*skrf.instances*)

This module contains commonly used instantiated objects's.

3.18.1 Standard Waveguide Bands

Frequency Objects

These are predefined *Frequency* objects that correspond to standard waveguide bands. This information is taken from the VDI Application Note 1002[1] . IEEE designators are taken from Spinner TD-00036[2] .

Object Name	Description
f_wr10	WR-10, 75-110 GHz
f_wr3	WR-3, 220-325 GHz
f_wr2p2	WR-2.2, 330-500 GHz
f_wr1p5	WR-1.5, 500-750 GHz
f_wr1	WR-1, 750-1100 GHz
…	…

RectangularWaveguide Objects

These are predefined *RectangularWaveguide* objects for standard waveguide bands.

Object Name	Description
wr10	WR-10, 75-110 GHz
wr3	WR-3, 220-325 GHz
wr2p2	WR-2.2, 330-500 GHz
wr1p5	WR-1.5, 500-750 GHz
wr1	WR-1, 750-1100 GHz
…	…

References

3.19 constants (*skrf.constants*)

This module contains constants, numerical approximations, and unit conversions

`skrf.constants.c`

Velocity of light constant (from scipy)

`skrf.constants.INF`

A very very large value (1e99)

`skrf.constants.ONE`

1 + epsilon where epsilon is small. Used to avoid numerical error.

`skrf.constants.ZERO`

0 + epsilon where epsilon is small. Used to avoid numerical error.

`skrf.constants.LOG_OF_NEG`

Very low but minus infinity value for numerical purposes.

`skrf.constants.K_BOLTZMANN`

Boltzmann constant (1.38064852e-23)

[1] VDI Application Note: VDI Waveguide Band Designations (VDI-1002) http://vadiodes.com/VDI/pdf/waveguidechart200908.pdf

[2] Spinner Technical Information: Cross Reference For Hollow Metallic Waveguides (TD-00036) https://www.spinner-group.com/images/download/technical_documents/SPINNER_TD00036.pdf

skrf.constants.**S_DEFINITIONS**

S-parameter definition labels:

- 'power' for power-waves definition,
- 'pseudo' for pseudo-waves definition.
- 'traveling' corresponds to the initial implementation.

skrf.constants.**S_DEF_DEFAULT**

Default S-parameter definition: 'power', for power-wave definition.

skrf.constants.**S_DEF_HFSS_DEFAULT = 'traveling'**

str(object='') -> str str(bytes_or_buffer[, encoding[, errors]]) -> str

Create a new string object from the given object. If encoding or errors is specified, then the object must expose a data buffer that will be decoded using the given encoding and error handler. Otherwise, returns the result of object.__str__() (if defined) or repr(object). encoding defaults to sys.getdefaultencoding(). errors defaults to 'strict'.

Default ANSYS HFSS S-parameter definition: 'traveling'

skrf.constants.**SweepTypeT**

Frequency sweep type, either "lin" or "log".

alias of `Literal`['lin', 'log']

skrf.constants.**FrequencyUnitT**

"Hz", "kHz", "MHz", "GHz", "THz" (case-insensitive).

Type
Frequency units

Frequency units: "Hz", "kHz", "MHz", "GHz", "THz" (case-insensitive). alias of `Literal`['Hz', 'kHz', 'MHz', 'GHz', 'THz']

skrf.constants.**FREQ_UNIT_DEFAULT**

Unit for newly constructed frequency objects, regardless of input units. Also interprets omitted Frequency input units. Set to None to preserve explicit/file units and interpret omitted input units as Hz.

to_meters(d[, unit, v_g])	Translate various units of distance into meters.

3.19.1 skrf.constants.to_meters

skrf.constants.**to_meters**(*d*, *unit='m'*, *v_g=None*)

Translate various units of distance into meters.

Parameters

- **d** (*number or array-like*) – value(s) to convert
- **unit** (*str*) – the unit to that x is in: ['m','cm','um','in','mil','s','us','ns','ps']
- **v_g** (*float*) – group velocity in m/s

Returns
d_m – distance in meters

Return type
number of array-like

3.20 data (*skrf.data*)

This Package provides data to be used in examples and testcases.

3.20.1 Material Properties

materials	Dictionary of materials properties

skrf.data.materials

skrf.data.**materials = {'al': {'resistivity(ohm*m)': 2.82e-08}, 'aluminum': {'resistivity(ohm*m)': 2.82e-08}, 'au': {'resistivity(ohm*m)': 2.44e-08}, 'copper': {'resistivity(ohm*m)': 1.68e-08}, 'cu': {'resistivity(ohm*m)': 1.68e-08}, 'duroid 5880': {'loss tangent': 0.004, 'relative permativity': 2.25}, 'gold': {'resistivity(ohm*m)': 2.44e-08}, 'lead': {'resistivity(ohm*m)': 2.192982456140351e-07}, 'mylar': {'loss tangent': 0.05, 'relative permativity': 3.1}, 'quartz': {'loss tangent': 0.00015, 'relative permativity': 3.8}, 'silicon': {'loss tangent': 0.0008, 'relative permativity': 11.68}, 'steel(stainless)': {'resistivity(ohm*m)': 9.090909090909e-07}, 'teflon': {'loss tangent': 0.0005, 'relative permativity': 2.1}}**

Dictionary of materials properties

Type
materials

3.20.2 Example/Test Networks

ring_slot	An n-port electrical network.
ring_slot_meas	An n-port electrical network.
line	An n-port electrical network.
tee	An n-port electrical network.
ind	An n-port electrical network.
open_2p	An n-port electrical network.
short_2p	An n-port electrical network.

skrf.data.ring_slot

skrf.data.**ring_slot = 2-Port Network: 'ring slot', 75.0-110.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]**

An n-port electrical network.

For instructions on how to create Network see `__init__()`. An n-port network[1] may be defined by three quantities

- network parameter matrix (s, z, or y-matrix)
- port characteristic impedance matrix
- frequency information

The `Network` class stores these data structures internally in the form of complex `numpy.ndarray`'s. These arrays are not interfaced directly but instead through the use of the properties:

[1] http://en.wikipedia.org/wiki/Two-port_network

Property	Meaning
s	Scattering parameter matrix.
z0	Characteristic impedance matrix.
f	Frequency vector.

Although these docs focus on s-parameters, other equivalent network representations such as z and y are available. Scalar projections of the complex network parameters are accessible through properties as well. These also return numpy.ndarray's.

Property	Meaning
s_re	Real part of the s-matrix.
s_im	Imaginary part of the s-matrix.
s_mag	Magnitude of the s-matrix.
s_db	Magnitude in log scale of the s-matrix.
s_deg	Phase of the s-matrix in degrees.

The following operations act on the networks s-matrix.

Operator	Function
+	Element-wise addition of the s-matrix.
-	Element-wise difference of the s-matrix.
*	Element-wise multiplication of the s-matrix.
/	Element-wise division of the s-matrix.
**	Cascading (only for 2-ports).
//	De-embedding (for 2-ports, see inv).

Different components of the Network can be visualized through various plotting methods. These methods can be used to plot individual elements of the s-matrix or all at once. For more info about plotting see the ../../tutorials/Plotting tutorial.

Method	Meaning
plot_s_smith()	Plot complex s-parameters on smith chart.
plot_s_re()	Plot real part of s-parameters vs frequency.
plot_s_im()	Plot imaginary part of s-parameters vs frequency.
plot_s_mag()	Plot magnitude of s-parameters vs frequency.
plot_s_db()	Plot magnitude (in dB) of s-parameters vs frequency.
plot_s_deg()	Plot phase of s-parameters (in degrees) vs frequency.
plot_s_deg_unwrap()	Plot phase of s-parameters (in unwrapped degrees) vs frequency.

Network objects can be created from a touchstone or pickle file (see __init__()), by a *Media* object, or manually by assigning the network properties directly. Network objects can be saved to disk in the form of touchstone files with the write_touchstone() method.

An exhaustive list of Network Methods and Properties (Attributes) are given below

References

skrf.data.ring_slot_meas

skrf.data.**ring_slot_meas = 1-Port Network: 'ring slot measured', 75.0-109.999999992 GHz, 101 pts, z0=[50.+0.j]**

An n-port electrical network.

For instructions on how to create Network see `__init__()`. An n-port network[1] may be defined by three quantities

- network parameter matrix (s, z, or y-matrix)
- port characteristic impedance matrix
- frequency information

The `Network` class stores these data structures internally in the form of complex `numpy.ndarray`'s. These arrays are not interfaced directly but instead through the use of the properties:

Property	Meaning
`s`	Scattering parameter matrix.
`z0`	Characteristic impedance matrix.
`f`	Frequency vector.

Although these docs focus on s-parameters, other equivalent network representations such as `z` and `y` are available. Scalar projections of the complex network parameters are accessible through properties as well. These also return `numpy.ndarray`'s.

Property	Meaning
`s_re`	Real part of the s-matrix.
`s_im`	Imaginary part of the s-matrix.
`s_mag`	Magnitude of the s-matrix.
`s_db`	Magnitude in log scale of the s-matrix.
`s_deg`	Phase of the s-matrix in degrees.

The following operations act on the networks s-matrix.

Operator	Function
+	Element-wise addition of the s-matrix.
-	Element-wise difference of the s-matrix.
*	Element-wise multiplication of the s-matrix.
/	Element-wise division of the s-matrix.
**	Cascading (only for 2-ports).
//	De-embedding (for 2-ports, see `inv`).

Different components of the `Network` can be visualized through various plotting methods. These methods can be used to plot individual elements of the s-matrix or all at once. For more info about plotting see the ../../tutorials/Plotting tutorial.

[1] http://en.wikipedia.org/wiki/Two-port_network

Method	Meaning
`plot_s_smith()`	Plot complex s-parameters on smith chart.
`plot_s_re()`	Plot real part of s-parameters vs frequency.
`plot_s_im()`	Plot imaginary part of s-parameters vs frequency.
`plot_s_mag()`	Plot magnitude of s-parameters vs frequency.
`plot_s_db()`	Plot magnitude (in dB) of s-parameters vs frequency.
`plot_s_deg()`	Plot phase of s-parameters (in degrees) vs frequency.
`plot_s_deg_unwrap()`	Plot phase of s-parameters (in unwrapped degrees) vs frequency.

`Network` objects can be created from a touchstone or pickle file (see `__init__()`), by a *Media* object, or manually by assigning the network properties directly. `Network` objects can be saved to disk in the form of touchstone files with the `write_touchstone()` method.

An exhaustive list of `Network` Methods and Properties (Attributes) are given below

References

skrf.data.line

skrf.data.**line = 2-Port Network: 'line', 75.0-110.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j]**

An n-port electrical network.

For instructions on how to create Network see `__init__()`. An n-port network[1] may be defined by three quantities

- network parameter matrix (s, z, or y-matrix)
- port characteristic impedance matrix
- frequency information

The `Network` class stores these data structures internally in the form of complex `numpy.ndarray`'s. These arrays are not interfaced directly but instead through the use of the properties:

Property	Meaning
`s`	Scattering parameter matrix.
`z0`	Characteristic impedance matrix.
`f`	Frequency vector.

Although these docs focus on s-parameters, other equivalent network representations such as `z` and `y` are available. Scalar projections of the complex network parameters are accessible through properties as well. These also return `numpy.ndarray`'s.

Property	Meaning
`s_re`	Real part of the s-matrix.
`s_im`	Imaginary part of the s-matrix.
`s_mag`	Magnitude of the s-matrix.
`s_db`	Magnitude in log scale of the s-matrix.
`s_deg`	Phase of the s-matrix in degrees.

The following operations act on the networks s-matrix.

[1] http://en.wikipedia.org/wiki/Two-port_network

Operator	Function
+	Element-wise addition of the s-matrix.
-	Element-wise difference of the s-matrix.
*	Element-wise multiplication of the s-matrix.
/	Element-wise division of the s-matrix.
**	Cascading (only for 2-ports).
//	De-embedding (for 2-ports, see `inv`).

Different components of the `Network` can be visualized through various plotting methods. These methods can be used to plot individual elements of the s-matrix or all at once. For more info about plotting see the ../../tutorials/Plotting tutorial.

Method	Meaning
`plot_s_smith()`	Plot complex s-parameters on smith chart.
`plot_s_re()`	Plot real part of s-parameters vs frequency.
`plot_s_im()`	Plot imaginary part of s-parameters vs frequency.
`plot_s_mag()`	Plot magnitude of s-parameters vs frequency.
`plot_s_db()`	Plot magnitude (in dB) of s-parameters vs frequency.
`plot_s_deg()`	Plot phase of s-parameters (in degrees) vs frequency.
`plot_s_deg_unwrap()`	Plot phase of s-parameters (in unwrapped degrees) vs frequency.

`Network` objects can be created from a touchstone or pickle file (see `__init__()`), by a *Media* object, or manually by assigning the network properties directly. `Network` objects can be saved to disk in the form of touchstone files with the `write_touchstone()` method.

An exhaustive list of `Network` Methods and Properties (Attributes) are given below

References

skrf.data.tee

skrf.data.tee = 3-Port Network: 'tee', 330.0-500.0 GHz, 201 pts, z0=[50.+0.j 50.+0.j 50.+0.j]

An n-port electrical network.

For instructions on how to create Network see `__init__()`. An n-port network[1] may be defined by three quantities

- network parameter matrix (s, z, or y-matrix)
- port characteristic impedance matrix
- frequency information

The `Network` class stores these data structures internally in the form of complex `numpy.ndarray`'s. These arrays are not interfaced directly but instead through the use of the properties:

Property	Meaning
`s`	Scattering parameter matrix.
`z0`	Characteristic impedance matrix.
`f`	Frequency vector.

[1] http://en.wikipedia.org/wiki/Two-port_network

Although these docs focus on s-parameters, other equivalent network representations such as `z` and `y` are available. Scalar projections of the complex network parameters are accessible through properties as well. These also return `numpy.ndarray`'s.

Property	Meaning
`s_re`	Real part of the s-matrix.
`s_im`	Imaginary part of the s-matrix.
`s_mag`	Magnitude of the s-matrix.
`s_db`	Magnitude in log scale of the s-matrix.
`s_deg`	Phase of the s-matrix in degrees.

The following operations act on the networks s-matrix.

Operator	Function
+	Element-wise addition of the s-matrix.
-	Element-wise difference of the s-matrix.
*	Element-wise multiplication of the s-matrix.
/	Element-wise division of the s-matrix.
**	Cascading (only for 2-ports).
//	De-embedding (for 2-ports, see `inv`).

Different components of the `Network` can be visualized through various plotting methods. These methods can be used to plot individual elements of the s-matrix or all at once. For more info about plotting see the ../../tutorials/Plotting tutorial.

Method	Meaning
`plot_s_smith()`	Plot complex s-parameters on smith chart.
`plot_s_re()`	Plot real part of s-parameters vs frequency.
`plot_s_im()`	Plot imaginary part of s-parameters vs frequency.
`plot_s_mag()`	Plot magnitude of s-parameters vs frequency.
`plot_s_db()`	Plot magnitude (in dB) of s-parameters vs frequency.
`plot_s_deg()`	Plot phase of s-parameters (in degrees) vs frequency.
`plot_s_deg_unwrap()`	Plot phase of s-parameters (in unwrapped degrees) vs frequency.

`Network` objects can be created from a touchstone or pickle file (see `__init__()`), by a *`Media`* object, or manually by assigning the network properties directly. `Network` objects can be saved to disk in the form of touchstone files with the `write_touchstone()` method.

An exhaustive list of `Network` Methods and Properties (Attributes) are given below

References

skrf.data.ind

`skrf.data.ind = 2-Port Network: 'ind', 1000000000.0-10000000000.0 Hz, 10 pts, z0=[50.+0.j 50.+0.j]`

An n-port electrical network.

For instructions on how to create Network see `__init__()`. An n-port network[1] may be defined by three quantities

[1] http://en.wikipedia.org/wiki/Two-port_network

- network parameter matrix (s, z, or y-matrix)
- port characteristic impedance matrix
- frequency information

The `Network` class stores these data structures internally in the form of complex `numpy.ndarray`'s. These arrays are not interfaced directly but instead through the use of the properties:

Property	Meaning
`s`	Scattering parameter matrix.
`z0`	Characteristic impedance matrix.
`f`	Frequency vector.

Although these docs focus on s-parameters, other equivalent network representations such as `z` and `y` are available. Scalar projections of the complex network parameters are accessible through properties as well. These also return `numpy.ndarray`'s.

Property	Meaning
`s_re`	Real part of the s-matrix.
`s_im`	Imaginary part of the s-matrix.
`s_mag`	Magnitude of the s-matrix.
`s_db`	Magnitude in log scale of the s-matrix.
`s_deg`	Phase of the s-matrix in degrees.

The following operations act on the networks s-matrix.

Operator	Function
+	Element-wise addition of the s-matrix.
-	Element-wise difference of the s-matrix.
*	Element-wise multiplication of the s-matrix.
/	Element-wise division of the s-matrix.
**	Cascading (only for 2-ports).
//	De-embedding (for 2-ports, see `inv`).

Different components of the `Network` can be visualized through various plotting methods. These methods can be used to plot individual elements of the s-matrix or all at once. For more info about plotting see the ../../tutorials/Plotting tutorial.

Method	Meaning
`plot_s_smith()`	Plot complex s-parameters on smith chart.
`plot_s_re()`	Plot real part of s-parameters vs frequency.
`plot_s_im()`	Plot imaginary part of s-parameters vs frequency.
`plot_s_mag()`	Plot magnitude of s-parameters vs frequency.
`plot_s_db()`	Plot magnitude (in dB) of s-parameters vs frequency.
`plot_s_deg()`	Plot phase of s-parameters (in degrees) vs frequency.
`plot_s_deg_unwrap()`	Plot phase of s-parameters (in unwrapped degrees) vs frequency.

`Network` objects can be created from a touchstone or pickle file (see `__init__()`), by a *Media* object, or manually by assigning the network properties directly. `Network` objects can be saved to disk in the form of touchstone

files with the `write_touchstone()` method.

An exhaustive list of `Network` Methods and Properties (Attributes) are given below

References

skrf.data.open_2p

`skrf.data.open_2p = 2-Port Network: 'open', 1000000000.0-10000000000.0 Hz, 10 pts, z0=[50.+0.j 50.+0.j]`

An n-port electrical network.

For instructions on how to create Network see `__init__()`. An n-port network[1] may be defined by three quantities

- network parameter matrix (s, z, or y-matrix)
- port characteristic impedance matrix
- frequency information

The `Network` class stores these data structures internally in the form of complex `numpy.ndarray`'s. These arrays are not interfaced directly but instead through the use of the properties:

Property	Meaning
`s`	Scattering parameter matrix.
`z0`	Characteristic impedance matrix.
`f`	Frequency vector.

Although these docs focus on s-parameters, other equivalent network representations such as `z` and `y` are available. Scalar projections of the complex network parameters are accessible through properties as well. These also return `numpy.ndarray`'s.

Property	Meaning
`s_re`	Real part of the s-matrix.
`s_im`	Imaginary part of the s-matrix.
`s_mag`	Magnitude of the s-matrix.
`s_db`	Magnitude in log scale of the s-matrix.
`s_deg`	Phase of the s-matrix in degrees.

The following operations act on the networks s-matrix.

Operator	Function
+	Element-wise addition of the s-matrix.
-	Element-wise difference of the s-matrix.
*	Element-wise multiplication of the s-matrix.
/	Element-wise division of the s-matrix.
**	Cascading (only for 2-ports).
//	De-embedding (for 2-ports, see `inv`).

[1] http://en.wikipedia.org/wiki/Two-port_network

Different components of the `Network` can be visualized through various plotting methods. These methods can be used to plot individual elements of the s-matrix or all at once. For more info about plotting see the ../../tutorials/Plotting tutorial.

Method	Meaning
`plot_s_smith()`	Plot complex s-parameters on smith chart.
`plot_s_re()`	Plot real part of s-parameters vs frequency.
`plot_s_im()`	Plot imaginary part of s-parameters vs frequency.
`plot_s_mag()`	Plot magnitude of s-parameters vs frequency.
`plot_s_db()`	Plot magnitude (in dB) of s-parameters vs frequency.
`plot_s_deg()`	Plot phase of s-parameters (in degrees) vs frequency.
`plot_s_deg_unwrap()`	Plot phase of s-parameters (in unwrapped degrees) vs frequency.

`Network` objects can be created from a touchstone or pickle file (see `__init__()`), by a *Media* object, or manually by assigning the network properties directly. `Network` objects can be saved to disk in the form of touchstone files with the `write_touchstone()` method.

An exhaustive list of `Network` Methods and Properties (Attributes) are given below

References

skrf.data.short_2p

skrf.data.short_2p = 2-Port Network: 'short', 1000000000.0-10000000000.0 Hz, 10 pts, z0=[50.+0.j 50.+0.j]

An n-port electrical network.

For instructions on how to create Network see `__init__()`. An n-port network[1] may be defined by three quantities

- network parameter matrix (s, z, or y-matrix)
- port characteristic impedance matrix
- frequency information

The `Network` class stores these data structures internally in the form of complex `numpy.ndarray`'s. These arrays are not interfaced directly but instead through the use of the properties:

Property	Meaning
`s`	Scattering parameter matrix.
`z0`	Characteristic impedance matrix.
`f`	Frequency vector.

Although these docs focus on s-parameters, other equivalent network representations such as `z` and `y` are available. Scalar projections of the complex network parameters are accessible through properties as well. These also return `numpy.ndarray`'s.

[1] http://en.wikipedia.org/wiki/Two-port_network

Property	Meaning
`s_re`	Real part of the s-matrix.
`s_im`	Imaginary part of the s-matrix.
`s_mag`	Magnitude of the s-matrix.
`s_db`	Magnitude in log scale of the s-matrix.
`s_deg`	Phase of the s-matrix in degrees.

The following operations act on the networks s-matrix.

Operator	Function
+	Element-wise addition of the s-matrix.
-	Element-wise difference of the s-matrix.
*	Element-wise multiplication of the s-matrix.
/	Element-wise division of the s-matrix.
**	Cascading (only for 2-ports).
//	De-embedding (for 2-ports, see `inv`).

Different components of the `Network` can be visualized through various plotting methods. These methods can be used to plot individual elements of the s-matrix or all at once. For more info about plotting see the ../../tutorials/Plotting tutorial.

Method	Meaning
`plot_s_smith()`	Plot complex s-parameters on smith chart.
`plot_s_re()`	Plot real part of s-parameters vs frequency.
`plot_s_im()`	Plot imaginary part of s-parameters vs frequency.
`plot_s_mag()`	Plot magnitude of s-parameters vs frequency.
`plot_s_db()`	Plot magnitude (in dB) of s-parameters vs frequency.
`plot_s_deg()`	Plot phase of s-parameters (in degrees) vs frequency.
`plot_s_deg_unwrap()`	Plot phase of s-parameters (in unwrapped degrees) vs frequency.

`Network` objects can be created from a touchstone or pickle file (see `__init__()`), by a *Media* object, or manually by assigning the network properties directly. `Network` objects can be saved to disk in the form of touchstone files with the `write_touchstone()` method.

An exhaustive list of `Network` Methods and Properties (Attributes) are given below

References

3.21 util (*skrf.util*)

Holds utilities that are general conveniences.

3.21.1 Time-related utilities

now_string()	Return a unique sortable string, representing the current time.
now_string_2_dt(s)	Converts the output of *now_string()* to a datetime object.

continues on next page

Table 188 – continued from previous page

ProgressBar(iterations[, label])	A progress bar based off of the notebook/ipython progress bar from PyMC.

skrf.util.now_string

skrf.util.**now_string**()

Return a unique sortable string, representing the current time.

Nice for generating date-time stamps to be used in file-names, the companion function *now_string_2_dt()* can be used to read these string back into datetime objects.

Returns
now – current date-time stamps.

Return type
string

See also

now_string_2_dt

skrf.util.now_string_2_dt

skrf.util.**now_string_2_dt**(*s*)

Converts the output of *now_string()* to a datetime object.

Parameters
s (*str*) – date-time stamps string as generated by *now_string()*

Returns
dt – date-time stamps

Return type
datetime

See also

now_string

skrf.util.ProgressBar

class skrf.util.**ProgressBar**(*iterations*, *label='iterations'*)

A progress bar based off of the notebook/ipython progress bar from PyMC.

Useful when waiting for long operations such as taking a large number of VNA measurements that may take a few minutes.

Examples

```
>>> from time import sleep
>>> pb = rf.ProgressBar(10)
>>> for idx in range(10):
>>>     sleep(1)
>>>     pb.animate(idx)
```

Parameters

- **iterations** (*int*)
- **label** (*str*)

Methods

__init__	Progress bar constructor.
animate	Animate the progress bar.
update_iteration	

skrf.util.ProgressBar.__init__

ProgressBar.**__init__**(*iterations*, *label='iterations'*)

Progress bar constructor.

Parameters

- **iterations** (*int*) – Number of expected iterations
- **label** (*str, optional*) – Progress bar label, by default "iterations"

skrf.util.ProgressBar.animate

ProgressBar.**animate**(*iteration*)

Animate the progress bar.

Parameters

iteration (*int*) – current iteration

skrf.util.ProgressBar.update_iteration

ProgressBar.**update_iteration**(*elapsed_iter*)

Parameters

elapsed_iter (*int*)

3.21.2 Array-related functions

find_nearest(array, value)	Find the nearest value in array.
find_nearest_index(array, value)	Find the nearest index for a value in array.
has_duplicate_value(value, values, index)	Check if there is another value of the current index in the list.
smooth(x[, window_len, window])	Smooth the data using a window with requested size.

skrf.util.find_nearest

skrf.util.**find_nearest**(*array*, *value*)

Find the nearest value in array.

Parameters

- **array** (*np.ndarray*) – array we are searching for a value in
- **value** (*element of the array*) – value to search for

Returns
found_value – the value that is numerically closest to *value*

Return type
an element of the array

skrf.util.find_nearest_index

skrf.util.**find_nearest_index**(*array*, *value*)

Find the nearest index for a value in array.

Parameters

- **array** (*np.ndarray*) – array we are searching for a value in
- **value** (*element of the array*) – value to search for

Returns
found_index – the index at which the numerically closest element to *value* was found at

Return type
int

References

taken from http://stackoverflow.com/questions/2566412/find-nearest-value-in-numpy-array

skrf.util.has_duplicate_value

skrf.util.**has_duplicate_value**(*value*, *values*, *index*)

Check if there is another value of the current index in the list.

Parameters

- **value** (*Any*) – any value in a list
- **values** (*Iterable*) – the iterable containing the values
- **index** (*int*) – the index of the current item we are checking for.

Returns
index – returns None if no duplicate found, or the index of the first found duplicate

Return type
bool or int

Examples

```
>>> rf.has_duplicate_value(0, [1, 2, 0, 3, 0], -1)  # -> 2
>>> rf.has_duplicate_value(0, [1, 2, 0, 3, 0], 2)  # -> 4
>>> rf.has_duplicate_value(3, [1, 2, 0, 3, 0], 0)  # -> 3
>>> rf.has_duplicate_value(3, [1, 2, 0, 3, 0], 3)  # -> False
```

skrf.util.smooth

skrf.util.**smooth**(*x*, *window_len=11*, *window='flat'*)

Smooth the data using a window with requested size.

Based on the function from the scipy cookbook[1]

[1] http://scipy-cookbook.readthedocs.io/items/SignalSmooth.html

This method is based on the convolution of a scaled window with the signal. The signal is prepared by introducing reflected copies of the signal (with the window size) in both ends so that transient parts are minimized in the beginning and end part of the output signal.

Parameters

- **x** (*numpy.array*) – the input signal
- **window_len** (*int, optional*) – the dimension of the smoothing window; should be an odd integer. Default is 11.
- **window** (*str, optional*) – the type of window from 'flat', 'hanning', 'hamming', 'bartlett', 'blackman' flat window will produce a moving average smoothing. Default is 'flat'

Returns

y – The smoothed signal

Return type

numpy.array

Examples

```
>>> t = linspace(-2, 2, 0.1)
>>> x = sin(t) + randn(len(t))*0.1
>>> y = smooth(x)
```

See also

numpy.hanning, numpy.hamming, numpy.bartlett, numpy.blackman, numpy.convolve, scipy.signal.lfilter

Note

length(output) != length(input). To correct this: *return y[(window_len/2-1):-(window_len/2)]* instead of just *y*.

References

3.21.3 File-related functions

get_fid(file, *args, **kwargs)	Return a file object, given a filename or file object.
get_extn(filename)	Get the extension from a filename.
basename_noext(filename)	Get the basename and strips extension.
git_version(modname)	Return output 'git describe', executed in a module's root directory.
unique_name(name, names[, exclude])	Pass in a name and a list of names, and increment with _## as necessary to ensure a unique name.
findReplace(directory, find, replace, ...)	Find/replace some txt in all files in a directory, recursively.
dict_2_recarray(d, delim, dtype)	Turn a dictionary of structured keys to a record array of objects.

skrf.util.get_fid

skrf.util.**get_fid**(*file*, **args*, ***kwargs*)

Return a file object, given a filename or file object.

Useful when you want to allow the arguments of a function to be either files or filenames

Parameters

- **file** (*str/unicode, Path, or file-object*) – file to open
- ***args** (arguments and keyword arguments to *open()*)
- ****kwargs** (arguments and keyword arguments to *open()*)

Returns
fid

Return type
file object

skrf.util.get_extn

skrf.util.**get_extn**(*filename*)

Get the extension from a filename.

The extension is defined as everything passed the last '.'. Returns None if it ain't got one

Parameters
filename (*string or Path*) – the filename

Returns
ext – either the extension (not including '.') or None if there isn't one

Return type
string, None

skrf.util.basename_noext

skrf.util.**basename_noext**(*filename*)

Get the basename and strips extension.

Parameters
filename (*string*) – the filename

Returns
basename – file basename (ie. without extension)

Return type
str

skrf.util.git_version

skrf.util.**git_version**(*modname*)

Return output 'git describe', executed in a module's root directory.

Parameters
modname (*str*) – module name

Returns
out – output of 'git describe'

Return type
str

skrf.util.unique_name

skrf.util.**unique_name**(*name*, *names*, *exclude=-1*)

Pass in a name and a list of names, and increment with _## as necessary to ensure a unique name.

Parameters

- **name** (*str*) – the chosen name, to be modified if necessary
- **names** (*list*) – list of names (str)
- **exclude** (*int, optional*) – the index of an item to be excluded from the search. Default is -1.

Returns
unique_name

Return type
str

skrf.util.findReplace

skrf.util.**findReplace**(*directory*, *find*, *replace*, *file_pattern*)

Find/replace some txt in all files in a directory, recursively.

This was found in[1] .

Parameters

- **directory** (*str*) – path of a directory
- **find** (*str*) – pattern to search for
- **replace** (*str*) – string to replace with
- **file_pattern** (*str*) – file pattern for filtering. Ex: '*.txt'.

Examples

```
>>> rf.findReplace('some_dir', 'find this', 'replace with this', '*.txt')
```

References

skrf.util.dict_2_recarray

skrf.util.**dict_2_recarray**(*d*, *delim*, *dtype*)

Turn a dictionary of structured keys to a record array of objects.

This is useful if you save data-base like meta-data in the form or file-naming conventions, aka 'the poor-mans database'

Parameters

- **d** (*dict*) – dictionary of structured keys
- **delim** (*str*) – delimiter string
- **dtype** (*list of tuple*) – list of type, where a type is tuple like ('type_name', type)

[1] http://stackoverflow.com/questions/4205854/python-way-to-recursively-find-and-replace-string-in-text-files

Returns
ra

Return type
numpy.array

Examples

Given a directory of networks like:

```
>>> ls
a1,0.0,0.0.s1p    a1,3.0,3.0.s1p    a2,3.0,-3.0.s1p   b1,-3.0,3.0.s1p
...
```

you can sort based on the values or each field, after defining their type with *dtype*. The *values* field accesses the objects.

```
>>> d = rf.read_all_networks('/tmp/')
>>> delim = ','
>>> dtype = [('name', object), ('voltage', float), ('current', float)]
>>> ra = dict_2_recarray(d=rf.ran(dir), delim=delim, dtype =dtype)
```

then you can sift like you do with numpy arrays

```
>>> ra[ra['voltage'] < 3]['values']
array([1-Port Network: 'a2,0.0,-3.0',  450-800 GHz, 101 pts, z0=[ 50.+0.j],
       1-Port Network: 'b1,0.0,3.0',  450-800 GHz, 101 pts, z0=[ 50.+0.j],
       1-Port Network: 'a1,0.0,-3.0',  450-800 GHz, 101 pts, z0=[ 50.+0.j],
```

3.21.4 General Purpose Objects

HomoList(list_)	A Homogeneous Sequence.
HomoDict(dict_)	A Homogeneous Mutable Mapping.

skrf.util.HomoList

class skrf.util.**HomoList**(*list_*)

A Homogeneous Sequence.

Provides a class for a list-like object which contains homogeneous values. Attributes of the values can be accessed through the attributes of HomoList. Searching is done like numpy arrays.

Initialized from a list of all the same type

```
>>> h = HomoDict([Foo(...), Foo(...)])
```

The individual values of *h* can be access in identical fashion to Lists.

```
>>> h[0]
```

Assuming that *Foo* has property *prop* and function *func* ...

Access elements' properties:

```
>>> h.prop
```

Access elements' functions:

```
>>> h.func()
```

Searching:

```
>>> h[h.prop == value]
>>> h[h.prop < value]
```

Multiple search:

```
>>> h[set(h.prop==value1) & set( h.prop2==value2)]
```

Combos:

```
>>> h[h.prop==value].func()
```

Methods

__init__	
count	
index	Raises ValueError if the value is not present.

skrf.util.HomoList.__init__

HomoList.**__init__**(*list_*)

skrf.util.HomoList.count

HomoList.**count**(*value*) → integer -- return number of occurrences of value

skrf.util.HomoList.index

HomoList.**index**(*value*[, *start*[, *stop*]]) → integer -- return first index of value.

Raises ValueError if the value is not present.

Supporting start and stop arguments is optional, but recommended.

skrf.util.HomoDict

class skrf.util.**HomoDict**(*dict_*)

A Homogeneous Mutable Mapping.

Provides a class for a dictionary-like object which contains homogeneous values. Attributes of the values can be accessed through the attributes of HomoDict. Searching is done like numpy arrays.

Initialized from a dictionary containing values of all the same type

```
>>> h = HomoDict({'a':Foo(...),'b': Foo(...), 'c':Foo(..)})
```

The individual values of *h* can be access in identical fashion to Dictionaries.

```
>>> h['key']
```

Assuming that *Foo* has property *prop* and function *func* ...

Access elements' properties:

```
>>> h.prop
```

Access elements' functions:

```
>>> h.func()
```

Searching:

```
>>> h[h.prop == value]
>>> h[h.prop < value]
```

Multiple search:

```
>>> h[set(h.prop==value1) & set( h.prop2==value2)]
```

Combos:

```
>>> h[h.prop==value].func()
```

Methods

__init__	
clear	
copy	
filter	Filter self based on kwargs
filter_nones	
get	
items	
keys	
pop	If key is not found, d is returned if given, otherwise KeyError is raised.
popitem	as a 2-tuple; but raise KeyError if D is empty.
setdefault	
update	If E present and has a .keys() method, does: for k in E.keys(): D[k] = E[k] If E present and lacks .keys() method, does: for (k, v) in E: D[k] = v In either case, this is followed by: for k, v in F.items(): D[k] = v
values	

skrf.util.HomoDict.__init__

HomoDict.**__init__**(*dict_*)

skrf.util.HomoDict.clear

HomoDict.**clear**() → None. Remove all items from D.

skrf.util.HomoDict.copy

HomoDict.**copy**()

skrf.util.HomoDict.filter

HomoDict.**filter**(***kwargs*)

Filter self based on kwargs

This is equivalent to:

```
>>> h = HomoDict(...)
>>> for k in kwargs:
>>>     h = h[k ==kwargs[k]]
>>> return h
```

prefixing the kwarg value with a '!' causes a not equal test (!=)

Examples

```
>>> h = HomoDict(...)
>>> h.filter(name='jean', age = '18', gender ='!female')
```

skrf.util.HomoDict.filter_nones

HomoDict.**filter_nones**()

skrf.util.HomoDict.get

HomoDict.**get**(*k*[, *d*]) → D[k] if k in D, else d. d defaults to None.

skrf.util.HomoDict.items

HomoDict.**items**() → a set-like object providing a view on D's items

skrf.util.HomoDict.keys

HomoDict.**keys**() → a set-like object providing a view on D's keys

skrf.util.HomoDict.pop

HomoDict.**pop**(*k*[, *d*]) → v, remove specified key and return the corresponding value.

If key is not found, d is returned if given, otherwise KeyError is raised.

skrf.util.HomoDict.popitem

HomoDict.**popitem**() → (k, v), remove and return some (key, value) pair
as a 2-tuple; but raise KeyError if D is empty.

skrf.util.HomoDict.setdefault

HomoDict.**setdefault**(*k*[, *d*]) → D.get(k,d), also set D[k]=d if k not in D

skrf.util.HomoDict.update

HomoDict.**update**([*E*,]**F) → None. Update D from mapping/iterable E and F.
If E present and has a .keys() method, does: for k in E.keys(): D[k] = E[k] If E present and lacks .keys() method, does: for (k, v) in E: D[k] = v In either case, this is followed by: for k, v in F.items(): D[k] = v

skrf.util.HomoDict.values

HomoDict.**values**() → an object providing a view on D's values

3.22 IO

This Package provides functions and objects for input/output.

Reading and writing touchstone files is supported through the `Touchstone` class, which can be more easily used through the Network constructor, *__init__()*

The general functions *read()* and *write()* can be used to read and write [almost] any skrf object to disk, using the `pickle` module. This should only be used for temporary storage, because pickling is not stable over time, as skrf evolves.

3.22.1 touchstone (*skrf.io.touchstone*)

Touchstone class and utilities

Touchstone(file[, encoding])	Class to read touchstone s-parameter files.

skrf.io.touchstone.Touchstone

class skrf.io.touchstone.**Touchstone**(*file*, *encoding=None*)

Class to read touchstone s-parameter files.

The reference for writing this class is the draft of the Touchstone(R) File Format Specification Rev 2.0[1] and Touchstone(R) File Format Specification Version 2.0[2] and Touchstone(R) File Format Specification Version 2.1[3]

References

Parameters

- **file** (*str | Path | TextIO*)
- **encoding** (*str | None*)

[1] https://ibis.org/interconnect_wip/touchstone_spec2_draft.pdf
[2] https://ibis.org/touchstone_ver2.0/touchstone_ver2_0.pdf
[3] https://ibis.org/touchstone_ver2.1/touchstone_ver2_1.pdf

Attributes

sparameters	Touchstone data in tabular format.
version	The version string.

skrf.io.touchstone.Touchstone.sparameters

property Touchstone.**sparameters**: ndarray

Touchstone data in tabular format.

Returns
Frequency and data array.

Return type
np.ndarray

skrf.io.touchstone.Touchstone.version

property Touchstone.**version**: str

The version string.

Returns
Version

Return type
str

Methods

__init__	constructor
get_comment_variables	Convert hfss variable comments to a dict of vars.
get_comments	Returns the comments which appear anywhere in the file.
get_format	Returns the file format string used for the given format.
get_gamma_z0	Extracts Z0 and Gamma comments from touchstone file (if provided).
get_noise_data	
get_noise_names	
get_sparameter_arrays	Returns the s-parameters as a tuple of arrays.
get_sparameter_data	Get the data of the s-parameter with the given format.
get_sparameter_names	Generate a list of column names for the s-parameter data.
load_file	Load the touchstone file into the internal data structures.

skrf.io.touchstone.Touchstone.__init__

Touchstone.**__init__**(*file*, *encoding=None*)

constructor

Parameters

- **file** (*str, Path, or file-object*) – touchstone file to load
- **encoding** (*str, optional*) – define the file encoding to use. Default value is None, meaning the encoding is guessed (ANSI, UTF-8 or Latin-1).

Examples

From filename

```
>>> t = rf.Touchstone('network.s2p')
```

File encoding can be specified to help parsing the special characters:

```
>>> t = rf.Touchstone('network.s2p', encoding='ISO-8859-1')
```

From file-object

```
>>> file = open('network.s2p')
>>> t = rf.Touchstone(file)
```

From a io.StringIO object

```
>>> link = 'https://raw.githubusercontent.com/scikit-rf/scikit-rf/master/examples/
    basic_touchstone_plotting/horn antenna.s1p'
>>> r = requests.get(link)
>>> stringio = io.StringIO(r.text)
>>> stringio.name = 'horn.s1p'  # must be provided for the Touchstone parser
>>> horn = rf.Touchstone(stringio)
```

skrf.io.touchstone.Touchstone.get_comment_variables

Touchstone.**get_comment_variables**()

Convert hfss variable comments to a dict of vars.

Returns
var_dict – Dictionary containing the comments

Return type
dict (numbers, units)

skrf.io.touchstone.Touchstone.get_comments

Touchstone.**get_comments**(*ignored_comments=None*)

Returns the comments which appear anywhere in the file.

Comment lines containing ignored comments are removed. By default these are comments which contain special meaning within skrf and are not user comments.

Returns
processed_comments

Return type
string

Parameters
ignored_comments (*list[str]*)

skrf.io.touchstone.Touchstone.get_format

Touchstone.**get_format**(*format='ri'*)

Returns the file format string used for the given format.

This is useful to get some information.

Returns
format

Return type
string

Parameters
format (*Literal['db', 'ri', 'ma', 'orig']*)

skrf.io.touchstone.Touchstone.get_gamma_z0

Touchstone.**get_gamma_z0**()

Extracts Z0 and Gamma comments from touchstone file (if provided).

Returns

- **gamma** (*complex np.ndarray*) – complex propagation constant
- **z0** (*np.ndarray*) – complex port impedance

skrf.io.touchstone.Touchstone.get_noise_data

Touchstone.**get_noise_data**()

skrf.io.touchstone.Touchstone.get_noise_names

Touchstone.**get_noise_names**()

skrf.io.touchstone.Touchstone.get_sparameter_arrays

Touchstone.**get_sparameter_arrays**()

Returns the s-parameters as a tuple of arrays.

The first element is the frequency vector (in Hz) and the s-parameters are a 3d numpy array. The values of the s-parameters are complex number.

Returns
param

Return type
tuple of arrays

Examples

```
>>> f, a = self.sgetparameter_arrays()
>>> s11 = a[:, 0, 0]
```

skrf.io.touchstone.Touchstone.get_sparameter_data

Touchstone.**get_sparameter_data**(*format='ri'*)

Get the data of the s-parameter with the given format.

Parameters

- **format** (*str*) – Format: ri, ma, db
- **are** (*supported formats*) – ri: data in real/imaginary ma: data in magnitude and angle (degree) db: data in log magnitude and angle (degree)

Returns
ret – list of numpy.arrays

Return type
list

skrf.io.touchstone.Touchstone.get_sparameter_names

Touchstone.**get_sparameter_names**(*format='ri'*)

Generate a list of column names for the s-parameter data. The names are different for each format.

Parameters
format (*str*) – Format: ri, ma, db

Returns
names – list of strings

Return type
list

skrf.io.touchstone.Touchstone.load_file

Touchstone.**load_file**(*fid*)

Load the touchstone file into the internal data structures.

Parameters
fid (*file object*)

Functions related to reading/writing touchstones.

hfss_touchstone_2_gamma_z0(filename)	Extracts Z0 and Gamma comments from touchstone file.
hfss_touchstone_2_media(filename)	Creates a `Media` object from a a HFSS-style Touchstone file with Gamma and Z0 comments.
hfss_touchstone_2_network(filename)	Creates a `Network` object from a a HFSS-style Touchstone file.
read_zipped_touchstones(ziparchive[, dir, ...])	similar to skrf.io.read_all_networks, which works for directories but only for Touchstones in ziparchives.

skrf.io.touchstone.hfss_touchstone_2_gamma_z0

skrf.io.touchstone.**hfss_touchstone_2_gamma_z0**(*filename*)

Extracts Z0 and Gamma comments from touchstone file.

Takes a HFSS-style Touchstone file with Gamma and Z0 comments and extracts a triplet of arrays being: (frequency, Gamma, Z0)

Parameters
filename (*string*) – the HFSS-style Touchstone file

Returns

- **f** (*np.ndarray*) – frequency vector (in Hz)
- **gamma** (*complex np.ndarray*) – complex propagation constant
- **z0** (*np.ndarray*) – complex port impedance

Return type
tuple[*ndarray*, *ndarray*, *ndarray*]

Examples

```
>>> f,gamm,z0 = rf.hfss_touchstone_2_gamma_z0('line.s2p')
```

skrf.io.touchstone.hfss_touchstone_2_media

skrf.io.touchstone.**hfss_touchstone_2_media**(*filename*)

Creates a `Media` object from a a HFSS-style Touchstone file with Gamma and Z0 comments.

Parameters
filename (*string*) – the HFSS-style Touchstone file

Returns
my_media – the transmission line model defined by the gamma, and z0 comments in the HFSS file.

Return type
Media object

Examples

```
>>> port1_media, port2_media = rf.hfss_touchstone_2_media('line.s2p')
```

See also

hfss_touchstone_2_gamma_z0
returns gamma, and z0

skrf.io.touchstone.hfss_touchstone_2_network

skrf.io.touchstone.**hfss_touchstone_2_network**(*filename*)

Creates a `Network` object from a a HFSS-style Touchstone file.

Parameters
filename (*string*) – the HFSS-style Touchstone file

Returns
my_network – the n-port network model

Return type
Network object

Examples

```
>>> my_network = rf.hfss_touchstone_2_network('DUT.s2p')
```

See also

hfss_touchstone_2_gamma_z0
returns gamma, and z0

skrf.io.touchstone.read_zipped_touchstones

skrf.io.touchstone.**read_zipped_touchstones**(*ziparchive*, *dir=''*, *encoding=None*)

similar to skrf.io.read_all_networks, which works for directories but only for Touchstones in ziparchives.

Parameters

- **ziparchive** (zipfile.ZipFile) – an zip archive file, containing Touchstone files and open for reading
- **dir** (*str*) – the directory of the ziparchive to read networks from, default is "" which reads only the root directory
- **encoding** (*str, optional*) – File encoding. Supported encodings are ISO-8859-1 and UTF-8. If omitted, encoding is detected for each file.

Returns

keys are touchstone filenames without extensions values are network objects created from the touchstone files

Return type

dict

3.22.2 general (*skrf.io.general*)

General input/output functions for reading and writing skrf objects

Pickle functions

The read/write methods use the pickle module. These should only be used for temporary storage.

read(file, *args, **kwargs)	Read skrf object[s] from a pickle file.
read_all([dir, sort, contains, f_unit, ...])	Read all skrf objects in a directory.
read_all_networks(*args, **kwargs)	Read all networks in a directory.
write(file, obj[, overwrite])	Write skrf object[s] to a file.
write_all(dict_objs[, dir])	Write a dictionary of skrf objects individual files in *dir*.
save_sesh(dict_objs[, file, module, ...])	Save all *skrf* objects in the local namespace.

skrf.io.general.read

skrf.io.general.**read**(*file*, **args*, ***kwargs*)

Read skrf object[s] from a pickle file.

Reads a skrf object that is written with *write()*, which uses the pickle module.

Parameters

- **file** (*str, Path, or file-object*) – name of file, or a file-object
- ***args** (*arguments and keyword arguments*) – passed through to pickle.load
- ****kwargs** (*arguments and keyword arguments*) – passed through to pickle.load

Note

If *file* is a:

- a file-object, it is left open
- a filename, then a file-object is opened and closed.
- a file-object and reading fails, then the position is reset back to 0 using seek if possible.

Examples

```
>>> n = rf.Network(f=[1,2,3],s=[1,1,1],z0=50)
>>> n.write('my_ntwk.ntwk')
>>> n_2 = rf.read('my_ntwk.ntwk')
```

See also

read
: read a skrf object

write
: write skrf object[s]

read_all
: read all skrf objects in a directory

write_all
: write dictionary of skrf objects to a directory

skrf.io.general.read_all

skrf.io.general.**read_all**(*dir='.'*, *sort=True*, *contains=None*, *f_unit=None*, *obj_type=None*, *files=None*, *recursive=False*)

Read all skrf objects in a directory.

Attempts to load all files in *dir*, using *read()*. Any file that is not readable by skrf is skipped. Optionally, simple filtering can be achieved through the use of *contains* argument.

Parameters

- **dir** (*str or Path, optional*) – the directory to load from, default '.'
- **sort** (*boolean, default is True*) – filenames sorted by https://docs.python.org/3/library/stdtypes.html#list.sort without arguments
- **contains** (*str, optional*) – if not None, only files containing this substring will be loaded
- **f_unit** (*['hz','khz','mhz','ghz','thz']*) – for all *Network* objects, set their frequencies's f_unit
- **obj_type** (*str*) – Name of skrf object types to read (ie 'Network')
- **files** (*list, optional*) – list of files to load, bypasses dir parameter.
- **recursive** (*bool, optional*) – If True, search in the specified directory and all other nested directories

Returns
out – dictionary containing all loaded skrf objects. keys are the filenames without extensions, and the values are the objects

Return type
dictionary

Examples

```
>>> rf.read_all('skrf/data/')
{'delay_short': 1-Port Network: 'delay_short',  75-110 GHz, 201 pts, z0=[ 50.+0.j],
'line': 2-Port Network: 'line',  75-110 GHz, 201 pts, z0=[ 50.+0.j  50.+0.j],
'ntwk1': 2-Port Network: 'ntwk1',  1-10 GHz, 91 pts, z0=[ 50.+0.j  50.+0.j],
'one_port': one port Calibration: 'one_port', 500-750 GHz, 201 pts, 4-ideals/4-
↪measured,
...
```

```
>>> rf.read_all('skrf/data/', obj_type = 'Network')
{'delay_short': 1-Port Network: 'delay_short',  75-110 GHz, 201 pts, z0=[ 50.+0.j],
'line': 2-Port Network: 'line',  75-110 GHz, 201 pts, z0=[ 50.+0.j  50.+0.j],
'ntwk1': 2-Port Network: 'ntwk1',  1-10 GHz, 91 pts, z0=[ 50.+0.j  50.+0.j],
```

```
>>> rf.read_all(files = ['skrf/data/delay_short.s1p', 'skrf/data/line.s2p'], obj_
↪type = 'Network')
{'delay_short': 1-Port Network: 'delay_short',  75-110 GHz, 201 pts, z0=[ 50.+0.j],
'line': 2-Port Network: 'line',  75-110 GHz, 201 pts, z0=[ 50.+0.j  50.+0.j]}
```

See also

read
read a skrf object

write
write skrf object[s]

read_all
read all skrf objects in a directory

write_all
write dictionary of skrf objects to a directory

skrf.io.general.read_all_networks

skrf.io.general.**read_all_networks**(**args*, ***kwargs*)

Read all networks in a directory.

This is a convenience function. It just calls:

```
read_all(*args,obj_type='Network', **kwargs)
```

See also

read_all

skrf.io.general.write

skrf.io.general.**write**(*file*, *obj*, *overwrite=True*)

Write skrf object[s] to a file.

This uses the pickle module to write skrf objects to a file. Note that you can write any pickl-able python object. For example, you can write a list or dictionary of *Network* objects or *Calibration* objects. This will write out a single file. If you would like to write out a separate file for each object, use *write_all()*.

Parameters

- **file** (*file, Path, or string*) – File or filename to which the data is saved. If file is a file-object, then the filename is unchanged. If file is a string, an appropriate extension will be appended to the file name if it does not already have an extension.
- **obj** (*an object, or list/dict of objects*) – object or list/dict of objects to write to disk
- **overwrite** (*Boolean*) – if file exists, should it be overwritten?

Note

If *file* is a string, but doesn't contain a suffix, one is chosen automatically. Here are the extensions:

skrf object	extension
Frequency	'.freq'
Network	'.ntwk'
NetworkSet	'.ns'
Calibration	'.cal'
Media	'.med'
other	'.p'

Note

To make the file written by this method cross-platform, the pickling protocol 2 is used. See pickle for more info.

Examples

Convert a touchstone file to a pickled Network,

```
>>> n = rf.Network('my_ntwk.s2p')
>>> rf.write('my_ntwk',n)
>>> n_red = rf.read('my_ntwk.ntwk')
```

Writing a list of different objects

```
>>> n = rf.Network('my_ntwk.s2p')
>>> ns = rf.NetworkSet([n,n,n])
>>> rf.write('out',[n,ns])
>>> n_red = rf.read('out.p')
```

See also

read
: read a skrf object

write
: write skrf object[s]

read_all
: read all skrf objects in a directory

write_all
: write dictionary of skrf objects to a directory

skrf.network.Network.write
: write method of Network

skrf.calibration.calibration.Calibration.write
: write method of Calibration

skrf.io.general.write_all

skrf.io.general.**write_all**(*dict_objs*, *dir='.'*, **args*, ***kwargs*)

Write a dictionary of skrf objects individual files in *dir*.

Each object is written to its own file. The filename used for each object is taken from its key in the dictionary. If no extension exists in the key, then one is added. See *write()* for a list of extensions. If you would like to write the dictionary to a single output file use *write()*.

Note

Any object in dict_objs that is pickl-able will be written.

Parameters

- **dict_objs** (*dict*) – dictionary of skrf objects
- **dir** (*str*) – directory to save skrf objects into
- ***args** – passed through to *write()*. *overwrite* option may be of use.
- ****kwargs** – passed through to *write()*. *overwrite* option may be of use.

See also

read
: read a skrf object

write
write skrf object[s]

read_all
read all skrf objects in a directory

write_all
write dictionary of skrf objects to a directory

Examples

Writing a diction of different skrf objects

```
>>> from skrf.data import line, short
>>> d = {'ring_slot':ring_slot, 'one_port_cal':one_port_cal}
>>> rf.write_all(d)
```

skrf.io.general.save_sesh

skrf.io.general.**save_sesh**(*dict_objs*, *file='skrfSesh.p'*, *module='skrf'*, *exclude_prefix='_'*)

Save all *skrf* objects in the local namespace.

This is used to save current workspace in a hurry, by passing it the output of `locals()` (see Examples). Note this can be used for other modules as well by passing a different *module* name.

Parameters

- **dict_objs** (*dict*) – dictionary containing *skrf* objects. See the Example.
- **file** (*str or file-object, optional*) – the file to save all objects to
- **module** (*str, optional*) – the module name to grep for.
- **exclude_prefix** (*str, optional*) – dont save objects which have this as a prefix.

See also

read
read a skrf object

write
write skrf object[s]

read_all
read all skrf objects in a directory

write_all
write dictionary of skrf objects to a directory

Examples

Write out all skrf objects in current namespace.

```
>>> rf.write_all(locals(), 'mysesh.p')
```

Spreadsheets

network_2_spreadsheet(ntwk[, file_name, ...])	Write a Network object to a spreadsheet, for your boss.
networkset_2_spreadsheet(ntwkset[, ...])	Write a NetworkSet object to a spreadsheet, for your boss.

skrf.io.general.network_2_spreadsheet

skrf.io.general.**network_2_spreadsheet**(*ntwk*, *file_name=None*, *file_type='excel'*, *form='db'*, **args*, ***kwargs*)

Write a Network object to a spreadsheet, for your boss.

Write the s-parameters of a network to a spreadsheet, in a variety of forms. This functions makes use of the pandas module, which in turn makes use of the xlrd module. These are imported during this function call. For more details about the file-writing functions see the *pandas.DataFrom.to_???* functions.

Note

The frequency unit used in the spreadsheet is take from *ntwk.frequency.unit*

Parameters

- **ntwk** (*Network* object) – the network to write
- **file_name** (*str, Path or None*) – the file_name to write. if None, ntwk.name is used.
- **file_type** (*['csv','excel','html']*) – the type of file to write. See *pandas.DataFrame.to_???* functions.
- **form** (*'db','ma','ri'*) – format to write data, * db = db, deg * ma = mag, deg * ri = real, imag
- ***args** – passed to *pandas.DataFrame.to_???* functions.
- ****kwargs** – passed to *pandas.DataFrame.to_???* functions.

See also

networkset_2_spreadsheet
writes a spreadsheet for many networks

skrf.io.general.networkset_2_spreadsheet

skrf.io.general.**networkset_2_spreadsheet**(*ntwkset*, *file_name=None*, *file_type='excel'*, **args*, ***kwargs*)

Write a NetworkSet object to a spreadsheet, for your boss.

Write the s-parameters of a each network in the networkset to a spreadsheet. If the *excel* file_type is used, then each network, is written to its own sheet, with the sheetname taken from the network *name* attribute. This functions makes use of the pandas module, which in turn makes use of the xlrd module. These are imported during this function.

Note

The frequency unit used in the spreadsheet is take from *ntwk.frequency.unit*

Parameters

- **ntwkset** (*NetworkSet* object) – the network to write
- **file_name** (*str, None*) – the file_name to write. if None, ntwk.name is used.
- **file_type** (*['csv','excel','html']*) – the type of file to write. See *pandas.DataFrame.to_???* functions.
- **form** (*'db','ma','ri'*) – format to write data, * db = db, deg * ma = mag, deg * ri = real, imag
- ***args** – passed to *pandas.DataFrame.to_???* functions.
- ****kwargs** – passed to *pandas.DataFrame.to_???* functions.

See also

networkset_2_spreadsheet
writes a spreadsheet for many networks

Pandas dataframe

network_2_dataframe(ntwk[, attrs, ports, ...])	Convert one or more attributes of a network to a pandas DataFrame.

skrf.io.general.network_2_dataframe

skrf.io.general.**network_2_dataframe**(*ntwk*, *attrs=None*, *ports=None*, *port_sep=None*)

Convert one or more attributes of a network to a pandas DataFrame.

Parameters

- **ntwk** (*Network* object) – the network to write
- **attrs** (*list Network attributes*) – like [‘s_db’,’s_deg’]
- **ports** (*list of tuples*) – list of port pairs to write. defaults to ntwk.port_tuples (like [(0,0)])
- **port_sep** (*string*) – defaults to None, which means a empty string “” is used for networks with lower than 10 ports. (s_db 11, s_db 21) For more than ten ports a “_” is used to avoid ambiguity. (s_db 1_1, s_db 2_1)

Returns
df

Return type
pandas DataFrame Object

Statistics

statistical_2_touchstone(file_name[, ...])	Converts Statistical file to a touchstone file.

skrf.io.general.statistical_2_touchstone

skrf.io.general.**statistical_2_touchstone**(*file_name*, *new_file_name=None*, *header_string='# GHz S RI R 50.0'*)

Converts Statistical file to a touchstone file.

Converts the file format used by Statistical and other Dylan Williams software to standard touchstone format.

Parameters

- **file_name** (*string*) – name of file to convert
- **new_file_name** (*string*) – name of new file to write out (including extension)
- **header_string** (*string*) – touchstone header written to first beginning of file

JSON

TouchstoneEncoder(*[, skipkeys, ...])	Serializes Network object by converting arrays to lists, splitting complex numbers into real and imaginary, and breaking down frequency objects into dicts.
to_json_string(network)	Dumps Network to JSON string.
from_json_string(obj_string)	Loads network object from JSON string representation.

skrf.io.general.TouchstoneEncoder

class skrf.io.general.**TouchstoneEncoder**(*, *skipkeys=False*, *ensure_ascii=True*, *check_circular=True*, *allow_nan=True*, *sort_keys=False*, *indent=None*, *separators=None*, *default=None*)

Serializes Network object by converting arrays to lists, splitting complex numbers into real and imaginary, and breaking down frequency objects into dicts.

Attributes

item_separator
key_separator

skrf.io.general.TouchstoneEncoder.item_separator

TouchstoneEncoder.**item_separator** = ', '

skrf.io.general.TouchstoneEncoder.key_separator

TouchstoneEncoder.**key_separator** = ': '

Methods

__init__	Constructor for JSONEncoder, with sensible defaults.
default	Implement this method in a subclass such that it returns a serializable object for `o`, or calls the base implementation (to raise a `TypeError`).
encode	Return a JSON string representation of a Python data structure.
iterencode	Encode the given object and yield each string representation as available.

skrf.io.general.TouchstoneEncoder.__init__

TouchstoneEncoder.**__init__**(*, *skipkeys=False*, *ensure_ascii=True*, *check_circular=True*, *allow_nan=True*, *sort_keys=False*, *indent=None*, *separators=None*, *default=None*)

Constructor for JSONEncoder, with sensible defaults.

If skipkeys is false, then it is a TypeError to attempt encoding of keys that are not str, int, float, bool or None. If skipkeys is True, such items are simply skipped.

If ensure_ascii is true, the output is guaranteed to be str objects with all incoming non-ASCII characters escaped. If ensure_ascii is false, the output can contain non-ASCII characters.

If check_circular is true, then lists, dicts, and custom encoded objects will be checked for circular references during encoding to prevent an infinite recursion (which would cause an RecursionError). Otherwise, no such check takes place.

If allow_nan is true, then NaN, Infinity, and -Infinity will be encoded as such. This behavior is not JSON specification compliant, but is consistent with most JavaScript based encoders and decoders. Otherwise, it will be a ValueError to encode such floats.

If sort_keys is true, then the output of dictionaries will be sorted by key; this is useful for regression tests to ensure that JSON serializations can be compared on a day-to-day basis.

If indent is a non-negative integer, then JSON array elements and object members will be pretty-printed with that indent level. An indent level of 0 will only insert newlines. None is the most compact representation.

If specified, separators should be an (item_separator, key_separator) tuple. The default is (', ', ': ') if *indent* is `None` and (',', ': ') otherwise. To get the most compact JSON representation, you should specify (',', ':') to eliminate whitespace.

If specified, default is a function that gets called for objects that can't otherwise be serialized. It should return a JSON encodable version of the object or raise a `TypeError`.

skrf.io.general.TouchstoneEncoder.default

TouchstoneEncoder.**default**(*obj*)

Implement this method in a subclass such that it returns a serializable object for `o`, or calls the base implementation (to raise a `TypeError`).

For example, to support arbitrary iterators, you could implement default like this:

```
def default(self, o):
    try:
        iterable = iter(o)
    except TypeError:
        pass
    else:
        return list(iterable)
    # Let the base class default method raise the TypeError
    return super().default(o)
```

skrf.io.general.TouchstoneEncoder.encode

TouchstoneEncoder.**encode**(*o*)

Return a JSON string representation of a Python data structure.

```
>>> from json.encoder import JSONEncoder
>>> JSONEncoder().encode({"foo": ["bar", "baz"]})
'{"foo": ["bar", "baz"]}'
```

skrf.io.general.TouchstoneEncoder.iterencode

TouchstoneEncoder.**iterencode**(*o*, *_one_shot=False*)

Encode the given object and yield each string representation as available.

For example:

```
for chunk in JSONEncoder().iterencode(bigobject):
    mysocket.write(chunk)
```

skrf.io.general.to_json_string

skrf.io.general.**to_json_string**(*network*)

Dumps Network to JSON string.

Faster than converting and saving as touchstone. Safer than pickling (no arbitrary code execution on load).

Parameters
network (*Network* object) – A Network object to be serialized and returned as a JSON string.

Returns
json – JSON string representation of a network object.

Return type
str

skrf.io.general.from_json_string

skrf.io.general.**from_json_string**(*obj_string*)

Loads network object from JSON string representation.

Parameters
obj_string (*str*) – JSON string representation of a network object.

Returns
ntwk – A Network object, rebuilt from JSON.

Return type
Network object

3.22.3 csv (*skrf.io.csv*)

Functions for reading and writing standard csv files

read_all_csv([dir, contains])	Read all CSV files in a directory.
AgilentCSV(filename, *args, **kwargs)	Agilent-style csv file representing either scalar traces vs frequency or complex data vs. frequency.

skrf.io.csv.read_all_csv

skrf.io.csv.**read_all_csv**(*dir='.'*, *contains=None*)

Read all CSV files in a directory.

Parameters

- **dir** (*str, optional*) – the directory to load from, default '.'
- **contains** (*str, optional*) – if not None, only files containing this substring will be loaded

Returns
out – dictionary containing all loaded CSV objects. keys are the filenames without extensions, and the values are the objects

Return type
dictionary

skrf.io.csv.AgilentCSV

class skrf.io.csv.**AgilentCSV**(*filename*, **args*, ***kwargs*)

Agilent-style csv file representing either scalar traces vs frequency or complex data vs. frequency.

Attributes

columns	list of str.
dataframe	Pandas DataFrame representation of csv file.
dict	Dictionary representation of csv file.
frequency	*Frequency*.
n_traces	int
networks	Reads a PNAX csv file, and returns a list of one-port Networks.
scalar_networks	Returns list of Networks for each column.

skrf.io.csv.AgilentCSV.columns

property AgilentCSV.**columns**

list of str.

This function is needed because Agilent allows the delimiter of a csv file (ie ') to be present in the header name. ridiculous.

If splitting the header fails, then a suitable list is returned of the correct length, which looks like:

```
['Freq(?)','filename-0','filename-1',..]
```

Type
List of column names

skrf.io.csv.AgilentCSV.dataframe

property AgilentCSV.**dataframe**

Pandas DataFrame representation of csv file.

Returns
df

Return type
pandas.DataFrame

skrf.io.csv.AgilentCSV.dict

property AgilentCSV.**dict**

Dictionary representation of csv file.

Returns
dict

Return type
dict

skrf.io.csv.AgilentCSV.frequency

property AgilentCSV.**frequency**

Frequency.

Type
Frequency object

skrf.io.csv.AgilentCSV.n_traces

property AgilentCSV.**n_traces**

int

Type
number of data traces

skrf.io.csv.AgilentCSV.networks

property AgilentCSV.**networks**

Reads a PNAX csv file, and returns a list of one-port Networks.

> **Note**
>
> Note this only works if csv is save in Real/Imaginary format for now

Parameters
filename (*str*) – filename

Returns
out – list of Networks representing the data contained in column pairs

Return type
list of *Network* objects

skrf.io.csv.AgilentCSV.scalar_networks

property AgilentCSV.**scalar_networks**

Returns list of Networks for each column.

> **Note**
>
> The data is stored in the Network's *.s* property, so its up to you to interpret results. if 'db' is in the column name then it is converted to linear before being store into *s*.

Returns
out – list of Networks representing the data contained in each column

Return type
list of *Network* objects

Methods

__init__	Init.
read	Reads data from file.

skrf.io.csv.AgilentCSV.__init__

AgilentCSV.**__init__**(*filename*, **args*, ***kwargs*)

Init.

Parameters

- **filename** (*str*) – filename
- ***args** – passed to Network.__init__ in *networks()* and *scalar_networks()*
- ****kwargs** – passed to Network.__init__ in *networks()* and *scalar_networks()*

skrf.io.csv.AgilentCSV.read

AgilentCSV.**read**()

Reads data from file.

This function returns a triplet containing the header, comments, and data.

Returns

- **header** (*str*) – The header string, which is the line following the 'BEGIN'
- **comments** (*str*) – All lines that begin with a '!'
- **data** (numpy.ndarray) – An array containing the data. The meaning of which depends on the header.

Reading/Writing Agilent

read_pna_csv(filename, *args, **kwargs)	Reads data from a csv file written by an Agilent PNA.
pna_csv_2_ntwks(filename)	Reads a PNAX csv file, and returns a list of one-port Networks.
pna_csv_2_ntwks3(filename)	Read a CSV file exported from an Agilent PNA in dB/deg format.
pna_csv_2_df(filename)	Reads data from a csv file written by an Agilent PNA as a pandas DataFrame.

skrf.io.csv.read_pna_csv

skrf.io.csv.**read_pna_csv**(*filename*, **args*, ***kwargs*)

Reads data from a csv file written by an Agilent PNA.

This function returns a triplet containing the header, comments, and data.

Parameters

- **filename** (*str*) – the file
- ***args**
- ****kwargs**

Returns

- **header** (*str*) – The header string, which is the line following the 'BEGIN'
- **comments** (*str*) – All lines that begin with a '!'
- **data** (numpy.ndarray) – An array containing the data. The meaning of which depends on the header.

Return type

tuple[str, str, *ndarray*]

See also

pna_csv_2_ntwks
Reads a csv file which contains s-parameter data

Examples

```
>>> header, comments, data = rf.read_pna_csv('myfile.csv')
```

skrf.io.csv.pna_csv_2_ntwks

skrf.io.csv.**pna_csv_2_ntwks**(*filename*)

Reads a PNAX csv file, and returns a list of one-port Networks.

Deprecated since version Use: *pna_csv_2_ntwks3()* instead.

> **Note**
>
> Note this only works if csv is save in Real/Imaginary format for now

Parameters
filename (*str*) – filename

Returns
out – list of Networks representing the data contained in column pairs

Return type
list of *Network* objects

skrf.io.csv.pna_csv_2_ntwks3

skrf.io.csv.**pna_csv_2_ntwks3**(*filename*)

Read a CSV file exported from an Agilent PNA in dB/deg format.

Parameters
filename (*str*) – full path or filename

Returns
out – 2-Port Network

Return type
n

skrf.io.csv.pna_csv_2_df

skrf.io.csv.**pna_csv_2_df**(*filename*)

Reads data from a csv file written by an Agilient PNA as a pandas DataFrame.

Parameters
filename (*string*) – filename

Returns
df

Return type
pandas.DataFrame

Reading/Writing R&S

read_zva_dat(filename, *args, **kwargs)	Reads data from a dat file written by a R&S ZVA in dB/deg or re/im format.
read_all_zva_dat([dir, contains])	Read all DAT files in a directory (from R&S ZVA).
zva_dat_2_ntwks(filename)	Read a dat file exported from a R&S ZVA in dB/deg or re/im format.

skrf.io.csv.read_zva_dat

skrf.io.csv.**read_zva_dat**(*filename*, **args*, ***kwargs*)

Reads data from a dat file written by a R&S ZVA in dB/deg or re/im format.

This function returns a triplet containing header, comments and data.

Parameters

- **filename** (*str*) – the file
- ***args**
- ****kwargs**

Returns

- **header** (*str*) – The header string, which is the line following the 'BEGIN'
- **data** (*numpy.ndarray*) – An array containing the data. The meaning of which depends on the header.

skrf.io.csv.read_all_zva_dat

skrf.io.csv.**read_all_zva_dat**(*dir='.'*, *contains=None*)

Read all DAT files in a directory (from R&S ZVA).

Parameters

- **dir** (*str, optional*) – the directory to load from, default '.'
- **contains** (*str, optional*) – if not None, only files containing this substring will be loaded

Returns

out – dictionary containing all loaded DAT objects. keys are the filenames without extensions, and the values are the objects

Return type

dictionary

skrf.io.csv.zva_dat_2_ntwks

skrf.io.csv.**zva_dat_2_ntwks**(*filename*)

Read a dat file exported from a R&S ZVA in dB/deg or re/im format.

Parameters

filename (*str*) – full path or filename

Returns

out – 2-Port Network

Return type

n

Reading/Writing Anritsu VectorStar

vectorstar_csv_2_ntwks(filename)	Reads a vectorstar csv file, and returns a list of one-port Networks.

continues on next page

Table 211 – continued from previous page

read_vectorstar_csv(filename, *args, **kwargs)	Reads data from a csv file written by an Anritsu VectorStar.

skrf.io.csv.vectorstar_csv_2_ntwks

skrf.io.csv.**vectorstar_csv_2_ntwks**(*filename*)

Reads a vectorstar csv file, and returns a list of one-port Networks.

> **Note**
>
> Note this only works if csv is save in Real/Imaginary format for now

Parameters
filename (*str*) – filename

Returns
out – list of Networks representing the data contained in column pairs

Return type
list of *Network* objects

skrf.io.csv.read_vectorstar_csv

skrf.io.csv.**read_vectorstar_csv**(*filename*, **args*, ***kwargs*)

Reads data from a csv file written by an Anritsu VectorStar.

Parameters

- **filename** (*str*) – the file
- ***args**
- ****kwargs**

Returns

- **header** (*str*) – The header string, which is the line just before the data
- **comments** (*str*) – All lines that begin with a '!'
- **data** (numpy.ndarray) – An array containing the data. The meaning of which depends on the header.

CHAPTER

FOUR

CONTRIBUTING TO SCIKIT-RF

4.1 Sponsoring the Project

It is possible to sponsor the maintainers and developers of the scikit-rf package, using the GitHub Sponsor option ("Sponsor this project") visible on the scikit-rf GitHub page.

Sponsoring is one more way to contribute to open source: financially supporting the people who build and maintain it. Funding individuals helps them keep doing important work, expands opportunities to participate, and gives developers the recognition they deserve.

4.2 Contributing to the Code

Note

if you feel that the instructions on this page are too complicated, but you still would like to contribute, do not hesitate to email us on the scikit-rf mailing list or join us in the scikit-rf Slack channel or the scikit-rf Matrix/Element channel to get some help.

skrf uses the "Fork + Pull" collaborative development model. If this is new to you, see GitHub's articles on forking and pull requests.

Basically, you will:

1. Fork the GitHub scikit-rf repository,
2. Make your modifications.
3. Then send a pull request (PR) for your additions to be merged into the main scikit-rf repository. Your proposal will be reviewed or discussed and you may receive some comments which only aim to make your contribution as great as possible!

Tip

When making your modification locally, you may need to work into a dedicated development environment in order to not interfere with the scikit-rf package that you have already installed. You can use for example anaconda environment. In order for the anaconda environment to find your local scikit-rf repo, use the convenient conda-develop command.

4.2.1 Prek

You can use *prek* to automate your code quality checks and perform automated fixes in **scikit-rf**.

To enable prek on your computer (make sure it is installed first), open a terminal and Navigate to the `scikit-rf/` directory that contains your clone of scikit-rf's repository, then run:

```
prek install
```

Note

Once prek has been installed for a repository, prek will run every time you try to commit a change. If any prek checks fail, or if prek changes any files, it will be necessary to redo *git add* on the changed files and *git commit* once again.

Tip

To commit a change without running prek, use the *-n* or *–no-verify* flag with git.

4.2.2 Basic git command-line workflow

The following is a basic example of the git commands that can be used to contribute to the code.

```
# create your own fork of scikit-rf in the GitHub interface, say ME/scikit-rf

# clone your new fork locally, using either:
git clone ME@github.com:ME/scikit-rf.git

# if you have ssh keys setup, or if not:
git clone https://github.com/scikit-rf/scikit-rf.git

# ... make your changes...

# commit changes locally
git commit -a

# push changes to your repo
git push origin

# create a pull request on github.com
```

4.2.3 Staying Synchronized

To keep your local version synchronized (up-to-date) with the scikit-rf repository, add a "remote" (call it "upstream"). From this remote, you can "fetch" and "merge" the official scikit-rf repo's changes into your local repo.

```
git remote add upstream https://github.com/scikit-rf/scikit-rf.git

# Fetch any new changes from the original repo
git fetch upstream
```

(continues on next page)

(continued from previous page)

```
# Merges any changes fetched into your working files
git merge upstream/master
```

4.2.4 Tests

Tests are vital for software reliability and maintainability. Writing tests often require additional effort but saves time in the long run. Tests enable us to quickly discover when we introduce new errors. It is also a way to provide examples of how functions and classes were originally intended to be used.

Before making a Pull Request, we advise contributors to run the tests locally to check if nothing has been broken following their modifications. In addition, we highly recommend providing new tests when adding new features.

The structure of the testing generally follows the conventions of numpy/scipy. Test cases live in the module, or sub-module, which they are testing, and are located in a directory called *tests*. So, the tests of the media module are located at *skrf/media/tests/*. Tests can be run most easily with pytest.

You probably **don't** want to run the tests for the virtual instruments `skrf.vi` with the rest of the tests, so these tests are excluded by default.

To run all the tests (except the virtual instruments)

```
cd scikit-rf
# Create environment and install dependencies, needed only once.
python -m venv .venv
pip install -e .[test,visa,netw,xlsx,plot,docs,testspice] --compile

# Activate Environment, needed for all following steps.
# on Linux Systems
. .venv/bin/activate

# on Windows
.\.venv\Scripts\activate

pytest
```

To run all tests *and* all tutorials and example notebooks in your current environment (recommended before making a pull request):

```
pytest --nbval-lax
```

If you want to test a single file or directory, you need to override the default pytest configuration and indicate the test path. For example, to run only the tests associated with the Network object (-v to increase the verbosity):

```
pytest -v -c "" skrf/tests/test_network.py
```

It is also possible to select some particular tests using the regex option (-k):

```
pytest -v -c "" skrf/calibration/tests/test_calibration.py -k "test_error_ntwk"
```

4.3 Contributing to the Documentation

4.3.1 Examples and Tutorials

Usage examples of scikit-rf are welcomed, especially when adding new features. We are using Jupyter Notebooks to write the examples and the tutorials, which are located in the `scikit-rf/docs/source/examples/` and `doc/source/examples` directories. These notebooks are then converted into webpages with the sphinx extension called nbsphinx.

The docs are automatically built and served by readthedocs when a Pull Request is accepted. The Python package requirements to build the docs are kept in `scikit-rf/pyproject.toml`.

Important

Before pushing to your repo and making a pull request, at a minimum you will need to clear the notebook outputs using the "Clear All Output" command in the notebook (or install nbstripout so that the output is not tracked in git (or the repo size would grow infinitely).

4.3.2 Reference (API) or static documentation

The documentation source files can be found in `doc/source/`.

The reference documentation for the functions, classes, and submodules are documented in docstrings following the conventions put forth by Numpy/Scipy docstring format. The documentation as a whole is generated using sphinx, and written using reStructed (.rst) Text.

Tip

If you want to write some .rst file yourself, please use an RST format editor and checker (ex: https://livesphinx.herokuapp.com/), as Sphinx is (very) picky with the syntax…

4.3.3 Building the documentation locally

Before making a pull request concerning the documentation, it is a good idea to test locally if your changes lead to the desired HTML output (sometimes some problems can arise during the conversion to HTML). The documentation is built by the following commands:

```
# be sure to be in the scikit-rf/doc directory
make html
```

The built docs then reside in `doc/build/html`.

4.4 Join the scikit-rf team!

Do you like using scikit-rf? Merchandise is available for you to express your love.

scikit-rf
Open Source RF Engineering
blam.

CHAPTER

FIVE

ACKNOWLEDGING OR CITING SCIKIT-RF

5.1 In Publications

If you use scikit-rf for work/research presented in a publication (whether directly, or as a dependency to another package), we ask that you acknowledge it. Perhaps with something like:

```
This research made use of scikit-rf, an open-source Python package for RF and Microwave
applications.
```

You can also include a link to www.scikit-rf.org (if the journal allows this) in addition to the above text.

For academic citations, please cite the IEEE Microwave Magazine article that some of the community members wrote. The IEEE-style citation for this paper is

A. Arsenovic *et al.*, "scikit-rf: An Open Source Python Package for Microwave Network Creation, Analysis, and Calibration [Speakers Corner]," in *IEEE Microwave Magazine*, vol. 23, no. 1, pp. 98-105, Jan. 2022, doi: 10.1109/MMM.2021.3117139.

and in BibTex format

```
@ARTICLE{9632487,
author={Arsenovic, Alexander and Hillairet, Julien and Anderson, Jackson and Forstén,␣
↪Henrik and Rieß, Vincent and Eller, Michael and Sauber, Noah and Weikle, Robert and␣
↪Barnhart, William and Forstmayr, Franz},
journal={IEEE Microwave Magazine},
title={scikit-rf: An Open Source Python Package for Microwave Network Creation, Analysis,
↪ and Calibration [Speakers Corner]},
year={2022},
volume={23},
number={1},
pages={98-105},
doi={10.1109/MMM.2021.3117139}}
```

5.2 In Presentations or Apps

If you are giving a presentation featuring work/research that makes use of scikit-rf or creating an app that uses it under the hood, we provide this "powered by" image [PNG] [SVG] :

Powered By

CHAPTER

SIX

PUBLICATIONS USING OR CITING SCIKIT-RF

This list has been extracted from a "scikit-rf" research on OpenAlex combined to the citations of the scikit-rf paper described above, and is certainly non-exhaustive. Please contact us or make a Pull Request if you want to add a reference in this list.

6.1 2026

- Ding, Wei, et al. "A Multi-Resolution Machine Learning Paradigm for Pixelated Metasurface Design." Laser & Photonics Reviews (2026): e02024.
- Fan, Guozhuang, et al. "Uncertainty quantification in microelectronic packaging using feedback-enhanced adaptive polynomial chaos expansion." Microelectronics Reliability 176 (2026): 115973.

6.2 2025

- Bezerra, Sávio Correia, et al. "Comparative Analysis of RVE and Complete Models of a Interdigitated Planar Capacitive Sensor with FEM." Journal of Integrated Circuits and Systems 20.3 (2025): 1-8.
- Doineau, Raphaël C. L-M., et al. "Self-Assembled Inorganic Nanomembrane Tubes: Rolled-Up Piezoelectrics for Microacoustic Wave-Based Actuators and Sensors." Advanced Materials (2025): e12619.
- Baek, Seungwoo. Process Development of Sub-5 Micrometer Spin-Coated Organic Dielectrics for Advanced Packaging. MS thesis. University of California, Los Angeles, 2025.
- Lin Htun, San, and Jillian L. Goldfarb. "Novel Reactor for In Situ Dielectric Constant Measurements of Fluids at High Temperature and Pressure." Journal of Chemical & Engineering Data 70.11 (2025): 4471-4482.
- Molina-Cuberos, Gregorio J., García-Collado, Ángel J. et al., "A Reconfigurable Metasurface for Linear-to-Circular Polarization Conversion Using Mechanical Rotation", Electronics, 2025
- Kuznetsov, Vadim, "Microstrip Line Modeling Taking into Account Dispersion Using a General-Purpose SPICE Simulator", Journal of Low Power Electronics and Applications, 2025
- Van de Lindt, J.G., Wukitch, S. et al., "Design workflow of a symmetric traveling wave antenna for fast ion production on DD tokamaks", Nuclear Fusion, 2025
- Gabe Hoshino, S. Knirck, et al., "First Axionlike Particle Results from a Broadband Search for Wavelike Dark Matter in the 44 to 52 eV Range with a Coaxial Dish Antenna", Physical Review Letters, 2025
- Paliwoda, Matt, Tejero, Erik, et al., "De-embedding a pair of dipoles to calibrate the mutual impedance plasma density diagnostic", Frontiers in Astronomy and Space Sciences, 2025
- Greenfield, Jonathan, Bell, Cynthia S., et al., "Kinetic inductance and non-linearity of MgB2 films at 4K", Applied Physics Letters, 2025

- Stepanov, I., et al. "Setup and first operation of the Wendelstein 7-X ICRH matching system." Fusion Engineering and Design 211 (2025): 114794. https://doi.org/10.1016/j.fusengdes.2024.114794
- Paliwoda, Matthew Crawford, Erik M. Tejero, and George R. Gatling. "De-embedding a pair of dipoles to calibrate the mutual impedance plasma density diagnostic." Frontiers in Astronomy and Space Sciences 12: 1519446.
- Hillairet, Julien. "WEST Ion Cyclotron Antenna electrical model digital twin." OSSFE 2025-Open-Source Software for Fusion Energy. Zenodo, 2025. https://dx.doi.org/10.5281/zenodo.15053680
- Mueller, Mark Niklas, and Martin Vechev. "Automated Benchmark Generation for Repository-Level Coding Tasks." ICLR 2025 Third Workshop on Deep Learning for Code. https://openreview.net/forum?id=BQA7dkV3iZ
- Caignet, F., et al. "Reflection to improve ESD protection simulation based on voltage-dependent dynamic model." (2025). https://hal.science/hal-04978913/
- Zhu, Jiacong, et al. "RFSoC receiver calibration system for 21-cm global spectrum experiments from space: The CosmoCube case." RAS Techniques and Instruments (2025): rzae064. https://doi.org/10.1093/rasti/rzae064
- Hoshino, Gabe, et al. "First Axion-Like Particle Results from a Broadband Search for Wave-Like Dark Matter in the 44 to 52$mu $ eV Range with a Coaxial Dish Antenna." arXiv preprint arXiv:2501.17119 (2025). https://doi.org/10.48550/arXiv.2501.17119
- Campbell, Bailey. "Distributed Hybrid Beamforming Techniques for Random Three Dimensional Arrays." (2025). https://etda.libraries.psu.edu/catalog/28168rdc233
- King, Benjamin, Nikolas Bruce, and Mahmoud Wagih. "Large-Area Conductor-Loaded PDMS Flexible Composites for Wireless and Chipless Electromagnetic Multiplexed Temperature Sensors." Advanced Science (2025): 2412066. https://doi.org/10.1002/advs.202412066
- Greenfield, J., et al. "Kinetic inductance and non-linearity of MgB2 films at 4K." Applied Physics Letters 126.2 (2025). https://doi.org/10.1063/5.0245866
- Xenidis, Nikolaos, et al. "Highly efficient hierarchically porous carbon-silica composite for sub-terahertz stealth and shielding applications." Computational and Structural Biotechnology Journal (2025). https://doi.org/10.1016/j.csbj.2025.02.021

6.3 2024

- Emser, A. L. , Metzger, C. et al., "Thin-film quartz for high-coherence piezoelectric phononic crystal resonators ", Physical Review Applied, 2024
- Prod'homme, Hugo, and Philipp del Hougne. "Updatable Closed-Form Evaluation of Arbitrarily Complex Multi-Port Network Connections." arXiv preprint arXiv:2412.17884 (2024). https://doi.org/10.48550/arXiv.2412.17884
- Gregory, Andrew P., Peter D. Woolliams, and Stephen M. Hanham. "Robust Algorithms for Fitting Q-Factor in the Complex Domain." IEEE Access (2024). https://doi.org/10.1109/ACCESS.2024.3514707
- Karime, Ahmad, Alberto Battistel, and Knut Möller. "Vector Network Analyzer (VNA) Measurements for Electrical Impedance Tomography." Current Directions in Biomedical Engineering. Vol. 10. No. 4. De Gruyter, 2024. https://doi.org/10.1515/cdbme-2024-2088
- Jepsen, Rasmus Alexander, Jan Henrik Ardenkjær-Larsen, and Vitaliy Zhurbenko. "Four-element LC-baluns for power matching arbitrary impedances." International Journal of Microwave and Wireless Technologies (2024): 1-9. https://doi.org/10.1017/S1759078724000722
- Sutantyo, Evan Kinanda, Dion Hayu Fandiantoro, and Arief Kurniawan. "Detection of Iron Corrosion in Concrete Using 1D Convolution." 2024 International Conference on Computer Engineering, Network, and Intelligent Multimedia (CENIM). IEEE, 2024. https://doi.org/10.1109/CENIM64038.2024.10882636

- Loffler, Jens, et al. "Broadband 11 0 GHz Multiport Scalar Network Analysis Methodology Using Port Extension Across Multiple VNAs." 2024 IEEE Asia-Pacific Microwave Conference (APMC). IEEE, 2024. https://doi.org/10.1109/APMC60911.2024.10867595
- Rodríguez Pérez, Jorge. "Construcción de un pequeño radiotelescopio para la detección de la línea del hidrógeno neutro." (2024). https://uvadoc.uva.es/handle/10324/71012
- Hassab, Youcef, et al. "Machine Learning Based Data Validation for Signal Integrity and Power Integrity Using Supervised Time Series Classification." IEEE Transactions on Electromagnetic Compatibility (2024). https://doi.org/10.1109/TEMC.2024.3474917
- Arsanjani, Arash, et al. "Miniaturized Ka-Band Metasurface Filter With Wide Out-of-Band Rejection up to the 5th Harmonic." 2024 54th European Microwave Conference (EuMC). IEEE, 2024. https://doi.org/10.23919/EuMC61614.2024.10732666
- Kanitkar, Abhijeet, et al. "Influence of On-Wafer Parasitic Effects on Mason's Gain of Down-Scaled InP HBTs." 2024 54th European Microwave Conference (EuMC). IEEE, 2024. https://doi.org/10.23919/EuMC61614.2024.10732142
- Löffler, Jens, et al. "High-Frequency Multiport Vector Network Analysis Methodology Using Port Extension Across Multiple VNAs." 2024 54th European Microwave Conference (EuMC). IEEE, 2024. https://doi.org/10.23919/EuMC61614.2024.10732150
- Moore, Christopher, et al. "Fabrication and Characterization of Submillimeter-Wave GaAs Quasi-Vertical Schottky Diodes Diffusion Bonded to Silicon." 2024 54th European Microwave Conference (EuMC). IEEE, 2024. https://doi.org/10.23919/EuMC61614.2024.10732102
- Ballandras, Sylvain, et al. "Filter performances for 5G bands based on SCAW design on POI." 2024 IEEE Ultrasonics, Ferroelectrics, and Frequency Control Joint Symposium (UFFC-JS). IEEE, 2024. https://doi.org/10.1109/UFFC-JS60046.2024.10793591
- Fodran, T. Commissioning the Auger Prime Radio Detector. Paving the Way for Mass Separation. Nijmegen: Radboud University Press, 2024. https://repository.ubn.ru.nl/bitstream/handle/2066/309692/309692.pdf?sequence=1
- Mendoza Ochoa, Leonardo Josue. "Detección de anomalías en un modelo craneal mediante la generación de imágenes médicas usando antenas de microondas." (2024). https://dspace.umh.es/handle/11000/33945
- Wood, S., N. Klimovich, and B-K. Tan. "Automated characterisation and operational insights of superconducting travelling wave parametric amplifiers: unveiling novel behaviours and enhancing tunability." Journal of Instrumentation 19.08 (2024): P08024. https://doi.org/10.1088/1748-0221/19/08/P08024/
- Yoon, Daeyeong, et al. "Estimation of Permittivity of Films Using the Free-Space Measurement Method at Ka-Band." IEEE Access (2024). https://doi.org/10.1109/ACCESS.2024.3445913
- Massingill, Kyle D. Development of Millimeter-Wave Instruments for Water Vapor Radiometry and Exploring the Regulation of Galaxy Evolution With High-Redshift ALMA Observations. Diss. Arizona State University, 2024. https://www.proquest.com/openview/559f33b22ba9742bd5365fb066a6e80f/1?cbl=18750&diss=y&pq-origsite=gscholar
- Yoppy, Yoppy, et al. "Improving the single current probe method of in-circuit impedance measurement." AIP Conference Proceedings. Vol. 3069. No. 1. AIP Publishing, 2024. https://doi.org/10.1063/5.0205694
- Dahms, Tayla, et al. "Measurement of Dielectric Properties of Thin Materials for Radomes Using Waveguide Cavities." Telecom. Vol. 5. No. 3. MDPI, 2024. https://doi.org/10.3390/telecom5030035
- Guerenneur, A., et al. "Equivalent circuit fitting method for microwave characterisation of low-k dielectric thin films." Measurement Science and Technology 35.10 (2024): 105602. https://doi.org/10.1088/1361-6501/ad6342
- Santos, Marlo Andrade. "Wireless system for passive surface acoustic wave sensors." (2024). http://dspace.sti.ufcg.edu.br:8080/jspui/handle/riufcg/39686

- Vazquez Aige, Marc. Electromagnetic study of two-dimensional materials with radar absorbing capabilities. MS thesis. Universitat Politècnica de Catalunya, 2024. https://upcommons.upc.edu/handle/2117/423342
- Takahashi, Hiroaki, et al. "Uncertainty Quantification and Variance-Based Sensitivity Analysis for Microstrip Line on IC-Substrate." 2024 International Conference on Broadband Communications for Next Generation Networks and Multimedia Applications (CoBCom). IEEE, 2024. https://doi.org/10.1109/CoBCom62281.2024.10631259
- Arsanjani, Arash, et al. "Modified semi-additive manufacturing of pcbs for enabling accurate device measurements at millimeter-wave and sub-terahertz frequencies." 2024 103rd ARFTG Microwave Measurement Conference (ARFTG). IEEE, 2024. https://doi.org/10.1109/ARFTG61196.2024.10660794
- Majed, Ismail, et al. "Cold-Termination Noise-Parameter Measurements at Cryogenic Temperatures." 2024 103rd ARFTG Microwave Measurement Conference (ARFTG). IEEE, 2024. https://doi.org/10.1109/ARFTG61196.2024.10661091
- Xenidis, Nikolaos, et al. "Dichroic absorption of aligned graphene-augmented inorganic nanofibers in the terahertz regime." Applied Materials Today 39 (2024): 102245. https://doi.org/10.1016/j.apmt.2024.102245
- Zohner, Alina, et al. "Performance Evaluation of a New Universal Time Domain Gating Algorithm." 2024 IEEE International Instrumentation and Measurement Technology Conference (I2MTC). IEEE, 2024. https://doi.org/10.1109/I2MTC60896.2024.10560830
- Krummenauer, Jan, et al. "Evaluating Deep Reinforcement Learning for Macromodel Synthesis." 2024 IEEE 28th Workshop on Signal and Power Integrity (SPI). IEEE, 2024.
- Dahms, Tayla, et al. "Measurement of Dielectric Properties of Thin Materials for Radomes Using Waveguide Cavities." Telecom. Vol. 5. No. 3. MDPI, 2024. https://doi.org/10.3390/telecom5030035
- Ferrand, Renaud, Victor Desangles, and Paul-Quentin Elias. "Non-intrusive measurements of plasma impedance in an electron-cyclotron resonance thruster." Physics of Plasmas 31.5 (2024). https://doi.org/10.1063/5.0191658
- J. W. Brooks, Matthew C. Paliwoda, Uncertainty analysis of the plasma impedance probe, 2024-05-01, https://doi.org/10.1063/5.0203675
- Yagmur Ozturk, Banaful Paul, Alebel Arage, et al., Characterization of Automotive Paint Permittivity for mmWave Radar Applications, 2024-04-01, https://doi.org/10.1109/tmtt.2023.3322928
- Laura Van Messem, Reinier Broucke, Hendrik Rogier, et al., Consistent and Cost-Effective PCB-Based D-Band Air-Filled Substrate-Integrated-Waveguide Filters, 2024-02-21, https://doi.org/10.1109/imfw59690.2024.10477120
- Shu-Ming Chang, Chelsea Swank, Andrew C. Kummel, et al., Free Space Dielectric Techniques for Diamond Composite Characterization, 2024-01-01, https://doi.org/10.1109/jmw.2023.3339255
- Yoppy Yoppy, R. Harry Arjadi, Yudhistira Yudhistira, et al., Improving the single current probe method of in-circuit impedance measurement, 2024-01-01, https://doi.org/10.1063/5.0205694
- Alex Arsenovic, A Spinor Model for Cascading Two-Port Networks in Conformal Geometric Algebra, 2024-01-01, https://doi.org/10.1007/978-3-031-55985-3_9
- Ziad Hatab, Michael Gadringer, Wolfgang Bsch, SymmetricReciprocalMatch Method for Vector Network Analyzer Calibration, 2024-01-01, https://doi.org/10.1109/tim.2024.3350124
- Nadeem Rather, Roy B. V. B. Simorangkir, John L. Buckley, et al., Deep Learning Assisted Robust Detection Techniques for a Chipless RFID Sensor Tag, 2024-01-01, https://doi.org/10.1109/tim.2023.3334378

6.4 2023

- A. Gascon Bravo, Salvador G. Garca, Alejandro Muoz Manterola, et al., Time Domain Simulation of Common Mode Ferrite Chokes at System Level, 2023-12-01, https://doi.org/10.1109/temc.2023.3309698
- Ilda Cahani, Marcus Stiemer, Mathematical optimization and machine learning to support PCB topology identification, 2023-12-01, https://doi.org/10.5194/ars-21-25-2023
- Ziad Hatab, Michael Gadringer, Wolfgang Bsch, Indirect Measurement of Switch Terms of a Vector Network Analyzer With Reciprocal Devices, 2023-11-01, https://doi.org/10.1109/lmwt.2023.3311032
- Kaisa Ryynnen, Vesa Lahtinen, Santeri Porrasmaa, et al., An Automated EM-Simulation Environment with Parameterized Layout Generation for Microwave Integrated Circuits, 2023-10-31, https://doi.org/10.1109/norcas58970.2023.10305468
- Sriram Ravula, Varun Gorti, Bo Deng, et al., One-Dimensional Deep Image Prior for Curve Fitting of S-Parameters from Electromagnetic Solvers, 2023-10-28, https://doi.org/10.1109/iccad57390.2023.10323916
- Laura Van Messem, Siddhartha Sinha, Ilja Ocket, et al., Air-filled SIW technology for mass-manufacturable and energy-efficient terahertz systems, 2023-10-04, https://doi.org/10.1038/s41598-023-43887-0
- Nadeem Rather, Roy B. V. B. Simorangkir, Cian ODonnell, et al., A Novel Robot Based Data Acquisition Methodology for Chipless RFID Systems, 2023-09-04, https://doi.org/10.1109/rfid-ta58140.2023.10290256
- Tuge Ayra, Anil zdemirli, Elif Apaydin, et al., Comparative Evaluation of Multiline TRL and 2X-Thru De-Embedding Implementation Methods on Printed Circuit Board Measurements, 2023-07-03, https://doi.org/10.1109/smacd58065.2023.10192140
- Vittorio Ugo Castrillo, Ivan Iudice, Domenico Pascarella, et al., A Real-Time Simulation Framework for Powerline Communications in More-Electric Aircraft and Spacecraft, 2023-06-19, https://doi.org/10.1109/metroaerospace57412.2023.10189965
- Barry van Jaarsveld, Sandra M. Hauswirth, Niko Wanders, Comment on hess-2022-430, 2023-06-05, https://doi.org/10.5194/hess-2022-430-rc2
- Shankkar Balasubramanian, Carsten Wulff, Trond Ytterdal, A 160-GHz Power Amplifier with 32-dB Gain and 9.8% Peak PAE in 28-nm FD-SOI, 2023-05-21, https://doi.org/10.1109/iscas46773.2023.10181360
- Ziad Hatab, Arezoo Abdi, Gregor Steinbauer, et al., Propagation Constant Measurement Based on a Single Transmission Line Standard Using a Two-Port VNA, 2023-05-07, https://doi.org/10.3390/s23094548
- Slawomir Simbierowicz, V. Yu. Monarkha, Merlin von Soosten, et al., Calibrated transmission and reflection from a multi-qubit microwave package, 2023-05-01, https://doi.org/10.1063/5.0144840
- Alessio Monti, S. Vellucci, Mirko Barbuto, et al., Line-Wave Waveguides: Design Procedure and Performance Comparison, 2023-03-26, https://doi.org/10.23919/eucap57121.2023.10133060
- Danny C. Price, Edward Tong, Adrian Sutinjo, et al., Measuring Noise Parameters Using an Open, Short, Load, and ?/8-Length Cable as Source Impedances, 2023-03-01, https://doi.org/10.1109/tmtt.2022.3225317
- Abraham Jos Hidalgo Snchez, Fabin Robledo Upegui, Una herramienta para el diseo de redes MSMN de banda ancha en lneas de transmisin basada en algoritmos heursticos de optimizacin comparados, 2023-02-24, https://doi.org/10.54139/revinguc.v29i2.171
- Barry van Jaarsveld, Sandra M. Hauswirth, Niko Wanders, Machine learning and Global Vegetation: Random Forests for Downscaling and Gapfilling, 2023-02-08, https://doi.org/10.5194/hess-2022-430
- Fabian Marxer, Antti Vepslinen, Shan W. Jolin, et al., Long-Distance Transmon Coupler with cz-Gate Fidelity above <mml:math xmlns:mml=http://www.w3.org/1998/Math/MathML display=inline overflow=scroll><mml:mn>99.8</mml:mn><mml:mi mathvariant=normal>%</mml:mi></mml:math>, 2023-02-06, https://doi.org/10.1103/prxquantum.4.010314

- Slawomir Simbierowicz, V. Yu. Monarkha, Merlin von Soosten, et al., Dataset for Calibrated transmission and reflection from a multi-qubit microwave package, 2023-02-01, https://doi.org/10.5281/zenodo.7593903
- Ivo Colmiais, Vtor Silva, Jrme Borme, et al., Extraction of Graphenes RF Impedance through Thru-Reflect-Line Calibration, 2023-01-14, https://doi.org/10.3390/mi14010215
- Keisuke Kawahara, Joe Sawada, Takumi Kamo, et al., Bandwidth Tripler: Broadband Signal Generation With an Image-Rejection Analog Multiplexer for Fiber Optic Transmitters, 2023-01-01, https://doi.org/10.1109/tmtt.2022.3190480
- Ziad Hatab, Michael Gadringer, Ahmad Bader Alothman Alterkawi, et al., Validation of the Reference Impedance in Multiline Calibration With Stepped Impedance Standards, 2023-01-01, https://doi.org/10.1109/ojim.2023.3315349
- Ziad Hatab, Michael Gadringer, Wolfgang Bsch, Propagation of Linear Uncertainties Through Multiline Thru-Reflect-Line Calibration, 2023-01-01, https://doi.org/10.1109/tim.2023.3296123
- Ziad Hatab, Michael Gadringer, Wolfgang Bsch, A Thru-Free Multiline Calibration, 2023-01-01, https://doi.org/10.1109/tim.2023.3308226

6.5 2022

- Jackson Anderson, Yanbo He, Bichoy Bahr, et al., Integrated acoustic resonators in commercial fin field-effect transistor technology, 2022-09-23, https://doi.org/10.1038/s41928-022-00827-6
- Daniel Gomez-Ibaez, Moustafa Elkolali, Ahmed Al-Tawil, et al., Design and Testing of a Low-Profile Pressure-Tolerant L-band Antenna, 2022-09-19, https://doi.org/10.1109/auv53081.2022.9965937
- Przemyslaw Bryndza, VNA measurement calibration in cryogenic environment, 2022-09-12, https://doi.org/10.23919/mikon54314.2022.9924695
- M. Amiri, Kevin Bandura, Anja Boskovic, et al., An Overview of CHIME, the Canadian Hydrogen Intensity Mapping Experiment, 2022-07-27, https://doi.org/10.3847/1538-4365/ac6fd9
- Mario Krattenmacher, Markus Mller, Pascal Kuthe, et al., DMT-core: A Python Toolkit for Semiconductor Device Engineers, 2022-07-04, https://doi.org/10.21105/joss.04298
- Miguel Sinusa Lozano, Laura Fernndez-Garca, D. Lpez-Romero, et al., SAW Resonators and Filters Based on Sc0.43Al0.57N on Single Crystal and Polycrystalline Diamond, 2022-06-30, https://doi.org/10.3390/mi13071061
- Giovanni Gugliandolo, K. Naishadham, Giovanni Crupi, et al., Design and Characterization of a Microwave Transducer for Gas Sensing Applications, 2022-03-29, https://doi.org/10.3390/chemosensors10040127
- Giovanni Gugliandolo, Zlatica Marinkovic, Giovanni Crupi, et al., Equivalent Circuit Model Extraction for a SAW Resonator: Below and above Room Temperature, 2022-03-26, https://doi.org/10.3390/s22072546
- Pedro Alberto Segura Chavez, Jrmy Bonhomme, Mohamed Lamine Fayal Bellaredj, et al., Love Wave Sensor with High Penetration Depth for Potential Application in Cell Monitoring, 2022-01-24, https://doi.org/10.3390/bios12020061
- Alex Arsenovic, J. Hillairet, Jackson Anderson, et al., scikit-rf: An Open Source Python Package for Microwave Network Creation, Analysis, and Calibration [Speakers Corner], 2022-01-01, https://doi.org/10.1109/mmm.2021.3117139
- Andreas Depold, Stefan Erhardt, Robert Weigel, et al., A 10 kHz to 6 GHz Low-Cost Vector Network Analyzer, 2021-12-17, https://doi.org/10.5194/ars-19-17-2021

6.6 2021

- J. Hillairet, P. Mollard, L. Colas, et al., WEST actively cooled load resilient ion cyclotron resonance heating system results, 2021-08-12, https://doi.org/10.1088/1741-4326/ac1759
- James Campion, Joachim Oberhammer, Silicon Micromachined Waveguide Calibration Standards for Terahertz Metrology, 2021-08-01, https://doi.org/10.1109/tmtt.2021.3091720

6.7 2020

- J. Hillairet, RF network analysis of the WEST ICRH antenna with the open-source python scikit-RF package, 2020-01-01, https://doi.org/10.1063/5.0013523
- Jing Zhang, Lingdong Chen, Biao Shen, et al., Ultra-high strength poly(N-(2-hydroxyethyl)acrylamide)/chitosan hydrogel with repelling and killing bacteria property, 2019-12-01, https://doi.org/10.1016/j.carbpol.2019.115160

6.8 2019

- Mohd Nazrin Md Isa, Shaiful Jahari Hashim, Fakhrul Zaman Rokhani, et al., Error correction and uncertainty measurement of short-open-load calibration standards on a new concept of software defined instrumentation for microwave network analysis, 2019-09-30, https://doi.org/10.21595/jme.2019.20329
- Thomas Blaschke, Filip Miljkovic, Jrgen Bajorath, Prediction of Different Classes of Promiscuous and Non-promiscuous Compounds Using Machine Learning and Nearest Neighbor Analysis, 2019-04-16, https://doi.org/10.1021/acsomega.9b00492
- Julian Koch, Simon Stisen, Jens Christian Refsgaard, et al., Modeling Depth of the Redox Interface at High Resolution at National Scale Using Random Forest and Residual Gaussian Simulation, 2019-02-01, https://doi.org/10.1029/2018wr023939
- Giovanna Nicora, Simone Marini, Ivan Limongelli, et al., A Semi-supervised Learning Approach for Pan-Cancer Somatic Genomic Variant Classification, 2019-01-01, https://doi.org/10.1007/978-3-030-21642-9_7
- Xinhua Gao, A Machine-learning-based Investigation of the Open Cluster M67, 2018-12-05, https://doi.org/10.3847/1538-4357/aae8dd

6.9 2018

- Ahmed Elmogi, Hannes Ramon, Joris Lambrecht, et al., Aerosol-Jet Printed Interconnects for 60-Gb/s CMOS Driver and Microring Modulator Transmitter Assembly, 2018-11-15, https://doi.org/10.1109/lpt.2018.2873056
- Sjoerd Op 'T Land, Mohamed Ramdani, Richard Perdriau, Dominant Coupling Mechanism for Integrated Circuit Immunity of SOIC Packages Up To 10 GHz, 2018-08-01, https://doi.org/10.1109/temc.2017.2756915
- J. Redford, Charles M. Bradford, Steven Hailey-Dunsheath, et al., The design and characterization of a 300 channel, optimized full-band millimeter filterbank for science with SuperSpec, 2018-07-09, https://doi.org/10.1117/12.2313666
- Viktor Doychinov, Christopher Russell, Nutapong Somjit, et al., Investigation of Implantable Antennas for Exploratory Neuroscience Studies, 2018-01-01, https://doi.org/10.1049/cp.2018.1445
- Matthew F. Bauwens, Micromachined On-Water Probes for Characterization of Terahertz Devices and Circuits, 2017-08-09, https://doi.org/10.18130/v3wj99

6.10 2012-2018

- Sjoerd Op 'T Land, Mohamed Ramdani, Richard Perdriau, et al., Field-to-Long-Segmented-Trace Coupling With Arbitrary Loads and a Transparent Upper Bound Using a Single Modified Taylor Cell, 2016-10-01, https://doi.org/10.1109/temc.2016.2566449
- Sean Bryan, George Che, Christopher Groppi, et al., A Compact Filter-Bank Waveguide Spectrometer for Millimeter Wavelengths, 2015-07-01, https://doi.org/10.1109/tthz.2015.2433919
- Souheil Nadri, R. Percy, Lin Kittiwatanakul, et al., Terahertz coded aperture mask using vanadium dioxide bowtie antenna array, 2014-09-05, https://doi.org/10.1117/12.2060899
- Alex Arsenovic, Lihan Chen, Matthew F. Bauwens, et al., An Experimental Technique for Calibration Uncertainty Analysis, 2013-01-01, https://doi.org/10.1109/tmtt.2012.2222908
- Sjoerd Op 'T Land, Richard Perdriau, Mohamed Ramdani, et al., Design of a 20 GHz DPI method for SOIC8, 2012-09-01, https://doi.org/10.1109/emceurope.2012.6396691

CHAPTER

SEVEN

GLOSSARY

switch terms

Described in[1] by Roger Marks, switch-terms account for the fact that the 8-term (aka *error-box*) model is overly simplified. The two error networks change slightly depending on which port is being excited. This is due to the internal switch within the VNA.

port impedance

The impedance of a given port on a microwave Network. Frequently this is the same as the characteristic impedance of a given transmission line mode, but may be arbitrarily set. terminating a port with any impedance other than its port impedance will cause reflections.

characteristic impedance

Ratio of voltage to current, for a single traveling wave along a transmission line.

[1] R.B. Marks, "Formulations of the Basic Vector Network Analyzer Error Model including Switch-Terms," in ARFTG Conference Digest-Fall, 50th, 1997, vol. 32, pp. 115-126.

CHAPTER

EIGHT

LICENSE

Copyright (c) 2010, Alexander Arsenovic All rights reserved.

Copyright (c) 2017, scikit-rf Developers All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the scikit-rf nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

BIBLIOGRAPHY

[QUCSa] http://qucs.sourceforge.net/docs/technical.pdf

[QUCSb] http://www.qucs.sourceforge.net/

[DBLS01] Djordjevic, R.M. Biljic, V.D. Likar-Smiljanic, T.K. Sarkar, Wideband frequency-domain characterization of FR-4 and time-domain causality, IEEE Trans. on EMC, vol. 43, N4, 2001, p. 662-667.

[SvDe01] C. Svensson, G.E. Dermer, Time domain modeling of lossy interconnects, IEEE Trans. on Advanced Packaging, May 2001, N2, Vol. 24, pp.191-196.

[GhNa84] G. Ghione and C. Naldi. "Analytical Formulas for Coplanar Lines in Hybrid and Monolithic MICs", Electronics Letters, Vol. 20, No. 4, February 16, 1984, pp. 179-181.

[GhNa83] G. Ghione and C. Naldi. "Parameters of Coplanar Waveguides with Lower Common Planes", Electronics Letters, Vol. 19, No. 18, September 1, 1983, pp. 734-735.

[Cohn60] S. B. Cohn, "Thickness Corrections for Capacitive obstacles and Strip Conductors", IRE Trans. on Microwave Theory and Techniques, Vol. MTT-8, November 1960, pp. 638-644.

[GGBB96] K. C. Gupta, R. Garg, I. J. Bahl, and P. Bhartia, Microstrip Lines and Slotlines, 2nd ed.Artech House, Inc., 1996.

[FGVM91] M. Y. Frankel, S. Gupta, J. A. Valdmanis, and G. A. Mourou, "Terahertz Attenuation and Dispersion Characteristics of Coplanar Transmission Lines" IEEE Trans. on Microwave Theory and Techniques, vol. 39, no. 6, pp. 910-916, June 1991.

[GMDK97] S. Gevorgian, T. Martinsson, A. Deleniv, E. Kollberg, and I. Vendik, "Simple and accurate dispersion expression for the effective dielectric constant of coplanar waveguides" in Proceedings of Microwaves, Antennas and Propagation, vol. 144, no. 2.IEE, Apr. 1997, pp. 145-148.

[Whee42] H. A. Wheeler, "Formulas for the Skin Effect," Proceedings of the IRE, vol. 30, no. 9, pp. 412-424, Sept. 1942.

[OwWu58] G. H. Owyang and T. T. Wu, "The Approximate Parameters of Slot Lines and Their Complement" IRE Transactions on Antennas and Propagation, pp. 49-55, Jan. 1958.

[Ghio93] G. Ghione, "A CAD-Oriented Analytical Model for the Losses of General Asymmetric Coplanar Lines in Hybrid and Monolithic MICs" IEEE Trans. on Microwave Theory and Techniques, vol. 41, no. 9, pp. 1499-1510, Sept. 1993.

[GhNa84] G. Ghione and C. Naldi. "Analytical Formulas for Coplanar Lines in Hybrid and Monolithic MICs", Electronics Letters, Vol. 20, No. 4, February 16, 1984, pp. 179-181.

[GhNa83] G. Ghione and C. Naldi. "Parameters of Coplanar Waveguides with Lower Common Planes", Electronics Letters, Vol. 19, No. 18, September 1, 1983, pp. 734-735.

[Cohn60] S. B. Cohn, "Thickness Corrections for Capacitive obstacles and Strip Conductors", IRE Trans. on Microwave Theory and Techniques, Vol. MTT-8, November 1960, pp. 638-644.

[GGBB96] K. C. Gupta, R. Garg, I. J. Bahl, and P. Bhartia, Microstrip Lines and Slotlines, 2nd ed.Artech House, Inc., 1996.

[VF] : https://www.microwaves101.com/encyclopedias/light-phase-and-group-velocities

[QUCS_tech] http://qucs.sourceforge.net/docs/technical.pdf

[QUCS_repo] https://github.com/Qucs/qucsator/blob/develop/src/components/microstrip/msline.cpp

[Hammerstad] E. Hammerstad and Ø. Jensen, "Accurate Models for Microstrip Computer-Aided Design", Symposium on Microwave Theory and Techniques, pp. 407-409, June 1980.

[Kirschning] M. Kirschning and R. H. Jansen, "Accurate Model for Effective Dielectric Constant of Microstrip with Validity up to Millimeter-Wave Frequencies", Electronics Letters, vol. 8, no. 6, pp. 272-273, Mar. 1982.

[Jansen] R. H. Jansen and M. Kirschning, "Arguments and an accurate Model for the Power-Current Formulation of Microstrip Characteristic Impedance", Archiv für Elektronik und Übertragungstechnik (AEÜ), vol. 37, pp. 108-112, 1983.

[Kobayashi] M. Kobayashi, "A Dispersion Formula Satisfying Recent Requirements in Microstrip CAD", IEEE Trans. on Microwave Theory and Techniques, vol. 36, no. 8, pp. 1246-1250, Aug. 1988.

[Schneider1969] M. V. Schneider, "Microstrip Lines for Microwave Integrated Circuits", The Bell System Technical Journal, vol. 48, pp. 1421-1444, May 1969.

[Schneider1972] M. V. Schneider, "Microstrip Dispersion", Proceedings of the IEEE, Letters, vol. 60, Jan. 1972, pp. 144-146.

[Wheeler1977] H. A. Wheeler, "Transmission-Line Properties of a Strip on a Dielectric Sheet on a Plane, IEEE Trans. on Microwave Theory and Techniques, vol. 25, no. 8, pp. 631-647, Aug. 1977.

[Wheeler1942] H. A. Wheeler, "Formulas for the Skin Effect," Proceedings of the IRE, vol. 30, no. 9, pp. 412-424, Sept. 1942.

[Yamashita] E. Yamashita, K. Atsuki, and T. Ueda, "An Approximate Dispersion Formula of Microstrip Lines for Computer Aided Design of Microwave Integrated Circuits", IEEE Trans. on Microwave Theory and Techniques, vol. 27, pp. 1036-1038, Dec. 1979.

[Svensson] C. Svensson, G.E. Dermer, Time domain modeling of lossy interconnects, IEEE Trans. on Advanced Packaging, May 2001, N2, Vol. 24, pp.191-196.

[Djordjevic] Djordjevic, R.M. Biljic, V.D. Likar-Smiljanic, T.K. Sarkar, Wideband frequency-domain characterization of FR-4 and time-domain causality, IEEE Trans. on EMC, vol. 43, N4, 2001, p. 662-667.

[ElSA20] Ellison J, Smith SB, Agili S., "Using a 2x-thru standard to achieve accurate de-embedding of measurements", Microwave Optical Technology Letter, 2020, https://doi.org/10.1002/mop.32098

[I3E3701] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG1/IEEEP3702xThru.m, commit 49ddd78cf68ad5a7c0aaa57a73415075b5178aa6

[I3E3702] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG1/IEEEP370mmZc2xthru.m commit 49ddd78cf68ad5a7c0aaa57a73415075b5178aa6

[I3E3703] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG1/IEEEP370Zc2xThru.m commit 49ddd78cf68ad5a7c0aaa57a73415075b5178aa6

[I3E3704] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG1/IEEEP370mmZc2xthru.m commit 49ddd78cf68ad5a7c0aaa57a73415075b5178aa6

[I3E3705] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG3/qualityCheckFrequencyDomain.m commit 8b8f3a3b5e41aeb4ab16110bbfb683ec52e70206

[ElSA20] Ellison J, Smith SB, Agili S., "Using a 2x-thru standard to achieve accurate de-embedding of measurements", Microwave Optical Technology Letter, 2020, https://doi.org/10.1002/mop.32098

[I3E370] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG1/IEEEP3702xThru.m, commit 49ddd78cf68ad5a7c0aaa57a73415075b5178aa6

[ElSA20] Ellison J, Smith SB, Agili S., "Using a 2x-thru standard to achieve accurate de-embedding of measurements", Microwave Optical Technology Letter, 2020, https://doi.org/10.1002/mop.32098

[I3E370] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG1/IEEEP370mmZc2xthru.m commit 49ddd78cf68ad5a7c0aaa57a73415075b5178aa6

[I3E370] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG1/IEEEP370Zc2xThru.m commit 49ddd78cf68ad5a7c0aaa57a73415075b5178aa6

[I3E370] https://opensource.ieee.org/elec-char/ieee-370/-/blob/master/TG1/IEEEP370mmZc2xthru.m commit 49ddd78cf68ad5a7c0aaa57a73415075b5178aa6

[MAT58] "Q-factor Measurement by using a Vector Network Analyser", A. P. Gregory, National Physical Laboratory Report MAT 58 (2021) https://eprintspublications.npl.co.uk/9304/, section 2.5, eqn. (31).

[MAT58] "Q-factor Measurement by using a Vector Network Analyser", A. P. Gregory, National Physical Laboratory Report MAT 58 (2021) https://eprintspublications.npl.co.uk/9304/

[MAT58] "Q-factor Measurement by using a Vector Network Analyser", A. P. Gregory, National Physical Laboratory Report MAT 58 (2021) https://eprintspublications.npl.co.uk/9304/ section 2.4, eqn. (28).

[MAT58] "Q-factor Measurement by using a Vector Network Analyser", A. P. Gregory, National Physical Laboratory Report MAT 58 (2021) https://eprintspublications.npl.co.uk/9304/

[MAT58] "Q-factor Measurement by using a Vector Network Analyser", A. P. Gregory, National Physical Laboratory Report MAT 58 (2021) https://eprintspublications.npl.co.uk/9304/, section 2.2, eqn. (21).

[MAT58] "Q-factor Measurement by using a Vector Network Analyser", A. P. Gregory, National Physical Laboratory Report MAT 58 (2021) https://eprintspublications.npl.co.uk/9304/

[Pozar] D. M. Pozar, Microwave engineering, 4th ed. J. Wiley, 2012.

[Galwas] B. A. Galwas, 'Scattering Matrix Description of Microwave Resonators', IEEE Trans. Microwave Theory Techn., vol. 31, no. 8, pp. 669–671, Aug. 1983, doi: 10.1109/TMTT.1983.1131566.

PYTHON MODULE INDEX

S

skrf.calibration.calibration, 1148
skrf.calibration.deembedding, 1394
skrf.circuit, 1485
skrf.constants, 1547
skrf.data, 1549
skrf.frequency, 836
skrf.instances, 1546
skrf.io.csv, 1586
skrf.io.general, 1575
skrf.io.touchstone, 1569
skrf.mathFunctions, 1514
skrf.media, 881
skrf.media.device, 1508
skrf.network, 633
skrf.networkSet, 846
skrf.plotting, 868
skrf.qfactor, 1474
skrf.taper, 1136
skrf.time, 1481
skrf.tlineFunctions, 1531
skrf.util, 1558
skrf.vi, 1443
skrf.vi.vna, 1452

INDEX

Symbols

__init__() (*skrf.calibration.calibration.Calibration method*), 1155
__init__() (*skrf.calibration.calibration.EightTerm method*), 1228
__init__() (*skrf.calibration.calibration.EnhancedResponse method*), 1385
__init__() (*skrf.calibration.calibration.LMR16 method*), 1348
__init__() (*skrf.calibration.calibration.LRM method*), 1251
__init__() (*skrf.calibration.calibration.LRRM method*), 1262
__init__() (*skrf.calibration.calibration.MRC method*), 1274
__init__() (*skrf.calibration.calibration.MultilineTRL method*), 1297
__init__() (*skrf.calibration.calibration.MultiportCal method*), 1364
__init__() (*skrf.calibration.calibration.MultiportSOLT method*), 1367
__init__() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1310
__init__() (*skrf.calibration.calibration.Normalization method*), 1358
__init__() (*skrf.calibration.calibration.OnePort method*), 1166
__init__() (*skrf.calibration.calibration.PHN method*), 1197
__init__() (*skrf.calibration.calibration.SDDL method*), 1176
__init__() (*skrf.calibration.calibration.SDDLWeikle method*), 1187
__init__() (*skrf.calibration.calibration.SOLT method*), 1217
__init__() (*skrf.calibration.calibration.SixteenTerm method*), 1336
__init__() (*skrf.calibration.calibration.TRL method*), 1285
__init__() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1324
__init__() (*skrf.calibration.calibration.TwelveTerm method*), 1207
__init__() (*skrf.calibration.calibration.TwoPortOnePath method*), 1375
__init__() (*skrf.calibration.calibration.UnknownThru method*), 1239
__init__() (*skrf.calibration.deembedding.AdmittanceCancel method*), 1405
__init__() (*skrf.calibration.deembedding.Deembedding method*), 1395
__init__() (*skrf.calibration.deembedding.IEEEP370 method*), 1408
__init__() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru method*), 1422
__init__() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru method*), 1439
__init__() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru method*), 1415
__init__() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru method*), 1430
__init__() (*skrf.calibration.deembedding.ImpedanceCancel method*), 1406
__init__() (*skrf.calibration.deembedding.Open method*), 1398
__init__() (*skrf.calibration.deembedding.OpenShort method*), 1397
__init__() (*skrf.calibration.deembedding.Short method*), 1401
__init__() (*skrf.calibration.deembedding.ShortOpen method*), 1399
__init__() (*skrf.calibration.deembedding.SplitPi method*), 1402
__init__() (*skrf.calibration.deembedding.SplitTee method*), 1403
__init__() (*skrf.circuit.Circuit method*), 1496
__init__() (*skrf.frequency.Frequency method*), 842
__init__() (*skrf.io.csv.AgilentCSV method*), 1588
__init__() (*skrf.io.general.TouchstoneEncoder method*), 1584
__init__() (*skrf.io.touchstone.Touchstone method*), 1570
__init__() (*skrf.media.circularWaveguide.CircularWaveguide method*), 987

__init__() (*skrf.media.coaxial.Coaxial method*), 1040
__init__() (*skrf.media.cpw.CPW method*), 1011
__init__() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1118
__init__() (*skrf.media.device.Device method*), 1509
__init__() (*skrf.media.device.DualCoupler method*), 1514
__init__() (*skrf.media.device.Hybrid method*), 1511
__init__() (*skrf.media.device.Hybrid180 method*), 1513
__init__() (*skrf.media.device.MatchedSymmetricCoupler method*), 1510
__init__() (*skrf.media.device.QuadratureHybrid method*), 1512
__init__() (*skrf.media.distributedCircuit.DistributedCircuit method*), 933
__init__() (*skrf.media.freespace.Freespace method*), 1093
__init__() (*skrf.media.media.DefinedGammaZ0 method*), 909
__init__() (*skrf.media.media.Media method*), 886
__init__() (*skrf.media.mline.MLine method*), 1066
__init__() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 961
__init__() (*skrf.network.Network method*), 680
__init__() (*skrf.networkSet.NetworkSet method*), 851
__init__() (*skrf.qfactor.Qfactor method*), 1478
__init__() (*skrf.taper.Exponential method*), 1144
__init__() (*skrf.taper.Klopfenstein method*), 1148
__init__() (*skrf.taper.Linear method*), 1141
__init__() (*skrf.taper.SmoothStep method*), 1146
__init__() (*skrf.taper.Taper1D method*), 1138
__init__() (*skrf.util.HomoDict method*), 1567
__init__() (*skrf.util.HomoList method*), 1566
__init__() (*skrf.util.ProgressBar method*), 1560
__init__() (*skrf.vectorFitting.VectorFitting method*), 1459
__init__() (*skrf.vectorFitting.VectorFittingParametric method*), 1472
__init__() (*skrf.vi.validators.BooleanValidator method*), 1451
__init__() (*skrf.vi.validators.DelimitedStrValidator method*), 1450
__init__() (*skrf.vi.validators.DictValidator method*), 1449
__init__() (*skrf.vi.validators.EnumValidator method*), 1448
__init__() (*skrf.vi.validators.FloatValidator method*), 1446
__init__() (*skrf.vi.validators.FreqValidator method*), 1447
__init__() (*skrf.vi.validators.IntValidator method*), 1445
__init__() (*skrf.vi.validators.SetValidator method*), 1448

A

a (*skrf.media.coaxial.Coaxial property*), 1036
a (*skrf.network.Network property*), 638
a_arcl (*skrf.network.Network property*), 639
a_arcl_unwrap (*skrf.network.Network property*), 639
a_db (*skrf.network.Network property*), 639
a_db10 (*skrf.network.Network property*), 639
a_deg (*skrf.network.Network property*), 639
a_deg_unwrap (*skrf.network.Network property*), 640
a_im (*skrf.network.Network property*), 640
a_mag (*skrf.network.Network property*), 640
a_rad (*skrf.network.Network property*), 640
a_rad_unwrap (*skrf.network.Network property*), 640
a_re (*skrf.network.Network property*), 641
a_time (*skrf.network.Network property*), 641
a_time_db (*skrf.network.Network property*), 641
a_time_impulse (*skrf.network.Network property*), 641
a_time_mag (*skrf.network.Network property*), 641
a_time_step (*skrf.network.Network property*), 642
a_vswr (*skrf.network.Network property*), 642
add_dc() (*skrf.calibration.deembedding.IEEEP370 static method*), 1409
add_dc() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1423
add_dc() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1440
add_dc() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1415
add_dc() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1431
add_markers_to_lines() (*in module skrf.plotting*), 879
add_noise_polar() (*skrf.network.Network method*), 682
add_noise_polar_flatband() (*skrf.network.Network method*), 682
add_polar_noise() (*skrf.networkSet.NetworkSet method*), 851
AdmittanceCancel (*class in skrf.calibration.deembedding*), 1404
AgilentCSV (*class in skrf.io.csv*), 1586
align_measured_ideals() (*in module skrf.calibration.calibration*), 1393
alpha (*skrf.media.circularWaveguide.CircularWaveguide property*), 981
alpha (*skrf.media.coaxial.Coaxial property*), 1036
alpha (*skrf.media.cpw.CPW property*), 1008
alpha (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1114
alpha (*skrf.media.distributedCircuit.DistributedCircuit property*), 929
alpha (*skrf.media.freespace.Freespace property*), 1088

alpha (*skrf.media.media.DefinedGammaZ0 property*), 905
alpha (*skrf.media.media.Media property*), 883
alpha (*skrf.media.mline.MLine property*), 1063
alpha (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 955
alpha_c (*skrf.media.circularWaveguide.CircularWaveguide property*), 981
alpha_c (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 955
alpha_conductor (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1114
alpha_dielectric (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1114
analyse_dielectric() (*skrf.media.cpw.CPW method*), 1012
analyse_dielectric() (*skrf.media.mline.MLine method*), 1067
analyse_dispersion() (*skrf.media.cpw.CPW method*), 1012
analyse_dispersion() (*skrf.media.mline.MLine method*), 1067
analyse_loss() (*skrf.media.cpw.CPW method*), 1013
analyse_loss() (*skrf.media.mline.MLine method*), 1068
analyse_quasi_static() (*skrf.media.cpw.CPW method*), 1013
analyse_quasi_static() (*skrf.media.mline.MLine method*), 1069
angular_weights() (*skrf.qfactor.Qfactor static method*), 1478
animate() (*skrf.networkSet.NetworkSet method*), 851
animate() (*skrf.util.ProgressBar method*), 1560
apply_cal() (*skrf.calibration.calibration.Calibration method*), 1155
apply_cal() (*skrf.calibration.calibration.EightTerm method*), 1228
apply_cal() (*skrf.calibration.calibration.EnhancedResponse method*), 1386
apply_cal() (*skrf.calibration.calibration.LMR16 method*), 1348
apply_cal() (*skrf.calibration.calibration.LRM method*), 1251
apply_cal() (*skrf.calibration.calibration.LRRM method*), 1263
apply_cal() (*skrf.calibration.calibration.MRC method*), 1274
apply_cal() (*skrf.calibration.calibration.MultilineTRL method*), 1298
apply_cal() (*skrf.calibration.calibration.MultiportCal method*), 1364
apply_cal() (*skrf.calibration.calibration.MultiportSOLT method*), 1367
apply_cal() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1312
apply_cal() (*skrf.calibration.calibration.Normalization method*), 1359
apply_cal() (*skrf.calibration.calibration.OnePort method*), 1166
apply_cal() (*skrf.calibration.calibration.PHN method*), 1197
apply_cal() (*skrf.calibration.calibration.SDDL method*), 1176
apply_cal() (*skrf.calibration.calibration.SDDLWeikle method*), 1187
apply_cal() (*skrf.calibration.calibration.SixteenTerm method*), 1336
apply_cal() (*skrf.calibration.calibration.SOLT method*), 1218
apply_cal() (*skrf.calibration.calibration.TRL method*), 1286
apply_cal() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1325
apply_cal() (*skrf.calibration.calibration.TwelveTerm method*), 1208
apply_cal() (*skrf.calibration.calibration.TwoPortOnePath method*), 1375
apply_cal() (*skrf.calibration.calibration.UnknownThru method*), 1240
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.Calibration method*), 1155
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.EightTerm method*), 1229
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.EnhancedResponse method*), 1386
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.LMR16 method*), 1348
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.LRM method*), 1251
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.LRRM method*), 1263
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.MRC method*), 1274
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.MultilineTRL method*), 1298
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1312
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.Normalization*

method), 1359
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.OnePort method*), 1166
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.PHN method*), 1197
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.SDDL method*), 1177
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.SDDLWeikle method*), 1187
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.SixteenTerm method*), 1337
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.SOLT method*), 1218
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.TRL method*), 1286
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1325
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.TwelveTerm method*), 1208
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.TwoPortOnePath method*), 1376
apply_cal_to_all_in_dir() (*skrf.calibration.calibration.UnknownThru method*), 1240
apply_cal_to_list() (*skrf.calibration.calibration.Calibration method*), 1155
apply_cal_to_list() (*skrf.calibration.calibration.EightTerm method*), 1229
apply_cal_to_list() (*skrf.calibration.calibration.EnhancedResponse method*), 1386
apply_cal_to_list() (*skrf.calibration.calibration.LMR16 method*), 1349
apply_cal_to_list() (*skrf.calibration.calibration.LRM method*), 1251
apply_cal_to_list() (*skrf.calibration.calibration.LRRM method*), 1263
apply_cal_to_list() (*skrf.calibration.calibration.MRC method*), 1274
apply_cal_to_list() (*skrf.calibration.calibration.MultilineTRL method*), 1299
apply_cal_to_list() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1312
apply_cal_to_list() (*skrf.calibration.calibration.Normalization method*), 1359
apply_cal_to_list() (*skrf.calibration.calibration.OnePort method*), 1166
apply_cal_to_list() (*skrf.calibration.calibration.PHN method*), 1198
apply_cal_to_list() (*skrf.calibration.calibration.SDDL method*), 1177
apply_cal_to_list() (*skrf.calibration.calibration.SDDLWeikle method*), 1187
apply_cal_to_list() (*skrf.calibration.calibration.SixteenTerm method*), 1337
apply_cal_to_list() (*skrf.calibration.calibration.SOLT method*), 1218
apply_cal_to_list() (*skrf.calibration.calibration.TRL method*), 1287
apply_cal_to_list() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1326
apply_cal_to_list() (*skrf.calibration.calibration.TwelveTerm method*), 1208
apply_cal_to_list() (*skrf.calibration.calibration.TwoPortOnePath method*), 1376
apply_cal_to_list() (*skrf.calibration.calibration.UnknownThru method*), 1240
apply_cal_to_network_set() (*skrf.calibration.calibration.Calibration method*), 1156
apply_cal_to_network_set() (*skrf.calibration.calibration.EightTerm method*), 1229
apply_cal_to_network_set() (*skrf.calibration.calibration.EnhancedResponse method*), 1386
apply_cal_to_network_set() (*skrf.calibration.calibration.LMR16 method*),

1349
apply_cal_to_network_set() (*skrf.calibration.calibration.LRM method*), 1251
apply_cal_to_network_set() (*skrf.calibration.calibration.LRRM method*), 1263
apply_cal_to_network_set() (*skrf.calibration.calibration.MRC method*), 1274
apply_cal_to_network_set() (*skrf.calibration.calibration.MultilineTRL method*), 1299
apply_cal_to_network_set() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1312
apply_cal_to_network_set() (*skrf.calibration.calibration.Normalization method*), 1359
apply_cal_to_network_set() (*skrf.calibration.calibration.OnePort method*), 1166
apply_cal_to_network_set() (*skrf.calibration.calibration.PHN method*), 1198
apply_cal_to_network_set() (*skrf.calibration.calibration.SDDL method*), 1177
apply_cal_to_network_set() (*skrf.calibration.calibration.SDDLWeikle method*), 1187
apply_cal_to_network_set() (*skrf.calibration.calibration.SixteenTerm method*), 1337
apply_cal_to_network_set() (*skrf.calibration.calibration.SOLT method*), 1218
apply_cal_to_network_set() (*skrf.calibration.calibration.TRL method*), 1287
apply_cal_to_network_set() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1326
apply_cal_to_network_set() (*skrf.calibration.calibration.TwelveTerm method*), 1208
apply_cal_to_network_set() (*skrf.calibration.calibration.TwoPortOnePath method*), 1376
apply_cal_to_network_set() (*skrf.calibration.calibration.UnknownThru method*), 1240
assert_frequency_equal() (*in module skrf.network*), 835
assert_frequency_exist() (*in module skrf.network*), 835
assert_nports_equal() (*in module skrf.network*), 835
assert_z0_at_ports_equal() (*in module skrf.network*), 836
assert_z0_equal() (*in module skrf.network*), 836
attenuator() (*skrf.media.circularWaveguide.CircularWaveguide method*), 988
attenuator() (*skrf.media.coaxial.Coaxial method*), 1040
attenuator() (*skrf.media.cpw.CPW method*), 1014
attenuator() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1118
attenuator() (*skrf.media.distributedCircuit.DistributedCircuit method*), 934
attenuator() (*skrf.media.freespace.Freespace method*), 1093
attenuator() (*skrf.media.media.DefinedGammaZ0 method*), 909
attenuator() (*skrf.media.media.Media method*), 886
attenuator() (*skrf.media.mline.MLine method*), 1069
attenuator() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 962
attribute() (*skrf.network.Network method*), 682
auto_fit() (*skrf.vectorFitting.VectorFitting method*), 1459
auto_fit() (*skrf.vectorFitting.VectorFittingParametric method*), 1473
average() (*in module skrf.network*), 830

B

b (*skrf.media.coaxial.Coaxial property*), 1036
basename_noext() (*in module skrf.util*), 1563
beta (*skrf.media.circularWaveguide.CircularWaveguide property*), 982
beta (*skrf.media.coaxial.Coaxial property*), 1036
beta (*skrf.media.cpw.CPW property*), 1008
beta (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1114
beta (*skrf.media.distributedCircuit.DistributedCircuit property*), 929
beta (*skrf.media.freespace.Freespace property*), 1088
beta (*skrf.media.media.DefinedGammaZ0 property*), 905
beta (*skrf.media.media.Media property*), 883
beta (*skrf.media.mline.MLine property*), 1063
beta (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 955
beta_phase (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1114
biased_error (*skrf.calibration.calibration.Calibration property*), 1149
biased_error (*skrf.calibration.calibration.EightTerm property*), 1222

biased_error (*skrf.calibration.calibration.EnhancedResponse property*), 1380
biased_error (*skrf.calibration.calibration.LMR16 property*), 1342
biased_error (*skrf.calibration.calibration.LRM property*), 1245
biased_error (*skrf.calibration.calibration.LRRM property*), 1257
biased_error (*skrf.calibration.calibration.MRC property*), 1268
biased_error (*skrf.calibration.calibration.MultilineTRL property*), 1291
biased_error (*skrf.calibration.calibration.NISTMultilineTRL property*), 1304
biased_error (*skrf.calibration.calibration.Normalization property*), 1353
biased_error (*skrf.calibration.calibration.OnePort property*), 1161
biased_error (*skrf.calibration.calibration.PHN property*), 1192
biased_error (*skrf.calibration.calibration.SDDL property*), 1171
biased_error (*skrf.calibration.calibration.SDDLWeikle property*), 1181
biased_error (*skrf.calibration.calibration.SixteenTerm property*), 1331
biased_error (*skrf.calibration.calibration.SOLT property*), 1212
biased_error (*skrf.calibration.calibration.TRL property*), 1279
biased_error (*skrf.calibration.calibration.TUGMultilineTRL property*), 1318
biased_error (*skrf.calibration.calibration.TwelveTerm property*), 1202
biased_error (*skrf.calibration.calibration.TwoPortOnePath property*), 1370
biased_error (*skrf.calibration.calibration.UnknownThru property*), 1234
BooleanValidator (*class in skrf.vi.validators*), 1451
BW (*skrf.qfactor.Qfactor property*), 1475
BW_scaled (*skrf.qfactor.Qfactor property*), 1476

C

c (*in module skrf.constants*), 1547
C (*skrf.circuit.Circuit property*), 1487
C (*skrf.media.coaxial.Coaxial property*), 1034
c (*skrf.media.device.Hybrid property*), 1511
c (*skrf.media.device.MatchedSymmetricCoupler property*), 1510
c (*skrf.media.device.QuadratureHybrid property*), 1512
C_F (*skrf.circuit.Circuit property*), 1487
CACHEDPROPERTIES (*skrf.circuit.Circuit attribute*), 1487
caled_ntwk_sets (*skrf.calibration.calibration.Calibration property*), 1150
caled_ntwk_sets (*skrf.calibration.calibration.EightTerm property*), 1223
caled_ntwk_sets (*skrf.calibration.calibration.EnhancedResponse property*), 1380
caled_ntwk_sets (*skrf.calibration.calibration.LMR16 property*), 1342
caled_ntwk_sets (*skrf.calibration.calibration.LRM property*), 1245
caled_ntwk_sets (*skrf.calibration.calibration.LRRM property*), 1257
caled_ntwk_sets (*skrf.calibration.calibration.MRC property*), 1269
caled_ntwk_sets (*skrf.calibration.calibration.MultilineTRL property*), 1292
caled_ntwk_sets (*skrf.calibration.calibration.NISTMultilineTRL property*), 1304
caled_ntwk_sets (*skrf.calibration.calibration.Normalization property*), 1354
caled_ntwk_sets (*skrf.calibration.calibration.OnePort property*), 1161
caled_ntwk_sets (*skrf.calibration.calibration.PHN property*), 1192
caled_ntwk_sets (*skrf.calibration.calibration.SDDL property*), 1171
caled_ntwk_sets (*skrf.calibration.calibration.SDDLWeikle property*), 1182
caled_ntwk_sets (*skrf.calibration.calibration.SixteenTerm property*), 1331
caled_ntwk_sets (*skrf.calibration.calibration.SOLT property*), 1213
caled_ntwk_sets (*skrf.calibration.calibration.TRL property*), 1280
caled_ntwk_sets (*skrf.calibration.calibration.TUGMultilineTRL property*), 1319
caled_ntwk_sets (*skrf.calibration.calibration.TwelveTerm property*), 1202
caled_ntwk_sets (*skrf.calibration.calibration.TwoPortOnePath property*), 1370
caled_ntwk_sets (*skrf.calibration.calibration.UnknownThru property*), 1234
caled_ntwks (*skrf.calibration.calibration.Calibration property*), 1150
caled_ntwks (*skrf.calibration.calibration.EightTerm property*), 1223
caled_ntwks (*skrf.calibration.calibration.EnhancedResponse property*), 1380
caled_ntwks (*skrf.calibration.calibration.LMR16 property*), 1343
caled_ntwks (*skrf.calibration.calibration.LRM property*), 1246
caled_ntwks (*skrf.calibration.calibration.LRRM property*), 1257
caled_ntwks (*skrf.calibration.calibration.MRC property*), 1269

caled_ntwks (*skrf.calibration.calibration.MultilineTRL property*), 1292
caled_ntwks (*skrf.calibration.calibration.NISTMultilineTRL property*), 1304
caled_ntwks (*skrf.calibration.calibration.Normalization property*), 1354
caled_ntwks (*skrf.calibration.calibration.OnePort property*), 1161
caled_ntwks (*skrf.calibration.calibration.PHN property*), 1192
caled_ntwks (*skrf.calibration.calibration.SDDL property*), 1171
caled_ntwks (*skrf.calibration.calibration.SDDLWeikle property*), 1182
caled_ntwks (*skrf.calibration.calibration.SixteenTerm property*), 1331
caled_ntwks (*skrf.calibration.calibration.SOLT property*), 1213
caled_ntwks (*skrf.calibration.calibration.TRL property*), 1280
caled_ntwks (*skrf.calibration.calibration.TUGMultilineTRL property*), 1319
caled_ntwks (*skrf.calibration.calibration.TwelveTerm property*), 1203
caled_ntwks (*skrf.calibration.calibration.TwoPortOnePath property*), 1370
caled_ntwks (*skrf.calibration.calibration.UnknownThru property*), 1234
Calibration (*class in skrf.calibration.calibration*), 1148
capacitor() (*skrf.media.circularWaveguide.CircularWaveguide method*), 988
capacitor() (*skrf.media.coaxial.Coaxial method*), 1041
capacitor() (*skrf.media.cpw.CPW method*), 1014
capacitor() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1119
capacitor() (*skrf.media.distributedCircuit.DistributedCircuit method*), 934
capacitor() (*skrf.media.freespace.Freespace method*), 1094
capacitor() (*skrf.media.media.DefinedGammaZ0 method*), 910
capacitor() (*skrf.media.media.Media method*), 887
capacitor() (*skrf.media.mline.MLine method*), 1069
capacitor() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 962
capacitor_q() (*skrf.media.circularWaveguide.CircularWaveguide method*), 988
capacitor_q() (*skrf.media.coaxial.Coaxial method*), 1041
capacitor_q() (*skrf.media.cpw.CPW method*), 1015
capacitor_q() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1119
capacitor_q() (*skrf.media.distributedCircuit.DistributedCircuit method*), 934
capacitor_q() (*skrf.media.freespace.Freespace method*), 1094
capacitor_q() (*skrf.media.media.DefinedGammaZ0 method*), 910
capacitor_q() (*skrf.media.media.Media method*), 887
capacitor_q() (*skrf.media.mline.MLine method*), 1070
capacitor_q() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 963
cascade() (*in module skrf.network*), 801
cascade_list() (*in module skrf.network*), 802
center (*skrf.frequency.Frequency property*), 837
center_idx (*skrf.frequency.Frequency property*), 838
center_scaled (*skrf.frequency.Frequency property*), 838
characteristic impedance, **1609**
check_bounds() (*skrf.vi.validators.FloatValidator method*), 1446
check_bounds() (*skrf.vi.validators.IntValidator method*), 1445
check_duplicate_names() (*skrf.circuit.Circuit class method*), 1498
check_frequency_equal() (*in module skrf.network*), 835
check_frequency_exist() (*in module skrf.network*), 834
check_monotonic_increasing() (*skrf.frequency.Frequency method*), 843
check_nports_equal() (*in module skrf.network*), 834
check_z0_equal() (*in module skrf.network*), 835
chopinhalf() (*in module skrf.network*), 833
Circuit (*class in skrf.circuit*), 1486
CircularWaveguide (*class in skrf.media.circularWaveguide*), 979
clear() (*skrf.util.HomoDict method*), 1568
Coaxial (*class in skrf.media.coaxial*), 1031
coefs (*skrf.calibration.calibration.Calibration property*), 1150
coefs (*skrf.calibration.calibration.EightTerm property*), 1223
coefs (*skrf.calibration.calibration.EnhancedResponse property*), 1380
coefs (*skrf.calibration.calibration.LMR16 property*), 1343
coefs (*skrf.calibration.calibration.LRM property*), 1246
coefs (*skrf.calibration.calibration.LRRM property*), 1257
coefs (*skrf.calibration.calibration.MRC property*), 1269
coefs (*skrf.calibration.calibration.MultilineTRL property*), 1292
coefs (*skrf.calibration.calibration.MultiportCal property*), 1363
coefs (*skrf.calibration.calibration.MultiportSOLT prop-*

erty), 1366
coefs (*skrf.calibration.calibration.NISTMultilineTRL property*), 1305
coefs (*skrf.calibration.calibration.Normalization property*), 1354
coefs (*skrf.calibration.calibration.OnePort property*), 1161
coefs (*skrf.calibration.calibration.PHN property*), 1192
coefs (*skrf.calibration.calibration.SDDL property*), 1171
coefs (*skrf.calibration.calibration.SDDLWeikle property*), 1182
coefs (*skrf.calibration.calibration.SixteenTerm property*), 1331
coefs (*skrf.calibration.calibration.SOLT property*), 1213
coefs (*skrf.calibration.calibration.TRL property*), 1280
coefs (*skrf.calibration.calibration.TUGMultilineTRL property*), 1319
coefs (*skrf.calibration.calibration.TwelveTerm property*), 1203
coefs (*skrf.calibration.calibration.TwoPortOnePath property*), 1370
coefs (*skrf.calibration.calibration.UnknownThru property*), 1234
coefs_12term (*skrf.calibration.calibration.Calibration property*), 1150
coefs_12term (*skrf.calibration.calibration.EightTerm property*), 1223
coefs_12term (*skrf.calibration.calibration.EnhancedResponse property*), 1381
coefs_12term (*skrf.calibration.calibration.LMR16 property*), 1343
coefs_12term (*skrf.calibration.calibration.LRM property*), 1246
coefs_12term (*skrf.calibration.calibration.LRRM property*), 1258
coefs_12term (*skrf.calibration.calibration.MRC property*), 1269
coefs_12term (*skrf.calibration.calibration.MultilineTRL property*), 1292
coefs_12term (*skrf.calibration.calibration.NISTMultilineTRL property*), 1305
coefs_12term (*skrf.calibration.calibration.Normalization property*), 1354
coefs_12term (*skrf.calibration.calibration.OnePort property*), 1162
coefs_12term (*skrf.calibration.calibration.PHN property*), 1193
coefs_12term (*skrf.calibration.calibration.SDDL property*), 1172
coefs_12term (*skrf.calibration.calibration.SDDLWeikle property*), 1182
coefs_12term (*skrf.calibration.calibration.SixteenTerm property*), 1332
coefs_12term (*skrf.calibration.calibration.SOLT property*), 1213
coefs_12term (*skrf.calibration.calibration.TRL property*), 1280
coefs_12term (*skrf.calibration.calibration.TUGMultilineTRL property*), 1319
coefs_12term (*skrf.calibration.calibration.TwelveTerm property*), 1203
coefs_12term (*skrf.calibration.calibration.TwoPortOnePath property*), 1371
coefs_12term (*skrf.calibration.calibration.UnknownThru property*), 1235
coefs_12term_ntwks (*skrf.calibration.calibration.Calibration property*), 1151
coefs_12term_ntwks (*skrf.calibration.calibration.EightTerm property*), 1224
coefs_12term_ntwks (*skrf.calibration.calibration.EnhancedResponse property*), 1381
coefs_12term_ntwks (*skrf.calibration.calibration.LMR16 property*), 1344
coefs_12term_ntwks (*skrf.calibration.calibration.LRM property*), 1247
coefs_12term_ntwks (*skrf.calibration.calibration.LRRM property*), 1258
coefs_12term_ntwks (*skrf.calibration.calibration.MRC property*), 1270
coefs_12term_ntwks (*skrf.calibration.calibration.MultilineTRL property*), 1293
coefs_12term_ntwks (*skrf.calibration.calibration.NISTMultilineTRL property*), 1305
coefs_12term_ntwks (*skrf.calibration.calibration.Normalization property*), 1355
coefs_12term_ntwks (*skrf.calibration.calibration.OnePort property*), 1162
coefs_12term_ntwks (*skrf.calibration.calibration.PHN property*), 1193
coefs_12term_ntwks (*skrf.calibration.calibration.SDDL property*), 1172
coefs_12term_ntwks (*skrf.calibration.calibration.SDDLWeikle property*), 1183
coefs_12term_ntwks (*skrf.calibration.calibration.SixteenTerm property*), 1332
coefs_12term_ntwks (*skrf.calibration.calibration.SOLT property*), 1214
coefs_12term_ntwks (*skrf.calibration.calibration.TRL property*), 1281
coefs_12term_ntwks (*skrf.calibration.calibration.TUGMultilineTRL property*), 1320
coefs_12term_ntwks (*skrf.calibration.calibration.TwelveTerm property*), 1204
coefs_12term_ntwks (*skrf.calibration.calibration.TwoPortOnePath property*), 1371
coefs_12term_ntwks (*skrf.calibration.calibration.UnknownThru property*), 1235

coefs_3term (*skrf.calibration.calibration.Calibration property*), 1151
coefs_3term (*skrf.calibration.calibration.EightTerm property*), 1224
coefs_3term (*skrf.calibration.calibration.EnhancedResponse property*), 1381
coefs_3term (*skrf.calibration.calibration.LMR16 property*), 1344
coefs_3term (*skrf.calibration.calibration.LRM property*), 1247
coefs_3term (*skrf.calibration.calibration.LRRM property*), 1258
coefs_3term (*skrf.calibration.calibration.MRC property*), 1270
coefs_3term (*skrf.calibration.calibration.MultilineTRL property*), 1293
coefs_3term (*skrf.calibration.calibration.NISTMultilineTRL property*), 1306
coefs_3term (*skrf.calibration.calibration.Normalization property*), 1355
coefs_3term (*skrf.calibration.calibration.OnePort property*), 1162
coefs_3term (*skrf.calibration.calibration.PHN property*), 1193
coefs_3term (*skrf.calibration.calibration.SDDL property*), 1172
coefs_3term (*skrf.calibration.calibration.SDDLWeikle property*), 1183
coefs_3term (*skrf.calibration.calibration.SixteenTerm property*), 1332
coefs_3term (*skrf.calibration.calibration.SOLT property*), 1214
coefs_3term (*skrf.calibration.calibration.TRL property*), 1281
coefs_3term (*skrf.calibration.calibration.TUGMultilineTRL property*), 1320
coefs_3term (*skrf.calibration.calibration.TwelveTerm property*), 1204
coefs_3term (*skrf.calibration.calibration.TwoPortOnePath property*), 1371
coefs_3term (*skrf.calibration.calibration.UnknownThru property*), 1235
coefs_3term_ntwks (*skrf.calibration.calibration.Calibration property*), 1151
coefs_3term_ntwks (*skrf.calibration.calibration.EightTerm property*), 1224
coefs_3term_ntwks (*skrf.calibration.calibration.EnhancedResponse property*), 1382
coefs_3term_ntwks (*skrf.calibration.calibration.LMR16 property*), 1344
coefs_3term_ntwks (*skrf.calibration.calibration.LRM property*), 1247
coefs_3term_ntwks (*skrf.calibration.calibration.LRRM property*), 1258
coefs_3term_ntwks (*skrf.calibration.calibration.MRC property*), 1270
coefs_3term_ntwks (*skrf.calibration.calibration.MultilineTRL property*), 1293
coefs_3term_ntwks (*skrf.calibration.calibration.NISTMultilineTRL property*), 1306
coefs_3term_ntwks (*skrf.calibration.calibration.Normalization property*), 1355
coefs_3term_ntwks (*skrf.calibration.calibration.OnePort property*), 1162
coefs_3term_ntwks (*skrf.calibration.calibration.PHN property*), 1193
coefs_3term_ntwks (*skrf.calibration.calibration.SDDL property*), 1173
coefs_3term_ntwks (*skrf.calibration.calibration.SDDLWeikle property*), 1183
coefs_3term_ntwks (*skrf.calibration.calibration.SixteenTerm property*), 1332
coefs_3term_ntwks (*skrf.calibration.calibration.SOLT property*), 1214
coefs_3term_ntwks (*skrf.calibration.calibration.TRL property*), 1281
coefs_3term_ntwks (*skrf.calibration.calibration.TUGMultilineTRL property*), 1320
coefs_3term_ntwks (*skrf.calibration.calibration.TwelveTerm property*), 1204
coefs_3term_ntwks (*skrf.calibration.calibration.TwoPortOnePath property*), 1371
coefs_3term_ntwks (*skrf.calibration.calibration.UnknownThru property*), 1236
coefs_8term (*skrf.calibration.calibration.Calibration property*), 1151
coefs_8term (*skrf.calibration.calibration.EightTerm property*), 1224
coefs_8term (*skrf.calibration.calibration.EnhancedResponse property*), 1382
coefs_8term (*skrf.calibration.calibration.LMR16 property*), 1344
coefs_8term (*skrf.calibration.calibration.LRM property*), 1247
coefs_8term (*skrf.calibration.calibration.LRRM property*), 1259
coefs_8term (*skrf.calibration.calibration.MRC property*), 1270
coefs_8term (*skrf.calibration.calibration.MultilineTRL property*), 1293
coefs_8term (*skrf.calibration.calibration.NISTMultilineTRL property*), 1306
coefs_8term (*skrf.calibration.calibration.Normalization property*), 1355
coefs_8term (*skrf.calibration.calibration.OnePort property*), 1163
coefs_8term (*skrf.calibration.calibration.PHN property*), 1194

coefs_8term (*skrf.calibration.calibration.SDDL property*), 1173
coefs_8term (*skrf.calibration.calibration.SDDLWeikle property*), 1183
coefs_8term (*skrf.calibration.calibration.SixteenTerm property*), 1333
coefs_8term (*skrf.calibration.calibration.SOLT property*), 1214
coefs_8term (*skrf.calibration.calibration.TRL property*), 1281
coefs_8term (*skrf.calibration.calibration.TUGMultilineTRL property*), 1320
coefs_8term (*skrf.calibration.calibration.TwelveTerm property*), 1204
coefs_8term (*skrf.calibration.calibration.TwoPortOnePath property*), 1372
coefs_8term (*skrf.calibration.calibration.UnknownThru property*), 1236
coefs_8term_ntwks (*skrf.calibration.calibration.Calibration property*), 1152
coefs_8term_ntwks (*skrf.calibration.calibration.EightTerm property*), 1225
coefs_8term_ntwks (*skrf.calibration.calibration.EnhancedResponse property*), 1382
coefs_8term_ntwks (*skrf.calibration.calibration.LMR16 property*), 1345
coefs_8term_ntwks (*skrf.calibration.calibration.LRM property*), 1248
coefs_8term_ntwks (*skrf.calibration.calibration.LRRM property*), 1259
coefs_8term_ntwks (*skrf.calibration.calibration.MRC property*), 1271
coefs_8term_ntwks (*skrf.calibration.calibration.MultilineTRL property*), 1294
coefs_8term_ntwks (*skrf.calibration.calibration.NISTMultilineTRL property*), 1306
coefs_8term_ntwks (*skrf.calibration.calibration.Normalization property*), 1356
coefs_8term_ntwks (*skrf.calibration.calibration.OnePort property*), 1163
coefs_8term_ntwks (*skrf.calibration.calibration.PHN property*), 1194
coefs_8term_ntwks (*skrf.calibration.calibration.SDDL property*), 1173
coefs_8term_ntwks (*skrf.calibration.calibration.SDDLWeikle property*), 1184
coefs_8term_ntwks (*skrf.calibration.calibration.SixteenTerm property*), 1333
coefs_8term_ntwks (*skrf.calibration.calibration.SOLT property*), 1215
coefs_8term_ntwks (*skrf.calibration.calibration.TRL property*), 1282
coefs_8term_ntwks (*skrf.calibration.calibration.TUGMultilineTRL property*), 1321
coefs_8term_ntwks (*skrf.calibration.calibration.TwelveTerm property*), 1205
coefs_8term_ntwks (*skrf.calibration.calibration.TwoPortOnePath property*), 1372
coefs_8term_ntwks (*skrf.calibration.calibration.UnknownThru property*), 1236
coefs_ntwks (*skrf.calibration.calibration.Calibration property*), 1152
coefs_ntwks (*skrf.calibration.calibration.EightTerm property*), 1225
coefs_ntwks (*skrf.calibration.calibration.EnhancedResponse property*), 1382
coefs_ntwks (*skrf.calibration.calibration.LMR16 property*), 1345
coefs_ntwks (*skrf.calibration.calibration.LRM property*), 1248
coefs_ntwks (*skrf.calibration.calibration.LRRM property*), 1259
coefs_ntwks (*skrf.calibration.calibration.MRC property*), 1271
coefs_ntwks (*skrf.calibration.calibration.MultilineTRL property*), 1294
coefs_ntwks (*skrf.calibration.calibration.NISTMultilineTRL property*), 1307
coefs_ntwks (*skrf.calibration.calibration.Normalization property*), 1356
coefs_ntwks (*skrf.calibration.calibration.OnePort property*), 1163
coefs_ntwks (*skrf.calibration.calibration.PHN property*), 1194
coefs_ntwks (*skrf.calibration.calibration.SDDL property*), 1173
coefs_ntwks (*skrf.calibration.calibration.SDDLWeikle property*), 1184
coefs_ntwks (*skrf.calibration.calibration.SixteenTerm property*), 1333
coefs_ntwks (*skrf.calibration.calibration.SOLT property*), 1215
coefs_ntwks (*skrf.calibration.calibration.TRL property*), 1282
coefs_ntwks (*skrf.calibration.calibration.TUGMultilineTRL property*), 1321
coefs_ntwks (*skrf.calibration.calibration.TwelveTerm property*), 1205
coefs_ntwks (*skrf.calibration.calibration.TwoPortOnePath property*), 1372
coefs_ntwks (*skrf.calibration.calibration.UnknownThru property*), 1236
coefs_to_ntwks() (*skrf.calibration.calibration.MultiportCal method*), 1364
coefs_to_ntwks() (*skrf.calibration.calibration.MultiportSOLT method*), 1367
columns (*skrf.io.csv.AgilentCSV property*), 1586
COM_receiver_noise_filter()

(*skrf.calibration.deembedding.IEEEP370 static method*), 1407
COM_receiver_noise_filter() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1422
COM_receiver_noise_filter() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1438
COM_receiver_noise_filter() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1414
COM_receiver_noise_filter() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1429
command() (*skrf.vi.vna.VNA static method*), 1453
complex2Scalar() (*in module skrf.mathFunctions*), 1521
complex_2_db() (*in module skrf.mathFunctions*), 1515
complex_2_db10() (*in module skrf.mathFunctions*), 1515
complex_2_degree() (*in module skrf.mathFunctions*), 1515
complex_2_magnitude() (*in module skrf.mathFunctions*), 1514
complex_2_quadrature() (*in module skrf.mathFunctions*), 1516
complex_2_radian() (*in module skrf.mathFunctions*), 1515
complex_2_reim() (*in module skrf.mathFunctions*), 1514
complex_components() (*in module skrf.mathFunctions*), 1516
COMPONENT_FUNC_DICT (*skrf.network.Network attribute*), 638
compute_switch_terms() (*in module skrf.calibration.calibration*), 1389
concat_ports() (*in module skrf.network*), 810
connect() (*in module skrf.network*), 799
connect_s() (*in module skrf.network*), 812
connections (*skrf.circuit.Circuit property*), 1489
connections_list (*skrf.circuit.Circuit property*), 1489
connections_nb (*skrf.circuit.Circuit property*), 1489
connections_pair (*skrf.circuit.Circuit property*), 1489
constant_coeff (*skrf.vectorFitting.VectorFitting attribute*), 1457
convert_12term_2_8term() (*in module skrf.calibration.calibration*), 1393
convert_8term_2_12term() (*in module skrf.calibration.calibration*), 1393
convert_pnacoefs_2_skrf() (*in module skrf.calibration.calibration*), 1394
convert_skrfcoefs_2_pna() (*in module skrf.calibration.calibration*), 1394
copy() (*skrf.frequency.Frequency method*), 843
copy() (*skrf.media.circularWaveguide.CircularWaveguide method*), 989
copy() (*skrf.media.coaxial.Coaxial method*), 1042
copy() (*skrf.media.cpw.CPW method*), 1015
copy() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1120
copy() (*skrf.media.distributedCircuit.DistributedCircuit method*), 935
copy() (*skrf.media.freespace.Freespace method*), 1095
copy() (*skrf.media.media.DefinedGammaZ0 method*), 910
copy() (*skrf.media.media.Media method*), 888
copy() (*skrf.media.mline.MLine method*), 1070
copy() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 963
copy() (*skrf.network.Network method*), 682
copy() (*skrf.networkSet.NetworkSet method*), 852
copy() (*skrf.util.HomoDict method*), 1568
copy_from() (*skrf.network.Network method*), 683
copy_subset() (*skrf.network.Network method*), 683
count() (*skrf.util.HomoList method*), 1566
cov() (*skrf.networkSet.NetworkSet method*), 852
CPW (*class in skrf.media.cpw*), 1005
crop() (*skrf.network.Network method*), 684
cropped() (*skrf.network.Network method*), 684
cross_ratio() (*in module skrf.mathFunctions*), 1523
currents() (*skrf.circuit.Circuit method*), 1498
currents_external() (*skrf.circuit.Circuit method*), 1499

D

dataframe (*skrf.io.csv.AgilentCSV property*), 1587
datetime_index() (*skrf.networkSet.NetworkSet method*), 852
db10_2_mag() (*in module skrf.mathFunctions*), 1518
db_2_mag() (*in module skrf.mathFunctions*), 1517
db_2_magnitude() (*in module skrf.mathFunctions*), 1517
db_2_np() (*in module skrf.mathFunctions*), 1519
db_per_100feet_2_db_per_100meter() (*in module skrf.mathFunctions*), 1520
DC() (*skrf.calibration.deembedding.IEEEP370 static method*), 1408
DC() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1422
DC() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1438
DC() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1414
DC() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1429
dc_interp() (*skrf.calibration.deembedding.IEEEP370 static method*), 1409

dc_interp() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1423
dc_interp() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1440
dc_interp() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1416
dc_interp() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1431
de_embed() (*in module skrf.network*), 802
deembed() (*skrf.calibration.deembedding.AdmittanceCancel method*), 1405
deembed() (*skrf.calibration.deembedding.Deembedding method*), 1395
deembed() (*skrf.calibration.deembedding.IEEEP370 method*), 1409
deembed() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru method*), 1424
deembed() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru method*), 1440
deembed() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru method*), 1416
deembed() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru method*), 1431
deembed() (*skrf.calibration.deembedding.ImpedanceCancel method*), 1406
deembed() (*skrf.calibration.deembedding.Open method*), 1398
deembed() (*skrf.calibration.deembedding.OpenShort method*), 1397
deembed() (*skrf.calibration.deembedding.Short method*), 1401
deembed() (*skrf.calibration.deembedding.ShortOpen method*), 1400
deembed() (*skrf.calibration.deembedding.SplitPi method*), 1402
deembed() (*skrf.calibration.deembedding.SplitTee method*), 1404
Deembedding (*class in skrf.calibration.deembedding*), 1395
default() (*skrf.io.general.TouchstoneEncoder method*), 1584
DefinedAEpTandZ0 (*class in skrf.media.definedAEpTandZ0*), 1111
DefinedGammaZ0 (*class in skrf.media.media*), 904
degree_2_radian() (*in module skrf.mathFunctions*), 1519
delay() (*skrf.network.Network method*), 685
delay_load() (*skrf.media.circularWaveguide.CircularWaveguide method*), 989
delay_load() (*skrf.media.coaxial.Coaxial method*), 1042
delay_load() (*skrf.media.cpw.CPW method*), 1015
delay_load() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1120
delay_load() (*skrf.media.distributedCircuit.DistributedCircuit method*), 935
delay_load() (*skrf.media.freespace.Freespace method*), 1095
delay_load() (*skrf.media.media.DefinedGammaZ0 method*), 911
delay_load() (*skrf.media.media.Media method*), 888
delay_load() (*skrf.media.mline.MLine method*), 1070
delay_load() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 963
delay_open() (*skrf.media.circularWaveguide.CircularWaveguide method*), 990
delay_open() (*skrf.media.coaxial.Coaxial method*), 1043
delay_open() (*skrf.media.cpw.CPW method*), 1016
delay_open() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1121
delay_open() (*skrf.media.distributedCircuit.DistributedCircuit method*), 936
delay_open() (*skrf.media.freespace.Freespace method*), 1096
delay_open() (*skrf.media.media.DefinedGammaZ0 method*), 911
delay_open() (*skrf.media.media.Media method*), 889
delay_open() (*skrf.media.mline.MLine method*), 1071
delay_open() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 964
delay_short() (*skrf.media.circularWaveguide.CircularWaveguide method*), 990
delay_short() (*skrf.media.coaxial.Coaxial method*), 1043
delay_short() (*skrf.media.cpw.CPW method*), 1017
delay_short() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1121
delay_short() (*skrf.media.distributedCircuit.DistributedCircuit method*), 936
delay_short() (*skrf.media.freespace.Freespace method*), 1096
delay_short() (*skrf.media.media.DefinedGammaZ0 method*), 912
delay_short() (*skrf.media.media.Media method*), 889
delay_short() (*skrf.media.mline.MLine method*), 1072
delay_short() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 965
DelimitedStrValidator (*class in skrf.vi.validators*), 1450
detect_span() (*in module skrf.time*), 1484
determine_line() (*in module skrf.calibration.calibration*), 1392
determine_reflect() (*in module skrf.calibration.calibration*), 1392
Device (*class in skrf.media.device*), 1508
df (*skrf.frequency.Frequency property*), 838
df_scaled (*skrf.frequency.Frequency property*), 838

dict (*skrf.io.csv.AgilentCSV property*), 1587
dict_2_recarray() (*in module skrf.util*), 1564
DictValidator (*class in skrf.vi.validators*), 1449
dim (*skrf.circuit.Circuit property*), 1489
dirac_delta() (*in module skrf.mathFunctions*), 1522
distance_2_electrical_length() (*in module skrf.tlineFunctions*), 1537
distributed_circuit_2_propagation_impedance() (*in module skrf.tlineFunctions*), 1542
DistributedCircuit (*class in skrf.media.distributedCircuit*), 927
drop_non_monotonic_increasing() (*skrf.frequency.Frequency method*), 844
drop_non_monotonic_increasing() (*skrf.network.Network method*), 685
DualCoupler (*class in skrf.media.device*), 1513
dut_termination() (*skrf.calibration.calibration.MultiportCal method*), 1364
dut_termination() (*skrf.calibration.calibration.MultiportSOLT method*), 1367
dw (*skrf.frequency.Frequency property*), 838

E

E_matrices() (*skrf.calibration.calibration.LMR16 method*), 1348
E_matrices() (*skrf.calibration.calibration.SixteenTerm method*), 1336
edge_labels (*skrf.circuit.Circuit property*), 1490
edges (*skrf.circuit.Circuit property*), 1490
effective_phase_deg (*skrf.calibration.calibration.TUGMultilineTRL property*), 1321
EightTerm (*class in skrf.calibration.calibration*), 1221
electrical_length() (*in module skrf.tlineFunctions*), 1536
electrical_length() (*skrf.media.circularWaveguide.CircularWaveguide method*), 991
electrical_length() (*skrf.media.coaxial.Coaxial method*), 1044
electrical_length() (*skrf.media.cpw.CPW method*), 1017
electrical_length() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1122
electrical_length() (*skrf.media.distributedCircuit.DistributedCircuit method*), 937
electrical_length() (*skrf.media.freespace.Freespace method*), 1097
electrical_length() (*skrf.media.media.DefinedGammaZ0 method*), 912
electrical_length() (*skrf.media.media.Media method*), 890
electrical_length() (*skrf.media.mline.MLine method*), 1072
electrical_length() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 965
electrical_length_2_distance() (*in module skrf.tlineFunctions*), 1538
element_wise_method() (*skrf.networkSet.NetworkSet method*), 852
embed() (*skrf.calibration.calibration.Calibration method*), 1156
embed() (*skrf.calibration.calibration.EightTerm method*), 1229
embed() (*skrf.calibration.calibration.EnhancedResponse method*), 1386
embed() (*skrf.calibration.calibration.LMR16 method*), 1349
embed() (*skrf.calibration.calibration.LRM method*), 1251
embed() (*skrf.calibration.calibration.LRRM method*), 1264
embed() (*skrf.calibration.calibration.MRC method*), 1275
embed() (*skrf.calibration.calibration.MultilineTRL method*), 1299
embed() (*skrf.calibration.calibration.MultiportCal method*), 1364
embed() (*skrf.calibration.calibration.MultiportSOLT method*), 1368
embed() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1312
embed() (*skrf.calibration.calibration.Normalization method*), 1360
embed() (*skrf.calibration.calibration.OnePort method*), 1166
embed() (*skrf.calibration.calibration.PHN method*), 1198
embed() (*skrf.calibration.calibration.SDDL method*), 1177
embed() (*skrf.calibration.calibration.SDDLWeikle method*), 1188
embed() (*skrf.calibration.calibration.SixteenTerm method*), 1337
embed() (*skrf.calibration.calibration.SOLT method*), 1218
embed() (*skrf.calibration.calibration.TRL method*), 1287
embed() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1326
embed() (*skrf.calibration.calibration.TwelveTerm method*), 1208
embed() (*skrf.calibration.calibration.TwoPortOnePath method*), 1376

embed() (*skrf.calibration.calibration.UnknownThru method*), 1240
encode() (*skrf.io.general.TouchstoneEncoder method*), 1585
EnhancedResponse (*class in skrf.calibration.calibration*), 1379
EnumValidator (*class in skrf.vi.validators*), 1447
ep (*skrf.media.circularWaveguide.CircularWaveguide property*), 982
ep (*skrf.media.freespace.Freespace property*), 1088
ep (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 955
ep_r_f (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1115
ep_with_rho (*skrf.media.freespace.Freespace property*), 1089
er_eff (*skrf.calibration.calibration.NISTMultilineTRL property*), 1307
er_eff (*skrf.calibration.calibration.TUGMultilineTRL property*), 1321
error_dict_2_network() (*in module skrf.calibration.calibration*), 1393
error_ntwk (*skrf.calibration.calibration.Calibration property*), 1152
error_ntwk (*skrf.calibration.calibration.EightTerm property*), 1225
error_ntwk (*skrf.calibration.calibration.EnhancedResponse property*), 1383
error_ntwk (*skrf.calibration.calibration.LMR16 property*), 1345
error_ntwk (*skrf.calibration.calibration.LRM property*), 1248
error_ntwk (*skrf.calibration.calibration.LRRM property*), 1259
error_ntwk (*skrf.calibration.calibration.MRC property*), 1271
error_ntwk (*skrf.calibration.calibration.MultilineTRL property*), 1294
error_ntwk (*skrf.calibration.calibration.NISTMultilineTRL property*), 1307
error_ntwk (*skrf.calibration.calibration.Normalization property*), 1356
error_ntwk (*skrf.calibration.calibration.OnePort property*), 1163
error_ntwk (*skrf.calibration.calibration.PHN property*), 1194
error_ntwk (*skrf.calibration.calibration.SDDL property*), 1174
error_ntwk (*skrf.calibration.calibration.SDDLWeikle property*), 1184
error_ntwk (*skrf.calibration.calibration.SixteenTerm property*), 1333
error_ntwk (*skrf.calibration.calibration.SOLT property*), 1215
error_ntwk (*skrf.calibration.calibration.TRL property*), 1282
error_ntwk (*skrf.calibration.calibration.TUGMultilineTRL property*), 1321
error_ntwk (*skrf.calibration.calibration.TwelveTerm property*), 1205
error_ntwk (*skrf.calibration.calibration.TwoPortOnePath property*), 1372
error_ntwk (*skrf.calibration.calibration.UnknownThru property*), 1237
evenodd2delta() (*in module skrf.network*), 833
Exponential (*class in skrf.taper*), 1142
extract_distance() (*skrf.media.circularWaveguide.CircularWaveguide method*), 991
extract_distance() (*skrf.media.coaxial.Coaxial method*), 1044
extract_distance() (*skrf.media.cpw.CPW method*), 1018
extract_distance() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1122
extract_distance() (*skrf.media.distributedCircuit.DistributedCircuit method*), 937
extract_distance() (*skrf.media.freespace.Freespace method*), 1097
extract_distance() (*skrf.media.media.DefinedGammaZ0 method*), 913
extract_distance() (*skrf.media.media.Media method*), 890
extract_distance() (*skrf.media.mline.MLine method*), 1073
extract_distance() (*skrf.media.rectangularWaveguide.RectangularWav method*), 965
extrapolate_to_dc() (*skrf.calibration.deembedding.IEEEP370 static method*), 1409
extrapolate_to_dc() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1424
extrapolate_to_dc() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1440
extrapolate_to_dc() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1416
extrapolate_to_dc() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1432
extrapolate_to_dc() (*skrf.network.Network method*), 685

F

f (*skrf.frequency.Frequency property*), 839
f (*skrf.network.Network property*), 642

f_cutoff (*skrf.media.circularWaveguide.CircularWaveguide property*), 982
f_cutoff (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 956
f_L_scaled (*skrf.qfactor.Qfactor property*), 1476
f_noise (*skrf.network.Network property*), 642
f_norm (*skrf.media.circularWaveguide.CircularWaveguide property*), 982
f_norm (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 956
f_scaled (*skrf.frequency.Frequency property*), 839
falsey (*skrf.vi.validators.BooleanValidator attribute*), 1451
family (*skrf.calibration.calibration.Calibration attribute*), 1152
family (*skrf.calibration.calibration.EightTerm attribute*), 1225
family (*skrf.calibration.calibration.EnhancedResponse attribute*), 1383
family (*skrf.calibration.calibration.LMR16 attribute*), 1345
family (*skrf.calibration.calibration.LRM attribute*), 1248
family (*skrf.calibration.calibration.LRRM attribute*), 1260
family (*skrf.calibration.calibration.MRC attribute*), 1271
family (*skrf.calibration.calibration.MultilineTRL attribute*), 1294
family (*skrf.calibration.calibration.MultiportCal attribute*), 1363
family (*skrf.calibration.calibration.MultiportSOLT attribute*), 1366
family (*skrf.calibration.calibration.NISTMultilineTRL attribute*), 1307
family (*skrf.calibration.calibration.Normalization attribute*), 1356
family (*skrf.calibration.calibration.OnePort attribute*), 1164
family (*skrf.calibration.calibration.PHN attribute*), 1195
family (*skrf.calibration.calibration.SDDL attribute*), 1174
family (*skrf.calibration.calibration.SDDLWeikle attribute*), 1184
family (*skrf.calibration.calibration.SixteenTerm attribute*), 1334
family (*skrf.calibration.calibration.SOLT attribute*), 1215
family (*skrf.calibration.calibration.TRL attribute*), 1282
family (*skrf.calibration.calibration.TUGMultilineTRL attribute*), 1321
family (*skrf.calibration.calibration.TwelveTerm attribute*), 1205
family (*skrf.calibration.calibration.TwoPortOnePath attribute*), 1373
family (*skrf.calibration.calibration.UnknownThru attribute*), 1237
feet_2_meter() (*in module skrf.mathFunctions*), 1520
filter() (*skrf.networkSet.NetworkSet method*), 853
filter() (*skrf.util.HomoDict method*), 1568
filter_nones() (*skrf.util.HomoDict method*), 1568
find_closest() (*in module skrf.mathFunctions*), 1529
find_correct_sign() (*in module skrf.mathFunctions*), 1529
find_n_peaks() (*in module skrf.time*), 1484
find_nearest() (*in module skrf.util*), 1560
find_nearest_index() (*in module skrf.util*), 1561
findReplace() (*in module skrf.util*), 1564
fit() (*skrf.qfactor.Qfactor method*), 1479
fitted_network() (*skrf.qfactor.Qfactor method*), 1479
fitted_s() (*skrf.qfactor.Qfactor method*), 1480
fix_param_shape() (*in module skrf.network*), 834
fix_z0_shape() (*in module skrf.network*), 826
flip() (*in module skrf.network*), 802
flip() (*skrf.network.Network method*), 686
flipped() (*skrf.network.Network method*), 687
FloatValidator (*class in skrf.vi.validators*), 1445
four_oneports_2_twoport() (*in module skrf.network*), 809
Freespace (*class in skrf.media.freespace*), 1086
freq_re (*skrf.vi.validators.FreqValidator attribute*), 1447
FREQ_UNIT_DEFAULT (*in module skrf.constants*), 1548
Frequency (*class in skrf.frequency*), 836
frequency (*skrf.calibration.calibration.Calibration property*), 1152
frequency (*skrf.calibration.calibration.EightTerm property*), 1225
frequency (*skrf.calibration.calibration.EnhancedResponse property*), 1383
frequency (*skrf.calibration.calibration.LMR16 property*), 1345
frequency (*skrf.calibration.calibration.LRM property*), 1248
frequency (*skrf.calibration.calibration.LRRM property*), 1260
frequency (*skrf.calibration.calibration.MRC property*), 1271
frequency (*skrf.calibration.calibration.MultilineTRL property*), 1294
frequency (*skrf.calibration.calibration.NISTMultilineTRL property*), 1307
frequency (*skrf.calibration.calibration.Normalization property*), 1356
frequency (*skrf.calibration.calibration.OnePort property*), 1164
frequency (*skrf.calibration.calibration.PHN property*),

1195
frequency (*skrf.calibration.calibration.SDDL property*), 1174
frequency (*skrf.calibration.calibration.SDDLWeikle property*), 1184
frequency (*skrf.calibration.calibration.SixteenTerm property*), 1334
frequency (*skrf.calibration.calibration.SOLT property*), 1215
frequency (*skrf.calibration.calibration.TRL property*), 1282
frequency (*skrf.calibration.calibration.TUGMultilineTRL property*), 1321
frequency (*skrf.calibration.calibration.TwelveTerm property*), 1205
frequency (*skrf.calibration.calibration.TwoPortOnePath property*), 1373
frequency (*skrf.calibration.calibration.UnknownThru property*), 1237
frequency (*skrf.io.csv.AgilentCSV property*), 1587
frequency (*skrf.media.media.DefinedGammaZ0 property*), 906
frequency (*skrf.network.Network property*), 642
FrequencyUnitT (*in module skrf.constants*), 1548
FreqValidator (*class in skrf.vi.validators*), 1446
from_attenuation_VF() (*skrf.media.coaxial.Coaxial class method*), 1045
from_citi() (*skrf.networkSet.NetworkSet class method*), 853
from_coefs() (*skrf.calibration.calibration.Calibration class method*), 1156
from_coefs() (*skrf.calibration.calibration.EightTerm class method*), 1229
from_coefs() (*skrf.calibration.calibration.EnhancedResponse class method*), 1386
from_coefs() (*skrf.calibration.calibration.LMR16 class method*), 1349
from_coefs() (*skrf.calibration.calibration.LRM class method*), 1252
from_coefs() (*skrf.calibration.calibration.LRRM class method*), 1264
from_coefs() (*skrf.calibration.calibration.MRC class method*), 1275
from_coefs() (*skrf.calibration.calibration.MultilineTRL class method*), 1299
from_coefs() (*skrf.calibration.calibration.NISTMultilineTRL class method*), 1312
from_coefs() (*skrf.calibration.calibration.Normalization class method*), 1360
from_coefs() (*skrf.calibration.calibration.OnePort class method*), 1167
from_coefs() (*skrf.calibration.calibration.PHN class method*), 1198
from_coefs() (*skrf.calibration.calibration.SDDL class method*), 1177
from_coefs() (*skrf.calibration.calibration.SDDLWeikle class method*), 1188
from_coefs() (*skrf.calibration.calibration.SixteenTerm class method*), 1337
from_coefs() (*skrf.calibration.calibration.SOLT class method*), 1218
from_coefs() (*skrf.calibration.calibration.TRL class method*), 1287
from_coefs() (*skrf.calibration.calibration.TUGMultilineTRL class method*), 1326
from_coefs() (*skrf.calibration.calibration.TwelveTerm class method*), 1208
from_coefs() (*skrf.calibration.calibration.TwoPortOnePath class method*), 1376
from_coefs() (*skrf.calibration.calibration.UnknownThru class method*), 1240
from_coefs_ntwks() (*skrf.calibration.calibration.Calibration class method*), 1156
from_coefs_ntwks() (*skrf.calibration.calibration.EightTerm class method*), 1229
from_coefs_ntwks() (*skrf.calibration.calibration.EnhancedResponse class method*), 1387
from_coefs_ntwks() (*skrf.calibration.calibration.LMR16 class method*), 1349
from_coefs_ntwks() (*skrf.calibration.calibration.LRM class method*), 1252
from_coefs_ntwks() (*skrf.calibration.calibration.LRRM class method*), 1264
from_coefs_ntwks() (*skrf.calibration.calibration.MRC class method*), 1275
from_coefs_ntwks() (*skrf.calibration.calibration.MultilineTRL class method*), 1299
from_coefs_ntwks() (*skrf.calibration.calibration.NISTMultilineTRL class method*), 1313
from_coefs_ntwks() (*skrf.calibration.calibration.Normalization class method*), 1360
from_coefs_ntwks() (*skrf.calibration.calibration.OnePort class method*), 1167
from_coefs_ntwks() (*skrf.calibration.calibration.PHN class method*), 1198
from_coefs_ntwks() (*skrf.calibration.calibration.SDDL class method*), 1177
from_coefs_ntwks() (*skrf.calibration.calibration.SDDLWeikle class method*), 1188
from_coefs_ntwks() (*skrf.calibration.calibration.SixteenTerm class method*), 1337
from_coefs_ntwks() (*skrf.calibration.calibration.SOLT class method*), 1219
from_coefs_ntwks() (*skrf.calibration.calibration.TRL class method*), 1287
from_coefs_ntwks() (*skrf.calibration.calibration.TUGMultilineTRL class method*), 1326
from_coefs_ntwks() (*skrf.calibration.calibration.TwelveTerm*

class method), 1209
from_coefs_ntwks() (*skrf.calibration.calibration.TwoPortOnePath class method*), 1376
from_coefs_ntwks() (*skrf.calibration.calibration.UnknownThru class method*), 1241
from_csv() (*skrf.media.coaxial.Coaxial class method*), 1046
from_csv() (*skrf.media.distributedCircuit.DistributedCircuit class method*), 938
from_csv() (*skrf.media.media.DefinedGammaZ0 class method*), 913
from_dbdeg() (*skrf.media.device.Hybrid class method*), 1511
from_dbdeg() (*skrf.media.device.MatchedSymmetricCoupler class method*), 1510
from_dbdeg() (*skrf.media.device.QuadratureHybrid class method*), 1512
from_dir() (*skrf.networkSet.NetworkSet class method*), 854
from_distributed_circuit() (*skrf.media.freespace.Freespace class method*), 1097
from_f() (*skrf.frequency.Frequency class method*), 844
from_json_string() (*in module skrf.io.general*), 1585
from_mdif() (*skrf.networkSet.NetworkSet class method*), 854
from_media() (*skrf.media.coaxial.Coaxial class method*), 1046
from_media() (*skrf.media.distributedCircuit.DistributedCircuit class method*), 938
from_s_dict() (*skrf.networkSet.NetworkSet class method*), 854
from_string() (*skrf.network.Network class method*), 687
from_z() (*skrf.network.Network class method*), 688
from_z0() (*skrf.media.circularWaveguide.CircularWaveguide class method*), 992
from_z0() (*skrf.media.rectangularWaveguide.RectangularWaveguide class method*), 966
from_Z0_Dout() (*skrf.media.coaxial.Coaxial class method*), 1044
from_zip() (*skrf.networkSet.NetworkSet class method*), 855
func_on_all_figs() (*in module skrf.plotting*), 880
func_on_networks() (*in module skrf.networkSet*), 867
func_on_parameter() (*skrf.network.Network method*), 688

G

G (*skrf.circuit.Circuit property*), 1487
G (*skrf.media.coaxial.Coaxial property*), 1034
g (*skrf.network.Network property*), 643
g_arcl (*skrf.network.Network property*), 643
g_arcl_unwrap (*skrf.network.Network property*), 643
g_db (*skrf.network.Network property*), 644
g_db0 (*skrf.network.Network property*), 644
g_deg (*skrf.network.Network property*), 644
g_deg_unwrap (*skrf.network.Network property*), 644
g_im (*skrf.network.Network property*), 644
g_mag (*skrf.network.Network property*), 645
g_opt (*skrf.network.Network property*), 645
g_rad (*skrf.network.Network property*), 645
g_rad_unwrap (*skrf.network.Network property*), 645
g_re (*skrf.network.Network property*), 645
g_time (*skrf.network.Network property*), 645
g_time_db (*skrf.network.Network property*), 646
g_time_impulse (*skrf.network.Network property*), 646
g_time_mag (*skrf.network.Network property*), 646
g_time_step (*skrf.network.Network property*), 646
g_vswr (*skrf.network.Network property*), 646
gain_circle() (*skrf.network.Network method*), 689
gamma (*skrf.calibration.calibration.NISTMultilineTRL property*), 1307
gamma (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
gamma (*skrf.media.circularWaveguide.CircularWaveguide property*), 982
gamma (*skrf.media.coaxial.Coaxial property*), 1036
gamma (*skrf.media.cpw.CPW property*), 1008
gamma (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1115
gamma (*skrf.media.distributedCircuit.DistributedCircuit property*), 930
gamma (*skrf.media.freespace.Freespace property*), 1089
gamma (*skrf.media.media.DefinedGammaZ0 property*), 906
gamma (*skrf.media.media.Media property*), 883
gamma (*skrf.media.mline.MLine property*), 1063
gamma (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 956
Gamma0_2_Gamma_in() (*in module skrf.tlineFunctions*), 1535
Gamma0_2_swr() (*in module skrf.tlineFunctions*), 1536
Gamma0_2_zin() (*in module skrf.tlineFunctions*), 1535
Gamma0_2_zl() (*in module skrf.tlineFunctions*), 1534
get() (*skrf.util.HomoDict method*), 1568
get_array_of() (*skrf.media.circularWaveguide.CircularWaveguide method*), 992
get_array_of() (*skrf.media.coaxial.Coaxial method*), 1046
get_array_of() (*skrf.media.cpw.CPW method*), 1018
get_array_of() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1122
get_array_of() (*skrf.media.distributedCircuit.DistributedCircuit method*), 938
get_array_of() (*skrf.media.freespace.Freespace method*), 1098

get_array_of() (*skrf.media.media.DefinedGammaZ0 method*), 914
get_array_of() (*skrf.media.media.Media method*), 890
get_array_of() (*skrf.media.mline.MLine method*), 1073
get_array_of() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 966
get_comment_variables() (*skrf.io.touchstone.Touchstone method*), 1571
get_comments() (*skrf.io.touchstone.Touchstone method*), 1571
get_extn() (*in module skrf.util*), 1563
get_fid() (*in module skrf.util*), 1563
get_format() (*skrf.io.touchstone.Touchstone method*), 1571
get_gamma_z0() (*skrf.io.touchstone.Touchstone method*), 1572
get_Hermitian_transpose() (*in module skrf.mathFunctions*), 1529
get_local_vectorfit() (*skrf.vectorFitting.VectorFittingParametric method*), 1473
get_model_order() (*skrf.vectorFitting.VectorFitting static method*), 1460
get_model_response() (*skrf.vectorFitting.VectorFitting method*), 1460
get_model_response() (*skrf.vectorFitting.VectorFittingParametric method*), 1473
get_noise_data() (*skrf.io.touchstone.Touchstone method*), 1572
get_noise_names() (*skrf.io.touchstone.Touchstone method*), 1572
get_rms_error() (*skrf.vectorFitting.VectorFitting method*), 1461
get_sparameter_arrays() (*skrf.io.touchstone.Touchstone method*), 1572
get_sparameter_data() (*skrf.io.touchstone.Touchstone method*), 1572
get_sparameter_names() (*skrf.io.touchstone.Touchstone method*), 1573
get_spurious() (*skrf.vectorFitting.VectorFitting static method*), 1461
get_window() (*in module skrf.time*), 1485
getset() (*in module skrf.networkSet*), 868
getz() (*skrf.calibration.deembedding.IEEEP370 static method*), 1410
getz() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1424
getz() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1441
getz() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1416
getz() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1432
git_version() (*in module skrf.util*), 1563
gmm2se() (*skrf.network.Network method*), 690
graph() (*skrf.circuit.Circuit method*), 1499
G_network() (*skrf.circuit.Circuit class method*), 1493
group_delay (*skrf.network.Network property*), 647

H

h (*skrf.network.Network property*), 647
h2s() (*in module skrf.network*), 823
h2z() (*in module skrf.network*), 823
h_arcl (*skrf.network.Network property*), 647
h_arcl_unwrap (*skrf.network.Network property*), 648
h_db (*skrf.network.Network property*), 648
h_db10 (*skrf.network.Network property*), 648
h_deg (*skrf.network.Network property*), 648
h_deg_unwrap (*skrf.network.Network property*), 648
h_im (*skrf.network.Network property*), 649
h_mag (*skrf.network.Network property*), 649
h_rad (*skrf.network.Network property*), 649
h_rad_unwrap (*skrf.network.Network property*), 649
h_re (*skrf.network.Network property*), 649
h_time (*skrf.network.Network property*), 650
h_time_db (*skrf.network.Network property*), 650
h_time_impulse (*skrf.network.Network property*), 650
h_time_mag (*skrf.network.Network property*), 650
h_time_step (*skrf.network.Network property*), 650
h_vswr (*skrf.network.Network property*), 651
has_duplicate_value() (*in module skrf.util*), 1561
has_params() (*skrf.networkSet.NetworkSet method*), 855
hfss_touchstone_2_gamma_z0() (*in module skrf.io.touchstone*), 1573
hfss_touchstone_2_media() (*in module skrf.io.touchstone*), 1574
hfss_touchstone_2_network() (*in module skrf.io.touchstone*), 1574
HomoDict (*class in skrf.util*), 1566
HomoList (*class in skrf.util*), 1565
Hybrid (*class in skrf.media.device*), 1510
Hybrid180 (*class in skrf.media.device*), 1512

I

ideal_coefs_12term() (*in module skrf.calibration.calibration*), 1390
IEEEP370 (*class in skrf.calibration.deembedding*), 1407
IEEEP370_MM_NZC_2xThru (*class in skrf.calibration.deembedding*), 1419
IEEEP370_MM_ZC_2xThru (*class in skrf.calibration.deembedding*), 1435
IEEEP370_SE_NZC_2xThru (*class in skrf.calibration.deembedding*), 1412

IEEEP370_SE_ZC_2xThru (*class in skrf.calibration.deembedding*), 1427
ifft() (*in module skrf.mathFunctions*), 1526
impedance_mismatch() (*in module skrf.network*), 832
impedance_mismatch() (*skrf.media.circularWaveguide.CircularWaveguide method*), 992
impedance_mismatch() (*skrf.media.coaxial.Coaxial method*), 1046
impedance_mismatch() (*skrf.media.cpw.CPW method*), 1018
impedance_mismatch() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1123
impedance_mismatch() (*skrf.media.distributedCircuit.DistributedCircuit method*), 938
impedance_mismatch() (*skrf.media.freespace.Freespace method*), 1098
impedance_mismatch() (*skrf.media.media.DefinedGammaZ0 method*), 914
impedance_mismatch() (*skrf.media.media.Media method*), 891
impedance_mismatch() (*skrf.media.mline.MLine method*), 1073
impedance_mismatch() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 966
ImpedanceCancel (*class in skrf.calibration.deembedding*), 1405
impulse_response() (*skrf.network.Network method*), 692
ind (*in module skrf.data*), 1554
index() (*skrf.util.HomoList method*), 1566
indexes() (*in module skrf.time*), 1484
inductor() (*skrf.media.circularWaveguide.CircularWaveguide method*), 992
inductor() (*skrf.media.coaxial.Coaxial method*), 1047
inductor() (*skrf.media.cpw.CPW method*), 1019
inductor() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1123
inductor() (*skrf.media.distributedCircuit.DistributedCircuit method*), 939
inductor() (*skrf.media.freespace.Freespace method*), 1099
inductor() (*skrf.media.media.DefinedGammaZ0 method*), 914
inductor() (*skrf.media.media.Media method*), 891
inductor() (*skrf.media.mline.MLine method*), 1074
inductor() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 967
inductor_q() (*skrf.media.circularWaveguide.CircularWaveguide method*), 993
inductor_q() (*skrf.media.coaxial.Coaxial method*), 1047
inductor_q() (*skrf.media.cpw.CPW method*), 1019
inductor_q() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1123
inductor_q() (*skrf.media.distributedCircuit.DistributedCircuit method*), 939
inductor_q() (*skrf.media.freespace.Freespace method*), 1099
inductor_q() (*skrf.media.media.DefinedGammaZ0 method*), 915
inductor_q() (*skrf.media.media.Media method*), 891
inductor_q() (*skrf.media.mline.MLine method*), 1074
inductor_q() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 967
INF (*in module skrf.constants*), 1547
inf_to_num() (*in module skrf.mathFunctions*), 1530
innerconnect() (*in module skrf.network*), 800
innerconnect_s() (*in module skrf.network*), 813
innerconnect_s_lstsq() (*in module skrf.network*), 814
input_impedance_at_theta() (*in module skrf.tlineFunctions*), 1540
interpolate() (*skrf.network.Network method*), 692
interpolate_frequency() (*skrf.networkSet.NetworkSet method*), 855
interpolate_from_network() (*skrf.networkSet.NetworkSet method*), 856
interpolate_from_params() (*skrf.networkSet.NetworkSet method*), 857
interpolate_self() (*skrf.network.Network method*), 694
intersections_dict (*skrf.circuit.Circuit property*), 1490
IntValidator (*class in skrf.vi.validators*), 1444
inv (*skrf.network.Network property*), 651
inv (*skrf.networkSet.NetworkSet property*), 848
inv() (*in module skrf.network*), 803
InvalidFrequencyWarning, 846
irfft() (*in module skrf.mathFunctions*), 1526
is_connected() (*skrf.circuit.Circuit method*), 1499
is_Hermitian() (*in module skrf.mathFunctions*), 1527
is_lossless() (*skrf.network.Network method*), 694
is_passive() (*skrf.network.Network method*), 695
is_passive() (*skrf.vectorFitting.VectorFitting method*), 1462
is_positive_definite() (*in module skrf.mathFunctions*), 1528
is_positive_semidefinite() (*in module skrf.mathFunctions*), 1528
is_reciprocal() (*skrf.network.Network method*), 695
is_square() (*in module skrf.mathFunctions*), 1527
is_symmetric() (*in module skrf.mathFunctions*), 1527

is_symmetric() (*skrf.network.Network method*), 696
isolator() (*skrf.media.circularWaveguide.CircularWaveguide method*), 993
isolator() (*skrf.media.coaxial.Coaxial method*), 1048
isolator() (*skrf.media.cpw.CPW method*), 1019
isolator() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1124
isolator() (*skrf.media.distributedCircuit.DistributedCircuit method*), 940
isolator() (*skrf.media.freespace.Freespace method*), 1099
isolator() (*skrf.media.media.DefinedGammaZ0 method*), 915
isolator() (*skrf.media.media.Media method*), 892
isolator() (*skrf.media.mline.MLine method*), 1074
isolator() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 968
item_separator (*skrf.io.general.TouchstoneEncoder attribute*), 1583
items() (*skrf.util.HomoDict method*), 1568
iterencode() (*skrf.io.general.TouchstoneEncoder method*), 1585

K

k0 (*skrf.media.circularWaveguide.CircularWaveguide property*), 983
k0 (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 956
K_BOLTZMANN (*in module skrf.constants*), 1547
kappa (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
kc (*skrf.media.circularWaveguide.CircularWaveguide property*), 983
kc (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 956
key_separator (*skrf.io.general.TouchstoneEncoder attribute*), 1584
keys() (*skrf.util.HomoDict method*), 1568
Klopfenstein (*class in skrf.taper*), 1146
kx (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 957
ky (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 957

L

L (*skrf.media.coaxial.Coaxial property*), 1034
labelXAxis() (*skrf.frequency.Frequency method*), 844
lambd (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
lambda_cutoff (*skrf.media.circularWaveguide.CircularWaveguide property*), 983
lambda_cutoff (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 957
lambda_guide (*skrf.media.circularWaveguide.CircularWaveguide property*), 984
lambda_guide (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 957
legend_off() (*in module skrf.plotting*), 880
LIZZIE (*in module skrf.data*), 1552
line() (*skrf.media.circularWaveguide.CircularWaveguide method*), 994
line() (*skrf.media.coaxial.Coaxial method*), 1048
line() (*skrf.media.cpw.CPW method*), 1020
line() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1124
line() (*skrf.media.distributedCircuit.DistributedCircuit method*), 940
line() (*skrf.media.freespace.Freespace method*), 1100
line() (*skrf.media.media.DefinedGammaZ0 method*), 915
line() (*skrf.media.media.Media method*), 892
line() (*skrf.media.mline.MLine method*), 1075
line() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 968
line_floating() (*skrf.media.circularWaveguide.CircularWaveguide method*), 994
line_floating() (*skrf.media.coaxial.Coaxial method*), 1048
line_floating() (*skrf.media.cpw.CPW method*), 1020
line_floating() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1125
line_floating() (*skrf.media.distributedCircuit.DistributedCircuit method*), 940
line_floating() (*skrf.media.freespace.Freespace method*), 1100
line_floating() (*skrf.media.media.DefinedGammaZ0 method*), 916
line_floating() (*skrf.media.media.Media method*), 893
line_floating() (*skrf.media.mline.MLine method*), 1075
line_floating() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 968
Linear (*class in skrf.taper*), 1139
LMR16 (*class in skrf.calibration.calibration*), 1340
load() (*skrf.media.circularWaveguide.CircularWaveguide method*), 995
load() (*skrf.media.coaxial.Coaxial method*), 1049
load() (*skrf.media.cpw.CPW method*), 1021
load() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1125
load() (*skrf.media.distributedCircuit.DistributedCircuit method*), 941
load() (*skrf.media.freespace.Freespace method*), 1101
load() (*skrf.media.media.DefinedGammaZ0 method*), 917
load() (*skrf.media.media.Media method*), 893

load() (*skrf.media.mline.MLine method*), 1076
load() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 969
load_calibration_archive() (*skrf.calibration.calibration.NISTMultilineTRL class method*), 1313
load_file() (*skrf.io.touchstone.Touchstone method*), 1573
load_impedance_2_reflection_coefficient() (*in module skrf.tlineFunctions*), 1540
load_impedance_2_reflection_coefficient_at_theta() (*in module skrf.tlineFunctions*), 1541
LOG_OF_NEG (*in module skrf.constants*), 1547
lossless_mismatch() (*skrf.media.circularWaveguide.CircularWaveguide method*), 995
lossless_mismatch() (*skrf.media.coaxial.Coaxial method*), 1050
lossless_mismatch() (*skrf.media.cpw.CPW method*), 1022
lossless_mismatch() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1126
lossless_mismatch() (*skrf.media.distributedCircuit.DistributedCircuit method*), 942
lossless_mismatch() (*skrf.media.freespace.Freespace method*), 1101
lossless_mismatch() (*skrf.media.media.DefinedGammaZ0 method*), 917
lossless_mismatch() (*skrf.media.media.Media method*), 894
lossless_mismatch() (*skrf.media.mline.MLine method*), 1077
lossless_mismatch() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 970
LRM (*class in skrf.calibration.calibration*), 1244
LRRM (*class in skrf.calibration.calibration*), 1255

M

mag_2_db() (*in module skrf.mathFunctions*), 1517
mag_2_db10() (*in module skrf.mathFunctions*), 1517
magnitude_2_db() (*in module skrf.mathFunctions*), 1516
makeErrorBox_v7() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru method*), 1432
makeErrorBox_v8() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru method*), 1432
makeStep() (*skrf.calibration.deembedding.IEEEP370 static method*), 1410
makeStep() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1425
makeStep() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1441
makeStep() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1417
makeStep() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1433
makeTL() (*skrf.calibration.deembedding.IEEEP370 static method*), 1410
makeTL() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1425
makeTL() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1441
makeTL() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1417
makeTL() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1433
match() (*skrf.media.circularWaveguide.CircularWaveguide method*), 996
match() (*skrf.media.coaxial.Coaxial method*), 1050
match() (*skrf.media.cpw.CPW method*), 1022
match() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1126
match() (*skrf.media.distributedCircuit.DistributedCircuit method*), 942
match() (*skrf.media.freespace.Freespace method*), 1102
match() (*skrf.media.media.DefinedGammaZ0 method*), 917
match() (*skrf.media.media.Media method*), 894
match() (*skrf.media.mline.MLine method*), 1077
match() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 970
MatchedSymmetricCoupler (*class in skrf.media.device*), 1509
materials (*in module skrf.data*), 1549
max_gain (*skrf.network.Network property*), 651
max_iterations (*skrf.vectorFitting.VectorFitting attribute*), 1457
max_stable_gain (*skrf.network.Network property*), 652
max_tol (*skrf.vectorFitting.VectorFitting attribute*), 1457
mean_s_db (*skrf.networkSet.NetworkSet property*), 848
measured_unisolated (*skrf.calibration.calibration.EightTerm property*), 1226
measured_unisolated (*skrf.calibration.calibration.LRM property*), 1248
measured_unisolated (*skrf.calibration.calibration.LRRM property*), 1260
measured_unisolated (*skrf.calibration.calibration.MRC property*), 1271

measured_unisolated (*skrf.calibration.calibration.MultilineTRL property*), 1295
measured_unisolated (*skrf.calibration.calibration.NISTMultilineTRL property*), 1307
measured_unisolated (*skrf.calibration.calibration.TRL property*), 1283
measured_unisolated (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
measured_unisolated (*skrf.calibration.calibration.UnknownThru property*), 1237
measured_unterminated (*skrf.calibration.calibration.EightTerm property*), 1226
measured_unterminated (*skrf.calibration.calibration.LMR16 property*), 1345
measured_unterminated (*skrf.calibration.calibration.LRM property*), 1248
measured_unterminated (*skrf.calibration.calibration.LRRM property*), 1260
measured_unterminated (*skrf.calibration.calibration.MRC property*), 1271
measured_unterminated (*skrf.calibration.calibration.MultilineTRL property*), 1295
measured_unterminated (*skrf.calibration.calibration.NISTMultilineTRL property*), 1307
measured_unterminated (*skrf.calibration.calibration.SixteenTerm property*), 1334
measured_unterminated (*skrf.calibration.calibration.TRL property*), 1283
measured_unterminated (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
measured_unterminated (*skrf.calibration.calibration.UnknownThru property*), 1237
Media (*class in skrf.media.media*), 881
media_at() (*skrf.taper.Exponential method*), 1144
media_at() (*skrf.taper.Klopfenstein method*), 1148
media_at() (*skrf.taper.Linear method*), 1142
media_at() (*skrf.taper.SmoothStep method*), 1146
media_at() (*skrf.taper.Taper1D method*), 1139
medias (*skrf.taper.Exponential property*), 1143
medias (*skrf.taper.Klopfenstein property*), 1147
medias (*skrf.taper.Linear property*), 1140
medias (*skrf.taper.SmoothStep property*), 1145
medias (*skrf.taper.Taper1D property*), 1137
meter_2_feet() (*in module skrf.mathFunctions*), 1520
MLine (*class in skrf.media.mline*), 1059
mode() (*skrf.media.circularWaveguide.CircularWaveguide method*), 996
mode() (*skrf.media.coaxial.Coaxial method*), 1050
mode() (*skrf.media.cpw.CPW method*), 1022
mode() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1127
mode() (*skrf.media.distributedCircuit.DistributedCircuit method*), 942
mode() (*skrf.media.freespace.Freespace method*), 1102
mode() (*skrf.media.media.DefinedGammaZ0 method*), 918
mode() (*skrf.media.media.Media method*), 895
mode() (*skrf.media.mline.MLine method*), 1077
mode() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 970
module
    skrf.calibration.calibration, 1148
    skrf.calibration.deembedding, 1394
    skrf.circuit, 1485
    skrf.constants, 1547
    skrf.data, 1548
    skrf.frequency, 836
    skrf.instances, 1546
    skrf.io.csv, 1586
    skrf.io.general, 1575
    skrf.io.touchstone, 1569
    skrf.mathFunctions, 1514
    skrf.media, 881
    skrf.media.device, 1508
    skrf.network, 633
    skrf.networkSet, 846
    skrf.plotting, 868
    skrf.qfactor, 1474
    skrf.taper, 1136
    skrf.time, 1481
    skrf.tlineFunctions, 1531
    skrf.util, 1558
    skrf.vi, 1443
    skrf.vi.vna, 1452
MRC (*class in skrf.calibration.calibration*), 1267
mu (*skrf.media.circularWaveguide.CircularWaveguide property*), 984
mu (*skrf.media.freespace.Freespace property*), 1089
mu (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 958
MultilineTRL (*in module skrf.calibration.calibration*), 1290

multiplier (*skrf.frequency.Frequency property*), 839
multiplier_dict (*skrf.frequency.Frequency attribute*), 840
multiply_noise() (*skrf.network.Network method*), 696
MultiportCal (*class in skrf.calibration.calibration*), 1362
MultiportSOLT (*class in skrf.calibration.calibration*), 1365

N

n (*skrf.network.Network property*), 653
n_oneports_2_nport() (*in module skrf.network*), 809
n_traces (*skrf.io.csv.AgilentCSV property*), 1587
n_twoports_2_nport() (*in module skrf.network*), 809
Network (*class in skrf.network*), 633
network (*skrf.circuit.Circuit property*), 1490
network (*skrf.taper.Exponential property*), 1143
network (*skrf.taper.Klopfenstein property*), 1147
network (*skrf.taper.Linear property*), 1140
network (*skrf.taper.SmoothStep property*), 1145
network (*skrf.taper.Taper1D property*), 1137
network (*skrf.vectorFitting.VectorFitting attribute*), 1456
network_2_dataframe() (*in module skrf.io.general*), 1582
network_2_spreadsheet() (*in module skrf.io.general*), 1581
networks (*skrf.io.csv.AgilentCSV property*), 1587
networks_dict() (*skrf.circuit.Circuit method*), 1499
networks_list() (*skrf.circuit.Circuit method*), 1500
networks_nb (*skrf.circuit.Circuit property*), 1490
NetworkSet (*class in skrf.networkSet*), 846
networkset_2_spreadsheet() (*in module skrf.io.general*), 1581
neuman() (*in module skrf.mathFunctions*), 1522
nf() (*skrf.network.Network method*), 696
nf_circle() (*skrf.network.Network method*), 697
nfdb_gs() (*skrf.network.Network method*), 698
nfmin (*skrf.network.Network property*), 653
nfmin_db (*skrf.network.Network property*), 653
NISTMultilineTRL (*class in skrf.calibration.calibration*), 1302
nodes_nb (*skrf.circuit.Circuit property*), 1490
noisy (*skrf.network.Network property*), 653
nonreciprocity() (*skrf.network.Network method*), 698
Normalization (*class in skrf.calibration.calibration*), 1352
normalized_directivity (*skrf.calibration.calibration.Calibration property*), 1153
normalized_directivity (*skrf.calibration.calibration.EightTerm property*), 1226
normalized_directivity (*skrf.calibration.calibration.EnhancedResponse property*), 1383
normalized_directivity (*skrf.calibration.calibration.LMR16 property*), 1345
normalized_directivity (*skrf.calibration.calibration.LRM property*), 1248
normalized_directivity (*skrf.calibration.calibration.LRRM property*), 1260
normalized_directivity (*skrf.calibration.calibration.MRC property*), 1271
normalized_directivity (*skrf.calibration.calibration.MultilineTRL property*), 1295
normalized_directivity (*skrf.calibration.calibration.NISTMultilineTRL property*), 1308
normalized_directivity (*skrf.calibration.calibration.Normalization property*), 1356
normalized_directivity (*skrf.calibration.calibration.OnePort property*), 1164
normalized_directivity (*skrf.calibration.calibration.PHN property*), 1195
normalized_directivity (*skrf.calibration.calibration.SDDL property*), 1174
normalized_directivity (*skrf.calibration.calibration.SDDLWeikle property*), 1184
normalized_directivity (*skrf.calibration.calibration.SixteenTerm property*), 1334
normalized_directivity (*skrf.calibration.calibration.SOLT property*), 1215
normalized_directivity (*skrf.calibration.calibration.TRL property*), 1283
normalized_directivity (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
normalized_directivity (*skrf.calibration.calibration.TwelveTerm property*), 1205
normalized_directivity (*skrf.calibration.calibration.TwoPortOnePath property*), 1373
normalized_directivity (*skrf.calibration.calibration.UnknownThru*

property), 1237
now_string() (*in module skrf.util*), 1559
now_string_2_dt() (*in module skrf.util*), 1559
np_2_db() (*in module skrf.mathFunctions*), 1519
npoints (*skrf.frequency.Frequency property*), 840
npoints (*skrf.media.circularWaveguide.CircularWaveguide property*), 984
npoints (*skrf.media.coaxial.Coaxial property*), 1037
npoints (*skrf.media.cpw.CPW property*), 1008
npoints (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1115
npoints (*skrf.media.distributedCircuit.DistributedCircuit property*), 930
npoints (*skrf.media.freespace.Freespace property*), 1090
npoints (*skrf.media.media.DefinedGammaZ0 property*), 906
npoints (*skrf.media.media.Media property*), 883
npoints (*skrf.media.mline.MLine property*), 1063
npoints (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 958
nports (*skrf.network.Network property*), 653
NRP() (*skrf.calibration.deembedding.IEEEP370 static method*), 1408
NRP() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1422
NRP() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1438
NRP() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1414
NRP() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1430
nstandards (*skrf.calibration.calibration.Calibration property*), 1153
nstandards (*skrf.calibration.calibration.EightTerm property*), 1226
nstandards (*skrf.calibration.calibration.EnhancedResponse property*), 1383
nstandards (*skrf.calibration.calibration.LMR16 property*), 1345
nstandards (*skrf.calibration.calibration.LRM property*), 1248
nstandards (*skrf.calibration.calibration.LRRM property*), 1260
nstandards (*skrf.calibration.calibration.MRC property*), 1272
nstandards (*skrf.calibration.calibration.MultilineTRL property*), 1295
nstandards (*skrf.calibration.calibration.NISTMultilineTRL property*), 1308
nstandards (*skrf.calibration.calibration.Normalization property*), 1356
nstandards (*skrf.calibration.calibration.OnePort property*), 1164
nstandards (*skrf.calibration.calibration.PHN property*), 1195
nstandards (*skrf.calibration.calibration.SDDL property*), 1174
nstandards (*skrf.calibration.calibration.SDDLWeikle property*), 1185
nstandards (*skrf.calibration.calibration.SixteenTerm property*), 1334
nstandards (*skrf.calibration.calibration.SOLT property*), 1215
nstandards (*skrf.calibration.calibration.TRL property*), 1283
nstandards (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
nstandards (*skrf.calibration.calibration.TwelveTerm property*), 1205
nstandards (*skrf.calibration.calibration.TwoPortOnePath property*), 1373
nstandards (*skrf.calibration.calibration.UnknownThru property*), 1237
nstd (*skrf.calibration.calibration.NISTMultilineTRL property*), 1308
ntwk (*skrf.media.device.Device property*), 1509
ntwk (*skrf.media.device.DualCoupler property*), 1513
ntwk (*skrf.media.device.Hybrid property*), 1511
ntwk (*skrf.media.device.Hybrid180 property*), 1513
ntwk (*skrf.media.device.MatchedSymmetricCoupler property*), 1510
ntwk (*skrf.media.device.QuadratureHybrid property*), 1512
ntwk_attr_2_df() (*skrf.networkSet.NetworkSet method*), 857
nudge() (*skrf.network.Network method*), 699
nudge_eig() (*in module skrf.mathFunctions*), 1530
null() (*in module skrf.mathFunctions*), 1523
number_of_ports (*skrf.network.Network property*), 653

O

ONE (*in module skrf.constants*), 1547
one_port_2_two_port() (*in module skrf.network*), 808
OnePort (*class in skrf.calibration.calibration*), 1159
Open (*class in skrf.calibration.deembedding*), 1397
Open() (*skrf.circuit.Circuit class method*), 1494
open() (*skrf.media.circularWaveguide.CircularWaveguide method*), 996
open() (*skrf.media.coaxial.Coaxial method*), 1051
open() (*skrf.media.cpw.CPW method*), 1023
open() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1127
open() (*skrf.media.distributedCircuit.DistributedCircuit method*), 943
open() (*skrf.media.freespace.Freespace method*), 1102
open() (*skrf.media.media.DefinedGammaZ0 method*), 918

open() (*skrf.media.media.Media method*), 895
open() (*skrf.media.mline.MLine method*), 1078
open() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 971
open_2p (*in module skrf.data*), 1556
OpenShort (*class in skrf.calibration.deembedding*), 1396
output_from_run (*skrf.calibration.calibration.Calibration property*), 1153
output_from_run (*skrf.calibration.calibration.EightTerm property*), 1226
output_from_run (*skrf.calibration.calibration.EnhancedResponse property*), 1383
output_from_run (*skrf.calibration.calibration.LMR16 property*), 1346
output_from_run (*skrf.calibration.calibration.LRM property*), 1249
output_from_run (*skrf.calibration.calibration.LRRM property*), 1260
output_from_run (*skrf.calibration.calibration.MRC property*), 1272
output_from_run (*skrf.calibration.calibration.MultilineTRL property*), 1295
output_from_run (*skrf.calibration.calibration.NISTMultilineTRL property*), 1308
output_from_run (*skrf.calibration.calibration.Normalization property*), 1357
output_from_run (*skrf.calibration.calibration.OnePort property*), 1164
output_from_run (*skrf.calibration.calibration.PHN property*), 1195
output_from_run (*skrf.calibration.calibration.SDDL property*), 1174
output_from_run (*skrf.calibration.calibration.SDDLWeikle property*), 1185
output_from_run (*skrf.calibration.calibration.SixteenTerm property*), 1334
output_from_run (*skrf.calibration.calibration.SOLT property*), 1216
output_from_run (*skrf.calibration.calibration.TRL property*), 1283
output_from_run (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
output_from_run (*skrf.calibration.calibration.TwelveTerm property*), 1205
output_from_run (*skrf.calibration.calibration.TwoPortOnePath property*), 1373
output_from_run (*skrf.calibration.calibration.UnknownThru property*), 1237
overlap() (*in module skrf.network*), 806
overlap() (*skrf.frequency.Frequency method*), 844
overlap_freq() (*in module skrf.frequency*), 845
overlap_multi() (*in module skrf.network*), 807

P

parallelconnect() (*in module skrf.network*), 803
params (*skrf.networkSet.NetworkSet property*), 848
params_types (*skrf.networkSet.NetworkSet property*), 848
params_values (*skrf.networkSet.NetworkSet property*), 849
passivity (*skrf.network.Network property*), 653
passivity() (*in module skrf.network*), 828
passivity_enforce() (*skrf.vectorFitting.VectorFitting method*), 1463
passivity_test() (*skrf.vectorFitting.VectorFitting method*), 1464
peelNPointsLossless() (*skrf.calibration.deembedding.IEEEP370 static method*), 1410
peelNPointsLossless() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1425
peelNPointsLossless() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1442
peelNPointsLossless() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1417
peelNPointsLossless() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1433
PHN (*class in skrf.calibration.calibration*), 1190
plot() (*skrf.frequency.Frequency method*), 845
plot() (*skrf.media.circularWaveguide.CircularWaveguide method*), 997
plot() (*skrf.media.coaxial.Coaxial method*), 1051
plot() (*skrf.media.cpw.CPW method*), 1023
plot() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1127
plot() (*skrf.media.distributedCircuit.DistributedCircuit method*), 943
plot() (*skrf.media.freespace.Freespace method*), 1103
plot() (*skrf.media.media.DefinedGammaZ0 method*), 919
plot() (*skrf.media.media.Media method*), 895
plot() (*skrf.media.mline.MLine method*), 1078
plot() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 971
plot() (*skrf.network.Network method*), 699
plot() (*skrf.vectorFitting.VectorFitting method*), 1465
plot_a_arcl() (*skrf.network.Network method*), 699
plot_a_arcl_unwrap() (*skrf.network.Network method*), 700
plot_a_complex() (*skrf.network.Network method*), 700
plot_a_db() (*skrf.network.Network method*), 701
plot_a_db10() (*skrf.network.Network method*), 701
plot_a_deg() (*skrf.network.Network method*), 702

plot_a_deg_unwrap() (*skrf.network.Network method*), 703
plot_a_im() (*skrf.network.Network method*), 703
plot_a_mag() (*skrf.network.Network method*), 704
plot_a_polar() (*skrf.network.Network method*), 704
plot_a_rad() (*skrf.network.Network method*), 705
plot_a_rad_unwrap() (*skrf.network.Network method*), 705
plot_a_re() (*skrf.network.Network method*), 706
plot_a_time() (*skrf.network.Network method*), 707
plot_a_time_db() (*skrf.network.Network method*), 707
plot_a_time_impulse() (*skrf.network.Network method*), 708
plot_a_time_mag() (*skrf.network.Network method*), 708
plot_a_time_step() (*skrf.network.Network method*), 709
plot_a_vswr() (*skrf.network.Network method*), 710
plot_attribute() (*skrf.network.Network method*), 710
plot_caled_ntwks() (*skrf.calibration.calibration.Calibration method*), 1156
plot_caled_ntwks() (*skrf.calibration.calibration.EightTerm method*), 1230
plot_caled_ntwks() (*skrf.calibration.calibration.EnhancedResponse method*), 1387
plot_caled_ntwks() (*skrf.calibration.calibration.LMR16 method*), 1349
plot_caled_ntwks() (*skrf.calibration.calibration.LRM method*), 1252
plot_caled_ntwks() (*skrf.calibration.calibration.LRRM method*), 1264
plot_caled_ntwks() (*skrf.calibration.calibration.MRC method*), 1275
plot_caled_ntwks() (*skrf.calibration.calibration.MultilineTRL method*), 1300
plot_caled_ntwks() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1313
plot_caled_ntwks() (*skrf.calibration.calibration.Normalization method*), 1360
plot_caled_ntwks() (*skrf.calibration.calibration.OnePort method*), 1167
plot_caled_ntwks() (*skrf.calibration.calibration.PHN method*), 1198
plot_caled_ntwks() (*skrf.calibration.calibration.SDDL method*), 1178
plot_caled_ntwks() (*skrf.calibration.calibration.SDDLWeikle method*), 1188
plot_caled_ntwks() (*skrf.calibration.calibration.SixteenTerm method*), 1338
plot_caled_ntwks() (*skrf.calibration.calibration.SOLT method*), 1219
plot_caled_ntwks() (*skrf.calibration.calibration.TRL method*), 1288
plot_caled_ntwks() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1327
plot_caled_ntwks() (*skrf.calibration.calibration.TwelveTerm method*), 1209
plot_caled_ntwks() (*skrf.calibration.calibration.TwoPortOnePath method*), 1377
plot_caled_ntwks() (*skrf.calibration.calibration.UnknownThru method*), 1241
plot_calibration_errors() (*skrf.calibration.calibration.Calibration method*), 1157
plot_calibration_errors() (*skrf.calibration.calibration.EightTerm method*), 1230
plot_calibration_errors() (*skrf.calibration.calibration.EnhancedResponse method*), 1387
plot_calibration_errors() (*skrf.calibration.calibration.LMR16 method*), 1350
plot_calibration_errors() (*skrf.calibration.calibration.LRM method*), 1252
plot_calibration_errors() (*skrf.calibration.calibration.LRRM method*), 1265
plot_calibration_errors() (*skrf.calibration.calibration.MRC method*), 1276
plot_calibration_errors() (*skrf.calibration.calibration.MultilineTRL method*), 1300
plot_calibration_errors() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1313
plot_calibration_errors() (*skrf.calibration.calibration.Normalization method*), 1360
plot_calibration_errors() (*skrf.calibration.calibration.OnePort method*), 1167
plot_calibration_errors() (*skrf.calibration.calibration.PHN method*), 1199
plot_calibration_errors() (*skrf.calibration.calibration.SDDL method*), 1178
plot_calibration_errors() (*skrf.calibration.calibration.SDDLWeikle method*), 1188
plot_calibration_errors() (*skrf.calibration.calibration.SixteenTerm method*), 1338
plot_calibration_errors() (*skrf.calibration.calibration.SOLT method*),

1219
plot_calibration_errors() (*skrf.calibration.calibration.TRL method*), 1288
plot_calibration_errors() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1327
plot_calibration_errors() (*skrf.calibration.calibration.TwelveTerm method*), 1209
plot_calibration_errors() (*skrf.calibration.calibration.TwoPortOnePath method*), 1377
plot_calibration_errors() (*skrf.calibration.calibration.UnknownThru method*), 1241
plot_check_impedance() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru method*), 1426
plot_check_impedance() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru method*), 1442
plot_check_impedance() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru method*), 1418
plot_check_impedance() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru method*), 1434
plot_check_residuals() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru method*), 1426
plot_check_residuals() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru method*), 1442
plot_check_residuals() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru method*), 1418
plot_check_residuals() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru method*), 1434
plot_complex_polar() (*in module skrf.plotting*), 873
plot_complex_rectangular() (*in module skrf.plotting*), 872
plot_contour() (*in module skrf.plotting*), 877
plot_convergence() (*skrf.vectorFitting.VectorFitting method*), 1465
plot_ep() (*skrf.media.freespace.Freespace method*), 1103
plot_ep_mu() (*skrf.media.freespace.Freespace method*), 1103
plot_g_arcl() (*skrf.network.Network method*), 711
plot_g_arcl_unwrap() (*skrf.network.Network method*), 711
plot_g_complex() (*skrf.network.Network method*), 712
plot_g_db() (*skrf.network.Network method*), 712
plot_g_db10() (*skrf.network.Network method*), 713
plot_g_deg() (*skrf.network.Network method*), 714
plot_g_deg_unwrap() (*skrf.network.Network method*), 714
plot_g_im() (*skrf.network.Network method*), 715
plot_g_mag() (*skrf.network.Network method*), 715
plot_g_polar() (*skrf.network.Network method*), 716
plot_g_rad() (*skrf.network.Network method*), 717
plot_g_rad_unwrap() (*skrf.network.Network method*), 717
plot_g_re() (*skrf.network.Network method*), 718
plot_g_time() (*skrf.network.Network method*), 718
plot_g_time_db() (*skrf.network.Network method*), 719
plot_g_time_impulse() (*skrf.network.Network method*), 720
plot_g_time_mag() (*skrf.network.Network method*), 720
plot_g_time_step() (*skrf.network.Network method*), 721
plot_g_vswr() (*skrf.network.Network method*), 721
plot_graph() (*skrf.circuit.Circuit method*), 1500
plot_h_arcl() (*skrf.network.Network method*), 722
plot_h_arcl_unwrap() (*skrf.network.Network method*), 723
plot_h_complex() (*skrf.network.Network method*), 723
plot_h_db() (*skrf.network.Network method*), 724
plot_h_db10() (*skrf.network.Network method*), 724
plot_h_deg() (*skrf.network.Network method*), 725
plot_h_deg_unwrap() (*skrf.network.Network method*), 726
plot_h_im() (*skrf.network.Network method*), 726
plot_h_mag() (*skrf.network.Network method*), 727
plot_h_polar() (*skrf.network.Network method*), 727
plot_h_rad() (*skrf.network.Network method*), 728
plot_h_rad_unwrap() (*skrf.network.Network method*), 728
plot_h_re() (*skrf.network.Network method*), 729
plot_h_time() (*skrf.network.Network method*), 730
plot_h_time_db() (*skrf.network.Network method*), 730
plot_h_time_impulse() (*skrf.network.Network method*), 731
plot_h_time_mag() (*skrf.network.Network method*), 731
plot_h_time_step() (*skrf.network.Network method*), 732
plot_h_vswr() (*skrf.network.Network method*), 733
plot_it_all() (*in module skrf.plotting*), 873
plot_it_all() (*skrf.network.Network method*), 733
plot_logsigma() (*in module skrf.plotting*), 877
plot_logsigma() (*skrf.networkSet.NetworkSet method*), 858
plot_minmax_bounds_component() (*in module skrf.plotting*), 874

plot_minmax_bounds_component() (*skrf.networkSet.NetworkSet method*), 858
plot_minmax_bounds_s_db() (*in module skrf.plotting*), 875
plot_minmax_bounds_s_db() (*skrf.networkSet.NetworkSet method*), 859
plot_minmax_bounds_s_db10() (*in module skrf.plotting*), 875
plot_minmax_bounds_s_db10() (*skrf.networkSet.NetworkSet method*), 859
plot_minmax_bounds_s_time_db() (*in module skrf.plotting*), 875
plot_minmax_bounds_s_time_db() (*skrf.networkSet.NetworkSet method*), 859
plot_mu() (*skrf.media.freespace.Freespace method*), 1103
plot_passivation() (*skrf.vectorFitting.VectorFitting method*), 1466
plot_passivity() (*in module skrf.plotting*), 877
plot_passivity() (*skrf.network.Network method*), 734
plot_polar() (*in module skrf.plotting*), 871
plot_prop_complex() (*skrf.network.Network method*), 734
plot_prop_polar() (*skrf.network.Network method*), 734
plot_reciprocity() (*skrf.network.Network method*), 735
plot_reciprocity2() (*skrf.network.Network method*), 735
plot_rectangular() (*in module skrf.plotting*), 871
plot_residuals() (*skrf.calibration.calibration.Calibration method*), 1157
plot_residuals() (*skrf.calibration.calibration.EightTerm method*), 1230
plot_residuals() (*skrf.calibration.calibration.EnhancedResponse method*), 1387
plot_residuals() (*skrf.calibration.calibration.LMR16 method*), 1350
plot_residuals() (*skrf.calibration.calibration.LRM method*), 1253
plot_residuals() (*skrf.calibration.calibration.LRRM method*), 1265
plot_residuals() (*skrf.calibration.calibration.MRC method*), 1276
plot_residuals() (*skrf.calibration.calibration.MultilineTRL method*), 1300
plot_residuals() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1314
plot_residuals() (*skrf.calibration.calibration.Normalization method*), 1361
plot_residuals() (*skrf.calibration.calibration.OnePort method*), 1168
plot_residuals() (*skrf.calibration.calibration.PHN method*), 1199
plot_residuals() (*skrf.calibration.calibration.SDDL method*), 1178
plot_residuals() (*skrf.calibration.calibration.SDDLWeikle method*), 1189
plot_residuals() (*skrf.calibration.calibration.SixteenTerm method*), 1338
plot_residuals() (*skrf.calibration.calibration.SOLT method*), 1219
plot_residuals() (*skrf.calibration.calibration.TRL method*), 1288
plot_residuals() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1327
plot_residuals() (*skrf.calibration.calibration.TwelveTerm method*), 1209
plot_residuals() (*skrf.calibration.calibration.TwoPortOnePath method*), 1377
plot_residuals() (*skrf.calibration.calibration.UnknownThru method*), 1241
plot_s_arcl() (*skrf.network.Network method*), 735
plot_s_arcl_unwrap() (*skrf.network.Network method*), 736
plot_s_complex() (*skrf.network.Network method*), 737
plot_s_db() (*skrf.network.Network method*), 737
plot_s_db10() (*skrf.network.Network method*), 738
plot_s_db_time() (*skrf.network.Network method*), 738
plot_s_deg() (*skrf.network.Network method*), 738
plot_s_deg_unwrap() (*skrf.network.Network method*), 739
plot_s_error() (*skrf.network.Network method*), 739
plot_s_im() (*skrf.network.Network method*), 740
plot_s_mag() (*skrf.network.Network method*), 740
plot_s_polar() (*skrf.network.Network method*), 741
plot_s_rad() (*skrf.network.Network method*), 742
plot_s_rad_unwrap() (*skrf.network.Network method*), 742
plot_s_re() (*skrf.network.Network method*), 743
plot_s_singular() (*skrf.vectorFitting.VectorFitting method*), 1468
plot_s_smith() (*skrf.network.Network method*), 743
plot_s_time() (*skrf.network.Network method*), 744
plot_s_time_db() (*skrf.network.Network method*), 745
plot_s_time_impulse() (*skrf.network.Network method*), 745
plot_s_time_mag() (*skrf.network.Network method*), 746
plot_s_time_step() (*skrf.network.Network method*), 746
plot_s_vswr() (*skrf.network.Network method*), 747
plot_smith() (*in module skrf.plotting*), 870
plot_t_arcl() (*skrf.network.Network method*), 748
plot_t_arcl_unwrap() (*skrf.network.Network method*), 748
plot_t_complex() (*skrf.network.Network method*), 749
plot_t_db() (*skrf.network.Network method*), 749

plot_t_db10() (*skrf.network.Network method*), 750
plot_t_deg() (*skrf.network.Network method*), 751
plot_t_deg_unwrap() (*skrf.network.Network method*), 751
plot_t_im() (*skrf.network.Network method*), 752
plot_t_mag() (*skrf.network.Network method*), 752
plot_t_polar() (*skrf.network.Network method*), 753
plot_t_rad() (*skrf.network.Network method*), 753
plot_t_rad_unwrap() (*skrf.network.Network method*), 754
plot_t_re() (*skrf.network.Network method*), 755
plot_t_time() (*skrf.network.Network method*), 755
plot_t_time_db() (*skrf.network.Network method*), 756
plot_t_time_impulse() (*skrf.network.Network method*), 756
plot_t_time_mag() (*skrf.network.Network method*), 757
plot_t_time_step() (*skrf.network.Network method*), 758
plot_t_vswr() (*skrf.network.Network method*), 758
plot_uncertainty_bounds_component() (*in module skrf.plotting*), 875
plot_uncertainty_bounds_component() (*skrf.networkSet.NetworkSet method*), 859
plot_uncertainty_bounds_s_db() (*in module skrf.plotting*), 876
plot_uncertainty_bounds_s_db() (*skrf.networkSet.NetworkSet method*), 860
plot_uncertainty_bounds_s_time_db() (*in module skrf.plotting*), 876
plot_uncertainty_bounds_s_time_db() (*skrf.networkSet.NetworkSet method*), 860
plot_uncertainty_decomposition() (*skrf.networkSet.NetworkSet method*), 860
plot_violin() (*skrf.networkSet.NetworkSet method*), 860
plot_y_arcl() (*skrf.network.Network method*), 759
plot_y_arcl_unwrap() (*skrf.network.Network method*), 759
plot_y_complex() (*skrf.network.Network method*), 760
plot_y_db() (*skrf.network.Network method*), 760
plot_y_db10() (*skrf.network.Network method*), 761
plot_y_deg() (*skrf.network.Network method*), 762
plot_y_deg_unwrap() (*skrf.network.Network method*), 762
plot_y_im() (*skrf.network.Network method*), 763
plot_y_mag() (*skrf.network.Network method*), 763
plot_y_polar() (*skrf.network.Network method*), 764
plot_y_rad() (*skrf.network.Network method*), 765
plot_y_rad_unwrap() (*skrf.network.Network method*), 765
plot_y_re() (*skrf.network.Network method*), 766
plot_y_time() (*skrf.network.Network method*), 766
plot_y_time_db() (*skrf.network.Network method*), 767
plot_y_time_impulse() (*skrf.network.Network method*), 768
plot_y_time_mag() (*skrf.network.Network method*), 768
plot_y_time_step() (*skrf.network.Network method*), 769
plot_y_vswr() (*skrf.network.Network method*), 769
plot_z_arcl() (*skrf.network.Network method*), 770
plot_z_arcl_unwrap() (*skrf.network.Network method*), 771
plot_z_complex() (*skrf.network.Network method*), 771
plot_z_db() (*skrf.network.Network method*), 772
plot_z_db10() (*skrf.network.Network method*), 772
plot_z_deg() (*skrf.network.Network method*), 773
plot_z_deg_unwrap() (*skrf.network.Network method*), 774
plot_z_im() (*skrf.network.Network method*), 774
plot_z_mag() (*skrf.network.Network method*), 775
plot_z_polar() (*skrf.network.Network method*), 775
plot_z_rad() (*skrf.network.Network method*), 776
plot_z_rad_unwrap() (*skrf.network.Network method*), 776
plot_z_re() (*skrf.network.Network method*), 777
plot_z_time() (*skrf.network.Network method*), 778
plot_z_time_db() (*skrf.network.Network method*), 778
plot_z_time_impulse() (*skrf.network.Network method*), 779
plot_z_time_mag() (*skrf.network.Network method*), 779
plot_z_time_step() (*skrf.network.Network method*), 780
plot_z_vswr() (*skrf.network.Network method*), 781
pna_csv_2_df() (*in module skrf.io.csv*), 1590
pna_csv_2_ntwks() (*in module skrf.io.csv*), 1590
pna_csv_2_ntwks3() (*in module skrf.io.csv*), 1590
poles (*skrf.vectorFitting.VectorFitting attribute*), 1457
pop() (*skrf.calibration.calibration.Calibration method*), 1157
pop() (*skrf.calibration.calibration.EightTerm method*), 1230
pop() (*skrf.calibration.calibration.EnhancedResponse method*), 1388
pop() (*skrf.calibration.calibration.LMR16 method*), 1350
pop() (*skrf.calibration.calibration.LRM method*), 1253
pop() (*skrf.calibration.calibration.LRRM method*), 1265
pop() (*skrf.calibration.calibration.MRC method*), 1276
pop() (*skrf.calibration.calibration.MultilineTRL method*), 1300
pop() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1314
pop() (*skrf.calibration.calibration.Normalization method*), 1361
pop() (*skrf.calibration.calibration.OnePort method*),

1168
pop() (*skrf.calibration.calibration.PHN method*), 1199
pop() (*skrf.calibration.calibration.SDDL method*), 1178
pop() (*skrf.calibration.calibration.SDDLWeikle method*), 1189
pop() (*skrf.calibration.calibration.SixteenTerm method*), 1339
pop() (*skrf.calibration.calibration.SOLT method*), 1220
pop() (*skrf.calibration.calibration.TRL method*), 1288
pop() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1327
pop() (*skrf.calibration.calibration.TwelveTerm method*), 1210
pop() (*skrf.calibration.calibration.TwoPortOnePath method*), 1377
pop() (*skrf.calibration.calibration.UnknownThru method*), 1242
pop() (*skrf.util.HomoDict method*), 1568
popitem() (*skrf.util.HomoDict method*), 1569
port impedance, **1609**
Port() (*skrf.circuit.Circuit class method*), 1494
port_indexes (*skrf.circuit.Circuit property*), 1491
port_modes (*skrf.network.Network property*), 654
port_names (*skrf.circuit.Circuit property*), 1491
port_names (*skrf.network.Network property*), 654
port_tuples (*skrf.network.Network property*), 655
port_z0 (*skrf.circuit.Circuit property*), 1491
PRIMARY_PROPERTIES (*skrf.network.Network attribute*), 638
ProgressBar (*class in skrf.util*), 1559
propagation_impedance_2_distributed_circuit() (*in module skrf.tlineFunctions*), 1542
proportional_coeff (*skrf.vectorFitting.VectorFitting attribute*), 1457
psd2TimeDomain() (*in module skrf.mathFunctions*), 1525

Q

Q_circle() (*skrf.qfactor.Qfactor method*), 1477
Q_unloaded() (*skrf.qfactor.Qfactor method*), 1477
Qfactor (*class in skrf.qfactor*), 1475
QuadratureHybrid (*class in skrf.media.device*), 1511

R

R (*skrf.media.coaxial.Coaxial property*), 1034
radian_2_degree() (*in module skrf.mathFunctions*), 1519
rand() (*skrf.networkSet.NetworkSet method*), 861
rand_c() (*in module skrf.mathFunctions*), 1524
rand_rng() (*in module skrf.mathFunctions*), 1524
random() (*skrf.media.circularWaveguide.CircularWaveguide method*), 997
random() (*skrf.media.coaxial.Coaxial method*), 1051
random() (*skrf.media.cpw.CPW method*), 1023
random() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1127
random() (*skrf.media.distributedCircuit.DistributedCircuit method*), 943
random() (*skrf.media.freespace.Freespace method*), 1103
random() (*skrf.media.media.DefinedGammaZ0 method*), 919
random() (*skrf.media.media.Media method*), 895
random() (*skrf.media.mline.MLine method*), 1078
random() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 971
rational_interp() (*in module skrf.mathFunctions*), 1525
read() (*in module skrf.io.general*), 1575
read() (*skrf.io.csv.AgilentCSV method*), 1588
read() (*skrf.network.Network method*), 781
read_all() (*in module skrf.io.general*), 1576
read_all_csv() (*in module skrf.io.csv*), 1586
read_all_networks() (*in module skrf.io.general*), 1577
read_all_zva_dat() (*in module skrf.io.csv*), 1591
read_npz() (*skrf.vectorFitting.VectorFitting method*), 1468
read_npz() (*skrf.vectorFitting.VectorFittingParametric method*), 1473
read_pna_csv() (*in module skrf.io.csv*), 1589
read_touchstone() (*skrf.network.Network method*), 782
read_vectorstar_csv() (*in module skrf.io.csv*), 1592
read_zipped_touchstones() (*in module skrf.io.touchstone*), 1575
read_zva_dat() (*in module skrf.io.csv*), 1591
reciprocity (*skrf.network.Network property*), 655
reciprocity() (*in module skrf.network*), 829
reciprocity2 (*skrf.network.Network property*), 655
RectangularWaveguide (*class in skrf.media.rectangularWaveguide*), 951
reduce_circuit() (*in module skrf.circuit*), 1506
reflection_coefficient_2_input_impedance() (*in module skrf.tlineFunctions*), 1539
reflection_coefficient_2_input_impedance_at_theta() (*in module skrf.tlineFunctions*), 1539
reflection_coefficient_2_propagation_constant() (*in module skrf.tlineFunctions*), 1539
reflection_coefficient_at_theta() (*in module skrf.tlineFunctions*), 1538
remove_and_cal() (*skrf.calibration.calibration.Calibration method*), 1157
remove_and_cal() (*skrf.calibration.calibration.EightTerm method*), 1231
remove_and_cal() (*skrf.calibration.calibration.EnhancedResponse method*), 1388
remove_and_cal() (*skrf.calibration.calibration.LMR16*

method), 1351
remove_and_cal() (*skrf.calibration.calibration.LRM method*), 1253
remove_and_cal() (*skrf.calibration.calibration.LRRM method*), 1265
remove_and_cal() (*skrf.calibration.calibration.MRC method*), 1276
remove_and_cal() (*skrf.calibration.calibration.MultilineTRL method*), 1301
remove_and_cal() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1314
remove_and_cal() (*skrf.calibration.calibration.Normalization method*), 1361
remove_and_cal() (*skrf.calibration.calibration.OnePort method*), 1168
remove_and_cal() (*skrf.calibration.calibration.PHN method*), 1199
remove_and_cal() (*skrf.calibration.calibration.SDDL method*), 1179
remove_and_cal() (*skrf.calibration.calibration.SDDLWeikle method*), 1189
remove_and_cal() (*skrf.calibration.calibration.SixteenTerm method*), 1339
remove_and_cal() (*skrf.calibration.calibration.SOLT method*), 1220
remove_and_cal() (*skrf.calibration.calibration.TRL method*), 1289
remove_and_cal() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1328
remove_and_cal() (*skrf.calibration.calibration.TwelveTerm method*), 1210
remove_and_cal() (*skrf.calibration.calibration.TwoPortOnePath method*), 1378
remove_and_cal() (*skrf.calibration.calibration.UnknownThru method*), 1242
renormalize() (*skrf.calibration.calibration.EightTerm method*), 1231
renormalize() (*skrf.calibration.calibration.LMR16 method*), 1351
renormalize() (*skrf.calibration.calibration.LRM method*), 1254
renormalize() (*skrf.calibration.calibration.LRRM method*), 1266
renormalize() (*skrf.calibration.calibration.MRC method*), 1277
renormalize() (*skrf.calibration.calibration.MultilineTRL method*), 1301
renormalize() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1314
renormalize() (*skrf.calibration.calibration.SixteenTerm method*), 1339
renormalize() (*skrf.calibration.calibration.TRL method*), 1289
renormalize() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1328
renormalize() (*skrf.calibration.calibration.UnknownThru method*), 1242
renormalize() (*skrf.network.Network method*), 782
renormalize_s() (*in module skrf.network*), 827
renumber() (*skrf.network.Network method*), 783
renumbered() (*skrf.network.Network method*), 785
resample() (*skrf.network.Network method*), 785
residual_ntwk_sets (*skrf.calibration.calibration.Calibration property*), 1153
residual_ntwk_sets (*skrf.calibration.calibration.EightTerm property*), 1226
residual_ntwk_sets (*skrf.calibration.calibration.EnhancedResponse property*), 1383
residual_ntwk_sets (*skrf.calibration.calibration.LMR16 property*), 1346
residual_ntwk_sets (*skrf.calibration.calibration.LRM property*), 1249
residual_ntwk_sets (*skrf.calibration.calibration.LRRM property*), 1260
residual_ntwk_sets (*skrf.calibration.calibration.MRC property*), 1272
residual_ntwk_sets (*skrf.calibration.calibration.MultilineTRL property*), 1295
residual_ntwk_sets (*skrf.calibration.calibration.NISTMultilineTRL property*), 1308
residual_ntwk_sets (*skrf.calibration.calibration.Normalization property*), 1357
residual_ntwk_sets (*skrf.calibration.calibration.OnePort property*), 1164
residual_ntwk_sets (*skrf.calibration.calibration.PHN property*), 1195
residual_ntwk_sets (*skrf.calibration.calibration.SDDL property*), 1174
residual_ntwk_sets (*skrf.calibration.calibration.SDDLWeikle property*), 1185
residual_ntwk_sets (*skrf.calibration.calibration.SixteenTerm property*), 1334
residual_ntwk_sets (*skrf.calibration.calibration.SOLT property*), 1216
residual_ntwk_sets (*skrf.calibration.calibration.TRL property*), 1283
residual_ntwk_sets (*skrf.calibration.calibration.TUGMultilineTRL property*), 1322
residual_ntwk_sets (*skrf.calibration.calibration.TwelveTerm property*), 1206
residual_ntwk_sets (*skrf.calibration.calibration.TwoPortOnePath property*), 1373
residual_ntwk_sets (*skrf.calibration.calibration.UnknownThru property*), 1237
residual_ntwks (*skrf.calibration.calibration.Calibration property*), 1153
residual_ntwks (*skrf.calibration.calibration.EightTerm property*), 1226

residual_ntwks (*skrf.calibration.calibration.EnhancedResponse property*), 1383
residual_ntwks (*skrf.calibration.calibration.LMR16 property*), 1346
residual_ntwks (*skrf.calibration.calibration.LRM property*), 1249
residual_ntwks (*skrf.calibration.calibration.LRRM property*), 1260
residual_ntwks (*skrf.calibration.calibration.MRC property*), 1272
residual_ntwks (*skrf.calibration.calibration.MultilineTRL property*), 1295
residual_ntwks (*skrf.calibration.calibration.NISTMultilineTRL property*), 1308
residual_ntwks (*skrf.calibration.calibration.Normalization property*), 1357
residual_ntwks (*skrf.calibration.calibration.OnePort property*), 1164
residual_ntwks (*skrf.calibration.calibration.PHN property*), 1195
residual_ntwks (*skrf.calibration.calibration.SDDL property*), 1174
residual_ntwks (*skrf.calibration.calibration.SDDLWeikle property*), 1185
residual_ntwks (*skrf.calibration.calibration.SixteenTerm property*), 1334
residual_ntwks (*skrf.calibration.calibration.SOLT property*), 1216
residual_ntwks (*skrf.calibration.calibration.TRL property*), 1283
residual_ntwks (*skrf.calibration.calibration.TUGMultilineTRL property*), 1323
residual_ntwks (*skrf.calibration.calibration.TwelveTerm property*), 1206
residual_ntwks (*skrf.calibration.calibration.TwoPortOnePath property*), 1373
residual_ntwks (*skrf.calibration.calibration.UnknownThru property*), 1238
residues (*skrf.vectorFitting.VectorFitting attribute*), 1457
resistor() (*skrf.media.circularWaveguide.CircularWaveguide method*), 997
resistor() (*skrf.media.coaxial.Coaxial method*), 1052
resistor() (*skrf.media.cpw.CPW method*), 1024
resistor() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1128
resistor() (*skrf.media.distributedCircuit.DistributedCircuit method*), 944
resistor() (*skrf.media.freespace.Freespace method*), 1104
resistor() (*skrf.media.media.DefinedGammaZ0 method*), 919
resistor() (*skrf.media.media.Media method*), 896
resistor() (*skrf.media.mline.MLine method*), 1079
resistor() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 972
rho (*skrf.media.circularWaveguide.CircularWaveguide property*), 984
rho (*skrf.media.freespace.Freespace property*), 1090
rho (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 958
ring_slot (*in module skrf.data*), 1549
ring_slot_meas (*in module skrf.data*), 1551
rn (*skrf.network.Network property*), 656
rotate() (*skrf.network.Network method*), 786
round_to() (*skrf.frequency.Frequency method*), 845
RRL (*skrf.media.coaxial.Coaxial property*), 1035
rsolve() (*in module skrf.mathFunctions*), 1530
run() (*skrf.calibration.calibration.Calibration method*), 1158
run() (*skrf.calibration.calibration.EightTerm method*), 1231
run() (*skrf.calibration.calibration.EnhancedResponse method*), 1388
run() (*skrf.calibration.calibration.LMR16 method*), 1351
run() (*skrf.calibration.calibration.LRM method*), 1254
run() (*skrf.calibration.calibration.LRRM method*), 1266
run() (*skrf.calibration.calibration.MRC method*), 1277
run() (*skrf.calibration.calibration.MultilineTRL method*), 1301
run() (*skrf.calibration.calibration.MultiportCal method*), 1365
run() (*skrf.calibration.calibration.MultiportSOLT method*), 1368
run() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1315
run() (*skrf.calibration.calibration.Normalization method*), 1362
run() (*skrf.calibration.calibration.OnePort method*), 1168
run() (*skrf.calibration.calibration.PHN method*), 1200
run() (*skrf.calibration.calibration.SDDL method*), 1179
run() (*skrf.calibration.calibration.SDDLWeikle method*), 1190
run() (*skrf.calibration.calibration.SixteenTerm method*), 1339
run() (*skrf.calibration.calibration.SOLT method*), 1220
run() (*skrf.calibration.calibration.TRL method*), 1289
run() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1328
run() (*skrf.calibration.calibration.TwelveTerm method*), 1210
run() (*skrf.calibration.calibration.TwoPortOnePath method*), 1378
run() (*skrf.calibration.calibration.UnknownThru method*), 1242
run_2port() (*skrf.calibration.calibration.MultiportCal*

method), 1365
run_2port() (*skrf.calibration.calibration.MultiportSOLT method*), 1368

S

s (*skrf.circuit.Circuit property*), 1491
s (*skrf.network.Network property*), 656
s2a() (*in module skrf.network*), 817
s2g() (*in module skrf.network*), 817
s2h() (*in module skrf.network*), 818
s2s() (*in module skrf.network*), 815
s2s_active() (*in module skrf.network*), 824
s2t() (*in module skrf.network*), 816
s2vswr_active() (*in module skrf.network*), 826
s2y() (*in module skrf.network*), 816
s2y_active() (*in module skrf.network*), 825
s2z() (*in module skrf.network*), 815
s2z_active() (*in module skrf.network*), 825
s_active() (*skrf.circuit.Circuit method*), 1500
s_active() (*skrf.network.Network method*), 786
s_arcl (*skrf.network.Network property*), 656
s_arcl_unwrap (*skrf.network.Network property*), 656
s_db (*skrf.network.Network property*), 656
s_db10 (*skrf.network.Network property*), 657
S_DEF_DEFAULT (*in module skrf.constants*), 1548
S_DEF_HFSS_DEFAULT (*in module skrf.constants*), 1548
S_DEFINITIONS (*in module skrf.constants*), 1547
s_deg (*skrf.network.Network property*), 657
s_deg_unwrap (*skrf.network.Network property*), 657
s_error() (*in module skrf.network*), 831
s_error() (*skrf.network.Network method*), 787
s_external (*skrf.circuit.Circuit property*), 1492
s_im (*skrf.network.Network property*), 657
s_invert (*skrf.network.Network property*), 657
s_mag (*skrf.network.Network property*), 658
s_power (*skrf.network.Network property*), 658
s_pseudo (*skrf.network.Network property*), 658
s_rad (*skrf.network.Network property*), 658
s_rad_unwrap (*skrf.network.Network property*), 659
s_re (*skrf.network.Network property*), 659
s_time (*skrf.network.Network property*), 659
s_time_db (*skrf.network.Network property*), 659
s_time_impulse (*skrf.network.Network property*), 659
s_time_mag (*skrf.network.Network property*), 660
s_time_step (*skrf.network.Network property*), 660
s_traveling (*skrf.network.Network property*), 660
s_vswr (*skrf.network.Network property*), 660
save_all_figs() (*in module skrf.plotting*), 879
save_calibration() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1315
save_sesh() (*in module skrf.io.general*), 1580
scalar2Complex() (*in module skrf.mathFunctions*), 1521
scalar_mat() (*skrf.networkSet.NetworkSet method*), 861
scalar_networks (*skrf.io.csv.AgilentCSV property*), 1588
scale_frequency_ticks() (*in module skrf.plotting*), 879
scrape_legend() (*in module skrf.plotting*), 880
SDDL (*class in skrf.calibration.calibration*), 1169
SDDLWeikle (*class in skrf.calibration.calibration*), 1180
se2gmm() (*skrf.network.Network method*), 787
section_at() (*skrf.taper.Exponential method*), 1144
section_at() (*skrf.taper.Klopfenstein method*), 1148
section_at() (*skrf.taper.Linear method*), 1142
section_at() (*skrf.taper.SmoothStep method*), 1146
section_at() (*skrf.taper.Taper1D method*), 1139
section_length (*skrf.taper.Exponential property*), 1143
section_length (*skrf.taper.Klopfenstein property*), 1147
section_length (*skrf.taper.Linear property*), 1140
section_length (*skrf.taper.SmoothStep property*), 1145
section_length (*skrf.taper.Taper1D property*), 1137
sections (*skrf.taper.Exponential property*), 1143
sections (*skrf.taper.Klopfenstein property*), 1147
sections (*skrf.taper.Linear property*), 1140
sections (*skrf.taper.SmoothStep property*), 1145
sections (*skrf.taper.Taper1D property*), 1137
sel() (*skrf.networkSet.NetworkSet method*), 862
SeriesImpedance() (*skrf.circuit.Circuit class method*), 1495
set_noise_a() (*skrf.network.Network method*), 789
set_rand_rng() (*in module skrf.mathFunctions*), 1524
set_wise_function() (*skrf.networkSet.NetworkSet method*), 863
setdefault() (*skrf.util.HomoDict method*), 1569
SetValidator (*class in skrf.vi.validators*), 1448
shade_bands() (*in module skrf.plotting*), 879
shiftNPoints() (*skrf.calibration.deembedding.IEEEP370 static method*), 1411
shiftNPoints() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2x static method*), 1426
shiftNPoints() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xT static method*), 1442
shiftNPoints() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xT static method*), 1418
shiftNPoints() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xTh static method*), 1434
shiftOnePort() (*skrf.calibration.deembedding.IEEEP370 static method*), 1411
shiftOnePort() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2x static method*), 1426
shiftOnePort() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xT static method*), 1443
shiftOnePort() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xT*

static method), 1418
shiftOnePort() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru static method*), 1434
Short (*class in skrf.calibration.deembedding*), 1400
short() (*skrf.media.circularWaveguide.CircularWaveguide method*), 998
short() (*skrf.media.coaxial.Coaxial method*), 1052
short() (*skrf.media.cpw.CPW method*), 1024
short() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1128
short() (*skrf.media.distributedCircuit.DistributedCircuit method*), 944
short() (*skrf.media.freespace.Freespace method*), 1104
short() (*skrf.media.media.DefinedGammaZ0 method*), 920
short() (*skrf.media.media.Media method*), 896
short() (*skrf.media.mline.MLine method*), 1079
short() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 972
short_2p (*in module skrf.data*), 1557
ShortOpen (*class in skrf.calibration.deembedding*), 1399
shunt() (*skrf.media.circularWaveguide.CircularWaveguide method*), 999
shunt() (*skrf.media.coaxial.Coaxial method*), 1053
shunt() (*skrf.media.cpw.CPW method*), 1025
shunt() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1129
shunt() (*skrf.media.distributedCircuit.DistributedCircuit method*), 945
shunt() (*skrf.media.freespace.Freespace method*), 1105
shunt() (*skrf.media.media.DefinedGammaZ0 method*), 920
shunt() (*skrf.media.media.Media method*), 897
shunt() (*skrf.media.mline.MLine method*), 1080
shunt() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 973
shunt_capacitor() (*skrf.media.circularWaveguide.CircularWaveguide method*), 999
shunt_capacitor() (*skrf.media.coaxial.Coaxial method*), 1053
shunt_capacitor() (*skrf.media.cpw.CPW method*), 1025
shunt_capacitor() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1130
shunt_capacitor() (*skrf.media.distributedCircuit.DistributedCircuit method*), 945
shunt_capacitor() (*skrf.media.freespace.Freespace method*), 1105
shunt_capacitor() (*skrf.media.media.DefinedGammaZ0 method*), 921
shunt_capacitor() (*skrf.media.media.Media method*), 898
shunt_capacitor() (*skrf.media.mline.MLine method*), 1080
shunt_capacitor() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 973
shunt_delay_load() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1000
shunt_delay_load() (*skrf.media.coaxial.Coaxial method*), 1054
shunt_delay_load() (*skrf.media.cpw.CPW method*), 1026
shunt_delay_load() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1130
shunt_delay_load() (*skrf.media.distributedCircuit.DistributedCircuit method*), 946
shunt_delay_load() (*skrf.media.freespace.Freespace method*), 1106
shunt_delay_load() (*skrf.media.media.DefinedGammaZ0 method*), 921
shunt_delay_load() (*skrf.media.media.Media method*), 898
shunt_delay_load() (*skrf.media.mline.MLine method*), 1081
shunt_delay_load() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 974
shunt_delay_open() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1000
shunt_delay_open() (*skrf.media.coaxial.Coaxial method*), 1054
shunt_delay_open() (*skrf.media.cpw.CPW method*), 1026
shunt_delay_open() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1131
shunt_delay_open() (*skrf.media.distributedCircuit.DistributedCircuit method*), 946
shunt_delay_open() (*skrf.media.freespace.Freespace method*), 1106
shunt_delay_open() (*skrf.media.media.DefinedGammaZ0 method*), 922
shunt_delay_open() (*skrf.media.media.Media method*), 899
shunt_delay_open() (*skrf.media.mline.MLine method*), 1081
shunt_delay_open() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 974
shunt_delay_short() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1001
shunt_delay_short() (*skrf.media.coaxial.Coaxial method*), 1055
shunt_delay_short() (*skrf.media.cpw.CPW method*), 1027
shunt_delay_short() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1131
shunt_delay_short() (*skrf.media.distributedCircuit.DistributedCircuit*

method), 947
shunt_delay_short() (*skrf.media.freespace.Freespace method*), 1107
shunt_delay_short() (*skrf.media.media.DefinedGammaZ0 method*), 922
shunt_delay_short() (*skrf.media.media.Media method*), 899
shunt_delay_short() (*skrf.media.mline.MLine method*), 1082
shunt_delay_short() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 975
shunt_inductor() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1001
shunt_inductor() (*skrf.media.coaxial.Coaxial method*), 1055
shunt_inductor() (*skrf.media.cpw.CPW method*), 1027
shunt_inductor() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1132
shunt_inductor() (*skrf.media.distributedCircuit.DistributedCircuit method*), 947
shunt_inductor() (*skrf.media.freespace.Freespace method*), 1107
shunt_inductor() (*skrf.media.media.DefinedGammaZ0 method*), 923
shunt_inductor() (*skrf.media.media.Media method*), 900
shunt_inductor() (*skrf.media.mline.MLine method*), 1082
shunt_inductor() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 975
shunt_resistor() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1002
shunt_resistor() (*skrf.media.coaxial.Coaxial method*), 1056
shunt_resistor() (*skrf.media.cpw.CPW method*), 1028
shunt_resistor() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1132
shunt_resistor() (*skrf.media.distributedCircuit.DistributedCircuit method*), 948
shunt_resistor() (*skrf.media.freespace.Freespace method*), 1108
shunt_resistor() (*skrf.media.media.DefinedGammaZ0 method*), 923
shunt_resistor() (*skrf.media.media.Media method*), 900
shunt_resistor() (*skrf.media.mline.MLine method*), 1083
shunt_resistor() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 976
ShuntAdmittance() (*skrf.circuit.Circuit class method*), 1495
si (*skrf.vi.validators.FreqValidator attribute*), 1447
signature() (*in module skrf.plotting*), 881
signature() (*skrf.networkSet.NetworkSet method*), 863
SixteenTerm (*class in skrf.calibration.calibration*), 1329
skin_depth() (*in module skrf.tlineFunctions*), 1543
skrf.calibration.calibration module, 1148
skrf.calibration.deembedding module, 1394
skrf.circuit module, 1485
skrf.constants module, 1547
skrf.data module, 1548
skrf.frequency module, 836
skrf.io.csv module, 1586
skrf.io.general module, 1575
skrf.io.touchstone module, 1569
skrf.mathFunctions module, 1514
skrf.media module, 881
skrf.network module, 633
skrf.networkSet module, 846
skrf.plotting module, 868
skrf.taper module, 1136
skrf.time module, 1481
skrf.tlineFunctions module, 1531
skrf.util module, 1558
skrf.vi module, 1443
skrf.vi.vna module, 1452

smith() (*in module skrf.plotting*), 869
smooth() (*in module skrf.util*), 1561
SmoothStep (*class in skrf.taper*), 1144
SOLT (*class in skrf.calibration.calibration*), 1211
solved_c (*skrf.calibration.calibration.LRRM property*), 1260
solved_l (*skrf.calibration.calibration.LRRM property*), 1261
solved_m (*skrf.calibration.calibration.LRRM property*), 1261
solved_r (*skrf.calibration.calibration.LRM property*), 1249
solved_r1 (*skrf.calibration.calibration.LRRM property*), 1261
solved_r2 (*skrf.calibration.calibration.LRRM property*), 1261
solved_reflect (*skrf.calibration.calibration.LMR16 property*), 1346
solved_through (*skrf.calibration.calibration.LMR16 property*), 1346
sort() (*skrf.networkSet.NetworkSet method*), 863
span (*skrf.frequency.Frequency property*), 840
span_scaled (*skrf.frequency.Frequency property*), 840
sparameters (*skrf.io.touchstone.Touchstone property*), 1570
split2xthru() (*skrf.calibration.deembedding.IEEEP370 method*), 1411
split2xthru() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru method*), 1427
split2xthru() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru method*), 1443
split2xthru() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru method*), 1419
split2xthru() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru method*), 1435
SplitPi (*class in skrf.calibration.deembedding*), 1401
SplitTee (*class in skrf.calibration.deembedding*), 1403
splitter() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1002
splitter() (*skrf.media.coaxial.Coaxial method*), 1056
splitter() (*skrf.media.cpw.CPW method*), 1028
splitter() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1133
splitter() (*skrf.media.distributedCircuit.DistributedCircuit method*), 948
splitter() (*skrf.media.freespace.Freespace method*), 1108
splitter() (*skrf.media.media.DefinedGammaZ0 method*), 924
splitter() (*skrf.media.media.Media method*), 901
splitter() (*skrf.media.mline.MLine method*), 1083
splitter() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 976
sqrt_known_sign() (*in module skrf.mathFunctions*), 1529
sqrt_phase_unwrap() (*in module skrf.mathFunctions*), 1518
stability (*skrf.network.Network property*), 661
stability_circle() (*skrf.network.Network method*), 789
start (*skrf.frequency.Frequency property*), 840
start_scaled (*skrf.frequency.Frequency property*), 840
statistical_2_touchstone() (*in module skrf.io.general*), 1583
std_s_db (*skrf.networkSet.NetworkSet property*), 849
stdev() (*in module skrf.network*), 830
step (*skrf.frequency.Frequency property*), 840
step_response() (*skrf.network.Network method*), 791
step_scaled (*skrf.frequency.Frequency property*), 840
stitch() (*in module skrf.network*), 806
stop (*skrf.frequency.Frequency property*), 841
stop_scaled (*skrf.frequency.Frequency property*), 841
stylely() (*in module skrf.plotting*), 878
subnetwork() (*in module skrf.network*), 807
subnetwork() (*skrf.network.Network method*), 791
subplot_params() (*in module skrf.plotting*), 878
surface_impedance() (*in module skrf.tlineFunctions*), 1544
surface_resistivity() (*in module skrf.tlineFunctions*), 1544
sweep_type (*skrf.frequency.Frequency property*), 841
SWEEP_TYPES (*in module skrf.constants*), 1548
switch terms, **1609**

T

t (*skrf.circuit.Circuit property*), 1488
t (*skrf.frequency.Frequency property*), 841
t (*skrf.media.device.Hybrid property*), 1511
t (*skrf.media.device.MatchedSymmetricCoupler property*), 1510
t (*skrf.media.device.QuadratureHybrid property*), 1512
t (*skrf.network.Network property*), 661
t2s() (*in module skrf.network*), 822
t2y() (*in module skrf.network*), 823
t2z() (*in module skrf.network*), 822
t_arcl (*skrf.network.Network property*), 661
t_arcl_unwrap (*skrf.network.Network property*), 662
t_db (*skrf.network.Network property*), 662
t_db10 (*skrf.network.Network property*), 662
t_deg (*skrf.network.Network property*), 662
t_deg_unwrap (*skrf.network.Network property*), 662
t_im (*skrf.network.Network property*), 663
t_mag (*skrf.network.Network property*), 663
T_matrices (*skrf.calibration.calibration.EightTerm property*), 1222
T_matrices (*skrf.calibration.calibration.LMR16 property*), 1342

T_matrices (*skrf.calibration.calibration.LRM property*), 1245
T_matrices (*skrf.calibration.calibration.LRRM property*), 1256
T_matrices (*skrf.calibration.calibration.MRC property*), 1268
T_matrices (*skrf.calibration.calibration.MultilineTRL property*), 1291
T_matrices (*skrf.calibration.calibration.NISTMultilineTRL property*), 1304
T_matrices (*skrf.calibration.calibration.SixteenTerm property*), 1330
T_matrices (*skrf.calibration.calibration.TRL property*), 1279
T_matrices (*skrf.calibration.calibration.TUGMultilineTRL property*), 1318
T_matrices (*skrf.calibration.calibration.UnknownThru property*), 1234
T_matrices() (*skrf.calibration.calibration.MultiportCal method*), 1364
T_matrices() (*skrf.calibration.calibration.MultiportSOLT method*), 1367
t_ns (*skrf.frequency.Frequency property*), 841
t_rad (*skrf.network.Network property*), 663
t_rad_unwrap (*skrf.network.Network property*), 663
t_re (*skrf.network.Network property*), 663
t_time (*skrf.network.Network property*), 664
t_time_db (*skrf.network.Network property*), 664
t_time_impulse (*skrf.network.Network property*), 664
t_time_mag (*skrf.network.Network property*), 664
t_time_step (*skrf.network.Network property*), 664
t_vswr (*skrf.network.Network property*), 665
tand_f (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1115
Taper1D (*class in skrf.taper*), 1136
tee (*in module skrf.data*), 1553
tee() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1003
tee() (*skrf.media.coaxial.Coaxial method*), 1057
tee() (*skrf.media.cpw.CPW method*), 1029
tee() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1133
tee() (*skrf.media.distributedCircuit.DistributedCircuit method*), 949
tee() (*skrf.media.freespace.Freespace method*), 1109
tee() (*skrf.media.media.DefinedGammaZ0 method*), 924
tee() (*skrf.media.media.Media method*), 901
tee() (*skrf.media.mline.MLine method*), 1084
tee() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 977
terminate() (*in module skrf.calibration.calibration*), 1390
terminate() (*skrf.calibration.calibration.EightTerm method*), 1231
terminate() (*skrf.calibration.calibration.LMR16 method*), 1351
terminate() (*skrf.calibration.calibration.LRM method*), 1254
terminate() (*skrf.calibration.calibration.LRRM method*), 1266
terminate() (*skrf.calibration.calibration.MRC method*), 1277
terminate() (*skrf.calibration.calibration.MultilineTRL method*), 1301
terminate() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1315
terminate() (*skrf.calibration.calibration.SixteenTerm method*), 1339
terminate() (*skrf.calibration.calibration.TRL method*), 1289
terminate() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1328
terminate() (*skrf.calibration.calibration.UnknownThru method*), 1243
terminate_2port() (*skrf.calibration.calibration.MultiportCal method*), 1365
terminate_2port() (*skrf.calibration.calibration.MultiportSOLT method*), 1368
terminate_nport() (*in module skrf.calibration.calibration*), 1391
theta() (*in module skrf.tlineFunctions*), 1532
theta_2_d() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1003
theta_2_d() (*skrf.media.coaxial.Coaxial method*), 1057
theta_2_d() (*skrf.media.cpw.CPW method*), 1029
theta_2_d() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1134
theta_2_d() (*skrf.media.distributedCircuit.DistributedCircuit method*), 949
theta_2_d() (*skrf.media.freespace.Freespace method*), 1109
theta_2_d() (*skrf.media.media.DefinedGammaZ0 method*), 925
theta_2_d() (*skrf.media.media.Media method*), 902
theta_2_d() (*skrf.media.mline.MLine method*), 1084
theta_2_d() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 977
thru() (*skrf.calibration.deembedding.IEEEP370 static method*), 1412
thru() (*skrf.calibration.deembedding.IEEEP370_MM_NZC_2xThru static method*), 1427
thru() (*skrf.calibration.deembedding.IEEEP370_MM_ZC_2xThru static method*), 1443
thru() (*skrf.calibration.deembedding.IEEEP370_SE_NZC_2xThru static method*), 1419
thru() (*skrf.calibration.deembedding.IEEEP370_SE_ZC_2xThru*

static method), 1435
thru() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1004
thru() (*skrf.media.coaxial.Coaxial method*), 1058
thru() (*skrf.media.cpw.CPW method*), 1030
thru() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1134
thru() (*skrf.media.distributedCircuit.DistributedCircuit method*), 950
thru() (*skrf.media.freespace.Freespace method*), 1110
thru() (*skrf.media.media.DefinedGammaZ0 method*), 925
thru() (*skrf.media.media.Media method*), 902
thru() (*skrf.media.mline.MLine method*), 1085
thru() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 978
time_gate() (*in module skrf.time*), 1482
time_gate() (*skrf.network.Network method*), 792
to_dataframe() (*skrf.network.Network method*), 792
to_dataframe() (*skrf.networkSet.NetworkSet method*), 864
to_dict() (*skrf.networkSet.NetworkSet method*), 864
to_json_string() (*in module skrf.io.general*), 1585
to_meters() (*in module skrf.constants*), 1548
to_meters() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1004
to_meters() (*skrf.media.coaxial.Coaxial method*), 1058
to_meters() (*skrf.media.cpw.CPW method*), 1030
to_meters() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1135
to_meters() (*skrf.media.distributedCircuit.DistributedCircuit method*), 950
to_meters() (*skrf.media.freespace.Freespace method*), 1110
to_meters() (*skrf.media.media.DefinedGammaZ0 method*), 926
to_meters() (*skrf.media.media.Media method*), 903
to_meters() (*skrf.media.mline.MLine method*), 1085
to_meters() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 978
to_s_dict() (*skrf.networkSet.NetworkSet method*), 865
total_error (*skrf.calibration.calibration.Calibration property*), 1153
total_error (*skrf.calibration.calibration.EightTerm property*), 1226
total_error (*skrf.calibration.calibration.EnhancedResponse property*), 1383
total_error (*skrf.calibration.calibration.LMR16 property*), 1346
total_error (*skrf.calibration.calibration.LRM property*), 1249
total_error (*skrf.calibration.calibration.LRRM property*), 1261
total_error (*skrf.calibration.calibration.MRC property*), 1272
total_error (*skrf.calibration.calibration.MultilineTRL property*), 1295
total_error (*skrf.calibration.calibration.NISTMultilineTRL property*), 1308
total_error (*skrf.calibration.calibration.Normalization property*), 1357
total_error (*skrf.calibration.calibration.OnePort property*), 1164
total_error (*skrf.calibration.calibration.PHN property*), 1195
total_error (*skrf.calibration.calibration.SDDL property*), 1174
total_error (*skrf.calibration.calibration.SDDLWeikle property*), 1185
total_error (*skrf.calibration.calibration.SixteenTerm property*), 1334
total_error (*skrf.calibration.calibration.SOLT property*), 1216
total_error (*skrf.calibration.calibration.TRL property*), 1283
total_error (*skrf.calibration.calibration.TUGMultilineTRL property*), 1323
total_error (*skrf.calibration.calibration.TwelveTerm property*), 1206
total_error (*skrf.calibration.calibration.TwoPortOnePath property*), 1373
total_error (*skrf.calibration.calibration.UnknownThru property*), 1238
Touchstone (*class in skrf.io.touchstone*), 1569
TouchstoneEncoder (*class in skrf.io.general*), 1583
TRL (*class in skrf.calibration.calibration*), 1278
truthy (*skrf.vi.validators.BooleanValidator attribute*), 1451
TUGMultilineTRL (*class in skrf.calibration.calibration*), 1316
TwelveTerm (*class in skrf.calibration.calibration*), 1201
two_port_error_vector_2_Ts() (*in module skrf.calibration.calibration*), 1393
two_port_reflect() (*in module skrf.network*), 832
twoport_to_nport() (*in module skrf.network*), 808
TwoPortOnePath (*class in skrf.calibration.calibration*), 1369

U

unbiased_error (*skrf.calibration.calibration.Calibration property*), 1154
unbiased_error (*skrf.calibration.calibration.EightTerm property*), 1227
unbiased_error (*skrf.calibration.calibration.EnhancedResponse property*), 1384
unbiased_error (*skrf.calibration.calibration.LMR16 property*), 1347

unbiased_error (*skrf.calibration.calibration.LRM property*), 1249
unbiased_error (*skrf.calibration.calibration.LRRM property*), 1261
unbiased_error (*skrf.calibration.calibration.MRC property*), 1272
unbiased_error (*skrf.calibration.calibration.MultilineTRL property*), 1296
unbiased_error (*skrf.calibration.calibration.NISTMultilineTRL property*), 1309
unbiased_error (*skrf.calibration.calibration.Normalization property*), 1357
unbiased_error (*skrf.calibration.calibration.OnePort property*), 1165
unbiased_error (*skrf.calibration.calibration.PHN property*), 1196
unbiased_error (*skrf.calibration.calibration.SDDL property*), 1175
unbiased_error (*skrf.calibration.calibration.SDDLWeikle property*), 1185
unbiased_error (*skrf.calibration.calibration.SixteenTerm property*), 1335
unbiased_error (*skrf.calibration.calibration.SOLT property*), 1216
unbiased_error (*skrf.calibration.calibration.TRL property*), 1284
unbiased_error (*skrf.calibration.calibration.TUGMultilineTRL property*), 1323
unbiased_error (*skrf.calibration.calibration.TwelveTerm property*), 1206
unbiased_error (*skrf.calibration.calibration.TwoPortOnePath property*), 1374
unbiased_error (*skrf.calibration.calibration.UnknownThru property*), 1238
uncertainty_ntwk_triplet() (*skrf.networkSet.NetworkSet method*), 865
unilateral_gain (*skrf.network.Network property*), 665
unique_name() (*in module skrf.util*), 1564
unit (*skrf.frequency.Frequency property*), 841
unit_dict (*skrf.frequency.Frequency attribute*), 841
UnknownThru (*class in skrf.calibration.calibration*), 1232
unterminate() (*in module skrf.calibration.calibration*), 1391
unterminate() (*skrf.calibration.calibration.EightTerm method*), 1232
unterminate() (*skrf.calibration.calibration.LMR16 method*), 1351
unterminate() (*skrf.calibration.calibration.LRM method*), 1254
unterminate() (*skrf.calibration.calibration.LRRM method*), 1266
unterminate() (*skrf.calibration.calibration.MRC method*), 1277
unterminate() (*skrf.calibration.calibration.MultilineTRL method*), 1301
unterminate() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1315
unterminate() (*skrf.calibration.calibration.SixteenTerm method*), 1340
unterminate() (*skrf.calibration.calibration.TRL method*), 1289
unterminate() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1328
unterminate() (*skrf.calibration.calibration.UnknownThru method*), 1243
unterminate_2port() (*skrf.calibration.calibration.MultiportCal method*), 1365
unterminate_2port() (*skrf.calibration.calibration.MultiportSOLT method*), 1368
unwrap_rad() (*in module skrf.mathFunctions*), 1518
update() (*skrf.util.HomoDict method*), 1569
update_coefs() (*skrf.calibration.calibration.Calibration method*), 1158
update_coefs() (*skrf.calibration.calibration.EightTerm method*), 1232
update_coefs() (*skrf.calibration.calibration.EnhancedResponse method*), 1388
update_coefs() (*skrf.calibration.calibration.LMR16 method*), 1351
update_coefs() (*skrf.calibration.calibration.LRM method*), 1254
update_coefs() (*skrf.calibration.calibration.LRRM method*), 1266
update_coefs() (*skrf.calibration.calibration.MRC method*), 1277
update_coefs() (*skrf.calibration.calibration.MultilineTRL method*), 1302
update_coefs() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1315
update_coefs() (*skrf.calibration.calibration.Normalization method*), 1362
update_coefs() (*skrf.calibration.calibration.OnePort method*), 1169
update_coefs() (*skrf.calibration.calibration.PHN method*), 1200
update_coefs() (*skrf.calibration.calibration.SDDL method*), 1179
update_coefs() (*skrf.calibration.calibration.SDDLWeikle method*), 1190
update_coefs() (*skrf.calibration.calibration.SixteenTerm method*), 1340
update_coefs() (*skrf.calibration.calibration.SOLT method*), 1220
update_coefs() (*skrf.calibration.calibration.TRL method*), 1290

update_coefs() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1329
update_coefs() (*skrf.calibration.calibration.TwelveTerm method*), 1210
update_coefs() (*skrf.calibration.calibration.TwoPortOnePath method*), 1378
update_coefs() (*skrf.calibration.calibration.UnknownThru method*), 1243
update_iteration() (*skrf.util.ProgressBar method*), 1560
update_networks() (*skrf.circuit.Circuit method*), 1501

V

v_g (*skrf.media.circularWaveguide.CircularWaveguide property*), 984
v_g (*skrf.media.coaxial.Coaxial property*), 1037
v_g (*skrf.media.cpw.CPW property*), 1008
v_g (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1115
v_g (*skrf.media.distributedCircuit.DistributedCircuit property*), 930
v_g (*skrf.media.freespace.Freespace property*), 1090
v_g (*skrf.media.media.DefinedGammaZ0 property*), 906
v_g (*skrf.media.media.Media property*), 883
v_g (*skrf.media.mline.MLine property*), 1063
v_g (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 959
v_p (*skrf.media.circularWaveguide.CircularWaveguide property*), 985
v_p (*skrf.media.coaxial.Coaxial property*), 1037
v_p (*skrf.media.cpw.CPW property*), 1009
v_p (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1116
v_p (*skrf.media.distributedCircuit.DistributedCircuit property*), 931
v_p (*skrf.media.freespace.Freespace property*), 1091
v_p (*skrf.media.media.DefinedGammaZ0 property*), 907
v_p (*skrf.media.media.Media property*), 884
v_p (*skrf.media.mline.MLine property*), 1064
v_p (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 959
validate_input() (*skrf.vi.validators.BooleanValidator method*), 1452
validate_input() (*skrf.vi.validators.DelimitedStrValidator method*), 1450
validate_input() (*skrf.vi.validators.DictValidator method*), 1449
validate_input() (*skrf.vi.validators.EnumValidator method*), 1448
validate_input() (*skrf.vi.validators.FloatValidator method*), 1446
validate_input() (*skrf.vi.validators.FreqValidator method*), 1447
validate_input() (*skrf.vi.validators.IntValidator method*), 1445
validate_input() (*skrf.vi.validators.SetValidator method*), 1449
validate_output() (*skrf.vi.validators.BooleanValidator method*), 1452
validate_output() (*skrf.vi.validators.DelimitedStrValidator method*), 1450
validate_output() (*skrf.vi.validators.DictValidator method*), 1450
validate_output() (*skrf.vi.validators.EnumValidator method*), 1448
validate_output() (*skrf.vi.validators.FloatValidator method*), 1446
validate_output() (*skrf.vi.validators.FreqValidator method*), 1447
validate_output() (*skrf.vi.validators.IntValidator method*), 1445
validate_output() (*skrf.vi.validators.SetValidator method*), 1449
value_vector (*skrf.taper.Exponential property*), 1143
value_vector (*skrf.taper.Klopfenstein property*), 1147
value_vector (*skrf.taper.Linear property*), 1140
value_vector (*skrf.taper.SmoothStep property*), 1145
value_vector (*skrf.taper.Taper1D property*), 1138
values() (*skrf.util.HomoDict method*), 1569
vector_fit() (*skrf.vectorFitting.VectorFitting method*), 1469
VectorFitting (*class in skrf.vectorFitting*), 1455
VectorFittingParametric (*class in skrf.vectorFitting*), 1471
vectorstar_csv_2_ntwks() (*in module skrf.io.csv*), 1592
verify_12term (*skrf.calibration.calibration.Calibration property*), 1154
verify_12term (*skrf.calibration.calibration.EightTerm property*), 1227
verify_12term (*skrf.calibration.calibration.EnhancedResponse property*), 1384
verify_12term (*skrf.calibration.calibration.LMR16 property*), 1347
verify_12term (*skrf.calibration.calibration.LRM property*), 1250
verify_12term (*skrf.calibration.calibration.LRRM property*), 1262
verify_12term (*skrf.calibration.calibration.MRC property*), 1273
verify_12term (*skrf.calibration.calibration.MultilineTRL property*), 1296
verify_12term (*skrf.calibration.calibration.NISTMultilineTRL property*), 1309
verify_12term (*skrf.calibration.calibration.Normalization property*), 1358
verify_12term (*skrf.calibration.calibration.OnePort*

property), 1165
verify_12term (*skrf.calibration.calibration.PHN property*), 1196
verify_12term (*skrf.calibration.calibration.SDDL property*), 1175
verify_12term (*skrf.calibration.calibration.SDDLWeikle property*), 1186
verify_12term (*skrf.calibration.calibration.SixteenTerm property*), 1335
verify_12term (*skrf.calibration.calibration.SOLT property*), 1217
verify_12term (*skrf.calibration.calibration.TRL property*), 1284
verify_12term (*skrf.calibration.calibration.TUGMultilineTRL property*), 1324
verify_12term (*skrf.calibration.calibration.TwelveTerm property*), 1207
verify_12term (*skrf.calibration.calibration.TwoPortOnePath property*), 1374
verify_12term (*skrf.calibration.calibration.UnknownThru property*), 1239
verify_12term_ntwk (*skrf.calibration.calibration.Calibration property*), 1154
verify_12term_ntwk (*skrf.calibration.calibration.EightTerm property*), 1227
verify_12term_ntwk (*skrf.calibration.calibration.EnhancedResponse property*), 1384
verify_12term_ntwk (*skrf.calibration.calibration.LMR16 property*), 1347
verify_12term_ntwk (*skrf.calibration.calibration.LRM property*), 1250
verify_12term_ntwk (*skrf.calibration.calibration.LRRM property*), 1262
verify_12term_ntwk (*skrf.calibration.calibration.MRC property*), 1273
verify_12term_ntwk (*skrf.calibration.calibration.MultilineTRL property*), 1296
verify_12term_ntwk (*skrf.calibration.calibration.NISTMultilineTRL property*), 1309
verify_12term_ntwk (*skrf.calibration.calibration.Normalization property*), 1358
verify_12term_ntwk (*skrf.calibration.calibration.OnePort property*), 1165
verify_12term_ntwk (*skrf.calibration.calibration.PHN property*), 1196
verify_12term_ntwk (*skrf.calibration.calibration.SDDL property*), 1175
verify_12term_ntwk (*skrf.calibration.calibration.SDDLWeikle property*), 1186
verify_12term_ntwk (*skrf.calibration.calibration.SixteenTerm property*), 1335
verify_12term_ntwk (*skrf.calibration.calibration.SOLT property*), 1217
verify_12term_ntwk (*skrf.calibration.calibration.TRL property*), 1284
verify_12term_ntwk (*skrf.calibration.calibration.TUGMultilineTRL property*), 1324
verify_12term_ntwk (*skrf.calibration.calibration.TwelveTerm property*), 1207
verify_12term_ntwk (*skrf.calibration.calibration.TwoPortOnePath property*), 1374
verify_12term_ntwk (*skrf.calibration.calibration.UnknownThru property*), 1239
version (*skrf.io.touchstone.Touchstone property*), 1570
voltage_current_propagation() (*in module skrf.tlineFunctions*), 1541
voltages() (*skrf.circuit.Circuit method*), 1502
voltages_external() (*skrf.circuit.Circuit method*), 1503
vswr_active() (*skrf.circuit.Circuit method*), 1503
vswr_active() (*skrf.network.Network method*), 793

W

w (*skrf.frequency.Frequency property*), 842
wall_clock_time (*skrf.vectorFitting.VectorFitting attribute*), 1457
white_gaussian_polar() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1005
white_gaussian_polar() (*skrf.media.coaxial.Coaxial method*), 1059
white_gaussian_polar() (*skrf.media.cpw.CPW method*), 1031
white_gaussian_polar() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1135
white_gaussian_polar() (*skrf.media.distributedCircuit.DistributedCircuit method*), 951
white_gaussian_polar() (*skrf.media.freespace.Freespace method*), 1111
white_gaussian_polar() (*skrf.media.media.DefinedGammaZ0 method*), 926
white_gaussian_polar() (*skrf.media.media.Media method*), 903
white_gaussian_polar() (*skrf.media.mline.MLine method*), 1086
white_gaussian_polar() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 979
windowed() (*skrf.network.Network method*), 793
write() (*in module skrf.io.general*), 1578
write() (*skrf.calibration.calibration.Calibration method*), 1158
write() (*skrf.calibration.calibration.EightTerm method*), 1232

write() (*skrf.calibration.calibration.EnhancedResponse method*), 1388
write() (*skrf.calibration.calibration.LMR16 method*), 1352
write() (*skrf.calibration.calibration.LRM method*), 1254
write() (*skrf.calibration.calibration.LRRM method*), 1266
write() (*skrf.calibration.calibration.MRC method*), 1277
write() (*skrf.calibration.calibration.MultilineTRL method*), 1302
write() (*skrf.calibration.calibration.NISTMultilineTRL method*), 1315
write() (*skrf.calibration.calibration.Normalization method*), 1362
write() (*skrf.calibration.calibration.OnePort method*), 1169
write() (*skrf.calibration.calibration.PHN method*), 1200
write() (*skrf.calibration.calibration.SDDL method*), 1179
write() (*skrf.calibration.calibration.SDDLWeikle method*), 1190
write() (*skrf.calibration.calibration.SixteenTerm method*), 1340
write() (*skrf.calibration.calibration.SOLT method*), 1220
write() (*skrf.calibration.calibration.TRL method*), 1290
write() (*skrf.calibration.calibration.TUGMultilineTRL method*), 1329
write() (*skrf.calibration.calibration.TwelveTerm method*), 1210
write() (*skrf.calibration.calibration.TwoPortOnePath method*), 1378
write() (*skrf.calibration.calibration.UnknownThru method*), 1243
write() (*skrf.network.Network method*), 794
write() (*skrf.networkSet.NetworkSet method*), 866
write_all() (*in module skrf.io.general*), 1579
write_csv() (*skrf.media.circularWaveguide.CircularWaveguide method*), 1005
write_csv() (*skrf.media.coaxial.Coaxial method*), 1059
write_csv() (*skrf.media.cpw.CPW method*), 1031
write_csv() (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 method*), 1136
write_csv() (*skrf.media.distributedCircuit.DistributedCircuit method*), 951
write_csv() (*skrf.media.freespace.Freespace method*), 1111
write_csv() (*skrf.media.media.DefinedGammaZ0 method*), 927
write_csv() (*skrf.media.media.Media method*), 904
write_csv() (*skrf.media.mline.MLine method*), 1086
write_csv() (*skrf.media.rectangularWaveguide.RectangularWaveguide method*), 979
write_mdif() (*skrf.networkSet.NetworkSet method*), 866
write_npz() (*skrf.vectorFitting.VectorFitting method*), 1470
write_npz() (*skrf.vectorFitting.VectorFittingParametric method*), 1474
write_spice_subcircuit_s() (*skrf.vectorFitting.VectorFitting method*), 1471
write_spreadsheet() (*skrf.network.Network method*), 795
write_spreadsheet() (*skrf.networkSet.NetworkSet method*), 867
write_to_json_string() (*skrf.network.Network method*), 795
write_touchstone() (*skrf.network.Network method*), 795

X

X (*skrf.circuit.Circuit property*), 1488
X_F (*skrf.circuit.Circuit property*), 1488

Y

Y (*skrf.media.coaxial.Coaxial property*), 1035
Y (*skrf.media.distributedCircuit.DistributedCircuit property*), 929
y (*skrf.network.Network property*), 665
y2s() (*in module skrf.network*), 820
y2t() (*in module skrf.network*), 821
y2z() (*in module skrf.network*), 821
y_active() (*skrf.circuit.Circuit method*), 1504
y_active() (*skrf.network.Network method*), 797
y_arcl (*skrf.network.Network property*), 666
y_arcl_unwrap (*skrf.network.Network property*), 666
y_db (*skrf.network.Network property*), 666
y_db10 (*skrf.network.Network property*), 666
y_deg (*skrf.network.Network property*), 667
y_deg_unwrap (*skrf.network.Network property*), 667
y_im (*skrf.network.Network property*), 667
Y_LABEL_DICT (*skrf.network.Network attribute*), 638
y_mag (*skrf.network.Network property*), 667
y_opt (*skrf.network.Network property*), 667
y_rad (*skrf.network.Network property*), 668
y_rad_unwrap (*skrf.network.Network property*), 668
y_re (*skrf.network.Network property*), 668
y_time (*skrf.network.Network property*), 668
y_time_db (*skrf.network.Network property*), 668
y_time_impulse (*skrf.network.Network property*), 669
y_time_mag (*skrf.network.Network property*), 669
y_time_step (*skrf.network.Network property*), 669
y_vswr (*skrf.network.Network property*), 669

Z

Z (*skrf.media.coaxial.Coaxial property*), 1035
Z (*skrf.media.distributedCircuit.DistributedCircuit property*), 929
z (*skrf.network.Network property*), 669
z0 (*skrf.calibration.calibration.NISTMultilineTRL property*), 1309
z0 (*skrf.circuit.Circuit property*), 1492
Z0 (*skrf.media.circularWaveguide.CircularWaveguide property*), 981
z0 (*skrf.media.circularWaveguide.CircularWaveguide property*), 985
Z0 (*skrf.media.coaxial.Coaxial property*), 1035
z0 (*skrf.media.coaxial.Coaxial property*), 1038
Z0 (*skrf.media.cpw.CPW property*), 1007
z0 (*skrf.media.cpw.CPW property*), 1009
Z0 (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1114
z0 (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1116
Z0 (*skrf.media.distributedCircuit.DistributedCircuit property*), 929
z0 (*skrf.media.distributedCircuit.DistributedCircuit property*), 931
Z0 (*skrf.media.freespace.Freespace property*), 1088
z0 (*skrf.media.freespace.Freespace property*), 1091
Z0 (*skrf.media.media.DefinedGammaZ0 property*), 905
z0 (*skrf.media.media.DefinedGammaZ0 property*), 907
Z0 (*skrf.media.media.Media property*), 882
z0 (*skrf.media.media.Media property*), 885
Z0 (*skrf.media.mline.MLine property*), 1062
z0 (*skrf.media.mline.MLine property*), 1064
Z0 (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 954
z0 (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 960
z0 (*skrf.network.Network property*), 670
z0_characteristic (*skrf.media.circularWaveguide.CircularWaveguide property*), 986
z0_characteristic (*skrf.media.coaxial.Coaxial property*), 1038
z0_characteristic (*skrf.media.cpw.CPW property*), 1010
z0_characteristic (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1116
z0_characteristic (*skrf.media.distributedCircuit.DistributedCircuit property*), 932
z0_characteristic (*skrf.media.freespace.Freespace property*), 1091
z0_characteristic (*skrf.media.media.DefinedGammaZ0 property*), 907
z0_characteristic (*skrf.media.media.Media property*), 885
z0_characteristic (*skrf.media.mline.MLine property*), 1065
z0_characteristic (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 960
Z0_f (*skrf.media.mline.MLine property*), 1062
z0_override (*skrf.media.circularWaveguide.CircularWaveguide property*), 986
z0_override (*skrf.media.coaxial.Coaxial property*), 1038
z0_override (*skrf.media.cpw.CPW property*), 1010
z0_override (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1117
z0_override (*skrf.media.distributedCircuit.DistributedCircuit property*), 932
z0_override (*skrf.media.freespace.Freespace property*), 1092
z0_override (*skrf.media.media.DefinedGammaZ0 property*), 908
z0_override (*skrf.media.media.Media property*), 885
z0_override (*skrf.media.mline.MLine property*), 1065
z0_override (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 960
z0_port (*skrf.media.circularWaveguide.CircularWaveguide property*), 986
z0_port (*skrf.media.coaxial.Coaxial property*), 1039
z0_port (*skrf.media.cpw.CPW property*), 1010
z0_port (*skrf.media.definedAEpTandZ0.DefinedAEpTandZ0 property*), 1117
z0_port (*skrf.media.distributedCircuit.DistributedCircuit property*), 932
z0_port (*skrf.media.freespace.Freespace property*), 1092
z0_port (*skrf.media.media.DefinedGammaZ0 property*), 908
z0_port (*skrf.media.media.Media property*), 885
z0_port (*skrf.media.mline.MLine property*), 1065
z0_port (*skrf.media.rectangularWaveguide.RectangularWaveguide property*), 960
z2a() (*in module skrf.network*), 819
z2s() (*in module skrf.network*), 818
z2t() (*in module skrf.network*), 819
z2y() (*in module skrf.network*), 819
z_active() (*skrf.circuit.Circuit method*), 1504
z_active() (*skrf.network.Network method*), 797
z_arcl (*skrf.network.Network property*), 670
z_arcl_unwrap (*skrf.network.Network property*), 670
z_db (*skrf.network.Network property*), 670
z_db10 (*skrf.network.Network property*), 671
z_deg (*skrf.network.Network property*), 671
z_deg_unwrap (*skrf.network.Network property*), 671
z_im (*skrf.network.Network property*), 671
z_mag (*skrf.network.Network property*), 671
z_opt (*skrf.network.Network property*), 672
z_rad (*skrf.network.Network property*), 672

z_rad_unwrap (*skrf.network.Network property*), 672
z_re (*skrf.network.Network property*), 672
z_time (*skrf.network.Network property*), 672
z_time_db (*skrf.network.Network property*), 672
z_time_impulse (*skrf.network.Network property*), 673
z_time_mag (*skrf.network.Network property*), 673
z_time_step (*skrf.network.Network property*), 673
z_vswr (*skrf.network.Network property*), 673
ZERO (*in module skrf.constants*), 1547
zipped_touchstone() (*skrf.network.Network class method*), 798
zl_2_Gamma0() (*in module skrf.tlineFunctions*), 1532
zl_2_Gamma_in() (*in module skrf.tlineFunctions*), 1533
zl_2_swr() (*in module skrf.tlineFunctions*), 1534
zl_2_total_loss() (*in module skrf.tlineFunctions*), 1534
zl_2_zin() (*in module skrf.tlineFunctions*), 1533
zva_dat_2_ntwks() (*in module skrf.io.csv*), 1591